

All Java Solutions

Count: 3371

1. 1.java

Path: LeetCode-main/solutions/1. Two Sum/1.java

```
class Solution {
    public int[] twoSum(int[] nums, int target) {
        Map<Integer, Integer> numToIndex = new HashMap<>();

        for (int i = 0; i < nums.length; ++i) {
            if (numToIndex.containsKey(target - nums[i]))
                return new int[] {numToIndex.get(target - nums[i]), i};
            numToIndex.put(nums[i], i);
        }

        throw new IllegalArgumentException();
    }
}
```

2. 10.java

Path: LeetCode-main/solutions/10. Regular Expression Matching/10.java

```
class Solution {
    public boolean isMatch(String s, String p) {
        final int m = s.length();
        final int n = p.length();
        // dp[i][j] := true if s[0..i] matches p[0..j]
        boolean[][] dp = new boolean[m + 1][n + 1];
        dp[0][0] = true;

        for (int j = 0; j < p.length(); ++j)
            if (p.charAt(j) == '*' && dp[0][j - 1])
                dp[0][j + 1] = true;

        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j)
                if (p.charAt(j) == '*') {
                    // The minimum index of '*' is 1.
                    final boolean noRepeat = dp[i + 1][j - 1];
                    final boolean doRepeat = isMatch(s, i, p, j - 1) && dp[i][j + 1];
                    dp[i + 1][j + 1] = noRepeat || doRepeat;
                } else if (isMatch(s, i, p, j)) {
                    dp[i + 1][j + 1] = dp[i][j];
                }

        return dp[m][n];
    }

    private boolean isMatch(final String s, int i, final String p, int j) {
        return j >= 0 && p.charAt(j) == '.' || s.charAt(i) == p.charAt(j);
    }
}
```

3. 100.java

Path: LeetCode-main/solutions/100. Same Tree/100.java

```
class Solution {
    public boolean isSameTree(TreeNode p, TreeNode q) {
        if (p == null || q == null)
            return p == q;
        return p.val == q.val &&          //
            isSameTree(p.left, q.left) && //
            isSameTree(p.right, q.right);
    }
}
```

4. 1000-2.java

Path: LeetCode-main/solutions/1000. Minimum Cost to Merge Stones/1000-2.java

```
class Solution {
    public int mergeStones(int[] stones, int K) {
        final int n = stones.length;
        if ((n - 1) % (K - 1) != 0)
            return -1;

        final int MAX = 1_000_000_000;
        // dp[i][j][k] := the minimum cost to merge stones[i..j] into k piles
        int[][][] dp = new int[n][n][K + 1];
        Arrays.stream(dp).forEach(A -> Arrays.stream(A).forEach(B -> Arrays.fill(B, MAX)));
        int[] prefix = new int[n + 1];

        for (int i = 0; i < n; ++i)
            dp[i][i][1] = 0;

        for (int i = 0; i < n; ++i)
            prefix[i + 1] = prefix[i] + stones[i];

        for (int d = 1; d < n; ++d)
            for (int i = 0; i + d < n; ++i) {
                final int j = i + d;
                for (int k = 2; k <= K; ++k)
                    for (int m = i; m < j; m += K - 1)
                        dp[i][j][k] = Math.min(dp[i][j][k], dp[i][m][1] + dp[m + 1][j][k - 1]);
                dp[i][j][1] = dp[i][j][K] + prefix[j + 1] - prefix[i];
            }

        return dp[0][n - 1][1];
    }
}
```

5. 1000-3.java

Path: LeetCode-main/solutions/1000. Minimum Cost to Merge Stones/1000-3.java

```
class Solution {
    public int mergeStones(int[] stones, int k) {
        final int n = stones.length;
        if ((n - 1) % (k - 1) != 0)
            return -1;

        int[][] mem = new int[n][n];
        int[] prefix = new int[n + 1];

        Arrays.stream(mem).forEach(A -> Arrays.fill(A, MAX));

        for (int i = 0; i < n; ++i)
            prefix[i + 1] = prefix[i] + stones[i];

        final int cost = mergeStones(stones, 0, n - 1, k, prefix, mem);
        return cost == MAX ? -1 : cost;
    }

    private static final int MAX = 1_000_000_000;

    // Returns the minimum cost to merge stones[i..j].
    private int mergeStones(final int[] stones, int i, int j, int k, int[] prefix, int[][] mem) {
        if (j - i + 1 < k)
            return 0;
        if (mem[i][j] != MAX)
            return mem[i][j];

        for (int m = i; m < j; m += k - 1)
            mem[i][j] = Math.min(mem[i][j], mergeStones(stones, i, m, k, prefix, mem) +
                                   mergeStones(stones, m + 1, j, k, prefix, mem));

        if ((j - i) % (k - 1) == 0)
            mem[i][j] += prefix[j + 1] - prefix[i];

        return mem[i][j];
    }
}
```

6. 1000-4.java

Path: LeetCode-main/solutions/1000. Minimum Cost to Merge Stones/1000-4.java

```
class Solution {
    public int mergeStones(int[] stones, int K) {
        final int n = stones.length;
        if ((n - 1) % (K - 1) != 0)
            return -1;

        final int MAX = 1_000_000_000;

        // dp[i][j] := the minimum cost to merge stones[i..j]
        int[][] dp = new int[n][n];
        Arrays.stream(dp).forEach(A -> Arrays.fill(A, MAX));
        int[] prefix = new int[n + 1];

        for (int i = 0; i < n; ++i)
            dp[i][i] = 0;

        for (int i = 0; i < n; ++i)
            prefix[i + 1] = prefix[i] + stones[i];

        for (int d = 1; d < n; ++d)
            for (int i = 0; i + d < n; ++i) {
                final int j = i + d;
                for (int m = i; m < j; m += K - 1)
                    dp[i][j] = Math.min(dp[i][j], dp[i][m] + dp[m + 1][j]);
                if (d % (K - 1) == 0)
                    dp[i][j] += prefix[j + 1] - prefix[i];
            }

        return dp[0][n - 1];
    }
}
```

7. 1000.java

Path: LeetCode-main/solutions/1000. Minimum Cost to Merge Stones/1000.java

```
class Solution {
    public int mergeStones(int[] stones, int K) {
        final int n = stones.length;
        int[][][] mem = new int[n][n][K + 1];
        int[] prefix = new int[n + 1];

        Arrays.stream(mem).forEach(A -> Arrays.stream(A).forEach(B -> Arrays.fill(B, MAX)));

        for (int i = 0; i < n; ++i)
            prefix[i + 1] = prefix[i] + stones[i];

        final int cost = mergeStones(stones, 0, n - 1, 1, K, prefix, mem);
        return cost == MAX ? -1 : cost;
    }

    private static final int MAX = 1_000_000_000;

    // Returns the minimum cost to merge stones[i..j] into k piles.
    private int mergeStones(final int[] stones, int i, int j, int k, int K, int[] prefix,
        int[][][] mem) {
        if ((j - i + 1 - k) % (K - 1) != 0)
            return MAX;
        if (i == j)
            return k == 1 ? 0 : MAX;
        if (mem[i][j][k] != MAX)
            return mem[i][j][k];
        if (k == 1)
            return mem[i][j][k] =
                mergeStones(stones, i, j, K, K, prefix, mem) + prefix[j + 1] - prefix[i];

        for (int m = i; m < j; m += K - 1)
            mem[i][j][k] =
                Math.min(mem[i][j][k], mergeStones(stones, i, m, 1, K, prefix, mem) +
                    mergeStones(stones, m + 1, j, k - 1, K, prefix, mem));

        return mem[i][j][k];
    }
}
```

8. 1001.java

Path: LeetCode-main/solutions/1001. Grid Illumination/1001.java

```
class Solution {
    public int[] gridIllumination(int n, int[][] lamps, int[][] queries) {
        List<Integer> ans = new ArrayList<>();
        Map<Integer, Integer> rows = new HashMap<>();
        Map<Integer, Integer> cols = new HashMap<>();
        Map<Integer, Integer> diag1 = new HashMap<>();
        Map<Integer, Integer> diag2 = new HashMap<>();
        Set<Long> lampsSet = new HashSet<>();

        for (int[] lamp : lamps) {
            final int i = lamp[0];
            final int j = lamp[1];
            if (lampsSet.add(hash(i, j))) {
                rows.merge(i, 1, Integer::sum);
                cols.merge(j, 1, Integer::sum);
                diag1.merge(i + j, 1, Integer::sum);
                diag2.merge(i - j, 1, Integer::sum);
            }
        }

        for (int[] query : queries) {
            final int i = query[0];
            final int j = query[1];
            if (rows.getOrDefault(i, 0) > 0 || cols.getOrDefault(j, 0) > 0 ||
                diag1.getOrDefault(i + j, 0) > 0 || diag2.getOrDefault(i - j, 0) > 0) {
                ans.add(1);
                for (int y = Math.max(0, i - 1); y < Math.min(n, i + 2); ++y)
                    for (int x = Math.max(0, j - 1); x < Math.min(n, j + 2); ++x)
                        if (lampsSet.remove(hash(y, x))) {
                            rows.merge(y, 1, Integer::sum);
                            cols.merge(x, 1, Integer::sum);
                            diag1.merge(y + x, 1, Integer::sum);
                            diag2.merge(y - x, 1, Integer::sum);
                        }
            } else {
                ans.add(0);
            }
        }

        return ans.stream().mapToInt(Integer::intValue).toArray();
    }

    private long hash(int i, int j) {
        return ((long) i << 32) + j;
    }
}
```


9. 1002.java

Path: LeetCode-main/solutions/1002. Find Common Characters/1002.java

```
class Solution {
    public List<String> commonChars(String[] words) {
        List<String> ans = new ArrayList<>();
        int[] commonCount = new int[26];
        Arrays.fill(commonCount, Integer.MAX_VALUE);

        for (final String word : words) {
            int[] count = new int[26];
            for (final char c : word.toCharArray())
                ++count[c - 'a'];
            for (int i = 0; i < 26; ++i)
                commonCount[i] = Math.min(commonCount[i], count[i]);
        }

        for (char c = 'a'; c <= 'z'; ++c)
            for (int i = 0; i < commonCount[c - 'a']; ++i)
                ans.add(String.valueOf(c));

        return ans;
    }
}
```

10. 1003.java

Path: LeetCode-main/solutions/1003. Check If Word Is Valid After Substitutions/1003.java

```
class Solution {
    public boolean isValid(String s) {
        Deque<Character> stack = new ArrayDeque<>();

        for (final char c : s.toCharArray())
            if (c == 'c') {
                if (stack.size() < 2)
                    return false;
                if (stack.peek() != 'b')
                    return false;
                stack.pop();
                if (stack.peek() != 'a')
                    return false;
                stack.pop();
            } else {
                stack.push(c);
            }
        return stack.isEmpty();
    }
}
```

11. 1004.java

Path: LeetCode-main/solutions/1004. Max Consecutive Ones III/1004.java

```
class Solution {
    public int longestOnes(int[] nums, int k) {
        int ans = 0;

        for (int l = 0, r = 0; r < nums.length; ++r) {
            if (nums[r] == 0)
                --k;
            while (k < 0)
                if (nums[l++] == 0)
                    ++k;
            ans = Math.max(ans, r - l + 1);
        }

        return ans;
    }
}
```

12. 1005.java

Path: LeetCode-main/solutions/1005. Maximize Sum Of Array After K Negations/1005.java

```
class Solution {
    public int largestSumAfterKNegations(int[] nums, int k) {
        Arrays.sort(nums);

        for (int i = 0; i < nums.length; ++i) {
            if (nums[i] > 0 || k == 0)
                break;
            nums[i] = -nums[i];
            --k;
        }

        return Arrays.stream(nums).sum() - (k % 2) * Arrays.stream(nums).min().getAsInt() * 2;
    }
}
```

13. 1006.java

Path: LeetCode-main/solutions/1006. Clumsy Factorial/1006.java

```
class Solution {
    public int clumsy(int n) {
        if (n == 1)
            return 1;
        if (n == 2)
            return 2;
        if (n == 3)
            return 6;
        if (n == 4)
            return 7;
        if (n % 4 == 1)
            return n + 2;
        if (n % 4 == 2)
            return n + 2;
        if (n % 4 == 3)
            return n - 1;
        return n + 1;
    }
}
```

14. 1007.java

Path: LeetCode-main/solutions/1007. Minimum Domino Rotations For Equal Row/1007.java

```
class Solution {
    public int minDominoRotations(int[] tops, int[] bottoms) {
        final int n = tops.length;
        int[] countTops = new int[7];
        int[] countBottoms = new int[7];
        int[] countBoth = new int[7];

        for (int i = 0; i < n; ++i) {
            ++countTops[tops[i]];
            ++countBottoms[bottoms[i]];
            if (tops[i] == bottoms[i])
                ++countBoth[tops[i]];
        }

        for (int i = 1; i <= 6; ++i)
            if (countTops[i] + countBottoms[i] - countBoth[i] == n)
                return n - Math.max(countTops[i], countBottoms[i]);

        return -1;
    }
}
```

15. 1008.java

Path: LeetCode-main/solutions/1008. Construct Binary Search Tree from Preorder Traversal/1008.java

```
class Solution {
    public TreeNode bstFromPreorder(int[] preorder) {
        TreeNode root = new TreeNode(preorder[0]);
        Deque<TreeNode> stack = new ArrayDeque<>(List.of(root));

        for (int i = 1; i < preorder.length; ++i) {
            TreeNode parent = stack.peek();
            TreeNode child = new TreeNode(preorder[i]);
            // Adjust the parent.
            while (!stack.isEmpty() && stack.peek().val < child.val)
                parent = stack.pop();
            // Create parent-child link according to BST property.
            if (parent.val > child.val)
                parent.left = child;
            else
                parent.right = child;
            stack.push(child);
        }

        return root;
    }
}
```

16. 1009.java

Path: LeetCode-main/solutions/1009. Complement of Base 10 Integer/1009.java

```
class Solution {  
    public int bitwiseComplement(int n) {  
        int mask = 1;  
        while (mask < n)  
            mask = (mask << 1) + 1;  
        return mask ^ n;  
    }  
}
```


17. 101.java

Path: LeetCode-main/solutions/101. Symmetric Tree/101.java

```
class Solution {
    public boolean isSymmetric(TreeNode root) {
        return isSymmetric(root, root);
    }

    private boolean isSymmetric(TreeNode p, TreeNode q) {
        if (p == null || q == null)
            return p == q;

        return p.val == q.val && isSymmetric(p.left, q.right) && isSymmetric(p.right, q.left);
    }
}
```

18. 1010.java

Path: LeetCode-main/solutions/1010. Pairs of Songs With Total Durations Divisible by 60/1010.java

```
class Solution {
    public int numPairsDivisibleBy60(int[] time) {
        int ans = 0;
        int[] count = new int[60];

        for (int t : time) {
            t %= 60;
            ans += count[(60 - t) % 60];
            ++count[t];
        }

        return ans;
    }
}
```

19. 1011.java

Path: LeetCode-main/solutions/1011. Capacity To Ship Packages Within D Days/1011.java

```
class Solution {
    public int shipWithinDays(int[] weights, int days) {
        int l = Arrays.stream(weights).max().getAsInt();
        int r = Arrays.stream(weights).sum();

        while (l < r) {
            final int m = (l + r) / 2;
            if (shipDays(weights, m) <= days)
                r = m;
            else
                l = m + 1;
        }

        return l;
    }

    private int shipDays(int[] weights, int shipCapacity) {
        int days = 1;
        int capacity = 0;
        for (final int weight : weights)
            if (capacity + weight > shipCapacity) {
                ++days;
                capacity = weight;
            } else {
                capacity += weight;
            }
        return days;
    }
}
```

20. 1012.java

Path: LeetCode-main/solutions/1012. Numbers With Repeated Digits/1012.java

```
class Solution {
    public int numDupDigitsAtMostN(int n) {
        return n - countSpecialNumbers(n);
    }

    // Same as 2376. Count Special Integers
    private int countSpecialNumbers(int n) {
        final int digitSize = (int) Math.log10(n) + 1;
        Integer[][][] mem = new Integer[digitSize + 1][1 << 10][2];
        return count(String.valueOf(n), 0, 0, true, mem) - 1; // - 0;
    }

    // Returns the number of special integers, considering the i-th digit, where
    // `used` is the bitmask of the used digits, and `tight` indicates if the
    // current digit is tightly bound.
    private int count(final String s, int i, int used, boolean tight, Integer[][][] mem) {
        if (i == s.length())
            return 1;
        if (mem[i][used][tight ? 1 : 0] != null)
            return mem[i][used][tight ? 1 : 0];

        int res = 0;
        final int maxDigit = tight ? s.charAt(i) - '0' : 9;

        for (int d = 0; d <= maxDigit; ++d) {
            // `d` is used.
            if ((used >> d & 1) == 1)
                continue;
            // Use `d` now.
            final boolean nextTight = tight && (d == maxDigit);
            if (used == 0 && d == 0) // Don't count leading 0s as used.
                res += count(s, i + 1, used, nextTight, mem);
            else
                res += count(s, i + 1, used | 1 << d, nextTight, mem);
        }

        return mem[i][used][tight ? 1 : 0] = res;
    }
}
```

21. 1013.java

Path: LeetCode-main/solutions/1013. Partition Array Into Three Parts With Equal Sum/1013.java

```
class Solution {
    public boolean canThreePartsEqualSum(int[] arr) {
        final int sum = Arrays.stream(arr).sum();
        if (sum % 3 != 0)
            return false;

        final int average = sum / 3;
        int partCount = 0;
        int partSum = 0;

        for (final int a : arr) {
            partSum += a;
            if (partSum == average) {
                ++partCount;
                partSum = 0;
            }
        }

        // edge case: arr = [0, 0, 0, 0] -> partCount = 4.
        return partCount >= 3;
    }
}
```

22. 1014.java

Path: LeetCode-main/solutions/1014. Best Sightseeing Pair/1014.java

```
class Solution {
    public int maxScoreSightseeingPair(int[] values) {
        int ans = 0;
        int bestPrev = 0;

        for (final int value : values) {
            ans = Math.max(ans, value + bestPrev);
            bestPrev = Math.max(bestPrev, value) - 1;
        }

        return ans;
    }
}
```

23. 1015.java

Path: LeetCode-main/solutions/1015. Smallest Integer Divisible by K/1015.java

```
class Solution {
    public int smallestRepunitDivByK(int k) {
        if (k % 10 != 1 && k % 10 != 3 && k % 10 != 7 && k % 10 != 9)
            return -1;

        Set<Integer> seen = new HashSet<>();
        int n = 0;

        for (int length = 1; length <= k; ++length) {
            n = (n * 10 + 1) % k;
            if (n == 0)
                return length;
            if (seen.contains(n))
                return -1;
            seen.add(n);
        }

        return -1;
    }
}
```

24. 1016.java

Path: LeetCode-main/solutions/1016. Binary String With Substrings Representing 1 To N/1016.java

```
class Solution {
    public boolean queryString(String s, int n) {
        if (n > 1511)
            return false;

        for (int i = n; i > n / 2; --i)
            if (!s.contains(Integer.toBinaryString(i)))
                return false;

        return true;
    }
}
```


25. 1017.java

Path: LeetCode-main/solutions/1017. Convert to Base -2/1017.java

```
class Solution {
    public String baseNeg2(int n) {
        StringBuilder sb = new StringBuilder();

        while (n != 0) {
            sb.append(n % 2);
            n = -(n >> 1);
        }

        return sb.length() > 0 ? sb.reverse().toString() : "0";
    }
}
```

26. 1018.java

Path: LeetCode-main/solutions/1018. Binary Prefix Divisible By 5/1018.java

```
class Solution {
    public List<Boolean> prefixesDivBy5(int[] nums) {
        List<Boolean> ans = new ArrayList<>();
        int curr = 0;

        for (final int num : nums) {
            curr = (curr * 2 + num) % 5;
            ans.add(curr % 5 == 0);
        }

        return ans;
    }
}
```

27. 1019.java

Path: LeetCode-main/solutions/1019. Next Greater Node In Linked List/1019.java

```
class Solution {
    public int[] nextLargerNodes(ListNode head) {
        List<Integer> ans = new ArrayList<>();
        Deque<Integer> stack = new ArrayDeque<>();

        for (; head != null; head = head.next) {
            while (!stack.isEmpty() && head.val > ans.get(stack.peek())) {
                int index = stack.pop();
                ans.set(index, head.val);
            }
            stack.push(ans.size());
            ans.add(head.val);
        }

        while (!stack.isEmpty())
            ans.set(stack.pop(), 0);

        return ans.stream().mapToInt(Integer::intValue).toArray();
    }
}
```

28. 102.java

Path: LeetCode-main/solutions/102. Binary Tree Level Order Traversal/102.java

```
class Solution {
    public List<List<Integer>> levelOrder(TreeNode root) {
        if (root == null)
            return new ArrayList<>();

        List<List<Integer>> ans = new ArrayList<>();
        Queue<TreeNode> q = new ArrayDeque<>(List.of(root));

        while (!q.isEmpty()) {
            List<Integer> currLevel = new ArrayList<>();
            for (int sz = q.size(); sz > 0; --sz) {
                TreeNode node = q.poll();
                currLevel.add(node.val);
                if (node.left != null)
                    q.offer(node.left);
                if (node.right != null)
                    q.offer(node.right);
            }
            ans.add(currLevel);
        }

        return ans;
    }
}
```

29. 1020.java

Path: LeetCode-main/solutions/1020. Number of Enclaves/1020.java

```
class Solution {
    public int numEnclaves(int[][] grid) {
        final int m = grid.length;
        final int n = grid[0].length;

        // Remove the lands connected to the edge.
        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j)
                if (i * j == 0 || i == m - 1 || j == n - 1)
                    if (grid[i][j] == 1)
                        dfs(grid, i, j);

        return Arrays.stream(grid).flatMapToInt(Arrays::stream).sum();
    }

    private void dfs(int[][] grid, int i, int j) {
        if (i < 0 || i == grid.length || j < 0 || j == grid[0].length)
            return;
        if (grid[i][j] == 0)
            return;
        grid[i][j] = 0;
        dfs(grid, i + 1, j);
        dfs(grid, i - 1, j);
        dfs(grid, i, j + 1);
        dfs(grid, i, j - 1);
    }
}
```

30. 1021.java

Path: LeetCode-main/solutions/1021. Remove Outermost Parentheses/1021.java

```
class Solution {
    public String removeOuterParentheses(String s) {
        StringBuilder sb = new StringBuilder();
        int opened = 0;

        for (final char c : s.toCharArray())
            if (c == '(') {
                if (++opened > 1)
                    sb.append(c);
            } else if (--opened > 0) { // c == ')'
                sb.append(c);
            }

        return sb.toString();
    }
}
```

31. 1022.java

Path: LeetCode-main/solutions/1022. Sum of Root To Leaf Binary Numbers/1022.java

```
class Solution {
    public int sumRootToLeaf(TreeNode root) {
        dfs(root, 0);
        return ans;
    }

    private int ans = 0;

    private void dfs(TreeNode root, int val) {
        if (root == null)
            return;
        val = val * 2 + root.val;
        if (root.left == null && root.right == null)
            ans += val;
        dfs(root.left, val);
        dfs(root.right, val);
    }
}
```

32. 1023.java

Path: LeetCode-main/solutions/1023. Camelcase Matching/1023.java

```
class Solution {
    public List<Boolean> camelMatch(String[] queries, String pattern) {
        List<Boolean> ans = new ArrayList<>();
        for (final String query : queries)
            ans.add(isMatch(query, pattern));
        return ans;
    }

    private boolean isMatch(final String query, final String pattern) {
        int j = 0;
        for (final char c : query.toCharArray())
            if (j < pattern.length() && c == pattern.charAt(j))
                ++j;
            else if (Character.isUpperCase(c))
                return false;
        return j == pattern.length();
    }
}
```


33. 1024.java

Path: LeetCode-main/solutions/1024. Video Stitching/1024.java

```
class Solution {
    public int videoStitching(int[][] clips, int time) {
        int ans = 0;
        int end = 0;
        int farthest = 0;

        Arrays.sort(clips, Comparator.comparingInt(clip -> clip[0]));

        int i = 0;
        while (farthest < time) {
            while (i < clips.length && clips[i][0] <= end)
                farthest = Math.max(farthest, clips[i++][1]);
            if (end == farthest)
                return -1;
            ++ans;
            end = farthest;
        }
        return ans;
    }
}
```

34. 1025.java

Path: LeetCode-main/solutions/1025. Divisor Game/1025.java

```
class Solution {  
    public boolean divisorGame(int n) {  
        return n % 2 == 0;  
    }  
}
```

35. 1026.java

Path: LeetCode-main/solutions/1026. Maximum Difference Between Node and Ancestor/1026.java

```
class Solution {
    public int maxAncestorDiff(TreeNode root) {
        return maxAncestorDiff(root, root.val, root.val);
    }

    // Returns |the maximum - the minimum| of the tree.
    private int maxAncestorDiff(TreeNode root, int mn, int mx) {
        if (root == null)
            return 0;
        mn = Math.min(mn, root.val);
        mx = Math.max(mx, root.val);
        final int l = maxAncestorDiff(root.left, mn, mx);
        final int r = maxAncestorDiff(root.right, mn, mx);
        return Math.max(mx - mn, Math.max(l, r));
    }
}
```

36. 1027.java

Path: LeetCode-main/solutions/1027. Longest Arithmetic Subsequence/1027.java

```
class Solution {
    public int longestArithSeqLength(int[] nums) {
        final int n = nums.length;
        int ans = 0;
        // dp[i][k] := the length of the longest arithmetic subsequence of nums[0..i]
        // with k = diff + 500
        int[][] dp = new int[n][1001];

        for (int i = 0; i < n; ++i)
            for (int j = 0; j < i; ++j) {
                final int k = nums[i] - nums[j] + 500;
                dp[i][k] = Math.max(2, dp[j][k] + 1);
                ans = Math.max(ans, dp[i][k]);
            }

        return ans;
    }
}
```

37. 1028.java

Path: LeetCode-main/solutions/1028. Recover a Tree From Preorder Traversal/1028.java

```
class Solution {
    public TreeNode recoverFromPreorder(String traversal) {
        return recoverFromPreorder(traversal, 0);
    }

    private int i = 0;

    private TreeNode recoverFromPreorder(final String traversal, int depth) {
        int nDashes = 0;
        while (i + nDashes < traversal.length() && traversal.charAt(i + nDashes) == '-')
            ++nDashes;
        if (nDashes != depth)
            return null;

        i += depth;
        final int start = i;
        while (i < traversal.length() && Character.isDigit(traversal.charAt(i)))
            ++i;

        return new TreeNode(Integer.valueOf(traversal.substring(start, i)),
                             recoverFromPreorder(traversal, depth + 1),
                             recoverFromPreorder(traversal, depth + 1));
    }
}
```

38. 1029.java

Path: LeetCode-main/solutions/1029. Two City Scheduling/1029.java

```
class Solution {
    public int twoCitySchedCost(int[][] costs) {
        final int n = costs.length / 2;
        int ans = 0;

        // How much money can we save if we fly a person to A instead of B?
        // To save money, we should
        // 1. Fly the person with the maximum saving to A.
        // 2. Fly the person with the minimum saving to B.

        // Sort `costs` in ascending order by the money saved if we fly a person to
        // B instead of A.
        Arrays.sort(costs, Comparator.comparingInt(cost -> cost[0] - cost[1]));

        for (int i = 0; i < n; ++i)
            ans += costs[i][0] + costs[i + n][1];

        return ans;
    }
}
```

39. 103-2.java

Path: LeetCode-main/solutions/103. Binary Tree Zigzag Level Order Traversal/103-2.java

```
class Solution {
    public List<List<Integer>> zigzagLevelOrder(TreeNode root) {
        if (root == null)
            return new ArrayList<>();

        List<List<Integer>> ans = new ArrayList<>();
        Deque<TreeNode> q = new ArrayDeque<>(List.of(root));
        boolean isLeftToRight = true;

        while (!q.isEmpty()) {
            List<Integer> currLevel = new ArrayList<>();
            for (int sz = q.size(); sz > 0; --sz) {
                TreeNode node = q.poll();
                if (isLeftToRight)
                    currLevel.add(node.val);
                else
                    currLevel.add(0, node.val);
                if (node.left != null)
                    q.offer(node.left);
                if (node.right != null)
                    q.offer(node.right);
            }
            ans.add(currLevel);
            isLeftToRight = !isLeftToRight;
        }

        return ans;
    }
}
```

40. 103.java

Path: LeetCode-main/solutions/103. Binary Tree Zigzag Level Order Traversal/103.java

```
class Solution {
    public List<List<Integer>> zigzagLevelOrder(TreeNode root) {
        if (root == null)
            return new ArrayList<>();

        List<List<Integer>> ans = new ArrayList<>();
        Deque<TreeNode> dq = new ArrayDeque<>(List.of(root));
        boolean isLeftToRight = true;

        while (!dq.isEmpty()) {
            List<Integer> currLevel = new ArrayList<>();
            for (int sz = dq.size(); sz > 0; --sz)
                if (isLeftToRight) {
                    TreeNode node = dq.pollFirst();
                    currLevel.add(node.val);
                    if (node.left != null)
                        dq.offerLast(node.left);
                    if (node.right != null)
                        dq.offerLast(node.right);
                } else {
                    TreeNode node = dq.pollLast();
                    currLevel.add(node.val);
                    if (node.right != null)
                        dq.offerFirst(node.right);
                    if (node.left != null)
                        dq.offerFirst(node.left);
                }
            ans.add(currLevel);
            isLeftToRight = !isLeftToRight;
        }

        return ans;
    }
}
```


41. 1030.java

Path: LeetCode-main/solutions/1030. Matrix Cells in Distance Order/1030.java

```
class Solution {
    public int[][] allCellsDistOrder(int rows, int cols, int rCenter, int cCenter) {
        final int[][] DIRS = {{0, 1}, {1, 0}, {0, -1}, {-1, 0}};
        List<int[]> ans = new ArrayList<>();
        boolean[][] seen = new boolean[rows][cols];
        Queue<Pair<Integer, Integer>> q = new LinkedList<>(Arrays.asList(new Pair<>(rCenter, cCenter)));
        seen[rCenter][cCenter] = true;

        while (!q.isEmpty()) {
            final int i = q.peek().getKey();
            final int j = q.poll().getValue();
            ans.add(new int[] {i, j});
            for (int[] dir : DIRS) {
                final int x = i + dir[0];
                final int y = j + dir[1];
                if (x < 0 || x == rows || y < 0 || y == cols)
                    continue;
                if (seen[x][y])
                    continue;
                seen[x][y] = true;
                q.offer(new Pair<>(x, y));
            }
        }

        return ans.toArray(int[][] ::new);
    }
}
```

42. 1031.java

Path: LeetCode-main/solutions/1031. Maximum Sum of Two Non-Overlapping Subarrays/1031.java

```
class Solution {
    public int maxSumTwoNoOverlap(int[] nums, int firstLen, int secondLen) {
        return Math.max(helper(nums, firstLen, secondLen), helper(nums, secondLen, firstLen));
    }

    private int helper(int[] nums, int l, int r) {
        final int n = nums.length;
        int[] left = new int[n];
        int sum = 0;

        for (int i = 0; i < n; ++i) {
            sum += nums[i];
            if (i >= l)
                sum -= nums[i - l];
            if (i >= l - 1)
                left[i] = i > 0 ? Math.max(left[i - 1], sum) : sum;
        }

        int[] right = new int[n];
        sum = 0;

        for (int i = n - 1; i >= 0; --i) {
            sum += nums[i];
            if (i <= n - r - 1)
                sum -= nums[i + r];
            if (i <= n - r)
                right[i] = i < n - 1 ? Math.max(right[i + 1], sum) : sum;
        }

        int ans = 0;

        for (int i = 0; i < n - 1; ++i)
            ans = Math.max(ans, left[i] + right[i + 1]);

        return ans;
    }
}
```

43. 1032.java

Path: LeetCode-main/solutions/1032. Stream of Characters/1032.java

```
class TrieNode {
    public TrieNode[] children = new TrieNode[26];
    public boolean isWord = false;
}

class StreamChecker {
    public StreamChecker(String[] words) {
        for (final String word : words)
            insert(word);
    }

    public boolean query(char letter) {
        letters.append(letter);
        TrieNode node = root;

        for (int i = letters.length() - 1; i >= 0; --i) {
            final int index = letters.charAt(i) - 'a';
            if (node.children[index] == null)
                return false;
            node = node.children[index];
            if (node.isWord)
                return true;
        }

        return false;
    }

    private TrieNode root = new TrieNode();
    private StringBuilder letters = new StringBuilder();

    private void insert(final String word) {
        TrieNode node = root;
        for (int i = word.length() - 1; i >= 0; --i) {
            final int index = word.charAt(i) - 'a';
            if (node.children[index] == null)
                node.children[index] = new TrieNode();
            node = node.children[index];
        }
        node.isWord = true;
    }
}
```

44. 1033.java

Path: LeetCode-main/solutions/1033. Moving Stones Until Consecutive/1033.java

```
class Solution {
    public int[] numMovesStones(int a, int b, int c) {
        int[] nums = new int[] {a, b, c};

        Arrays.sort(nums);

        if (nums[2] - nums[0] == 2)
            return new int[] {0, 0};
        return new int[] {Math.min(nums[1] - nums[0], nums[2] - nums[1]) <= 2 ? 1 : 2,
            nums[2] - nums[0] - 2};
    }
}
```

45. 1034.java

Path: LeetCode-main/solutions/1034. Coloring A Border/1034.java

```
class Solution {
    public int[][] colorBorder(int[][] grid, int r0, int c0, int color) {
        dfs(grid, r0, c0, grid[r0][c0]);

        for (int i = 0; i < grid.length; ++i)
            for (int j = 0; j < grid[0].length; ++j)
                if (grid[i][j] < 0)
                    grid[i][j] = color;

        return grid;
    }

    private void dfs(int[][] grid, int i, int j, int startColor) {
        if (i < 0 || i == grid.length || j < 0 || j == grid[0].length)
            return;
        if (grid[i][j] != startColor)
            return;

        grid[i][j] = -startColor;
        dfs(grid, i + 1, j, startColor);
        dfs(grid, i - 1, j, startColor);
        dfs(grid, i, j + 1, startColor);
        dfs(grid, i, j - 1, startColor);

        // If this cell is already on the boarder, it must be painted later.
        if (i == 0 || i == grid.length - 1 || j == 0 || j == grid[0].length - 1)
            return;

        if (Math.abs(grid[i + 1][j]) == startColor && //
            Math.abs(grid[i - 1][j]) == startColor && //
            Math.abs(grid[i][j + 1]) == startColor && //
            Math.abs(grid[i][j - 1]) == startColor)
            grid[i][j] = startColor;
    }
}
```

46. 1035.java

Path: LeetCode-main/solutions/1035. Uncrossed Lines/1035.java

```
class Solution {
    public int maxUncrossedLines(int[] nums1, int[] nums2) {
        final int m = nums1.length;
        final int n = nums2.length;
        int[][] dp = new int[m + 1][n + 1];

        for (int i = 1; i <= m; ++i)
            for (int j = 1; j <= n; ++j)
                dp[i][j] = nums1[i - 1] == nums2[j - 1] ? dp[i - 1][j - 1] + 1
                    : Math.max(dp[i - 1][j], dp[i][j - 1]);

        return dp[m][n];
    }
}
```

47. 1036.java

Path: LeetCode-main/solutions/1036. Escape a Large Maze/1036.java

```
class Solution {
    public boolean isEscapePossible(int[][] blocked, int[] source, int[] target) {
        Set<Long> blockedSet = new HashSet<>();
        for (int[] b : blocked)
            blockedSet.add(hash(b[0], b[1]));

        return dfs(blockedSet, source[0], source[1], hash(target[0], target[1]), new HashSet<>()) &&
            dfs(blockedSet, target[0], target[1], hash(source[0], source[1]), new HashSet<>());
    }

    private boolean dfs(Set<Long> blockedSet, int i, int j, long target, Set<Long> seen) {
        if (i < 0 || i >= 1e6 || j < 0 || j >= 1e6 || blockedSet.contains(hash(i, j)) ||
            seen.contains(hash(i, j)))
            return false;

        seen.add(hash(i, j));
        if (seen.size() > (1 + 199) * 199 / 2 || hash(i, j) == target)
            return true;

        return
            dfs(blockedSet, i + 1, j, target, seen) || //
            dfs(blockedSet, i - 1, j, target, seen) || //
            dfs(blockedSet, i, j + 1, target, seen) || //
            dfs(blockedSet, i, j - 1, target, seen);
    }

    private long hash(int i, int j) {
        return ((long) i << 16) + j;
    }
}
```

48. 1037.java

Path: LeetCode-main/solutions/1037. Valid Boomerang/1037.java

```
class Solution {  
    public boolean isBoomerang(int[][] points) {  
        return  
            (points[1][0] - points[0][0]) * (points[2][1] - points[1][1]) != //  
            (points[1][1] - points[0][1]) * (points[2][0] - points[1][0]);  
    }  
}
```


49. 1039.java

Path: LeetCode-main/solutions/1039. Minimum Score Triangulation of Polygon/1039.java

```
class Solution {
    public int minScoreTriangulation(int[] values) {
        final int n = values.length;
        int[][] dp = new int[n][n];

        for (int j = 2; j < n; ++j)
            for (int i = j - 2; i >= 0; --i) {
                dp[i][j] = Integer.MAX_VALUE;
                for (int k = i + 1; k < j; ++k)
                    dp[i][j] = Math.min(dp[i][j], dp[i][k] + values[i] * values[k] * values[j] + dp[k][j]);
            }

        return dp[0][n - 1];
    }
}
```

50. 104.java

Path: LeetCode-main/solutions/104. Maximum Depth of Binary Tree/104.java

```
class Solution {  
    public int maxDepth(TreeNode root) {  
        if (root == null)  
            return 0;  
        return 1 + Math.max(maxDepth(root.left), maxDepth(root.right));  
    }  
}
```

51. 1040.java

Path: LeetCode-main/solutions/1040. Moving Stones Until Consecutive II/1040.java

```
class Solution {
    public int[] numMovesStonesII(int[] stones) {
        final int n = stones.length;
        int minMoves = n;

        Arrays.sort(stones);

        for (int l = 0, r = 0; r < n; ++r) {
            while (stones[r] - stones[l] + 1 > n)
                ++l;
            int alreadyStored = r - l + 1;
            if (alreadyStored == n - 1 && stones[r] - stones[l] + 1 == n - 1)
                minMoves = Math.min(minMoves, 2);
            else
                minMoves = Math.min(minMoves, n - alreadyStored);
        }

        return new int[] {
            minMoves, Math.max(stones[n - 1] - stones[1] - n + 2, stones[n - 2] - stones[0] - n + 2)};
    }
}
```

52. 1041.java

Path: LeetCode-main/solutions/1041. Robot Bounded In Circle/1041.java

```
class Solution {
    public boolean isRobotBounded(String instructions) {
        int x = 0;
        int y = 0;
        int d = 0;
        int[][] directions = new int[][] {{0, 1}, {1, 0}, {0, -1}, {-1, 0}};

        for (char instruction : instructions.toCharArray()) {
            if (instruction == 'G') {
                x += directions[d][0];
                y += directions[d][1];
            } else if (instruction == 'L')
                d = (d + 3) % 4;
            else
                d = (d + 1) % 4;
        }

        return x == 0 && y == 0 || d > 0;
    }
}
```

53. 1042.java

Path: LeetCode-main/solutions/1042. Flower Planting With No Adjacent/1042.java

```
class Solution {
    public int[] gardenNoAdj(int n, int[][] paths) {
        int[] ans = new int[n]; // ans[i] := 1, 2, 3, or 4
        List<Integer>[] graph = new List[n];
        Arrays.setAll(graph, i -> new ArrayList<>());

        for (int[] path : paths) {
            final int u = path[0] - 1;
            final int v = path[1] - 1;
            graph[u].add(v);
            graph[v].add(u);
        }

        for (int u = 0; u < n; ++u) {
            int used = 0;
            for (final int v : graph[u])
                used |= 1 << ans[v];
            ans[u] = getFirstUnusedType(used);
        }

        return ans;
    }

    private int getFirstUnusedType(int used) {
        for (int type = 1; type <= 4; ++type)
            if ((used >> type & 1) == 0)
                return type;
        throw new IllegalArgumentException();
    }
}
```

54. 1043.java

Path: LeetCode-main/solutions/1043. Partition Array for Maximum Sum/1043.java

```
class Solution {
    public int maxSumAfterPartitioning(int[] arr, int k) {
        final int n = arr.length;
        int[] dp = new int[n + 1];

        for (int i = 1; i <= n; ++i) {
            int mx = Integer.MIN_VALUE;
            for (int j = 1; j <= Math.max(i, k); ++j) {
                mx = Math.max(mx, arr[i - j]);
                dp[i] = Math.max(dp[i], dp[i - j] + mx * j);
            }
        }

        return dp[n];
    }
}
```

55. 1044.java

Path: LeetCode-main/solutions/1044. Longest Duplicate Substring/1044.java

```
class Solution {
    public String longestDupSubString(String s) {
        final int n = s.length();
        int[] pows = new int[n];
        int bestStart = -1;
        int l = 1;
        int r = n;

        pows[0] = 1;
        for (int i = 1; i < n; ++i)
            pows[i] = (int) (pows[i - 1] * BASE % HASH);

        while (l < r) {
            final int m = (l + r) / 2;
            final int start = getStart(s, m, pows);
            if (start == -1) {
                r = m;
            } else {
                bestStart = start;
                l = m + 1;
            }
        }

        if (bestStart == -1)
            return "";
        if (getStart(s, l, pows) == -1)
            return s.substring(bestStart, bestStart + l - 1);
        return s.substring(bestStart, bestStart + l);
    }

    private static final long BASE = 26;
    private static final long HASH = 1_000_000_007;

    private static int val(char c) {
        return c - 'a';
    }

    // k := the length of the substring to be hashed
    private int getStart(final String s, int k, int[] pows) {
        Map<Long, List<Integer>> hashToStarts = new HashMap<>();
        long h = 0;

        // Compute the hash value of s[:k].
        for (int i = 0; i < k; ++i)
            h = (h * BASE % HASH + val(s.charAt(i))) % HASH;
        hashToStarts.put(h, new ArrayList<>());
        hashToStarts.get(h).add(0);

        // Compute the rolling hash by Rabin Karp.
        for (int i = k; i < s.length(); ++i) {
            final int startIndex = i - k + 1;
            h = ((h - (long) (pows[k - 1]) * val(s.charAt(i - k))) % HASH + HASH) % HASH;
            h = (h * BASE + val(s.charAt(i))) % HASH;
            if (hashToStarts.containsKey(h)) {
                final String currSub = s.substring(startIndex, startIndex + k);
                for (final int start : hashToStarts.get(h))
                    if (s.substring(start, start + k).equals(currSub))
                        return start;
            }
            hashToStarts.put(h, new ArrayList<>());
            hashToStarts.get(h).add(startIndex);
        }

        return -1;
    }
}
```

56. 1046.java

Path: LeetCode-main/solutions/1046. Last Stone Weight/1046.java

```
class Solution {
    public int lastStoneWeight(int[] stones) {
        Queue<Integer> pq = new PriorityQueue<>(Collections.reverseOrder());

        for (final int stone : stones)
            pq.offer(stone);

        while (pq.size() >= 2) {
            final int n1 = pq.poll();
            final int n2 = pq.poll();
            if (n1 != n2)
                pq.offer(n1 - n2);
        }

        return pq.isEmpty() ? 0 : pq.peek();
    }
}
```


57. 1047.java

Path: LeetCode-main/solutions/1047. Remove All Adjacent Duplicates In String/1047.java

```
class Solution {
    public String removeDuplicates(final String S) {
        StringBuilder sb = new StringBuilder();

        for (final char c : S.toCharArray()) {
            final int n = sb.length();
            if (n > 0 && sb.charAt(n - 1) == c)
                sb.deleteCharAt(n - 1);
            else
                sb.append(c);
        }

        return sb.toString();
    }
}
```

58. 1048-2.java

Path: LeetCode-main/solutions/1048. Longest String Chain/1048-2.java

```
class Solution {
    public int longestStrChain(String[] words) {
        int ans = 0;
        Map<String, Integer> dp = new HashMap<>();

        Arrays.sort(words, Comparator.comparingInt(String::length));

        for (final String word : words) {
            int bestLength = 0;
            for (int i = 0; i < word.length(); ++i) {
                final String pred = word.substring(0, i) + word.substring(i + 1);
                bestLength = Math.max(bestLength, dp.getOrDefault(pred, 0) + 1);
            }
            dp.put(word, bestLength);
            ans = Math.max(ans, bestLength);
        }

        return ans;
    }
}
```

59. 1048.java

Path: LeetCode-main/solutions/1048. Longest String Chain/1048.java

```
class Solution {
    public int longestStrChain(String[] words) {
        Set<String> wordsSet = new HashSet<>(Arrays.asList(words));
        int ans = 0;

        for (final String word : words)
            ans = Math.max(ans, longestStrChain(word, wordsSet));

        return ans;
    }
    // dp[s] := the longest string chain, where s is the last word
    private Map<String, Integer> dp = new HashMap<>();

    private int longestStrChain(final String s, Set<String> wordsSet) {
        if (dp.containsKey(s))
            return dp.get(s);

        int ans = 1;

        for (int i = 0; i < s.length(); ++i) {
            final String pred = s.substring(0, i) + s.substring(i + 1);
            if (wordsSet.contains(pred))
                ans = Math.max(ans, longestStrChain(pred, wordsSet) + 1);
        }

        dp.put(s, ans);
        return ans;
    }
}
```

60. 1049.java

Path: LeetCode-main/solutions/1049. Last Stone Weight II/1049.java

```
class Solution {
    public int lastStoneWeightII(int[] stones) {
        final int sum = Arrays.stream(stones).sum();
        boolean[] dp = new boolean[sum + 1];
        dp[0] = true;
        int s = 0;

        for (int stone : stones)
            for (int w = sum / 2; w > 0; --w) {
                if (w >= stone)
                    dp[w] = dp[w] || dp[w - stone];
                if (dp[w])
                    s = Math.max(s, w);
            }

        return sum - 2 * s;
    }
}
```

61. 105.java

Path: LeetCode-main/solutions/105. Construct Binary Tree from Preorder and Inorder Traversal/105.java

```
class Solution {
    public TreeNode buildTree(int[] preorder, int[] inorder) {
        Map<Integer, Integer> inToIndex = new HashMap<>();

        for (int i = 0; i < inorder.length; ++i)
            inToIndex.put(inorder[i], i);

        return build(preorder, 0, preorder.length - 1, inorder, 0, inorder.length - 1, inToIndex);
    }

    private TreeNode build(int[] preorder, int preStart, int preEnd, int[] inorder, int inStart,
                           int inEnd, Map<Integer, Integer> inToIndex) {
        if (preStart > preEnd)
            return null;

        final int rootVal = preorder[preStart];
        final int rootInIndex = inToIndex.get(rootVal);
        final int leftSize = rootInIndex - inStart;

        TreeNode root = new TreeNode(rootVal);
        root.left = build(preorder, preStart + 1, preStart + leftSize, inorder, inStart,
                          rootInIndex - 1, inToIndex);
        root.right = build(preorder, preStart + leftSize + 1, preEnd, inorder, rootInIndex + 1, inEnd,
                           inToIndex);

        return root;
    }
}
```

62. 1051.java

Path: LeetCode-main/solutions/1051. Height Checker/1051.java

```
class Solution {
    public int heightChecker(int[] heights) {
        int ans = 0;
        int currentHeight = 1;
        int[] count = new int[101];

        for (int height : heights)
            ++count[height];

        for (int height : heights) {
            while (count[currentHeight] == 0)
                ++currentHeight;
            if (height != currentHeight)
                ++ans;
            --count[currentHeight];
        }

        return ans;
    }
}
```

63. 1052.java

Path: LeetCode-main/solutions/1052. Grumpy Bookstore Owner/1052.java

```
class Solution {
    public int maxSatisfied(int[] customers, int[] grumpy, int X) {
        int satisfied = 0;
        int madeSatisfied = 0;
        int windowSatisfied = 0;

        for (int i = 0; i < customers.length; ++i) {
            if (grumpy[i] == 0)
                satisfied += customers[i];
            else
                windowSatisfied += customers[i];
            if (i >= X && grumpy[i - X] == 1)
                windowSatisfied -= customers[i - X];
            madeSatisfied = Math.max(madeSatisfied, windowSatisfied);
        }

        return satisfied + madeSatisfied;
    }
}
```

64. 1053.java

Path: LeetCode-main/solutions/1053. Previous Permutation With One Swap/1053.java

```
class Solution {
    public int[] prevPermOpt1(int[] arr) {
        final int n = arr.length;
        int l = n - 2;
        int r = n - 1;

        while (l >= 0 && arr[l] <= arr[l + 1])
            l--;
        if (l < 0)
            return arr;
        while (arr[r] >= arr[l] || arr[r] == arr[r - 1])
            r--;
        swap(arr, l, r);

        return arr;
    }

    private void swap(int[] arr, int l, int r) {
        int temp = arr[l];
        arr[l] = arr[r];
        arr[r] = temp;
    }
}
```


65. 1054.java

Path: LeetCode-main/solutions/1054. Distant Barcodes/1054.java

```
class Solution {
    public int[] rearrangeBarcodes(int[] barcodes) {
        int[] ans = new int[barcodes.length];
        int[] count = new int[10001];
        int maxCount = 0;
        int maxNum = 0;

        for (final int b : barcodes)
            ++count[b];

        for (int i = 1; i < 10001; ++i)
            if (count[i] > maxCount) {
                maxCount = count[i];
                maxNum = i;
            }

        fillAns(ans, count, maxNum, barcodes.length);
        for (int num = 1; num < 10001; ++num)
            fillAns(ans, count, num, barcodes.length);

        return ans;
    }

    private int i = 0; // ans' index

    private void fillAns(int[] ans, int[] count, int num, int n) {
        while (count[num]-- > 0) {
            ans[i] = num;
            i = i + 2 < n ? i + 2 : 1;
        }
    }
}
```

66. 1055-2.java

Path: LeetCode-main/solutions/1055. Shortest Way to Form String/1055-2.java

```
class Solution {
    public int shortestWay(String source, String target) {
        final int m = source.length();
        final int n = target.length();
        // dp[i][c] := the earliest index >= i s.t. source[index] = c
        // dp[i][c] := -1 if c isn't in the source
        int[][] dp = new int[m][26];

        Arrays.stream(dp).forEach(A -> Arrays.fill(A, -1));

        dp[m - 1][source.charAt(m - 1) - 'a'] = m - 1;
        for (int i = m - 2; i >= 0; --i) {
            dp[i] = dp[i + 1].clone();
            dp[i][source.charAt(i) - 'a'] = i;
        }

        int ans = 0;
        int i = 0; // source's index

        for (final char c : target.toCharArray()) {
            if (dp[0][c - 'a'] == -1)
                return -1;
            // If there are no c's left in source that occur more than i times but
            // there are c's from earlier in the subsequence, add 1 to subsequence
            // count and reset source's index to 0.
            if (dp[i][c - 'a'] == -1) {
                ++ans;
                i = 0;
            }
            // Continue taking letters from the subsequence.
            i = dp[i][c - 'a'] + 1;
            if (i == m) {
                ++ans;
                i = 0;
            }
        }

        return ans + (i == 0 ? 0 : 1);
    }
}
```

67. 1055.java

Path: LeetCode-main/solutions/1055. Shortest Way to Form String/1055.java

```
class Solution {
    public int shortestWay(String source, String target) {
        int ans = 0;

        for (int i = 0; i < target.length(); i) {
            final int prevIndex = i;
            for (int j = 0; j < source.length(); ++j)
                if (i < target.length() && source.charAt(j) == target.charAt(i))
                    ++i;
            // All chars in source didn't match target[i].
            if (i == prevIndex)
                return -1;
            ++ans;
        }
        return ans;
    }
}
```

68. 1056.java

Path: LeetCode-main/solutions/1056. Confusing Number/1056.java

```
class Solution {
    public boolean confusingNumber(int n) {
        final String s = String.valueOf(n);
        final char[] rotated = {'0', '1', 'x', 'x', 'x', 'x', '9', 'x', '8', '6'};
        StringBuilder sb = new StringBuilder();

        for (int i = s.length() - 1; i >= 0; --i) {
            if (rotated[s.charAt(i) - '0'] == 'x')
                return false;
            sb.append(rotated[s.charAt(i) - '0']);
        }

        return !sb.toString().equals(s);
    }
}
```

69. 1057.java

Path: LeetCode-main/solutions/1057. Campus Bikes/1057.java

```
class Solution {
    public int[] assignBikes(int[][] workers, int[][] bikes) {
        final int n = workers.length;
        final int m = bikes.length;
        int[] ans = new int[n];
        boolean[] usedBikes = new boolean[m];
        // buckets[k] := (i, j), where k = dist(workers[i], bikes[j])
        List<Pair<Integer, Integer>>[] buckets = new List[2001];

        for (int i = 0; i < 2001; ++i)
            buckets[i] = new ArrayList<>();

        for (int i = 0; i < n; ++i)
            for (int j = 0; j < m; ++j)
                buckets[dist(workers[i], bikes[j])].add(new Pair<>(i, j));

        Arrays.fill(ans, -1);

        for (int k = 0; k < 2001; ++k)
            for (Pair<Integer, Integer> pair : buckets[k]) {
                final int i = pair.getKey();
                final int j = pair.getValue();
                if (ans[i] == -1 && !usedBikes[j]) {
                    ans[i] = j;
                    usedBikes[j] = true;
                }
            }

        return ans;
    }

    private int dist(int[] p1, int[] p2) {
        return Math.abs(p1[0] - p2[0]) + Math.abs(p1[1] - p2[1]);
    }
}
```

70. 1058.java

Path: LeetCode-main/solutions/1058. Minimize Rounding Error to Meet Target/1058.java

```
class Solution {
    public String minimizeError(String[] prices, int target) {
        // A[i] := (costCeil - costFloor, costCeil, costFloor)
        // The lower the costCeil - costFloor is, the cheaper to ceil it.
        List<double[]> A = new ArrayList<>();
        int sumFloored = 0;
        int sumCeiled = 0;

        for (final String p : prices) {
            final double price = Double.parseDouble(p);
            final int floored = (int) Math.floor(price);
            final int ceiled = (int) Math.ceil(price);
            sumFloored += floored;
            sumCeiled += ceiled;
            final double costFloor = price - (double) floored;
            final double costCeil = (double) ceiled - price;
            A.add(new double[] {costCeil - costFloor, costCeil, costFloor});
        }

        if (sumFloored > target || sumCeiled < target)
            return "-1";

        Collections.sort(A, new Comparator<double[]>() {
            @Override
            public int compare(double[] a, double[] b) {
                return Double.compare(a[0], b[0]);
            }
        });

        double sumError = 0.0;
        final int nCeiled = target - sumFloored;
        for (int i = 0; i < nCeiled; ++i)
            sumError += A.get(i)[1];
        for (int i = nCeiled; i < A.size(); ++i)
            sumError += A.get(i)[2];

        return String.format("%.3f", sumError);
    }
}
```

71. 1059.java

Path: LeetCode-main/solutions/1059. All Paths from Source Lead to Destination/1059.java

```
enum State { INIT, VISITING, VISITED }

class Solution {
    public boolean leadsToDestination(int n, int[][] edges, int source, int destination) {
        List<Integer>[] graph = new List[n];
        State[] states = new State[n];
        Arrays.setAll(graph, i -> new ArrayList<>());

        for (int[] edge : edges) {
            final int u = edge[0];
            final int v = edge[1];
            graph[u].add(v);
        }

        return acyclic(graph, source, destination, states);
    }

    private boolean acyclic(List<Integer>[] graph, int u, int dest, State[] states) {
        if (graph[u].isEmpty())
            return u == dest;
        if (states[u] == State.VISITING)
            return false;
        if (states[u] == State.VISITED)
            return true;

        states[u] = State.VISITING;
        for (final int v : graph[u])
            if (!acyclic(graph, v, dest, states))
                return false;
        states[u] = State.VISITED;

        return true;
    }
}
```

72. 106.java

Path: LeetCode-main/solutions/106. Construct Binary Tree from Inorder and Postorder Traversal/106.java

```
class Solution {
    public TreeNode buildTree(int[] inorder, int[] postorder) {
        Map<Integer, Integer> inToIndex = new HashMap<>();

        for (int i = 0; i < inorder.length; ++i)
            inToIndex.put(inorder[i], i);

        return build(inorder, 0, inorder.length - 1, postorder, 0, postorder.length - 1, inToIndex);
    }

    private TreeNode build(int[] inorder, int inStart, int inEnd, int[] postorder, int postStart, int postEnd,
        Map<Integer, Integer> inToIndex) {
        if (inStart > inEnd)
            return null;

        final int rootVal = postorder[postEnd];
        final int rootInIndex = inToIndex.get(rootVal);
        final int leftSize = rootInIndex - inStart;

        TreeNode root = new TreeNode(rootVal);
        root.left = build(inorder, inStart, rootInIndex - 1, postorder, postStart,
            postStart + leftSize - 1, inToIndex);
        root.right = build(inorder, rootInIndex + 1, inEnd, postorder, postStart + leftSize,
            postEnd - 1, inToIndex);
        return root;
    }
}
```


73. 1060.java

Path: LeetCode-main/solutions/1060. Missing Element in Sorted Array/1060.java

```
class Solution {
    public int missingElement(int[] nums, int k) {
        int l = 0;
        int r = nums.length;

        // Find the first index l s.t. nMissing(l) >= k
        while (l < r) {
            final int m = (l + r) / 2;
            if (nMissing(nums, m) >= k)
                r = m;
            else
                l = m + 1;
        }

        return nums[l - 1] + (k - nMissing(nums, l - 1));
    }

    // the number of missing numbers in [nums[0], nums[i]]
    private int nMissing(int[] nums, int i) {
        return nums[i] - nums[0] - i;
    }
}
```

74. 1061.java

Path: LeetCode-main/solutions/1061. Lexicographically Smallest Equivalent String/1061.java

```
class UnionFind {
    public UnionFind(int n) {
        id = new int[n];
        for (int i = 0; i < n; ++i)
            id[i] = i;
    }

    public void union(int u, int v) {
        final int i = find(u);
        final int j = find(v);
        if (i > j)
            id[i] = j;
        else
            id[j] = i;
    }

    public int find(int u) {
        return id[u] == u ? u : (id[u] = find(id[u]));
    }

    private int[] id;
}

class Solution {
    public String smallestEquivalentString(String s1, String s2, String baseStr) {
        StringBuilder sb = new StringBuilder();
        UnionFind uf = new UnionFind(26);

        for (int i = 0; i < s1.length(); ++i)
            uf.union(s1.charAt(i) - 'a', s2.charAt(i) - 'a');

        for (final char c : baseStr.toCharArray())
            sb.append((char) ('a' + uf.find(c - 'a')));

        return sb.toString();
    }
}
```

75. 1062.java

Path: LeetCode-main/solutions/1062. Longest Repeating Substring/1062.java

```
class Solution {
    public int longestRepeatingSubstring(String s) {
        final int n = s.length();
        int ans = 0;
        // dp[i][j] := the number of repeating characters of s[0..i) and s[0..j)
        int[][] dp = new int[n + 1][n + 1];

        for (int i = 1; i <= n; ++i)
            for (int j = i + 1; j <= n; ++j)
                if (s.charAt(i - 1) == s.charAt(j - 1)) {
                    dp[i][j] = 1 + dp[i - 1][j - 1];
                    ans = Math.max(ans, dp[i][j]);
                }

        return ans;
    }
}
```

76. 1063.java

Path: LeetCode-main/solutions/1063. Number of Valid Subarrays/1063.java

```
class Solution {
    public int validSubarrays(int[] nums) {
        // For each `num` in `nums`, each element x in the stack can be the leftmost
        // element s.t. [x, num] forms a valid subarray, so the size of the stack is
        // the number of valid subarrays ending in the current number.
        //
        // e.g. nums = [1, 3, 2]
        // num = 1, stack = [1] -> valid subarray is [1]
        // num = 3, stack = [1, 3] -> valid subarrays are [1, 3], [3]
        // num = 2, stack = [1, 2] -> valid subarrays are [1, 3, 2], [2]
        int ans = 0;
        Deque<Integer> stack = new ArrayDeque<>();

        for (final int num : nums) {
            while (!stack.isEmpty() && stack.peek() > num)
                stack.pop();
            stack.push(num);
            ans += stack.size();
        }

        return ans;
    }
}
```

77. 1064.java

Path: LeetCode-main/solutions/1064. Fixed Point/1064.java

```
class Solution {
    public int fixedPoint(int[] arr) {
        int l = 0;
        int r = arr.length - 1;

        // Since arr[i] is strictly increasing, arr[i] - i will also be increasing.
        // Therefore, binary search `arr` for the first arr[i] - i = 0.
        while (l < r) {
            final int m = (l + r) / 2;
            if (arr[m] - m >= 0)
                r = m;
            else
                l = m + 1;
        }
        return arr[l] == l ? l : -1;
    }
}
```

78. 1065.java

Path: LeetCode-main/solutions/1065. Index Pairs of a String/1065.java

```
class TrieNode {
    public TrieNode[] children = new TrieNode[26];
    public boolean isWord = false;
}

class Solution {
    public int[][] indexPairs(String text, String[] words) {
        List<int[]> ans = new ArrayList<>();
        TrieNode root = new TrieNode();

        for (final String word : words) {
            TrieNode node = root;
            for (final char c : word.toCharArray()) {
                final int i = c - 'a';
                if (node.children[i] == null)
                    node.children[i] = new TrieNode();
                node = node.children[i];
            }
            node.isWord = true;
        }

        for (int i = 0; i < text.length(); ++i) {
            TrieNode node = root;
            for (int j = i; j < text.length(); ++j) {
                final int index = text.charAt(j) - 'a';
                if (node.children[index] == null)
                    break;
                node = node.children[index];
                if (node.isWord)
                    ans.add(new int[] {i, j});
            }
        }

        return ans.toArray(int[][] ::new);
    }
}
```

79. 1066.java

Path: LeetCode-main/solutions/1066. Campus Bikes II/1066.java

```
class Solution {
    public int assignBikes(int[][] workers, int[][] bikes) {
        int[] mem = new int[1 << bikes.length];
        Arrays.fill(mem, Integer.MAX_VALUE);
        return assignBikes(workers, bikes, 0, 0, mem);
    }

    // Returns the minimum Manhattan distances to assign bikes to
    // workers[workerIndex..n), where `used` is the bitmask of the used bikes.
    private int assignBikes(int[][] workers, int[][] bikes, int workerIndex, int used, int[] mem) {
        if (workerIndex == workers.length)
            return 0;
        if (mem[used] != Integer.MAX_VALUE)
            return mem[used];

        for (int bikeIndex = 0; bikeIndex < bikes.length; bikeIndex++)
            if ((used >> bikeIndex & 1) == 0)
                mem[used] = Math.min(mem[used], dist(workers[workerIndex], bikes[bikeIndex]) +
                                     assignBikes(workers, bikes, workerIndex + 1,
                                                  used | (1 << bikeIndex), mem));

        return mem[used];
    }

    private int dist(int[] p1, int[] p2) {
        return Math.abs(p1[0] - p2[0]) + Math.abs(p1[1] - p2[1]);
    }
}
```

80. 1067.java

Path: LeetCode-main/solutions/1067. Digit Count in Range/1067.java

```
class Solution {
    public int digitsCount(int d, int low, int high) {
        return countDigit(high, d) - countDigit(low - 1, d);
    }

    private int countDigit(int n, int d) {
        int count = 0;

        for (int pow10 = 1; pow10 <= n; pow10 *= 10) {
            final int divisor = pow10 * 10;
            final int quotient = n / divisor;
            final int remainder = n % divisor;
            if (quotient > 0)
                count += quotient * pow10;
            if (d == 0)
                count -= pow10;
            if (remainder >= d * pow10)
                count += Math.min(remainder - d * pow10 + 1, pow10);
        }

        return count;
    }
}
```


81. 107.java

Path: LeetCode-main/solutions/107. Binary Tree Level Order Traversal II/107.java

```
class Solution {
    public List<List<Integer>> levelOrderBottom(TreeNode root) {
        if (root == null)
            return new ArrayList<>();

        List<List<Integer>> ans = new ArrayList<>();
        Queue<TreeNode> q = new ArrayDeque<>(List.of(root));

        while (!q.isEmpty()) {
            List<Integer> currLevel = new ArrayList<>();
            for (int sz = q.size(); sz > 0; --sz) {
                TreeNode node = q.poll();
                currLevel.add(node.val);
                if (node.left != null)
                    q.offer(node.left);
                if (node.right != null)
                    q.offer(node.right);
            }
            ans.add(currLevel);
        }

        Collections.reverse(ans);
        return ans;
    }
}
```

82. 1071-2.java

Path: LeetCode-main/solutions/1071. Greatest Common Divisor of Strings/1071-2.java

```
class Solution {
    public String gcdOfStrings(String str1, String str2) {
        if (!(str1 + str2).equals(str2 + str1))
            return "";
        final int g = gcd(str1.length(), str2.length());
        return str1.substring(0, g);
    }

    private int gcd(int a, int b) {
        return b == 0 ? a : gcd(b, a % b);
    }
}
```

83. 1071.java

Path: LeetCode-main/solutions/1071. Greatest Common Divisor of Strings/1071.java

```
class Solution {
    public String gcdOfStrings(String str1, String str2) {
        for (int sz = Math.min(str1.length(), str2.length()); sz > 0; --sz)
            if (isDivisible(str1, str2, sz))
                return str1.substring(0, sz);
        return "";
    }

    // Returns true if str1 and str2 are divisible by str1[0..sz).
    private boolean isDivisible(final String str1, final String str2, int sz) {
        if (str1.length() % sz > 0 || str2.length() % sz > 0)
            return false;
        final String gcd = str1.substring(0, sz);
        return str1.replace(gcd, "").isEmpty() && str2.replace(gcd, "").isEmpty();
    }
}
```

84. 1072.java

Path: LeetCode-main/solutions/1072. Flip Columns For Maximum Number of Equal Rows/1072.java

```
class Solution {
    public int maxEqualRowsAfterFlips(int[][] matrix) {
        final int m = matrix.length;
        final int n = matrix[0].length;
        int ans = 0;
        int[] flip = new int[n];
        Set<Integer> seen = new HashSet<>();

        for (int i = 0; i < m; ++i) {
            if (seen.contains(i))
                continue;
            int count = 0;
            for (int j = 0; j < n; ++j)
                flip[j] = 1 ^ matrix[i][j];
            for (int k = 0; k < m; ++k)
                if (Arrays.equals(matrix[k], matrix[i]) || Arrays.equals(matrix[k], flip)) {
                    seen.add(k);
                    ++count;
                }
            ans = Math.max(ans, count);
        }
        return ans;
    }
}
```

85. 1073.java

Path: LeetCode-main/solutions/1073. Adding Two Negabinary Numbers/1073.java

```
class Solution {
    public int[] addNegabinary(int[] arr1, int[] arr2) {
        Deque<Integer> ans = new ArrayDeque<>();
        int carry = 0;
        int i = arr1.length - 1;
        int j = arr2.length - 1;

        while (carry != 0 || i >= 0 || j >= 0) {
            if (i >= 0)
                carry += arr1[i--];
            if (j >= 0)
                carry += arr2[j--];
            ans.addFirst(carry & 1);
            carry = -(carry >> 1);
        }

        while (ans.size() > 1 && ans.getFirst() == 0)
            ans.pollFirst();

        return ans.stream().mapToInt(Integer::intValue).toArray();
    }
}
```

86. 1074.java

Path: LeetCode-main/solutions/1074. Number of Submatrices That Sum to Target/1074.java

```
class Solution {
    public int numSubmatrixSumTarget(int[][] matrix, int target) {
        final int m = matrix.length;
        final int n = matrix[0].length;
        int ans = 0;

        // Transfer each row in the matrix to the prefix sum.
        for (int[] row : matrix)
            for (int i = 1; i < n; ++i)
                row[i] += row[i - 1];

        for (int baseCol = 0; baseCol < n; ++baseCol)
            for (int j = baseCol; j < n; ++j) {
                Map<Integer, Integer> prefixCount = new HashMap<>();
                prefixCount.put(0, 1);
                int sum = 0;
                for (int i = 0; i < m; ++i) {
                    if (baseCol > 0)
                        sum -= matrix[i][baseCol - 1];
                    sum += matrix[i][j];
                    ans += prefixCount.getOrDefault(sum - target, 0);
                    prefixCount.merge(sum, 1, Integer::sum);
                }
            }
        return ans;
    }
}
```

87. 1078.java

Path: LeetCode-main/solutions/1078. Occurrences After Bigram/1078.java

```
class Solution {
    public String[] findOcurrences(String text, String first, String second) {
        List<String> ans = new ArrayList<>();
        String[] words = text.split(" ");

        for (int i = 0; i + 2 < words.length; ++i)
            if (first.equals(words[i]) && second.equals(words[i + 1]))
                ans.add(words[i + 2]);

        return ans.toArray(new String[0]);
    }
}
```

88. 1079.java

Path: LeetCode-main/solutions/1079. Letter Tile Possibilities/1079.java

```
class Solution {
    public int numTilePossibilities(String tiles) {
        int[] count = new int[26];

        for (final char t : tiles.toCharArray())
            ++count[t - 'A'];

        return dfs(count);
    }

    private int dfs(int[] count) {
        int possibleSequences = 0;

        for (int i = 0; i < 26; ++i) {
            if (count[i] == 0)
                continue;
            // Put c in the current position. We only care about the number of possible
            // sequences of letters but don't care about the actual combination.
            --count[i];
            possibleSequences += 1 + dfs(count);
            ++count[i];
        }

        return possibleSequences;
    }
}
```


89. 108.java

Path: LeetCode-main/solutions/108. Convert Sorted Array to Binary Search Tree/108.java

```
class Solution {
    public TreeNode sortedArrayToBST(int[] nums) {
        return build(nums, 0, nums.length - 1);
    }

    private TreeNode build(int[] nums, int l, int r) {
        if (l > r)
            return null;
        final int m = (l + r) / 2;
        return new TreeNode(nums[m], build(nums, l, m - 1), build(nums, m + 1, r));
    }
}
```

90. 1080.java

Path: LeetCode-main/solutions/1080. Insufficient Nodes in Root to Leaf Paths/1080.java

```
class Solution {
    public TreeNode sufficientSubset(TreeNode root, int limit) {
        if (root == null)
            return null;
        if (root.left == null && root.right == null)
            return root.val < limit ? null : root;
        root.left = sufficientSubset(root.left, limit - root.val);
        root.right = sufficientSubset(root.right, limit - root.val);
        return root.left == null && root.right == null ? null : root;
    }
}
```

91. 1081.java

Path: LeetCode-main/solutions/1081. Smallest Subsequence of Distinct Characters/1081.java

```
class Solution {
    public String smallestSubsequence(String text) {
        StringBuilder sb = new StringBuilder();
        int[] count = new int[128];
        boolean[] used = new boolean[128];

        for (final char c : text.toCharArray())
            ++count[c];

        for (final char c : text.toCharArray()) {
            --count[c];
            if (used[c])
                continue;
            while (sb.length() > 0 && last(sb) > c && count[last(sb)] > 0) {
                used[last(sb)] = false;
                sb.setLength(sb.length() - 1);
            }
            used[c] = true;
            sb.append(c);
        }

        return sb.toString();
    }

    private char last(StringBuilder sb) {
        return sb.charAt(sb.length() - 1);
    }
}
```

92. 1085.java

Path: LeetCode-main/solutions/1085. Sum of Digits in the Minimum Number/1085.java

```
class Solution {
    public int sumOfDigits(int[] nums) {
        int mn = Arrays.stream(nums).min().getAsInt();
        int sum = 0;

        while (mn > 0) {
            sum += mn % 10;
            mn /= 10;
        }

        return sum & 1 ^ 1;
    }
}
```

93. 1086.java

Path: LeetCode-main/solutions/1086. High Five/1086.java

```
class Solution {
    public int[][] highFive(int[][] items) {
        List<int[]> ans = new ArrayList<>();
        Map<Integer, PriorityQueue<Integer>> idToScores = new TreeMap<>();

        for (int[] item : items) {
            final int id = item[0];
            final int score = item[1];
            idToScores.putIfAbsent(id, new PriorityQueue<>());
            PriorityQueue<Integer> scores = idToScores.get(id);
            scores.add(score);
            if (scores.size() > 5)
                scores.poll();
        }

        for (Map.Entry<Integer, PriorityQueue<Integer>> entry : idToScores.entrySet()) {
            final int id = entry.getKey();
            PriorityQueue<Integer> scores = entry.getValue();
            int sum = 0;
            while (!scores.isEmpty())
                sum += scores.poll();
            ans.add(new int[] {id, sum / 5});
        }

        return ans.toArray(int[][] ::new);
    }
}
```

94. 1087.java

Path: LeetCode-main/solutions/1087. Brace Expansion/1087.java

```
class Solution {
    public String[] expand(String s) {
        List<String> ans = new ArrayList<>();

        dfs(s, 0, new StringBuilder(), ans);
        Collections.sort(ans);
        return ans.toArray(new String[0]);
    }

    private void dfs(final String s, int i, StringBuilder sb, List<String> ans) {
        if (i == s.length()) {
            ans.add(sb.toString());
            return;
        }
        if (s.charAt(i) == '{') {
            final int nextRightBraceIndex = s.indexOf("}", i);
            for (final String c : s.substring(i + 1, nextRightBraceIndex).split(",")) {
                sb.append(c);
                dfs(s, nextRightBraceIndex + 1, sb, ans);
                sb.deleteCharAt(sb.length() - 1);
            }
        } else { // s[i] != '{'
            sb.append(s.charAt(i));
            dfs(s, i + 1, sb, ans);
            sb.deleteCharAt(sb.length() - 1);
        }
    }
}
```

95. 1088.java

Path: LeetCode-main/solutions/1088. Confusing Number II/1088.java

```
class Solution {
    public int confusingNumberII(int n) {
        return dfs(n, 0, 0, 1);
    }

    private int[][] digitToRotated = {{0, 0}, {1, 1}, {6, 9}, {8, 8}, {9, 6}};

    int dfs(int n, long num, long rotatedNum, long unit) {
        int ans = num == rotatedNum ? 0 : 1;
        // Add one more digit
        for (int[] pair : digitToRotated) {
            final int digit = pair[0];
            final int rotated = pair[1];
            if (digit == 0 && num == 0)
                continue;
            final long nextNum = num * 10 + digit;
            if (nextNum > n)
                break;
            ans += dfs(n, nextNum, rotated * unit + rotatedNum, unit * 10);
        }
        return ans;
    }
}
```

96. 1089.java

Path: LeetCode-main/solutions/1089. Duplicate Zeros/1089.java

```
class Solution {
    public void duplicateZeros(int[] arr) {
        int zeros = 0;

        for (int a : arr)
            if (a == 0)
                ++zeros;

        for (int i = arr.length - 1, j = arr.length + zeros - 1; i < j; --i, --j) {
            if (j < arr.length)
                arr[j] = arr[i];
            if (arr[i] == 0)
                if (--j < arr.length)
                    arr[j] = arr[i];
        }
    }
}
```


97. 109-2.java

Path: LeetCode-main/solutions/109. Convert Sorted List to Binary Search Tree/109-2.java

```
class Solution {
    public TreeNode sortedListToBST(ListNode head) {
        List<Integer> arr = new ArrayList<>();

        // Construct the array.
        for (ListNode curr = head; curr != null; curr = curr.next)
            arr.add(curr.val);

        return helper(arr, 0, arr.size() - 1);
    }

    private TreeNode helper(List<Integer> arr, int l, int r) {
        if (l > r)
            return null;
        final int m = (l + r) / 2;
        TreeNode root = new TreeNode(arr.get(m));
        root.left = helper(arr, l, m - 1);
        root.right = helper(arr, m + 1, r);
        return root;
    }
}
```

98. 109-3.java

Path: LeetCode-main/solutions/109. Convert Sorted List to Binary Search Tree/109-3.java

```
class Solution {
    public TreeNode sortedListToBST(ListNode head) {
        this.head = head;
        return helper(0, getLength(head) - 1);
    }

    private ListNode head;

    private TreeNode helper(int l, int r) {
        if (l > r)
            return null;

        final int m = (l + r) / 2;

        // Simulate inorder traversal: recursively form the left half.
        TreeNode left = helper(l, m - 1);

        // Once the left half is traversed, process the current node.
        TreeNode root = new TreeNode(head.val);
        root.left = left;

        // Maintain the invariance.
        head = head.next;

        // Simulate inorder traversal: recursively form the right half.
        root.right = helper(m + 1, r);
        return root;
    }

    private int getLength(ListNode head) {
        int length = 0;
        for (ListNode curr = head; curr != null; curr = curr.next)
            ++length;
        return length;
    }
}
```

99. 109.java

Path: LeetCode-main/solutions/109. Convert Sorted List to Binary Search Tree/109.java

```
class Solution {
    public TreeNode sortedListToBST(ListNode head) {
        if (head == null)
            return null;
        if (head.next == null)
            return new TreeNode(head.val);

        ListNode mid = findMid(head);
        TreeNode root = new TreeNode(mid.val);
        root.left = sortedListToBST(head);
        root.right = sortedListToBST(mid.next);
        return root;
    }

    private ListNode findMid(ListNode head) {
        ListNode prev = null;
        ListNode slow = head;
        ListNode fast = head;

        while (fast != null && fast.next != null) {
            prev = slow;
            slow = slow.next;
            fast = fast.next.next;
        }
        prev.next = null;

        return slow;
    }
}
```

100. 1091.java

Path: LeetCode-main/solutions/1091. Shortest Path in Binary Matrix/1091.java

```
class Solution {
    public int shortestPathBinaryMatrix(int[][] grid) {
        final int n = grid.length;
        if (grid[0][0] == 0 && n == 1)
            return 1;
        if (grid[0][0] == 1 || grid[n - 1][n - 1] == 1)
            return -1;

        final int[][] DIRS = {{-1, -1}, {-1, 0}, {-1, 1}, {0, -1}, {0, 1}, {1, -1}, {1, 0}, {1, 1}};
        Queue<Pair<Integer, Integer>> q = new ArrayDeque<>(List.of(new Pair<>(0, 0)));
        boolean[][] seen = new boolean[n][n];
        seen[0][0] = true;

        for (int step = 1; !q.isEmpty(); ++step)
            for (int sz = q.size(); sz > 0; --sz) {
                final int i = q.peek().getKey();
                final int j = q.poll().getValue();
                for (int[] dir : DIRS) {
                    final int x = i + dir[0];
                    final int y = j + dir[1];
                    if (x < 0 || x == n || y < 0 || y == n)
                        continue;
                    if (grid[x][y] != 0 || seen[x][y])
                        continue;
                    if (x == n - 1 && y == n - 1)
                        return step + 1;
                    q.offer(new Pair<>(x, y));
                    seen[x][y] = true;
                }
            }
        return -1;
    }
}
```

101. 1092.java

Path: LeetCode-main/solutions/1092. Shortest Common Supersequence/1092.java

```
class Solution {
    public String shortestCommonSupersequence(String str1, String str2) {
        StringBuilder sb = new StringBuilder();
        int i = 0; // str1's index
        int j = 0; // str2's index

        for (final char c : lcs(str1, str2).toCharArray()) {
            // Append the letters that are not part of the LCS.
            while (str1.charAt(i) != c)
                sb.append(str1.charAt(i++));
            while (str2.charAt(j) != c)
                sb.append(str2.charAt(j++));
            // Append the letter of the LCS and match it with str1 and str2.
            sb.append(c);
            ++i;
            ++j;
        }

        // Append the remaining letters.
        return sb.toString() + str1.substring(i) + str2.substring(j);
    }

    private String lcs(final String a, final String b) {
        final int m = a.length();
        final int n = b.length();
        // dp[i][j] := the length of LCS(a[0..i), b[0..j))
        StringBuilder[][] dp = new StringBuilder[m + 1][n + 1];

        for (final StringBuilder[] row : dp)
            for (int i = 0; i < row.length; ++i)
                row[i] = new StringBuilder();

        for (int i = 1; i <= m; ++i)
            for (int j = 1; j <= n; ++j)
                if (a.charAt(i - 1) == b.charAt(j - 1))
                    dp[i][j].append(dp[i - 1][j - 1]).append(a.charAt(i - 1));
                else
                    dp[i][j] = dp[i - 1][j].length() > dp[i][j - 1].length() ? dp[i - 1][j] : dp[i][j - 1];

        return dp[m][n].toString();
    }
}
```

102. 1093.java

Path: LeetCode-main/solutions/1093. Statistics from a Large Sample/1093.java

```
class Solution {
    public double[] sampleStats(int[] count) {
        final int n = Arrays.stream(count).sum();
        return new double[] {
            getMinimum(count), //
            getMaximum(count), //
            getMean(count, n), //
            (getLeftMedian(count, n) + getRightMedian(count, n)) / 2.0, //
            getMode(count),
        };
    }

    private double getMinimum(int[] count) {
        for (int i = 0; i < count.length; ++i)
            if (count[i] > 0)
                return i;
        return -1;
    }

    private double getMaximum(int[] count) {
        for (int i = count.length - 1; i >= 0; --i)
            if (count[i] > 0)
                return i;
        return -1;
    }

    private double getMean(int[] count, double n) {
        double mean = 0;
        for (int i = 0; i < count.length; ++i)
            mean += ((long) i * (long) count[i]) / n;
        return mean;
    }

    private double getLeftMedian(int[] count, double n) {
        int numCount = 0;
        for (int i = 0; i < count.length; ++i) {
            numCount += count[i];
            if (numCount >= n / 2)
                return i;
        }
        return -1;
    }

    private double getRightMedian(int[] count, double n) {
        int numCount = 0;
        for (int i = count.length - 1; i >= 0; --i) {
            numCount += count[i];
            if (numCount >= n / 2)
                return i;
        }
        return -1;
    }

    private double getMode(int[] count) {
        int mode = -1;
        int maxCount = 0;
        for (int i = 0; i < count.length; ++i)
            if (count[i] > maxCount) {
                maxCount = count[i];
                mode = i;
            }
        return mode;
    }
}
```

103. 1094-2.java

Path: LeetCode-main/solutions/1094. Car Pooling/1094-2.java

```
class Solution {
    public boolean carPooling(int[][] trips, int capacity) {
        int currentPassengers = 0;
        int[] line = new int[1001];

        for (int[] trip : trips) {
            final int nPassengers = trip[0];
            final int start = trip[1];
            final int end = trip[2];
            line[start] += nPassengers;
            line[end] -= nPassengers;
        }

        for (final int passengerChange : line) {
            currentPassengers += passengerChange;
            if (currentPassengers > capacity)
                return false;
        }

        return true;
    }
}
```

104. 1094.java

Path: LeetCode-main/solutions/1094. Car Pooling/1094.java

```
class Solution {
    public boolean carPooling(int[][] trips, int capacity) {
        int currentPassengers = 0;
        Map<Integer, Integer> line = new TreeMap<>();

        for (int[] trip : trips) {
            final int nPassengers = trip[0];
            final int start = trip[1];
            final int end = trip[2];
            line.merge(start, nPassengers, Integer::sum);
            line.merge(end, -nPassengers, Integer::sum);
        }

        for (final int passengerChange : line.values()) {
            currentPassengers += passengerChange;
            if (currentPassengers > capacity)
                return false;
        }

        return true;
    }
}
```


105. 1095.java

Path: LeetCode-main/solutions/1095. Find in Mountain Array/1095.java

```
/**
 * // This is MountainArray's API interface.
 * // You should not implement it, or speculate about its implementation
 * interface MountainArray {
 *     public int get(int index) {}
 *     public int length() {}
 * }
 */

class Solution {
    public int findInMountainArray(int target, MountainArray mountainArr) {
        final int n = mountainArr.length();
        final int peakIndex = peakIndexInMountainArray(mountainArr, 0, n - 1);

        final int leftIndex = searchLeft(mountainArr, target, 0, peakIndex);
        if (mountainArr.get(leftIndex) == target)
            return leftIndex;

        final int rightIndex = searchRight(mountainArr, target, peakIndex + 1, n - 1);
        if (mountainArr.get(rightIndex) == target)
            return rightIndex;

        return -1;
    }

    // 852. Peak Index in a Mountain Array
    private int peakIndexInMountainArray(MountainArray A, int l, int r) {
        while (l < r) {
            final int m = (l + r) / 2;
            if (A.get(m) < A.get(m + 1))
                l = m + 1;
            else
                r = m;
        }
        return l;
    }

    private int searchLeft(MountainArray A, int target, int l, int r) {
        while (l < r) {
            final int m = (l + r) / 2;
            if (A.get(m) < target)
                l = m + 1;
            else
                r = m;
        }
        return l;
    }

    private int searchRight(MountainArray A, int target, int l, int r) {
        while (l < r) {
            final int m = (l + r) / 2;
            if (A.get(m) > target)
                l = m + 1;
            else
                r = m;
        }
        return l;
    }
}
```

106. 1096.java

Path: LeetCode-main/solutions/1096. Brace Expansion II/1096.java

```
class Solution {
    public List<String> braceExpansionII(String expression) {
        return dfs(expression, 0, expression.length() - 1);
    }

    private List<String> dfs(final String expression, int s, int e) {
        TreeSet<String> ans = new TreeSet<>();
        List<List<String>> groups = new ArrayList<>();
        groups.add(new ArrayList<>());
        int layer = 0;
        int left = 0;

        for (int i = s; i <= e; ++i)
            if (expression.charAt(i) == '{' && ++layer == 1)
                left = i + 1;
            else if (expression.charAt(i) == '}' && --layer == 0)
                merge(groups, dfs(expression, left, i - 1));
            else if (expression.charAt(i) == ',' && layer == 0)
                groups.add(new ArrayList<>());
            else if (layer == 0)
                merge(groups, new ArrayList<>(List.of(String.valueOf(expression.charAt(i)))));

        for (final List<String> group : groups)
            for (final String word : group)
                ans.add(word);

        return new ArrayList<>(ans);
    }

    void merge(List<List<String>> groups, List<String> group) {
        if (groups.get(groups.size() - 1).isEmpty()) {
            groups.set(groups.size() - 1, group);
            return;
        }

        List<String> mergedGroup = new ArrayList<>();

        for (final String word1 : groups.get(groups.size() - 1))
            for (final String word2 : group)
                mergedGroup.add(word1 + word2);

        groups.set(groups.size() - 1, mergedGroup);
    }
}
```

107. 1099.java

Path: LeetCode-main/solutions/1099. Two Sum Less Than K/1099.java

```
class Solution {
    public int twoSumLessThanK(int[] nums, int k) {
        if (nums.length < 2)
            return -1;

        int ans = -1; // Note the constraint that nums[i] > 0.
        int l = 0;
        int r = nums.length - 1;

        Arrays.sort(nums);

        while (l < r)
            if (nums[l] + nums[r] < k) {
                ans = Math.max(ans, nums[l] + nums[r]);
                ++l;
            } else {
                --r;
            }

        return ans;
    }
}
```

108. 11.java

Path: LeetCode-main/solutions/11. Container With Most Water/11.java

```
class Solution {
    public int maxArea(int[] height) {
        int ans = 0;
        int l = 0;
        int r = height.length - 1;

        while (l < r) {
            final int minHeight = Math.min(height[l], height[r]);
            ans = Math.max(ans, minHeight * (r - l));
            if (height[l] < height[r])
                ++l;
            else
                --r;
        }

        return ans;
    }
}
```

109. 110-2.java

Path: LeetCode-main/solutions/110. Balanced Binary Tree/110-2.java

```
class Solution {
    public boolean isBalanced(TreeNode root) {
        maxDepth(root);
        return ans;
    }

    private boolean ans = true;

    // Returns the height of root and sets ans to false if root unbalanced.
    public int maxDepth(TreeNode root) {
        if (root == null || !ans)
            return 0;
        final int left = maxDepth(root.left);
        final int right = maxDepth(root.right);
        if (Math.abs(left - right) > 1)
            ans = false;
        return Math.max(left, right) + 1;
    }
}
```

110. 110-3.java

Path: LeetCode-main/solutions/110. Balanced Binary Tree/110-3.java

```
class Solution {
    public boolean isBalanced(TreeNode root) {
        return maxDepth(root) != -1;
    }

    // Returns the height of root if root is balanced; otherwise, returns -1.
    private int maxDepth(TreeNode root) {
        if (root == null)
            return 0;

        final int left = maxDepth(root.left);
        if (left == -1)
            return -1;
        final int right = maxDepth(root.right);
        if (right == -1)
            return -1;
        if (Math.abs(left - right) > 1)
            return -1;

        return 1 + Math.max(left, right);
    }
}
```

111. 110.java

Path: LeetCode-main/solutions/110. Balanced Binary Tree/110.java

```
class Solution {
    public boolean isBalanced(TreeNode root) {
        if (root == null)
            return true;
        return Math.abs(maxDepth(root.left) - maxDepth(root.right)) <= 1 && //
            isBalanced(root.left) && //
            isBalanced(root.right);
    }

    private int maxDepth(TreeNode root) {
        if (root == null)
            return 0;
        return 1 + Math.max(maxDepth(root.left), maxDepth(root.right));
    }
}
```

112. 1100.java

Path: LeetCode-main/solutions/1100. Find K-Length Substrings With No Repeated Characters/1100.java

```
class Solution {
    public int numKLenSubstrNoRepeats(String s, int k) {
        int ans = 0;
        int unique = 0;
        int[] count = new int[26];

        for (int i = 0; i < s.length(); ++i) {
            if (++count[s.charAt(i) - 'a'] == 1)
                ++unique;
            if (i >= k && --count[s.charAt(i - k) - 'a'] == 0)
                --unique;
            if (unique == k)
                ++ans;
        }
        return ans;
    }
}
```


113. 1101.java

Path: LeetCode-main/solutions/1101. The Earliest Moment When Everyone Become Friends/1101.java

```
class UnionFind {
    public UnionFind(int n) {
        count = n;
        id = new int[n];
        rank = new int[n];
        for (int i = 0; i < n; ++i)
            id[i] = i;
    }

    public void unionByRank(int u, int v) {
        final int i = find(u);
        final int j = find(v);
        if (i == j)
            return;
        if (rank[i] < rank[j]) {
            id[i] = j;
        } else if (rank[i] > rank[j]) {
            id[j] = i;
        } else {
            id[i] = j;
            ++rank[j];
        }
        --count;
    }

    public int getCount() {
        return count;
    }

    private int count;
    private int[] id;
    private int[] rank;

    private int find(int u) {
        return id[u] == u ? u : (id[u] = find(id[u]));
    }
}

class Solution {
    public int earliestAcq(int[][] logs, int n) {
        UnionFind uf = new UnionFind(n);

        // Sort `logs` by timestamp.
        Arrays.sort(logs, Comparator.comparingInt(log -> log[0]));

        for (int[] log : logs) {
            final int timestamp = log[0];
            final int x = log[1];
            final int y = log[2];
            uf.unionByRank(x, y);
            if (uf.getCount() == 1)
                return timestamp;
        }

        return -1;
    }
}
```

114. 1102.java

Path: LeetCode-main/solutions/1102. Path With Maximum Minimum Value/1102.java

```
class Solution {
    public int maximumMinimumPath(int[][] grid) {
        record T(int i, int j, int val) {}
        final int[][] DIRS = {{0, 1}, {1, 0}, {0, -1}, {-1, 0}};
        final int m = grid.length;
        final int n = grid[0].length;
        int ans = grid[0][0];
        boolean[][] seen = new boolean[m][n];
        Queue<T> maxHeap = new PriorityQueue<>(Comparator.comparingInt(T::val).reversed()) {
            { offer(new T(0, 0, grid[0][0])); }
        };

        while (!maxHeap.isEmpty()) {
            final int i = maxHeap.peek().i;
            final int j = maxHeap.peek().j;
            final int val = maxHeap.poll().val;
            ans = Math.min(ans, val);
            if (i == m - 1 && j == n - 1)
                return ans;
            seen[i][j] = true;
            for (int[] dir : DIRS) {
                final int x = i + dir[0];
                final int y = j + dir[1];
                if (x < 0 || x == m || y < 0 || y == n)
                    continue;
                if (seen[x][y])
                    continue;
                maxHeap.offer(new T(x, y, grid[x][y]));
            }
        }

        throw new IllegalArgumentException();
    }
}
```

115. 1103.java

Path: LeetCode-main/solutions/1103. Distribute Candies to People/1103.java

```
class Solution {
    public int[] distributeCandies(int candies, int num_people) {
        int[] ans = new int[num_people];
        long c = (long) candies;
        long n = (long) num_people;
        int rows = (int) (-n + Math.sqrt(n * n + 8 * n * n * c)) / (int) (2 * n * n);
        int accumN = rows * (rows - 1) * num_people / 2;

        for (int i = 0; i < n; ++i)
            ans[i] = accumN + rows * (i + 1);

        int givenCandies = (num_people * num_people * rows * rows + num_people * rows) / 2;
        candies -= givenCandies;

        for (int i = 0, lastGiven = rows * num_people + 1; candies > 0; ++i, ++lastGiven) {
            int actualGiven = Math.min(lastGiven, candies);
            candies -= actualGiven;
            ans[i] += actualGiven;
        }

        return ans;
    }
}
```

116. 1104.java

Path: LeetCode-main/solutions/1104. Path In Zigzag Labelled Binary Tree/1104.java

```
class Solution {
    public List<Integer> pathInZigZagTree(int label) {
        LinkedList<Integer> ans = new LinkedList<>();
        int level = 0;

        for (int l = 0; l < 21; ++l)
            if (Math.pow(2, l) > label) {
                level = l - 1;
                break;
            }

        if (level % 2 == 1)
            label = boundarySum(level) - label;

        for (int l = level; l >= 0; --l) {
            ans.addFirst(l % 2 == 0 ? label : boundarySum(l) - label);
            label /= 2;
        }

        return new ArrayList<>(ans);
    }

    private int boundarySum(int level) {
        return (int) Math.pow(2, level) + (int) Math.pow(2, level + 1) - 1;
    }
}
```

117. 1105.java

Path: LeetCode-main/solutions/1105. Filling Bookcase Shelves/1105.java

```
class Solution {
    public int minHeightShelves(int[][] books, int shelfWidth) {
        final int n = books.length;
        // dp[i] := the minimum height to place the first i books
        int[] dp = new int[n + 1];
        Arrays.fill(dp, Integer.MAX_VALUE);
        dp[0] = 0;

        for (int i = 0; i < books.length; ++i) {
            int sumThickness = 0;
            int maxHeight = 0;
            // Place books[j..i] on a new shelf.
            for (int j = i; j >= 0; --j) {
                final int thickness = books[j][0];
                final int height = books[j][1];
                sumThickness += thickness;
                if (sumThickness > shelfWidth)
                    break;
                maxHeight = Math.max(maxHeight, height);
                dp[i + 1] = Math.min(dp[i + 1], dp[j] + maxHeight);
            }
        }
        return dp[n];
    }
}
```

118. 1106.java

Path: LeetCode-main/solutions/1106. Parsing A Boolean Expression/1106.java

```
class Solution {
    public boolean parseBoolExpr(String expression) {
        return dfs(expression, 0, expression.length() - 1);
    }

    private boolean dfs(final String expression, int s, int e) {
        if (s == e)
            return expression.charAt(s) == 't';

        List<Boolean> exps = new ArrayList<>();
        int layer = 0;
        int left = 0;
        char op = ' ';

        for (int i = s; i <= e; ++i) {
            char c = expression.charAt(i);
            if (layer == 0 && (c == '!' || c == '&' || c == '|'))
                op = c;
            else if (c == '(' && ++layer == 1)
                left = i + 1;
            else if (c == ')' && --layer == 0)
                exps.add(dfs(expression, left, i - 1));
            else if (c == ',' && layer == 1) {
                exps.add(dfs(expression, left, i - 1));
                left = i + 1;
            }
        }

        if (op == '&') {
            boolean ans = true;
            for (boolean exp : exps)
                ans &= exp;
            return ans;
        }

        if (op == '|') {
            boolean ans = false;
            for (boolean exp : exps)
                ans |= exp;
            return ans;
        }

        return !exps.get(0);
    }
}
```

119. 1108.java

Path: LeetCode-main/solutions/1108. Defanging an IP Address/1108.java

```
class Solution {  
    public String defangIPAddr(String address) {  
        return address.replace(".", "[.]");  
    }  
}
```

120. 1109.java

Path: LeetCode-main/solutions/1109. Corporate Flight Bookings/1109.java

```
class Solution {
    public int[] corpFlightBookings(int[][] bookings, int n) {
        int[] ans = new int[n];

        for (int[] booking : bookings) {
            ans[booking[0] - 1] += booking[2];
            if (booking[1] < n)
                ans[booking[1]] -= booking[2];
        }

        for (int i = 1; i < n; ++i)
            ans[i] += ans[i - 1];

        return ans;
    }
}
```


121. 111-2.java

Path: LeetCode-main/solutions/111. Minimum Depth of Binary Tree/111-2.java

```
class Solution {
    public int minDepth(TreeNode root) {
        if (root == null)
            return 0;

        Queue<TreeNode> q = new ArrayDeque<>(List.of(root));

        for (int step = 1; !q.isEmpty(); ++step)
            for (int sz = q.size(); sz > 0; --sz) {
                TreeNode node = q.poll();
                if (node.left == null && node.right == null)
                    return step;
                if (node.left != null)
                    q.offer(node.left);
                if (node.right != null)
                    q.offer(node.right);
            }

        throw new IllegalArgumentException();
    }
}
```

122. 111.java

Path: LeetCode-main/solutions/111. Minimum Depth of Binary Tree/111.java

```
class Solution {
    public int minDepth(TreeNode root) {
        if (root == null)
            return 0;
        if (root.left == null)
            return minDepth(root.right) + 1;
        if (root.right == null)
            return minDepth(root.left) + 1;
        return Math.min(minDepth(root.left), minDepth(root.right)) + 1;
    }
}
```

123. 1110.java

Path: LeetCode-main/solutions/1110. Delete Nodes And Return Forest/1110.java

```
class Solution {
    public List<TreeNode> delNodes(TreeNode root, int[] to_delete) {
        List<TreeNode> ans = new ArrayList<>();
        Set<Integer> toDeleteSet = Arrays.stream(to_delete).boxed().collect(Collectors.toSet());
        dfs(root, toDeleteSet, true, ans);
        return ans;
    }

    private TreeNode dfs(TreeNode root, Set<Integer> toDeleteSet, boolean isRoot,
        List<TreeNode> ans) {
        if (root == null)
            return null;

        final boolean deleted = toDeleteSet.contains(root.val);
        if (isRoot && !deleted)
            ans.add(root);

        // If root is deleted, both children have the possibility to be a new root
        root.left = dfs(root.left, toDeleteSet, deleted, ans);
        root.right = dfs(root.right, toDeleteSet, deleted, ans);
        return deleted ? null : root;
    }
}
```

124. 1111.java

Path: LeetCode-main/solutions/1111. Maximum Nesting Depth of Two Valid Parentheses Strings/1111.java

```
class Solution {
    public int[] maxDepthAfterSplit(String seq) {
        int[] ans = new int[seq.length()];
        int depth = 1;

        // Put all odd-depth parentheses in one group and even-depth parentheses in
        // the other group.
        for (int i = 0; i < seq.length(); ++i)
            if (seq.charAt(i) == '(')
                ans[i] = ++depth % 2;
            else
                ans[i] = depth-- % 2;

        return ans;
    }
}
```

125. 1114.java

Path: LeetCode-main/solutions/1114. Print in Order/1114.java

```
class Foo {
    public void first(Runnable printFirst) throws InterruptedException {
        printFirst.run();
        firstDone.incrementAndGet();
    }

    public void second(Runnable printSecond) throws InterruptedException {
        while (firstDone.get() != 1)
            ;
        printSecond.run();
        secondDone.incrementAndGet();
    }

    public void third(Runnable printThird) throws InterruptedException {
        while (secondDone.get() != 1)
            ;
        printThird.run();
    }

    private AtomicInteger firstDone = new AtomicInteger();
    private AtomicInteger secondDone = new AtomicInteger();
}
```

126. 1115.java

Path: LeetCode-main/solutions/1115. Print FooBar Alternately/1115.java

```
class FooBar {
    public FooBar(int n) {
        this.n = n;
    }

    public void foo(Runnable printFoo) throws InterruptedException {
        for (int i = 0; i < n; ++i) {
            fooSemaphore.acquire();
            printFoo.run();
            barSemaphore.release();
        }
    }

    public void bar(Runnable printBar) throws InterruptedException {
        for (int i = 0; i < n; ++i) {
            barSemaphore.acquire();
            printBar.run();
            fooSemaphore.release();
        }
    }

    private int n;
    private Semaphore fooSemaphore = new Semaphore(1);
    private Semaphore barSemaphore = new Semaphore(0);
}
```

127. 1116.java

Path: LeetCode-main/solutions/1116. Print Zero Even Odd/1116.java

```
class ZeroEvenOdd {
    public ZeroEvenOdd(int n) {
        this.n = n;
    }

    // printNumber.accept(x) outputs "x", where x is an integer.
    public void zero(IntConsumer printNumber) throws InterruptedException {
        for (int i = 0; i < n; ++i) {
            zeroSemaphore.acquire();
            printNumber.accept(0);
            (i % 2 == 0 ? oddSemaphore : evenSemaphore).release();
        }
    }

    public void even(IntConsumer printNumber) throws InterruptedException {
        for (int i = 2; i <= n; i += 2) {
            evenSemaphore.acquire();
            printNumber.accept(i);
            zeroSemaphore.release();
        }
    }

    public void odd(IntConsumer printNumber) throws InterruptedException {
        for (int i = 1; i <= n; i += 2) {
            oddSemaphore.acquire();
            printNumber.accept(i);
            zeroSemaphore.release();
        }
    }

    private int n;
    private Semaphore zeroSemaphore = new Semaphore(1);
    private Semaphore evenSemaphore = new Semaphore(0);
    private Semaphore oddSemaphore = new Semaphore(0);
}
```

128. 1118.java

Path: LeetCode-main/solutions/1118. Number of Days in a Month/1118.java

```
class Solution {
    public int numberOfDays(int year, int month) {
        final int[] days = {0, 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};
        return month == 2 && isLeapYear(year) ? 29 : days[month];
    }

    private boolean isLeapYear(int year) {
        return year % 4 == 0 && year % 100 != 0 || year % 400 == 0;
    }
}
```


129. 1119.java

Path: LeetCode-main/solutions/1119. Remove Vowels from a String/1119.java

```
class Solution {  
    public String removeVowels(String s) {  
        return s.replaceAll("[aeiou]", "");  
    }  
}
```

130. 112.java

Path: LeetCode-main/solutions/112. Path Sum/112.java

```
class Solution {
    public boolean hasPathSum(TreeNode root, int sum) {
        if (root == null)
            return false;
        if (root.val == sum && root.left == null && root.right == null)
            return true;
        return
            hasPathSum(root.left, sum - root.val) || //
            hasPathSum(root.right, sum - root.val);
    }
}
```

131. 1120.java

Path: LeetCode-main/solutions/1120. Maximum Average Subtree/1120.java

```
class Solution {
    public double maximumAverageSubtree(TreeNode root) {
        return maximumAverage(root).maxAverage;
    }

    private record T(int sum, int count, double maxAverage) {}

    private T maximumAverage(TreeNode root) {
        if (root == null)
            return new T(0, 0, 0.0);

        T left = maximumAverage(root.left);
        T right = maximumAverage(root.right);

        final int sum = root.val + left.sum + right.sum;
        final int count = 1 + left.count + right.count;
        final double maxAverage =
            Math.max(sum / (double) count, Math.max(left.maxAverage, right.maxAverage));
        return new T(sum, count, maxAverage);
    }
}
```

132. 1121.java

Path: LeetCode-main/solutions/1121. Divide Array Into Increasing Sequences/1121.java

```
class Solution {
    public boolean canDivideIntoSubsequences(int[] nums, int k) {
        // Find the number with the maxFreq, we need at least maxFreq * k elements
        // e.g. nums = [1, 2, 2, 3, 4], we have maxFreq = 2 (two 2s), so we have to
        // Split nums into two subsequences say k = 3, the minimum length of nums is 2 x
        // 3 = 6, which is impossible if nums.size() = 5
        final int n = nums.length;
        int freq = 1;
        int maxFreq = 1;

        for (int i = 1; i < n; ++i) {
            freq = nums[i - 1] < nums[i] ? 1 : ++freq;
            maxFreq = Math.max(maxFreq, freq);
        }

        return n >= maxFreq * k;
    }
}
```

133. 1122.java

Path: LeetCode-main/solutions/1122. Relative Sort Array/1122.java

```
class Solution {
    public int[] relativeSortArray(int[] arr1, int[] arr2) {
        int[] ans = new int[arr1.length];
        int[] count = new int[1001];
        int i = 0;

        for (int a : arr1)
            ++count[a];

        for (int a : arr2)
            while (count[a]-- > 0)
                ans[i++] = a;

        for (int num = 0; num < 1001; ++num)
            while (count[num]-- > 0)
                ans[i++] = num;

        return ans;
    }
}
```

134. 1123.java

Path: LeetCode-main/solutions/1123. Lowest Common Ancestor of Deepest Leaves/1123.java

```
class Solution {
    public TreeNode lcaDeepestLeaves(TreeNode root) {
        return dfs(root).lca;
    }

    private record T(TreeNode lca, int depth) {}

    private T dfs(TreeNode root) {
        if (root == null)
            return new T(null, 0);
        T left = dfs(root.left);
        T right = dfs(root.right);
        if (left.depth > right.depth)
            return new T(left.lca, left.depth + 1);
        if (left.depth < right.depth)
            return new T(right.lca, right.depth + 1);
        return new T(root, left.depth + 1);
    }
}
```

135. 1125.java

Path: LeetCode-main/solutions/1125. Smallest Sufficient Team/1125.java

```
class Solution {
    public int[] smallestSufficientTeam(String[] req_skills, List<List<String>> people) {
        final int n = req_skills.length;
        final int nSkills = 1 << n;
        Map<String, Integer> skillToId = new HashMap();
        // dp[i] := the minimum people's indices to cover skillset of mask i
        List<Integer>[] dp = new List[nSkills];
        dp[0] = new ArrayList<>();

        for (int i = 0; i < req_skills.length; ++i)
            skillToId.put(req_skills[i], i);

        for (int i = 0; i < people.size(); ++i) {
            final int currSkill = getSkill(people.get(i), skillToId);
            for (int j = 0; j < nSkills; ++j) {
                if (dp[j] == null)
                    continue;
                final int newSkillSet = currSkill | j;
                if (newSkillSet == j) // Adding people[i] doesn't increase skill set
                    continue;
                if (dp[newSkillSet] == null || dp[newSkillSet].size() > dp[j].size() + 1) {
                    dp[newSkillSet] = new ArrayList<>(dp[j]);
                    dp[newSkillSet].add(i);
                }
            }
        }

        return dp[nSkills - 1].stream().mapToInt(Integer::intValue).toArray();
    }

    private int getSkill(List<String> person, Map<String, Integer> skillToId) {
        int mask = 0;
        for (final String skill : person)
            if (skillToId.containsKey(skill))
                mask |= 1 << skillToId.get(skill);
        return mask;
    }
}
```

136. 1128.java

Path: LeetCode-main/solutions/1128. Number of Equivalent Domino Pairs/1128.java

```
class Solution {
    public int numEquivDominoPairs(int[][] dominoes) {
        int ans = 0;
        Map<Integer, Integer> count = new HashMap<>();

        for (int[] domino : dominoes) {
            int key = Math.min(domino[0], domino[1]) * 10 + Math.max(domino[0], domino[1]);
            ans += count.getOrDefault(key, 0);
            count.merge(key, 1, Integer::sum);
        }

        return ans;
    }
}
```


137. 1129.java

Path: LeetCode-main/solutions/1129. Shortest Path with Alternating Colors/1129.java

```
enum Color { INIT, RED, BLUE }

class Solution {
    public int[] shortestAlternatingPaths(int n, int[][] redEdges, int[][] blueEdges) {
        int[] ans = new int[n];
        Arrays.fill(ans, -1);
        // graph[u] := [(v, edgeColor)]
        List<Pair<Integer, Color>>[] graph = new List[n];
        // [(u, prevColor)]
        Queue<Pair<Integer, Color>> q = new ArrayDeque<>(List.of(new Pair<>(0, Color.INIT)));
        Arrays.setAll(graph, i -> new ArrayList<>());

        for (int[] edge : redEdges) {
            final int u = edge[0];
            final int v = edge[1];
            graph[u].add(new Pair<>(v, Color.RED));
        }

        for (int[] edge : blueEdges) {
            final int u = edge[0];
            final int v = edge[1];
            graph[u].add(new Pair<>(v, Color.BLUE));
        }

        for (int step = 0; !q.isEmpty(); ++step)
            for (int sz = q.size(); sz > 0; --sz) {
                final int u = q.peek().getKey();
                Color prevColor = q.poll().getValue();
                ans[u] = ans[u] == -1 ? step : ans[u];
                for (int i = 0; i < graph[u].size(); ++i) {
                    Pair<Integer, Color> node = graph[u].get(i);
                    final int v = node.getKey();
                    Color edgeColor = node.getValue();
                    if (v == -1 || edgeColor == prevColor)
                        continue;
                    q.add(new Pair<>(v, edgeColor));
                    // Mark (u, v) as used.
                    graph[u].set(i, new Pair<>(-1, edgeColor));
                }
            }

        return ans;
    }
}
```

138. 113.java

Path: LeetCode-main/solutions/113. Path Sum II/113.java

```
class Solution {
    public List<List<Integer>> pathSum(TreeNode root, int sum) {
        List<List<Integer>> ans = new ArrayList<>();
        dfs(root, sum, new ArrayList<>(), ans);
        return ans;
    }

    private void dfs(TreeNode root, int sum, List<Integer> path, List<List<Integer>> ans) {
        if (root == null)
            return;
        if (root.val == sum && root.left == null && root.right == null) {
            path.add(root.val);
            ans.add(new ArrayList<>(path));
            path.remove(path.size() - 1);
            return;
        }

        path.add(root.val);
        dfs(root.left, sum - root.val, path, ans);
        dfs(root.right, sum - root.val, path, ans);
        path.remove(path.size() - 1);
    }
}
```

139. 1130-2.java

Path: LeetCode-main/solutions/1130. Minimum Cost Tree From Leaf Values/1130-2.java

```
class Solution {
    public int mctFromLeafValues(int[] arr) {
        int ans = 0;
        Deque<Integer> stack = new ArrayDeque<>();
        stack.push(Integer.MAX_VALUE);

        for (final int a : arr) {
            while (stack.peek() <= a) {
                final int mid = stack.pop();
                // Multiply mid with next greater element in the array,
                // On the left (stack[-1]) or on the right (current number a)
                ans += Math.min(stack.peek(), a) * mid;
            }
            stack.push(a);
        }

        while (stack.size() > 2)
            ans += stack.pop() * stack.peek();

        return ans;
    }
}
```

140. 1130.java

Path: LeetCode-main/solutions/1130. Minimum Cost Tree From Leaf Values/1130.java

```
class Solution {
    public int mctFromLeafValues(int[] arr) {
        final int n = arr.length;
        // dp[i][j] := the minimum cost of arr[i..j]
        int[][] dp = new int[n][n];
        // maxVal[i][j] := the maximum value of arr[i..j]
        int[][] maxVal = new int[n][n];

        for (int i = 0; i < n; ++i)
            maxVal[i][i] = arr[i];

        for (int d = 1; d < n; ++d)
            for (int i = 0; i + d < n; ++i) {
                final int j = i + d;
                maxVal[i][j] = Math.max(maxVal[i][j - 1], maxVal[i + 1][j]);
            }

        for (int d = 1; d < n; ++d)
            for (int i = 0; i + d < n; ++i) {
                final int j = i + d;
                dp[i][j] = Integer.MAX_VALUE;
                for (int k = i; k < j; ++k)
                    dp[i][j] = Math.min(dp[i][j], dp[i][k] + dp[k + 1][j] + maxVal[i][k] * maxVal[k + 1][j]);
            }

        return dp[0][n - 1];
    }
}
```

141. 1131.java

Path: LeetCode-main/solutions/1131. Maximum of Absolute Value Expression/1131.java

```
class Solution {
    public int maxAbsValExpr(int[] arr1, int[] arr2) {
        final int n = arr1.length;
        int[] a = new int[n];
        int[] b = new int[n];
        int[] c = new int[n];
        int[] d = new int[n];

        for (int i = 0; i < n; ++i) {
            a[i] = arr1[i] + arr2[i] + i;
            b[i] = arr1[i] + arr2[i] - i;
            c[i] = arr1[i] - arr2[i] + i;
            d[i] = arr1[i] - arr2[i] - i;
        }

        return Math.max(Math.max(diff(a), diff(b)), Math.max(diff(c), diff(d)));
    }

    private int diff(int[] nums) {
        final int mn = Arrays.stream(nums).min().getAsInt();
        final int mx = Arrays.stream(nums).max().getAsInt();
        return mx - mn;
    }
}
```

142. 1133.java

Path: LeetCode-main/solutions/1133. Largest Unique Number/1133.java

```
class Solution {
    public int largestUniqueNumber(int[] nums) {
        final int MAX = 1000;
        int[] count = new int[MAX + 1];

        for (final int num : nums)
            ++count[num];

        for (int num = MAX; num >= 0; --num)
            if (count[num] == 1)
                return num;

        return -1;
    }
}
```

143. 1134.java

Path: LeetCode-main/solutions/1134. Armstrong Number/1134.java

```
class Solution {  
    public boolean isArmstrong(int n) {  
        final String s = String.valueOf(n);  
        final int k = s.length();  
        for (final char c : s.toCharArray())  
            n -= Math.pow(c - '0', k);  
        return n == 0;  
    }  
}
```

144. 1135.java

Path: LeetCode-main/solutions/1135. Connecting Cities With Minimum Cost/1135.java

```
class UnionFind {
    public UnionFind(int n) {
        id = new int[n];
        rank = new int[n];
        for (int i = 0; i < n; ++i)
            id[i] = i;
    }

    public void unionByRank(int u, int v) {
        final int i = find(u);
        final int j = find(v);
        if (i == j)
            return;
        if (rank[i] < rank[j]) {
            id[i] = j;
        } else if (rank[i] > rank[j]) {
            id[j] = i;
        } else {
            id[i] = j;
            ++rank[j];
        }
    }

    public int find(int u) {
        return id[u] == u ? u : (id[u] = find(id[u]));
    }

    private int[] id;
    private int[] rank;
}

class Solution {
    public int minimumCost(int n, int[][] connections) {
        int ans = 0;
        UnionFind uf = new UnionFind(n + 1);

        // Sort by cost.
        Arrays.sort(connections, Comparator.comparingInt(connection -> connection[2]));

        for (int[] connection : connections) {
            final int u = connection[0];
            final int v = connection[1];
            final int cost = connection[2];
            if (uf.find(u) == uf.find(v))
                continue;
            uf.unionByRank(u, v);
            ans += cost;
        }

        final int root = uf.find(1);
        for (int i = 1; i <= n; ++i)
            if (uf.find(i) != root)
                return -1;

        return ans;
    }
}
```


145. 1136-2.java

Path: LeetCode-main/solutions/1136. Parallel Courses/1136-2.java

```
class Solution {
    public int minimumSemesters(int n, int[][] relations) {
        List<Integer>[] graph = new List[n];
        int[] inDegrees = new int[n];
        Arrays.setAll(graph, i -> new ArrayList<>());

        // Build the graph.
        for (int[] relation : relations) {
            final int u = relation[0] - 1;
            final int v = relation[1] - 1;
            graph[u].add(v);
            ++inDegrees[v];
        }

        // Perform topological sorting.
        Queue<Integer> q = IntStream.range(0, n)
            .filter(i -> inDegrees[i] == 0)
            .boxed()
            .collect(Collectors.toCollection(ArrayDeque::new));

        int step = 0;
        for (; !q.isEmpty(); ++step)
            for (int sz = q.size(); sz > 0; --sz) {
                final int u = q.poll();
                --n;
                for (final int v : graph[u])
                    if (--inDegrees[v] == 0)
                        q.offer(v);
            }

        return n == 0 ? step : -1;
    }
}
```

146. 1136.java

Path: LeetCode-main/solutions/1136. Parallel Courses/1136.java

```
enum State { INIT, VISITING, VISITED }

class Solution {
    public int minimumSemesters(int n, int[][] relations) {
        List<Integer>[] graph = new List[n];
        State[] states = new State[n];
        int[] depth = new int[n];
        Arrays.fill(depth, 1);
        Arrays.setAll(graph, i -> new ArrayList<>());

        for (int[] relation : relations) {
            final int u = relation[0] - 1;
            final int v = relation[1] - 1;
            graph[u].add(v);
        }

        for (int i = 0; i < n; ++i)
            if (hasCycle(graph, i, states, depth))
                return -1;

        return Arrays.stream(depth).max().getAsInt();
    }

    private boolean hasCycle(List<Integer>[] graph, int u, State[] states, int[] depth) {
        if (states[u] == State.VISITING)
            return true;
        if (states[u] == State.VISITED)
            return false;
        states[u] = State.VISITING;
        for (final int v : graph[u]) {
            if (hasCycle(graph, v, states, depth))
                return true;
            depth[u] = Math.max(depth[u], 1 + depth[v]);
        }
        states[u] = State.VISITED;
        return false;
    }
}
```

147. 1137-2.java

Path: LeetCode-main/solutions/1137. N-th Tribonacci Number/1137-2.java

```
class Solution {
    public int tribonacci(int n) {
        int[] dp = {0, 1, 1};

        for (int i = 3; i <= n; ++i)
            dp[i % 3] = dp[0] + dp[1] + dp[2];

        return dp[n % 3];
    }
}
```

148. 1137.java

Path: LeetCode-main/solutions/1137. N-th Tribonacci Number/1137.java

```
class Solution {
    public int tribonacci(int n) {
        if (n < 2)
            return n;

        int[] dp = {0, 1, 1};

        for (int i = 3; i <= n; ++i) {
            final int next = dp[0] + dp[1] + dp[2];
            dp[0] = dp[1];
            dp[1] = dp[2];
            dp[2] = next;
        }

        return dp[2];
    }
}
```

149. 1139.java

Path: LeetCode-main/solutions/1139. Largest 1-Bordered Square/1139.java

```
class Solution {
    public int largest1BorderedSquare(int[][] grid) {
        final int m = grid.length;
        final int n = grid[0].length;

        // leftOnes[i][j] := consecutive 1s in the left of grid[i][j]
        int[][] leftOnes = new int[m][n];
        // topOnes[i][j] := consecutive 1s in the top of grid[i][j]
        int[][] topOnes = new int[m][n];

        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j)
                if (grid[i][j] == 1) {
                    leftOnes[i][j] = j == 0 ? 1 : 1 + leftOnes[i][j - 1];
                    topOnes[i][j] = i == 0 ? 1 : 1 + topOnes[i - 1][j];
                }

        for (int sz = Math.min(m, n); sz > 0; --sz)
            for (int i = 0; i + sz - 1 < m; ++i)
                for (int j = 0; j + sz - 1 < n; ++j) {
                    final int x = i + sz - 1;
                    final int y = j + sz - 1;
                    // If grid[i..x][j..y] has all 1s on its border.
                    if (Math.min(leftOnes[i][y], leftOnes[x][y]) >= sz &&
                        Math.min(topOnes[x][j], topOnes[x][y]) >= sz)
                        return sz * sz;
                }

        return 0;
    }
};
```

150. 114-2.java

Path: LeetCode-main/solutions/114. Flatten Binary Tree to Linked List/114-2.java

```
class Solution {
    public void flatten(TreeNode root) {
        if (root == null)
            return;

        Deque<TreeNode> stack = new ArrayDeque<>();
        stack.push(root);

        while (!stack.isEmpty()) {
            root = stack.pop();
            if (root.right != null)
                stack.push(root.right);
            if (root.left != null)
                stack.push(root.left);
            if (!stack.isEmpty())
                root.right = stack.peek();
            root.left = null;
        }
    }
}
```

151. 114-3.java

Path: LeetCode-main/solutions/114. Flatten Binary Tree to Linked List/114-3.java

```
class Solution {
    public void flatten(TreeNode root) {
        if (root == null)
            return;

        while (root != null) {
            if (root.left != null) {
                // Find the rightmost root.
                TreeNode rightmost = root.left;
                while (rightmost.right != null)
                    rightmost = rightmost.right;
                // Rewire the connections.
                rightmost.right = root.right;
                root.right = root.left;
                root.left = null;
            }
            // Move on to the right side of the tree.
            root = root.right;
        }
    }
}
```

152. 114.java

Path: LeetCode-main/solutions/114. Flatten Binary Tree to Linked List/114.java

```
class Solution {
    public void flatten(TreeNode root) {
        if (root == null)
            return;

        flatten(root.left);
        flatten(root.right);

        TreeNode left = root.left; // flattened left
        TreeNode right = root.right; // flattened right

        root.left = null;
        root.right = left;

        // Connect the original right subtree to the end of the new right subtree.
        TreeNode rightmost = root;
        while (rightmost.right != null)
            rightmost = rightmost.right;
        rightmost.right = right;
    }
}
```


153. 1140.java

Path: LeetCode-main/solutions/1140. Stone Game II/1140.java

```
class Solution {
    public int stoneGameII(int[] piles) {
        final int n = piles.length;
        int[][] mem = new int[n][n];
        int[] suffix = new int[n]; // suffix[i] := sum(piles[i..n))
        Arrays.stream(mem).forEach(A -> Arrays.fill(A, -1));
        suffix[n - 1] = piles[n - 1];
        for (int i = n - 2; i >= 0; --i)
            suffix[i] = suffix[i + 1] + piles[i];
        return stoneGameII(suffix, 0, 1, mem);
    }

    // Returns the maximum number of stones Alice can get from piles[i..n) with M.
    private int stoneGameII(int[] suffix, int i, int M, int[][] mem) {
        if (i + 2 * M >= suffix.length)
            return suffix[i];
        if (mem[i][M] != -1)
            return mem[i][M];

        int opponent = suffix[i];

        for (int X = 1; X <= 2 * M; ++X)
            opponent = Math.min(opponent, stoneGameII(suffix, i + X, Math.max(M, X), mem));

        return mem[i][M] = suffix[i] - opponent;
    }
}
```

154. 1143.java

Path: LeetCode-main/solutions/1143. Longest Common Subsequence/1143.java

```
class Solution {
    public int longestCommonSubsequence(String text1, String text2) {
        final int m = text1.length();
        final int n = text2.length();
        // dp[i][j] := the length of LCS(text1[0..i), text2[0..j))
        int[][] dp = new int[m + 1][n + 1];

        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j)
                dp[i + 1][j + 1] = text1.charAt(i) == text2.charAt(j)
                    ? 1 + dp[i][j]
                    : Math.max(dp[i][j + 1], dp[i + 1][j]);

        return dp[m][n];
    }
}
```

155. 1144.java

Path: LeetCode-main/solutions/1144. Decrease Elements To Make Array Zigzag/1144.java

```
class Solution {
    public int movesToMakeZigzag(int[] nums) {
        int[] decreasing = new int[2];

        for (int i = 0; i < nums.length; ++i) {
            int l = i > 0 ? nums[i - 1] : 1001;
            int r = i + 1 < nums.length ? nums[i + 1] : 1001;
            decreasing[i % 2] += Math.max(0, nums[i] - Math.min(l, r) + 1);
        }

        return Math.min(decreasing[0], decreasing[1]);
    }
}
```

156. 1145.java

Path: LeetCode-main/solutions/1145. Binary Tree Coloring Game/1145.java

```
class Solution {
    public boolean btreeGameWinningMove(TreeNode root, int n, int x) {
        count(root, x);
        return Math.max(Math.max(leftCount, rightCount), n - leftCount - rightCount - 1) > n / 2;
    }

    private int leftCount; // the number of nodes of n's left
    private int rightCount; // the number of nodes of n's right

    private int count(TreeNode root, int x) {
        if (root == null)
            return 0;
        final int l = count(root.left, x);
        final int r = count(root.right, x);
        if (root.val == x) {
            leftCount = l;
            rightCount = r;
        }
        return 1 + l + r;
    }
}
```

157. 1146.java

Path: LeetCode-main/solutions/1146. Snapshot Array/1146.java

```
class SnapshotArray {
    public SnapshotArray(int length) {
        snaps = new List[length];
        for (int i = 0; i < length; ++i) {
            snaps[i] = new ArrayList<>();
            snaps[i].add(new int[] {0, 0});
        }
    }

    public void set(int index, int val) {
        int[] snap = snaps[index].get(snaps[index].size() - 1);
        if (snap[0] == snap_id)
            snap[1] = val;
        else
            snaps[index].add(new int[] {snap_id, val});
    }

    public int snap() {
        return snap_id++;
    }

    public int get(int index, int snap_id) {
        int i = Collections.binarySearch(snaps[index], new int[] {snap_id, 0},
                                         Comparator.comparingInt(snap -> snap[0]));
        if (i < 0)
            i = -i - 2;
        return snaps[index].get(i)[1];
    }

    private List<int[]>[] snaps;
    private int snap_id = 0;
}
```

158. 1147.java

Path: LeetCode-main/solutions/1147. Longest Chunked Palindrome Decomposition/1147.java

```
class Solution {
    public int longestDecomposition(String text) {
        final int n = text.length();

        int count = 0;
        int l = 0;

        for (int r = 1; 2 * r <= n; ++r)
            if (text.substring(n - r).startsWith(text.substring(l, r))) {
                count += 2;
                l = r;
            }

        return count + (2 * l < n ? 1 : 0);
    }
}
```

159. 115-2.java

Path: LeetCode-main/solutions/115. Distinct Subsequences/115-2.java

```
class Solution {
    public int numDistinct(String s, String t) {
        final int m = s.length();
        final int n = t.length();
        long[] dp = new long[n + 1];
        dp[0] = 1;

        for (int i = 1; i <= m; ++i)
            for (int j = n; j >= 1; --j)
                if (s.charAt(i - 1) == t.charAt(j - 1))
                    dp[j] += dp[j - 1];

        return (int) dp[n];
    }
}
```

160. 115.java

Path: LeetCode-main/solutions/115. Distinct Subsequences/115.java

```
class Solution {
    public int numDistinct(String s, String t) {
        final int m = s.length();
        final int n = t.length();
        long[][] dp = new long[m + 1][n + 1];

        for (int i = 0; i <= m; ++i)
            dp[i][0] = 1;

        for (int i = 1; i <= m; ++i)
            for (int j = 1; j <= n; ++j)
                if (s.charAt(i - 1) == t.charAt(j - 1))
                    dp[i][j] = dp[i - 1][j - 1] + dp[i - 1][j];
                else
                    dp[i][j] = dp[i - 1][j];

        return (int) dp[m][n];
    }
}
```


161. 1150.java

Path: LeetCode-main/solutions/1150. Check If a Number Is Majority Element in a Sorted Array/1150.java

```
class Solution {
    public boolean isMajorityElement(int[] nums, int target) {
        final int n = nums.length;
        final int i = firstGreaterEqual(nums, target);
        return i + n / 2 < n && nums[i + n / 2] == target;
    }

    private int firstGreaterEqual(int[] arr, int target) {
        int l = 0;
        int r = arr.length;
        while (l < r) {
            final int m = (l + r) / 2;
            if (arr[m] >= target)
                r = m;
            else
                l = m + 1;
        }
        return l;
    }
}
```

162. 1151.java

Path: LeetCode-main/solutions/1151. Minimum Swaps to Group All 1's Together/1151.java

```
class Solution {
    public int minSwaps(int[] data) {
        final int k = (int) Arrays.stream(data).filter(a -> a == 1).count();
        int ones = 0; // the number of ones in the window
        int maxOnes = 0; // the maximum number of ones in the window

        for (int i = 0; i < data.length; ++i) {
            if (i >= k && data[i - k] == 1)
                --ones;
            if (data[i] == 1)
                ++ones;
            maxOnes = Math.max(maxOnes, ones);
        }

        return k - maxOnes;
    }
}
```

163. 1153.java

Path: LeetCode-main/solutions/1153. String Transforms Into Another String/1153.java

```
class Solution {
    public boolean canConvert(String str1, String str2) {
        if (str1.equals(str2))
            return true;

        Map<Character, Character> mappings = new HashMap<>();

        for (int i = 0; i < str1.length(); ++i) {
            final char a = str1.charAt(i);
            final char b = str2.charAt(i);
            if (mappings.getOrDefault(a, b) != b)
                return false;
            mappings.put(a, b);
        }

        // No letter in the str1 maps to > 1 letter in the str2 and there is at
        // least one temporary letter can break any loops.
        return new HashSet<>(mappings.values()).size() < 26;
    }
}
```

164. 1154.java

Path: LeetCode-main/solutions/1154. Day of the Year/1154.java

```
class Solution {
    public int dayOfYear(String date) {
        int ans = 0;
        int year = Integer.valueOf(date.substring(0, 4));
        int month = Integer.valueOf(date.substring(5, 7));
        int day = Integer.valueOf(date.substring(8));
        int[] days = new int[] {31, isLeapYear(year) ? 29 : 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};

        for (int i = 0; i < month - 1; ++i)
            ans += days[i];

        return ans + day;
    }

    private boolean isLeapYear(int year) {
        return (year % 4 == 0 && year % 100 != 0) || year % 400 == 0;
    }
}
```

165. 1155.java

Path: LeetCode-main/solutions/1155. Number of Dice Rolls With Target Sum/1155.java

```
class Solution {
    public int numRollsToTarget(int n, int k, int target) {
        final int MOD = 1_000_000_007;
        int[] dp = new int[target + 1];
        dp[0] = 1;

        while (n-- > 0) { // n dices
            int[] newDp = new int[target + 1];
            for (int i = 1; i <= k; ++i) // numbers 1, 2, ..., k
                for (int t = i; t <= target; ++t) { // all the possible targets
                    newDp[t] += dp[t - i];
                    newDp[t] %= MOD;
                }
            dp = newDp;
        }

        return dp[target];
    }
}
```

166. 1156.java

Path: LeetCode-main/solutions/1156. Swap For Longest Repeated Character Substring/1156.java

```
class Solution {
    public int maxRepOpt1(String text) {
        int ans = 0;
        int[] count = new int[26];
        List<int[]> groups = new ArrayList<>();

        for (final char c : text.toCharArray())
            ++count[c - 'a'];

        groups.add(new int[] {text.charAt(0), 1});

        for (int i = 1; i < text.length(); ++i)
            if (text.charAt(i) == text.charAt(i - 1))
                ++groups.get(groups.size() - 1)[1];
            else
                groups.add(new int[] {text.charAt(i), 1});

        for (int[] group : groups)
            ans = Math.max(ans, Math.min(group[1] + 1, count[group[0] - 'a']));

        for (int i = 1; i + 1 < groups.size(); ++i)
            if (groups.get(i - 1)[0] == groups.get(i + 1)[0] && groups.get(i)[1] == 1)
                ans = Math.max(ans, Math.min(groups.get(i - 1)[1] + groups.get(i + 1)[1] + 1,
                                                count[groups.get(i - 1)[0] - 'a']));

        return ans;
    }
}
```

167. 1157.java

Path: LeetCode-main/solutions/1157. Online Majority Element In Subarray/1157.java

```
class MajorityChecker {
    public MajorityChecker(int[] arr) {
        this.arr = arr;
        for (int i = 0; i < arr.length; ++i) {
            if (!numToIndices.containsKey(arr[i]))
                numToIndices.put(arr[i], new ArrayList<>());
            numToIndices.get(arr[i]).add(i);
        }
    }

    public int query(int left, int right, int threshold) {
        for (int i = 0; i < TIMES; ++i) {
            final int randIndex = rand.nextInt(right - left + 1) + left;
            final int num = arr[randIndex];
            List<Integer> indices = numToIndices.get(num);
            final int l = firstGreaterEqual(indices, left);
            final int r = firstGreaterEqual(indices, right + 1);
            if (r - l >= threshold)
                return num;
        }
        return -1;
    }

    private static final int TIMES = 20; // 2^TIMES >> |arr|
    private int[] arr;
    private Map<Integer, List<Integer>> numToIndices = new HashMap<>();
    private Random rand = new Random();

    private int firstGreaterEqual(List<Integer> A, int target) {
        final int i = Collections.binarySearch(A, target);
        return i < 0 ? -i - 1 : i;
    }
}
```

168. 116-2.java

Path: LeetCode-main/solutions/116. Populating Next Right Pointers in Each Node/116-2.java

```
class Solution {
    public Node connect(Node root) {
        Node node = root; // the node that is above the current needling

        while (node != null && node.left != null) {
            Node dummy = new Node(); // a dummy node before needling
            // Needle the children of the node.
            for (Node needle = dummy; node != null; node = node.next) {
                needle.next = node.left;
                needle = needle.next;
                needle.next = node.right;
                needle = needle.next;
            }
            node = dummy.next; // Move the node to the next level.
        }

        return root;
    }
}
```


169. 116.java

Path: LeetCode-main/solutions/116. Populating Next Right Pointers in Each Node/116.java

```
class Solution {
    public Node connect(Node root) {
        if (root == null)
            return null;
        connectTwoNodes(root.left, root.right);
        return root;
    }

    private void connectTwoNodes(Node p, Node q) {
        if (p == null)
            return;
        p.next = q;
        connectTwoNodes(p.left, p.right);
        connectTwoNodes(q.left, q.right);
        connectTwoNodes(p.right, q.left);
    }
}
```

170. 1160.java

Path: LeetCode-main/solutions/1160. Find Words That Can Be Formed by Characters/1160.java

```
class Solution {
    public int countCharacters(String[] words, String chars) {
        int ans = 0;
        int[] count = new int[26];

        for (final char c : chars.toCharArray())
            ++count[c - 'a'];

        for (final String word : words) {
            int[] tempCount = count.clone();
            for (final char c : word.toCharArray())
                if (--tempCount[c - 'a'] < 0) {
                    ans -= word.length();
                    break;
                }
            ans += word.length();
        }

        return ans;
    }
}
```

171. 1161-2.java

Path: LeetCode-main/solutions/1161. Maximum Level Sum of a Binary Tree/1161-2.java

```
class Solution {
    public int maxLevelSum(TreeNode root) {
        // levelSums[i] := the sum of level (i + 1) (1-indexed)
        List<Integer> levelSums = new ArrayList<>();
        dfs(root, 0, levelSums);
        return 1 + IntStream.range(0, levelSums.size())
            .reduce(0, (a, b) -> levelSums.get(a) < levelSums.get(b) ? b : a);
    }

    private void dfs(TreeNode root, int level, List<Integer> levelSums) {
        if (root == null)
            return;
        if (levelSums.size() == level)
            levelSums.add(0);
        levelSums.set(level, levelSums.get(level) + root.val);
        dfs(root.left, level + 1, levelSums);
        dfs(root.right, level + 1, levelSums);
    }
}
```

172. 1161.java

Path: LeetCode-main/solutions/1161. Maximum Level Sum of a Binary Tree/1161.java

```
class Solution {
    public int maxLevelSum(TreeNode root) {
        int ans = 0;
        int maxLevelSum = Integer.MIN_VALUE;
        Queue<TreeNode> q = new LinkedList<>(Arrays.asList(root));

        for (int level = 1; !q.isEmpty(); ++level) {
            int levelSum = 0;
            for (int sz = q.size(); sz > 0; --sz) {
                TreeNode node = q.poll();
                levelSum += node.val;
                if (node.left != null)
                    q.offer(node.left);
                if (node.right != null)
                    q.offer(node.right);
            }
            if (levelSum > maxLevelSum) {
                maxLevelSum = levelSum;
                ans = level;
            }
        }

        return ans;
    }
}
```

173. 1162.java

Path: LeetCode-main/solutions/1162. As Far from Land as Possible/1162.java

```
class Solution {
    public int maxDistance(int[][] grid) {
        final int[][] DIRS = {{0, 1}, {1, 0}, {0, -1}, {-1, 0}};
        final int m = grid.length;
        final int n = grid[0].length;
        Queue<Pair<Integer, Integer>> q = new ArrayDeque<>();
        int water = 0;

        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j)
                if (grid[i][j] == 0)
                    ++water;
                else
                    q.offer(new Pair<>(i, j));

        if (water == 0 || water == m * n)
            return -1;

        int ans = 0;

        for (int d = 0; !q.isEmpty(); ++d)
            for (int sz = q.size(); sz > 0; --sz) {
                Pair<Integer, Integer> pair = q.poll();
                final int i = pair.getKey();
                final int j = pair.getValue();
                ans = d;
                for (int[] dir : DIRS) {
                    final int x = i + dir[0];
                    final int y = j + dir[1];
                    if (x < 0 || x == m || y < 0 || y == n)
                        continue;
                    if (grid[x][y] > 0)
                        continue;
                    q.offer(new Pair<>(x, y));
                    grid[x][y] = 2; // Mark as visited.
                }
            }

        return ans;
    }
}
```

174. 1163.java

Path: LeetCode-main/solutions/1163. Last Substring in Lexicographical Order/1163.java

```
class Solution {
    public String lastSubstring(String s) {
        int i = 0;
        int j = 1;
        int k = 0; // the number of the same letters of s[i..n) and s[j..n)

        while (j + k < s.length())
            if (s.charAt(i + k) == s.charAt(j + k)) {
                ++k;
            } else if (s.charAt(i + k) > s.charAt(j + k)) {
                // Skip s[j..j + k) and advance to s[j + k + 1] to find a possible
                // lexicographically larger substring since s[i..i + k) == s[j..j + k)
                // and s[i + k] > s[j + k].
                j = j + k + 1;
                k = 0;
            } else {
                // Skip s[i..i + k) and advance to s[i + k + 1] or s[j] to find a
                // possible lexicographically larger substring since
                // s[i..i + k) == s[j..j + k) and s[i + k] < s[j + k].
                // Note that it's unnecessary to explore s[i + k + 1..j) if
                // i + k + 1 < j since they are already explored by j.
                i = Math.max(i + k + 1, j);
                j = i + 1;
                k = 0;
            }
        return s.substring(i);
    }
}
```

175. 1165.java

Path: LeetCode-main/solutions/1165. Single-Row Keyboard/1165.java

```
class Solution {
    public int calculateTime(String keyboard, String word) {
        int ans = 0;
        int prevIndex = 0;
        int[] letterToIndex = new int[26];

        for (int i = 0; i < keyboard.length(); ++i)
            letterToIndex[keyboard.charAt(i) - 'a'] = i;

        for (final char c : word.toCharArray()) {
            final int currIndex = letterToIndex[c - 'a'];
            ans += Math.abs(currIndex - prevIndex);
            prevIndex = currIndex;
        }

        return ans;
    }
}
```

176. 1166.java

Path: LeetCode-main/solutions/1166. Design File System/1166.java

```
class TrieNode {
    public Map<String, TrieNode> children = new HashMap<>();
    public int value;
    public TrieNode(int value) {
        this.value = value;
    }
}

class FileSystem {
    public boolean createPath(String path, int value) {
        final String[] subpaths = path.split("/");
        TrieNode node = root;

        for (int i = 1; i < subpaths.length - 1; ++i) {
            if (!node.children.containsKey(subpaths[i]))
                return false;
            node = node.children.get(subpaths[i]);
        }

        final String lastSubpath = subpaths[subpaths.length - 1];
        if (node.children.containsKey(lastSubpath))
            return false;
        node.children.put(lastSubpath, new TrieNode(value));
        return true;
    }

    public int get(String path) {
        final String[] subpaths = path.split("/");
        TrieNode node = root;
        for (int i = 1; i < subpaths.length; ++i) {
            if (!node.children.containsKey(subpaths[i]))
                return -1;
            node = node.children.get(subpaths[i]);
        }
        return node.value;
    }
}

private TrieNode root = new TrieNode(0);
}
```


177. 1167.java

Path: LeetCode-main/solutions/1167. Minimum Cost to Connect Sticks/1167.java

```
class Solution {
    public int connectSticks(int[] sticks) {
        int ans = 0;
        Queue<Integer> minHeap = new PriorityQueue<>();

        for (final int stick : sticks)
            minHeap.offer(stick);

        while (minHeap.size() > 1) {
            final int x = minHeap.poll();
            final int y = minHeap.poll();
            ans += x + y;
            minHeap.offer(x + y);
        }

        return ans;
    }
}
```

178. 1168.java

Path: LeetCode-main/solutions/1168. Optimize Water Distribution in a Village/1168.java

```
class Solution {
    public int minCostToSupplyWater(int n, int[] wells, int[][] pipes) {
        int ans = 0;
        List<Pair<Integer, Integer>>[] graph = new List[n + 1];
        Queue<Pair<Integer, Integer>> minHeap =
            new PriorityQueue<>(Comparator.comparingInt(Pair::getKey)); // (d, u)

        for (int i = 0; i <= n; ++i)
            graph[i] = new ArrayList<>();

        for (int[] pipe : pipes) {
            final int u = pipe[0];
            final int v = pipe[1];
            final int w = pipe[2];
            graph[u].add(new Pair<>(v, w));
            graph[v].add(new Pair<>(u, w));
        }

        // Connect virtual 0 with nodes 1 to n.
        for (int i = 0; i < n; ++i) {
            graph[0].add(new Pair<>(i + 1, wells[i]));
            minHeap.offer(new Pair<>(wells[i], i + 1));
        }

        Set<Integer> mst = new HashSet<>(Arrays.asList(0));

        while (mst.size() < n + 1) {
            final int d = minHeap.peek().getKey();
            final int u = minHeap.poll().getValue();
            if (mst.contains(u))
                continue;
            // Add the new vertex.
            mst.add(u);
            ans += d;
            // Expand if possible.
            for (Pair<Integer, Integer> pair : graph[u]) {
                final int v = pair.getKey();
                final int w = pair.getValue();
                if (!mst.contains(v))
                    minHeap.offer(new Pair<>(w, v));
            }
        }

        return ans;
    }
}
```

179. 1169.java

Path: LeetCode-main/solutions/1169. Invalid Transactions/1169.java

```
class Solution {
    public List<String> invalidTransactions(String[] transactions) {
        List<String> ans = new ArrayList<>();
        Map<String, List<Trans>> nameToTrans = new HashMap<>();

        for (final String t : transactions) {
            Trans trans = getTrans(t);
            nameToTrans.putIfAbsent(trans.name, new ArrayList<>());
            nameToTrans.get(trans.name).add(trans);
        }

        for (final String t : transactions) {
            Trans currTrans = getTrans(t);
            if (currTrans.amount > 1000) {
                ans.add(t);
            } else if (nameToTrans.containsKey(currTrans.name)) {
                // Iterate through all the transactions with the same name, check if
                // they're within 60 minutes in a different city.
                for (Trans trans : nameToTrans.get(currTrans.name))
                    if (Math.abs(trans.time - currTrans.time) <= 60 && !trans.city.equals(currTrans.city)) {
                        ans.add(t);
                        break;
                    }
            }
        }

        return ans;
    }

    private record Trans(String name, int time, int amount, String city) {}

    private Trans getTrans(final String t) {
        String[] s = t.split(","); // [name, time, amount, city]
        return new Trans(s[0], Integer.parseInt(s[1]), Integer.parseInt(s[2]), s[3]);
    }
}
```

180. 117.java

Path: LeetCode-main/solutions/117. Populating Next Right Pointers in Each Node II/117.java

```
class Solution {
    public Node connect(Node root) {
        Node node = root; // the node that is above the current needling

        while (node != null) {
            Node dummy = new Node(); // a dummy node before needling
            // Needle the children of the node.
            for (Node needle = dummy; node != null; node = node.next) {
                if (node.left != null) { // Needle the left child.
                    needle.next = node.left;
                    needle = needle.next;
                }
                if (node.right != null) { // Needle the right child.
                    needle.next = node.right;
                    needle = needle.next;
                }
            }
            node = dummy.next; // Move the node to the next level.
        }

        return root;
    }
}
```

181. 1170.java

Path: LeetCode-main/solutions/1170. Compare Strings by Frequency of the Smallest Character/1170.java

```
class Solution {
    public int[] numSmallerByFrequency(String[] queries, String[] words) {
        int[] ans = new int[queries.length];
        int[] wordsFreq = new int[words.length];

        for (int i = 0; i < words.length; ++i)
            wordsFreq[i] = f(words[i]);
        Arrays.sort(wordsFreq);

        for (int i = 0; i < queries.length; ++i) {
            final int freq = f(queries[i]);
            ans[i] = words.length - firstGreater(wordsFreq, 0, wordsFreq.length, freq);
        }

        return ans;
    }

    private int f(final String word) {
        int count = 0;
        char currentChar = 'z' + 1;

        for (final char c : word.toCharArray())
            if (c < currentChar) {
                currentChar = c;
                count = 1;
            } else if (c == currentChar) {
                ++count;
            }

        return count;
    }

    private int firstGreater(int[] nums, int l, int r, int value) {
        while (l < r) {
            final int m = (l + r) / 2;
            if (nums[m] > value)
                r = m;
            else
                l = m + 1;
        }
        return l;
    }
}
```

182. 1171.java

Path: LeetCode-main/solutions/1171. Remove Zero Sum Consecutive Nodes from Linked List/1171.java

```
class Solution {
    public ListNode removeZeroSumSublists(ListNode head) {
        ListNode dummy = new ListNode(0, head);
        int prefix = 0;
        Map<Integer, ListNode> prefixToNode = new HashMap<>();
        prefixToNode.put(0, dummy);

        for (; head != null; head = head.next) {
            prefix += head.val;
            prefixToNode.put(prefix, head);
        }

        prefix = 0;

        for (head = dummy; head != null; head = head.next) {
            prefix += head.val;
            head.next = prefixToNode.get(prefix).next;
        }

        return dummy.next;
    }
}
```

183. 1172.java

Path: LeetCode-main/solutions/1172. Dinner Plate Stacks/1172.java

```
class DinnerPlates {
    public DinnerPlates(int capacity) {
        this.capacity = capacity;
        minHeap.offer(0);
    }

    public void push(int val) {
        final int index = minHeap.peek();
        // Add a new stack on demand.
        if (index == stacks.size())
            stacks.add(new ArrayDeque<>());
        // Push the new value.
        stacks.get(index).push(val);
        // If the stack pushed is full, remove its candidacy from `minHeap`.
        if (stacks.get(index).size() == capacity) {
            minHeap.poll();
            // If `minHeap` is empty, the next available stack index is |stacks|.
            if (minHeap.isEmpty())
                minHeap.offer(stacks.size());
        }
    }

    public int pop() {
        // Remove empty stacks from the back.
        while (!stacks.isEmpty() && stacks.get(stacks.size() - 1).isEmpty())
            stacks.remove(stacks.size() - 1);
        if (stacks.isEmpty())
            return -1;
        return popAtStack(stacks.size() - 1);
    }

    public int popAtStack(int index) {
        if (index >= stacks.size() || stacks.get(index).isEmpty())
            return -1;
        // If the stack is going to have space, add its candidacy to `minHeap`.
        if (stacks.get(index).size() == capacity)
            minHeap.offer(index);
        return stacks.get(index).pop();
    }

    private final int capacity;
    private List<Deque<Integer>> stacks = new ArrayList<>();
    private Queue<Integer> minHeap = new PriorityQueue<>();
}
```

184. 1175.java

Path: LeetCode-main/solutions/1175. Prime Arrangements/1175.java

```
class Solution {
    public int numPrimeArrangements(int n) {
        final int count = countPrimes(n);
        return (int) (factorial(count) * factorial(n - count) % MOD);
    }

    private static final int MOD = 1_000_000_007;

    private int countPrimes(int n) {
        boolean[] prime = new boolean[n + 1];
        Arrays.fill(prime, 2, n + 1, true);

        for (int i = 0; i * i <= n; ++i)
            if (prime[i])
                for (int j = i * i; j <= n; j += i)
                    prime[j] = false;

        int count = 0;

        for (boolean p : prime)
            if (p)
                ++count;

        return count;
    }

    private long factorial(int n) {
        long res = 1;
        for (int i = 2; i <= n; ++i)
            res = res * i % MOD;
        return res;
    }
}
```


185. 1176.java

Path: LeetCode-main/solutions/1176. Diet Plan Performance/1176.java

```
class Solution {
    public int dietPlanPerformance(int[] calories, int k, int lower, int upper) {
        int ans = 0;
        int sum = 0;

        for (int i = 0; i < calories.length; ++i) {
            sum += calories[i];
            if (i < k - 1)
                continue;
            if (i >= k)
                sum -= calories[i - k];
            if (sum < lower)
                --ans;
            else if (sum > upper)
                ++ans;
        }
        return ans;
    }
}
```

186. 1177.java

Path: LeetCode-main/solutions/1177. Can Make Palindrome from Substring/1177.java

```
class Solution {
    public List<Boolean> canMakePaliQueries(String s, int[][] queries) {
        List<Boolean> ans = new ArrayList<>();
        int[] dp = new int[s.length() + 1];

        for (int i = 1; i <= s.length(); ++i)
            dp[i] = dp[i - 1] ^ 1 << s.charAt(i - 1) - 'a';

        for (int[] query : queries) {
            int odds = Integer.bitCount(dp[query[1] + 1] ^ dp[query[0]]);
            ans.add(odds / 2 <= query[2]);
        }

        return ans;
    }
}
```

187. 1178.java

Path: LeetCode-main/solutions/1178. Number of Valid Words for Each Puzzle/1178.java

```
class Solution {
    public List<Integer> findNumOfValidWords(String[] words, String[] puzzles) {
        List<Integer> ans = new ArrayList<>();
        Map<Integer, Integer> binaryCount = new HashMap<>();

        for (final String word : words) {
            int mask = 0;
            for (char c : word.toCharArray())
                mask |= 1 << (c - 'a');
            binaryCount.merge(mask, 1, Integer::sum);
        }

        for (final String puzzle : puzzles) {
            int valid = 0;
            final int n = puzzle.length() - 1;
            for (int i = 0; i < (1 << n); ++i) {
                int mask = 1 << puzzle.charAt(0) - 'a';
                for (int j = 0; j < n; ++j)
                    if ((i >> j & 1) == 1)
                        mask |= 1 << puzzle.charAt(j + 1) - 'a';
                if (binaryCount.containsKey(mask))
                    valid += binaryCount.get(mask);
            }
            ans.add(valid);
        }

        return ans;
    }
}
```

188. 118.java

Path: LeetCode-main/solutions/118. Pascal's Triangle/118.java

```
class Solution {
    public List<List<Integer>> generate(int numRows) {
        List<List<Integer>> ans = new ArrayList<>();

        for (int i = 0; i < numRows; ++i) {
            Integer[] temp = new Integer[i + 1];
            Arrays.fill(temp, 1);
            ans.add(Arrays.asList(temp));
        }

        for (int i = 2; i < numRows; ++i)
            for (int j = 1; j < ans.get(i).size() - 1; ++j)
                ans.get(i).set(j, ans.get(i - 1).get(j - 1) + ans.get(i - 1).get(j));

        return ans;
    }
}
```

189. 1180.java

Path: LeetCode-main/solutions/1180. Count Substrings with Only One Distinct Letter/1180.java

```
class Solution {
    public int countLetters(String s) {
        int ans = 0;
        int dp = 0;          // the length of the running letter
        char letter = '@';   // the running letter

        for (final char c : s.toCharArray()) {
            if (c == letter) {
                ++dp;
            } else {
                dp = 1;
                letter = c;
            }
            // Add the number of substrings ending in the current letter.
            ans += dp;
        }

        return ans;
    }
}
```

190. 1182.java

Path: LeetCode-main/solutions/1182. Shortest Distance to Target Color/1182.java

```
class Solution {
    public List<Integer> shortestDistanceColor(int[] colors, int[][] queries) {
        final int NUM_COLOR = 3;
        final int n = colors.length;
        List<Integer> ans = new ArrayList<>();
        // left[i][c] := the closest index of color c in index i to the left
        int[][] left = new int[n][NUM_COLOR + 1];
        // right[i][c] := the closest index of color c in index i to the right
        int[][] right = new int[n][NUM_COLOR + 1];

        int[] colorToClosestIndex = {0, -1, -1, -1}; // 0-indexed, -1 := N/A
        for (int i = 0; i < n; ++i) {
            colorToClosestIndex[colors[i]] = i;
            for (int c = 1; c <= NUM_COLOR; ++c)
                left[i][c] = colorToClosestIndex[c];
        }

        colorToClosestIndex = new int[] {0, -1, -1, -1}; // Reset
        for (int i = n - 1; i >= 0; --i) {
            colorToClosestIndex[colors[i]] = i;
            for (int c = 1; c <= NUM_COLOR; ++c)
                right[i][c] = colorToClosestIndex[c];
        }

        for (int[] query : queries) {
            final int i = query[0];
            final int c = query[1];
            final int leftDist = left[i][c] == -1 ? Integer.MAX_VALUE : i - left[i][c];
            final int rightDist = right[i][c] == -1 ? Integer.MAX_VALUE : right[i][c] - i;
            final int minDist = Math.min(leftDist, rightDist);
            ans.add(minDist == Integer.MAX_VALUE ? -1 : minDist);
        }

        return ans;
    }
}
```

191. 1183-2.java

Path: LeetCode-main/solutions/1183. Maximum Number of Ones/1183-2.java

```
class Solution {
    public int maximumNumberOfOnes(int width, int height, int sideLength, int maxOnes) {
        int ans = 0;
        List<Integer> subCount = new ArrayList<>();

        for (int i = 0; i < sideLength; ++i)
            for (int j = 0; j < sideLength; ++j)
                subCount.add(getCount(width, i, sideLength) * getCount(height, j, sideLength));

        Collections.sort(subCount, Collections.reverseOrder());

        for (int i = 0; i < maxOnes; ++i)
            ans += subCount.get(i);

        return ans;
    }

    private int getCount(int length, int index, int sideLength) {
        return (length - index - 1) / sideLength + 1;
    }
}
```

192. 1183.java

Path: LeetCode-main/solutions/1183. Maximum Number of Ones/1183.java

```
class Solution {
    public int maximumNumberOfOnes(int width, int height, int sideLength, int maxOnes) {
        int ans = 0;
        int[][] submatrix = new int[sideLength][sideLength];
        Queue<Integer> maxHeap = new PriorityQueue<>(Comparator.reverseOrder());

        for (int i = 0; i < width; ++i)
            for (int j = 0; j < height; ++j)
                ++submatrix[i % sideLength][j % sideLength];

        for (int[] row : submatrix)
            for (final int a : row)
                maxHeap.offer(a);

        for (int i = 0; i < maxOnes; ++i)
            ans += maxHeap.poll();

        return ans;
    }
}
```


193. 1184.java

Path: LeetCode-main/solutions/1184. Distance Between Bus Stops/1184.java

```
class Solution {
    public int distanceBetweenBusStops(int[] distance, int start, int destination) {
        int clockwise = 0;
        int counterclockwise = 0;

        if (start > destination) {
            int temp = start;
            start = destination;
            destination = temp;
        }

        for (int i = 0; i < distance.length; ++i) {
            if (i >= start && i < destination)
                clockwise += distance[i];
            else
                counterclockwise += distance[i];
        }

        return Math.min(clockwise, counterclockwise);
    }
}
```

194. 1185.java

Path: LeetCode-main/solutions/1185. Day of the Week/1185.java

```
class Solution {
    public String dayOfTheWeek(int day, int month, int year) {
        String[] week = {"Sunday", "Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday"};
        int[] days = {31, isLeapYear(year) ? 29 : 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};
        int count = 0;

        for (int i = 1971; i < year; ++i)
            count += i % 4 == 0 ? 366 : 365;
        for (int i = 0; i < month - 1; ++i)
            count += days[i];
        count += day;

        return week[(count + 4) % 7];
    }

    private boolean isLeapYear(int year) {
        return (year % 4 == 0 && year % 100 != 0) || year % 400 == 0;
    }
}
```

195. 1186-2.java

Path: LeetCode-main/solutions/1186. Maximum Subarray Sum with One Deletion/1186-2.java

```
class Solution {
    // Similar to 53. Maximum Subarray
    public int maximumSum(int[] arr) {
        final int MIN = Integer.MIN_VALUE / 2;
        int ans = MIN;
        int zero = MIN; // no deletion
        int one = MIN;  // <= 1 deletion

        for (final int a : arr) {
            one = Math.max(a, Math.max(one + a, zero /*delete a*/));
            zero = Math.max(a, zero + a);
            ans = Math.max(ans, one);
        }

        return ans;
    }
}
```

196. 1186.java

Path: LeetCode-main/solutions/1186. Maximum Subarray Sum with One Deletion/1186.java

```
class Solution {
    // Similar to 53. Maximum Subarray
    public int maximumSum(int[] arr) {
        // dp[0][i] := the maximum sum subarray ending in i (no deletion)
        // dp[1][i] := the maximum sum subarray ending in i (at most 1 deletion)
        int[][] dp = new int[2][arr.length];

        dp[0][0] = arr[0];
        dp[1][0] = arr[0];
        for (int i = 1; i < arr.length; ++i) {
            dp[0][i] = Math.max(arr[i], dp[0][i - 1] + arr[i]);
            dp[1][i] = Math.max(arr[i], Math.max(dp[1][i - 1] + arr[i], dp[0][i - 1] /*delete arr[i]*/));
        }

        return Arrays.stream(dp[1]).max().getAsInt();
    }
}
```

197. 1187.java

Path: LeetCode-main/solutions/1187. Make Array Strictly Increasing/1187.java

```
class Solution {
    public int makeArrayIncreasing(int[] arr1, int[] arr2) {
        // dp[i] := the minimum steps to reach i at previous round
        Map<Integer, Integer> dp = new HashMap<>();
        dp.put(-1, 0);

        Arrays.sort(arr2);

        for (final int a : arr1) {
            Map<Integer, Integer> newDp = new HashMap<>();
            for (final int val : dp.keySet()) {
                final int steps = dp.get(val);
                // It's possible to use the value in the arr1.
                if (a > val)
                    newDp.put(a, Math.min(newDp.getOrDefault(a, Integer.MAX_VALUE), steps));
                // Also try the value in the arr2.
                final int i = firstGreater(arr2, val);
                if (i < arr2.length)
                    newDp.put(arr2[i], Math.min(newDp.getOrDefault(arr2[i], Integer.MAX_VALUE), steps + 1));
            }
            if (newDp.isEmpty())
                return -1;
            dp = newDp;
        }

        return Collections.min(dp.values());
    }

    private int firstGreater(int[] arr, int target) {
        final int i = Arrays.binarySearch(arr, target + 1);
        return i < 0 ? -i - 1 : i;
    }
}
```

198. 1188.java

Path: LeetCode-main/solutions/1188. Design Bounded Blocking Queue/1188.java

```
class BoundedBlockingQueue {
    public BoundedBlockingQueue(int capacity) {
        this.enqueueSemaphore = new Semaphore(capacity);
    }

    public void enqueue(int element) throws InterruptedException {
        enqueueSemaphore.acquire();
        q.offer(element);
        dequeueSemaphore.release();
    }

    public int dequeue() throws InterruptedException {
        dequeueSemaphore.acquire();
        final int element = q.poll();
        enqueueSemaphore.release();
        return element;
    }

    public int size() {
        return q.size();
    }

    private Queue<Integer> q = new ArrayDeque<>();
    private Semaphore enqueueSemaphore;
    private Semaphore dequeueSemaphore = new Semaphore(0);
}
```

199. 1189.java

Path: LeetCode-main/solutions/1189. Maximum Number of Balloons/1189.java

```
class Solution {
    public int maxNumberOfBalloons(String text) {
        int ans = Integer.MAX_VALUE;
        int[] count = new int[26];

        for (char c : text.toCharArray())
            ++count[c - 'a'];

        for (char c : new char[] { 'b', 'a', 'n' })
            ans = Math.min(ans, count[c - 'a']);

        for (char c : new char[] { 'o', 'l' })
            ans = Math.min(ans, count[c - 'a'] / 2);

        return ans;
    }
}
```

200. 119.java

Path: LeetCode-main/solutions/119. Pascal's Triangle II/119.java

```
class Solution {
    public List<Integer> getRow(int rowIndex) {
        Integer[] ans = new Integer[rowIndex + 1];
        Arrays.fill(ans, 1);

        for (int i = 2; i < rowIndex + 1; ++i)
            for (int j = 1; j < i; ++j)
                ans[i - j] += ans[i - j - 1];

        return Arrays.asList(ans);
    }
}
```


201. 1190-2.java

Path: LeetCode-main/solutions/1190. Reverse Substrings Between Each Pair of Parentheses/1190-2.java

```
class Solution {
    public String reverseParentheses(String s) {
        StringBuilder sb = new StringBuilder();
        Deque<Integer> stack = new ArrayDeque<>();
        Map<Integer, Integer> pair = new HashMap<>();

        for (int i = 0; i < s.length(); ++i)
            if (s.charAt(i) == '(') {
                stack.push(i);
            } else if (s.charAt(i) == ')') {
                final int j = stack.pop();
                pair.put(i, j);
                pair.put(j, i);
            }

        for (int i = 0, d = 1; i < s.length(); i += d)
            if (s.charAt(i) == '(' || s.charAt(i) == ')') {
                i = pair.get(i);
                d = -d;
            } else {
                sb.append(s.charAt(i));
            }

        return sb.toString();
    }
}
```

202. 1190.java

Path: LeetCode-main/solutions/1190. Reverse Substrings Between Each Pair of Parentheses/1190.java

```
class Solution {
    public String reverseParentheses(String s) {
        Deque<Integer> stack = new ArrayDeque<>();
        StringBuilder sb = new StringBuilder();

        for (final char c : s.toCharArray())
            if (c == '(') {
                stack.push(sb.length());
            } else if (c == ')') {
                // Reverse the corresponding substring between ().
                StringBuilder reversed = new StringBuilder();
                for (int sz = sb.length() - stack.pop(); sz > 0; --sz) {
                    reversed.append(sb.charAt(sb.length() - 1));
                    sb.deleteCharAt(sb.length() - 1);
                }
                sb.append(reversed);
            } else {
                sb.append(c);
            }

        return sb.toString();
    }
}
```

203. 1191.java

Path: LeetCode-main/solutions/1191. K-Concatenation Maximum Sum/1191.java

```
class Solution {
    public int kConcatenationMaxSum(int[] arr, int k) {
        final int MOD = 1_000_000_007;
        final int sz = arr.length * (k == 1 ? 1 : 2);
        final int sum = Arrays.stream(arr).sum();
        // The concatenated array will be [arr1, arr2, ..., arrk].
        // If sum(arr) > 0 and k > 2, then arr2, ..., arr(k - 1) should be included.
        // Equivalently, maxSubarraySum is from arr1 and arrk.
        if (sum > 0 && k > 2)
            return (int) ((kadane(arr, sz) + sum * (long) (k - 2)) % MOD);
        return kadane(arr, sz) % MOD;
    }

    private int kadane(int[] arr, int sz) {
        int ans = 0;
        int sum = 0;
        for (int i = 0; i < sz; ++i) {
            final int a = arr[i % arr.length];
            sum = Math.max(a, sum + a);
            ans = Math.max(ans, sum);
        }
        return ans;
    }
}
```

204. 1192.java

Path: LeetCode-main/solutions/1192. Critical Connections in a Network/1192.java

```
class Solution {
    public List<List<Integer>> criticalConnections(int n, List<List<Integer>> connections) {
        List<List<Integer>> ans = new ArrayList<>();
        List<Integer>[] graph = new List[n];
        Arrays.setAll(graph, i -> new ArrayList<>());

        for (List<Integer> connection : connections) {
            final int u = connection.get(0);
            final int v = connection.get(1);
            graph[u].add(v);
            graph[v].add(u);
        }

        // rank[i] := the minimum node that node i can reach with forward edges
        // Initialize with NO_RANK = -2 to indicate not visited.
        int[] rank = new int[n];
        Arrays.fill(rank, NO_RANK);
        getRank(graph, 0, 0, rank, ans);
        return ans;
    }

    private static final int NO_RANK = -2;

    // Gets the minimum rank that u can reach with forward edges.
    private int getRank(List<Integer>[] graph, int u, int myRank, int[] rank,
                        List<List<Integer>> ans) {
        if (rank[u] != NO_RANK) // The rank is already been determined.
            return rank[u];

        rank[u] = myRank;
        int minRank = myRank;

        for (final int v : graph[u]) {
            // visited || parent (that's why NO_RANK = -2 instead of -1)
            if (rank[u] == rank.length || rank[v] == myRank - 1)
                continue;
            final int nextRank = getRank(graph, v, myRank + 1, rank, ans);
            // (u, v) is the only way for u go to v.
            if (nextRank == myRank + 1)
                ans.add(Arrays.asList(u, v));
            minRank = Math.min(minRank, nextRank);
        }

        rank[u] = rank.length; // Mark as visited.
        return minRank;
    }
}
```

205. 1195.java

Path: LeetCode-main/solutions/1195. Fizz Buzz Multithreaded/1195.java

```
class FizzBuzz {
    public FizzBuzz(int n) {
        this.n = n;
    }

    // printFizz.run() outputs "fizz".
    public void fizz(Runnable printFizz) throws InterruptedException {
        for (int i = 1; i <= n; ++i)
            if (i % 3 == 0 && i % 15 != 0) {
                fizzSemaphore.acquire();
                printFizz.run();
                numberSemaphore.release();
            }
    }

    // printBuzz.run() outputs "buzz".
    public void buzz(Runnable printBuzz) throws InterruptedException {
        for (int i = 1; i <= n; ++i)
            if (i % 5 == 0 && i % 15 != 0) {
                buzzSemaphore.acquire();
                printBuzz.run();
                numberSemaphore.release();
            }
    }

    // printFizzBuzz.run() outputs "fizzbuzz".
    public void fizzbuzz(Runnable printFizzBuzz) throws InterruptedException {
        for (int i = 1; i <= n; ++i)
            if (i % 15 == 0) {
                fizzbuzzSemaphore.acquire();
                printFizzBuzz.run();
                numberSemaphore.release();
            }
    }

    // printNumber.accept(x) outputs "x", where x is an integer.
    public void number(IntConsumer printNumber) throws InterruptedException {
        for (int i = 1; i <= n; ++i) {
            numberSemaphore.acquire();
            if (i % 15 == 0)
                fizzbuzzSemaphore.release();
            else if (i % 3 == 0)
                fizzSemaphore.release();
            else if (i % 5 == 0)
                buzzSemaphore.release();
            else {
                printNumber.accept(i);
                numberSemaphore.release();
            }
        }
    }

    private int n;
    private Semaphore fizzSemaphore = new Semaphore(0);
    private Semaphore buzzSemaphore = new Semaphore(0);
    private Semaphore fizzbuzzSemaphore = new Semaphore(0);
    private Semaphore numberSemaphore = new Semaphore(1);
}
```

206. 1196.java

Path: LeetCode-main/solutions/1196. How Many Apples Can You Put into the Basket/1196.java

```
class Solution {
    public int maxNumberOfApples(int[] weight) {
        int sum = 0;

        Arrays.sort(weight);

        for (int i = 0; i < weight.length; ++i) {
            sum += weight[i];
            if (sum > 5000)
                return i;
        }

        return weight.length;
    }
}
```

207. 1197.java

Path: LeetCode-main/solutions/1197. Minimum Knight Moves/1197.java

```
class Solution {
    public int minKnightMoves(int x, int y) {
        return dp(Math.abs(x), Math.abs(y));
    }

    private Map<Pair<Integer, Integer>, Integer> mem = new HashMap<>();

    private int dp(int x, int y) {
        if (x + y == 0) // (0, 0)
            return 0;
        if (x + y == 2) // (0, 2), (1, 1), (2, 0)
            return 2;
        Pair<Integer, Integer> key = new Pair<>(x, y);
        if (mem.containsKey(key))
            return mem.get(key);

        final int minMove = 1 + Math.min(
            dp(Math.abs(x - 2), Math.abs(y - 1)), //
            dp(Math.abs(x - 1), Math.abs(y - 2)));

        mem.put(key, minMove);
        return minMove;
    }
}
```


209. 1199.java

Path: LeetCode-main/solutions/1199. Minimum Time to Build Blocks/1199.java

```
class Solution {
    public int minBuildTime(int[] blocks, int split) {
        Queue<Integer> minHeap = new PriorityQueue<>();

        for (final int block : blocks)
            minHeap.offer(block);

        while (minHeap.size() > 1) {
            minHeap.poll();           // the minimum
            final int x = minHeap.poll(); // the second minimum
            minHeap.offer(x + split);
        }

        return minHeap.poll();
    }
}
```

210. 12-2.java

Path: LeetCode-main/solutions/12. Integer to Roman/12-2.java

```
class Solution {
    public String intToRoman(int num) {
        final String[] M = {"", "M", "MM", "MMM"};
        final String[] C = {"", "C", "CC", "CCC", "CD", "D", "DC", "DCC", "DCCC", "CM"};
        final String[] X = {"", "X", "XX", "XXX", "XL", "L", "LX", "LXX", "LXXX", "XC"};
        final String[] I = {"", "I", "II", "III", "IV", "V", "VI", "VII", "VIII", "IX"};
        return M[num / 1000] + C[num % 1000 / 100] + X[num % 100 / 10] + I[num % 10];
    }
}
```

211. 12.java

Path: LeetCode-main/solutions/12. Integer to Roman/12.java

```
class Solution {
    public String intToRoman(int num) {
        final int[] values = {1000, 900, 500, 400, 100, 90, 50, 40, 10, 9, 5, 4, 1};
        final String[] symbols = {"M", "CM", "D", "CD", "C", "XC", "L",
                                   "XL", "X", "IX", "V", "IV", "I"};

        StringBuilder sb = new StringBuilder();

        for (int i = 0; i < values.length; ++i) {
            if (num == 0)
                break;
            while (num >= values[i]) {
                num -= values[i];
                sb.append(symbols[i]);
            }
        }

        return sb.toString();
    }
}
```

212. 120.java

Path: LeetCode-main/solutions/120. Triangle/120.java

```
class Solution {
    public int minimumTotal(List<List<Integer>> triangle) {
        for (int i = triangle.size() - 2; i >= 0; --i)
            for (int j = 0; j <= i; ++j)
                triangle.get(i).set(j, triangle.get(i).get(j) + Math.min(triangle.get(i + 1).get(j),
                                                                            triangle.get(i + 1).get(j + 1)));
        return triangle.get(0).get(0);
    }
}
```

213. 1200.java

Path: LeetCode-main/solutions/1200. Minimum Absolute Difference/1200.java

```
class Solution {
    public List<List<Integer>> minimumAbsDifference(int[] arr) {
        List<List<Integer>> ans = new ArrayList<>();
        int mn = Integer.MAX_VALUE;

        Arrays.sort(arr);

        for (int i = 1; i < arr.length; ++i) {
            final int diff = arr[i] - arr[i - 1];
            if (diff < mn) {
                mn = diff;
                ans.clear();
            }
            if (diff == mn)
                ans.add(Arrays.asList(arr[i - 1], arr[i]));
        }

        return ans;
    }
}
```

214. 1201.java

Path: LeetCode-main/solutions/1201. Ugly Number III/1201.java

```
class Solution {
    public int nthUglyNumber(int n, long a, long b, long c) {
        final long ab = a * b / gcd(a, b);
        final long ac = a * c / gcd(a, c);
        final long bc = b * c / gcd(b, c);
        final long abc = a * bc / gcd(a, bc);
        int l = 1;
        int r = 2_000_000_000;

        while (l < r) {
            final int m = (l + r) / 2;
            if (m / a + m / b + m / c - m / ab - m / ac - m / bc + m / abc >= n)
                r = m;
            else
                l = m + 1;
        }

        return l;
    }

    private long gcd(long a, long b) {
        return b == 0 ? a : gcd(b, a % b);
    }
}
```

215. 1202.java

Path: LeetCode-main/solutions/1202. Smallest String With Swaps/1202.java

```
class UnionFind {
    public UnionFind(int n) {
        id = new int[n];
        rank = new int[n];
        for (int i = 0; i < n; ++i)
            id[i] = i;
    }

    public void unionByRank(int u, int v) {
        final int i = find(u);
        final int j = find(v);
        if (i == j)
            return;
        if (rank[i] < rank[j]) {
            id[i] = j;
        } else if (rank[i] > rank[j]) {
            id[j] = i;
        } else {
            id[i] = j;
            ++rank[j];
        }
    }

    public int find(int u) {
        return id[u] == u ? u : (id[u] = find(id[u]));
    }

    private int[] id;
    private int[] rank;
}

class Solution {
    public String smallestStringWithSwaps(String s, List<List<Integer>> pairs) {
        StringBuilder sb = new StringBuilder();
        UnionFind uf = new UnionFind(s.length());
        Map<Integer, Queue<Character>> indexToLetters = new HashMap<>();

        for (List<Integer> pair : pairs) {
            final int a = pair.get(0);
            final int b = pair.get(1);
            uf.unionByRank(a, b);
        }

        for (int i = 0; i < s.length(); ++i)
            indexToLetters.computeIfAbsent(uf.find(i), k -> new PriorityQueue<>()).offer(s.charAt(i));

        for (int i = 0; i < s.length(); ++i)
            sb.append(indexToLetters.get(uf.find(i)).poll());

        return sb.toString();
    }
}
```

216. 1203.java

Path: LeetCode-main/solutions/1203. Sort Items by Groups Respecting Dependencies/1203.java

```
class Solution {
    public int[] sortItems(int n, int m, int[] group, List<List<Integer>> beforeItems) {
        List<Integer>[] graph = new List[n + m];
        int[] inDegrees = new int[n + m];

        for (int i = 0; i < graph.length; ++i)
            graph[i] = new ArrayList<>();

        // Build the graph by remapping the k-th group to k + n imaginary node.
        for (int i = 0; i < group.length; ++i) {
            if (group[i] == -1)
                continue;
            graph[group[i] + n].add(i);
            ++inDegrees[i];
        }

        for (int i = 0; i < beforeItems.size(); ++i)
            for (final int b : beforeItems.get(i)) {
                final int u = group[b] == -1 ? b : group[b] + n;
                final int v = group[i] == -1 ? i : group[i] + n;
                if (u == v) { // u and v are already in the same group.
                    graph[b].add(i);
                    ++inDegrees[i];
                } else {
                    graph[u].add(v);
                    ++inDegrees[v];
                }
            }

        // Perform topological sorting.
        List<Integer> ans = new ArrayList<>();

        for (int i = 0; i < n + m; ++i)
            if (inDegrees[i] == 0) // inDegrees[i] == -1 means visited.
                dfs(graph, i, inDegrees, n, ans);

        return ans.size() == n ? ans.stream().mapToInt(Integer::intValue).toArray() : new int[] {};
    }

    private void dfs(List<Integer>[] graph, int u, int[] inDegrees, int n, List<Integer> ans) {
        if (u < n)
            ans.add(u);

        inDegrees[u] = -1; // Mark as visited.

        for (final int v : graph[u])
            if (--inDegrees[v] == 0)
                dfs(graph, v, inDegrees, n, ans);
    }
}
```


217. 1206.java

Path: LeetCode-main/solutions/1206. Design Skiplist/1206.java

```
class Node {
    public int val;
    public Node next;
    public Node down;
    public Node(int val, Node next, Node down) {
        this.val = val;
        this.next = next;
        this.down = down;
    }
}

class Skiplist {
    public boolean search(int target) {
        for (Node node = dummy; node != null; node = node.down) {
            node = advance(node, target);
            if (node.next != null && node.next.val == target)
                return true;
        }
        return false;
    }

    public void add(int num) {
        // Collect nodes that are before the insertion point.
        Deque<Node> nodes = new ArrayDeque<>();
        for (Node node = dummy; node != null; node = node.down) {
            node = advance(node, num);
            nodes.push(node);
        }

        Node down = null;
        boolean shouldInsert = true;
        while (shouldInsert && !nodes.isEmpty()) {
            Node prev = nodes.poll();
            prev.next = new Node(num, prev.next, down);
            down = prev.next;
            shouldInsert = Math.random() < 0.5;
        }

        // Create a topmost new level dummy that points to the existing dummy.
        if (shouldInsert)
            dummy = new Node(-1, null, dummy);
    }

    public boolean erase(int num) {
        boolean found = false;
        for (Node node = dummy; node != null; node = node.down) {
            node = advance(node, num);
            if (node.next != null && node.next.val == num) {
                node.next = node.next.next;
                found = true;
            }
        }
        return found;
    }

    private Node dummy = new Node(-1, null, null);

    private Node advance(Node node, int target) {
        while (node.next != null && node.next.val < target)
            node = node.next;
        return node;
    }
}
```

218. 1207.java

Path: LeetCode-main/solutions/1207. Unique Number of Occurrences/1207.java

```
class Solution {
    public boolean uniqueOccurrences(int[] arr) {
        Map<Integer, Integer> count = new HashMap<>();
        Set<Integer> occurrences = new HashSet<>();

        for (final int a : arr)
            count.merge(a, 1, Integer::sum);

        for (final int value : count.values())
            if (!occurrences.add(value))
                return false;

        return true;
    }
}
```

219. 1208.java

Path: LeetCode-main/solutions/1208. Get Equal Substrings Within Budget/1208.java

```
class Solution {
    public int equalSubstring(String s, String t, int maxCost) {
        int j = 0;
        for (int i = 0; i < s.length(); ++i) {
            maxCost -= Math.abs(s.charAt(i) - t.charAt(i));
            if (maxCost < 0)
                maxCost += Math.abs(s.charAt(j) - t.charAt(j++));
        }
        return s.length() - j;
    }
}
```

220. 1209.java

Path: LeetCode-main/solutions/1209. Remove All Adjacent Duplicates in String II/1209.java

```
class Item {
    public char c;
    public int freq;
    public Item(char c, int freq) {
        this.c = c;
        this.freq = freq;
    }
}

class Solution {
    public String removeDuplicates(String s, int k) {
        StringBuilder sb = new StringBuilder();
        LinkedList<Item> stack = new LinkedList<>();

        for (final char c : s.toCharArray()) {
            if (!stack.isEmpty() && stack.peek().c == c)
                ++stack.peek().freq;
            else
                stack.push(new Item(c, 1));
            if (stack.peek().freq == k)
                stack.pop();
        }

        while (!stack.isEmpty()) {
            Item item = stack.pop();
            for (int i = 0; i < item.freq; ++i)
                sb.append(item.c);
        }

        return sb.reverse().toString();
    }
}
```

221. 121.java

Path: LeetCode-main/solutions/121. Best Time to Buy and Sell Stock/121.java

```
class Solution {
    public int maxProfit(int[] prices) {
        int sellOne = 0;
        int holdOne = Integer.MIN_VALUE;

        for (final int price : prices) {
            sellOne = Math.max(sellOne, holdOne + price);
            holdOne = Math.max(holdOne, -price);
        }

        return sellOne;
    }
}
```

222. 1210.java

Path: LeetCode-main/solutions/1210. Minimum Moves to Reach Target with Rotations/1210.java

```
enum Pos { HORIZONTAL, VERTICAL }

class Solution {
    public int minimumMoves(int[][] grid) {
        record T(int x, int y, Pos pos) {}
        final int n = grid.length;
        Queue<T> q = new ArrayDeque<>(List.of(new T(0, 0, Pos.HORIZONTAL)));
        boolean[][][] seen = new boolean[n][n][2];
        seen[0][0][Pos.HORIZONTAL.ordinal()] = true;

        for (int step = 0; !q.isEmpty(); ++step)
            for (int sz = q.size(); sz > 0; --sz) {
                final int x = q.peek().x;
                final int y = q.peek().y;
                final Pos pos = q.poll().pos;
                if (x == n - 1 && y == n - 2 && pos == Pos.HORIZONTAL)
                    return step;
                if (canMoveRight(grid, x, y, pos) && !seen[x][y + 1][pos.ordinal()]) {
                    q.offer(new T(x, y + 1, pos));
                    seen[x][y + 1][pos.ordinal()] = true;
                }
                if (canMoveDown(grid, x, y, pos) && !seen[x + 1][y][pos.ordinal()]) {
                    q.offer(new T(x + 1, y, pos));
                    seen[x + 1][y][pos.ordinal()] = true;
                }
                final Pos newPos = pos == Pos.HORIZONTAL ? Pos.VERTICAL : Pos.HORIZONTAL;
                if ((canRotateClockwise(grid, x, y, pos) || canRotateCounterclockwise(grid, x, y, pos)) &&
                    !seen[x][y][newPos.ordinal()]) {
                    q.offer(new T(x, y, newPos));
                    seen[x][y][newPos.ordinal()] = true;
                }
            }

        return -1;
    }

    private boolean canMoveRight(int[][] grid, int x, int y, Pos pos) {
        if (pos == Pos.HORIZONTAL)
            return y + 2 < grid.length && grid[x][y + 2] == 0;
        return y + 1 < grid.length && grid[x][y + 1] == 0 && grid[x + 1][y + 1] == 0;
    }

    private boolean canMoveDown(int[][] grid, int x, int y, Pos pos) {
        if (pos == Pos.VERTICAL)
            return x + 2 < grid.length && grid[x + 2][y] == 0;
        return x + 1 < grid.length && grid[x + 1][y] == 0 && grid[x + 1][y + 1] == 0;
    }

    private boolean canRotateClockwise(int[][] grid, int x, int y, Pos pos) {
        return pos == Pos.HORIZONTAL && x + 1 < grid.length && grid[x + 1][y + 1] == 0 &&
            grid[x + 1][y] == 0;
    }

    private boolean canRotateCounterclockwise(int[][] grid, int x, int y, Pos pos) {
        return pos == Pos.VERTICAL && y + 1 < grid.length && grid[x + 1][y + 1] == 0 &&
            grid[x][y + 1] == 0;
    }
}
```

223. 1213.java

Path: LeetCode-main/solutions/1213. Intersection of Three Sorted Arrays/1213.java

```
class Solution {
    public List<Integer> arraysIntersection(int[] arr1, int[] arr2, int[] arr3) {
        List<Integer> ans = new ArrayList<>();
        int i = 0;
        int j = 0;
        int k = 0;

        while (i < arr1.length && j < arr2.length && k < arr3.length) {
            final int mn = Math.min(arr1[i], Math.min(arr2[j], arr3[k]));
            if (arr1[i] == mn && arr2[j] == mn && arr3[k] == mn) {
                ans.add(mn);
                ++i;
                ++j;
                ++k;
            } else if (arr1[i] == mn) {
                ++i;
            } else if (arr2[j] == mn) {
                ++j;
            } else {
                ++k;
            }
        }

        return ans;
    }
}
```

224. 1214.java

Path: LeetCode-main/solutions/1214. Two Sum BSTs/1214.java

```
class BSTIterator {
    BSTIterator(TreeNode root, boolean leftToRight) {
        this.leftToRight = leftToRight;
        pushUntilNull(root);
    }

    public boolean hasNext() {
        return !stack.isEmpty();
    }

    public int next() {
        TreeNode root = stack.pop();
        pushUntilNull(leftToRight ? root.right : root.left);
        return root.val;
    }

    private Deque<TreeNode> stack = new ArrayDeque<>();
    private boolean leftToRight;

    private void pushUntilNull(TreeNode root) {
        while (root != null) {
            stack.push(root);
            root = leftToRight ? root.left : root.right;
        }
    }
}

class Solution {
    public boolean twoSumBSTs(TreeNode root1, TreeNode root2, int target) {
        BSTIterator bst1 = new BSTIterator(root1, true);
        BSTIterator bst2 = new BSTIterator(root2, false);

        for (int l = bst1.next(), r = bst2.next(); true;) {
            final int sum = l + r;
            if (sum == target)
                return true;
            if (sum < target) {
                if (!bst1.hasNext())
                    return false;
                l = bst1.next();
            } else {
                if (!bst2.hasNext())
                    return false;
                r = bst2.next();
            }
        }
    }
}
```


225. 1215-2.java

Path: LeetCode-main/solutions/1215. Stepping Numbers/1215-2.java

```
class Solution {
    public List<Integer> countSteppingNumbers(int low, int high) {
        List<Integer> ans = new ArrayList<>();
        if (low == 0)
            ans.add(0);

        for (long i = 1; i <= 9; ++i)
            dfs(i, low, high, ans);

        Collections.sort(ans);
        return ans;
    }

    private void dfs(long curr, int low, int high, List<Integer> ans) {
        if (curr > high)
            return;
        if (curr >= low)
            ans.add((int) curr);

        final long lastDigit = curr % 10;
        if (lastDigit > 0)
            dfs(curr * 10 + lastDigit - 1, low, high, ans);
        if (lastDigit < 9)
            dfs(curr * 10 + lastDigit + 1, low, high, ans);
    }
}
```

226. 1215.java

Path: LeetCode-main/solutions/1215. Stepping Numbers/1215.java

```
class Solution {
    public List<Integer> countSteppingNumbers(int low, int high) {
        List<Integer> ans = new ArrayList<>();
        if (low == 0)
            ans.add(0);

        Queue<Long> q = new ArrayDeque<>();

        for (long i = 1; i <= 9; ++i)
            q.offer(i);

        while (!q.isEmpty()) {
            final long curr = q.poll();
            if (curr > high)
                continue;
            if (curr >= low)
                ans.add((int) curr);
            final long lastDigit = curr % 10;
            if (lastDigit > 0)
                q.offer(curr * 10 + lastDigit - 1);
            if (lastDigit < 9)
                q.offer(curr * 10 + lastDigit + 1);
        }

        return ans;
    }
}
```

227. 1216.java

Path: LeetCode-main/solutions/1216. Valid Palindrome III/1216.java

```
class Solution {
    public boolean isValidPalindrome(String s, int k) {
        return s.length() - longestPalindromeSubseq(s) <= k;
    }

    // Same as 516. Longest Palindromic Subsequence
    private int longestPalindromeSubseq(final String s) {
        final int n = s.length();
        // dp[i][j] := the length of LPS(s[i..j])
        int[][] dp = new int[n][n];

        for (int i = 0; i < n; ++i)
            dp[i][i] = 1;

        for (int d = 1; d < n; ++d)
            for (int i = 0; i + d < n; ++i) {
                final int j = i + d;
                if (s.charAt(i) == s.charAt(j))
                    dp[i][j] = 2 + dp[i + 1][j - 1];
                else
                    dp[i][j] = Math.max(dp[i + 1][j], dp[i][j - 1]);
            }

        return dp[0][n - 1];
    }
}
```

228. 1217.java

Path: LeetCode-main/solutions/1217. Play with Chips/1217.java

```
class Solution {  
    public int minCostToMoveChips(int[] position) {  
        int[] count = new int[2];  
        for (final int p : position)  
            ++count[p % 2];  
        return Math.min(count[0], count[1]);  
    }  
}
```

229. 1218.java

Path: LeetCode-main/solutions/1218. Longest Arithmetic Subsequence of Given Difference/1218.java

```
class Solution {
    public int longestSubsequence(int[] arr, int difference) {
        int ans = 0;
        Map<Integer, Integer> lengthAt = new HashMap<>();

        for (final int a : arr) {
            lengthAt.put(a, lengthAt.getOrDefault(a - difference, 0) + 1);
            ans = Math.max(ans, lengthAt.get(a));
        }

        return ans;
    }
}
```

230. 1219.java

Path: LeetCode-main/solutions/1219. Path with Maximum Gold/1219.java

```
class Solution {
    public int getMaximumGold(int[][] grid) {
        int ans = 0;

        for (int i = 0; i < grid.length; ++i)
            for (int j = 0; j < grid[0].length; ++j)
                ans = Math.max(ans, dfs(grid, i, j));

        return ans;
    }

    private int dfs(int[][] grid, int i, int j) {
        if (i < 0 || j < 0 || i == grid.length || j == grid[0].length)
            return 0;
        if (grid[i][j] == 0)
            return 0;

        final int gold = grid[i][j];
        grid[i][j] = 0; // Mark as visited.
        final int maxPath = Math.max(Math.max(dfs(grid, i + 1, j), dfs(grid, i - 1, j)),
                                     Math.max(dfs(grid, i, j + 1), dfs(grid, i, j - 1)));
        grid[i][j] = gold;
        return gold + maxPath;
    }
}
```

231. 122.java

Path: LeetCode-main/solutions/122. Best Time to Buy and Sell Stock II/122.java

```
class Solution {
    public int maxProfit(int[] prices) {
        int sell = 0;
        int hold = Integer.MIN_VALUE;

        for (final int price : prices) {
            sell = Math.max(sell, hold + price);
            hold = Math.max(hold, sell - price);
        }

        return sell;
    }
}
```

232. 1222.java

Path: LeetCode-main/solutions/1222. Queens That Can Attack the King/1222.java

```
class Solution {
    public List<List<Integer>> queensAttacktheKing(int[][] queens, int[] king) {
        List<List<Integer>> ans = new ArrayList<>();
        Set<Integer> queensSet = new HashSet<>();

        for (int[] queen : queens)
            queensSet.add(hash(queen[0], queen[1]));

        int[][] directions =
            new int[][] {{-1, -1}, {-1, 0}, {-1, 1}, {0, -1}, {0, 1}, {1, -1}, {1, 0}, {1, 1}};
        for (int[] d : directions)
            for (int i = king[0] + d[0], j = king[1] + d[1]; 0 <= i && i < 8 && 0 <= j && j < 8;
                 i += d[0], j += d[1])
                if (queensSet.contains(hash(i, j))) {
                    ans.add(Arrays.asList(i, j));
                    break;
                }

        return ans;
    }

    private int hash(int i, int j) {
        return i * 8 + j;
    }
}
```


233. 1224.java

Path: LeetCode-main/solutions/1224. Maximum Equal Frequency/1224.java

```
class Solution {
    public int maxEqualFreq(int[] nums) {
        int ans = 0;
        int maxFreq = 0;
        Map<Integer, Integer> count = new HashMap<>();
        Map<Integer, Integer> freq = new HashMap<>();

        for (int i = 0; i < nums.length; ++i) {
            final int currentFreq = count.getDefault(nums[i], 0);
            freq.merge(currentFreq, -1, Integer::sum);
            final int updatedFreq = currentFreq + 1;
            count.put(nums[i], updatedFreq);
            freq.merge(updatedFreq, 1, Integer::sum);
            maxFreq = Math.max(maxFreq, updatedFreq);
            if (maxFreq == 1 || maxFreq * freq.get(maxFreq) == i ||
                (maxFreq - 1) * (freq.get(maxFreq - 1) + 1) == i)
                ans = i + 1;
        }

        return ans;
    }
}
```

234. 1227.java

Path: LeetCode-main/solutions/1227. Airplane Seat Assignment Probability/1227.java

```
class Solution {  
    public double nthPersonGetsNthSeat(int n) {  
        return n == 1 ? 1 : 0.5;  
    }  
}
```

235. 1228.java

Path: LeetCode-main/solutions/1228. Missing Number In Arithmetic Progression/1228.java

```
class Solution {
    public int missingNumber(int[] arr) {
        final int n = arr.length;
        final int delta = (arr[n - 1] - arr[0]) / n;
        int l = 0;
        int r = n - 1;

        while (l < r) {
            final int m = (l + r) / 2;
            if (arr[m] == arr[0] + m * delta)
                l = m + 1;
            else
                r = m;
        }

        return arr[0] + l * delta;
    }
}
```

236. 1229.java

Path: LeetCode-main/solutions/1229. Meeting Scheduler/1229.java

```
class Solution {
    public List<Integer> minAvailableDuration(int[][] slots1, int[][] slots2, int duration) {
        Arrays.sort(slots1, Comparator.comparingInt(slot -> slot[0]));
        Arrays.sort(slots2, Comparator.comparingInt(slot -> slot[0]));

        int i = 0; // slots1's index
        int j = 0; // slots2's index

        while (i < slots1.length && j < slots2.length) {
            final int start = Math.max(slots1[i][0], slots2[j][0]);
            final int end = Math.min(slots1[i][1], slots2[j][1]);
            if (start + duration <= end)
                return Arrays.asList(start, start + duration);
            if (slots1[i][1] < slots2[j][1])
                ++i;
            else
                ++j;
        }

        return new ArrayList<>();
    }
}
```

237. 123.java

Path: LeetCode-main/solutions/123. Best Time to Buy and Sell Stock III/123.java

```
class Solution {
    public int maxProfit(int[] prices) {
        int sellTwo = 0;
        int holdTwo = Integer.MIN_VALUE;
        int sellOne = 0;
        int holdOne = Integer.MIN_VALUE;

        for (final int price : prices) {
            sellTwo = Math.max(sellTwo, holdTwo + price);
            holdTwo = Math.max(holdTwo, sellOne - price);
            sellOne = Math.max(sellOne, holdOne + price);
            holdOne = Math.max(holdOne, -price);
        }

        return sellTwo;
    }
}
```

238. 1230-2.java

Path: LeetCode-main/solutions/1230. Toss Strange Coins/1230-2.java

```
class Solution {
    public double probabilityOfHeads(double[] prob, int target) {
        // dp[j] := the probability of tossing the coins so far with j heads
        double[] dp = new double[target + 1];
        dp[0] = 1.0;

        for (final double p : prob)
            for (int j = target; j >= 0; --j)
                dp[j] = (j > 0 ? dp[j - 1] * p : 0) + dp[j] * (1 - p);

        return dp[target];
    }
}
```

239. 1230.java

Path: LeetCode-main/solutions/1230. Toss Strange Coins/1230.java

```
class Solution {
    public double probabilityOfHeads(double[] prob, int target) {
        // dp[i][j] := the probability of tossing the first i coins with j heads
        double[][] dp = new double[prob.length + 1][target + 1];
        dp[0][0] = 1.0;

        for (int i = 1; i <= prob.length; ++i)
            for (int j = 0; j <= target; ++j)
                dp[i][j] = (j > 0 ? dp[i - 1][j - 1] * prob[i - 1] : 0) + dp[i - 1][j] * (1 - prob[i - 1]);

        return dp[prob.length][target];
    }
}
```

240. 1231.java

Path: LeetCode-main/solutions/1231. Divide Chocolate/1231.java

```
class Solution {
    public int maximizeSweetness(int[] sweetness, int k) {
        int l = sweetness.length / (k + 1);
        int r = Arrays.stream(sweetness).sum() / (k + 1);

        while (l < r) {
            final int m = (l + r) / 2;
            if (canEat(sweetness, k, m))
                l = m + 1;
            else
                r = m;
        }

        return canEat(sweetness, k, l) ? l : l - 1;
    }

    // Returns true if can eat m sweetness (the minimum sweetness of each piece).
    private boolean canEat(int[] sweetness, int k, int m) {
        int pieces = 0;
        int sum = 0; // the running sum

        for (final int s : sweetness) {
            sum += s;
            if (sum >= m) {
                if (++pieces > k)
                    return true;
                sum = 0;
            }
        }

        return false;
    }
}
```


241. 1232.java

Path: LeetCode-main/solutions/1232. Check If It Is a Straight Line/1232.java

```
class Solution {
    public boolean checkStraightLine(int[][] coordinates) {
        int x0 = coordinates[0][0];
        int y0 = coordinates[0][1];
        int x1 = coordinates[1][0];
        int y1 = coordinates[1][1];
        int dx = x1 - x0;
        int dy = y1 - y0;

        for (int i = 2; i < coordinates.length; ++i) {
            int x = coordinates[i][0];
            int y = coordinates[i][1];
            if ((x - x0) * dy != (y - y0) * dx)
                return false;
        }
        return true;
    }
}
```

242. 1233.java

Path: LeetCode-main/solutions/1233. Remove Sub-Folders from the Filesystem/1233.java

```
class Solution {
    public List<String> removeSubfolders(String[] folder) {
        List<String> ans = new ArrayList<>();
        String prev = "";

        Arrays.sort(folder);

        for (final String f : folder) {
            if (!prev.isEmpty() && f.startsWith(prev) && f.charAt(prev.length()) == '/')
                continue;
            ans.add(f);
            prev = f;
        }

        return ans;
    }
}
```

243. 1234.java

Path: LeetCode-main/solutions/1234. Replace the Substring for Balanced String/1234.java

```
class Solution {
    public int balancedString(String s) {
        final int n = s.length();
        final int k = n / 4;
        int ans = n;
        int[] count = new int[128];

        for (final char c : s.toCharArray())
            ++count[c];

        for (int i = 0, j = 0; i < n; ++i) {
            --count[s.charAt(i)];
            while (j < n && count['Q'] <= k && count['W'] <= k && count['E'] <= k && count['R'] <= k) {
                ans = Math.min(ans, i - j + 1);
                ++count[s.charAt(j)];
                ++j;
            }
        }

        return ans;
    }
}
```

244. 1235-2.java

Path: LeetCode-main/solutions/1235. Maximum Profit in Job Scheduling/1235-2.java

```
class Solution {
    public int jobScheduling(int[] startTime, int[] endTime, int[] profit) {
        record Job(int startTime, int endTime, int profit) {}
        final int n = startTime.length;
        // dp[i] := the maximum profit to schedule jobs[i..n)
        int[] dp = new int[n + 1];
        Job[] jobs = new Job[n];

        for (int i = 0; i < n; ++i)
            jobs[i] = new Job(startTime[i], endTime[i], profit[i]);

        Arrays.sort(jobs, Comparator.comparingInt(Job::startTime));

        for (int i = 0; i < n; ++i)
            startTime[i] = jobs[i].startTime;

        for (int i = n - 1; i >= 0; --i) {
            final int j = firstGreaterEqual(startTime, i + 1, jobs[i].endTime);
            final int pick = jobs[i].profit + dp[j];
            final int skip = dp[i + 1];
            dp[i] = Math.max(pick, skip);
        }

        return dp[0];
    }

    private int firstGreaterEqual(int[] arr, int startFrom, int target) {
        int l = startFrom;
        int r = arr.length;
        while (l < r) {
            final int m = (l + r) / 2;
            if (arr[m] >= target)
                r = m;
            else
                l = m + 1;
        }
        return l;
    }
}
```

245. 1235-3.java

Path: LeetCode-main/solutions/1235. Maximum Profit in Job Scheduling/1235-3.java

```
class Solution {
    public int jobScheduling(int[] startTime, int[] endTime, int[] profit) {
        final int n = startTime.length;
        Job[] jobs = new Job[n];

        for (int i = 0; i < n; ++i)
            jobs[i] = new Job(startTime[i], endTime[i], profit[i]);

        Arrays.sort(jobs, Comparator.comparingInt(Job::startTime));

        // Will use binary search to find the first available startTime
        for (int i = 0; i < n; ++i)
            startTime[i] = jobs[i].startTime;

        return getMaxProfit(jobs);
    }

    private record Job(int startTime, int endTime, int profit) {}

    private int getMaxProfit(Job[] jobs) {
        int maxProfit = 0;
        Queue<Job> minHeap = new PriorityQueue<>(Comparator.comparingInt(Job::endTime));

        for (Job job : jobs) {
            while (!minHeap.isEmpty() && job.startTime >= minHeap.peek().endTime)
                maxProfit = Math.max(maxProfit, minHeap.poll().profit);
            minHeap.offer(new Job(job.startTime, job.endTime, job.profit + maxProfit));
        }

        while (!minHeap.isEmpty())
            maxProfit = Math.max(maxProfit, minHeap.poll().profit);

        return maxProfit;
    }
}
```

246. 1235.java

Path: LeetCode-main/solutions/1235. Maximum Profit in Job Scheduling/1235.java

```
class Solution {
    public int jobScheduling(int[] startTime, int[] endTime, int[] profit) {
        final int n = startTime.length;
        int[] mem = new int[n + 1];
        Job[] jobs = new Job[n];

        for (int i = 0; i < n; ++i)
            jobs[i] = new Job(startTime[i], endTime[i], profit[i]);

        Arrays.sort(jobs, Comparator.comparingInt(Job::startTime));

        // Will use binary search to find the first available start time.
        for (int i = 0; i < n; ++i)
            startTime[i] = jobs[i].startTime;

        return jobScheduling(jobs, startTime, 0, mem);
    }

    private record Job(int startTime, int endTime, int profit) {}

    private int jobScheduling(Job[] jobs, int[] startTime, int i, int[] mem) {
        if (i == jobs.length)
            return 0;
        if (mem[i] > 0)
            return mem[i];

        final int j = firstGreaterEqual(startTime, i + 1, jobs[i].endTime);
        final int pick = jobs[i].profit + jobScheduling(jobs, startTime, j, mem);
        final int skip = jobScheduling(jobs, startTime, i + 1, mem);
        return mem[i] = Math.max(pick, skip);
    }

    private int firstGreaterEqual(int[] arr, int startFrom, int target) {
        int l = startFrom;
        int r = arr.length;
        while (l < r) {
            final int m = (l + r) / 2;
            if (arr[m] >= target)
                r = m;
            else
                l = m + 1;
        }
        return l;
    }
}
```

247. 1236.java

Path: LeetCode-main/solutions/1236. Web Crawler/1236.java

```
/**
 * // This is the HtmlParser's API interface.
 * // You should not implement it, or speculate about its implementation
 * interface HtmlParser {
 *     public List<String> getUrls(String url) {}
 * }
 */

class Solution {
    public List<String> crawl(String startUrl, HtmlParser htmlParser) {
        Queue<String> q = new ArrayDeque<>(List.of(startUrl));
        Set<String> seen = new HashSet<>(Arrays.asList(startUrl));
        final String hostname = startUrl.split("/")[2];

        while (!q.isEmpty()) {
            final String currUrl = q.poll();
            for (final String url : htmlParser.getUrls(currUrl)) {
                if (seen.contains(url))
                    continue;
                if (url.contains(hostname)) {
                    q.offer(url);
                    seen.add(url);
                }
            }
        }

        return new ArrayList<>(seen);
    }
}
```

248. 1237.java

Path: LeetCode-main/solutions/1237. Find Positive Integer Solution for a Given Equation/1237.java

```
class Solution {
    public List<List<Integer>> findSolution(CustomFunction customfunction, int z) {
        List<List<Integer>> ans = new LinkedList<>();
        int x = 1;
        int y = 1000;

        while (x <= 1000 && y >= 1) {
            int f = customfunction.f(x, y);
            if (f < z)
                ++x;
            else if (f > z)
                --y;
            else
                ans.add(Arrays.asList(x++, y--));
        }

        return ans;
    }
}
```


249. 1238.java

Path: LeetCode-main/solutions/1238. Circular Permutation in Binary Representation/1238.java

```
class Solution {
    public List<Integer> circularPermutation(int n, int start) {
        List<Integer> ans = new ArrayList<>();

        for (int i = 0; i < 1 << n; ++i)
            ans.add(start ^ i ^ i >> 1);

        return ans;
    }
}
```

250. 124.java

Path: LeetCode-main/solutions/124. Binary Tree Maximum Path Sum/124.java

```
class Solution {
    public int maxPathSum(TreeNode root) {
        maxPathSumDownFrom(root);
        return ans;
    }

    private int ans = Integer.MIN_VALUE;

    // Returns the maximum path sum starting from the current root, where
    // root.val is always included.
    private int maxPathSumDownFrom(TreeNode root) {
        if (root == null)
            return 0;

        final int l = Math.max(maxPathSumDownFrom(root.left), 0);
        final int r = Math.max(maxPathSumDownFrom(root.right), 0);
        ans = Math.max(ans, root.val + l + r);
        return root.val + Math.max(l, r);
    }
}
```

251. 1240.java

Path: LeetCode-main/solutions/1240. Tiling a Rectangle with the Fewest Squares/1240.java

```
class Solution {
    public int tilingRectangle(int n, int m) {
        return tilingRectangle(n, m, 0, new int[m]);
    }

    private static final int BASE = 13;
    private Map<Long, Integer> dp = new HashMap<>();

    private int tilingRectangle(int n, int m, long hashedHeights, int[] heights) {
        if (dp.containsKey(hashedHeights))
            return dp.get(hashedHeights);

        final int minHeight = Arrays.stream(heights).min().getAsInt();
        if (minHeight == n) // All filled.
            return 0;

        int ans = m * n;
        int start = -1;

        for (int i = 0; i < m; ++i)
            if (heights[i] == minHeight) {
                start = i;
                break;
            }

        // Try to put square of different size that doesn't exceed the width/height.
        for (int sz = 1; sz <= Math.min(m - start, n - minHeight); ++sz) {
            // heights[start..start + sz) must has the same height.
            if (heights[start + sz - 1] != minHeight)
                break;
            // Put a square of size `sz` to cover heights[start..start + sz).
            for (int i = start; i < start + sz; ++i)
                heights[i] += sz;
            ans = Math.min(ans, tilingRectangle(n, m, hash(heights), heights));
            for (int i = start; i < start + sz; ++i)
                heights[i] -= sz;
        }

        dp.put(hashedHeights, 1 + ans);
        return 1 + ans;
    }

    private long hash(int[] heights) {
        long hashed = 0;
        for (int i = heights.length - 1; i >= 0; --i)
            hashed = hashed * BASE + heights[i];
        return hashed;
    }
}
```

252. 1243.java

Path: LeetCode-main/solutions/1243. Array Transformation/1243.java

```
class Solution {
    public List<Integer> transformArray(int[] arr) {
        if (arr.length < 3)
            return Arrays.stream(arr).boxed().toList();

        int[] ans = new int[0];

        while (!Arrays.equals(ans, arr)) {
            ans = arr.clone();
            for (int i = 1; i < arr.length - 1; ++i)
                if (ans[i - 1] > ans[i] && ans[i] < ans[i + 1])
                    ++arr[i];
                else if (ans[i - 1] < ans[i] && ans[i] > ans[i + 1])
                    --arr[i];
        }

        return Arrays.stream(ans).boxed().toList();
    }
}
```

253. 1244.java

Path: LeetCode-main/solutions/1244. Design A Leaderboard/1244.java

```
class Leaderboard {
    public void addScore(int playerId, int score) {
        idToScore.merge(playerId, score, Integer::sum);
    }

    public int top(int K) {
        int ans = 0;
        Queue<Integer> minHeap = new PriorityQueue<>();

        for (final int score : idToScore.values()) {
            minHeap.offer(score);
            if (minHeap.size() > K)
                minHeap.poll();
        }

        while (!minHeap.isEmpty())
            ans += minHeap.poll();

        return ans;
    }

    public void reset(int playerId) {
        idToScore.remove(playerId);
    }

    private Map<Integer, Integer> idToScore = new HashMap<>();
}
```

254. 1245.java

Path: LeetCode-main/solutions/1245. Tree Diameter/1245.java

```
class Solution {
    public int treeDiameter(int[][] edges) {
        final int n = edges.length;
        List<Integer>[] tree = new List[n + 1];

        for (int i = 0; i < tree.length; ++i)
            tree[i] = new ArrayList<>();

        for (int[] edge : edges) {
            final int u = edge[0];
            final int v = edge[1];
            tree[u].add(v);
            tree[v].add(u);
        }

        maxDepth(tree, 0, -1);
        return ans;
    }

    private int ans = 0;

    private int maxDepth(List<Integer>[] tree, int u, int prev) {
        int maxDepth1 = 0; // the maximum depth
        int maxDepth2 = -1; // the second maximum depth

        for (final int v : tree[u]) {
            if (v == prev)
                continue;
            final int depth = maxDepth(tree, v, u);
            if (depth > maxDepth1) {
                maxDepth2 = maxDepth1;
                maxDepth1 = depth;
            } else if (depth > maxDepth2) {
                maxDepth2 = depth;
            }
        }

        ans = Math.max(ans, maxDepth1 + maxDepth2);
        return 1 + maxDepth1;
    }
}
```

255. 1246.java

Path: LeetCode-main/solutions/1246. Palindrome Removal/1246.java

```
class Solution {
    public int minimumMoves(int[] arr) {
        final int n = arr.length;
        // dp[i][j] := the minimum number of moves to remove all numbers from arr[i..j]
        int[][] dp = new int[n][n];
        Arrays.stream(dp).forEach(A -> Arrays.fill(A, n));

        for (int i = 0; i < n; ++i)
            dp[i][i] = 1;

        for (int i = 0; i + 1 < n; ++i)
            dp[i][i + 1] = arr[i] == arr[i + 1] ? 1 : 2;

        for (int d = 2; d < n; ++d)
            for (int i = 0; i + d < n; ++i) {
                final int j = i + d;
                // Remove arr[i] and arr[j] within the move of removing
                // arr[i + 1..j - 1]
                if (arr[i] == arr[j])
                    dp[i][j] = dp[i + 1][j - 1];
                // Try all the possible partitions.
                for (int k = i; k < j; ++k)
                    dp[i][j] = Math.min(dp[i][j], dp[i][k] + dp[k + 1][j]);
            }

        return dp[0][n - 1];
    }
}
```

256. 1247.java

Path: LeetCode-main/solutions/1247. Minimum Swaps to Make Strings Equal/1247.java

```
class Solution {
    public int minimumSwap(String s1, String s2) {
        // ("xx", "yy") = (2 "xy"s) -> 1 swap
        // ("yy", "xx") = (2 "yx"s) -> 1 swap
        // ("xy", "yx") = (1 "xy" and 1 "yx") -> 2 swaps
        int xy = 0; // the number of indices i's s.t. s1[i] = 'x' and s2[i] 'y'
        int yx = 0; // the number of indices i's s.t. s1[i] = 'y' and s2[i] 'x'

        for (int i = 0; i < s1.length(); ++i) {
            if (s1.charAt(i) == s2.charAt(i))
                continue;
            if (s1.charAt(i) == 'x')
                ++xy;
            else
                ++yx;
        }

        if ((xy + yx) % 2 == 1)
            return -1;
        return xy / 2 + yx / 2 + (xy % 2 == 0 ? 0 : 2);
    }
}
```


257. 1248.java

Path: LeetCode-main/solutions/1248. Count Number of Nice Subarrays/1248.java

```
class Solution {
    public int numberOfSubarrays(int[] nums, int k) {
        return numberOfSubarraysAtMost(nums, k) - numberOfSubarraysAtMost(nums, k - 1);
    }

    private int numberOfSubarraysAtMost(int[] nums, int k) {
        int ans = 0;

        for (int l = 0, r = 0; r <= nums.length;)
            if (k >= 0) {
                ans += r - l;
                if (r == nums.length)
                    break;
                if (nums[r] % 2 == 1)
                    --k;
                ++r;
            } else {
                if (nums[l] % 2 == 1)
                    ++k;
                ++l;
            }
        return ans;
    }
}
```

258. 1249.java

Path: LeetCode-main/solutions/1249. Minimum Remove to Make Valid Parentheses/1249.java

```
class Solution {
    public String minRemoveToMakeValid(String s) {
        Deque<Integer> stack = new ArrayDeque<>(); // unpaired '(' indices
        StringBuilder sb = new StringBuilder(s);

        for (int i = 0; i < s.length(); ++i)
            if (sb.charAt(i) == '(') {
                stack.push(i); // Record the unpaired '(' index.
            } else if (sb.charAt(i) == ')') {
                if (stack.isEmpty())
                    sb.setCharAt(i, '#'); // Mark the unpaired ')' as '#'.
                else
                    stack.pop(); // Find a pair!
            }

        // Mark the unpaired '(' as '#'.
        while (!stack.isEmpty())
            sb.setCharAt(stack.pop(), '#');

        return sb.toString().replaceAll("#", "");
    }
}
```

259. 125.java

Path: LeetCode-main/solutions/125. Valid Palindrome/125.java

```
class Solution {
    public boolean isPalindrome(String s) {
        int l = 0;
        int r = s.length() - 1;

        while (l < r) {
            while (l < r && !Character.isLetterOrDigit(s.charAt(l)))
                ++l;
            while (l < r && !Character.isLetterOrDigit(s.charAt(r)))
                --r;
            if (Character.toLowerCase(s.charAt(l)) != Character.toLowerCase(s.charAt(r)))
                return false;
            ++l;
            --r;
        }

        return true;
    }
}
```

260. 1252-2.java

Path: LeetCode-main/solutions/1252. Cells with Odd Values in a Matrix/1252-2.java

```
class Solution {
    public int oddCells(int m, int n, int[][] indices) {
        // rows[i] and cols[i] :=
        // 1. true (flipped even times)
        // 2. false (flipped odd times)
        boolean[] rows = new boolean[n];
        boolean[] cols = new boolean[n];

        for (int[] index : indices) {
            rows[index[0]] = rows[index[0]] ^ true;
            cols[index[1]] = cols[index[1]] ^ true;
        }

        int oddRowCount = 0;
        int oddColsCount = 0;

        for (boolean row : rows)
            if (row)
                ++oddRowCount;

        for (boolean col : cols)
            if (col)
                ++oddColsCount;

        final int evenRowCount = m - oddRowCount;
        final int evenColsCount = n - oddColsCount;
        return oddRowCount * evenColsCount + oddColsCount * evenRowCount;
    }
}
```


262. 1253.java

Path: LeetCode-main/solutions/1253. Reconstruct a 2-Row Binary Matrix/1253.java

```
class Solution {
    public List<List<Integer>> reconstructMatrix(int upper, int lower, int[] colsum) {
        if (upper + lower != Arrays.stream(colsum).sum())
            return new ArrayList<>();

        int count = 0;
        for (int c : colsum)
            if (c == 2)
                ++count;

        if (Math.min(upper, lower) < count)
            return new ArrayList<>();

        int[][] ans = new int[2][colsum.length];

        for (int j = 0; j < colsum.length; ++j)
            if (colsum[j] == 2) {
                ans[0][j] = 1;
                ans[1][j] = 1;
                --upper;
                --lower;
            }

        for (int j = 0; j < colsum.length; ++j) {
            if (colsum[j] == 1 && upper > 0) {
                ans[0][j] = 1;
                --colsum[j];
                --upper;
            }

            if (colsum[j] == 1 && lower > 0) {
                ans[1][j] = 1;
                --lower;
            }
        }

        return new ArrayList(Arrays.asList(ans[0], ans[1]));
    }
}
```

263. 1254.java

Path: LeetCode-main/solutions/1254. Number of Closed Islands/1254.java

```
class Solution {
    public int closedIsland(int[][] grid) {
        final int m = grid.length;
        final int n = grid[0].length;

        // Remove the lands connected to the edge.
        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j)
                if (i * j == 0 || i == m - 1 || j == n - 1)
                    if (grid[i][j] == 0)
                        dfs(grid, i, j);

        int ans = 0;

        // Reduce to 200. Number of Islands
        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j)
                if (grid[i][j] == 0) {
                    dfs(grid, i, j);
                    ++ans;
                }

        return ans;
    }

    private void dfs(int[][] grid, int i, int j) {
        if (i < 0 || i == grid.length || j < 0 || j == grid[0].length)
            return;
        if (grid[i][j] == 1)
            return;
        grid[i][j] = 1;
        dfs(grid, i + 1, j);
        dfs(grid, i - 1, j);
        dfs(grid, i, j + 1);
        dfs(grid, i, j - 1);
    }
}
```

264. 1255.java

Path: LeetCode-main/solutions/1255. Maximum Score Words Formed by Letters/1255.java

```
class Solution {
    public int maxScoreWords(String[] words, char[] letters, int[] score) {
        int[] count = new int[26];
        for (final char c : letters)
            ++count[c - 'a'];
        return dfs(words, 0, count, score);
    }

    // Returns the maximum score you can get from words[s..n).
    private int dfs(String[] words, int s, int[] count, int[] score) {
        int ans = 0;
        for (int i = s; i < words.length; ++i) {
            final int earned = useWord(words, i, count, score);
            if (earned > 0)
                ans = Math.max(ans, earned + dfs(words, i + 1, count, score));
            unuseWord(words, i, count);
        }
        return ans;
    }

    int useWord(String[] words, int i, int[] count, int[] score) {
        boolean isValid = true;
        int earned = 0;
        for (final char c : words[i].toCharArray()) {
            if (--count[c - 'a'] < 0)
                isValid = false;
            earned += score[c - 'a'];
        }
        return isValid ? earned : -1;
    }

    void unuseWord(String[] words, int i, int[] count) {
        for (final char c : words[i].toCharArray())
            ++count[c - 'a'];
    }
}
```


265. 1256.java

Path: LeetCode-main/solutions/1256. Encode Number/1256.java

```
class Solution {  
    public String encode(int num) {  
        return Integer.toBinaryString(num + 1).substring(1);  
    }  
}
```

266. 1257.java

Path: LeetCode-main/solutions/1257. Smallest Common Region/1257.java

```
class Solution {
    public String findSmallestRegion(List<List<String>> regions, String region1, String region2) {
        Map<String, String> parent = new HashMap<>();
        Set<String> ancestors = new HashSet<>(); // region1's ancestors

        for (List<String> region : regions)
            for (int i = 1; i < region.size(); ++i)
                parent.put(region.get(i), region.get(0));

        // Add all the region1's ancestors.
        while (region1 != null) {
            ancestors.add(region1);
            region1 = parent.get(region1); // Region1 becomes null in the end.
        }

        // Go up from region2 until meet any of region1's ancestors.
        while (!ancestors.contains(region2))
            region2 = parent.get(region2);

        return region2;
    }
}
```

267. 1258.java

Path: LeetCode-main/solutions/1258. Synonymous Sentences/1258.java

```
class Solution {
    public List<String> generateSentences(List<List<String>> synonyms, String text) {
        Set<String> ans = new TreeSet<>();
        Map<String, List<String>> graph = new HashMap<>();
        Queue<String> q = new ArrayDeque<>(List.of(text));

        for (List<String> synonym : synonyms) {
            final String s = synonym.get(0);
            final String t = synonym.get(1);
            graph.putIfAbsent(s, new ArrayList<>());
            graph.putIfAbsent(t, new ArrayList<>());
            graph.get(s).add(t);
            graph.get(t).add(s);
        }

        while (!q.isEmpty()) {
            final String u = q.poll();
            ans.add(u);
            String[] words = u.split("\\s");
            for (int i = 0; i < words.length; ++i)
                for (final String synonym : graph.getOrDefault(words[i], new ArrayList<>())) {
                    // Replace words[i] with its synonym.
                    words[i] = synonym;
                    final String newText = String.join(" ", words);
                    if (!ans.contains(newText))
                        q.offer(newText);
                }
        }

        return new ArrayList<>(ans);
    }
}
```

268. 1259.java

Path: LeetCode-main/solutions/1259. Handshakes That Don't Cross/1259.java

```
class Solution {
    public int numberOfWays(int numPeople) {
        final long MOD = 1_000_000_007;
        // dp[i] := the number of ways i handshakes could occur s.t. none of the
        // handshakes cross
        long[] dp = new long[numPeople / 2 + 1];
        dp[0] = 1;

        for (int i = 1; i <= numPeople / 2; ++i)
            for (int j = 0; j < i; ++j) {
                dp[i] += dp[j] * dp[i - 1 - j];
                dp[i] %= MOD;
            }

        return (int) dp[numPeople / 2];
    }
}
```

269. 126.java

Path: LeetCode-main/solutions/126. Word Ladder II/126.java

```
class Solution {
    public List<List<String>> findLadders(String beginWord, String endWord, List<String> wordList) {
        Set<String> wordSet = new HashSet<>(wordList);
        if (!wordSet.contains(endWord))
            return new ArrayList<>();

        // {"hit": ["hot"], "hot": ["dot", "lot"], ...}
        Map<String, List<String>> graph = new HashMap<>();

        // Build the graph from the beginWord to the endWord.
        if (!bfs(beginWord, endWord, wordSet, graph))
            return new ArrayList<>();

        List<List<String>> ans = new ArrayList<>();
        List<String> path = new ArrayList<>(List.of(beginWord));
        dfs(graph, beginWord, endWord, path, ans);
        return ans;
    }

    private boolean bfs(final String beginWord, final String endWord, Set<String> wordSet,
                       Map<String, List<String>> graph) {
        Set<String> currentLevelWords = new HashSet<>();
        currentLevelWords.add(beginWord);
        boolean reachEndWord = false;

        while (!currentLevelWords.isEmpty()) {
            for (final String word : currentLevelWords)
                wordSet.remove(word);
            Set<String> nextLevelWords = new HashSet<>();
            for (final String parent : currentLevelWords) {
                graph.putIfAbsent(parent, new ArrayList<>());
                for (final String child : getChildren(parent, wordSet)) {
                    if (wordSet.contains(child)) {
                        nextLevelWords.add(child);
                        graph.get(parent).add(child);
                    }
                    if (child.equals(endWord))
                        reachEndWord = true;
                }
            }
            if (reachEndWord)
                return true;
            currentLevelWords = nextLevelWords;
        }

        return false;
    }

    private List<String> getChildren(final String parent, Set<String> wordSet) {
        List<String> children = new ArrayList<>();
        StringBuilder sb = new StringBuilder(parent);

        for (int i = 0; i < sb.length(); ++i) {
            final char cache = sb.charAt(i);
            for (char c = 'a'; c <= 'z'; ++c) {
                if (c == cache)
                    continue;
                sb.setCharAt(i, c);
                final String child = sb.toString();
                if (wordSet.contains(child))
                    children.add(child);
            }
            sb.setCharAt(i, cache);
        }

        return children;
    }

    private void dfs(Map<String, List<String>> graph, final String word, final String endWord,
                    List<String> path, List<List<String>> ans) {
        if (word.equals(endWord)) {
            ans.add(new ArrayList<>(path));
            return;
        }
        if (!graph.containsKey(word))
            return;

        for (final String child : graph.get(word)) {
            path.add(child);
            dfs(graph, child, endWord, path, ans);
        }
    }
}
```

```
        path.remove(path.size() - 1);  
    }  
}
```

270. 1260.java

Path: LeetCode-main/solutions/1260. Shift 2D Grid/1260.java

```
class Solution {
    public List<List<Integer>> shiftGrid(int[][] grid, int k) {
        final int m = grid.length;
        final int n = grid[0].length;
        List<List<Integer>> ans = new ArrayList<>();
        int[][] arr = new int[m][n];

        k %= m * n;

        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j) {
                final int index = (i * n + j + k) % (m * n);
                final int x = index / n;
                final int y = index % n;
                arr[x][y] = grid[i][j];
            }

        for (int[] row : arr)
            ans.add(Arrays.stream(row).boxed().collect(Collectors.toList()));

        return ans;
    }
}
```

271. 1261.java

Path: LeetCode-main/solutions/1261. Find Elements in a Contaminated Binary Tree/1261.java

```
class FindElements {
    public FindElements(TreeNode root) {
        dfs(root, 0);
    }

    public boolean find(int target) {
        return vals.contains(target);
    }

    private Set<Integer> vals = new HashSet<>();

    private void dfs(TreeNode root, int val) {
        if (root == null)
            return;

        root.val = val;
        vals.add(val);
        dfs(root.left, val * 2 + 1);
        dfs(root.right, val * 2 + 2);
    }
}
```


272. 1262.java

Path: LeetCode-main/solutions/1262. Greatest Sum Divisible by Three/1262.java

```
class Solution {
    public int maxSumDivThree(int[] nums) {
        int[] dp = new int[3]; // dp[i] := the maximum sum so far s.t. sum % 3 == i

        for (final int num : nums)
            for (final int sum : dp.clone())
                dp[(sum + num) % 3] = Math.max(dp[(sum + num) % 3], sum + num);

        return dp[0];
    }
}
```

273. 1263.java

Path: LeetCode-main/solutions/1263. Minimum Moves to Move a Box to Their Target Location/1263.java

```
class Solution {
    public int minPushBox(char[][] grid) {
        record T(int boxX, int boxY, int playerX, int playerY) {}
        final int m = grid.length;
        final int n = grid[0].length;
        int[] box = {-1, -1};
        int[] player = {-1, -1};
        int[] target = {-1, -1};

        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j)
                if (grid[i][j] == 'B')
                    box = new int[] {i, j};
                else if (grid[i][j] == 'S')
                    player = new int[] {i, j};
                else if (grid[i][j] == 'T')
                    target = new int[] {i, j};

        Queue<T> q = new ArrayDeque<>(List.of(new T(box[0], box[1], player[0], player[1])));
        boolean[][][] seen = new boolean[m][n][m][n];
        seen[box[0]][box[1]][player[0]][player[1]] = true;

        for (int step = 0; !q.isEmpty(); ++step)
            for (int sz = q.size(); sz > 0; --sz) {
                final int boxX = q.peek().boxX;
                final int boxY = q.peek().boxY;
                final int playerX = q.peek().playerX;
                final int playerY = q.poll().playerY;
                if (boxX == target[0] && boxY == target[1])
                    return step;
                for (int k = 0; k < 4; ++k) {
                    final int nextBoxX = boxX + DIRS[k][0];
                    final int nextBoxY = boxY + DIRS[k][1];
                    if (isInvalid(grid, nextBoxX, nextBoxY))
                        continue;
                    if (seen[nextBoxX][nextBoxY][boxX][boxY])
                        continue;
                    final int fromX = boxX + DIRS[(k + 2) % 4][0];
                    final int fromY = boxY + DIRS[(k + 2) % 4][1];
                    if (isInvalid(grid, fromX, fromY))
                        continue;
                    if (canGoTo(grid, playerX, playerY, fromX, fromY, boxX, boxY)) {
                        seen[nextBoxX][nextBoxY][boxX][boxY] = true;
                        q.offer(new T(nextBoxX, nextBoxY, boxX, boxY));
                    }
                }
            }

        return -1;
    }

    private static final int[][] DIRS = {{0, 1}, {1, 0}, {0, -1}, {-1, 0}};

    // Returns true if (playerX, playerY) can go to (fromX, fromY).
    private boolean canGoTo(char[][] grid, int playerX, int playerY, int fromX, int fromY, int boxX,
        int boxY) {
        Queue<Pair<Integer, Integer>> q = new ArrayDeque<>(List.of(new Pair<>(playerX, playerY)));
        boolean[][] seen = new boolean[grid.length][grid[0].length];
        seen[playerX][playerY] = true;

        while (!q.isEmpty()) {
            final int i = q.peek().getKey();
            final int j = q.poll().getValue();
            if (i == fromX && j == fromY)
                return true;
            for (int[] dir : DIRS) {
                final int x = i + dir[0];
                final int y = j + dir[1];
                if (isInvalid(grid, x, y))
                    continue;
                if (seen[x][y])
                    continue;
                if (x == boxX && y == boxY)
                    continue;
                q.offer(new Pair<>(x, y));
                seen[x][y] = true;
            }
        }

        return false;
    }
}
```

```
}  
private boolean isInvalid(char[][] grid, int playerX, int playerY) {  
    return playerX < 0 || playerX == grid.length || playerY < 0 || playerY == grid[0].length ||  
        grid[playerX][playerY] == '#';  
}  
}
```

274. 1266.java

Path: LeetCode-main/solutions/1266. Minimum Time Visiting All Points/1266.java

```
class Solution {
    public int minTimeToVisitAllPoints(int[][] points) {
        int ans = 0;

        for (int i = 1; i < points.length; ++i)
            ans += Math.max(Math.abs(points[i][0] - points[i - 1][0]),
                            Math.abs(points[i][1] - points[i - 1][1]));

        return ans;
    }
}
```


276. 1268.java

Path: LeetCode-main/solutions/1268. Search Suggestions System/1268.java

```
class TrieNode {
    public TrieNode[] children = new TrieNode[26];
    public String word;
}

class Solution {
    public List<List<String>> suggestedProducts(String[] products, String searchWord) {
        List<List<String>> ans = new ArrayList<>();

        for (final String product : products)
            insert(product);

        TrieNode node = root;

        for (final char c : searchWord.toCharArray()) {
            if (node == null || node.children[c - 'a'] == null) {
                node = null;
                ans.add(new ArrayList<>());
                continue;
            }
            node = node.children[c - 'a'];
            ans.add(search(node));
        }

        return ans;
    }

    private TrieNode root = new TrieNode();

    private void insert(final String word) {
        TrieNode node = root;
        for (final char c : word.toCharArray()) {
            final int i = c - 'a';
            if (node.children[i] == null)
                node.children[i] = new TrieNode();
            node = node.children[i];
        }
        node.word = word;
    }

    private List<String> search(TrieNode node) {
        List<String> res = new ArrayList<>();
        dfs(node, res);
        return res;
    }

    private void dfs(TrieNode node, List<String> ans) {
        if (ans.size() == 3)
            return;
        if (node == null)
            return;
        if (node.word != null)
            ans.add(node.word);
        for (TrieNode child : node.children)
            dfs(child, ans);
    }
}
```

277. 1269.java

Path: LeetCode-main/solutions/1269. Number of Ways to Stay in the Same Place After Some Steps/1269.java

```
class Solution {
    public int numWays(int steps, int arrLen) {
        final int MOD = 1_000_000_007;
        final int n = Math.min(arrLen, steps / 2 + 1);
        // dp[i] := the number of ways to stay at index i
        long[] dp = new long[n];
        dp[0] = 1;

        while (steps-- > 0) {
            long[] newDp = new long[n];
            for (int i = 0; i < n; ++i) {
                newDp[i] = dp[i];
                if (i - 1 >= 0)
                    newDp[i] += dp[i - 1];
                if (i + 1 < n)
                    newDp[i] += dp[i + 1];
                newDp[i] %= MOD;
            }
            dp = newDp;
        }

        return (int) dp[0];
    }
}
```

278. 127.java

Path: LeetCode-main/solutions/127. Word Ladder/127.java

```
class Solution {
    public int ladderLength(String beginWord, String endWord, List<String> wordList) {
        Set<String> wordSet = new HashSet<>(wordList);
        if (!wordSet.contains(endWord))
            return 0;

        Queue<String> q = new ArrayDeque<>(List.of(beginWord));

        for (int step = 1; !q.isEmpty(); ++step)
            for (int sz = q.size(); sz > 0; --sz) {
                StringBuilder sb = new StringBuilder(q.poll());
                for (int i = 0; i < sb.length(); ++i) {
                    final char cache = sb.charAt(i);
                    for (char c = 'a'; c <= 'z'; ++c) {
                        sb.setCharAt(i, c);
                        final String word = sb.toString();
                        if (word.equals(endWord))
                            return step + 1;
                        if (wordSet.contains(word)) {
                            q.offer(word);
                            wordSet.remove(word);
                        }
                    }
                    sb.setCharAt(i, cache);
                }
            }
        return 0;
    }
}
```


279. 1271.java

Path: LeetCode-main/solutions/1271. Hexspeak/1271.java

```
class Solution {  
    public String toHexspeak(String num) {  
        final long n = Long.parseLong(num);  
        final String ans = Long.toHexString(n).toUpperCase().replace('1', 'I').replace('0', 'O');  
        return ans.chars().anyMatch(c -> Character.isDigit(c)) ? "ERROR" : ans;  
    }  
}
```

280. 1272.java

Path: LeetCode-main/solutions/1272. Remove Interval/1272.java

```
class Solution {
    public List<List<Integer>> removeInterval(int[][] intervals, int[] toBeRemoved) {
        List<List<Integer>> ans = new ArrayList<>();

        for (int[] interval : intervals) {
            final int a = interval[0];
            final int b = interval[1];
            if (a >= toBeRemoved[1] || b <= toBeRemoved[0]) {
                ans.add(Arrays.asList(a, b));
            } else { // a < toBeRemoved[1] && b > toBeRemoved[0]
                if (a < toBeRemoved[0])
                    ans.add(Arrays.asList(a, toBeRemoved[0]));
                if (b > toBeRemoved[1])
                    ans.add(Arrays.asList(toBeRemoved[1], b));
            }
        }

        return ans;
    }
}
```

281. 1273.java

Path: LeetCode-main/solutions/1273. Delete Tree Nodes/1273.java

```
class Solution {
    public int deleteTreeNodes(int nodes, int[] parent, int[] value) {
        List<Integer>[] tree = new List[nodes];

        for (int i = 0; i < tree.length; ++i)
            tree[i] = new ArrayList<>();

        for (int i = 1; i < parent.length; ++i)
            tree[parent[i]].add(i);

        return dfs(tree, 0, value).count;
    }

    private record T(int sum, int count) {}

    private T dfs(List<Integer>[] tree, int u, int[] value) {
        int sum = value[u]; // the root value
        int count = 1;      // this root

        for (final int v : tree[u]) {
            final T t = dfs(tree, v, value);
            sum += t.sum;
            count += t.count;
        }

        if (sum == 0) // Delete this root.
            return new T(0, 0); // So, its count = 0.
        return new T(sum, count);
    }
}
```

282. 1274.java

Path: LeetCode-main/solutions/1274. Number of Ships in a Rectangle/1274.java

```
/**
 * // This is Sea's API interface.
 * // You should not implement it, or speculate about its implementation
 * class Sea {
 *     public boolean hasShips(int[] topRight, int[] bottomLeft);
 * }
 */

class Solution {
    public int countShips(Sea sea, int[] topRight, int[] bottomLeft) {
        if (topRight[0] < bottomLeft[0] || topRight[1] < bottomLeft[1])
            return 0;
        if (!sea.hasShips(topRight, bottomLeft))
            return 0;

        // sea.hashShips(topRight, bottomLeft) == true
        if (topRight[0] == bottomLeft[0] && topRight[1] == bottomLeft[1])
            return 1;

        final int mx = (topRight[0] + bottomLeft[0]) / 2;
        final int my = (topRight[1] + bottomLeft[1]) / 2;
        int ans = 0;
        // the top-right
        ans += countShips(sea, topRight, new int[] {mx + 1, my + 1});
        // the bottom-right
        ans += countShips(sea, new int[] {topRight[0], my}, new int[] {mx + 1, bottomLeft[1]});
        // the top-left
        ans += countShips(sea, new int[] {mx, topRight[1]}, new int[] {bottomLeft[0], my + 1});
        // the bottom-left
        ans += countShips(sea, new int[] {mx, my}, bottomLeft);
        return ans;
    }
}
```

283. 1275.java

Path: LeetCode-main/solutions/1275. Find Winner on a Tic Tac Toe Game/1275.java

```
class Solution {
    public String tictactoe(int[][] moves) {
        int[][] row = new int[2][3];
        int[][] col = new int[2][3];
        int[] diag1 = new int[2];
        int[] diag2 = new int[2];

        for (int i = 0; i < moves.length; ++i) {
            final int r = moves[i][0];
            final int c = moves[i][1];
            final int j = i & 1;
            if (++row[j][r] == 3 || ++col[j][c] == 3 || r == c && ++diag1[j] == 3 ||
                r + c == 2 && ++diag2[j] == 3)
                return j == 0 ? "A" : "B";
        }

        return moves.length == 9 ? "Draw" : "Pending";
    }
}
```

284. 1276.java

Path: LeetCode-main/solutions/1276. Number of Burgers with No Waste of Ingredients/1276.java

```
class Solution {
    public List<Integer> numOfBurgers(int tomatoSlices, int cheeseSlices) {
        if (tomatoSlices % 2 == 1 || tomatoSlices < 2 * cheeseSlices || tomatoSlices > cheeseSlices * 4)
            return new ArrayList<>();

        int jumboBurgers = (tomatoSlices - 2 * cheeseSlices) / 2;

        return List.of(jumboBurgers, cheeseSlices - jumboBurgers);
    }
}
```

285. 1277.java

Path: LeetCode-main/solutions/1277. Count Square Submatrices with All Ones/1277.java

```
class Solution {
    public int countSquares(int[][] matrix) {
        for (int i = 0; i < matrix.length; ++i)
            for (int j = 0; j < matrix[0].length; ++j)
                if (matrix[i][j] == 1 && i > 0 && j > 0)
                    matrix[i][j] +=
                        Math.min(matrix[i - 1][j - 1], Math.min(matrix[i - 1][j], matrix[i][j - 1]));
        return Arrays.stream(matrix).flatMapToInt(Arrays::stream).sum();
    }
}
```

286. 1278-2.java

Path: LeetCode-main/solutions/1278. Palindrome Partitioning III/1278-2.java

```
class Solution {
    public int palindromePartition(String s, int K) {
        final int n = s.length();
        // dp[i][k] := the minimum cost to make k palindromes by s[0..i)
        int[][] dp = new int[n + 1][K + 1];
        Arrays.stream(dp).forEach(A -> Arrays.fill(A, n));
        // cost[i][j] := the minimum cost to make s[i..j] palindrome
        int[][] cost = new int[n][n];

        for (int d = 1; d < n; ++d)
            for (int i = 0, j = d; j < n; ++i, ++j)
                cost[i][j] = (s.charAt(i) == s.charAt(j)) ? 0 : 1 + cost[i + 1][j - 1];

        for (int i = 1; i <= n; ++i)
            dp[i][1] = cost[0][i - 1];

        for (int k = 2; k <= K; ++k)
            for (int i = k; i <= n; ++i)
                for (int j = k - 1; j < i; ++j)
                    dp[i][k] = Math.min(dp[i][k], dp[j][k - 1] + cost[j][i - 1]);

        return dp[n][K];
    }
}
```


287. 1278.java

Path: LeetCode-main/solutions/1278. Palindrome Partitioning III/1278.java

```
class Solution {
    public int palindromePartition(String s, int k) {
        final int n = s.length();
        int[][] mem = new int[n + 1][k + 1];
        // cost[i][j] := the minimum cost to make s[i..j] palindrome
        int[][] cost = new int[n][n];

        Arrays.stream(mem).forEach(A -> Arrays.fill(A, n));

        for (int d = 1; d < n; ++d)
            for (int i = 0, j = d; j < n; ++i, ++j)
                cost[i][j] = (s.charAt(i) == s.charAt(j)) ? 0 : 1 + cost[i + 1][j - 1];

        return palindromePartition(n, k, cost, mem);
    }

    // Returns the minimum cost to make k palindromes by s[0..i].
    private int palindromePartition(int n, int k, int[][] cost, int[][] mem) {
        if (k == 1)
            return cost[0][n - 1];
        if (mem[n][k] < n)
            return mem[n][k];

        // Try all the possible partitions.
        for (int i = k - 1; i < n; ++i)
            mem[n][k] = Math.min(mem[n][k], //
                palindromePartition(i, k - 1, cost, mem) + cost[i][n - 1]);

        return mem[n][k];
    }
}
```

288. 1279.java

Path: LeetCode-main/solutions/1279. Traffic Light Controlled Intersection/1279.java

```
class TrafficLight {
    public void carArrived(
        // ID of the car
        int carId,
        // ID of the road the car travels on. Can be 1 (road A) or 2 (road B).
        int roadId,
        // direction of the car
        int direction,
        // Use turnGreen() to turn light to green on current road.
        Runnable turnGreen,
        // Use crossCar() to make car cross the intersection.
        Runnable crossCar) {
        synchronized (this) {
            if (canPassRoadId != roadId) {
                canPassRoadId = roadId;
                turnGreen.run();
            }
            crossCar.run();
        }
    }

    private int canPassRoadId = 1; // 1 := road A, 2 := road B
}
```

289. 128.java

Path: LeetCode-main/solutions/128. Longest Consecutive Sequence/128.java

```
class Solution {
    public int longestConsecutive(int[] nums) {
        int ans = 0;
        Set<Integer> seen = Arrays.stream(nums).boxed().collect(Collectors.toSet());

        for (int num : seen) {
            // `num` is the start of a sequence.
            if (seen.contains(num - 1))
                continue;
            int length = 1;
            while (seen.contains(++num))
                ++length;
            ans = Math.max(ans, length);
        }

        return ans;
    }
}
```

290. 1281.java

Path: LeetCode-main/solutions/1281. Subtract the Product and Sum of Digits of an Integer/1281.java

```
class Solution {
    public int subtractProductAndSum(int n) {
        int prod = 1;
        int summ = 0;

        for (; n > 0; n /= 10) {
            prod *= n % 10;
            summ += n % 10;
        }

        return prod - summ;
    }
}
```

291. 1282.java

Path: LeetCode-main/solutions/1282. Group the People Given the Group Size They Belong To/1282.java

```
class Solution {
    public List<List<Integer>> groupThePeople(int[] groupSizes) {
        List<List<Integer>> ans = new ArrayList<>();
        Map<Integer, List<Integer>> groupSizeToIndices = new HashMap<>();

        for (int i = 0; i < groupSizes.length; ++i) {
            final int groupSize = groupSizes[i];
            groupSizeToIndices.putIfAbsent(groupSize, new ArrayList<>());
            groupSizeToIndices.get(groupSize).add(i);
        }

        for (Map.Entry<Integer, List<Integer>> entry : groupSizeToIndices.entrySet()) {
            final int groupSize = entry.getKey();
            List<Integer> indices = entry.getValue();
            List<Integer> groupIndices = new ArrayList<>();
            for (final int index : indices) {
                groupIndices.add(index);
                if (groupIndices.size() == groupSize) {
                    ans.add(new ArrayList<>(groupIndices));
                    groupIndices.clear();
                }
            }
        }

        return ans;
    }
}
```

292. 1283.java

Path: LeetCode-main/solutions/1283. Find the Smallest Divisor Given a Threshold/1283.java

```
class Solution {
    public int smallestDivisor(int[] nums, int threshold) {
        int l = 1;
        int r = Arrays.stream(nums).max().getAsInt();

        while (l < r) {
            final int m = (l + r) / 2;
            if (sumDivision(nums, m) <= threshold)
                r = m;
            else
                l = m + 1;
        }

        return l;
    }

    private int sumDivision(int[] nums, int m) {
        int sum = 0;
        for (final int num : nums)
            sum += (num - 1) / m + 1;
        return sum;
    }
}
```

293. 1284.java

Path: LeetCode-main/solutions/1284. Minimum Number of Flips to Convert Binary Matrix to Zero Matrix/1284.java

```
class Solution {
    public int minFlips(int[][] mat) {
        final int m = mat.length;
        final int n = mat[0].length;
        final int hash = getHash(mat, m, n);
        if (hash == 0)
            return 0;

        final int[][] DIRS = {{0, 1}, {1, 0}, {0, -1}, {-1, 0}};
        Queue<Integer> q = new ArrayDeque<>(List.of(hash));
        Set<Integer> seen = new HashSet<>(Arrays.asList(hash));

        for (int step = 1; !q.isEmpty(); ++step) {
            for (int sz = q.size(); sz > 0; --sz) {
                final int curr = q.poll();
                for (int i = 0; i < m; ++i) {
                    for (int j = 0; j < n; ++j) {
                        int next = curr ^ 1 << (i * n + j);
                        // Flie the four neighbors.
                        for (int[] dir : DIRS) {
                            final int x = i + dir[0];
                            final int y = j + dir[1];
                            if (x < 0 || x == m || y < 0 || y == n)
                                continue;
                            next ^= 1 << (x * n + y);
                        }
                        if (next == 0)
                            return step;
                        if (seen.contains(next))
                            continue;
                        q.offer(next);
                        seen.add(next);
                    }
                }
            }
        }

        return -1;
    }

    private int getHash(int[][] mat, int m, int n) {
        int hash = 0;
        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j)
                if (mat[i][j] == 1)
                    hash |= 1 << (i * n + j);
        return hash;
    }
}
```

294. 1286.java

Path: LeetCode-main/solutions/1286. Iterator for Combination/1286.java

```
class CombinationIterator {
    public CombinationIterator(String characters, int combinationLength) {
        final int n = characters.length();
        final int k = combinationLength;

        // Generate bitmasks from 0..00 to 1..11
        for (int mask = 0; mask < 1 << n; mask++) {
            // Use bitmasks with k 1-bits
            if (Integer.bitCount(mask) == k) {
                // Convert bitmask into combination
                // 111 --> "abc", 000 --> ""
                // 110 --> "ab", 101 --> "ac", 011 --> "bc"
                StringBuilder curr = new StringBuilder();
                for (int j = 0; j < n; j++) {
                    if ((mask & (1 << n - j - 1)) != 0) {
                        curr.append(characters.charAt(j));
                    }
                }
                combinations.push(curr.toString());
            }
        }
    }

    public String next() {
        return combinations.pop();
    }

    public boolean hasNext() {
        return (!combinations.isEmpty());
    }

    private Deque<String> combinations = new ArrayDeque<String>();
}
```


295. 1287.java

Path: LeetCode-main/solutions/1287. Element Appearing More Than 25% In Sorted Array/1287.java

```
class Solution {  
    public int findSpecialInteger(int[] arr) {  
        final int n = arr.length;  
        final int quarter = n / 4;  
  
        for (int i = 0; i < n - quarter; ++i)  
            if (arr[i] == arr[i + quarter])  
                return arr[i];  
  
        throw new IllegalArgumentException();  
    }  
}
```

296. 1288.java

Path: LeetCode-main/solutions/1288. Remove Covered Intervals/1288.java

```
class Solution {
    public int removeCoveredIntervals(int[][] intervals) {
        Arrays.sort(intervals, Comparator.comparingInt((int[] interval) -> interval[0])
            .thenComparingInt((int[] interval) -> - interval[1]));

        int ans = 0;
        int prevEnd = 0;

        for (int[] interval : intervals)
            // The current interval is not covered by the previous one.
            if (prevEnd < interval[1]) {
                prevEnd = interval[1];
                ++ans;
            }
        return ans;
    }
}
```

297. 1289.java

Path: LeetCode-main/solutions/1289. Minimum Falling Path Sum II/1289.java

```
class Solution {
    public int minFallingPathSum(int[][] grid) {
        final int n = grid.length;

        for (int i = 1; i < n; ++i) {
            Pair<Integer, Integer>[] twoMinNumAndIndexes = getTwoMinNumAndIndexes(grid[i - 1]);
            final int firstMinNum = twoMinNumAndIndexes[0].getKey();
            final int firstMinIndex = twoMinNumAndIndexes[0].getValue();
            final int secondMinNum = twoMinNumAndIndexes[1].getKey();
            for (int j = 0; j < n; ++j)
                if (j == firstMinIndex)
                    grid[i][j] += secondMinNum;
                else
                    grid[i][j] += firstMinNum;
        }

        return Arrays.stream(grid[n - 1]).min().getAsInt();
    }

    private Pair<Integer, Integer>[] getTwoMinNumAndIndexes(int[] arr) {
        List<Pair<Integer, Integer>> numAndIndexes = new ArrayList<>();
        for (int i = 0; i < arr.length; ++i)
            numAndIndexes.add(new Pair<>(arr[i], i));
        Collections.sort(numAndIndexes, Comparator.comparingInt(Pair::getKey));
        return new Pair[] {numAndIndexes.get(0), numAndIndexes.get(1)};
    }
}
```

298. 129.java

Path: LeetCode-main/solutions/129. Sum Root to Leaf Numbers/129.java

```
class Solution {
    public int sumNumbers(TreeNode root) {
        dfs(root, 0);
        return ans;
    }

    private int ans = 0;

    private void dfs(TreeNode root, int path) {
        if (root == null)
            return;
        if (root.left == null && root.right == null) {
            ans += path * 10 + root.val;
            return;
        }

        dfs(root.left, path * 10 + root.val);
        dfs(root.right, path * 10 + root.val);
    }
}
```

299. 1290.java

Path: LeetCode-main/solutions/1290. Convert Binary Number in a Linked List to Integer/1290.java

```
class Solution {  
    public int getDecimalValue(ListNode head) {  
        int ans = 0;  
  
        for (; head != null; head = head.next)  
            ans = ans * 2 + head.val;  
  
        return ans;  
    }  
}
```

300. 1291.java

Path: LeetCode-main/solutions/1291. Sequential Digits/1291.java

```
class Solution {
    public List<Integer> sequentialDigits(int low, int high) {
        List<Integer> ans = new ArrayList<>();
        Queue<Integer> q = new ArrayDeque<>(List.of(1, 2, 3, 4, 5, 6, 7, 8, 9));

        while (!q.isEmpty()) {
            final int num = q.poll();
            if (num > high)
                return ans;
            if (low <= num && num <= high)
                ans.add(num);
            final int lastDigit = num % 10;
            if (lastDigit < 9)
                q.offer(num * 10 + lastDigit + 1);
        }

        return ans;
    }
}
```

301. 1292.java

Path: LeetCode-main/solutions/1292. Maximum Side Length of a Square with Sum Less than or Equal to Threshold/1292.java

```
class Solution {
    public int maxSideLength(int[][] mat, int threshold) {
        final int m = mat.length;
        final int n = mat[0].length;
        int ans = 0;
        int[][] prefix = new int[m + 1][n + 1];

        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j)
                prefix[i + 1][j + 1] = mat[i][j] + prefix[i][j + 1] + prefix[i + 1][j] - prefix[i][j];

        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j)
                for (int length = ans; length < Math.min(m - i, n - j); ++length) {
                    if (squareSum(prefix, i, j, i + length, j + length) > threshold)
                        break;
                    ans = Math.max(ans, length + 1);
                }

        return ans;
    }

    private int squareSum(int[][] prefix, int r1, int c1, int r2, int c2) {
        return prefix[r2 + 1][c2 + 1] - prefix[r1][c2 + 1] - prefix[r2 + 1][c1] + prefix[r1][c1];
    }
}
```

302. 1293.java

Path: LeetCode-main/solutions/1293. Shortest Path in a Grid with Obstacles Elimination/1293.java

```
class Solution {
    public int shortestPath(int[][] grid, int k) {
        record T(int i, int j, int eliminate) {}
        final int m = grid.length;
        final int n = grid[0].length;
        if (m == 1 && n == 1)
            return 0;

        final int[][] DIRS = {{0, 1}, {1, 0}, {0, -1}, {-1, 0}};
        Queue<T> q = new ArrayDeque<>(List.of(new T(0, 0, k)));
        boolean[][][] seen = new boolean[m][n][k + 1];
        seen[0][0][k] = true;

        for (int step = 1; !q.isEmpty(); ++step)
            for (int sz = q.size(); sz > 0; --sz) {
                final int i = q.peek().i;
                final int j = q.peek().j;
                final int eliminate = q.poll().eliminate;
                for (int l = 0; l < 4; ++l) {
                    final int x = i + DIRS[l][0];
                    final int y = j + DIRS[l][1];
                    if (x < 0 || x == m || y < 0 || y == n)
                        continue;
                    if (x == m - 1 && y == n - 1)
                        return step;
                    if (grid[x][y] == 1 && eliminate == 0)
                        continue;
                    final int newEliminate = eliminate - grid[x][y];
                    if (seen[x][y][newEliminate])
                        continue;
                    q.offer(new T(x, y, newEliminate));
                    seen[x][y][newEliminate] = true;
                }
            }
        return -1;
    }
}
```


303. 1295.java

Path: LeetCode-main/solutions/1295. Find Numbers with Even Number of Digits/1295.java

```
class Solution {
    public int findNumbers(int[] nums) {
        int ans = 0;

        for (int num : nums)
            if (9 < num && num < 100 || 999 < num && num < 10000 || num == 100000)
                ++ans;

        return ans;
    }
}
```

304. 1296.java

Path: LeetCode-main/solutions/1296. Divide Array in Sets of K Consecutive Numbers/1296.java

```
class Solution {
    public boolean isPossibleDivide(int[] nums, int k) {
        TreeMap<Integer, Integer> count = new TreeMap<>();

        for (final int num : nums)
            count.merge(num, 1, Integer::sum);

        for (final int start : count.keySet()) {
            final int value = count.getDefault(start, 0);
            if (value > 0)
                for (int i = start; i < start + k; ++i)
                    if (count.merge(i, -value, Integer::sum) < 0)
                        return false;
        }

        return true;
    }
}
```

305. 1297.java

Path: LeetCode-main/solutions/1297. Maximum Number of Occurrences of a Substring/1297.java

```
class Solution {
    public int maxFreq(String s, int maxLetters, int minSize, int maxSize) {
        // Greedily consider strings with `minSize`, so ignore `maxSize`.
        int ans = 0;
        int letters = 0;
        int[] count = new int[26];
        Map<String, Integer> substringCount = new HashMap<>();

        for (int l = 0, r = 0; r < s.length(); ++r) {
            if (++count[s.charAt(r) - 'a'] == 1)
                ++letters;
            while (letters > maxLetters || r - l + 1 > minSize)
                if (--count[s.charAt(l++) - 'a'] == 0)
                    --letters;
            if (r - l + 1 == minSize)
                ans = Math.max(ans, substringCount.merge(s.substring(l, l + minSize), 1, Integer::sum));
        }

        return ans;
    }
}
```

306. 1298.java

Path: LeetCode-main/solutions/1298. Maximum Candies You Can Get from Boxes/1298.java

```
class Solution {
    public int maxCandies(int[] status, int[] candies, int[][] keys, int[][] containedBoxes,
        int[] initialBoxes) {
        int ans = 0;
        Queue<Integer> q = new ArrayDeque<>();
        boolean[] reachedClosedBoxes = new boolean[status.length];

        pushBoxesIfPossible(initialBoxes, status, q, reachedClosedBoxes);

        while (!q.isEmpty()) {
            final int currBox = q.poll();

            // Add the candies.
            ans += candies[currBox];

            // Push `reachedClosedBoxes` by `key` obtained in this turn and change
            // their statuses.
            for (final int key : keys[currBox]) {
                if (status[key] == 0 && reachedClosedBoxes[key])
                    q.offer(key);
                status[key] = 1; // boxes[key] is now open.
            }

            // Push the boxes contained in `currBox`.
            pushBoxesIfPossible(containedBoxes[currBox], status, q, reachedClosedBoxes);
        }

        return ans;
    }

    private void pushBoxesIfPossible(int[] boxes, int[] status, Queue<Integer> q,
        boolean[] reachedClosedBoxes) {
        for (final int box : boxes)
            if (status[box] == 1)
                q.offer(box);
            else
                reachedClosedBoxes[box] = true;
    }
}
```

307. 1299.java

Path: LeetCode-main/solutions/1299. Replace Elements with Greatest Element on Right Side/1299.java

```
class Solution {
    public int[] replaceElements(int[] arr) {
        int maxOfRight = -1;
        for (int i = arr.length - 1; i >= 0; --i) {
            int a = arr[i];
            arr[i] = maxOfRight;
            maxOfRight = Math.max(maxOfRight, a);
        }
        return arr;
    }
}
```

308. 13.java

Path: LeetCode-main/solutions/13. Roman to Integer/13.java

```
class Solution {
    public int romanToInt(String s) {
        int ans = 0;
        int[] roman = new int[128];

        roman['I'] = 1;
        roman['V'] = 5;
        roman['X'] = 10;
        roman['L'] = 50;
        roman['C'] = 100;
        roman['D'] = 500;
        roman['M'] = 1000;

        for (int i = 0; i + 1 < s.length(); ++i)
            if (roman[s.charAt(i)] < roman[s.charAt(i + 1)])
                ans -= roman[s.charAt(i)];
            else
                ans += roman[s.charAt(i)];

        return ans + roman[s.charAt(s.length() - 1)];
    }
}
```

309. 130-2.java

Path: LeetCode-main/solutions/130. Surrounded Regions/130-2.java

```
class Solution {
    public void solve(char[][] board) {
        if (board.length == 0)
            return;

        final int m = board.length;
        final int n = board[0].length;

        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j)
                if (i * j == 0 || i == m - 1 || j == n - 1)
                    dfs(board, i, j);

        for (char[] row : board)
            for (int i = 0; i < row.length; ++i)
                if (row[i] == '*')
                    row[i] = 'O';
                else if (row[i] == 'O')
                    row[i] = 'X';
    }

    // Marks the grids with 'O' that stretch from the four sides to '*'.
    private void dfs(char[][] board, int i, int j) {
        if (i < 0 || i == board.length || j < 0 || j == board[0].length)
            return;
        if (board[i][j] != 'O')
            return;
        board[i][j] = '*';
        dfs(board, i + 1, j);
        dfs(board, i - 1, j);
        dfs(board, i, j + 1);
        dfs(board, i, j - 1);
    }
}
```

310. 130.java

Path: LeetCode-main/solutions/130. Surrounded Regions/130.java

```
class Solution {
    public void solve(char[][] board) {
        if (board.length == 0)
            return;

        final int[][] DIRS = {{0, 1}, {1, 0}, {0, -1}, {-1, 0}};
        final int m = board.length;
        final int n = board[0].length;
        Queue<Pair<Integer, Integer>> q = new ArrayDeque<>();

        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j)
                if (i * j == 0 || i == m - 1 || j == n - 1)
                    if (board[i][j] == 'O') {
                        q.offer(new Pair<>(i, j));
                        board[i][j] = '*';
                    }

        // Mark the grids that stretch from the four sides with '*'.
        while (!q.isEmpty()) {
            final int i = q.peek().getKey();
            final int j = q.poll().getValue();
            for (int[] dir : DIRS) {
                final int x = i + dir[0];
                final int y = j + dir[1];
                if (x < 0 || x == m || y < 0 || y == n)
                    continue;
                if (board[x][y] != 'O')
                    continue;
                q.offer(new Pair<>(x, y));
                board[x][y] = '*';
            }
        }

        for (char[] row : board)
            for (int i = 0; i < row.length; ++i)
                if (row[i] == '*')
                    row[i] = 'O';
                else if (row[i] == 'O')
                    row[i] = 'X';
        }
    }
}
```


311. 1300.java

Path: LeetCode-main/solutions/1300. Sum of Mutated Array Closest to Target/1300.java

```
class Solution {
    public int findBestValue(int[] arr, int target) {
        final int n = arr.length;
        final double err = 1e-9;

        int prefix = 0;

        Arrays.sort(arr);

        for (int i = 0; i < n; ++i) {
            int ans = (int) Math.round(((float) target - prefix - err) / (n - i));
            if (ans <= arr[i])
                return ans;
            prefix += arr[i];
        }

        return arr[n - 1];
    }
}
```

312. 1301.java

Path: LeetCode-main/solutions/1301. Number of Paths with Max Score/1301.java

```
class Solution {
    public int[] pathsWithMaxScore(List<String> board) {
        final int MOD = 1_000_000_007;
        final int[][] DIRS = {{0, 1}, {1, 0}, {1, 1}};
        final int n = board.size();
        // dp[i][j] := the maximum sum from (n - 1, n - 1) to (i, j)
        int[][] dp = new int[n + 1][n + 1];
        Arrays.stream(dp).forEach(A -> Arrays.fill(A, -1));
        // count[i][j] := the number of paths to get dp[i][j] from (n - 1, n - 1) to (i, j)
        int[][] count = new int[n + 1][n + 1];

        dp[0][0] = 0;
        dp[n - 1][n - 1] = 0;
        count[n - 1][n - 1] = 1;

        for (int i = n - 1; i >= 0; --i)
            for (int j = n - 1; j >= 0; --j) {
                if (board.get(i).charAt(j) == 'S' || board.get(i).charAt(j) == 'X')
                    continue;
                for (int[] dir : DIRS) {
                    final int x = i + dir[0];
                    final int y = j + dir[1];
                    if (dp[i][j] < dp[x][y]) {
                        dp[i][j] = dp[x][y];
                        count[i][j] = count[x][y];
                    } else if (dp[i][j] == dp[x][y]) {
                        count[i][j] += count[x][y];
                        count[i][j] %= MOD;
                    }
                }
                // If there's path(s) from 'S' to (i, j) and the cell is not 'E'.
                if (dp[i][j] != -1 && board.get(i).charAt(j) != 'E') {
                    dp[i][j] += board.get(i).charAt(j) - '0';
                    dp[i][j] %= MOD;
                }
            }

        return new int[] {dp[0][0], count[0][0]};
    }
}
```

313. 1302.java

Path: LeetCode-main/solutions/1302. Deepest Leaves Sum/1302.java

```
class Solution {
    public int deepestLeavesSum(TreeNode root) {
        int ans = 0;
        Queue<TreeNode> q = new ArrayDeque<>(List.of(root));

        while (!q.isEmpty()) {
            ans = 0;
            for (int sz = q.size(); sz > 0; --sz) {
                TreeNode node = q.poll();
                ans += node.val;
                if (node.left != null)
                    q.offer(node.left);
                if (node.right != null)
                    q.offer(node.right);
            }
        }
        return ans;
    }
}
```

314. 1304.java

Path: LeetCode-main/solutions/1304. Find N Unique Integers Sum up to Zero/1304.java

```
class Solution {  
    public int[] sumZero(int n) {  
        int[] ans = new int[n];  
  
        for (int i = 0; i < n; ++i)  
            ans[i] = i * 2 - n + 1;  
  
        return ans;  
    }  
}
```

315. 1305.java

Path: LeetCode-main/solutions/1305. All Elements in Two Binary Search Trees/1305.java

```
class BSTIterator {
    public BSTIterator(TreeNode root) {
        pushLeftsUntilNull(root);
    }

    public int peek() {
        return stack.peek().val;
    }

    public void next() {
        pushLeftsUntilNull(stack.pop().right);
    }

    public boolean hasNext() {
        return !stack.isEmpty();
    }

    private Deque<TreeNode> stack = new ArrayDeque<>();

    private void pushLeftsUntilNull(TreeNode node) {
        while (node != null) {
            stack.push(node);
            node = node.left;
        }
    }
}

class Solution {
    public List<Integer> getAllElements(TreeNode root1, TreeNode root2) {
        List<Integer> ans = new ArrayList<>();
        BSTIterator bstIterator1 = new BSTIterator(root1);
        BSTIterator bstIterator2 = new BSTIterator(root2);

        while (bstIterator1.hasNext() && bstIterator2.hasNext())
            if (bstIterator1.peek() < bstIterator2.peek()) {
                ans.add(bstIterator1.peek());
                bstIterator1.next();
            } else {
                ans.add(bstIterator2.peek());
                bstIterator2.next();
            }

        while (bstIterator1.hasNext()) {
            ans.add(bstIterator1.peek());
            bstIterator1.next();
        }

        while (bstIterator2.hasNext()) {
            ans.add(bstIterator2.peek());
            bstIterator2.next();
        }

        return ans;
    }
}
```

316. 1306-2.java

Path: LeetCode-main/solutions/1306. Jump Game III/1306-2.java

```
class Solution {
    public boolean canReach(int[] arr, int start) {
        return canReach(arr, start, new boolean[arr.length]);
    }

    private boolean canReach(int[] arr, int node, boolean[] seen) {
        if (node < 0 || node >= arr.length)
            return false;
        if (seen[node])
            return false;
        if (arr[node] == 0)
            return true;
        seen[node] = true;
        return canReach(arr, node + arr[node], seen) || canReach(arr, node - arr[node], seen);
    }
}
```

317. 1306.java

Path: LeetCode-main/solutions/1306. Jump Game III/1306.java

```
class Solution {
    public boolean canReach(int[] arr, int start) {
        final int n = arr.length;
        Queue<Integer> q = new ArrayDeque<>(List.of(start));
        boolean[] seen = new boolean[n];

        while (!q.isEmpty()) {
            final int node = q.poll();
            if (arr[node] == 0)
                return true;
            if (seen[node])
                continue;
            // Check the available next steps.
            if (node - arr[node] >= 0)
                q.offer(node - arr[node]);
            if (node + arr[node] < n)
                q.offer(node + arr[node]);
            seen[node] = true;
        }

        return false;
    }
}
```

318. 1307.java

Path: LeetCode-main/solutions/1307. Verbal Arithmetic Puzzle/1307.java

```
class Solution {
    public boolean isSolvable(String[] words, String result) {
        rows = words.length + 1;
        for (final String word : words)
            cols = Math.max(cols, word.length());
        cols = Math.max(cols, result.length());
        return dfs(words, result, 0, 0, 0);
    }

    private Map<Character, Integer> letterToDigit = new HashMap<>();
    private boolean[] usedDigit = new boolean[10];
    private int rows = 0;
    private int cols = 0;

    private boolean dfs(String[] words, String result, int row, int col, int sum) {
        if (col == cols)
            return sum == 0;
        if (row == rows)
            return sum % 10 == 0 && dfs(words, result, 0, col + 1, sum / 10);

        String word = row == rows - 1 ? result : words[row];
        if (col >= word.length())
            return dfs(words, result, row + 1, col, sum);

        char letter = word.charAt(word.length() - col - 1);
        int sign = row == rows - 1 ? -1 : 1;

        if (letterToDigit.containsKey(letter) &&
            (letterToDigit.get(letter) > 0 || col < word.length() - 1))
            return dfs(words, result, row + 1, col, sum + sign * letterToDigit.get(letter));

        for (int digit = 0; digit < 10; ++digit)
            if (!usedDigit[digit] && (digit > 0 || col < word.length() - 1)) {
                letterToDigit.put(letter, digit);
                usedDigit[digit] = true;
                if (dfs(words, result, row + 1, col, sum + sign * letterToDigit.get(letter)))
                    return true;
                usedDigit[digit] = false;
                letterToDigit.remove(letter);
            }

        return false;
    }
}
```


319. 1309.java

Path: LeetCode-main/solutions/1309. Decrypt String from Alphabet to Integer Mapping/1309.java

```
class Solution {
    public String freqAlphabets(String s) {
        String ans = "";

        for (int i = 0; i < s.length(); i++) {
            if (i + 2 < s.length() && s.charAt(i + 2) == '#') {
                ans += (char) (Integer.valueOf(s.substring(i, i + 2)) + 'a' - 1);
                i += 3;
            } else {
                ans += (char) ((s.charAt(i) - '0') + 'a' - 1);
                i += 1;
            }
        }

        return ans;
    }
}
```

320. 131.java

Path: LeetCode-main/solutions/131. Palindrome Partitioning/131.java

```
class Solution {
    public List<List<String>> partition(String s) {
        List<List<String>> ans = new ArrayList<>();
        dfs(s, 0, new ArrayList<>(), ans);
        return ans;
    }

    private void dfs(final String s, int start, List<String> path, List<List<String>> ans) {
        if (start == s.length()) {
            ans.add(new ArrayList<>(path));
            return;
        }

        for (int i = start; i < s.length(); ++i)
            if (isPalindrome(s, start, i)) {
                path.add(s.substring(start, i + 1));
                dfs(s, i + 1, path, ans);
                path.remove(path.size() - 1);
            }
    }

    private boolean isPalindrome(final String s, int l, int r) {
        while (l < r)
            if (s.charAt(l++) != s.charAt(r--))
                return false;
        return true;
    }
}
```

321. 1310.java

Path: LeetCode-main/solutions/1310. XOR Queries of a Subarray/1310.java

```
class Solution {
    public int[] xorQueries(int[] arr, int[][] queries) {
        int[] ans = new int[queries.length];
        int[] xors = new int[arr.length + 1];

        for (int i = 0; i < arr.length; ++i)
            xors[i + 1] = xors[i] ^ arr[i];

        int i = 0;
        for (int[] query : queries) {
            final int left = query[0];
            final int right = query[1];
            ans[i++] = xors[left] ^ xors[right + 1];
        }

        return ans;
    }
}
```

322. 1311.java

Path: LeetCode-main/solutions/1311. Get Watched Videos by Your Friends/1311.java

```
class Solution {
    public List<String> watchedVideosByFriends(List<List<String>> watchedVideos, int[][] friends,
                                              int id, int level) {
        Queue<Integer> q = new ArrayDeque<>(List.of(id));
        boolean[] seen = new boolean[friends.length];
        seen[id] = true;
        Map<String, Integer> count = new HashMap<>();

        for (int i = 0; i < level; ++i)
            for (int sz = q.size(); sz > 0; --sz) {
                for (final int friend : friends[q.peek()])
                    if (!seen[friend]) {
                        seen[friend] = true;
                        q.offer(friend);
                    }
                q.poll();
            }

        for (final int friend : q)
            for (final String video : watchedVideos.get(friend))
                count.merge(video, 1, Integer::sum);

        List<String> ans = new ArrayList<>(count.keySet());
        ans.sort((a, b)
            -> count.get(a).equals(count.get(b)) ? a.compareTo(b)
            : count.get(a).compareTo(count.get(b)));

        return ans;
    }
}
```

323. 1312.java

Path: LeetCode-main/solutions/1312. Minimum Insertion Steps to Make a String Palindrome/1312.java

```
class Solution {
    public int minInsertions(String s) {
        return s.length() - longestPalindromeSubseq(s);
    }

    // Same as 516. Longest Palindromic Subsequence
    public int longestPalindromeSubseq(String s) {
        final int n = s.length();
        // dp[i][j] := the length of LPS(s[i..j])
        int[][] dp = new int[n][n];

        for (int i = 0; i < n; ++i)
            dp[i][i] = 1;

        for (int d = 1; d < n; ++d)
            for (int i = 0; i + d < n; ++i) {
                final int j = i + d;
                if (s.charAt(i) == s.charAt(j))
                    dp[i][j] = 2 + dp[i + 1][j - 1];
                else
                    dp[i][j] = Math.max(dp[i + 1][j], dp[i][j - 1]);
            }

        return dp[0][n - 1];
    }
}
```

324. 1313.java

Path: LeetCode-main/solutions/1313. Decompress Run-Length Encoded List/1313.java

```
class Solution {
    public int[] decompressRLElist(int[] nums) {
        List<Integer> ans = new ArrayList<>();

        for (int i = 0; i < nums.length; i += 2)
            for (int freq = 0; freq < nums[i]; ++freq)
                ans.add(nums[i + 1]);

        return ans.stream().mapToInt(Integer::intValue).toArray();
    }
}
```

325. 1314.java

Path: LeetCode-main/solutions/1314. Matrix Block Sum/1314.java

```
class Solution {
    public int[][] matrixBlockSum(int[][] mat, int k) {
        final int m = mat.length;
        final int n = mat[0].length;
        int[][] ans = new int[m][n];
        int[][] prefix = new int[m + 1][n + 1];

        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j)
                prefix[i + 1][j + 1] = mat[i][j] + prefix[i][j + 1] + prefix[i + 1][j] - prefix[i][j];

        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j) {
                final int r1 = Math.max(0, i - k) + 1;
                final int c1 = Math.max(0, j - k) + 1;
                final int r2 = Math.min(m - 1, i + k) + 1;
                final int c2 = Math.min(n - 1, j + k) + 1;
                ans[i][j] =
                    prefix[r2][c2] - prefix[r2][c1 - 1] - prefix[r1 - 1][c2] + prefix[r1 - 1][c1 - 1];
            }

        return ans;
    }
}
```

326. 1315.java

Path: LeetCode-main/solutions/1315. Sum of Nodes with Even-Valued Grandparent/1315.java

```
class Solution {
    public int sumEvenGrandparent(TreeNode root) {
        return dfs(root, 1, 1); // The parent and the grandparent are odd at first.
    }

    private int dfs(TreeNode root, int p, int gp) {
        if (root == null)
            return 0;
        return (gp % 2 == 0 ? root.val : 0) + //
            dfs(root.left, root.val, p) +    //
            dfs(root.right, root.val, p);
    }
}
```


327. 1316.java

Path: LeetCode-main/solutions/1316. Distinct Echo Substrings/1316.java

```
class Solution {
    public int distinctEchoSubstrings(String text) {
        Set<String> seen = new HashSet<>();

        for (int k = 1; k <= text.length() / 2; ++k) { // the target length
            int same = 0;
            for (int l = 0, r = k; r < text.length(); ++l, ++r) {
                if (text.charAt(l) == text.charAt(r))
                    ++same;
                else
                    same = 0;
                if (same == k) {
                    seen.add(text.substring(l - k + 1, l + 1));
                    // Move the window thus leaving a letter behind, so we need to
                    // decrease the counter.
                    --same;
                }
            }
        }

        return seen.size();
    }
}
```

328. 1317.java

Path: LeetCode-main/solutions/1317. Convert Integer to the Sum of Two No-Zero Integers/1317.java

```
class Solution {
    public int[] getNoZeroIntegers(int n) {
        for (int A = 1; A < n; ++A) {
            int B = n - A;
            if (!String.valueOf(A).contains("0") && !String.valueOf(B).contains("0"))
                return new int[] {A, B};
        }

        throw new IllegalArgumentException();
    }
}
```

329. 1318.java

Path: LeetCode-main/solutions/1318. Minimum Flips to Make a OR b Equal to c/1318.java

```
class Solution {
    public int minFlips(int a, int b, int c) {
        final int MAX_BIT = 30;
        int ans = 0;

        for (int i = 0; i < MAX_BIT; ++i)
            if ((c >> i & 1) == 1)
                ans += ((a >> i & 1) == 0 && (b >> i & 1) == 0) ? 1 : 0;
            else // (c >> i & 1) == 0
                ans += ((a >> i & 1) == 1 ? 1 : 0) + ((b >> i & 1) == 1 ? 1 : 0);

        return ans;
    }
}
```

330. 1319.java

Path: LeetCode-main/solutions/1319. Number of Operations to Make Network Connected/1319.java

```
class Solution {
    public int makeConnected(int n, int[][] connections) {
        // To connect n nodes, we need at least n - 1 edges
        if (connections.length < n - 1)
            return -1;

        int numOfConnected = 0;
        List<Integer>[] graph = new List[n];
        Set<Integer> seen = new HashSet<>();
        Arrays.setAll(graph, i -> new ArrayList<>());

        for (int[] connection : connections) {
            final int u = connection[0];
            final int v = connection[1];
            graph[u].add(v);
            graph[v].add(u);
        }

        for (int i = 0; i < n; ++i)
            if (seen.add(i)) {
                dfs(graph, i, seen);
                ++numOfConnected;
            }

        return numOfConnected - 1;
    }

    private void dfs(List<Integer>[] graph, int u, Set<Integer> seen) {
        for (final int v : graph[u])
            if (seen.add(v))
                dfs(graph, v, seen);
    }
}
```

331. 132.java

Path: LeetCode-main/solutions/132. Palindrome Partitioning II/132.java

```
class Solution {
    public int minCut(String s) {
        final int n = s.length();
        // isPalindrome[i][j] := true if s[i..j] is a palindrome
        boolean[][] isPalindrome = new boolean[n][n];
        for (boolean[] row : isPalindrome)
            Arrays.fill(row, true);
        // dp[i] := the minimum cuts needed for a palindrome partitioning of s[0..i]
        int[] dp = new int[n];
        Arrays.fill(dp, n);

        for (int l = 2; l <= n; ++l)
            for (int i = 0, j = l - 1; j < n; ++i, ++j)
                isPalindrome[i][j] = s.charAt(i) == s.charAt(j) && isPalindrome[i + 1][j - 1];

        for (int i = 0; i < n; ++i) {
            if (isPalindrome[0][i]) {
                dp[i] = 0;
                continue;
            }

            // Try all the possible partitions.
            for (int j = 0; j < i; ++j)
                if (isPalindrome[j + 1][i])
                    dp[i] = Math.min(dp[i], dp[j] + 1);
        }

        return dp[n - 1];
    }
}
```

332. 1320.java

Path: LeetCode-main/solutions/1320. Minimum Distance to Type a Word Using Two Fingers/1320.java

```
class Solution {
    public int minimumDistance(String word) {
        Integer[][][] mem = new Integer[27][27][word.length()];
        return minimumDistance(word, 26, 26, 0, mem);
    }

    // Returns the minimum distance to type `word`, where the left finger is on
    // the i-th letter, the right finger is on the j-th letter, and word[0..k)
    // have been written.
    private int minimumDistance(final String word, int i, int j, int k, Integer[][][] mem) {
        if (k == word.length())
            return 0;
        if (mem[i][j][k] != null)
            return mem[i][j][k];
        final int c = word.charAt(k) - 'A';
        final int moveLeft = dist(i, c) + minimumDistance(word, c, j, k + 1, mem);
        final int moveRight = dist(j, c) + minimumDistance(word, i, c, k + 1, mem);
        return mem[i][j][k] = Math.min(moveLeft, moveRight);
    }

    private int dist(int a, int b) {
        if (a == 26) // the first hovering state
            return 0;
        final int x1 = a / 6;
        final int y1 = a % 6;
        final int x2 = b / 6;
        final int y2 = b % 6;
        return Math.abs(x1 - x2) + Math.abs(y1 - y2);
    }
}
```

333. 1323.java

Path: LeetCode-main/solutions/1323. Maximum 69 Number/1323.java

```
class Solution {  
    public int maximum69Number(int num) {  
        char[] ans = String.valueOf(num).toCharArray();  
  
        for (int i = 0; i < ans.length; ++i)  
            if (ans[i] == '6') {  
                ans[i] = '9';  
                break;  
            }  
  
        return Integer.valueOf(String.valueOf(ans));  
    }  
}
```

334. 1324.java

Path: LeetCode-main/solutions/1324. Print Words Vertically/1324.java

```
class Solution {
    public List<String> printVertically(String s) {
        List<String> ans = new ArrayList<>();
        String[] words = s.split(" ");
        int maxLength = 0;

        for (final String word : words)
            maxLength = Math.max(maxLength, word.length());

        for (int i = 0; i < maxLength; ++i) {
            StringBuilder sb = new StringBuilder();
            for (final String word : words)
                sb.append(i < word.length() ? word.charAt(i) : ' ');
            while (sb.charAt(sb.length() - 1) == ' ')
                sb.deleteCharAt(sb.length() - 1);
            ans.add(sb.toString());
        }

        return ans;
    }
}
```


335. 1325.java

Path: LeetCode-main/solutions/1325. Delete Leaves With a Given Value/1325.java

```
class Solution {
    public TreeNode removeLeafNodes(TreeNode root, int target) {
        if (root == null)
            return null;
        root.left = removeLeafNodes(root.left, target);
        root.right = removeLeafNodes(root.right, target);
        return isLeaf(root) && root.val == target ? null : root;
    }

    private boolean isLeaf(TreeNode root) {
        return root.left == null && root.right == null;
    }
}
```

336. 1326.java

Path: LeetCode-main/solutions/1326. Minimum Number of Taps to Open to Water a Garden/1326.java

```
class Solution {
    public int minTaps(int n, int[] ranges) {
        int[] nums = new int[n + 1];

        for (int i = 0; i <= n; ++i) {
            int l = Math.max(0, i - ranges[i]);
            int r = Math.min(n, i + ranges[i]);
            nums[l] = Math.max(nums[l], r - l);
        }

        int ans = 0;
        int end = 0;
        int farthest = 0;

        for (int i = 0; i < n; i++) {
            farthest = Math.max(farthest, i + nums[i]);
            if (i == end) {
                ++ans;
                end = farthest;
            }
        }

        return end == n ? ans : -1;
    }
}
```

337. 1328.java

Path: LeetCode-main/solutions/1328. Break a Palindrome/1328.java

```
class Solution {
    public String breakPalindrome(String palindrome) {
        if (palindrome.length() == 1)
            return "";

        StringBuilder sb = new StringBuilder(palindrome);

        for (int i = 0; i < palindrome.length() / 2; ++i)
            if (palindrome.charAt(i) != 'a') {
                sb.setCharAt(i, 'a');
                return sb.toString();
            }

        sb.setCharAt(sb.length() - 1, 'b');
        return sb.toString();
    }
}
```

338. 1329.java

Path: LeetCode-main/solutions/1329. Sort the Matrix Diagonally/1329.java

```
class Solution {
    public int[][] diagonalSort(int[][] mat) {
        final int m = mat.length;
        final int n = mat[0].length;

        Map<Integer, Queue<Integer>> count = new HashMap<>();

        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j)
                count.computeIfAbsent(i - j, k -> new PriorityQueue<>()).add(mat[i][j]);

        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j)
                mat[i][j] = count.get(i - j).poll();

        return mat;
    }
}
```

339. 133-2.java

Path: LeetCode-main/solutions/133. Clone Graph/133-2.java

```
class Solution {
    public Node cloneGraph(Node node) {
        if (node == null)
            return null;
        if (map.containsKey(node))
            return map.get(node);

        Node newNode = new Node(node.val);
        map.put(node, newNode);

        for (Node neighbor : node.neighbors)
            newNode.neighbors.add(cloneGraph(neighbor));

        return newNode;
    }

    private Map<Node, Node> map = new HashMap<>();
}
```

340. 133.java

Path: LeetCode-main/solutions/133. Clone Graph/133.java

```
class Solution {
    public Node cloneGraph(Node node) {
        if (node == null)
            return null;

        Queue<Node> q = new ArrayDeque<>(List.of(node));
        Map<Node, Node> map = new HashMap<>();
        map.put(node, new Node(node.val));

        while (!q.isEmpty()) {
            Node u = q.poll();
            for (Node v : u.neighbors) {
                if (!map.containsKey(v)) {
                    map.put(v, new Node(v.val));
                    q.offer(v);
                }
                map.get(u).neighbors.add(map.get(v));
            }
        }

        return map.get(node);
    }
}
```

341. 1330.java

Path: LeetCode-main/solutions/1330. Reverse Subarray To Maximize Array Value/1330.java

```
class Solution {
    public int maxValueAfterReverse(int[] nums) {
        int total = 0;
        int mn = Integer.MAX_VALUE;
        int mx = Integer.MIN_VALUE;

        for (int i = 0; i + 1 < nums.length; ++i) {
            final int a = nums[i];
            final int b = nums[i + 1];
            total += Math.abs(a - b);
            mn = Math.min(mn, Math.max(a, b));
            mx = Math.max(mx, Math.min(a, b));
        }
        int diff = Math.max(0, (mx - mn) * 2);

        for (int i = 0; i + 1 < nums.length; ++i) {
            final int a = nums[i];
            final int b = nums[i + 1];
            final int headDiff = -Math.abs(a - b) + Math.abs(nums[0] - b);
            final int tailDiff = -Math.abs(a - b) + Math.abs(nums[nums.length - 1] - a);
            diff = Math.max(diff, Math.max(headDiff, tailDiff));
        }

        return total + diff;
    }
}
```

342. 1331.java

Path: LeetCode-main/solutions/1331. Rank Transform of an Array/1331.java

```
class Solution {
    public int[] arrayRankTransform(int[] arr) {
        int[] sortedArr = arr.clone();
        Map<Integer, Integer> rank = new HashMap<>();

        Arrays.sort(sortedArr);

        for (final int a : sortedArr)
            rank.putIfAbsent(a, rank.size() + 1);

        for (int i = 0; i < arr.length; ++i)
            arr[i] = rank.get(arr[i]);

        return arr;
    }
}
```


343. 1332.java

Path: LeetCode-main/solutions/1332. Remove Palindromic Subsequences/1332.java

```
class Solution {  
    public int removePalindromeSub(String s) {  
        return s.equals(new StringBuilder(s).reverse().toString()) ? 1 : 2;  
    }  
}
```

344. 1333.java

Path: LeetCode-main/solutions/1333. Filter Restaurants by Vegan-Friendly, Price and Distance/1333.java

```
class Solution {
    public List<Integer> filterRestaurants(int[][] restaurants, int veganFriendly, int maxPrice,
                                           int maxDistance) {
        return Arrays.stream(restaurants)
            .filter(r -> r[2] >= veganFriendly && r[3] <= maxPrice && r[4] <= maxDistance)
            .sorted(Comparator.comparingInt((int[] r) -> - r[1]).thenComparingInt((int[] r) -> - r[0]))
            .map(r -> r[0])
            .collect(Collectors.toList());
    }
}
```

345. 1334.java

Path: LeetCode-main/solutions/1334. Find the City With the Smallest Number of Neighbors at a Threshold Distance/1334.java

```
class Solution {
    public int findTheCity(int n, int[][] edges, int distanceThreshold) {
        int ans = -1;
        int minCitiesCount = n;
        int[][] dist = floydWarshall(n, edges, distanceThreshold);

        for (int i = 0; i < n; ++i) {
            int citiesCount = 0;
            for (int j = 0; j < n; ++j)
                if (dist[i][j] <= distanceThreshold)
                    ++citiesCount;
            if (citiesCount <= minCitiesCount) {
                ans = i;
                minCitiesCount = citiesCount;
            }
        }

        return ans;
    }

    private int[][] floydWarshall(int n, int[][] edges, int distanceThreshold) {
        int[][] dist = new int[n][n];
        Arrays.stream(dist).forEach(A -> Arrays.fill(A, distanceThreshold + 1));

        for (int i = 0; i < n; ++i)
            dist[i][i] = 0;

        for (int[] edge : edges) {
            final int u = edge[0];
            final int v = edge[1];
            final int w = edge[2];
            dist[u][v] = w;
            dist[v][u] = w;
        }

        for (int k = 0; k < n; ++k)
            for (int i = 0; i < n; ++i)
                for (int j = 0; j < n; ++j)
                    dist[i][j] = Math.min(dist[i][j], dist[i][k] + dist[k][j]);

        return dist;
    }
}
```

346. 1335.java

Path: LeetCode-main/solutions/1335. Minimum Difficulty of a Job Schedule/1335.java

```
class Solution {
    public int minDifficulty(int[] jobDifficulty, int d) {
        final int n = jobDifficulty.length;
        if (n < d)
            return -1;

        // dp[i][k] := the minimum difficulty to schedule the first i jobs in k days
        int[][] dp = new int[n + 1][d + 1];
        Arrays.stream(dp).forEach(A -> Arrays.fill(A, Integer.MAX_VALUE / 2));
        dp[0][0] = 0;

        for (int i = 1; i <= n; ++i)
            for (int k = 1; k <= d; ++k) {
                // max(job[j + 1..i])
                int maxDifficulty = 0;
                for (int j = i - 1; j >= k - 1; --j) {
                    maxDifficulty = Math.max(maxDifficulty, jobDifficulty[j]); // 1-based
                    dp[i][k] = Math.min(dp[i][k], dp[j][k - 1] + maxDifficulty); // 0-based
                }
            }

        return dp[n][d];
    }
}
```

347. 1337.java

Path: LeetCode-main/solutions/1337. The K Weakest Rows in a Matrix/1337.java

```
class Solution {
    public int[] kWeakestRows(int[][] mat, int k) {
        int[] ans = new int[k];
        Pair<Integer, Integer>[] rowSums = new Pair[mat.length];

        for (int i = 0; i < mat.length; ++i)
            rowSums[i] = new Pair<>(Arrays.stream(mat[i]).sum(), i);

        Arrays.sort(rowSums, Comparator.comparingInt(Pair<Integer, Integer>::getKey)
            .thenComparingInt(Pair<Integer, Integer>::getValue));

        for (int i = 0; i < k; ++i)
            ans[i] = rowSums[i].getValue();

        return ans;
    }
}
```

348. 1338.java

Path: LeetCode-main/solutions/1338. Reduce Array Size to The Half/1338.java

```
class Solution {
    public int minSetSize(int[] arr) {
        final int n = arr.length;
        int sum = 0;
        Map<Integer, Integer> count = new HashMap<>();
        List<Map.Entry<Integer, Integer>> numAndFreqs = new ArrayList<>();

        for (final int a : arr)
            count.merge(a, 1, Integer::sum);

        for (Map.Entry<Integer, Integer> entry : count.entrySet())
            numAndFreqs.add(entry);

        numAndFreqs.sort(Comparator.comparingInt(Map.Entry<Integer, Integer>::getValue).reversed());

        for (int i = 0; i < numAndFreqs.size(); ++i) {
            sum += numAndFreqs.get(i).getValue();
            if (sum >= n / 2)
                return i + 1;
        }

        throw new IllegalArgumentException();
    }
}
```

349. 1339.java

Path: LeetCode-main/solutions/1339. Maximum Product of Splitted Binary Tree/1339.java

```
class Solution {
    public int maxProduct(TreeNode root) {
        final int MOD = 1_000_000_007;
        long ans = 0;
        List<Integer> allSums = new ArrayList<>();
        final long totalSum = treeSum(root, allSums);

        for (final long sum : allSums)
            ans = Math.max(ans, sum * (totalSum - sum));

        return (int) (ans % MOD);
    }

    private int treeSum(TreeNode root, List<Integer> allSums) {
        if (root == null)
            return 0;

        final int leftSum = treeSum(root.left, allSums);
        final int rightSum = treeSum(root.right, allSums);
        final int sum = root.val + leftSum + rightSum;
        allSums.add(sum);
        return sum;
    }
}
```

350. 134.java

Path: LeetCode-main/solutions/134. Gas Station/134.java

```
class Solution {
    public int canCompleteCircuit(int[] gas, int[] cost) {
        final int gasSum = Arrays.stream(gas).sum();
        final int costSum = Arrays.stream(cost).sum();
        if (gasSum - costSum < 0)
            return -1;

        int ans = 0;
        int sum = 0;

        // Try to start from each index.
        for (int i = 0; i < gas.length; ++i) {
            sum += gas[i] - cost[i];
            if (sum < 0) {
                sum = 0;
                ans = i + 1; // Start from the next index.
            }
        }
        return ans;
    }
}
```


351. 1340.java

Path: LeetCode-main/solutions/1340. Jump Game V/1340.java

```
class Solution {
    public int maxJumps(int[] arr, int d) {
        final int n = arr.length;
        // dp[i] := the maximum jumps starting from arr[i]
        int[] dp = new int[n];
        // a decreasing stack that stores indices
        Deque<Integer> stack = new ArrayDeque<>();

        for (int i = 0; i <= n; ++i) {
            while (!stack.isEmpty() && (i == n || arr[stack.peek()] < arr[i])) {
                List<Integer> indices = new ArrayList<>(List.of(stack.pop()));
                while (!stack.isEmpty() && arr[stack.peek()] == arr[indices.get(0)])
                    indices.add(stack.pop());
                for (final int j : indices) {
                    if (i < n && i - j <= d)
                        // Can jump from i to j.
                        dp[i] = Math.max(dp[i], dp[j] + 1);
                    if (!stack.isEmpty() && j - stack.peek() <= d)
                        // Can jump from stack.peek() to j
                        dp[stack.peek()] = Math.max(dp[stack.peek()], dp[j] + 1);
                }
            }
            stack.push(i);
        }

        return Arrays.stream(dp).max().getAsInt() + 1;
    }
}
```

352. 1342.java

Path: LeetCode-main/solutions/1342. Number of Steps to Reduce a Number to Zero/1342.java

```
class Solution {
    public int numberOfSteps(int num) {
        if (num == 0)
            return 0;
        final int subtractSteps = Integer.bitCount(num);
        final int divideSteps = 31 - Integer.numberOfLeadingZeros(num);
        return subtractSteps + divideSteps;
    }
}
```

353. 1343.java

Path: LeetCode-main/solutions/1343. Number of Sub-arrays of Size K and Average Greater than or Equal to Threshold/1343.java

```
class Solution {
    public int numOfSubarrays(int[] arr, int k, int threshold) {
        int ans = 0;
        int windowSum = 0;

        for (int i = 0; i < arr.length; ++i) {
            windowSum += arr[i];
            if (i >= k)
                windowSum -= arr[i - k];
            if (i >= k - 1 && windowSum / k >= threshold)
                ++ans;
        }

        return ans;
    }
}
```

354. 1344.java

Path: LeetCode-main/solutions/1344. Angle Between Hands of a Clock/1344.java

```
class Solution {
    public double angleClock(int hour, int minutes) {
        final double hourHand = (hour % 12 + minutes / 60.0) * 30;
        final double minuteHand = minutes * 6;
        final double diff = Math.abs(hourHand - minuteHand);
        return Math.min(diff, 360 - diff);
    }
}
```

355. 1345.java

Path: LeetCode-main/solutions/1345. Jump Game IV/1345.java

```
class Solution {
    public int minJumps(int[] arr) {
        final int n = arr.length;
        // {a: indices}
        Map<Integer, List<Integer>> graph = new HashMap<>();
        Queue<Integer> q = new ArrayDeque<>(List.of(0));
        boolean[] seen = new boolean[n];
        seen[0] = true;

        for (int i = 0; i < n; ++i) {
            graph.putIfAbsent(arr[i], new ArrayList<>());
            graph.get(arr[i]).add(i);
        }

        for (int step = 0; !q.isEmpty(); ++step) {
            for (int sz = q.size(); sz > 0; --sz) {
                final int i = q.poll();
                if (i == n - 1)
                    return step;
                seen[i] = true;
                final int u = arr[i];
                if (i + 1 < n)
                    graph.get(u).add(i + 1);
                if (i - 1 >= 0)
                    graph.get(u).add(i - 1);
                for (final int v : graph.get(u)) {
                    if (seen[v])
                        continue;
                    q.offer(v);
                }
                graph.get(u).clear();
            }
        }

        throw new IllegalArgumentException();
    }
}
```

356. 1346.java

Path: LeetCode-main/solutions/1346. Check If N and Its Double Exist/1346.java

```
class Solution {
    public boolean checkIfExist(int[] arr) {
        Set<Integer> seen = new HashSet<>();

        for (final int a : arr) {
            if (seen.contains(a * 2) || a % 2 == 0 && seen.contains(a / 2))
                return true;
            seen.add(a);
        }

        return false;
    }
}
```

357. 1347.java

Path: LeetCode-main/solutions/1347. Minimum Number of Steps to Make Two Strings Anagram/1347.java

```
class Solution {
    public int minSteps(String s, String t) {
        int[] count = new int[26];

        for (final char c : s.toCharArray())
            ++count[c - 'a'];

        for (final char c : t.toCharArray())
            --count[c - 'a'];

        return Arrays.stream(count).map(Math::abs).sum() / 2;
    }
}
```

358. 1348.java

Path: LeetCode-main/solutions/1348. Tweet Counts Per Frequency/1348.java

```
class TweetCounts {
    public void recordTweet(String tweetName, int time) {
        tweetNameToTimeCount.putIfAbsent(tweetName, new TreeMap<>());
        tweetNameToTimeCount.get(tweetName).merge(time, 1, Integer::sum);
    }

    public List<Integer> getTweetCountsPerFrequency(String freq, String tweetName, int startTime,
                                                    int endTime) {
        final int chunkSize = freq.equals("minute") ? 60 : freq.equals("hour") ? 3600 : 86400;
        int[] counts = new int[(endTime - startTime) / chunkSize + 1];
        TreeMap<Integer, Integer> timeCount = tweetNameToTimeCount.get(tweetName);

        for (Map.Entry<Integer, Integer> entry : timeCount.subMap(startTime, endTime + 1).entrySet()) {
            final int index = (entry.getKey() - startTime) / chunkSize;
            counts[index] += entry.getValue();
        }

        return Arrays.stream(counts).boxed().collect(Collectors.toList());
    }

    private Map<String, TreeMap<Integer, Integer>> tweetNameToTimeCount = new HashMap<>();
}
```


359. 1349.java

Path: LeetCode-main/solutions/1349. Maximum Students Taking Exam/1349.java

```
class Solution {
    public int maxStudents(char[][] seats) {
        int studentsCount = 0;

        for (char[] seat : seats)
            for (final char s : seat)
                if (s == '.')
                    ++studentsCount;

        return studentsCount - hungarian(seats);
    }

    private static final int[][] DIRS = {{-1, -1}, {0, -1}, {1, -1}, {-1, 1}, {0, 1}, {1, 1}};

    private int hungarian(char[][] seats) {
        final int m = seats.length;
        final int n = seats[0].length;
        int count = 0;
        int[][] seen = new int[m][n];
        int[][] match = new int[m][n];
        Arrays.stream(match).forEach(A -> Arrays.fill(A, -1));

        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j)
                if (seats[i][j] == '.' && match[i][j] == -1) {
                    final int sessionId = i * n + j;
                    seen[i][j] = sessionId;
                    if (dfs(seats, i, j, sessionId, seen, match))
                        ++count;
                }

        return count;
    }

    private boolean dfs(char[][] seats, int i, int j, int sessionId, int[][] seen, int[][] match) {
        final int m = seats.length;
        final int n = seats[0].length;

        for (int[] dir : DIRS) {
            final int x = i + dir[0];
            final int y = j + dir[1];
            if (x < 0 || x == m || y < 0 || y == n)
                continue;
            if (seats[x][y] != '.' || seen[x][y] == sessionId)
                continue;
            seen[x][y] = sessionId;
            if (match[x][y] == -1 ||
                dfs(seats, match[x][y] / n, match[x][y] % n, sessionId, seen, match)) {
                match[x][y] = i * n + j;
                match[i][j] = x * n + y;
                return true;
            }
        }

        return false;
    }
}
```

360. 135.java

Path: LeetCode-main/solutions/135. Candy/135.java

```
class Solution {
    public int candy(int[] ratings) {
        final int n = ratings.length;

        int ans = 0;
        int[] l = new int[n];
        int[] r = new int[n];
        Arrays.fill(l, 1);
        Arrays.fill(r, 1);

        for (int i = 1; i < n; ++i)
            if (ratings[i] > ratings[i - 1])
                l[i] = l[i - 1] + 1;

        for (int i = n - 2; i >= 0; --i)
            if (ratings[i] > ratings[i + 1])
                r[i] = r[i + 1] + 1;

        for (int i = 0; i < n; ++i)
            ans += Math.max(l[i], r[i]);

        return ans;
    }
}
```

361. 1351.java

Path: LeetCode-main/solutions/1351. Count Negative Numbers in a Sorted Matrix/1351.java

```
class Solution {
    public int countNegatives(int[][] grid) {
        final int m = grid.length;
        final int n = grid[0].length;
        int ans = 0;
        int i = m - 1;
        int j = 0;

        while (i >= 0 && j < n) {
            if (grid[i][j] < 0) {
                ans += n - j;
                --i;
            } else {
                ++j;
            }
        }

        return ans;
    }
}
```

362. 1352.java

Path: LeetCode-main/solutions/1352. Product of the Last K Numbers/1352.java

```
class ProductOfNumbers {
    public ProductOfNumbers() {
        prefix = new ArrayList<>(List.of(1));
    }

    public void add(int num) {
        if (num == 0)
            prefix = new ArrayList<>(List.of(1));
        else
            prefix.add(prefix.get(prefix.size() - 1) * num);
    }

    public int getProduct(int k) {
        return k >= prefix.size() ? 0
            : prefix.get(prefix.size() - 1) / prefix.get(prefix.size() - k - 1);
    }

    private List<Integer> prefix = new ArrayList<>();
}
```

363. 1353.java

Path: LeetCode-main/solutions/1353. Maximum Number of Events That Can Be Attended/1353.java

```
class Solution {
    public int maxEvents(int[][] events) {
        int ans = 0;
        int d = 0; // the current day
        int i = 0; // events' index
        Queue<Integer> minHeap = new PriorityQueue<>();

        Arrays.sort(events, Comparator.comparingInt(event -> event[0]));

        while (!minHeap.isEmpty() || i < events.length) {
            // If no events are available to attend today, let time flies to the next
            // available event.
            if (minHeap.isEmpty())
                d = events[i][0];
            // All the events starting from today are newly available.
            while (i < events.length && events[i][0] == d)
                minHeap.offer(events[i++][1]);
            // Greedily attend the event that'll end the earliest since it has higher
            // chance can't be attended in the future.
            minHeap.poll();
            ++ans;
            ++d;
            // Pop the events that can't be attended.
            while (!minHeap.isEmpty() && minHeap.peek() < d)
                minHeap.poll();
        }

        return ans;
    }
}
```

364. 1354.java

Path: LeetCode-main/solutions/1354. Construct Target Array With Multiple Sums/1354.java

```
class Solution {
    public boolean isPossible(int[] target) {
        if (target.length == 1)
            return target[0] == 1;

        long sum = Arrays.stream(target).asLongStream().sum();
        Queue<Integer> maxHeap = new PriorityQueue<>(Collections.reverseOrder());

        for (final int num : target)
            maxHeap.offer(num);

        while (maxHeap.peek() > 1) {
            final long mx = maxHeap.poll();
            final long restSum = sum - mx;
            // Only occurs if n == 2.
            if (restSum == 1)
                return true;
            final long updated = mx % restSum;
            // updated == 0 (invalid) or didn't change.
            if (updated == 0 || updated == mx)
                return false;
            maxHeap.offer((int) updated);
            sum = sum - mx + updated;
        }

        return true;
    }
}
```

365. 1356.java

Path: LeetCode-main/solutions/1356. Sort Integers by The Number of 1 Bits/1356.java

```
class Solution {  
    public int[] sortByBits(int[] arr) {  
        return Arrays.stream(arr)  
            .boxed()  
            .sorted(Comparator.comparingInt(Integer::bitCount).thenComparingInt(a -> a))  
            .mapToInt(Integer::intValue)  
            .toArray();  
    }  
}
```

366. 1357.java

Path: LeetCode-main/solutions/1357. Apply Discount Every n Orders/1357.java

```
class Cashier {
    public Cashier(int n, int discount, int[] products, int[] prices) {
        this.n = n;
        this.discount = discount;
        for (int i = 0; i < products.length; ++i)
            productToPrice.put(products[i], prices[i]);
    }

    public double getBill(int[] product, int[] amount) {
        ++count;
        int total = 0;
        for (int i = 0; i < product.length; ++i)
            total += productToPrice.get(product[i]) * amount[i];
        return count % n == 0 ? total * (1 - discount / 100.0) : total;
    }

    private int n;
    private int discount;
    private Map<Integer, Integer> productToPrice = new HashMap<>();
    private int count = 0;
}
```


367. 1358-2.java

Path: LeetCode-main/solutions/1358. Number of Substrings Containing All Three Characters/1358-2.java

```
class Solution {
    // Similar to 3. Longest SubString Without Repeating Characters
    public int numberOfSubstrings(String s) {
        int ans = 0;
        // lastSeen[c] := the index of the last time c appeared
        int[] lastSeen = new int[3];
        Arrays.fill(lastSeen, -1);

        for (int i = 0; i < s.length(); ++i) {
            lastSeen[s.charAt(i) - 'a'] = i;
            // s[0..i], s[1..i], s[Math.min(lastSeen)..i] are satisfied strings.
            ans += 1 + Arrays.stream(lastSeen).min().getAsInt();
        }

        return ans;
    }
}
```

368. 1358.java

Path: LeetCode-main/solutions/1358. Number of Substrings Containing All Three Characters/1358.java

```
class Solution {
    // Similar to 3. Longest SubString Without Repeating Characters
    public int numberOfSubstrings(String s) {
        int ans = 0;
        int[] count = new int[3];

        int l = 0;
        for (final char c : s.toCharArray()) {
            ++count[c - 'a'];
            while (count[0] > 0 && count[1] > 0 && count[2] > 0)
                --count[s.charAt(l++) - 'a'];
            // s[0..r], s[1..r], ..., s[l - 1..r] are satisfied strings.
            ans += l;
        }

        return ans;
    }
}
```

369. 1359.java

Path: LeetCode-main/solutions/1359. Count All Valid Pickup and Delivery Options/1359.java

```
class Solution {  
    public int countOrders(int n) {  
        final int MOD = 1_000_000_007;  
        long ans = 1;  
  
        for (int i = 1; i <= n; ++i)  
            ans = ans * i * (i * 2 - 1) % MOD;  
  
        return (int) ans;  
    }  
}
```

370. 136.java

Path: LeetCode-main/solutions/136. Single Number/136.java

```
class Solution {  
    public int singleNumber(int[] nums) {  
        int ans = 0;  
  
        for (final int num : nums)  
            ans ^= num;  
  
        return ans;  
    }  
}
```

371. 1360.java

Path: LeetCode-main/solutions/1360. Number of Days Between Two Dates/1360.java

```
class Solution {
    public int daysBetweenDates(String date1, String date2) {
        return Math.abs(daysFrom1971(date1) - daysFrom1971(date2));
    }

    private static final int[] days = {0, 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};

    private int daysFrom1971(final String date) {
        final int year = Integer.valueOf(date.substring(0, 4));
        final int month = Integer.valueOf(date.substring(5, 7));
        final int day = Integer.valueOf(date.substring(8));
        int res = 0;
        for (int i = 1971; i < year; ++i)
            res += isLeapYear(i) ? 366 : 365;
        for (int i = 0; i < month; ++i)
            res += days[i];
        if (month > 2 && isLeapYear(year))
            ++res;
        return res + day;
    }

    private boolean isLeapYear(int year) {
        return year % 4 == 0 && year % 100 != 0 || year % 400 == 0;
    }
}
```

372. 1361.java

Path: LeetCode-main/solutions/1361. Validate Binary Tree Nodes/1361.java

```
class Solution {
    public boolean validateBinaryTreeNodes(int n, int[] leftChild, int[] rightChild) {
        int[] inDegrees = new int[n];
        int root = -1;

        // If the in-degree of any node > 1, return false.
        for (final int child : leftChild)
            if (child != -1 && ++inDegrees[child] == 2)
                return false;

        for (final int child : rightChild)
            if (child != -1 && ++inDegrees[child] == 2)
                return false;

        // Find the root (the node with in-degree == 0).
        for (int i = 0; i < n; ++i)
            if (inDegrees[i] == 0)
                if (root == -1)
                    root = i;
                else
                    return false; // There're multiple roots.

        // Didn't find the root.
        if (root == -1)
            return false;

        return countNodes(root, leftChild, rightChild) == n;
    }

    private int countNodes(int root, int[] leftChild, int[] rightChild) {
        if (root == -1)
            return 0;
        return 1 + //
            countNodes(leftChild[root], leftChild, rightChild) +
            countNodes(rightChild[root], leftChild, rightChild);
    }
}
```

373. 1362.java

Path: LeetCode-main/solutions/1362. Closest Divisors/1362.java

```
class Solution {
    public int[] closestDivisors(int num) {
        for (int root = (int) Math.sqrt(num + 2); root > 0; --root)
            for (int cand : new int[] {num + 1, num + 2})
                if (cand % root == 0)
                    return new int[] {root, cand / root};

        throw new IllegalArgumentException();
    }
}
```

374. 1363.java

Path: LeetCode-main/solutions/1363. Largest Multiple of Three/1363.java

```
class Solution {
    public String largestMultipleOfThree(int[] digits) {
        StringBuilder ans = new StringBuilder();
        int[] mod1 = new int[] {1, 4, 7, 2, 5, 8};
        int[] mod2 = new int[] {2, 5, 8, 1, 4, 7};
        int[] count = new int[10];
        int sum = 0;

        for (int digit : digits) {
            ++count[digit];
            sum += digit;
        }

        while (sum % 3 != 0)
            for (int i : sum % 3 == 1 ? mod1 : mod2)
                if (count[i] > 0) {
                    --count[i];
                    sum -= i;
                    break;
                }

        for (int digit = 9; digit >= 0; --digit)
            ans.append(Character.toString('0' + digit).repeat(count[digit]));

        return ans.length() > 0 && ans.charAt(0) == '0' ? "0" : ans.toString();
    }
}
```


375. 1365.java

Path: LeetCode-main/solutions/1365. How Many Numbers Are Smaller Than the Current Number/1365.java

```
class Solution {
    public int[] smallerNumbersThanCurrent(int[] nums) {
        final int MAX = 100;
        int[] ans = new int[nums.length];
        int[] count = new int[MAX + 1];

        for (final int num : nums)
            ++count[num];

        for (int i = 1; i <= MAX; ++i)
            count[i] += count[i - 1];

        for (int i = 0; i < nums.length; ++i)
            ans[i] = nums[i] == 0 ? 0 : count[nums[i] - 1];

        return ans;
    }
}
```

376. 1366.java

Path: LeetCode-main/solutions/1366. Rank Teams by Votes/1366.java

```
class Team {
    public char name;
    public int[] rank;
    public Team(char name, int teamSize) {
        this.name = name;
        this.rank = new int[teamSize];
    }
}

class Solution {
    public String rankTeams(String[] votes) {
        final int teamSize = votes[0].length();
        StringBuilder sb = new StringBuilder();
        Team[] teams = new Team[26];

        for (int i = 0; i < 26; ++i)
            teams[i] = new Team((char) ('A' + i), teamSize);

        for (final String vote : votes)
            for (int i = 0; i < teamSize; ++i)
                ++teams[vote.charAt(i) - 'A'].rank[i];

        Arrays.sort(teams, new Comparator<Team>() {
            @Override
            public int compare(Team a, Team b) {
                for (int i = 0; i < a.rank.length; ++i)
                    if (a.rank[i] > b.rank[i])
                        return -1;
                    else if (a.rank[i] < b.rank[i])
                        return 1;
                return a.name - b.name;
            }
        });

        for (int i = 0; i < teamSize; ++i)
            sb.append(teams[i].name);

        return sb.toString();
    }
}
```

377. 1367.java

Path: LeetCode-main/solutions/1367. Linked List in Binary Tree/1367.java

```
class Solution {
    public boolean isSubPath(ListNode head, TreeNode root) {
        if (root == null)
            return false;
        return isContinuousSubPath(head, root) || isSubPath(head, root.left) ||
            isSubPath(head, root.right);
    }

    private boolean isContinuousSubPath(ListNode head, TreeNode root) {
        if (head == null)
            return true;
        if (root == null)
            return false;
        return head.val == root.val &&
            (isContinuousSubPath(head.next, root.left) || isContinuousSubPath(head.next, root.right));
    }
}
```

378. 1368.java

Path: LeetCode-main/solutions/1368. Minimum Cost to Make at Least One Valid Path in a Grid/1368.java

```
class Solution {
    public int minCost(int[][] grid) {
        final int m = grid.length;
        final int n = grid[0].length;
        int[][] mem = new int[m][n];
        Arrays.stream(mem).forEach(A -> Arrays.fill(A, -1));
        Queue<Pair<Integer, Integer>> q = new ArrayDeque<>();

        dfs(grid, 0, 0, /*cost=*/0, q, mem);

        for (int cost = 1; !q.isEmpty(); ++cost)
            for (int sz = q.size(); sz > 0; --sz) {
                Pair<Integer, Integer> pair = q.poll();
                final int i = pair.getKey();
                final int j = pair.getValue();
                for (int[] dir : DIRS)
                    dfs(grid, i + dir[0], j + dir[1], cost, q, mem);
            }

        return mem[m - 1][n - 1];
    }

    private static final int[][] DIRS = {{0, 1}, {0, -1}, {1, 0}, {-1, 0}};

    private void dfs(int[][] grid, int i, int j, int cost, Queue<Pair<Integer, Integer>> q,
                     int[][] mem) {
        if (i < 0 || i == grid.length || j < 0 || j == grid[0].length)
            return;
        if (mem[i][j] != -1)
            return;

        mem[i][j] = cost;
        q.add(new Pair<>(i, j));
        int[] dir = DIRS[grid[i][j] - 1];
        dfs(grid, i + dir[0], j + dir[1], cost, q, mem);
    }
}
```

379. 137-2.java

Path: LeetCode-main/solutions/137. Single Number II/137-2.java

```
class Solution {
    public int singleNumber(int[] nums) {
        int ones = 0;
        int twos = 0;

        for (final int num : nums) {
            ones ^= (num & ~twos);
            twos ^= (num & ~ones);
        }

        return ones;
    }
}
```

380. 137.java

Path: LeetCode-main/solutions/137. Single Number II/137.java

```
class Solution {
    public int singleNumber(int[] nums) {
        int ans = 0;

        for (int i = 0; i < 32; ++i) {
            int sum = 0;
            for (final int num : nums)
                sum += num >> i & 1;
            sum %= 3;
            ans |= sum << i;
        }

        return ans;
    }
}
```

381. 1370.java

Path: LeetCode-main/solutions/1370. Increasing Decreasing String/1370.java

```
class Solution {
    public String sortString(String s) {
        StringBuilder sb = new StringBuilder();
        int[] count = new int[26];

        for (final char c : s.toCharArray())
            ++count[c - 'a'];

        while (sb.length() < s.length()) {
            for (int i = 0; i < 26; i++) {
                if (count[i] == 0)
                    continue;
                sb.append((char) (i + 'a'));
                --count[i];
            }
            for (int i = 25; i >= 0; i--) {
                if (count[i] == 0)
                    continue;
                sb.append((char) (i + 'a'));
                --count[i];
            }
        }
        return sb.toString();
    }
}
```

382. 1371.java

Path: LeetCode-main/solutions/1371. Find the Longest Substring Containing Vowels in Even Counts/1371.java

```
class Solution {
    public int findTheLongestSubstring(String s) {
        final String VOWELS = "aeiou";
        int ans = 0;
        int prefix = 0; // the binary prefix
        Map<Integer, Integer> prefixToIndex = new HashMap<>();
        prefixToIndex.put(0, -1);

        for (int i = 0; i < s.length(); ++i) {
            final int index = VOWELS.indexOf(s.charAt(i));
            if (index != -1)
                prefix ^= 1 << index;
            prefixToIndex.putIfAbsent(prefix, i);
            ans = Math.max(ans, i - prefixToIndex.get(prefix));
        }

        return ans;
    }
}
```


383. 1372.java

Path: LeetCode-main/solutions/1372. Longest ZigZag Path in a Binary Tree/1372.java

```
class Solution {
    public int longestZigZag(TreeNode root) {
        return dfs(root).subtreeMax;
    }

    private record T(int leftMax, int rightMax, int subtreeMax) {}

    private T dfs(TreeNode root) {
        if (root == null)
            return new T(-1, -1, -1);
        T left = dfs(root.left);
        T right = dfs(root.right);
        final int leftZigZag = left.rightMax + 1;
        final int rightZigZag = right.leftMax + 1;
        final int subtreeMax =
            Math.max(Math.max(leftZigZag, rightZigZag), Math.max(left.subtreeMax, right.subtreeMax));
        return new T(leftZigZag, rightZigZag, subtreeMax);
    }
}
```

384. 1373.java

Path: LeetCode-main/solutions/1373. Maximum Sum BST in Binary Tree/1373.java

```
class Solution {
    public int maxSumBST(TreeNode root) {
        traverse(root);
        return ans;
    }

    private record T(boolean isBST, Integer mx, Integer mn, Integer sum) {}

    private int ans = 0;

    private T traverse(TreeNode root) {
        if (root == null)
            return new T(true, Integer.MIN_VALUE, Integer.MAX_VALUE, 0);

        T left = traverse(root.left);
        T right = traverse(root.right);

        if (!left.isBST || !right.isBST)
            return new T(false, null, null, null);
        if (root.val <= left.mx || root.val >= right.mn)
            return new T(false, null, null, null);

        // The `root` is a valid BST.
        final int sum = root.val + left.sum + right.sum;
        ans = Math.max(ans, sum);
        return new T(true, Math.max(root.val, right.mx), Math.min(root.val, left.mn), sum);
    }
}
```

385. 1374.java

Path: LeetCode-main/solutions/1374. Generate a String With Characters That Have Odd Counts/1374.java

```
class Solution {
    public String generateTheString(int n) {
        StringBuilder sb = new StringBuilder(n);
        for (int i = 0; i < n; ++i)
            sb.append('a');
        if (n % 2 == 0)
            sb.setCharAt(n - 1, 'b');
        return sb.toString();
    }
}
```

386. 1375.java

Path: LeetCode-main/solutions/1375. Number of Times Binary String Is Prefix-Aligned/1375.java

```
class Solution {
    public int numTimesAllBlue(int[] flips) {
        int ans = 0;
        int rightmost = 0;

        for (int i = 0; i < flips.length; ++i) {
            rightmost = Math.max(rightmost, flips[i]);
            // Math.max(flips[0..i]) = rightmost = i + 1,
            // so flips[0..i] is a permutation of 1, 2, ..., i + 1.
            if (rightmost == i + 1)
                ++ans;
        }

        return ans;
    }
}
```

387. 1376.java

Path: LeetCode-main/solutions/1376. Time Needed to Inform All Employees/1376.java

```
class Solution {
    public int numOfMinutes(int n, int headID, int[] manager, int[] informTime) {
        int ans = 0;
        Map<Integer, Integer> mem = new HashMap<>();

        for (int i = 0; i < n; ++i)
            ans = Math.max(ans, dfs(i, headID, manager, informTime, mem));

        return ans;
    }

    int dfs(int i, int headID, int[] manager, int[] informTime, Map<Integer, Integer> mem) {
        if (mem.containsKey(i))
            return mem.get(i);
        if (i == headID)
            return 0;

        final int parent = manager[i];
        mem.put(i, informTime[parent] + dfs(parent, headID, manager, informTime, mem));
        return mem.get(i);
    }
}
```

388. 1377.java

Path: LeetCode-main/solutions/1377. Frog Position After T Seconds/1377.java

```
class Solution {
    public double frogPosition(int n, int[][] edges, int t, int target) {
        List<Integer>[] tree = new List[n + 1];
        Queue<Integer> q = new ArrayDeque<>(List.of(1));
        boolean[] seen = new boolean[n + 1];
        double[] prob = new double[n + 1];

        seen[1] = true;
        prob[1] = 1.0;

        for (int i = 1; i <= n; ++i)
            tree[i] = new ArrayList<>();

        for (int[] edge : edges) {
            final int u = edge[0];
            final int v = edge[1];
            tree[u].add(v);
            tree[v].add(u);
        }

        while (!q.isEmpty() && t-- > 0)
            for (int sz = q.size(); sz > 0; --sz) {
                final int a = q.poll();
                int nChildren = 0;
                for (final int b : tree[a])
                    if (!seen[b])
                        ++nChildren;
                for (final int b : tree[a]) {
                    if (seen[b])
                        continue;
                    seen[b] = true;
                    prob[b] = prob[a] / nChildren;
                    q.add(b);
                }
                if (nChildren > 0)
                    prob[a] = 0.0;
            }

        return prob[target];
    }
}
```

389. 1379.java

Path: LeetCode-main/solutions/1379. Find a Corresponding Node of a Binary Tree in a Clone of That Tree/1379.java

```
class Solution {
    public final TreeNode getTargetCopy(final TreeNode original, final TreeNode cloned,
                                       final TreeNode target) {
        dfs(original, cloned, target);
        return ans;
    }

    private TreeNode ans = null;

    private void dfs(TreeNode original, TreeNode cloned, TreeNode target) {
        if (ans != null)
            return;
        if (original == null)
            return;
        if (original == target) {
            ans = cloned;
            return;
        }
        dfs(original.left, cloned.left, target);
        dfs(original.right, cloned.right, target);
    }
}
```

390. 138.java

Path: LeetCode-main/solutions/138. Copy List with Random Pointer/138.java

```
class Solution {
    public Node copyRandomList(Node head) {
        if (head == null)
            return null;
        if (map.containsKey(head))
            return map.get(head);

        Node newNode = new Node(head.val);
        map.put(head, newNode);
        newNode.next = copyRandomList(head.next);
        newNode.random = copyRandomList(head.random);
        return newNode;
    }

    private Map<Node, Node> map = new HashMap<>();
}
```


391. 1380.java

Path: LeetCode-main/solutions/1380. Lucky Numbers in a Matrix/1380.java

```
class Solution {
    public List<Integer> luckyNumbers(int[][] matrix) {
        for (int[] row : matrix) {
            final int minIndex = getMinIndex(row);
            if (row[minIndex] == maxNumOfColumn(matrix, minIndex))
                return List.of(row[minIndex]);
        }
        return new ArrayList<>();
    }

    private int getMinIndex(int[] row) {
        int minIndex = 0;
        for (int j = 0; j < row.length; ++j)
            if (row[j] < row[minIndex])
                minIndex = j;
        return minIndex;
    }

    private int maxNumOfColumn(int[][] matrix, int j) {
        int res = 0;
        for (int i = 0; i < matrix.length; ++i)
            res = Math.max(res, matrix[i][j]);
        return res;
    }
}
```

392. 1381.java

Path: LeetCode-main/solutions/1381. Design a Stack With Increment Operation/1381.java

```
class CustomStack {
    public CustomStack(int maxSize) {
        this.maxSize = maxSize;
    }

    public void push(int x) {
        if (stack.size() == maxSize)
            return;
        stack.push(x);
        pendingIncrements.add(0);
    }

    public int pop() {
        if (stack.isEmpty())
            return -1;
        final int i = stack.size() - 1;
        final int pendingIncrement = pendingIncrements.get(i);
        pendingIncrements.remove(i);
        if (i > 0)
            pendingIncrements.set(i - 1, pendingIncrements.get(i - 1) + pendingIncrement);
        return stack.pop() + pendingIncrement;
    }

    public void increment(int k, int val) {
        if (stack.isEmpty())
            return;
        final int i = Math.min(k - 1, stack.size() - 1);
        pendingIncrements.set(i, pendingIncrements.get(i) + val);
    }

    private int maxSize;
    private Deque<Integer> stack = new ArrayDeque<>();
    // pendingIncrements[i] := the pending increment for stack[0..i].
    private List<Integer> pendingIncrements = new ArrayList<>();
}
```

393. 1382.java

Path: LeetCode-main/solutions/1382. Balance a Binary Search Tree/1382.java

```
class Solution {
    public TreeNode balanceBST(TreeNode root) {
        List<Integer> nums = new ArrayList<>();
        inorder(root, nums);
        return build(nums, 0, nums.size() - 1);
    }

    private void inorder(TreeNode root, List<Integer> nums) {
        if (root == null)
            return;
        inorder(root.left, nums);
        nums.add(root.val);
        inorder(root.right, nums);
    }

    // Same as 108. Convert Sorted Array to Binary Search Tree
    private TreeNode build(List<Integer> nums, int l, int r) {
        if (l > r)
            return null;
        final int m = (l + r) / 2;
        return new TreeNode(nums.get(m), build(nums, l, m - 1), build(nums, m + 1, r));
    }
}
```

394. 1383.java

Path: LeetCode-main/solutions/1383. Maximum Performance of a Team/1383.java

```
class Solution {
    // Similar to 857. Minimum Cost to Hire K Workers
    public int maxPerformance(int n, int[] speed, int[] efficiency, int k) {
        final int MOD = 1_000_000_007;
        long ans = 0;
        long speedSum = 0;
        // (efficiency[i], speed[i]) sorted by efficiency[i] in descending order
        Pair<Integer, Integer>[] A = new Pair[n];
        Queue<Integer> minHeap = new PriorityQueue<>();

        for (int i = 0; i < n; ++i)
            A[i] = new Pair<>(efficiency[i], speed[i]);

        Arrays.sort(A, Comparator.comparingInt(Pair::getKey).reversed());

        for (Pair<Integer, Integer> a : A) {
            final int e = a.getKey();
            final int s = a.getValue();
            minHeap.offer(s);
            speedSum += s;
            if (minHeap.size() > k)
                speedSum -= minHeap.poll();
            ans = Math.max(ans, speedSum * e);
        }

        return (int) (ans % MOD);
    }
}
```

395. 1385.java

Path: LeetCode-main/solutions/1385. Find the Distance Value Between Two Arrays/1385.java

```
class Solution {
    public int findTheDistanceValue(int[] arr1, int[] arr2, int d) {
        int ans = 0;

        Arrays.sort(arr2);

        for (final int a : arr1) {
            int i = Arrays.binarySearch(arr2, a);
            i = i < 0 ? -i - 1 : i;
            if ((i == arr2.length || arr2[i] - a > d) && (i == 0 || a - arr2[i - 1] > d))
                ++ans;
        }

        return ans;
    }
}
```

396. 1386.java

Path: LeetCode-main/solutions/1386. Cinema Seat Allocation/1386.java

```
class Solution {
    public int maxNumberOfFamilies(int n, int[][] reservedSeats) {
        int ans = 0;
        Map<Integer, Integer> rowToSeats = new HashMap<>();

        for (int[] reservedSeat : reservedSeats) {
            final int row = reservedSeat[0];
            final int seat = reservedSeat[1];
            rowToSeats.put(row, rowToSeats.getOrDefault(row, 0) | 1 << (seat - 1));
        }

        for (final int seats : rowToSeats.values())
            if ((seats & 0b0111111110) == 0)
                // Can fit 2 four-person groups.
                ans += 2;
            else if ((seats & 0b0111100000) == 0 || // The left is not occupied.
                    (seats & 0b0001111000) == 0 || // The middle is not occupied.
                    (seats & 0b0000011110) == 0) // The right is not occupied.
                // Can fit 1 four-person group.
                ans += 1;

        // Any empty row can fit 2 four-person groups.
        return ans + (n - rowToSeats.size()) * 2;
    }
}
```

397. 1387.java

Path: LeetCode-main/solutions/1387. Sort Integers by The Power Value/1387.java

```
class Solution {
    public int getKth(int lo, int hi, int k) {
        int[][] powAndVals = new int[hi - lo + 1][2]; // (pow, val)

        for (int i = lo; i <= hi; ++i)
            powAndVals[i - lo] = new int[] {getPow(i), i};

        Arrays.sort(powAndVals, Comparator.comparingInt(powAndVal -> powAndVal[0]));
        return powAndVals[k - 1][1];
    }

    private int getPow(int n) {
        if (n == 1)
            return 0;
        return 1 + (n % 2 == 0 ? getPow(n / 2) : getPow(n * 3 + 1));
    }
}
```

398. 1389.java

Path: LeetCode-main/solutions/1389. Create Target Array in the Given Order/1389.java

```
class Solution {
    public int[] createTargetArray(int[] nums, int[] index) {
        List<Integer> ans = new ArrayList<>();
        for (int i = 0; i < nums.length; i++)
            ans.add(index[i], nums[i]);
        return ans.stream().mapToInt(Integer::intValue).toArray();
    }
}
```


399. 139-2.java

Path: LeetCode-main/solutions/139. Word Break/139-2.java

```
class Solution {
    public boolean wordBreak(String s, List<String> wordDict) {
        return wordBreak(s, 0, new HashSet<>(wordDict), new Boolean[s.length()]);
    }

    // Returns true if s[i..n) can be segmented.
    private boolean wordBreak(final String s, int i, Set<String> wordSet, Boolean[] mem) {
        if (i == s.length())
            return true;
        if (mem[i] != null)
            return mem[i];

        for (int j = i + 1; j <= s.length(); ++j)
            if (wordSet.contains(s.substring(i, j)) && wordBreak(s, j, wordSet, mem))
                return mem[i] = true;

        return mem[i] = false;
    }
}
```

400. 139-3.java

Path: LeetCode-main/solutions/139. Word Break/139-3.java

```
class Solution {
    public boolean wordBreak(String s, List<String> wordDict) {
        final int n = s.length();
        Set<String> wordSet = new HashSet<>(wordDict);
        // dp[i] := true if s[0..i) can be segmented
        boolean[] dp = new boolean[n + 1];
        dp[0] = true;

        for (int i = 1; i <= n; ++i)
            for (int j = 0; j < i; ++j)
                // s[0..j) can be segmented and s[j..i) is in `wordSet`, so s[0..i) can
                // be segmented.
                if (dp[j] && wordSet.contains(s.substring(j, i))) {
                    dp[i] = true;
                    break;
                }

        return dp[n];
    }
}
```

401. 139-4.java

Path: LeetCode-main/solutions/139. Word Break/139-4.java

```
class Solution {
    public boolean wordBreak(String s, List<String> wordDict) {
        final int n = s.length();
        final int maxLength = getMaxLength(wordDict);
        Set<String> wordSet = new HashSet<>(wordDict);
        // dp[i] := true if s[0..i) can be segmented
        boolean[] dp = new boolean[n + 1];
        dp[0] = true;

        for (int i = 1; i <= n; ++i)
            for (int j = i - 1; j >= 0; --j) {
                if (i - j > maxLength)
                    break;
                // s[0..j) can be segmented and s[j..i) is in the wordSet, so s[0..i)
                // can be segmented.
                if (dp[j] && wordSet.contains(s.substring(j, i))) {
                    dp[i] = true;
                    break;
                }
            }

        return dp[n];
    }

    private int getMaxLength(List<String> wordDict) {
        return wordDict.stream().mapToInt(String::length).max().getAsInt();
    }
}
```

402. 139.java

Path: LeetCode-main/solutions/139. Word Break/139.java

```
class Solution {
    public boolean wordBreak(String s, List<String> wordDict) {
        return wordBreak(s, new HashSet<>(wordDict), new HashMap<>());
    }

    private boolean wordBreak(final String s, Set<String> wordSet, Map<String, Boolean> mem) {
        if (mem.containsKey(s))
            return mem.get(s);
        if (wordSet.contains(s)) {
            mem.put(s, true);
            return true;
        }

        // 1 <= prefix.length() < s.length()
        for (int i = 1; i < s.length(); ++i) {
            final String prefix = s.substring(0, i);
            final String suffix = s.substring(i);
            if (wordSet.contains(prefix) && wordBreak(suffix, wordSet, mem)) {
                mem.put(s, true);
                return true;
            }
        }

        mem.put(s, false);
        return false;
    }
}
```

403. 1390.java

Path: LeetCode-main/solutions/1390. Four Divisors/1390.java

```
class Solution {
    public int sumFourDivisors(int[] nums) {
        int ans = 0;

        for (int num : nums) {
            int divisor = 0;
            for (int i = 2; i * i <= num; ++i)
                if (num % i == 0) {
                    if (divisor == 0)
                        divisor = i;
                    else {
                        divisor = 0;
                        break;
                    }
                }
            if (divisor > 0 && divisor * divisor < num)
                ans += 1 + num + divisor + num / divisor;
        }

        return ans;
    }
}
```

404. 1391.java

Path: LeetCode-main/solutions/1391. Check if There is a Valid Path in a Grid/1391.java

```
class Solution {
    public boolean hasValidPath(int[][] grid) {
        final int m = grid.length;
        final int n = grid[0].length;
        // g := upscaled grid
        boolean[][] g = new boolean[m * 3][n * 3];

        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j)
                switch (grid[i][j]) {
                    case 1:
                        g[i * 3 + 1][j * 3 + 0] = true;
                        g[i * 3 + 1][j * 3 + 1] = true;
                        g[i * 3 + 1][j * 3 + 2] = true;
                        break;
                    case 2:
                        g[i * 3 + 0][j * 3 + 1] = true;
                        g[i * 3 + 1][j * 3 + 1] = true;
                        g[i * 3 + 2][j * 3 + 1] = true;
                        break;
                    case 3:
                        g[i * 3 + 1][j * 3 + 0] = true;
                        g[i * 3 + 1][j * 3 + 1] = true;
                        g[i * 3 + 2][j * 3 + 1] = true;
                        break;
                    case 4:
                        g[i * 3 + 1][j * 3 + 1] = true;
                        g[i * 3 + 1][j * 3 + 2] = true;
                        g[i * 3 + 2][j * 3 + 1] = true;
                        break;
                    case 5:
                        g[i * 3 + 0][j * 3 + 1] = true;
                        g[i * 3 + 1][j * 3 + 0] = true;
                        g[i * 3 + 1][j * 3 + 1] = true;
                        break;
                    case 6:
                        g[i * 3 + 0][j * 3 + 1] = true;
                        g[i * 3 + 1][j * 3 + 1] = true;
                        g[i * 3 + 1][j * 3 + 2] = true;
                        break;
                }

        return dfs(g, 1, 1);
    }

    private boolean dfs(boolean[][] g, int i, int j) {
        if (i < 0 || i == g.length || j < 0 || j == g[0].length)
            return false;
        if (!g[i][j]) // There's no path here.
            return false;
        if (i == g.length - 2 && j == g[0].length - 2)
            return true;

        g[i][j] = false; // Mark as visited.
        return dfs(g, i + 1, j) || dfs(g, i - 1, j) || dfs(g, i, j + 1) || dfs(g, i, j - 1);
    }
}
```

405. 1392.java

Path: LeetCode-main/solutions/1392. Longest Happy Prefix/1392.java

```
class Solution {
    public String longestPrefix(String s) {
        final int BASE = 26;
        final long HASH = 8_417_508_174_513L;
        final int n = s.length();
        int maxLength = 0;
        long pow = 1;
        long prefixHash = 0; // the hash of s[0..i]
        long suffixHash = 0; // the hash of s[j..n)

        for (int i = 0, j = n - 1; i < n - 1; ++i, --j) {
            prefixHash = (prefixHash * BASE + val(s.charAt(i))) % HASH;
            suffixHash = (val(s.charAt(j)) * pow + suffixHash) % HASH;
            pow = pow * BASE % HASH;
            if (prefixHash == suffixHash)
                maxLength = i + 1;
        }

        return s.substring(0, maxLength);
    }

    private int val(char c) {
        return c - 'a';
    }
}
```

406. 1394.java

Path: LeetCode-main/solutions/1394. Find Lucky Integer in an Array/1394.java

```
class Solution {
    public int findLucky(int[] arr) {
        int[] count = new int[arr.length + 1];

        for (final int a : arr)
            if (a <= arr.length)
                ++count[a];

        for (int i = arr.length; i >= 1; --i)
            if (count[i] == i)
                return i;

        return -1;
    }
}
```


407. 1395.java

Path: LeetCode-main/solutions/1395. Count Number of Teams/1395.java

```
class Solution {
    public int numTeams(int[] rating) {
        int ans = 0;

        for (int i = 1; i < rating.length - 1; ++i) {
            // Calculate soldiers on the left with less/greater ratings.
            int leftLess = 0;
            int leftGreater = 0;
            for (int j = 0; j < i; ++j)
                if (rating[j] < rating[i])
                    ++leftLess;
                else if (rating[j] > rating[i])
                    ++leftGreater;
            // Calculate soldiers on the right with less/greater ratings.
            int rightLess = 0;
            int rightGreater = 0;
            for (int j = i + 1; j < rating.length; ++j)
                if (rating[j] < rating[i])
                    ++rightLess;
                else if (rating[j] > rating[i])
                    ++rightGreater;
            ans += leftLess * rightGreater + leftGreater * rightLess;
        }

        return ans;
    }
}
```

408. 1396.java

Path: LeetCode-main/solutions/1396. Design Underground System/1396.java

```
class CheckIn {
    public String stationName;
    public int time;
    public CheckIn(String stationName, int time) {
        this.stationName = stationName;
        this.time = time;
    }
}

class CheckOut {
    public int numTrips;
    public int totalTime;
    public CheckOut(int numTrips, int totalTime) {
        this.numTrips = numTrips;
        this.totalTime = totalTime;
    }
}

class UndergroundSystem {
    public void checkIn(int id, String stationName, int t) {
        checkIns.put(id, new CheckIn(stationName, t));
    }

    public void checkOut(int id, String stationName, int t) {
        final CheckIn checkIn = checkIns.get(id);
        checkIns.remove(id);
        final String route = checkIn.stationName + "->" + stationName;
        checkOuts.putIfAbsent(route, new CheckOut(0, 0));
        ++checkOuts.get(route).numTrips;
        checkOuts.get(route).totalTime += t - checkIn.time;
    }

    public double getAverageTime(String startStation, String endStation) {
        final CheckOut checkOut = checkOuts.get(startStation + "->" + endStation);
        return checkOut.totalTime / (double) checkOut.numTrips;
    }

    private Map<Integer, CheckIn> checkIns = new HashMap<>(); // {id: (stationName, time)}
    private Map<String, CheckOut> checkOuts = new HashMap<>(); // {route: (numTrips, totalTime)}
}
```

409. 1397.java

Path: LeetCode-main/solutions/1397. Find All Good Strings/1397.java

```
class Solution {
    public int findGoodStrings(int n, String s1, String s2, String evil) {
        Integer[][][] mem = new Integer[n][evil.length()][2][2];
        // nextMatchedCount[i][j] := the number of next matched evil count, where
        // there're j matches with `evil` and the current letter is ('a' + j)
        Integer[][] nextMatchedCount = new Integer[evil.length()][26];
        return count(s1, s2, evil, 0, 0, true, true, getLPS(evil), nextMatchedCount, mem);
    }

    private static final int MOD = 1_000_000_007;

    // Returns the number of good strings for s[i..n), where there're j matches
    // with `evil`, `isS1Prefix` indicates if the current letter is tightly bound
    // for `s1` and `isS2Prefix` indicates if the current letter is tightly bound
    // for `s2`.
    private int count(final String s1, final String s2, final String evil, int i,
                      int matchedEvilCount, boolean isS1Prefix, boolean isS2Prefix, int[] evilLPS,
                      Integer[][] nextMatchedCount, Integer[][][] mem) {
        // s[0..i) contains `evil`, so don't consider any ongoing strings.
        if (matchedEvilCount == evil.length())
            return 0;
        // Run out of strings, so contribute one.
        if (i == s1.length())
            return 1;
        final int k1 = isS1Prefix ? 1 : 0;
        final int k2 = isS2Prefix ? 1 : 0;
        if (mem[i][matchedEvilCount][k1][k2] != null)
            return mem[i][matchedEvilCount][k1][k2];
        mem[i][matchedEvilCount][k1][k2] = 0;
        final char minChar = isS1Prefix ? s1.charAt(i) : 'a';
        final char maxChar = isS2Prefix ? s2.charAt(i) : 'z';
        for (char c = minChar; c <= maxChar; ++c) {
            final int nextMatchedEvilCount =
                getNextMatchedEvilCount(nextMatchedCount, evil, matchedEvilCount, c, evilLPS);
            mem[i][matchedEvilCount][k1][k2] +=
                count(s1, s2, evil, i + 1, nextMatchedEvilCount, isS1Prefix && c == s1.charAt(i),
                    isS2Prefix && c == s2.charAt(i), evilLPS, nextMatchedCount, mem);
            mem[i][matchedEvilCount][k1][k2] %= MOD;
        }
        return mem[i][matchedEvilCount][k1][k2];
    }

    // Returns the lps array, where lps[i] is the length of the longest prefix of
    // pattern[0..i] which is also a suffix of this substring.
    private int[] getLPS(final String pattern) {
        int[] lps = new int[pattern.length()];
        for (int i = 1, j = 0; i < pattern.length(); ++i) {
            while (j > 0 && pattern.charAt(j) != pattern.charAt(i))
                j = lps[j - 1];
            if (pattern.charAt(i) == pattern.charAt(j))
                lps[i] = ++j;
        }
        return lps;
    }

    // j := the next index we're trying to match with `currLetter`
    private int getNextMatchedEvilCount(Integer[][] nextMatchedCount, final String evil, int j,
                                       char currChar, int[] lps) {
        if (nextMatchedCount[j][currChar - 'a'] != null)
            return nextMatchedCount[j][currChar - 'a'];
        while (j > 0 && evil.charAt(j) != currChar)
            j = lps[j - 1];
        return nextMatchedCount[j][currChar - 'a'] = (evil.charAt(j) == currChar ? j + 1 : j);
    }
}
```

410. 1399.java

Path: LeetCode-main/solutions/1399. Count Largest Group/1399.java

```
class Solution {
    public int countLargestGroup(int n) {
        int[] count = new int[9 * 4 + 1];
        for (int i = 1; i <= n; ++i)
            ++count[getDigitSum(i)];
        final int mx = Arrays.stream(count).max().getAsInt();
        return (int) Arrays.stream(count).filter(c -> c == mx).count();
    }

    private int getDigitSum(int num) {
        int digitSum = 0;
        while (num > 0) {
            digitSum += num % 10;
            num /= 10;
        }
        return digitSum;
    }
}
```

411. 14.java

Path: LeetCode-main/solutions/14. Longest Common Prefix/14.java

```
class Solution {
    public String longestCommonPrefix(String[] strs) {
        if (strs.length == 0)
            return "";

        for (int i = 0; i < strs[0].length(); ++i)
            for (int j = 1; j < strs.length; ++j)
                if (i == strs[j].length() || strs[j].charAt(i) != strs[0].charAt(i))
                    return strs[0].substring(0, i);

        return strs[0];
    }
}
```

412. 140.java

Path: LeetCode-main/solutions/140. Word Break II/140.java

```
class Solution {
    public List<String> wordBreak(String s, List<String> wordDict) {
        Set<String> wordSet = new HashSet<>(wordDict);
        Map<String, List<String>> mem = new HashMap<>();
        return wordBreak(s, wordSet, mem);
    }

    private List<String> wordBreak(final String s, Set<String> wordSet,
                                   Map<String, List<String>> mem) {
        if (mem.containsKey(s))
            return mem.get(s);

        List<String> ans = new ArrayList<>();

        // 1 <= prefix.length() < s.length()
        for (int i = 1; i < s.length(); ++i) {
            final String prefix = s.substring(0, i);
            final String suffix = s.substring(i);
            if (wordSet.contains(prefix))
                for (final String word : wordBreak(suffix, wordSet, mem))
                    ans.add(prefix + " " + word);
        }

        // `wordSet` contains the whole string s, so don't add any space.
        if (wordSet.contains(s))
            ans.add(s);

        mem.put(s, ans);
        return ans;
    }
}
```

413. 1400.java

Path: LeetCode-main/solutions/1400. Construct K Palindrome Strings/1400.java

```
class Solution {
    public boolean canConstruct(String s, int k) {
        // If |s| < k, we cannot construct k strings from the s.
        if (s.length() < k)
            return false;

        int[] count = new int[26];

        for (final char c : s.toCharArray())
            count[c - 'a'] ^= 1;

        // If the number of letters that have odd counts > k, the minimum number of
        // palindromic strings we can construct is > k.
        return Arrays.stream(count).filter(c -> c % 2 == 1).count() <= k;
    }
}
```

414. 1402.java

Path: LeetCode-main/solutions/1402. Reducing Dishes/1402.java

```
class Solution {
    public int maxSatisfaction(int[] satisfaction) {
        int ans = 0;
        int sumSatisfaction = 0;

        satisfaction = Arrays.stream(satisfaction)
                               .boxed()
                               .sorted(Collections.reverseOrder())
                               .mapToInt(Integer::intValue)
                               .toArray();

        for (final int s : satisfaction) {
            sumSatisfaction += s;
            if (sumSatisfaction <= 0)
                return ans;
            ans += sumSatisfaction;
        }

        return ans;
    }
}
```


415. 1403.java

Path: LeetCode-main/solutions/1403. Minimum Subsequence in Non-Increasing Order/1403.java

```
class Solution {
    public List<Integer> minSubsequence(int[] nums) {
        List<Integer> ans = new ArrayList<>();
        Queue<Integer> maxHeap = new PriorityQueue<>(Collections.reverseOrder());
        for (final int num : nums)
            maxHeap.offer(num);
        int half = Arrays.stream(nums).sum() / 2;

        while (half >= 0) {
            ans.add(maxHeap.peek());
            half -= maxHeap.poll();
        }

        return ans;
    }
}
```

416. 1404.java

Path: LeetCode-main/solutions/1404. Number of Steps to Reduce a Number in Binary Representation to One/1404.java

```
class Solution {
    public int numSteps(String s) {
        int ans = 0;
        StringBuilder sb = new StringBuilder(s);

        // All the trailing 0s can be popped by 1 step.
        while (sb.charAt(sb.length() - 1) == '0') {
            sb.deleteCharAt(sb.length() - 1);
            ++ans;
        }

        if (sb.toString().equals("1"))
            return ans;

        // `s` is now odd, so add 1 to `s` and cost 1 step.
        ++ans;

        // All the 1s will become 0s and can be popped by 1 step.
        // All the 0s will become 1s and can be popped by 2 steps (adding 1 then
        // dividing by 2).
        for (final char c : sb.toString().toCharArray())
            ans += c == '1' ? 1 : 2;

        return ans;
    }
}
```

417. 1406-2.java

Path: LeetCode-main/solutions/1406. Stone Game III/1406-2.java

```
class Solution {
    public String stoneGameIII(int[] stoneValue) {
        final int n = stoneValue.length;
        // dp[i] := the maximum relative score Alice can make with stoneValue[i..n)
        int[] dp = new int[n + 1];
        Arrays.fill(dp, 0, n, Integer.MIN_VALUE / 2);

        for (int i = n - 1; i >= 0; --i) {
            int sum = 0;
            for (int j = i; j < i + 3 && j < n; ++j) {
                sum += stoneValue[j];
                dp[i] = Math.max(dp[i], sum - dp[j + 1]);
            }
        }

        final int score = dp[0];
        return score > 0 ? "Alice" : score < 0 ? "Bob" : "Tie";
    }
}
```

418. 1406.java

Path: LeetCode-main/solutions/1406. Stone Game III/1406.java

```
class Solution {
    public String stoneGameIII(int[] stoneValue) {
        int[] mem = new int[stoneValue.length];
        Arrays.fill(mem, Integer.MIN_VALUE);
        final int score = stoneGameIII(stoneValue, 0, mem);
        return score > 0 ? "Alice" : score < 0 ? "Bob" : "Tie";
    }

    // Returns the maximum relative score Alice can make from stoneValue[i..n).
    private int stoneGameIII(int[] stoneValue, int i, int[] mem) {
        if (i == stoneValue.length)
            return 0;
        if (mem[i] > Integer.MIN_VALUE)
            return mem[i];

        int sum = 0;
        for (int j = i; j < i + 3 && j < stoneValue.length; ++j) {
            sum += stoneValue[j];
            mem[i] = Math.max(mem[i], sum - stoneGameIII(stoneValue, j + 1, mem));
        }

        return mem[i];
    };
}
```

419. 1408-2.java

Path: LeetCode-main/solutions/1408. String Matching in an Array/1408-2.java

```
class TrieNode {
    public TrieNode[] children = new TrieNode[26];
}

class Trie {
    public void insert(final String word) {
        TrieNode node = root;
        for (final char c : word.toCharArray()) {
            final int i = c - 'a';
            if (node.children[i] == null)
                node.children[i] = new TrieNode();
            node = node.children[i];
        }
    }

    public boolean search(final String word) {
        TrieNode node = root;
        for (final char c : word.toCharArray()) {
            final int i = c - 'a';
            if (node.children[i] == null)
                return false;
            node = node.children[i];
        }
        return true;
    }

    private TrieNode root = new TrieNode();
}

class Solution {
    public List<String> stringMatching(String[] words) {
        List<String> ans = new ArrayList<>();
        Trie trie = new Trie();

        Arrays.sort(words, Comparator.comparingInt(String::length).reversed());

        for (final String word : words) {
            if (trie.search(word))
                ans.add(word);
            for (int i = 0; i < word.length(); ++i)
                trie.insert(word.substring(i));
        }

        return ans;
    }
}
```

420. 1408-3.java

Path: LeetCode-main/solutions/1408. String Matching in an Array/1408-3.java

```
class TrieNode {
    public TrieNode[] children = new TrieNode[26];
    public int count = 0;
}

class Trie {
    public void insert(final String word) {
        TrieNode node = root;
        for (final char c : word.toCharArray()) {
            final int i = c - 'a';
            if (node.children[i] == null)
                node.children[i] = new TrieNode();
            node = node.children[i];
            ++node.count;
        }
    }

    public boolean search(final String word) {
        TrieNode node = root;
        for (final char c : word.toCharArray()) {
            final int i = c - 'a';
            if (node.children[i] == null)
                return false;
            node = node.children[i];
        }
        return node.count > 1;
    }

    private TrieNode root = new TrieNode();
}

class Solution {
    public List<String> stringMatching(String[] words) {
        List<String> ans = new ArrayList<>();
        Trie trie = new Trie();

        for (final String word : words)
            for (int i = 0; i < word.length(); ++i)
                trie.insert(word.substring(i));

        for (final String word : words)
            if (trie.search(word))
                ans.add(word);

        return ans;
    }
}
```

421. 1408.java

Path: LeetCode-main/solutions/1408. String Matching in an Array/1408.java

```
class Solution {
    public List<String> stringMatching(String[] words) {
        List<String> ans = new ArrayList<>();
        for (final String a : words)
            for (final String b : words)
                if (a.length() < b.length() && b.indexOf(a) != -1) {
                    ans.add(a);
                    break;
                }
        return ans;
    }
}
```

422. 1409.java

Path: LeetCode-main/solutions/1409. Queries on a Permutation With Key/1409.java

```
class FenwickTree {
    public FenwickTree(int n) {
        sums = new int[n + 1];
    }

    public void add(int i, int delta) {
        while (i < sums.length) {
            sums[i] += delta;
            i += lowbit(i);
        }
    }

    public int get(int i) {
        int sum = 0;
        while (i > 0) {
            sum += sums[i];
            i -= lowbit(i);
        }
        return sum;
    }

    private int[] sums;

    private static int lowbit(int i) {
        return i & -i;
    }
}

class Solution {
    public int[] processQueries(int[] queries, int m) {
        int[] ans = new int[queries.length];
        // Map [-m, m] to [0, 2 * m].
        FenwickTree tree = new FenwickTree(2 * m + 1);
        Map<Integer, Integer> numToIndex = new HashMap<>();

        for (int num = 1; num <= m; ++num) {
            numToIndex.put(num, num + m);
            tree.add(num + m, 1);
        }

        int nextEmptyIndex = m; // Map 0 to m.

        for (int i = 0; i < queries.length; ++i) {
            final int query = queries[i];
            final int index = numToIndex.get(query);
            ans[i] = tree.get(index - 1);
            // Move `query` from `index` to `nextEmptyIndex`.
            tree.add(index, -1);
            tree.add(nextEmptyIndex, 1);
            numToIndex.put(query, nextEmptyIndex--);
        }

        return ans;
    }
}
```


423. 141.java

Path: LeetCode-main/solutions/141. Linked List Cycle/141.java

```
class Solution {
    public boolean hasCycle(ListNode head) {
        ListNode slow = head;
        ListNode fast = head;

        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
            if (slow == fast)
                return true;
        }

        return false;
    }
}
```

424. 1410.java

Path: LeetCode-main/solutions/1410. HTML Entity Parser/1410.java

```
class Solution {
    public String entityParser(String text) {
        Map<String, String> entryToChar =
            Map.of("&quot;", "\"", "&apos;", "'", "&gt;", ">", "&lt;", "<", "&frasl;", "/");

        for (Map.Entry<String, String> entry : entryToChar.entrySet()) {
            final String entity = entry.getKey();
            final String c = entry.getValue();
            text = text.replaceAll(entity, c);
        }

        // Process '&' in last.
        return text.replaceAll("&", "&");
    }
}
```

425. 1411.java

Path: LeetCode-main/solutions/1411. Number of Ways to Paint $N \times 3$ Grid/1411.java

```
class Solution {
    public int numOfWays(int n) {
        final int MOD = 1_000_000_007;
        long color2 = 6; // 121, 131, 212, 232, 313, 323
        long color3 = 6; // 123, 132, 213, 231, 312, 321

        for (int i = 1; i < n; ++i) {
            final long nextColor2 = color2 * 3 + color3 * 2;
            final long nextColor3 = color2 * 2 + color3 * 2;
            color2 = nextColor2 % MOD;
            color3 = nextColor3 % MOD;
        }

        return (int) ((color2 + color3) % MOD);
    }
}
```

426. 1413.java

Path: LeetCode-main/solutions/1413. Minimum Value to Get Positive Step by Step Sum/1413.java

```
class Solution {  
    public int minStartValue(int[] nums) {  
        int sum = 0;  
        int minSum = 0;  
  
        for (final int num : nums)  
            minSum = Math.min(minSum, sum += num);  
  
        return 1 - minSum;  
    }  
}
```

427. 1414-2.java

Path: LeetCode-main/solutions/1414. Find the Minimum Number of Fibonacci Numbers Whose Sum Is K/1414-2.java

```
class Solution {
    public int findMinFibonacciNumbers(int k) {
        int ans = 0;
        int a = 1; // F_1
        int b = 1; // F_2

        while (b <= k) {
            // a, b = F_{i + 1}, F_{i + 2}
            // -> a, b = F_{i + 2}, F_{i + 3}
            final int temp = a;
            a = b;
            b = a + temp;
        }

        while (a > 0) {
            if (a <= k) {
                k -= a;
                ++ans;
            }
            // a, b = F_{i + 2}, F_{i + 3}
            // -> a, b = F_{i + 1}, F_{i + 2}
            final int temp = a;
            a = b - a;
            b = temp;
        }

        return ans;
    }
}
```

428. 1414.java

Path: LeetCode-main/solutions/1414. Find the Minimum Number of Fibonacci Numbers Whose Sum Is K/1414.java

```
class Solution {
    public int findMinFibonacciNumbers(int k) {
        if (k < 2) // k == 0 || k == 1
            return k;

        int a = 1; // F_1
        int b = 1; // F_2

        while (b <= k) {
            // a, b = F_{i + 1}, F_{i + 2}
            // -> a, b = F_{i + 2}, F_{i + 3}
            final int temp = a;
            a = b;
            b = a + temp;
        }

        return 1 + findMinFibonacciNumbers(k - a);
    }
}
```

429. 1415.java

Path: LeetCode-main/solutions/1415. The k-th Lexicographical String of All Happy Strings of Length n/1415.java

```
class Solution {
    public String getHappyString(int n, int k) {
        Map<Character, String> nextLetters = Map.of('a', "bc", 'b', "ac", 'c', "ab");
        Queue<String> q = new ArrayDeque<>(List.of("a", "b", "c"));

        while (q.peek().length() != n) {
            final String u = q.poll();
            for (final char nextLetter : nextLetters.get(u.charAt(u.length() - 1)).toCharArray())
                q.offer(u + nextLetter);
        }

        if (q.size() < k)
            return "";

        for (int i = 0; i < k - 1; ++i)
            q.poll();
        return q.poll();
    }
}
```

430. 1417.java

Path: LeetCode-main/solutions/1417. Reformat The String/1417.java

```
class Solution {
    public String reformat(String s) {
        List<Character> A = new ArrayList<>();
        List<Character> B = new ArrayList<>();

        for (final char c : s.toCharArray())
            if (Character.isAlphabetic(c))
                A.add(c);
            else
                B.add(c);

        if (A.size() < B.size()) {
            List<Character> temp = A;
            A = B;
            B = temp;
        }
        if (A.size() - B.size() > 1)
            return "";

        StringBuilder sb = new StringBuilder();

        for (int i = 0; i < B.size(); ++i) {
            sb.append(A.get(i));
            sb.append(B.get(i));
        }

        if (A.size() > B.size())
            sb.append(A.get(A.size() - 1));
        return sb.toString();
    }
}
```


431. 1419.java

Path: LeetCode-main/solutions/1419. Minimum Number of Frogs Croaking/1419.java

```
class Solution {
    public int minNumberOfFrogs(String croakOfFrogs) {
        final String CROAK = "croak";
        int ans = 0;
        int frogs = 0;
        int[] count = new int[5];

        for (final char c : croakOfFrogs.toCharArray()) {
            ++count[CROAK.indexOf(c)];
            for (int i = 1; i < 5; ++i)
                if (count[i] > count[i - 1])
                    return -1;
            if (c == 'c')
                ++frogs;
            else if (c == 'k')
                --frogs;
            ans = Math.max(ans, frogs);
        }

        return frogs == 0 ? ans : -1;
    }
}
```

432. 142.java

Path: LeetCode-main/solutions/142. Linked List Cycle II/142.java

```
class Solution {
    public ListNode detectCycle(ListNode head) {
        ListNode slow = head;
        ListNode fast = head;

        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
            if (slow == fast) {
                slow = head;
                while (slow != fast) {
                    slow = slow.next;
                    fast = fast.next;
                }
                return slow;
            }
        }
        return null;
    }
}
```

433. 1420-2.java

Path: LeetCode-main/solutions/1420. Build Array Where You Can Find The Maximum Exactly K Comparisons/1420-2.java

```
class Solution {
    public int numOfArrays(int n, int m, int k) {
        final int MOD = 1_000_000_007;
        // dp[i][j][k] := the number of ways to build an array of length i, where j
        // is the maximum number and k is the `search_cost`
        int[][][] dp = new int[n + 1][m + 1][k + 1];
        // prefix[i][j][k] := sum(dp[i][x][k]), where 1 <= x <= j
        int[][][] prefix = new int[n + 1][m + 1][k + 1];

        for (int j = 1; j <= m; ++j) {
            dp[1][j][1] = 1;
            prefix[1][j][1] = j;
        }

        for (int i = 2; i <= n; ++i) // for each length
            for (int j = 1; j <= m; ++j) // for each max value
                for (int cost = 1; cost <= k; ++cost) { // for each cost
                    // 1. Appending any of [1, j] in the i-th position doesn't change the
                    //    maximum and cost.
                    // 2. Appending j in the i-th position makes j the new max and cost 1.
                    dp[i][j][cost] =
                        (int) (((long) j * dp[i - 1][j][cost] + prefix[i - 1][j - 1][cost - 1]) % MOD);
                    prefix[i][j][cost] = (dp[i][j][cost] + prefix[i][j - 1][cost]) % MOD;
                }

        int ans = 0;
        for (int j = 1; j <= m; ++j) {
            ans += dp[n][j][k];
            ans %= MOD;
        }
        return ans;
    }
}
```

434. 1420.java

Path: LeetCode-main/solutions/1420. Build Array Where You Can Find The Maximum Exactly K Comparisons/1420.java

```
class Solution {
    public int numOfArrays(int n, int m, int k) {
        final int MOD = 1_000_000_007;
        // dp[i][j][k] := the number of ways to build an array of length i, where j
        // is the maximum number and k is `search_cost`
        int[][][] dp = new int[n + 1][m + 1][k + 1];

        for (int j = 1; j <= m; ++j)
            dp[1][j][1] = 1;

        for (int i = 2; i <= n; ++i) // for each length
            for (int j = 1; j <= m; ++j) // for each max value
                for (int cost = 1; cost <= k; ++cost) { // for each cost
                    // 1. Appending any of [1, j] in the i-th position doesn't change the
                    //    maximum and cost.
                    dp[i][j][cost] = (int) ((long) j * dp[i - 1][j][cost] % MOD);
                    // 2. Appending j in the i-th position makes j the new max and cost 1.
                    for (int prevMax = 1; prevMax < j; ++prevMax) {
                        dp[i][j][cost] += dp[i - 1][prevMax][cost - 1];
                        dp[i][j][cost] %= MOD;
                    }
                }

        int ans = 0;
        for (int j = 1; j <= m; ++j) {
            ans += dp[n][j][k];
            ans %= MOD;
        }
        return ans;
    }
}
```

435. 1422.java

Path: LeetCode-main/solutions/1422. Maximum Score After Splitting a String/1422.java

```
class Solution {
    public int maxScore(String s) {
        int ans = 0;
        int zeros = 0;
        int ones = (int) s.chars().filter(c -> c == '1').count();

        for (int i = 0; i + 1 < s.length(); ++i) {
            if (s.charAt(i) == '0')
                ++zeros;
            else
                --ones;
            ans = Math.max(ans, zeros + ones);
        }

        return ans;
    }
}
```

436. 1423.java

Path: LeetCode-main/solutions/1423. Maximum Points You Can Obtain from Cards/1423.java

```
class Solution {
    public int maxScore(int[] cardPoints, int k) {
        final int n = cardPoints.length;
        final int sum = Arrays.stream(cardPoints).sum();
        int windowSum = 0;
        for (int i = 0; i < n - k; ++i)
            windowSum += cardPoints[i];

        int ans = sum - windowSum;

        for (int i = 0; i < k; ++i) {
            windowSum -= cardPoints[i];
            windowSum += cardPoints[i + n - k];
            ans = Math.max(ans, sum - windowSum);
        }

        return ans;
    }
}
```

437. 1424.java

Path: LeetCode-main/solutions/1424. Diagonal Traverse II/1424.java

```
class Solution {
    public int[] findDiagonalOrder(List<List<Integer>> nums) {
        List<Integer> ans = new ArrayList<>();
        Map<Integer, List<Integer>> keyToNums = new HashMap<>(); // key := row + column
        int maxKey = 0;

        for (int i = 0; i < nums.size(); ++i)
            for (int j = 0; j < nums.get(i).size(); ++j) {
                final int key = i + j;
                keyToNums.putIfAbsent(key, new ArrayList<>());
                keyToNums.get(key).add(nums.get(i).get(j));
                maxKey = Math.max(key, maxKey);
            }

        for (int i = 0; i <= maxKey; ++i)
            for (int j = keyToNums.get(i).size() - 1; j >= 0; --j)
                ans.add(keyToNums.get(i).get(j));

        return ans.stream().mapToInt(Integer::intValue).toArray();
    }
}
```

438. 1425.java

Path: LeetCode-main/solutions/1425. Constrained Subsequence Sum/1425.java

```
class Solution {
    public int constrainedSubsetSum(int[] nums, int k) {
        // dp[i] := the maximum the sum of non-empty subsequences in nums[0..i]
        int[] dp = new int[nums.length];
        // dq stores dp[i - k], dp[i - k + 1], ..., dp[i - 1] whose values are > 0
        // in decreasing order.
        Deque<Integer> dq = new ArrayDeque<>();

        for (int i = 0; i < nums.length; ++i) {
            if (dq.isEmpty())
                dp[i] = nums[i];
            else
                dp[i] = Math.max(dq.peekFirst(), 0) + nums[i];
            while (!dq.isEmpty() && dq.peekLast() < dp[i])
                dq.pollLast();
            dq.offerLast(dp[i]);
            if (i >= k && dp[i - k] == dq.peekFirst())
                dq.pollFirst();
        }

        return Arrays.stream(dp).max().getAsInt();
    }
}
```


439. 1426.java

Path: LeetCode-main/solutions/1426. Counting Elements/1426.java

```
class Solution {
    public int countElements(int[] arr) {
        int ans = 0;
        Map<Integer, Integer> count = new HashMap<>();

        for (final int a : arr)
            count.merge(a, 1, Integer::sum);

        for (Map.Entry<Integer, Integer> entry : count.entrySet())
            if (count.containsKey(entry.getKey() + 1))
                ans += entry.getValue();

        return ans;
    }
}
```

440. 1427.java

Path: LeetCode-main/solutions/1427. Perform String Shifts/1427.java

```
class Solution {
    public String stringShift(String s, int[][] shift) {
        final int n = s.length();
        int move = 0;

        for (int[] pair : shift) {
            final int direction = pair[0];
            final int amount = pair[1];
            if (direction == 0)
                move -= amount;
            else
                move += amount;
        }

        move = ((move % n) + n) % n;
        return s.substring(n - move) + s.substring(0, n - move);
    }
}
```

441. 1428-2.java

Path: LeetCode-main/solutions/1428. Leftmost Column with at Least a One/1428-2.java

```
/**
 * // This is the BinaryMatrix's API interface.
 * // You should not implement it, or speculate about its implementation
 * interface BinaryMatrix {
 *     public int get(int row, int col) {}
 *     public List<Integer> dimensions {}
 * };
 */

class Solution {
    public int leftMostColumnWithOne(BinaryMatrix binaryMatrix) {
        final List<Integer> dimensions = binaryMatrix.dimensions();
        final int m = dimensions.get(0);
        final int n = dimensions.get(1);
        int ans = -1;
        int i = 0;
        int j = n - 1;

        while (i < m && j >= 0)
            if (binaryMatrix.get(i, j) == 1)
                ans = j--;
            else
                ++i;

        return ans;
    }
}
```

442. 1428.java

Path: LeetCode-main/solutions/1428. Leftmost Column with at Least a One/1428.java

```
/**
 * // This is the BinaryMatrix's API interface.
 * // You should not implement it, or speculate about its implementation
 * interface BinaryMatrix {
 *   public int get(int row, int col) {}
 *   public List<Integer> dimensions {}
 * };
 */

class Solution {
    public int leftMostColumnWithOne(BinaryMatrix binaryMatrix) {
        final List<Integer> dimensions = binaryMatrix.dimensions();
        final int m = dimensions.get(0);
        final int n = dimensions.get(1);
        int ans = -1;
        int l = 0;
        int r = n - 1;

        while (l <= r) {
            final int mid = (l + r) / 2;
            if (existOne(binaryMatrix, m, mid)) {
                ans = mid;
                r = mid - 1;
            } else {
                l = mid + 1;
            }
        }

        return ans;
    }

    private boolean existOne(BinaryMatrix binaryMatrix, int m, int col) {
        for (int i = 0; i < m; ++i)
            if (binaryMatrix.get(i, col) == 1)
                return true;
        return false;
    }
}
```

443. 1429.java

Path: LeetCode-main/solutions/1429. First Unique Number/1429.java

```
class FirstUnique {
    public FirstUnique(int[] nums) {
        for (final int num : nums)
            add(num);
    }

    public int showFirstUnique() {
        return unique.isEmpty() ? -1 : unique.iterator().next();
    }

    public void add(int value) {
        if (!seen.contains(value)) {
            unique.add(value);
            seen.add(value);
        } else if (unique.contains(value)) {
            // We have added this value before, and this is the second time we're
            // adding it. So, erase the value from `unique`.
            unique.remove(value);
        }
    }

    private Set<Integer> seen = new HashSet<>();
    private Set<Integer> unique = new LinkedHashSet<>();
}
```

444. 143.java

Path: LeetCode-main/solutions/143. Reorder List/143.java

```
class Solution {
    public void reorderList(ListNode head) {
        if (head == null || head.next == null)
            return;

        ListNode mid = findMid(head);
        ListNode reversed = reverse(mid);
        merge(head, reversed);
    }

    private ListNode findMid(ListNode head) {
        ListNode prev = null;
        ListNode slow = head;
        ListNode fast = head;

        while (fast != null && fast.next != null) {
            prev = slow;
            slow = slow.next;
            fast = fast.next.next;
        }
        prev.next = null;

        return slow;
    }

    private ListNode reverse(ListNode head) {
        ListNode prev = null;
        ListNode curr = head;

        while (curr != null) {
            ListNode next = curr.next;
            curr.next = prev;
            prev = curr;
            curr = next;
        }

        return prev;
    }

    private void merge(ListNode l1, ListNode l2) {
        while (l2 != null) {
            ListNode next = l1.next;
            l1.next = l2;
            l1 = l2;
            l2 = next;
        }
    }
}
```

445. 1430.java

Path: LeetCode-main/solutions/1430. Check If a String Is a Valid Sequence from Root to Leaves Path in a Binary Tree/1430.java

```
class Solution {
    public boolean isValidSequence(TreeNode root, int[] arr) {
        return isValidSequence(root, arr, 0);
    }

    private boolean isValidSequence(TreeNode root, int[] arr, int i) {
        if (root == null)
            return false;
        if (i == arr.length - 1)
            return root.val == arr[i] && root.left == null && root.right == null;
        return root.val == arr[i] &&
            (isValidSequence(root.left, arr, i + 1) || isValidSequence(root.right, arr, i + 1));
    }
}
```

446. 1431.java

Path: LeetCode-main/solutions/1431. Kids With the Greatest Number of Candies/1431.java

```
class Solution {
    public List<Boolean> kidsWithCandies(int[] candies, int extraCandies) {
        List<Boolean> ans = new ArrayList<>();
        final int maxCandy = Arrays.stream(candies).max().getAsInt();

        for (final int candy : candies)
            ans.add(candy + extraCandies >= maxCandy);

        return ans;
    }
}
```


447. 1432.java

Path: LeetCode-main/solutions/1432. Max Difference You Can Get From Changing an Integer/1432.java

```
class Solution {
    public int maxDiff(int num) {
        final String s = String.valueOf(num);
        final int firstNot9 = firstNot(s, '9', '9');
        final int firstNot01 = firstNot(s, '0', '1');
        final String a = s.replace(s.charAt(firstNot9), '9');
        final String b = s.replace(s.charAt(firstNot01), firstNot01 == 0 ? '1' : '0');
        return Integer.parseInt(a) - Integer.parseInt(b);
    }

    private int firstNot(final String s, char a, char b) {
        for (int i = 0; i < s.length(); ++i)
            if (s.charAt(i) != a && s.charAt(i) != b)
                return i;
        return 0;
    }
}
```

448. 1433-2.java

Path: LeetCode-main/solutions/1433. Check If a String Can Break Another String/1433-2.java

```
class Solution {
    public boolean checkIfCanBreak(String s1, String s2) {
        int[] count = new int[26];

        for (final char c : s1.toCharArray())
            ++count[c - 'a'];

        for (final char c : s2.toCharArray())
            --count[c - 'a'];

        for (int i = 1; i < 26; ++i)
            count[i] += count[i - 1];

        return Arrays.stream(count).allMatch(c -> c <= 0) || Arrays.stream(count).allMatch(c -> c >= 0);
    }
}
```

449. 1433.java

Path: LeetCode-main/solutions/1433. Check If a String Can Break Another String/1433.java

```
class Solution {
    public boolean checkIfCanBreak(String s1, String s2) {
        int[] count1 = new int[26];
        int[] count2 = new int[26];

        for (final char c : s1.toCharArray())
            ++count1[c - 'a'];

        for (final char c : s2.toCharArray())
            ++count2[c - 'a'];

        return canBreak(count1, count2) || canBreak(count2, count1);
    }

    // Returns true if count1 can break count2.
    private boolean canBreak(int[] count1, int[] count2) {
        int diff = 0;
        for (int i = 0; i < 26; ++i) {
            diff += count2[i] - count1[i];
            // count2 is alphabetically greater than count1.
            if (diff < 0)
                return false;
        }
        return true;
    }
}
```

450. 1434-2.java

Path: LeetCode-main/solutions/1434. Number of Ways to Wear Different Hats to Each Other/1434-2.java

```
class Solution {
    public int numberWays(List<List<Integer>> hats) {
        final int MOD = 1_000_000_007;
        final int nHats = hats.size();
        final int nPeople = 40;
        final int nAssignments = 1 << nPeople;
        List<Integer>[] hatToPeople = new List[nHats + 1];
        // dp[i][j] := the number of ways to assign hats 1, 2, ..., i to people,
        // where j is the bitmask of the current assignment
        int[][] dp = new int[nHats + 1][nAssignments];
        dp[0][0] = 1;

        for (int i = 1; i <= nHats; ++i)
            hatToPeople[i] = new ArrayList<>();

        for (int i = 0; i < nPeople; ++i)
            for (final int hat : hats.get(i))
                hatToPeople[hat].add(i);

        for (int h = 1; h <= nHats; ++h)
            for (int j = 0; j < nAssignments; ++j) {
                // We can cover the assignment j in 2 ways:
                // 1. Assign the first h - 1 hats to people without using the hat `h`.
                dp[h][j] += dp[h - 1][j];
                dp[h][j] %= MOD;
                for (final int p : hatToPeople[h])
                    if ((j >> p & 1) == 1) {
                        // 2. Assign the first h - 1 hats to people without the person `p`
                        // and assign the hat `h` to the person `p`.
                        dp[h][j] += dp[h - 1][j ^ 1 << p];
                        dp[h][j] %= MOD;
                    }
            }

        return dp[nHats][nAssignments - 1];
    }
}
```

451. 1434-3.java

Path: LeetCode-main/solutions/1434. Number of Ways to Wear Different Hats to Each Other/1434-3.java

```
class Solution {
    public int numberWays(List<List<Integer>> hats) {
        final int MOD = 1_000_000_007;
        final int nHats = hats.size();
        final int nPeople = hats.get(0).size();
        final int nAssignments = 1 << nPeople;
        List<Integer>[] hatToPeople = new List[nHats + 1];
        // dp[i] := the number of ways to assign the hats so far to people, where i
        // is the bitmask of the current assignment
        int[] dp = new int[nAssignments];
        dp[0] = 1;

        for (int i = 1; i <= nHats; ++i)
            hatToPeople[i] = new ArrayList<>();

        for (int i = 0; i < nPeople; ++i)
            for (final int hat : hats.get(i))
                hatToPeople[hat].add(i);

        for (int h = 1; h <= nHats; ++h)
            for (int j = nAssignments - 1; j >= 0; --j)
                for (final int p : hatToPeople[h])
                    if ((j >> p & 1) == 1) {
                        dp[j] += dp[j ^ 1 << p];
                        dp[j] %= MOD;
                    }

        return dp[nAssignments - 1];
    }
}
```

452. 1434.java

Path: LeetCode-main/solutions/1434. Number of Ways to Wear Different Hats to Each Other/1434.java

```
class Solution {
    public int numberWays(List<List<Integer>> hats) {
        final int nHats = hats.size();
        final int nPeople = hats.size();
        List<Integer>[] hatToPeople = new List[nHats + 1];
        Integer[][] mem = new Integer[nHats + 1][1 << nPeople];

        for (int i = 1; i <= nHats; ++i)
            hatToPeople[i] = new ArrayList<>();

        for (int i = 0; i < nPeople; ++i)
            for (final int hat : hats.get(i))
                hatToPeople[hat].add(i);

        return numberWays(hats, 0, 1, hatToPeople, mem);
    }

    private static final int MOD = 1_000_000_007;

    // Returns the number of ways to assign hats 1, 2, ..., h to people, where
    // `assignment` is the bitmask of the current assignment.
    private int numberWays(List<List<Integer>> hats, int assignment, int h,
        List<Integer>[] hatToPeople, Integer[][] mem) {
        // All the people are assigned.
        if (assignment == (1 << hats.size()) - 1)
            return 1;
        if (h > 40)
            return 0;
        if (mem[h][assignment] != null)
            return mem[h][assignment];

        // Don't wear the hat `h`.
        int ans = numberWays(hats, assignment, h + 1, hatToPeople, mem);

        for (final int p : hatToPeople[h]) {
            // The person `p` was assigned the hat `h` before.
            if ((assignment >> p & 1) == 1)
                continue;
            // Assign the hat `h` to the person `p`.
            ans += numberWays(hats, assignment | 1 << p, h + 1, hatToPeople, mem);
            ans %= MOD;
        }

        return mem[h][assignment] = ans;
    }
}
```

453. 1439.java

Path: LeetCode-main/solutions/1439. Find the Kth Smallest Sum of a Matrix With Sorted Rows/1439.java

```
class Solution {
    public int kthSmallest(int[][] mat, int k) {
        int[] row = mat[0];

        for (int i = 1; i < mat.length; ++i)
            row = kSmallestPairSums(row, mat[i], k);

        return row[k - 1];
    }

    private record T(int i, int j, int sum) {}

    // Similar to 373. Find K Pairs with Smallest Sums
    private int[] kSmallestPairSums(int[] nums1, int[] nums2, int k) {
        List<Integer> ans = new ArrayList<>();
        Queue<T> minHeap = new PriorityQueue<>(Comparator.comparingInt(T::sum));

        for (int i = 0; i < k && i < nums1.length; ++i)
            minHeap.offer(new T(i, 0, nums1[i] + nums2[0]));

        while (!minHeap.isEmpty() && ans.size() < k) {
            final int i = minHeap.peek().i;
            final int j = minHeap.poll().j;
            ans.add(nums1[i] + nums2[j]);
            if (j + 1 < nums2.length)
                minHeap.offer(new T(i, j + 1, nums1[i] + nums2[j + 1]));
        }

        return ans.stream().mapToInt(Integer::intValue).toArray();
    }
}
```

454. 144-2.java

Path: LeetCode-main/solutions/144. Binary Tree Preorder Traversal/144-2.java

```
class Solution {
    public List<Integer> preorderTraversal(TreeNode root) {
        if (root == null)
            return new ArrayList<>();

        List<Integer> ans = new ArrayList<>();
        Deque<TreeNode> stack = new ArrayDeque<>();
        stack.push(root);

        while (!stack.isEmpty()) {
            root = stack.pop();
            ans.add(root.val);
            if (root.right != null)
                stack.push(root.right);
            if (root.left != null)
                stack.push(root.left);
        }

        return ans;
    }
}
```


455. 144.java

Path: LeetCode-main/solutions/144. Binary Tree Preorder Traversal/144.java

```
class Solution {
    public List<Integer> preorderTraversal(TreeNode root) {
        List<Integer> ans = new ArrayList<>();
        preorder(root, ans);
        return ans;
    }

    private void preorder(TreeNode root, List<Integer> ans) {
        if (root == null)
            return;

        ans.add(root.val);
        preorder(root.left, ans);
        preorder(root.right, ans);
    }
}
```

456. 1443.java

Path: LeetCode-main/solutions/1443. Minimum Time to Collect All Apples in a Tree/1443.java

```
class Solution {
    public int minTime(int n, int[][] edges, List<Boolean> hasApple) {
        List<Integer>[] graph = new List[n];
        Arrays.setAll(graph, i -> new ArrayList<>());

        for (int[] edge : edges) {
            final int u = edge[0];
            final int v = edge[1];
            graph[u].add(v);
            graph[v].add(u);
        }

        return dfs(graph, 0, new boolean[n], hasApple);
    }

    private int dfs(List<Integer>[] graph, int u, boolean[] seen, List<Boolean> hasApple) {
        seen[u] = true;
        int totalCost = 0;

        for (final int v : graph[u]) {
            if (seen[v])
                continue;
            final int cost = dfs(graph, v, seen, hasApple);
            if (cost > 0 || hasApple.get(v))
                totalCost += cost + 2;
        }

        return totalCost;
    }
}
```

457. 1444.java

Path: LeetCode-main/solutions/1444. Number of Ways of Cutting a Pizza/1444.java

```
class Solution {
    public int ways(String[] pizza, int k) {
        final int M = pizza.length;
        final int N = pizza[0].length();
        int[][][] mem = new int[M][N][k];
        int[][] prefix = new int[M + 1][N + 1];

        Arrays.stream(mem).forEach(A -> Arrays.stream(A).forEach(B -> Arrays.fill(B, -1)));

        for (int i = 0; i < M; ++i)
            for (int j = 0; j < N; ++j)
                prefix[i + 1][j + 1] = (pizza[i].charAt(j) == 'A' ? 1 : 0) + prefix[i][j + 1] +
                    prefix[i + 1][j] - prefix[i][j];

        return ways(0, 0, k - 1, M, N, prefix, mem);
    }

    private static final int MOD = 1_000_000_007;

    // Returns the number of ways to cut pizza[m..M][n..N) with k cuts.
    private int ways(int m, int n, int k, int M, int N, int[][] prefix, int[][][] mem) {
        if (k == 0)
            return hasApple(prefix, m, M, n, N) ? 1 : 0;
        if (mem[m][n][k] != -1)
            return mem[m][n][k];

        mem[m][n][k] = 0;

        for (int i = m + 1; i < M; ++i) // Cut horizontally.
            if (hasApple(prefix, m, i, n, N) && hasApple(prefix, i, M, n, N)) {
                mem[m][n][k] += ways(i, n, k - 1, M, N, prefix, mem);
                mem[m][n][k] %= MOD;
            }

        for (int j = n + 1; j < N; ++j) // Cut vertically.
            if (hasApple(prefix, m, M, n, j) && hasApple(prefix, m, M, j, N)) {
                mem[m][n][k] += ways(m, j, k - 1, M, N, prefix, mem);
                mem[m][n][k] %= MOD;
            }

        return mem[m][n][k];
    }

    // Returns true if pizza[row1..row2)[col1..col2) has apple.
    private boolean hasApple(int[][] prefix, int row1, int row2, int col1, int col2) {
        return (prefix[row2][col2] - prefix[row1][col2] - //
            prefix[row2][col1] + prefix[row1][col1]) > 0;
    }
}
```

458. 1446.java

Path: LeetCode-main/solutions/1446. Consecutive Characters/1446.java

```
class Solution {  
    public int maxPower(String s) {  
        int ans = 1;  
        int count = 1;  
  
        for (int i = 1; i < s.length(); ++i) {  
            count = s.charAt(i) == s.charAt(i - 1) ? count + 1 : 1;  
            ans = Math.max(ans, count);  
        }  
  
        return ans;  
    }  
}
```

459. 1447.java

Path: LeetCode-main/solutions/1447. Simplified Fractions/1447.java

```
class Solution {
    public List<String> simplifiedFractions(int n) {
        List<String> ans = new ArrayList<>();
        for (int denominator = 2; denominator <= n; ++denominator)
            for (int numerator = 1; numerator < denominator; ++numerator)
                if (gcd(denominator, numerator) == 1)
                    ans.add(String.valueOf(numerator) + "/" + String.valueOf(denominator));
        return ans;
    }

    private int gcd(int a, int b) {
        return b == 0 ? a : gcd(b, a % b);
    }
}
```

460. 1448.java

Path: LeetCode-main/solutions/1448. Count Good Nodes in Binary Tree/1448.java

```
class Solution {
    public int goodNodes(TreeNode root) {
        return goodNodes(root, Integer.MIN_VALUE);
    }

    private int goodNodes(TreeNode root, int mx) {
        if (root == null)
            return 0;

        final int newMax = Math.max(mx, root.val);
        return (root.val >= mx ? 1 : 0) + goodNodes(root.left, newMax) + goodNodes(root.right, newMax);
    }
}
```

461. 1449.java

Path: LeetCode-main/solutions/1449. Form Largest Integer With Digits That Add up to Target/1449.java

```
class Solution {
    public String largestNumber(int[] cost, int target) {
        // dp[i] := the maximum length that cost i can achieve
        int[] dp = new int[target + 1];
        Arrays.fill(dp, Integer.MIN_VALUE);
        dp[0] = 0; // If cost = 0, the best choice is the empty string "".

        for (int i = 1; i <= target; ++i)
            for (int d = 0; d < 9; ++d)
                if (i >= cost[d])
                    dp[i] = Math.max(dp[i], dp[i - cost[d]] + 1);

        if (dp[target] < 0)
            return "0";

        StringBuilder sb = new StringBuilder();

        // Greedily build the ans string.
        for (int d = 8; d >= 0; --d)
            while (target >= cost[d] && dp[target] == dp[target - cost[d]] + 1) {
                target -= cost[d];
                sb.append(1 + d);
            }

        return sb.toString();
    }
}
```

462. 145-2.java

Path: LeetCode-main/solutions/145. Binary Tree Postorder Traversal/145-2.java

```
class Solution {
    public List<Integer> postorderTraversal(TreeNode root) {
        if (root == null)
            return new ArrayList<>();

        List<Integer> ans = new ArrayList<>();
        Deque<TreeNode> stack = new ArrayDeque<>();
        stack.push(root);

        while (!stack.isEmpty()) {
            root = stack.pop();
            ans.add(root.val);
            if (root.left != null)
                stack.push(root.left);
            if (root.right != null)
                stack.push(root.right);
        }

        Collections.reverse(ans);
        return ans;
    }
}
```


463. 145.java

Path: LeetCode-main/solutions/145. Binary Tree Postorder Traversal/145.java

```
class Solution {
    public List<Integer> postorderTraversal(TreeNode root) {
        List<Integer> ans = new ArrayList<>();
        postorder(root, ans);
        return ans;
    }

    private void postorder(TreeNode root, List<Integer> ans) {
        if (root == null)
            return;

        postorder(root.left, ans);
        postorder(root.right, ans);
        ans.add(root.val);
    }
}
```

464. 1453.java

Path: LeetCode-main/solutions/1453. Maximum Number of Darts Inside of a Circular Dartboard/1453.java

```
class Point {
    public double x = 0;
    public double y = 0;
    public Point(double x, double y) {
        this.x = x;
        this.y = y;
    }
}

class Solution {
    public int numPoints(int[][] darts, int r) {
        int ans = 1;
        List<Point> points = convertToPoints(darts);

        for (int i = 0; i < points.size(); ++i)
            for (int j = i + 1; j < points.size(); ++j)
                for (Point c : getCircles(points.get(i), points.get(j), r)) {
                    int count = 0;
                    for (Point point : points)
                        if (dist(c, point) - r <= ERR)
                            ++count;
                    ans = Math.max(ans, count);
                }

        return ans;
    }

    private static final double ERR = 1e-6;

    private List<Point> convertToPoints(int[][] darts) {
        List<Point> points = new ArrayList<>();
        for (int[] dart : darts)
            points.add(new Point(dart[0], dart[1]));
        return points;
    }

    private Point[] getCircles(Point p, Point q, int r) {
        if (dist(p, q) - 2.0 * r > ERR)
            return new Point[] {};
        Point m = new Point((p.x + q.x) / 2, (p.y + q.y) / 2);
        final double distCM = Math.sqrt(Math.pow(r, 2) - Math.pow(dist(p, q) / 2, 2));
        final double alpha = Math.atan2(p.y - q.y, q.x - p.x);
        return new Point[] {new Point(m.x - distCM * Math.sin(alpha), m.y - distCM * Math.cos(alpha)),
                             new Point(m.x + distCM * Math.sin(alpha), m.y + distCM * Math.cos(alpha))};
    }

    private double dist(Point p, Point q) {
        return Math.sqrt(Math.pow(p.x - q.x, 2) + Math.pow(p.y - q.y, 2));
    }
}
```

465. 1457.java

Path: LeetCode-main/solutions/1457. Pseudo-Palindromic Paths in a Binary Tree/1457.java

```
class Solution {
    public int pseudoPalindromicPaths(TreeNode root) {
        dfs(root, 0);
        return ans;
    }

    private int ans = 0;

    private void dfs(TreeNode root, int path) {
        if (root == null)
            return;
        if (root.left == null && root.right == null) {
            path ^= 1 << root.val;
            if ((path & (path - 1)) == 0)
                ++ans;
            return;
        }

        dfs(root.left, path ^ 1 << root.val);
        dfs(root.right, path ^ 1 << root.val);
    }
}
```

466. 1458.java

Path: LeetCode-main/solutions/1458. Max Dot Product of Two Subsequences/1458.java

```
class Solution {
    public int maxDotProduct(int[] nums1, int[] nums2) {
        final int m = nums1.length;
        final int n = nums2.length;
        // dp[i][j] := the maximum dot product of the two subsequences nums[0..i)
        // and nums2[0..j)
        int[][] dp = new int[m + 1][n + 1];
        Arrays.stream(dp).forEach(A -> Arrays.fill(A, Integer.MIN_VALUE));

        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j)
                dp[i + 1][j + 1] = Math.max(Math.max(dp[i][j + 1], dp[i + 1][j]),
                                             Math.max(0, dp[i][j]) + nums1[i] * nums2[j]);

        return dp[m][n];
    }
}
```

467. 146.java

Path: LeetCode-main/solutions/146. LRU Cache/146.java

```
class Node {
    public int key;
    public int value;
    public Node(int key, int value) {
        this.key = key;
        this.value = value;
    }
}

class LRUCache {
    public LRUCache(int capacity) {
        this.capacity = capacity;
    }

    public int get(int key) {
        if (!keyToNode.containsKey(key))
            return -1;

        Node node = keyToNode.get(key);
        cache.remove(node);
        cache.add(node);
        return node.value;
    }

    public void put(int key, int value) {
        if (keyToNode.containsKey(key)) {
            keyToNode.get(key).value = value;
            get(key);
            return;
        }

        if (cache.size() == capacity) {
            Node lastNode = cache.iterator().next();
            cache.remove(lastNode);
            keyToNode.remove(lastNode.key);
        }

        Node node = new Node(key, value);
        cache.add(node);
        keyToNode.put(key, node);
    }

    private int capacity;
    private Set<Node> cache = new LinkedHashSet<>();
    private Map<Integer, Node> keyToNode = new HashMap<>();
}
```

468. 1460.java

Path: LeetCode-main/solutions/1460. Make Two Arrays Equal by Reversing Subarrays/1460.java

```
class Solution {
    public boolean canBeEqual(int[] target, int[] arr) {
        return Arrays.stream(arr)
            .boxed()
            .collect(Collectors.groupingBy(Integer::intValue, Collectors.counting()))
            .equals(Arrays.stream(target).boxed().collect(
                Collectors.groupingBy(Integer::intValue, Collectors.counting())));
    }
}
```

469. 1461.java

Path: LeetCode-main/solutions/1461. Check If a String Contains All Binary Codes of Size K/1461.java

```
class Solution {
    public boolean hasAllCodes(String s, int k) {
        final int n = 1 << k;
        if (s.length() < n)
            return false;

        // used[i] := true if i is a substring of `s`
        boolean[] used = new boolean[n];

        int window = k == 1 ? 0 : Integer.parseInt(s.substring(0, k - 1), 2);
        for (int i = k - 1; i < s.length(); ++i) {
            // Include the s[i].
            window = (window << 1) + (s.charAt(i) - '0');
            // Discard the s[i - k].
            window &= n - 1;
            used[window] = true;
        }

        return IntStream.range(0, used.length).mapToObj(i -> used[i]).allMatch(u -> u);
    }
}
```

470. 1462-2.java

Path: LeetCode-main/solutions/1462. Course Schedule IV/1462-2.java

```
class Solution {
    public List<Boolean> checkIfPrerequisite(int numCourses, int[][] prerequisites, int[][] queries) {
        List<Boolean> ans = new ArrayList<>();
        // isPrerequisite[i][j] := true if course i is a prerequisite of course j
        boolean[][] isPrerequisite = new boolean[numCourses][numCourses];

        for (int[] prerequisite : prerequisites) {
            final int u = prerequisite[0];
            final int v = prerequisite[1];
            isPrerequisite[u][v] = true;
        }

        for (int k = 0; k < numCourses; ++k)
            for (int i = 0; i < numCourses; ++i)
                for (int j = 0; j < numCourses; ++j)
                    isPrerequisite[i][j] =
                        isPrerequisite[i][k] || (isPrerequisite[i][k] && isPrerequisite[k][j]);

        for (int[] query : queries) {
            final int u = query[0];
            final int v = query[1];
            ans.add(isPrerequisite[u][v]);
        }

        return ans;
    }
}
```


471. 1462.java

Path: LeetCode-main/solutions/1462. Course Schedule IV/1462.java

```
class Solution {
    public List<Boolean> checkIfPrerequisite(int numCourses, int[][] prerequisites, int[][] queries) {
        List<Boolean> ans = new ArrayList<>();
        List<Integer>[] graph = new List[numCourses];

        for (int i = 0; i < numCourses; ++i)
            graph[i] = new ArrayList<>();

        for (int[] prerequisite : prerequisites) {
            final int u = prerequisite[0];
            final int v = prerequisite[1];
            graph[u].add(v);
        }

        // isPrerequisite[i][j] := true if course i is a prerequisite of course j
        boolean[][] isPrerequisite = new boolean[numCourses][numCourses];

        // DFS from every course.
        for (int i = 0; i < numCourses; ++i)
            dfs(graph, i, isPrerequisite[i]);

        for (int[] query : queries) {
            final int u = query[0];
            final int v = query[1];
            ans.add(isPrerequisite[u][v]);
        }

        return ans;
    }

    public void dfs(List<Integer>[] graph, int u, boolean[] used) {
        for (final int v : graph[u]) {
            if (used[v])
                continue;
            used[v] = true;
            dfs(graph, v, used);
        }
    }
}
```

472. 1463-2.java

Path: LeetCode-main/solutions/1463. Cherry Pickup II/1463-2.java

```
class Solution {
    public int cherryPickup(int[][] grid) {
        final int m = grid.length;
        final int n = grid[0].length;
        // dp[x][y1][y2] := the maximum cherries we can collect, where the robot #1
        // is on (x, y1) and the robot #2 is on (x, y2)
        int[][][] dp = new int[m + 1][n][n];

        for (int x = m - 1; x >= 0; --x)
            for (int y1 = 0; y1 < n; ++y1)
                for (int y2 = 0; y2 < n; ++y2) {
                    final int currRow = grid[x][y1] + (y1 == y2 ? 0 : 1) * grid[x][y2];
                    for (int d1 = Math.max(0, y1 - 1); d1 < Math.min(n, y1 + 2); ++d1)
                        for (int d2 = Math.max(0, y2 - 1); d2 < Math.min(n, y2 + 2); ++d2)
                            dp[x][y1][y2] = Math.max(dp[x][y1][y2], currRow + dp[x + 1][d1][d2]);
                }

        return dp[0][0][n - 1];
    }
}
```

473. 1463.java

Path: LeetCode-main/solutions/1463. Cherry Pickup II/1463.java

```
class Solution {
    public int cherryPickup(int[][] grid) {
        final int m = grid.length;
        final int n = grid[0].length;
        int[][][] mem = new int[m][n][n];
        Arrays.stream(mem).forEach(A -> Arrays.stream(A).forEach(B -> Arrays.fill(B, -1)));
        return dp(grid, 0, 0, n - 1, mem);
    }

    // Returns the maximum cherries we can collect, where robot #1 is on (x, y1)
    // and robot #2 is on (x, y2).
    private int dp(int[][] grid, int x, int y1, int y2, int[][][] mem) {
        if (x == grid.length)
            return 0;
        if (y1 < 0 || y1 == grid[0].length || y2 < 0 || y2 == grid[0].length)
            return 0;
        if (mem[x][y1][y2] != -1)
            return mem[x][y1][y2];

        final int currRow = grid[x][y1] + (y1 == y2 ? 0 : grid[x][y2]);

        for (int d1 = -1; d1 <= 1; ++d1)
            for (int d2 = -1; d2 <= 1; ++d2)
                mem[x][y1][y2] = Math.max(mem[x][y1][y2], currRow + dp(grid, x + 1, y1 + d1, y2 + d2, mem));

        return mem[x][y1][y2];
    }
}
```

474. 1464.java

Path: LeetCode-main/solutions/1464. Maximum Product of Two Elements in an Array/1464.java

```
class Solution {
    public int maxProduct(int[] nums) {
        int max1 = 0;
        int max2 = 0;

        for (final int num : nums)
            if (num > max1) {
                max2 = max1;
                max1 = num;
            } else if (num > max2) {
                max2 = num;
            }

        return (max1 - 1) * (max2 - 1);
    }
}
```

475. 1465.java

Path: LeetCode-main/solutions/1465. Maximum Area of a Piece of Cake After Horizontal and Vertical Cuts/1465.java

```
class Solution {
    public int maxArea(int h, int w, int[] horizontalCuts, int[] verticalCuts) {
        final int MOD = 1_000_000_007;
        Arrays.sort(horizontalCuts);
        Arrays.sort(verticalCuts);

        // the maximum gap of each direction
        int maxGapX = Math.max(verticalCuts[0], w - verticalCuts[verticalCuts.length - 1]);
        int maxGapY = Math.max(horizontalCuts[0], h - horizontalCuts[horizontalCuts.length - 1]);

        for (int i = 1; i < verticalCuts.length; ++i)
            maxGapX = Math.max(maxGapX, verticalCuts[i] - verticalCuts[i - 1]);

        for (int i = 1; i < horizontalCuts.length; ++i)
            maxGapY = Math.max(maxGapY, horizontalCuts[i] - horizontalCuts[i - 1]);

        return (int) ((long) maxGapX * maxGapY % MOD);
    }
}
```

476. 1466.java

Path: LeetCode-main/solutions/1466. Reorder Routes to Make All Paths Lead to the City Zero/1466.java

```
class Solution {
    public int minReorder(int n, int[][] connections) {
        List<Integer>[] graph = new List[n];
        Arrays.setAll(graph, i -> new ArrayList<>());

        for (int[] connection : connections) {
            final int u = connection[0];
            final int v = connection[1];
            graph[u].add(v);
            graph[v].add(-u); // - := u -> v
        }

        return dfs(graph, 0, -1);
    }

    private int dfs(List<Integer>[] graph, int u, int prev) {
        int change = 0;

        for (final int v : graph[u]) {
            if (Math.abs(v) == prev)
                continue;
            if (v > 0)
                ++change;
            change += dfs(graph, Math.abs(v), u);
        }

        return change;
    }
}
```

477. 1467.java

Path: LeetCode-main/solutions/1467. Probability of a Two Boxes Having The Same Number of Distinct Balls/1467.java

```
enum BoxCase { EQUAL_BALLS, EQUAL_DISTANT_BALLS }

class Solution {
    public double getProbability(int[] balls) {
        final int n = Arrays.stream(balls).sum() / 2;
        return cases(balls, 0, 0, 0, 0, 0, n, BoxCase.EQUAL_DISTANT_BALLS) /
            cases(balls, 0, 0, 0, 0, 0, n, BoxCase.EQUAL_BALLS);
    }

    private int[] fact = {1, 1, 2, 6, 24, 120, 720};

    // Assume we have two boxes A and B.
    double cases(int[] balls, int i, int ballsCountA, int ballsCountB, int colorsCountA,
        int colorsCountB, int n, BoxCase boxCase) {
        if (ballsCountA > n || ballsCountB > n)
            return 0;
        if (i == balls.length)
            return boxCase == BoxCase.EQUAL_BALLS ? 1 : (colorsCountA == colorsCountB ? 1 : 0);

        double ans = 0;

        // balls taken from A for `balls[i]`
        for (int ballsTakenA = 0; ballsTakenA <= balls[i]; ++ballsTakenA) {
            final int ballsTakenB = balls[i] - ballsTakenA;
            final int newcolorsCountA = colorsCountA + (ballsTakenA > 0 ? 1 : 0);
            final int newcolorsCountB = colorsCountB + (ballsTakenB > 0 ? 1 : 0);
            ans += cases(balls, i + 1, ballsCountA + ballsTakenA, ballsCountB + ballsTakenB,
                newcolorsCountA, newcolorsCountB, n, boxCase) /
                (fact[ballsTakenA] * fact[ballsTakenB]);
        }

        return ans;
    }
}
```

478. 1469.java

Path: LeetCode-main/solutions/1469. Find All The Lonely Nodes/1469.java

```
class Solution {
    public List<Integer> getLonelyNodes(TreeNode root) {
        List<Integer> ans = new ArrayList<>();

        dfs(root, false, ans);

        return ans;
    }

    private void dfs(TreeNode root, boolean isLonely, List<Integer> ans) {
        if (root == null)
            return;
        if (isLonely)
            ans.add(root.val);

        dfs(root.left, root.right == null, ans);
        dfs(root.right, root.left == null, ans);
    }
}
```


479. 147.java

Path: LeetCode-main/solutions/147. Insertion Sort List/147.java

```
class Solution {
    public ListNode insertionSortList(ListNode head) {
        ListNode dummy = new ListNode(0);
        ListNode prev = dummy; // the last and thus largest of the sorted list

        while (head != null) { // the current inserting node
            ListNode next = head.next; // Cache the next inserting node.
            if (prev.val >= head.val)
                prev = dummy; // Move `prev` to the front.
            while (prev.next != null && prev.next.val < head.val)
                prev = prev.next;
            head.next = prev.next;
            prev.next = head;
            head = next; // Update the current inserting node.
        }

        return dummy.next;
    }
}
```

480. 1470.java

Path: LeetCode-main/solutions/1470. Shuffle the Array/1470.java

```
class Solution {
    public int[] shuffle(int[] nums, int n) {
        int[] ans = new int[2 * n];
        for (int i = 0; i < n; ++i) {
            ans[i * 2] = nums[i];
            ans[i * 2 + 1] = nums[i + n];
        }
        return ans;
    }
}
```

481. 1471.java

Path: LeetCode-main/solutions/1471. The k Strongest Values in an Array/1471.java

```
class Solution {
    public int[] getStrongest(int[] arr, int k) {
        Arrays.sort(arr);

        final int n = arr.length;
        final int median = arr[(n - 1) / 2];
        int[] ans = new int[k];

        for (int i = 0, l = 0, r = n - 1; i < k; ++i)
            if (median - arr[l] > arr[r] - median)
                ans[i] = arr[l++];
            else
                ans[i] = arr[r--];

        return ans;
    }
}
```

482. 1472-2.java

Path: LeetCode-main/solutions/1472. Design Browser History/1472-2.java

```
class BrowserHistory {
    public BrowserHistory(String homepage) {
        visit(homepage);
    }

    public void visit(String url) {
        history.push(url);
        future = new ArrayDeque<>();
    }

    public String back(int steps) {
        while (history.size() > 1 && steps-- > 0)
            future.push(history.pop());
        return history.peek();
    }

    public String forward(int steps) {
        while (!future.isEmpty() && steps-- > 0)
            history.push(future.pop());
        return history.peek();
    }

    private Deque<String> history = new ArrayDeque<>();
    private Deque<String> future;
}
```

483. 1472-3.java

Path: LeetCode-main/solutions/1472. Design Browser History/1472-3.java

```
class Node {
    public Node prev;
    public Node next;
    public final String url;
    public Node(final String url) {
        this.url = url;
    }
}

class BrowserHistory {
    public BrowserHistory(String homepage) {
        curr = new Node(homepage);
    }

    public void visit(String url) {
        curr.next = new Node(url);
        curr.next.prev = curr;
        curr = curr.next;
    }

    public String back(int steps) {
        while (curr.prev != null && steps-- > 0)
            curr = curr.prev;
        return curr.url;
    }

    public String forward(int steps) {
        while (curr.next != null && steps-- > 0)
            curr = curr.next;
        return curr.url;
    }

    private Node curr;
}
```

484. 1472.java

Path: LeetCode-main/solutions/1472. Design Browser History/1472.java

```
class BrowserHistory {
    public BrowserHistory(String homepage) {
        visit(homepage);
    }

    public void visit(String url) {
        if (++index < urls.size())
            urls.set(index, url);
        else
            urls.add(url);
        lastIndex = index;
    }

    public String back(int steps) {
        index = Math.max(0, index - steps);
        return urls.get(index);
    }

    public String forward(int steps) {
        index = Math.min(lastIndex, index + steps);
        return urls.get(index);
    }

    private List<String> urls = new ArrayList<>();
    private int index = -1;
    private int lastIndex = -1;
}
```

485. 1473.java

Path: LeetCode-main/solutions/1473. Paint House III/1473.java

```
class Solution {
    public int minCost(int[] houses, int[][] cost, int m, int n, int target) {
        int[][][] mem = new int[target + 1][m][n + 1];
        // Initialize `prevColor` to 0 (the virtual neighbor).
        final int c = minCost(houses, cost, m, n, target, 0, 0, mem);
        return c == MAX ? -1 : c;
    }

    private static final int MAX = 1_000_001;

    // Returns the minimum cost to paint houses[i..m) into k neighborhoods, where
    // there are houses[i - 1] colors = prevColor.
    public int minCost(int[] houses, int[][] cost, int m, int n, int k, int i, int prevColor,
        int[][][] mem) {
        if (i == m || k < 0)
            return k == 0 ? 0 : MAX;
        if (mem[k][i][prevColor] > 0)
            return mem[k][i][prevColor];
        if (houses[i] > 0) // The house was painted last year.
            return minCost(houses, cost, m, n, k - (prevColor == houses[i] ? 0 : 1), i + 1, houses[i],
                mem);

        int ans = MAX;

        // Try to paint the houses[i] with each color in 1..n.
        for (int color = 1; color <= n; ++color)
            ans = Math.min(ans, cost[i][color - 1] + minCost(houses, cost, m, n,
                k - (prevColor == color ? 0 : 1), i + 1,
                color, mem));

        return mem[k][i][prevColor] = ans;
    }
}
```

486. 1474.java

Path: LeetCode-main/solutions/1474. Delete N Nodes After M Nodes of a Linked List/1474.java

```
class Solution {
    public ListNode deleteNodes(ListNode head, int m, int n) {
        ListNode curr = head;
        ListNode prev = null; // prev.next == curr

        while (curr != null) {
            // Set the m-th node as `prev`.
            for (int i = 0; i < m & curr != null; ++i) {
                prev = curr;
                curr = curr.next;
            }
            // Set the (m + n + 1)-th node as `curr`.
            for (int i = 0; i < n && curr != null; ++i)
                curr = curr.next;
            // Delete the nodes [m + 1..n - 1].
            prev.next = curr;
        }

        return head;
    }
}
```


487. 1475.java

Path: LeetCode-main/solutions/1475. Final Prices With a Special Discount in a Shop/1475.java

```
class Solution {
    public int[] finalPrices(int[] prices) {
        int[] ans = prices.clone();
        Deque<Integer> stack = new ArrayDeque<>();

        for (int j = 0; j < prices.length; ++j) {
            // stack[-1] := i in the problem description.
            while (!stack.isEmpty() && prices[j] <= prices[stack.peek()])
                ans[stack.pop()] -= prices[j];
            stack.push(j);
        }

        return ans;
    }
}
```

488. 1476.java

Path: LeetCode-main/solutions/1476. Subrectangle Queries/1476.java

```
class SubrectangleQueries {
    public SubrectangleQueries(int[][] rectangle) {
        this.rectangle = rectangle;
    }

    public void updateSubrectangle(int row1, int col1, int row2, int col2, int newValue) {
        updates.add(new int[] {row1, col1, row2, col2, newValue});
    }

    public int getValue(int row, int col) {
        for (int i = updates.size() - 1; i >= 0; --i) {
            int[] update = updates.get(i);
            final int r1 = update[0];
            final int c1 = update[1];
            final int r2 = update[2];
            final int c2 = update[3];
            final int v = update[4];
            if (r1 <= row && row <= r2 && c1 <= col && col <= c2)
                return v;
        }
        return rectangle[row][col];
    }

    private int[][] rectangle;
    private List<int[]> updates = new ArrayList<>(); // [r1, c1, r2, c2, v]
}
```

489. 1477-2.java

Path: LeetCode-main/solutions/1477. Find Two Non-overlapping Sub-arrays Each With Target Sum/1477-2.java

```
class Solution {
    public int minSumOfLengths(int[] arr, int target) {
        int ans = Integer.MAX_VALUE;
        int sum = 0; // the window sum
        // best[i] := the minimum length of subarrays in arr[0..i] that have
        // sum = target
        int[] best = new int[arr.length];
        Arrays.fill(best, Integer.MAX_VALUE);

        for (int l = 0, r = 0; r < arr.length; ++r) {
            sum += arr[r]; // Expand the window.
            while (sum > target)
                sum -= arr[l++]; // Shrink the window.
            if (sum == target) {
                if (l > 0 && best[l - 1] != Integer.MAX_VALUE)
                    ans = Math.min(ans, best[l - 1] + r - l + 1);
                best[r] = Math.min(best[r], r - l + 1);
            }
            if (r > 0)
                best[r] = Math.min(best[r], best[r - 1]);
        }

        return ans == Integer.MAX_VALUE ? -1 : ans;
    }
}
```

490. 1477.java

Path: LeetCode-main/solutions/1477. Find Two Non-overlapping Sub-arrays Each With Target Sum/1477.java

```
class Solution {
    public int minSumOfLengths(int[] arr, int target) {
        int ans = Integer.MAX_VALUE;
        int leftLength = Integer.MAX_VALUE;
        int prefix = 0;
        Map<Integer, Integer> prefixToIndex = new HashMap<>();
        prefixToIndex.put(0, -1);

        for (int i = 0; i < arr.length; ++i) {
            prefix += arr[i];
            prefixToIndex.put(prefix, i);
        }

        prefix = 0;

        for (int i = 0; i < arr.length; ++i) {
            prefix += arr[i];
            if (prefixToIndex.containsKey(prefix - target))
                leftLength = Math.min(leftLength, i - prefixToIndex.get(prefix - target));
            if (leftLength < Integer.MAX_VALUE)
                if (prefixToIndex.containsKey(prefix + target))
                    ans = Math.min(ans, leftLength + prefixToIndex.get(prefix + target) - i);
        }

        return ans == Integer.MAX_VALUE ? -1 : ans;
    }
}
```

491. 1478.java

Path: LeetCode-main/solutions/1478. Allocate Mailboxes/1478.java

```
class Solution {
    public int minDistance(int[] houses, int k) {
        final int n = houses.length;
        int[][] mem = new int[n][k + 1];
        // cost[i][j] := the minimum cost to allocate mailboxes between houses[i]
        // and houses[j]
        int[][] cost = new int[n][n];

        Arrays.stream(mem).forEach(A -> Arrays.fill(A, Integer.MAX_VALUE));
        Arrays.sort(houses);

        for (int i = 0; i < n; ++i)
            for (int j = i + 1; j < n; ++j) {
                int median = houses[(i + j) / 2];
                for (int x = i; x <= j; ++x)
                    cost[i][j] += Math.abs(houses[x] - median);
            }

        return minDistance(houses, 0, k, cost, mem);
    }

    private static final int MAX = 1_000_000;

    // Returns the minimum distance to allocate k mailboxes for houses[i..n).
    private int minDistance(int[] houses, int i, int k, int[][] cost, int[][] mem) {
        if (i == houses.length && k == 0)
            return 0;
        if (i == houses.length || k == 0)
            return MAX;
        if (mem[i][k] != Integer.MAX_VALUE)
            return mem[i][k];

        for (int j = i; j < houses.length; ++j)
            mem[i][k] = Math.min(mem[i][k], cost[i][j] + minDistance(houses, j + 1, k - 1, cost, mem));

        return mem[i][k];
    }
}
```

492. 148.java

Path: LeetCode-main/solutions/148. Sort List/148.java

```
class Solution {
    public ListNode sortList(ListNode head) {
        final int length = getLength(head);
        ListNode dummy = new ListNode(0, head);

        for (int k = 1; k < length; k *= 2) {
            ListNode curr = dummy.next;
            ListNode tail = dummy;
            while (curr != null) {
                ListNode l = curr;
                ListNode r = split(l, k);
                curr = split(r, k);
                ListNode[] merged = merge(l, r);
                tail.next = merged[0];
                tail = merged[1];
            }
        }

        return dummy.next;
    }

    private int getLength(ListNode head) {
        int length = 0;
        for (ListNode curr = head; curr != null; curr = curr.next)
            ++length;
        return length;
    }

    private ListNode split(ListNode head, int k) {
        while (--k > 0 && head != null)
            head = head.next;
        ListNode rest = head == null ? null : head.next;
        if (head != null)
            head.next = null;
        return rest;
    }

    private ListNode[] merge(ListNode l1, ListNode l2) {
        ListNode dummy = new ListNode(0);
        ListNode tail = dummy;

        while (l1 != null && l2 != null) {
            if (l1.val > l2.val) {
                ListNode temp = l1;
                l1 = l2;
                l2 = temp;
            }
            tail.next = l1;
            l1 = l1.next;
            tail = tail.next;
        }
        tail.next = l1 == null ? l2 : l1;
        while (tail.next != null)
            tail = tail.next;

        return new ListNode[] {dummy.next, tail};
    }
}
```

493. 1480.java

Path: LeetCode-main/solutions/1480. Running Sum of 1d Array/1480.java

```
class Solution {  
    public int[] runningSum(int[] nums) {  
        int[] ans = new int[nums.length];  
        int sum = 0;  
        for (int i = 0; i < nums.length; ++i)  
            ans[i] = sum += nums[i];  
        return ans;  
    }  
}
```

494. 1481.java

Path: LeetCode-main/solutions/1481. Least Number of Unique Integers after K Removals/1481.java

```
class Solution {
    public int findLeastNumOfUniqueInts(int[] arr, int k) {
        Map<Integer, Integer> count = new HashMap<>();

        for (final int a : arr)
            count.merge(a, 1, Integer::sum);

        Queue<Integer> minHeap = new PriorityQueue<>(count.values());

        // Greedily remove the k least frequent numbers to have the least number of unique integers.
        while (k > 0)
            k -= minHeap.poll();

        return minHeap.size() + (k < 0 ? 1 : 0);
    }
}
```


495. 1482.java

Path: LeetCode-main/solutions/1482. Minimum Number of Days to Make m Bouquets/1482.java

```
class Solution {
    public int minDays(int[] bloomDay, int m, int k) {
        if (bloomDay.length < (long) m * k)
            return -1;

        int l = Arrays.stream(bloomDay).min().getAsInt();
        int r = Arrays.stream(bloomDay).max().getAsInt();

        while (l < r) {
            final int mid = (l + r) / 2;
            if (getBouquetCount(bloomDay, k, mid) >= m)
                r = mid;
            else
                l = mid + 1;
        }

        return l;
    }

    // Returns the number of bouquets (k flowers needed) can be made after the
    // `waitingDays`..
    private int getBouquetCount(int[] bloomDay, int k, int waitingDays) {
        int bouquetCount = 0;
        int requiredFlowers = k;
        for (final int day : bloomDay)
            if (day > waitingDays) {
                // Reset `requiredFlowers` since there was not enough adjacent flowers.
                requiredFlowers = k;
            } else if (--requiredFlowers == 0) {
                // Use k adjacent flowers to make a bouquet.
                ++bouquetCount;
                requiredFlowers = k;
            }
        return bouquetCount;
    }
}
```

496. 1483.java

Path: LeetCode-main/solutions/1483. Kth Ancestor of a Tree Node/1483.java

```
class TreeAncestor {
    public TreeAncestor(int n, int[] parent) {
        this.maxLevel = 32 - Integer.numberOfLeadingZeros(n);
        this.dp = new int[n][maxLevel];

        // Node i's 2^0 ancestor is its direct parent.
        for (int i = 0; i < n; ++i)
            dp[i][0] = parent[i];

        for (int j = 1; j < maxLevel; ++j)
            for (int i = 0; i < n; ++i)
                if (dp[i][j - 1] == -1) // There's no such ancestor.
                    dp[i][j] = -1;
                else // A(i, 2^j) = A(A(i, 2^{j - 1}), 2^{j - 1})
                    dp[i][j] = dp[dp[i][j - 1]][j - 1];
    }

    public int getKthAncestor(int node, int k) {
        for (int j = 0; j < maxLevel && node != -1; ++j)
            if ((k >> j & 1) == 1)
                node = dp[node][j];
        return node;
    }

    private final int maxLevel;
    private int[][] dp; // dp[i][j] := node i's 2^j-th ancestor
}
```

497. 1485.java

Path: LeetCode-main/solutions/1485. Clone Binary Tree With Random Pointer/1485.java

```
class Solution {
    public NodeCopy copyRandomBinaryTree(Node root) {
        if (root == null)
            return null;
        if (map.containsKey(root))
            return map.get(root);

        NodeCopy newNode = new NodeCopy(root.val);
        map.put(root, newNode);

        newNode.left = copyRandomBinaryTree(root.left);
        newNode.right = copyRandomBinaryTree(root.right);
        newNode.random = copyRandomBinaryTree(root.random);
        return newNode;
    }

    private Map<Node, NodeCopy> map = new HashMap<>();
}
```

498. 1486.java

Path: LeetCode-main/solutions/1486. XOR Operation in an Array/1486.java

```
class Solution {
    public int xorOperation(int n, int start) {
        int ans = 0;
        for (int i = 0; i < n; ++i)
            ans ^= start + 2 * i;
        return ans;
    }
}
```

499. 1487.java

Path: LeetCode-main/solutions/1487. Making File Names Unique/1487.java

```
class Solution {
    public String[] getFolderNames(String[] names) {
        String[] ans = new String[names.length];
        Map<String, Integer> nameToSuffix = new HashMap<>();

        for (int i = 0; i < names.length; ++i) {
            final String name = names[i];
            if (nameToSuffix.containsKey(name)) {
                int suffix = nameToSuffix.get(name);
                String newName = getName(name, ++suffix);
                while (nameToSuffix.containsKey(newName))
                    newName = getName(name, ++suffix);
                nameToSuffix.put(name, suffix);
                nameToSuffix.put(newName, 0);
                ans[i] = newName;
            } else {
                nameToSuffix.put(name, 0);
                ans[i] = name;
            }
        }

        return ans;
    }

    private String getName(final String name, int suffix) {
        return name + "(" + String.valueOf(suffix) + ")";
    }
}
```

500. 1488.java

Path: LeetCode-main/solutions/1488. Avoid Flood in The City/1488.java

```
class Solution {
    public int[] avoidFlood(int[] rains) {
        int[] ans = new int[rains.length];
        Arrays.fill(ans, -1);
        Map<Integer, Integer> lakeIdToFullDay = new HashMap<>();
        TreeSet<Integer> emptyDays = new TreeSet<>(); // indices of rains[i] == 0

        for (int i = 0; i < rains.length; ++i) {
            final int lakeId = rains[i];
            if (lakeId == 0) {
                emptyDays.add(i);
                continue;
            }
            if (lakeIdToFullDay.containsKey(lakeId)) {
                final int fullDay = lakeIdToFullDay.get(lakeId);
                // The lake was full in a previous day. Greedily find the closest day
                // to make the lake empty.
                Integer emptyDay = emptyDays.higher(fullDay);
                if (emptyDay == null) // Not found.
                    return new int[] {};
                // Empty the lake at this day.
                ans[emptyDay] = lakeId;
                emptyDays.remove(emptyDay);
            }
            // The lake with `lakeId` becomes full at the day `i`.
            lakeIdToFullDay.put(lakeId, i);
        }

        // Empty an arbitrary lake if there are remaining empty days.
        for (final int emptyDay : emptyDays)
            ans[emptyDay] = 1;

        return ans;
    }
}
```

501. 1489.java

Path: LeetCode-main/solutions/1489. Find Critical and Pseudo-Critical Edges in Minimum Spanning Tree/1489.java

```
class UnionFind {
    public UnionFind(int n) {
        id = new int[n];
        rank = new int[n];
        for (int i = 0; i < n; ++i)
            id[i] = i;
    }

    public void unionByRank(int u, int v) {
        final int i = find(u);
        final int j = find(v);
        if (i == j)
            return;
        if (rank[i] < rank[j]) {
            id[i] = j;
        } else if (rank[i] > rank[j]) {
            id[j] = i;
        } else {
            id[i] = j;
            ++rank[j];
        }
    }

    public int find(int u) {
        return id[u] == u ? u : (id[u] = find(id[u]));
    }

    private int[] id;
    private int[] rank;
}

class Solution {
    public List<List<Integer>> findCriticalAndPseudoCriticalEdges(int n, int[][] edges) {
        List<Integer> criticalEdges = new ArrayList<>();
        List<Integer> pseudoCriticalEdges = new ArrayList<>();

        // Record the index information, so edges[i] := (u, v, weight, index).
        for (int i = 0; i < edges.length; ++i)
            edges[i] = new int[] {edges[i][0], edges[i][1], edges[i][2], i};

        // Sort by the weight.
        Arrays.sort(edges, Comparator.comparingInt(edge -> edge[2]));

        final int mstWeight = getMSTWeight(n, edges, new int[] {}, -1);

        for (int[] edge : edges) {
            final int index = edge[3];
            // Deleting the `edge` increases the MST's weight or makes the MST
            // invalid.
            if (getMSTWeight(n, edges, new int[] {}, index) > mstWeight)
                criticalEdges.add(index);
            // If an edge can be in any MST, we can always add `edge` to the edge set.
            else if (getMSTWeight(n, edges, edge, -1) == mstWeight)
                pseudoCriticalEdges.add(index);
        }

        return List.of(criticalEdges, pseudoCriticalEdges);
    }

    private int getMSTWeight(int n, int[][] edges, int[] firstEdge, int deletedEdgeIndex) {
        int mstWeight = 0;
        UnionFind uf = new UnionFind(n);

        if (firstEdge.length == 4) {
            uf.unionByRank(firstEdge[0], firstEdge[1]);
            mstWeight += firstEdge[2];
        }

        for (int[] edge : edges) {
            final int u = edge[0];
            final int v = edge[1];
            final int weight = edge[2];
            final int index = edge[3];
            if (index == deletedEdgeIndex)
                continue;
            if (uf.find(u) == uf.find(v))
                continue;
            uf.unionByRank(u, v);
            mstWeight += weight;
        }
    }
}
```

```
        final int root = uf.find(0);
        for (int i = 0; i < n; ++i)
            if (uf.find(i) != root)
                return Integer.MAX_VALUE;
    }
    return mstWeight;
}
```


502. 149.java

Path: LeetCode-main/solutions/149. Max Points on a Line/149.java

```
class Solution {
    public int maxPoints(int[][] points) {
        int ans = 0;

        for (int i = 0; i < points.length; ++i) {
            Map<Pair<Integer, Integer>, Integer> slopeCount = new HashMap<>();
            int[] p1 = points[i];
            int samePoints = 1;
            int maxPoints = 0; // the maximum number of points with the same slope
            for (int j = i + 1; j < points.length; ++j) {
                int[] p2 = points[j];
                if (p1[0] == p2[0] && p1[1] == p2[1])
                    ++samePoints;
                else {
                    Pair<Integer, Integer> slope = getSlope(p1, p2);
                    slopeCount.merge(slope, 1, Integer::sum);
                    maxPoints = Math.max(maxPoints, slopeCount.get(slope));
                }
            }
            ans = Math.max(ans, samePoints + maxPoints);
        }

        return ans;
    }

    private Pair<Integer, Integer> getSlope(int[] p, int[] q) {
        final int dx = p[0] - q[0];
        final int dy = p[1] - q[1];
        if (dx == 0)
            return new Pair<>(0, p[0]);
        if (dy == 0)
            return new Pair<>(p[1], 0);
        final int d = gcd(dx, dy);
        return new Pair<>(dx / d, dy / y);
    }

    private int gcd(int a, int b) {
        return b == 0 ? a : gcd(b, a % b);
    }
}
```

503. 1490.java

Path: LeetCode-main/solutions/1490. Clone N-ary Tree/1490.java

```
class Solution {
    public Node cloneTree(Node root) {
        if (root == null)
            return null;
        if (map.containsKey(root))
            return map.get(root);

        Node newNode = new Node(root.val);
        map.put(root, newNode);

        for (Node child : root.children)
            newNode.children.add(cloneTree(child));

        return newNode;
    }

    private Map<Node, Node> map = new HashMap<>();
}
```

504. 1491.java

Path: LeetCode-main/solutions/1491. Average Salary Excluding the Minimum and Maximum Salary/1491.java

```
class Solution {  
    public double average(int[] salary) {  
        final double sum = Arrays.stream(salary).sum();  
        final int mx = Arrays.stream(salary).max().getAsInt();  
        final int mn = Arrays.stream(salary).min().getAsInt();  
        return (sum - mx - mn) / (salary.length - 2);  
    }  
}
```

505. 1492.java

Path: LeetCode-main/solutions/1492. The kth Factor of n/1492.java

```
class Solution {
    public int kthFactor(int n, int k) {
        // If i is a divisor of n, then n / i is also a divisor of n. So, we can
        // find all the divisors of n by processing the numbers <= sqrt(n).
        int factor = 1;
        int i = 0; // the i-th factor

        for (; factor * factor < n; ++factor)
            if (n % factor == 0 && ++i == k)
                return factor;

        for (factor = n / factor; factor >= 1; --factor)
            if (n % factor == 0 && ++i == k)
                return n / factor;

        return -1;
    }
}
```

506. 1493-2.java

Path: LeetCode-main/solutions/1493. Longest Subarray of 1's After Deleting One Element/1493-2.java

```
class Solution {
    public int longestSubarray(int[] nums) {
        int l = 0;
        int zeros = 0;

        for (final int num : nums) {
            if (num == 0)
                ++zeros;
            if (zeros > 1 && nums[l++] == 0)
                --zeros;
        }

        return nums.length - l - 1;
    }
}
```

507. 1493.java

Path: LeetCode-main/solutions/1493. Longest Subarray of 1's After Deleting One Element/1493.java

```
class Solution {
    public int longestSubarray(int[] nums) {
        int ans = 0;
        int zeros = 0;

        for (int l = 0, r = 0; r < nums.length; ++r) {
            if (nums[r] == 0)
                ++zeros;
            while (zeros == 2)
                if (nums[l++] == 0)
                    --zeros;
            ans = Math.max(ans, r - l);
        }
        return ans;
    }
}
```

508. 1494.java

Path: LeetCode-main/solutions/1494. Parallel Courses II/1494.java

```
class Solution {
    public int minNumberOfSemesters(int n, int[][] relations, int k) {
        // dp[i] := the minimum number of semesters to take the courses, where i is
        // the bitmask of the taken courses
        int[] dp = new int[1 << n];
        Arrays.fill(dp, n);
        // prereq[i] := the bitmask of all the dependencies of the i-th course
        int[] prereq = new int[n];

        for (int[] r : relations) {
            final int prevCourse = r[0] - 1;
            final int nextCourse = r[1] - 1;
            prereq[nextCourse] |= 1 << prevCourse;
        }

        dp[0] = 0; // Don't need time to finish 0 course.

        for (int i = 0; i < dp.size(); ++i) {
            // the bitmask of all the courses can be taken
            int coursesCanBeTaken = 0;
            // Can take the j-th course if i contains all of j's prerequisites.
            for (int j = 0; j < n; ++j)
                if ((i & prereq[j]) == prereq[j])
                    coursesCanBeTaken |= 1 << j;
            // Don't take any course which is already taken.
            // (i represents set of courses that are already taken)
            coursesCanBeTaken &= ~i;
            // Enumerate every bitmask subset of `coursesCanBeTaken`.
            for (int s = coursesCanBeTaken; s > 0; s = (s - 1) & coursesCanBeTaken)
                if (Integer.bitCount(s) <= k)
                    // Any combination of courses (if <= k) can be taken now.
                    // i | s := combining courses taken with courses can be taken.
                    dp[i | s] = Math.min(dp[i | s], dp[i] + 1);
        }

        return dp[(1 << n) - 1];
    }
}
```

509. 1497.java

Path: LeetCode-main/solutions/1497. Check If Array Pairs Are Divisible by k/1497.java

```
class Solution {
    public boolean canArrange(int[] arr, int k) {
        int[] count = new int[k];

        for (int a : arr) {
            a %= k;
            ++count[a < 0 ? a + k : a];
        }

        if (count[0] % 2 != 0)
            return false;

        for (int i = 1; i <= k / 2; i++)
            if (count[i] != count[k - i])
                return false;

        return true;
    }
}
```


510. 1498.java

Path: LeetCode-main/solutions/1498. Number of Subsequences That Satisfy the Given Sum Condition/1498.java

```
class Solution {
    public int numSubseq(int[] nums, int target) {
        final int MOD = 1_000_000_007;
        final int n = nums.length;
        int ans = 0;
        int[] pows = new int[n]; // pows[i] = 2^i % MOD
        pows[0] = 1;

        for (int i = 1; i < n; ++i)
            pows[i] = pows[i - 1] * 2 % MOD;

        Arrays.sort(nums);

        for (int l = 0, r = n - 1; l <= r;)
            if (nums[l] + nums[r] <= target) {
                ans += pows[r - l];
                ans %= MOD;
                ++l;
            } else {
                --r;
            }

        return ans;
    }
}
```

511. 1499-2.java

Path: LeetCode-main/solutions/1499. Max Value of Equation/1499-2.java

```
class Solution {
    public int findMaxValueOfEquation(int[][] points, int k) {
        int ans = Integer.MIN_VALUE;
        Deque<Pair<Integer, Integer>> maxQ = new ArrayDeque<>(); // (y - x, x)

        for (int[] point : points) {
            final int x = point[0];
            final int y = point[1];
            // Remove the invalid points, xj - xi > k
            while (!maxQ.isEmpty() && x - maxQ.peekFirst().getValue() > k)
                maxQ.pollFirst();
            if (!maxQ.isEmpty())
                ans = Math.max(ans, x + y + maxQ.peekFirst().getKey());
            // Remove the points that contribute less value and have a bigger x.
            while (!maxQ.isEmpty() && y - x >= maxQ.peekLast().getKey())
                maxQ.pollLast();
            maxQ.offerLast(new Pair<>(y - x, x));
        }

        return ans;
    }
}
```

512. 1499.java

Path: LeetCode-main/solutions/1499. Max Value of Equation/1499.java

```
class Solution {
    public int findMaxValueOfEquation(int[][] points, int k) {
        int ans = Integer.MIN_VALUE;
        // (y - x, x)
        Queue<Pair<Integer, Integer>> maxHeap =
            new PriorityQueue<>(Comparator.comparing(Pair::getKey, Comparator.reverseOrder()));

        for (int[] p : points) {
            final int x = p[0];
            final int y = p[1];
            while (!maxHeap.isEmpty() && x - maxHeap.peek().getValue() > k)
                maxHeap.poll();
            if (!maxHeap.isEmpty())
                ans = Math.max(ans, x + y + maxHeap.peek().getKey());
            maxHeap.offer(new Pair<>(y - x, x));
        }

        return ans;
    }
}
```

513. 15.java

Path: LeetCode-main/solutions/15. 3Sum/15.java

```
class Solution {
    public List<List<Integer>> threeSum(int[] nums) {
        if (nums.length < 3)
            return new ArrayList<>();

        List<List<Integer>> ans = new ArrayList<>();

        Arrays.sort(nums);

        for (int i = 0; i + 2 < nums.length; ++i) {
            if (i > 0 && nums[i] == nums[i - 1])
                continue;
            // Choose nums[i] as the first number in the triplet, then search the
            // remaining numbers in [i + 1, n - 1].
            int l = i + 1;
            int r = nums.length - 1;
            while (l < r) {
                final int sum = nums[i] + nums[l] + nums[r];
                if (sum == 0) {
                    ans.add(Arrays.asList(nums[i], nums[l++], nums[r--]));
                    while (l < r && nums[l] == nums[l - 1])
                        ++l;
                    while (l < r && nums[r] == nums[r + 1])
                        --r;
                } else if (sum < 0) {
                    ++l;
                } else {
                    --r;
                }
            }
        }

        return ans;
    }
}
```

514. 150.java

Path: LeetCode-main/solutions/150. Evaluate Reverse Polish Notation/150.java

```
class Solution {
    public int evalRPN(String[] tokens) {
        final Map<String, BinaryOperator<Long>> op = Map.of(
            "+", (a, b) -> a + b, "-", (a, b) -> a - b, "**", (a, b) -> a * b, "/", (a, b) -> a / b);
        Deque<Long> stack = new ArrayDeque<>();

        for (final String token : tokens)
            if (op.containsKey(token)) {
                final long b = stack.pop();
                final long a = stack.pop();
                stack.push(op.get(token).apply(a, b));
            } else {
                stack.push(Long.parseLong(token));
            }

        return stack.pop().intValue();
    }
}
```

515. 1500.java

Path: LeetCode-main/solutions/1500. Design a File Sharing System/1500.java

```
class FileSharing {
    public FileSharing(int m) {}

    public int join(List<Integer> ownedChunks) {
        final int userId =
            availableUserIds.isEmpty() ? userToChunks.size() + 1 : availableUserIds.poll();
        userToChunks.put(userId, new HashSet<>(ownedChunks));
        for (final int chunk : ownedChunks) {
            chunkToUsers.putIfAbsent(chunk, new TreeSet<>());
            chunkToUsers.get(chunk).add(userId);
        }
        return userId;
    }

    public void leave(int userID) {
        for (final int chunk : userToChunks.get(userID)) {
            chunkToUsers.get(chunk).remove(userID);
            if (chunkToUsers.get(chunk).isEmpty())
                chunkToUsers.remove(chunk);
        }
        userToChunks.remove(userID);
        availableUserIds.offer(userID);
    }

    public List<Integer> request(int userID, int chunkID) {
        if (!chunkToUsers.containsKey(chunkID))
            return new ArrayList<>();
        List<Integer> userIds = new ArrayList<>(chunkToUsers.get(chunkID));
        userToChunks.get(userID).add(chunkID);
        chunkToUsers.get(chunkID).add(userID);
        return userIds;
    }

    private Map<Integer, Set<Integer>> userToChunks = new HashMap<>();
    private Map<Integer, Set<Integer>> chunkToUsers = new HashMap<>();
    private Queue<Integer> availableUserIds = new PriorityQueue<>();
}
```

516. 1504-2.java

Path: LeetCode-main/solutions/1504. Count Submatrices With All Ones/1504-2.java

```
class Solution {
    public int numSubmat(int[][] mat) {
        int ans = 0;
        int[] hist = new int[mat[0].length];

        for (int[] row : mat) {
            for (int i = 0; i < row.length; ++i)
                hist[i] = row[i] == 0 ? 0 : hist[i] + 1;
            ans += count(hist);
        }

        return ans;
    }

    private int count(int[] hist) {
        // submatrices[i] := the number of submatrices, where the i-th column is the
        // right border
        int[] submatrices = new int[hist.length];
        Deque<Integer> stack = new ArrayDeque<>();

        for (int i = 0; i < hist.length; ++i) {
            while (!stack.isEmpty() && hist[stack.peek()] >= hist[i])
                stack.pop();
            if (!stack.isEmpty()) {
                final int prevIndex = stack.peek();
                submatrices[i] = submatrices[prevIndex] + hist[i] * (i - prevIndex);
            } else {
                submatrices[i] = hist[i] * (i + 1);
            }
            stack.push(i);
        }

        return Arrays.stream(submatrices).sum();
    }
}
```

517. 1504.java

Path: LeetCode-main/solutions/1504. Count Submatrices With All Ones/1504.java

```
class Solution {
    public int numSubmat(int[][] mat) {
        final int m = mat.length;
        final int n = mat[0].length;
        int ans = 0;

        for (int baseRow = 0; baseRow < m; ++baseRow) {
            int[] row = new int[n];
            Arrays.fill(row, 1);
            for (int i = baseRow; i < m; ++i) {
                for (int j = 0; j < n; ++j)
                    row[j] &= mat[i][j];
                ans += count(row);
            }
        }

        return ans;
    }

    private int count(int[] row) {
        int res = 0;
        int length = 0;
        for (final int num : row) {
            length = num == 0 ? 0 : length + 1;
            res += length;
        }
        return res;
    }
}
```


518. 1506.java

Path: LeetCode-main/solutions/1506. Find Root of N-Ary Tree/1506.java

```
class Solution {
    public Node findRoot(List<Node> tree) {
        int sum = 0;

        for (Node node : tree) {
            sum ^= node.val;
            for (Node child : node.children)
                sum ^= child.val;
        }

        for (Node node : tree)
            if (node.val == sum)
                return node;

        throw new IllegalArgumentException();
    }
}
```

519. 1507.java

Path: LeetCode-main/solutions/1507. Reformat Date/1507.java

```
class Solution {
    public String reformatDate(String date) {
        Map<String, String> months = getMonths();
        String[] words = date.split("\\s+");
        final String day =
            (words[0].length() == 4) ? words[0].substring(0, 2) : "0" + words[0].substring(0, 1);
        final String month = months.get(words[1]);
        final String year = words[2];
        return year + "-" + month + "-" + day;
    }

    private Map<String, String> getMonths() {
        Map<String, String> months = new HashMap<>();
        months.put("Jan", "01");
        months.put("Feb", "02");
        months.put("Mar", "03");
        months.put("Apr", "04");
        months.put("May", "05");
        months.put("Jun", "06");
        months.put("Jul", "07");
        months.put("Aug", "08");
        months.put("Sep", "09");
        months.put("Oct", "10");
        months.put("Nov", "11");
        months.put("Dec", "12");
        return months;
    }
}
```

520. 1509.java

Path: LeetCode-main/solutions/1509. Minimum Difference Between Largest and Smallest Value in Three Moves/1509.java

```
class Solution {
    public int minDifference(int[] nums) {
        final int n = nums.length;
        if (n < 5)
            return 0;

        int ans = Integer.MAX_VALUE;

        Arrays.sort(nums);

        // 1. Change nums[0..i) to nums[i].
        // 2. Change nums[n - 3 + i..n) to nums[n - 4 + i].
        for (int i = 0; i <= 3; ++i)
            ans = Math.min(ans, nums[n - 4 + i] - nums[i]);

        return ans;
    }
}
```

521. 151.java

Path: LeetCode-main/solutions/151. Reverse Words in a String/151.java

```
class Solution {
    public String reverseWords(String s) {
        StringBuilder sb = new StringBuilder(s).reverse(); // Reverse the whole string.
        reverseWords(sb, sb.length()); // Reverse each word.
        return cleanSpaces(sb, sb.length()); // Clean up the spaces.
    }

    private void reverseWords(StringBuilder sb, int n) {
        int i = 0;
        int j = 0;

        while (i < n) {
            while (i < j || i < n && sb.charAt(i) == ' ') // Skip the spaces.
                ++i;
            while (j < i || j < n && sb.charAt(j) != ' ') // Skip the spaces.
                ++j;
            reverse(sb, i, j - 1); // Reverse the word.
        }
    }

    // Trim leading, trailing, and middle spaces
    private String cleanSpaces(StringBuilder sb, int n) {
        int i = 0;
        int j = 0;

        while (j < n) {
            while (j < n && sb.charAt(j) == ' ') // Skip the spaces.
                ++j;
            while (j < n && sb.charAt(j) != ' ') // Keep non spaces
                sb.setCharAt(i++, sb.charAt(j++));
            while (j < n && sb.charAt(j) == ' ') // Skip the spaces.
                ++j;
            if (j < n) // Keep only one space.
                sb.setCharAt(i++, ' ');
        }

        return sb.substring(0, i).toString();
    }

    private void reverse(StringBuilder sb, int l, int r) {
        while (l < r) {
            final char temp = sb.charAt(l);
            sb.setCharAt(l++, sb.charAt(r));
            sb.setCharAt(r--, temp);
        }
    }
}
```

522. 1510.java

Path: LeetCode-main/solutions/1510. Stone Game IV/1510.java

```
class Solution {
    public boolean winnerSquareGame(int n) {
        // dp[i] := the winning result for n = i
        boolean[] dp = new boolean[n + 1];

        for (int i = 1; i <= n; ++i)
            for (int j = 1; j * j <= i; ++j)
                if (!dp[i - j * j]) { // Removing j^2 stones make the opponent lose.
                    dp[i] = true;      // So, we win.
                    break;
                }

        return dp[n];
    }
}
```

523. 1514.java

Path: LeetCode-main/solutions/1514. Path with Maximum Probability/1514.java

```
class Solution {
    public double maxProbability(int n, int[][] edges, double[] succProb, int start, int end) {
        // {a: [(b, probability_ab)]}
        List<Pair<Integer, Double>>[] graph = new List[n];
        // (the probability to reach u, u)
        Queue<Pair<Double, Integer>> maxHeap =
            new PriorityQueue<>((a, b) -> Double.compare(b.getKey(), a.getKey())) {
                { offer(new Pair<>(1.0, start)); }
            };
        boolean[] seen = new boolean[n];
        Arrays.setAll(graph, i -> new ArrayList<>());

        for (int i = 0; i < edges.length; ++i) {
            final int u = edges[i][0];
            final int v = edges[i][1];
            final double prob = succProb[i];
            graph[u].add(new Pair<>(v, prob));
            graph[v].add(new Pair<>(u, prob));
        }

        while (!maxHeap.isEmpty()) {
            final double prob = maxHeap.peek().getKey();
            final int u = maxHeap.poll().getValue();
            if (u == end)
                return prob;
            if (seen[u])
                continue;
            seen[u] = true;
            for (Pair<Integer, Double> node : graph[u]) {
                final int nextNode = node.getKey();
                final double edgeProb = node.getValue();
                if (seen[nextNode])
                    continue;
                maxHeap.add(new Pair<>(prob * edgeProb, nextNode));
            }
        }

        return 0;
    }
}
```

524. 1515.java

Path: LeetCode-main/solutions/1515. Best Position for a Service Centre/1515.java

```
class Solution {
    public double getMinDistSum(int[][] positions) {
        final double ERR = 1e-6;
        double currX = 50;
        double currY = 50;
        double ans = distSum(positions, currX, currY);
        double step = 1;

        while (step > ERR) {
            boolean shouldDecreaseStep = true;
            for (double[] DIRS : new double[][] {{0, step}, {0, -step}, {step, 0}, {-step, 0}}) {
                final double x = currX + DIRS[0];
                final double y = currY + DIRS[1];
                final double newDistSum = distSum(positions, x, y);
                if (newDistSum < ans) {
                    ans = newDistSum;
                    currX = x;
                    currY = y;
                    shouldDecreaseStep = false;
                }
            }
            if (shouldDecreaseStep)
                step /= 10;
        }

        return ans;
    }

    private double distSum(int[][] positions, double a, double b) {
        double sum = 0;
        for (int[] p : positions)
            sum += Math.sqrt(Math.pow(a - p[0], 2) + Math.pow(b - p[1], 2));
        return sum;
    }
}
```

525. 1519.java

Path: LeetCode-main/solutions/1519. Number of Nodes in the Sub-Tree With the Same Label/1519.java

```
class Solution {
    public int[] countSubTrees(int n, int[][] edges, String labels) {
        int[] ans = new int[n];
        List<Integer>[] graph = new List[n];
        Arrays.setAll(graph, i -> new ArrayList<>());

        for (int[] edge : edges) {
            final int u = edge[0];
            final int v = edge[1];
            graph[u].add(v);
            graph[v].add(u);
        }

        dfs(graph, 0, -1, labels, ans);
        return ans;
    }

    private int[] dfs(List<Integer>[] graph, int u, int prev, final String labels, int[] ans) {
        // count[i] := the number of letters down from ('a' + i)
        int[] count = new int[26];

        for (final int v : graph[u]) {
            if (v == prev)
                continue;
            int[] childCount = dfs(graph, v, u, labels, ans);
            for (int i = 0; i < 26; ++i)
                count[i] += childCount[i];
        }

        ans[u] = ++count[labels.charAt(u) - 'a']; // the u itself
        return count;
    }
}
```


526. 152.java

Path: LeetCode-main/solutions/152. Maximum Product Subarray/152.java

```
class Solution {
    public int maxProduct(int[] nums) {
        int ans = nums[0];
        int dpMin = nums[0]; // the minimum so far
        int dpMax = nums[0]; // the maximum so far

        for (int i = 1; i < nums.length; ++i) {
            final int num = nums[i];
            final int prevMin = dpMin; // dpMin[i - 1]
            final int prevMax = dpMax; // dpMax[i - 1]
            if (num < 0) {
                dpMin = Math.min(prevMax * num, num);
                dpMax = Math.max(prevMin * num, num);
            } else {
                dpMin = Math.min(prevMin * num, num);
                dpMax = Math.max(prevMax * num, num);
            }
            ans = Math.max(ans, dpMax);
        }

        return ans;
    }
}
```

527. 1521.java

Path: LeetCode-main/solutions/1521. Find a Value of a Mysterious Function Closest to Target/1521.java

```
class Solution {
    public int closestToTarget(int[] arr, int target) {
        int ans = Integer.MAX_VALUE;
        // all the values of subarrays that end in the previous number
        Set<Integer> prev = new HashSet<>();

        for (final int num : arr) {
            HashSet<Integer> curr = new HashSet<>(Arrays.asList(num));
            // Extend each subarray that ends in the previous number. Due to
            // monotonicity of the AND operation, the size of `curr` will be at most
            // Integer.bitCount(num) + 1.
            for (final int val : prev)
                curr.add(val & num);
            for (final int val : curr)
                ans = Math.min(ans, Math.abs(target - val));
            prev = curr;
        }

        return ans;
    }
}
```

528. 1522.java

Path: LeetCode-main/solutions/1522. Diameter of N-Ary Tree/1522.java

```
class Solution {
    public int diameter(Node root) {
        maxDepth(root);
        return ans;
    }

    private int ans = 0;

    // Returns the maximum depth of the subtree rooted at `root`.
    private int maxDepth(Node root) {
        int maxSubDepth1 = 0;
        int maxSubDepth2 = 0;
        for (Node child : root.children) {
            final int maxSubDepth = maxDepth(child);
            if (maxSubDepth > maxSubDepth1) {
                maxSubDepth2 = maxSubDepth1;
                maxSubDepth1 = maxSubDepth;
            } else if (maxSubDepth > maxSubDepth2) {
                maxSubDepth2 = maxSubDepth;
            }
        }
        ans = Math.max(ans, maxSubDepth1 + maxSubDepth2);
        return 1 + maxSubDepth1;
    }
}
```

529. 1525.java

Path: LeetCode-main/solutions/1525. Number of Good Ways to Split a String/1525.java

```
class Solution {
    public int numSplits(String s) {
        final int n = s.length();
        int ans = 0;
        int[] prefix = new int[n];
        int[] suffix = new int[n];
        Set<Character> seen = new HashSet<>();

        for (int i = 0; i < n; ++i) {
            seen.add(s.charAt(i));
            prefix[i] = seen.size();
        }

        seen.clear();

        for (int i = n - 1; i >= 0; --i) {
            seen.add(s.charAt(i));
            suffix[i] = seen.size();
        }

        for (int i = 0; i + 1 < n; ++i)
            if (prefix[i] == suffix[i + 1])
                ++ans;

        return ans;
    }
}
```

530. 1526.java

Path: LeetCode-main/solutions/1526. Minimum Number of Increments on Subarrays to Form a Target Array/1526.java

```
class Solution {
    public int minNumberOperations(int[] target) {
        int ans = target[0];

        for (int i = 1; i < target.length; ++i)
            if (target[i] > target[i - 1])
                ans += target[i] - target[i - 1];

        return ans;
    }
}
```

531. 1529.java

Path: LeetCode-main/solutions/1529. Minimum Suffix Flips/1529.java

```
class Solution {
    public int minFlips(String target) {
        int ans = 0;
        int state = 0;

        for (final char c : target.toCharArray())
            if (c - '0' != state) {
                state = c - '0';
                ++ans;
            }

        return ans;
    }
}
```

532. 153.java

Path: LeetCode-main/solutions/153. Find Minimum in Rotated Sorted Array/153.java

```
class Solution {
    public int findMin(int[] nums) {
        int l = 0;
        int r = nums.length - 1;

        while (l < r) {
            final int m = (l + r) / 2;
            if (nums[m] < nums[r])
                r = m;
            else
                l = m + 1;
        }

        return nums[l];
    }
}
```

533. 1530.java

Path: LeetCode-main/solutions/1530. Number of Good Leaf Nodes Pairs/1530.java

```
class Solution {
    public int countPairs(TreeNode root, int distance) {
        dfs(root, distance);

        return ans;
    }

    private int ans = 0;

    private int[] dfs(TreeNode root, int distance) {
        int[] d = new int[distance + 1]; // {distance: the number of leaf nodes}
        if (root == null)
            return d;
        if (root.left == null && root.right == null) {
            d[0] = 1;
            return d;
        }

        int[] dl = dfs(root.left, distance);
        int[] dr = dfs(root.right, distance);

        for (int i = 0; i < distance; ++i)
            for (int j = 0; j < distance; ++j)
                if (i + j + 2 <= distance)
                    ans += dl[i] * dr[j];

        for (int i = 0; i < distance; ++i)
            d[i + 1] = dl[i] + dr[i];

        return d;
    }
}
```


534. 1531.java

Path: LeetCode-main/solutions/1531. String Compression II/1531.java

```
class Solution {
    public int getLengthOfOptimalCompression(String s, int k) {
        int[][] mem = new int[s.length()][k + 1];
        Arrays.stream(mem).forEach(A -> Arrays.fill(A, MAX));
        return compression(s, 0, k, mem);
    }

    private static final int MAX = 101;

    // Returns the length of the optimal compression of s[i..n) with at most k
    // deletion.
    private int compression(final String s, int i, int k, int[][] mem) {
        if (k < 0)
            return MAX;
        if (i == s.length() || s.length() - i <= k)
            return 0;
        if (mem[i][k] != MAX)
            return mem[i][k];

        int maxFreq = 0;
        int[] count = new int[128];

        // Make letters in s[i..j] be the same.
        // Keep the letter that has the maximum frequency in this range and remove
        // the other letters.
        for (int j = i; j < s.length(); ++j) {
            maxFreq = Math.max(maxFreq, ++count[s.charAt(j)]);
            mem[i][k] = Math.min( //
                mem[i][k], //
                getLength(maxFreq) + compression(s, j + 1, k - (j - i + 1 - maxFreq), mem));
        }

        return mem[i][k];
    }

    // Returns the length to compress `maxFreq`.
    private int getLength(int maxFreq) {
        if (maxFreq == 1)
            return 1; // c
        if (maxFreq < 10)
            return 2; // [1-9]c
        if (maxFreq < 100)
            return 3; // [1-9][0-9]c
        return 4; // [1-9][0-9][0-9]c
    }
}
```

535. 1533-2.java

Path: LeetCode-main/solutions/1533. Find the Index of the Large Integer/1533-2.java

```
/**
 * // This is ArrayReader's API interface.
 * // You should not implement it, or speculate about its implementation
 * interface ArrayReader {
 *     // Compares the sum of arr[l..r] with the sum of arr[x..y]
 *     // return 1 if sum(arr[l..r]) > sum(arr[x..y])
 *     // return 0 if sum(arr[l..r]) == sum(arr[x..y])
 *     // return -1 if sum(arr[l..r]) < sum(arr[x..y])
 *     public int compareSub(int l, int r, int x, int y) {}
 *
 *     // Returns the length of the array
 *     public int length() {}
 * }
 */

class Solution {
    public int getIndex(ArrayReader reader) {
        int l = 0;
        int r = reader.length() - 1;

        while (l < r) {
            final int m = (l + r) / 2;
            final int res = (r - l + 1) % 2 == 0 ? reader.compareSub(l, m, m + 1, r) //
                : reader.compareSub(l, m, m, r);

            if (res == -1)
                l = m + 1;
            else // res == 1 || res == 0
                r = m;
        }

        return l;
    }
}
```

536. 1533.java

Path: LeetCode-main/solutions/1533. Find the Index of the Large Integer/1533.java

```
/**
 * // This is ArrayReader's API interface.
 * // You should not implement it, or speculate about its implementation
 * interface ArrayReader {
 *     // Compares the sum of arr[l..r] with the sum of arr[x..y]
 *     // return 1 if sum(arr[l..r]) > sum(arr[x..y])
 *     // return 0 if sum(arr[l..r]) == sum(arr[x..y])
 *     // return -1 if sum(arr[l..r]) < sum(arr[x..y])
 *     public int compareSub(int l, int r, int x, int y) {}
 *
 *     // Returns the length of the array
 *     public int length() {}
 * }
 */

class Solution {
    public int getIndex(ArrayReader reader) {
        int l = 0;
        int r = reader.length() - 1;

        while (l < r) {
            final int m = (l + r) / 2;
            if ((r - l) % 2 == 0) {
                final int res = reader.compareSub(l, m - 1, m + 1, r);
                if (res == 0)
                    return m;
                if (res == 1) {
                    r = m - 1;
                } else { // res == -1
                    l = m + 1;
                }
            } else {
                final int res = reader.compareSub(l, m, m + 1, r);
                // res is either 1 or -1.
                if (res == 1) {
                    r = m;
                } else { // res == -1
                    l = m + 1;
                }
            }
        }

        return l;
    }
}
```


538. 1535.java

Path: LeetCode-main/solutions/1535. Find the Winner of an Array Game/1535.java

```
class Solution {
    public int getWinner(int[] arr, int k) {
        int ans = arr[0];
        int wins = 0;

        for (int i = 1; i < arr.length && wins < k; ++i)
            if (arr[i] > ans) {
                ans = arr[i];
                wins = 1;
            } else {
                ++wins;
            }

        return ans;
    }
}
```

539. 1536.java

Path: LeetCode-main/solutions/1536. Minimum Swaps to Arrange a Binary Grid/1536.java

```
class Solution {
    public int minSwaps(int[][] grid) {
        final int n = grid.length;
        int ans = 0;
        // suffixZeros[i] := the number of suffix zeros in the i-th row
        int[] suffixZeros = new int[n];

        for (int i = 0; i < grid.length; ++i)
            suffixZeros[i] = getSuffixZeroCount(grid[i]);

        for (int i = 0; i < n; ++i) {
            final int neededZeros = n - 1 - i;
            // Get the first row with suffix zeros >= `neededZeros` in suffixZeros[i:..n).
            final int j = getFirstRowWithEnoughZeros(suffixZeros, i, neededZeros);
            if (j == -1)
                return -1;
            // Move the rows[j] to the rows[i].
            for (int k = j; k > i; --k)
                suffixZeros[k] = suffixZeros[k - 1];
            ans += j - i;
        }

        return ans;
    }

    private int getSuffixZeroCount(int[] row) {
        for (int i = row.length - 1; i >= 0; --i)
            if (row[i] == 1)
                return row.length - i - 1;
        return row.length;
    }

    // Returns first row that has suffix zeros > `neededZeros` in suffixZeros[i:..n).
    private int getFirstRowWithEnoughZeros(int[] suffixZeros, int i, int neededZeros) {
        for (int j = i; j < suffixZeros.length; ++j)
            if (suffixZeros[j] >= neededZeros)
                return j;
        return -1;
    }
}
```

540. 1537.java

Path: LeetCode-main/solutions/1537. Get the Maximum Score/1537.java

```
class Solution {
    public int maxSum(int[] nums1, int[] nums2) {
        final int MOD = 1_000_000_007;
        // Keep the running the sum of `nums1` and `nums2` before the next rendezvous.
        // Since `nums1` and `nums2` are increasing, move forward on the smaller one
        // to ensure we don't miss any rendezvous. When meet rendezvous, choose the
        // better path.
        long ans = 0;
        // sum(nums1) in (the prevoious rendezvous, the next rendezvous)
        long sum1 = 0;
        // sum(nums2) in (the prevoious rendezvous, the next rendezvous)
        int i = 0; // nums1's index
        int j = 0; // nums2's index

        while (i < nums1.length && j < nums2.length)
            if (nums1[i] < nums2[j]) {
                sum1 += nums1[i++];
            } else if (nums1[i] > nums2[j]) {
                sum2 += nums2[j++];
            } else { // An rendezvous happens.
                ans += Math.max(sum1, sum2) + nums1[i];
                sum1 = 0;
                sum2 = 0;
                ++i;
                ++j;
            }

        while (i < nums1.length)
            sum1 += nums1[i++];

        while (j < nums2.length)
            sum2 += nums2[j++];

        return (int) ((ans + Math.max(sum1, sum2)) % MOD);
    }
}
```

541. 1538.java

Path: LeetCode-main/solutions/1538. Guess the Majority in a Hidden Array/1538.java

```
/**
 * // This is the ArrayReader's API interface.
 * // You should not implement it, or speculate about its implementation
 * interface ArrayReader {
 *     // Compares 4 different elements in the array
 *     // Returns 4 if the values of the 4 elements are the same (0 or 1).
 *     // Returns 2 if three elements have a value equal to 0 and one element has
 *     //         value equal to 1 or vice versa.
 *     // Returns 0 if two element have a value equal to 0 and two elements have
 *     //         a value equal to 1.
 *     public int query(int a, int b, int c, int d);
 *
 *     // Returns the length of the array
 *     public int length();
 * };
 */

class Solution {
    public int guessMajority(ArrayReader reader) {
        final int n = reader.length();
        final int query0123 = reader.query(0, 1, 2, 3);
        final int query1234 = reader.query(1, 2, 3, 4);
        // the number of numbers that are same as `nums[0]`
        int zeros = 1;
        // the number of numbers that are different from `nums[0]`
        int nonZeros = 0;
        // any index i s.t. nums[i] != nums[0]
        int indexNot0 = -1;

        // Find which group nums[1..3] belong to.
        for (int i = 1; i <= 3; ++i) {
            List<Integer> abcd = getABCD(i);
            if (reader.query(abcd.get(0), abcd.get(1), abcd.get(2), abcd.get(3)) == query1234) {
                ++zeros;
            } else {
                ++nonZeros;
                indexNot0 = i;
            }
        }

        // Find which group nums[4..n] belong to.
        for (int i = 4; i < n; ++i)
            if (reader.query(1, 2, 3, i) == query0123) {
                ++zeros;
            } else {
                ++nonZeros;
                indexNot0 = i;
            }

        if (zeros == nonZeros)
            return -1;
        if (zeros > nonZeros)
            return 0;
        return indexNot0;
    }

    // Returns [0..4] except i.
    private List<Integer> getABCD(int i) {
        List<Integer> abcd = new ArrayList<>(List.of(0));
        for (int j = 1; j <= 3; ++j)
            if (j != i)
                abcd.add(j);
        abcd.add(4);
        return abcd;
    }
}
```


542. 1539.java

Path: LeetCode-main/solutions/1539. Kth Missing Positive Number/1539.java

```
class Solution {
    public int findKthPositive(int[] arr, int k) {
        int l = 0;
        int r = arr.length;

        // Find the first index l s.t. nMissing(l) = A[l] - l - 1 >= k.
        while (l < r) {
            final int m = (l + r) / 2;
            if (arr[m] - m - 1 >= k)
                r = m;
            else
                l = m + 1;
        }

        // The k-th missing positive
        // = A[l - 1] + k - nMissing(l - 1)
        // = A[l - 1] + k - (A[l - 1] - (l - 1) - 1)
        // = A[l - 1] + k - (A[l - 1] - l)
        // = l + k
        return l + k;
    }
}
```

543. 154.java

Path: LeetCode-main/solutions/154. Find Minimum in Rotated Sorted Array II/154.java

```
class Solution {
    public int findMin(int[] nums) {
        int l = 0;
        int r = nums.length - 1;

        while (l < r) {
            final int m = (l + r) / 2;
            if (nums[m] == nums[r])
                --r;
            else if (nums[m] < nums[r])
                r = m;
            else
                l = m + 1;
        }
        return nums[l];
    }
}
```

544. 1540-2.java

Path: LeetCode-main/solutions/1540. Can Convert String in K Moves/1540-2.java

```
class Solution {
    public boolean canConvertString(String s, String t, int k) {
        if (s.length() != t.length())
            return false;

        // e.g. s = "aab", t = "bbc", so shiftCount[1] = 3
        // 1. a -> b, need 1 move.
        // 2. a -> b, need 1 + 26 moves.
        // 3. b -> c, need 1 + 26 * 2 moves.
        int[] shiftCount = new int[26];

        for (int i = 0; i < s.length(); ++i) {
            final int shift = (t.charAt(i) - s.charAt(i) + 26) % 26;
            if (shift == 0)
                continue;
            if (shift + 26 * shiftCount[shift] > k)
                return false;
            ++shiftCount[shift];
        }

        return true;
    }
}
```

545. 1540.java

Path: LeetCode-main/solutions/1540. Can Convert String in K Moves/1540.java

```
class Solution {
    public boolean canConvertString(String s, String t, int k) {
        if (s.length() != t.length())
            return false;

        // e.g. s = "aab", t = "bbc", so shiftCount[1] = 3
        // 1. a -> b, need 1 move.
        // 2. a -> b, need 1 + 26 moves.
        // 3. b -> c, need 1 + 26 * 2 moves.
        int[] shiftCount = new int[26];

        for (int i = 0; i < s.length(); ++i)
            ++shiftCount[(t.charAt(i) - s.charAt(i) + 26) % 26];

        for (int shift = 1; shift < 26; ++shift)
            if (shift + 26 * (shiftCount[shift] - 1) > k)
                return false;

        return true;
    }
}
```

546. 1541.java

Path: LeetCode-main/solutions/1541. Minimum Insertions to Balance a Parentheses String/1541.java

```
class Solution {
    public int minInsertions(String s) {
        int neededRight = 0; // Increment by 2 for each '('.
        int missingLeft = 0; // Increment by 1 for each missing '('.
        int missingRight = 0; // Increment by 1 for each missing ')'.

        for (final char c : s.toCharArray())
            if (c == '(') {
                if (neededRight % 2 == 1) {
                    // e.g. "()(..."
                    ++missingRight;
                    --neededRight;
                }
                neededRight += 2;
            } else if (--neededRight < 0) { // c == ')'
                // e.g. "()))..."
                ++missingLeft;
                neededRight += 2;
            }

        return neededRight + missingLeft + missingRight;
    }
}
```

547. 1542.java

Path: LeetCode-main/solutions/1542. Find Longest Awesome Substring/1542.java

```
class Solution {
    public int longestAwesome(String s) {
        int ans = 0;
        int prefix = 0; // the binary prefix
        int[] prefixToIndex = new int[1024];
        Arrays.fill(prefixToIndex, s.length());
        prefixToIndex[0] = -1;

        for (int i = 0; i < s.length(); ++i) {
            prefix ^= 1 << s.charAt(i) - '0';
            ans = Math.max(ans, i - prefixToIndex[prefix]);
            for (int j = 0; j < 10; ++j)
                ans = Math.max(ans, i - prefixToIndex[prefix ^ 1 << j]);
            prefixToIndex[prefix] = Math.min(prefixToIndex[prefix], i);
        }

        return ans;
    }
}
```

548. 1544.java

Path: LeetCode-main/solutions/1544. Make The String Great/1544.java

```
class Solution {
    public String makeGood(String s) {
        StringBuilder sb = new StringBuilder();
        for (char c : s.toCharArray())
            if (sb.length() > 0 && isBadPair(sb.charAt(sb.length() - 1), c))
                sb.deleteCharAt(sb.length() - 1);
            else
                sb.append(c);
        return sb.toString();
    }

    private boolean isBadPair(char a, char b) {
        return a != b && Character.toLowerCase(a) == Character.toLowerCase(b);
    }
}
```

549. 1545.java

Path: LeetCode-main/solutions/1545. Find Kth Bit in Nth Binary String/1545.java

```
class Solution {
    public char findKthBit(int n, int k) {
        if (n == 1)
            return '0';
        final int midIndex = (int) Math.pow(2, n - 1); // 1-indexed
        if (k == midIndex)
            return '1';
        if (k < midIndex)
            return findKthBit(n - 1, k);
        return findKthBit(n - 1, midIndex * 2 - k) == '0' ? '1' : '0';
    }
}
```


550. 1546.java

Path: LeetCode-main/solutions/1546. Maximum Number of Non-Overlapping Subarrays With Sum Equals Target/1546.java

```
class Solution {
    public static int maxNonOverlapping(int[] nums, int target) {
        // Ending the subarray ASAP always has a better result.
        int ans = 0;
        int prefix = 0;
        Set<Integer> prefixes = new HashSet<>(Arrays.asList(0));

        // Greedily find the subarrays that equal to the target.
        for (int num : nums) {
            prefix += num;
            // Check if there is a subarray ends in here and equals to the target.
            if (prefixes.contains(prefix - target)) {
                // Find one and discard all the prefixes that have been used.
                ans++;
                prefix = 0;
                prefixes = new HashSet<>(Arrays.asList(0));
            } else {
                prefixes.add(prefix);
            }
        }

        return ans;
    }
}
```

551. 1547-2.java

Path: LeetCode-main/solutions/1547. Minimum Cost to Cut a Stick/1547-2.java

```
class Solution {
    public int minCost(int n, int[] cuts) {
        int[] arr = new int[cuts.length + 2];
        System.arraycopy(cuts, 0, arr, 1, cuts.length);
        arr[0] = 0;
        arr[arr.length - 1] = n;

        Arrays.sort(arr);

        // dp[i][j] := minCost(cuts[i..j])
        int[][] dp = new int[arr.length][arr.length];

        for (int d = 2; d < arr.length; ++d)
            for (int i = 0; i + d < arr.length; ++i) {
                final int j = i + d;
                dp[i][j] = Integer.MAX_VALUE;
                for (int k = i + 1; k < j; ++k)
                    dp[i][j] = Math.min(dp[i][j], arr[j] - arr[i] + dp[i][k] + dp[k][j]);
            }

        return dp[0][arr.length - 1];
    }
}
```

552. 1547.java

Path: LeetCode-main/solutions/1547. Minimum Cost to Cut a Stick/1547.java

```
class Solution {
    public int minCost(int n, int[] cuts) {
        int[] extendedCuts = new int[cuts.length + 2];
        int[][] mem = new int[extendedCuts.length][extendedCuts.length];
        System.arraycopy(cuts, 0, extendedCuts, 1, cuts.length);
        extendedCuts[0] = 0;
        extendedCuts[extendedCuts.length - 1] = n;
        Arrays.sort(extendedCuts);
        Arrays.stream(mem).forEach(A -> Arrays.fill(A, Integer.MAX_VALUE));
        return minCost(extendedCuts, 0, extendedCuts.length - 1, mem);
    }

    // Returns minCost(cuts[i..j]).
    private int minCost(int[] cuts, int i, int j, int[][] mem) {
        if (j - i <= 1)
            return 0;
        if (mem[i][j] != Integer.MAX_VALUE)
            return mem[i][j];

        for (int k = i + 1; k < j; ++k)
            mem[i][j] = Math.min(mem[i][j],
                                   cuts[j] - cuts[i] + minCost(cuts, i, k, mem) + minCost(cuts, k, j, mem));

        return mem[i][j];
    }
}
```

553. 1548.java

Path: LeetCode-main/solutions/1548. The Most Similar Path in a Graph/1548.java

```
class Solution {
    public List<Integer> mostSimilar(int n, int[][] roads, String[] names, String[] targetPath) {
        this.names = names;
        this.targetPath = targetPath;
        // cost[i][j] := the minimum cost to start from names[i] in path[j]
        this.cost = new int[names.length][targetPath.length];
        // next[i][j] := the best next of names[i] in path[j]
        this.next = new int[names.length][targetPath.length];
        this.graph = new List[n];

        Arrays.stream(cost).forEach(a -> Arrays.fill(a, -1));
        Arrays.setAll(graph, i -> new ArrayList<>());

        for (int[] road : roads) {
            graph[road[0]].add(road[1]);
            graph[road[1]].add(road[0]);
        }

        int minDist = Integer.MAX_VALUE;
        int start = 0;

        // Start from different names
        for (int i = 0; i < names.length; ++i) {
            final int dist = dfs(i, 0);
            if (dist < minDist) {
                minDist = dist;
                start = i;
            }
        }

        List<Integer> ans = new ArrayList<>();

        while (ans.size() < targetPath.length) {
            ans.add(start);
            start = next[start][ans.size() - 1];
        }

        return ans;
    }

    private String[] names;
    private String[] targetPath;
    private int[][] cost;
    private int[][] next;
    private List<Integer>[] graph;

    private int dfs(int nameIndex, int pathIndex) {
        if (cost[nameIndex][pathIndex] != -1)
            return cost[nameIndex][pathIndex];

        final int editDist = names[nameIndex].equals(targetPath[pathIndex]) ? 0 : 1;
        if (pathIndex == targetPath.length - 1)
            return editDist;

        int minDist = Integer.MAX_VALUE;

        for (final int v : graph[nameIndex]) {
            final int dist = dfs(v, pathIndex + 1);
            if (dist < minDist) {
                minDist = dist;
                next[nameIndex][pathIndex] = v;
            }
        }

        return cost[nameIndex][pathIndex] = editDist + minDist;
    }
}
```

554. 155.java

Path: LeetCode-main/solutions/155. Min Stack/155.java

```
class MinStack {
    public void push(int x) {
        if (stack.isEmpty())
            stack.push(new int[] {x, x});
        else
            stack.push(new int[] {x, Math.min(x, stack.peek()[1])});
    }

    public void pop() {
        stack.pop();
    }

    public int top() {
        return stack.peek()[0];
    }

    public int getMin() {
        return stack.peek()[1];
    }

    private Stack<int[]> stack = new Stack<>(); // {x, min}
}
```

555. 1550.java

Path: LeetCode-main/solutions/1550. Three Consecutive Odds/1550.java

```
class Solution {  
    public boolean threeConsecutiveOdds(int[] arr) {  
        int count = 0;  
        for (final int a : arr) {  
            count = a % 2 == 0 ? 0 : count + 1;  
            if (count == 3)  
                return true;  
        }  
        return false;  
    }  
}
```

556. 1551.java

Path: LeetCode-main/solutions/1551. Minimum Operations to Make Array Equal/1551.java

```
class Solution {
    public int minOperations(int n) {
        //      median := median of arr
        //      diffs[i] := median - arr[i] where i <= i <= n / 2
        //      ans := sum(diffs)
        // e.g.
        // n = 5, arr = [1, 3, 5, 7, 9], diffs = [4, 2]
        //      ans = (4 + 2) * 2 / 2 = 6
        // n = 6, arr = [1, 3, 5, 7, 9, 11], diffs = [5, 3, 1]
        //      ans = (5 + 1) * 3 / 2 = 9
        final int halfSize = n / 2;
        final int median = (arr(n) + arr(1)) / 2;
        final int firstDiff = median - arr(1);
        final int lastDiff = median - arr(halfSize);
        return (firstDiff + lastDiff) * halfSize / 2;
    }

    // Returns the i-th element of `arr`, where 1 <= i <= n.
    private int arr(int i) {
        return (i - 1) * 2 + 1;
    }
}
```

557. 1552.java

Path: LeetCode-main/solutions/1552. Magnetic Force Between Two Balls/1552.java

```
class Solution {
    public int maxDistance(int[] position, int m) {
        Arrays.sort(position);

        int l = 1;
        int r = position[position.length - 1] - position[0];

        while (l < r) {
            final int mid = r - (r - l) / 2;
            if (numBalls(position, mid) >= m) // There're too many balls.
                l = mid;
            else // There're too few balls.
                r = mid - 1;
        }

        return l;
    }

    private int numBalls(int[] position, int force) {
        int balls = 0;
        int prevPosition = -force;
        for (final int pos : position)
            if (pos - prevPosition >= force) {
                balls++;
                prevPosition = pos;
            }
        return balls;
    }
}
```


558. 1553.java

Path: LeetCode-main/solutions/1553. Minimum Number of Days to Eat N Oranges/1553.java

```
class Solution {
    public int minDays(int n) {
        if (n <= 1)
            return n;
        if (dp.containsKey(n))
            return dp.get(n);

        final int res = 1 + Math.min(minDays(n / 3) + n % 3, //
                                     minDays(n / 2) + n % 2);
        dp.put(n, res);
        return res;
    }
    private Map<Integer, Integer> dp = new HashMap<>();
}
```

559. 1554-2.java

Path: LeetCode-main/solutions/1554. Strings Differ by One Character/1554-2.java

```
class Solution {
    public boolean differByOne(String[] dict) {
        final int m = dict[0].length();
        int[] wordToHash = new int[dict.length];

        for (int i = 0; i < dict.length; ++i)
            wordToHash[i] = getHash(dict[i]);

        // Compute the hash without each letter.
        // e.g. hash of "abc" = 26^2 * 'a' + 26 * 'b' + 'c'
        // newHash of "a*c" = hash - 26 * 'b'
        int coefficient = 1;
        for (int j = m - 1; j >= 0; --j) {
            Map<Integer, List<Integer>> newHashToIndices = new HashMap<>();
            for (int i = 0; i < dict.length; ++i) {
                final int newHash =
                    (int) ((wordToHash[i] - (long) coefficient * val(dict[i].charAt(j)) % HASH + HASH) %
                        HASH);
                if (newHashToIndices.containsKey(newHash)) {
                    for (final int index : newHashToIndices.get(newHash)) {
                        if (dict[i].substring(0, j).equals(dict[index].substring(0, j)) &&
                            dict[i].substring(j + 1).equals(dict[index].substring(j + 1))) {
                            return true;
                        }
                    }
                }
                newHashToIndices.putIfAbsent(newHash, new ArrayList<>());
                newHashToIndices.get(newHash).add(i);
            }
            coefficient = (int) (coefficient * BASE % HASH);
        }

        return false;
    }

    private static final long BASE = 26;
    private static final int HASH = 1_000_000_007;

    private static int val(char c) {
        return c - 'a';
    }

    // Returns the hash of `s`. Assume the length of `s` is m.
    // e.g. getHash(s) = 26^(m - 1) * s[0] + 26^(m - 2) * s[1] + ... + s[m - 1].
    private int getHash(String s) {
        int hash = 0;
        for (final char c : s.toCharArray()) {
            hash = (int) ((hash * BASE + val(c)) % HASH);
        }
        return hash;
    }
}
```

560. 1556.java

Path: LeetCode-main/solutions/1556. Thousand Separator/1556.java

```
class Solution {
    public String thousandSeparator(int n) {
        final String s = Integer.toString(n);
        StringBuilder ans = new StringBuilder();

        for (int i = 0; i < s.length(); ++i) {
            if (i > 0 && (s.length() - i) % 3 == 0)
                ans.append('.');
            ans.append(s.charAt(i));
        }

        return ans.toString();
    }
}
```

561. 1557.java

Path: LeetCode-main/solutions/1557. Minimum Number of Vertices to Reach All Nodes/1557.java

```
class Solution {
    public List<Integer> findSmallestSetOfVertices(int n, List<List<Integer>> edges) {
        List<Integer> ans = new ArrayList<>();
        int[] inDegrees = new int[n];

        for (List<Integer> edge : edges)
            ++inDegrees[edge.get(1)];

        for (int i = 0; i < inDegrees.length; ++i)
            if (inDegrees[i] == 0)
                ans.add(i);

        return ans;
    }
}
```

562. 1558.java

Path: LeetCode-main/solutions/1558. Minimum Numbers of Function Calls to Make Target Array/1558.java

```
class Solution {
    public int minOperations(int[] nums) {
        final int mx = Arrays.stream(nums).max().getAsInt();
        return Arrays.stream(nums).map(num -> Integer.bitCount(num)).sum() +
            (mx == 0 ? 0 : (int) (Math.log(mx) / Math.log(2)));
    }
}
```

563. 1559.java

Path: LeetCode-main/solutions/1559. Detect Cycles in 2D Grid/1559.java

```
class Solution {
    public boolean containsCycle(char[][] grid) {
        final int m = grid.length;
        final int n = grid[0].length;
        boolean[][] seen = new boolean[m][n];

        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j) {
                if (seen[i][j])
                    continue;
                if (dfs(grid, i, j, -1, -1, grid[i][j], seen))
                    return true;
            }

        return false;
    }

    private static final int[][] DIRS = {{0, 1}, {1, 0}, {0, -1}, {-1, 0}};

    private boolean dfs(char[][] grid, int i, int j, int prevI, int prevJ, char c, boolean[][] seen) {
        seen[i][j] = true;

        for (int[] dir : DIRS) {
            final int x = i + dir[0];
            final int y = j + dir[1];
            if (x < 0 || x == grid.length || y < 0 || y == grid[0].length)
                continue;
            if (x == prevI && y == prevJ)
                continue;
            if (grid[x][y] != c)
                continue;
            if (seen[x][y])
                return true;
            if (dfs(grid, x, y, i, j, c, seen))
                return true;
        }

        return false;
    }
}
```

564. 156-2.java

Path: LeetCode-main/solutions/156. Binary Tree Upside Down/156-2.java

```
class Solution {
    public TreeNode upsideDownBinaryTree(TreeNode root) {
        TreeNode prevRoot = null;
        TreeNode prevRightChild = null;

        while (root != null) {
            TreeNode nextRoot = root.left; // Cache the next root.
            root.left = prevRightChild;
            prevRightChild = root.right;
            root.right = prevRoot;
            prevRoot = root; // Record the previous root.
            root = nextRoot; // Update the root.
        }

        return prevRoot;
    }
}
```

565. 156.java

Path: LeetCode-main/solutions/156. Binary Tree Upside Down/156.java

```
class Solution {
    public TreeNode upsideDownBinaryTree(TreeNode root) {
        if (root == null || root.left == null)
            return root;

        TreeNode newRoot = upsideDownBinaryTree(root.left);
        root.left.left = root.right; // 2's left = 3 (root's right)
        root.left.right = root;      // 2's right = 1 (root)
        root.left = null;
        root.right = null;
        return newRoot;
    }
}
```


566. 1560.java

Path: LeetCode-main/solutions/1560. Most Visited Sector in a Circular Track/1560.java

```
class Solution {
    public List<Integer> mostVisited(int n, int[] rounds) {
        // 1. If start <= end, [start, end] is the most visited.
        //
        //      s ----- n
        // 1 ----- n
        // 1 ----- e
        //
        // 2. If start > end, [1, end] and [start, n] are the most visited.
        //
        //      s -- n
        // 1 ----- n
        // 1 ----- e
        final int start = rounds[0];
        final int end = rounds[rounds.length - 1];
        List<Integer> ans = new ArrayList<>();
        for (int i = 1; i <= n; ++i)
            if (start <= end) {
                if (start <= i && i <= end)
                    ans.add(i);
            } else { // start > end
                if (i >= start || i <= end)
                    ans.add(i);
            }
        return ans;
    }
}
```

567. 1561.java

Path: LeetCode-main/solutions/1561. Maximum Number of Coins You Can Get/1561.java

```
class Solution {
    public int maxCoins(int[] piles) {
        Arrays.sort(piles);

        int ans = 0;

        // piles = [S S M L M L], pick all the M.
        for (int i = piles.length / 3; i < piles.length; i += 2)
            ans += piles[i];

        return ans;
    }
}
```

568. 1562.java

Path: LeetCode-main/solutions/1562. Find Latest Group of Size M/1562.java

```
class Solution {
    public int findLatestStep(int[] arr, int m) {
        if (arr.length == m)
            return arr.length;

        int ans = -1;
        int step = 0;
        // sizes[i] := the size of the group starting from i or ending in i
        // (1-indexed)
        int[] sizes = new int[arr.length + 2];

        for (final int i : arr) {
            ++step;
            // In the previous step, there exists a group with a size of m.
            if (sizes[i - 1] == m || sizes[i + 1] == m)
                ans = step - 1;
            final int head = i - sizes[i - 1];
            final int tail = i + sizes[i + 1];
            sizes[head] = tail - head + 1;
            sizes[tail] = tail - head + 1;
        }

        return ans;
    }
}
```

569. 1563.java

Path: LeetCode-main/solutions/1563. Stone Game V/1563.java

```
class Solution {
    public int stoneGameV(int[] stoneValue) {
        final int n = stoneValue.length;
        int[][] mem = new int[n][n];
        int[] prefix = new int[n + 1];
        Arrays.stream(mem).forEach(A -> Arrays.fill(A, Integer.MIN_VALUE));
        for (int i = 0; i < n; ++i)
            prefix[i + 1] = prefix[i] + stoneValue[i];
        return stoneGameV(stoneValue, 0, n - 1, prefix, mem);
    }

    // Returns the maximum score that Alice can obtain from stoneValue[i..j].
    private int stoneGameV(int[] stoneValue, int i, int j, int[] prefix, int[][] mem) {
        if (i == j)
            return 0;
        if (mem[i][j] != Integer.MIN_VALUE)
            return mem[i][j];

        // Try all the possible partitions.
        for (int p = i; p < j; ++p) {
            // sum(stoneValue[i..p])
            final int leftSum = prefix[p + 1] - prefix[i];
            final int throwRight = leftSum + stoneGameV(stoneValue, i, p, prefix, mem);
            // sum(stoneValue[p + 1..j])
            final int rightSum = prefix[j + 1] - prefix[p + 1];
            final int throwLeft = rightSum + stoneGameV(stoneValue, p + 1, j, prefix, mem);
            if (leftSum < rightSum)
                mem[i][j] = Math.max(mem[i][j], throwRight);
            else if (leftSum > rightSum)
                mem[i][j] = Math.max(mem[i][j], throwLeft);
            else
                mem[i][j] = Math.max(Math.max(mem[i][j], throwLeft), throwRight);
        }

        return mem[i][j];
    }
}
```

570. 1564-2.java

Path: LeetCode-main/solutions/1564. Put Boxes Into the Warehouse I/1564-2.java

```
class Solution {
    public int maxBoxesInWarehouse(int[] boxes, int[] warehouse) {
        boxes = Arrays.stream(boxes)
            .boxed()
            .sorted(Collections.reverseOrder())
            .mapToInt(Integer::intValue)
            .toArray();

        int i = 0; // warehouse's index

        for (final int box : boxes)
            if (i < warehouse.length && warehouse[i] >= box)
                ++i;

        return i;
    }
}
```

571. 1564.java

Path: LeetCode-main/solutions/1564. Put Boxes Into the Warehouse I/1564.java

```
class Solution {
    public int maxBoxesInWarehouse(int[] boxes, int[] warehouse) {
        int[] realWarehouse = new int[warehouse.length];
        realWarehouse[0] = warehouse[0];

        for (int i = 1; i < warehouse.length; i++)
            realWarehouse[i] = Math.min(realWarehouse[i - 1], warehouse[i]);

        Arrays.sort(boxes);
        int i = 0; // boxes' index
        for (int j = realWarehouse.length - 1; j >= 0; j--)
            if (i < boxes.length && boxes[i] <= realWarehouse[j])
                ++i;

        return i;
    }
}
```

572. 1566.java

Path: LeetCode-main/solutions/1566. Detect Pattern of Length M Repeated K or More Times/1566.java

```
class Solution {
    public boolean containsPattern(int[] arr, int m, int k) {
        int count = 0;
        for (int i = m; i < arr.length; ++i) {
            count = arr[i] == arr[i - m] ? count + 1 : 0;
            if (count == m * k - m)
                return true;
        }
        return false;
    }
}
```

573. 1567.java

Path: LeetCode-main/solutions/1567. Maximum Length of Subarray With Positive Product/1567.java

```
class Solution {
    public int getMaxLen(int[] nums) {
        int ans = 0;
        // the maximum length of subarrays ending in `num` with a negative product
        int neg = 0;
        // the maximum length of subarrays ending in `num` with a positive product
        int pos = 0;

        for (final int num : nums) {
            pos = num == 0 ? 0 : pos + 1;
            neg = num == 0 || neg == 0 ? 0 : neg + 1;
            if (num < 0) {
                final int temp = pos;
                pos = neg;
                neg = temp;
            }
            ans = Math.max(ans, pos);
        }

        return ans;
    }
}
```


574. 1568.java

Path: LeetCode-main/solutions/1568. Minimum Number of Days to Disconnect Island/1568.java

```
class Solution {
    public int minDays(int[][] grid) {
        if (disconnected(grid))
            return 0;

        // Try to remove 1 land.
        for (int i = 0; i < grid.length; ++i)
            for (int j = 0; j < grid[0].length; ++j)
                if (grid[i][j] == 1) {
                    grid[i][j] = 0;
                    if (disconnected(grid))
                        return 1;
                    grid[i][j] = 1;
                }

        // Remove 2 lands.
        return 2;
    }

    private final int[][] DIRS = {{0, 1}, {1, 0}, {0, -1}, {-1, 0}};

    private boolean disconnected(int[][] grid) {
        int islandsCount = 0;
        boolean[][] seen = new boolean[grid.length][grid[0].length];
        for (int i = 0; i < grid.length; ++i)
            for (int j = 0; j < grid[0].length; ++j) {
                if (grid[i][j] == 0 || seen[i][j])
                    continue;
                if (++islandsCount > 1)
                    return true;
                dfs(grid, i, j, seen);
            }

        return islandsCount != 1;
    }

    private void dfs(int[][] grid, int i, int j, boolean[][] seen) {
        seen[i][j] = true;
        for (int[] dir : DIRS) {
            int x = i + dir[0];
            int y = j + dir[1];
            if (x < 0 || x == grid.length || y < 0 || y == grid[0].length)
                continue;
            if (grid[x][y] == 0 || seen[x][y])
                continue;
            dfs(grid, x, y, seen);
        }
    }
}
```

575. 1569.java

Path: LeetCode-main/solutions/1569. Number of Ways to Reorder Array to Get Same BST/1569.java

```
class Solution {
    public int numOfWays(int[] nums) {
        comb = generate(nums.length + 1);
        return ways(Arrays.stream(nums).boxed().collect(Collectors.toList())) - 1;
    }

    private static final int MOD = 1_000_000_007;
    // comb[n][k] := C(n, k)
    private List<List<Integer>> comb;

    private int ways(List<Integer> nums) {
        if (nums.size() <= 2)
            return 1;

        List<Integer> left = new ArrayList<>();
        List<Integer> right = new ArrayList<>();

        for (int i = 1; i < nums.size(); ++i)
            if (nums.get(i) < nums.get(0))
                left.add(nums.get(i));
            else
                right.add(nums.get(i));

        long ans = comb.get(nums.size() - 1).get(left.size());
        ans = (ans * ways(left)) % MOD;
        ans = (ans * ways(right)) % MOD;
        return (int) ans;
    }

    // Same as 118. Pascal's Triangle
    public List<List<Integer>> generate(int numRows) {
        List<List<Integer>> comb = new ArrayList<>();

        for (int i = 0; i < numRows; ++i) {
            Integer[] temp = new Integer[i + 1];
            Arrays.fill(temp, 1);
            comb.add(Arrays.asList(temp));
        }

        for (int i = 2; i < numRows; ++i)
            for (int j = 1; j < comb.get(i).size() - 1; ++j)
                comb.get(i).set(j, (comb.get(i - 1).get(j - 1) + comb.get(i - 1).get(j)) % MOD);

        return comb;
    }
}
```

576. 157.java

Path: LeetCode-main/solutions/157. Read N Characters Given Read4/157.java

```
/**
 * The read4 API is defined in the parent class Reader4.
 *   int read4(char[] buf4);
 */

public class Solution extends Reader4 {
    /**
     * @param buf Destination buffer
     * @param n Number of characters to read
     * @return The number of actual characters read
     */
    public int read(char[] buf, int n) {
        char[] buf4 = new char[4];
        int i4 = 0; // buf4's index
        int n4 = 0; // buf4's size
        int i = 0; // buf's index

        while (i < n) {
            if (i4 == n4) { // All the characters in the buf4 are consumed.
                i4 = 0; // Reset the buf4's index.
                n4 = read4(buf4); // Read <= 4 characters from the file to the buf4.
                if (n4 == 0) // Reach the EOF.
                    return i;
            }
            buf[i++] = buf4[i4++];
        }

        return i;
    }
}
```

577. 1570-2.java

Path: LeetCode-main/solutions/1570. Dot Product of Two Sparse Vectors/1570-2.java

```
class SparseVector {
    SparseVector(int[] nums) {
        for (int i = 0; i < nums.length; ++i)
            if (nums[i] != 0)
                v.add(new T(i, nums[i]));
    }

    // Returns the dot product of the two sparse vectors.
    public int dotProduct(SparseVector vec) {
        int ans = 0;

        for (int i = 0, j = 0; i < v.size() && j < vec.v.size(); )
            if (v.get(i).index == vec.v.get(j).index)
                ans += v.get(i++).num * vec.v.get(j++).num;
            else if (v.get(i).index < vec.v.get(j).index)
                ++i;
            else
                ++j;

        return ans;
    }

    private record T(int index, int num) {}
    private List<T> v = new ArrayList<>(); // [(index, num)]
}
```

578. 1570.java

Path: LeetCode-main/solutions/1570. Dot Product of Two Sparse Vectors/1570.java

```
class SparseVector {
    SparseVector(int[] nums) {
        for (int i = 0; i < nums.length; ++i)
            if (nums[i] != 0)
                indexToNum.put(i, nums[i]);
    }

    // Returns the dot product of the two sparse vectors.
    public int dotProduct(SparseVector vec) {
        if (indexToNum.size() < vec.indexToNum.size())
            return vec.dotProduct(this);

        int ans = 0;

        for (final int index : vec.indexToNum.keySet())
            if (indexToNum.containsKey(index))
                ans += vec.indexToNum.get(index) * indexToNum.get(index);

        return ans;
    }

    private Map<Integer, Integer> indexToNum = new HashMap<>();
}
```

579. 1572.java

Path: LeetCode-main/solutions/1572. Matrix Diagonal Sum/1572.java

```
class Solution {
    public int diagonalSum(int[][] mat) {
        final int n = mat.length;
        int ans = 0;

        for (int i = 0; i < n; ++i) {
            ans += mat[i][i];
            ans += mat[n - 1 - i][i];
        }

        return n % 2 == 0 ? ans : ans - mat[n / 2][n / 2];
    }
}
```

580. 1573.java

Path: LeetCode-main/solutions/1573. Number of Ways to Split a String/1573.java

```
class Solution {
    public int numWays(String s) {
        final int MOD = 1_000_000_007;
        final int ones = (int) s.chars().filter(c -> c == '1').count();
        if (ones % 3 != 0)
            return 0;
        if (ones == 0) {
            final long n = s.length();
            return (int) ((n - 1) * (n - 2) / 2 % MOD);
        }

        int s1End = -1;
        int s2Start = -1;
        int s2End = -1;
        int s3Start = -1;
        int onesSoFar = 0;

        for (int i = 0; i < s.length(); ++i) {
            if (s.charAt(i) == '1')
                ++onesSoFar;
            if (s1End == -1 && onesSoFar == ones / 3)
                s1End = i;
            else if (s2Start == -1 && onesSoFar == ones / 3 + 1)
                s2Start = i;
            if (s2End == -1 && onesSoFar == ones / 3 * 2)
                s2End = i;
            else if (s3Start == -1 && onesSoFar == ones / 3 * 2 + 1)
                s3Start = i;
        }

        return (int) ((long) (s2Start - s1End) * (s3Start - s2End) % MOD);
    }
}
```

581. 1574.java

Path: LeetCode-main/solutions/1574. Shortest Subarray to be Removed to Make Array Sorted/1574.java

```
class Solution {
    public int findLengthOfShortestSubarray(int[] arr) {
        final int n = arr.length;
        int l = 0;
        int r = n - 1;

        // arr[0..l] is non-decreasing prefix.
        while (l < n - 1 && arr[l + 1] >= arr[l])
            ++l;
        // arr[r..n - 1] is non-decreasing suffix.
        while (r > 0 && arr[r - 1] <= arr[r])
            --r;
        // Remove arr[l + 1..n - 1] or arr[0..r - 1]
        int ans = Math.min(n - 1 - l, r);

        // Since arr[0..l] and arr[r..n - 1] are non-decreasing, we place pointers
        // at the rightmost indices, l and n - 1, and greedily shrink them toward
        // the leftmost indices, 0 and r, respectively. By removing arr[i + 1..j],
        // we ensure that `arr` becomes non-decreasing.
        int i = l;
        int j = n - 1;
        while (i >= 0 && j >= r && j > i) {
            if (arr[i] <= arr[j])
                --j;
            else
                --i;
            ans = Math.min(ans, j - i);
        }

        return ans;
    }
}
```


582. 1575-2.java

Path: LeetCode-main/solutions/1575. Count All Possible Routes/1575-2.java

```
class Solution {
    public int countRoutes(int[] locations, int start, int finish, int fuel) {
        final int MOD = 1_000_000_007;
        int n = locations.length;
        // dp[i][j] := the number of ways to reach the `finish` city from the i-th
        // city with `j` fuel
        int[][] dp = new int[n][fuel + 1];

        for (int f = 0; f <= fuel; ++f)
            dp[finish][f] = 1;

        for (int f = 0; f <= fuel; ++f)
            for (int i = 0; i < n; ++i)
                for (int j = 0; j < n; ++j) {
                    if (i == j)
                        continue;
                    final int requiredFuel = Math.abs(locations[i] - locations[j]);
                    if (requiredFuel <= f) {
                        dp[i][f] += dp[j][f - requiredFuel];
                        dp[i][f] %= MOD;
                    }
                }

        return dp[start][fuel];
    }
}
```

583. 1575.java

Path: LeetCode-main/solutions/1575. Count All Possible Routes/1575.java

```
class Solution {
    public int countRoutes(int[] locations, int start, int finish, int fuel) {
        Integer[][] mem = new Integer[locations.length][fuel + 1];
        return countRoutes(locations, start, finish, fuel, mem);
    }

    private static final int MOD = 1_000_000_007;

    // Returns the number of ways to reach the `finish` city from the i-th city
    // with `fuel` fuel.
    private int countRoutes(int[] locations, int i, int finish, int fuel, Integer[][] mem) {
        if (fuel < 0)
            return 0;
        if (mem[i][fuel] != null)
            return mem[i][fuel];

        int res = (i == finish) ? 1 : 0;
        for (int j = 0; j < locations.length; ++j) {
            if (j == i)
                continue;
            res += countRoutes(locations, j, finish, fuel - Math.abs(locations[i] - locations[j]), mem);
            res %= MOD;
        }

        return mem[i][fuel] = res;
    }
}
```

584. 1576.java

Path: LeetCode-main/solutions/1576. Replace All ?'s to Avoid Consecutive Repeating Characters/1576.java

```
class Solution {
    public String modifyString(String s) {
        StringBuilder sb = new StringBuilder();

        for (int i = 0; i < s.length(); ++i)
            if (s.charAt(i) == '?')
                sb.append(nextAvailable(sb, s, i));
            else
                sb.append(s.charAt(i));

        return sb.toString();
    }

    private char nextAvailable(StringBuilder sb, final String s, int i) {
        char c = 'a';
        while ((i > 0 && sb.charAt(i - 1) == c) || (i + 1 < s.length() && c == s.charAt(i + 1)))
            ++c;
        return c;
    }
}
```

585. 1577.java

Path: LeetCode-main/solutions/1577. Number of Ways Where Square of Number Is Equal to Product of Two Numbers/1577.java

```
class Solution {
    public int numTriplets(int[] nums1, int[] nums2) {
        return countTriplets(nums1, nums2) + countTriplets(nums2, nums1);
    }

    // Returns the number of triplet (i, j, k) if  $A[i]^2 == B[j] * B[k]$ .
    private int countTriplets(int[] A, int[] B) {
        int res = 0;
        Map<Integer, Integer> count = new HashMap<>();

        for (final int b : B)
            count.merge(b, 1, Integer::sum);

        for (final int a : A) {
            long target = (long) a * a;
            for (Map.Entry<Integer, Integer> entry : count.entrySet()) {
                final int b = entry.getKey();
                final int freq = entry.getValue();
                if (target % b > 0 || !count.containsKey((int) (target / b)))
                    continue;
                if (target / b == b)
                    res += freq * (freq - 1);
                else
                    res += freq * count.get((int) (target / b));
            }
        }

        return res / 2;
    }
}
```

586. 1578.java

Path: LeetCode-main/solutions/1578. Minimum Time to Make Rope Colorful/1578.java

```
class Solution {
    public int minCost(String colors, int[] neededTime) {
        int ans = 0;
        int maxNeededTime = neededTime[0];

        for (int i = 1; i < colors.length(); ++i)
            if (colors.charAt(i) == colors.charAt(i - 1)) {
                ans += Math.min(maxNeededTime, neededTime[i]);
                // For each continuous group, Bob needs to remove every balloon except the one with the max
                // `neededTime`. So, he should hold the balloon with the highest `neededTime` in his hand.
                maxNeededTime = Math.max(maxNeededTime, neededTime[i]);
            } else {
                // If the current balloon is different from the previous one, discard the balloon from the
                // previous group and hold the new one in hand.
                maxNeededTime = neededTime[i];
            }

        return ans;
    }
}
```

587. 1579.java

Path: LeetCode-main/solutions/1579. Remove Max Number of Edges to Keep Graph Fully Traversable/1579.java

```
class UnionFind {
    public UnionFind(int n) {
        count = n;
        id = new int[n];
        rank = new int[n];
        for (int i = 0; i < n; ++i)
            id[i] = i;
    }

    public boolean unionByRank(int u, int v) {
        final int i = find(u);
        final int j = find(v);
        if (i == j)
            return false;
        if (rank[i] < rank[j]) {
            id[i] = j;
        } else if (rank[i] > rank[j]) {
            id[j] = i;
        } else {
            id[i] = j;
            ++rank[j];
        }
        --count;
        return true;
    }

    public int getCount() {
        return count;
    }

    private int count;
    private int[] id;
    private int[] rank;

    private int find(int u) {
        return id[u] == u ? u : (id[u] = find(id[u]));
    }
}

class Solution {
    public int maxNumEdgesToRemove(int n, int[][] edges) {
        UnionFind alice = new UnionFind(n);
        UnionFind bob = new UnionFind(n);
        int requiredEdges = 0;

        // Greedily put type 3 edges in the front.
        Arrays.sort(edges, Comparator.comparingInt(edge -> - edge[0]));

        for (int[] edge : edges) {
            final int type = edge[0];
            final int u = edge[1] - 1;
            final int v = edge[2] - 1;
            switch (type) {
                case 3: // Can be traversed by Alice and Bob.
                    // Note that we should use | instead of || because if the first
                    // expression is true, short-circuiting will skip the second
                    // expression.
                    if (alice.unionByRank(u, v) | bob.unionByRank(u, v))
                        ++requiredEdges;
                    break;
                case 2: // Can be traversed by Bob.
                    if (bob.unionByRank(u, v))
                        ++requiredEdges;
                    break;
                case 1: // Can be traversed by Alice.
                    if (alice.unionByRank(u, v))
                        ++requiredEdges;
            }
        }

        return alice.getCount() == 1 && bob.getCount() == 1 ? edges.length - requiredEdges : -1;
    }
}
```

588. 158.java

Path: LeetCode-main/solutions/158. Read N Characters Given Read4 II - Call multiple times/158.java

```
/**
 * The read4 API is defined in the parent class Reader4.
 *   int read4(char[] buf4);
 */

public class Solution extends Reader4 {
    /**
     * @param buf Destination buffer
     * @param n   Number of characters to read
     * @return    The number of actual characters read
     */
    public int read(char[] buf, int n) {
        int i = 0; // buf's index

        while (i < n) {
            if (i4 == n4) { // All the characters in the buf4 are consumed.
                i4 = 0; // Reset the buf4's index.
                n4 = read4(buf4); // Read <= 4 characters from the file to the buf4.
                if (n4 == 0) // Reach the EOF.
                    return i;
            }
            buf[i++] = buf4[i4++];
        }

        return i;
    }

    private char[] buf4 = new char[4];
    private int i4 = 0; // buf4's index
    private int n4 = 0; // buf4's size
}
```

589. 1580.java

Path: LeetCode-main/solutions/1580. Put Boxes Into the Warehouse II/1580.java

```
class Solution {
    public int maxBoxesInWarehouse(int[] boxes, int[] warehouse) {
        int l = 0;
        int r = warehouse.length - 1;

        boxes = Arrays.stream(boxes)
            .boxed()
            .sorted(Collections.reverseOrder())
            .mapToInt(Integer::intValue)
            .toArray();

        for (final int box : boxes) {
            if (l > r)
                return warehouse.length;
            if (box <= warehouse[l])
                ++l;
            else if (box <= warehouse[r])
                --r;
        }

        return l + (warehouse.length - r - 1);
    }
}
```


591. 1583.java

Path: LeetCode-main/solutions/1583. Count Unhappy Friends/1583.java

```
class Solution {
    public int unhappyFriends(int n, int[][] preferences, int[][] pairs) {
        int ans = 0;
        int[] matches = new int[n];
        Map<Integer, Integer>[] prefer = new Map[n];

        for (int[] pair : pairs) {
            final int x = pair[0];
            final int y = pair[1];
            matches[x] = y;
            matches[y] = x;
        }

        for (int i = 0; i < n; ++i)
            prefer[i] = new HashMap<>();

        for (int i = 0; i < n; ++i)
            for (int j = 0; j < n - 1; ++j)
                prefer[i].put(preferences[i][j], j);

        for (int x = 0; x < n; ++x)
            for (final int u : preferences[x]) {
                final int y = matches[x];
                final int v = matches[u];
                if (prefer[x].get(u) < prefer[x].get(y) && prefer[u].get(x) < prefer[u].get(v)) {
                    ++ans;
                    break;
                }
            }

        return ans;
    }
}
```

592. 1584.java

Path: LeetCode-main/solutions/1584. Min Cost to Connect All Points/1584.java

```
class Solution {
    public int minCostConnectPoints(int[][] points) {
        // dist[i] := the minimum distance to connect the points[i]
        int[] dist = new int[points.length];
        Arrays.fill(dist, Integer.MAX_VALUE);
        int ans = 0;

        for (int i = 0; i < points.length - 1; ++i) {
            for (int j = i + 1; j < points.length; ++j) {
                // Try to connect the points[i] with the points[j].
                dist[j] = Math.min(dist[j], Math.abs(points[i][0] - points[j][0]) +
                                    Math.abs(points[i][1] - points[j][1]));

                // Swap the points[j] (the point with the minimum distance) with the
                // points[i + 1].
                if (dist[j] < dist[i + 1]) {
                    final int[] tempPoint = points[j];
                    points[j] = points[i + 1];
                    points[i + 1] = tempPoint;
                    final int tempDist = dist[j];
                    dist[j] = dist[i + 1];
                    dist[i + 1] = tempDist;
                }
            }
            ans += dist[i + 1];
        }

        return ans;
    }
}
```

593. 1585.java

Path: LeetCode-main/solutions/1585. Check If String Is Transformable With Substring Sort Operations/1585.java

```
class Solution {
    public boolean isTransformable(String s, String t) {
        if (!Arrays.equals(getCount(s), getCount(t)))
            return false;

        Queue<Integer>[] positions = new Queue[10];
        for (int i = 0; i < 10; i++)
            positions[i] = new LinkedList<>();

        for (int i = 0; i < s.length(); i++)
            positions[s.charAt(i) - '0'].offer(i);

        // For each digit in `t`, check if we can put this digit in `s` at the same
        // position as `t`. Ensure that all the left digits are equal to or greater
        // than it. This is because the only operation we can perform is sorting in
        // ascending order. If there is a digit to the left that is smaller than it,
        // we can never move it to the same position as in `t`. However, if all the
        // digits to its left are equal to or greater than it, we can move it one
        // position to the left until it reaches the same position as in `t`.
        for (final char c : t.toCharArray()) {
            final int d = c - '0';
            final int front = positions[d].poll();
            for (int smaller = 0; smaller < d; ++smaller)
                if (!positions[smaller].isEmpty() && positions[smaller].peek() < front)
                    return false;
        }

        return true;
    }

    private int[] getCount(String s) {
        int[] count = new int[10];
        for (char c : s.toCharArray())
            count[c - '0']++;
        return count;
    }
}
```

594. 1586.java

Path: LeetCode-main/solutions/1586. Binary Search Tree Iterator II/1586.java

```
class BSTIterator {
    public BSTIterator(TreeNode root) {
        pushLeftsUntilNull(root);
    }

    public boolean hasNext() {
        return !nexts.isEmpty();
    }

    public int next() {
        Pair<TreeNode, Boolean> pair = nexts.pop();
        TreeNode root = pair.getKey();
        boolean fromNext = pair.getValue();
        if (fromNext)
            pushLeftsUntilNull(root.right);
        prevsAndCurr.push(root);
        return root.val;
    }

    public boolean hasPrev() {
        return prevsAndCurr.size() > 1;
    }

    public int prev() {
        nexts.push(new Pair<>(prevsAndCurr.pop(), /*fromNext=*/false));
        return prevsAndCurr.peek().val;
    }

    private void pushLeftsUntilNull(TreeNode root) {
        while (root != null) {
            nexts.push(new Pair<>(root, /*fromNext=*/true));
            root = root.left;
        }
    }

    private Deque<TreeNode> prevsAndCurr = new ArrayDeque<>();
    private Deque<Pair<TreeNode, /*fromNext=*/Boolean>> nexts = new ArrayDeque<>();
}
```

595. 1588.java

Path: LeetCode-main/solutions/1588. Sum of All Odd Length Subarrays/1588.java

```
class Solution {
    public int sumOddLengthSubarrays(int[] arr) {
        int ans = 0;
        // Maintain two sums of subarrays ending in the previous index.
        // Each time we meet a new number, we'll consider "how many times" it should
        // contribute to the newly built subarrays by calculating the number of
        // previous even/odd-length subarrays.
        int prevEvenSum = 0; // the sum of even-length subarrays
        int prevOddSum = 0;  // the sum of odd-length subarrays

        for (int i = 0; i < arr.length; ++i) {
            // (i + 1) / 2 := the number of previous odd-length subarrays.
            final int currEvenSum = prevOddSum + ((i + 1) / 2) * arr[i];
            // i / 2 + 1 := the number of previous even-length subarrays
            // (including 0).
            final int currOddSum = prevEvenSum + (i / 2 + 1) * arr[i];
            ans += currOddSum;
            prevEvenSum = currEvenSum;
            prevOddSum = currOddSum;
        }

        return ans;
    }
}
```

596. 1589.java

Path: LeetCode-main/solutions/1589. Maximum Sum Obtained of Any Permutation/1589.java

```
class Solution {
    public int maxSumRangeQuery(int[] nums, int[][] requests) {
        final int MOD = 1_000_000_007;
        long ans = 0;
        // count[i] := the number of times nums[i] has been requested
        int[] count = new int[nums.length];

        for (int[] request : requests) {
            final int start = request[0];
            final int end = request[1];
            ++count[start];
            if (end + 1 < nums.length)
                --count[end + 1];
        }

        for (int i = 1; i < nums.length; ++i)
            count[i] += count[i - 1];

        Arrays.sort(count);
        Arrays.sort(nums);

        for (int i = 0; i < nums.length; ++i) {
            ans += (long) nums[i] * count[i];
            ans %= MOD;
        }

        return (int) ans;
    }
}
```

597. 159.java

Path: LeetCode-main/solutions/159. Longest Substring with At Most Two Distinct Characters/159.java

```
class Solution {
    public int lengthOfLongestSubstringTwoDistinct(String s) {
        int ans = 0;
        int distinct = 0;
        int[] count = new int[128];

        for (int l = 0, r = 0; r < s.length(); ++r) {
            if (++count[s.charAt(r)] == 1)
                ++distinct;
            while (distinct == 3)
                if (--count[s.charAt(l++)] == 0)
                    --distinct;
            ans = Math.max(ans, r - l + 1);
        }
        return ans;
    }
}
```


598. 1590.java

Path: LeetCode-main/solutions/1590. Make Sum Divisible by P/1590.java

```
class Solution {
    public int minSubarray(int[] nums, int p) {
        final long sum = Arrays.stream(nums).asLongStream().sum();
        final int remainder = (int) (sum % p);
        if (remainder == 0)
            return 0;

        int ans = nums.length;
        int prefix = 0;
        Map<Integer, Integer> prefixToIndex = new HashMap<>();
        prefixToIndex.put(0, -1);

        for (int i = 0; i < nums.length; ++i) {
            prefix += nums[i];
            prefix %= p;
            final int target = (prefix - remainder + p) % p;
            if (prefixToIndex.containsKey(target))
                ans = Math.min(ans, i - prefixToIndex.get(target));
            prefixToIndex.put(prefix, i);
        }

        return ans == nums.length ? -1 : ans;
    }
}
```

599. 1591.java

Path: LeetCode-main/solutions/1591. Strange Printer II/1591.java

```
enum State { INIT, VISITING, VISITED }

class Solution {
    public boolean isPrintable(int[][] targetGrid) {
        final int MAX_COLOR = 60;
        final int m = targetGrid.length;
        final int n = targetGrid[0].length;
        // graph[u] := {v1, v2} means v1 and v2 cover u
        Set<Integer>[] graph = new HashSet[MAX_COLOR + 1];

        for (int color = 1; color <= MAX_COLOR; ++color) {
            // Get the rectangle of the current color.
            int minI = m;
            int minJ = n;
            int maxI = -1;
            int maxJ = -1;
            for (int i = 0; i < m; ++i)
                for (int j = 0; j < n; ++j)
                    if (targetGrid[i][j] == color) {
                        minI = Math.min(minI, i);
                        minJ = Math.min(minJ, j);
                        maxI = Math.max(maxI, i);
                        maxJ = Math.max(maxJ, j);
                    }
            // Add any color covering the current as the children.
            graph[color] = new HashSet<>();
            for (int i = minI; i <= maxI; ++i)
                for (int j = minJ; j <= maxJ; ++j)
                    if (targetGrid[i][j] != color) {
                        graph[color].add(targetGrid[i][j]);
                    }
        }

        State[] states = new State[MAX_COLOR + 1];

        for (int color = 1; color <= MAX_COLOR; ++color)
            if (hasCycle(graph, color, states))
                return false;

        return true;
    }

    private boolean hasCycle(Set<Integer>[] graph, int u, State[] states) {
        if (states[u] == State.VISITING)
            return true;
        if (states[u] == State.VISITED)
            return false;
        states[u] = State.VISITING;
        for (int v : graph[u])
            if (hasCycle(graph, v, states))
                return true;
        states[u] = State.VISITED;
        return false;
    }
}
```

600. 1592.java

Path: LeetCode-main/solutions/1592. Rearrange Spaces Between Words/1592.java

```
class Solution {
    public String reorderSpaces(String text) {
        final String[] words = text.trim().split("\\s+");
        final int spaces = (int) text.chars().filter(c -> c == ' ').count();
        final int n = words.length;
        final int gapSize = n == 1 ? 0 : spaces / (n - 1);
        final int remains = n == 1 ? spaces : spaces % (n - 1);
        return String.join(" ".repeat(gapSize), words) + " ".repeat(remains);
    }
}
```

601. 1593.java

Path: LeetCode-main/solutions/1593. Split a String Into the Max Number of Unique Substrings/1593.java

```
class Solution {
    public int maxUniqueSplit(String s) {
        dfs(s, 0, new HashSet<>());
        return ans;
    }

    private int ans = 0;

    private void dfs(final String s, int start, Set<String> seen) {
        if (start == s.length()) {
            ans = Math.max(ans, seen.size());
            return;
        }

        for (int i = start + 1; i <= s.length(); ++i) {
            final String cand = s.substring(start, i);
            if (seen.contains(cand))
                continue;
            seen.add(cand);
            dfs(s, i, seen);
            seen.remove(cand);
        }
    }
}
```

602. 1594.java

Path: LeetCode-main/solutions/1594. Maximum Non Negative Product in a Matrix/1594.java

```
class Solution {
    public int maxProductPath(int[][] grid) {
        final int MOD = 1_000_000_007;
        final int m = grid.length;
        final int n = grid[0].length;
        // dpMin[i][j] := the minimum product from (0, 0) to (i, j)
        // dpMax[i][j] := the maximum product from (0, 0) to (i, j)
        long[][] dpMin = new long[m][n];
        long[][] dpMax = new long[m][n];

        dpMin[0][0] = dpMax[0][0] = grid[0][0];

        for (int i = 1; i < m; ++i)
            dpMin[i][0] = dpMax[i][0] = dpMin[i - 1][0] * grid[i][0];

        for (int j = 1; j < n; ++j)
            dpMin[0][j] = dpMax[0][j] = dpMin[0][j - 1] * grid[0][j];

        for (int i = 1; i < m; ++i)
            for (int j = 1; j < n; ++j)
                if (grid[i][j] < 0) {
                    dpMin[i][j] = Math.max(dpMax[i - 1][j], dpMax[i][j - 1]) * grid[i][j];
                    dpMax[i][j] = Math.min(dpMin[i - 1][j], dpMin[i][j - 1]) * grid[i][j];
                } else {
                    dpMin[i][j] = Math.min(dpMin[i - 1][j], dpMin[i][j - 1]) * grid[i][j];
                    dpMax[i][j] = Math.max(dpMax[i - 1][j], dpMax[i][j - 1]) * grid[i][j];
                }

        final long mx = Math.max(dpMin[m - 1][n - 1], dpMax[m - 1][n - 1]);
        return mx < 0 ? -1 : (int) (mx % MOD);
    }
}
```

603. 1595.java

Path: LeetCode-main/solutions/1595. Minimum Cost to Connect Two Groups of Points/1595.java

```
class Solution {
    public int connectTwoGroups(List<List<Integer>> cost) {
        final int m = cost.size();
        final int n = cost.get(0).size();
        Integer[][] mem = new Integer[m][1 << n];
        // minCosts[j] := the minimum cost of connecting group2's point j
        int[] minCosts = new int[n];

        for (int j = 0; j < n; ++j) {
            int minCostIndex = 0;
            for (int i = 1; i < m; ++i)
                if (cost.get(i).get(j) < cost.get(minCostIndex).get(j))
                    minCostIndex = i;
            minCosts[j] = cost.get(minCostIndex).get(j);
        }

        return connectTwoGroups(cost, 0, 0, minCosts, mem);
    }

    // Returns the minimum cost to connect group1's points[i..n) with group2's
    // points, where `mask` is the bitmask of the connected points in group2.
    private int connectTwoGroups(List<List<Integer>> cost, int i, int mask, int[] minCosts,
                                Integer[][] mem) {
        if (i == cost.size()) {
            // All the points in group 1 are connected, so greedily assign the
            // minimum cost for the unconnected points of group2.
            int res = 0;
            for (int j = 0; j < cost.get(0).size(); ++j)
                if ((mask >> j & 1) == 0)
                    res += minCosts[j];
            return res;
        }
        if (mem[i][mask] != null)
            return mem[i][mask];

        int res = Integer.MAX_VALUE;
        for (int j = 0; j < cost.get(0).size(); ++j)
            res = Math.min(res, cost.get(i).get(j) +
                           connectTwoGroups(cost, i + 1, mask | 1 << j, minCosts, mem));
        return mem[i][mask] = res;
    }
}
```

604. 1597.java

Path: LeetCode-main/solutions/1597. Build Binary Expression Tree From Infix Expression/1597.java

```
class Solution {
    public Node expTree(String s) {
        Deque<Node> nodes = new ArrayDeque<>();
        Deque<Character> ops = new ArrayDeque<>(); // [operators | parentheses]

        for (final char c : s.toCharArray())
            if (Character.isDigit(c)) {
                nodes.push(new Node(c));
            } else if (c == '(') {
                ops.push(c);
            } else if (c == ')') {
                while (ops.peek() != '(')
                    nodes.push(buildNode(ops.pop(), nodes.pop(), nodes.pop()));
                ops.pop(); // Remove '('
            } else { // c == '+' || c == '-' || c == '*' || c == '/'
                while (!ops.isEmpty() && compare(ops.peek(), c))
                    nodes.push(buildNode(ops.pop(), nodes.pop(), nodes.pop()));
                ops.push(c);
            }

        while (!ops.isEmpty())
            nodes.push(buildNode(ops.pop(), nodes.pop(), nodes.pop()));

        return nodes.peek();
    }

    private Node buildNode(char op, Node right, Node left) {
        return new Node(op, left, right);
    }

    // Returns true if op1 is a operator and priority(op1) >= priority(op2).
    boolean compare(char op1, char op2) {
        if (op1 == '(' || op1 == ')')
            return false;
        return op1 == '*' || op1 == '/' || op2 == '+' || op2 == '-';
    }
}
```

605. 1598.java

Path: LeetCode-main/solutions/1598. Crawler Log Folder/1598.java

```
class Solution {
    public int minOperations(String[] logs) {
        int ans = 0;

        for (final String log : logs) {
            if (log.equals("../"))
                continue;
            if (log.equals("./"))
                ans = Math.max(0, ans - 1);
            else
                ++ans;
        }

        return ans;
    }
}
```


606. 1599.java

Path: LeetCode-main/solutions/1599. Maximum Profit of Operating a Centennial Wheel/1599.java

```
class Solution {
    public int minOperationsMaxProfit(int[] customers, int boardingCost, int runningCost) {
        int waiting = 0;
        int profit = 0;
        int maxProfit = 0;
        int rotate = 0;
        int maxRotate = -1;
        int i = 0;

        while (waiting > 0 || i < customers.length) {
            if (i < customers.length)
                waiting += customers[i++];
            // Onboard new customers.
            final int newOnboard = Math.min(waiting, 4);
            waiting -= newOnboard;
            profit += newOnboard * boardingCost - runningCost;
            ++rotate;
            if (profit > maxProfit) {
                maxProfit = profit;
                maxRotate = rotate;
            }
        }

        return maxRotate;
    }
}
```

607. 16.java

Path: LeetCode-main/solutions/16. 3Sum Closest/16.java

```
class Solution {
    public int threeSumClosest(int[] nums, int target) {
        int ans = nums[0] + nums[1] + nums[2];

        Arrays.sort(nums);

        for (int i = 0; i + 2 < nums.length; ++i) {
            if (i > 0 && nums[i] == nums[i - 1])
                continue;
            // Choose nums[i] as the first number in the triplet, then search the
            // remaining numbers in [i + 1, n - 1].
            int l = i + 1;
            int r = nums.length - 1;
            while (l < r) {
                final int sum = nums[i] + nums[l] + nums[r];
                if (sum == target)
                    return sum;
                if (Math.abs(sum - target) < Math.abs(ans - target))
                    ans = sum;
                if (sum < target)
                    ++l;
                else
                    --r;
            }
        }

        return ans;
    }
}
```

608. 160.java

Path: LeetCode-main/solutions/160. Intersection of Two Linked Lists/160.java

```
class Solution {
    public ListNode getIntersectionNode(ListNode headA, ListNode headB) {
        ListNode a = headA;
        ListNode b = headB;

        while (a != b) {
            a = a == null ? headB : a.next;
            b = b == null ? headA : b.next;
        }

        return a;
    }
}
```

609. 1600.java

Path: LeetCode-main/solutions/1600. Throne Inheritance/1600.java

```
class ThroneInheritance {
    public ThroneInheritance(String kingName) {
        this.kingName = kingName;
    }

    public void birth(String parentName, String childName) {
        family.putIfAbsent(parentName, new ArrayList<>());
        family.get(parentName).add(childName);
    }

    public void death(String name) {
        dead.add(name);
    }

    public List<String> getInheritanceOrder() {
        List<String> ans = new ArrayList<>();
        dfs(kingName, ans);
        return ans;
    }

    private Set<String> dead = new HashSet<>();
    private Map<String, List<String>> family = new HashMap<>();
    private String kingName;

    private void dfs(final String name, List<String> ans) {
        if (!dead.contains(name))
            ans.add(name);
        if (!family.containsKey(name))
            return;

        for (final String child : family.get(name))
            dfs(child, ans);
    }
}
```

610. 1601.java

Path: LeetCode-main/solutions/1601. Maximum Number of Achievable Transfer Requests/1601.java

```
class Solution {
    public int maximumRequests(int n, int[][] requests) {
        dfs(0, 0, requests, new int[n]);

        return ans;
    }

    private int ans = 0;

    private void dfs(int i, int processedReqs, int[][] requests, int[] degrees) {
        if (i == requests.length) {
            if (Arrays.stream(degrees).allMatch(d -> d == 0))
                ans = Math.max(ans, processedReqs);
            return;
        }

        // Skip the requests[i].
        dfs(i + 1, processedReqs, requests, degrees);

        // Process the requests[i].
        --degrees[requests[i][0]];
        ++degrees[requests[i][1]];
        dfs(i + 1, processedReqs + 1, requests, degrees);
        --degrees[requests[i][1]];
        ++degrees[requests[i][0]];
    }
}
```

611. 1602.java

Path: LeetCode-main/solutions/1602. Find Nearest Right Node in Binary Tree/1602.java

```
class Solution {
    public TreeNode findNearestRightNode(TreeNode root, TreeNode u) {
        dfs(root, u, 0);
        return ans;
    }

    private TreeNode ans = null;
    private int targetDepth = -1;

    private void dfs(TreeNode root, TreeNode u, int depth) {
        if (root == null)
            return;
        if (root == u) {
            targetDepth = depth;
            return;
        }
        if (depth == targetDepth && ans == null) {
            ans = root;
            return;
        }
        dfs(root.left, u, depth + 1);
        dfs(root.right, u, depth + 1);
    }
}
```

612. 1603.java

Path: LeetCode-main/solutions/1603. Design Parking System/1603.java

```
class ParkingSystem {
    public ParkingSystem(int big, int medium, int small) {
        count = new int[] {big, medium, small};
    }

    public boolean addCar(int carType) {
        return count[carType - 1]-- > 0;
    }

    private int[] count;
}
```

613. 1604.java

Path: LeetCode-main/solutions/1604. Alert Using Same Key-Card Three or More Times in a One Hour Period/1604.java

```
class Solution {
    public List<String> alertNames(String[] keyName, String[] keyTime) {
        List<String> ans = new ArrayList<>();
        HashMap<String, List<Integer>> nameToMinutes = new HashMap<>();

        for (int i = 0; i < keyName.length; i++) {
            final int minutes = getMinutes(keyTime[i]);
            nameToMinutes.putIfAbsent(keyName[i], new ArrayList<>());
            nameToMinutes.get(keyName[i]).add(minutes);
        }

        for (Map.Entry<String, List<Integer>> entry : nameToMinutes.entrySet()) {
            final String name = entry.getKey();
            List<Integer> minutes = entry.getValue();
            if (hasAlert(minutes))
                ans.add(name);
        }

        Collections.sort(ans);
        return ans;
    }

    private boolean hasAlert(List<Integer> minutes) {
        if (minutes.size() > 70)
            return true;
        Collections.sort(minutes);
        for (int i = 2; i < minutes.size(); i++)
            if (minutes.get(i - 2) + 60 >= minutes.get(i))
                return true;
        return false;
    }

    private int getMinutes(String time) {
        final int h = Integer.parseInt(time.substring(0, 2));
        final int m = Integer.parseInt(time.substring(3));
        return 60 * h + m;
    }
}
```


614. 1605.java

Path: LeetCode-main/solutions/1605. Find Valid Matrix Given Row and Column Sums/1605.java

```
class Solution {
    public int[][] restoreMatrix(int[] rowSum, int[] colSum) {
        final int m = rowSum.length;
        final int n = colSum.length;
        int[][] ans = new int[m][n];

        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j) {
                ans[i][j] = Math.min(rowSum[i], colSum[j]);
                rowSum[i] -= ans[i][j];
                colSum[j] -= ans[i][j];
            }

        return ans;
    }
}
```

615. 1606.java

Path: LeetCode-main/solutions/1606. Find Servers That Handled Most Number of Requests/1606.java

```
class Solution {
    public List<Integer> busiestServers(int k, int[] arrival, int[] load) {
        List<Integer> ans = new ArrayList<>();
        int[] times = new int[k];
        TreeSet<Integer> idleServers = new TreeSet<>();
        // (endTime, server)
        Queue<Pair<Integer, Integer>> minHeap =
            new PriorityQueue<>(Comparator.comparingInt(Pair::getKey));

        for (int i = 0; i < k; ++i)
            idleServers.add(i);

        for (int i = 0; i < arrival.length; ++i) {
            // Pop all the servers that are available now.
            while (!minHeap.isEmpty() && minHeap.peek().getKey() <= arrival[i]) {
                idleServers.add(minHeap.peek().getValue());
                minHeap.poll();
            }
            // Get the next available server.
            final int server = getNextAvailableServer(idleServers, i, k);
            if (server == -1)
                continue;
            ++times[server];
            minHeap.offer(new Pair<>(arrival[i] + load[i], server));
            idleServers.remove(server);
        }

        final int busiest = Arrays.stream(times).max().getAsInt();
        for (int i = 0; i < k; ++i)
            if (times[i] == busiest)
                ans.add(i);
        return ans;
    }

    private int getNextAvailableServer(TreeSet<Integer> idleServers, int ithRequest, int k) {
        if (idleServers.isEmpty())
            return -1;
        Integer server = idleServers.ceiling(ithRequest % k);
        return server == null ? idleServers.first() : server;
    }
}
```

616. 1608.java

Path: LeetCode-main/solutions/1608. Special Array With X Elements Greater Than or Equal X/1608.java

```
class Solution {
    public int specialArray(int[] nums) {
        Arrays.sort(nums);

        if (nums[0] >= nums.length)
            return nums.length;

        for (int i = 1; i < nums.length; ++i) {
            final int count = nums.length - i;
            if (nums[i - 1] < count && nums[i] >= count)
                return count;
        }

        return -1;
    }
}
```

617. 1609.java

Path: LeetCode-main/solutions/1609. Even Odd Tree/1609.java

```
class Solution {
    public boolean isEvenOddTree(TreeNode root) {
        Queue<TreeNode> q = new LinkedList<>(Arrays.asList(root));
        boolean isEven = true;

        for (; !q.isEmpty(); isEven = !isEven) {
            int prevVal = isEven ? Integer.MIN_VALUE : Integer.MAX_VALUE;
            for (int sz = q.size(); sz > 0; --sz) {
                TreeNode node = q.poll();
                if (isEven && (node.val % 2 == 0 || node.val <= prevVal))
                    return false; // invalid case on even level
                if (!isEven && (node.val % 2 == 1 || node.val >= prevVal))
                    return false; // invalid case on odd level
                prevVal = node.val;
                if (node.left != null)
                    q.offer(node.left);
                if (node.right != null)
                    q.offer(node.right);
            }
        }
        return true;
    }
}
```

618. 161.java

Path: LeetCode-main/solutions/161. One Edit Distance/161.java

```
class Solution {
    public boolean isOneEditDistance(String s, String t) {
        final int m = s.length();
        final int n = t.length();
        if (m > n) // Make sure that |s| <= |t|.
            return isOneEditDistance(t, s);

        for (int i = 0; i < m; ++i)
            if (s.charAt(i) != t.charAt(i)) {
                if (m == n)
                    return s.substring(i + 1).equals(t.substring(i + 1)); // Replace s[i] with t[i].
                return s.substring(i).equals(t.substring(i + 1)); // Delete t[i].
            }

        return m + 1 == n; // Delete t[-1].
    }
}
```

619. 1610.java

Path: LeetCode-main/solutions/1610. Maximum Number of Visible Points/1610.java

```
class Solution {
    public int visiblePoints(List<List<Integer>> points, int angle, List<Integer> location) {
        final int posX = location.get(0);
        final int posY = location.get(1);
        int maxVisible = 0;
        int same = 0;
        List<Double> pointAngles = new ArrayList<>();

        for (List<Integer> p : points) {
            final int x = p.get(0);
            final int y = p.get(1);
            if (x == posX && y == posY)
                ++same;
            else
                pointAngles.add(getAngle(y - posY, x - posX));
        }

        Collections.sort(pointAngles);

        final int n = pointAngles.size();
        for (int i = 0; i < n; ++i)
            pointAngles.add(pointAngles.get(i) + 360);

        for (int l = 0, r = 0; r < pointAngles.size(); ++r) {
            while (pointAngles.get(r) - pointAngles.get(l) > angle)
                ++l;
            maxVisible = Math.max(maxVisible, r - l + 1);
        }

        return maxVisible + same;
    }

    private double getAngle(int dy, int dx) {
        return Math.atan2(dy, dx) * 180 / Math.PI;
    }
}
```

620. 1611.java

Path: LeetCode-main/solutions/1611. Minimum One Bit Operations to Make Integers Zero/1611.java

```
class Solution {
    public int minimumOneBitOperations(int n) {
        // Observation: e.g. n = 2^2
        //      100 (2^2 needs 2^3 - 1 ops)
        // op1 -> 101
        // op2 -> 111
        // op1 -> 110
        // op2 -> 010 (2^1 needs 2^2 - 1 ops)
        // op1 -> 011
        // op2 -> 001 (2^0 needs 2^1 - 1 ops)
        // op1 -> 000
        //
        // So 2^k needs 2^(k + 1) - 1 ops. Note this is reversible, i.e., 0 -> 2^k
        // also takes 2^(k + 1) - 1 ops.

        // e.g. n = 1XXX, our first goal is to change 1XXX -> 1100.
        // - If the second bit is 1, you only need to consider the cost of turning
        //   the last 2 bits to 0.
        // - If the second bit is 0, you need to add up the cost of flipping the
        //   second bit from 0 to 1.
        // XOR determines the cost minimumOneBitOperations(1XXX^1100) accordingly.
        // Then, 1100 -> 0100 needs 1 op. Finally, 0100 -> 0 needs 2^3 - 1 ops.
        if (n == 0)
            return 0;
        // x is the largest 2^k <= n.
        // x | x >> 1 -> x >> 1 needs 1 op.
        //   x >> 1 -> 0      needs x = 2^k - 1 ops.
        int x = 1;
        while (x * 2 <= n)
            x <<= 1;
        return minimumOneBitOperations(n ^ (x | x >> 1)) + 1 + x - 1;
    }
}
```

621. 1612.java

Path: LeetCode-main/solutions/1612. Check If Two Expression Trees are Equivalent/1612.java

```
class Solution {
    public boolean checkEquivalence(Node root1, Node root2) {
        int[] count = new int[26];
        dfs(root1, count, 1);
        dfs(root2, count, -1);
        return Arrays.stream(count).filter(c -> c != 0).count() == 0;
    }

    private void dfs(Node root, int[] count, int add) {
        if (root == null)
            return;
        if ('a' <= root.val && root.val <= 'z')
            count[root.val - 'a'] += add;
        dfs(root.left, count, add);
        dfs(root.right, count, add);
    }
}
```


622. 1614.java

Path: LeetCode-main/solutions/1614. Maximum Nesting Depth of the Parentheses/1614.java

```
class Solution {  
    public int maxDepth(String s) {  
        int ans = 0;  
        int opened = 0;  
  
        for (final char c : s.toCharArray())  
            if (c == '(')  
                ans = Math.max(ans, ++opened);  
            else if (c == ')')  
                --opened;  
  
        return ans;  
    }  
}
```

623. 1615.java

Path: LeetCode-main/solutions/1615. Maximal Network Rank/1615.java

```
class Solution {
    public int maximalNetworkRank(int n, int[][] roads) {
        int[] degrees = new int[n];

        for (int[] road : roads) {
            final int u = road[0];
            final int v = road[1];
            ++degrees[u];
            ++degrees[v];
        }

        // Find the first maximum and the second maximum degrees.
        int maxDegree1 = 0;
        int maxDegree2 = 0;
        for (final int degree : degrees)
            if (degree > maxDegree1) {
                maxDegree2 = maxDegree1;
                maxDegree1 = degree;
            } else if (degree > maxDegree2) {
                maxDegree2 = degree;
            }

        // There can be multiple nodes with `maxDegree1` or `maxDegree2`.
        // Find the counts of such nodes.
        int countMaxDegree1 = 0;
        int countMaxDegree2 = 0;
        for (final int degree : degrees)
            if (degree == maxDegree1)
                ++countMaxDegree1;
            else if (degree == maxDegree2)
                ++countMaxDegree2;

        if (countMaxDegree1 == 1) {
            // 1. If there is only one node with degree = `maxDegree1`, then we'll
            // need to use the node with degree = `maxDegree2`. The answer in general
            // will be (maxDegree1 + maxDegree2), but if the two nodes that we're
            // considering are connected, then we'll have to subtract 1.
            final int edgeCount = getEdgeCount(roads, degrees, maxDegree1, maxDegree2) +
                                getEdgeCount(roads, degrees, maxDegree2, maxDegree1);
            return maxDegree1 + maxDegree2 - (countMaxDegree2 == edgeCount ? 1 : 0);
        } else {
            // 2. If there are more than one node with degree = `maxDegree1`, then we
            // can consider `maxDegree1` twice, and we don't need to use `maxDegree2`.
            // The answer in general will be 2 * maxDegree1, but if the two nodes that
            // we're considering are connected, then we'll have to subtract 1.
            final int edgeCount = getEdgeCount(roads, degrees, maxDegree1, maxDegree1);
            final int maxPossibleEdgeCount = countMaxDegree1 * (countMaxDegree1 - 1) / 2;
            return 2 * maxDegree1 - (maxPossibleEdgeCount == edgeCount ? 1 : 0);
        }
    }

    // Returns the number of edges (u, v) where degrees[u] == degreeU and
    // degrees[v] == degreeV.
    private int getEdgeCount(int[][] roads, int[] degrees, int degreeU, int degreeV) {
        int edgeCount = 0;
        for (int[] road : roads) {
            final int u = road[0];
            final int v = road[1];
            if (degrees[u] == degreeU && degrees[v] == degreeV)
                ++edgeCount;
        }
        return edgeCount;
    }
}
```

624. 1616.java

Path: LeetCode-main/solutions/1616. Split Two Strings to Make Palindrome/1616.java

```
class Solution {
    public boolean checkPalindromeFormation(String a, String b) {
        return check(a, b) || check(b, a);
    }

    private boolean check(String a, String b) {
        for (int i = 0, j = a.length() - 1; i < j; ++i, --j)
            if (a.charAt(i) != b.charAt(j))
                // a[0:i] + a[i..j] + b[j + 1:] or
                // a[0:i] + b[i..j] + b[j + 1:]
                return isPalindrome(a, i, j) || isPalindrome(b, i, j);
        return true;
    }

    private boolean isPalindrome(String s, int i, int j) {
        while (i < j)
            if (s.charAt(i++) != s.charAt(j--))
                return false;
        return true;
    }
}
```

625. 1617.java

Path: LeetCode-main/solutions/1617. Count Subtrees With Max Distance Between Cities/1617.java

```
class Solution {
    public int[] countSubgraphsForEachDiameter(int n, int[][] edges) {
        final int maxMask = 1 << n;
        final int[][] dist = floydWarshall(n, edges);
        int[] ans = new int[n - 1];

        // mask := the subset of the cities
        for (int mask = 0; mask < maxMask; ++mask) {
            int maxDist = getMaxDist(mask, dist, n);
            if (maxDist > 0)
                ++ans[maxDist - 1];
        }

        return ans;
    }

    private int[][] floydWarshall(int n, int[][] edges) {
        int[][] dist = new int[n][n];
        for (int i = 0; i < n; ++i)
            Arrays.fill(dist[i], n);

        for (int i = 0; i < n; ++i)
            dist[i][i] = 0;

        for (int[] edge : edges) {
            final int u = edge[0] - 1;
            final int v = edge[1] - 1;
            dist[u][v] = 1;
            dist[v][u] = 1;
        }

        for (int k = 0; k < n; ++k)
            for (int i = 0; i < n; ++i)
                for (int j = 0; j < n; ++j)
                    dist[i][j] = Math.min(dist[i][j], dist[i][k] + dist[k][j]);

        return dist;
    }

    private int getMaxDist(int mask, int[][] dist, int n) {
        int maxDist = 0;
        int edgeCount = 0;
        int cityCount = 0;
        for (int u = 0; u < n; ++u) {
            if ((mask >> u & 1) == 0) // u is not in the subset.
                continue;
            ++cityCount;
            for (int v = u + 1; v < n; ++v) {
                if ((mask >> v & 1) == 0) // v is not in the subset.
                    continue;
                if (dist[u][v] == 1) // u and v are connected.
                    ++edgeCount;
                maxDist = Math.max(maxDist, dist[u][v]);
            }
        }
        return edgeCount == cityCount - 1 ? maxDist : 0;
    }
}
```

626. 1618.java

Path: LeetCode-main/solutions/1618. Maximum Font to Fit a Sentence in a Screen/1618.java

```
/**
 * // This is the FontInfo's API interface.
 * // You should not implement it, or speculate about its implementation
 * interface FontInfo {
 *     // Return the width of char ch when fontSize is used.
 *     public int getWidth(int fontSize, char ch) {}
 *     // Return Height of any char when fontSize is used.
 *     public int getHeight(int fontSize)
 * }
 */
class Solution {
    public int maxFont(String text, int w, int h, int[] fonts, FontInfo fontInfo) {
        int[] count = new int[26];

        for (final char c : text.toCharArray())
            ++count[c - 'a'];

        int l = 0;
        int r = fonts.length - 1;

        while (l < r) {
            final int m = (l + r + 1) / 2;
            if (fontInfo.getHeight(fonts[m]) <= h && getWidthSum(count, fonts[m], fontInfo) <= w)
                l = m;
            else
                r = m - 1;
        }

        return getWidthSum(count, fonts[l], fontInfo) <= w ? fonts[l] : -1;
    }

    private int getWidthSum(int[] count, int font, FontInfo fontInfo) {
        int width = 0;
        for (int i = 0; i < 26; ++i)
            width += count[i] * fontInfo.getWidth(font, (char) ('a' + i));
        return width;
    }
}
```

627. 1619.java

Path: LeetCode-main/solutions/1619. Mean of Array After Removing Some Elements/1619.java

```
class Solution {  
    public double trimMean(int[] arr) {  
        Arrays.sort(arr);  
        final int offset = arr.length / 20;  
        return Arrays.stream(Arrays.copyOfRange(arr, offset, arr.length - offset)).average().orElse(0);  
    }  
}
```

628. 162.java

Path: LeetCode-main/solutions/162. Find Peak Element/162.java

```
class Solution {
    public int findPeakElement(int[] nums) {
        int l = 0;
        int r = nums.length - 1;

        while (l < r) {
            final int m = (l + r) / 2;
            if (nums[m] >= nums[m + 1])
                r = m;
            else
                l = m + 1;
        }

        return l;
    }
}
```

629. 1620.java

Path: LeetCode-main/solutions/1620. Coordinate With Maximum Network Quality/1620.java

```
class Solution {
    public int[] bestCoordinate(int[][] towers, int radius) {
        final int MAX = 50;
        final int n = towers.length;
        int[] ans = new int[2];
        int maxQuality = 0;

        for (int i = 0; i <= MAX; ++i) {
            for (int j = 0; j <= MAX; ++j) {
                int qualitySum = 0;
                for (int[] tower : towers) {
                    double d = dist(tower, i, j);
                    if (d <= radius) {
                        final int q = tower[2];
                        qualitySum += (int) (q / (1 + d));
                    }
                }
                if (qualitySum > maxQuality) {
                    maxQuality = qualitySum;
                    ans[0] = i;
                    ans[1] = j;
                }
            }
        }

        return ans;
    }

    // Returns the distance between the tower and the coordinate.
    private double dist(int[] tower, int i, int j) {
        return Math.sqrt(Math.pow(tower[0] - i, 2) + Math.pow(tower[1] - j, 2));
    }
}
```


630. 1621.java

Path: LeetCode-main/solutions/1621. Number of Sets of K Non-Overlapping Line Segments/1621.java

[illegible]

631. 1622.java

Path: LeetCode-main/solutions/1622. Fancy Sequence/1622.java

```
class Fancy {
    // To undo  $a * val + b$  and get the original value, we append  $(val - b) / a$ .
    // By Fermat's little theorem:
    //  $a^{(p - 1)} \equiv 1 \pmod{p}$ 
    //  $a^{(p - 2)} \equiv a^{(-1)} \pmod{p}$ 
    // So,  $(val - b) / a \equiv (val - b) * a^{(p - 2)} \pmod{p}$ 
    public void append(int val) {
        final long x = (val - b + MOD) % MOD;
        vals.add(x * modPow(a, MOD - 2) % MOD);
    }

    // If the value is  $a * val + b$ , then the value after adding by `inc` will be
    //  $a * val + b + inc$ . So, we adjust b to  $b + inc$ .
    public void addAll(int inc) {
        b = (b + inc) % MOD;
    }

    // If the value is  $a * val + b$ , then the value after multiplying by `m` will
    // be  $a * m * val + b * m$ . So, we adjust a to  $a * m$  and b to  $b * m$ .
    public void multAll(int m) {
        a = (a * m) % MOD;
        b = (b * m) % MOD;
    }

    public int getIndex(int idx) {
        return idx >= vals.size() ? -1 : (int) ((a * vals.get(idx) + b) % MOD);
    }

    private static final int MOD = 1_000_000_007;
    // For each `val` in `vals`, it actually represents  $a * val + b$ .
    private List<Long> vals = new ArrayList<>();
    private long a = 1;
    private long b = 0;

    private int modPow(long x, long n) {
        if (n == 0)
            return 1;
        if (n % 2 == 1)
            return (int) (x * modPow(x % MOD, (n - 1)) % MOD);
        return modPow(x * x % MOD, (n / 2)) % MOD;
    }
}
```

632. 1624.java

Path: LeetCode-main/solutions/1624. Largest Substring Between Two Equal Characters/1624.java

```
class Solution {
    public int maxLengthBetweenEqualCharacters(String s) {
        int ans = -1;
        int[] lastSeen = new int[26];
        Arrays.fill(lastSeen, -1);

        for (int i = 0; i < s.length(); ++i) {
            final int c = s.charAt(i) - 'a';
            if (lastSeen[c] == -1)
                lastSeen[c] = i;
            else
                ans = Math.max(ans, i - lastSeen[c] - 1);
        }

        return ans;
    }
}
```

633. 1625.java

Path: LeetCode-main/solutions/1625. Lexicographically Smallest String After Applying Operations/1625.java

```
class Solution {
    public String findLexSmallestString(String s, int a, int b) {
        ans = s;

        dfs(s, a, b, new HashSet<>());

        return ans;
    }

    private String ans;

    private void dfs(String s, int a, int b, Set<String> seen) {
        if (seen.contains(s))
            return;

        seen.add(s);
        if (ans.compareTo(s) > 0)
            ans = s;

        dfs(add(s, a), a, b, seen);
        dfs(rotate(s, b), a, b, seen);
    }

    private String add(final String s, int a) {
        StringBuilder sb = new StringBuilder(s);
        for (int i = 1; i < sb.length(); i += 2)
            sb.setCharAt(i, (char) ('0' + (s.charAt(i) - '0' + a) % 10));
        return sb.toString();
    }

    private String rotate(final String s, int b) {
        final int n = s.length();
        return s.substring(n - b, n) + s.substring(0, n - b);
    }
}
```

634. 1626.java

Path: LeetCode-main/solutions/1626. Best Team With No Conflicts/1626.java

```
class Solution {
    public int bestTeamScore(int[] scores, int[] ages) {
        record Player(int age, int score) {}
        final int n = scores.length;
        Player[] players = new Player[n];
        // dp[i] := the maximum score of choosing the players[0..i] with the
        // players[i] being selected
        int[] dp = new int[n];

        for (int i = 0; i < n; ++i)
            players[i] = new Player(ages[i], scores[i]);

        Arrays.sort(players, Comparator.comparing(Player::age, Comparator.reverseOrder())
            .thenComparing(Player::score, Comparator.reverseOrder()));

        for (int i = 0; i < n; ++i) {
            // For each player, choose it first.
            dp[i] = players[i].score;
            // players[j].age >= players[i].age since we sort in descending order.
            // So, we only have to check that players[j].score >= players[i].score.
            for (int j = 0; j < i; ++j)
                if (players[j].score >= players[i].score)
                    dp[i] = Math.max(dp[i], dp[j] + players[i].score);
        }

        return Arrays.stream(dp).max().getAsInt();
    }
}
```

635. 1627.java

Path: LeetCode-main/solutions/1627. Graph Connectivity With Threshold/1627.java

```
class UnionFind {
    public UnionFind(int n) {
        id = new int[n];
        rank = new int[n];
        for (int i = 0; i < n; ++i)
            id[i] = i;
    }

    public boolean unionByRank(int u, int v) {
        final int i = find(u);
        final int j = find(v);
        if (i == j)
            return false;
        if (rank[i] < rank[j]) {
            id[i] = j;
        } else if (rank[i] > rank[j]) {
            id[j] = i;
        } else {
            id[i] = j;
            ++rank[j];
        }
        return true;
    }

    public int find(int u) {
        return id[u] == u ? u : (id[u] = find(id[u]));
    }

    private int[] id;
    private int[] rank;
}

class Solution {
    public List<Boolean> areConnected(int n, int threshold, int[][] queries) {
        List<Boolean> ans = new ArrayList<>();
        UnionFind uf = new UnionFind(n + 1);

        for (int z = threshold + 1; z <= n; ++z)
            for (int x = z * 2; x <= n; x += z)
                uf.unionByRank(z, x);

        for (int[] query : queries) {
            final int a = query[0];
            final int b = query[1];
            ans.add(uf.find(a) == uf.find(b));
        }

        return ans;
    }
}
```

636. 1628.java

Path: LeetCode-main/solutions/1628. Design an Expression Tree With Evaluate Function/1628.java

```
/**
 * This is the interface for the expression tree Node.
 * You should not remove it, and you can define some classes to implement it.
 */

abstract class Node {
    public abstract int evaluate();
    // define your fields here
};

class ExpNode extends Node {
    public ExpNode(String val, ExpNode left, ExpNode right) {
        this.val = val;
        this.left = left;
        this.right = right;
    }

    public int evaluate() {
        if (left == null && right == null)
            return Integer.parseInt(val);
        return op.get(val).apply(left.evaluate(), right.evaluate());
    }

    private static final Map<String, BinaryOperator<Integer>> op =
        Map.of("+", (a, b) -> a + b, "-", (a, b) -> a - b, "*", (a, b) -> a * b, "/", (a, b) -> a / b);
    private final String val;
    private final ExpNode left;
    private final ExpNode right;
}

/**
 * This is the TreeBuilder class.
 * You can treat it as the driver code that takes the postfix input
 * and returns the expression tree representing it as a Node.
 */

class TreeBuilder {
    Node buildTree(String[] postfix) {
        Deque<ExpNode> stack = new ArrayDeque<>();

        for (final String val : postfix)
            if (val.equals("+") || val.equals("-") || val.equals("*") || val.equals("/")) {
                ExpNode right = stack.pop();
                ExpNode left = stack.pop();
                stack.push(new ExpNode(val, left, right));
            } else {
                stack.push(new ExpNode(val, null, null));
            }

        return stack.pop();
    }
}
```

637. 1629.java

Path: LeetCode-main/solutions/1629. Slowest Key/1629.java

```
class Solution {
    public char slowestKey(int[] releaseTimes, String keysPressed) {
        char ans = keysPressed.charAt(0);
        int maxDuration = releaseTimes[0];

        for (int i = 1; i < keysPressed.length(); ++i) {
            final int duration = releaseTimes[i] - releaseTimes[i - 1];
            if (duration > maxDuration || (duration == maxDuration && keysPressed.charAt(i) > ans)) {
                ans = keysPressed.charAt(i);
                maxDuration = duration;
            }
        }

        return ans;
    }
}
```


638. 163.java

Path: LeetCode-main/solutions/163. Missing Ranges/163.java

```
class Solution {
    public List<List<Integer>> findMissingRanges(int[] nums, int lower, int upper) {
        if (nums.length == 0)
            return List.of(getRange(lower, upper));

        List<List<Integer>> ans = new ArrayList<>();

        if (nums[0] > lower)
            ans.add(getRange(lower, nums[0] - 1));

        for (int i = 1; i < nums.length; ++i)
            if (nums[i] > nums[i - 1] + 1)
                ans.add(getRange(nums[i - 1] + 1, nums[i] - 1));

        if (nums[nums.length - 1] < upper)
            ans.add(getRange(nums[nums.length - 1] + 1, upper));

        return ans;
    }

    private List<Integer> getRange(int lo, int hi) {
        if (lo == hi)
            return List.of(lo, lo);
        return List.of(lo, hi);
    }
}
```

639. 1630.java

Path: LeetCode-main/solutions/1630. Arithmetic Subarrays/1630.java

```
class Solution {
    public List<Boolean> checkArithmeticSubarrays(int[] nums, int[] l, int[] r) {
        List<Boolean> ans = new ArrayList<>();

        for (int i = 0; i < l.length; ++i)
            ans.add(isArithmetic(nums, l[i], r[i]));

        return ans;
    }

    private boolean isArithmetic(int[] nums, int l, int r) {
        if (r - l < 2)
            return true;

        int mn = Integer.MAX_VALUE;
        int mx = Integer.MIN_VALUE;
        Set<Integer> numsSet = new HashSet<>();

        for (int i = l; i <= r; ++i) {
            mn = Math.min(mn, nums[i]);
            mx = Math.max(mx, nums[i]);
            numsSet.add(nums[i]);
        }

        if ((mx - mn) % (r - l) != 0)
            return false;

        final int interval = (mx - mn) / (r - l);

        for (int k = 1; k <= r - l; ++k)
            if (!numsSet.contains(mn + k * interval))
                return false;

        return true;
    }
}
```

640. 1631.java

Path: LeetCode-main/solutions/1631. Path With Minimum Effort/1631.java

```
class Solution {
    public int minimumEffortPath(int[][] heights) {
        // d := the maximum difference of (i, j) and its neighbors
        record T(int i, int j, int d) {}
        final int[][] DIRS = {{0, 1}, {1, 0}, {0, -1}, {-1, 0}};
        final int m = heights.length;
        final int n = heights[0].length;
        Queue<T> minHeap = new PriorityQueue<>(Comparator.comparingInt(T::d));
        // dist[i][j] := the maximum absolute difference to reach (i, j)
        int[][] diff = new int[m][n];
        Arrays.stream(diff).forEach(A -> Arrays.fill(A, Integer.MAX_VALUE));
        boolean[][] seen = new boolean[m][n];

        minHeap.offer(new T(0, 0, 0));
        diff[0][0] = 0;

        while (!minHeap.isEmpty()) {
            final int i = minHeap.peek().i;
            final int j = minHeap.peek().j;
            final int d = minHeap.poll().d;
            if (i == m - 1 && j == n - 1)
                return d;
            seen[i][j] = true;
            for (int[] dir : DIRS) {
                final int x = i + dir[0];
                final int y = j + dir[1];
                if (x < 0 || x == m || y < 0 || y == n)
                    continue;
                if (seen[x][y])
                    continue;
                final int newDiff = Math.abs(heights[i][j] - heights[x][y]);
                final int maxDiff = Math.max(diff[i][j], newDiff);
                if (diff[x][y] > maxDiff) {
                    diff[x][y] = maxDiff;
                    minHeap.offer(new T(x, y, maxDiff));
                }
            }
        }

        throw new IllegalArgumentException();
    }
}
```

641. 1632.java

Path: LeetCode-main/solutions/1632. Rank Transform of a Matrix/1632.java

```
class UnionFind {
    public void union(int u, int v) {
        id.putIfAbsent(u, u);
        id.putIfAbsent(v, v);
        final int i = find(u);
        final int j = find(v);
        if (i != j)
            id.put(i, j);
    }

    public Map<Integer, List<Integer>> getGroupIdToValues() {
        Map<Integer, List<Integer>> groupIdToValues = new HashMap<>();
        for (Map.Entry<Integer, Integer> entry : id.entrySet()) {
            final int u = entry.getKey();
            final int i = find(u);
            groupIdToValues.putIfAbsent(i, new ArrayList<>());
            groupIdToValues.get(i).add(u);
        }
        return groupIdToValues;
    }

    private Map<Integer, Integer> id = new HashMap<>();

    private int find(int u) {
        return id.getOrDefault(u, u) == u ? u : find(id.get(u));
    }
}

class Solution {
    public int[][] matrixRankTransform(int[][] matrix) {
        final int m = matrix.length;
        final int n = matrix[0].length;
        int[][] ans = new int[m][n];
        // {val: [(i, j)]}
        TreeMap<Integer, List<Pair<Integer, Integer>>> valToGrids = new TreeMap<>();
        // rank[i] := the maximum rank of the row or column so far
        int[] maxRankSoFar = new int[m + n];

        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j) {
                final int val = matrix[i][j];
                valToGrids.putIfAbsent(val, new ArrayList<>());
                valToGrids.get(val).add(new Pair<>(i, j));
            }

        for (Map.Entry<Integer, List<Pair<Integer, Integer>>> entry : valToGrids.entrySet()) {
            final int val = entry.getKey();
            List<Pair<Integer, Integer>> grids = entry.getValue();
            UnionFind uf = new UnionFind();
            for (Pair<Integer, Integer> grid : grids) {
                final int i = grid.getKey();
                final int j = grid.getValue();
                // Union i-th row with j-th col.
                uf.union(i, j + m);
            }
            for (List<Integer> values : uf.getGroupIdToValues().values()) {
                // Get the maximum rank of all the included rows and columns.
                int maxRank = 0;
                for (final int i : values)
                    maxRank = Math.max(maxRank, maxRankSoFar[i]);
                // Update all the rows and columns to maxRank + 1.
                for (final int i : values)
                    maxRankSoFar[i] = maxRank + 1;
            }
            for (Pair<Integer, Integer> grid : grids) {
                final int i = grid.getKey();
                final int j = grid.getValue();
                ans[i][j] = maxRankSoFar[i];
            }
        }

        return ans;
    }
}
```

642. 1634.java

Path: LeetCode-main/solutions/1634. Add Two Polynomials Represented as Linked Lists/1634.java

```
/**
 * Definition for polynomial singly-linked list.
 * class PolyNode {
 *   int coefficient, power;
 *   PolyNode next = null;
 *   PolyNode() {}
 *   PolyNode(int x, int y) { this.coefficient = x; this.power = y; }
 *   PolyNode(int x, int y, PolyNode next) {
 *     this.coefficient = x;
 *     this.power = y;
 *     this.next = next;
 *   }
 * }
 */

class Solution {
    public PolyNode addPoly(PolyNode poly1, PolyNode poly2) {
        PolyNode dummy = new PolyNode();
        PolyNode curr = dummy;
        PolyNode p = poly1; // poly1's pointer
        PolyNode q = poly2; // poly2's pointer

        while (p != null && q != null) {
            if (p.power > q.power) {
                curr.next = new PolyNode(p.coefficient, p.power);
                curr = curr.next;
                p = p.next;
            } else if (p.power < q.power) {
                curr.next = new PolyNode(q.coefficient, q.power);
                curr = curr.next;
                q = q.next;
            } else { // p.power == q.power
                final int sumCoefficient = p.coefficient + q.coefficient;
                if (sumCoefficient != 0) {
                    curr.next = new PolyNode(sumCoefficient, p.power);
                    curr = curr.next;
                }
                p = p.next;
                q = q.next;
            }
        }

        while (p != null) {
            curr.next = new PolyNode(p.coefficient, p.power);
            curr = curr.next;
            p = p.next;
        }

        while (q != null) {
            curr.next = new PolyNode(q.coefficient, q.power);
            curr = curr.next;
            q = q.next;
        }

        return dummy.next;
    }
}
```

643. 1636.java

Path: LeetCode-main/solutions/1636. Sort Array by Increasing Frequency/1636.java

```
class Solution {
    public int[] frequencySort(int[] nums) {
        record T(int num, int freq) {}
        int[] ans = new int[nums.length];
        int ansIndex = 0;
        Map<Integer, Integer> count = new HashMap<>();
        Queue<T> heap = new PriorityQueue<>(
            Comparator.comparingInt(T::freq).thenComparing(T::num, Comparator.reverseOrder()));

        for (final int num : nums)
            count.merge(num, 1, Integer::sum);

        for (Map.Entry<Integer, Integer> entry : count.entrySet())
            heap.offer(new T(entry.getKey(), entry.getValue()));

        while (!heap.isEmpty()) {
            final int num = heap.peek().num;
            final int freq = heap.poll().freq;
            for (int i = 0; i < freq; ++i)
                ans[ansIndex++] = num;
        }

        return ans;
    }
}
```

644. 1637.java

Path: LeetCode-main/solutions/1637. Widest Vertical Area Between Two Points Containing No Points/1637.java

```
class Solution {
    public int maxWidthOfVerticalArea(int[][] points) {
        int ans = 0;
        int[] xs = new int[points.length];

        for (int i = 0; i < points.length; ++i)
            xs[i] = points[i][0];

        Arrays.sort(xs);

        for (int i = 1; i < xs.length; ++i)
            ans = Math.max(ans, xs[i] - xs[i - 1]);

        return ans;
    }
}
```

645. 1638.java

Path: LeetCode-main/solutions/1638. Count Substrings That Differ by One Character/1638.java

```
class Solution {
    public int countSubstrings(String s, String t) {
        int ans = 0;

        for (int i = 0; i < s.length(); ++i)
            ans += count(s, t, i, 0);

        for (int j = 1; j < t.length(); ++j)
            ans += count(s, t, 0, j);

        return ans;
    }

    // Returns the number of substrings of s[i..n) and t[j..n) that differ by one
    // letter.
    private int count(final String s, final String t, int i, int j) {
        int res = 0;
        // the number of substrings starting at s[i] and t[j] ending in the current
        // index with zero different letter
        int dp0 = 0;
        // the number of substrings starting at s[i] and t[j] ending in the current
        // index with one different letter
        int dp1 = 0;

        for (; i < s.length() && j < t.length(); ++i, ++j) {
            if (s.charAt(i) == t.charAt(j)) {
                ++dp0;
            } else {
                dp1 = dp0 + 1;
                dp0 = 0;
            }
            res += dp1;
        }

        return res;
    }
}
```


646. 1639-2.java

Path: LeetCode-main/solutions/1639. Number of Ways to Form a Target String Given a Dictionary/1639-2.java

```
class Solution {
    public int numWays(String[] words, String target) {
        final int MOD = 1_000_000_007;
        final int wordLength = words[0].length();
        // dp[i][j] := the number of ways to form the first i characters of the
        // `target` using the j first characters in each word
        int[][] dp = new int[target.length() + 1][wordLength + 1];
        // counts[j] := the count map of words[i][j], where 0 <= i < |words|
        int[][] counts = new int[wordLength][26];

        for (int i = 0; i < wordLength; ++i)
            for (final String word : words)
                ++counts[i][word.charAt(i) - 'a'];

        dp[0][0] = 1;

        for (int i = 0; i <= target.length(); ++i)
            for (int j = 0; j < wordLength; ++j) {
                if (i < target.length())
                    // Pick the character target[i] from word[j].
                    dp[i + 1][j + 1] = (int) ((long) dp[i][j] * counts[j][target.charAt(i) - 'a'] % MOD);
                // Skip the word[j].
                dp[i][j + 1] = (dp[i][j + 1] + dp[i][j]) % MOD;
            }

        return dp[target.length()][wordLength];
    }
}
```

647. 1639-3.java

Path: LeetCode-main/solutions/1639. Number of Ways to Form a Target String Given a Dictionary/1639-3.java

```
class Solution {
    public int numWays(String[] words, String target) {
        final int MOD = 1_000_000_007;
        final int wordLength = words[0].length();
        // dp[i] := the number of ways to form the first i characters of `target`
        long[] dp = new long[target.length() + 1];
        dp[0] = 1;

        for (int j = 0; j < wordLength; ++j) {
            int[] count = new int[26];
            for (final String word : words)
                ++count[word.charAt(j) - 'a'];
            for (int i = target.length(); i > 0; --i) {
                dp[i] += dp[i - 1] * count[target.charAt(i - 1) - 'a'];
                dp[i] %= MOD;
            }
        }

        return (int) dp[target.length()];
    }
}
```

648. 1639.java

Path: LeetCode-main/solutions/1639. Number of Ways to Form a Target String Given a Dictionary/1639.java

```
class Solution {
    public int numWays(String[] words, String target) {
        final int wordLength = words[0].length();
        Integer[][] mem = new Integer[target.length()][wordLength];
        // counts[j] := the count map of words[i][j], where 0 <= i < |words|
        int[][] counts = new int[wordLength][26];

        for (int i = 0; i < wordLength; ++i)
            for (final String word : words)
                ++counts[i][word.charAt(i) - 'a'];

        return numWays(target, 0, 0, counts, mem);
    }

    private static final int MOD = 1_000_000_007;

    // Returns the number of ways to form target[i..n) using word[j..n).
    private int numWays(final String target, int i, int j, int[][] counts, Integer[][] mem) {
        if (i == target.length())
            return 1;
        if (j == counts.length)
            return 0;
        if (mem[i][j] != null)
            return mem[i][j];
        return mem[i][j] = (int) (((long) numWays(target, i + 1, j + 1, counts, mem) *
                                   (counts[j][target.charAt(i) - 'a']) +
                                   numWays(target, i, j + 1, counts, mem)) %
                               MOD);
    }
}
```

649. 164.java

Path: LeetCode-main/solutions/164. Maximum Gap/164.java

```
class Bucket {
    public int mn;
    public int mx;
    public Bucket(int mn, int mx) {
        this.mn = mn;
        this.mx = mx;
    }
}

class Solution {
    public int maximumGap(int[] nums) {
        if (nums.length < 2)
            return 0;

        final int mn = Arrays.stream(nums).min().getAsInt();
        final int mx = Arrays.stream(nums).max().getAsInt();
        if (mn == mx)
            return 0;

        final int gap = (int) Math.ceil((double) (mx - mn) / (nums.length - 1));
        final int bucketsLength = (mx - mn) / gap + 1;
        Bucket[] buckets = new Bucket[bucketsLength];

        for (int i = 0; i < buckets.length; ++i)
            buckets[i] = new Bucket(Integer.MAX_VALUE, Integer.MIN_VALUE);

        for (final int num : nums) {
            final int i = (num - mn) / gap;
            buckets[i].mn = Math.min(buckets[i].mn, num);
            buckets[i].mx = Math.max(buckets[i].mx, num);
        }

        int ans = 0;
        int prevMax = mn;

        for (final Bucket bucket : buckets) {
            if (bucket.mn == Integer.MAX_VALUE) // empty bucket
                continue;
            ans = Math.max(ans, bucket.mn - prevMax);
            prevMax = bucket.mx;
        }

        return ans;
    }
}
```

650. 1640.java

Path: LeetCode-main/solutions/1640. Check Array Formation Through Concatenation/1640.java

```
class Solution {
    public boolean canFormArray(int[] arr, int[][] pieces) {
        List<Integer> concatenated = new ArrayList<>();
        Map<Integer, int[]> startToPiece = new HashMap<>();

        for (int[] piece : pieces)
            startToPiece.put(piece[0], piece);

        for (final int a : arr)
            for (final int num : startToPiece.getOrDefault(a, new int[]{}))
                concatenated.add(num);

        return Arrays.equals(concatenated.stream().mapToInt(Integer::intValue).toArray(), arr);
    }
}
```

651. 1641-2.java

Path: LeetCode-main/solutions/1641. Count Sorted Vowel Strings/1641-2.java

```
class Solution {  
    public int countVowelStrings(int n) {  
        return (n + 4) * (n + 3) * (n + 2) * (n + 1) / 24;  
    }  
}
```

652. 1641.java

Path: LeetCode-main/solutions/1641. Count Sorted Vowel Strings/1641.java

```
class Solution {
    public int countVowelStrings(int n) {
        // dp[0] := the number of lexicographically sorted string that end in 'a'
        // dp[1] := the number of lexicographically sorted string that end in 'e'
        // dp[2] := the number of lexicographically sorted string that end in 'i'
        // dp[3] := the number of lexicographically sorted string that end in 'o'
        // dp[4] := the number of lexicographically sorted string that end in 'u'
        int[] dp = new int[5];
        Arrays.fill(dp, 1);

        for (int i = 2; i <= n; ++i) {
            int[] newDp = new int[5];
            for (int j = 0; j < 5; ++j)
                for (int k = 0; k <= j; ++k)
                    newDp[j] += dp[k];
            dp = newDp;
        }

        return Arrays.stream(dp).sum();
    }
}
```

653. 1642.java

Path: LeetCode-main/solutions/1642. Furthest Building You Can Reach/1642.java

```
class Solution {
    public int furthestBuilding(int[] heights, int bricks, int ladders) {
        Queue<Integer> minHeap = new PriorityQueue<>();

        for (int i = 1; i < heights.length; ++i) {
            final int diff = heights[i] - heights[i - 1];
            if (diff <= 0)
                continue;
            minHeap.offer(diff);
            // If we run out of ladders, greedily use as less bricks as possible.
            if (minHeap.size() > ladders)
                bricks -= minHeap.poll();
            if (bricks < 0)
                return i - 1;
        }

        return heights.length - 1;
    }
}
```


654. 1643.java

Path: LeetCode-main/solutions/1643. Kth Smallest Instructions/1643.java

```
class Solution {
    public String kthSmallestPath(int[] destination, int k) {
        StringBuilder sb = new StringBuilder();
        int v = destination[0];
        int h = destination[1];
        final int totalSteps = v + h;
        final int[][] comb = getComb(totalSteps - 1, v);

        for (int i = 0; i < totalSteps; ++i) {
            // If 'H' is picked, we can reach ranks [1, availableRank].
            final int availableRank = comb[h + v - 1][v];
            if (availableRank >= k) { // Should pick 'H'.
                sb.append('H');
                --h;
            } else { // Should pick 'V'.
                k -= availableRank;
                sb.append('V');
                --v;
            }
        }

        return sb.toString();
    }

    private int[][] getComb(int n, int k) {
        int[][] comb = new int[n + 1][k + 1];
        for (int i = 0; i <= n; ++i)
            comb[i][0] = 1;
        for (int i = 1; i <= n; ++i)
            for (int j = 1; j <= k; ++j)
                comb[i][j] = comb[i - 1][j] + comb[i - 1][j - 1];
        return comb;
    }
}
```

655. 1644-2.java

Path: LeetCode-main/solutions/1644. Lowest Common Ancestor of a Binary Tree II/1644-2.java

```
class Solution {
    public TreeNode lowestCommonAncestor(TreeNode root, TreeNode p, TreeNode q) {
        TreeNode ans = getLCA(root, p, q);
        if (ans == p) // Search q in the subtree rooted at p.
            return getLCA(p, q, q) == null ? null : ans;
        if (ans == q) // Search p in the subtree rooted at q.
            return getLCA(q, p, p) == null ? null : ans;
        return ans; // (ans != p && ans != q) || ans == null
    }

    private TreeNode getLCA(TreeNode root, TreeNode p, TreeNode q) {
        if (root == null || root == p || root == q)
            return root;
        TreeNode left = getLCA(root.left, p, q);
        TreeNode right = getLCA(root.right, p, q);
        if (left != null && right != null)
            return root;
        return left == null ? right : left;
    }
}
```

656. 1644.java

Path: LeetCode-main/solutions/1644. Lowest Common Ancestor of a Binary Tree II/1644.java

```
class Solution {
    public TreeNode lowestCommonAncestor(TreeNode root, TreeNode p, TreeNode q) {
        TreeNode lca = getLCA(root, p, q);
        return seenP && seenQ ? lca : null;
    }

    private boolean seenP = false;
    private boolean seenQ = false;

    private TreeNode getLCA(TreeNode root, TreeNode p, TreeNode q) {
        if (root == null)
            return null;
        // Need to traverse the entire tree to update `seenP` and `seenQ`.
        TreeNode left = getLCA(root.left, p, q);
        TreeNode right = getLCA(root.right, p, q);
        if (root == p) {
            seenP = true;
            return root;
        }
        if (root == q) {
            seenQ = true;
            return root;
        }
        if (left != null && right != null)
            return root;
        return left == null ? right : left;
    }
}
```

657. 1646.java

Path: LeetCode-main/solutions/1646. Get Maximum in Generated Array/1646.java

```
class Solution {
    public int getMaximumGenerated(int n) {
        if (n == 0)
            return 0;
        if (n == 1)
            return 1;

        int[] nums = new int[n + 1];
        nums[1] = 1;

        for (int i = 1; (2 * i + 1) <= n; ++i) {
            nums[2 * i] = nums[i];
            nums[2 * i + 1] = nums[i] + nums[i + 1];
        }

        return Arrays.stream(nums).max().getAsInt();
    }
}
```

658. 1647.java

Path: LeetCode-main/solutions/1647. Minimum Deletions to Make Character Frequencies Unique/1647.java

```
class Solution {
    public int minDeletions(String s) {
        int ans = 0;
        int[] count = new int[26];
        Set<Integer> usedFreq = new HashSet<>();

        for (final char c : s.toCharArray())
            ++count[c - 'a'];

        for (int freq : count) {
            while (freq > 0 && usedFreq.contains(freq)) {
                --freq; // Delete ('a' + i).
                ++ans;
            }
            usedFreq.add(freq);
        }

        return ans;
    }
}
```

659. 1648.java

Path: LeetCode-main/solutions/1648. Sell Diminishing-Valued Colored Balls/1648.java

```
class Solution {
    public int maxProfit(int[] inventory, int orders) {
        final int MOD = 1_000_000_007;
        long ans = 0;
        long largestCount = 1;

        Arrays.sort(inventory);

        for (int i = inventory.length - 1; i >= 0; --i, ++largestCount)
            if (i == 0 || inventory[i] > inventory[i - 1]) {
                // If we are at the first inventory, or inventory[i] > inventory[i + 1].
                // In either case, we will pick inventory[i..i + largestCount - 1].
                final long pick = (i == 0) ? inventory[i] : inventory[i] - inventory[i - 1];
                if (largestCount * pick >= orders) {
                    // We have run out of orders, so we need to recalculate the number of
                    // balls that we actually pick for inventory[i..i + largestCount - 1].
                    final long actualPick = orders / largestCount;
                    final long remaining = orders % largestCount;
                    return (int) ((ans +
                        largestCount * trapezoid(inventory[i], inventory[i] - actualPick + 1) +
                        remaining * (inventory[i] - actualPick)) %
                        MOD);
                }
                ans += largestCount * trapezoid(inventory[i], inventory[i] - pick + 1);
                ans %= MOD;
                orders -= largestCount * pick;
            }

        throw new IllegalArgumentException();
    }

    private long trapezoid(long a, long b) {
        return (a + b) * (a - b + 1) / 2;
    }
}
```

660. 165.java

Path: LeetCode-main/solutions/165. Compare Version Numbers/165.java

```
class Solution {
    public int compareVersion(String version1, String version2) {
        final String[] levels1 = version1.split("\\.");
        final String[] levels2 = version2.split("\\.");
        final int length = Math.max(levels1.length, levels2.length);

        for (int i = 0; i < length; ++i) {
            final Integer v1 = i < levels1.length ? Integer.parseInt(levels1[i]) : 0;
            final Integer v2 = i < levels2.length ? Integer.parseInt(levels2[i]) : 0;
            final int compare = v1.compareTo(v2);
            if (compare != 0)
                return compare;
        }

        return 0;
    }
}
```

661. 1650.java

Path: LeetCode-main/solutions/1650. Lowest Common Ancestor of a Binary Tree III/1650.java

```
class Solution {
    // Same as 160. Intersection of Two Linked Lists
    public Node lowestCommonAncestor(Node p, Node q) {
        Node a = p;
        Node b = q;

        while (a != b) {
            a = a == null ? q : a.parent;
            b = b == null ? p : b.parent;
        }

        return a;
    }
}
```


662. 1652.java

Path: LeetCode-main/solutions/1652. Defuse the Bomb/1652.java

```
class Solution {
    public int[] decrypt(int[] code, int k) {
        final int n = code.length;
        int[] ans = new int[n];
        if (k == 0)
            return ans;

        int sum = 0;
        int start = k > 0 ? 1 : n + k; // the start of the next k numbers
        int end = k > 0 ? k : n - 1; // the end of the next k numbers

        for (int i = start; i <= end; ++i)
            sum += code[i];

        for (int i = 0; i < n; ++i) {
            ans[i] = sum;
            sum -= code[start++ % n];
            sum += code[++end % n];
        }

        return ans;
    }
}
```

663. 1653.java

Path: LeetCode-main/solutions/1653. Minimum Deletions to Make String Balanced/1653.java

```
class Solution {
    // Same as 926. Flip String to Monotone Increasing
    public int minimumDeletions(String s) {
        // the number of characters to be deleted to make subString so far balanced
        int dp = 0;
        int countB = 0;

        for (final char c : s.toCharArray())
            if (c == 'a')
                // 1. Delete 'a'.
                // 2. Keep 'a' and delete the previous 'b's.
                dp = Math.min(dp + 1, countB);
            else
                ++countB;

        return dp;
    }
}
```

664. 1654.java

Path: LeetCode-main/solutions/1654. Minimum Jumps to Reach Home/1654.java

```
enum Direction { FORWARD, BACKWARD }

class Solution {
    public int minimumJumps(int[] forbidden, int a, int b, int x) {
        int furthest = x + a + b;
        Set<Integer> seenForward = new HashSet<>();
        Set<Integer> seenBackward = new HashSet<>();

        for (final int pos : forbidden) {
            seenForward.add(pos);
            seenBackward.add(pos);
            furthest = Math.max(furthest, pos + a + b);
        }

        // (direction, position)
        Queue<Pair<Direction, Integer>> q = new ArrayDeque<>(List.of(new Pair<>(Direction.FORWARD, 0)));

        for (int ans = 0; !q.isEmpty(); ++ans)
            for (int sz = q.size(); sz > 0; --sz) {
                Direction dir = q.peek().getKey();
                final int pos = q.poll().getValue();
                if (pos == x)
                    return ans;
                final int forward = pos + a;
                final int backward = pos - b;
                if (forward <= furthest && seenForward.add(forward))
                    q.offer(new Pair<>(Direction.FORWARD, forward));
                // It cannot jump backward twice in a row.
                if (dir == Direction.FORWARD && backward >= 0 && seenBackward.add(backward))
                    q.offer(new Pair<>(Direction.BACKWARD, backward));
            }

        return -1;
    }
}
```

665. 1655.java

Path: LeetCode-main/solutions/1655. Distribute Repeating Integers/1655.java

```
class Solution {
    public boolean canDistribute(int[] nums, int[] quantity) {
        List<Integer> freqs = getFreqs(nums);
        // validDistribution[i][j] := true if it's possible to distribute the i-th
        // freq into a subset of quantity represented by the bitmask j
        boolean[][] validDistribution = getValidDistribution(freqs, quantity);
        final int n = freqs.size();
        final int m = quantity.length;
        final int maxMask = 1 << m;
        // dp[i][j] := true if it's possible to distribute freqs[i..n), where j is
        // the bitmask of the selected quantity
        boolean[][] dp = new boolean[n + 1][maxMask];
        dp[n][maxMask - 1] = true;

        for (int i = n - 1; i >= 0; --i)
            for (int mask = 0; mask < maxMask; ++mask) {
                dp[i][mask] = dp[i + 1][mask];
                final int availableMask = ~mask & (maxMask - 1);
                for (int submask = availableMask; submask > 0; submask = (submask - 1) & availableMask)
                    if (validDistribution[i][submask])
                        dp[i][mask] = dp[i][mask] || dp[i + 1][mask | submask];
            }

        return dp[0][0];
    }

    private List<Integer> getFreqs(int[] nums) {
        List<Integer> freqs = new ArrayList<>();
        Map<Integer, Integer> count = new HashMap<>();
        for (final int num : nums)
            count.merge(num, 1, Integer::sum);
        return new ArrayList<>(count.values());
    }

    boolean[][] getValidDistribution(List<Integer> freqs, int[] quantity) {
        final int maxMask = 1 << quantity.length;
        boolean[][] validDistribution = new boolean[freqs.size()][maxMask];
        for (int i = 0; i < freqs.size(); ++i)
            for (int mask = 0; mask < maxMask; ++mask)
                if (freqs.get(i) >= getQuantitySum(quantity, mask))
                    validDistribution[i][mask] = true;
        return validDistribution;
    }

    // Returns the sum of the selected quantity represented by `mask`.
    int getQuantitySum(int[] quantity, int mask) {
        int sum = 0;
        for (int i = 0; i < quantity.length; ++i)
            if ((mask >> i & 1) == 1)
                sum += quantity[i];
        return sum;
    }
}
```

666. 1656.java

Path: LeetCode-main/solutions/1656. Design an Ordered Stream/1656.java

```
class OrderedStream {
    public OrderedStream(int n) {
        this.values = new String[n];
    }

    public List<String> insert(int idKey, String value) {
        --idKey; // Converts to 0-indexed
        values[idKey] = value;
        if (idKey > i)
            return new ArrayList<>();
        while (i < values.length && values[i] != null && !values[i].isEmpty())
            i++;
        List<String> res = new ArrayList<>();
        for (int j = idKey; j < i; ++j)
            res.add(values[j]);
        return res;
    }

    private String[] values;
    private int i = 0; // values' index (0-indexed)
}
```

667. 1657.java

Path: LeetCode-main/solutions/1657. Determine if Two Strings Are Close/1657.java

```
class Solution {
    public boolean closeStrings(String word1, String word2) {
        if (word1.length() != word2.length())
            return false;

        Map<Character, Integer> count1 = new HashMap<>();
        Map<Character, Integer> count2 = new HashMap<>();

        for (final char c : word1.toCharArray())
            count1.merge(c, 1, Integer::sum);

        for (final char c : word2.toCharArray())
            count2.merge(c, 1, Integer::sum);

        if (!count1.keySet().equals(count2.keySet()))
            return false;

        List<Integer> freqs1 = new ArrayList<>(count1.values());
        List<Integer> freqs2 = new ArrayList<>(count2.values());

        Collections.sort(freqs1);
        Collections.sort(freqs2);
        return freqs1.equals(freqs2);
    }
}
```

668. 1658.java

Path: LeetCode-main/solutions/1658. Minimum Operations to Reduce X to Zero/1658.java

```
class Solution {
    public int minOperations(int[] nums, int x) {
        final int targetSum = Arrays.stream(nums).sum() - x;
        if (targetSum == 0)
            return nums.length;
        final int maxLen = maxSubArrayLen(nums, targetSum);
        return maxLen == -1 ? -1 : nums.length - maxLen;
    }

    // Same as 325. Maximum Size Subarray Sum Equals k
    private int maxSubArrayLen(int[] nums, int k) {
        int res = -1;
        int prefix = 0;
        Map<Integer, Integer> prefixToIndex = new HashMap<>();
        prefixToIndex.put(0, -1);

        for (int i = 0; i < nums.length; ++i) {
            prefix += nums[i];
            final int target = prefix - k;
            if (prefixToIndex.containsKey(target))
                res = Math.max(res, i - prefixToIndex.get(target));
            // No need to check the existence of the prefix since it's unique.
            prefixToIndex.put(prefix, i);
        }

        return res;
    }
}
```

669. 1659.java

Path: LeetCode-main/solutions/1659. Maximize Grid Happiness/1659.java

```
class Solution {
    public int getMaxGridHappiness(int m, int n, int introvertsCount, int extrovertsCount) {
        final int twoToThePowerOfN = (int) Math.pow(2, n);
        int[][][][][] mem = new int[m * n][twoToThePowerOfN][twoToThePowerOfN][introvertsCount + 1][extrovertsCount + 1];
        return getMaxGridHappiness(m, n, 0, 0, 0, introvertsCount, extrovertsCount, mem);
    }

    // Calculates the cost based on left and up neighbors.
    //
    // The `diff` parameter represents the happiness change due to the current
    // placed person in (i, j). We add `diff` each time we encounter a neighbor
    // (left or up) who is already placed.
    //
    // 1. If the neighbor is an introvert, we subtract 30 from cost.
    // 2. If the neighbor is an extrovert, we add 20 to from cost.
    private int getPlacementCost(int n, int i, int j, int inMask, int exMask, int diff) {
        int cost = 0;
        if (i > 0) {
            if (((1 << (n - 1)) & inMask) > 0)
                cost += diff - 30;
            if (((1 << (n - 1)) & exMask) > 0)
                cost += diff + 20;
        }
        if (j > 0) {
            if ((1 & inMask) > 0)
                cost += diff - 30;
            if ((1 & exMask) > 0)
                cost += diff + 20;
        }
        return cost;
    }

    private int getMaxGridHappiness(int m, int n, int pos, int inMask, int exMask, int inCount,
                                    int exCount, int[][][][][] mem) {
        // `inMask` is the placement of introvert people in the last n cells.
        // e.g. if we have m = 2, n = 3, i = 1, j = 1, then inMask = 0b101 means
        //
        // ? 1 0
        // 1 x ? (x := current position)
        final int i = pos / n;
        final int j = pos % n;
        if (i == m)
            return 0;
        if (mem[pos][inMask][exMask][inCount][exCount] > 0)
            return mem[pos][inMask][exMask][inCount][exCount];

        final int shiftedInMask = (inMask << 1) & ((1 << n) - 1);
        final int shiftedExMask = (exMask << 1) & ((1 << n) - 1);

        final int skip =
            getMaxGridHappiness(m, n, pos + 1, shiftedInMask, shiftedExMask, inCount, exCount, mem);
        final int placeIntrovert =
            inCount > 0 ? 120 + getPlacementCost(n, i, j, inMask, exMask, -30) +
                getMaxGridHappiness(m, n, pos + 1, shiftedInMask | 1, shiftedExMask,
                                    inCount - 1, exCount, mem)
                : Integer.MIN_VALUE;
        final int placeExtrovert =
            exCount > 0 ? 40 + getPlacementCost(n, i, j, inMask, exMask, 20) +
                getMaxGridHappiness(m, n, pos + 1, shiftedInMask, shiftedExMask | 1,
                                    inCount, exCount - 1, mem)
                : Integer.MIN_VALUE;
        return mem[pos][inMask][exMask][inCount][exCount] =
            Math.max(skip, Math.max(placeIntrovert, placeExtrovert));
    }
}
```


670. 166.java

Path: LeetCode-main/solutions/166. Fraction to Recurring Decimal/166.java

```
class Solution {
    public String fractionToDecimal(int numerator, int denominator) {
        if (numerator == 0)
            return "0";

        StringBuilder sb = new StringBuilder();

        if (numerator < 0 ^ denominator < 0)
            sb.append("-");

        long n = Math.abs((long) numerator);
        long d = Math.abs((long) denominator);
        sb.append(n / d);

        if (n % d == 0)
            return sb.toString();

        sb.append(".");
        Map<Long, Integer> seen = new HashMap<>();

        for (long r = n % d; r > 0; r %= d) {
            if (seen.containsKey(r)) {
                sb.insert(seen.get(r), "(");
                sb.append(")");
                break;
            }
            seen.put(r, sb.length());
            r *= 10;
            sb.append(r / d);
        }

        return sb.toString();
    }
}
```

671. 1660.java

Path: LeetCode-main/solutions/1660. Correct a Binary Tree/1660.java

```
class Solution {
    public TreeNode correctBinaryTree(TreeNode root) {
        if (root == null)
            return null;
        if (root.right != null && seen.contains(root.right.val))
            return null;
        seen.add(root.val);
        root.right = correctBinaryTree(root.right);
        root.left = correctBinaryTree(root.left);
        return root;
    }

    private Set<Integer> seen = new HashSet<>();
}
```

672. 1662.java

Path: LeetCode-main/solutions/1662. Check If Two String Arrays are Equivalent/1662.java

```
class Solution {
    public boolean arrayStringsAreEqual(String[] word1, String[] word2) {
        int i = 0; // word1's index
        int j = 0; // word2's index
        int a = 0; // word1[i]'s index
        int b = 0; // word2[j]'s index

        while (i < word1.length && j < word2.length) {
            if (word1[i].charAt(a) != word2[j].charAt(b))
                return false;
            if (++a == word1[i].length()) {
                ++i;
                a = 0;
            }
            if (++b == word2[j].length()) {
                ++j;
                b = 0;
            }
        }

        return i == word1.length && j == word2.length;
    }
}
```

673. 1663.java

Path: LeetCode-main/solutions/1663. Smallest String With A Given Numeric Value/1663.java

```
class Solution {
    public String getSmallestString(int n, int k) {
        StringBuilder sb = new StringBuilder();

        for (int i = 0; i < n; ++i) {
            final int remainingLetters = n - 1 - i;
            final int rank = Math.max(1, k - remainingLetters * 26);
            sb.append((char) ('a' + rank - 1));
            k -= rank;
        }

        return sb.toString();
    }
}
```

674. 1664-2.java

Path: LeetCode-main/solutions/1664. Ways to Make a Fair Array/1664-2.java

```
class Solution {
    public int waysToMakeFair(int[] nums) {
        final int n = nums.length;
        int ans = 0;
        // l[0] := the sum of even-indexed nums[0..i)
        // l[1] := the sum of odd-indexed nums[0..i)
        // r[0] := the sum of even-indexed nums[i + 1..n)
        // r[1] := the sum of odd-indexed nums[i + 1..n)
        int[] l = new int[2];
        int[] r = new int[2];

        for (int i = 0; i < n; ++i)
            r[i % 2] += nums[i];

        for (int i = 0; i < n; ++i) {
            r[i % 2] -= nums[i];
            if (l[0] + r[1] == l[1] + r[0])
                ++ans;
            l[i % 2] += nums[i];
        }

        return ans;
    }
}
```

675. 1664.java

Path: LeetCode-main/solutions/1664. Ways to Make a Fair Array/1664.java

```
class Solution {
    public int waysToMakeFair(int[] nums) {
        final int n = nums.length;
        int ans = 0;
        int[] even = new int[n + 1]; // the sum of even-indexed nums[0..i)
        int[] odd = new int[n + 1];  // the sum of odd-indexed nums[0..i)

        for (int i = 1; i <= n; ++i) {
            odd[i] = odd[i - 1];
            even[i] = even[i - 1];
            if (i % 2 == 0)
                even[i] += nums[i - 1];
            else
                odd[i] += nums[i - 1];
        }

        final int sum = even[n] + odd[n];

        for (int i = 0; i < n; ++i) {
            final int evenSum = even[i] + odd[n] - odd[i + 1];
            final int oddSum = sum - nums[i] - evenSum;
            if (evenSum == oddSum)
                ++ans;
        }

        return ans;
    }
}
```

676. 1665.java

Path: LeetCode-main/solutions/1665. Minimum Initial Energy to Finish Tasks/1665.java

```
class Solution {
    public int minimumEffort(int[][] tasks) {
        int ans = 0;
        int prevSaved = 0;

        Arrays.sort(tasks, Comparator.comparingInt(task -> task[0] - task[1]));

        for (int[] task : tasks) {
            final int actual = task[0];
            final int minimum = task[1];
            if (prevSaved < minimum) {
                ans += minimum - prevSaved;
                prevSaved = minimum - actual;
            } else {
                prevSaved -= actual;
            }
        }

        return ans;
    }
}
```

677. 1666.java

Path: LeetCode-main/solutions/1666. Change the Root of a Binary Tree/1666.java

```
class Solution {
    public Node flipBinaryTree(Node root, Node leaf) {
        return reroot(root, leaf, null);
    }

    private Node reroot(Node root, Node node, Node newParent) {
        Node oldParent = node.parent;
        node.parent = newParent;

        // Clean up the child if it's the new parent.
        if (node.left == newParent)
            node.left = null;
        if (node.right == newParent)
            node.right = null;

        // We meet the original root, so we're done.
        if (node == root)
            return node;

        if (node.left != null)
            node.right = node.left;
        node.left = reroot(root, oldParent, node);

        return node;
    }
}
```


678. 1668.java

Path: LeetCode-main/solutions/1668. Maximum Repeating Substring/1668.java

```
class Solution {  
    public int maxRepeating(String sequence, String word) {  
        int ans = 1;  
        while (sequence.contains(word.repeat(ans)))  
            ++ans;  
        return ans - 1;  
    }  
}
```

679. 1669.java

Path: LeetCode-main/solutions/1669. Merge In Between Linked Lists/1669.java

```
class Solution {
    public ListNode mergeInBetween(ListNode list1, int a, int b, ListNode list2) {
        ListNode nodeBeforeA = list1;
        for (int i = 0; i < a - 1; ++i)
            nodeBeforeA = nodeBeforeA.next;

        ListNode nodeB = nodeBeforeA.next;
        for (int i = 0; i < b - a; ++i)
            nodeB = nodeB.next;

        nodeBeforeA.next = list2;
        ListNode lastNodeInList2 = list2;

        while (lastNodeInList2.next != null)
            lastNodeInList2 = lastNodeInList2.next;

        lastNodeInList2.next = nodeB.next;
        nodeB.next = null;
        return list1;
    }
}
```

680. 167.java

Path: LeetCode-main/solutions/167. Two Sum II - Input array is sorted/167.java

```
class Solution {  
    public int[] twoSum(int[] numbers, int target) {  
        int l = 0;  
        int r = numbers.length - 1;  
  
        while (numbers[l] + numbers[r] != target)  
            if (numbers[l] + numbers[r] < target)  
                ++l;  
            else  
                --r;  
  
        return new int[] {l + 1, r + 1};  
    }  
}
```

681. 1670.java

Path: LeetCode-main/solutions/1670. Design Front Middle Back Queue/1670.java

```
class FrontMiddleBackQueue {
    public void pushFront(int val) {
        frontQueue.offerFirst(val);
        moveFrontToBackIfNeeded();
    }

    public void pushMiddle(int val) {
        if (| frontQueue | == | backQueue |)
            backQueue.offerFirst(val);
        else
            frontQueue.offerLast(val);
    }

    public void pushBack(int val) {
        backQueue.offerLast(val);
        moveBackToFrontIfNeeded();
    }

    public int popFront() {
        if (!frontQueue.isEmpty()) {
            final int x = frontQueue.removeFirst();
            moveBackToFrontIfNeeded();
            return x;
        }
        if (!backQueue.isEmpty())
            return backQueue.pollFirst();
        return -1;
    }

    public int popMiddle() {
        if (frontQueue.isEmpty() && backQueue.isEmpty())
            return -1;
        if (frontQueue.size() + 1 == backQueue.size())
            return backQueue.pollFirst();
        // |frontQueue| == |backQueue|
        return frontQueue.pollLast();
    }

    public int popBack() {
        if (backQueue.isEmpty())
            return -1;
        final int x = backQueue.removeLast();
        moveFrontToBackIfNeeded();
        return x;
    }

    private void moveFrontToBackIfNeeded() {
        if (frontQueue.size() - 1 == backQueue.size())
            backQueue.offerFirst(frontQueue.pollLast());
    }

    private void moveBackToFrontIfNeeded() {
        if (frontQueue.size() + 2 == backQueue.size())
            frontQueue.offerLast(backQueue.pollFirst());
    }

    private Deque<Integer> frontQueue = new ArrayDeque<>();
    private Deque<Integer> backQueue = new ArrayDeque<>();
}
```

682. 1671.java

Path: LeetCode-main/solutions/1671. Minimum Number of Removals to Make Mountain Array/1671.java

```
class Solution {
    public int minimumMountainRemovals(int[] nums) {
        int[] l = lengthOfLIS(nums);
        int[] r = reversed(lengthOfLIS(reversed(nums)));
        int maxMountainSeq = 0;

        for (int i = 0; i < nums.length; ++i)
            if (l[i] > 1 && r[i] > 1)
                maxMountainSeq = Math.max(maxMountainSeq, l[i] + r[i] - 1);

        return nums.length - maxMountainSeq;
    }

    // Similar to 300. Longest Increasing Subsequence
    private int[] lengthOfLIS(int[] nums) {
        // tails[i] := the minimum tail of all the increasing subsequences with
        // length i + 1
        List<Integer> tails = new ArrayList<>();
        // dp[i] := the length of LIS ending in nums[i]
        int[] dp = new int[nums.length];
        for (int i = 0; i < nums.length; ++i) {
            final int num = nums[i];
            if (tails.isEmpty() || num > tails.get(tails.size() - 1))
                tails.add(num);
            else
                tails.set(firstGreaterEqual(tails, num), num);
            dp[i] = tails.size();
        }
        return dp;
    }

    private int firstGreaterEqual(List<Integer> arr, int target) {
        final int i = Collections.binarySearch(arr, target);
        return i < 0 ? -i - 1 : i;
    }

    private int[] reversed(int[] nums) {
        int[] arr = nums.clone();
        int l = 0;
        int r = nums.length - 1;
        while (l < r)
            swap(arr, l++, r--);
        return arr;
    }

    private void swap(int[] arr, int i, int j) {
        final int temp = arr[i];
        arr[i] = arr[j];
        arr[j] = temp;
    }
}
```

683. 1672.java

Path: LeetCode-main/solutions/1672. Richest Customer Wealth/1672.java

```
class Solution {  
    public int maximumWealth(int[][] accounts) {  
        return Arrays.stream(accounts)  
            .mapToInt(account -> Arrays.stream(account).sum())  
            .max()  
            .getAsInt();  
    }  
}
```

684. 1673.java

Path: LeetCode-main/solutions/1673. Find the Most Competitive Subsequence/1673.java

```
class Solution {
    public int[] mostCompetitive(int[] nums, int k) {
        int[] ans = new int[k];
        Deque<Integer> stack = new ArrayDeque<>();

        for (int i = 0; i < nums.length; ++i) {
            // If |stack| - 1 + |nums[i..n]| >= k, then it means we still have enough
            // numbers, and we can safely pop an element from stack.
            while (!stack.isEmpty() && stack.peek() > nums[i] && stack.size() - 1 + nums.length - i >= k)
                stack.pop();
            if (stack.size() < k)
                stack.push(nums[i]);
        }

        for (int i = 0; i < k; i++)
            ans[i] = stack.pollLast();

        return ans;
    }
}
```

685. 1674.java

Path: LeetCode-main/solutions/1674. Minimum Moves to Make Array Complementary/1674.java

```
class Solution {
    public int minMoves(int[] nums, int limit) {
        final int n = nums.length;
        int ans = n;
        // delta[i] := the number of moves needed when target goes from i - 1 to i
        int[] delta = new int[limit * 2 + 2];

        for (int i = 0; i < n / 2; ++i) {
            final int a = nums[i];
            final int b = nums[n - 1 - i];
            --delta[Math.min(a, b) + 1];
            --delta[a + b];
            ++delta[a + b + 1];
            ++delta[Math.max(a, b) + limit + 1];
        }

        // Initially, we need `moves` when the target is 2.
        for (int i = 2, moves = n; i <= limit * 2; ++i) {
            moves += delta[i];
            ans = Math.min(ans, moves);
        }

        return ans;
    }
}
```


686. 1675.java

Path: LeetCode-main/solutions/1675. Minimize Deviation in Array/1675.java

```
class Solution {
    public int minimumDeviation(int[] nums) {
        int ans = Integer.MAX_VALUE;
        int mn = Integer.MAX_VALUE;
        Queue<Integer> maxHeap = new PriorityQueue<>(Collections.reverseOrder());

        for (final int num : nums) {
            final int evenNum = num % 2 == 0 ? num : num * 2;
            mn = Math.min(mn, evenNum);
            maxHeap.offer(evenNum);
        }

        while (maxHeap.peek() % 2 == 0) {
            final int mx = maxHeap.poll();
            ans = Math.min(ans, mx - mn);
            mn = Math.min(mn, mx / 2);
            maxHeap.offer(mx / 2);
        }

        return Math.min(ans, maxHeap.peek() - mn);
    }
}
```

687. 1676.java

Path: LeetCode-main/solutions/1676. Lowest Common Ancestor of a Binary Tree IV/1676.java

```
class Solution {
    public TreeNode lowestCommonAncestor(TreeNode root, TreeNode[] nodes) {
        Set<TreeNode> nodesSet = new HashSet<>(Arrays.asList(nodes));
        return lca(root, nodesSet);
    }

    private TreeNode lca(TreeNode root, Set<TreeNode> nodesSet) {
        if (root == null)
            return null;
        if (nodesSet.contains(root))
            return root;
        TreeNode left = lca(root.left, nodesSet);
        TreeNode right = lca(root.right, nodesSet);
        if (left != null && right != null)
            return root;
        return left == null ? right : left;
    }
}
```

688. 1678.java

Path: LeetCode-main/solutions/1678. Goal Parser Interpretation/1678.java

```
class Solution {
    public String interpret(String command) {
        StringBuilder sb = new StringBuilder();
        for (int i = 0; i < command.length(); i++)
            if (command.charAt(i) == 'G') {
                sb.append("G");
                ++i;
            } else if (command.charAt(i + 1) == ')') {
                sb.append("o");
                i += 2;
            } else {
                sb.append("al");
                i += 4;
            }
        return sb.toString();
    }
}
```

689. 1679.java

Path: LeetCode-main/solutions/1679. Max Number of K-Sum Pairs/1679.java

```
class Solution {
    public int maxOperations(int[] nums, int k) {
        int ans = 0;
        HashMap<Integer, Integer> count = new HashMap<>();

        for (final int num : nums)
            count.merge(num, 1, Integer::sum);

        for (final int num : count.keySet()) {
            final int complement = k - num;
            if (count.containsKey(complement))
                ans += Math.min(count.get(num), count.get(complement));
        }

        return ans / 2;
    }
}
```

690. 168.java

Path: LeetCode-main/solutions/168. Excel Sheet Column Title/168.java

```
class Solution {  
    public String convertToTitle(int n) {  
        return n == 0 ? "" : convertToTitle((n - 1) / 26) + (char) ('A' + ((n - 1) % 26));  
    }  
}
```

691. 1680-2.java

Path: LeetCode-main/solutions/1680. Concatenation of Consecutive Binary Numbers/1680-2.java

```
class Solution {
    public int concatenatedBinary(int n) {
        final int MOD = 1_000_000_007;
        long ans = 0;
        int numberOfBits = 0;

        for (int i = 1; i <= n; ++i) {
            if (Integer.bitCount(i) == 1)
                ++numberOfBits;
            ans = ((ans << numberOfBits) % MOD + i) % MOD;
        }

        return (int) ans;
    }
}
```

692. 1680.java

Path: LeetCode-main/solutions/1680. Concatenation of Consecutive Binary Numbers/1680.java

```
class Solution {
    public int concatenatedBinary(int n) {
        final int MOD = 1_000_000_007;
        long ans = 0;

        for (int i = 1; i <= n; ++i)
            ans = ((ans << numberOfBits(i)) % MOD + i) % MOD;

        return (int) ans;
    }

    private int numberOfBits(int n) {
        return (int) (Math.log(n) / Math.log(2)) + 1;
    }
}
```

693. 1681.java

Path: LeetCode-main/solutions/1681. Minimum Incompatibility/1681.java

```
class Solution {
    public int minimumIncompatibility(int[] nums, int k) {
        final int MAX_COMPATIBILITY = (16 - 1) * (16 / 2);
        final int n = nums.length;
        final int subsetSize = n / k;
        final int maxMask = 1 << n;
        final int[] incompatibilities = getIncompatibilities(nums, subsetSize);
        // dp[i] := the minimum possible sum of incompatibilities of the subset
        // of numbers represented by the bitmask i
        int[] dp = new int[maxMask];
        Arrays.fill(dp, MAX_COMPATIBILITY);
        dp[0] = 0;

        for (int mask = 1; mask < maxMask; ++mask) {
            // The number of 1s in `mask` isn't a multiple of `subsetSize`.
            if (Integer.bitCount(mask) % subsetSize != 0)
                continue;
            // https://cp-algorithms.com/algebra/all-submasks.html
            for (int submask = mask; submask > 0; submask = (submask - 1) & mask)
                if (incompatibilities[submask] != -1) // valid submask
                    dp[mask] = Math.min(dp[mask], dp[mask - submask] + incompatibilities[submask]);
        }

        return dp[maxMask - 1] == MAX_COMPATIBILITY ? -1 : dp[maxMask - 1];
    }

    private static final int MAX_NUM = 16;

    // Returns an incompatibilities array where
    // * incompatibilities[i] := the incompatibility of the subset of numbers
    //   represented by the bitmask i
    // * incompatibilities[i] := -1 if the number of 1s in the bitmask i is not
    //   `subsetSize`
    private int[] getIncompatibilities(int[] nums, int subsetSize) {
        final int maxMask = 1 << nums.length;
        int[] incompatibilities = new int[maxMask];
        Arrays.fill(incompatibilities, -1);
        for (int mask = 0; mask < maxMask; ++mask)
            if (Integer.bitCount(mask) == subsetSize && isUnique(nums, mask, subsetSize))
                incompatibilities[mask] = getIncompatibility(nums, mask);
        return incompatibilities;
    }

    // Returns true if the numbers selected by `mask` are unique.
    //
    // e.g. If we call isUnique(0b1010, 2, [1, 2, 1, 4]), `used` variable
    // will be 0b1, which only has one 1 (less than `subsetSize`). In this case,
    // we should return false.
    private boolean isUnique(int[] nums, int mask, int subsetSize) {
        int used = 0;
        for (int i = 0; i < nums.length; ++i)
            if ((mask >> i & 1) == 1)
                used |= 1 << nums[i];
        return Integer.bitCount(used) == subsetSize;
    }

    // Returns the incompatibility of the selected numbers represented by the
    // `mask`.
    private int getIncompatibility(int[] nums, int mask) {
        int mn = MAX_NUM;
        int mx = 0;
        for (int i = 0; i < nums.length; ++i)
            if ((mask >> i & 1) == 1) {
                mx = Math.max(mx, nums[i]);
                mn = Math.min(mn, nums[i]);
            }
        return mx - mn;
    }
}
```


694. 1682-2.java

Path: LeetCode-main/solutions/1682. Longest Palindromic Subsequence II/1682-2.java

```
class Solution {
    public int longestPalindromeSubseq(String s) {
        final int n = s.length();
        // dp[i][j][k] := the length of LPS(s[i..j]), where the previous letter is
        // ('a' + k).
        int[][][] dp = new int[n][n][27];

        for (int d = 1; d < n; ++d)
            for (int i = 0; i + d < n; ++i)
                for (int k = 0; k <= 26; ++k) {
                    final int j = i + d;
                    if (s.charAt(i) == s.charAt(j) && s.charAt(i) != 'a' + k)
                        dp[i][j][k] = dp[i + 1][j - 1][s.charAt(i) - 'a'] + 2;
                    else
                        dp[i][j][k] = Math.max(dp[i + 1][j][k], dp[i][j - 1][k]);
                }

        return dp[0][n - 1][26];
    }
}
```

695. 1682.java

Path: LeetCode-main/solutions/1682. Longest Palindromic Subsequence II/1682.java

```
class Solution {
    public int longestPalindromeSubseq(String s) {
        final int n = s.length();
        int[][][] mem = new int[n][n][27];
        return lps(s, 0, n - 1, 26, mem);
    }

    // Returns the length of LPS(s[i..j]), where the previous letter is ('a' + k).
    private int lps(final String s, int i, int j, int k, int[][][] mem) {
        if (i >= j)
            return 0;
        if (mem[i][j][k] > 0)
            return mem[i][j][k];
        if (s.charAt(i) == s.charAt(j) && s.charAt(i) != (char) (k + 'a'))
            return mem[i][j][k] = lps(s, i + 1, j - 1, s.charAt(i) - 'a', mem) + 2;
        return mem[i][j][k] = Math.max(lps(s, i + 1, j, k, mem), lps(s, i, j - 1, k, mem));
    }
}
```

696. 1684.java

Path: LeetCode-main/solutions/1684. Count the Number of Consistent Strings/1684.java

```
class Solution {  
    public int countConsistentStrings(String allowed, String[] words) {  
        return (int) Arrays.stream(words)  
            .filter(word -> word.matches(String.format("[%s]*", allowed)))  
            .count();  
    }  
}
```

697. 1685.java

Path: LeetCode-main/solutions/1685. Sum of Absolute Differences in a Sorted Array/1685.java

```
class Solution {
    public int[] getSumAbsoluteDifferences(int[] nums) {
        final int n = nums.length;
        int[] ans = new int[n];
        int[] prefix = new int[n];
        int[] suffix = new int[n];

        prefix[0] = nums[0];
        for (int i = 1; i < n; ++i)
            prefix[i] = prefix[i - 1] + nums[i];

        suffix[n - 1] = nums[n - 1];
        for (int i = n - 2; i >= 0; --i)
            suffix[i] = suffix[i + 1] + nums[i];

        for (int i = 0; i < nums.length; ++i) {
            final int left = nums[i] * (i + 1) - prefix[i];
            final int right = suffix[i] - nums[i] * (n - i);
            ans[i] = left + right;
        }

        return ans;
    }
}
```

698. 1686.java

Path: LeetCode-main/solutions/1686. Stone Game VI/1686.java

```
class Solution {
    public int stoneGameVI(int[] aliceValues, int[] bobValues) {
        final int n = aliceValues.length;
        int[][] values = new int[n][2];

        for (int i = 0; i < n; ++i)
            values[i] = new int[] {aliceValues[i], bobValues[i]};

        Arrays.sort(values, Comparator.comparingInt(value -> - value[0] - value[1]));

        int a = 0;
        int b = 0;

        for (int i = 0; i < n; ++i)
            if (i % 2 == 0)
                a += values[i][0];
            else
                b += values[i][1];

        return Integer.compare(a, b);
    }
}
```

699. 1687.java

Path: LeetCode-main/solutions/1687. Delivering Boxes from Storage to Ports/1687.java

```
class Solution {
    public int boxDelivering(int[][] boxes, int portsCount, int maxBoxes, int maxWeight) {
        final int n = boxes.length;
        // dp[i] := the minimum trips to deliver boxes[0..i) and return to the
        // storage
        int[] dp = new int[n + 1];
        int trips = 2;
        int weight = 0;

        for (int l = 0, r = 0; r < n; ++r) {
            weight += boxes[r][1];

            // The current box is different from the previous one, need to make one
            // more trip.
            if (r > 0 && boxes[r][0] != boxes[r - 1][0])
                ++trips;

            while (r - l + 1 > maxBoxes || weight > maxWeight ||
                // Loading boxes[l] in the previous turn is always no bad than
                // loading it in this turn.
                (l < r && dp[l + 1] == dp[l])) {
                weight -= boxes[l][1];
                if (boxes[l][0] != boxes[l + 1][0])
                    --trips;
                ++l;
            }

            // the minimum trips to deliver boxes[0..r]
            // = the minimum trips to deliver boxes[0..l) +
            //   trips to deliver boxes[l..r]
            dp[r + 1] = dp[l] + trips;
        }

        return dp[n];
    }
}
```

700. 1688.java

Path: LeetCode-main/solutions/1688. Count of Matches in Tournament/1688.java

```
class Solution {  
    public int numberOfMatches(int n) {  
        return n - 1;  
    }  
}
```

701. 1689.java

Path: LeetCode-main/solutions/1689. Partitioning Into Minimum Number Of Deci-Binary Numbers/1689.java

```
class Solution {  
    public int minPartitions(String n) {  
        return Character.getNumericValue(n.chars().max().getAsInt());  
    }  
}
```


702. 169.java

Path: LeetCode-main/solutions/169. Majority Element/169.java

```
class Solution {
    public int majorityElement(int[] nums) {
        Integer ans = null;
        int count = 0;

        for (final int num : nums) {
            if (count == 0)
                ans = num;
            count += num == ans ? 1 : -1;
        }

        return ans;
    }
}
```

703. 1690-2.java

Path: LeetCode-main/solutions/1690. Stone Game VII/1690-2.java

```
class Solution {
    public int stoneGameVII(int[] stones) {
        final int n = stones.length;
        // dp[i][j] := the maximum score you can get more than your opponent in stones[i..j]
        int[][] dp = new int[n][n];
        int[] prefix = new int[n + 1];

        for (int i = 0; i < n; ++i)
            prefix[i + 1] = stones[i] + prefix[i];

        for (int d = 1; d < n; ++d)
            for (int i = 0; i + d < n; ++i) {
                final int j = i + d;
                dp[i][j] = Math.max(prefix[j + 1] - prefix[i + 1] - dp[i + 1][j],
                                     prefix[j] - prefix[i] - dp[i][j - 1]);
            }

        return dp[0][n - 1];
    }
}
```

704. 1690.java

Path: LeetCode-main/solutions/1690. Stone Game VII/1690.java

```
class Solution {
    public int stoneGameVII(int[] stones) {
        final int n = stones.length;
        Integer[][] mem = new Integer[n][n];
        int[] prefix = new int[n + 1];
        for (int i = 0; i < n; ++i)
            prefix[i + 1] = prefix[i] + stones[i];
        return stoneGameVII(stones, 0, n - 1, prefix, mem);
    }

    // Returns the maximum score you can get more than your opponent in
    // stones[i..j].
    private int stoneGameVII(int[] stones, int i, int j, int[] prefix, Integer[][] mem) {
        if (i == j)
            return 0;
        if (mem[i][j] != null)
            return mem[i][j];
        // Remove stones[i] to get sum(stones[i + 1..j]).
        final int removeLeft = prefix[j + 1] - prefix[i + 1] - //
            stoneGameVII(stones, i + 1, j, prefix, mem);

        // Remove stones[j] to get sum(stones[i..j - 1]).
        final int removeRight = prefix[j] - prefix[i] - //
            stoneGameVII(stones, i, j - 1, prefix, mem);
        return mem[i][j] = Math.max(removeLeft, removeRight);
    }
}
```

705. 1691.java

Path: LeetCode-main/solutions/1691. Maximum Height by Stacking Cuboids/1691.java

```
class Solution {
    public int maxHeight(int[][] cuboids) {
        // For each cuboid, sort it so that c[0] <= c[1] <= c[2].
        for (int[] cuboid : cuboids)
            Arrays.sort(cuboid);

        Arrays.sort(cuboids, new Comparator<int[]>() {
            @Override
            public int compare(int[] a, int[] b) {
                if (a[0] != b[0])
                    return a[0] - b[0];
                if (a[1] != b[1])
                    return a[1] - b[1];
                return a[2] - b[2];
            }
        });

        // dp[i] := the maximum height with cuboids[i] in the bottom
        int[] dp = new int[cuboids.length];

        for (int i = 0; i < cuboids.length; ++i)
            dp[i] = cuboids[i][2];

        for (int i = 1; i < cuboids.length; ++i)
            for (int j = 0; j < i; ++j)
                if (cuboids[j][0] <= cuboids[i][0] && //
                    cuboids[j][1] <= cuboids[i][1] && //
                    cuboids[j][2] <= cuboids[i][2])
                    dp[i] = Math.max(dp[i], dp[j] + cuboids[i][2]);

        return Arrays.stream(dp).max().getAsInt();
    }
}
```

706. 1692-2.java

Path: LeetCode-main/solutions/1692. Count Ways to Distribute Candies/1692-2.java

```
class Solution {
    public int waysToDistribute(int n, int k) {
        final int MOD = 1_000_000_007;
        long[][] dp = new long[k + 1][n + 1];

        for (int i = 0; i <= k; ++i)
            dp[i][i] = 1;

        for (int i = 1; i <= k; ++i)
            for (int j = i + 1; j <= n; ++j)
                dp[i][j] = (dp[i - 1][j - 1] + i * dp[i][j - 1]) % MOD;

        return (int) dp[k][n];
    }
}
```

707. 1692.java

Path: LeetCode-main/solutions/1692. Count Ways to Distribute Candies/1692.java

```
class Solution {
    public int waysToDistribute(int n, int k) {
        final int MOD = 1_000_000_007;
        // dp[i][j] := the number of ways to distribute i candies to j bags
        // dp[i][j] = 0, if i < j
        //           = 1, if i == j
        //           = dp[i - 1][j - 1] (the new candy occupies a bag)
        //           + dp[i - 1][j] * j (the new candy is in any of the j bags)
        long[][] dp = new long[n + 1][k + 1];

        for (int i = 0; i <= k; ++i)
            dp[i][i] = 1;

        for (int i = 1; i <= n; ++i)
            for (int j = 1; j <= k; ++j)
                dp[i][j] = (dp[i - 1][j - 1] + dp[i - 1][j] * j) % MOD;

        return (int) dp[n][k];
    }
}
```

708. 1694.java

Path: LeetCode-main/solutions/1694. Reformat Phone Number/1694.java

```
class Solution {
    public String reformatNumber(String number) {
        StringBuilder sb = new StringBuilder();

        number = number.replaceAll("[-\s]", "");

        int i = 0; // number's index
        for (i = 0; i + 4 < number.length(); i += 3)
            sb.append(number.substring(i, i + 3) + '-');

        final int countFinalDigits = number.length() - i;
        if (countFinalDigits < 4)
            sb.append(number.substring(i));
        else // countFinalDigits == 4
            sb.append(number.substring(i, i + 2) + '-' + number.substring(i + 2));

        return sb.toString();
    }
}
```

709. 1695.java

Path: LeetCode-main/solutions/1695. Maximum Erasure Value/1695.java

```
class Solution {
    public int maximumUniqueSubarray(int[] nums) {
        int ans = 0;
        int score = 0;
        Set<Integer> seen = new HashSet<>();

        for (int l = 0, r = 0; r < nums.length; ++r) {
            while (!seen.add(nums[r])) {
                score -= nums[l];
                seen.remove(nums[l++]);
            }
            score += nums[r];
            ans = Math.max(ans, score);
        }

        return ans;
    }
}
```


710. 1696.java

Path: LeetCode-main/solutions/1696. Jump Game VI/1696.java

```
class Solution {
    public int maxResult(int[] nums, int k) {
        // Stores dp[i] within the bounds.
        Deque<Integer> maxQ = new ArrayDeque<>(List.of(0));
        // dp[i] := the maximum score to consider nums[0..i]
        int[] dp = new int[nums.length];
        dp[0] = nums[0];

        for (int i = 1; i < nums.length; ++i) {
            // Pop the index if it's out-of-bounds.
            if (maxQ.peekFirst() + k < i)
                maxQ.pollFirst();
            dp[i] = dp[maxQ.peekFirst()] + nums[i];
            // Pop indices that won't be chosen in the future.
            while (!maxQ.isEmpty() && dp[maxQ.peekLast()] <= dp[i])
                maxQ.pollLast();
            maxQ.offerLast(i);
        }

        return dp[nums.length - 1];
    }
}
```

711. 1697.java

Path: LeetCode-main/solutions/1697. Checking Existence of Edge Length Limited Paths/1697.java

```
class UnionFind {
    public UnionFind(int n) {
        id = new int[n];
        rank = new int[n];
        for (int i = 0; i < n; ++i)
            id[i] = i;
    }

    public void unionByRank(int u, int v) {
        final int i = find(u);
        final int j = find(v);
        if (i == j)
            return;
        if (rank[i] < rank[j]) {
            id[i] = j;
        } else if (rank[i] > rank[j]) {
            id[j] = i;
        } else {
            id[i] = j;
            ++rank[j];
        }
    }

    public int find(int u) {
        return id[u] == u ? u : (id[u] = find(id[u]));
    }

    private int[] id;
    private int[] rank;
}

class Solution {
    public boolean[] distanceLimitedPathsExist(int n, int[][] edgeList, int[][] queries) {
        boolean[] ans = new boolean[queries.length];
        int[][] qs = new int[queries.length][4];
        UnionFind uf = new UnionFind(n);

        for (int i = 0; i < queries.length; ++i) {
            qs[i][0] = queries[i][0];
            qs[i][1] = queries[i][1];
            qs[i][2] = queries[i][2];
            qs[i][3] = i;
        }

        Arrays.sort(qs, Comparator.comparingInt(q -> q[2]));
        Arrays.sort(edgeList, Comparator.comparingInt(edge -> edge[2]));

        int i = 0; // i := edgeList's index
        for (int[] q : qs) {
            // Union edges whose distances < limit (q[2]).
            while (i < edgeList.length && edgeList[i][2] < q[2])
                uf.unionByRank(edgeList[i][0], edgeList[i++][1]);
            if (uf.find(q[0]) == uf.find(q[1]))
                ans[q[3]] = true;
        }

        return ans;
    }
}
```

712. 1698.java

Path: LeetCode-main/solutions/1698. Number of Distinct Substrings in a String/1698.java

```
class Solution {
    public int countDistinct(String s) {
        final int n = s.length();
        int ans = 0;
        long[] pow = new long[n + 1];    // pow[i] := BASE^i
        long[] hashes = new long[n + 1]; // hashes[i] := the hash of s[0..i)

        pow[0] = 1;
        for (int i = 1; i <= n; ++i) {
            pow[i] = pow[i - 1] * BASE % HASH;
            hashes[i] = (hashes[i - 1] * BASE + val(s.charAt(i - 1))) % HASH;
        }

        for (int length = 1; length <= n; ++length) {
            Set<Long> seen = new HashSet<>();
            for (int i = 0; i + length <= n; ++i)
                seen.add(getHash(i, i + length, hashes, pow));
            ans += seen.size();
        }

        return ans;
    }

    private static final int BASE = 26;
    private static final int HASH = 1_000_000_007;

    private static int val(char c) {
        return c - 'a';
    }

    // Returns the hash of s[l..r).
    private long getHash(int l, int r, long[] hashes, long[] pow) {
        final long hash = (hashes[r] - hashes[l] * pow[r - l]) % HASH;
        return hash < 0 ? hash + HASH : hash;
    }
}
```

713. 17-2.java

Path: LeetCode-main/solutions/17. Letter Combinations of a Phone Number/17-2.java

```
class Solution {
    public List<String> letterCombinations(String digits) {
        if (digits.isEmpty())
            return new ArrayList<>();

        List<String> ans = new ArrayList<>();
        ans.add("");
        final String[] digitToLetters = {"", "", "abc", "def", "ghi",
                                          "jkl", "mno", "pqrs", "tuv", "wxyz"};

        for (final char d : digits.toCharArray()) {
            List<String> temp = new ArrayList<>();
            for (final String s : ans)
                for (final char c : digitToLetters[d - '0'].toCharArray())
                    temp.add(s + c);
            ans = temp;
        }
        return ans;
    }
}
```

714. 17.java

Path: LeetCode-main/solutions/17. Letter Combinations of a Phone Number/17.java

```
class Solution {
    public List<String> letterCombinations(String digits) {
        if (digits.isEmpty())
            return new ArrayList<>();

        List<String> ans = new ArrayList<>();

        dfs(digits, 0, new StringBuilder(), ans);
        return ans;
    }

    private static final String[] digitToLetters = {"", "", "abc", "def", "ghi",
        "jkl", "mno", "pqrs", "tuv", "wxyz"};

    private void dfs(String digits, int i, StringBuilder sb, List<String> ans) {
        if (i == digits.length()) {
            ans.add(sb.toString());
            return;
        }

        for (final char c : digitToLetters[digits.charAt(i) - '0'].toCharArray()) {
            sb.append(c);
            dfs(digits, i + 1, sb, ans);
            sb.deleteCharAt(sb.length() - 1);
        }
    }
}
```

715. 170.java

Path: LeetCode-main/solutions/170. Two Sum III - Data structure design/170.java

```
class TwoSum {
    public void add(int number) {
        count.merge(number, 1, Integer::sum);
    }

    public boolean find(int value) {
        for (Map.Entry<Integer, Integer> entry : count.entrySet()) {
            final int key = entry.getKey();
            final int remain = value - key;
            if (key == remain && entry.getValue() > 1)
                return true;
            if (key != remain && count.containsKey(remain))
                return true;
        }
        return false;
    }

    private HashMap<Integer, Integer> count = new HashMap<>();
}
```

716. 1700.java

Path: LeetCode-main/solutions/1700. Number of Students Unable to Eat Lunch/1700.java

```
class Solution {
    public int countStudents(int[] students, int[] sandwiches) {
        int[] count = new int[2];

        for (final int student : students)
            ++count[student];

        for (int i = 0; i < sandwiches.length; ++i) {
            if (count[sandwiches[i]] == 0)
                return sandwiches.length - i;
            --count[sandwiches[i]];
        }

        return 0;
    }
}
```

717. 1701.java

Path: LeetCode-main/solutions/1701. Average Waiting Time/1701.java

```
class Solution {
    public double averageWaitingTime(int[][] customers) {
        double wait = 0;
        double curr = 0;

        for (int[] c : customers) {
            curr = Math.max(curr, 1.0 * c[0]) + c[1];
            wait += curr - c[0];
        }

        return 1.0 * wait / customers.length;
    }
}
```


718. 1702.java

Path: LeetCode-main/solutions/1702. Maximum Binary String After Change/1702.java

```
class Solution {
    public String maximumBinaryString(String binary) {
        // e.g. binary = "100110"
        // Do Operation 2 -> "100011"
        // Do Operation 1 -> "111011"
        // So, the index of the only '0' is prefixOnes + zeros - 1.
        final int zeros = (int) binary.chars().filter(c -> c == '0').count();
        final int prefixOnes = binary.indexOf('0');

        // Make the entire String as 1s.
        StringBuilder sb = new StringBuilder("1".repeat(binary.length()));

        // Make the only '0' if necessary.
        if (prefixOnes != -1)
            sb.setCharAt(prefixOnes + zeros - 1, '0');
        return sb.toString();
    }
}
```

719. 1703.java

Path: LeetCode-main/solutions/1703. Minimum Adjacent Swaps for K Consecutive Ones/1703.java

```
class Solution {
    public int minMoves(int[] nums, int k) {
        List<Integer> ones = new ArrayList<>();

        for (int i = 0; i < nums.length; ++i)
            if (nums[i] == 1)
                ones.add(i);

        final int median = ones.get(getMedIndex(0, k));
        int moves = 0;
        for (int i = 0; i < k; ++i)
            moves += Math.abs(ones.get(i) - median);

        int ans = moves;

        for (int i = 1; i <= ones.size() - k; ++i) {
            final int oldMedianIndex = ones.get(getMedIndex(i - 1, k));
            final int newMedianIndex = ones.get(getMedIndex(i, k));
            if (k % 2 == 1)
                moves += newMedianIndex - oldMedianIndex;
            moves -= newMedianIndex - ones.get(i - 1);
            moves += ones.get(i + k - 1) - newMedianIndex;
            ans = Math.min(ans, moves);
        }

        return ans - nThSum((k - 1) / 2) - nThSum(k / 2);
    }

    // Returns the median index of [i..i + k).
    private int getMedIndex(int i, int k) {
        return (i + (i + k - 1)) / 2;
    }

    // Returns 1 + 2 + ... + n
    private int nThSum(int n) {
        return n * (n + 1) / 2;
    }
}
```

720. 1704.java

Path: LeetCode-main/solutions/1704. Determine if String Halves Are Alike/1704.java

```
class Solution {
    public boolean halvesAreAlike(String s) {
        final String a = s.substring(0, s.length() / 2);
        final String b = s.substring(s.length() / 2);
        final int aVowelsCount = (int) a.chars().filter(c -> isVowel((char) c)).count();
        final int bVowelsCount = (int) b.chars().filter(c -> isVowel((char) c)).count();
        return aVowelsCount == bVowelsCount;
    }

    private boolean isVowel(char c) {
        return "aeiouAEIOU".indexOf(c) != -1;
    }
}
```

721. 1705.java

Path: LeetCode-main/solutions/1705. Maximum Number of Eaten Apples/1705.java

```
class Solution {
    public int eatenApples(int[] apples, int[] days) {
        final int n = apples.length;
        int ans = 0;
        // (the rotten day, the number of apples)
        Queue<Pair<Integer, Integer>> minHeap =
            new PriorityQueue<>(Comparator.comparingInt(Pair::getKey));

        for (int i = 0; i < n || !minHeap.isEmpty(); ++i) {
            // Remove the rotten apples.
            while (!minHeap.isEmpty() && minHeap.peek().getKey() <= i)
                minHeap.poll();
            // Add today's apples.
            if (i < n && apples[i] > 0)
                minHeap.offer(new Pair<>(i + days[i], apples[i]));
            // Eat one apple today.
            if (!minHeap.isEmpty()) {
                final int rottenDay = minHeap.peek().getKey();
                final int numApples = minHeap.poll().getValue();
                if (numApples > 1)
                    minHeap.offer(new Pair<>(rottenDay, numApples - 1));
                ++ans;
            }
        }

        return ans;
    }
}
```

722. 1706.java

Path: LeetCode-main/solutions/1706. Where Will the Ball Fall/1706.java

```
class Solution {
    public int[] findBall(int[][] grid) {
        final int m = grid.length;
        final int n = grid[0].length;
        // dp[i] := status of the i-th column
        // -1 := empty, 0 := b0, 1 := b1, ...
        int[] dp = new int[n];
        // ans[i] := the i-th ball's final position
        int[] ans = new int[n];
        Arrays.fill(ans, -1);

        for (int i = 0; i < n; ++i)
            dp[i] = i;

        for (int i = 0; i < m; ++i) {
            int[] newDp = new int[n];
            Arrays.fill(newDp, -1);
            for (int j = 0; j < n; ++j) {
                // out-of-bounds
                if (j + grid[i][j] < 0 || j + grid[i][j] == n)
                    continue;
                // Stuck
                if (grid[i][j] == 1 && grid[i][j + 1] == -1 || grid[i][j] == -1 && grid[i][j - 1] == 1)
                    continue;
                newDp[j + grid[i][j]] = dp[j];
            }
            dp = newDp;
        }

        for (int i = 0; i < n; ++i)
            if (dp[i] != -1)
                ans[dp[i]] = i;

        return ans;
    }
}
```

723. 1707.java

Path: LeetCode-main/solutions/1707. Maximum XOR With an Element From Array/1707.java

```
class TrieNode {
    public TrieNode[] children = new TrieNode[2];
}

class BitTrie {
    public BitTrie(int maxBit) {
        this.maxBit = maxBit;
    }

    public void insert(int num) {
        TrieNode node = root;
        for (int i = maxBit; i >= 0; --i) {
            final int bit = (int) (num >> i & 1);
            if (node.children[bit] == null)
                node.children[bit] = new TrieNode();
            node = node.children[bit];
        }
    }

    public int getMaxXor(int num) {
        int maxXor = 0;
        TrieNode node = root;
        for (int i = maxBit; i >= 0; --i) {
            final int bit = (int) (num >> i & 1);
            final int toggleBit = bit ^ 1;
            if (node.children[toggleBit] != null) {
                maxXor = maxXor | 1 << i;
                node = node.children[toggleBit];
            } else if (node.children[bit] != null) {
                node = node.children[bit];
            } else { // There's nothing in the Bit Trie.
                return 0;
            }
        }
        return maxXor;
    }

    private int maxBit;
    private TrieNode root = new TrieNode();
}

class Solution {
    public int[] maximizeXor(int[] nums, int[][] queries) {
        int[] ans = new int[queries.length];
        Arrays.fill(ans, -1);
        final int maxNumInNums = Arrays.stream(nums).max().getAsInt();
        final int maxNumInQuery = Arrays.stream(queries).mapToInt(query -> query[0]).max().getAsInt();
        final int maxBit = (int) (Math.log(Math.max(maxNumInNums, maxNumInQuery)) / Math.log(2));
        BitTrie bitTrie = new BitTrie(maxBit);

        Arrays.sort(nums);

        int i = 0; // nums' index
        for (IndexedQuery indexedQuery : getIndexedQueries(queries)) {
            final int queryIndex = indexedQuery.queryIndex;
            final int x = indexedQuery.x;
            final int m = indexedQuery.m;
            while (i < nums.length && nums[i] <= m)
                bitTrie.insert(nums[i++]);
            if (i > 0 && nums[i - 1] <= m)
                ans[queryIndex] = bitTrie.getMaxXor(x);
        }

        return ans;
    }

    private record IndexedQuery(int queryIndex, int x, int m){};

    private IndexedQuery[] getIndexedQueries(int[][] queries) {
        IndexedQuery[] indexedQueries = new IndexedQuery[queries.length];
        for (int i = 0; i < queries.length; ++i)
            indexedQueries[i] = new IndexedQuery(i, queries[i][0], queries[i][1]);
        Arrays.sort(indexedQueries, Comparator.comparingInt(IndexedQuery::m));
        return indexedQueries;
    }
}
```

724. 1708.java

Path: LeetCode-main/solutions/1708. Largest Subarray Length K/1708.java

```
class Solution {  
    public int[] largestSubarray(int[] nums, int k) {  
        int maxIndex = 0;  
        for (int i = 1; i <= nums.length - k; ++i)  
            if (nums[i] > nums[maxIndex])  
                maxIndex = i;  
        return Arrays.copyOfRange(nums, maxIndex, maxIndex + k);  
    }  
}
```

725. 171.java

Path: LeetCode-main/solutions/171. Excel Sheet Column Number/171.java

```
class Solution {  
    public int titleToNumber(String columnTitle) {  
        return columnTitle.chars().reduce(0, (subtotal, c) -> subtotal * 26 + c - '@');  
    }  
}
```


726. 1710.java

Path: LeetCode-main/solutions/1710. Maximum Units on a Truck/1710.java

```
class Solution {
    public int maximumUnits(int[][] boxTypes, int truckSize) {
        int ans = 0;

        Arrays.sort(boxTypes, Comparator.comparingInt(boxType -> - boxType[1]));

        for (int[] boxType : boxTypes) {
            final int boxes = boxType[0];
            final int units = boxType[1];
            if (boxes >= truckSize)
                return ans + truckSize * units;
            ans += boxes * units;
            truckSize -= boxes;
        }

        return ans;
    }
}
```

727. 1711.java

Path: LeetCode-main/solutions/1711. Count Good Meals/1711.java

```
class Solution {
    public int countPairs(int[] deliciousness) {
        final int MOD = 1_000_000_007;
        final int MAX_BIT = 20 + 1;
        final int MAX_POWER = (int) Math.pow(2, MAX_BIT);
        int ans = 0;
        Map<Integer, Integer> count = new HashMap<>();

        for (final int d : deliciousness) {
            for (int power = 1; power <= MAX_POWER; power *= 2) {
                ans += count.getDefault(power - d, 0);
                ans %= MOD;
            }
            count.merge(d, 1, Integer::sum);
        }

        return ans;
    }
}
```

728. 1712-2.java

Path: LeetCode-main/solutions/1712. Ways to Split Array Into Three Subarrays/1712-2.java

```
class Solution {
    public int waysToSplit(int[] nums) {
        final int MOD = 1_000_000_007;
        final int n = nums.length;
        int ans = 0;
        int[] prefix = nums.clone();

        for (int i = 1; i < n; ++i)
            prefix[i] += prefix[i - 1];

        for (int i = 0, j = 0, k = 0; i < n - 2; ++i) {
            // Find the first index j s.t.
            // left = prefix[i] <= mid = prefix[j] - prefix[i]
            j = Math.max(j, i + 1);
            while (j < n - 1 && prefix[i] > prefix[j] - prefix[i])
                ++j;
            // Find the first index k s.t.
            // mid = prefix[k] - prefix[i] > right = prefix[-1] - prefix[k]
            k = Math.max(k, j);
            while (k < n - 1 && prefix[k] - prefix[i] <= prefix[prefix.length - 1] - prefix[k])
                ++k;
            ans += k - j;
            ans %= MOD;
        }

        return ans;
    }
}
```

729. 1712.java

Path: LeetCode-main/solutions/1712. Ways to Split Array Into Three Subarrays/1712.java

```
class Solution {
    public int waysToSplit(int[] nums) {
        final int MOD = 1_000_000_007;
        final int n = nums.length;
        int ans = 0;
        int[] prefix = nums.clone();

        for (int i = 1; i < n; ++i)
            prefix[i] += prefix[i - 1];

        for (int i = 0; i < n - 2; ++i) {
            final int j = firstGreaterEqual(prefix, i);
            if (j == n - 1)
                break;
            final int mid = prefix[j] - prefix[i];
            final int right = prefix[prefix.length - 1] - prefix[j];
            if (mid > right)
                continue;
            final int k = firstGreater(prefix, i);
            ans = (ans + k - j) % MOD;
        }

        return ans;
    }

    // Finds the first index j s.t.
    // mid = prefix[j] - prefix[i] >= left = prefix[i].
    private int firstGreaterEqual(int[] prefix, int i) {
        int l = i + 1;
        int r = prefix.length - 1;
        while (l < r) {
            final int m = (l + r) / 2;
            if (prefix[m] - prefix[i] >= prefix[i])
                r = m;
            else
                l = m + 1;
        }
        return l;
    };

    // Finds the first index k s.t.
    // mid = prefix[k] - prefix[i] > right = prefix[-1] - prefix[k].
    private int firstGreater(int[] prefix, int i) {
        int l = i + 1;
        int r = prefix.length - 1;
        while (l < r) {
            final int m = (l + r) / 2;
            if (prefix[m] - prefix[i] > prefix[prefix.length - 1] - prefix[m])
                r = m;
            else
                l = m + 1;
        }
        return l;
    };
};
```

730. 1713.java

Path: LeetCode-main/solutions/1713. Minimum Operations to Make a Subsequence/1713.java

```
class Solution {
    public int minOperations(int[] target, int[] arr) {
        List<Integer> indices = new ArrayList<>();
        Map<Integer, Integer> numToIndex = new HashMap<>();

        for (int i = 0; i < target.length; ++i)
            numToIndex.put(target[i], i);

        for (final int a : arr)
            if (numToIndex.containsKey(a))
                indices.add(numToIndex.get(a));

        return target.length - lengthOfLIS(indices);
    }

    // Same as 300. Longest Increasing Subsequence
    private int lengthOfLIS(List<Integer> nums) {
        // tails[i] := the minimum tail of all the increasing subsequences with
        // length i + 1
        List<Integer> tails = new ArrayList<>();
        for (final int num : nums)
            if (tails.isEmpty() || num > tails.get(tails.size() - 1))
                tails.add(num);
            else
                tails.set(firstGreaterEqual(tails, num), num);
        return tails.size();
    }

    private int firstGreaterEqual(List<Integer> arr, int target) {
        final int i = Collections.binarySearch(arr, target);
        return i < 0 ? -i - 1 : i;
    }
}
```

731. 1714.java

Path: LeetCode-main/solutions/1714. Sum Of Special Evenly-Spaced Elements In Array/1714.java

```
class Solution {
    public int[] solve(int[] nums, int[][] queries) {
        final int MOD = 1_000_000_007;
        final int n = nums.length;
        final int sqrtN = (int) Math.sqrt(n);
        int[] ans = new int[queries.length];
        // prefix[x][y] = sum(nums[x + ay]), where a >= 0 and x + ay < n
        int[][] prefix = new int[n][sqrtN];

        // Set prefix[i][j] to nums[i] to indicate the sequence starts with nums[i].
        for (int i = 0; i < n; ++i)
            for (int j = 0; j < sqrtN; ++j)
                prefix[i][j] = nums[i];

        for (int x = n - 1; x >= 0; --x)
            for (int y = 1; y < sqrtN; ++y)
                if (x + y < n) {
                    prefix[x][y] += prefix[x + y][y];
                    prefix[x][y] %= MOD;
                }

        for (int i = 0; i < queries.length; ++i) {
            final int x = queries[i][0];
            final int y = queries[i][1];
            if (y < sqrtN) {
                ans[i] = prefix[x][y];
            } else {
                int sum = 0;
                for (int j = x; j < n; j += y)
                    sum = (sum + nums[j]) % MOD;
                ans[i] = sum;
            }
        }

        return ans;
    }
}
```

732. 1716.java

Path: LeetCode-main/solutions/1716. Calculate Money in Leetcode Bank/1716.java

```
class Solution {
    public int totalMoney(int n) {
        final int weeks = n / 7;
        final int firstWeek = trapezoid(1, 7);
        final int lastFullWeek = trapezoid(1 + weeks - 1, 7 + weeks - 1);
        final int remainingDays = trapezoid(1 + weeks, n % 7 + weeks);
        return (firstWeek + lastFullWeek) * weeks / 2 + remainingDays;
    }

    // Returns sum(a..b).
    private int trapezoid(int a, int b) {
        return (a + b) * (b - a + 1) / 2;
    }
}
```

733. 1717.java

Path: LeetCode-main/solutions/1717. Maximum Score From Removing Substrings/1717.java

```
class Solution {
    public int maximumGain(String s, int x, int y) {
        // The assumption that gain("ab") > gain("ba") while removing "ba" first is
        // optimal is contradicted. Only "b(ab)a" satisfies the condition of
        // preventing two "ba" removals, but after removing "ab", we can still
        // remove one "ba", resulting in a higher gain. Thus, removing "ba" first is
        // not optimal.
        return x > y ? gain(s, "ab", x, "ba", y) : gain(s, "ba", y, "ab", x);
    }

    // Returns the points gained by first removing sub1 ("ab" | "ba") from s with
    // point1, then removing sub2 ("ab" | "ba") from s with point2.
    private int gain(final String s, final String sub1, int point1, final String sub2, int point2) {
        int points = 0;
        Stack<Character> stack1 = new Stack<>();
        Stack<Character> stack2 = new Stack<>();

        // Remove "sub1" from s with point1 gain.
        for (final char c : s.toCharArray())
            if (!stack1.isEmpty() && stack1.peek() == sub1.charAt(0) && c == sub1.charAt(1)) {
                stack1.pop();
                points += point1;
            } else {
                stack1.push(c);
            }

        // Remove "sub2" from s with point2 gain.
        for (final char c : stack1)
            if (!stack2.isEmpty() && stack2.peek() == sub2.charAt(0) && c == sub2.charAt(1)) {
                stack2.pop();
                points += point2;
            } else {
                stack2.push(c);
            }

        return points;
    }
}
```


734. 1718.java

Path: LeetCode-main/solutions/1718. Construct the Lexicographically Largest Valid Sequence/1718.java

```
class Solution {
    public int[] constructDistancedSequence(int n) {
        int[] ans = new int[2 * n - 1];
        dfs(n, 0, 0, ans);
        return ans;
    }

    private boolean dfs(int n, int i, int mask, int[] ans) {
        if (i == ans.length)
            return true;
        if (ans[i] > 0)
            return dfs(n, i + 1, mask, ans);

        // Greedily fill in `ans` in descending order.
        for (int num = n; num >= 1; --num) {
            if ((mask >> num & 1) == 1)
                continue;
            if (num == 1) {
                ans[i] = num;
                if (dfs(n, i + 1, mask | 1 << num, ans))
                    return true;
                ans[i] = 0;
            } else { // num in [2, n]
                if (i + num >= ans.length || ans[i + num] > 0)
                    continue;
                ans[i] = num;
                ans[i + num] = num;
                if (dfs(n, i + 1, mask | 1 << num, ans))
                    return true;
                ans[i + num] = 0;
                ans[i] = 0;
            }
        }
    }

    return false;
}
```

735. 1719.java

Path: LeetCode-main/solutions/1719. Number Of Ways To Reconstruct A Tree/1719.java

```
class Solution {
    public int checkWays(int[][] pairs) {
        final int MAX = 501;
        Map<Integer, List<Integer>> graph = new HashMap<>();
        int[] degrees = new int[MAX];
        boolean[][] connected = new boolean[MAX][MAX];

        for (int[] pair : pairs) {
            final int u = pair[0];
            final int v = pair[1];
            graph.putIfAbsent(u, new ArrayList<>());
            graph.putIfAbsent(v, new ArrayList<>());
            graph.get(u).add(v);
            graph.get(v).add(u);
            ++degrees[u];
            ++degrees[v];
            connected[u][v] = true;
            connected[v][u] = true;
        }

        // For each node, sort its children by degrees in descending order.
        for (final int u : graph.keySet())
            graph.get(u).sort(Comparator.comparingInt(a -> - degrees[a]));

        final int root = getRoot(degrees, graph.keySet().size());
        if (root == -1)
            return 0;
        if (!dfs(graph, root, degrees, connected, new ArrayList<>(), new boolean[MAX]))
            return 0;
        return hasMoreThanOneWay ? 2 : 1;
    }

    private boolean hasMoreThanOneWay = false;

    // Returns the root by finding the node with a degree that equals to n - 1.
    private int getRoot(int[] degrees, int n) {
        for (int i = 1; i < degrees.length; ++i)
            if (degrees[i] == n - 1)
                return i;
        return -1;
    }

    // Returns true if each node rooted at u is connected to all of its ancestors.
    private boolean dfs(Map<Integer, List<Integer>> graph, int u, int[] degrees,
        boolean[][] connected, List<Integer> ancestors, boolean[] seen) {
        seen[u] = true;

        for (final int ancestor : ancestors)
            if (!connected[u][ancestor])
                return false;

        ancestors.add(u);

        for (final int v : graph.get(u)) {
            if (seen[v])
                continue;
            // We can swap u with v, so there are more than one way.
            if (degrees[v] == degrees[u])
                hasMoreThanOneWay = true;
            if (!dfs(graph, v, degrees, connected, ancestors, seen))
                return false;
        }

        ancestors.remove(ancestors.size() - 1);
        return true;
    }
}
```

736. 172.java

Path: LeetCode-main/solutions/172. Factorial Trailing Zeroes/172.java

```
class Solution {  
    public int trailingZeroes(int n) {  
        return n == 0 ? 0 : n / 5 + trailingZeroes(n / 5);  
    }  
}
```

737. 1720.java

Path: LeetCode-main/solutions/1720. Decode XORed Array/1720.java

```
class Solution {
    public int[] decode(int[] encoded, int first) {
        int[] ans = new int[encoded.length + 1];
        ans[0] = first;

        for (int i = 0; i < encoded.length; ++i)
            ans[i + 1] = ans[i] ^ encoded[i];

        return ans;
    }
}
```

738. 1721.java

Path: LeetCode-main/solutions/1721. Swapping Nodes in a Linked List/1721.java

```
class Solution {
    public ListNode swapNodes(ListNode head, int k) {
        ListNode p = null; // Points the k-th node from the beginning.
        ListNode q = null; // Points the k-th node from the end.

        for (ListNode curr = head; curr != null; curr = curr.next) {
            if (q != null)
                q = q.next;
            if (--k == 0) {
                p = curr;
                q = head;
            }
        }

        final int temp = p.val;
        p.val = q.val;
        q.val = temp;
        return head;
    }
}
```

739. 1722.java

Path: LeetCode-main/solutions/1722. Minimize Hamming Distance After Swap Operations/1722.java

```
class UnionFind {
    public UnionFind(int n) {
        id = new int[n];
        rank = new int[n];
        for (int i = 0; i < n; ++i)
            id[i] = i;
    }

    public void unionByRank(int u, int v) {
        final int i = find(u);
        final int j = find(v);
        if (i == j)
            return;
        if (rank[i] < rank[j]) {
            id[i] = j;
        } else if (rank[i] > rank[j]) {
            id[j] = i;
        } else {
            id[i] = j;
            ++rank[j];
        }
    }

    public int find(int u) {
        return id[u] == u ? u : (id[u] = find(id[u]));
    }

    private int[] id;
    private int[] rank;
}

class Solution {
    public int minimumHammingDistance(int[] source, int[] target, int[][] allowedSwaps) {
        final int n = source.length;
        int ans = 0;
        UnionFind uf = new UnionFind(n);
        Map<Integer, Integer>[] groupIdToCount = new Map[n];

        for (int i = 0; i < n; ++i)
            groupIdToCount[i] = new HashMap<>();

        for (int[] allowedSwap : allowedSwaps) {
            final int a = allowedSwap[0];
            final int b = allowedSwap[1];
            uf.unionByRank(a, b);
        }

        for (int i = 0; i < n; ++i)
            groupIdToCount[uf.find(i)].merge(source[i], 1, Integer::sum);

        for (int i = 0; i < n; ++i) {
            final int groupId = uf.find(i);
            Map<Integer, Integer> count = groupIdToCount[groupId];
            if (!count.containsKey(target[i]))
                ++ans;
            else if (count.merge(target[i], -1, Integer::sum) == 0)
                count.remove(target[i]);
        }

        return ans;
    }
}
```

740. 1723.java

Path: LeetCode-main/solutions/1723. Find Minimum Time to Finish All Jobs/1723.java

```
class Solution {
    public int minimumTimeRequired(int[] jobs, int k) {
        ans = Arrays.stream(jobs).sum();
        int[] times = new int[k];

        Arrays.sort(jobs);
        Collections.reverse(Arrays.asList(jobs));
        dfs(jobs, 0, times);
        return ans;
    }

    private int ans = 0;

    private void dfs(int[] jobs, int s, int[] times) {
        if (s == jobs.length) {
            ans = Math.min(ans, Arrays.stream(times).max().getAsInt());
            return;
        }
        for (int i = 0; i < times.length; ++i) {
            if (times[i] + jobs[s] >= ans)
                continue;
            times[i] += jobs[s];
            dfs(jobs, s + 1, times);
            times[i] -= jobs[s];
            if (times[i] == 0)
                return;
        }
    }
}
```

741. 1724.java

Path: LeetCode-main/solutions/1724. Checking Existence of Edge Length Limited Paths II/1724.java

```
class UnionFind {
    public UnionFind(int n) {
        id = new TreeMap[n];
        for (int i = 0; i < n; ++i) {
            id[i] = new TreeMap<>();
            id[i].put(0, i);
        }
    }

    public void union(int u, int v, int limit) {
        final int i = find(u, limit);
        final int j = find(v, limit);
        if (i == j)
            return;
        id[i].put(limit, j);
    }

    public int find(int u, int limit) {
        // the maximum key of id[u] <= limit
        final int floorKey = id[u].floorKey(limit);
        final int i = id[u].get(floorKey);
        if (i == u)
            return u;
        // Recursively find i's id.
        final int j = find(i, limit);
        id[u].put(limit, j);
        return j;
    }

    // id[i]'s (key, value) := (limit, id of node i <= limit)
    private TreeMap<Integer, Integer>[] id;
}

class DistanceLimitedPathsExist {
    public DistanceLimitedPathsExist(int n, int[][] edgeList) {
        uf = new UnionFind(n);

        Arrays.sort(edgeList, Comparator.comparingInt(edge -> edge[2]));

        for (int[] edge : edgeList) {
            final int u = edge[0];
            final int v = edge[1];
            final int d = edge[2];
            uf.union(u, v, d);
        }

        public boolean query(int p, int q, int limit) {
            return uf.find(p, limit - 1) == uf.find(q, limit - 1);
        }

        private UnionFind uf;
    }
}
```


742. 1725.java

Path: LeetCode-main/solutions/1725. Number Of Rectangles That Can Form The Largest Square/1725.java

```
class Solution {
    public int countGoodRectangles(int[][] rectangles) {
        int[] minSides = new int[rectangles.length];

        for (int i = 0; i < rectangles.length; ++i) {
            final int x = rectangles[i][0];
            final int y = rectangles[i][1];
            minSides[i] = Math.min(x, y);
        }

        final int maxLen = Arrays.stream(minSides).max().getAsInt();
        return (int) Arrays.stream(minSides).filter(minSide -> minSide == maxLen).count();
    }
}
```

743. 1726.java

Path: LeetCode-main/solutions/1726. Tuple with Same Product/1726.java

```
class Solution {
    public int tupleSameProduct(int[] nums) {
        int ans = 0;
        Map<Integer, Integer> count = new HashMap<>();

        for (int i = 0; i < nums.length; ++i)
            for (int j = 0; j < i; ++j) {
                final int prod = nums[i] * nums[j];
                ans += count.getOrDefault(prod, 0) * 8;
                count.merge(prod, 1, Integer::sum);
            }

        return ans;
    }
}
```

744. 1727.java

Path: LeetCode-main/solutions/1727. Largest Submatrix With Rearrangements/1727.java

```
class Solution {
    public int largestSubmatrix(int[][] matrix) {
        final int n = matrix[0].length;
        int ans = 0;
        int[] hist = new int[n];

        for (int[] row : matrix) {
            // Accumulate the histogram if possible.
            for (int i = 0; i < n; ++i)
                hist[i] = row[i] == 0 ? 0 : hist[i] + 1;

            // Get the sorted histogram.
            int[] sortedHist = hist.clone();
            Arrays.sort(sortedHist);

            // Greedily calculate the answer.
            for (int i = 0; i < n; ++i)
                ans = Math.max(ans, sortedHist[i] * (n - i));
        }

        return ans;
    }
}
```

745. 1728.java

Path: LeetCode-main/solutions/1728. Cat and Mouse II/1728.java

```
class Solution {
    public boolean canMouseWin(String[] grid, int catJump, int mouseJump) {
        final int m = grid.length;
        final int n = grid[0].length();
        int nFloors = 0;
        int cat = 0; // cat's position
        int mouse = 0; // mouse's position

        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j) {
                if (grid[i].charAt(j) != '#')
                    ++nFloors;
                if (grid[i].charAt(j) == 'C')
                    cat = hash(i, j, n);
                else if (grid[i].charAt(j) == 'M')
                    mouse = hash(i, j, n);
            }

        Boolean[][][] mem = new Boolean[m * n][m * n][nFloors * 2];
        return canMouseWin(grid, cat, mouse, 0, catJump, mouseJump, m, n, nFloors, mem);
    }

    private static final int[][] DIRS = {{0, 1}, {1, 0}, {0, -1}, {-1, 0}};

    // Returns true if the mouse can win, where the cat is on (i / 8, i % 8), the
    // mouse is on (j / 8, j % 8), and the turns is k.
    private boolean canMouseWin(String[] grid, int cat, int mouse, int turn, int catJump,
                                int mouseJump, int m, int n, int nFloors, Boolean[][][] mem) {
        // We already search the whole touchable grid.
        if (turn == nFloors * 2)
            return false;
        if (mem[cat][mouse][turn] != null)
            return mem[cat][mouse][turn];

        if (turn % 2 == 0) {
            // the mouse's turn
            int i = mouse / n;
            int j = mouse % n;
            for (int[] dir : DIRS) {
                for (int jump = 0; jump <= mouseJump; ++jump) {
                    int x = i + dir[0] * jump;
                    int y = j + dir[1] * jump;
                    if (x < 0 || x == m || y < 0 || y == n)
                        break;
                    if (grid[x].charAt(y) == '#')
                        break;
                    // The mouse eats the food, so the mouse wins.
                    if (grid[x].charAt(y) == 'F')
                        return mem[cat][mouse][turn] = true;
                    if (canMouseWin(grid, cat, hash(x, y, n), turn + 1, catJump, mouseJump, m, n, nFloors,
                                    mem))
                        return mem[cat][mouse][turn] = true;
                }
            }
            // The mouse can't win, so the mouse loses.
            return mem[cat][mouse][turn] = false;
        } else {
            // the cat's turn
            final int i = cat / n;
            final int j = cat % n;
            for (int[] dir : DIRS) {
                for (int jump = 0; jump <= catJump; ++jump) {
                    final int x = i + dir[0] * jump;
                    final int y = j + dir[1] * jump;
                    if (x < 0 || x == m || y < 0 || y == n)
                        break;
                    if (grid[x].charAt(y) == '#')
                        break;
                    // The cat eats the food, so the mouse loses.
                    if (grid[x].charAt(y) == 'F')
                        return mem[cat][mouse][turn] = false;
                    final int nextCat = hash(x, y, n);
                    if (nextCat == mouse)
                        return mem[cat][mouse][turn] = false;
                    if (!canMouseWin(grid, nextCat, mouse, turn + 1, catJump, mouseJump, m, n, nFloors, mem))
                        return mem[cat][mouse][turn] = false;
                }
            }
            // The cat can't win, so the mouse wins.
            return mem[cat][mouse][turn] = true;
        }
    }
}
```

```
}  
private int hash(int i, int j, int n) {  
    return i * n + j;  
}  
}
```

746. 173-2.java

Path: LeetCode-main/solutions/173. Binary Search Tree Iterator/173-2.java

```
class BSTIterator {
    public BSTIterator(TreeNode root) {
        pushLeftsUntilNull(root);
    }

    public int next() {
        TreeNode root = stack.pop();
        pushLeftsUntilNull(root.right);
        return root.val;
    }

    public boolean hasNext() {
        return !stack.isEmpty();
    }

    private Deque<TreeNode> stack = new ArrayDeque<>();

    private void pushLeftsUntilNull(TreeNode root) {
        while (root != null) {
            stack.push(root);
            root = root.left;
        }
    }
}
```

747. 173.java

Path: LeetCode-main/solutions/173. Binary Search Tree Iterator/173.java

```
class BSTIterator {
    public BSTIterator(TreeNode root) {
        inorder(root);
    }

    public int next() {
        return vals.get(i++);
    }

    public boolean hasNext() {
        return i < vals.size();
    }

    private int i = 0;
    private List<Integer> vals = new ArrayList<>();

    private void inorder(TreeNode root) {
        if (root == null)
            return;
        inorder(root.left);
        vals.add(root.val);
        inorder(root.right);
    }
}
```

748. 1730.java

Path: LeetCode-main/solutions/1730. Shortest Path to Get Food/1730.java

```
class Solution {
    public int getFood(char[][] grid) {
        final int[][] DIRS = {{0, 1}, {1, 0}, {0, -1}, {-1, 0}};
        final int m = grid.length;
        final int n = grid[0].length;
        Queue<Pair<Integer, Integer>> q = new ArrayDeque<>(List.of(getStartLocation(grid)));

        for (int ans = 0; !q.isEmpty(); ++ans)
            for (int sz = q.size(); sz > 0; --sz) {
                final int i = q.peek().getKey();
                final int j = q.poll().getValue();
                for (int[] dir : DIRS) {
                    final int x = i + dir[0];
                    final int y = j + dir[1];
                    if (x < 0 || x == m || y < 0 || y == n)
                        continue;
                    if (grid[x][y] == 'X')
                        continue;
                    if (grid[x][y] == '#')
                        return ans + 1;
                    q.add(new Pair<>(x, y));
                    grid[x][y] = 'X'; // Mark as visited.
                }
            }

        return -1;
    }

    private Pair<Integer, Integer> getStartLocation(char[][] grid) {
        for (int i = 0; i < grid.length; ++i)
            for (int j = 0; j < grid[0].length; ++j)
                if (grid[i][j] == '*')
                    return new Pair<>(i, j);
        throw new IllegalArgumentException();
    }
}
```


749. 1732.java

Path: LeetCode-main/solutions/1732. Find the Highest Altitude/1732.java

```
class Solution {
    public int largestAltitude(int[] gain) {
        int ans = 0;
        int currAltitude = 0;
        for (final int g : gain) {
            currAltitude += g;
            ans = Math.max(ans, currAltitude);
        }
        return ans;
    }
}
```

750. 1733.java

Path: LeetCode-main/solutions/1733. Minimum Number of People to Teach/1733.java

```
class Solution {
    public int minimumTeachings(int n, int[][] languages, int[][] friendships) {
        List<Set<Integer>> languageSets = new ArrayList<>();
        Set<Integer> needTeach = new HashSet<>();
        Map<Integer, Integer> languageCount = new HashMap<>();

        for (int[] language : languages)
            languageSets.add(new HashSet<>(Arrays.stream(language).boxed().toList()));

        // Find friends that can't communicate.
        for (int[] friendship : friendships) {
            final int u = friendship[0] - 1;
            final int v = friendship[1] - 1;
            if (cantTalk(languageSets, u, v)) {
                needTeach.add(u);
                needTeach.add(v);
            }
        }

        // Find the most popular language.
        for (int u : needTeach)
            for (final int language : languageSets.get(u))
                languageCount.merge(language, 1, Integer::sum);

        // Teach the most popular language to people who don't understand.
        int maxCount = 0;
        for (int freq : languageCount.values())
            maxCount = Math.max(maxCount, freq);

        return needTeach.size() - maxCount;
    }

    // Returns true if u can't talk with v.
    private boolean cantTalk(List<Set<Integer>> languageSets, int u, int v) {
        for (int language : languageSets.get(u))
            if (languageSets.get(v).contains(language))
                return false;
        return true;
    }
}
```

751. 1734.java

Path: LeetCode-main/solutions/1734. Decode XORed Permutation/1734.java

```
class Solution {
    public int[] decode(int[] encoded) {
        // Our goal is to find the value of a1, which will allow us to decode a2,
        // a3, ..., an. This can be achieved by performing XOR operation between
        // each element in `encoded` and a1.
        //
        // e.g. n = 3, perm = [a1, a2, a3] is a permutation of [1, 2, 3]
        //      encoded = [a1^a2, a2^a3]
        //      accumulatedEncoded = [a1^a2, a1^a3]
        //      a1 = (a1^a2)^(a1^a3)^(a1^a2^a3)
        //      a2 = a1^(a1^a2)
        //      a3 = a2^(a2^a3)
        final int n = encoded.length + 1;
        int nXors = 0;
        for (int i = 1; i <= n; i++)
            nXors ^= i;

        // Instead of constructing the array, we can track of the running XOR value
        // of `accumulatedEncoded`.
        int runningXors = 0;
        int xors = 0; // xors(accumulatedEncoded)

        for (final int encode : encoded) {
            runningXors ^= encode;
            xors ^= runningXors;
        }

        int[] ans = new int[encoded.length + 1];
        ans[0] = xors ^ nXors;

        for (int i = 0; i < encoded.length; i++)
            ans[i + 1] = ans[i] ^ encoded[i];

        return ans;
    }
}
```

752. 1735.java

Path: LeetCode-main/solutions/1735. Count Ways to Make Array With Product/1735.java

```
class Solution {
    public int[] waysToFillArray(int[][] queries) {
        final int MAX = 10_000;
        final int MAX_FREQ = 13; //  $2^{13} = 8192 < \text{MAX}$ 
        final int[] minPrimeFactors = sieveEratosthenes(MAX + 1);
        final long[][] factAndInvFact = getFactAndInvFact(MAX + MAX_FREQ - 1);
        final long[] fact = factAndInvFact[0];
        final long[] invFact = factAndInvFact[1];
        int[] ans = new int[queries.length];

        for (int i = 0; i < queries.length; ++i) {
            final int n = queries[i][0];
            final int k = queries[i][1];
            int res = 1;
            for (final int freq : getPrimeFactorsCount(k, minPrimeFactors).values())
                res = (int) ((long) res * nCk(n - 1 + freq, freq, fact, invFact) % MOD);
            ans[i] = res;
        }

        return ans;
    }

    private static final int MOD = 1_000_000_007;

    // Gets the minimum prime factor of i, where  $1 < i \leq n$ .
    private int[] sieveEratosthenes(int n) {
        int[] minPrimeFactors = new int[n + 1];
        for (int i = 2; i <= n; ++i)
            minPrimeFactors[i] = i;
        for (int i = 2; i * i < n; ++i)
            if (minPrimeFactors[i] == i) // `i` is prime.
                for (int j = i * i; j < n; j += i)
                    minPrimeFactors[j] = Math.min(minPrimeFactors[j], i);
        return minPrimeFactors;
    }

    private Map<Integer, Integer> getPrimeFactorsCount(int num, int[] minPrimeFactors) {
        Map<Integer, Integer> count = new HashMap<>();
        while (num > 1) {
            final int divisor = minPrimeFactors[num];
            while (num % divisor == 0) {
                num /= divisor;
                count.put(divisor, count.merge(divisor, 1, Integer::sum));
            }
        }
        return count;
    }

    private long[][] getFactAndInvFact(int n) {
        long[] fact = new long[n + 1];
        long[] invFact = new long[n + 1];
        long[] inv = new long[n + 1];
        fact[0] = invFact[0] = 1;
        inv[0] = inv[1] = 1;
        for (int i = 1; i <= n; ++i) {
            if (i >= 2)
                inv[i] = MOD - MOD / i * inv[MOD % i] % MOD;
            fact[i] = fact[i - 1] * i % MOD;
            invFact[i] = invFact[i - 1] * inv[i] % MOD;
        }
        return new long[][] {fact, invFact};
    }

    private int nCk(int n, int k, long[] fact, long[] invFact) {
        return (int) (fact[n] * invFact[k] % MOD * invFact[n - k] % MOD);
    }
}
```

753. 1736.java

Path: LeetCode-main/solutions/1736. Latest Time by Replacing Hidden Digits/1736.java

```
class Solution {
    public String maximumTime(String time) {
        char[] ans = time.toCharArray();
        if (time.charAt(0) == '?')
            ans[0] = time.charAt(1) == '?' || time.charAt(1) < '4' ? '2' : '1';
        if (time.charAt(1) == '?')
            ans[1] = ans[0] == '2' ? '3' : '9';
        if (time.charAt(3) == '?')
            ans[3] = '5';
        if (time.charAt(4) == '?')
            ans[4] = '9';
        return new String(ans);
    }
}
```

754. 1737.java

Path: LeetCode-main/solutions/1737. Change Minimum Characters to Satisfy One of Three Conditions/1737.java

```
class Solution {
    public int minCharacters(String a, String b) {
        final int m = a.length();
        final int n = b.length();
        int[] countA = new int[26];
        int[] countB = new int[26];

        for (final char c : a.toCharArray())
            ++countA[c - 'a'];

        for (final char c : b.toCharArray())
            ++countB[c - 'a'];

        int ans = Integer.MAX_VALUE;
        int prevA = 0; // the number of characters in a <= c
        int prevB = 0; // the number of characters in b <= c

        for (char c = 'a'; c <= 'z'; ++c) {
            // the condition 3
            ans = Math.min(ans, m + n - countA[c - 'a'] - countB[c - 'a']);
            // the conditions 1 and 2
            if (c > 'a')
                ans = Math.min(ans, Math.min(m - prevA + prevB, n - prevB + prevA));
            prevA += countA[c - 'a'];
            prevB += countB[c - 'a'];
        }

        return ans;
    }
}
```

755. 1738.java

Path: LeetCode-main/solutions/1738. Find Kth Largest XOR Coordinate Value/1738.java

```
class Solution {
    public int kthLargestValue(int[][] matrix, int k) {
        final int m = matrix.length;
        final int n = matrix[0].length;
        int[][] xors = new int[m + 1][n + 1];
        Queue<Integer> minHeap = new PriorityQueue<>();

        for (int i = 1; i <= m; ++i)
            for (int j = 1; j <= n; ++j) {
                xors[i][j] = xors[i - 1][j] ^ xors[i][j - 1] ^ xors[i - 1][j - 1] ^ matrix[i - 1][j - 1];
                minHeap.offer(xors[i][j]);
                if (minHeap.size() > k)
                    minHeap.poll();
            }

        return minHeap.peek();
    }
}
```

756. 1739.java

Path: LeetCode-main/solutions/1739. Building Boxes/1739.java

```
class Solution {
    public int minimumBoxes(int n) {
        int nBoxes = 0;
        int nextTouchings = 0; // j
        int currLevelBoxes = 0; // 1 + 2 + ... + j

        // Find the minimum j s.t. `nBoxes` = 1 + (1 + 2) + ... + (1 + 2 + ... + j)
        // >= n.
        while (nBoxes < n) {
            ++nextTouchings;
            currLevelBoxes += nextTouchings;
            nBoxes += currLevelBoxes;
        }

        // If nBoxes = n, the answer is `currLevelBoxes` = 1 + 2 + ... + j.
        if (nBoxes == n)
            return currLevelBoxes;

        // Otherwise, need to remove the boxes in the current level and rebuild it.
        nBoxes -= currLevelBoxes;
        currLevelBoxes -= nextTouchings;
        nextTouchings = 0;

        while (nBoxes < n) {
            ++nextTouchings;
            nBoxes += nextTouchings;
        }

        return currLevelBoxes + nextTouchings;
    }
}
```


757. 174.java

Path: LeetCode-main/solutions/174. Dungeon Game/174.java

```
class Solution {
    public int calculateMinimumHP(int[][] dungeon) {
        final int m = dungeon.length;
        final int n = dungeon[0].length;
        int[][] dp = new int[m + 1][n + 1];
        Arrays.stream(dp).forEach(A -> Arrays.fill(A, Integer.MAX_VALUE));
        dp[m][n - 1] = 1;
        dp[m - 1][n] = 1;

        for (int i = m - 1; i >= 0; --i)
            for (int j = n - 1; j >= 0; --j) {
                dp[i][j] = Math.min(dp[i + 1][j], dp[i][j + 1]) - dungeon[i][j];
                dp[i][j] = Math.max(dp[i][j], 1);
            }

        return dp[0][0];
    }
}
```

758. 1740.java

Path: LeetCode-main/solutions/1740. Find Distance in a Binary Tree/1740.java

```
class Solution {
    public int findDistance(TreeNode root, int p, int q) {
        TreeNode lca = getLCA(root, p, q);
        return dist(lca, p) + dist(lca, q);
    }

    private TreeNode getLCA(TreeNode root, int p, int q) {
        if (root == null || root.val == p || root.val == q)
            return root;

        TreeNode l = getLCA(root.left, p, q);
        TreeNode r = getLCA(root.right, p, q);

        if (l != null && r != null)
            return root;
        return l == null ? r : l;
    }

    private int dist(TreeNode lca, int target) {
        if (lca == null)
            return 10000;
        if (lca.val == target)
            return 0;
        return 1 + Math.min(dist(lca.left, target), dist(lca.right, target));
    }
}
```

759. 1742.java

Path: LeetCode-main/solutions/1742. Maximum Number of Balls in a Box/1742.java

```
class Solution {
    public int countBalls(int lowLimit, int highLimit) {
        final int maxDigitSum = 9 * 5; // 99999
        int ans = 0;
        int[] count = new int[maxDigitSum + 1];

        for (int num = lowLimit; num <= highLimit; ++num)
            ans = Math.max(ans, ++count[getDigitSum(num)]);

        return ans;
    }

    private int getDigitSum(int num) {
        int digitSum = 0;
        while (num > 0) {
            digitSum += num % 10;
            num /= 10;
        }
        return digitSum;
    }
}
```

760. 1743.java

Path: LeetCode-main/solutions/1743. Restore the Array From Adjacent Pairs/1743.java

```
class Solution {
    public int[] restoreArray(int[][] adjacentPairs) {
        int[] ans = new int[adjacentPairs.length + 1];
        int i = 0; // ans' index
        Map<Integer, List<Integer>> numToAdjs = new HashMap<>();

        for (int[] pair : adjacentPairs) {
            numToAdjs.putIfAbsent(pair[0], new ArrayList<>());
            numToAdjs.putIfAbsent(pair[1], new ArrayList<>());
            numToAdjs.get(pair[0]).add(pair[1]);
            numToAdjs.get(pair[1]).add(pair[0]);
        }

        for (Map.Entry<Integer, List<Integer>> entry : numToAdjs.entrySet())
            if (entry.getValue().size() == 1) {
                ans[i++] = entry.getKey();
                ans[i++] = entry.getValue().get(0);
                break;
            }

        while (i < adjacentPairs.length + 1) {
            final int tail = ans[i - 1];
            final int prev = ans[i - 2];
            List<Integer> adjs = numToAdjs.get(tail);
            if (adjs.get(0) == prev)
                ans[i++] = adjs.get(1);
            else
                ans[i++] = adjs.get(0);
        }

        return ans;
    }
}
```

761. 1744.java

Path: LeetCode-main/solutions/1744. Can You Eat Your Favorite Candy on Your Favorite Day?/1744.java

```
class Solution {
    public boolean[] canEat(int[] candiesCount, int[][] queries) {
        final int n = candiesCount.length;
        boolean[] ans = new boolean[queries.length];
        long[] prefix = new long[n + 1];

        for (int i = 0; i < n; ++i)
            prefix[i + 1] = prefix[i] + candiesCount[i];

        for (int i = 0; i < queries.length; ++i) {
            final int type = queries[i][0];
            final long day = (long) queries[i][1];
            final long cap = (long) queries[i][2];
            // the minimum day required to eat
            final long minDay = prefix[type] / cap;
            // the maximum day required to eat
            final long maxDay = prefix[type + 1] - 1;
            ans[i] = minDay <= day && day <= maxDay;
        }

        return ans;
    }
}
```

762. 1745-2.java

Path: LeetCode-main/solutions/1745. Palindrome Partitioning IV/1745-2.java

```
class Solution {
    public boolean checkPartitioning(String s) {
        final int n = s.length();
        // dp[i][j] := true if s[i..j] is a palindrome
        boolean[][] dp = new boolean[n + 1][n + 1];

        for (int i = 0; i < n; ++i)
            dp[i][i] = true;

        for (int d = 1; d < n; ++d)
            for (int i = 0; i + d < n; ++i) {
                final int j = i + d;
                if (s.charAt(i) == s.charAt(j))
                    dp[i][j] = i + 1 > j - 1 || dp[i + 1][j - 1];
            }

        for (int i = 0; i < n; ++i)
            for (int j = i + 1; j < n; ++j)
                if (dp[0][i] && dp[i + 1][j] && dp[j + 1][n - 1])
                    return true;

        return false;
    }
}
```

763. 1745.java

Path: LeetCode-main/solutions/1745. Palindrome Partitioning IV/1745.java

```
class Solution {
    public boolean checkPartitioning(String s) {
        final int n = s.length();
        Boolean[][] mem = new Boolean[n][n];

        for (int i = 0; i < n; ++i)
            for (int j = i + 1; j < n; ++j)
                if (isPalindrome(s, 0, i, mem) && //
                    isPalindrome(s, i + 1, j, mem) && //
                    isPalindrome(s, j + 1, n - 1, mem))
                    return true;

        return false;
    }

    // Returns true if s[i..j] is a palindrome.
    // Returns false if s[i..j] is not a palindrome.
    private boolean isPalindrome(final String s, int i, int j, Boolean[][] mem) {
        if (i > j)
            return true;
        if (mem[i][j] != null)
            return mem[i][j];
        if (s.charAt(i) == s.charAt(j))
            return mem[i][j] = isPalindrome(s, i + 1, j - 1, mem);
        return mem[i][j] = false;
    }
}
```

764. 1746.java

Path: LeetCode-main/solutions/1746. Maximum Subarray Sum After One Operation/1746.java

```
class Solution {
    public int maxSumAfterOperation(int[] nums) {
        int ans = Integer.MIN_VALUE;
        int regular = 0;
        int squared = 0;

        for (final int num : nums) {
            squared = Math.max(num * num, Math.max(regular + num * num, squared + num));
            regular = Math.max(num, regular + num);
            ans = Math.max(ans, squared);
        }

        return ans;
    }
}
```


765. 1748.java

Path: LeetCode-main/solutions/1748. Sum of Unique Elements/1748.java

```
class Solution {
    public int sumOfUnique(int[] nums) {
        final int MAX = 100;
        int ans = 0;
        int[] count = new int[MAX + 1];

        for (final int num : nums)
            ++count[num];

        for (int i = 1; i <= MAX; ++i)
            if (count[i] == 1)
                ans += i;

        return ans;
    }
}
```

766. 1749-2.java

Path: LeetCode-main/solutions/1749. Maximum Absolute Sum of Any Subarray/1749-2.java

```
class Solution {
    public int maxAbsoluteSum(int[] nums) {
        int sum = 0;
        int maxPrefix = 0;
        int minPrefix = 0;

        for (final int num : nums) {
            sum += num;
            maxPrefix = Math.max(maxPrefix, sum);
            minPrefix = Math.min(minPrefix, sum);
        }

        return maxPrefix - minPrefix;
    }
}
```

767. 1749.java

Path: LeetCode-main/solutions/1749. Maximum Absolute Sum of Any Subarray/1749.java

```
class Solution {
    public int maxAbsoluteSum(int[] nums) {
        int ans = Integer.MIN_VALUE;
        int maxSum = 0;
        int minSum = 0;

        for (final int num : nums) {
            maxSum = Math.max(num, maxSum + num);
            minSum = Math.min(num, minSum + num);
            ans = Math.max(ans, Math.max(maxSum, -minSum));
        }

        return ans;
    }
}
```

768. 1750.java

Path: LeetCode-main/solutions/1750. Minimum Length of String After Deleting Similar Ends/1750.java

```
class Solution {
    public int minimumLength(String s) {
        int i = 0;
        int j = s.length() - 1;

        while (i < j && s.charAt(i) == s.charAt(j)) {
            final char c = s.charAt(i);
            while (i <= j && s.charAt(i) == c)
                ++i;
            while (i <= j && s.charAt(j) == c)
                --j;
        }

        return j - i + 1;
    }
}
```

769. 1751.java

Path: LeetCode-main/solutions/1751. Maximum Number of Events That Can Be Attended II/1751.java

```
class Solution {
    public int maxValue(int[][] events, int k) {
        Integer[][] mem = new Integer[events.length][k + 1];
        Arrays.sort(events, Comparator.comparingInt((int[] event) -> event[0])
            .thenComparing((int[] event) -> event[1]));
        return maxValue(events, 0, k, mem);
    }

    private int maxValue(int[][] events, int i, int k, Integer[][] mem) {
        if (k == 0 || i == events.length)
            return 0;
        if (mem[i][k] != null)
            return mem[i][k];

        // Binary search `events` to find the first index j
        // s.t. events[j][0] > events[i][1].
        final int j = firstGreaterEqual(events, i + 1, events[i][1] + 1);
        return mem[i][k] = Math.max(events[i][2] + maxValue(events, j, k - 1, mem),
            maxValue(events, i + 1, k, mem));
    }

    // Finds the first index l s.t events[l][0] >= target.
    private int firstGreaterEqual(int[][] events, int l, int target) {
        int r = events.length;
        while (l < r) {
            final int m = (l + r) / 2;
            if (events[m][0] >= target)
                r = m;
            else
                l = m + 1;
        }
        return l;
    }
}
```

770. 1752.java

Path: LeetCode-main/solutions/1752. Check if Array Is Sorted and Rotated/1752.java

```
class Solution {
    public boolean check(int[] nums) {
        final int n = nums.length;
        int rotates = 0;

        for (int i = 0; i < n; ++i)
            if (nums[i] > nums[(i + 1) % n] && ++rotates > 1)
                return false;

        return true;
    }
}
```

771. 1753.java

Path: LeetCode-main/solutions/1753. Maximum Score From Removing Stones/1753.java

```
class Solution {  
    public int maximumScore(int a, int b, int c) {  
        // the maximum <= the minimum + the middle  
        final int x = (a + b + c) / 2;  
        // the maximum > the minimum + the middle  
        final int y = a + b + c - Math.max(a, Math.max(b, c));  
        return Math.min(x, y);  
    }  
}
```

772. 1754.java

Path: LeetCode-main/solutions/1754. Largest Merge Of Two Strings/1754.java

```
class Solution {
    public String largestMerge(String word1, String word2) {
        if (word1.isEmpty())
            return word2;
        if (word2.isEmpty())
            return word1;
        return word1.compareTo(word2) > 0 ? word1.charAt(0) + largestMerge(word1.substring(1), word2)
            : word2.charAt(0) + largestMerge(word1, word2.substring(1));
    }
}
```


773. 1755.java

Path: LeetCode-main/solutions/1755. Closest Subsequence Sum/1755.java

```
class Solution {
    public int minAbsDifference(int[] nums, int goal) {
        final int n = nums.length / 2;
        final int[] lNums = Arrays.copyOfRange(nums, 0, n);
        final int[] rNums = Arrays.copyOfRange(nums, n, nums.length);
        int ans = Integer.MAX_VALUE;
        List<Integer> lSums = new ArrayList<>();
        List<Integer> rSums = new ArrayList<>();

        dfs(lNums, 0, 0, lSums);
        dfs(rNums, 0, 0, rSums);
        Collections.sort(rSums);

        for (final int lSum : lSums) {
            final int i = firstGreaterEqual(rSums, goal - lSum);
            if (i < rSums.size()) // 2^n
                ans = Math.min(ans, Math.abs(goal - lSum - rSums.get(i)));
            if (i > 0)
                ans = Math.min(ans, Math.abs(goal - lSum - rSums.get(i - 1)));
        }

        return ans;
    }

    private void dfs(int[] arr, int i, int path, List<Integer> sums) {
        if (i == arr.length) {
            sums.add(path);
            return;
        }
        dfs(arr, i + 1, path + arr[i], sums);
        dfs(arr, i + 1, path, sums);
    }

    private int firstGreaterEqual(List<Integer> arr, int target) {
        final int i = Collections.binarySearch(arr, target);
        return i < 0 ? -i - 1 : i;
    }
}
```

774. 1758.java

Path: LeetCode-main/solutions/1758. Minimum Changes To Make Alternating Binary String/1758.java

```
class Solution {
    public int minOperations(String s) {
        int cost10 = 0; // the cost to make s "1010"

        for (int i = 0; i < s.length(); ++i)
            if (s.charAt(i) - '0' == i % 2)
                ++cost10;

        final int cost01 = s.length() - cost10; // the cost to make s "0101"
        return Math.min(cost10, cost01);
    }
}
```

775. 1759.java

Path: LeetCode-main/solutions/1759. Count Number of Homogenous Substrings/1759.java

```
class Solution {
    public int countHomogenous(String s) {
        final int MOD = 1_000_000_007;
        int ans = 0;
        int count = 0;
        char currentChar = '@';

        for (final char c : s.toCharArray()) {
            count = c == currentChar ? count + 1 : 1;
            currentChar = c;
            ans += count;
            ans %= MOD;
        }

        return ans;
    }
}
```

776. 1760.java

Path: LeetCode-main/solutions/1760. Minimum Limit of Balls in a Bag/1760.java

```
class Solution {
    public int minimumSize(int[] nums, int maxOperations) {
        int l = 1;
        int r = Arrays.stream(nums).max().getAsInt();

        while (l < r) {
            final int m = (l + r) / 2;
            if (numOperations(nums, m) <= maxOperations)
                r = m;
            else
                l = m + 1;
        }

        return l;
    }

    // Returns the number of operations required to make m penalty.
    private int numOperations(int[] nums, int m) {
        int operations = 0;
        for (final int num : nums)
            operations += (num - 1) / m;
        return operations;
    }
}
```

777. 1761.java

Path: LeetCode-main/solutions/1761. Minimum Degree of a Connected Trio in a Graph/1761.java

```
class Solution {
    public int minTrioDegree(int n, int[][] edges) {
        int ans = Integer.MAX_VALUE;
        Set<Integer>[] graph = new Set[n];
        int[] degrees = new int[n];

        for (int i = 0; i < n; ++i)
            graph[i] = new HashSet<>();

        for (int[] edge : edges) {
            final int u = edge[0] - 1;
            final int v = edge[1] - 1;
            // Store the mapping from `min(u, v)` to `max(u, v)` to speed up.
            graph[Math.min(u, v)].add(Math.max(u, v));
            ++degrees[u];
            ++degrees[v];
        }

        for (int u = 0; u < n; u++)
            for (final int v : graph[u])
                for (final int w : graph[u])
                    if (graph[v].contains(w))
                        ans = Math.min(ans, degrees[u] + degrees[v] + degrees[w] - 6);

        return ans == Integer.MAX_VALUE ? -1 : ans;
    }
}
```

778. 1762.java

Path: LeetCode-main/solutions/1762. Buildings With an Ocean View/1762.java

```
class Solution {
    public int[] findBuildings(int[] heights) {
        Deque<Integer> stack = new ArrayDeque<>();

        for (int i = 0; i < heights.length; ++i) {
            while (!stack.isEmpty() && heights[stack.peek()] <= heights[i])
                stack.pop();
            stack.push(i);
        }

        int[] ans = new int[stack.size()];
        for (int i = 0; i < ans.length; ++i)
            ans[ans.length - 1 - i] = stack.pop();
        return ans;
    }
}
```

779. 1763.java

Path: LeetCode-main/solutions/1763. Longest Nice Substring/1763.java

```
class Solution {
    public String longestNiceSubstring(String s) {
        if (s.length() < 2)
            return "";

        Set<Character> seen = s.chars().mapToObj(c -> (char) c).collect(Collectors.toSet());

        for (int i = 0; i < s.length(); ++i)
            if (!seen.contains(toggleCase(s.charAt(i)))) {
                final String prefix = longestNiceSubstring(s.substring(0, i));
                final String suffix = longestNiceSubstring(s.substring(i + 1));
                return prefix.length() >= suffix.length() ? prefix : suffix;
            }

        return s;
    }

    private char toggleCase(char c) {
        return Character.isLowerCase(c) ? Character.toUpperCase(c) : Character.toLowerCase(c);
    }
}
```

780. 1764.java

Path: LeetCode-main/solutions/1764. Form Array by Concatenating Subarrays of Another Array/1764.java

```
class Solution {
    public boolean canChoose(int[][] groups, int[] nums) {
        int i = 0; // groups' index
        int j = 0; // nums' index

        while (i < groups.length && j < nums.length)
            if (isMatch(groups[i], nums, j)) {
                j += groups[i].length;
                ++i;
            } else {
                ++j;
            }

        return i == groups.length;
    }

    // Returns true if group == nums[j..j + |group|].
    private boolean isMatch(int[] group, int[] nums, int j) {
        if (j + group.length > nums.length)
            return false;
        for (int i = 0; i < group.length; ++i)
            if (group[i] != nums[j + i])
                return false;
        return true;
    }
}
```


781. 1765.java

Path: LeetCode-main/solutions/1765. Map of Highest Peak/1765.java

```
class Solution {
    public int[][] highestPeak(int[][] isWater) {
        final int[][] DIRS = {{0, 1}, {1, 0}, {0, -1}, {-1, 0}};
        final int m = isWater.length;
        final int n = isWater[0].length;
        int[][] ans = new int[m][n];
        Arrays.stream(ans).forEach(A -> Arrays.fill(A, -1));
        Queue<Pair<Integer, Integer>> q = new ArrayDeque<>();

        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j)
                if (isWater[i][j] == 1) {
                    q.offer(new Pair<>(i, j));
                    ans[i][j] = 0;
                }

        while (!q.isEmpty()) {
            final int i = q.peek().getKey();
            final int j = q.poll().getValue();
            for (int[] dir : DIRS) {
                final int x = i + dir[0];
                final int y = j + dir[1];
                if (x < 0 || x == m || y < 0 || y == n)
                    continue;
                if (ans[x][y] != -1)
                    continue;
                ans[x][y] = ans[i][j] + 1;
                q.offer(new Pair<>(x, y));
            }
        }

        return ans;
    }
}
```

782. 1766.java

Path: LeetCode-main/solutions/1766. Tree of Coprimes/1766.java

```
class Solution {
    public int[] getCoprimes(int[] nums, int[][] edges) {
        int[] ans = new int[nums.length];
        Arrays.fill(ans, -1);
        List<Integer>[] tree = new List[nums.length];
        // stacks[i] := (node, depth)s of nodes with value i
        Deque<Pair<Integer, Integer>>[] stacks = new Deque[MAX + 1];

        for (int i = 0; i < nums.length; ++i)
            tree[i] = new ArrayList<>();

        for (int i = 1; i <= MAX; ++i)
            stacks[i] = new ArrayDeque<>();

        for (int[] edge : edges) {
            final int u = edge[0];
            final int v = edge[1];
            tree[u].add(v);
            tree[v].add(u);
        }

        dfs(tree, 0, /*prev=*/-1, /*depth=*/0, nums, stacks, ans);
        return ans;
    }

    private static final int MAX = 50;

    private void dfs(List<Integer>[] tree, int u, int prev, int depth, int[] nums,
                    Deque<Pair<Integer, Integer>>[] stacks, int[] ans) {
        ans[u] = getAncestor(u, stacks, nums);
        stacks[nums[u]].push(new Pair<>(u, depth));

        for (final int v : tree[u])
            if (v != prev)
                dfs(tree, v, u, depth + 1, nums, stacks, ans);

        stacks[nums[u]].pop();
    }

    private int getAncestor(int u, Deque<Pair<Integer, Integer>>[] stacks, int[] nums) {
        int maxNode = -1;
        int maxDepth = -1;
        for (int i = 1; i <= MAX; ++i)
            if (!stacks[i].isEmpty() && stacks[i].peek().getValue() > maxDepth && gcd(nums[u], i) == 1) {
                maxNode = stacks[i].peek().getKey();
                maxDepth = stacks[i].peek().getValue();
            }
        return maxNode;
    }

    private int gcd(int a, int b) {
        return b == 0 ? a : gcd(b, a % b);
    }
}
```

783. 1768.java

Path: LeetCode-main/solutions/1768. Merge Strings Alternately/1768.java

```
class Solution {
    public String mergeAlternately(String word1, String word2) {
        final int n = Math.min(word1.length(), word2.length());
        StringBuilder sb = new StringBuilder();

        for (int i = 0; i < n; ++i) {
            sb.append(word1.charAt(i));
            sb.append(word2.charAt(i));
        }

        return sb.append(word1.substring(n)).append(word2.substring(n)).toString();
    }
}
```

784. 1769.java

Path: LeetCode-main/solutions/1769. Minimum Number of Operations to Move All Balls to Each Box/1769.java

```
class Solution {
    public int[] minOperations(String boxes) {
        int[] ans = new int[boxes.length()];

        for (int i = 0, count = 0, moves = 0; i < boxes.length(); ++i) {
            ans[i] += moves;
            count += boxes.charAt(i) == '1' ? 1 : 0;
            moves += count;
        }

        for (int i = boxes.length() - 1, count = 0, moves = 0; i >= 0; --i) {
            ans[i] += moves;
            count += boxes.charAt(i) == '1' ? 1 : 0;
            moves += count;
        }

        return ans;
    }
}
```

785. 1770.java

Path: LeetCode-main/solutions/1770. Maximum Score from Performing Multiplication Operations/1770.java

```
class Solution {
    public int maximumScore(int[] nums, int[] multipliers) {
        Integer[][] mem = new Integer[multipliers.length][multipliers.length];
        return maximumScore(nums, 0, multipliers, 0, mem);
    }

    private int maximumScore(int[] nums, int s, int[] multipliers, int i, Integer[][] mem) {
        if (i == multipliers.length)
            return 0;
        if (mem[s][i] != null)
            return mem[s][i];

        // The number of nums picked on the start side is s.
        // The number of nums picked on the end side is i - s.
        // So, e = n - (i - s) - 1.
        final int e = nums.length - (i - s) - 1;
        final int pickStart = nums[s] * multipliers[i] + //
            maximumScore(nums, s + 1, multipliers, i + 1, mem);
        final int pickEnd = nums[e] * multipliers[i] + //
            maximumScore(nums, s, multipliers, i + 1, mem);
        return mem[s][i] = Math.max(pickStart, pickEnd);
    }
}
```

786. 1771.java

Path: LeetCode-main/solutions/1771. Maximize Palindrome Length From Subsequences/1771.java

```
class Solution {
    public int longestPalindrome(String word1, String word2) {
        final String s = word1 + word2;
        final int n = s.length();
        int ans = 0;
        // dp[i][j] := the length of LPS(s[i..j])
        int[][] dp = new int[n][n];

        for (int i = 0; i < n; ++i)
            dp[i][i] = 1;

        for (int d = 1; d < n; ++d)
            for (int i = 0; i + d < n; ++i) {
                final int j = i + d;
                if (s.charAt(i) == s.charAt(j)) {
                    dp[i][j] = 2 + dp[i + 1][j - 1];
                    if (i < word1.length() && j >= word1.length())
                        ans = Math.max(ans, dp[i][j]);
                } else {
                    dp[i][j] = Math.max(dp[i + 1][j], dp[i][j - 1]);
                }
            }

        return ans;
    }
}
```

787. 1772.java

Path: LeetCode-main/solutions/1772. Sort Features by Popularity/1772.java

```
class Solution {
    public String[] sortFeatures(String[] features, String[] responses) {
        String[] ans = new String[features.length];
        int[][] featCount = new int[features.length][]; // {i: count[features[i]]}
        Map<String, Integer> count = new HashMap<>();

        for (final String res : responses)
            for (final String token : new HashSet<>(Arrays.asList(res.split(" "))))
                count.merge(token, 1, Integer::sum);

        for (int i = 0; i < features.length; ++i)
            featCount[i] = new int[] {i, count.getDefault(features[i], 0)};

        Arrays.sort(featCount,
            Comparator.comparingInt((int[] a) -> - a[1]).thenComparingInt((int[] a) -> a[0]));

        for (int i = 0; i < features.length; ++i)
            ans[i] = features[featCount[i][0]];

        return ans;
    }
}
```

788. 1773.java

Path: LeetCode-main/solutions/1773. Count Items Matching a Rule/1773.java

```
class Solution {
    public int countMatches(List<List<String>> items, String ruleKey, String ruleValue) {
        if (ruleKey.equals("type"))
            return count(items, 0, ruleValue);
        if (ruleKey.equals("color"))
            return count(items, 1, ruleValue);
        return count(items, 2, ruleValue);
    }

    private int count(List<List<String>> items, int index, final String ruleValue) {
        return (int) items.stream()
            .map(item -> item.get(index))
            .filter(s -> s.equals(ruleValue))
            .count();
    }
}
```


789. 1774.java

Path: LeetCode-main/solutions/1774. Closest Dessert Cost/1774.java

```
class Solution {
    public int closestCost(int[] baseCosts, int[] toppingCosts, int target) {
        for (final int baseCost : baseCosts)
            dfs(toppingCosts, 0, target, baseCost);
        return ans;
    }

    private int ans = Integer.MAX_VALUE;

    private void dfs(int[] toppingCosts, int i, int target, int currCost) {
        if (Math.abs(currCost - target) < Math.abs(ans - target))
            ans = currCost;
        if (i == toppingCosts.length || currCost >= target)
            return;

        for (int k = 0; k < 3; ++k)
            dfs(toppingCosts, i + 1, target, currCost + k * toppingCosts[i]);
    }
}
```

790. 1775.java

Path: LeetCode-main/solutions/1775. Equal Sum Arrays With Minimum Number of Operations/1775.java

```
class Solution {
    public int minOperations(int[] nums1, int[] nums2) {
        if (nums1.length * 6 < nums2.length || nums2.length * 6 < nums1.length)
            return -1;

        int sum1 = Arrays.stream(nums1).sum();
        int sum2 = Arrays.stream(nums2).sum();
        if (sum1 > sum2)
            return minOperations(nums2, nums1);

        int ans = 0;
        // increasing in `nums1` and decreasing in `nums2`
        int[] count = new int[6];

        Arrays.stream(nums1).forEach(num -> ++count[6 - num]);
        Arrays.stream(nums2).forEach(num -> ++count[num - 1]);

        for (int i = 5; sum2 > sum1; i) {
            while (count[i] == 0)
                --i;
            sum1 += i;
            --count[i];
            ++ans;
        }

        return ans;
    }
}
```

791. 1776.java

Path: LeetCode-main/solutions/1776. Car Fleet II/1776.java

```
class Solution {
    public double[] getCollisionTimes(int[][] cars) {
        double[] ans = new double[cars.length];
        Deque<Car> stack = new ArrayDeque<>();

        for (int i = cars.length - 1; i >= 0; --i) {
            final int pos = cars[i][0];
            final int speed = cars[i][1];
            while (!stack.isEmpty() &&
                (speed <= stack.peek().speed ||
                 getCollisionTime(stack.peek(), pos, speed) >= stack.peek().collisionTime))
                stack.pop();
            if (stack.isEmpty()) {
                stack.push(new Car(pos, speed, Integer.MAX_VALUE));
                ans[i] = -1;
            } else {
                final double collisionTime = getCollisionTime(stack.peek(), pos, speed);
                stack.push(new Car(pos, speed, collisionTime));
                ans[i] = collisionTime;
            }
        }

        return ans;
    }

    private record Car(int pos, int speed, double collisionTime) {}

    private double getCollisionTime(Car car, int pos, int speed) {
        return (double) (car.pos - pos) / (speed - car.speed);
    }
}
```

792. 1778.java

Path: LeetCode-main/solutions/1778. Shortest Path in a Hidden Grid/1778.java

```
/**
 * // This is the GridMaster's API interface.
 * // You should not implement it, or speculate about its implementation
 * class GridMaster {
 *     boolean canMove(char direction);
 *     void move(char direction);
 *     boolean isTarget();
 * }
 */

enum Grid { UNVISITED, START, TARGET, BLOCKED, EMPTY }

class Solution {
    public int findShortestPath(GridMaster master) {
        final int m = 501;
        final int startX = m;
        final int startY = m;
        Grid[][] grid = new Grid[m * 2][m * 2];
        Arrays.stream(grid).forEach(A -> Arrays.fill(A, Grid.UNVISITED));

        // Build the grid information by DFS.
        dfs(master, grid, startX, startY);

        Queue<Pair<Integer, Integer>> q = new ArrayDeque<>(List.of(new Pair<>(startX, startY)));
        grid[startX][startY] = Grid.BLOCKED;

        // Find the steps by BFS.
        for (int step = 1; !q.isEmpty(); ++step)
            for (int sz = q.size(); sz > 0; --sz) {
                final int i = q.peek().getKey();
                final int j = q.poll().getValue();
                for (int[] dir : DIRS) {
                    final int x = i + dir[0];
                    final int y = j + dir[1];
                    if (grid[x][y] == Grid.TARGET)
                        return step;
                    if (grid[x][y] == Grid.BLOCKED)
                        continue;
                    grid[x][y] = Grid.BLOCKED;
                    q.offer(new Pair<>(x, y));
                }
            }

        return -1;
    }

    private static final int[][] DIRS = {{0, 1}, {1, 0}, {0, -1}, {-1, 0}};
    private static final char[] charTable = {'R', 'D', 'L', 'U'};

    private void dfs(GridMaster master, Grid[][] grid, int i, int j) {
        if (grid[i][j] != Grid.UNVISITED)
            return;
        if (master.isTarget())
            grid[i][j] = Grid.TARGET;
        else
            grid[i][j] = Grid.EMPTY;

        for (int k = 0; k < 4; ++k) {
            final int x = i + DIRS[k][0];
            final int y = j + DIRS[k][1];
            final char d = charTable[k];
            final char undoD = charTable[(k + 2) % 4];
            if (master.canMove(d)) {
                master.move(d);
                dfs(master, grid, x, y);
                master.move(undoD);
            } else {
                grid[x][y] = Grid.BLOCKED;
            }
        }
    }
}
```

793. 1779.java

Path: LeetCode-main/solutions/1779. Find Nearest Point That Has the Same X or Y Coordinate/1779.java

```
class Solution {
    public int nearestValidPoint(int x, int y, int[][] points) {
        int ans = -1;
        int minDist = Integer.MAX_VALUE;

        for (int i = 0; i < points.length; ++i) {
            final int dx = x - points[i][0];
            final int dy = y - points[i][1];
            if (dx == 0 || dy == 0) {
                final int dist = Math.abs(dx + dy);
                if (dist < minDist) {
                    minDist = dist;
                    ans = i;
                }
            }
        }

        return ans;
    }
}
```

794. 1780.java

Path: LeetCode-main/solutions/1780. Check if Number is a Sum of Powers of Three/1780.java

```
class Solution {
    public boolean checkPowersOfThree(int n) {
        while (n > 1) {
            final int r = n % 3;
            if (r == 2)
                return false;
            n /= 3;
        }
        return true;
    }
}
```

795. 1781.java

Path: LeetCode-main/solutions/1781. Sum of Beauty of All Substrings/1781.java

```
class Solution {
    public int beautySum(String s) {
        int ans = 0;

        for (int i = 0; i < s.length(); ++i) {
            int[] count = new int[26];
            for (int j = i; j < s.length(); ++j) {
                ++count[s.charAt(j) - 'a'];
                ans += Arrays.stream(count).max().getAsInt() - getMinFreq(count);
            }
        }

        return ans;
    }

    // Returns the minimum frequency > 0.
    private int getMin(int[] count) {
        int minFreq = Integer.MAX_VALUE;
        for (final int freq : count)
            if (freq > 0)
                minFreq = min(minFreq, freq);
        return minFreq;
    }
}
```

796. 1782.java

Path: LeetCode-main/solutions/1782. Count Pairs Of Nodes/1782.java

```
class Solution {
    public int[] countPairs(int n, int[][] edges, int[] queries) {
        int[] ans = new int[queries.length];

        // count[i] := the number of edges of node i
        int[] count = new int[n + 1];

        // shared[i][j] := the number of edges incident to i or j, where i < j
        Map<Integer, Integer>[] shared = new Map[n + 1];

        for (int i = 1; i <= n; ++i)
            shared[i] = new HashMap<>();

        for (int[] edge : edges) {
            final int u = edge[0];
            final int v = edge[1];
            ++count[u];
            ++count[v];
            shared[Math.min(u, v)].merge(Math.max(u, v), 1, Integer::sum);
        }

        int[] sortedCount = count.clone();
        Arrays.sort(sortedCount);

        int k = 0;
        for (final int query : queries) {
            for (int i = 1, j = n; i < j;)
                if (sortedCount[i] + sortedCount[j] > query)
                    // sortedCount[i] + sortedCount[j] > query
                    // sortedCount[i + 1] + sortedCount[j] > query
                    // ...
                    // sortedCount[j - 1] + sortedCount[j] > query
                    // So, there are (j - 1) - i + 1 = j - i pairs > query
                    ans[k] += (j - i);
                else
                    ++i;
            for (int i = 1; i <= n; ++i)
                for (Map.Entry<Integer, Integer> p : shared[i].entrySet()) {
                    final int j = p.getKey();
                    final int sh = p.getValue();
                    if (count[i] + count[j] > query && count[i] + count[j] - sh <= query)
                        --ans[k];
                }
            ++k;
        }

        return ans;
    }
}
```


797. 1784.java

Path: LeetCode-main/solutions/1784. Check if Binary String Has at Most One Segment of Ones/1784.java

```
class Solution {  
    public boolean checkOnesSegment(String s) {  
        return !s.contains("01");  
    }  
}
```

798. 1785.java

Path: LeetCode-main/solutions/1785. Minimum Elements to Add to Form a Given Sum/1785.java

```
class Solution {  
    public int minElements(int[] nums, int limit, int goal) {  
        final long sum = Arrays.stream(nums).asLongStream().sum();  
        final double diff = Math.abs(goal - sum);  
        return (int) Math.ceil(diff / limit);  
    }  
}
```

799. 1786.java

Path: LeetCode-main/solutions/1786. Number of Restricted Paths From First to Last Node/1786.java

```
class Solution {
    public int countRestrictedPaths(int n, int[][] edges) {
        List<Pair<Integer, Integer>>[] graph = new List[n];
        Arrays.setAll(graph, i -> new ArrayList<>());

        for (int[] edge : edges) {
            final int u = edge[0] - 1;
            final int v = edge[1] - 1;
            final int w = edge[2];
            graph[u].add(new Pair<>(v, w));
            graph[v].add(new Pair<>(u, w));
        }

        return dijkstra(graph, 0, n - 1);
    }

    private int dijkstra(List<Pair<Integer, Integer>>[] graph, int src, int dst) {
        final int MOD = 1_000_000_007;
        // ways[i] := the number of restricted path from i to n
        long[] ways = new long[graph.length];
        // dist[i] := the distance to the last node of i
        long[] dist = new long[graph.length];
        Arrays.fill(dist, Long.MAX_VALUE);

        ways[dst] = 1;
        dist[dst] = 0;
        // (d, u)
        Queue<Pair<Long, Integer>> minHeap =
            new PriorityQueue<>(Comparator.comparingLong(Pair::getKey));
        minHeap.offer(new Pair<>(dist[dst], dst));

        while (!minHeap.isEmpty()) {
            final long d = minHeap.peek().getKey();
            final int u = minHeap.poll().getValue();
            if (d > dist[u])
                continue;
            for (Pair<Integer, Integer> pair : graph[u]) {
                final int v = pair.getKey();
                final int w = pair.getValue();
                if (d + w < dist[v]) {
                    dist[v] = d + w;
                    minHeap.offer(new Pair<>(dist[v], v));
                }
                if (dist[v] < dist[u]) {
                    ways[u] += ways[v];
                    ways[u] %= MOD;
                }
            }
        }

        return (int) ways[src];
    }
}
```

800. 1787.java

Path: LeetCode-main/solutions/1787. Make the XOR of All Segments Equal to Zero/1787.java

```
class Solution {
    public int minChanges(int[] nums, int k) {
        final int MAX = 1024;
        final int n = nums.length;
        // counts[i] := the counter that maps at the i-th position
        Map<Integer, Integer>[] counts = new Map[k];
        // dp[i][j] := the minimum number of elements to change s.t. XOR(nums[i..k - 1]) is j
        int[][] dp = new int[k][MAX];

        for (int i = 0; i < k; ++i)
            counts[i] = new HashMap<>();

        for (int i = 0; i < n; ++i)
            counts[i % k].merge(nums[i], 1, Integer::sum);

        Arrays.stream(dp).forEach(A -> Arrays.fill(A, n));

        // Initialize the DP array.
        for (int j = 0; j < MAX; ++j)
            dp[k - 1][j] = countAt(n, k, k - 1) - counts[k - 1].getOrDefault(j, 0);

        for (int i = k - 2; i >= 0; --i) {
            // The worst-case scenario is changing all the i-th position numbers to a
            // non-existent value in the current bucket.
            final int changeAll = countAt(n, k, i) + Arrays.stream(dp[i + 1]).min().getAsInt();
            for (int j = 0; j < MAX; ++j) {
                dp[i][j] = changeAll;
                for (Map.Entry<Integer, Integer> entry : counts[i].entrySet()) {
                    final int num = entry.getKey();
                    final int freq = entry.getValue();
                    // the cost to change every number in the i-th position to `num`
                    final int cost = countAt(n, k, i) - freq;
                    dp[i][j] = Math.min(dp[i][j], dp[i + 1][j ^ num] + cost);
                }
            }
        }

        return dp[0][0];
    }

    private int countAt(int n, int k, int i) {
        return n / k + (n % k > i ? 1 : 0);
    }
}
```

801. 1788.java

Path: LeetCode-main/solutions/1788. Maximize the Beauty of the Garden/1788.java

```
class Solution {
    public int maximumBeauty(int[] flowers) {
        int ans = Integer.MIN_VALUE;
        int prefix = 0;
        Map<Integer, Integer> flowerToPrefix = new HashMap<>();

        for (final int flower : flowers) {
            if (flowerToPrefix.containsKey(flower))
                ans = Math.max(ans, prefix - flowerToPrefix.get(flower) + flower * 2);
            prefix += Math.max(0, flower);
            flowerToPrefix.putIfAbsent(flower, prefix);
        }

        return ans;
    }
}
```

802. 179.java

Path: LeetCode-main/solutions/179. Largest Number/179.java

```
class Solution {
    public String largestNumber(int[] nums) {
        final String s = Arrays.stream(nums)
                                .mapToObj(String::valueOf)
                                .sorted((a, b) -> (b + a).compareTo(a + b))
                                .collect(Collectors.joining(""));
        return s.startsWith("00") ? "0" : s;
    }
}
```

803. 1790.java

Path: LeetCode-main/solutions/1790. Check if One String Swap Can Make Strings Equal/1790.java

```
class Solution {
    public boolean areAlmostEqual(String s1, String s2) {
        List<Integer> diffIndices = new ArrayList<>();

        for (int i = 0; i < s1.length(); ++i)
            if (s1.charAt(i) != s2.charAt(i))
                diffIndices.add(i);

        return diffIndices.isEmpty() ||
            (diffIndices.size() == 2 &&
             s1.charAt(diffIndices.get(0)) == s2.charAt(diffIndices.get(1)) &&
             s1.charAt(diffIndices.get(1)) == s2.charAt(diffIndices.get(0)));
    }
}
```

804. 1791.java

Path: LeetCode-main/solutions/1791. Find Center of Star Graph/1791.java

```
class Solution {  
    public int findCenter(int[][] edges) {  
        return edges[0][0] == edges[1][0] || edges[0][0] == edges[1][1] //  
            ? edges[0][0]  
            : edges[0][1];  
    }  
}
```


805. 1792.java

Path: LeetCode-main/solutions/1792. Maximum Average Pass Ratio/1792.java

```
class Solution {
    public double maxAverageRatio(int[][] classes, int extraStudents) {
        // (extra pass ratio, pass, total)
        PriorityQueue<T> maxHeap =
            new PriorityQueue<>((a, b) -> Double.compare(b.extraPassRatio, a.extraPassRatio));

        for (int[] c : classes) {
            final int pass = c[0];
            final int total = c[1];
            maxHeap.offer(new T(getExtraPassRatio(pass, total), pass, total));
        }

        for (int i = 0; i < extraStudents; ++i) {
            final int pass = maxHeap.peek().pass;
            final int total = maxHeap.poll().total;
            maxHeap.offer(new T(getExtraPassRatio(pass + 1, total + 1), pass + 1, total + 1));
        }

        double ratioSum = 0;

        while (!maxHeap.isEmpty())
            ratioSum += maxHeap.peek().pass / (double) maxHeap.poll().total;

        return ratioSum / classes.length;
    }

    // Returns the extra pass ratio if a brilliant student joins.
    private double getExtraPassRatio(int pass, int total) {
        return (pass + 1) / (double) (total + 1) - pass / (double) total;
    }

    private record T(double extraPassRatio, int pass, int total){};
}
```

806. 1793-2.java

Path: LeetCode-main/solutions/1793. Maximum Score of a Good Subarray/1793-2.java

```
class Solution {
    public int maximumScore(int[] nums, int k) {
        final int n = nums.length;
        int ans = nums[k];
        int mn = nums[k];
        int i = k;
        int j = k;

        // Greedily expand the window and decrease the minimum as slow as possible.
        while (i > 0 || j < n - 1) {
            if (i == 0)
                ++j;
            else if (j == n - 1)
                --i;
            else if (nums[i - 1] < nums[j + 1])
                ++j;
            else
                --i;
            mn = Math.min(mn, Math.min(nums[i], nums[j]));
            ans = Math.max(ans, mn * (j - i + 1));
        }

        return ans;
    }
}
```

807. 1793.java

Path: LeetCode-main/solutions/1793. Maximum Score of a Good Subarray/1793.java

```
class Solution {
    // Similar to 84. Largest Rectangle in Histogram
    public int maximumScore(int[] nums, int k) {
        int ans = 0;
        Deque<Integer> stack = new ArrayDeque<>();

        for (int i = 0; i <= nums.length; ++i) {
            while (!stack.isEmpty() && (i == nums.length || nums[stack.peek()] > nums[i])) {
                final int h = nums[stack.pop()];
                final int w = stack.isEmpty() ? i : i - stack.peek() - 1;
                if ((stack.isEmpty() || stack.peek() + 1 <= k) && i - 1 >= k)
                    ans = Math.max(ans, h * w);
            }
            stack.push(i);
        }

        return ans;
    }
}
```

808. 1794.java

Path: LeetCode-main/solutions/1794. Count Pairs of Equal Substrings With Minimum Difference/1794.java

```
class Solution {
    public int countQuadruples(String s1, String s2) {
        // To minimize j - a, the length of the substring should be 1. This is
        // because for substrings with a size greater than 1, a will decrease,
        // causing j - a to become larger.
        int ans = 0;
        int diff = Integer.MAX_VALUE; // diff := j - a
        int[] firstJ = new int[26];
        int[] lastA = new int[26];
        Arrays.fill(firstJ, -1);
        Arrays.fill(lastA, -1);

        for (int j = s1.length() - 1; j >= 0; --j)
            firstJ[s1.charAt(j) - 'a'] = j;

        for (int a = 0; a < s2.length(); ++a)
            lastA[s2.charAt(a) - 'a'] = a;

        for (int i = 0; i < 26; ++i) {
            if (firstJ[i] == -1 || lastA[i] == -1)
                continue;
            if (firstJ[i] - lastA[i] < diff) {
                diff = firstJ[i] - lastA[i];
                ans = 0;
            }
            if (firstJ[i] - lastA[i] == diff)
                ++ans;
        }

        return ans;
    }
}
```

809. 1796.java

Path: LeetCode-main/solutions/1796. Second Largest Digit in a String/1796.java

```
class Solution {
    public int secondHighest(String s) {
        int maxDigit = -1;
        int secondMaxDigit = -1;

        for (final char c : s.toCharArray())
            if (Character.isDigit(c)) {
                final int digit = Character.getNumericValue(c);
                if (digit > maxDigit) {
                    secondMaxDigit = maxDigit;
                    maxDigit = digit;
                } else if (maxDigit > digit && digit > secondMaxDigit) {
                    secondMaxDigit = digit;
                }
            }

        return secondMaxDigit;
    }
}
```

810. 1797.java

Path: LeetCode-main/solutions/1797. Design Authentication Manager/1797.java

```
public class AuthenticationManager {
    public AuthenticationManager(int timeToLive) {
        this.timeToLive = timeToLive;
    }

    public void generate(String tokenId, int currentTime) {
        tokenIdToExpiryTime.put(tokenId, currentTime);
        times.add(currentTime);
    }

    public void renew(String tokenId, int currentTime) {
        Integer expiryTime = tokenIdToExpiryTime.get(tokenId);
        if (expiryTime == null || currentTime >= expiryTime + timeToLive)
            return;
        times.remove(expiryTime);
        tokenIdToExpiryTime.put(tokenId, currentTime);
        times.add(currentTime);
    }

    public int countUnexpiredTokens(int currentTime) {
        final int expiryTimeThreshold = currentTime - timeToLive + 1;
        // Remove expired tokens.
        times.headSet(expiryTimeThreshold).clear();
        return times.size();
    }

    private final int timeToLive;
    private final Map<String, Integer> tokenIdToExpiryTime = new HashMap<>();
    private final TreeSet<Integer> times = new TreeSet<>();
}
```

811. 1798.java

Path: LeetCode-main/solutions/1798. Maximum Number of Consecutive Values You Can Make/1798.java

```
class Solution {
    public int getMaximumConsecutive(int[] coins) {
        int ans = 1; // the next value we want to make

        Arrays.sort(coins);

        for (final int coin : coins) {
            if (coin > ans)
                return ans;
            ans += coin;
        }

        return ans;
    }
}
```

812. 1799.java

Path: LeetCode-main/solutions/1799. Maximize Score After N Operations/1799.java

```
class Solution {
    public int maxScore(int[] nums) {
        final int n = nums.length / 2;
        int[][] mem = new int[n + 1][1 << (n * 2)];
        return maxScore(nums, 1, 0, mem);
    }

    // Returns the maximum score you can receive after performing the k to n
    // operations, where `mask` is the bitmask of the chosen numbers.
    private int maxScore(int[] nums, int k, int mask, int[][] mem) {
        if (k == mem.length)
            return 0;
        if (mem[k][mask] > 0)
            return mem[k][mask];

        for (int i = 0; i < nums.length; ++i)
            for (int j = i + 1; j < nums.length; ++j) {
                final int chosenMask = 1 << i | 1 << j;
                if ((mask & chosenMask) == 0)
                    mem[k][mask] = Math.max(mem[k][mask], k * gcd(nums[i], nums[j]) +
                                                maxScore(nums, k + 1, mask | chosenMask, mem));
            }

        return mem[k][mask];
    }

    private int gcd(int a, int b) {
        return b == 0 ? a : gcd(b, a % b);
    }
}
```


813. 18.java

Path: LeetCode-main/solutions/18. 4Sum/18.java

```
class Solution {
    public List<List<Integer>> fourSum(int[] nums, int target) {
        List<List<Integer>> ans = new ArrayList<>();
        Arrays.sort(nums);
        nSum(nums, 4, target, 0, nums.length - 1, new ArrayList<>(), ans);
        return ans;
    }

    // Finds n numbers that add up to the target in [l, r].
    private void nSum(int[] nums, long n, long target, int l, int r, List<Integer> path,
        List<List<Integer>> ans) {
        if (r - l + 1 < n || target < nums[l] * n || target > nums[r] * n)
            return;
        if (n == 2) {
            // Similar to the sub procedure in 15. 3Sum
            while (l < r) {
                final int sum = nums[l] + nums[r];
                if (sum == target) {
                    path.add(nums[l]);
                    path.add(nums[r]);
                    ans.add(new ArrayList<>(path));
                    path.remove(path.size() - 1);
                    path.remove(path.size() - 1);
                    ++l;
                    --r;
                    while (l < r && nums[l] == nums[l - 1])
                        ++l;
                    while (l < r && nums[r] == nums[r + 1])
                        --r;
                } else if (sum < target) {
                    ++l;
                } else {
                    --r;
                }
            }
            return;
        }
        for (int i = l; i <= r; ++i) {
            if (i > l && nums[i] == nums[i - 1])
                continue;
            path.add(nums[i]);
            nSum(nums, n - 1, target - nums[i], i + 1, r, path, ans);
            path.remove(path.size() - 1);
        }
    }
}
```

814. 1800.java

Path: LeetCode-main/solutions/1800. Maximum Ascending Subarray Sum/1800.java

```
class Solution {
    public int maxAscendingSum(int[] nums) {
        int ans = 0;
        int sum = nums[0];

        for (int i = 1; i < nums.length; ++i)
            if (nums[i] > nums[i - 1]) {
                sum += nums[i];
            } else {
                ans = Math.max(ans, sum);
                sum = nums[i];
            }

        return Math.max(ans, sum);
    }
}
```

815. 1801.java

Path: LeetCode-main/solutions/1801. Number of Orders in the Backlog/1801.java

```
class Solution {
    public int getNumberOfBacklogOrders(int[][] orders) {
        final int MOD = 1_000_000_007;
        int ans = 0;
        Queue<int[]> buysMaxHeap = new PriorityQueue<>(Comparator.comparingInt(a -> - a[0]));
        Queue<int[]> sellsMinHeap = new PriorityQueue<>(Comparator.comparingInt(a -> a[0]));

        for (int[] order : orders) {
            if (order[2] == 0)
                buysMaxHeap.offer(order);
            else
                sellsMinHeap.offer(order);
            while (!buysMaxHeap.isEmpty() && !sellsMinHeap.isEmpty() &&
                buysMaxHeap.peek()[0] >= sellsMinHeap.peek()[0]) {
                final int minAmount = Math.min(buysMaxHeap.peek()[1], sellsMinHeap.peek()[1]);
                buysMaxHeap.peek()[1] -= minAmount;
                sellsMinHeap.peek()[1] -= minAmount;
                if (buysMaxHeap.peek()[1] == 0)
                    buysMaxHeap.poll();
                if (sellsMinHeap.peek()[1] == 0)
                    sellsMinHeap.poll();
            }
        }

        while (!buysMaxHeap.isEmpty()) {
            ans += buysMaxHeap.poll()[1];
            ans %= MOD;
        }

        while (!sellsMinHeap.isEmpty()) {
            ans += sellsMinHeap.poll()[1];
            ans %= MOD;
        }

        return ans;
    }
}
```

816. 1802.java

Path: LeetCode-main/solutions/1802. Maximum Value at a Given Index in a Bounded Array/1802.java

```
class Solution {
    public int maxValue(int n, int index, int maxSum) {
        maxSum -= n;

        int l = 0;
        int r = maxSum;

        // Find the first value x s.t. if A[index] = x, then sum(A) >= maxSum.
        while (l < r) {
            final int m = (l + r) / 2;
            if (getSum(n, index, m) >= maxSum)
                r = m;
            else
                l = m + 1;
        }

        return getSum(n, index, l) > maxSum ? l : l + 1;
    }

    // Returns the minimum sum if nums[index] = x.
    private long getSum(int n, int index, int x) {
        long l = Math.min(index, x - 1);
        long r = Math.min(n - index, x);
        long lSum = ((x - 1) + (x - 1 - l + 1)) * l / 2;
        long rSum = (x + (x - r + 1)) * r / 2;
        return lSum + rSum;
    }
}
```

817. 1803.java

Path: LeetCode-main/solutions/1803. Count Pairs With XOR in a Range/1803.java

```
class TrieNode {
    public TrieNode[] children = new TrieNode[2];
    public int count = 0;
}

class Solution {
    public int countPairs(int[] nums, int low, int high) {
        int ans = 0;

        for (final int num : nums) {
            ans += getCount(num, high + 1) - getCount(num, low);
            insert(num);
        }

        return ans;
    }

    private final int HEIGHT = 14;
    private TrieNode root = new TrieNode();

    private void insert(int num) {
        TrieNode node = root;
        for (int i = HEIGHT; i >= 0; --i) {
            final int bit = num >> i & 1;
            if (node.children[bit] == null)
                node.children[bit] = new TrieNode();
            node = node.children[bit];
            ++node.count;
        }
    }

    // Returns the number of numbers < limit.
    private int getCount(int num, int limit) {
        int count = 0;
        TrieNode node = root;
        for (int i = HEIGHT; i >= 0; --i) {
            final int bit = num >> i & 1;
            final int bitLimit = ((limit >> i) & 1);
            if (bitLimit == 1) {
                if (node.children[bit] != null)
                    count += node.children[bit].count;
                node = node.children[bit ^ 1];
            } else {
                node = node.children[bit];
            }
            if (node == null)
                break;
        }
        return count;
    }
}
```

818. 1804.java

Path: LeetCode-main/solutions/1804. Implement Trie II (Prefix Tree)/1804.java

```
class TrieNode {
    public TrieNode[] children = new TrieNode[26];
    public int prefixCount = 0;
    public int wordCount = 0;
}

class Trie {
    public void insert(final String word) {
        TrieNode node = root;
        for (final char c : word.toCharArray()) {
            final int i = c - 'a';
            if (node.children[i] == null)
                node.children[i] = new TrieNode();
            node = node.children[i];
            ++node.prefixCount;
        }
        ++node.wordCount;
    }

    public int countWordsEqualTo(final String word) {
        TrieNode node = find(word);
        return node == null ? 0 : node.wordCount;
    }

    public int countWordsStartingWith(final String prefix) {
        TrieNode node = find(prefix);
        return node == null ? 0 : node.prefixCount;
    }

    public void erase(final String word) {
        TrieNode node = root;
        for (final char c : word.toCharArray()) {
            final int i = c - 'a';
            node = node.children[i];
            --node.prefixCount;
        }
        --node.wordCount;
    }

    private TrieNode root = new TrieNode();

    private TrieNode find(final String s) {
        TrieNode node = root;
        for (final char c : s.toCharArray()) {
            final int i = c - 'a';
            if (node.children[i] == null)
                return null;
            node = node.children[i];
        }
        return node;
    }
}
```

819. 1805.java

Path: LeetCode-main/solutions/1805. Number of Different Integers in a String/1805.java

```
class Solution {
    public int numDifferentIntegers(String word) {
        HashSet<String> nums = new HashSet<>();
        StringBuilder sb = new StringBuilder();

        for (final char c : word.toCharArray())
            if (Character.isDigit(c)) {
                sb.append(c);
            } else if (sb.length() > 0) {
                nums.add(removeLeadingZeros(sb.toString()));
                sb = new StringBuilder();
            }

        if (sb.length() > 0)
            nums.add(removeLeadingZeros(sb.toString()));
        return nums.size();
    }

    private String removeLeadingZeros(final String s) {
        int index = 0;
        while (index < s.length() && s.charAt(index) == '0')
            ++index;
        return index == s.length() ? "0" : s.substring(index);
    }
}
```

820. 1806.java

Path: LeetCode-main/solutions/1806. Minimum Number of Operations to Reinitialize a Permutation/1806.java

```
class Solution {
    public int reinitializePermutation(int n) {
        int ans = 0;
        int i = 1;

        do {
            if (i < n / 2)
                i = i * 2;
            else
                i = (i - n / 2) * 2 + 1;
            ++ans;
        } while (i != 1);

        return ans;
    }
}
```


821. 1807.java

Path: LeetCode-main/solutions/1807. Evaluate the Bracket Pairs of a String/1807.java

```
class Solution {
    public String evaluate(String s, List<List<String>> knowledge) {
        StringBuilder sb = new StringBuilder();
        Map<String, String> map = new HashMap<>();

        for (List<String> list : knowledge)
            map.put("(" + list.get(0) + ")", list.get(1));

        for (int i = 0; i < s.length(); ++i) {
            final char c = s.charAt(i);
            if (c == '(') {
                final int j = s.indexOf(')', i);
                sb.append(map.getOrDefault(s.substring(i, j + 1), "?"));
                i = j;
            } else {
                sb.append(c);
            }
        }

        return sb.toString();
    }
}
```

822. 1808.java

Path: LeetCode-main/solutions/1808. Maximize Number of Nice Divisors/1808.java

```
class Solution {
    public int maxNiceDivisors(int primeFactors) {
        if (primeFactors <= 3)
            return primeFactors;
        if (primeFactors % 3 == 0)
            return (int) (modPow(3, primeFactors / 3) % MOD);
        if (primeFactors % 3 == 1)
            return (int) (4 * modPow(3, (primeFactors - 4) / 3) % MOD);
        return (int) (2 * modPow(3, (primeFactors - 2) / 3) % MOD);
    }

    private static final int MOD = 1_000_000_007;

    private long modPow(long x, long n) {
        if (n == 0)
            return 1;
        if (n % 2 == 1)
            return x * modPow(x, n - 1) % MOD;
        return modPow(x * x % MOD, n / 2);
    }
}
```

823. 1810.java

Path: LeetCode-main/solutions/1810. Minimum Path Cost in a Hidden Grid/1810.java

```
/**
 * // This is the GridMaster's API interface.
 * // You should not implement it, or speculate about its implementation
 * class GridMaster {
 *     boolean canMove(char direction);
 *     int move(char direction);
 *     boolean isTarget();
 * }
 */

class Solution {
    public int findShortestPath(GridMaster master) {
        final int m = 100;
        final int startX = m;
        final int startY = m;
        int[] target = {m * 2, m * 2};
        int[][] grid = new int[m * 2][m * 2];
        boolean[][] seen = new boolean[m * 2][m * 2];
        Arrays.stream(grid).forEach(A -> Arrays.fill(A, -1));

        // Build the grid information by DFS.
        dfs(master, grid, startX, startY, target);

        Queue<int[]> minHeap = new PriorityQueue<>(Comparator.comparingInt(a -> a[2])) {
            { offer(new int[] {startX, startY, 0}); }
        };

        // Find the steps by BFS.
        while (!minHeap.isEmpty()) {
            final int i = minHeap.peek()[0];
            final int j = minHeap.peek()[1];
            final int cost = minHeap.poll()[2];
            if (i == target[0] && j == target[1])
                return cost;
            if (seen[i][j])
                continue;
            seen[i][j] = true;
            for (int[] dir : DIRS) {
                final int x = i + dir[0];
                final int y = j + dir[1];
                if (x < 0 || x == 2 * m || y < 0 || y == 2 * m)
                    continue;
                if (seen[x][y] || grid[x][y] == -1)
                    continue;
                final int nextCost = cost + grid[x][y];
                minHeap.offer(new int[] {x, y, nextCost});
            }
        }

        return -1;
    }

    private static final int[][] DIRS = {{0, 1}, {1, 0}, {0, -1}, {-1, 0}};
    private static final char[] charTable = {'R', 'D', 'L', 'U'};

    private void dfs(GridMaster master, int[][] grid, int i, int j, int[] target) {
        if (master.isTarget()) {
            target[0] = i;
            target[1] = j;
        }

        for (int k = 0; k < 4; ++k) {
            final int x = i + DIRS[k][0];
            final int y = j + DIRS[k][1];
            final char d = charTable[k];
            final char undoD = charTable[(k + 2) % 4];
            if (master.canMove(d) && grid[x][y] == -1) {
                grid[x][y] = master.move(d);
                dfs(master, grid, x, y, target);
                master.move(undoD);
            }
        }
    }
}
```

824. 1812.java

Path: LeetCode-main/solutions/1812. Determine Color of a Chessboard Square/1812.java

```
class Solution {  
    public boolean squareIsWhite(String coordinates) {  
        final char letter = coordinates.charAt(0);  
        final char digit = coordinates.charAt(1);  
        return letter % 2 != digit % 2;  
    }  
}
```

825. 1813.java

Path: LeetCode-main/solutions/1813. Sentence Similarity III/1813.java

```
class Solution {
    public boolean areSentencesSimilar(String sentence1, String sentence2) {
        if (sentence1.length() == sentence2.length())
            return sentence1.equals(sentence2);

        String[] words1 = sentence1.split(" ");
        String[] words2 = sentence2.split(" ");
        final int m = words1.length;
        final int n = words2.length;
        if (m > n)
            return areSentencesSimilar(sentence2, sentence1);

        int i = 0; // words1's index
        while (i < m && words1[i].equals(words2[i]))
            ++i;
        while (i < m && words1[i].equals(words2[i + n - m]))
            ++i;
        return i == m;
    }
}
```

826. 1814.java

Path: LeetCode-main/solutions/1814. Count Nice Pairs in an Array/1814.java

```
class Solution {
    public int countNicePairs(int[] nums) {
        final int MOD = 1_000_000_007;
        long ans = 0;
        Map<Integer, Long> count = new HashMap<>();

        for (final int num : nums)
            count.merge(num - rev(num), 1L, Long::sum);

        for (final long freq : count.values()) {
            ans += freq * (freq - 1) / 2;
            ans %= MOD;
        }

        return (int) ans;
    }

    private int rev(int n) {
        int x = 0;
        while (n > 0) {
            x = x * 10 + (n % 10);
            n /= 10;
        }
        return x;
    }
}
```

827. 1815.java

Path: LeetCode-main/solutions/1815. Maximum Number of Groups Getting Fresh Donuts/1815.java

```
class Solution {
    public int maxHappyGroups(int batchSize, int[] groups) {
        int happy = 0;
        int[] freq = new int[batchSize];

        for (int g : groups) {
            g %= batchSize;
            if (g == 0) {
                ++happy;
            } else if (freq[batchSize - g] > 0) {
                --freq[batchSize - g];
                ++happy;
            } else {
                ++freq[g];
            }
        }

        return happy + dp(freq, 0, batchSize);
    }

    private Map<String, Integer> mem = new HashMap<>();

    // Returns the maximum number of partitions can be formed.
    private int dp(int[] freq, int remainder, int batchSize) {
        final String hashed = hash(freq);
        if (mem.containsKey(hashed))
            return mem.get(hashed);

        int ans = 0;

        if (Arrays.stream(freq).anyMatch(f -> f != 0)) {
            for (int i = 0; i < freq.length; ++i)
                if (freq[i] > 0) {
                    int[] newFreq = freq.clone();
                    --newFreq[i];
                    ans = Math.max(ans, dp(newFreq, (remainder + i) % batchSize, batchSize));
                }
            if (remainder == 0)
                ++ans;
        }

        mem.put(hash(freq), ans);
        return ans;
    }

    private String hash(int[] freq) {
        StringBuilder sb = new StringBuilder();
        for (final int f : freq)
            sb.append("#").append(f);
        return sb.toString();
    }
}
```

828. 1816.java

Path: LeetCode-main/solutions/1816. Truncate Sentence/1816.java

```
class Solution {  
    public String truncateSentence(String s, int k) {  
        String[] words = s.split(" ");  
        String[] truncated = new String[k];  
  
        for (int i = 0; i < k; ++i)  
            truncated[i] = words[i];  
  
        return String.join(" ", truncated);  
    }  
}
```


829. 1817.java

Path: LeetCode-main/solutions/1817. Finding the Users Active Minutes/1817.java

```
class Solution {
    public int[] findingUsersActiveMinutes(int[][] logs, int k) {
        int[] ans = new int[k];
        Map<Integer, Set<Integer>> idToTimes = new HashMap<>();

        for (int[] log : logs) {
            idToTimes.putIfAbsent(log[0], new HashSet<>());
            idToTimes.get(log[0]).add(log[1]);
        }

        for (final int id : idToTimes.keySet())
            ++ans[idToTimes.get(id).size() - 1];

        return ans;
    }
}
```

830. 1818.java

Path: LeetCode-main/solutions/1818. Minimum Absolute Sum Difference/1818.java

```
class Solution {
    public int minAbsoluteSumDiff(int[] nums1, int[] nums2) {
        final int MOD = 1_000_000_007;
        long sumDiff = 0;
        long maxDecrement = 0;
        TreeSet<Integer> sorted = new TreeSet<>();

        for (final int num : nums1)
            sorted.add(num);

        for (int i = 0; i < nums1.length; ++i) {
            final long currDiff = (long) Math.abs(nums1[i] - nums2[i]);
            sumDiff += currDiff;
            Integer ceiling = sorted.ceiling(nums2[i]);
            Integer floor = sorted.floor(nums2[i]);
            if (ceiling != null)
                maxDecrement = Math.max(maxDecrement, currDiff - (long) Math.abs(ceiling - nums2[i]));
            if (floor != null)
                maxDecrement = Math.max(maxDecrement, currDiff - (long) Math.abs(floor - nums2[i]));
        }

        return (int) ((sumDiff - maxDecrement) % MOD);
    }
}
```

831. 1819.java

Path: LeetCode-main/solutions/1819. Number of Different Subsequences GCDs/1819.java

```
class Solution {
    public int countDifferentSubsequenceGCDs(int[] nums) {
        final int maxNum = Arrays.stream(nums).max().getAsInt();
        int ans = 0;
        // factor[i] := the GCD of numbers having factor i
        int[] factor = new int[maxNum + 1];

        for (final int num : nums)
            for (int i = 1; i * i <= num; ++i)
                if (num % i == 0) {
                    final int j = num / i;
                    factor[i] = gcd(factor[i], num);
                    factor[j] = gcd(factor[j], num);
                }

        for (int i = 1; i <= maxNum; ++i)
            if (factor[i] == i)
                ++ans;

        return ans;
    }

    private int gcd(int a, int b) {
        return b == 0 ? a : gcd(b, a % b);
    }
}
```

832. 1820.java

Path: LeetCode-main/solutions/1820. Maximum Number of Accepted Invitations/1820.java

```
class Solution {
    public int maximumInvitations(int[][] grid) {
        final int m = grid.length;
        final int n = grid[0].length;
        int ans = 0;
        int[] mates = new int[n]; // mates[i] := the i-th girl's mate

        Arrays.fill(mates, -1);

        for (int i = 0; i < m; ++i)
            if (canInvite(grid, i, new boolean[n], mates))
                ++ans;

        return ans;
    }

    // Returns true if the i-th boy can make an invitation.
    private boolean canInvite(int[][] grid, int i, boolean[] seen, int[] mates) {
        // The i-th boy asks each girl.
        for (int j = 0; j < seen.length; ++j) {
            if (grid[i][j] == 0 || seen[j])
                continue;
            seen[j] = true;
            if (mates[j] == -1 || canInvite(grid, mates[j], seen, mates)) {
                mates[j] = i; // Match the j-th girl with i-th boy.
                return true;
            }
        }

        return false;
    }
}
```

833. 1822.java

Path: LeetCode-main/solutions/1822. Sign of the Product of an Array/1822.java

```
class Solution {  
    public int arraySign(int[] nums) {  
        int sign = 1;  
  
        for (final int num : nums) {  
            if (num == 0)  
                return 0;  
            if (num < 0)  
                sign = -sign;  
        }  
  
        return sign;  
    }  
}
```

834. 1823-2.java

Path: LeetCode-main/solutions/1823. Find the Winner of the Circular Game/1823-2.java

```
class Solution {
    public int findTheWinner(int n, int k) {
        // Converts back to 1-indexed.
        return f(n, k) + 1;
    }

    // e.g. n = 4, k = 2.
    // By using 0-indexed notation, we have the following circle:
    //
    // 0 -> 1 -> 2 -> 3 -> 0
    //      x
    //      0 -> 1 -> 2 -> 0
    //
    // After the first round, 1 is removed.
    // So, 2 becomes 0, 3 becomes 1, and 0 becomes 2.
    // Let's denote that oldIndex = f(n, k) and newIndex = f(n - 1, k).
    // By observation, we know f(n, k) = (f(n - 1, k) + k) % n.
    private int f(int n, int k) {
        if (n == 1)
            return 0;
        return (f(n - 1, k) + k) % n;
    }
}
```

835. 1823-3.java

Path: LeetCode-main/solutions/1823. Find the Winner of the Circular Game/1823-3.java

```
class Solution {
    public int findTheWinner(int n, int k) {
        // Converts back to 1-indexed.
        return f(n, k) + 1;
    }

    // e.g. n = 4, k = 2.
    // By using 0-indexed notation, we have the following circle:
    //
    // 0 -> 1 -> 2 -> 3 -> 0
    //      x
    //      0 -> 1 -> 2 -> 0
    //
    // After the first round, 1 is removed.
    // So, 2 becomes 0, 3 becomes 1, and 0 becomes 2.
    // Let's denote that oldIndex = f(n, k) and newIndex = f(n - 1, k).
    // By observation, we know f(n, k) = (f(n - 1, k) + k) % n.
    private int f(int n, int k) {
        int ans = 0; // f(1, k)
        // Computes f(i, k) based on f(i - 1, k).
        for (int i = 2; i <= n; ++i)
            ans = (ans + k) % i;
        return ans;
    }
}
```

836. 1823.java

Path: LeetCode-main/solutions/1823. Find the Winner of the Circular Game/1823.java

```
class Solution {
    public int findTheWinner(int n, int k) {
        // friends[i] := true if i-th friend is left
        boolean[] friends = new boolean[n];

        int friendCount = n;
        int fp = n; // friends' index

        while (friendCount > 1) {
            for (int i = 0; i < k; ++i, ++fp)
                while (friends[fp % n]) // The friend is not there.
                    ++fp; // Point to the next one.
            friends[(fp - 1) % n] = true;
            --friendCount;
        }

        for (fp = 0; friends[fp]; ++fp)
            ;
        return fp + 1;
    }
}
```


837. 1824.java

Path: LeetCode-main/solutions/1824. Minimum Sideway Jumps/1824.java

```
class Solution {
    public int minSideJumps(int[] obstacles) {
        final int INF = 1_000_000;
        // dp[i] := the minimum jump to reach the i-th lane
        int[] dp = {INF, 1, 0, 1};

        for (final int obstacle : obstacles) {
            if (obstacle > 0)
                dp[obstacle] = INF;
            for (int i = 1; i <= 3; ++i) // the current
                if (i != obstacle)
                    for (int j = 1; j <= 3; ++j) // the previous
                        dp[i] = Math.min(dp[i], dp[j] + (i == j ? 0 : 1));
        }

        return Arrays.stream(dp).min().getAsInt();
    }
}
```

838. 1825-2.java

Path: LeetCode-main/solutions/1825. Finding MK Average/1825-2.java

```
class MyMap {
    public TreeMap<Integer, Integer> map = new TreeMap<>();
    public int size = 0;
    public long sum = 0;
}

class MKAverage {
    public MKAverage(int m, int k) {
        this.m = m;
        this.k = k;
        this.MID_SIZE = m - 2 * k;
    }

    public void addElement(int num) {
        q.offer(num);
        add(num);

        if (q.size() > m)
            remove(q.poll());
    }

    public int calculateMKAverage() {
        return q.size() == m ? (int) (mid.sum / MID_SIZE) : -1;
    }

    private final int m;
    private final int k;
    private final int MID_SIZE;
    private Queue<Integer> q = new ArrayDeque<>();
    private MyMap bot = new MyMap();
    private MyMap mid = new MyMap();
    private MyMap top = new MyMap();

    private void add(int num) {
        add(bot, num);
        if (bot.size > k)
            add(mid, remove(bot, bot.map.lastKey()));
        if (mid.size > MID_SIZE)
            add(top, remove(mid, mid.map.lastKey()));
    }

    private void remove(int num) {
        if (bot.map.containsKey(num))
            remove(bot, num);
        else if (mid.map.containsKey(num))
            remove(mid, num);
        else
            remove(top, num);

        if (bot.size < k)
            add(bot, remove(mid, mid.map.firstKey()));
        if (mid.size < MID_SIZE)
            add(mid, remove(top, top.map.firstKey()));
    }

    private void add(MyMap m, int num) {
        m.map.merge(num, 1, Integer::sum);
        ++m.size;
        m.sum += num;
    }

    private int remove(MyMap m, int num) {
        m.map.merge(num, -1, Integer::sum);
        if (m.map.get(num) == 0)
            m.map.remove(num);
        --m.size;
        m.sum -= num;
        return num;
    }
}
```

839. 1825.java

Path: LeetCode-main/solutions/1825. Finding MK Average/1825.java

```
class MKAverage {
    public MKAverage(int m, int k) {
        this.m = m;
        this.k = k;
    }

    public void addElement(int num) {
        q.offer(num);
        add(mid, num);
        midSum += num;

        if (q.size() > m) {
            final int removed = q.poll();
            if (top.containsKey(removed)) {
                remove(top, removed);
                --topSize;
            } else if (mid.containsKey(removed)) {
                remove(mid, removed);
                midSum -= removed;
            } else {
                remove(bot, removed);
                --botSize;
            }
        }

        // Move item(s) from `mid` to `top` to fill k slots.
        while (!mid.isEmpty() && topSize < k) {
            midSum -= mid.lastKey();
            add(top, remove(mid, mid.lastKey()));
            ++topSize;
        }

        // Rebalance `mid` and `top`.
        while (!mid.isEmpty() && mid.lastKey() > top.firstKey()) {
            midSum -= mid.lastKey();
            midSum += top.firstKey();
            add(top, remove(mid, mid.lastKey()));
            add(mid, remove(top, top.firstKey()));
        }

        // Move item(s) from `mid` to `bot` to fill k slots.
        while (!mid.isEmpty() && botSize < k) {
            midSum -= mid.firstKey();
            add(bot, remove(mid, mid.firstKey()));
            ++botSize;
        }

        // Rebalance mid and bot
        while (!mid.isEmpty() && mid.firstKey() < bot.lastKey()) {
            midSum -= mid.firstKey();
            midSum += bot.lastKey();
            add(bot, remove(mid, mid.firstKey()));
            add(mid, remove(bot, bot.lastKey()));
        }
    }

    public int calculateMKAverage() {
        return q.size() == m ? (int) (midSum / (m - 2 * k)) : -1;
    }

    private final int m;
    private final int k;
    private Queue<Integer> q = new ArrayDeque<>();
    private TreeMap<Integer, Integer> top = new TreeMap<>();
    private TreeMap<Integer, Integer> mid = new TreeMap<>();
    private TreeMap<Integer, Integer> bot = new TreeMap<>();
    private int topSize = 0;
    private int botSize = 0;
    private long midSum = 0;

    private void add(TreeMap<Integer, Integer> map, int num) {
        map.merge(num, 1, Integer::sum);
    }

    private int remove(TreeMap<Integer, Integer> map, int num) {
        map.put(num, map.get(num) - 1);
        if (map.get(num) == 0)
            map.remove(num);
        return num;
    }
}
```


840. 1826.java

Path: LeetCode-main/solutions/1826. Faulty Sensor/1826.java

```
class Solution {
    public int badSensor(int[] sensor1, int[] sensor2) {
        final boolean oneDefect = canReplace(sensor2, sensor1);
        final boolean twoDefect = canReplace(sensor1, sensor2);
        if (oneDefect && twoDefect)
            return -1;
        if (!oneDefect && !twoDefect)
            return -1;
        return oneDefect ? 1 : 2;
    }

    private boolean canReplace(int[] A, int[] B) {
        int i = 0; // A's index
        int j = 0; // B's index
        int droppedValue = -1;

        while (i < A.length)
            if (A[i] == B[j]) {
                ++i;
                ++j;
            } else {
                droppedValue = A[i];
                ++i;
            }

        return j == B.length - 1 && B[j] != droppedValue;
    }
}
```

841. 1827.java

Path: LeetCode-main/solutions/1827. Minimum Operations to Make the Array Increasing/1827.java

```
class Solution {
    public int minOperations(int[] nums) {
        int ans = 0;
        int last = 0;

        for (final int num : nums) {
            ans += Math.max(0, last - num + 1);
            last = Math.max(num, last + 1);
        }

        return ans;
    }
}
```

842. 1828.java

Path: LeetCode-main/solutions/1828. Queries on Number of Points Inside a Circle/1828.java

```
class Solution {
    public int[] countPoints(int[][] points, int[][] queries) {
        int[] ans = new int[queries.length];

        for (int i = 0; i < queries.length; ++i) {
            final int xj = queries[i][0];
            final int yj = queries[i][1];
            final int rj = queries[i][2];
            int count = 0;
            for (int[] point : points) {
                final int xi = point[0];
                final int yi = point[1];
                if (squared(xi - xj) + squared(yi - yj) <= squared(rj))
                    ++count;
            }
            ans[i] = count;
        }

        return ans;
    }

    private int squared(int x) {
        return x * x;
    }
}
```

843. 1829.java

Path: LeetCode-main/solutions/1829. Maximum XOR for Each Query/1829.java

```
class Solution {
    public int[] getMaximumXor(int[] nums, int maximumBit) {
        final int n = nums.length;
        final int mx = (1 << maximumBit) - 1;
        int[] ans = new int[n];
        int xors = 0;

        for (int i = 0; i < n; ++i) {
            xors ^= nums[i];
            ans[n - 1 - i] = xors ^ mx;
        }

        return ans;
    }
}
```


844. 1830.java

Path: LeetCode-main/solutions/1830. Minimum Number of Operations to Make String Sorted/1830.java

```
class Solution {
    public int makeStringSorted(String s) {
        final int MOD = 1_000_000_007;
        final int n = s.length();
        final long[][] factAndInvFact = getFactAndInvFact(n);
        final long[] fact = factAndInvFact[0];
        final long[] invFact = factAndInvFact[1];
        int ans = 0;
        int[] count = new int[26];

        for (int i = n - 1; i >= 0; --i) {
            final int order = s.charAt(i) - 'a';
            ++count[order];
            long perm = Arrays.stream(count, 0, order).sum() * fact[n - 1 - i] % MOD;
            for (int j = 0; j < 26; ++j)
                perm = perm * invFact[count[j]] % MOD;
            ans += perm;
            ans %= MOD;
        }

        return ans;
    }

    private static final int MOD = 1_000_000_007;

    private long[][] getFactAndInvFact(int n) {
        long[] fact = new long[n + 1];
        long[] invFact = new long[n + 1];
        long[] inv = new long[n + 1];
        fact[0] = invFact[0] = 1;
        inv[0] = inv[1] = 1;
        for (int i = 1; i <= n; ++i) {
            if (i >= 2)
                inv[i] = MOD - MOD / i * inv[MOD % i] % MOD;
            fact[i] = fact[i - 1] * i % MOD;
            invFact[i] = invFact[i - 1] * inv[i] % MOD;
        }
        return new long[][] {fact, invFact};
    }
}
```

845. 1832-2.java

Path: LeetCode-main/solutions/1832. Check if the Sentence Is Pangram/1832-2.java

```
class Solution {  
    public boolean checkIfPangram(String sentence) {  
        int seen = 0;  
  
        for (final char c : sentence.toCharArray())  
            seen |= 1 << c - 'a';  
  
        return seen == (1 << 26) - 1;  
    }  
}
```

846. 1832.java

Path: LeetCode-main/solutions/1832. Check if the Sentence Is Pangram/1832.java

```
class Solution {  
    public boolean checkIfPangram(String sentence) {  
        Set<Character> seen = new HashSet<>();  
  
        for (final char c : sentence.toCharArray())  
            seen.add(c);  
  
        return seen.size() == 26;  
    }  
}
```

847. 1833.java

Path: LeetCode-main/solutions/1833. Maximum Ice Cream Bars/1833.java

```
class Solution {  
    public int maxIceCream(int[] costs, int coins) {  
        Arrays.sort(costs);  
  
        for (int i = 0; i < costs.length; ++i)  
            if (coins >= costs[i])  
                coins -= costs[i];  
            else  
                return i;  
  
        return costs.length;  
    }  
}
```

848. 1834.java

Path: LeetCode-main/solutions/1834. Single-Threaded CPU/1834.java

```
class Solution {
    public int[] getOrder(int[][] tasks) {
        record T(int procTime, int index) {}
        final int n = tasks.length;
        int[][] A = new int[n][3];

        for (int i = 0; i < n; ++i) {
            A[i][0] = tasks[i][0];
            A[i][1] = tasks[i][1];
            A[i][2] = i;
        }

        int[] ans = new int[n];
        int ansIndex = 0;
        Queue<T> minHeap =
            new PriorityQueue<>(Comparator.comparingInt(T::procTime).thenComparingInt(T::index));
        int i = 0; // tasks' index
        long time = 0; // the current time

        Arrays.sort(A, Comparator.comparingInt(a -> a[0]));

        while (i < n || !minHeap.isEmpty()) {
            if (minHeap.isEmpty())
                time = Math.max(time, (long) A[i][0]);
            while (i < n && time >= (long) A[i][0]) {
                minHeap.offer(new T(A[i][1], A[i][2]));
                ++i;
            }
            final int procTime = minHeap.peek().procTime;
            final int index = minHeap.poll().index;
            time += procTime;
            ans[ansIndex++] = index;
        }

        return ans;
    }
}
```

849. 1835.java

Path: LeetCode-main/solutions/1835. Find XOR Sum of All Pairs Bitwise AND/1835.java

```
class Solution {
    public int getXORSum(int[] arr1, int[] arr2) {
        final int xors1 = Arrays.stream(arr1).reduce((a, b) -> a ^ b).getAsInt();
        final int xors2 = Arrays.stream(arr2).reduce((a, b) -> a ^ b).getAsInt();
        return xors1 & xors2;
    }
}
```

850. 1836.java

Path: LeetCode-main/solutions/1836. Remove Duplicates From an Unsorted Linked List/1836.java

```
class Solution {
    public ListNode deleteDuplicatesUnsorted(ListNode head) {
        ListNode dummy = new ListNode(0, head);
        Map<Integer, Integer> count = new HashMap<>();

        for (ListNode curr = head; curr != null; curr = curr.next)
            count.merge(curr.val, 1, Integer::sum);

        ListNode curr = dummy;

        while (curr != null) {
            while (curr.next != null && count.containsKey(curr.next.val) && count.get(curr.next.val) > 1)
                curr.next = curr.next.next;
            curr = curr.next;
        }

        return dummy.next;
    }
}
```

851. 1837.java

Path: LeetCode-main/solutions/1837. Sum of Digits in Base K/1837.java

```
class Solution {
    public int sumBase(int n, int k) {
        int ans = 0;

        while (n > 0) {
            ans += n % k;
            n /= k;
        }

        return ans;
    }
}
```


852. 1838.java

Path: LeetCode-main/solutions/1838. Frequency of the Most Frequent Element/1838.java

```
class Solution {
    public int maxFrequency(int[] nums, int k) {
        int ans = 0;
        long sum = 0;

        Arrays.sort(nums);

        for (int l = 0, r = 0; r < nums.length; ++r) {
            sum += nums[r];
            while (sum + k < (long) nums[r] * (r - l + 1))
                sum -= nums[l++];
            ans = Math.max(ans, r - l + 1);
        }

        return ans;
    }
}
```

853. 1839.java

Path: LeetCode-main/solutions/1839. Longest Substring Of All Vowels in Order/1839.java

```
class Solution {
    public int longestBeautifulSubstring(String word) {
        int ans = 0;
        int count = 1;

        for (int l = 0, r = 1; r < word.length(); ++r) {
            final char curr = word.charAt(r);
            final char prev = word.charAt(r - 1);
            if (curr >= prev) {
                if (curr > prev)
                    ++count;
                if (count == 5)
                    ans = Math.max(ans, r - l + 1);
            } else {
                count = 1;
                l = r;
            }
        }

        return ans;
    }
}
```

854. 1840.java

Path: LeetCode-main/solutions/1840. Maximum Building Height/1840.java

```
class Solution {
    public int maxBuilding(int n, int[][] restrictions) {
        final int k = restrictions.length;
        int[][] A = new int[k + 2][2];
        System.arraycopy(restrictions, 0, A, 0, k);
        A[k] = new int[] {1, 0};
        A[k + 1] = new int[] {n, n - 1};

        Arrays.sort(A, Comparator.comparingInt((int[] a) -> a[0]).thenComparingInt(a -> a[1]));

        for (int i = 1; i < A.length; ++i) {
            final int dist = A[i][0] - A[i - 1][0];
            A[i][1] = Math.min(A[i][1], A[i - 1][1] + dist);
        }

        for (int i = A.length - 2; i >= 0; --i) {
            final int dist = A[i + 1][0] - A[i][0];
            A[i][1] = Math.min(A[i][1], A[i + 1][1] + dist);
        }

        int ans = 0;

        for (int i = 1; i < A.length; ++i) {
            final int l = A[i - 1][0];
            final int r = A[i][0];
            final int hL = A[i - 1][1];
            final int hR = A[i][1];
            ans = Math.max(ans, Math.max(hL, hR) + (r - l - Math.abs(hL - hR)) / 2);
        }

        return ans;
    }
}
```

855. 1842.java

Path: LeetCode-main/solutions/1842. Next Palindrome Using Same Digits/1842.java

```
class Solution {
    public String nextPalindrome(String num) {
        final int n = num.length();
        int[] arr = new int[n / 2];

        for (int i = 0; i < arr.length; ++i)
            arr[i] = num.charAt(i) - '0';

        if (!nextPermutation(arr))
            return "";

        StringBuilder sb = new StringBuilder();

        for (final int a : arr)
            sb.append(a);

        if (n % 2 == 1)
            return sb.toString() + num.charAt(n / 2) + sb.reverse().toString();
        return sb.toString() + sb.reverse().toString();
    }

    private boolean nextPermutation(int[] nums) {
        final int n = nums.length;

        // From the back to the front, find the first num < nums[i + 1].
        int i;
        for (i = n - 2; i >= 0; --i)
            if (nums[i] < nums[i + 1])
                break;

        if (i < 0)
            return false;

        // From the back to the front, find the first num > nums[i] and swap it with
        // nums[i].
        for (int j = n - 1; j > i; --j)
            if (nums[j] > nums[i]) {
                swap(nums, i, j);
                break;
            }

        // Reverse nums[i + 1..n - 1].
        reverse(nums, i + 1, n - 1);
        return true;
    }

    private void reverse(int[] nums, int l, int r) {
        while (l < r)
            swap(nums, l++, r--);
    }

    private void swap(int[] nums, int i, int j) {
        final int temp = nums[i];
        nums[i] = nums[j];
        nums[j] = temp;
    }
}
```

856. 1844.java

Path: LeetCode-main/solutions/1844. Replace All Digits with Characters/1844.java

```
class Solution {
    public String replaceDigits(String s) {
        char[] chars = s.toCharArray();
        for (int i = 1; i < chars.length; i += 2)
            chars[i] += chars[i - 1] - '0';
        return String.valueOf(chars);
    }
}
```

857. 1845.java

Path: LeetCode-main/solutions/1845. Seat Reservation Manager/1845.java

```
class SeatManager {
    public SeatManager(int n) {}

    public int reserve() {
        if (minHeap.isEmpty())
            return ++num;
        return minHeap.poll();
    }

    public void unreserve(int seatNumber) {
        minHeap.offer(seatNumber);
    }

    private Queue<Integer> minHeap = new PriorityQueue<>();
    private int num = 0;
}
```

858. 1846.java

Path: LeetCode-main/solutions/1846. Maximum Element After Decreasing and Rearranging/1846.java

```
class Solution {
    public int maximumElementAfterDecrementingAndRearranging(int[] arr) {
        Arrays.sort(arr);
        arr[0] = 1;

        for (int i = 1; i < arr.length; ++i)
            arr[i] = Math.min(arr[i], arr[i - 1] + 1);

        return arr[arr.length - 1];
    }
}
```

859. 1847.java

Path: LeetCode-main/solutions/1847. Closest Room/1847.java

```
class Solution {
    public int[] closestRoom(int[][] rooms, int[][] queries) {
        int[] ans = new int[queries.length];
        Integer[] indices = new Integer[queries.length];
        TreeSet<Integer> roomIds = new TreeSet<>();

        for (int i = 0; i < queries.length; ++i)
            indices[i] = i;

        Arrays.sort(rooms, Comparator.comparingInt(room -> - room[1]));
        Arrays.sort(indices, Comparator.comparingInt(index -> - queries[index][1]));

        int i = 0; // rooms' index
        for (final int index : indices) {
            while (i < rooms.length && rooms[i][1] >= queries[index][1])
                roomIds.add(rooms[i++][0]);
            ans[index] = searchClosestRoomId(roomIds, queries[index][0]);
        }

        return ans;
    }

    private int searchClosestRoomId(TreeSet<Integer> roomIds, int preferred) {
        Integer floor = roomIds.floor(preferred);
        Integer ceiling = roomIds.ceiling(preferred);
        final int id1 = floor == null ? -1 : floor;
        final int id2 = ceiling == null ? -1 : ceiling;
        if (id1 == -1)
            return id2;
        if (id2 == -1)
            return id1;
        if (Math.abs(preferred - id1) <= Math.abs(preferred - id2))
            return id1;
        return id2;
    }
}
```


860. 1848.java

Path: LeetCode-main/solutions/1848. Minimum Distance to the Target Element/1848.java

```
class Solution {
    public int getMinDistance(int[] nums, int target, int start) {
        int ans = Integer.MAX_VALUE;

        for (int i = 0; i < nums.length; ++i)
            if (nums[i] == target)
                ans = Math.min(ans, Math.abs(i - start));

        return ans;
    }
}
```

861. 1849.java

Path: LeetCode-main/solutions/1849. Splitting a String Into Descending Consecutive Values/1849.java

```
class Solution {
    public boolean splitString(String s) {
        return isValid(s, 0, -1, 0);
    }

    private boolean isValid(final String s, int start, long prev, int segment) {
        if (start == s.length() && segment > 1)
            return true;

        long curr = 0;
        for (int i = start; i < s.length(); ++i) {
            curr = curr * 10 + s.charAt(i) - '0';
            if (curr > 999999999)
                return false;
            if ((prev == -1 || curr == prev - 1) && isValid(s, i + 1, curr, segment + 1))
                return true;
        }
        return false;
    }
}
```

862. 1850.java

Path: LeetCode-main/solutions/1850. Minimum Adjacent Swaps to Reach the Kth Smallest Number/1850.java

```
class Solution {
    public int getMinSwaps(String num, int k) {
        int[] original = new int[num.length()];

        for (int i = 0; i < original.length; ++i)
            original[i] = num.charAt(i) - '0';

        int[] permutated = original.clone();

        for (int i = 0; i < k; ++i)
            nextPermutation(permutated);

        return countSteps(original, permutated);
    }

    public void nextPermutation(int[] nums) {
        final int n = nums.length;

        // From the back to the front, find the first num < nums[i + 1].
        int i;
        for (i = n - 2; i >= 0; --i)
            if (nums[i] < nums[i + 1])
                break;

        // From the back to the front, find the first num > nums[i] and swap it with nums[i].
        if (i >= 0)
            for (int j = n - 1; j > i; --j)
                if (nums[j] > nums[i]) {
                    swap(nums, i, j);
                    break;
                }

        // Reverse nums[i + 1..n - 1].
        reverse(nums, i + 1, n - 1);
    }

    private void reverse(int[] nums, int l, int r) {
        while (l < r)
            swap(nums, l++, r--);
    }

    private void swap(int[] nums, int i, int j) {
        final int temp = nums[i];
        nums[i] = nums[j];
        nums[j] = temp;
    }

    private int countSteps(int[] A, int[] B) {
        int count = 0;

        for (int i = 0, j = 0; i < A.length; ++i) {
            j = i;
            while (A[i] != B[j])
                ++j;
            while (i < j) {
                swap(B, j, j - 1);
                --j;
                ++count;
            }
        }

        return count;
    }
}
```

863. 1851.java

Path: LeetCode-main/solutions/1851. Minimum Interval to Include Each Query/1851.java

```
class Solution {
    public int[] minInterval(int[][] intervals, int[] queries) {
        record T(int size, int right) {}
        int[] ans = new int[queries.length];
        Arrays.fill(ans, -1);
        Queue<T> minHeap = new PriorityQueue<T>(Comparator.comparingInt(T::size));
        Integer[] indices = new Integer[queries.length];

        for (int i = 0; i < queries.length; ++i)
            indices[i] = i;

        Arrays.sort(intervals, Comparator.comparingInt(interval -> interval[0]));
        Arrays.sort(indices, Comparator.comparingInt(index -> queries[index]));

        int i = 0; // intervals' index
        for (final int index : indices) {
            while (i < intervals.length && intervals[i][0] <= queries[index]) {
                minHeap.offer(new T(intervals[i][1] - intervals[i][0] + 1, intervals[i][1]));
                ++i;
            }
            while (!minHeap.isEmpty() && minHeap.peek().right < queries[index])
                minHeap.poll();
            if (!minHeap.isEmpty())
                ans[index] = minHeap.peek().size;
        }

        return ans;
    }
}
```

864. 1852.java

Path: LeetCode-main/solutions/1852. Distinct Numbers in Each Subarray/1852.java

```
class Solution {
    public int[] distinctNumbers(int[] nums, int k) {
        int[] ans = new int[nums.length - k + 1];
        int distinct = 0;
        Map<Integer, Integer> count = new HashMap<>();

        for (int i = 0; i < nums.length; ++i) {
            if (count.merge(nums[i], 1, Integer::sum) == 1)
                ++distinct;
            if (i >= k && count.merge(nums[i - k], -1, Integer::sum) == 0)
                --distinct;
            if (i >= k - 1)
                ans[i - k + 1] = distinct;
        }

        return ans;
    }
}
```

865. 1854.java

Path: LeetCode-main/solutions/1854. Maximum Population Year/1854.java

```
class Solution {
    public int maximumPopulation(int[][] logs) {
        final int MIN_YEAR = 1950;
        final int MAX_YEAR = 2050;
        int ans = 0;
        int maxPopulation = 0;
        int runningPopulation = 0;
        // population[i] := the population of year i
        int[] population = new int[MAX_YEAR + 1];

        for (int[] log : logs) {
            final int birth = log[0];
            final int death = log[1];
            ++population[birth];
            --population[death];
        }

        for (int year = MIN_YEAR; year <= MAX_YEAR; ++year) {
            runningPopulation += population[year];
            if (runningPopulation > maxPopulation) {
                maxPopulation = runningPopulation;
                ans = year;
            }
        }

        return ans;
    }
}
```

866. 1855-2.java

Path: LeetCode-main/solutions/1855. Maximum Distance Between a Pair of Values/1855-2.java

```
class Solution {  
    public int maxDistance(int[] nums1, int[] nums2) {  
        int i = 0;  
        int j = 0;  
  
        for (; i < nums1.length && j < nums2.length; ++j)  
            if (nums1[i] > nums2[j])  
                ++i;  
  
        return i == j ? 0 : j - i - 1;  
    }  
}
```

867. 1855.java

Path: LeetCode-main/solutions/1855. Maximum Distance Between a Pair of Values/1855.java

```
class Solution {
    public int maxDistance(int[] nums1, int[] nums2) {
        int ans = 0;
        int i = 0;
        int j = 0;

        while (i < nums1.length && j < nums2.length)
            if (nums1[i] > nums2[j])
                ++i;
            else
                ans = Math.max(ans, j++ - i);

        return ans;
    }
}
```


868. 1856.java

Path: LeetCode-main/solutions/1856. Maximum Subarray Min-Product/1856.java

```
class Solution {
    public int maxSumMinProduct(int[] nums) {
        final int MOD = 1_000_000_007;
        long ans = 0;
        Deque<Integer> stack = new ArrayDeque<>();
        long[] prefix = new long[nums.length + 1];

        for (int i = 0; i < nums.length; ++i)
            prefix[i + 1] = prefix[i] + nums[i];

        for (int i = 0; i <= nums.length; ++i) {
            while (!stack.isEmpty() && (i == nums.length || nums[stack.peek()] > nums[i])) {
                final int minVal = nums[stack.pop()];
                final long sum = stack.isEmpty() ? prefix[i] : prefix[i] - prefix[stack.peek() + 1];
                ans = Math.max(ans, minVal * sum);
            }
            stack.push(i);
        }

        return (int) (ans % MOD);
    }
}
```

869. 1857.java

Path: LeetCode-main/solutions/1857. Largest Color Value in a Directed Graph/1857.java

```
class Solution {
    public int largestPathValue(String colors, int[][] edges) {
        final int n = colors.length();
        int ans = 0;
        int processed = 0;
        List<Integer>[] graph = new List[n];
        int[] inDegrees = new int[n];
        int[][] count = new int[n][26];
        Arrays.setAll(graph, i -> new ArrayList<>());

        // Build the graph.
        for (int[] edge : edges) {
            final int u = edge[0];
            final int v = edge[1];
            graph[u].add(v);
            ++inDegrees[v];
        }

        // Perform topological sorting.
        Queue<Integer> q = IntStream.range(0, n)
            .filter(i -> inDegrees[i] == 0)
            .boxed()
            .collect(Collectors.toCollection(ArrayDeque::new));

        while (!q.isEmpty()) {
            final int out = q.poll();
            ++processed;
            ans = Math.max(ans, ++count[out][colors.charAt(out) - 'a']);
            for (final int in : graph[out]) {
                for (int i = 0; i < 26; ++i)
                    count[in][i] = Math.max(count[in][i], count[out][i]);
                if (--inDegrees[in] == 0)
                    q.offer(in);
            }
        }

        return processed == n ? ans : -1;
    }
}
```

870. 1858.java

Path: LeetCode-main/solutions/1858. Longest Word With All Prefixes/1858.java

```
class TrieNode {
    public TrieNode[] children = new TrieNode[26];
    public boolean isWord = false;
}

class Solution {
    public String longestWord(String[] words) {
        String ans = "";

        for (final String word : words)
            insert(word);

        for (final String word : words) {
            if (!allPrefixed(word))
                continue;
            if (ans.length() < word.length() ||
                (ans.length() == word.length() && ans.compareTo(word) > 0))
                ans = word;
        }

        return ans;
    }

    private TrieNode root = new TrieNode();

    private void insert(final String word) {
        TrieNode node = root;
        for (final char c : word.toCharArray()) {
            final int i = c - 'a';
            if (node.children[i] == null)
                node.children[i] = new TrieNode();
            node = node.children[i];
        }
        node.isWord = true;
    }

    private boolean allPrefixed(final String word) {
        TrieNode node = root;
        for (final char c : word.toCharArray()) {
            final int i = c - 'a';
            node = node.children[i];
            if (!node.isWord)
                return false;
        }
        return true;
    }
}
```

871. 1859.java

Path: LeetCode-main/solutions/1859. Sorting the Sentence/1859.java

```
class Solution {
    public String sortSentence(String s) {
        String[] words = s.split(" ");

        Arrays.sort(words, (a, b) -> a.charAt(a.length() - 1) - b.charAt(b.length() - 1));

        StringBuilder sb = new StringBuilder(trim(words[0]));

        for (int i = 1; i < words.length; ++i)
            sb.append(" ").append(trim(words[i]));

        return sb.toString();
    }

    private String trim(final String s) {
        return s.substring(0, s.length() - 1);
    }
}
```

872. 186.java

Path: LeetCode-main/solutions/186. Reverse Words in a String II/186.java

```
class Solution {
    public void reverseWords(char[] s) {
        reverse(s, 0, s.length - 1); // Reverse the whole string.
        reverseWords(s, s.length);    // Reverse each word.
    }

    private void reverse(char[] s, int l, int r) {
        while (l < r) {
            final char c = s[l];
            s[l++] = s[r];
            s[r--] = c;
        }
    }

    private void reverseWords(char[] s, int n) {
        int i = 0;
        int j = 0;

        while (i < n) {
            while (i < j || i < n && s[i] == ' ') // Skip the spaces.
                ++i;
            while (j < i || j < n && s[j] != ' ') // Skip the spaces.
                ++j;
            reverse(s, i, j - 1); // Reverse the word.
        }
    }
}
```

873. 1860.java

Path: LeetCode-main/solutions/1860. Incremental Memory Leak/1860.java

```
class Solution {
    public int[] memLeak(int memory1, int memory2) {
        int i = 1;

        while (memory1 >= i || memory2 >= i) {
            if (memory1 >= memory2)
                memory1 -= i;
            else
                memory2 -= i;
            ++i;
        }

        return new int[] {i, memory1, memory2};
    }
}
```

874. 1861.java

Path: LeetCode-main/solutions/1861. Rotating the Box/1861.java

```
class Solution {
    public char[][] rotateTheBox(char[][] box) {
        final int m = box.length;
        final int n = box[0].length;
        char[][] ans = new char[n][m];
        Arrays.stream(ans).forEach(A -> Arrays.fill(A, '.'));

        for (int i = 0; i < m; ++i)
            for (int j = n - 1, k = n - 1; j >= 0; --j)
                if (box[i][j] != '.') {
                    if (box[i][j] == '*')
                        k = j;
                    ans[k--][m - i - 1] = box[i][j];
                }
        return ans;
    }
}
```

875. 1862.java

Path: LeetCode-main/solutions/1862. Sum of Floored Pairs/1862.java

```
class Solution {
    public int sumOfFlooredPairs(int[] nums) {
        final int MOD = 1_000_000_007;
        final int MAX = Arrays.stream(nums).max().getAsInt();
        long ans = 0;
        // count[i] := the number of `nums` <= i
        int[] count = new int[MAX + 1];

        for (final int num : nums)
            ++count[num];

        for (int i = 1; i <= MAX; ++i)
            count[i] += count[i - 1];

        for (int i = 1; i <= MAX; ++i)
            if (count[i] > count[i - 1]) {
                long sum = 0;
                for (int j = 1; i * j <= MAX; ++j) {
                    final int lo = i * j - 1;
                    final int hi = i * (j + 1) - 1;
                    sum += (count[Math.min(hi, MAX)] - count[lo]) * j;
                }
                ans += sum * (count[i] - count[i - 1]);
                ans %= MOD;
            }

        return (int) ans;
    }
}
```


876. 1863-2.java

Path: LeetCode-main/solutions/1863. Sum of All Subset XOR Totals/1863-2.java

```
class Solution {  
    public int subsetXORSum(int[] nums) {  
        return Arrays.stream(nums).reduce((a, b) -> a | b).getAsInt() << nums.length - 1;  
    }  
}
```

877. 1863.java

Path: LeetCode-main/solutions/1863. Sum of All Subset XOR Totals/1863.java

```
class Solution {
    public int subsetXORSum(int[] nums) {
        return dfs(nums, 0, 0);
    }

    private int dfs(int[] nums, int i, int xors) {
        if (i == nums.length)
            return xors;

        final int x = dfs(nums, i + 1, xors);
        final int y = dfs(nums, i + 1, nums[i] ^ xors);
        return x + y;
    }
}
```

878. 1864.java

Path: LeetCode-main/solutions/1864. Minimum Number of Swaps to Make the Binary String Alternating/1864.java

```
class Solution {
    public int minSwaps(String s) {
        final int ones = (int) s.chars().filter(c -> c == '1').count();
        final int zeros = s.length() - ones;
        if (Math.abs(ones - zeros) > 1)
            return -1;
        if (ones > zeros)
            return countSwaps(s, '1');
        if (zeros > ones)
            return countSwaps(s, '0');
        return Math.min(countSwaps(s, '1'), countSwaps(s, '0'));
    }

    private int countSwaps(final String s, char curr) {
        int swaps = 0;
        for (final char c : s.toCharArray()) {
            if (c != curr)
                ++swaps;
            curr ^= 1;
        }
        return swaps / 2;
    }
}
```

879. 1865.java

Path: LeetCode-main/solutions/1865. Finding Pairs With a Certain Sum/1865.java

```
class FindSumPairs {
    public FindSumPairs(int[] nums1, int[] nums2) {
        this.nums1 = nums1;
        this.nums2 = nums2;
        for (final int num : nums2)
            count2.merge(num, 1, Integer::sum);
    }

    public void add(int index, int val) {
        count2.merge(nums2[index], -1, Integer::sum);
        nums2[index] += val;
        count2.merge(nums2[index], 1, Integer::sum);
    }

    public int count(int tot) {
        int ans = 0;
        for (final int num : nums1)
            ans += count2.getDefault(tot - num, 0);
        return ans;
    }

    private int[] nums1;
    private int[] nums2;
    private Map<Integer, Integer> count2 = new HashMap<>();
}
```

880. 1866.java

Path: LeetCode-main/solutions/1866. Number of Ways to Rearrange Sticks With K Sticks Visible/1866.java

```
class Solution {
    public int rearrangeSticks(int n, int k) {
        if (n == k)
            return 1;
        if (k == 0)
            return 0;
        if (dp[n][k] != 0)
            return dp[n][k];
        return dp[n][k] = (int) (((long) rearrangeSticks(n - 1, k - 1) +
                                (long) rearrangeSticks(n - 1, k) * (n - 1)) %
                                MOD);
    }

    private static final int MOD = 1_000_000_007;
    private int[][] dp = new int[1001][1001];
}
```

881. 1868.java

Path: LeetCode-main/solutions/1868. Product of Two Run-Length Encoded Arrays/1868.java

```
class Solution {
    public List<List<Integer>> findRLEArray(int[][] encoded1, int[][] encoded2) {
        List<List<Integer>> ans = new ArrayList<>();
        int i = 0; // encoded1's index
        int j = 0; // encoded2's index

        while (i < encoded1.length && j < encoded2.length) {
            final int mult = encoded1[i][0] * encoded2[j][0];
            final int minFreq = Math.min(encoded1[i][1], encoded2[j][1]);
            if (!ans.isEmpty() && mult == ans.get(ans.size() - 1).get(0))
                ans.get(ans.size() - 1).set(1, ans.get(ans.size() - 1).get(1) + minFreq);
            else
                ans.add(Arrays.asList(mult, minFreq));
            encoded1[i][1] -= minFreq;
            encoded2[j][1] -= minFreq;
            if (encoded1[i][1] == 0)
                ++i;
            if (encoded2[j][1] == 0)
                ++j;
        }

        return ans;
    }
}
```

882. 1869.java

Path: LeetCode-main/solutions/1869. Longer Contiguous Segments of Ones than Zeros/1869.java

```
class Solution {
    public boolean checkZeroOnes(String s) {
        int longestOnes = 0;
        int longestZeros = 0;
        int currentOnes = 0;
        int currentZeros = 0;

        for (final char c : s.toCharArray())
            if (c == '0') {
                currentOnes = 0;
                longestZeros = Math.max(longestZeros, ++currentZeros);
            } else {
                currentZeros = 0;
                longestOnes = Math.max(longestOnes, ++currentOnes);
            }

        return longestOnes > longestZeros;
    }
}
```

883. 187.java

Path: LeetCode-main/solutions/187. Repeated DNA Sequences/187.java

```
class Solution {
    public List<String> findRepeatedDnaSequences(String s) {
        Set<String> ans = new HashSet<>();
        Set<String> seen = new HashSet<>();

        for (int i = 0; i + 10 <= s.length(); ++i) {
            final String seq = s.substring(i, i + 10);
            if (seen.contains(seq))
                ans.add(seq);
            seen.add(seq);
        }

        return new ArrayList<>(ans);
    }
}
```


884. 1870.java

Path: LeetCode-main/solutions/1870. Minimum Speed to Arrive on Time/1870.java

```
class Solution {
    public int minSpeedOnTime(int[] dist, double hour) {
        int ans = -1;
        int l = 1;
        int r = 10_000_000;

        while (l <= r) {
            final int m = (l + r) / 2;
            if (time(dist, hour, m) > hour) {
                l = m + 1;
            } else {
                ans = m;
                r = m - 1;
            }
        }

        return ans;
    }

    private double time(int[] dist, double hour, int speed) {
        double sum = 0;
        for (int i = 0; i < dist.length - 1; ++i)
            sum += Math.ceil((double) dist[i] / speed);
        return sum + (double) dist[dist.length - 1] / speed;
    }
}
```

885. 1871.java

Path: LeetCode-main/solutions/1871. Jump Game VII/1871.java

```
class Solution {
    public boolean canReach(String s, int minJump, int maxJump) {
        int count = 0;
        boolean dp[] = new boolean[s.length()];
        dp[0] = true;

        for (int i = minJump; i < s.length(); ++i) {
            count += dp[i - minJump] ? 1 : 0;
            if (i - maxJump > 0)
                count -= dp[i - maxJump - 1] ? 1 : 0;
            dp[i] = count > 0 && s.charAt(i) == '0';
        }

        return dp[dp.length - 1];
    }
}
```

886. 1872.java

Path: LeetCode-main/solutions/1872. Stone Game VIII/1872.java

```
class Solution {
    public int stoneGameVIII(int[] stones) {
        final int n = stones.length;
        int[] prefix = stones.clone();
        // dp[i] := the maximum score difference the current player can get when the
        // game starts at i, i.e. stones[0..i] are merged into the value prefix[i]
        int[] dp = new int[n];
        Arrays.fill(dp, Integer.MIN_VALUE);

        for (int i = 1; i < prefix.length; ++i)
            prefix[i] += prefix[i - 1];

        // Must take all when there're only two stones left.
        dp[n - 2] = prefix[n - 1];

        for (int i = n - 3; i >= 0; --i)
            dp[i] = Math.max(dp[i + 1], prefix[i + 1] - dp[i + 1]);

        return dp[0];
    }
}
```

887. 1874.java

Path: LeetCode-main/solutions/1874. Minimize Product Sum of Two Arrays/1874.java

```
class Solution {
    public int minProductSum(int[] nums1, int[] nums2) {
        final int n = nums1.length;

        int ans = 0;

        Arrays.sort(nums1);
        Arrays.sort(nums2);

        for (int i = 0; i < n; ++i)
            ans += nums1[i] * nums2[n - 1 - i];

        return ans;
    }
}
```

888. 1876.java

Path: LeetCode-main/solutions/1876. Substrings of Size Three with Distinct Characters/1876.java

```
class Solution {
    public int countGoodSubstrings(String s) {
        int ans = 0;

        for (int i = 0; i < s.length() - 2; ++i) {
            final char a = s.charAt(i);
            final char b = s.charAt(i + 1);
            final char c = s.charAt(i + 2);
            if (a == b || a == c || b == c)
                continue;
            ++ans;
        }

        return ans;
    }
}
```

889. 1877.java

Path: LeetCode-main/solutions/1877. Minimize Maximum Pair Sum in Array/1877.java

```
class Solution {  
    public int minPairSum(int[] nums) {  
        int ans = 0;  
  
        Arrays.sort(nums);  
  
        for (int i = 0, j = nums.length - 1; i < j;)  
            ans = Math.max(ans, nums[i++] + nums[j--]);  
  
        return ans;  
    }  
}
```

890. 1878.java

Path: LeetCode-main/solutions/1878. Get Biggest Three Rhombus Sums in a Grid/1878.java

```
class Solution {
    public int[] getBiggestThree(int[][] grid) {
        final int m = grid.length;
        final int n = grid[0].length;
        TreeSet<Integer> sums = new TreeSet<>();

        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j)
                for (int sz = 0; i + sz < m && i - sz >= 0 && j + 2 * sz < n; ++sz) {
                    final int sum = sz == 0 ? grid[i][j] : getSum(grid, i, j, sz);
                    sums.add(sum);
                    if (sums.size() > 3)
                        sums.pollFirst();
                }

        return sums.descendingSet().stream().mapToInt(Integer::intValue).toArray();
    }

    // Returns the sum of the rhombus, where the top grid is (i, j) and the edge
    // size is `sz`.
    private int getSum(int[][] grid, int i, int j, int sz) {
        int x = i;
        int y = j;
        int sum = 0;

        // Go left down.
        for (int k = 0; k < sz; ++k)
            sum += grid[--x][++y];

        // Go right down.
        for (int k = 0; k < sz; ++k)
            sum += grid[++x][++y];

        // Go right up.
        for (int k = 0; k < sz; ++k)
            sum += grid[++x][--y];

        // Go left up.
        for (int k = 0; k < sz; ++k)
            sum += grid[--x][--y];

        return sum;
    }
}
```

891. 1879.java

Path: LeetCode-main/solutions/1879. Minimum XOR Sum of Two Arrays/1879.java

```
class Solution {
    public int minimumXORSum(int[] nums1, int[] nums2) {
        int[] mem = new int[1 << nums2.length];
        Arrays.fill(mem, Integer.MAX_VALUE);
        return minimumXORSum(nums1, nums2, 0, mem);
    }

    private int minimumXORSum(int[] nums1, int[] nums2, int mask, int[] mem) {
        final int i = Integer.bitCount(mask);
        if (i == nums1.length)
            return 0;
        if (mem[mask] < Integer.MAX_VALUE)
            return mem[mask];

        for (int j = 0; j < nums2.length; ++j)
            if ((mask >> j & 1) == 0)
                mem[mask] = Math.min(mem[mask], (nums1[i] ^ nums2[j]) +
                                         minimumXORSum(nums1, nums2, mask | 1 << j, mem));

        return mem[mask];
    }
}
```


892. 188.java

Path: LeetCode-main/solutions/188. Best Time to Buy and Sell Stock IV/188.java

```
class Solution {
    public int maxProfit(int k, int[] prices) {
        if (k >= prices.length / 2) {
            int sell = 0;
            int hold = Integer.MIN_VALUE;

            for (final int price : prices) {
                sell = Math.max(sell, hold + price);
                hold = Math.max(hold, sell - price);
            }

            return sell;
        }

        int[] sell = new int[k + 1];
        int[] hold = new int[k + 1];
        Arrays.fill(hold, Integer.MIN_VALUE);

        for (final int price : prices)
            for (int i = k; i > 0; --i) {
                sell[i] = Math.max(sell[i], hold[i] + price);
                hold[i] = Math.max(hold[i], sell[i - 1] - price);
            }

        return sell[k];
    }
}
```

893. 1880.java

Path: LeetCode-main/solutions/1880. Check if Word Equals Summation of Two Words/1880.java

```
class Solution {
    public boolean isSumEqual(String firstWord, String secondWord, String targetWord) {
        final int first = getNumber(firstWord);
        final int second = getNumber(secondWord);
        final int target = getNumber(targetWord);
        return first + second == target;
    }

    private int getNumber(String word) {
        int num = 0;
        for (final char c : word.toCharArray())
            num = num * 10 + (c - 'a');
        return num;
    }
}
```

894. 1881.java

Path: LeetCode-main/solutions/1881. Maximum Value after Insertion/1881.java

```
class Solution {
    public String maxValue(String n, int x) {
        boolean isNegative = n.charAt(0) == '-';

        for (int i = 0; i < n.length(); ++i)
            if (!isNegative && n.charAt(i) - '0' < x || isNegative && n.charAt(i) - '0' > x)
                return n.substring(0, i) + x + n.substring(i);

        return n + x;
    }
}
```

895. 1882.java

Path: LeetCode-main/solutions/1882. Process Tasks Using Servers/1882.java

```
class T {
    public int weight;
    public int index;
    public int freeTime;
    public T(int weight, int index, int freeTime) {
        this.weight = weight;
        this.index = index;
        this.freeTime = freeTime;
    }
}

class Solution {
    public int[] assignTasks(int[] servers, int[] tasks) {
        final int n = servers.length;
        final int m = tasks.length;
        int[] ans = new int[m];
        Queue<T> free = new PriorityQueue<>(
            Comparator.comparing((T t) -> t.weight).thenComparing((T t) -> t.index));
        Queue<T> used = new PriorityQueue<>(Comparator.comparing((T t) -> t.freeTime)
            .thenComparing((T t) -> t.weight)
            .thenComparing((T t) -> t.index));

        for (int i = 0; i < n; ++i)
            free.offer(new T(servers[i], i, 0));

        for (int i = 0; i < m; ++i) { // i := the current time
            final int executionTime = tasks[i];
            // Poll all servers that'll be free at time i.
            while (!used.isEmpty() && used.peek().freeTime <= i)
                free.offer(used.poll());
            if (free.isEmpty()) {
                T server = used.poll();
                ans[i] = server.index;
                server.freeTime += executionTime;
                used.offer(server);
            } else {
                T server = free.poll();
                ans[i] = server.index;
                server.freeTime = i + executionTime;
                used.offer(server);
            }
        }

        return ans;
    }
}
```

896. 1883.java

Path: LeetCode-main/solutions/1883. Minimum Skips to Arrive at Meeting On Time/1883.java

```
class Solution {
    public int minSkips(int[] dist, int speed, int hoursBefore) {
        final double INF = 1e7;
        final double EPS = 1e-9;
        final int n = dist.length;
        // dp[i][j] := the minimum time, where i is the number of roads we traversed
        // so far and j is the number of skips we did
        double[][] dp = new double[n + 1][n + 1];
        Arrays.stream(dp).forEach(A -> Arrays.fill(A, INF));
        dp[0][0] = 0;

        for (int i = 1; i <= n; ++i) {
            final double d = dist[i - 1];
            dp[i][0] = Math.ceil(dp[i - 1][0] + d / speed - EPS);
            for (int j = 1; j <= i; ++j)
                dp[i][j] =
                    Math.min(dp[i - 1][j - 1] + d / speed, Math.ceil(dp[i - 1][j] + d / speed - EPS));
        }

        for (int j = 0; j <= n; ++j)
            if (dp[n][j] <= hoursBefore)
                return j;

        return -1;
    }
}
```

897. 1884.java

Path: LeetCode-main/solutions/1884. Egg Drop With 2 Eggs and N Floors/1884.java

```
class Solution {
    public int twoEggDrop(int n) {
        return superEggDrop(2, n);
    }

    // Same as 887. Super Egg Drop
    private int superEggDrop(int k, int n) {
        int[][] mem = new int[k + 1][n + 1];
        Arrays.stream(mem).forEach(A -> Arrays.fill(A, -1));
        return drop(k, n, mem);
    }

    // Returns the minimum number of moves to know f with k eggs and n floors.
    private int drop(int k, int n, int[][] mem) {
        if (k == 0) // no eggs -> done
            return 0;
        if (k == 1) // one egg -> drop from 1-th floor to n-th floor
            return n;
        if (n == 0) // no floor -> done
            return 0;
        if (n == 1) // one floor -> drop from that floor
            return 1;
        if (mem[k][n] != -1)
            return mem[k][n];

        int l = 1;
        int r = n + 1;

        while (l < r) {
            final int m = (l + r) / 2;
            final int broken = drop(k - 1, m - 1, mem);
            final int unbroken = drop(k, n - m, mem);
            if (broken >= unbroken)
                r = m;
            else
                l = m + 1;
        }

        return mem[k][n] = 1 + drop(k - 1, l - 1, mem);
    }
}
```

898. 1885.java

Path: LeetCode-main/solutions/1885. Count Pairs in Two Arrays/1885.java

```
class Solution {
    public long countPairs(int[] nums1, int[] nums2) {
        long ans = 0;
        int[] arr = new int[nums1.length]; // arr[i] = nums1[i] - nums2[i]

        for (int i = 0; i < arr.length; ++i)
            arr[i] = nums1[i] - nums2[i];

        Arrays.sort(arr);

        for (int i = 0; i < arr.length; ++i) {
            final int index = firstGreater(arr, -arr[i]);
            ans += arr.length - Math.max(i + 1, index);
        }

        return ans;
    }

    private int firstGreater(int[] arr, int target) {
        int l = 0;
        int r = arr.length;
        while (l < r) {
            final int m = (l + r) / 2;
            if (arr[m] > target)
                r = m;
            else
                l = m + 1;
        }
        return l;
    }
}
```

899. 1886.java

Path: LeetCode-main/solutions/1886. Determine Whether Matrix Can Be Obtained By Rotation/1886.java

```
class Solution {
    public boolean findRotation(int[][] mat, int[][] target) {
        for (int i = 0; i < 4; ++i) {
            if (Arrays.deepEquals(mat, target))
                return true;
            rotate(mat);
        }
        return false;
    }

    private void rotate(int[][] mat) {
        for (int i = 0, j = mat.length - 1; i < j; ++i, --j) {
            int[] temp = mat[i];
            mat[i] = mat[j];
            mat[j] = temp;
        }
        for (int i = 0; i < mat.length; ++i)
            for (int j = i + 1; j < mat.length; ++j) {
                final int temp = mat[i][j];
                mat[i][j] = mat[j][i];
                mat[j][i] = temp;
            }
    }
}
```


900. 1887.java

Path: LeetCode-main/solutions/1887. Reduction Operations to Make the Array Elements Equal/1887.java

```
class Solution {
    public int reductionOperations(int[] nums) {
        int ans = 0;

        Arrays.sort(nums);

        for (int i = nums.length - 1; i > 0; --i)
            if (nums[i] != nums[i - 1])
                ans += nums.length - i;

        return ans;
    }
}
```

901. 1888.java

Path: LeetCode-main/solutions/1888. Minimum Number of Flips to Make the Binary String Alternating/1888.java

```
class Solution {
    public int minFlips(String s) {
        final int n = s.length();
        // count[0][0] := the number of 0s in the even indices
        // count[0][1] := the number of 0s in the odd indices
        // count[1][0] := the number of 1s in the even indices
        // count[1][1] := the number of 1s in the odd indices
        int[][] count = new int[2][2];

        for (int i = 0; i < n; ++i)
            ++count[s.charAt(i) - '0'][i % 2];

        // min(make all 0s in the even indices + make all 1s in the odd indices,
        //      make all 1s in the even indices + make all 0s in the odd indices)
        int ans = Math.min(count[1][0] + count[0][1], count[0][0] + count[1][1]);

        for (int i = 0; i < n; ++i) {
            --count[s.charAt(i) - '0'][i % 2];
            ++count[s.charAt(i) - '0'][(n + i) % 2];
            ans = Math.min(ans, Math.min(count[1][0] + count[0][1], count[0][0] + count[1][1]));
        }

        return ans;
    }
}
```

902. 1889.java

Path: LeetCode-main/solutions/1889. Minimum Space Wasted From Packaging/1889.java

```
class Solution {
    public int minWastedSpace(int[] packages, int[][] boxes) {
        final int MOD = 1_000_000_007;
        final long INF = (long) 1e11;
        final long packagesSum = Arrays.stream(packages).asLongStream().sum();
        long minBoxesSum = INF;

        Arrays.sort(packages);

        for (int[] box : boxes) {
            Arrays.sort(box);
            if (box[box.length - 1] < packages[packages.length - 1])
                continue;
            long accu = 0;
            long i = 0;
            for (final int b : box) {
                final long j = firstGreaterEqual(packages, b + 1);
                accu += b * (j - i);
                i = j;
            }
            minBoxesSum = Math.min(minBoxesSum, accu);
        }

        return minBoxesSum == INF ? -1 : (int) ((minBoxesSum - packagesSum) % MOD);
    }

    private int firstGreaterEqual(int[] arr, int target) {
        int l = 0;
        int r = arr.length;
        while (l < r) {
            final int m = (l + r) / 2;
            if (arr[m] >= target)
                r = m;
            else
                l = m + 1;
        }
        return l;
    }
}
```

903. 189.java

Path: LeetCode-main/solutions/189. Rotate Array/189.java

```
class Solution {
    public void rotate(int[] nums, int k) {
        k %= nums.length;
        reverse(nums, 0, nums.length - 1);
        reverse(nums, 0, k - 1);
        reverse(nums, k, nums.length - 1);
    }

    private void reverse(int[] nums, int l, int r) {
        while (l < r)
            swap(nums, l++, r--);
    }

    private void swap(int[] nums, int l, int r) {
        final int temp = nums[l];
        nums[l] = nums[r];
        nums[r] = temp;
    }
}
```

904. 1891.java

Path: LeetCode-main/solutions/1891. Cutting Ribbons/1891.java

```
class Solution {
    public int maxLength(int[] ribbons, int k) {
        int l = 1;
        int r = (int) (Arrays.stream(ribbons).asLongStream().sum() / k) + 1;

        while (l < r) {
            final int m = (l + r) / 2;
            if (!isCutPossible(ribbons, m, k))
                r = m;
            else
                l = m + 1;
        }

        return l - 1;
    }

    private boolean isCutPossible(int[] ribbons, int length, int k) {
        int count = 0;
        for (final int ribbon : ribbons)
            count += ribbon / length;
        return count >= k;
    }
}
```

905. 1893-2.java

Path: LeetCode-main/solutions/1893. Check if All the Integers in a Range Are Covered/1893-2.java

```
class Solution {
    public boolean isCovered(int[][] ranges, int left, int right) {
        int[] seen = new int[52];

        for (int[] range : ranges) {
            ++seen[range[0]];
            --seen[range[1] + 1];
        }

        for (int i = 1; i < 52; ++i)
            seen[i] += seen[i - 1];

        for (int i = left; i <= right; ++i)
            if (seen[i] == 0)
                return false;

        return true;
    }
}
```

906. 1893.java

Path: LeetCode-main/solutions/1893. Check if All the Integers in a Range Are Covered/1893.java

```
class Solution {
    public boolean isCovered(int[][] ranges, int left, int right) {
        for (int i = left; i <= right; ++i) {
            boolean seen = false;
            for (int[] range : ranges)
                if (i >= range[0] && i <= range[1]) {
                    seen = true;
                    break;
                }
            if (!seen)
                return false;
        }
        return true;
    }
}
```

907. 1894.java

Path: LeetCode-main/solutions/1894. Find the Student that Will Replace the Chalk/1894.java

```
class Solution {
    public int chalkReplacer(int[] chalk, int k) {
        k %= Arrays.stream(chalk).asLongStream().sum();
        if (k == 0)
            return 0;

        for (int i = 0; i < chalk.length; ++i) {
            k -= chalk[i];
            if (k < 0)
                return i;
        }

        throw new IllegalArgumentException();
    }
}
```


908. 1895.java

Path: LeetCode-main/solutions/1895. Largest Magic Square/1895.java

```
class Solution {
    public int largestMagicSquare(int[][] grid) {
        final int m = grid.length;
        final int n = grid[0].length;
        // prefixRow[i][j] := the sum of the first j numbers in the i-th row
        int[][] prefixRow = new int[m][n + 1];
        // prefixCol[i][j] := the sum of the first j numbers in the i-th column
        int[][] prefixCol = new int[n][m + 1];

        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j) {
                prefixRow[i][j + 1] = prefixRow[i][j] + grid[i][j];
                prefixCol[j][i + 1] = prefixCol[j][i] + grid[i][j];
            }

        for (int k = Math.min(m, n); k >= 2; --k)
            if (containsMagicSquare(grid, prefixRow, prefixCol, k))
                return k;

        return 1;
    }

    // Returns true if the grid contains any magic square of size k x k.
    private boolean containsMagicSquare(int[][] grid, int[][] prefixRow, int[][] prefixCol, int k) {
        for (int i = 0; i + k - 1 < grid.length; ++i)
            for (int j = 0; j + k - 1 < grid[0].length; ++j)
                if (isMagicSquare(grid, prefixRow, prefixCol, i, j, k))
                    return true;
        return false;
    }

    // Returns true if grid[i..i + k][j..j + k] is a magic square.
    private boolean isMagicSquare(int[][] grid, int[][] prefixRow, int[][] prefixCol, int i, int j,
                                   int k) {
        int diag = 0;
        int antiDiag = 0;
        for (int d = 0; d < k; ++d) {
            diag += grid[i + d][j + d];
            antiDiag += grid[i + d][j + k - 1 - d];
        }
        if (diag != antiDiag)
            return false;
        for (int d = 0; d < k; ++d) {
            if (getSum(prefixRow, i + d, j, j + k - 1) != diag)
                return false;
            if (getSum(prefixCol, j + d, i, i + k - 1) != diag)
                return false;
        }
        return true;
    }

    // Returns sum(grid[i][l..r]) or sum(grid[l..r][i]).
    private int getSum(int[][] prefix, int i, int l, int r) {
        return prefix[i][r + 1] - prefix[i][l];
    }
}
```

909. 1896.java

Path: LeetCode-main/solutions/1896. Minimum Cost to Change the Final Value of Expression/1896.java

```
class Solution {
    public int minOperationsToFlip(String expression) {
        // [(the expression, the cost to toggle the expression)]
        Deque<Pair<Character, Integer>> stack = new ArrayDeque<>();
        Pair<Character, Integer> lastPair = null;

        for (final char e : expression.toCharArray()) {
            if (e == '(' || e == '&' || e == '|') {
                // These aren't expressions, so the cost is meaningless.
                stack.push(new Pair<>(e, 0));
                continue;
            }
            if (e == ')') {
                lastPair = stack.pop();
                stack.pop(); // Pop '('.
            } else { // e == '0' || e == '1'
                // Store the '0' or '1'. The cost to change their values is just 1,
                // whether it's changing '0' to '1' or '1' to '0'.
                lastPair = new Pair<>(e, 1);
            }
            if (!stack.isEmpty() && (stack.peek().getKey() == '&' || stack.peek().getKey() == '|')) {
                final char op = stack.pop().getKey();
                final char a = stack.peek().getKey();
                final int costA = stack.pop().getValue();
                final char b = lastPair.getKey();
                final int costB = lastPair.getValue();
                // Determine the cost to toggle op(a, b).
                if (op == '&') {
                    if (a == '0' && b == '0')
                        // Change '&' to '|' and a|b to '1'.
                        lastPair = new Pair<>('0', 1 + Math.min(costA, costB));
                    else if (a == '0' && b == '1')
                        // Change '&' to '|'.
                        lastPair = new Pair<>('0', 1);
                    else if (a == '1' && b == '0')
                        // Change '&' to '|'.
                        lastPair = new Pair<>('0', 1);
                    else // a == '1' and b == '1'
                        // Change a|b to '0'.
                        lastPair = new Pair<>('1', Math.min(costA, costB));
                } else { // op == '|'
                    if (a == '0' && b == '0')
                        // Change a|b to '1'.
                        lastPair = new Pair<>('0', Math.min(costA, costB));
                    else if (a == '0' && b == '1')
                        // Change '|' to '&'.
                        lastPair = new Pair<>('1', 1);
                    else if (a == '1' && b == '0')
                        // Change '|' to '&'.
                        lastPair = new Pair<>('1', 1);
                    else // a == '1' and b == '1'
                        // Change '|' to '&' and a|b to '0'.
                        lastPair = new Pair<>('1', 1 + Math.min(costA, costB));
                }
            }
            stack.push(lastPair);
        }
        return stack.peek().getValue();
    }
}
```

910. 1897.java

Path: LeetCode-main/solutions/1897. Redistribute Characters to Make All Strings Equal/1897.java

```
class Solution {
    public boolean makeEqual(String[] words) {
        int[] count = new int[26];

        for (final String word : words)
            for (final char c : word.toCharArray())
                ++count[c - 'a'];

        return Arrays.stream(count).allMatch(c -> c % words.length == 0);
    }
}
```

911. 1898.java

Path: LeetCode-main/solutions/1898. Maximum Number of Removable Characters/1898.java

```
class Solution {
    public int maximumRemovals(String s, String p, int[] removable) {
        int l = 0;
        int r = removable.length + 1;

        while (l < r) {
            final int m = (l + r) / 2;
            final String removed = remove(s, removable, m);
            if (isSubsequence(p, removed))
                l = m + 1;
            else
                r = m;
        }

        return l - 1;
    }

    private String remove(final String s, int[] removable, int k) {
        StringBuilder sb = new StringBuilder(s);
        for (int i = 0; i < k; ++i)
            sb.setCharAt(removable[i], '*');
        return sb.toString();
    }

    private boolean isSubsequence(final String p, final String s) {
        int i = 0; // p's index
        for (int j = 0; j < s.length(); ++j)
            if (p.charAt(i) == s.charAt(j))
                if (++i == p.length())
                    return true;
        return false;
    }
}
```

912. 1899.java

Path: LeetCode-main/solutions/1899. Merge Triplets to Form Target Triplet/1899.java

```
class Solution {
    public boolean mergeTriplets(int[][] triplets, int[] target) {
        int[] merged = new int[target.length];

        for (int[] triplet : triplets)
            if (triplet[0] <= target[0] && triplet[1] <= target[1] && triplet[2] <= target[2])
                for (int i = 0; i < target.length; ++i)
                    merged[i] = Math.max(merged[i], triplet[i]);

        return Arrays.equals(merged, target);
    }
}
```

913. 19.java

Path: LeetCode-main/solutions/19. Remove Nth Node From End of List/19.java

```
class Solution {
    public ListNode removeNthFromEnd(ListNode head, int n) {
        ListNode slow = head;
        ListNode fast = head;

        while (n-- > 0)
            fast = fast.next;
        if (fast == null)
            return head.next;

        while (fast.next != null) {
            slow = slow.next;
            fast = fast.next;
        }
        slow.next = slow.next.next;

        return head;
    }
}
```

914. 190.java

Path: LeetCode-main/solutions/190. Reverse Bits/190.java

```
class Solution {
    // You need treat n as an unsigned value
    public int reverseBits(int n) {
        int ans = 0;

        for (int i = 0; i < 32; ++i)
            if ((n >> i & 1) == 1)
                ans |= 1 << 31 - i;

        return ans;
    }
}
```

915. 1900.java

Path: LeetCode-main/solutions/1900. The Earliest and Latest Rounds Where Players Compete/1900.java

```
class Solution {
    public int[] earliestAndLatest(int n, int firstPlayer, int secondPlayer) {
        int[][][] mem = new int[n + 1][n + 1][n + 1][2];
        return solve(firstPlayer, n - secondPlayer + 1, n, mem);
    }

    // Returns the (earliest, latest) pair, the first player is the l-th player
    // from the front, the second player is the r-th player from the end, and
    // there're k people.
    private int[] solve(int l, int r, int k, int[][][] mem) {
        if (l == r)
            return new int[] {1, 1};
        if (l > r)
            return solve(r, l, k, mem);
        if (!Arrays.equals(mem[l][r][k], new int[] {0, 0}))
            return mem[l][r][k];

        int a = Integer.MAX_VALUE;
        int b = Integer.MIN_VALUE;

        // Enumerate all the possible positions.
        for (int i = 1; i <= l; ++i)
            for (int j = l - i + 1; j <= r - i; ++j) {
                if (i + j > (k + 1) / 2 || i + j < l + r - k / 2)
                    continue;
                int[] res = solve(i, j, (k + 1) / 2, mem);
                a = Math.min(a, res[0] + 1);
                b = Math.max(b, res[1] + 1);
            }

        return mem[l][r][k] = new int[] {a, b};
    }
}
```


916. 1901.java

Path: LeetCode-main/solutions/1901. Find a Peak Element II/1901.java

```
class Solution {
    public int[] findPeakGrid(int[][] mat) {
        int l = 0;
        int r = mat.length - 1;

        while (l < r) {
            final int m = (l + r) / 2;
            if (Arrays.stream(mat[m]).max().getAsInt() >= Arrays.stream(mat[m + 1]).max().getAsInt())
                r = m;
            else
                l = m + 1;
        }

        return new int[] {l, getMaxIndex(mat[l])};
    }

    private int getMaxIndex(int[] arr) {
        int[] res = {0, arr[0]};
        for (int i = 1; i < arr.length; ++i)
            if (arr[i] > res[1])
                res = new int[] {i, arr[i]};
        return res[0];
    }
}
```

917. 1902.java

Path: LeetCode-main/solutions/1902. Depth of BST Given Insertion Order/1902.java

```
class Solution {
    public int maxDepthBST(int[] order) {
        int ans = 1;
        TreeMap<Integer, Integer> valToDepth = new TreeMap<>();

        for (final int val : order) {
            Map.Entry<Integer, Integer> l = valToDepth.floorEntry(val);
            Map.Entry<Integer, Integer> r = valToDepth.ceilingEntry(val);
            final int leftDepth = l == null ? 0 : l.getValue();
            final int rightDepth = r == null ? 0 : r.getValue();
            final int depth = Math.max(leftDepth, rightDepth) + 1;
            ans = Math.max(ans, depth);
            valToDepth.put(val, depth);
        }

        return ans;
    }
}
```

918. 1903.java

Path: LeetCode-main/solutions/1903. Largest Odd Number in String/1903.java

```
class Solution {  
    public String largestOddNumber(String num) {  
        for (int i = num.length() - 1; i >= 0; --i)  
            if ((num.charAt(i) - '0') % 2 == 1)  
                return num.substring(0, i + 1);  
        return "";  
    }  
}
```

919. 1904.java

Path: LeetCode-main/solutions/1904. The Number of Full Rounds You Have Played/1904.java

```
class Solution {
    public int numberOfRounds(String loginTime, String logoutTime) {
        final int start = getMinutes(loginTime);
        int finish = getMinutes(logoutTime);
        if (start > finish)
            finish += 60 * 24;
        return Math.max(0, finish / 15 - (start + 14) / 15);
    }

    private int getMinutes(final String s) {
        return 60 * Integer.valueOf(s.substring(0, 2)) + Integer.valueOf(s.substring(3));
    }
}
```

920. 1905.java

Path: LeetCode-main/solutions/1905. Count Sub Islands/1905.java

```
class Solution {
    public int countSubIslands(int[][] grid1, int[][] grid2) {
        int ans = 0;

        for (int i = 0; i < grid2.length; ++i)
            for (int j = 0; j < grid2[0].length; ++j)
                if (grid2[i][j] == 1)
                    ans += dfs(grid1, grid2, i, j);

        return ans;
    }

    private int dfs(int[][] grid1, int[][] grid2, int i, int j) {
        if (i < 0 || i == grid1.length || j < 0 || j == grid2[0].length)
            return 1;
        if (grid2[i][j] != 1)
            return 1;

        grid2[i][j] = 2; // Mark 2 as visited.

        return
            dfs(grid1, grid2, i + 1, j) & dfs(grid1, grid2, i - 1, j) & //
            dfs(grid1, grid2, i, j + 1) & dfs(grid1, grid2, i, j - 1) & grid1[i][j];
    }
}
```

921. 1906.java

Path: LeetCode-main/solutions/1906. Minimum Absolute Difference Queries/1906.java

```
class Solution {
    public int[] minDifference(int[] nums, int[][] queries) {
        int[] ans = new int[queries.length];
        List<Integer>[] numToIndices = new List[101];

        for (int i = 1; i <= 100; ++i)
            numToIndices[i] = new ArrayList<>();

        for (int i = 0; i < nums.length; ++i)
            numToIndices[nums[i]].add(i);

        if (numToIndices[nums[0]].size() == nums.length) {
            Arrays.fill(ans, -1);
            return ans;
        }

        for (int i = 0; i < queries.length; ++i) {
            final int l = queries[i][0];
            final int r = queries[i][1];
            int prevNum = -1;
            int minDiff = 101;
            for (int num = 1; num <= 100; ++num) {
                List<Integer> indices = numToIndices[num];
                final int j = firstGreaterEqual(indices, l);
                if (j == indices.size() || indices.get(j) > r)
                    continue;
                if (prevNum != -1)
                    minDiff = Math.min(minDiff, num - prevNum);
                prevNum = num;
            }
            ans[i] = minDiff == 101 ? -1 : minDiff;
        }

        return ans;
    }

    private int firstGreaterEqual(List<Integer> A, int target) {
        final int i = Collections.binarySearch(A, target);
        return i < 0 ? -i - 1 : i;
    }
}
```

922. 1908.java

Path: LeetCode-main/solutions/1908. Game of Nim/1908.java

```
class Solution {  
    public boolean nimGame(int[] piles) {  
        return Arrays.stream(piles).reduce((a, b) -> a ^ b).getAsInt() > 0;  
    }  
}
```

923. 1909.java

Path: LeetCode-main/solutions/1909. Remove One Element to Make the Array Strictly Increasing/1909.java

```
class Solution {
    public boolean canBeIncreasing(int[] nums) {
        boolean removed = false;

        for (int i = 1; i < nums.length; ++i)
            if (nums[i - 1] >= nums[i]) {
                if (removed)
                    return false;
                removed = true; // Remove nums[i - 1].
                if (i > 1 && nums[i - 2] >= nums[i])
                    nums[i] = nums[i - 1]; // Remove nums[i] instead.
            }

        return true;
    }
}
```


924. 191-2.java

Path: LeetCode-main/solutions/191. Number of 1 Bits/191-2.java

```
class Solution {  
    // You need to treat n as an unsigned value  
    public int hammingWeight(int n) {  
        return Integer.bitCount(n);  
    }  
}
```

925. 191.java

Path: LeetCode-main/solutions/191. Number of 1 Bits/191.java

```
class Solution {
    // You need to treat n as an unsigned value
    public int hammingWeight(int n) {
        int ans = 0;

        for (int i = 0; i < 32; ++i)
            if (((n >> i) & 1) == 1)
                ++ans;

        return ans;
    }
}
```

926. 1910.java

Path: LeetCode-main/solutions/1910. Remove All Occurrences of a Substring/1910.java

```
class Solution {
    public String removeOccurrences(String s, String part) {
        final int n = s.length();
        final int k = part.length();

        StringBuilder sb = new StringBuilder(s);
        int j = 0; // sb's index

        for (int i = 0; i < n; ++i) {
            sb.setCharAt(j++, s.charAt(i));
            if (j >= k && sb.substring(j - k, j).toString().equals(part))
                j -= k;
        }

        return sb.substring(0, j).toString();
    }
}
```

927. 1911.java

Path: LeetCode-main/solutions/1911. Maximum Alternating Subsequence Sum/1911.java

```
class Solution {
    public long maxAlternatingSum(int[] nums) {
        long even = 0; // the maximum alternating sum ending in an even index
        long odd = 0;  // the maximum alternating sum ending in an odd index

        for (final int num : nums) {
            even = Math.max(even, odd + num);
            odd = even - num;
        }

        return even;
    }
}
```

928. 1912.java

Path: LeetCode-main/solutions/1912. Design Movie Rental System/1912.java

```
class MovieRentingSystem {
    public MovieRentingSystem(int n, int[][] entries) {
        for (int[] e : entries) {
            final int shop = e[0];
            final int movie = e[1];
            final int price = e[2];
            unrented.putIfAbsent(movie, new TreeSet<>(comparator));
            unrented.get(movie).add(new Entry(price, shop, movie));
            shopAndMovieToPrice.put(new Pair<>(shop, movie), price);
        }
    }

    public List<Integer> search(int movie) {
        return unrented.getDefault(movie, Collections.emptySet())
            .stream()
            .limit(5)
            .map(e -> e.shop)
            .collect(Collectors.toList());
    }

    public void rent(int shop, int movie) {
        final int price = shopAndMovieToPrice.get(new Pair<>(shop, movie));
        unrented.get(movie).remove(new Entry(price, shop, movie));
        rented.add(new Entry(price, shop, movie));
    }

    public void drop(int shop, int movie) {
        final int price = shopAndMovieToPrice.get(new Pair<>(shop, movie));
        unrented.get(movie).add(new Entry(price, shop, movie));
        rented.remove(new Entry(price, shop, movie));
    }

    public List<List<Integer>> report() {
        return rented.stream().limit(5).map(e -> List.of(e.shop, e.movie)).collect(Collectors.toList());
    }

    private record Entry(int price, int shop, int movie) {}

    private Comparator<Entry> comparator = Comparator.comparingInt(Entry::price)
        .thenComparingInt(Entry::shop)
        .thenComparingInt(Entry::movie);

    // {movie: (price, shop)}
    private Map<Integer, Set<Entry>> unrented = new HashMap<>();

    // {(shop, movie): price}
    private Map<Pair<Integer, Integer>, Integer> shopAndMovieToPrice = new HashMap<>();

    // (price, shop, movie)
    private Set<Entry> rented = new TreeSet<>(comparator);
}
```

929. 1913.java

Path: LeetCode-main/solutions/1913. Maximum Product Difference Between Two Pairs/1913.java

```
class Solution {
    public int maxProductDifference(int[] nums) {
        int max1 = Integer.MIN_VALUE;
        int max2 = Integer.MIN_VALUE;
        int min1 = Integer.MAX_VALUE;
        int min2 = Integer.MAX_VALUE;

        for (final int num : nums) {
            if (num > max1) {
                max2 = max1;
                max1 = num;
            } else if (num > max2) {
                max2 = num;
            }
            if (num < min1) {
                min2 = min1;
                min1 = num;
            } else if (num < min2) {
                min2 = num;
            }
        }

        return max1 * max2 - min1 * min2;
    }
}
```

930. 1914.java

Path: LeetCode-main/solutions/1914. Cyclically Rotating a Grid/1914.java

```
class Solution {
    public int[][] rotateGrid(int[][] grid, int k) {
        final int m = grid.length;
        final int n = grid[0].length;
        int t = 0; // the top
        int l = 0; // the left
        int b = m - 1; // the bottom
        int r = n - 1; // the right

        while (t < b && l < r) {
            final int elementInThisLayer = 2 * (b - t + 1) + 2 * (r - l + 1) - 4;
            final int netRotations = k % elementInThisLayer;
            for (int rotate = 0; rotate < netRotations; ++rotate) {
                final int topLeft = grid[t][l];
                for (int j = l; j < r; ++j)
                    grid[t][j] = grid[t][j + 1];
                for (int i = t; i < b; ++i)
                    grid[i][r] = grid[i + 1][r];
                for (int j = r; j > l; --j)
                    grid[b][j] = grid[b][j - 1];
                for (int i = b; i > t; --i)
                    grid[i][l] = grid[i - 1][l];
                grid[t + 1][l] = topLeft;
            }
            ++t;
            ++l;
            --b;
            --r;
        }

        return grid;
    }
}
```

931. 1915.java

Path: LeetCode-main/solutions/1915. Number of Wonderful Substrings/1915.java

```
class Solution {
    public long wonderfulSubstrings(String word) {
        long ans = 0;
        int prefix = 0;           // the binary prefix
        int[] count = new int[1024]; // the binary prefix count
        count[0] = 1;           // the empty string ""

        for (final char c : word.toCharArray()) {
            prefix ^= 1 << c - 'a';
            // All the letters occur even number of times.
            ans += count[prefix];
            // ('a' + i) occurs odd number of times.
            for (int i = 0; i < 10; ++i)
                ans += count[prefix ^ 1 << i];
            ++count[prefix];
        }

        return ans;
    }
}
```


932. 1918.java

Path: LeetCode-main/solutions/1918. Kth Smallest Subarray Sum/1918.java

```
class Solution {
    public int kthSmallestSubarraySum(int[] nums, int k) {
        int l = 0;
        int r = Arrays.stream(nums).sum();

        while (l < r) {
            final int m = (l + r) / 2;
            if (numSubarrayLessThan(nums, m) >= k)
                r = m;
            else
                l = m + 1;
        }

        return l;
    }

    private int numSubarrayLessThan(int[] nums, int m) {
        int res = 0;
        int sum = 0;
        for (int l = 0, r = 0; r < nums.length; ++r) {
            sum += nums[r];
            while (sum > m)
                sum -= nums[l++];
            res += r - l + 1;
        }
        return res;
    }
}
```

933. 1920.java

Path: LeetCode-main/solutions/1920. Build Array from Permutation/1920.java

```
class Solution {
    public int[] buildArray(int[] nums) {
        final int n = nums.length;

        for (int i = 0; i < n; ++i)
            nums[i] += n * (nums[nums[i]] % n);

        for (int i = 0; i < n; ++i)
            nums[i] /= n;

        return nums;
    }
}
```

934. 1921.java

Path: LeetCode-main/solutions/1921. Eliminate Maximum Number of Monsters/1921.java

```
class Solution {
    public int eliminateMaximum(int[] dist, int[] speed) {
        final int n = dist.length;
        int[] arrivalTime = new int[n];

        for (int i = 0; i < n; ++i)
            arrivalTime[i] = (dist[i] - 1) / speed[i];

        Arrays.sort(arrivalTime);

        for (int i = 0; i < n; ++i)
            if (i > arrivalTime[i])
                return i;

        return n;
    }
}
```

935. 1922.java

Path: LeetCode-main/solutions/1922. Count Good Numbers/1922.java

```
class Solution {
    public int countGoodNumbers(long n) {
        return (int) (modPow(4 * 5, n / 2) * (n % 2 == 0 ? 1 : 5) % MOD);
    }

    private static final int MOD = 1_000_000_007;

    private long modPow(long x, long n) {
        if (n == 0)
            return 1;
        if (n % 2 == 1)
            return x * modPow(x, n - 1) % MOD;
        return modPow(x * x % MOD, n / 2);
    }
}
```

936. 1923.java

Path: LeetCode-main/solutions/1923. Longest Common Subpath/1923.java

```
class Solution {
    public int longestCommonSubpath(int n, int[][] paths) {
        int l = 0;
        int r = paths[0].length;

        while (l < r) {
            final int m = l + (r - l + 1) / 2;
            if (checkCommonSubpath(paths, m))
                l = m;
            else
                r = m - 1;
        }

        return l;
    }

    private static final long BASE = 165_131L;
    private static final long HASH = 8_417_508_174_513L;

    // Returns true if there's a common subpath of length m for all the paths.
    private boolean checkCommonSubpath(int[][] paths, int m) {
        Set<Long>[] hashSets = new Set[paths.length];

        // Calculate the hash values for subpaths of length m for every path.
        for (int i = 0; i < paths.length; ++i)
            hashSets[i] = rabinKarp(paths[i], m);

        // Check if there is a common subpath of length m.
        for (final long subpathHash : hashSets[0])
            if (Arrays.stream(hashSets).allMatch(hashSet -> hashSet.contains(subpathHash)))
                return true;

        return false;
    }

    // Returns the hash values for subpaths of length m in the path.
    private Set<Long> rabinKarp(int[] path, int m) {
        Set<Long> hashes = new HashSet<>();
        long maxPower = 1;
        long hash = 0;
        for (int i = 0; i < path.length; ++i) {
            hash = (hash * BASE + path[i]) % HASH;
            if (i >= m)
                hash = (hash - path[i - m] * maxPower % HASH + HASH) % HASH;
            else
                maxPower = maxPower * BASE % HASH;
            if (i >= m - 1)
                hashes.add(hash);
        }
        return hashes;
    }
}
```

937. 1924.java

Path: LeetCode-main/solutions/1924. Erect the Fence II/1924.java

```
class Point {
    public double x;
    public double y;
    public Point(double x, double y) {
        this.x = x;
        this.y = y;
    }
}

class Disk {
    public Point center;
    public double radius;
    public Disk(Point center, double radius) {
        this.center = center;
        this.radius = radius;
    }
}

class Solution {
    public double[] outerTrees(int[][] trees) {
        Point[] points = new Point[trees.length];
        for (int i = 0; i < trees.length; ++i)
            points[i] = new Point(trees[i][0], trees[i][1]);
        Disk disk = welzl(points, 0, new ArrayList<>());
        return new double[] {disk.center.x, disk.center.y, disk.radius};
    }

    // Returns the smallest disk that encloses points[i..n].
    //
    // https://en.wikipedia.org/wiki/Smallest-disk_problem#Welzl's_algorithm
    private Disk welzl(Point[] points, int i, List<Point> planePoints) {
        if (i == points.length || planePoints.size() == 3)
            return trivial(planePoints);
        Disk disk = welzl(points, i + 1, planePoints);
        if (inside(disk, points[i]))
            return disk;
        return welzl(points, i + 1, addPlanePoints(planePoints, points[i]));
    }

    private List<Point> addPlanePoints(List<Point> planePoints, Point point) {
        List<Point> newPlanePoints = new ArrayList<>(planePoints);
        newPlanePoints.add(point);
        return newPlanePoints;
    }

    // Returns the smallest disk that encloses `planePoints`.
    private Disk trivial(List<Point> planePoints) {
        if (planePoints.isEmpty())
            return null;
        if (planePoints.size() == 1)
            return new Disk(new Point(planePoints.get(0).x, planePoints.get(0).y), 0);
        if (planePoints.size() == 2)
            return getDisk(planePoints.get(0), planePoints.get(1));

        Disk disk01 = getDisk(planePoints.get(0), planePoints.get(1));
        if (inside(disk01, planePoints.get(2)))
            return disk01;

        Disk disk02 = getDisk(planePoints.get(0), planePoints.get(2));
        if (inside(disk02, planePoints.get(1)))
            return disk02;

        Disk disk12 = getDisk(planePoints.get(1), planePoints.get(2));
        if (inside(disk12, planePoints.get(0)))
            return disk12;

        return getDisk(planePoints.get(0), planePoints.get(1), planePoints.get(2));
    }

    // Returns the smallest disk that encloses the points A and B.
    private Disk getDisk(Point A, Point B) {
        final double x = (A.x + B.x) / 2;
        final double y = (A.y + B.y) / 2;
        return new Disk(new Point(x, y), distance(A, B) / 2);
    }

    // Returns the smallest disk that encloses the points A, B, and C.
    private Disk getDisk(Point A, Point B, Point C) {
        // Calculate midpoints.
        Point mAB = new Point((A.x + B.x) / 2, (A.y + B.y) / 2);
    }
}
```

```

    Point mBC = new Point((B.x + C.x) / 2, (B.y + C.y) / 2);

    // Calculate the slopes and the perpendicular slopes.
    final double slopeAB = (B.y - A.y) / (B.x - A.x);
    final double slopeBC = (C.y - B.y) / (C.x - B.x);
    final double perpSlopeAB = -1 / slopeAB;
    final double perpSlopeBC = -1 / slopeBC;

    // Calculate the center.
    final double x =
        (perpSlopeBC * mBC.x - perpSlopeAB * mAB.x + mAB.y - mBC.y) / (perpSlopeBC - perpSlopeAB);
    final double y = perpSlopeAB * (x - mAB.x) + mAB.y;
    Point center = new Point(x, y);
    return new Disk(center, distance(center, A));
}

// Returns true if the point is inside the disk.
private boolean inside(Disk disk, Point point) {
    return disk != null && distance(disk.center, point) <= disk.radius;
}

private double distance(Point A, Point B) {
    final double dx = A.x - B.x;
    final double dy = A.y - B.y;
    return Math.sqrt(dx * dx + dy * dy);
}
}

```


939. 1926.java

Path: LeetCode-main/solutions/1926. Nearest Exit from Entrance in Maze/1926.java

```
class Solution {
    public int nearestExit(char[][] maze, int[] entrance) {
        final int[][] DIRS = {{0, 1}, {1, 0}, {0, -1}, {-1, 0}};
        final int m = maze.length;
        final int n = maze[0].length;
        Queue<Pair<Integer, Integer>> q =
            new ArrayDeque<>(List.of(new Pair<>(entrance[0], entrance[1])));
        boolean[][] seen = new boolean[m][n];
        seen[entrance[0]][entrance[1]] = true;

        for (int step = 1; !q.isEmpty(); ++step)
            for (int sz = q.size(); sz > 0; --sz) {
                final int i = q.peek().getKey();
                final int j = q.poll().getValue();
                for (int[] dir : DIRS) {
                    final int x = i + dir[0];
                    final int y = j + dir[1];
                    if (x < 0 || x == m || y < 0 || y == n)
                        continue;
                    if (seen[x][y] || maze[x][y] == '+')
                        continue;
                    if (x == 0 || x == m - 1 || y == 0 || y == n - 1)
                        return step;
                    q.offer(new Pair<>(x, y));
                    seen[x][y] = true;
                }
            }
        return -1;
    }
}
```

940. 1927.java

Path: LeetCode-main/solutions/1927. Sum Game/1927.java

```
class Solution {
    public boolean sumGame(String num) {
        final int n = num.length();
        double ans = 0.0;

        for (int i = 0; i < n / 2; ++i)
            ans += getExpectation(num.charAt(i));

        for (int i = n / 2; i < n; ++i)
            ans -= getExpectation(num.charAt(i));

        return ans != 0.0;
    }

    private double getExpectation(char c) {
        return c == '?' ? 4.5 : c - '0';
    }
}
```

941. 1928.java

Path: LeetCode-main/solutions/1928. Minimum Cost to Reach Destination in Time/1928.java

```
class Solution {
    public int minCost(int maxTime, int[][] edges, int[] passingFees) {
        final int n = passingFees.length;
        List<Pair<Integer, Integer>>[] graph = new List[n];
        Arrays.setAll(graph, i -> new ArrayList<>());

        for (int[] edge : edges) {
            final int u = edge[0];
            final int v = edge[1];
            final int t = edge[2];
            graph[u].add(new Pair<>(v, t));
            graph[v].add(new Pair<>(u, t));
        }

        return dijkstra(graph, 0, n - 1, maxTime, passingFees);
    }

    private int dijkstra(List<Pair<Integer, Integer>>[] graph, int src, int dst, int maxTime,
                        int[] passingFees) {
        int[] cost = new int[graph.length]; // cost[i] := the minimum cost to reach the i-th city
        int[] dist = new int[graph.length]; // dist[i] := the minimum distance to reach the i-th city
        Arrays.fill(cost, Integer.MAX_VALUE);
        Arrays.fill(dist, maxTime + 1);

        cost[0] = passingFees[0];
        dist[0] = 0;
        Queue<int[]> minHeap = new PriorityQueue<>(Comparator.comparingInt(a -> a[0])) {
            { offer(new int[] {cost[src], dist[src], src}); } // (cost[u], dist[u], u)
        };

        while (!minHeap.isEmpty()) {
            final int currCost = minHeap.peek()[0];
            final int d = minHeap.peek()[1];
            final int u = minHeap.poll()[2];
            if (u == dst)
                return cost[dst];
            if (d > dist[u] && currCost > cost[u])
                continue;
            for (Pair<Integer, Integer> pair : graph[u]) {
                final int v = pair.getKey();
                final int w = pair.getValue();
                if (d + w > maxTime)
                    continue;
                // Go from x -> y
                if (currCost + passingFees[v] < cost[v]) {
                    cost[v] = currCost + passingFees[v];
                    dist[v] = d + w;
                    minHeap.offer(new int[] {cost[v], dist[v], v});
                } else if (d + w < dist[v]) {
                    dist[v] = d + w;
                    minHeap.offer(new int[] {currCost + passingFees[v], dist[v], v});
                }
            }
        }

        return -1;
    }
}
```

942. 1929.java

Path: LeetCode-main/solutions/1929. Concatenation of Array/1929.java

```
class Solution {  
    public int[] getConcatenation(int[] nums) {  
        final int n = nums.length;  
  
        int[] ans = new int[n * 2];  
  
        for (int i = 0; i < n; ++i)  
            ans[i] = ans[i + n] = nums[i];  
  
        return ans;  
    }  
}
```

943. 1930.java

Path: LeetCode-main/solutions/1930. Unique Length-3 Palindromic Subsequences/1930.java

```
class Solution {
    public int countPalindromicSubsequence(String s) {
        int ans = 0;
        int[] first = new int[26];
        int[] last = new int[26];

        Arrays.fill(first, s.length());
        Arrays.fill(last, -1);

        for (int i = 0; i < s.length(); ++i) {
            final int index = s.charAt(i) - 'a';
            first[index] = Math.min(first[index], i);
            last[index] = i;
        }

        for (int i = 0; i < 26; ++i)
            if (first[i] < last[i])
                ans += s.substring(first[i] + 1, last[i]).chars().distinct().count();

        return ans;
    }
}
```

944. 1931.java

Path: LeetCode-main/solutions/1931. Painting a Grid With Three Different Colors/1931.java

```
class Solution {
    public int colorTheGrid(int m, int n) {
        this.m = m;
        this.n = n;
        return dp(0, 0, 0, 0);
    }

    private static final int MOD = 1_000_000_007;
    private int m;
    private int n;
    private int[][] mem = new int[1000][1024];

    private int dp(int r, int c, int prevColMask, int currColMask) {
        if (c == n)
            return 1;
        if (mem[c][prevColMask] != 0)
            return mem[c][prevColMask];
        if (r == m)
            return dp(0, c + 1, currColMask, 0);

        int ans = 0;

        // 1 := red, 2 := green, 3 := blue
        for (int color = 1; color <= 3; ++color) {
            if (getColor(prevColMask, r) == color)
                continue;
            if (r > 0 && getColor(currColMask, r - 1) == color)
                continue;
            ans += dp(r + 1, c, prevColMask, setColor(currColMask, r, color));
            ans %= MOD;
        }

        if (r == 0)
            mem[c][prevColMask] = ans;

        return ans;
    }

    // e.g.  01 10 11 11 11
    //       R  G  B  B  B
    // getColor(0110111111, 3) -> G
    private int getColor(int mask, int r) {
        return mask >> r * 2 & 3;
    }

    private int setColor(int mask, int r, int color) {
        return mask | color << r * 2;
    }
}
```

945. 1932.java

Path: LeetCode-main/solutions/1932. Merge BSTs to Create Single BST/1932.java

```
class Solution {
    public TreeNode canMerge(List<TreeNode> trees) {
        Map<Integer, TreeNode> valToNode = new HashMap<>(); // {val: node}
        Map<Integer, Integer> count = new HashMap<>();      // {val: freq}

        for (TreeNode tree : trees) {
            valToNode.put(tree.val, tree);
            count.merge(tree.val, 1, Integer::sum);
            if (tree.left != null)
                count.merge(tree.left.val, 1, Integer::sum);
            if (tree.right != null)
                count.merge(tree.right.val, 1, Integer::sum);
        }

        for (TreeNode tree : trees)
            if (count.get(tree.val) == 1) {
                if (isValidBST(tree, null, null, valToNode) && valToNode.size() <= 1)
                    return tree;
                return null;
            }

        return null;
    }

    private boolean isValidBST(TreeNode tree, TreeNode minNode, TreeNode maxNode,
                               Map<Integer, TreeNode> valToNode) {
        if (tree == null)
            return true;
        if (minNode != null && tree.val <= minNode.val)
            return false;
        if (maxNode != null && tree.val >= maxNode.val)
            return false;
        if (tree.left == null && tree.right == null && valToNode.containsKey(tree.val)) {
            final int val = tree.val;
            tree.left = valToNode.get(val).left;
            tree.right = valToNode.get(val).right;
            valToNode.remove(val);
        }

        return
            isValidBST(tree.left, minNode, tree, valToNode) && //
            isValidBST(tree.right, tree, maxNode, valToNode);
    }
}
```

946. 1933.java

Path: LeetCode-main/solutions/1933. Check if String Is Decomposable Into Value-Equal Substrings/1933.java

```
class Solution {
    public boolean isDecomposable(String s) {
        int twos = 0;
        int groupLength = 0;
        char letter = '@'; // the running letter

        for (final char c : s.toCharArray())
            if (c == letter) {
                ++groupLength;
            } else {
                if (groupLength % 3 == 1)
                    return false;
                if (groupLength % 3 == 2 && ++twos > 1)
                    return false;
                groupLength = 1;
                letter = c;
            }

        // Check the final group.
        if (groupLength % 3 == 1)
            return false;
        if (groupLength % 3 == 2 && ++twos > 1)
            return false;
        return twos == 1;
    }
}
```


947. 1935.java

Path: LeetCode-main/solutions/1935. Maximum Number of Words You Can Type/1935.java

```
class Solution {
    public int canBeTypedWords(String text, String brokenLetters) {
        int ans = 0;
        boolean[] broken = new boolean[26];

        for (final char c : brokenLetters.toCharArray())
            broken[c - 'a'] = true;

        for (final String word : text.split(" "))
            ans += canBeTyped(word, broken);

        return ans;
    }

    private int canBeTyped(final String word, boolean[] broken) {
        for (final char c : word.toCharArray())
            if (broken[c - 'a'])
                return 0;
        return 1;
    }
}
```

948. 1936.java

Path: LeetCode-main/solutions/1936. Add Minimum Number of Rungs/1936.java

```
class Solution {
    public int addRungs(int[] rungs, int dist) {
        int ans = 0;
        int prev = 0;

        for (final int rung : rungs) {
            ans += (rung - prev - 1) / dist;
            prev = rung;
        }

        return ans;
    }
}
```

949. 1937.java

Path: LeetCode-main/solutions/1937. Maximum Number of Points with Cost/1937.java

```
class Solution {
    public long maxPoints(int[][] points) {
        final int n = points[0].length;
        // dp[j] := the maximum number of points you can have if points[i][j] is the
        // most recent cell you picked
        long[] dp = new long[n];

        for (int[] row : points) {
            long[] leftToRight = new long[n];
            long runningMax = 0;

            for (int j = 0; j < n; ++j) {
                runningMax = Math.max(runningMax - 1, dp[j]);
                leftToRight[j] = runningMax;
            }

            long[] rightToLeft = new long[n];
            runningMax = 0;

            for (int j = n - 1; j >= 0; --j) {
                runningMax = Math.max(runningMax - 1, dp[j]);
                rightToLeft[j] = runningMax;
            }

            for (int j = 0; j < n; ++j)
                dp[j] = Math.max(leftToRight[j], rightToLeft[j]) + row[j];
        }

        return Arrays.stream(dp).max().getAsLong();
    }
}
```

950. 1938.java

Path: LeetCode-main/solutions/1938. Maximum Genetic Difference Query/1938.java

```
class TrieNode {
    public TrieNode[] children = new TrieNode[2];
    public int count = 0;
}

class Trie {
    public void update(int num, int val) {
        TrieNode node = root;
        for (int i = HEIGHT; i >= 0; --i) {
            final int bit = (num >> i) & 1;
            if (node.children[bit] == null)
                node.children[bit] = new TrieNode();
            node = node.children[bit];
            node.count += val;
        }
    }

    public int query(int num) {
        int ans = 0;
        TrieNode node = root;
        for (int i = HEIGHT; i >= 0; --i) {
            final int bit = (num >> i) & 1;
            final int targetBit = bit ^ 1;
            if (node.children[targetBit] != null && node.children[targetBit].count > 0) {
                ans += 1 << i;
                node = node.children[targetBit];
            } else {
                node = node.children[bit ^ 1];
            }
        }
        return ans;
    }

    private static final int HEIGHT = 17;
    TrieNode root = new TrieNode();
}

class Solution {
    public int[] maxGeneticDifference(int[] parents, int[][] queries) {
        final int n = parents.length;
        int[] ans = new int[queries.length];
        int rootVal = -1;
        List<Integer>[] tree = new List[n];

        for (int i = 0; i < n; ++i)
            tree[i] = new ArrayList<>();

        // {node: (index, val)}
        Map<Integer, List<Pair<Integer, Integer>>> nodeToQueries = new HashMap<>();
        Trie trie = new Trie();

        for (int i = 0; i < parents.length; ++i)
            if (parents[i] == -1)
                rootVal = i;
            else
                tree[parents[i]].add(i);

        for (int i = 0; i < queries.length; ++i) {
            final int node = queries[i][0];
            final int val = queries[i][1];
            nodeToQueries.putIfAbsent(node, new ArrayList<>());
            nodeToQueries.get(node).add(new Pair<>(i, val));
        }

        dfs(rootVal, trie, tree, nodeToQueries, ans);
        return ans;
    }

    private void dfs(int node, Trie trie, List<Integer>[] tree,
                    Map<Integer, List<Pair<Integer, Integer>>> nodeToQueries, int[] ans) {
        trie.update(node, 1);

        if (nodeToQueries.containsKey(node))
            for (Pair<Integer, Integer> query : nodeToQueries.get(node)) {
                final int i = query.getKey();
                final int val = query.getValue();
                ans[i] = trie.query(val);
            }

        for (final int child : tree[node])
```

```
        dfs(child, trie, tree, nodeToQueries, ans);
    trie.update(node, -1);
}
```

951. 1940.java

Path: LeetCode-main/solutions/1940. Longest Common Subsequence Between Sorted Arrays/1940.java

```
class Solution {
    public List<Integer> longestCommonSubsequence(int[][] arrays) {
        final int MAX = 100;
        List<Integer> ans = new ArrayList<>();
        int[] count = new int[MAX + 1];

        for (int[] array : arrays)
            for (final int a : array)
                if (++count[a] == arrays.length)
                    ans.add(a);

        return ans;
    }
}
```

952. 1941.java

Path: LeetCode-main/solutions/1941. Check if All Characters Have Equal Number of Occurrences/1941.java

```
class Solution {
    public boolean areOccurrencesEqual(String s) {
        int[] count = new int[26];
        for (final char c : s.toCharArray())
            ++count[c - 'a'];
        return equalFreq(count, count[s.charAt(0) - 'a']);
    }

    private boolean equalFreq(int[] count, int theFreq) {
        return Arrays.stream(count).allMatch(freq -> freq == 0 || freq == theFreq);
    }
}
```

953. 1942.java

Path: LeetCode-main/solutions/1942. The Number of the Smallest Unoccupied Chair/1942.java

```
class Solution {
    public int smallestChair(int[][] times, int targetFriend) {
        int nextUnsatChair = 0;
        PriorityQueue<Integer> emptyChairs = new PriorityQueue<>();
        PriorityQueue<Pair<Integer, Integer>> occupied =
            new PriorityQueue<>(Comparator.comparingInt(Pair::getKey));

        for (int i = 0; i < times.length; ++i) {
            int[] time = times[i];
            time = Arrays.copyOf(time, time.length + 1);
            time[time.length - 1] = i;
            times[i] = time;
        }

        Arrays.sort(times, Comparator.comparingInt(time -> time[0]));

        for (int[] time : times) {
            final int arrival = time[0];
            final int leaving = time[1];
            final int i = time[2];
            while (!occupied.isEmpty() && occupied.peek().getKey() <= arrival)
                emptyChairs.add(occupied.poll().getValue());
            if (i == targetFriend)
                return emptyChairs.isEmpty() ? nextUnsatChair : emptyChairs.peek();
            if (emptyChairs.isEmpty())
                occupied.add(new Pair<>(leaving, nextUnsatChair++));
            else
                occupied.add(new Pair<>(leaving, emptyChairs.poll()));
        }

        throw new IllegalArgumentException();
    }
}
```


954. 1943.java

Path: LeetCode-main/solutions/1943. Describe the Painting/1943.java

```
class Solution {
    public List<List<Long>> splitPainting(int[][] segments) {
        List<List<Long>> ans = new ArrayList<>();
        int prevIndex = 0;
        long runningMix = 0;
        TreeMap<Integer, Long> line = new TreeMap<>();

        for (int[] segment : segments) {
            final int start = segment[0];
            final int end = segment[1];
            final int color = segment[2];
            line.merge(start, (long) color, Long::sum);
            line.merge(end, (long) -color, Long::sum);
        }

        for (Map.Entry<Integer, Long> entry : line.entrySet()) {
            final int i = entry.getKey();
            final long mix = entry.getValue();
            if (runningMix > 0)
                ans.add(List.of((long) prevIndex, (long) i, runningMix));
            runningMix += mix;
            prevIndex = i;
        }

        return ans;
    }
}
```

955. 1944.java

Path: LeetCode-main/solutions/1944. Number of Visible People in a Queue/1944.java

```
class Solution {
    public int[] canSeePersonsCount(int[] heights) {
        final int n = heights.length;
        int[] ans = new int[n];
        Deque<Integer> stack = new ArrayDeque<>();

        for (int i = 0; i < n; ++i) {
            while (!stack.isEmpty() && heights[stack.peek()] <= heights[i])
                ++ans[stack.pop()];
            if (!stack.isEmpty())
                ++ans[stack.peek()];
            stack.push(i);
        }

        return ans;
    }
}
```

956. 1945.java

Path: LeetCode-main/solutions/1945. Sum of Digits of String After Convert/1945.java

```
class Solution {
    public int getLucky(String s, int k) {
        int ans = convert(s);
        for (int i = 1; i < k; ++i)
            ans = getDigitSum(ans);
        return ans;
    }

    private int convert(final String s) {
        int sum = 0;
        for (final char c : s.toCharArray()) {
            final int val = c - 'a' + 1;
            // Do one transform to prevent integer overflow.
            sum += val < 10 ? val : (val % 10 + val / 10);
        }
        return sum;
    }

    private int getDigitSum(int num) {
        int digitSum = 0;
        while (num > 0) {
            digitSum += num % 10;
            num /= 10;
        }
        return digitSum;
    }
}
```

957. 1946.java

Path: LeetCode-main/solutions/1946. Largest Number After Mutating Substring/1946.java

```
class Solution {
    public String maximumNumber(String num, int[] change) {
        StringBuilder sb = new StringBuilder(num);
        boolean mutated = false;

        for (int i = 0; i < num.length(); ++i) {
            final int d = num.charAt(i) - '0';
            final char c = (char) ('0' + Math.max(d, change[d]));
            sb.setCharAt(i, c);
            if (mutated && d > change[d])
                return sb.toString();
            if (d < change[d])
                mutated = true;
        }

        return sb.toString();
    }
}
```

958. 1947.java

Path: LeetCode-main/solutions/1947. Maximum Compatibility Score Sum/1947.java

```
class Solution {
    public int maxCompatibilitySum(int[][] students, int[][] mentors) {
        dfs(students, mentors, 0, /*score=*/0, new boolean[students.length]);
        return ans;
    }

    private int ans = 0;

    private void dfs(int[][] students, int[][] mentors, int i, int scoreSum, boolean[] used) {
        if (i == students.length) {
            ans = Math.max(ans, scoreSum);
            return;
        }
        for (int j = 0; j < students.length; ++j) {
            if (used[j])
                continue;
            used[j] = true; // The `mentors[j]` is used.
            dfs(students, mentors, i + 1, scoreSum + getScore(students[i], mentors[j]), used);
            used[j] = false;
        }
    }

    private int getScore(int[] student, int[] mentor) {
        int score = 0;
        for (int i = 0; i < student.length; ++i)
            if (student[i] == mentor[i])
                ++score;
        return score;
    }
}
```

959. 1948.java

Path: LeetCode-main/solutions/1948. Delete Duplicate Folders in System/1948.java

```
class TrieNode {
    public Map<String, TrieNode> children = new HashMap<>();
    public boolean deleted = false;
}

class Solution {
    public List<List<String>> deleteDuplicateFolder(List<List<String>> paths) {
        List<List<String>> ans = new ArrayList<>();
        Map<String, List<TrieNode>> subtreeToNodes = new HashMap<>();

        Collections.sort(paths,
            Comparator.<List<String>>comparingInt(List::size).thenComparing((a, b) -> {
                for (int i = 0; i < Math.min(a.size(), b.size()); ++i) {
                    final int c = a.get(i).compareTo(b.get(i));
                    if (c != 0)
                        return c;
                }
                return 0;
            }));

        for (List<String> path : paths) {
            TrieNode node = root;
            for (final String s : path) {
                node.children.putIfAbsent(s, new TrieNode());
                node = node.children.get(s);
            }
        }

        buildSubtreeToRoots(root, subtreeToNodes);

        for (List<TrieNode> nodes : subtreeToNodes.values())
            if (nodes.size() > 1)
                for (TrieNode node : nodes)
                    node.deleted = true;

        constructPath(root, new ArrayList<>(), ans);
        return ans;
    }

    private TrieNode root = new TrieNode();

    private StringBuilder buildSubtreeToRoots(TrieNode node,
        Map<String, List<TrieNode>> subtreeToNodes) {
        StringBuilder sb = new StringBuilder("(");
        for (final String s : node.children.keySet()) {
            TrieNode child = node.children.get(s);
            sb.append(s).append(buildSubtreeToRoots(child, subtreeToNodes));
        }
        sb.append(")");
        final String subtree = sb.toString();
        if (!subtree.equals("(")) {
            subtreeToNodes.putIfAbsent(subtree, new ArrayList<>());
            subtreeToNodes.get(subtree).add(node);
        }
        return sb;
    }

    private void constructPath(TrieNode node, List<String> path, List<List<String>> ans) {
        for (final String s : node.children.keySet()) {
            TrieNode child = node.children.get(s);
            if (!child.deleted) {
                path.add(s);
                constructPath(child, path, ans);
                path.remove(path.size() - 1);
            }
        }
        if (!path.isEmpty())
            ans.add(new ArrayList<>(path));
    }
}
```

960. 1950.java

Path: LeetCode-main/solutions/1950. Maximum of Minimum Values in All Subarrays/1950.java

```
class Solution {
    // Similar to 1950. Maximum of Minimum Values in All Subarrays
    public int[] findMaximums(int[] nums) {
        final int n = nums.length;
        int[] ans = new int[n];
        // prevMin[i] := the index k s.t.
        // nums[k] is the previous minimum in nums[0..n)
        int[] prevMin = new int[n];
        // nextMin[i] := the index k s.t.
        // nums[k] is the next minimum in nums[i + 1..n)
        int[] nextMin = new int[n];
        Deque<Integer> stack = new ArrayDeque<>();

        Arrays.fill(prevMin, -1);
        Arrays.fill(nextMin, n);

        for (int i = 0; i < n; ++i) {
            while (!stack.isEmpty() && nums[stack.peek()] > nums[i]) {
                final int index = stack.pop();
                nextMin[index] = i;
            }
            if (!stack.isEmpty())
                prevMin[i] = stack.peek();
            stack.push(i);
        }

        // For each nums[i], let l = nextMin[i] + 1 and r = nextMin[i] - 1.
        // nums[i] is the minimum in nums[l..r].
        // So, the ans[r - l + 1] will be at least nums[i].
        for (int i = 0; i < n; ++i) {
            final int sz = nextMin[i] - prevMin[i] - 1;
            ans[sz - 1] = Math.max(ans[sz - 1], nums[i]);
        }

        // ans[i] should always >= ans[i + 1..n).
        for (int i = n - 2; i >= 0; --i)
            ans[i] = Math.max(ans[i], ans[i + 1]);

        return ans;
    }
}
```

961. 1952.java

Path: LeetCode-main/solutions/1952. Three Divisors/1952.java

```
class Solution {
    public boolean isThree(int n) {
        if (n == 1)
            return false;
        // The numbers with exactly three divisors are perfect squares of a prime
        // number.
        final int root = (int) Math.sqrt(n);
        return root * root == n && isPrime(root);
    }

    private boolean isPrime(int num) {
        for (int i = 2; i <= Math.sqrt(num); ++i)
            if (num % i == 0)
                return false;
        return true;
    }
}
```


962. 1953.java

Path: LeetCode-main/solutions/1953. Maximum Number of Weeks for Which You Can Work/1953.java

```
class Solution {
    public long numberOfWeeks(int[] milestones) {
        // The best strategy is to pick "max, nonMax, max, nonMax, ...".
        final int mx = Arrays.stream(milestones).max().getAsInt();
        final long sum = Arrays.stream(milestones).asLongStream().sum();
        final long nonMax = sum - mx;
        return Math.min(sum, 2 * nonMax + 1);
    }
}
```

963. 1954.java

Path: LeetCode-main/solutions/1954. Minimum Garden Perimeter to Collect Enough Apples/1954.java

```
class Solution {
    public long minimumPerimeter(long neededApples) {
        long l = 1;
        long r = 100_000; // \sqrt [3] {10^{15}}

        while (l < r) {
            final long m = (l + r) / 2;
            if (numApples(m) >= neededApples)
                r = m;
            else
                l = m + 1;
        }

        return l * 8;
    }

    // Returns the number of apples at the k-th level.
    // k := the level making perimeter = 8k
    // p(k) := the number of apples at the k-th level on the perimeter
    // n(k) := the number of apples at the k-th level not on the perimeter
    //
    // p(1) = 1 + 2
    // p(2) = 3 + 2 + 3 + 4
    // p(3) = 5 + 4 + 3 + 4 + 5 + 6
    // p(4) = 7 + 6 + 5 + 4 + 5 + 6 + 7 + 8
    // p(k) = k + 2(k+1) + 2(k+2) + ... + 2(k+k-1) + 2k
    //       = k + 2k^2 + 2k(k-1)/2
    //       = k + 2k^2 + k^2 - k = 3k^2
    //
    // n(k) = p(1) + p(2) + p(3) + ... + p(k)
    //       = 3*1 + 3*4 + 3*9 + ... + 3k^2
    //       = 3 * (1 + 4 + 9 + ... + k^2)
    //       = 3 * k(k+1)(2k+1)/6 = k(k+1)(2k+1)/2
    // So, the number of apples at the k-th level should be
    // k(k+1)(2k+1)/2 * 4 = 2k(k+1)(2k+1)
    private long numApples(long k) {
        return 2L * k * (k + 1) * (2 * k + 1);
    }
}
```

964. 1955-2.java

Path: LeetCode-main/solutions/1955. Count Number of Special Subsequences/1955-2.java

```
class Solution {
    public int countSpecialSubsequences(int[] nums) {
        final int MOD = 1_000_000_007;
        final int n = nums.length;
        // dp[i][j] := the number of increasing subsequences of the first i numbers
        // that end in j
        long[][] dp = new long[n][3];

        if (nums[0] == 0)
            dp[0][0] = 1;

        for (int i = 1; i < n; ++i) {
            for (int ending = 0; ending < 3; ++ending)
                dp[i][ending] = dp[i - 1][ending];

            if (nums[i] == 0)
                // 1. The number of the previous subsequences that end in 0.
                // 2. Append a 0 to the previous subsequences that end in 0.
                // 3. Start a new subsequence from this 0.
                dp[i][0] = dp[i - 1][0] * 2 + 1;
            else if (nums[i] == 1)
                // 1. The number of the previous subsequences that end in 1.
                // 2. Append a 1 to the previous subsequences that end in 1.
                // 3. Append a 1 to the previous subsequences that end in 0.
                dp[i][1] = dp[i - 1][1] * 2 + dp[i - 1][0];
            else // nums[i] == 2
                // 1. The number of the previous subsequences that end in 2.
                // 2. Append a 2 to the previous subsequences that end in 2.
                // 3. Append a 2 to the previous subsequences that end in 1.
                dp[i][2] = dp[i - 1][2] * 2 + dp[i - 1][1];

            for (int ending = 0; ending < 3; ++ending)
                dp[i][ending] %= MOD;
        }

        return (int) dp[n - 1][2];
    }
}
```

965. 1955-3.java

Path: LeetCode-main/solutions/1955. Count Number of Special Subsequences/1955-3.java

```
class Solution {
    public int countSpecialSubsequences(int[] nums) {
        final int MOD = 1_000_000_007;
        final int n = nums.length;
        // dp[j] := the number of increasing subsequences of the numbers so far that
        // end in j
        long[] dp = new long[3];

        if (nums[0] == 0)
            dp[0] = 1;

        for (int i = 1; i < n; ++i) {
            if (nums[i] == 0)
                dp[0] = dp[0] * 2 + 1;
            else if (nums[i] == 1)
                dp[1] = dp[1] * 2 + dp[0];
            else // nums[i] == 2
                dp[2] = dp[2] * 2 + dp[1];

            for (int ending = 0; ending < 3; ++ending)
                dp[ending] %= MOD;
        }

        return (int) dp[2];
    }
}
```

966. 1955.java

Path: LeetCode-main/solutions/1955. Count Number of Special Subsequences/1955.java

```
class Solution {
    public int countSpecialSubsequences(int[] nums) {
        Integer[][] mem = new Integer[nums.length][4];
        return countSpecialSubsequences(nums, 0, -1, mem);
    }

    private static final int MOD = 1_000_000_007;

    // Returns the number of increasing subsequences of the first i numbers, where
    // the previous number is `prev`.
    int countSpecialSubsequences(int[] nums, int i, int prev, Integer[][] mem) {
        if (i == nums.length)
            return prev == 2 ? 1 : 0;
        final int j = prev + 1; // Map -1/0/1/2 to 0/1/2/3 respectively.
        if (mem[i][j] != null)
            return mem[i][j];

        long res = 0;

        // Don't include `nums[i]`.
        res += countSpecialSubsequences(nums, i + 1, prev, mem);

        // Include `nums[i]`.
        if (nums[i] == prev)
            res += countSpecialSubsequences(nums, i + 1, prev, mem);
        if (prev == -1 && nums[i] == 0)
            res += countSpecialSubsequences(nums, i + 1, 0, mem);
        if (prev == 0 && nums[i] == 1)
            res += countSpecialSubsequences(nums, i + 1, 1, mem);
        if (prev == 1 && nums[i] == 2)
            res += countSpecialSubsequences(nums, i + 1, 2, mem);

        res %= MOD;
        return mem[i][j] = (int) res;
    }
}
```

967. 1956.java

Path: LeetCode-main/solutions/1956. Minimum Time For K Virus Variants to Spread/1956.java

```
class Solution {
    public int minDayskVariants(int[][] points, int k) {
        final int MAX = 100;
        int ans = Integer.MAX_VALUE;

        for (int a = 1; a <= MAX; ++a)
            for (int b = 1; b <= MAX; ++b) {
                // Stores the k minimum distances of points that can reach (a, b).
                Queue<Integer> maxHeap = new PriorityQueue<>(Collections.reverseOrder());
                for (int[] point : points) {
                    final int x = point[0];
                    final int y = point[1];
                    maxHeap.offer(Math.abs(x - a) + Math.abs(y - b));
                    if (maxHeap.size() > k)
                        maxHeap.poll();
                }
                ans = Math.min(ans, maxHeap.peek());
            }
        return ans;
    }
}
```

968. 1957.java

Path: LeetCode-main/solutions/1957. Delete Characters to Make Fancy String/1957.java

```
class Solution {
    public String makeFancyString(String s) {
        StringBuilder sb = new StringBuilder();
        for (char c : s.toCharArray())
            if (sb.length() < 2 || //
                sb.charAt(sb.length() - 1) != c || sb.charAt(sb.length() - 2) != c)
                sb.append(c);
        return sb.toString();
    }
}
```

969. 1958.java

Path: LeetCode-main/solutions/1958. Check if Move is Legal/1958.java

```
class Solution {
    public boolean checkMove(char[][] board, int rMove, int cMove, char color) {
        final int[][] DIRS = {{-1, -1}, {-1, 0}, {-1, 1}, {0, -1}, {0, 1}, {1, -1}, {1, 0}, {1, 1}};

        for (int k = 0; k < 8; ++k) {
            final int dx = dir[0][0];
            final int dy = dir[0][1];
            int cellsCount = 2;
            int i = rMove + dx;
            int j = cMove + dy;
            while (0 <= i && i < 8 && 0 <= j && j < 8) {
                // There are no free cells in between.
                if (board[i][j] == '.')
                    break;
                // Need >= 3 cells.
                if (cellsCount == 2 && board[i][j] == color)
                    break;
                // >= 3 cells.
                if (board[i][j] == color)
                    return true;
                i += dx;
                j += dy;
                ++cellsCount;
            }
        }
        return false;
    }
}
```


970. 1959.java

Path: LeetCode-main/solutions/1959. Minimum Total Space Wasted With K Resizing Operations/1959.java

```
class Solution {
    public int minSpaceWastedKResizing(int[] nums, int k) {
        Integer[][] mem = new Integer[nums.length][k + 1];
        return minSpaceWasted(nums, 0, k, mem);
    }

    private static final int MAX = 200_000_000;

    // Returns the minimum space wasted for nums[i..n) if you can resize k times.
    private int minSpaceWasted(int[] nums, int i, int k, Integer[][] mem) {
        if (i == nums.length)
            return 0;
        if (k == -1)
            return MAX;
        if (mem[i][k] != null)
            return mem[i][k];

        int res = MAX;
        int sum = 0;
        int maxNum = nums[i];

        for (int j = i; j < nums.length; ++j) {
            sum += nums[j];
            maxNum = Math.max(maxNum, nums[j]);
            final int wasted = maxNum * (j - i + 1) - sum;
            res = Math.min(res, minSpaceWasted(nums, j + 1, k - 1, mem) + wasted);
        }

        return mem[i][k] = res;
    }
}
```

971. 1960-2.java

Path: LeetCode-main/solutions/1960. Maximum Product of the Length of Two Palindromic Substrings/1960-2.java

```
class Solution {
    public long maxProduct(String s) {
        final int n = s.length();
        long ans = 1;
        long[] pows = new long[n + 1]; // pows[i] := BASE^i % HASH
        // hashL[i] = the hash of the first i letters of s, where hashL[i] =
        // (26^(i - 1) * s[0] + 26^(i - 2) * s[1] + ... + s[i - 1]) % HASH
        long[] hashL = new long[n + 1];
        // hashR[i] = the hash of the last i letters of s, where hashR[i] =
        // (26^(i - 1) * s[-1] + 26^(i - 2) * s[-2] + ... + s[-i]) % HASH
        long[] hashR = new long[n + 1];
        // maxLeft[i] := the maximum odd length of palindromes in s[0..i]
        int[] maxLeft = new int[n];
        // maxRight[i] := the maximum odd length of palindromes in s[i..n - 1]
        int[] maxRight = new int[n];

        pows[0] = 1;
        for (int i = 1; i <= n; ++i)
            pows[i] = pows[i - 1] * BASE % HASH;

        for (int i = 1; i <= n; ++i)
            hashL[i] = (hashL[i - 1] * BASE + val(s.charAt(i - 1))) % HASH;

        for (int i = n - 1; i >= 0; --i)
            hashR[i] = (hashR[i + 1] * BASE + val(s.charAt(i))) % HASH;

        int maxLength = 1;
        for (int r = 0; r < n; ++r) {
            final int l = (r - maxLength - 2) + 1;
            if (l >= 0 && isPalindrome(l, r + 1, hashL, hashR, pows))
                maxLength += 2;
            maxLeft[r] = maxLength;
        }

        maxLength = 1;
        for (int l = n - 1; l >= 0; --l) {
            final int r = (l + maxLength + 2) - 1;
            if (r < n && isPalindrome(l, r + 1, hashL, hashR, pows))
                maxLength += 2;
            maxRight[l] = maxLength;
        }

        for (int i = 1; i < n; ++i)
            ans = Math.max(ans, (long) maxLeft[i - 1] * maxRight[i]);

        return ans;
    }

    private static final int BASE = 26;
    private static final int HASH = 1_000_000_007;

    private static int val(char c) {
        return c - 'a';
    }

    // Returns true if s[l..r) is a palindrome.
    private boolean isPalindrome(int l, int r, long[] hashL, long[] hashR, long[] pows) {
        return getLeftRollingHash(l, r, hashL, pows) == getRightRollingHash(l, r, hashR, pows);
    }

    // Returns the left rolling hash of s[l..r).
    private long getLeftRollingHash(int l, int r, long[] hashL, long[] pows) {
        final long h = (hashL[r] - hashL[l] * pows[r - l]) % HASH;
        return h < 0 ? h + HASH : h;
    }

    // Returns the right rolling hash of s[l..r).
    private long getRightRollingHash(int l, int r, long[] hashR, long[] pows) {
        final long h = (hashR[l] - hashR[r] * pows[r - l]) % HASH;
        return h < 0 ? h + HASH : h;
    }
}
```

972. 1960.java

Path: LeetCode-main/solutions/1960. Maximum Product of the Length of Two Palindromic Substrings/1960.java

```
class Solution {
    public long maxProduct(String s) {
        final int n = s.length();
        long ans = 1;
        // maxLeft[i] := the maximum odd length of palindromes in s[0..i]
        int[] maxLeft = manacher(s, n);
        // maxRight[i] := the maximum odd length of palindromes in s[i..n - 1]
        int[] maxRight = manacher(new StringBuilder(s).reverse().toString(), n);

        reverse(maxRight, 0, n - 1);

        for (int i = 1; i < n; ++i)
            ans = Math.max(ans, (long) maxLeft[i - 1] * maxRight[i]);

        return ans;
    }

    private int[] manacher(final String s, int n) {
        int[] maxExtends = new int[n];
        int[] leftToRight = new int[n];
        Arrays.fill(leftToRight, 1);
        int center = 0;

        for (int i = 0; i < n; ++i) {
            final int r = center + maxExtends[center] - 1;
            final int mirrorIndex = center - (i - center);
            int extend = i > r ? 1 : Math.min(maxExtends[mirrorIndex], r - i + 1);
            while (i - extend >= 0 && i + extend < n && s.charAt(i - extend) == s.charAt(i + extend)) {
                leftToRight[i + extend] = 2 * extend + 1;
                ++extend;
            }
            maxExtends[i] = extend;
            if (i + maxExtends[i] >= r)
                center = i;
        }

        for (int i = 1; i < n; ++i)
            leftToRight[i] = Math.max(leftToRight[i], leftToRight[i - 1]);

        return leftToRight;
    }

    private void reverse(int[] arr, int l, int r) {
        while (l < r)
            swap(arr, l++, r--);
    }

    private void swap(int[] arr, int i, int j) {
        final int temp = arr[i];
        arr[i] = arr[j];
        arr[j] = temp;
    }
}
```

973. 1961.java

Path: LeetCode-main/solutions/1961. Check If String Is a Prefix of Array/1961.java

```
class Solution {
    public boolean isPrefixString(String s, String[] words) {
        StringBuilder sb = new StringBuilder();
        for (final String word : words) {
            sb.append(word);
            // Do not call `toString()` unless the length matches.
            if (sb.length() == s.length() && sb.toString().equals(s))
                return true;
        }
        return false;
    }
}
```

974. 1962.java

Path: LeetCode-main/solutions/1962. Remove Stones to Minimize the Total/1962.java

```
class Solution {
    public int minStoneSum(int[] piles, int k) {
        int ans = Arrays.stream(piles).sum();
        Queue<Integer> maxHeap = new PriorityQueue<>(Collections.reverseOrder());

        for (final int pile : piles)
            maxHeap.offer(pile);

        for (int i = 0; i < k; ++i) {
            final int maxPile = maxHeap.poll();
            maxHeap.offer(maxPile - maxPile / 2);
            ans -= maxPile / 2;
        }

        return ans;
    }
}
```

975. 1963.java

Path: LeetCode-main/solutions/1963. Minimum Number of Swaps to Make the String Balanced/1963.java

```
class Solution {
    public int minSwaps(String s) {
        // Cancel out all the matched pairs, then we'll be left with "]]]..[[[".
        // The answer is ceil(the number of unmatched pairs / 2).
        int unmatched = 0;

        for (final char c : s.toCharArray())
            if (c == '[')
                ++unmatched;
            else if (unmatched > 0) // c == ']' and there's a match.
                --unmatched;

        return (unmatched + 1) / 2;
    }
}
```

976. 1964.java

Path: LeetCode-main/solutions/1964. Find the Longest Valid Obstacle Course at Each Position/1964.java

```
class Solution {
    // Similar to 300. Longest Increasing Subsequence
    public int[] longestObstacleCourseAtEachPosition(int[] obstacles) {
        List<Integer> ans = new ArrayList<>();
        // tails[i] := the minimum tail of all the increasing subsequences with
        // length i + 1
        List<Integer> tails = new ArrayList<>();

        for (final int obstacle : obstacles)
            if (tails.isEmpty() || obstacle >= tails.get(tails.size() - 1)) {
                tails.add(obstacle);
                ans.add(tails.size());
            } else {
                final int index = firstGreater(tails, obstacle);
                tails.set(index, obstacle);
                ans.add(index + 1);
            }

        return ans.stream().mapToInt(Integer::intValue).toArray();
    }

    private int firstGreater(List<Integer> arr, int target) {
        int l = 0;
        int r = arr.size();
        while (l < r) {
            final int m = (l + r) / 2;
            if (arr.get(m) > target)
                r = m;
            else
                l = m + 1;
        }
        return l;
    }
}
```

977. 1966.java

Path: LeetCode-main/solutions/1966. Binary Searchable Numbers in an Unsorted Array/1966.java

```
class Solution {
    public int binarySearchableNumbers(int[] nums) {
        final int n = nums.length;
        int ans = 0;
        // prefixMaxs[i] := Math.max(nums[0..i])
        int[] prefixMaxs = new int[n];
        // suffixMins[i] := Math.min(nums[i + 1..n])
        int[] suffixMins = new int[n];

        // Fill in `prefixMaxs`.
        prefixMaxs[0] = Integer.MIN_VALUE;
        for (int i = 1; i < n; ++i)
            prefixMaxs[i] = Math.max(prefixMaxs[i - 1], nums[i - 1]);

        // Fill in `suffixMins`.
        suffixMins[n - 1] = Integer.MAX_VALUE;
        for (int i = n - 2; i >= 0; --i)
            suffixMins[i] = Math.min(suffixMins[i + 1], nums[i + 1]);

        for (int i = 0; i < n; ++i)
            if (prefixMaxs[i] < nums[i] && nums[i] < suffixMins[i])
                ++ans;

        return ans;
    }
}
```


978. 1967.java

Path: LeetCode-main/solutions/1967. Number of Strings That Appear as Substrings in Word/1967.java

```
class Solution {  
    public int numOfStrings(String[] patterns, String word) {  
        return (int) Arrays.stream(patterns).filter(pattern -> word.contains(pattern)).count();  
    }  
}
```

979. 1968.java

Path: LeetCode-main/solutions/1968. Array With Elements Not Equal to Average of Neighbors/1968.java

```
class Solution {
    public int[] rearrangeArray(int[] nums) {
        Arrays.sort(nums);
        for (int i = 1; i < nums.length; i += 2) {
            final int temp = nums[i];
            nums[i] = nums[i - 1];
            nums[i - 1] = temp;
        }
        return nums;
    }
}
```

980. 1969.java

Path: LeetCode-main/solutions/1969. Minimum Non-Zero Product of the Array Elements/1969.java

```
class Solution {
    public int minNonZeroProduct(int p) {
        // Can always turn  $[1..2^p - 1]$  to  $[1, 1, \dots, 2^p - 2, 2^p - 2, 2^p - 1]$ .
        final long n = 1L << p;
        final long halfCount = n / 2 - 1;
        return (int) (modPow(n - 2, halfCount) * ((n - 1) % MOD) % MOD);
    }

    private static final int MOD = 1_000_000_007;

    private long modPow(long x, long n) {
        if (n == 0)
            return 1L;
        x %= MOD;
        if (n % 2 == 1)
            return x * modPow(x, n - 1) % MOD;
        return modPow(x * x, n / 2) % MOD;
    }
}
```

981. 1970.java

Path: LeetCode-main/solutions/1970. Last Day Where You Can Still Cross/1970.java

```
class Solution {
    public int latestDayToCross(int row, int col, int[][] cells) {
        int ans = 0;
        int l = 1;
        int r = cells.length - 1;

        while (l <= r) {
            final int m = (l + r) / 2;
            if (canWalk(m, row, col, cells)) {
                ans = m;
                l = m + 1;
            } else {
                r = m - 1;
            }
        }

        return ans;
    }

    private static final int[][] DIRS = {{0, 1}, {1, 0}, {0, -1}, {-1, 0}};

    private boolean canWalk(int day, int row, int col, int[][] cells) {
        int[][] matrix = new int[row][col];
        for (int i = 0; i < day; ++i) {
            final int x = cells[i][0] - 1;
            final int y = cells[i][1] - 1;
            matrix[x][y] = 1;
        }

        Queue<Pair<Integer, Integer>> q = new ArrayDeque<>();

        for (int j = 0; j < col; ++j)
            if (matrix[0][j] == 0) {
                q.offer(new Pair<>(0, j));
                matrix[0][j] = 1;
            }

        while (!q.isEmpty()) {
            final int i = q.peek().getKey();
            final int j = q.poll().getValue();
            for (int[] dir : DIRS) {
                final int x = i + dir[0];
                final int y = j + dir[1];
                if (x < 0 || x == row || y < 0 || y == col)
                    continue;
                if (matrix[x][y] == 1)
                    continue;
                if (x == row - 1)
                    return true;
                q.offer(new Pair<>(x, y));
                matrix[x][y] = 1;
            }
        }

        return false;
    }
}
```

982. 1971.java

Path: LeetCode-main/solutions/1971. Find if Path Exists in Graph/1971.java

```
class UnionFind {
    public UnionFind(int n) {
        id = new int[n];
        rank = new int[n];
        for (int i = 0; i < n; ++i)
            id[i] = i;
    }

    public void unionByRank(int u, int v) {
        final int i = find(u);
        final int j = find(v);
        if (i == j)
            return;
        if (rank[i] < rank[j]) {
            id[i] = j;
        } else if (rank[i] > rank[j]) {
            id[j] = i;
        } else {
            id[i] = j;
            ++rank[j];
        }
    }

    public int find(int u) {
        return id[u] == u ? u : (id[u] = find(id[u]));
    }

    private int[] id;
    private int[] rank;
}

class Solution {
    public boolean validPath(int n, int[][] edges, int source, int destination) {
        UnionFind uf = new UnionFind(n);

        for (int[] edge : edges) {
            final int u = edge[0];
            final int v = edge[1];
            uf.unionByRank(u, v);
        }

        return uf.find(source) == uf.find(destination);
    }
}
```

983. 1973.java

Path: LeetCode-main/solutions/1973. Count Nodes Equal to Sum of Descendants/1973.java

```
class Solution {
    public int equalToDescendants(TreeNode root) {
        return dfs(root).count;
    }

    private T dfs(TreeNode root) {
        if (root == null)
            return new T(0, 0);
        T left = dfs(root.left);
        T right = dfs(root.right);
        return new T(root.val + left.sum + right.sum,
                     left.count + right.count + (root.val == left.sum + right.sum ? 1 : 0));
    }

    private record T(long sum, int count){};
}
```

984. 1974.java

Path: LeetCode-main/solutions/1974. Minimum Time to Type Word Using Special Typewriter/1974.java

```
class Solution {
    public int minTimeToType(String word) {
        int moves = 0;
        char letter = 'a';

        for (final char c : word.toCharArray()) {
            final int diff = Math.abs(c - letter);
            moves += Math.min(diff, 26 - diff);
            letter = c;
        }

        return moves + word.length();
    }
}
```

985. 1975.java

Path: LeetCode-main/solutions/1975. Maximum Matrix Sum/1975.java

```
class Solution {
    public long maxMatrixSum(int[][] matrix) {
        long absSum = 0;
        int minAbs = Integer.MAX_VALUE;
        // 0 := even number of negatives
        // 1 := odd number of negatives
        int oddNeg = 0;

        for (int[] row : matrix)
            for (final int num : row) {
                absSum += Math.abs(num);
                minAbs = Math.min(minAbs, Math.abs(num));
                if (num < 0)
                    oddNeg ^= 1;
            }

        return absSum - oddNeg * minAbs * 2;
    }
}
```


986. 1976.java

Path: LeetCode-main/solutions/1976. Number of Ways to Arrive at Destination/1976.java

```
class Solution {
    public int countPaths(int n, int[][] roads) {
        List<Pair<Integer, Integer>>[] graph = new List[n];

        for (int i = 0; i < n; i++)
            graph[i] = new ArrayList<>();

        for (int[] road : roads) {
            final int u = road[0];
            final int v = road[1];
            final int w = road[2];
            graph[u].add(new Pair<>(v, w));
            graph[v].add(new Pair<>(u, w));
        }

        return dijkstra(graph, 0, n - 1);
    }

    private int dijkstra(List<Pair<Integer, Integer>>[] graph, int src, int dst) {
        final int MOD = 1_000_000_007;
        long[] ways = new long[graph.length];
        Arrays.fill(ways, 0);
        long[] dist = new long[graph.length];
        Arrays.fill(dist, Long.MAX_VALUE);

        ways[src] = 1;
        dist[src] = 0;
        Queue<Pair<Long, Integer>> minHeap =
            new PriorityQueue<>(Comparator.comparingLong(Pair::getKey)) {
                { offer(new Pair<>(dist[src], src)); }
            };

        while (!minHeap.isEmpty()) {
            final long d = minHeap.peek().getKey();
            final int u = minHeap.poll().getValue();
            if (d > dist[u])
                continue;
            for (Pair<Integer, Integer> pair : graph[u]) {
                final int v = pair.getKey();
                final int w = pair.getValue();
                if (d + w < dist[v]) {
                    dist[v] = d + w;
                    ways[v] = ways[u];
                    minHeap.offer(new Pair<>(dist[v], v));
                } else if (d + w == dist[v]) {
                    ways[v] += ways[u];
                    ways[v] %= MOD;
                }
            }
        }

        return (int) ways[dst];
    }
}
```

987. 1977.java

Path: LeetCode-main/solutions/1977. Number of Ways to Separate Numbers/1977.java

```
class Solution {
    public int numberOfCombinations(String num) {
        if (num.charAt(0) == '0')
            return 0;

        final int MOD = 1_000_000_007;
        final int n = num.length();
        // dp[i][k] := the number of possible lists of integers ending in num[i] with
        // the length of the last number being 1..k
        long[][] dp = new long[n][n + 1];
        // lcs[i][j] := the number of the same digits in num[i..n) and num[j..n)
        int[][] lcs = new int[n + 1][n + 1];

        for (int i = n - 1; i >= 0; --i)
            for (int j = i + 1; j < n; ++j)
                if (num.charAt(i) == num.charAt(j))
                    lcs[i][j] = lcs[i + 1][j + 1] + 1;

        for (int i = 0; i < n; ++i)
            for (int k = 1; k <= i + 1; ++k) {
                dp[i][k] += dp[i][k - 1];
                dp[i][k] %= MOD;
                // The last number is num[s..i].
                final int s = i - k + 1;
                if (num.charAt(s) == '0')
                    // the number of possible lists of integers ending in num[i] with the
                    // length of the last number being k
                    continue;
                if (s == 0) {
                    // the whole string
                    dp[i][k] += 1;
                    continue;
                }
                if (s < k) {
                    // The length k is not enough, so add the number of possible lists of
                    // integers in num[0..s - 1].
                    dp[i][k] += dp[s - 1][s];
                    continue;
                }
                final int l = lcs[s - k][s];
                if (l >= k || num.charAt(s - k + l) <= num.charAt(s + l))
                    // Have enough length k and num[s - k..s - 1] <= num[j..i].
                    dp[i][k] += dp[s - 1][k];
                else
                    // Have enough length k but num[s - k..s - 1] > num[j..i].
                    dp[i][k] += dp[s - 1][k - 1];
            }

        return (int) dp[n - 1][n] % MOD;
    }
}
```

988. 1979.java

Path: LeetCode-main/solutions/1979. Find Greatest Common Divisor of Array/1979.java

```
class Solution {
    public int findGCD(int[] nums) {
        final int mn = Arrays.stream(nums).min().getAsInt();
        final int mx = Arrays.stream(nums).max().getAsInt();
        return gcd(mn, mx);
    }

    private int gcd(int a, int b) {
        return b == 0 ? a : gcd(b, a % b);
    }
}
```

989. 198-2.java

Path: LeetCode-main/solutions/198. House Robber/198-2.java

```
class Solution {
    public int rob(int[] nums) {
        int prev1 = 0; // dp[i - 1]
        int prev2 = 0; // dp[i - 2]

        for (final int num : nums) {
            final int dp = Math.max(prev1, prev2 + num);
            prev2 = prev1;
            prev1 = dp;
        }

        return prev1;
    }
}
```

990. 198.java

Path: LeetCode-main/solutions/198. House Robber/198.java

```
class Solution {
    public int rob(int[] nums) {
        final int n = nums.length;
        if (n == 0)
            return 0;
        if (n == 1)
            return nums[0];

        // dp[i] := the maximum money of robbing nums[0..i]
        int[] dp = new int[n];
        dp[0] = nums[0];
        dp[1] = Math.max(nums[0], nums[1]);

        for (int i = 2; i < n; ++i)
            dp[i] = Math.max(dp[i - 1], dp[i - 2] + nums[i]);

        return dp[n - 1];
    }
}
```

991. 1980-2.java

Path: LeetCode-main/solutions/1980. Find Unique Binary String/1980-2.java

```
class Solution {
    public String findDifferentBinaryString(String[] nums) {
        StringBuilder sb = new StringBuilder();

        // Flip the i-th bit for each nums[i] so that `ans` is unique.
        for (int i = 0; i < nums.length; ++i)
            sb.append(nums[i].charAt(i) == '0' ? '1' : '0');

        return sb.toString();
    }
}
```

992. 1980.java

Path: LeetCode-main/solutions/1980. Find Unique Binary String/1980.java

```
class Solution {
    public String findDifferentBinaryString(String[] nums) {
        final int bitSize = nums[0].length();
        final int maxNum = 1 << bitSize;
        Set<Integer> numsSet = Arrays.stream(nums)
            .mapToInt(num -> Integer.parseInt(num, 2))
            .boxed()
            .collect(Collectors.toSet());

        for (int num = 0; num < maxNum; ++num)
            if (!numsSet.contains(num))
                return String.format("%" + bitSize + "s", Integer.toBinaryString(num)).replace(' ', '0');

        throw new IllegalArgumentException();
    }
}
```

993. 1981.java

Path: LeetCode-main/solutions/1981. Minimize the Difference Between Target and Chosen Elements/1981.java

```
class Solution {
    public int minimizeTheDifference(int[][] mat, int target) {
        final int minSum = Arrays.stream(mat).mapToInt(A -> Arrays.stream(A).min().getAsInt()).sum();
        if (minSum >= target) // No need to consider any larger combination.
            return minSum - target;

        final int maxSum = Arrays.stream(mat).mapToInt(A -> Arrays.stream(A).max().getAsInt()).sum();
        Integer[][] mem = new Integer[mat.length][maxSum + 1];
        return minimizeTheDifference(mat, 0, 0, target, mem);
    }

    private int minimizeTheDifference(int[][] mat, int i, int sum, int target, Integer[][] mem) {
        if (i == mat.length)
            return Math.abs(sum - target);
        if (mem[i][sum] != null)
            return mem[i][sum];
        int res = Integer.MAX_VALUE;
        for (final int num : mat[i])
            res = Math.min(res, minimizeTheDifference(mat, i + 1, sum + num, target, mem));
        return mem[i][sum] = res;
    }
}
```


994. 1982.java

Path: LeetCode-main/solutions/1982. Find Array Given Subset Sums/1982.java

```
class Solution {
    public int[] recoverArray(int n, int[] sums) {
        Arrays.sort(sums);
        return recover(sums).stream().mapToInt(Integer::intValue).toArray();
    }

    private List<Integer> recover(int[] sums) {
        if (sums.length == 1) // sums[0] must be 0.
            return new ArrayList<>();

        Map<Integer, Long> count = Arrays.stream(sums).boxed().collect(
            Collectors.groupingBy(Function.identity(), Collectors.counting()));

        // Either num or -num must be in the final array.
        // num + sumsExcludingNum = sumsIncludingNum
        // -num + sumsIncludingNum = sumsExcludingNum
        final int num = sums[1] - sums[0];
        int i = 0; // sumsExcludingNum/sumsIncludingNum's index
        int[] sumsExcludingNum = new int[sums.length / 2];
        int[] sumsIncludingNum = new int[sums.length / 2];
        boolean chooseSumsIncludingNum = false;

        for (final int sum : sums) {
            if (count.get(sum) == 0)
                continue;
            count.merge(sum, -1L, Long::sum);
            count.merge(sum + num, -1L, Long::sum);
            sumsExcludingNum[i] = sum;
            sumsIncludingNum[i] = sum + num;
            ++i;
            if (sum + num == 0)
                chooseSumsIncludingNum = true;
        }

        // Choose `sumsExcludingNum` by default since we want to gradually strip
        // `num` from each sum in `sums` to have the final array. However, we should
        // always choose the group of sums with 0 since it's a must-have.
        List<Integer> recovered = recover(chooseSumsIncludingNum ? sumsIncludingNum : sumsExcludingNum);
        recovered.add(chooseSumsIncludingNum ? -num : num);
        return recovered;
    }
}
```

995. 1983.java

Path: LeetCode-main/solutions/1983. Widest Pair of Indices With Equal Range Sum/1983.java

```
class Solution {
    public int widestPairOfIndices(int[] nums1, int[] nums2) {
        int ans = 0;
        int prefix = 0;
        Map<Integer, Integer> prefixToIndex = new HashMap<>();
        prefixToIndex.put(0, -1);

        for (int i = 0; i < nums1.length; ++i) {
            prefix += nums1[i] - nums2[i];
            if (prefixToIndex.containsKey(prefix))
                ans = Math.max(ans, i - prefixToIndex.get(prefix));
            else
                prefixToIndex.put(prefix, i);
        }

        return ans;
    }
}
```

996. 1984.java

Path: LeetCode-main/solutions/1984. Minimum Difference Between Highest and Lowest of K Scores/1984.java

```
class Solution {
    public int minimumDifference(int[] nums, int k) {
        Arrays.sort(nums);

        int ans = nums[k - 1] - nums[0];

        for (int i = k; i < nums.length; ++i)
            ans = Math.min(ans, nums[i] - nums[i - k + 1]);

        return ans;
    }
}
```

997. 1985.java

Path: LeetCode-main/solutions/1985. Find the Kth Largest Integer in the Array/1985.java

```
class Solution {
    // Similar to 215. Kth Largest Element in an Array
    public String kthLargestNumber(String[] nums, int k) {
        Queue<String> minHeap = new PriorityQueue<>(
            Comparator.comparingInt(String::length).thenComparing(String::compareTo));

        for (final String num : nums) {
            minHeap.offer(num);
            if (minHeap.size() > k)
                minHeap.poll();
        }

        return minHeap.poll();
    }
}
```

998. 1986.java

Path: LeetCode-main/solutions/1986. Minimum Number of Work Sessions to Finish the Tasks/1986.java

```
class Solution {
    public int minSessions(int[] tasks, int sessionTime) {
        for (int numSessions = 1; numSessions <= tasks.length; ++numSessions)
            if (dfs(tasks, 0, new int[numSessions], sessionTime))
                return numSessions;
        throw new IllegalArgumentException();
    }

    // Returns true if we can assign tasks[s..n) to `sessions`. Note that `sessions`
    // may be occupied by some tasks.
    private boolean dfs(int[] tasks, int s, int[] sessions, int sessionTime) {
        if (s == tasks.length)
            return true;

        for (int i = 0; i < sessions.length; ++i) {
            // Can't assign the tasks[s] to this session.
            if (sessions[i] + tasks[s] > sessionTime)
                continue;
            // Assign the tasks[s] to this session.
            sessions[i] += tasks[s];
            if (dfs(tasks, s + 1, sessions, sessionTime))
                return true;
            // Backtracking.
            sessions[i] -= tasks[s];
            // If it's the first time we assign the tasks[s] to this session, then future
            // `session`s can't satisfy either.
            if (sessions[i] == 0)
                return false;
        }
        return false;
    }
}
```

999. 1987.java

Path: LeetCode-main/solutions/1987. Number of Unique Good Subsequences/1987.java

```
class Solution {
    // Similar to 940. Distinct Subsequences II
    public int numberOfUniqueGoodSubsequences(String binary) {
        final int MOD = 1_000_000_007;
        // endsIn[i] := the number of subsequence that end in ('0' + i)
        int[] endsIn = new int[2];

        for (final char c : binary.toCharArray()) {
            endsIn[c - '0'] = (endsIn[0] + endsIn[1]) % MOD;
            // Don't count '0' since we want to avoid the leading zeros case.
            // However, we can always count '1'.
            if (c == '1')
                ++endsIn[1];
        }

        // Count '0' in the end.
        return (endsIn[0] + endsIn[1] + (binary.indexOf('0') == -1 ? 0 : 1)) % MOD;
    }
}
```

1000. 1989.java

Path: LeetCode-main/solutions/1989. Maximum Number of People That Can Be Caught in Tag/1989.java

```
class Solution {
    public int catchMaximumAmountofPeople(int[] team, int dist) {
        int ans = 0;
        int i = 0; // 0s index
        int j = 0; // 1s index

        while (i < team.length && j < team.length)
            if (i + dist < j || team[i] != 0) {
                // Find the next 0 that can be caught by 1.
                ++i;
            } else if (j + dist < i || team[j] != 1) {
                // Find the next 1 that can catch 0.
                ++j;
            } else {
                // team[j] catches team[i], so move both.
                ++ans;
                ++i;
                ++j;
            }

        return ans;
    }
}
```

1001. 199-2.java

Path: LeetCode-main/solutions/199. Binary Tree Right Side View/199-2.java

```
class Solution {
    public List<Integer> rightSideView(TreeNode root) {
        List<Integer> ans = new ArrayList<>();
        dfs(root, 0, ans);
        return ans;
    }

    private void dfs(TreeNode root, int depth, List<Integer> ans) {
        if (root == null)
            return;

        if (depth == ans.size())
            ans.add(root.val);
        dfs(root.right, depth + 1, ans);
        dfs(root.left, depth + 1, ans);
    }
}
```


1002. 199.java

Path: LeetCode-main/solutions/199. Binary Tree Right Side View/199.java

```
class Solution {
    public List<Integer> rightSideView(TreeNode root) {
        if (root == null)
            return new ArrayList<>();

        List<Integer> ans = new ArrayList<>();
        Queue<TreeNode> q = new ArrayDeque<>(List.of(root));

        while (!q.isEmpty()) {
            final int size = q.size();
            for (int i = 0; i < size; ++i) {
                TreeNode node = q.poll();
                if (i == size - 1)
                    ans.add(node.val);
                if (node.left != null)
                    q.offer(node.left);
                if (node.right != null)
                    q.offer(node.right);
            }
        }
        return ans;
    }
}
```

1003. 1991.java

Path: LeetCode-main/solutions/1991. Find the Middle Index in Array/1991.java

```
class Solution {
    public int findMiddleIndex(int[] nums) {
        int prefix = 0;
        int suffix = Arrays.stream(nums).sum();

        for (int i = 0; i < nums.length; ++i) {
            suffix -= nums[i];
            if (prefix == suffix)
                return i;
            prefix += nums[i];
        }

        return -1;
    }
}
```

1004. 1992.java

Path: LeetCode-main/solutions/1992. Find All Groups of Farmland/1992.java

```
class Solution {
    public int[][] findFarmland(int[][] land) {
        List<int[]> ans = new ArrayList<>();

        for (int i = 0; i < land.length; ++i)
            for (int j = 0; j < land[0].length; ++j)
                if (land[i][j] == 1) {
                    int[] cell = new int[] {i, j};
                    dfs(land, i, j, cell);
                    ans.add(new int[] {i, j, cell[0], cell[1]});
                }

        return ans.stream().toArray(int[][] ::new);
    }

    private void dfs(int[][] land, int i, int j, int[] cell) {
        if (i < 0 || i == land.length || j < 0 || j == land[0].length)
            return;
        if (land[i][j] != 1)
            return;
        land[i][j] = 2; // Mark as visited.
        cell[0] = Math.max(cell[0], i);
        cell[1] = Math.max(cell[1], j);
        dfs(land, i + 1, j, cell);
        dfs(land, i, j + 1, cell);
    }
}
```

1005. 1993.java

Path: LeetCode-main/solutions/1993. Operations on Tree/1993.java

```
class Node {
    public List<Integer> children = new ArrayList<>();
    public int lockedBy = -1;
}

class LockingTree {
    public LockingTree(int[] parent) {
        this.parent = parent;
        nodes = new Node[parent.length];
        for (int i = 0; i < parent.length; ++i)
            nodes[i] = new Node();
        for (int i = 1; i < parent.length; ++i)
            nodes[parent[i]].children.add(i);
    }

    public boolean lock(int num, int user) {
        if (nodes[num].lockedBy != -1)
            return false;
        nodes[num].lockedBy = user;
        return true;
    }

    public boolean unlock(int num, int user) {
        if (nodes[num].lockedBy != user)
            return false;
        nodes[num].lockedBy = -1;
        return true;
    }

    public boolean upgrade(int num, int user) {
        if (nodes[num].lockedBy != -1)
            return false;
        if (!anyLockedDescendant(num))
            return false;

        // Walk up the hierarchy to ensure that there are no locked ancestors.
        for (int i = num; i != -1; i = parent[i])
            if (nodes[i].lockedBy != -1)
                return false;

        unlockDescendants(num);
        nodes[num].lockedBy = user;
        return true;
    }

    private int[] parent;
    private Node[] nodes;

    private boolean anyLockedDescendant(int i) {
        return nodes[i].lockedBy != -1 ||
            nodes[i].children.stream().anyMatch(child -> anyLockedDescendant(child));
    }

    private void unlockDescendants(int i) {
        nodes[i].lockedBy = -1;
        for (final int child : nodes[i].children)
            unlockDescendants(child);
    }
}
```

1006. 1994.java

Path: LeetCode-main/solutions/1994. The Number of Good Subsets/1994.java

```
class Solution {
    public int numberOfGoodSubsets(int[] nums) {
        final int[] primes = {2, 3, 5, 7, 11, 13, 17, 19, 23, 29};
        final int n = 1 << primes.length;
        final int maxNum = Arrays.stream(nums).max().getAsInt();
        long[] dp = new long[n];
        int[] count = new int[maxNum + 1];

        dp[0] = 1;

        for (final int num : nums)
            ++count[num];

        for (int num = 2; num <= maxNum; ++num) {
            if (count[num] == 0)
                continue;
            if (num % 4 == 0 || num % 9 == 0 || num % 25 == 0)
                continue;
            final int numPrimesMask = getPrimesMask(num, primes);
            for (int primesMask = 0; primesMask < n; ++primesMask) {
                if ((primesMask & numPrimesMask) > 0)
                    continue;
                final int nextPrimesMask = primesMask | numPrimesMask;
                dp[nextPrimesMask] += dp[primesMask] * count[num];
                dp[nextPrimesMask] %= MOD;
            }
        }

        return (int) (modPow(2, count[1]) * ((Arrays.stream(dp).sum() - 1) % MOD) % MOD);
    }

    private static final int MOD = 1_000_000_007;

    private int getPrimesMask(int num, int[] primes) {
        int primesMask = 0;
        for (int i = 0; i < primes.length; ++i)
            if (num % primes[i] == 0)
                primesMask |= 1 << i;
        return primesMask;
    }

    private long modPow(long x, long n) {
        if (n == 0)
            return 1;
        if (n % 2 == 1)
            return x * modPow(x, n - 1) % MOD;
        return modPow(x * x % MOD, n / 2);
    }
}
```

1007. 1995-2.java

Path: LeetCode-main/solutions/1995. Count Special Quadruplets/1995-2.java

```
class Solution {
    public int countQuadruplets(int[] nums) {
        final int n = nums.length;
        int ans = 0;
        Map<Integer, Integer> count = new HashMap<>();

        for (int c = n - 1; c > 1; --c) { // `c` also represents `d`.
            for (int b = c - 1; b > 0; --b)
                for (int a = b - 1; a >= 0; --a)
                    ans += count.getDefault(nums[a] + nums[b] + nums[c], 0);
            count.merge(nums[c], 1, Integer::sum); // c := d
        }

        return ans;
    }
}
```

1008. 1995-3.java

Path: LeetCode-main/solutions/1995. Count Special Quadruplets/1995-3.java

```
class Solution {
    public int countQuadruplets(int[] nums) {
        final int n = nums.length;
        int ans = 0;
        Map<Integer, Integer> count = new HashMap<>();

        //    nums[a] + nums[b] + nums[c] == nums[d]
        // => nums[a] + nums[b] == nums[d] - nums[c]
        for (int b = n - 1; b > 0; --b) { // `b` also represents `c`.
            for (int a = b - 1; a >= 0; --a)
                ans += count.getDefault(nums[a] + nums[b], 0);
            for (int d = n - 1; d > b; --d)
                count.merge(nums[d] - nums[b], 1, Integer::sum); // b := c
        }

        return ans;
    }
}
```

1009. 1995.java

Path: LeetCode-main/solutions/1995. Count Special Quadruplets/1995.java

```
class Solution {  
    public int countQuadruplets(int[] nums) {  
        final int n = nums.length;  
        int ans = 0;  
  
        for (int a = 0; a < n; ++a)  
            for (int b = a + 1; b < n; ++b)  
                for (int c = b + 1; c < n; ++c)  
                    for (int d = c + 1; d < n; ++d)  
                        if (nums[a] + nums[b] + nums[c] == nums[d])  
                            ++ans;  
  
        return ans;  
    }  
}
```


1010. 1996-2.java

Path: LeetCode-main/solutions/1996. The Number of Weak Characters in the Game/1996-2.java

```
class Solution {
    public int numberOfWeakCharacters(int[][] properties) {
        int ans = 0;
        final int maxAttack = Arrays.stream(properties).mapToInt(p -> p[0]).max().getAsInt();
        // maxDefenses[i] := the maximum defense for the i-th attack
        int[] maxDefenses = new int[maxAttack + 2];

        for (int[] property : properties) {
            final int attack = property[0];
            final int defense = property[1];
            maxDefenses[attack] = Math.max(maxDefenses[attack], defense);
        }

        for (int i = maxAttack; i >= 1; --i)
            maxDefenses[i] = Math.max(maxDefenses[i], maxDefenses[i + 1]);

        for (int[] property : properties) {
            final int attack = property[0];
            final int defense = property[1];
            if (maxDefenses[attack + 1] > defense)
                ++ans;
        }

        return ans;
    }
}
```

1011. 1996.java

Path: LeetCode-main/solutions/1996. The Number of Weak Characters in the Game/1996.java

```
class Solution {
    public int numberOfWeakCharacters(int[][] properties) {
        // Sort properties by `attack` in descending order, then by `defense` in
        // ascending order.
        Arrays.sort(properties, Comparator.comparingInt((int[] property) -> - property[0])
            .thenComparingInt((int[] property) -> property[1]));

        int ans = 0;
        int maxDefense = 0;

        for (int[] property : properties) {
            final int defense = property[1];
            if (defense < maxDefense)
                ++ans;
            maxDefense = Math.max(maxDefense, defense);
        }

        return ans;
    }
}
```

1012. 1997.java

Path: LeetCode-main/solutions/1997. First Day Where You Have Been in All the Rooms/1997.java

```
class Solution {
    public int firstDayBeenInAllRooms(int[] nextVisit) {
        final int MOD = 1_000_000_007;
        final int n = nextVisit.length;
        // dp[i] := the number of days to visit room i for the first time
        int[] dp = new int[n];

        // Whenever we visit i, visit times of room[0..i - 1] are all even.
        // Therefore, the rooms before i can be seen as reset and we can safely
        // reuse dp[0..i - 1] as first-time visit to get second-time visit.
        for (int i = 1; i < n; ++i)
            // The total days to visit room[i] is the sum of
            // * dp[i - 1]: 1st-time visit room[i - 1]
            // * 1: visit room[nextVisit[i - 1]]
            // * dp[i - 1] - dp[nextVisit[i - 1]]: 2-time visit room[i - 1]
            // * 1: visit room[i]
            dp[i] = (int) ((2L * dp[i - 1] - dp[nextVisit[i - 1]] + 2 + MOD) % MOD);

        return dp[n - 1];
    }
}
```

1013. 1998.java

Path: LeetCode-main/solutions/1998. GCD Sort of an Array/1998.java

```
class UnionFind {
    public UnionFind(int n) {
        id = new int[n];
        rank = new int[n];
        for (int i = 0; i < n; ++i)
            id[i] = i;
    }

    public void unionByRank(int u, int v) {
        final int i = find(u);
        final int j = find(v);
        if (i == j)
            return;
        if (rank[i] < rank[j]) {
            id[i] = j;
        } else if (rank[i] > rank[j]) {
            id[j] = i;
        } else {
            id[i] = j;
            ++rank[j];
        }
    }

    public int find(int u) {
        return id[u] == u ? u : (id[u] = find(id[u]));
    }

    private int[] id;
    private int[] rank;
}

class Solution {
    public boolean gcdSort(int[] nums) {
        final int mx = Arrays.stream(nums).max().getAsInt();
        final int[] minPrimeFactors = sieveEratosthenes(mx + 1);
        UnionFind uf = new UnionFind(mx + 1);

        for (final int num : nums)
            for (final int primeFactor : getPrimeFactors(num, minPrimeFactors))
                uf.unionByRank(num, primeFactor);

        int[] sortedNums = nums.clone();
        Arrays.sort(sortedNums);

        for (int i = 0; i < nums.length; ++i)
            // Can't swap nums[i] with sortedNums[i].
            if (uf.find(nums[i]) != uf.find(sortedNums[i]))
                return false;

        return true;
    }

    // Gets the minimum prime factor of i, where 1 < i <= n.
    private int[] sieveEratosthenes(int n) {
        int[] minPrimeFactors = new int[n + 1];
        for (int i = 2; i <= n; ++i)
            minPrimeFactors[i] = i;
        for (int i = 2; i * i < n; ++i)
            if (minPrimeFactors[i] == i) // `i` is prime.
                for (int j = i * i; j < n; j += i)
                    minPrimeFactors[j] = Math.min(minPrimeFactors[j], i);
        return minPrimeFactors;
    }

    private List<Integer> getPrimeFactors(int num, int[] minPrimeFactors) {
        List<Integer> primeFactors = new ArrayList<>();
        while (num > 1) {
            final int divisor = minPrimeFactors[num];
            primeFactors.add(divisor);
            while (num % divisor == 0)
                num /= divisor;
        }
        return primeFactors;
    }
}
```

1014. 1999-2.java

Path: LeetCode-main/solutions/1999. Smallest Greater Multiple Made of Two Digits/1999-2.java

```
class Solution {
    public int findInteger(int k, int digit1, int digit2) {
        return findInteger(k, digit1, digit2, 0);
    }

    private int findInteger(int k, int d1, int d2, long x) {
        if (x > Integer.MAX_VALUE)
            return -1;
        if (x > k && x % k == 0)
            return (int) x;
        // Skip if d1/d2 and x are zero.
        final int a = (x + d1 == 0) ? -1 : findInteger(k, d1, d2, x * 10 + d1);
        final int b = (x + d2 == 0) ? -1 : findInteger(k, d1, d2, x * 10 + d2);
        if (a == -1)
            return b;
        if (b == -1)
            return a;
        return Math.min(a, b);
    }
}
```

1015. 1999.java

Path: LeetCode-main/solutions/1999. Smallest Greater Multiple Made of Two Digits/1999.java

```
class Solution {
    public int findInteger(int k, int digit1, int digit2) {
        final int minDigit = Math.min(digit1, digit2);
        final int maxDigit = Math.max(digit1, digit2);
        final int[] digits =
            minDigit == maxDigit ? new int[] {minDigit} : new int[] {minDigit, maxDigit};
        Queue<Integer> q = new ArrayDeque<>();

        for (final int digit : digits)
            q.offer(digit);

        while (!q.isEmpty()) {
            final int u = q.poll();
            if (u > k && u % k == 0)
                return u;
            if (u == 0)
                continue;
            for (final int digit : digits) {
                final long nextNum = u * 10L + digit;
                if (nextNum > Integer.MAX_VALUE)
                    continue;
                q.offer((int) nextNum);
            }
        }

        return -1;
    }
}
```

1016. 2.java

Path: LeetCode-main/solutions/2. Add Two Numbers/2.java

```
class Solution {
    public ListNode addTwoNumbers(ListNode l1, ListNode l2) {
        ListNode dummy = new ListNode(0);
        ListNode curr = dummy;
        int carry = 0;

        while (l1 != null || l2 != null || carry > 0) {
            if (l1 != null) {
                carry += l1.val;
                l1 = l1.next;
            }
            if (l2 != null) {
                carry += l2.val;
                l2 = l2.next;
            }
            curr.next = new ListNode(carry % 10);
            carry /= 10;
            curr = curr.next;
        }

        return dummy.next;
    }
}
```

1017. 20.java

Path: LeetCode-main/solutions/20. Valid Parentheses/20.java

```
class Solution {
    public boolean isValid(String s) {
        Deque<Character> stack = new ArrayDeque<>();

        for (final char c : s.toCharArray())
            if (c == '(')
                stack.push(')');
            else if (c == '{')
                stack.push('}');
            else if (c == '[')
                stack.push(']');
            else if (stack.isEmpty() || stack.pop() != c)
                return false;

        return stack.isEmpty();
    }
}
```


1018. 200-2.java

Path: LeetCode-main/solutions/200. Number of Islands/200-2.java

```
class Solution {
    public int numIslands(char[][] grid) {
        int ans = 0;

        for (int i = 0; i < grid.length; ++i)
            for (int j = 0; j < grid[0].length; ++j)
                if (grid[i][j] == '1') {
                    dfs(grid, i, j);
                    ++ans;
                }

        return ans;
    }

    private void dfs(char[][] grid, int i, int j) {
        if (i < 0 || i == grid.length || j < 0 || j == grid[0].length)
            return;
        if (grid[i][j] != '1')
            return;

        grid[i][j] = '2'; // Mark '2' as visited.
        dfs(grid, i + 1, j);
        dfs(grid, i - 1, j);
        dfs(grid, i, j + 1);
        dfs(grid, i, j - 1);
    }
}
```

1019. 200.java

Path: LeetCode-main/solutions/200. Number of Islands/200.java

```
class Solution {
    public int numIslands(char[][] grid) {
        int ans = 0;

        for (int i = 0; i < grid.length; ++i)
            for (int j = 0; j < grid[0].length; ++j)
                if (grid[i][j] == '1') {
                    bfs(grid, i, j);
                    ++ans;
                }

        return ans;
    }

    private static final int[][] DIRS = {{0, 1}, {1, 0}, {0, -1}, {-1, 0}};

    private void bfs(char[][] grid, int r, int c) {
        Queue<Pair<Integer, Integer>> q = new ArrayDeque<>(List.of(new Pair<>(r, c)));
        grid[r][c] = '2'; // Mark '2' as visited.
        while (!q.isEmpty()) {
            final int i = q.peek().getKey();
            final int j = q.poll().getValue();
            for (int[] dir : DIRS) {
                final int x = i + dir[0];
                final int y = j + dir[1];
                if (x < 0 || x == grid.length || y < 0 || y == grid[0].length)
                    continue;
                if (grid[x][y] != '1')
                    continue;
                q.offer(new Pair<>(x, y));
                grid[x][y] = '2'; // Mark '2' as visited.
            }
        }
    }
}
```

1020. 2000.java

Path: LeetCode-main/solutions/2000. Reverse Prefix of Word/2000.java

```
class Solution {  
    public String reversePrefix(String word, char ch) {  
        final int i = word.indexOf(ch) + 1;  
        return new StringBuilder(word.substring(0, i)) //  
            .reverse() //  
            .append(word.substring(i)) //  
            .toString();  
    }  
}
```

1021. 2001-2.java

Path: LeetCode-main/solutions/2001. Number of Pairs of Interchangeable Rectangles/2001-2.java

```
class Solution {
    public long interchangeableRectangles(int[][] rectangles) {
        long ans = 0;
        Map<Pair<Integer, Integer>, Integer> ratioCount = new HashMap<>();

        for (int[] rectangle : rectangles) {
            final int width = rectangle[0];
            final int height = rectangle[1];
            final int d = gcd(width, height);
            ratioCount.merge(new Pair<>(width / d, height / d), 1, Integer::sum);
        }

        for (final int count : ratioCount.values())
            ans += (long) count * (count - 1) / 2;

        return ans;
    }

    private int gcd(int a, int b) {
        return b == 0 ? a : gcd(b, a % b);
    }
}
```

1022. 2001.java

Path: LeetCode-main/solutions/2001. Number of Pairs of Interchangeable Rectangles/2001.java

```
class Solution {
    public long interchangeableRectangles(int[][] rectangles) {
        long ans = 0;
        Map<Double, Integer> ratioCount = new HashMap<>();

        for (int[] rectangle : rectangles) {
            final int width = rectangle[0];
            final int height = rectangle[1];
            ratioCount.merge(1.0 * width / height, 1, Integer::sum);
        }

        for (final int count : ratioCount.values())
            ans += 1L * count * (count - 1) / 2;

        return ans;
    }
}
```

1023. 2002.java

Path: LeetCode-main/solutions/2002. Maximum Product of the Length of Two Palindromic Subsequences/2002.java

```
class Solution {
    public int maxProduct(String s) {
        dfs(s, 0, new StringBuilder(), new StringBuilder());
        return ans;
    }

    private int ans = 0;

    private void dfs(final String s, int i, StringBuilder sb1, StringBuilder sb2) {
        if (i == s.length()) {
            if (isPalindrome(sb1) && isPalindrome(sb2))
                ans = Math.max(ans, sb1.length() * sb2.length());
            return;
        }

        final int sb1Length = sb1.length();
        dfs(s, i + 1, sb1.append(s.charAt(i)), sb2);
        sb1.setLength(sb1Length);

        final int sb2Length = sb2.length();
        dfs(s, i + 1, sb1, sb2.append(s.charAt(i)));
        sb2.setLength(sb2Length);

        dfs(s, i + 1, sb1, sb2);
    }

    private boolean isPalindrome(StringBuilder sb) {
        int i = 0;
        int j = sb.length() - 1;
        while (i < j) {
            if (sb.charAt(i) != sb.charAt(j))
                return false;
            ++i;
            --j;
        }
        return true;
    }
}
```

1024. 2003.java

Path: LeetCode-main/solutions/2003. Smallest Missing Genetic Value in Each Subtree/2003.java

```
class Solution {
    public int[] smallestMissingValueSubtree(int[] parents, int[] nums) {
        final int n = parents.length;
        int[] ans = new int[n];
        Arrays.fill(ans, 1);
        List<Integer>[] tree = new List[n];
        Set<Integer> seen = new HashSet<>();
        int minMiss = 1;

        for (int i = 0; i < n; ++i)
            tree[i] = new ArrayList<>();

        for (int i = 1; i < n; ++i)
            tree[parents[i]].add(i);

        final int nodeThatsOne = getNode(nums);
        if (nodeThatsOne == -1)
            return ans;

        int u = nodeThatsOne;
        int prev = -1; // the u that just handled

        // Upward from `nodeThatsOne` to the root `u`.
        while (u != -1) {
            for (final int v : tree[u])
                if (v != prev)
                    dfs(v, tree, seen, nums);
            seen.add(nums[u]);
            while (seen.contains(minMiss))
                ++minMiss;
            ans[u] = minMiss;
            prev = u;
            u = parents[u];
        }

        return ans;
    }

    private void dfs(int u, List<Integer>[] tree, Set<Integer> seen, int[] nums) {
        seen.add(nums[u]);
        for (final int v : tree[u])
            dfs(v, tree, seen, nums);
    }

    private int getNode(int[] nums) {
        for (int i = 0; i < nums.length; ++i)
            if (nums[i] == 1)
                return i;
        return -1;
    }
}
```

1025. 2005.java

Path: LeetCode-main/solutions/2005. Subtree Removal Game with Fibonacci Tree/2005.java

```
class Solution {  
    public boolean findGameWinner(int n) {  
        return n % 6 != 1;  
    }  
}
```


1026. 2006.java

Path: LeetCode-main/solutions/2006. Count Number of Pairs With Absolute Difference K/2006.java

```
class Solution {
    public int countKDifference(int[] nums, int k) {
        final int MAX = 100;
        int ans = 0;
        int[] count = new int[MAX + 1];

        for (final int num : nums)
            ++count[num];

        for (int i = k + 1; i <= MAX; ++i)
            ans += count[i] * count[i - k];

        return ans;
    }
}
```

1027. 2007.java

Path: LeetCode-main/solutions/2007. Find Original Array From Doubled Array/2007.java

```
class Solution {
    public int[] findOriginalArray(int[] changed) {
        List<Integer> ans = new ArrayList<>();
        Queue<Integer> q = new ArrayDeque<>();

        Arrays.sort(changed);

        for (final int num : changed)
            if (!q.isEmpty() && num == q.peek()) {
                q.poll();
            } else {
                q.offer(num * 2);
                ans.add(num);
            }

        return q.isEmpty() ? ans.stream().mapToInt(Integer::intValue).toArray() : new int[] {};
    }
}
```

1028. 2008-2.java

Path: LeetCode-main/solutions/2008. Maximum Earnings From Taxi/2008-2.java

```
class Solution {
    public long maxTaxiEarnings(int n, int[][] rides) {
        List<Pair<Integer, Integer>>[] endToStartAndEarns = new List[n + 1];
        // dp[i] := the maximum dollars you can earn ending in i
        long[] dp = new long[n + 1];

        for (int i = 1; i <= n; ++i)
            endToStartAndEarns[i] = new ArrayList<>();

        for (int[] ride : rides) {
            final int start = ride[0];
            final int end = ride[1];
            final int tip = ride[2];
            final int earn = end - start + tip;
            endToStartAndEarns[end].add(new Pair<>(start, earn));
        }

        for (int i = 1; i <= n; ++i) {
            dp[i] = dp[i - 1];
            for (Pair<Integer, Integer> pair : endToStartAndEarns[i]) {
                final int start = pair.getKey();
                final int earn = pair.getValue();
                dp[i] = Math.max(dp[i], dp[start] + earn);
            }
        }

        return dp[n];
    }
}
```

1029. 2008.java

Path: LeetCode-main/solutions/2008. Maximum Earnings From Taxi/2008.java

```
class Solution {
    public long maxTaxiEarnings(int n, int[][] rides) {
        List<Pair<Integer, Integer>>[] startToEndAndEarns = new List[n];
        // dp[i] := the maximum dollars you can earn starting at i
        long[] dp = new long[n + 1];

        for (int i = 1; i < n; ++i)
            startToEndAndEarns[i] = new ArrayList<>();

        for (int[] ride : rides) {
            final int start = ride[0];
            final int end = ride[1];
            final int tip = ride[2];
            final int earn = end - start + tip;
            startToEndAndEarns[start].add(new Pair<>(end, earn));
        }

        for (int i = n - 1; i >= 1; --i) {
            dp[i] = dp[i + 1];
            for (Pair<Integer, Integer> pair : startToEndAndEarns[i]) {
                final int end = pair.getKey();
                final int earn = pair.getValue();
                dp[i] = Math.max(dp[i], dp[end] + earn);
            }
        }

        return dp[1];
    }
}
```

1030. 2009.java

Path: LeetCode-main/solutions/2009. Minimum Number of Operations to Make Array Continuous/2009.java

```
class Solution {
    public int minOperations(int[] nums) {
        final int n = nums.length;
        int ans = n;

        Arrays.sort(nums);
        nums = Arrays.stream(nums).distinct().toArray();

        for (int i = 0; i < nums.length; ++i) {
            final int start = nums[i];
            final int end = start + n - 1;
            final int index = firstGreater(nums, end);
            final int uniqueLength = index - i;
            ans = Math.min(ans, n - uniqueLength);
        }

        return ans;
    }

    private int firstGreater(int[] arr, int target) {
        final int i = Arrays.binarySearch(arr, target + 1);
        return i < 0 ? -i - 1 : i;
    }
}
```

1031. 201.java

Path: LeetCode-main/solutions/201. Bitwise AND of Numbers Range/201.java

```
class Solution {
    public int rangeBitwiseAnd(int m, int n) {
        int shiftBits = 0;

        while (m != n) {
            m >>= 1;
            n >>= 1;
            ++shiftBits;
        }

        return m << shiftBits;
    }
}
```

1032. 2011.java

Path: LeetCode-main/solutions/2011. Final Value of Variable After Performing Operations/2011.java

```
class Solution {  
    public int finalValueAfterOperations(String[] operations) {  
        int ans = 0;  
  
        for (final String op : operations)  
            ans += op.charAt(1) == '+' ? 1 : -1;  
  
        return ans;  
    }  
}
```

1033. 2012.java

Path: LeetCode-main/solutions/2012. Sum of Beauty in the Array/2012.java

```
class Solution {
    public int sumOfBeauties(int[] nums) {
        final int n = nums.length;
        int ans = 0;
        int[] minOfRight = new int[n];
        minOfRight[n - 1] = nums[n - 1];

        for (int i = n - 2; i >= 2; --i)
            minOfRight[i] = Math.min(nums[i], minOfRight[i + 1]);

        int maxOfLeft = nums[0];

        for (int i = 1; i <= n - 2; ++i) {
            if (maxOfLeft < nums[i] && nums[i] < minOfRight[i + 1])
                ans += 2;
            else if (nums[i - 1] < nums[i] && nums[i] < nums[i + 1])
                ans += 1;
            maxOfLeft = Math.max(maxOfLeft, nums[i]);
        }

        return ans;
    }
}
```


1034. 2013.java

Path: LeetCode-main/solutions/2013. Detect Squares/2013.java

```
class DetectSquares {
    public void add(int[] point) {
        pointCount.merge(getHash(point[0], point[1]), 1, Integer::sum);
    }

    public int count(int[] point) {
        final int x1 = point[0];
        final int y1 = point[1];
        int ans = 0;

        for (final int hash : pointCount.keySet()) {
            final int count = pointCount.get(hash);
            final int x3 = hash >> 10;
            final int y3 = hash & 1023;
            if (x1 != x3 && Math.abs(x1 - x3) == Math.abs(y1 - y3)) {
                final int p = getHash(x1, y3);
                final int q = getHash(x3, y1);
                if (pointCount.containsKey(p) && pointCount.containsKey(q))
                    ans += count * pointCount.get(p) * pointCount.get(q);
            }
        }

        return ans;
    }

    private Map<Integer, Integer> pointCount = new HashMap<>();

    private int getHash(int i, int j) {
        return i << 10 | j;
    }
}
```

1035. 2014.java

Path: LeetCode-main/solutions/2014. Longest Subsequence Repeated k Times/2014.java

```
class Solution {
    public String longestSubsequenceRepeatedK(String s, int k) {
        String ans = "";
        int[] count = new int[26];
        List<Character> possibleChars = new ArrayList<>();
        // Stores subsequences, where the length grows by 1 each time.
        Queue<String> q = new ArrayDeque<>(List.of(""));

        for (final char c : s.toCharArray())
            ++count[c - 'a'];

        for (char c = 'a'; c <= 'z'; ++c)
            if (count[c - 'a'] >= k)
                possibleChars.add(c);

        while (!q.isEmpty()) {
            final String currSubseq = q.poll();
            if (currSubseq.length() * k > s.length())
                return ans;
            for (final char c : possibleChars) {
                final String newSubseq = currSubseq + c;
                if (isSubsequence(newSubseq, s, k)) {
                    q.offer(newSubseq);
                    ans = newSubseq;
                }
            }
        }

        return ans;
    }

    private boolean isSubsequence(final String subseq, final String s, int k) {
        int i = 0; // subseq's index
        for (final char c : s.toCharArray())
            if (c == subseq.charAt(i))
                if (++i == subseq.length()) {
                    if (--k == 0)
                        return true;
                    i = 0;
                }
        return false;
    }
}
```

1036. 2015.java

Path: LeetCode-main/solutions/2015. Average Height of Buildings in Each Segment/2015.java

```
class Solution {
    public int[][] averageHeightOfBuildings(int[][] buildings) {
        List<int[]> ans = new ArrayList<>();
        List<Pair<Integer, Integer>> events = new ArrayList<>();

        for (int[] b : buildings) {
            final int start = b[0];
            final int end = b[1];
            final int height = b[2];
            events.add(new Pair<>(start, height));
            events.add(new Pair<>(end, -height));
        }

        Collections.sort(events, Comparator.comparingInt(Pair::getKey));

        int prev = 0;
        int count = 0;
        int sumHeight = 0;

        for (Pair<Integer, Integer> event : events) {
            final int curr = event.getKey();
            final int height = event.getValue();
            if (sumHeight > 0 && curr > prev) {
                final int avgHeight = sumHeight / count;
                if (!ans.isEmpty() && ans.get(ans.size() - 1)[1] == prev &&
                    avgHeight == ans.get(ans.size() - 1)[2])
                    ans.get(ans.size() - 1)[1] = curr;
                else
                    ans.add(new int[] {prev, curr, avgHeight});
            }
            sumHeight += height;
            count += height > 0 ? 1 : -1;
            prev = curr;
        }

        return ans.stream().toArray(int[][] ::new);
    }
}
```

1037. 2016.java

Path: LeetCode-main/solutions/2016. Maximum Difference Between Increasing Elements/2016.java

```
class Solution {
    public int maximumDifference(int[] nums) {
        int ans = -1;
        int mn = nums[0];

        for (int i = 1; i < nums.length; ++i) {
            if (nums[i] > mn)
                ans = Math.max(ans, nums[i] - mn);
            mn = Math.min(mn, nums[i]);
        }

        return ans;
    }
}
```

1038. 2017.java

Path: LeetCode-main/solutions/2017. Grid Game/2017.java

```
class Solution {
    public long gridGame(int[][] grid) {
        final int n = grid[0].length;
        long ans = Long.MAX_VALUE;
        long sumRow0 = Arrays.stream(grid[0]).asLongStream().sum();
        long sumRow1 = 0;

        for (int i = 0; i < n; ++i) {
            sumRow0 -= grid[0][i];
            ans = Math.min(ans, Math.max(sumRow0, sumRow1));
            sumRow1 += grid[1][i];
        }

        return ans;
    }
}
```

1039. 2018.java

Path: LeetCode-main/solutions/2018. Check if Word Can Be Placed In Crossword/2018.java

```
class Solution {
    public boolean placeWordInCrossword(char[][] board, String word) {
        for (char[][] state : new char[][][] {board, getRotated(board)})
            for (char[] chars : state)
                for (final String token : String.valueOf(chars).split("#"))
                    for (final String letters :
                        new String[] {word, new StringBuilder(word).reverse().toString()})
                        if (letters.length() == token.length())
                            if (canFit(letters, token))
                                return true;
        return false;
    }

    private char[][] getRotated(char[][] board) {
        final int m = board.length;
        final int n = board[0].length;
        char[][] rotated = new char[n][m];
        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j)
                rotated[j][i] = board[i][j];
        return rotated;
    }

    private boolean canFit(final String letters, final String token) {
        for (int i = 0; i < letters.length(); ++i)
            if (token.charAt(i) != ' ' && token.charAt(i) != letters.charAt(i))
                return false;
        return true;
    }
}
```

1040. 2019.java

Path: LeetCode-main/solutions/2019. The Score of Students Solving Math Expression/2019.java

```
class Solution {
    public int scoreOfStudents(String s, int[] answers) {
        final int n = s.length() / 2 + 1;
        int ans = 0;
        Set<Integer>[][] dp = new Set[n][n];
        Map<Integer, Integer> count = new HashMap<>();

        for (int i = 0; i < n; ++i)
            for (int j = i; j < n; ++j)
                dp[i][j] = new HashSet<>();

        for (int i = 0; i < n; ++i)
            dp[i][i].add(s.charAt(i * 2) - '0');

        for (int d = 1; d < n; ++d)
            for (int i = 0; i + d < n; ++i) {
                final int j = i + d;
                for (int k = i; k < j; ++k) {
                    final char op = s.charAt(k * 2 + 1);
                    for (final int a : dp[i][k])
                        for (final int b : dp[k + 1][j]) {
                            final int res = func(op, a, b);
                            if (res <= 1000)
                                dp[i][j].add(res);
                        }
                }
            }

        final int correctAnswer = eval(s);

        for (final int answer : answers)
            count.merge(answer, 1, Integer::sum);

        for (final int answer : count.keySet())
            if (answer == correctAnswer)
                ans += 5 * count.get(answer);
            else if (dp[0][n - 1].contains(answer))
                ans += 2 * count.get(answer);

        return ans;
    }

    private int eval(final String s) {
        int ans = 0;
        int currNum = 0;
        int prevNum = 0;
        char op = '+';

        for (int i = 0; i < s.length(); ++i) {
            final char c = s.charAt(i);
            if (Character.isDigit(c))
                currNum = currNum * 10 + (c - '0');
            if (!Character.isDigit(c) || i == s.length() - 1) {
                if (op == '+') {
                    ans += prevNum;
                    prevNum = currNum;
                } else if (op == '*') {
                    prevNum = prevNum * currNum;
                }
                op = c;
                currNum = 0;
            }
        }

        return ans + prevNum;
    }

    private int func(char op, int a, int b) {
        if (op == '+')
            return a + b;
        return a * b;
    }
}
```

1041. 202.java

Path: LeetCode-main/solutions/202. Happy Number/202.java

```
class Solution {
    public boolean isHappy(int n) {
        int slow = squaredSum(n);
        int fast = squaredSum(squaredSum(n));

        while (slow != fast) {
            slow = squaredSum(slow);
            fast = squaredSum(squaredSum(fast));
        }

        return slow == 1;
    }

    private int squaredSum(int n) {
        int sum = 0;
        while (n > 0) {
            sum += Math.pow(n % 10, 2);
            n /= 10;
        }
        return sum;
    }
}
```


1042. 2021.java

Path: LeetCode-main/solutions/2021. Brightest Position on Street/2021.java

```
class Solution {
    public int brightestPosition(int[][] lights) {
        int ans = Integer.MAX_VALUE;
        int maxBrightness = -1;
        int currBrightness = 0;
        Map<Integer, Integer> line = new TreeMap<>();

        for (int[] light : lights) {
            final int position = light[0];
            final int range = light[1];
            line.merge(position - range, 1, Integer::sum);
            line.merge(position + range + 1, -1, Integer::sum);
        }

        for (Map.Entry<Integer, Integer> entry : line.entrySet()) {
            final int pos = entry.getKey();
            final int brightness = entry.getValue();
            currBrightness += brightness;
            if (currBrightness > maxBrightness) {
                maxBrightness = currBrightness;
                ans = pos;
            }
        }

        return ans;
    }
}
```

1043. 2022.java

Path: LeetCode-main/solutions/2022. Convert 1D Array Into 2D Array/2022.java

```
class Solution {
    public int[][] construct2DArray(int[] original, int m, int n) {
        if (original.length != m * n)
            return new int[][] {};

        int[][] ans = new int[m][n];

        for (int i = 0; i < original.length; ++i)
            ans[i / n][i % n] = original[i];

        return ans;
    }
}
```

1044. 2023.java

Path: LeetCode-main/solutions/2023. Number of Pairs of Strings With Concatenation Equal to Target/2023.java

```
class Solution {
    public int numOfPairs(String[] nums, String target) {
        final int n = target.length();
        int ans = 0;
        Map<String, Integer> count = new HashMap<>();

        for (final String num : nums) {
            final int k = num.length();
            if (k >= n)
                continue;
            if (target.substring(0, k).equals(num))
                ans += count.getDefault(target.substring(k), 0);
            if (target.substring(n - k).equals(num))
                ans += count.getDefault(target.substring(0, n - k), 0);
            count.merge(num, 1, Integer::sum);
        }

        return ans;
    }
}
```

1045. 2024.java

Path: LeetCode-main/solutions/2024. Maximize the Confusion of an Exam/2024.java

```
class Solution {
    public int maxConsecutiveAnswers(String answerKey, int k) {
        int ans = 0;
        int maxCount = 0;
        int[] count = new int[2];

        for (int l = 0, r = 0; r < answerKey.length(); ++r) {
            maxCount = Math.max(maxCount, ++count[answerKey.charAt(r) == 'T' ? 1 : 0]);
            while (maxCount + k < r - l + 1)
                --count[answerKey.charAt(l++) == 'T' ? 1 : 0];
            ans = Math.max(ans, r - l + 1);
        }

        return ans;
    }
}
```

1046. 2025.java

Path: LeetCode-main/solutions/2025. Maximum Number of Ways to Partition an Array/2025.java

```
class Solution {
    public int waysToPartition(int[] nums, int k) {
        final int n = nums.length;
        final long sum = Arrays.stream(nums).asLongStream().sum();
        long prefix = 0;
        // count of sum(A[0..k)) - sum(A[k..n)) for k in [0..i)
        Map<Long, Integer> l = new HashMap<>();
        // count of sum(A[0..k)) - sum(A[k..n)) for k in [i..n)
        Map<Long, Integer> r = new HashMap<>();

        for (int pivot = 1; pivot < n; ++pivot) {
            prefix += nums[pivot - 1];
            final long suffix = sum - prefix;
            r.merge(prefix - suffix, 1, Integer::sum);
        }

        int ans = r.getDefault(0L, 0);
        prefix = 0;

        for (final int num : nums) {
            final long change = (long) k - num;
            ans = Math.max(ans, l.getDefault(change, 0) + r.getDefault(-change, 0));
            prefix += num;
            final long suffix = sum - prefix;
            final long diff = prefix - suffix;
            r.merge(diff, -1, Integer::sum);
            l.merge(diff, 1, Integer::sum);
        }

        return ans;
    }
}
```

1047. 2027.java

Path: LeetCode-main/solutions/2027. Minimum Moves to Convert String/2027.java

```
class Solution {
    public int minimumMoves(String s) {
        int ans = 0;

        for (int i = 0; i < s.length(); i++)
            if (s.charAt(i) == '0') {
                ++i;
            } else {
                ++ans;
                i += 3;
            }

        return ans;
    }
}
```

1048. 2028.java

Path: LeetCode-main/solutions/2028. Find Missing Observations/2028.java

```
class Solution {
    public int[] missingRolls(int[] rolls, int mean, int n) {
        final int targetSum = (rolls.length + n) * mean;
        int missingSum = targetSum - Arrays.stream(rolls).sum();
        if (missingSum > n * 6 || missingSum < n)
            return new int[] {};

        int[] ans = new int[n];
        Arrays.fill(ans, missingSum / n);
        missingSum %= n;
        for (int i = 0; i < missingSum; ++i)
            ++ans[i];

        return ans;
    }
}
```

1049. 2029.java

Path: LeetCode-main/solutions/2029. Stone Game IX/2029.java

```
class Solution {
    public boolean stoneGameIX(int[] stones) {
        int[] count = new int[3];

        for (final int stone : stones)
            ++count[stone % 3];

        if (count[0] % 2 == 0)
            return Math.min(count[1], count[2]) > 0;
        return Math.abs(count[1] - count[2]) > 2;
    }
}
```


1050. 203.java

Path: LeetCode-main/solutions/203. Remove Linked List Elements/203.java

```
class Solution {
    public ListNode removeElements(ListNode head, int val) {
        ListNode dummy = new ListNode(0, head);
        ListNode prev = dummy;

        for (; head != null; head = head.next)
            if (head.val != val) {
                prev.next = head;
                prev = prev.next;
            }
        prev.next = null; // In case that the last value equals `val`.

        return dummy.next;
    }
}
```

1051. 2030.java

Path: LeetCode-main/solutions/2030. Smallest K-Length Subsequence With Occurrences of a Letter/2030.java

```
class Solution {
    public String smallestSubsequence(String s, int k, char letter, int repetition) {
        StringBuilder sb = new StringBuilder();
        Deque<Character> stack = new ArrayDeque<>();
        int required = repetition;
        int nLetters = (int) s.chars().filter(c -> c == letter).count();

        for (int i = 0; i < s.length(); ++i) {
            final char c = s.charAt(i);
            while (!stack.isEmpty() && stack.peek() > c && stack.size() + s.length() - i - 1 >= k &&
                (stack.peek() != letter || nLetters > required))
                if (stack.pop() == letter)
                    ++required;
            if (stack.size() < k)
                if (c == letter) {
                    stack.push(c);
                    --required;
                } else if (k - stack.size() > required) {
                    stack.push(c);
                }
            if (c == letter)
                --nLetters;
        }

        for (final char c : stack)
            sb.append(c);

        return sb.reverse().toString();
    }
}
```

1052. 2031.java

Path: LeetCode-main/solutions/2031. Count Subarrays With More Ones Than Zeros/2031.java

```
class FenwickTree {
    public FenwickTree(int n) {
        sums = new int[n + 1];
    }

    public void add(int i, int delta) {
        while (i < sums.length) {
            sums[i] += delta;
            i += lowbit(i);
        }
    }

    public int get(int i) {
        int sum = 0;
        while (i > 0) {
            sum += sums[i];
            i -= lowbit(i);
        }
        return sum;
    }

    private int[] sums;

    private static int lowbit(int i) {
        return i & -i;
    }
}

class Solution {
    public int subarraysWithMoreZerosThanOnes(int[] nums) {
        final int MOD = 1_000_000_007;
        final int n = nums.length;
        int ans = 0;
        int prefix = 0;
        FenwickTree tree = new FenwickTree(2 * n + 1);
        tree.add(remap(0, n), 1);

        for (final int num : nums) {
            prefix += num == 0 ? -1 : 1;
            ans += tree.get(remap(prefix - 1, n));
            ans %= MOD;
            tree.add(remap(prefix, n), 1);
        }

        return ans;
    }

    private int remap(int i, int n) {
        return i + n + 1;
    }
}
```

1053. 2032.java

Path: LeetCode-main/solutions/2032. Two Out of Three/2032.java

```
class Solution {
    public List<Integer> twoOutOfThree(int[] nums1, int[] nums2, int[] nums3) {
        List<Integer> ans = new ArrayList<>();
        int[] count = new int[101];

        for (int[] nums : new int[][] {nums1, nums2, nums3})
            update(count, nums);

        for (int i = 1; i <= 100; ++i)
            if (count[i] >= 2)
                ans.add(i);

        return ans;
    }

    private void update(int[] count, int[] nums) {
        for (final int num : Arrays.stream(nums).boxed().collect(Collectors.toSet()))
            ++count[num];
    }
}
```

1054. 2033.java

Path: LeetCode-main/solutions/2033. Minimum Operations to Make a Uni-Value Grid/2033.java

```
class Solution {
    public int minOperations(int[][] grid, int x) {
        final int m = grid.length;
        final int n = grid[0].length;
        int[] arr = new int[m * n];
        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j)
                arr[i * n + j] = grid[i][j];
        if (Arrays.stream(arr).anyMatch(a -> (a - arr[0]) % x != 0))
            return -1;

        int ans = 0;

        Arrays.sort(arr);

        for (final int a : arr)
            ans += Math.abs(a - arr[arr.length / 2]) / x;

        return ans;
    }
}
```

1055. 2034.java

Path: LeetCode-main/solutions/2034. Stock Price Fluctuation/2034.java

```
class StockPrice {
    public void update(int timestamp, int price) {
        if (timestampToPrice.containsKey(timestamp)) {
            final int prevPrice = timestampToPrice.get(timestamp);
            if (pricesCount.merge(prevPrice, -1, Integer::sum) == 0)
                pricesCount.remove(prevPrice);
        }
        timestampToPrice.put(timestamp, price);
        pricesCount.merge(price, 1, Integer::sum);
    }

    public int current() {
        return timestampToPrice.lastEntry().getValue();
    }

    public int maximum() {
        return pricesCount.lastKey();
    }

    public int minimum() {
        return pricesCount.firstKey();
    }

    private TreeMap<Integer, Integer> timestampToPrice = new TreeMap<>();
    private TreeMap<Integer, Integer> pricesCount = new TreeMap<>();
}
```

1056. 2035.java

Path: LeetCode-main/solutions/2035. Partition Array Into Two Arrays to Minimize Sum Difference/2035.java

```
class Solution {
    public int minimumDifference(int[] nums) {
        final int n = nums.length / 2;
        final int sum = Arrays.stream(nums).sum();
        final int goal = sum / 2;
        final int[] lNums = Arrays.copyOfRange(nums, 0, n);
        final int[] rNums = Arrays.copyOfRange(nums, n, nums.length);
        int ans = Integer.MAX_VALUE;
        List<Integer>[] lSums = new List[n + 1];
        List<Integer>[] rSums = new List[n + 1];

        for (int i = 0; i <= n; ++i) {
            lSums[i] = new ArrayList<>();
            rSums[i] = new ArrayList<>();
        }

        dfs(lNums, 0, 0, 0, lSums);
        dfs(rNums, 0, 0, 0, rSums);

        for (int lCount = 0; lCount <= n; ++lCount) {
            List<Integer> l = lSums[lCount];
            List<Integer> r = rSums[n - lCount];
            Collections.sort(r);
            for (final int lSum : l) {
                final int i = firstGreaterEqual(r, goal - lSum);
                if (i < r.size()) {
                    final int sumPartOne = sum - lSum - r.get(i);
                    final int sumPartTwo = sum - sumPartOne;
                    ans = Math.min(ans, Math.abs(sumPartOne - sumPartTwo));
                }
                if (i > 0) {
                    final int sumPartOne = sum - lSum - r.get(i - 1);
                    final int sumPartTwo = sum - sumPartOne;
                    ans = Math.min(ans, Math.abs(sumPartOne - sumPartTwo));
                }
            }
        }

        return ans;
    }

    private void dfs(int[] arr, int i, int count, int path, List<Integer>[] sums) {
        if (i == arr.length) {
            sums[count].add(path);
            return;
        }
        dfs(arr, i + 1, count + 1, path + arr[i], sums);
        dfs(arr, i + 1, count, path, sums);
    }

    private int firstGreaterEqual(List<Integer> arr, int target) {
        final int i = Collections.binarySearch(arr, target);
        return i < 0 ? -i - 1 : i;
    }
}
```

1057. 2036.java

Path: LeetCode-main/solutions/2036. Maximum Alternating Subarray Sum/2036.java

```
class Solution {
    public long maximumAlternatingSubarraySum(int[] nums) {
        long ans = Integer.MIN_VALUE;
        long even = 0; // the subarray sum starting from an even index
        long odd = 0;  // the subarray sum starting from an odd index

        for (int i = 0; i < nums.length; ++i) {
            if (i % 2 == 0) // Must pick.
                even += nums[i];
            else // Start a fresh subarray or subtract `nums[i]`.
                even = Math.max(0, even - nums[i]);
            ans = Math.max(ans, even);
        }

        for (int i = 1; i < nums.length; ++i) {
            if (i % 2 == 1) // Must pick.
                odd += nums[i];
            else // Start a fresh subarray or subtract `nums[i]`.
                odd = Math.max(0, odd - nums[i]);
            ans = Math.max(ans, odd);
        }

        return ans;
    }
}
```


1058. 2037.java

Path: LeetCode-main/solutions/2037. Minimum Number of Moves to Seat Everyone/2037.java

```
class Solution {
    public int minMovesToSeat(int[] seats, int[] students) {
        int res = 0;

        Arrays.sort(seats);
        Arrays.sort(students);

        for (int i = 0; i < seats.length; ++i)
            res += Math.abs(seats[i] - students[i]);

        return res;
    }
}
```

1059. 2038.java

Path: LeetCode-main/solutions/2038. Remove Colored Pieces if Both Neighbors are the Same Color/2038.java

```
class Solution {
    public boolean winnerOfGame(String colors) {
        int countAAA = 0;
        int countBBB = 0;

        for (int i = 1; i + 1 < colors.length(); ++i)
            if (colors.charAt(i - 1) == colors.charAt(i) && colors.charAt(i) == colors.charAt(i + 1))
                if (colors.charAt(i) == 'A')
                    ++countAAA;
                else
                    ++countBBB;

        return countAAA > countBBB;
    }
}
```

1060. 2039.java

Path: LeetCode-main/solutions/2039. The Time When the Network Becomes Idle/2039.java

```
class Solution {
    public int networkBecomesIdle(int[][] edges, int[] patience) {
        final int n = patience.length;
        int ans = 0;
        List<Integer>[] graph = new List[n];
        Queue<Integer> q = new ArrayDeque<>(List.of(0));
        int[] dist = new int[n]; // dist[i] := the distance between i and 0
        Arrays.fill(dist, Integer.MAX_VALUE);
        dist[0] = 0;
        Arrays.setAll(graph, i -> new ArrayList<>());

        for (int[] edge : edges) {
            final int u = edge[0];
            final int v = edge[1];
            graph[u].add(v);
            graph[v].add(u);
        }

        while (!q.isEmpty())
            for (int sz = q.size(); sz > 0; --sz) {
                final int u = q.poll();
                for (final int v : graph[u])
                    if (dist[v] == Integer.MAX_VALUE) {
                        dist[v] = dist[u] + 1;
                        q.offer(v);
                    }
            }

        for (int i = 1; i < n; ++i) {
            final int numResending = (dist[i] * 2 - 1) / patience[i];
            final int lastResendingTime = patience[i] * numResending;
            final int lastArrivingTime = lastResendingTime + dist[i] * 2;
            ans = Math.max(ans, lastArrivingTime);
        }

        return ans + 1;
    }
}
```

1061. 204.java

Path: LeetCode-main/solutions/204. Count Primes/204.java

```
class Solution {
    public int countPrimes(int n) {
        if (n <= 2)
            return 0;
        final boolean[] isPrime = sieveEratosthenes(n);
        int ans = 0;
        return (int) IntStream.range(0, isPrime.length)
            .mapToObj(i -> isPrime[i])
            .filter(p -> p)
            .count();
    }

    private boolean[] sieveEratosthenes(int n) {
        boolean[] isPrime = new boolean[n];
        Arrays.fill(isPrime, true);
        isPrime[0] = false;
        isPrime[1] = false;
        for (int i = 2; i * i < n; ++i)
            if (isPrime[i])
                for (int j = i * i; j < n; j += i)
                    isPrime[j] = false;
        return isPrime;
    }
}
```

1062. 2040.java

Path: LeetCode-main/solutions/2040. Kth Smallest Product of Two Sorted Arrays/2040.java

```
class Solution {
    public long kthSmallestProduct(int[] nums1, int[] nums2, long k) {
        List<Integer> A1 = new ArrayList<>();
        List<Integer> A2 = new ArrayList<>();
        List<Integer> B1 = new ArrayList<>();
        List<Integer> B2 = new ArrayList<>();

        seperate(nums1, A1, A2);
        seperate(nums2, B1, B2);

        final long negCount = A1.size() * B2.size() + A2.size() * B1.size();
        int sign = 1;

        if (k > negCount) {
            k -= negCount; // Find the (k - negCount)-th positive.
        } else {
            k = negCount - k + 1; // Find the (negCount - k + 1)-th abs(negative).
            sign = -1;
            List<Integer> temp = B1;
            B1 = B2;
            B2 = temp;
        }

        long l = 0;
        long r = (long) 1e10;

        while (l < r) {
            final long m = (l + r) / 2;
            if (numProductNoGreaterThanOrEqualTo(A1, B1, m) + numProductNoGreaterThanOrEqualTo(A2, B2, m) >= k)
                r = m;
            else
                l = m + 1;
        }

        return sign * l;
    }

    private void seperate(int[] arr, List<Integer> A1, List<Integer> A2) {
        for (final int a : arr)
            if (a < 0)
                A1.add(-a);
            else
                A2.add(a);
        Collections.reverse(A1); // Reverse to sort ascending
    }

    private long numProductNoGreaterThanOrEqualTo(List<Integer> A, List<Integer> B, long m) {
        long count = 0;
        int j = B.size() - 1;
        // For each a, find the first index j s.t. a * B[j] <= m
        // So numProductNoGreaterThanOrEqualTo m for this row will be j + 1
        for (final long a : A) {
            while (j >= 0 && a * B.get(j) > m)
                --j;
            count += j + 1;
        }
        return count;
    }
}
```

1063. 2042.java

Path: LeetCode-main/solutions/2042. Check if Numbers Are Ascending in a Sentence/2042.java

```
class Solution {
    public boolean areNumbersAscending(String s) {
        int prev = 0;

        for (final String token : s.split(" "))
            if (Character.isDigit(token.charAt(0))) {
                final int num = Integer.parseInt(token);
                if (num <= prev)
                    return false;
                prev = num;
            }

        return true;
    }
}
```

1064. 2043.java

Path: LeetCode-main/solutions/2043. Simple Bank System/2043.java

```
public class Bank {
    public Bank(long[] balance) {
        this.balance = balance;
    }

    public boolean transfer(int account1, int account2, long money) {
        if (!isValid(account2))
            return false;
        return withdraw(account1, money) && deposit(account2, money);
    }

    public boolean deposit(int account, long money) {
        if (!isValid(account))
            return false;
        balance[account - 1] += money;
        return true;
    }

    public boolean withdraw(int account, long money) {
        if (!isValid(account))
            return false;
        if (balance[account - 1] < money)
            return false;
        balance[account - 1] -= money;
        return true;
    }

    private long[] balance;

    private boolean isValid(int account) {
        return 1 <= account && account <= balance.length;
    }
}
```

1065. 2044.java

Path: LeetCode-main/solutions/2044. Count Number of Maximum Bitwise-OR Subsets/2044.java

```
class Solution {
    public int countMaxOrSubsets(int[] nums) {
        final int ors = Arrays.stream(nums).reduce((a, b) -> a | b).getAsInt();
        dfs(nums, 0, 0, ors);
        return ans;
    }

    private int ans = 0;

    private void dfs(int[] nums, int i, int path, final int ors) {
        if (i == nums.length) {
            if (path == ors)
                ++ans;
            return;
        }

        dfs(nums, i + 1, path, ors);
        dfs(nums, i + 1, path | nums[i], ors);
    }
}
```


1066. 2045.java

Path: LeetCode-main/solutions/2045. Second Minimum Time to Reach Destination/2045.java

```
class Solution {
    public int secondMinimum(int n, int[][] edges, int time, int change) {
        List<Integer>[] graph = new List[n + 1];
        // (index, time)
        Queue<Pair<Integer, Integer>> q = new ArrayDeque<>(List.of(new Pair<>(1, 0)));
        // minTime[u][0] := the first minimum time to reach the node u
        // minTime[u][1] := the second minimum time to reach the node u
        int[][] minTime = new int[n + 1][2];
        Arrays.stream(minTime).forEach(A -> Arrays.fill(A, Integer.MAX_VALUE));
        minTime[1][0] = 0;

        for (int i = 1; i <= n; ++i)
            graph[i] = new ArrayList<>();

        for (int[] edge : edges) {
            final int u = edge[0];
            final int v = edge[1];
            graph[u].add(v);
            graph[v].add(u);
        }

        while (!q.isEmpty()) {
            final int u = q.peek().getKey();
            final int prevTime = q.poll().getValue();
            // Start from green.
            // If `numChangeSignal` is odd, now red.
            // If `numChangeSignal` is even, now green.
            final int numChangeSignal = prevTime / change;
            final int waitTime = numChangeSignal % 2 == 0 ? 0 : change - prevTime % change;
            final int newTime = prevTime + waitTime + time;
            for (final int v : graph[u])
                if (newTime < minTime[v][0]) {
                    minTime[v][0] = newTime;
                    q.offer(new Pair<>(v, newTime));
                } else if (minTime[v][0] < newTime && newTime < minTime[v][1]) {
                    if (v == n)
                        return newTime;
                    minTime[v][1] = newTime;
                    q.offer(new Pair<>(v, newTime));
                }
        }

        throw new IllegalArgumentException();
    }
}
```

1067. 2046.java

Path: LeetCode-main/solutions/2046. Sort Linked List Already Sorted Using Absolute Values/2046.java

```
class Solution {
    public ListNode sortLinkedList(ListNode head) {
        ListNode prev = head;
        ListNode curr = head.next;

        while (curr != null)
            if (curr.val < 0) {
                prev.next = curr.next;
                curr.next = head;
                head = curr;
                curr = prev.next;
            } else {
                prev = curr;
                curr = curr.next;
            }
        return head;
    }
}
```

1068. 2047-2.java

Path: LeetCode-main/solutions/2047. Number of Valid Words in a Sentence/2047-2.java

```
class Solution {
    public int countValidWords(String sentence) {
        final String regex = "[a-z]*([a-z]-[a-z])?[a-z]*[!,.]?";
        int ans = 0;

        for (final String token : sentence.trim().split("\\s+"))
            if (token.matches(regex))
                ++ans;

        return ans;
    }
}
```

1069. 2047.java

Path: LeetCode-main/solutions/2047. Number of Valid Words in a Sentence/2047.java

```
class Solution {
    public int countValidWords(String sentence) {
        int ans = 0;

        for (final String token : sentence.trim().split("\\s+"))
            if (isValid(token))
                ++ans;

        return ans;
    }

    private boolean isValid(final String token) {
        int countHyphen = 0;
        for (int i = 0; i < token.length(); ++i) {
            final char c = token.charAt(i);
            if (Character.isDigit(c))
                return false;
            if (c == '-') {
                if (i == 0 || !Character.isLowerCase(token.charAt(i - 1)))
                    return false;
                if (i + 1 == token.length() || !Character.isLowerCase(token.charAt(i + 1)))
                    return false;
                if (++countHyphen > 1)
                    return false;
            } else if (c == '!' || c == '.' || c == ',') {
                if (i != token.length() - 1)
                    return false;
            }
        }
        return true;
    }
}
```

1070. 2048.java

Path: LeetCode-main/solutions/2048. Next Greater Numerically Balanced Number/2048.java

```
class Solution {
    public int nextBeautifulNumber(int n) {
        while (!isBalance(++n))
            ;
        return n;
    }

    private boolean isBalance(int num) {
        int[] count = new int[10];
        while (num > 0) {
            if (num % 10 == 0)
                return false;
            ++count[num % 10];
            num /= 10;
        }
        for (int i = 1; i < 10; ++i)
            if (count[i] > 0 && count[i] != i)
                return false;
        return true;
    }
}
```

1071. 2049.java

Path: LeetCode-main/solutions/2049. Count Nodes With the Highest Score/2049.java

```
class Solution {
    public int countHighestScoreNodes(int[] parents) {
        List<Integer>[] tree = new List[parents.length];

        for (int i = 0; i < tree.length; ++i)
            tree[i] = new ArrayList<>();

        for (int i = 0; i < parents.length; ++i) {
            if (parents[i] == -1)
                continue;
            tree[parents[i]].add(i);
        }

        dfs(tree, 0);
        return ans;
    }

    private int ans = 0;
    private long maxScore = 0;

    private int dfs(List<Integer>[] tree, int u) {
        int count = 1;
        long score = 1;
        for (final int v : tree[u]) {
            final int childCount = dfs(tree, v);
            count += childCount;
            score *= childCount;
        }
        final int aboveCount = tree.length - count;
        score *= Math.max(aboveCount, 1);
        if (score > maxScore) {
            maxScore = score;
            ans = 1;
        } else if (score == maxScore) {
            ++ans;
        }
        return count;
    }
}
```

1072. 205.java

Path: LeetCode-main/solutions/205. Isomorphic Strings/205.java

```
class Solution {
    public boolean isIsomorphic(String s, String t) {
        Map<Character, Integer> charToIndex_s = new HashMap<>();
        Map<Character, Integer> charToIndex_t = new HashMap<>();

        for (Integer i = 0; i < s.length(); ++i)
            if (charToIndex_s.put(s.charAt(i), i) != charToIndex_t.put(t.charAt(i), i))
                return false;

        return true;
    }
}
```

1073. 2050.java

Path: LeetCode-main/solutions/2050. Parallel Courses III/2050.java

```
class Solution {
    public int minimumTime(int n, int[][] relations, int[] time) {
        List<Integer>[] graph = new List[n];
        int[] inDegrees = new int[n];
        int[] dist = time.clone();
        Arrays.setAll(graph, i -> new ArrayList<>());

        // Build the graph.
        for (int[] r : relations) {
            final int u = r[0] - 1;
            final int v = r[1] - 1;
            graph[u].add(v);
            ++inDegrees[v];
        }

        // Perform topological sorting.
        Queue<Integer> q = IntStream.range(0, n)
            .filter(i -> inDegrees[i] == 0)
            .boxed()
            .collect(Collectors.toCollection(ArrayDeque::new));

        while (!q.isEmpty()) {
            final int u = q.poll();
            for (final int v : graph[u]) {
                dist[v] = Math.max(dist[v], dist[u] + time[v]);
                if (--inDegrees[v] == 0)
                    q.offer(v);
            }
        }

        return Arrays.stream(dist).max().getAsInt();
    }
}
```


1074. 2052.java

Path: LeetCode-main/solutions/2052. Minimum Cost to Separate Sentence Into Rows/2052.java

```
class Solution {
    public int minimumCost(String sentence, int k) {
        if (sentence.length() <= k)
            return 0;

        String[] words = sentence.split(" ");
        // dp[i] := the minimum cost of the first i words
        int[] dp = new int[words.length + 1];

        for (int i = 1; i <= words.length; ++i) {
            int n = words[i - 1].length(); // the length of the current row
            dp[i] = dp[i - 1] + (k - n) * (k - n);
            // Gradually add words[j - 1], words[j - 2], ....
            for (int j = i - 1; j > 0; --j) {
                n += words[j - 1].length() + 1;
                if (n > k)
                    break;
                dp[i] = Math.min(dp[i], dp[j - 1] + (k - n) * (k - n));
            }
        }

        int lastRowLen = words[words.length - 1].length();
        int i = words.length - 2;

        while (i > 0 && lastRowLen + words[i].length() + 1 <= k)
            lastRowLen += words[i--].length();

        return Arrays.stream(dp, i + 1, words.length).min().getAsInt();
    }
}
```

1075. 2053.java

Path: LeetCode-main/solutions/2053. Kth Distinct String in an Array/2053.java

```
class Solution {
    public String kthDistinct(String[] arr, int k) {
        Map<String, Integer> count = new HashMap<>();

        for (final String a : arr)
            count.merge(a, 1, Integer::sum);

        for (final String a : arr)
            if (count.get(a) == 1 && --k == 0)
                return a;

        return "";
    }
}
```

1076. 2054.java

Path: LeetCode-main/solutions/2054. Two Best Non-Overlapping Events/2054.java

```
class Solution {
    public int maxTwoEvents(int[][] events) {
        record Event(int time, int value, int isStart) {}
        int ans = 0;
        int maxValue = 0;
        Event[] evts = new Event[events.length * 2];

        for (int i = 0; i < events.length; ++i) {
            final int start = events[i][0];
            final int end = events[i][1];
            final int value = events[i][2];
            evts[i * 2] = new Event(start, value, 1);
            evts[i * 2 + 1] = new Event(end + 1, value, 0);
        }

        Arrays.sort(evts, Comparator.comparingInt(Event::time).thenComparingInt(Event::isStart));

        for (Event evt : evts)
            if (evt.isStart == 1)
                ans = Math.max(ans, evt.value + maxValue);
            else
                maxValue = Math.max(maxValue, evt.value);

        return ans;
    }
}
```

1077. 2055-2.java

Path: LeetCode-main/solutions/2055. Plates Between Candles/2055-2.java

```
class Solution {
    public int[] platesBetweenCandles(String s, int[][] queries) {
        final int n = s.length();
        int[] ans = new int[queries.length];
        int[] closestLeftCandle = new int[n];
        int[] closestRightCandle = new int[n];
        int[] candleCount = new int[n]; // candleCount[i] := the number of candles in s[0..i]
        int candle = -1;
        int count = 0;

        for (int i = 0; i < n; ++i) {
            if (s.charAt(i) == '|') {
                candle = i;
                ++count;
            }
            closestLeftCandle[i] = candle;
            candleCount[i] = count;
        }

        candle = -1;
        for (int i = n - 1; i >= 0; --i) {
            if (s.charAt(i) == '|')
                candle = i;
            closestRightCandle[i] = candle;
        }

        for (int i = 0; i < queries.length; ++i) {
            final int left = queries[i][0];
            final int right = queries[i][1];
            final int l = closestRightCandle[left];
            final int r = closestLeftCandle[right];
            if (i == -1 || r == -1 || l > r)
                continue;
            final int lengthBetweenCandles = r - l + 1;
            final int numCandles = candleCount[r] - candleCount[l] + 1;
            ans[i] = lengthBetweenCandles - numCandles;
        }

        return ans;
    }
}
```

1078. 2055.java

Path: LeetCode-main/solutions/2055. Plates Between Candles/2055.java

```
class Solution {
    public int[] platesBetweenCandles(String s, int[][] queries) {
        int[] ans = new int[queries.length];
        List<Integer> indices = new ArrayList<>(); // indices of '|'

        for (int i = 0; i < s.length(); ++i)
            if (s.charAt(i) == '|')
                indices.add(i);

        for (int i = 0; i < queries.length; ++i) {
            final int left = queries[i][0];
            final int right = queries[i][1];
            final int l = firstGreaterEqual(indices, left);
            final int r = firstGreaterEqual(indices, right + 1) - 1;
            if (l < r) {
                final int lengthBetweenCandles = indices.get(r) - indices.get(l) + 1;
                final int numCandles = r - l + 1;
                ans[i] = lengthBetweenCandles - numCandles;
            }
        }

        return ans;
    }

    private int firstGreaterEqual(List<Integer> indices, int target) {
        final int i = Collections.binarySearch(indices, target);
        return i < 0 ? -i - 1 : i;
    }
}
```

1079. 2056.java

Path: LeetCode-main/solutions/2056. Number of Valid Move Combinations On Chessboard/2056.java

```
class Solution {
    public int countCombinations(String[] pieces, int[][] positions) {
        final int n = pieces.length;
        Set<Long> hashedBoards = new HashSet<>();
        // Stores all possible move combinations for `pieces`. Each element is a
        // vector of moves, one for each piece in the input order. e.g., if pieces =
        // ["rook", "bishop"], one element might be [[1,0], [1,1]], representing a
        // rook moving right and a bishop moving diagonally up-right.
        List<List<int[]>> combMoves = new ArrayList<>();

        getCombMoves(pieces, 0, new ArrayList<>(), combMoves);

        for (List<int[]> combMove : combMoves)
            dfs(positions, n, combMove, (1 << n) - 1, hashedBoards);

        return hashedBoards.size();
    }

    private static Map<String, int[][]> MOVES = new HashMap<>();

    static {
        MOVES.put("rook", new int[][] {{1, 0}, {-1, 0}, {0, 1}, {0, -1}});
        MOVES.put("bishop", new int[][] {{1, 1}, {1, -1}, {-1, 1}, {-1, -1}});
        MOVES.put("queen",
            new int[][] {{1, 0}, {-1, 0}, {0, 1}, {0, -1}, {1, 1}, {1, -1}, {-1, 1}, {-1, -1}});
    }

    // Generates all possible combinations of moves for the given pieces.
    private void getCombMoves(String[] pieces, int ithPiece, List<int[]> path,
        List<List<int[]>> combMoves) {
        if (ithPiece == pieces.length) {
            combMoves.add(new ArrayList<>(path));
            return;
        }

        for (int[] move : MOVES.get(pieces[ithPiece])) {
            path.add(move);
            getCombMoves(pieces, ithPiece + 1, path, combMoves);
            path.remove(path.size() - 1);
        }
    }

    // Performs a depth-first search to explore all possible board states.
    private void dfs(int[][] board, int n, List<int[]> combMove, int activeMask,
        Set<Long> hashedBoards) {
        if (activeMask == 0)
            return;
        hashedBoards.add(getHash(board));

        for (int nextActiveMask = 1; nextActiveMask < 1 << n; ++nextActiveMask) {
            if ((activeMask & nextActiveMask) != nextActiveMask)
                continue;

            // Copy the board.
            int[][] nextBoard = new int[n][];
            for (int i = 0; i < n; ++i)
                nextBoard[i] = board[i].clone();

            // Move the pieces that are active in this turn.
            for (int i = 0; i < n; ++i)
                if ((nextActiveMask >> i & 1) == 1) {
                    nextBoard[i][0] += combMove.get(i)[0];
                    nextBoard[i][1] += combMove.get(i)[1];
                }

            // No two or more pieces occupy the same square.
            if (getUniqueSize(nextBoard) < n)
                continue;

            // Every piece needs to be in the boundary.
            if (Arrays.stream(nextBoard).allMatch(p -> 1 <= p[0] && p[0] <= 8 && 1 <= p[1] && p[1] <= 8))
                dfs(nextBoard, n, combMove, nextActiveMask, hashedBoards);
        }
    }

    // Generates a unique hash for the given board state.
    private long getHash(int[][] board) {
        long hash = 0;
        for (int[] pos : board)
            hash = (hash * 64) + (pos[0] - 1 << 3) + (pos[1] - 1);
    }
}
```

```
        return hash;
    }

    // Counts the number of unique positions on the board occupied by the pieces.
    private int getUniqueSize(int[][] board) {
        Set<Integer> unique = new HashSet<>();
        for (int[] pos : board)
            unique.add(pos[0] * 8 + pos[1]);
        return unique.size();
    }
}
```

1080. 2057.java

Path: LeetCode-main/solutions/2057. Smallest Index With Equal Value/2057.java

```
class Solution {  
    public int smallestEqual(int[] nums) {  
        for (int i = 0; i < nums.length; ++i)  
            if (nums[i] == i % 10)  
                return i;  
        return -1;  
    }  
}
```


1081. 2058.java

Path: LeetCode-main/solutions/2058. Find the Minimum and Maximum Number of Nodes Between Critical Points/2058.java

```
class Solution {
    public int[] nodesBetweenCriticalPoints(ListNode head) {
        int minDistance = Integer.MAX_VALUE;
        int firstMaIndex = -1;
        int prevMaIndex = -1;
        int index = 1;
        ListNode prev = head;        // Point to the index 0.
        ListNode curr = head.next;   // Point to the index 1.

        while (curr.next != null) {
            if (curr.val > prev.val && curr.val > curr.next.val ||
                curr.val < prev.val && curr.val < curr.next.val) {
                if (firstMaIndex == -1) // Only assign once.
                    firstMaIndex = index;
                if (prevMaIndex != -1)
                    minDistance = Math.min(minDistance, index - prevMaIndex);
                prevMaIndex = index;
            }
            prev = curr;
            curr = curr.next;
            ++index;
        }

        if (minDistance == Integer.MAX_VALUE)
            return new int[] {-1, -1};
        return new int[] {minDistance, prevMaIndex - firstMaIndex};
    }
}
```

1082. 2059.java

Path: LeetCode-main/solutions/2059. Minimum Operations to Convert Number/2059.java

```
class Solution {
    public int minimumOperations(int[] nums, int start, int goal) {
        Queue<Integer> q = new ArrayDeque<>(List.of(start));
        boolean[] seen = new boolean[1001];
        seen[start] = true;

        for (int step = 1; !q.isEmpty(); ++step)
            for (int sz = q.size(); sz > 0; --sz) {
                final int x = q.poll();
                for (final int num : nums) {
                    for (final int res : new int[] {x + num, x - num, x ^ num}) {
                        if (res == goal)
                            return step;
                        if (res < 0 || res > 1000 || seen[res])
                            continue;
                        seen[res] = true;
                        q.offer(res);
                    }
                }
            }
        return -1;
    }
}
```

1083. 206-2.java

Path: LeetCode-main/solutions/206. Reverse Linked List/206-2.java

```
class Solution {  
    public ListNode reverseList(ListNode head) {  
        ListNode prev = null;  
  
        while (head != null) {  
            ListNode next = head.next;  
            head.next = prev;  
            prev = head;  
            head = next;  
        }  
  
        return prev;  
    }  
}
```

1084. 206.java

Path: LeetCode-main/solutions/206. Reverse Linked List/206.java

```
class Solution {  
    public ListNode reverseList(ListNode head) {  
        if (head == null || head.next == null)  
            return head;  
  
        ListNode newHead = reverseList(head.next);  
        head.next.next = head;  
        head.next = null;  
        return newHead;  
    }  
}
```

1085. 2060.java

Path: LeetCode-main/solutions/2060. Check if an Original String Exists Given Two Encoded Strings/2060.java

```
class Solution {
    public boolean possiblyEquals(String s1, String s2) {
        Map<Integer, Boolean>[][] mem = new Map[s1.length() + 1][s2.length() + 1];
        for (int i = 0; i <= s1.length(); ++i)
            for (int j = 0; j <= s2.length(); ++j)
                mem[i][j] = new HashMap<>();
        return f(s1, s2, 0, 0, 0, mem);
    }

    // Returns true if s1[i..n) matches s2[j..n), accounting for the padding
    // difference. Here, `paddingDiff` represents the signed padding. A positive
    // `paddingDiff` indicates that s1 has an additional number of offset bytes
    // compared to s2.
    private boolean f(final String s1, final String s2, int i, int j, int paddingDiff,
        Map<Integer, Boolean>[][] mem) {
        if (mem[i][j].containsKey(paddingDiff))
            return mem[i][j].get(paddingDiff);
        if (i == s1.length() && j == s2.length())
            return paddingDiff == 0;
        if (i < s1.length() && Character.isDigit(s1.charAt(i))) {
            // Add padding on s1.
            final int nextLetterIndex = getNextLetterIndex(s1, i);
            for (final int num : getNums(s1.substring(i, nextLetterIndex)))
                if (f(s1, s2, nextLetterIndex, j, paddingDiff + num, mem))
                    return true;
        } else if (j < s2.length() && Character.isDigit(s2.charAt(j))) {
            // Add padding on s2.
            final int nextLetterIndex = getNextLetterIndex(s2, j);
            for (final int num : getNums(s2.substring(j, nextLetterIndex)))
                if (f(s1, s2, i, nextLetterIndex, paddingDiff - num, mem))
                    return true;
        } else if (paddingDiff > 0) {
            // `s1` has more padding, so j needs to catch up.
            if (j < s2.length())
                return f(s1, s2, i, j + 1, paddingDiff - 1, mem);
        } else if (paddingDiff < 0) {
            // `s2` has more padding, so i needs to catch up.
            if (i < s1.length())
                return f(s1, s2, i + 1, j, paddingDiff + 1, mem);
        } else { // paddingDiff == 0
            // There's no padding difference, so consume the next letter.
            if (i < s1.length() && j < s2.length() && s1.charAt(i) == s2.charAt(j))
                return f(s1, s2, i + 1, j + 1, 0, mem);
        }
        mem[i][j].put(paddingDiff, false);
        return false;
    }

    private int getNextLetterIndex(final String s, int i) {
        int j = i;
        while (j < s.length() && Character.isDigit(s.charAt(j)))
            ++j;
        return j;
    }

    private List<Integer> getNums(final String s) {
        List<Integer> nums = new ArrayList<>(List.of(Integer.parseInt(s)));
        if (s.length() == 2) {
            nums.add(Integer.parseInt(s.substring(0, 1)) + Integer.parseInt(s.substring(1, 2)));
        } else if (s.length() == 3) {
            nums.add(Integer.parseInt(s.substring(0, 1)) + Integer.parseInt(s.substring(1, 3)));
            nums.add(Integer.parseInt(s.substring(0, 2)) + Integer.parseInt(s.substring(2, 3)));
            nums.add(Integer.parseInt(s.substring(0, 1)) + Integer.parseInt(s.substring(1, 2)) +
                Integer.parseInt(s.substring(2, 3)));
        }
        return nums;
    }
}
```

1086. 2061.java

Path: LeetCode-main/solutions/2061. Number of Spaces Cleaning Robot Cleaned/2061.java

```
class Solution {
    public int numberOfCleanRooms(int[][] room) {
        final int[][] DIRS = {{0, 1}, {1, 0}, {0, -1}, {-1, 0}};
        final int m = room.length;
        final int n = room[0].length;
        int ans = 1;
        int i = 0;
        int j = 0;
        int state = 0; // 0 := right, 1 := down, 2 := left, 3 := up
        int[][] seen = new int[m][n]; // seen[i][j] := bitmask
        seen[i][j] |= 1 << state;
        room[i][j] = 2; // 2 := cleaned

        while (true) {
            final int x = i + DIRS[state][0];
            final int y = j + DIRS[state][1];
            if (x < 0 || x == m || y < 0 || y == n || room[x][y] == 1) {
                // Turn 90 degrees clockwise.
                state = (state + 1) % 4;
            } else {
                // Walk to (x, y).
                if (room[x][y] == 0) {
                    ++ans;
                    room[x][y] = 2;
                }
                i = x;
                j = y;
            }
            if ((seen[i][j] >> state & 1) == 1)
                return ans;
            seen[i][j] |= (1 << state);
        }
    }
}
```

1087. 2062.java

Path: LeetCode-main/solutions/2062. Count Vowel Substrings of a String/2062.java

```
class Solution {
    public int countVowelSubstrings(String word) {
        return countVowelSubstringsAtMost(word, 5) - countVowelSubstringsAtMost(word, 4);
    }

    private int countVowelSubstringsAtMost(final String s, int goal) {
        int ans = 0;
        int k = goal;
        int[] count = new int[26];

        for (int l = 0, r = 0; r < s.length(); ++r) {
            if (!isVowel(s.charAt(r))) { // Fresh start.
                l = r + 1;
                k = goal;
                count = new int[26];
                continue;
            }
            if (++count[s.charAt(r) - 'a'] == 1)
                --k;
            while (k == -1)
                if (--count[s.charAt(l++) - 'a'] == 0)
                    ++k;
            ans += r - l + 1; // s[l..r], s[l + 1..r], ..., s[r]
        }

        return ans;
    }

    private boolean isVowel(char c) {
        return "aeiou".indexOf(c) != -1;
    }
}
```

1088. 2063-2.java

Path: LeetCode-main/solutions/2063. Vowels of All Substrings/2063-2.java

```
class Solution {
    public long countVowels(String word) {
        long ans = 0;

        for (int i = 0; i < word.length(); ++i)
            if (isVowel(word.charAt(i)))
                ans += (i + 1) * (long) (word.length() - i);

        return ans;
    }

    private boolean isVowel(char c) {
        return "aeiou".indexOf(c) != -1;
    }
}
```


1089. 2063.java

Path: LeetCode-main/solutions/2063. Vowels of All Substrings/2063.java

```
class Solution {
    public long countVowels(String word) {
        // dp[i] := the sum of the number of vowels of word[0..i), ...,
        // word[i - 1..i)
        long[] dp = new long[word.length() + 1];

        for (int i = 1; i <= word.length(); ++i) {
            dp[i] = dp[i - 1];
            if (isVowel(word.charAt(i - 1)))
                dp[i] += i;
        }

        return Arrays.stream(dp).sum();
    }

    private boolean isVowel(char c) {
        return "aeiou".indexOf(c) != -1;
    }
}
```

1090. 2064.java

Path: LeetCode-main/solutions/2064. Minimized Maximum of Products Distributed to Any Store/2064.java

```
class Solution {
    public int minimizedMaximum(int n, int[] quantities) {
        int l = 1;
        int r = Arrays.stream(quantities).max().getAsInt();

        while (l < r) {
            final int m = (l + r) / 2;
            if (numStores(quantities, m) <= n)
                r = m;
            else
                l = m + 1;
        }

        return l;
    }

    private int numStores(int[] quantities, int m) {
        // ceil(q / m)
        return Arrays.stream(quantities).reduce(0, (subtotal, q) -> subtotal + (q - 1) / m + 1);
    }
}
```

1091. 2065.java

Path: LeetCode-main/solutions/2065. Maximum Path Quality of a Graph/2065.java

```
class Solution {
    public int maximalPathQuality(int[] values, int[][] edges, int maxTime) {
        final int n = values.length;
        List<Pair<Integer, Integer>>[] graph = new List[n];
        int[] seen = new int[n];
        seen[0] = 1;
        Arrays.setAll(graph, i -> new ArrayList<>());

        for (int[] edge : edges) {
            final int u = edge[0];
            final int v = edge[1];
            final int time = edge[2];
            graph[u].add(new Pair<>(v, time));
            graph[v].add(new Pair<>(u, time));
        }

        dfs(graph, values, 0, values[0], maxTime, seen);
        return ans;
    }

    private int ans = 0;

    private void dfs(List<Pair<Integer, Integer>>[] graph, int[] values, int u, int quality,
                     int remainingTime, int[] seen) {
        if (u == 0)
            ans = Math.max(ans, quality);
        for (Pair<Integer, Integer> pair : graph[u]) {
            final int v = pair.getKey();
            final int time = pair.getValue();
            if (time > remainingTime)
                continue;
            final int newQuality = quality + values[v] * (seen[v] == 0 ? 1 : 0);
            ++seen[v];
            dfs(graph, values, v, newQuality, remainingTime - time, seen);
            --seen[v];
        }
    }
}
```

1092. 2067.java

Path: LeetCode-main/solutions/2067. Number of Equal Count Substrings/2067.java

```
class Solution {
    public int equalCountSubstrings(String s, int count) {
        final int maxUnique = s.chars().mapToObj(c -> (char) c).collect(Collectors.toSet()).size();
        int ans = 0;

        for (int unique = 1; unique <= maxUnique; ++unique) {
            final int windowSize = unique * count;
            int[] lettersCount = new int[26];
            int uniqueCount = 0;
            for (int i = 0; i < s.length(); ++i) {
                if (++lettersCount[s.charAt(i) - 'a'] == count)
                    ++uniqueCount;
                if (i >= windowSize && --lettersCount[s.charAt(i - windowSize) - 'a'] == count - 1)
                    --uniqueCount;
                ans += uniqueCount == unique ? 1 : 0;
            }
        }

        return ans;
    }
}
```

1093. 2068.java

Path: LeetCode-main/solutions/2068. Check Whether Two Strings are Almost Equivalent/2068.java

```
class Solution {
    public boolean checkAlmostEquivalent(String word1, String word2) {
        int[] count = new int[26];

        for (final char c : word1.toCharArray())
            ++count[c - 'a'];

        for (final char c : word2.toCharArray())
            --count[c - 'a'];

        return Arrays.stream(count).allMatch(freq -> Math.abs(freq) <= 3);
    }
}
```

1094. 2069.java

Path: LeetCode-main/solutions/2069. Walking Robot Simulation II/2069.java

```
class Robot {
    public Robot(int width, int height) {
        pos.add(new Pair<>(new int[] {0, 0}, "South"));
        for (int i = 1; i < width; ++i)
            pos.add(new Pair<>(new int[] {i, 0}, "East"));
        for (int j = 1; j < height; ++j)
            pos.add(new Pair<>(new int[] {width - 1, j}, "North"));
        for (int i = width - 2; i >= 0; --i)
            pos.add(new Pair<>(new int[] {i, height - 1}, "West"));
        for (int j = height - 2; j > 0; --j)
            pos.add(new Pair<>(new int[] {0, j}, "South"));
    }

    public void step(int num) {
        isOrigin = false;
        i = (i + num) % pos.size();
    }

    public int[] getPos() {
        return pos.get(i).getKey();
    }

    public String getDir() {
        return isOrigin ? "East" : pos.get(i).getValue();
    }

    private boolean isOrigin = true;
    private int i = 0;
    private List<Pair<int[], String>> pos = new ArrayList<>();
}
```

1095. 207.java

Path: LeetCode-main/solutions/207. Course Schedule/207.java

```
enum State { INIT, VISITING, VISITED }

class Solution {
    public boolean canFinish(int numCourses, int[][] prerequisites) {
        List<Integer>[] graph = new List[numCourses];
        State[] states = new State[numCourses];

        for (int i = 0; i < numCourses; ++i)
            graph[i] = new ArrayList<>();

        for (int[] prerequisite : prerequisites) {
            final int u = prerequisite[1];
            final int v = prerequisite[0];
            graph[u].add(v);
        }

        for (int i = 0; i < numCourses; ++i)
            if (hasCycle(graph, i, states))
                return false;

        return true;
    }

    private boolean hasCycle(List<Integer>[] graph, int u, State[] states) {
        if (states[u] == State.VISITING)
            return true;
        if (states[u] == State.VISITED)
            return false;
        states[u] = State.VISITING;
        for (final int v : graph[u])
            if (hasCycle(graph, v, states))
                return true;
        states[u] = State.VISITED;
        return false;
    }
}
```

1096. 2070.java

Path: LeetCode-main/solutions/2070. Most Beautiful Item for Each Query/2070.java

```
class Solution {
    public int[] maximumBeauty(int[][] items, int[] queries) {
        int[] ans = new int[queries.length];
        int[] prices = new int[items.length];
        int[] maxBeautySoFar = new int[items.length + 1];

        Arrays.sort(items, Comparator.comparingInt(item -> item[0]));

        for (int i = 0; i < items.length; ++i)
            maxBeautySoFar[i + 1] = Math.max(maxBeautySoFar[i], items[i][1]);

        for (int i = 0; i < queries.length; ++i) {
            final int index = firstGreater(items, queries[i]);
            ans[i] = maxBeautySoFar[index];
        }

        return ans;
    }

    private int firstGreater(int[][] items, int q) {
        int l = 0;
        int r = items.length;
        while (l < r) {
            final int m = (l + r) / 2;
            if (items[m][0] > q)
                r = m;
            else
                l = m + 1;
        }
        return l;
    }
}
```


1097. 2071.java

Path: LeetCode-main/solutions/2071. Maximum Number of Tasks You Can Assign/2071.java

```
class Solution {
    public int maxTaskAssign(int[] tasks, int[] workers, int pills, int strength) {
        int ans = 0;
        int l = 0;
        int r = Math.min(tasks.length, workers.length);

        Arrays.sort(tasks);
        Arrays.sort(workers);

        while (l <= r) {
            final int m = (l + r) / 2;
            if (canComplete(tasks, workers, pills, strength, m)) {
                ans = m;
                l = m + 1;
            } else {
                r = m - 1;
            }
        }

        return ans;
    }

    // Returns true if we can finish k tasks.
    private boolean canComplete(int[] tasks, int[] workers, int pillsLeft, int strength, int k) {
        // k strongest workers
        TreeMap<Integer, Integer> sortedWorkers = new TreeMap<>();
        for (int i = workers.length - k; i < workers.length; ++i)
            sortedWorkers.merge(workers[i], 1, Integer::sum);

        // Out of the k smallest tasks, start from the biggest one.
        for (int i = k - 1; i >= 0; --i) {
            // Find the first worker that has strength >= tasks[i].
            Integer lo = sortedWorkers.ceilingKey(tasks[i]);
            if (lo != null) {
                sortedWorkers.merge(lo, -1, Integer::sum);
                if (sortedWorkers.get(lo) == 0) {
                    sortedWorkers.remove(lo);
                }
            } else if (pillsLeft > 0) {
                // Find the first worker that has strength >= tasks[i] - strength.
                lo = sortedWorkers.ceilingKey(tasks[i] - strength);
                if (lo != null) {
                    sortedWorkers.merge(lo, -1, Integer::sum);
                    if (sortedWorkers.get(lo) == 0) {
                        sortedWorkers.remove(lo);
                    }
                }
                --pillsLeft;
            } else {
                return false;
            }
        }
        return true;
    }
}
```

1098. 2073.java

Path: LeetCode-main/solutions/2073. Time Needed to Buy Tickets/2073.java

```
class Solution {
    public int timeRequiredToBuy(int[] tickets, int k) {
        int ans = 0;

        for (int i = 0; i < tickets.length; ++i)
            if (i <= k)
                ans += Math.min(tickets[i], tickets[k]);
            else
                ans += Math.min(tickets[i], tickets[k] - 1);

        return ans;
    }
}
```

1099. 2074.java

Path: LeetCode-main/solutions/2074. Reverse Nodes in Even Length Groups/2074.java

```
class Solution {
    public ListNode reverseEvenLengthGroups(ListNode head) {
        // prev -> (head -> ... -> tail) -> next -> ...
        ListNode dummy = new ListNode(0, head);
        ListNode prev = dummy;
        ListNode tail = head;
        ListNode next = head.next;
        int groupLength = 1;

        while (true) {
            if (groupLength % 2 == 1) {
                prev.next = head;
                prev = tail;
            } else {
                tail.next = null;
                prev.next = reverse(head);
                // Prev -> (tail -> ... -> head) -> next -> ...
                head.next = next;
                prev = head;
            }
            if (next == null)
                break;
            head = next;
            Pair<ListNode, Integer> res = getTailAndLength(head, groupLength + 1);
            tail = res.getKey();
            next = tail.next;
            groupLength = res.getValue();
        }

        return dummy.next;
    }

    private Pair<ListNode, Integer> getTailAndLength(ListNode head, int groupLength) {
        int length = 1;
        ListNode tail = head;
        while (length < groupLength && tail.next != null) {
            tail = tail.next;
            ++length;
        }
        return new Pair<>(tail, length);
    }

    ListNode reverse(ListNode head) {
        ListNode prev = null;
        while (head != null) {
            ListNode next = head.next;
            head.next = prev;
            prev = head;
            head = next;
        }
        return prev;
    }
}
```

1100. 2076.java

Path: LeetCode-main/solutions/2076. Process Restricted Friend Requests/2076.java

```
class UnionFind {
    public UnionFind(int n) {
        id = new int[n];
        rank = new int[n];
        for (int i = 0; i < n; ++i)
            id[i] = i;
    }

    public void unionByRank(int u, int v) {
        final int i = find(u);
        final int j = find(v);
        if (i == j)
            return;
        if (rank[i] < rank[j]) {
            id[i] = j;
        } else if (rank[i] > rank[j]) {
            id[j] = i;
        } else {
            id[i] = j;
            ++rank[j];
        }
    }

    public int find(int u) {
        return id[u] == u ? u : (id[u] = find(id[u]));
    }

    private int[] id;
    private int[] rank;
}

class Solution {
    public boolean[] friendRequests(int n, int[][] restrictions, int[][] requests) {
        boolean[] ans = new boolean[requests.length];
        UnionFind uf = new UnionFind(n);

        for (int i = 0; i < requests.length; ++i) {
            final int pu = uf.find(requests[i][0]);
            final int pv = uf.find(requests[i][1]);
            boolean isValid = true;
            if (pu != pv)
                for (int[] restriction : restrictions) {
                    final int px = uf.find(restriction[0]);
                    final int py = uf.find(restriction[1]);
                    if (pu == px && pv == py || pu == py && pv == px) {
                        isValid = false;
                        break;
                    }
                }
            ans[i] = isValid;
            if (isValid)
                uf.unionByRank(pu, pv);
        }

        return ans;
    }
}
```

1101. 2078.java

Path: LeetCode-main/solutions/2078. Two Furthest Houses With Different Colors/2078.java

```
class Solution {
    public int maxDistance(int[] colors) {
        final int n = colors.length;
        int i = 0; // the leftmost index, where colors[i] != colors[n - 1]
        int j = n - 1; // the rightmost index, where colors[j] != colors[0]
        while (colors[i] == colors[n - 1])
            ++i;
        while (colors[j] == colors[0])
            --j;
        return Math.max(n - 1 - i, j);
    }
}
```

1102. 208.java

Path: LeetCode-main/solutions/208. Implement Trie (Prefix Tree)/208.java

```
class TrieNode {
    public TrieNode[] children = new TrieNode[26];
    public boolean isWord = false;
}

class Trie {
    public void insert(final String word) {
        TrieNode node = root;
        for (final char c : word.toCharArray()) {
            final int i = c - 'a';
            if (node.children[i] == null)
                node.children[i] = new TrieNode();
            node = node.children[i];
        }
        node.isWord = true;
    }

    public boolean search(final String word) {
        TrieNode node = find(word);
        return node != null && node.isWord;
    }

    public boolean startsWith(final String prefix) {
        return find(prefix) != null;
    }

    private TrieNode root = new TrieNode();

    private TrieNode find(final String prefix) {
        TrieNode node = root;
        for (final char c : prefix.toCharArray()) {
            final int i = c - 'a';
            if (node.children[i] == null)
                return null;
            node = node.children[i];
        }
        return node;
    }
}
```

1103. 2083.java

Path: LeetCode-main/solutions/2083. Substrings That Begin and End With the Same Letter/2083.java

```
class Solution {  
    public long numberOfSubstrings(String s) {  
        long ans = 0;  
        int[] count = new int[26];  
  
        for (final char c : s.toCharArray())  
            ans += ++count[c - 'a'];  
  
        return ans;  
    }  
}
```

1104. 2085.java

Path: LeetCode-main/solutions/2085. Count Common Words With One Occurrence/2085.java

```
class Solution {
    public int countWords(String[] words1, String[] words2) {
        Map<String, Integer> count = new HashMap<>();

        for (final String word : words1)
            count.merge(word, 1, Integer::sum);

        for (final String word : words2)
            if (count.containsKey(word) && count.get(word) < 2)
                count.merge(word, -1, Integer::sum);

        return (int) count.values().stream().filter(v -> v == 0).count();
    }
}
```


1105. 2086.java

Path: LeetCode-main/solutions/2086. Minimum Number of Buckets Required to Collect Rainwater from Houses/2086.java

```
class Solution {
    public int minimumBuckets(String street) {
        final char[] A = street.toCharArray();

        for (int i = 0; i < A.length; ++i)
            if (A[i] == 'H') {
                if (i > 0 && A[i - 1] == 'B')
                    continue;
                if (i + 1 < A.length && A[i + 1] == '.')
                    // Always prefer place a bucket in (i + 1) because it enhances the
                    // possibility to collect the upcoming houses.
                    A[i + 1] = 'B';
                else if (i > 0 && A[i - 1] == '.')
                    A[i - 1] = 'B';
                else
                    return -1;
            }

        return (int) new String(A).chars().filter(a -> a == 'B').count();
    }
}
```

1106. 2087.java

Path: LeetCode-main/solutions/2087. Minimum Cost Homecoming of a Robot in a Grid/2087.java

```
class Solution {
    public int minCost(int[] startPos, int[] homePos, int[] rowCosts, int[] colCosts) {
        int ans = 0;
        int i = startPos[0];
        int j = startPos[1];
        int x = homePos[0];
        int y = homePos[1];

        while (i != x)
            ans += i < x ? rowCosts[++i] : rowCosts[--i];
        while (j != y)
            ans += j < y ? colCosts[++j] : colCosts[--j];

        return ans;
    }
}
```

1107. 2088.java

Path: LeetCode-main/solutions/2088. Count Fertile Pyramids in a Land/2088.java

```
class Solution {
    public int countPyramids(int[][] grid) {
        return count(reversed(grid)) + count(grid);
    }

    // dp[i][j] := the maximum height of the pyramid for which it is the apex
    private int count(int[][] dp) {
        int ans = 0;
        for (int i = dp.length - 2; i >= 0; --i)
            for (int j = 1; j + 1 < dp[0].length; ++j)
                if (dp[i][j] == 1) {
                    dp[i][j] = Math.min(dp[i + 1][j - 1], Math.min(dp[i + 1][j], dp[i + 1][j + 1])) + 1;
                    ans += dp[i][j] - 1;
                }
        return ans;
    }

    private int[][] reversed(int[][] grid) {
        int[][] A = new int[grid.length][];
        for (int i = 0; i < grid.length; ++i)
            A[i] = grid[grid.length - i - 1].clone();
        return A;
    }
}
```

1108. 2089.java

Path: LeetCode-main/solutions/2089. Find Target Indices After Sorting Array/2089.java

```
class Solution {
    public List<Integer> targetIndices(int[] nums, int target) {
        List<Integer> ans = new ArrayList<>();
        final int count = (int) Arrays.stream(nums).filter(num -> num == target).count();
        int lessThan = (int) Arrays.stream(nums).filter(num -> num < target).count();

        for (int i = 0; i < count; ++i)
            ans.add(lessThan++);

        return ans;
    }
}
```

1109. 209.java

Path: LeetCode-main/solutions/209. Minimum Size Subarray Sum/209.java

```
class Solution {
    public int minSubArrayLen(int target, int[] nums) {
        int ans = Integer.MAX_VALUE;
        int sum = 0;

        for (int l = 0, r = 0; r < nums.length; ++r) {
            sum += nums[r];
            while (sum >= target) {
                ans = Math.min(ans, r - l + 1);
                sum -= nums[l++];
            }
        }

        return ans == Integer.MAX_VALUE ? 0 : ans;
    }
}
```

1110. 2090.java

Path: LeetCode-main/solutions/2090. K Radius Subarray Averages/2090.java

```
class Solution {
    public int[] getAverages(int[] nums, int k) {
        final int n = nums.length;
        final int size = 2 * k + 1;
        int[] ans = new int[n];
        Arrays.fill(ans, -1);
        if (size > n)
            return ans;

        long sum = 0;

        for (int i = 0; i < size; ++i)
            sum += nums[i];

        for (int i = k; i + k < n; ++i) {
            ans[i] = (int) (sum / size);
            if (i + k + 1 < n)
                sum += nums[i + k + 1] - nums[i - k];
        }

        return ans;
    }
}
```

1111. 2091.java

Path: LeetCode-main/solutions/2091. Removing Minimum and Maximum From Array/2091.java

```
class Solution {
    public int minimumDeletions(int[] nums) {
        final int n = nums.length;

        int mn = Integer.MAX_VALUE;
        int mx = Integer.MIN_VALUE;
        int minIndex = -1;
        int maxIndex = -1;

        for (int i = 0; i < n; ++i) {
            if (nums[i] < mn) {
                mn = nums[i];
                minIndex = i;
            }
            if (nums[i] > mx) {
                mx = nums[i];
                maxIndex = i;
            }
        }

        final int a = Math.min(minIndex, maxIndex);
        final int b = Math.max(minIndex, maxIndex);

        // min(delete from front and back,
        //      delete from front,
        //      delete from back)
        return Math.min(a + 1 + n - b, Math.min(b + 1, n - a));
    }
}
```

1112. 2092.java

Path: LeetCode-main/solutions/2092. Find All People With Secret/2092.java

```
class UnionFind {
    public UnionFind(int n) {
        id = new int[n];
        rank = new int[n];
        for (int i = 0; i < n; ++i)
            id[i] = i;
    }

    public void unionByRank(int u, int v) {
        final int i = find(u);
        final int j = find(v);
        if (i == j)
            return;
        if (rank[i] < rank[j]) {
            id[i] = j;
        } else if (rank[i] > rank[j]) {
            id[j] = i;
        } else {
            id[i] = j;
            ++rank[j];
        }
    }

    public boolean connected(int u, int v) {
        return find(u) == find(v);
    }

    public void reset(int u) {
        id[u] = u;
    }

    private int[] id;
    private int[] rank;

    private int find(int u) {
        return id[u] == u ? u : (id[u] = find(id[u]));
    }
}

class Solution {
    public List<Integer> findAllPeople(int n, int[][] meetings, int firstPerson) {
        List<Integer> ans = new ArrayList<>();
        UnionFind uf = new UnionFind(n);
        TreeMap<Integer, List<Pair<Integer, Integer>>> timeToPairs = new TreeMap<>();

        uf.unionByRank(0, firstPerson);

        for (int[] meeting : meetings) {
            final int x = meeting[0];
            final int y = meeting[1];
            final int time = meeting[2];
            timeToPairs.putIfAbsent(time, new ArrayList<>());
            timeToPairs.get(time).add(new Pair<>(x, y));
        }

        for (List<Pair<Integer, Integer>> pairs : timeToPairs.values()) {
            Set<Integer> peopleUnioned = new HashSet<>();
            for (Pair<Integer, Integer> pair : pairs) {
                final int x = pair.getKey();
                final int y = pair.getValue();
                uf.unionByRank(x, y);
                peopleUnioned.add(x);
                peopleUnioned.add(y);
            }
            for (final int person : peopleUnioned)
                if (!uf.connected(person, 0))
                    uf.reset(person);
        }

        for (int i = 0; i < n; ++i)
            if (uf.connected(i, 0))
                ans.add(i);

        return ans;
    }
}
```


1113. 2093.java

Path: LeetCode-main/solutions/2093. Minimum Cost to Reach City With Discounts/2093.java

```
class Solution {
    public int minimumCost(int n, int[][] highways, int discounts) {
        record T(int u, int d, int leftDiscounts) {}
        List<Pair<Integer, Integer>>[] graph = new List[n];
        Queue<T> minHeap = new PriorityQueue<>(Comparator.comparingInt(T::d));
        Map<Integer, Integer> minDiscounts = new HashMap<>();

        for (int i = 0; i < graph.length; i++)
            graph[i] = new ArrayList<>();

        for (int[] h : highways) {
            final int city1 = h[0];
            final int city2 = h[1];
            final int toll = h[2];
            graph[city1].add(new Pair<>(city2, toll));
            graph[city2].add(new Pair<>(city1, toll));
        }

        minHeap.offer(new T(0, 0, discounts));

        while (!minHeap.isEmpty()) {
            final int u = minHeap.peek().u;
            final int d = minHeap.peek().d;
            final int leftDiscounts = minHeap.poll().leftDiscounts;
            if (u == n - 1)
                return d;
            if (minDiscounts.getOrDefault(u, -1) >= leftDiscounts)
                continue;
            minDiscounts.put(u, leftDiscounts);
            for (Pair<Integer, Integer> pair : graph[u]) {
                final int v = pair.getKey();
                final int w = pair.getValue();
                minHeap.offer(new T(v, d + w, leftDiscounts));
                if (leftDiscounts > 0)
                    minHeap.offer(new T(v, d + w / 2, leftDiscounts - 1));
            }
        }
        return -1;
    }
}
```

1114. 2094.java

Path: LeetCode-main/solutions/2094. Finding 3-Digit Even Numbers/2094.java

```
class Solution {
    public int[] findEvenNumbers(int[] digits) {
        List<Integer> ans = new ArrayList<>();
        int[] count = new int[10];

        for (final int digit : digits)
            ++count[digit];

        // Try to construct `abc`.
        for (int a = 1; a <= 9; ++a)
            for (int b = 0; b <= 9; ++b)
                for (int c = 0; c <= 8; c += 2)
                    if (count[a] > 0 && count[b] > (b == a ? 1 : 0) &&
                        count[c] > (c == a ? 1 : 0) + (c == b ? 1 : 0))
                        ans.add(a * 100 + b * 10 + c);

        return ans.stream().mapToInt(Integer::intValue).toArray();
    }
}
```

1115. 2095.java

Path: LeetCode-main/solutions/2095. Delete the Middle Node of a Linked List/2095.java

```
class Solution {
    public ListNode deleteMiddle(ListNode head) {
        ListNode dummy = new ListNode(0, head);
        ListNode slow = dummy;
        ListNode fast = dummy;

        while (fast.next != null && fast.next.next != null) {
            slow = slow.next;
            fast = fast.next.next;
        }

        // Delete the middle node.
        slow.next = slow.next.next;
        return dummy.next;
    }
}
```

1116. 2096-2.java

Path: LeetCode-main/solutions/2096. Step-By-Step Directions From a Binary Tree Node to Another/2096-2.java

```
class Solution {
    public String getDirections(TreeNode root, int startValue, int destValue) {
        StringBuilder pathToStart = new StringBuilder();
        StringBuilder pathToDest = new StringBuilder();

        dfs(root, startValue, pathToStart);
        dfs(root, destValue, pathToDest);

        while (pathToStart.length() > 0 && pathToDest.length() > 0 &&
            pathToStart.charAt(pathToStart.length() - 1) ==
            pathToDest.charAt(pathToDest.length() - 1)) {
            pathToStart.setLength(pathToStart.length() - 1);
            pathToDest.setLength(pathToDest.length() - 1);
        }

        return "U".repeat(pathToStart.length()) + pathToDest.reverse().toString();
    }

    // Builds the string in reverse order to avoid creating a new copy.
    private boolean dfs(TreeNode root, int val, StringBuilder sb) {
        if (root.val == val)
            return true;
        if (root.left != null && dfs(root.left, val, sb))
            sb.append("L");
        else if (root.right != null && dfs(root.right, val, sb))
            sb.append("R");
        return sb.length() > 0;
    }
}
```

1117. 2096.java

Path: LeetCode-main/solutions/2096. Step-By-Step Directions From a Binary Tree Node to Another/2096.java

```
class Solution {
    public String getDirections(TreeNode root, int startValue, int destValue) {
        // Only this subtree matters.
        dfs(lca(root, startValue, destValue), startValue, destValue, new StringBuilder());
        return "U".repeat(pathToStart.length()) + pathToDest;
    }

    private String pathToStart = "";
    private String pathToDest = "";

    private TreeNode lca(TreeNode root, int p, int q) {
        if (root == null || root.val == p || root.val == q)
            return root;
        TreeNode left = lca(root.left, p, q);
        TreeNode right = lca(root.right, p, q);
        if (left != null && right != null)
            return root;
        return left == null ? right : left;
    }

    private void dfs(TreeNode root, int p, int q, StringBuilder path) {
        if (root == null)
            return;
        if (root.val == p)
            pathToStart = path.toString();
        if (root.val == q)
            pathToDest = path.toString();
        dfs(root.left, p, q, path.append('L'));
        path.deleteCharAt(path.length() - 1);
        dfs(root.right, p, q, path.append('R'));
        path.deleteCharAt(path.length() - 1);
    }
}
```

1118. 2097.java

Path: LeetCode-main/solutions/2097. Valid Arrangement of Pairs/2097.java

```
class Solution {
    public int[][] validArrangement(int[][] pairs) {
        List<int[]> ans = new ArrayList<>();
        Map<Integer, Deque<Integer>> graph = new HashMap<>();
        Map<Integer, Integer> outDegree = new HashMap<>();
        Map<Integer, Integer> inDegrees = new HashMap<>();

        for (int[] pair : pairs) {
            final int start = pair[0];
            final int end = pair[1];
            graph.putIfAbsent(start, new ArrayDeque<>());
            graph.get(start).push(end);
            outDegree.merge(start, 1, Integer::sum);
            inDegrees.merge(end, 1, Integer::sum);
        }

        final int startNode = getStartNode(graph, outDegree, inDegrees, pairs);
        euler(graph, startNode, ans);
        Collections.reverse(ans);
        return ans.stream().toArray(int[][] ::new);
    }

    private int getStartNode(Map<Integer, Deque<Integer>> graph, Map<Integer, Integer> outDegree,
                             Map<Integer, Integer> inDegrees, int[][] pairs) {
        for (final int u : graph.keySet())
            if (outDegree.getOrDefault(u, 0) - inDegrees.getOrDefault(u, 0) == 1)
                return u;
        return pairs[0][0]; // Arbitrarily choose a node.
    }

    private void euler(Map<Integer, Deque<Integer>> graph, int u, List<int[]> ans) {
        Deque<Integer> stack = graph.get(u);
        while (stack != null && !stack.isEmpty()) {
            final int v = stack.pop();
            euler(graph, v, ans);
            ans.add(new int[] {u, v});
        }
    }
}
```

1119. 2098.java

Path: LeetCode-main/solutions/2098. Subsequence of Size K With the Largest Even Sum/2098.java

```
class Solution {
    public long largestEvenSum(int[] nums, int k) {
        Arrays.sort(nums);

        long sum = 0;
        int minOdd = -1;
        int minEven = -1;
        int maxOdd = -1;
        int maxEven = -1;

        for (int i = nums.length - 1; i + k >= nums.length; --i) {
            sum += nums[i];
            if (nums[i] % 2 == 1)
                minOdd = nums[i];
            else
                minEven = nums[i];
        }

        if (sum % 2 == 0)
            return sum;

        for (int i = 0; i + k < nums.length; ++i)
            if (nums[i] % 2 == 1)
                maxOdd = nums[i];
            else
                maxEven = nums[i];

        long ans = -1;

        if (maxEven >= 0 && minOdd >= 0)
            ans = Math.max(ans, sum + maxEven - minOdd);
        if (maxOdd >= 0 && minEven >= 0)
            ans = Math.max(ans, sum + maxOdd - minEven);
        return ans;
    }
}
```

1120. 2099.java

Path: LeetCode-main/solutions/2099. Find Subsequence of Length K With the Largest Sum/2099.java

```
class Solution {
    public int[] maxSubsequence(int[] nums, int k) {
        int[] ans = new int[k];
        int[] arr = nums.clone();
        Arrays.sort(arr);
        final int threshold = arr[arr.length - k];
        final int larger = (int) Arrays.stream(nums).filter(num -> num > threshold).count();
        int equal = k - larger;

        int i = 0;
        for (final int num : nums)
            if (num > threshold) {
                ans[i++] = num;
            } else if (num == threshold && equal > 0) {
                ans[i++] = num;
                --equal;
            }

        return ans;
    }
}
```


1121. 21.java

Path: LeetCode-main/solutions/21. Merge Two Sorted Lists/21.java

```
class Solution {
    public ListNode mergeTwoLists(ListNode list1, ListNode list2) {
        if (list1 == null || list2 == null)
            return list1 == null ? list2 : list1;
        if (list1.val > list2.val) {
            ListNode temp = list1;
            list1 = list2;
            list2 = temp;
        }
        list1.next = mergeTwoLists(list1.next, list2);
        return list1;
    }
}
```

1122. 210-2.java

Path: LeetCode-main/solutions/210. Course Schedule II/210-2.java

```
class Solution {
    public int[] findOrder(int numCourses, int[][] prerequisites) {
        List<Integer> ans = new ArrayList<>();
        List<Integer>[] graph = new List[numCourses];
        int[] inDegrees = new int[numCourses];

        for (int i = 0; i < numCourses; ++i)
            graph[i] = new ArrayList<>();

        // Build the graph.
        for (int[] prerequisite : prerequisites) {
            final int u = prerequisite[1];
            final int v = prerequisite[0];
            graph[u].add(v);
            ++inDegrees[v];
        }

        // Perform topological sorting.
        Queue<Integer> q = IntStream.range(0, numCourses)
            .filter(i -> inDegrees[i] == 0)
            .boxed()
            .collect(Collectors.toCollection(ArrayDeque::new));

        while (!q.isEmpty()) {
            final int u = q.poll();
            ans.add(u);
            for (final int v : graph[u])
                if (--inDegrees[v] == 0)
                    q.offer(v);
        }

        if (ans.size() == numCourses)
            return ans.stream().mapToInt(Integer::intValue).toArray();
        return new int[] {};
    }
}
```

1123. 210.java

Path: LeetCode-main/solutions/210. Course Schedule II/210.java

```
enum State { INIT, VISITING, VISITED }

class Solution {
    public int[] findOrder(int numCourses, int[][] prerequisites) {
        Deque<Integer> ans = new ArrayDeque<>();
        List<Integer>[] graph = new List[numCourses];
        State[] states = new State[numCourses];

        for (int i = 0; i < numCourses; ++i)
            graph[i] = new ArrayList<>();

        for (int[] prerequisite : prerequisites) {
            final int u = prerequisite[1];
            final int v = prerequisite[0];
            graph[u].add(v);
        }

        for (int i = 0; i < numCourses; ++i)
            if (hasCycle(graph, i, states, ans))
                return new int[] {};

        return ans.stream().mapToInt(Integer::intValue).toArray();
    }

    private boolean hasCycle(List<Integer>[] graph, int u, State[] states, Deque<Integer> ans) {
        if (states[u] == State.VISITING)
            return true;
        if (states[u] == State.VISITED)
            return false;
        states[u] = State.VISITING;
        for (final int v : graph[u])
            if (hasCycle(graph, v, states, ans))
                return true;
        states[u] = State.VISITED;
        ans.addFirst(u);
        return false;
    }
}
```

1124. 2100.java

Path: LeetCode-main/solutions/2100. Find Good Days to Rob the Bank/2100.java

```
class Solution {
    public List<Integer> goodDaysToRobBank(int[] security, int time) {
        final int n = security.length;
        List<Integer> ans = new ArrayList<>();
        // dec[i] := the number of continuous decreasing numbers before i
        int[] dec = new int[n];
        // inc[i] := the number of continuous increasing numbers after i
        int[] inc = new int[n];

        for (int i = 1; i < n; ++i)
            if (security[i - 1] >= security[i])
                dec[i] = dec[i - 1] + 1;

        for (int i = n - 2; i >= 0; --i)
            if (security[i] <= security[i + 1])
                inc[i] = inc[i + 1] + 1;

        for (int i = 0; i < n; ++i)
            if (dec[i] >= time && inc[i] >= time)
                ans.add(i);

        return ans;
    }
}
```

1125. 2101.java

Path: LeetCode-main/solutions/2101. Detonate the Maximum Bombs/2101.java

```
class Solution {
    public int maximumDetonation(int[][] bombs) {
        final int n = bombs.length;
        int ans = 0;
        List<Integer>[] graph = new List[n];
        Arrays.setAll(graph, i -> new ArrayList<>());

        for (int i = 0; i < n; ++i) {
            for (int j = 0; j < n; ++j) {
                if (i == j)
                    continue;
                final long ri = bombs[i][2];
                if (ri * ri >= squaredDist(bombs, i, j))
                    graph[i].add(j);
            }
        }

        for (int i = 0; i < n; ++i) {
            Set<Integer> seen = new HashSet<>(Arrays.asList(i));
            dfs(graph, i, seen);
            ans = Math.max(ans, seen.size());
        }

        return ans;
    }

    private void dfs(List<Integer>[] graph, int u, Set<Integer> seen) {
        for (final int v : graph[u]) {
            if (seen.contains(v))
                continue;
            seen.add(v);
            dfs(graph, v, seen);
        }
    }

    private long squaredDist(int[][] bombs, int i, int j) {
        return (long) (bombs[i][0] - bombs[j][0]) * (bombs[i][0] - bombs[j][0]) +
            (long) (bombs[i][1] - bombs[j][1]) * (bombs[i][1] - bombs[j][1]);
    }
};
```

1126. 2102.java

Path: LeetCode-main/solutions/2102. Sequentially Ordinal Rank Tracker/2102.java

```
class SORTracker {
    public void add(String name, int score) {
        l.offer(new Location(name, score));
        if (l.size() > k + 1)
            r.offer(l.poll());
    }

    public String get() {
        final String name = l.peek().name;
        if (!r.isEmpty())
            l.offer(r.poll());
        ++k;
        return name;
    }

    private record Location(String name, int score) {}

    private Queue<Location> l =
        new PriorityQueue<>(Comparator.comparingInt(Location::score)
            .thenComparing(Location::name, Comparator.reverseOrder()));
    private Queue<Location> r =
        new PriorityQueue<>(Comparator.comparingInt(Location::score, Comparator.reverseOrder())
            .thenComparing(Location::name));
    private int k;
}
```

1127. 2103.java

Path: LeetCode-main/solutions/2103. Rings and Rods/2103.java

```
class Solution {
    public int countPoints(String rings) {
        int[] colors = new int[10];

        for (int i = 0; i < rings.length(); i += 2) {
            final char c = rings.charAt(i);
            final int color = c == 'R' ? 1 : c == 'G' ? 2 : 4;
            colors[rings.charAt(i + 1) - '0'] |= color;
        }

        return (int) Arrays.stream(colors).filter(c -> c != 0).count();
    }
}
```

1128. 2104.java

Path: LeetCode-main/solutions/2104. Sum of Subarray Ranges/2104.java

```
class Solution {
    public long subArrayRanges(int[] nums) {
        Pair<int[], int[]> gt = getPrevNext(nums, (a, b) -> a < b);
        Pair<int[], int[]> lt = getPrevNext(nums, (a, b) -> a > b);
        int[] prevGt = gt.getKey();
        int[] nextGt = gt.getValue();
        int[] prevLt = lt.getKey();
        int[] nextLt = lt.getValue();
        long ans = 0;

        for (int i = 0; i < nums.length; ++i) {
            ans += (long) nums[i] * (i - prevGt[i]) * (nextGt[i] - i);
            ans -= (long) nums[i] * (i - prevLt[i]) * (nextLt[i] - i);
        }

        return ans;
    }

    // Returns `prev` and `next`, that store the indices of the nearest numbers
    // that are smaller or larger than the current number depending on `op`.
    private Pair<int[], int[]> getPrevNext(int[] nums, BiFunction<Integer, Integer, Boolean> op) {
        final int n = nums.length;
        int[] prev = new int[n];
        int[] next = new int[n];
        Arrays.fill(prev, -1);
        Arrays.fill(next, n);
        Deque<Integer> stack = new ArrayDeque<>();
        for (int i = 0; i < n; ++i) {
            while (!stack.isEmpty() && op.apply(nums[stack.peek()], nums[i]))
                next[stack.pop()] = i;
            if (!stack.isEmpty())
                prev[i] = stack.peek();
            stack.push(i);
        }
        return new Pair<>(prev, next);
    }
}
```


1129. 2105.java

Path: LeetCode-main/solutions/2105. Watering Plants II/2105.java

```
class Solution {
    public int minimumRefill(int[] plants, int capacityA, int capacityB) {
        int ans = 0;
        int i = 0;
        int j = plants.length - 1;
        int canA = capacityA;
        int canB = capacityB;

        while (i < j) {
            ans += (canA < plants[i] ? 1 : 0) + (canB < plants[j] ? 1 : 0);
            if (canA < plants[i])
                canA = capacityA;
            if (canB < plants[j])
                canB = capacityB;
            canA -= plants[i++];
            canB -= plants[j--];
        }

        return ans + (i == j && Math.max(canA, canB) < plants[i] ? 1 : 0);
    }
}
```

1130. 2106.java

Path: LeetCode-main/solutions/2106. Maximum Fruits Harvested After at Most K Steps/2106.java

```
class Solution {
    public int maxTotalFruits(int[][] fruits, int startPos, int k) {
        final int maxRight = Math.max(startPos, fruits[fruits.length - 1][0]);
        int ans = 0;
        int[] amounts = new int[1 + maxRight];
        int[] prefix = new int[2 + maxRight];

        for (int[] f : fruits)
            amounts[f[0]] = f[1];

        for (int i = 0; i + 1 < prefix.length; ++i)
            prefix[i + 1] = prefix[i] + amounts[i];

        // Go right first.
        final int maxRightSteps = Math.min(maxRight - startPos, k);
        for (int rightSteps = 0; rightSteps <= maxRightSteps; ++rightSteps) {
            final int leftSteps = Math.max(0, k - 2 * rightSteps); // Turn left
            ans = Math.max(ans, getFruits(startPos, maxRight, leftSteps, rightSteps, prefix));
        }

        // Go left first.
        final int maxLeftSteps = Math.min(startPos, k);
        for (int leftSteps = 0; leftSteps <= maxLeftSteps; ++leftSteps) {
            final int rightSteps = Math.max(0, k - 2 * leftSteps); // Turn right
            ans = Math.max(ans, getFruits(startPos, maxRight, leftSteps, rightSteps, prefix));
        }

        return ans;
    }

    private int getFruits(int startPos, int maxRight, int leftSteps, int rightSteps, int[] prefix) {
        final int l = Math.max(0, startPos - leftSteps);
        final int r = Math.min(maxRight, startPos + rightSteps);
        return prefix[r + 1] - prefix[l];
    }
}
```

1131. 2107.java

Path: LeetCode-main/solutions/2107. Number of Unique Flavors After Sharing K Candies/2107.java

```
class Solution {
    public int shareCandies(int[] candies, int k) {
        int ans = 0;
        Map<Integer, Integer> count = new HashMap<>();

        for (final int candy : candies)
            count.merge(candy, 1, Integer::sum);

        int unique = count.size();

        for (int i = 0; i < candies.length; ++i) {
            if (count.merge(candies[i], -1, Integer::sum) == 0) {
                count.remove(candies[i]);
                --unique;
            }
            if (i >= k) {
                count.merge(candies[i - k], 1, Integer::sum);
                if (count.get(candies[i - k]) == 1)
                    ++unique;
            }
            if (i >= k - 1)
                ans = Math.max(ans, unique);
        }

        return ans;
    }
}
```

1132. 2108.java

Path: LeetCode-main/solutions/2108. Find First Palindromic String in the Array/2108.java

```
class Solution {
    public String firstPalindrome(String[] words) {
        for (final String word : words)
            if (isPalindrome(word))
                return word;
        return "";
    }

    private boolean isPalindrome(final String s) {
        int i = 0;
        int j = s.length() - 1;
        while (i < j)
            if (s.charAt(i++) != s.charAt(j--))
                return false;
        return true;
    }
}
```

1133. 2109.java

Path: LeetCode-main/solutions/2109. Adding Spaces to a String/2109.java

```
class Solution {
    public String addSpaces(String s, int[] spaces) {
        StringBuilder sb = new StringBuilder();
        int j = 0; // spaces' index

        for (int i = 0; i < s.length(); ++i) {
            if (j < spaces.length && i == spaces[j]) {
                sb.append(' ');
                ++j;
            }
            sb.append(s.charAt(i));
        }

        return sb.toString();
    }
}
```

1134. 211.java

Path: LeetCode-main/solutions/211. Add and Search Word - Data structure design/211.java

```
class TrieNode {
    public TrieNode[] children = new TrieNode[26];
    public boolean isWord = false;
}

class WordDictionary {
    public void addWord(final String word) {
        TrieNode node = root;
        for (final char c : word.toCharArray()) {
            final int i = c - 'a';
            if (node.children[i] == null)
                node.children[i] = new TrieNode();
            node = node.children[i];
        }
        node.isWord = true;
    }

    public boolean search(final String word) {
        return dfs(word, 0, root);
    }

    private TrieNode root = new TrieNode();

    private boolean dfs(final String word, int s, TrieNode node) {
        if (s == word.length())
            return node.isWord;
        if (word.charAt(s) != '.') {
            TrieNode next = node.children[word.charAt(s) - 'a'];
            return next == null ? false : dfs(word, s + 1, next);
        }

        // If word[s] == '.', then search all the 26 children.
        for (int i = 0; i < 26; ++i)
            if (node.children[i] != null && dfs(word, s + 1, node.children[i]))
                return true;

        return false;
    }
}
```

1135. 2110.java

Path: LeetCode-main/solutions/2110. Number of Smooth Descent Periods of a Stock/2110.java

```
class Solution {
    public long getDescentPeriods(int[] prices) {
        long ans = 1; // prices[0]
        int dp = 1;

        for (int i = 1; i < prices.size(); ++i) {
            if (prices[i] == prices[i - 1] - 1)
                ++dp;
            else
                dp = 1;
            ans += dp;
        }

        return ans;
    }
}
```

1136. 2111.java

Path: LeetCode-main/solutions/2111. Minimum Operations to Make the Array K-Increasing/2111.java

```
class Solution {
    public int kIncreasing(int[] arr, int k) {
        int ans = 0;

        for (int i = 0; i < k; ++i) {
            List<Integer> arr = new ArrayList<>();
            for (int j = i; j < arr.length; j += k)
                arr.add(arr[j]);
            ans += numReplaced(arr);
        }

        return ans;
    }

    private int numReplaced(List<Integer> arr) {
        List<Integer> tails = new ArrayList<>();
        for (final int a : arr)
            if (tails.isEmpty() || tails.get(tails.size() - 1) <= a)
                tails.add(a);
            else
                tails.set(firstGreater(tails, a), a);
        return arr.size() - tails.size();
    }

    private int firstGreater(List<Integer> arr, int target) {
        final int i = Collections.binarySearch(arr, target + 1);
        return i < 0 ? -i - 1 : i;
    }
}
```


1137. 2113.java

Path: LeetCode-main/solutions/2113. Elements in Array After Removing and Replacing Elements/2113.java

```
class Solution {
    public int[] elementInNums(int[] nums, int[][] queries) {
        int[] ans = new int[queries.length];

        for (int i = 0; i < queries.length; ++i) {
            final int time = queries[i][0];
            final int index = queries[i][1];
            ans[i] = f(nums, time % (2 * nums.length), index);
        }

        return ans;
    }

    private int f(int[] arr, int time, int index) {
        final int n = arr.length;
        if (time < n) { // [0, 1, 2] -> [1, 2] -> [2]
            index += time;
            return index >= n ? -1 : arr[index];
        } else { // [] -> [0] -> [0, 1]
            return index >= time - n ? -1 : arr[index];
        }
    }
}
```

1138. 2114.java

Path: LeetCode-main/solutions/2114. Maximum Number of Words Found in Sentences/2114.java

```
class Solution {  
    public int mostWordsFound(String[] sentences) {  
        return 1 + Stream.of(sentences)  
            .mapToInt(s -> (int) s.chars().filter(c -> c == ' ').count())  
            .max()  
            .getAsInt();  
    }  
}
```

1139. 2115.java

Path: LeetCode-main/solutions/2115. Find All Possible Recipes from Given Supplies/2115.java

```
class Solution {
    public List<String> findAllRecipes(String[] recipes, List<List<String>> ingredients,
                                     String[] supplies) {
        List<String> ans = new ArrayList<>();
        Set<String> suppliesSet = new HashSet<>();
        for (final String supply : supplies)
            suppliesSet.add(supply);
        Map<String, List<String>> graph = new HashMap<>();
        Map<String, Integer> inDegrees = new HashMap<>();

        // Build the graph.
        for (int i = 0; i < recipes.length; ++i)
            for (final String ingredient : ingredients.get(i))
                if (!suppliesSet.contains(ingredient)) {
                    graph.putIfAbsent(ingredient, new ArrayList<>());
                    graph.get(ingredient).add(recipes[i]);
                    inDegrees.merge(recipes[i], 1, Integer::sum);
                }

        // Perform topological sorting.
        Queue<String> q = Arrays.stream(recipes)
                                .filter(recipe -> inDegrees.getOrDefault(recipe, 0) == 0)
                                .collect(Collectors.toCollection(ArrayDeque::new));

        while (!q.isEmpty()) {
            final String u = q.poll();
            ans.add(u);
            if (!graph.containsKey(u))
                continue;
            for (final String v : graph.get(u)) {
                inDegrees.merge(v, -1, Integer::sum);
                if (inDegrees.get(v) == 0)
                    q.offer(v);
            }
        }

        return ans;
    }
}
```

1140. 2116.java

Path: LeetCode-main/solutions/2116. Check if a Parentheses String Can Be Valid/2116.java

```
class Solution {
    public boolean canBeValid(String s, String locked) {
        if (s.length() % 2 == 1)
            return false;

        return check(s, locked, true) && check(new StringBuilder(s).reverse().toString(),
                                                new StringBuilder(locked).reverse().toString(), false);
    }

    private boolean check(final String s, final String locked, boolean isForward) {
        int changeable = 0;
        int l = 0;
        int r = 0;

        for (int i = 0; i < s.length(); ++i) {
            final char c = s.charAt(i);
            final char lock = locked.charAt(i);
            if (lock == '0')
                ++changeable;
            else if (c == '(')
                ++l;
            else // c == ')'
                ++r;
            if (isForward && changeable + l - r < 0)
                return false;
            if (!isForward && changeable + r - l < 0)
                return false;
        }

        return true;
    }
}
```

1141. 2117.java

Path: LeetCode-main/solutions/2117. Abbreviating the Product of a Range/2117.java

```
class Solution {
    public String abbreviateProduct(int left, int right) {
        final long maxSuf = 100_000_000_000L;
        double prod = 1.0;
        long suf = 1;
        int countDigits = 0;
        int countZeros = 0;

        for (int num = left; num <= right; ++num) {
            prod *= num;
            while (prod >= 1.0) {
                prod /= 10;
                ++countDigits;
            }
            suf *= num;
            while (suf % 10 == 0) {
                suf /= 10;
                ++countZeros;
            }
            if (suf > maxSuf)
                suf %= maxSuf;
        }

        if (countDigits - countZeros <= 10) {
            final long tens = (long) Math.pow(10, countDigits - countZeros);
            return String.valueOf((long) (prod * tens + 0.5)) + "e" + String.valueOf(countZeros);
        }

        final String pre = String.valueOf((long) (prod * Math.pow(10, 5)));
        String sufStr = String.valueOf(suf);
        sufStr = sufStr.substring(sufStr.length() - 5);
        return pre + "..." + sufStr + "e" + String.valueOf(countZeros);
    }
}
```

1142. 2119-2.java

Path: LeetCode-main/solutions/2119. A Number After a Double Reversal/2119-2.java

```
class Solution {  
    public boolean isSameAfterReversals(int num) {  
        return num == 0 || num % 10 > 0;  
    }  
}
```

1143. 2119.java

Path: LeetCode-main/solutions/2119. A Number After a Double Reversal/2119.java

```
class Solution {
    public boolean isSameAfterReversals(int num) {
        final int reversed1 = getReversed(num);
        final int reversed2 = getReversed(reversed1);
        return reversed2 == num;
    }

    private int getReversed(int num) {
        int reversed = 0;
        while (num > 0) {
            reversed = reversed * 10 + num % 10;
            num /= 10;
        }
        return reversed;
    }
}
```

1144. 212.java

Path: LeetCode-main/solutions/212. Word Search II/212.java

```
class TrieNode {
    public TrieNode[] children = new TrieNode[26];
    public String word;
}

class Solution {
    public List<String> findWords(char[][] board, String[] words) {
        for (final String word : words)
            insert(word);

        List<String> ans = new ArrayList<>();

        for (int i = 0; i < board.length; ++i)
            for (int j = 0; j < board[0].length; ++j)
                dfs(board, i, j, root, ans);

        return ans;
    }

    private TrieNode root = new TrieNode();

    private void insert(final String word) {
        TrieNode node = root;
        for (final char c : word.toCharArray()) {
            final int i = c - 'a';
            if (node.children[i] == null)
                node.children[i] = new TrieNode();
            node = node.children[i];
        }
        node.word = word;
    }

    private void dfs(char[][] board, int i, int j, TrieNode node, List<String> ans) {
        if (i < 0 || i == board.length || j < 0 || j == board[0].length)
            return;
        if (board[i][j] == '*')
            return;

        final char c = board[i][j];
        TrieNode child = node.children[c - 'a'];
        if (child == null)
            return;
        if (child.word != null) {
            ans.add(child.word);
            child.word = null;
        }

        board[i][j] = '*';
        dfs(board, i + 1, j, child, ans);
        dfs(board, i - 1, j, child, ans);
        dfs(board, i, j + 1, child, ans);
        dfs(board, i, j - 1, child, ans);
        board[i][j] = c;
    }
}
```


1145. 2120.java

Path: LeetCode-main/solutions/2120. Execution of All Suffix Instructions Staying in a Grid/2120.java

```
class Solution {
    public int[] executeInstructions(int n, int[] startPos, String s) {
        final int m = s.length();
        final int uMost = startPos[0] + 1;
        final int dMost = n - startPos[0];
        final int lMost = startPos[1] + 1;
        final int rMost = n - startPos[1];
        Map<Character, Pair<Integer, Integer>> moves = new HashMap<>();
        moves.put('L', new Pair<>(0, -1));
        moves.put('R', new Pair<>(0, 1));
        moves.put('U', new Pair<>(-1, 0));
        moves.put('D', new Pair<>(1, 0));

        int[] ans = new int[m];
        Map<Integer, Integer> reachX = new HashMap<>();
        Map<Integer, Integer> reachY = new HashMap<>();
        reachX.put(0, m);
        reachY.put(0, m);
        int x = 0;
        int y = 0;

        for (int i = m - 1; i >= 0; --i) {
            Pair<Integer, Integer> pair = moves.get(s.charAt(i));
            final int dx = pair.getKey();
            final int dy = pair.getValue();
            x -= dx;
            y -= dy;
            reachX.put(x, i);
            reachY.put(y, i);
            final int out = Math.min(Math.min(reachX.getDefault(x - uMost, Integer.MAX_VALUE),
                                                reachX.getDefault(x + dMost, Integer.MAX_VALUE)),
                                    Math.min(reachY.getDefault(y - lMost, Integer.MAX_VALUE),
                                                reachY.getDefault(y + rMost, Integer.MAX_VALUE)));
            ans[i] = out == Integer.MAX_VALUE ? m - i : out - i - 1;
        }

        return ans;
    }
}
```

1146. 2121.java

Path: LeetCode-main/solutions/2121. Intervals Between Identical Elements/2121.java

```
class Solution {
    public long[] getDistances(int[] arr) {
        final int n = arr.length;
        long[] ans = new long[n];
        long[] prefix = new long[n];
        long[] suffix = new long[n];
        Map<Integer, List<Integer>> numToIndices = new HashMap<>();

        for (int i = 0; i < n; ++i) {
            numToIndices.putIfAbsent(arr[i], new ArrayList<>());
            numToIndices.get(arr[i]).add(i);
        }

        for (List<Integer> indices : numToIndices.values()) {
            for (int i = 1; i < indices.size(); ++i) {
                final int currIndex = indices.get(i);
                final int prevIndex = indices.get(i - 1);
                prefix[currIndex] += prefix[prevIndex] + i * (currIndex - prevIndex);
            }
            for (int i = indices.size() - 2; i >= 0; --i) {
                final int currIndex = indices.get(i);
                final int prevIndex = indices.get(i + 1);
                suffix[currIndex] += suffix[prevIndex] + (indices.size() - i - 1) * (prevIndex - currIndex);
            }
        }

        for (int i = 0; i < n; ++i)
            ans[i] = prefix[i] + suffix[i];

        return ans;
    }
}
```

1147. 2122.java

Path: LeetCode-main/solutions/2122. Recover the Original Array/2122.java

```
class Solution {
    public int[] recoverArray(int[] nums) {
        final int n = nums.length;
        Map<Integer, Integer> count = new HashMap<>();

        for (final int num : nums)
            count.merge(num, 1, Integer::sum);

        Arrays.sort(nums);

        for (int i = 1; i < n; ++i) {
            final int x = nums[i] - nums[0]; // 2 * k
            if (x <= 0 || x % 2 == 1)
                continue;
            Map<Integer, Integer> countCopy = new HashMap<>();
            countCopy.putAll(count);
            int[] arr = getArray(nums, x, countCopy);
            if (arr.length == n / 2)
                return arr;
        }

        throw new IllegalArgumentException();
    }

    private int[] getArray(int[] nums, int x, Map<Integer, Integer> count) {
        List<Integer> arr = new ArrayList<>();
        for (final int num : nums) {
            if (count.getOrDefault(num, 0) == 0)
                continue;
            if (count.getOrDefault(num + x, 0) == 0)
                return new int[] {};
            count.merge(num, -1, Integer::sum);
            count.merge(num + x, -1, Integer::sum);
            arr.add(num + x / 2);
        }
        return arr.stream().mapToInt(Integer::intValue).toArray();
    }
}
```

1148. 2123.java

Path: LeetCode-main/solutions/2123. Minimum Operations to Remove Adjacent Ones in Matrix/2123.java

```
class Solution {
    public int minimumOperations(int[][] grid) {
        final int m = grid.length;
        final int n = grid[0].length;
        int ans = 0;
        int[][] seen = new int[m][n];
        int[][] match = new int[m][n];
        Arrays.stream(match).forEach(A -> Arrays.fill(A, -1));

        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j)
                if (grid[i][j] == 1 && match[i][j] == -1) {
                    final int sessionId = i * n + j;
                    seen[i][j] = sessionId;
                    if (dfs(grid, i, j, sessionId, seen, match))
                        ++ans;
                }

        return ans;
    }

    private static final int[][] DIRS = {{0, 1}, {1, 0}, {0, -1}, {-1, 0}};

    private boolean dfs(final int[][] grid, int i, int j, int sessionId, int[][] seen,
                        int[][] match) {
        final int m = grid.length;
        final int n = grid[0].length;

        for (int[] dir : DIRS) {
            final int x = i + dir[0];
            final int y = j + dir[1];
            if (x < 0 || x == m || y < 0 || y == n)
                continue;
            if (grid[x][y] == 0 || seen[x][y] == sessionId)
                continue;
            seen[x][y] = sessionId;
            if (match[x][y] == -1 ||
                dfs(grid, match[x][y] / n, match[x][y] % n, sessionId, seen, match)) {
                match[x][y] = i * n + j;
                match[i][j] = x * n + y;
                return true;
            }
        }

        return false;
    }
}
```

1149. 2124.java

Path: LeetCode-main/solutions/2124. Check if All A's Appears Before All B's/2124.java

```
class Solution {  
    public boolean checkString(String s) {  
        return !s.contains("ba");  
    }  
}
```

1150. 2125.java

Path: LeetCode-main/solutions/2125. Number of Laser Beams in a Bank/2125.java

```
class Solution {
    public int numberOfBeams(String[] bank) {
        int ans = 0;
        int prevOnes = 0;

        for (final String row : bank) {
            final int ones = (int) row.chars().filter(c -> c == '1').count();
            if (ones > 0) {
                ans += prevOnes * ones;
                prevOnes = ones;
            }
        }

        return ans;
    }
}
```

1151. 2126.java

Path: LeetCode-main/solutions/2126. Destroying Asteroids/2126.java

```
class Solution {
    public boolean asteroidsDestroyed(int mass, int[] asteroids) {
        Arrays.sort(asteroids);

        long m = mass;

        for (final int asteroid : asteroids)
            if (m >= asteroid)
                m += asteroid;
            else
                return false;

        return true;
    }
}
```

1152. 2127.java

Path: LeetCode-main/solutions/2127. Maximum Employees to Be Invited to a Meeting/2127.java

```
enum State { INIT, VISITING, VISITED }

class Solution {
    public int maximumInvitations(int[] favorite) {
        final int n = favorite.length;
        int sumComponentsLength = 0; // the component: a -> b -> c <-> x <- y
        List<Integer>[] graph = new List[n];
        int[] inDegrees = new int[n];
        int[] maxChainLength = new int[n];
        Arrays.fill(maxChainLength, 1);
        Arrays.setAll(graph, i -> new ArrayList<>());

        // Build the graph.
        for (int i = 0; i < n; ++i) {
            graph[i].add(favorite[i]);
            ++inDegrees[favorite[i]];
        }

        // Perform topological sorting.
        Queue<Integer> q = IntStream.range(0, n)
            .filter(i -> inDegrees[i] == 0)
            .boxed()
            .collect(Collectors.toCollection(ArrayDeque::new));

        while (!q.isEmpty()) {
            final int u = q.poll();
            for (final int v : graph[u]) {
                if (--inDegrees[v] == 0)
                    q.offer(v);
                maxChainLength[v] = Math.max(maxChainLength[v], 1 + maxChainLength[u]);
            }
        }

        for (int i = 0; i < n; ++i)
            if (favorite[favorite[i]] == i)
                // i <-> favorite[i] (the cycle's length = 2)
                sumComponentsLength += maxChainLength[i] + maxChainLength[favorite[i]];

        int[] parent = new int[n];
        Arrays.fill(parent, -1);
        boolean[] seen = new boolean[n];
        State[] states = new State[n];

        for (int i = 0; i < n; ++i)
            if (!seen[i])
                findCycle(graph, i, parent, seen, states);

        return Math.max(sumComponentsLength / 2, maxCycleLength);
    }

    private int maxCycleLength = 0; // the cycle : a -> b -> c -> a

    private void findCycle(List<Integer>[] graph, int u, int[] parent, boolean[] seen,
        State[] states) {
        seen[u] = true;
        states[u] = State.VISITING;

        for (final int v : graph[u]) {
            if (!seen[v]) {
                parent[v] = u;
                findCycle(graph, v, parent, seen, states);
            } else if (states[v] == State.VISITING) {
                // Find the cycle's length.
                int curr = u;
                int cycleLength = 1;
                while (curr != v) {
                    curr = parent[curr];
                    ++cycleLength;
                }
                maxCycleLength = Math.max(maxCycleLength, cycleLength);
            }
        }

        states[u] = State.VISITED;
    }
}
```


1153. 2128.java

Path: LeetCode-main/solutions/2128. Remove All Ones With Row and Column Flips/2128.java

```
class Solution {
    public boolean removeOnes(int[][] grid) {
        int[] revRow = getRevRow(grid[0]);
        return Arrays.stream(grid).allMatch(
            row -> Arrays.equals(row, grid[0]) || Arrays.equals(row, revRow));
    }

    private int[] getRevRow(int[] row) {
        int[] revRow = new int[row.length];
        for (int i = 0; i < row.length; ++i)
            revRow[i] = row[i] ^ 1;
        return revRow;
    }
}
```

1154. 2129.java

Path: LeetCode-main/solutions/2129. Capitalize the Title/2129.java

```
class Solution {
    public String capitalizeTitle(String title) {
        StringBuilder sb = new StringBuilder(title.toLowerCase());

        int i = 0; // Point to the start of a word.
        int j = 0; // Point to the end of a word.

        while (j < sb.length()) {
            while (j < sb.length() && sb.charAt(j) != ' ')
                ++j;
            if (j - i > 2)
                sb.setCharAt(i, Character.toUpperCase(sb.charAt(i)));
            i = j + 1;
            ++j; // Skip the spaces.
        }

        return sb.toString();
    }
}
```

1155. 213.java

Path: LeetCode-main/solutions/213. House Robber II/213.java

```
class Solution {
    public int rob(int[] nums) {
        if (nums.length == 0)
            return 0;
        if (nums.length == 1)
            return nums[0];
        return Math.max(rob(nums, 0, nums.length - 2), rob(nums, 1, nums.length - 1));
    }

    private int rob(int[] nums, int l, int r) {
        int prev1 = 0; // dp[i - 1]
        int prev2 = 0; // dp[i - 2]

        for (int i = l; i <= r; ++i) {
            final int dp = Math.max(prev1, prev2 + nums[i]);
            prev2 = prev1;
            prev1 = dp;
        }

        return prev1;
    }
}
```

1156. 2130.java

Path: LeetCode-main/solutions/2130. Maximum Twin Sum of a Linked List/2130.java

```
class Solution {
    public int pairSum(ListNode head) {
        int ans = 0;
        ListNode slow = head;
        ListNode fast = head;

        // `slow` points to the start of the second half.
        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
        }

        // `tail` points to the end of the reversed second half.
        ListNode tail = reverseList(slow);

        while (tail != null) {
            ans = Math.max(ans, head.val + tail.val);
            head = head.next;
            tail = tail.next;
        }

        return ans;
    }

    private ListNode reverseList(ListNode head) {
        ListNode prev = null;
        while (head != null) {
            ListNode next = head.next;
            head.next = prev;
            prev = head;
            head = next;
        }
        return prev;
    }
}
```

1157. 2131.java

Path: LeetCode-main/solutions/2131. Longest Palindrome by Concatenating Two Letter Words/2131.java

```
class Solution {
    public int longestPalindrome(String[] words) {
        int ans = 0;
        int[][] count = new int[26][26];

        for (final String word : words) {
            final int i = word.charAt(0) - 'a';
            final int j = word.charAt(1) - 'a';
            if (count[j][i] > 0) {
                ans += 4;
                --count[j][i];
            } else {
                ++count[i][j];
            }
        }

        for (int i = 0; i < 26; ++i)
            if (count[i][i] > 0)
                return ans + 2;

        return ans;
    }
}
```

1158. 2132.java

Path: LeetCode-main/solutions/2132. Stamping the Grid/2132.java

```
class Solution {
    public boolean possibleToStamp(int[][] grid, int stampHeight, int stampWidth) {
        final int m = grid.length;
        final int n = grid[0].length;
        // A[i][j] := the number of 1s in grid[0..i][0..j]
        int[][] A = new int[m + 1][n + 1];
        // B[i][j] := the number of ways to stamp the submatrix in [0..i][0..j]
        int[][] B = new int[m + 1][n + 1];
        // fit[i][j] := 1 if the stamps can fit with the right-bottom at (i, j)
        int[][] fit = new int[m][n];

        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j) {
                A[i + 1][j + 1] = A[i + 1][j] + A[i][j + 1] - A[i][j] + grid[i][j];
                if (i + 1 >= stampHeight && j + 1 >= stampWidth) {
                    final int x = i - stampHeight + 1;
                    final int y = j - stampWidth + 1;
                    if (A[i + 1][j + 1] - A[x][j + 1] - A[i + 1][y] + A[x][y] == 0)
                        fit[i][j] = 1;
                }
            }

        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j)
                B[i + 1][j + 1] = B[i + 1][j] + B[i][j + 1] - B[i][j] + fit[i][j];

        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j)
                if (grid[i][j] == 0) {
                    final int x = Math.min(i + stampHeight, m);
                    final int y = Math.min(j + stampWidth, n);
                    if (B[x][y] - B[i][y] - B[x][j] + B[i][j] == 0)
                        return false;
                }

        return true;
    }
}
```

1159. 2133.java

Path: LeetCode-main/solutions/2133. Check if Every Row and Column Contains All Numbers/2133.java

```
class Solution {
    public boolean checkValid(int[][] matrix) {
        final int n = matrix.length;

        for (int i = 0; i < n; ++i) {
            Set<Integer> row = new HashSet<>();
            Set<Integer> col = new HashSet<>();
            for (int j = 0; j < n; ++j) {
                row.add(matrix[i][j]);
                col.add(matrix[j][i]);
            }
            if (Math.min(row.size(), col.size()) < n)
                return false;
        }
        return true;
    }
}
```

1160. 2134.java

Path: LeetCode-main/solutions/2134. Minimum Swaps to Group All 1's Together II/2134.java

```
class Solution {
    public int minSwaps(int[] nums) {
        final int n = nums.length;
        final int k = (int) Arrays.stream(nums).filter(a -> a == 1).count();
        int ones = 0;    // the number of ones in the window
        int maxOnes = 0; // the maximum number of ones in the window

        for (int i = 0; i < n * 2; ++i) {
            if (i >= k && nums[(i - k) % n] == 1)
                --ones;
            if (nums[i % n] == 1)
                ++ones;
            maxOnes = Math.max(maxOnes, ones);
        }
        return k - maxOnes;
    }
}
```


1161. 2135.java

Path: LeetCode-main/solutions/2135. Count Words Obtained After Adding a Letter/2135.java

```
class Solution {
    public int wordCount(String[] startWords, String[] targetWords) {
        int ans = 0;
        Set<Integer> seen = Arrays.stream(startWords).map(this::getMask).collect(Collectors.toSet());

        for (final String w : targetWords) {
            final int mask = getMask(w);
            for (final char c : w.toCharArray())
                // Toggle one character.
                if (seen.contains(mask ^ 1 << c - 'a')) {
                    ++ans;
                    break;
                }
        }

        return ans;
    }

    private int getMask(final String s) {
        int mask = 0;
        for (final char c : s.toCharArray())
            mask ^= 1 << c - 'a';
        return mask;
    }
}
```

1162. 2136.java

Path: LeetCode-main/solutions/2136. Earliest Possible Day of Full Bloom/2136.java

```
class Solution {
    public int earliestFullBloom(int[] plantTime, int[] growTime) {
        record Seed(int p, int g) {}
        final int n = plantTime.length;
        int ans = 0;
        int time = 0;
        Seed[] seeds = new Seed[n];

        for (int i = 0; i < plantTime.length; ++i)
            seeds[i] = new Seed(plantTime[i], growTime[i]);

        Arrays.sort(seeds, Comparator.comparingInt(Seed::g).reversed());

        for (Seed seed : seeds) {
            time += seed.p;
            ans = Math.max(ans, time + seed.g);
        }

        return ans;
    }
}
```

1163. 2137.java

Path: LeetCode-main/solutions/2137. Pour Water Between Buckets to Make Water Levels Equal/2137.java

```
class Solution {
    public double equalizeWater(int[] buckets, int loss) {
        final double ERR = 1e-5;
        final double PERCENTAGE = (100 - loss) / (double) 100;
        double l = 0.0;
        double r = Arrays.stream(buckets).max().getAsInt();

        while (r - l > ERR) {
            final double m = (l + r) / 2;
            if (canFill(buckets, m, PERCENTAGE))
                l = m;
            else
                r = m;
        }

        return l;
    }

    private boolean canFill(int[] buckets, double target, double PERCENTAGE) {
        double extra = 0.0;
        double need = 0.0;
        for (final int bucket : buckets)
            if (bucket > target)
                extra += bucket - target;
            else
                need += target - bucket;
        return extra * PERCENTAGE >= need;
    }
}
```

1164. 2138.java

Path: LeetCode-main/solutions/2138. Divide a String Into Groups of Size k/2138.java

```
class Solution {
    public String[] divideString(String s, int k, char fill) {
        String[] ans = new String[(s.length() + k - 1) / k];

        for (int i = 0, j = 0; i < s.length(); i += k)
            ans[j++] = i + k > s.length()
                ? s.substring(i) + String.valueOf(fill).repeat(i + k - s.length())
                : s.substring(i, i + k);

        return ans;
    }
}
```

1165. 2139.java

Path: LeetCode-main/solutions/2139. Minimum Moves to Reach Target Score/2139.java

```
class Solution {
    public int minMoves(int target, int maxDoubles) {
        int steps = 0;

        while (target > 1 && maxDoubles > 0) {
            if (target % 2 == 1) {
                --target;
            } else {
                target /= 2;
                --maxDoubles;
            }
            ++steps;
        }
        return steps + target - 1;
    }
}
```

1166. 214.java

Path: LeetCode-main/solutions/214. Shortest Palindrome/214.java

```
class Solution {  
    public String shortestPalindrome(String s) {  
        final String t = new StringBuilder(s).reverse().toString();  
  
        for (int i = 0; i < t.length(); ++i)  
            if (s.startsWith(t.substring(i)))  
                return t.substring(0, i) + s;  
  
        return t + s;  
    }  
}
```

1167. 2140.java

Path: LeetCode-main/solutions/2140. Solving Questions With Brainpower/2140.java

```
class Solution {
    public long mostPoints(int[][] questions) {
        final int n = questions.length;
        // dp[i] := the maximum points starting from questions[i]
        long[] dp = new long[n + 1];

        for (int i = n - 1; i >= 0; --i) {
            final int points = questions[i][0];
            final int brainpower = questions[i][1];
            final int nextIndex = i + brainpower + 1;
            final long nextPoints = nextIndex < n ? dp[nextIndex] : 0;
            dp[i] = Math.max(points + nextPoints, dp[i + 1]);
        }

        return dp[0];
    }
}
```

1168. 2141.java

Path: LeetCode-main/solutions/2141. Maximum Running Time of N Computers/2141.java

```
class Solution {
    public long maxRunTime(int n, int[] batteries) {
        long sum = Arrays.stream(batteries).asLongStream().sum();

        Arrays.sort(batteries);

        // The maximum battery is greater than the average, so it can last forever.
        // Reduce the problem from size n to size n - 1.
        int i = batteries.length - 1;
        while (batteries[i] > sum / n) {
            sum -= batteries[i--];
            --n;
        }

        // If the maximum battery <= average running time, it won't be waste, and so
        // do smaller batteries.
        return sum / n;
    }
}
```


1169. 2143.java

Path: LeetCode-main/solutions/2143. Choose Numbers From Two Arrays in Range/2143.java

```
class Solution {
    public int countSubranges(int[] nums1, int[] nums2) {
        final int MOD = 1_000_000_007;
        int ans = 0;
        // {sum, count}, add if choose from nums1, minus if choose from nums2
        Map<Integer, Integer> dp = new HashMap<>();

        for (int i = 0; i < nums1.length; ++i) {
            Map<Integer, Integer> newDp = new HashMap<>();
            // edge case: nums1[i] == nums2[i] == 0, so can't put them in the
            // initializer list.
            newDp.merge(nums1[i], 1, Integer::sum);
            newDp.merge(-nums2[i], 1, Integer::sum);

            for (final int prevSum : dp.keySet()) {
                final int count = dp.get(prevSum);
                final int chooseNums1 = prevSum + nums1[i];
                newDp.put(chooseNums1, (newDp.getOrDefault(chooseNums1, 0) + count) % MOD);
                final int chooseNums2 = prevSum - nums2[i];
                newDp.put(chooseNums2, (newDp.getOrDefault(chooseNums2, 0) + count) % MOD);
            }

            dp = newDp;
            if (dp.containsKey(0)) {
                ans += dp.get(0);
                ans %= MOD;
            }
        }

        return ans;
    }
}
```

1170. 2144.java

Path: LeetCode-main/solutions/2144. Minimum Cost of Buying Candies With Discount/2144.java

```
class Solution {
    public int minimumCost(int[] cost) {
        int ans = 0;

        cost = Arrays.stream(cost)
            .boxed()
            .sorted(Collections.reverseOrder())
            .mapToInt(Integer::intValue)
            .toArray();

        for (int i = 0; i < cost.length; ++i)
            if (i % 3 != 2)
                ans += cost[i];

        return ans;
    }
}
```

1171. 2145-2.java

Path: LeetCode-main/solutions/2145. Count the Hidden Sequences/2145-2.java

```
class Solution {
    public int numberOfArrays(int[] differences, int lower, int upper) {
        long prefix = 0;
        long mn = 0; // Starts from 0.
        long mx = 0; // Starts from 0.

        for (final int d : differences) {
            prefix += d;
            mn = Math.min(mn, prefix);
            mx = Math.max(mx, prefix);
        }

        return (int) Math.max(0L, (upper - lower) - (mx - mn) + 1);
    }
}
```

1172. 2145.java

Path: LeetCode-main/solutions/2145. Count the Hidden Sequences/2145.java

```
class Solution {
    public int numberOfArrays(int[] differences, int lower, int upper) {
        // Starts from 0, so prefix[0] = 0.
        // Changing prefix[0] to any other number doesn't affect the answer.
        long[] prefix = new long[differences.length + 1];

        for (int i = 0; i < differences.length; ++i)
            prefix[i + 1] += prefix[i] + differences[i];

        final long mx = Arrays.stream(prefix).max().getAsLong();
        final long mn = Arrays.stream(prefix).min().getAsLong();
        return (int) Math.max(0L, (upper - lower) - (mx - mn) + 1);
    }
}
```

1173. 2146.java

Path: LeetCode-main/solutions/2146. K Highest Ranked Items Within a Price Range/2146.java

```
class Solution {
    public List<List<Integer>> highestRankedKItems(int[][] grid, int[] pricing, int[] start, int k) {
        final int[][] DIRS = {{0, 1}, {1, 0}, {0, -1}, {-1, 0}};
        final int m = grid.length;
        final int n = grid[0].length;
        final int low = pricing[0];
        final int high = pricing[1];
        final int row = start[0];
        final int col = start[1];
        List<List<Integer>> ans = new ArrayList<>();

        if (low <= grid[row][col] && grid[row][col] <= high) {
            ans.add(Arrays.asList(row, col));
            if (k == 1)
                return ans;
        }

        Queue<Pair<Integer, Integer>> q = new ArrayDeque<>(List.of(new Pair<>(row, col)));
        boolean[][] seen = new boolean[m][n];
        seen[row][col] = true; // Mark as visited.

        while (!q.isEmpty()) {
            List<List<Integer>> neighbors = new ArrayList<>();
            for (int sz = q.size(); sz > 0; --sz) {
                final int i = q.peek().getKey();
                final int j = q.poll().getValue();
                for (int[] dir : DIRS) {
                    final int x = i + dir[0];
                    final int y = j + dir[1];
                    if (x < 0 || x == m || y < 0 || y == n)
                        continue;
                    if (grid[x][y] == 0 || seen[x][y])
                        continue;
                    if (low <= grid[x][y] && grid[x][y] <= high)
                        neighbors.add(Arrays.asList(x, y));
                    q.offer(new Pair<>(x, y));
                    seen[x][y] = true;
                }
            }
            Collections.sort(neighbors, new Comparator<List<Integer>>() {
                @Override
                public int compare(List<Integer> a, List<Integer> b) {
                    final int x1 = a.get(0);
                    final int y1 = a.get(1);
                    final int x2 = b.get(0);
                    final int y2 = b.get(1);
                    if (grid[x1][y1] != grid[x2][y2])
                        return grid[x1][y1] - grid[x2][y2];
                    return x1 == x2 ? y1 - y2 : x1 - x2;
                }
            });
            for (List<Integer> neighbor : neighbors) {
                if (ans.size() < k)
                    ans.add(neighbor);
                if (ans.size() == k)
                    return ans;
            }
        }

        return ans;
    }
}
```

1174. 2147.java

Path: LeetCode-main/solutions/2147. Number of Ways to Divide a Long Corridor/2147.java

```
class Solution {
    public int numberOfWays(String corridor) {
        final int MOD = 1_000_000_007;
        long ans = 1;
        int prevSeat = -1;
        int numSeats = 0;

        for (int i = 0; i < corridor.length(); ++i) {
            if (corridor.charAt(i) == 'S') {
                if (++numSeats > 2 && numSeats % 2 == 1)
                    ans = ans * (i - prevSeat) % MOD;
                prevSeat = i;
            }
        }

        return numSeats > 1 && numSeats % 2 == 0 ? (int) ans : 0;
    }
}
```

1175. 2148.java

Path: LeetCode-main/solutions/2148. Count Elements With Strictly Smaller and Greater Elements/2148.java

```
class Solution {  
    public int countElements(int[] nums) {  
        final int mn = Arrays.stream(nums).min().getAsInt();  
        final int mx = Arrays.stream(nums).max().getAsInt();  
        return (int) Arrays.stream(nums).filter(num -> mn < num && num < mx).count();  
    }  
}
```

1176. 2149.java

Path: LeetCode-main/solutions/2149. Rearrange Array Elements by Sign/2149.java

```
class Solution {
    public int[] rearrangeArray(int[] nums) {
        int[] ans = new int[nums.length];
        List<Integer> pos = new ArrayList<>();
        List<Integer> neg = new ArrayList<>();

        for (final int num : nums)
            (num > 0 ? pos : neg).add(num);

        for (int i = 0; i < pos.size(); ++i) {
            ans[i * 2] = pos.get(i);
            ans[i * 2 + 1] = neg.get(i);
        }

        return ans;
    }
}
```


1177. 215-2.java

Path: LeetCode-main/solutions/215. Kth Largest Element in an Array/215-2.java

```
class Solution {
    public int findKthLargest(int[] nums, int k) {
        return quickSelect(nums, 0, nums.length - 1, k);
    }

    private int quickSelect(int[] nums, int l, int r, int k) {
        final int pivot = nums[r];

        int nextSwapped = l;
        for (int i = l; i < r; ++i)
            if (nums[i] >= pivot)
                swap(nums, nextSwapped++, i);
        swap(nums, nextSwapped, r);

        final int count = nextSwapped - l + 1; // the number of `nums` >= pivot
        if (count == k)
            return nums[nextSwapped];
        if (count > k)
            return quickSelect(nums, l, nextSwapped - 1, k);
        return quickSelect(nums, nextSwapped + 1, r, k - count);
    }

    private void swap(int[] nums, int i, int j) {
        final int temp = nums[i];
        nums[i] = nums[j];
        nums[j] = temp;
    }
}
```

1178. 215-3.java

Path: LeetCode-main/solutions/215. Kth Largest Element in an Array/215-3.java

```
class Solution {
    public int findKthLargest(int[] nums, int k) {
        return quickSelect(nums, 0, nums.length - 1, k);
    }

    private int quickSelect(int[] nums, int l, int r, int k) {
        final int randIndex = new Random().nextInt(r - l + 1) + l;
        swap(nums, randIndex, r);
        final int pivot = nums[r];

        int nextSwapped = l;
        for (int i = l; i < r; ++i)
            if (nums[i] >= pivot)
                swap(nums, nextSwapped++, i);
        swap(nums, nextSwapped, r);

        final int count = nextSwapped - l + 1; // the number of `nums` >= pivot
        if (count == k)
            return nums[nextSwapped];
        if (count > k)
            return quickSelect(nums, l, nextSwapped - 1, k);
        return quickSelect(nums, nextSwapped + 1, r, k - count);
    }

    private void swap(int[] nums, int i, int j) {
        final int temp = nums[i];
        nums[i] = nums[j];
        nums[j] = temp;
    }
}
```

1179. 215.java

Path: LeetCode-main/solutions/215. Kth Largest Element in an Array/215.java

```
class Solution {
    public int findKthLargest(int[] nums, int k) {
        Queue<Integer> minHeap = new PriorityQueue<>();

        for (final int num : nums) {
            minHeap.offer(num);
            while (minHeap.size() > k)
                minHeap.poll();
        }

        return minHeap.peek();
    }
}
```

1180. 2150.java

Path: LeetCode-main/solutions/2150. Find All Lonely Numbers in the Array/2150.java

```
class Solution {
    public List<Integer> findLonely(int[] nums) {
        List<Integer> ans = new ArrayList<>();
        Map<Integer, Integer> count = new HashMap<>();

        for (final int num : nums)
            count.merge(num, 1, Integer::sum);

        for (final int num : count.keySet())
            if (count.get(num) == 1 && !count.containsKey(num - 1) && !count.containsKey(num + 1))
                ans.add(num);

        return ans;
    }
}
```

1181. 2151-2.java

Path: LeetCode-main/solutions/2151. Maximum Good People Based on Statements/2151-2.java

```
class Solution {
    public int maximumGood(int[][] statements) {
        final int maxMask = 1 << statements.length;
        int ans = 0;

        for (int mask = 0; mask < maxMask; ++mask)
            if (isValid(statements, mask))
                ans = Math.max(ans, Integer.bitCount(mask));

        return ans;
    }

    private boolean isValid(int[][] statements, int mask) {
        for (int i = 0; i < statements.length; ++i) {
            if ((mask >> i & 1) == 0) // The i-th person is bad, so no need to check.
                continue;
            for (int j = 0; j < statements.length; ++j) {
                if (statements[i][j] == 2)
                    continue;
                if (statements[i][j] != (mask >> j & 1))
                    return false;
            }
        }
        return true;
    }
}
```

1182. 2151.java

Path: LeetCode-main/solutions/2151. Maximum Good People Based on Statements/2151.java

```
class Solution {
    public int maximumGood(int[][] statements) {
        dfs(statements, new ArrayList<>(), 0, 0);
        return ans;
    }

    private int ans = 0;

    private void dfs(int[][] statements, List<Integer> good, int i, int count) {
        if (i == statements.length) {
            if (isValid(statements, good))
                ans = Math.max(ans, count);
            return;
        }

        good.add(0); // Assume the i-th person is bad.
        dfs(statements, good, i + 1, count);
        good.set(good.size() - 1, 1); // Assume the i-th person is good.
        dfs(statements, good, i + 1, count + 1);
        good.remove(good.size() - 1);
    }

    private boolean isValid(int[][] statements, List<Integer> good) {
        for (int i = 0; i < good.size(); ++i) {
            if (good.get(i) == 0) // The i-th person is bad, so no need to check.
                continue;
            for (int j = 0; j < statements.length; ++j) {
                if (statements[i][j] == 2)
                    continue;
                if (statements[i][j] != good.get(j))
                    return false;
            }
        }
        return true;
    }
}
```

1183. 2152.java

Path: LeetCode-main/solutions/2152. Minimum Number of Lines to Cover Points/2152.java

```
class Solution {
    public int minimumLines(int[][] points) {
        final int allCovered = (1 << points.length) - 1;
        int[] mem = new int[allCovered];
        Arrays.fill(mem, -1);
        return dfs(points, 0, allCovered, mem);
    }

    private int dfs(int[][] points, int covered, int allCovered, int[] mem) {
        if (covered == allCovered)
            return 0;
        if (mem[covered] != -1)
            return mem[covered];

        final int n = points.length;
        int ans = n / 2 + ((n & 1) == 1 ? 1 : 0);

        for (int i = 0; i < n; ++i) {
            if ((covered >> i & 1) == 1)
                continue;
            for (int j = 0; j < n; ++j) {
                if (i == j)
                    continue;
                // Connect the points[i] with the points[j].
                int newCovered = covered | 1 << i | 1 << j;
                // Mark the points covered by this line.
                Pair<Integer, Integer> slope = getSlope(points[i], points[j]);
                for (int k = 0; k < n; ++k)
                    if (getSlope(points[i], points[k]).equals(slope))
                        newCovered |= 1 << k;
                ans = Math.min(ans, 1 + dfs(points, newCovered, allCovered, mem));
            }
        }

        return mem[covered] = ans;
    }

    private Pair<Integer, Integer> getSlope(int[] p, int[] q) {
        final int dx = p[0] - q[0];
        final int dy = p[1] - q[1];
        if (dx == 0)
            return new Pair<>(0, p[0]);
        if (dy == 0)
            return new Pair<>(p[1], 0);
        final int d = gcd(dx, dy);
        final int x = dx / d;
        final int y = dy / d;
        return x > 0 ? new Pair<>(x, y) : new Pair<>(-x, -y);
    }

    private int gcd(int a, int b) {
        return b == 0 ? a : gcd(b, a % b);
    }
}
```

1184. 2155-2.java

Path: LeetCode-main/solutions/2154. Keep Multiplying Found Values by Two/2155-2.java

```
class Solution {
    public int findFinalValue(int[] nums, int original) {
        boolean[] seen = new boolean[1001];

        for (final int num : nums)
            seen[num] = true;

        while (original < 1001 && seen[original])
            original *= 2;

        return original;
    }
}
```


1185. 2155.java

Path: LeetCode-main/solutions/2154. Keep Multiplying Found Values by Two/2155.java

```
class Solution {  
    public int findFinalValue(int[] nums, int original) {  
        Set<Integer> numsSet = Arrays.stream(nums).boxed().collect(Collectors.toSet());  
        while (numsSet.contains(original))  
            original *= 2;  
        return original;  
    }  
}
```

1186. 2155.java

Path: LeetCode-main/solutions/2155. All Divisions With the Highest Score of a Binary Array/2155.java

```
class Solution {
    public List<Integer> maxScoreIndices(int[] nums) {
        final int ones = Arrays.stream(nums).sum();
        final int zeros = nums.length - ones;
        // the division at index 0
        List<Integer> ans = new ArrayList<>(List.of(0));
        int leftZeros = 0;
        int leftOnes = 0;
        int maxScore = ones; // `leftZeros` + `rightOnes`

        for (int i = 0; i < nums.length; ++i) {
            leftZeros += nums[i] == 0 ? 1 : 0;
            leftOnes += nums[i] == 1 ? 1 : 0;
            final int rightOnes = ones - leftOnes;
            final int score = leftZeros + rightOnes;
            if (maxScore == score) {
                ans.add(i + 1);
            } else if (maxScore < score) {
                maxScore = score;
                ans = new ArrayList<>(List.of(i + 1));
            }
        }

        return ans;
    }
}
```

1187. 2156.java

Path: LeetCode-main/solutions/2156. Find Substring With Given Hash Value/2156.java

```
class Solution {
    public String subStrHash(String s, int power, int modulo, int k, int hashValue) {
        long maxPower = 1;
        long hash = 0;
        int bestLeft = -1;

        for (int i = s.length() - 1; i >= 0; --i) {
            hash = (hash * power + val(s.charAt(i))) % modulo;
            if (i + k < s.length())
                hash = (hash - val(s.charAt(i + k)) * maxPower % modulo + modulo) % modulo;
            else
                maxPower = maxPower * power % modulo;
            if (hash == hashValue)
                bestLeft = i;
        }

        return s.substring(bestLeft, bestLeft + k);
    }

    private int val(char c) {
        return c - 'a' + 1;
    }
}
```

1188. 2157.java

Path: LeetCode-main/solutions/2157. Groups of Strings/2157.java

```
class UnionFind {
    public UnionFind(int n) {
        count = n;
        id = new int[n];
        sz = new int[n];
        for (int i = 0; i < n; ++i)
            id[i] = i;
        for (int i = 0; i < n; ++i)
            sz[i] = 1;
    }

    public void unionBySize(int u, int v) {
        final int i = find(u);
        final int j = find(v);
        if (i == j)
            return;
        if (sz[i] < sz[j]) {
            sz[j] += sz[i];
            id[i] = j;
        } else {
            sz[i] += sz[j];
            id[j] = i;
        }
        --count;
    }

    public int getCount() {
        return count;
    }

    public int getMaxSize() {
        return Arrays.stream(sz).max().getAsInt();
    }

    private int count;
    private int[] id;
    private int[] sz;

    private int find(int u) {
        return id[u] == u ? u : (id[u] = find(id[u]));
    }
}

class Solution {
    public int[] groupStrings(String[] words) {
        UnionFind uf = new UnionFind(words.length);
        Map<Integer, Integer> maskToIndex = new HashMap<>();
        Map<Integer, Integer> deletedMaskToIndex = new HashMap<>();

        for (int i = 0; i < words.length; ++i) {
            final int mask = getMask(words[i]);
            for (int j = 0; j < 26; ++j)
                if ((mask >> j & 1) == 1) {
                    // Going to delete this bit.
                    final int m = mask ^ 1 << j;
                    if (maskToIndex.containsKey(m))
                        uf.unionBySize(i, maskToIndex.get(m));
                    if (deletedMaskToIndex.containsKey(m))
                        uf.unionBySize(i, deletedMaskToIndex.get(m));
                    else
                        deletedMaskToIndex.put(m, i);
                } else {
                    // Going to add this bit.
                    final int m = mask | 1 << j;
                    if (maskToIndex.containsKey(m))
                        uf.unionBySize(i, maskToIndex.get(m));
                }
            maskToIndex.put(mask, i);
        }

        return new int[] {uf.getCount(), uf.getMaxSize()};
    }

    private int getMask(final String s) {
        int mask = 0;
        for (final char c : s.toCharArray())
            mask |= 1 << c - 'a';
        return mask;
    }
}
```

1189. 2158.java

Path: LeetCode-main/solutions/2158. Amount of New Area Painted Each Day/2158.java

```
enum Type { ENTERING, LEAVING }

class Solution {
    public int[] amountPainted(int[][] paint) {
        record Event(int day, int index, Type type) {}
        final int n = paint.length;
        final int minDay = Arrays.stream(paint).mapToInt(x -> x[0]).min().getAsInt();
        final int maxDay = Arrays.stream(paint).mapToInt(x -> x[1]).max().getAsInt();
        int[] ans = new int[n];
        // Stores the indices of paints that are available now.
        TreeSet<Integer> runningIndices = new TreeSet<>();
        List<Event> events = new ArrayList<>();

        for (int i = 0; i < n; ++i) {
            final int start = paint[i][0];
            final int end = paint[i][1];
            events.add(new Event(start, i, Type.ENTERING)); // 1 := entering
            events.add(new Event(end, i, Type.LEAVING));    // -1 := leaving
        }

        Collections.sort(events, Comparator.comparingInt(Event::day));

        int i = 0; // events' index
        for (int day = minDay; day < maxDay; ++day) {
            while (i < events.size() && events.get(i).day == day) {
                if (events.get(i).type == Type.ENTERING)
                    runningIndices.add(events.get(i).index);
                else
                    runningIndices.remove(events.get(i).index);
                ++i;
            }
            if (!runningIndices.isEmpty())
                ++ans[runningIndices.first()];
        }

        return ans;
    }
}
```

1190. 216.java

Path: LeetCode-main/solutions/216. Combination Sum III/216.java

```
class Solution {
    public List<List<Integer>> combinationSum3(int k, int n) {
        List<List<Integer>> ans = new ArrayList<>();
        dfs(k, n, 1, new ArrayList<>(), ans);
        return ans;
    }

    private void dfs(int k, int n, int s, List<Integer> path, List<List<Integer>> ans) {
        if (k == 0 && n == 0) {
            ans.add(new ArrayList<>(path));
            return;
        }
        if (k == 0 || n < 0)
            return;

        for (int i = s; i <= 9; ++i) {
            path.add(i);
            dfs(k - 1, n - i, i + 1, path, ans);
            path.remove(path.size() - 1);
        }
    }
}
```

1191. 2160.java

Path: LeetCode-main/solutions/2160. Minimum Sum of Four Digit Number After Splitting Digits/2160.java

```
class Solution {  
    public int minimumSum(int num) {  
        char[] chars = String.valueOf(num).toCharArray();  
        Arrays.sort(chars);  
        return (chars[0] - '0') * 10 + (chars[2] - '0') + (chars[1] - '0') * 10 + (chars[3] - '0');  
    }  
}
```

1192. 2161.java

Path: LeetCode-main/solutions/2161. Partition Array According to Given Pivot/2161.java

```
class Solution {
    public int[] pivotArray(int[] nums, int pivot) {
        int[] ans = new int[nums.length];
        int i = 0; // ans' index

        for (final int num : nums)
            if (num < pivot)
                ans[i++] = num;

        for (final int num : nums)
            if (num == pivot)
                ans[i++] = num;

        for (final int num : nums)
            if (num > pivot)
                ans[i++] = num;

        return ans;
    }
}
```


1193. 2162.java

Path: LeetCode-main/solutions/2162. Minimum Cost to Set Cooking Time/2162.java

```
class Solution {
    public int minCostSetTime(int startAt, int moveCost, int pushCost, int targetSeconds) {
        int ans = Integer.MAX_VALUE;
        int mins = targetSeconds > 5999 ? 99 : targetSeconds / 60;
        int secs = targetSeconds - mins * 60;

        while (secs < 100) {
            ans = Math.min(ans, getCost(startAt, moveCost, pushCost, mins, secs));
            --mins;
            secs += 60;
        }

        return ans;
    }

    private int getCost(int startAt, int moveCost, int pushCost, int mins, int secs) {
        int cost = 0;
        char curr = (char) ('0' + startAt);
        for (final char c : String.valueOf(mins * 100 + secs).toCharArray())
            if (c == curr) {
                cost += pushCost;
            } else {
                cost += moveCost + pushCost;
                curr = c;
            }
        return cost;
    }
};
```

1194. 2163.java

Path: LeetCode-main/solutions/2163. Minimum Difference in Sums After Removal of Elements/2163.java

```
class Solution {
    public long minimumDifference(int[] nums) {
        final int n = nums.length / 3;
        long ans = Long.MAX_VALUE;
        long leftSum = 0;
        long rightSum = 0;
        // The left part should be as small as possible.
        Queue<Integer> maxHeap = new PriorityQueue<>(Collections.reverseOrder());
        // The right part should be as big as possible.
        Queue<Integer> minHeap = new PriorityQueue<>();
        // minLeftSum[i] := the minimum of the sum of n nums in nums[0..i)
        long[] minLeftSum = new long[nums.length];

        for (int i = 0; i < 2 * n; ++i) {
            maxHeap.offer(nums[i]);
            leftSum += nums[i];
            if (maxHeap.size() == n + 1)
                leftSum -= maxHeap.poll();
            if (maxHeap.size() == n)
                minLeftSum[i] = leftSum;
        }

        for (int i = nums.length - 1; i >= n; --i) {
            minHeap.offer(nums[i]);
            rightSum += nums[i];
            if (minHeap.size() == n + 1)
                rightSum -= minHeap.poll();
            if (minHeap.size() == n)
                ans = Math.min(ans, minLeftSum[i - 1] - rightSum);
        }

        return ans;
    }
}
```

1195. 2164.java

Path: LeetCode-main/solutions/2164. Sort Even and Odd Indices Independently/2164.java

```
class Solution {
    public int[] sortEvenOdd(int[] nums) {
        final int n = nums.length;
        int[] ans = new int[n];
        int[] evenCount = new int[101];
        int[] oddCount = new int[101];

        for (int i = 0; i < n; ++i)
            if (i % 2 == 1)
                ++oddCount[nums[i]];
            else
                ++evenCount[nums[i]];

        int ansIndex = 0;
        for (int i = 1; i < 101; ++i)
            while (evenCount[i]-- > 0) {
                ans[ansIndex] = i;
                ansIndex += 2;
            }

        ansIndex = 1;
        for (int i = 100; i > 0; --i)
            while (oddCount[i]-- > 0) {
                ans[ansIndex] = i;
                ansIndex += 2;
            }

        return ans;
    }
}
```

1196. 2165.java

Path: LeetCode-main/solutions/2165. Smallest Value of the Rearranged Number/2165.java

```
class Solution {
    public long smallestNumber(long num) {
        String[] digits = String.valueOf(Math.abs(num)).split("");
        String s = Stream.of(digits).sorted().collect(Collectors.joining());
        StringBuilder sb = new StringBuilder(s);
        if (num <= 0)
            return -1 * Long.parseLong(sb.reverse().toString());
        if (sb.charAt(0) == '0') {
            final int firstNonZeroIndex = sb.lastIndexOf("0") + 1;
            sb.setCharAt(0, sb.charAt(firstNonZeroIndex));
            sb.setCharAt(firstNonZeroIndex, '0');
        }
        return Long.parseLong(sb.toString());
    }
}
```

1197. 2166.java

Path: LeetCode-main/solutions/2166. Design Bitset/2166.java

```
class Bitset {
    public Bitset(int size) {
        for (int i = 0; i < size; ++i) {
            sb.append('0');
            rb.append('1');
        }
    }

    public void fix(int idx) {
        if (sb.charAt(idx) == '0')
            ++cnt;
        sb.setCharAt(idx, '1');
        rb.setCharAt(idx, '0');
    }

    public void unfix(int idx) {
        if (sb.charAt(idx) == '1')
            --cnt;
        sb.setCharAt(idx, '0');
        rb.setCharAt(idx, '1');
    }

    public void flip() {
        StringBuilder temp = sb;
        sb = rb;
        rb = temp;
        cnt = sb.length() - cnt;
    }

    public boolean all() {
        return cnt == sb.length();
    }

    public boolean one() {
        return cnt > 0;
    }

    public int count() {
        return cnt;
    }

    public String toString() {
        return sb.toString();
    }

    private StringBuilder sb = new StringBuilder(); // the original
    private StringBuilder rb = new StringBuilder(); // the reversed
    private int cnt = 0;
}
```

1198. 2167-2.java

Path: LeetCode-main/solutions/2167. Minimum Time to Remove All Cars Containing Illegal Goods/2167-2.java

```
class Solution {
    public int minimumTime(String s) {
        final int n = s.length();
        int ans = n;
        int left = 0; // the minimum time to remove the illegal cars so far

        for (int i = 0; i < n; ++i) {
            left = Math.min(left + (s.charAt(i) - '0') * 2, i + 1);
            ans = Math.min(ans, left + n - 1 - i);
        }

        return ans;
    }
}
```

1199. 2167.java

Path: LeetCode-main/solutions/2167. Minimum Time to Remove All Cars Containing Illegal Goods/2167.java

```
class Solution {
    public int minimumTime(String s) {
        final int n = s.length();
        // left[i] := the minimum time to remove the illegal cars of s[0..i]
        int[] left = new int[n];
        left[0] = s.charAt(0) - '0';
        // dp[i] := the minimum time to remove the illegal cars of s[0..i] optimally
        // + the time to remove the illegal cars of s[i + 1..n) consecutively
        // Note that the way to remove the illegal cars in the right part
        // doesn't need to be optimal since:
        //   `left | illegal cars | n - 1 - k` will be covered in
        //   `left' | n - 1 - i` later.
        int[] dp = new int[n];
        Arrays.fill(dp, n);
        dp[0] = left[0] + n - 1;

        for (int i = 1; i < n; ++i) {
            left[i] = Math.min(left[i - 1] + (s.charAt(i) - '0') * 2, i + 1);
            dp[i] = Math.min(dp[i], left[i] + n - 1 - i);
        }

        return Arrays.stream(dp).min().getAsInt();
    }
}
```

1200. 2168.java

Path: LeetCode-main/solutions/2168. Unique Substrings With Equal Digit Frequency/2168.java

```
class Solution {
    public int equalDigitFrequency(String s) {
        final int n = s.length();
        int[][] counts = new int[n][]; // counts[i] := the counter map of s[0..i]
        int[] count = new int[10];
        long[] pows = new long[n + 1]; // pows[i] := BASE^i % HASH
        // hash[i] = the hash of the first i letters of s, where hash[i] =
        // (26^(i - 1) * s[0] + 26^(i - 2) * s[1] + ... + s[i - 1]) % HASH
        long[] hash = new long[n + 1];
        Set<Integer> seen = new HashSet<>();

        pows[0] = 1;

        for (int i = 0; i < n; ++i) {
            ++count[s.charAt(i) - '0'];
            counts[i] = count.clone();
            pows[i + 1] = pows[i] * BASE % HASH;
            hash[i + 1] = (hash[i] * BASE + val(s.charAt(i))) % HASH;
        }

        for (int i = 0; i < n; ++i)
            for (int j = i; j < n; ++j)
                if (isSameFreq(counts, i, j))
                    seen.add(getRollingHash(i, j + 1, hash, pows));

        return seen.size();
    }

    private static final int MAX = 1001;
    private static final int BASE = 11;
    private static final int HASH = 1_000_000_007;

    private static int val(char c) {
        return c - '0' + 1;
    }

    // Returns true if s[i..j] has the same digit frequency.
    private boolean isSameFreq(int[][] counts, int i, int j) {
        int[] count = counts[j].clone();
        if (i > 0)
            for (int num = 0; num < 10; ++num)
                count[num] -= counts[i - 1][num];
        return equalFreq(count);
    }

    private boolean equalFreq(int[] count) {
        int minfreq = MAX;
        int maxfreq = 0;
        for (final int freq : count)
            if (freq > 0) {
                minfreq = Math.min(minfreq, freq);
                maxfreq = Math.max(maxfreq, freq);
            }
        return minfreq == maxfreq;
    }

    // Returns the rolling hash of s[l..r].
    private int getRollingHash(int l, int r, long[] hash, long[] pows) {
        final long h = (hash[r] - hash[l] * pows[r - l]) % HASH;
        return (int) (h < 0 ? h + HASH : h);
    }
}
```


1201. 2169.java

Path: LeetCode-main/solutions/2169. Count Operations to Obtain Zero/2169.java

```
class Solution {
    public int countOperations(int num1, int num2) {
        int ans = 0;

        while (num1 > 0 && num2 > 0) {
            if (num1 < num2) {
                final int temp = num1;
                num1 = num2;
                num2 = temp;
            }
            ans += num1 / num2;
            num1 %= num2;
        }

        return ans;
    }
}
```

1202. 217.java

Path: LeetCode-main/solutions/217. Contains Duplicate/217.java

```
class Solution {
    public boolean containsDuplicate(int[] nums) {
        Set<Integer> seen = new HashSet<>();

        for (final int num : nums)
            if (!seen.add(num))
                return true;

        return false;
    }
}
```

1203. 2170.java

Path: LeetCode-main/solutions/2170. Minimum Operations to Make the Array Alternating/2170.java

```
class T {
    public Map<Integer, Integer> count = new HashMap<>();
    public int mx = 0;
    public int secondMax = 0;
    public int maxFreq = 0;
    public int secondMaxFreq = 0;
}

class Solution {
    public int minimumOperations(int[] nums) {
        T[] ts = new T[2];
        ts[0] = new T();
        ts[1] = new T();

        for (int i = 0; i < nums.length; ++i) {
            T t = ts[i % 2];
            t.count.merge(nums[i], 1, Integer::sum);
            final int freq = t.count.get(nums[i]);
            if (freq > t.maxFreq) {
                t.maxFreq = freq;
                t.mx = nums[i];
            } else if (freq > t.secondMaxFreq) {
                t.secondMaxFreq = freq;
                t.secondMax = nums[i];
            }
        }

        if (ts[0].mx == ts[1].mx)
            return nums.length -
                Math.max(ts[0].maxFreq + ts[1].secondMaxFreq, ts[1].maxFreq + ts[0].secondMaxFreq);
        return nums.length - (ts[0].maxFreq + ts[1].maxFreq);
    }
}
```

1204. 2171.java

Path: LeetCode-main/solutions/2171. Removing Minimum Number of Magic Beans/2171.java

```
class Solution {
    public long minimumRemoval(int[] beans) {
        final long n = beans.length;
        final long sum = Arrays.stream(beans).asLongStream().sum();
        long ans = Long.MAX_VALUE;

        Arrays.sort(beans);

        for (int i = 0; i < n; ++i)
            ans = Math.min(ans, sum - (n - i) * beans[i]);

        return ans;
    }
}
```

1205. 2172.java

Path: LeetCode-main/solutions/2172. Maximum AND Sum of Array/2172.java

```
class Solution {
    public int maximumANDSum(int[] nums, int numSlots) {
        final int n = 2 * numSlots;
        final int nSelected = 1 << n;
        // dp[i] := the maximum value, where i is the bitmask of the selected
        // numbers
        int[] dp = new int[nSelected];
        int[] arr = Arrays.copyOf(nums, n);

        for (int mask = 1; mask < nSelected; ++mask) {
            final int selected = Integer.bitCount(mask);
            final int slot = (selected + 1) / 2; // (1, 2) -> 1, (3, 4) -> 2
            for (int i = 0; i < n; ++i)
                if ((mask >> i & 1) == 1) // Assign `arr[i]` to the `slot`-th slot.
                    dp[mask] = Math.max(dp[mask], dp[mask ^ 1 << i] + (slot & arr[i]));
        }

        return dp[nSelected - 1];
    }
}
```

1206. 2174-2.java

Path: LeetCode-main/solutions/2174. Remove All Ones With Row and Column Flips II/2174-2.java

```
class Solution {
    public int removeOnes(int[][] grid) {
        final int m = grid.length;
        final int n = grid[0].length;
        final int maxMask = 1 << m * n;
        // dp[i] := the minimum number of operations to remove all 1s from the grid,
        // where `i` is the bitmask of the state of the grid
        int[] dp = new int[1 << m * n];
        Arrays.fill(dp, Integer.MAX_VALUE / 2);
        dp[0] = 0;

        for (int mask = 0; mask < maxMask; ++mask)
            for (int i = 0; i < m; ++i)
                for (int j = 0; j < n; ++j)
                    if (grid[i][j] == 1) {
                        int newMask = mask;
                        // Set the cells in the same row with 0.
                        for (int k = 0; k < n; ++k)
                            newMask &= ~(1 << i * n + k);
                        // Set the cells in the same column with 0.
                        for (int k = 0; k < m; ++k)
                            newMask &= ~(1 << k * n + j);
                        dp[mask] = Math.min(dp[mask], 1 + dp[newMask]);
                    }

        return dp[encode(grid, m, n)];
    }

    private int encode(int[][] grid, int m, int n) {
        int encoded = 0;
        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j)
                if (grid[i][j] == 1)
                    encoded |= 1 << i * n + j;
        return encoded;
    }
}
```

1207. 2174.java

Path: LeetCode-main/solutions/2174. Remove All Ones With Row and Column Flips II/2174.java

```
class Solution {
    public int removeOnes(int[][] grid) {
        final int m = grid.length;
        final int n = grid[0].length;
        int[] mem = new int[1 << m * n];
        Arrays.fill(mem, Integer.MAX_VALUE);
        return removeOnes(encode(grid, m, n), m, n, mem);
    }

    // Returns the minimum number of operations to remove all 1s from the grid,
    // where `mask` is the bitmask of the state of the grid.
    private int removeOnes(int mask, int m, int n, int[] mem) {
        if (mask == 0)
            return 0;
        if (mem[mask] < Integer.MAX_VALUE)
            return mem[mask];

        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j)
                if ((mask >> i * n + j & 1) == 1) { // grid[i][j] == 1
                    int newMask = mask;
                    // Set the cells in the same row with 0.
                    for (int k = 0; k < n; ++k)
                        newMask &= ~(1 << i * n + k);
                    // Set the cells in the same column with 0.
                    for (int k = 0; k < m; ++k)
                        newMask &= ~(1 << k * n + j);
                    mem[mask] = Math.min(mem[mask], 1 + removeOnes(newMask, m, n, mem));
                }

        return mem[mask];
    }

    private int encode(int[][] grid, int m, int n) {
        int encoded = 0;
        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j)
                if (grid[i][j] == 1)
                    encoded |= 1 << i * n + j;
        return encoded;
    }
}
```

1208. 2176.java

Path: LeetCode-main/solutions/2176. Count Equal and Divisible Pairs in an Array/2176.java

```
class Solution {
    public int countPairs(int[] nums, int k) {
        int ans = 0;
        Map<Integer, List<Integer>> numToIndices = new HashMap<>();

        for (int i = 0; i < nums.length; ++i) {
            numToIndices.putIfAbsent(nums[i], new ArrayList<>());
            numToIndices.get(nums[i]).add(i);
        }

        for (List<Integer> indices : numToIndices.values()) {
            Map<Integer, Integer> gcds = new HashMap<>();
            for (final int i : indices) {
                final int gcd_i = gcd(i, k);
                for (final int gcd_j : gcds.keySet())
                    if (gcd_i * gcd_j % k == 0)
                        ans += gcds.get(gcd_j);
                gcds.merge(gcd_i, 1, Integer::sum);
            }
        }

        return ans;
    }

    private int gcd(int a, int b) {
        return b == 0 ? a : gcd(b, a % b);
    }
}
```


1209. 2177.java

Path: LeetCode-main/solutions/2177. Find Three Consecutive Integers That Sum to a Given Number/2177.java

```
class Solution {
    public long[] sumOfThree(long num) {
        if (num % 3 != 0)
            return new long[] {};
        final long x = num / 3;
        return new long[] {x - 1, x, x + 1};
    }
}
```

1210. 2178.java

Path: LeetCode-main/solutions/2178. Maximum Split of Positive Even Integers/2178.java

```
class Solution {
    public List<Long> maximumEvenSplit(long finalSum) {
        if (finalSum % 2 == 1)
            return new ArrayList<>();

        List<Long> ans = new ArrayList<>();
        long needSum = finalSum;
        long even = 2;

        while (needSum - even >= even + 2) {
            ans.add(even);
            needSum -= even;
            even += 2;
        }

        ans.add(needSum);
        return ans;
    }
}
```

1211. 2179.java

Path: LeetCode-main/solutions/2179. Count Good Triplets in an Array/2179.java

```
class FenwickTree {
    public FenwickTree(int n) {
        sums = new int[n + 1];
    }

    public void add(int i, int delta) {
        while (i < sums.length) {
            sums[i] += delta;
            i += lowbit(i);
        }
    }

    public int get(int i) {
        int sum = 0;
        while (i > 0) {
            sum += sums[i];
            i -= lowbit(i);
        }
        return sum;
    }

    private int[] sums;

    private static int lowbit(int i) {
        return i & -i;
    }
}

class Solution {
    public long goodTriplets(int[] nums1, int[] nums2) {
        final int n = nums1.length;
        long ans = 0;
        Map<Integer, Integer> numToIndex = new HashMap<>();
        int[] arr = new int[n];
        // leftSmaller[i] := the number of arr[j] < arr[i], where 0 <= j < i
        int[] leftSmaller = new int[n];
        // rightLarger[i] := the number of arr[j] > arr[i], where i < j < n
        int[] rightLarger = new int[n];
        FenwickTree tree1 = new FenwickTree(n); // Calculates `leftSmaller`.
        FenwickTree tree2 = new FenwickTree(n); // Calculates `rightLarger`.

        for (int i = 0; i < n; ++i)
            numToIndex.put(nums1[i], i);

        // Remap each number in `nums2` to the according index in `nums1` as `arr`.
        // So the problem is to find the number of increasing tripets in `arr`.
        for (int i = 0; i < n; ++i)
            arr[i] = numToIndex.get(nums2[i]);

        for (int i = 0; i < n; ++i) {
            leftSmaller[i] = tree1.get(arr[i]);
            tree1.add(arr[i] + 1, 1);
        }

        for (int i = n - 1; i >= 0; --i) {
            rightLarger[i] = tree2.get(n) - tree2.get(arr[i]);
            tree2.add(arr[i] + 1, 1);
        }

        for (int i = 0; i < n; ++i)
            ans += (long) leftSmaller[i] * rightLarger[i];

        return ans;
    }
}
```

1212. 218.java

Path: LeetCode-main/solutions/218. The Skyline Problem/218.java

```
class Solution {
    public List<List<Integer>> getSkyline(int[][] buildings) {
        final int n = buildings.length;
        if (n == 0)
            return new ArrayList<>();
        if (n == 1) {
            final int left = buildings[0][0];
            final int right = buildings[0][1];
            final int height = buildings[0][2];
            List<List<Integer>> ans = new ArrayList<>();
            ans.add(new ArrayList<>(List.of(left, height)));
            ans.add(new ArrayList<>(List.of(right, 0)));
            return ans;
        }

        List<List<Integer>> leftSkyline = getSkyline(Arrays.copyOfRange(buildings, 0, n / 2));
        List<List<Integer>> rightSkyline = getSkyline(Arrays.copyOfRange(buildings, n / 2, n));
        return merge(leftSkyline, rightSkyline);
    }

    private List<List<Integer>> merge(List<List<Integer>> left, List<List<Integer>> right) {
        List<List<Integer>> ans = new ArrayList<>();
        int i = 0; // left's index
        int j = 0; // right's index
        int leftY = 0;
        int rightY = 0;

        while (i < left.size() && j < right.size())
            // Choose the point with the smaller x.
            if (left.get(i).get(0) < right.get(j).get(0)) {
                leftY = left.get(i).get(1); // Update the ongoing `leftY`.
                addPoint(ans, left.get(i).get(0), Math.max(left.get(i++).get(1), rightY));
            } else {
                rightY = right.get(j).get(1); // Update the ongoing `rightY`.
                addPoint(ans, right.get(j).get(0), Math.max(right.get(j++).get(1), leftY));
            }

        while (i < left.size())
            addPoint(ans, left.get(i).get(0), left.get(i++).get(1));

        while (j < right.size())
            addPoint(ans, right.get(j).get(0), right.get(j++).get(1));

        return ans;
    }

    private void addPoint(List<List<Integer>> ans, int x, int y) {
        if (!ans.isEmpty() && ans.get(ans.size() - 1).get(0) == x) {
            ans.get(ans.size() - 1).set(1, y);
            return;
        }
        if (!ans.isEmpty() && ans.get(ans.size() - 1).get(1) == y)
            return;
        ans.add(new ArrayList<>(List.of(x, y)));
    }
}
```

1213. 2180.java

Path: LeetCode-main/solutions/2180. Count Integers With Even Digit Sum/2180.java

```
class Solution {
    public int countEven(int num) {
        if (getDigitSum(num) % 2 == 0)
            return num / 2;
        return (num - 1) / 2;
    }

    private int getDigitSum(int num) {
        int digitSum = 0;
        while (num > 0) {
            digitSum += num % 10;
            num /= 10;
        }
        return digitSum;
    }
}
```

1214. 2181-2.java

Path: LeetCode-main/solutions/2181. Merge Nodes in Between Zeros/2181-2.java

```
class Solution {
    public ListNode mergeNodes(ListNode head) {
        ListNode curr = head.next;

        while (curr != null) {
            ListNode running = curr;
            int sum = 0;
            while (running.val > 0) {
                sum += running.val;
                running = running.next;
            }
            curr.val = sum;
            curr.next = running.next;
            curr = running.next;
        }

        return head.next;
    }
}
```

1215. 2181.java

Path: LeetCode-main/solutions/2181. Merge Nodes in Between Zeros/2181.java

```
class Solution {
    public ListNode mergeNodes(ListNode head) {
        if (head == null)
            return null;
        if (head.next.val == 0) {
            ListNode node = new ListNode(head.val);
            node.next = mergeNodes(head.next.next);
            return node;
        }

        ListNode next = mergeNodes(head.next);
        next.val += head.val;
        return next;
    }
}
```

1216. 2182.java

Path: LeetCode-main/solutions/2182. Construct String With Repeat Limit/2182.java

```
class Solution {
    public String repeatLimitedString(String s, int repeatLimit) {
        StringBuilder sb = new StringBuilder();
        int[] count = new int[26];

        for (final char c : s.toCharArray())
            ++count[c - 'a'];

        while (true) {
            final boolean addOne = !sb.isEmpty() && shouldAddOne(sb, count);
            final int i = getLargestChar(sb, count);
            if (i == -1)
                break;
            final int repeats = addOne ? 1 : Math.min(count[i], repeatLimit);
            sb.append(String.valueOf((char) ('a' + i)).repeat(repeats));
            count[i] -= repeats;
        }

        return sb.toString();
    }

    private boolean shouldAddOne(StringBuilder sb, int[] count) {
        for (int i = 25; i >= 0; --i)
            if (count[i] > 0)
                return sb.charAt(sb.length() - 1) == 'a' + i;
        return false;
    }

    private int getLargestChar(StringBuilder sb, int[] count) {
        for (int i = 25; i >= 0; --i)
            if (count[i] > 0 && (sb.isEmpty() || sb.charAt(sb.length() - 1) != 'a' + i))
                return i;
        return -1;
    }
}
```


1217. 2183.java

Path: LeetCode-main/solutions/2183. Count Array Pairs Divisible by K/2183.java

```
class Solution {
    public long countPairs(int[] nums, int k) {
        long ans = 0;
        Map<Integer, Integer> gcds = new HashMap<>();

        for (final int num : nums) {
            final int gcd_i = gcd(num, k);
            for (final int gcd_j : gcds.keySet())
                if ((long) gcd_i * gcd_j % k == 0)
                    ans += gcds.get(gcd_j);
            gcds.merge(gcd_i, 1, Integer::sum);
        }

        return ans;
    }

    private int gcd(int a, int b) {
        return b == 0 ? a : gcd(b, a % b);
    }
}
```

1218. 2184.java

Path: LeetCode-main/solutions/2184. Number of Ways to Build Sturdy Brick Wall/2184.java

```
class Solution {
    public int buildWall(int height, int width, int[] bricks) {
        final int MOD = 1_000_000_007;
        // Stores the valid rows in bitmask.
        List<Integer> rows = new ArrayList<>();
        buildRows(width, bricks, 0, rows);

        final int n = rows.size();
        // dp[i] := the number of ways to build `h` height walls with rows[i] in the bottom
        long[] dp = new long[n];
        // graph[i] := the valid neighbors of rows[i]
        List<Integer>[] graph = new List[n];

        Arrays.fill(dp, 1);
        Arrays.setAll(graph, i -> new ArrayList<>());

        for (int i = 0; i < n; ++i)
            for (int j = 0; j < n; ++j)
                if ((rows.get(i) & rows.get(j)) == 0)
                    graph[i].add(j);

        for (int h = 2; h <= height; ++h) {
            long[] newDp = new long[n];
            for (int i = 0; i < n; ++i)
                for (final int v : graph[i]) {
                    newDp[i] += dp[v];
                    newDp[i] %= MOD;
                }
            dp = newDp;
        }

        return (int) (Arrays.stream(dp).sum() % MOD);
    }

    private void buildRows(int width, int[] bricks, int path, List<Integer> rows) {
        for (final int brick : bricks)
            if (brick == width)
                rows.add(path);
            else if (brick < width) {
                final int newWidth = width - brick;
                buildRows(newWidth, bricks, path | 1 << newWidth, rows);
            }
    }
}
```

1219. 2185.java

Path: LeetCode-main/solutions/2185. Counting Words With a Given Prefix/2185.java

```
class Solution {  
    public int prefixCount(String[] words, String pref) {  
        return (int) Arrays.stream(words).filter(w -> w.startsWith(pref)).count();  
    }  
}
```

1220. 2186.java

Path: LeetCode-main/solutions/2186. Minimum Number of Steps to Make Two Strings Anagram II/2186.java

```
class Solution {  
    public int minSteps(String s, String t) {  
        int[] count = new int[26];  
        s.chars().forEach(c -> ++count[c - 'a']);  
        t.chars().forEach(c -> --count[c - 'a']);  
        return Arrays.stream(count).map(Math::abs).sum();  
    }  
}
```

1221. 2187.java

Path: LeetCode-main/solutions/2187. Minimum Time to Complete Trips/2187.java

```
class Solution {
    public long minimumTime(int[] time, int totalTrips) {
        long l = 1;
        long r = (long) Arrays.stream(time).min().getAsInt() * totalTrips;

        while (l < r) {
            final long m = (l + r) / 2;
            if (numTrips(time, m) >= totalTrips)
                r = m;
            else
                l = m + 1;
        }

        return l;
    }

    private long numTrips(int[] time, long m) {
        return Arrays.stream(time).asLongStream().reduce(0L, (subtotal, t) -> subtotal + m / t);
    }
}
```

1222. 2188.java

Path: LeetCode-main/solutions/2188. Minimum Time to Finish the Race/2188.java

```
class Solution {
    public int minimumFinishTime(int[][] tires, int changeTime, int numLaps) {
        // singleTire[i] := the minimum time to finish i laps without changing tire
        int[] singleTire = new int[numLaps + 1];
        // dp[i] := the minimum time to finish i laps
        int[] dp = new int[numLaps + 1];

        Arrays.fill(singleTire, Integer.MAX_VALUE / 2);
        Arrays.fill(dp, Integer.MAX_VALUE / 2);

        for (int i = 0; i < tires.length; ++i) {
            final int f = tires[i][0];
            final int r = tires[i][1];
            int sumSecs = 0;
            int rPower = 1;
            for (int j = 1; j <= numLaps; ++j) {
                // the time to use the same tire for the next lap >=
                // the time to change a new tire + f
                if ((long) f * rPower >= changeTime + f)
                    break;
                sumSecs += f * rPower;
                rPower *= r;
                singleTire[j] = Math.min(singleTire[j], sumSecs);
            }

            dp[0] = 0;
            for (int i = 1; i <= numLaps; ++i)
                for (int j = 1; j <= i; ++j)
                    dp[i] = Math.min(dp[i], dp[i - j] + changeTime + singleTire[j]);
        }

        return dp[numLaps] - changeTime;
    }
}
```

1223. 2189.java

Path: LeetCode-main/solutions/2189. Number of Ways to Build House of Cards/2189.java

```
class Solution {
    public int houseOfCards(int n) {
        // dp[i] := the number of valid result for i cards
        int[] dp = new int[n + 1];
        dp[0] = 1;

        for (int baseCards = 2; baseCards <= n; baseCards += 3)
            for (int i = n; i >= baseCards; --i)
                // Use `baseCards` as the base, so we're left with `i - baseCards` cards.
                dp[i] += dp[i - baseCards];

        return dp[n];
    }
}
```

1224. 219.java

Path: LeetCode-main/solutions/219. Contains Duplicate II/219.java

```
class Solution {
    public boolean containsNearbyDuplicate(int[] nums, int k) {
        Set<Integer> seen = new HashSet<>();

        for (int i = 0; i < nums.length; ++i) {
            if (!seen.add(nums[i]))
                return true;
            if (i >= k)
                seen.remove(nums[i - k]);
        }

        return false;
    }
}
```


1225. 2190.java

Path: LeetCode-main/solutions/2190. Most Frequent Number Following Key In an Array/2190.java

```
class Solution {
    public int mostFrequent(int[] nums, int key) {
        int[] count = new int[1001];
        int ans = 0;

        for (int i = 0; i + 1 < nums.length; ++i)
            if (nums[i] == key)
                ++count[nums[i + 1]];

        for (int i = 1; i < 1001; ++i)
            if (count[i] > count[ans])
                ans = i;

        return ans;
    }
}
```

1226. 2191-2.java

Path: LeetCode-main/solutions/2191. Sort the Jumbled Numbers/2191-2.java

```
class Solution {
    public int[] sortJumbled(int[] mapping, int[] nums) {
        List<Integer> ans = new ArrayList<>();
        TreeMap<Integer, List<Integer>> mappedToOriginalNums = new TreeMap<>();

        for (final int num : nums) {
            final int mapped = getMapped(num, mapping);
            mappedToOriginalNums.putIfAbsent(mapped, new ArrayList<>());
            mappedToOriginalNums.get(mapped).add(num);
        }

        for (List<Integer> originalNums : mappedToOriginalNums.values())
            ans.addAll(originalNums);

        return ans.stream().mapToInt(Integer::intValue).toArray();
    }

    private int getMapped(int num, int[] mapping) {
        StringBuilder sb = new StringBuilder();
        for (final char c : String.valueOf(num).toCharArray())
            sb.append(mapping[c - '0']);
        return Integer.parseInt(sb.toString());
    }
}
```

1227. 2191.java

Path: LeetCode-main/solutions/2191. Sort the Jumbled Numbers/2191.java

```
class Solution {
    public int[] sortJumbled(int[] mapping, int[] nums) {
        int[] ans = new int[nums.length];
        List<int[]> A = new ArrayList<>(); // (mapped, index, num)

        for (int i = 0; i < nums.length; ++i)
            A.add(new int[] {getMapped(nums[i], mapping), i, nums[i]});

        Collections.sort(
            A, Comparator.comparingInt((int[] a) -> a[0]).thenComparingInt((int[] a) -> a[1]));
        return A.stream().mapToInt(a -> a[2]).toArray();
    }

    private int getMapped(int num, int[] mapping) {
        StringBuilder sb = new StringBuilder();
        for (final char c : String.valueOf(num).toCharArray())
            sb.append(mapping[c - '0']);
        return Integer.parseInt(sb.toString());
    }
}
```

1228. 2192-2.java

Path: LeetCode-main/solutions/2192. All Ancestors of a Node in a Directed Acyclic Graph/2192-2.java

```
class Solution {
    public List<List<Integer>> getAncestors(int n, int[][] edges) {
        List<List<Integer>> ans = new ArrayList<>();
        List<Integer>[] graph = new List[n];
        TreeSet<Integer>[] ancestors = new TreeSet[n]; // {u: {ancestors}}
        int[] inDegrees = new int[n];

        for (int i = 0; i < n; ++i) {
            graph[i] = new ArrayList<>();
            ancestors[i] = new TreeSet<>();
        }

        // Build the graph.
        for (int[] edge : edges) {
            final int u = edge[0];
            final int v = edge[1];
            graph[u].add(v);
            ++inDegrees[v];
        }

        // Perform topological sorting.
        Queue<Integer> q = IntStream.range(0, n)
            .filter(i -> inDegrees[i] == 0)
            .boxed()
            .collect(Collectors.toCollection(ArrayDeque::new));

        while (!q.isEmpty())
            for (int sz = q.size(); sz > 0; --sz) {
                final int u = q.poll();
                for (final int v : graph[u]) {
                    ancestors[v].add(u);
                    ancestors[v].addAll(ancestors[u]);
                    if (--inDegrees[v] == 0)
                        q.offer(v);
                }
            }

        for (TreeSet<Integer> nodes : ancestors)
            ans.add(new ArrayList<>(nodes));

        return ans;
    }
}
```

1229. 2192.java

Path: LeetCode-main/solutions/2192. All Ancestors of a Node in a Directed Acyclic Graph/2192.java

```
class Solution {
    public List<List<Integer>> getAncestors(int n, int[][] edges) {
        List<List<Integer>> ans = new ArrayList<>();
        List<Integer>[] graph = new List[n];

        for (int i = 0; i < n; ++i) {
            ans.add(new ArrayList<>());
            graph[i] = new ArrayList<>();
        }

        for (int[] edge : edges) {
            final int u = edge[0];
            final int v = edge[1];
            graph[u].add(v);
        }

        for (int i = 0; i < n; ++i)
            dfs(graph, i, i, new boolean[n], ans);

        return ans;
    }

    private void dfs(List<Integer>[] graph, int u, int ancestor, boolean[] seen,
                     List<List<Integer>> ans) {
        seen[u] = true;
        for (final int v : graph[u]) {
            if (seen[v])
                continue;
            ans.get(v).add(ancestor);
            dfs(graph, v, ancestor, seen, ans);
        }
    }
}
```

1230. 2193.java

Path: LeetCode-main/solutions/2193. Minimum Number of Moves to Make Palindrome/2193.java

```
class Solution {
    public int minMovesToMakePalindrome(String s) {
        int ans = 0;
        StringBuilder sb = new StringBuilder(s);

        while (sb.length() > 1) {
            // Greedily match the last digit.
            final int i = sb.indexOf(sb.substring(sb.length() - 1));
            if (i == sb.length() - 1) {
                // s[i] is the middle letter.
                ans += i / 2;
            } else {
                sb.deleteCharAt(i);
                ans += i; // Swap the matched letter to the left.
            }
            sb.deleteCharAt(sb.length() - 1);
        }

        return ans;
    }
}
```

1231. 2194.java

Path: LeetCode-main/solutions/2194. Cells in a Range on an Excel Sheet/2194.java

```
class Solution {
    public List<String> cellsInRange(String s) {
        List<String> ans = new ArrayList<>();
        final char startCol = s.charAt(0);
        final char endCol = s.charAt(3);
        final char startRow = s.charAt(1);
        final char endRow = s.charAt(4);

        for (char col = startCol; col <= endCol; ++col)
            for (char row = startRow; row <= endRow; ++row)
                ans.add("" + col + row);

        return ans;
    }
}
```

1232. 2195.java

Path: LeetCode-main/solutions/2195. Append K Integers With Minimal Sum/2195.java

```
class Solution {
    public long minimalKSum(int[] nums, int k) {
        long ans = 0;
        Arrays.sort(nums);

        if (nums[0] > 1) {
            final int l = 1;
            final int r = Math.min(k, nums[0] - 1);
            ans += (long) (l + r) * (r - l + 1) / 2;
            k -= r - l + 1;
            if (k == 0)
                return ans;
        }

        for (int i = 0; i + 1 < nums.length; ++i) {
            if (nums[i] == nums[i + 1])
                continue;
            final int l = nums[i] + 1;
            final int r = Math.min(nums[i] + k, nums[i + 1] - 1);
            ans += (long) (l + r) * (r - l + 1) / 2;
            k -= r - l + 1;
            if (k == 0)
                return ans;
        }

        if (k > 0) {
            final int l = nums[nums.length - 1] + 1;
            final int r = nums[nums.length - 1] + k;
            ans += (long) (l + r) * (r - l + 1) / 2;
        }

        return ans;
    }
}
```


1233. 2196.java

Path: LeetCode-main/solutions/2196. Create Binary Tree From Descriptions/2196.java

```
class Solution {
    public TreeNode createBinaryTree(int[][] descriptions) {
        Map<TreeNode, TreeNode> childToParent = new HashMap<>();
        Map<Integer, TreeNode> valToNode = new HashMap<>();

        for (int[] d : descriptions) {
            final int p = d[0];
            final int c = d[1];
            final int isLeft = d[2];
            TreeNode parent = valToNode.getOrDefault(p, new TreeNode(p));
            TreeNode child = valToNode.getOrDefault(c, new TreeNode(c));
            valToNode.put(p, parent);
            valToNode.put(c, child);
            childToParent.put(child, parent);
            if (isLeft == 1)
                parent.left = child;
            else
                parent.right = child;
        }

        // Pick a random node and traverse upwardly.
        TreeNode root = childToParent.keySet().iterator().next();
        while (childToParent.containsKey(root))
            root = childToParent.get(root);
        return root;
    }
}
```

1234. 2197.java

Path: LeetCode-main/solutions/2197. Replace Non-Coprime Numbers in Array/2197.java

```
class Solution {
    public List<Integer> replaceNonCoprimes(int[] nums) {
        LinkedList<Integer> ans = new LinkedList<>();

        for (int num : nums) {
            while (!ans.isEmpty() && gcd(ans.getLast(), num) > 1)
                num = lcm(ans.removeLast(), num);
            ans.addLast(num);
        }

        return ans;
    }

    private int gcd(int a, int b) {
        return b == 0 ? a : gcd(b, a % b);
    }

    private int lcm(int a, int b) {
        return a * (b / gcd(a, b));
    }
}
```

1235. 2198.java

Path: LeetCode-main/solutions/2198. Number of Single Divisor Triplets/2198.java

```
class Solution {
    public long singleDivisorTriplet(int[] nums) {
        final int MAX = 100;
        long ans = 0;
        int[] count = new int[MAX + 1];

        for (final int num : nums)
            ++count[num];

        for (int a = 1; a <= MAX; ++a)
            for (int b = a; count[a] > 0 && b <= MAX; ++b)
                for (int c = b; count[b] > 0 && c <= MAX; ++c) {
                    final int sum = a + b + c;
                    if (divisible(sum, a) + divisible(sum, b) + divisible(sum, c) != 1)
                        continue;
                    if (a == b)
                        ans += (long) count[a] * (count[a] - 1) / 2 * count[c];
                    else if (b == c)
                        ans += (long) count[b] * (count[b] - 1) / 2 * count[a];
                    else
                        ans += (long) count[a] * count[b] * count[c];
                }

        return ans * 6;
    }

    private int divisible(int sum, int num) {
        return sum % num == 0 ? 1 : 0;
    }
}
```

1236. 22.java

Path: LeetCode-main/solutions/22. Generate Parentheses/22.java

```
class Solution {
    public List<String> generateParenthesis(int n) {
        List<String> ans = new ArrayList<>();
        dfs(n, n, new StringBuilder(), ans);
        return ans;
    }

    private void dfs(int l, int r, StringBuilder sb, List<String> ans) {
        if (l == 0 && r == 0) {
            ans.add(sb.toString());
            return;
        }

        if (l > 0) {
            sb.append("(");
            dfs(l - 1, r, sb, ans);
            sb.deleteCharAt(sb.length() - 1);
        }
        if (l < r) {
            sb.append(")");
            dfs(l, r - 1, sb, ans);
            sb.deleteCharAt(sb.length() - 1);
        }
    }
}
```

1237. 220-2.java

Path: LeetCode-main/solutions/220. Contains Duplicate III/220-2.java

```
class Solution {
    public boolean containsNearbyAlmostDuplicate(int[] nums, int indexDiff, int valueDiff) {
        if (nums.length == 0 || indexDiff <= 0 || valueDiff < 0)
            return false;

        final long mn = Arrays.stream(nums).min().getAsInt();
        final long diff = (long) valueDiff + 1; // In case that `valueDiff` equals 0.
        // Use Long because of corner case Integer.MAX_VALUE - (-1) will overflow
        HashMap<Long, Long> bucket = new HashMap<>();

        for (int i = 0; i < nums.length; ++i) {
            final long num = (long) nums[i];
            final long key = getKey(nums[i], mn, diff);
            if (bucket.containsKey(key)) // the current bucket
                return true;
            if (bucket.containsKey(key - 1) &&
                num - bucket.get(key - 1) < diff) // the left adjacent bucket
                return true;
            if (bucket.containsKey(key + 1) &&
                bucket.get(key + 1) - num < diff) // the right adjacent bucket
                return true;
            bucket.put(key, num);
            if (i >= indexDiff)
                bucket.remove(getKey(nums[i - indexDiff], mn, diff));
        }

        return false;
    }

    private long getKey(int num, long mn, long diff) {
        return ((long) num - mn) / diff;
    }
}
```

1238. 220.java

Path: LeetCode-main/solutions/220. Contains Duplicate III/220.java

```
class Solution {
    public boolean containsNearbyAlmostDuplicate(int[] nums, int indexDiff, int valueDiff) {
        TreeSet<Long> set = new TreeSet<>();

        for (int i = 0; i < nums.length; ++i) {
            final long num = (long) nums[i];
            final Long ceiling = set.ceiling(num); // The smallest num >= nums[i]
            if (ceiling != null && ceiling - num <= valueDiff)
                return true;
            final Long floor = set.floor(num); // The largest num <= nums[i]
            if (floor != null && num - floor <= valueDiff)
                return true;
            set.add(num);
            if (i >= indexDiff)
                set.remove((long) nums[i - indexDiff]);
        }

        return false;
    }
}
```

1239. 2200.java

Path: LeetCode-main/solutions/2200. Find All K-Distant Indices in an Array/2200.java

```
class Solution {
    public List<Integer> findKDistantIndices(int[] nums, int key, int k) {
        final int n = nums.length;
        List<Integer> ans = new ArrayList<>();

        for (int i = 0, j = 0; i < n; ++i) {
            // the first index j s.t. nums[j] == key and j >= i - k
            while (j < n && (nums[j] != key || j < i - k))
                ++j;
            if (j == n)
                break;
            if (Math.abs(i - j) <= k)
                ans.add(i);
        }

        return ans;
    }
}
```

1240. 2201.java

Path: LeetCode-main/solutions/2201. Count Artifacts That Can Be Extracted/2201.java

```
class Solution {
    public int digArtifacts(int n, int[][] artifacts, int[][] dig) {
        Set<Integer> digged = new HashSet<>();

        for (int[] d : dig)
            digged.add(hash(d[0], d[1]));

        return (int) Arrays.stream(artifacts).filter(a -> canExtract(a, digged)).count();
    }

    private int hash(int i, int j) {
        return i << 16 | j;
    }

    private boolean canExtract(int[] a, Set<Integer> digged) {
        for (int i = a[0]; i <= a[2]; ++i)
            for (int j = a[1]; j <= a[3]; ++j)
                if (!digged.contains(hash(i, j)))
                    return false;
        return true;
    }
}
```


1241. 2202.java

Path: LeetCode-main/solutions/2202. Maximize the Topmost Element After K Moves/2202.java

```
class Solution {
    public int maximumTop(int[] nums, int k) {
        final int n = nums.length;
        // After taking k elements, if there's something left, then return nums[k].
        // Otherwise, return -1>
        if (k == 0 || k == 1)
            return n == k ? -1 : nums[k];
        // Remove then add even number of times.
        if (n == 1)
            return k % 2 == 0 ? nums[0] : -1;
        // Take min(n, k - 1) elements and put the largest one back.
        final int mx = firstMAX(nums, k - 1);
        if (k >= n)
            return mx;
        return Math.max(mx, nums[k]);
    }

    private int firstMAX(int[] nums, int k) {
        int mx = 0;
        for (int i = 0; i < nums.length && i < k; ++i)
            mx = Math.max(mx, nums[i]);
        return mx;
    }
}
```

1242. 2203.java

Path: LeetCode-main/solutions/2203. Minimum Weighted Subgraph With the Required Paths/2203.java

```
class Solution {
    public long minimumWeight(int n, int[][] edges, int src1, int src2, int dest) {
        List<Pair<Integer, Integer>>[] graph = new List[n];
        List<Pair<Integer, Integer>>[] reversedGraph = new List[n];

        for (int i = 0; i < n; ++i) {
            graph[i] = new ArrayList<>();
            reversedGraph[i] = new ArrayList<>();
        }

        for (int[] edge : edges) {
            final int u = edge[0];
            final int v = edge[1];
            final int w = edge[2];
            graph[u].add(new Pair<>(v, w));
            reversedGraph[v].add(new Pair<>(u, w));
        }

        long[] fromSrc1 = dijkstra(graph, src1);
        long[] fromSrc2 = dijkstra(graph, src2);
        long[] fromDest = dijkstra(reversedGraph, dest);
        long ans = MAX;

        for (int i = 0; i < n; ++i) {
            if (fromSrc1[i] == MAX || fromSrc2[i] == MAX || fromDest[i] == MAX)
                continue;
            ans = Math.min(ans, fromSrc1[i] + fromSrc2[i] + fromDest[i]);
        }

        return ans == MAX ? -1 : ans;
    }

    private static long MAX = (long) 1e10;

    private long[] dijkstra(List<Pair<Integer, Integer>>[] graph, int src) {
        long[] dist = new long[graph.length];
        Arrays.fill(dist, MAX);

        dist[src] = 0;
        Queue<Pair<Long, Integer>> minHeap =
            new PriorityQueue<>(Comparator.comparingLong(Pair::getKey)) {
                {
                    offer(new Pair<>(dist[src], src)); // (d, u)
                }
            };

        while (!minHeap.isEmpty()) {
            final long d = minHeap.peek().getKey();
            final int u = minHeap.poll().getValue();
            if (d > dist[u])
                continue;
            for (Pair<Integer, Integer> pair : graph[u]) {
                final int v = pair.getKey();
                final int w = pair.getValue();
                if (d + w < dist[v]) {
                    dist[v] = d + w;
                    minHeap.offer(new Pair<>(dist[v], v));
                }
            }
        }

        return dist;
    }
}
```

1243. 2204.java

Path: LeetCode-main/solutions/2204. Distance to a Cycle in Undirected Graph/2204.java

```
class Solution {
    public int[] distanceToCycle(int n, int[][] edges) {
        int[] ans = new int[n];
        List<Integer>[] graph = new List[n];
        Arrays.setAll(graph, i -> new ArrayList<>());

        for (int[] edge : edges) {
            final int u = edge[0];
            final int v = edge[1];
            graph[u].add(v);
            graph[v].add(u);
        }

        // rank[i] := the minimum node that node i can reach with forward edges
        // Initialize with NO_RANK = -2 to indicate not visited.
        int[] rank = new int[n];
        Arrays.fill(rank, NO_RANK);
        List<Integer> cycle = new ArrayList<>();
        getRank(graph, 0, 0, rank, cycle);

        Queue<Integer> q = cycle.stream().collect(Collectors.toCollection(ArrayDeque::new));
        boolean[] seen = new boolean[n];
        for (final int u : cycle)
            seen[u] = true;

        for (int step = 1; !q.isEmpty(); ++step)
            for (int sz = q.size(); sz > 0; --sz) {
                final int u = q.poll();
                for (final int v : graph[u]) {
                    if (seen[v])
                        continue;
                    q.offer(v);
                    seen[v] = true;
                    ans[v] = step;
                }
            }

        return ans;
    }

    private static final int NO_RANK = -2;

    // The minRank that u can reach with forward edges
    private int getRank(List<Integer>[] graph, int u, int currRank, int[] rank, List<Integer> cycle) {
        if (rank[u] != NO_RANK) // The rank is already determined
            return rank[u];

        rank[u] = currRank;
        int minRank = currRank;

        for (final int v : graph[u]) {
            // Visited || parent (that's why NO_RANK = -2 instead of -1)
            if (rank[u] == rank.length || rank[v] == currRank - 1)
                continue;
            final int nextRank = getRank(graph, v, currRank + 1, rank, cycle);
            // NextRank should > currRank if there's no cycle
            if (nextRank <= currRank)
                cycle.add(v);
            minRank = Math.min(minRank, nextRank);
        }

        rank[u] = rank.length; // Mark as visited.
        return minRank;
    }
}
```

1244. 2206.java

Path: LeetCode-main/solutions/2206. Divide Array Into Equal Pairs/2206.java

```
class Solution {
    public boolean divideArray(int[] nums) {
        int[] count = new int[501];

        for (final int num : nums)
            ++count[num];

        return Arrays.stream(count).allMatch(c -> c % 2 == 0);
    }
}
```

1245. 2207.java

Path: LeetCode-main/solutions/2207. Maximize Number of Subsequences in a String/2207.java

```
class Solution {
    public long maximumSubsequenceCount(String text, String pattern) {
        long ans = 0;
        int count0 = 0; // the count of the letter pattern[0]
        int count1 = 0; // the count of the letter pattern[1]

        for (final char c : text.toCharArray()) {
            if (c == pattern.charAt(1)) {
                ans += count0;
                ++count1;
            }
            if (c == pattern.charAt(0))
                ++count0;
        }

        // It is optimal to add pattern[0] at the beginning or add pattern[1] at the
        // end of the text.
        return ans + Math.max(count0, count1);
    }
}
```

1246. 2208.java

Path: LeetCode-main/solutions/2208. Minimum Operations to Halve Array Sum/2208.java

```
class Solution {
    public int halveArray(int[] nums) {
        final double halfSum = Arrays.stream(nums).asDoubleStream().sum() / 2;
        int ans = 0;
        double runningSum = 0;
        Queue<Double> maxHeap = new PriorityQueue<>(Collections.reverseOrder());

        for (final double num : nums)
            maxHeap.offer(num);

        while (runningSum < halfSum) {
            final double maxValue = maxHeap.poll() / 2;
            runningSum += maxValue;
            maxHeap.offer(maxValue);
            ++ans;
        }

        return ans;
    }
}
```

1247. 2209-2.java

Path: LeetCode-main/solutions/2209. Minimum White Tiles After Covering With Carpets/2209-2.java

```
class Solution {
    public int minimumWhiteTiles(String floor, int numCarpets, int carpetLen) {
        final int n = floor.length();
        // dp[i][j] := the minimum number of visible white tiles of floor[i..n)
        // after covering at most j carpets
        int[][] dp = new int[n + 1][numCarpets + 1];

        for (int i = n - 1; i >= 0; --i)
            dp[i][0] = floor.charAt(i) - '0' + dp[i + 1][0];

        for (int i = n - 1; i >= 0; --i)
            for (int j = 1; j <= numCarpets; ++j) {
                final int cover = i + carpetLen < n ? dp[i + carpetLen][j - 1] : 0;
                final int skip = floor.charAt(i) - '0' + dp[i + 1][j];
                dp[i][j] = Math.min(cover, skip);
            }

        return dp[0][numCarpets];
    }
}
```

1248. 2209.java

Path: LeetCode-main/solutions/2209. Minimum White Tiles After Covering With Carpets/2209.java

```
class Solution {
    public int minimumWhiteTiles(String floor, int numCarpets, int carpetLen) {
        int[][] mem = new int[floor.length() + 1][numCarpets + 1];
        Arrays.stream(mem).forEach(A -> Arrays.fill(A, MAX));
        return minimumWhiteTiles(floor, 0, numCarpets, carpetLen, mem);
    }

    private static final int MAX = 1000;

    // Returns the minimum number of visible white tiles of floor[i..n) after
    // covering at most j carpets.
    int minimumWhiteTiles(final String floor, int i, int j, int carpetLen, int[][] mem) {
        if (j < 0)
            return MAX;
        if (i >= floor.length())
            return 0;
        if (mem[i][j] != MAX)
            return mem[i][j];
        return mem[i][j] =
            Math.min(minimumWhiteTiles(floor, i + carpetLen, j - 1, carpetLen, mem),
                    minimumWhiteTiles(floor, i + 1, j, carpetLen, mem) + floor.charAt(i) - '0');
    }
}
```


1249. 221-2.java

Path: LeetCode-main/solutions/221. Maximal Square/221-2.java

```
class Solution {
    public int maximalSquare(char[][] matrix) {
        final int m = matrix.length;
        final int n = matrix[0].length;
        int[] dp = new int[n];
        int maxLength = 0;
        int prev = 0; // dp[i - 1][j - 1]

        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j) {
                final int cache = dp[j];
                if (i == 0 || j == 0 || matrix[i][j] == '0')
                    dp[j] = matrix[i][j] == '1' ? 1 : 0;
                else
                    dp[j] = Math.min(prev, Math.min(dp[j], dp[j - 1])) + 1;
                maxLength = Math.max(maxLength, dp[j]);
                prev = cache;
            }

        return maxLength * maxLength;
    }
}
```

1250. 221.java

Path: LeetCode-main/solutions/221. Maximal Square/221.java

```
class Solution {
    public int maximalSquare(char[][] matrix) {
        final int m = matrix.length;
        final int n = matrix[0].length;
        int[][] dp = new int[m][n];
        int maxLength = 0;

        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j) {
                if (i == 0 || j == 0 || matrix[i][j] == '0')
                    dp[i][j] = matrix[i][j] == '1' ? 1 : 0;
                else
                    dp[i][j] = Math.min(dp[i - 1][j - 1], Math.min(dp[i - 1][j], dp[i][j - 1])) + 1;
                maxLength = Math.max(maxLength, dp[i][j]);
            }

        return maxLength * maxLength;
    }
}
```

1251. 2210.java

Path: LeetCode-main/solutions/2210. Count Hills and Valleys in an Array/2210.java

```
class Solution {
    public int countHillValley(int[] nums) {
        int ans = 0;
        int left = nums[0];

        for (int i = 1; i + 1 < nums.length; ++i)
            if (left < nums[i] && nums[i] > nums[i + 1] || // the hill
                left > nums[i] && nums[i] < nums[i + 1]) { // the valley
                ++ans;
                left = nums[i];
            }

        return ans;
    }
}
```

1252. 2211.java

Path: LeetCode-main/solutions/2211. Count Collisions on a Road/2211.java

```
class Solution {
    public int countCollisions(String directions) {
        int ans = 0;
        int l = 0;
        int r = directions.length() - 1;

        while (l < directions.length() && directions.charAt(l) == 'L')
            ++l;

        while (r >= 0 && directions.charAt(r) == 'R')
            --r;

        for (int i = l; i <= r; ++i)
            if (directions.charAt(i) != 'S')
                ++ans;

        return ans;
    }
}
```

1253. 2212.java

Path: LeetCode-main/solutions/2212. Maximum Points in an Archery Competition/2212.java

```
class Solution {
    public int[] maximumBobPoints(int numArrows, int[] aliceArrows) {
        final int FULL_MASK = (1 << 12) - 1;
        int maxPoint = 0;
        int maxMask = 0;

        for (int mask = 0; mask < FULL_MASK; ++mask) {
            Pair<Boolean, Integer> pair = getShorableAndPoint(mask, numArrows, aliceArrows);
            final boolean shorable = pair.getKey();
            final int point = pair.getValue();
            if (shorable && point > maxPoint) {
                maxPoint = point;
                maxMask = mask;
            }
        }

        return getBobsArrows(maxMask, numArrows, aliceArrows);
    }

    private Pair<Boolean, Integer> getShorableAndPoint(int mask, int leftArrows, int[] aliceArrows) {
        int point = 0;
        for (int i = 0; i < 12; ++i)
            if ((mask >> i & 1) == 1) {
                leftArrows -= aliceArrows[i] + 1;
                point += i;
            }
        return new Pair<>(leftArrows >= 0, point);
    }

    int[] getBobsArrows(int mask, int leftArrows, int[] aliceArrows) {
        int[] bobsArrows = new int[12];
        for (int i = 0; i < 12; ++i)
            if ((mask >> i & 1) == 1) {
                bobsArrows[i] = aliceArrows[i] + 1;
                leftArrows -= aliceArrows[i] + 1;
            }
        bobsArrows[0] = leftArrows;
        return bobsArrows;
    }
}
```

1254. 2214.java

Path: LeetCode-main/solutions/2214. Minimum Health to Beat Game/2214.java

```
class Solution {  
    public long minimumHealth(int[] damage, int armor) {  
        final long sum = Arrays.stream(damage).asLongStream().sum();  
        final int maxDamage = Arrays.stream(damage).max().getAsInt();  
        return 1 + sum - Math.min(maxDamage, armor);  
    }  
}
```

1255. 2215.java

Path: LeetCode-main/solutions/2215. Find the Difference of Two Arrays/2215.java

```
class Solution {
    public List<List<Integer>> findDifference(int[] nums1, int[] nums2) {
        Set<Integer> set1 = Arrays.stream(nums1).boxed().collect(Collectors.toSet());
        Set<Integer> set2 = Arrays.stream(nums2).boxed().collect(Collectors.toSet());
        Arrays.stream(nums1).forEach(set2::remove);
        Arrays.stream(nums2).forEach(set1::remove);
        return Arrays.asList(new ArrayList<>(set1), new ArrayList<>(set2));
    }
}
```

1256. 2216.java

Path: LeetCode-main/solutions/2216. Minimum Deletions to Make Array Beautiful/2216.java

```
class Solution {
    public int minDeletion(int[] nums) {
        int ans = 0;

        for (int i = 0; i + 1 < nums.length; ++i)
            // i - ans := the index after deletion
            if (nums[i] == nums[i + 1] && (i - ans) % 2 == 0)
                ++ans;

        return ans + (((nums.length - ans) & 1) == 1 ? 1 : 0);
    }
}
```


1257. 2217.java

Path: LeetCode-main/solutions/2217. Find Palindrome With Fixed Length/2217.java

```
class Solution {
    public long[] kthPalindrome(int[] queries, int intLength) {
        final int start = (int) Math.pow(10, (intLength + 1) / 2 - 1);
        final int end = (int) Math.pow(10, (intLength + 1) / 2);
        final int mul = (int) Math.pow(10, intLength / 2);
        long[] ans = new long[queries.length];

        for (int i = 0; i < queries.length; ++i)
            if (start + queries[i] > end)
                ans[i] = -1;
            else
                ans[i] = getKthPalindrome(queries[i], start, mul, intLength);

        return ans;
    }

    private long getKthPalindrome(int q, int start, int mul, int intLength) {
        final long prefix = start + q - 1;
        return prefix * mul + reverse(intLength % 2 == 0 ? prefix : prefix / 10);
    }

    long reverse(long num) {
        long res = 0;
        while (num > 0) {
            res = res * 10 + num % 10;
            num /= 10;
        }
        return res;
    }
}
```

1258. 2218.java

Path: LeetCode-main/solutions/2218. Maximum Value of K Coins From Piles/2218.java

```
class Solution {
    public int maxValueOfCoins(List<List<Integer>> piles, int k) {
        Integer[][] mem = new Integer[piles.size()][k + 1];
        return maxValueOfCoins(piles, 0, k, mem);
    }

    // Returns the maximum value of picking k coins from piles[i..n)
    private int maxValueOfCoins(List<List<Integer>> piles, int i, int k, Integer[][] mem) {
        if (i == piles.size() || k == 0)
            return 0;
        if (mem[i][k] != null)
            return mem[i][k];

        // Pick no coins from the current pile.
        int res = maxValueOfCoins(piles, i + 1, k, mem);
        int val = 0; // the coins picked from the current pile

        // Try to pick 1, 2, ..., k coins from the current pile.
        for (int j = 0; j < Math.min(piles.get(i).size(), k); ++j) {
            val += piles.get(i).get(j);
            res = Math.max(res, val + maxValueOfCoins(piles, i + 1, k - j - 1, mem));
        }

        return mem[i][k] = res;
    }
}
```

1259. 2219.java

Path: LeetCode-main/solutions/2219. Maximum Sum Score of Array/2219.java

```
class Solution {
    public long maximumSumScore(int[] nums) {
        long ans = Long.MIN_VALUE;
        long prefix = 0;
        long sum = Arrays.stream(nums).asLongStream().sum();

        for (final int num : nums) {
            prefix += num;
            ans = Math.max(ans, Math.max(prefix, sum - prefix + num));
        }

        return ans;
    }
}
```

1260. 222-2.java

Path: LeetCode-main/solutions/222. Count Complete Tree Nodes/222-2.java

```
class Solution {
    public int countNodes(TreeNode root) {
        if (root == null)
            return 0;

        TreeNode left = root;
        TreeNode right = root;
        int heightL = 0;
        int heightR = 0;

        while (left != null) {
            ++heightL;
            left = left.left;
        }

        while (right != null) {
            ++heightR;
            right = right.right;
        }

        if (heightL == heightR) // `root` is a complete tree.
            return (int) Math.pow(2, heightL) - 1;
        return 1 + countNodes(root.left) + countNodes(root.right);
    }
}
```

1261. 222.java

Path: LeetCode-main/solutions/222. Count Complete Tree Nodes/222.java

```
class Solution {  
    public int countNodes(TreeNode root) {  
        if (root == null)  
            return 0;  
        return 1 + countNodes(root.left) + countNodes(root.right);  
    }  
}
```

1262. 2220.java

Path: LeetCode-main/solutions/2220. Minimum Bit Flips to Convert Number/2220.java

```
class Solution {  
    public int minBitFlips(int start, int goal) {  
        return Integer.bitCount(start ^ goal);  
    }  
}
```

1263. 2221.java

Path: LeetCode-main/solutions/2221. Find Triangular Sum of an Array/2221.java

```
class Solution {
    public int triangularSum(int[] nums) {
        for (int sz = nums.length; sz > 0; --sz)
            for (int i = 0; i + 1 < sz; ++i)
                nums[i] = (nums[i] + nums[i + 1]) % 10;
        return nums[0];
    }
}
```

1264. 2222.java

Path: LeetCode-main/solutions/2222. Number of Ways to Select Buildings/2222.java

```
class Solution {
    public long numberOfWays(String s) {
        long ans = 0;
        // before[i] := the number of i before the current digit
        int[] before = new int[2];
        // after[i] := the number of i after the current digit
        int[] after = new int[2];
        after[0] = (int) s.chars().filter(c -> c == '0').count();
        after[1] = s.length() - after[0];

        for (final char c : s.toCharArray()) {
            final int num = c - '0';
            --after[num];
            if (num == 0)
                ans += before[1] * after[1];
            else
                ans += before[0] * after[0];
            ++before[num];
        }

        return ans;
    }
}
```


1265. 2223.java

Path: LeetCode-main/solutions/2223. Sum of Scores of Built Strings/2223.java

```
class Solution {
    public long sumScores(String s) {
        final int n = s.length();
        // https://cp-algorithms.com/string/z-function.html#implementation
        int[] z = new int[n];
        // [l, r] := the indices of the rightmost segment match
        int l = 0;
        int r = 0;

        for (int i = 1; i < n; ++i) {
            if (i < r)
                z[i] = Math.min(r - i, z[i - l]);
            while (i + z[i] < n && s.charAt(z[i]) == s.charAt(i + z[i]))
                ++z[i];
            if (i + z[i] > r) {
                l = i;
                r = i + z[i];
            }
        }

        return Arrays.stream(z).asLongStream().sum() + n;
    }
}
```

1266. 2224.java

Path: LeetCode-main/solutions/2224. Minimum Number of Operations to Convert Time/2224.java

```
class Solution {
    public int convertTime(String current, String correct) {
        final int[] ops = {60, 15, 5, 1};
        int diff = getMinutes(correct) - getMinutes(current);
        int ans = 0;

        for (final int op : ops) {
            ans += diff / op;
            diff %= op;
        }

        return ans;
    }

    private int getMinutes(final String s) {
        return Integer.parseInt(s.substring(0, 2)) * 60 + Integer.parseInt(s.substring(3));
    }
}
```

1267. 2225.java

Path: LeetCode-main/solutions/2225. Find Players With Zero or One Losses/2225.java

```
class Solution {
    public List<List<Integer>> findWinners(int[][] matches) {
        List<List<Integer>> ans = Arrays.asList(new ArrayList<>(), new ArrayList<>());
        Map<Integer, Integer> lossesCount = new TreeMap<>();

        for (int[] m : matches) {
            final int winner = m[0];
            final int loser = m[1];
            if (!lossesCount.containsKey(winner))
                lossesCount.put(winner, 0);
            lossesCount.merge(loser, 1, Integer::sum);
        }

        for (final int player : lossesCount.keySet()) {
            final int nLosses = lossesCount.get(player);
            if (nLosses < 2)
                ans.get(nLosses).add(player);
        }

        return ans;
    }
}
```

1268. 2226.java

Path: LeetCode-main/solutions/2226. Maximum Candies Allocated to K Children/2226.java

```
class Solution {
    public int maximumCandies(int[] candies, long k) {
        int l = 1;
        int r = (int) (Arrays.stream(candies).asLongStream().sum() / k);

        while (l < r) {
            final int m = (l + r) / 2;
            if (numChildren(candies, m) < k)
                r = m;
            else
                l = m + 1;
        }

        return numChildren(candies, l) >= k ? l : l - 1;
    }

    private long numChildren(int[] candies, int m) {
        return Arrays.stream(candies).asLongStream().reduce(0L, (subtotal, c) -> subtotal + c / m);
    }
}
```

1269. 2227-2.java

Path: LeetCode-main/solutions/2227. Encrypt and Decrypt Strings/2227-2.java

```
class Encrypter {
    public Encrypter(char[] keys, String[] values, String[] dictionary) {
        for (int i = 0; i < keys.length; ++i)
            keyToValue.put(keys[i], values[i]);

        for (final String word : dictionary)
            encryptedCount.merge(encrypt(word), 1, Integer::sum);
    }

    public String encrypt(String word1) {
        StringBuilder sb = new StringBuilder();
        for (final char c : word1.toCharArray())
            sb.append(keyToValue.get(c));
        return sb.toString();
    }

    public int decrypt(String word2) {
        return encryptedCount.getDefault(word2, 0);
    }

    private Map<Character, String> keyToValue = new HashMap<>();
    private Map<String, Integer> encryptedCount = new HashMap<>();
}
```

1270. 2227.java

Path: LeetCode-main/solutions/2227. Encrypt and Decrypt Strings/2227.java

```
class TrieNode {
    public TrieNode[] children = new TrieNode[26];
    public boolean isWord = false;
}

class Encrypter {
    public Encrypter(char[] keys, String[] values, String[] dictionary) {
        for (int i = 0; i < keys.length; ++i) {
            final char key = keys[i];
            final String value = values[i];
            keyToValue.put(key, value);
            valueToKeys.putIfAbsent(value, new ArrayList<>());
            valueToKeys.get(value).add(key);
        }
        for (final String word : dictionary)
            insert(word);
    }

    public String encrypt(String word1) {
        StringBuilder sb = new StringBuilder();
        for (final char c : word1.toCharArray())
            sb.append(keyToValue.get(c));
        return sb.toString();
    }

    public int decrypt(String word2) {
        return find(word2, 0, root);
    }

    private Map<Character, String> keyToValue = new HashMap<>();
    private Map<String, List<Character>> valueToKeys = new HashMap<>();
    private TrieNode root = new TrieNode();

    void insert(final String word) {
        TrieNode node = root;
        for (final char c : word.toCharArray()) {
            final int i = c - 'a';
            if (node.children[i] == null)
                node.children[i] = new TrieNode();
            node = node.children[i];
        }
        node.isWord = true;
    }

    int find(final String word, int i, TrieNode node) {
        final String value = word.substring(i, i + 2);
        if (!valueToKeys.containsKey(value))
            return 0;

        int ans = 0;
        if (i + 2 == word.length()) {
            for (final char key : valueToKeys.get(value)) {
                TrieNode child = node.children[key - 'a'];
                if (child != null && child.isWord)
                    ++ans;
            }
            return ans;
        }

        for (final char key : valueToKeys.get(value)) {
            if (node.children[key - 'a'] == null)
                continue;
            ans += find(word, i + 2, node.children[key - 'a']);
        }

        return ans;
    }
}
```

1271. 2229.java

Path: LeetCode-main/solutions/2229. Check if an Array Is Consecutive/2229.java

```
class Solution {  
    public boolean isConsecutive(int[] nums) {  
        final int n = nums.length;  
        final int mx = Arrays.stream(nums).max().getAsInt();  
        final int mn = Arrays.stream(nums).min().getAsInt();  
        return mx - mn + 1 == n && Arrays.stream(nums).boxed().collect(Collectors.toSet()).size() == n;  
    }  
}
```

1272. 223.java

Path: LeetCode-main/solutions/223. Rectangle Area/223.java

```
class Solution {
    public int computeArea(long A, long B, long C, long D, long E, long F, long G, long H) {
        final long x = Math.max(A, E) < Math.min(C, G) ? (Math.min(C, G) - Math.max(A, E)) : 0;
        final long y = Math.max(B, F) < Math.min(D, H) ? (Math.min(D, H) - Math.max(B, F)) : 0;
        return (int) ((C - A) * (D - B) + (G - E) * (H - F) - x * y);
    }
}
```


1273. 2231.java

Path: LeetCode-main/solutions/2231. Largest Number After Digit Swaps by Parity/2231.java

```
class Solution {
    public int largestInteger(int num) {
        final String s = String.valueOf(num);
        int ans = 0;
        // maxHeap[0] := the odd digits
        // maxHeap[1] := the even digits
        Queue<Integer>[] maxHeap = new Queue[2];

        for (int i = 0; i < 2; ++i)
            maxHeap[i] = new PriorityQueue<>(Comparator.reverseOrder());

        for (final char c : s.toCharArray()) {
            final int digit = c - '0';
            maxHeap[digit % 2].offer(digit);
        }

        for (final char c : s.toCharArray()) {
            final int i = c - '0' & 1;
            ans = (ans * 10 + maxHeap[i].poll());
        }

        return ans;
    }
}
```

1274. 2232.java

Path: LeetCode-main/solutions/2232. Minimize Result by Adding Parentheses to Expression/2232.java

```
class Solution {
    public String minimizeResult(String expression) {
        final int plusIndex = expression.indexOf('+');
        final String left = expression.substring(0, plusIndex);
        final String right = expression.substring(plusIndex + 1);
        String ans = "";
        int mn = Integer.MAX_VALUE;

        // the expression -> a * (b + c) * d
        for (int i = 0; i < left.length(); ++i)
            for (int j = 0; j < right.length(); ++j) {
                final int a = i == 0 ? 1 : Integer.parseInt(left.substring(0, i));
                final int b = Integer.parseInt(left.substring(i));
                final int c = Integer.parseInt(right.substring(0, j + 1));
                final int d = j == right.length() - 1 ? 1 : Integer.parseInt(right.substring(j + 1));
                final int val = a * (b + c) * d;
                if (val < mn) {
                    mn = val;
                    ans = (i == 0 ? "" : String.valueOf(a)) + '(' + String.valueOf(b) + '+' +
                        String.valueOf(c) + ')' + (j == right.length() - 1 ? "" : String.valueOf(d));
                }
            }
        return ans;
    }
}
```

1275. 2233.java

Path: LeetCode-main/solutions/2233. Maximum Product After K Increments/2233.java

```
class Solution {
    public int maximumProduct(int[] nums, int k) {
        final int MOD = 1_000_000_007;
        long ans = 1;
        Queue<Integer> minHeap = new PriorityQueue<>();

        for (final int num : nums)
            minHeap.offer(num);

        for (int i = 0; i < k; ++i) {
            final int minNum = minHeap.poll();
            minHeap.offer(minNum + 1);
        }

        while (!minHeap.isEmpty()) {
            ans *= minHeap.poll();
            ans %= MOD;
        }

        return (int) ans;
    }
}
```

1276. 2234.java

Path: LeetCode-main/solutions/2234. Maximum Total Beauty of the Gardens/2234.java

```
class Solution {
    public long maximumBeauty(int[] flowers, long newFlowers, int target, int full, int partial) {
        final int n = flowers.length;

        // If a garden is already complete, clamp it to the target.
        for (int i = 0; i < n; ++i)
            flowers[i] = Math.min(flowers[i], target);
        Arrays.sort(flowers);

        // All gardens are complete, so nothing we can do.
        if (flowers[0] == target)
            return (long) n * full;

        // Having many new flowers maximizes the beauty value.
        if (newFlowers >= (long) n * target - Arrays.stream(flowers).asLongStream().sum())
            return Math.max((long) n * full, (n - 1L) * full + (target - 1L) * partial);

        long ans = 0;
        long leftFlowers = newFlowers;
        // cost[i] := the cost to make flowers[0..i] the same
        long[] cost = new long[flowers.length];

        for (int i = 1; i < flowers.length; ++i)
            // Plant (flowers[i] - flowers[i - 1]) flowers for flowers[0..i - 1].
            cost[i] = cost[i - 1] + i * (flowers[i] - flowers[i - 1]);

        int i = flowers.length - 1; // flowers' index (flowers[i + 1..n] are complete)
        while (flowers[i] == target)
            --i;

        for (; leftFlowers >= 0; --i) {
            // To maximize the minimum number of incomplete flowers, we find the first
            // index j that we can't make flowers[0..j] equal to flowers[j], then we
            // know we can make flowers[0..j - 1] equal to flowers[j - 1]. In the
            // meantime, evenly increase each of them to seek a bigger minimum value.
            final int j = firstGreater(cost, i, leftFlowers);
            final long minIncomplete = flowers[j - 1] + (leftFlowers - cost[j - 1]) / j;
            ans = Math.max(ans, (n - 1L - i) * full + (long) minIncomplete * partial);
            leftFlowers -= Math.max(0, target - flowers[i]);
        }

        return ans;
    }

    private int firstGreater(long[] A, int maxIndex, long target) {
        final int i = Arrays.binarySearch(A, 0, maxIndex + 1, target + 1);
        return i < 0 ? -i - 1 : i;
    }
}
```

1277. 2235.java

Path: LeetCode-main/solutions/2235. Add Two Integers/2235.java

```
class Solution {  
    public int sum(int num1, int num2) {  
        return num1 + num2;  
    }  
}
```

1278. 2236.java

Path: LeetCode-main/solutions/2236. Root Equals Sum of Children/2236.java

```
class Solution {  
    public boolean checkTree(TreeNode root) {  
        return root.val == root.left.val + root.right.val;  
    }  
}
```

1279. 2237.java

Path: LeetCode-main/solutions/2237. Count Positions on Street With Required Brightness/2237.java

```
class Solution {
    public int meetRequirement(int n, int[][] lights, int[] requirement) {
        int ans = 0;
        int currBrightness = 0;
        int[] change = new int[n + 1];

        for (int[] light : lights) {
            final int position = light[0];
            final int range = light[1];
            ++change[Math.max(0, position - range)];
            --change[Math.min(n, position + range + 1)];
        }

        for (int i = 0; i < n; ++i) {
            currBrightness += change[i];
            if (currBrightness >= requirement[i])
                ++ans;
        }

        return ans;
    }
}
```

1280. 2239.java

Path: LeetCode-main/solutions/2239. Find Closest Number to Zero/2239.java

```
class Solution {
    public int findClosestNumber(int[] nums) {
        int ans = 0;
        int mn = Integer.MAX_VALUE;

        for (final int num : nums)
            if (Math.abs(num) < mn) {
                mn = Math.abs(num);
                ans = num;
            } else if (Math.abs(num) == mn && num > ans) {
                ans = num;
            }

        return ans;
    }
}
```


1281. 224.java

Path: LeetCode-main/solutions/224. Basic Calculator/224.java

```
class Solution {
    public int calculate(String s) {
        int ans = 0;
        int num = 0;
        int sign = 1;
        // stack.peek() := the current environment's sign
        Deque<Integer> stack = new ArrayDeque<>();
        stack.push(sign);

        for (final char c : s.toCharArray())
            if (Character.isDigit(c))
                num = num * 10 + (c - '0');
            else if (c == '(')
                stack.push(sign);
            else if (c == ')')
                stack.pop();
            else if (c == '+' || c == '-') {
                ans += sign * num;
                sign = (c == '+' ? 1 : -1) * stack.peek();
                num = 0;
            }
        return ans + sign * num;
    }
}
```

1282. 2240.java

Path: LeetCode-main/solutions/2240. Number of Ways to Buy Pens and Pencils/2240.java

```
class Solution {
    public long waysToBuyPensPencils(int total, int cost1, int cost2) {
        long ans = 0;
        final int maxPen = total / cost1;

        for (int i = 0; i <= maxPen; ++i)
            ans += (total - i * cost1) / cost2 + 1;

        return ans;
    }
}
```

1283. 2241.java

Path: LeetCode-main/solutions/2241. Design an ATM Machine/2241.java

```
class ATM {
    public void deposit(int[] banknotesCount) {
        for (int i = 0; i < 5; ++i)
            bank[i] += banknotesCount[i];
    }

    public int[] withdraw(int amount) {
        int[] withdrew = new int[5];

        for (int i = 4; i >= 0; --i) {
            withdrew[i] = (int) Math.min(bank[i], (long) amount / banknotes[i]);
            amount -= withdrew[i] * banknotes[i];
        }

        if (amount > 0)
            return new int[] {-1};

        for (int i = 0; i < 5; ++i)
            bank[i] -= withdrew[i];
        return withdrew;
    }

    private int[] banknotes = {20, 50, 100, 200, 500};
    private long[] bank = new long[5];
}
```

1284. 2242.java

Path: LeetCode-main/solutions/2242. Maximum Score of a Node Sequence/2242.java

```
class Solution {
    public int maximumScore(int[] scores, int[][] edges) {
        final int n = scores.length;
        int ans = -1;
        Queue<Integer>[] graph = new Queue[n];

        for (int i = 0; i < n; ++i)
            graph[i] = new PriorityQueue<>(Comparator.comparingInt(a -> scores[a]));

        for (int[] edge : edges) {
            final int u = edge[0];
            final int v = edge[1];
            graph[u].offer(v);
            graph[v].offer(u);
            if (graph[u].size() > 3)
                graph[u].poll();
            if (graph[v].size() > 3)
                graph[v].poll();
        }

        // To find the target sequence: a - u - v - b, enumerate each edge (u, v),
        // and find a (u's child) and b (v's child). That's why we find the 3
        // children that have the highest scores because one of the 3 children is
        // guaranteed to be valid.
        for (int[] edge : edges) {
            final int u = edge[0];
            final int v = edge[1];
            for (final int a : graph[u])
                for (final int b : graph[v])
                    if (a != b && a != v && b != u)
                        ans = Math.max(ans, scores[a] + scores[u] + scores[v] + scores[b]);
        }

        return ans;
    }
}
```

1285. 2243.java

Path: LeetCode-main/solutions/2243. Calculate Digit Sum of a String/2243.java

```
class Solution {
    public String digitSum(String s, int k) {
        while (s.length() > k) {
            StringBuilder sb = new StringBuilder();
            for (int i = 0; i < s.length(); i += k) {
                int sum = 0;
                for (int j = i; j < Math.min(s.length(), i + k); ++j)
                    sum += s.charAt(j) - '0';
                sb.append(sum);
            }
            s = sb.toString();
        }
        return s;
    }
}
```

1286. 2244.java

Path: LeetCode-main/solutions/2244. Minimum Rounds to Complete All Tasks/2244.java

```
class Solution {
    public int minimumRounds(int[] tasks) {
        int ans = 0;
        Map<Integer, Integer> count = new HashMap<>();

        for (final int task : tasks)
            count.merge(task, 1, Integer::sum);

        // freq = 1 -> it's impossible
        // freq = 2 -> needs 1 round
        // freq = 3 -> needs 1 round
        // freq = 3k -> needs k rounds
        // freq = 3k + 1 = 3 * (k - 1) + 2 * 2 -> needs k + 1 rounds
        // freq = 3k + 2 = 3 * k + 2 * 1 -> needs k + 1 rounds
        for (final int freq : count.values())
            if (freq == 1)
                return -1;
            else
                ans += (freq + 2) / 3;

        return ans;
    }
}
```

1287. 2245.java

Path: LeetCode-main/solutions/2245. Maximum Trailing Zeros in a Cornered Path/2245.java

```
class Solution {
    public int maxTrailingZeros(int[][] grid) {
        final int m = grid.length;
        final int n = grid[0].length;
        // leftPrefix2[i][j] := the number of 2 in grid[i][0..j]
        // leftPrefix5[i][j] := the number of 5 in grid[i][0..j]
        // topPrefix2[i][j] := the number of 2 in grid[0..i][j]
        // topPrefix5[i][j] := the number of 5 in grid[0..i][j]
        int[][] leftPrefix2 = new int[m][n];
        int[][] leftPrefix5 = new int[m][n];
        int[][] topPrefix2 = new int[m][n];
        int[][] topPrefix5 = new int[m][n];

        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j) {
                leftPrefix2[i][j] = getCount(grid[i][j], 2);
                leftPrefix5[i][j] = getCount(grid[i][j], 5);
                if (j > 0) {
                    leftPrefix2[i][j] += leftPrefix2[i][j - 1];
                    leftPrefix5[i][j] += leftPrefix5[i][j - 1];
                }
            }

        for (int j = 0; j < n; ++j)
            for (int i = 0; i < m; ++i) {
                topPrefix2[i][j] = getCount(grid[i][j], 2);
                topPrefix5[i][j] = getCount(grid[i][j], 5);
                if (i > 0) {
                    topPrefix2[i][j] += topPrefix2[i - 1][j];
                    topPrefix5[i][j] += topPrefix5[i - 1][j];
                }
            }

        int ans = 0;
        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j) {
                final int curr2 = getCount(grid[i][j], 2);
                final int curr5 = getCount(grid[i][j], 5);
                final int l2 = leftPrefix2[i][j];
                final int l5 = leftPrefix5[i][j];
                final int r2 = leftPrefix2[i][n - 1] - (j > 0 ? leftPrefix2[i][j - 1] : 0);
                final int r5 = leftPrefix5[i][n - 1] - (j > 0 ? leftPrefix5[i][j - 1] : 0);
                final int t2 = topPrefix2[i][j];
                final int t5 = topPrefix5[i][j];
                final int d2 = topPrefix2[m - 1][j] - (i > 0 ? topPrefix2[i - 1][j] : 0);
                final int d5 = topPrefix5[m - 1][j] - (i > 0 ? topPrefix5[i - 1][j] : 0);
                ans = Math.max(ans, Math.max(Math.max(Math.min(l2 + t2 - curr2, l5 + t5 - curr5),
                                                            Math.min(r2 + t2 - curr2, r5 + t5 - curr5)),
                                                            Math.max(Math.min(l2 + d2 - curr2, l5 + d5 - curr5),
                                                            Math.min(r2 + d2 - curr2, r5 + d5 - curr5))));
            }

        return ans;
    }

    private int getCount(int num, int factor) {
        int count = 0;
        while (num % factor == 0) {
            num /= factor;
            ++count;
        }
        return count;
    }
}
```

1288. 2246.java

Path: LeetCode-main/solutions/2246. Longest Path With Different Adjacent Characters/2246.java

```
class Solution {
    public int longestPath(int[] parent, String s) {
        final int n = parent.length;
        List<Integer>[] graph = new List[n];
        Arrays.setAll(graph, i -> new ArrayList<>());

        for (int i = 1; i < n; ++i)
            graph[parent[i]].add(i);

        longestPathDownFrom(graph, 0, s);
        return ans;
    }

    private int ans = 0;

    private int longestPathDownFrom(List<Integer>[] graph, int u, final String s) {
        int max1 = 0;
        int max2 = 0;

        for (final int v : graph[u]) {
            final int res = longestPathDownFrom(graph, v, s);
            if (s.charAt(u) == s.charAt(v))
                continue;
            if (res > max1) {
                max2 = max1;
                max1 = res;
            } else if (res > max2) {
                max2 = res;
            }
        }

        ans = Math.max(ans, 1 + max1 + max2);
        return 1 + max1;
    }
}
```


1289. 2247.java

Path: LeetCode-main/solutions/2247. Maximum Cost of Trip With K Highways/2247.java

```
class Solution {
    public int maximumCost(int n, int[][] highways, int k) {
        if (k + 1 > n)
            return -1;

        int ans = -1;
        Integer[][] mem = new Integer[n][1 << n];
        List<Pair<Integer, Integer>>[] graph = new List[n];
        Arrays.setAll(graph, i -> new ArrayList<>());

        for (int[] h : highways) {
            final int u = h[0];
            final int v = h[1];
            final int w = h[2];
            graph[u].add(new Pair<>(v, w));
            graph[v].add(new Pair<>(u, w));
        }

        for (int i = 0; i < n; ++i)
            ans = Math.max(ans, maximumCost(graph, i, 1 << i, k, mem));

        return ans;
    }

    // Returns the maximum cost of trip starting from u, where `mask` is the
    // bitmask of the visited cities.
    private int maximumCost(List<Pair<Integer, Integer>>[] graph, int u, int mask, int k,
        Integer[][] mem) {
        if (Integer.bitCount(mask) == k + 1)
            return 0;
        if (mem[u][mask] != null)
            return mem[u][mask];

        int res = -1;
        for (Pair<Integer, Integer> pair : graph[u]) {
            final int v = pair.getKey();
            final int w = pair.getValue();
            if ((mask >> v & 1) == 1)
                continue;
            final int nextCost = maximumCost(graph, v, mask | 1 << v, k, mem);
            if (nextCost != -1)
                res = Math.max(res, w + nextCost);
        }

        return mem[u][mask] = res;
    }
}
```

1290. 2248.java

Path: LeetCode-main/solutions/2248. Intersection of Multiple Arrays/2248.java

```
class Solution {
    public List<Integer> intersection(int[][] nums) {
        final int MAX = 1000;
        List<Integer> ans = new ArrayList<>();
        int[] count = new int[MAX + 1];

        for (int[] arr : nums)
            for (final int a : arr)
                ++count[a];

        for (int i = 1; i <= MAX; ++i)
            if (count[i] == nums.length)
                ans.add(i);

        return ans;
    }
}
```

1291. 2249-2.java

Path: LeetCode-main/solutions/2249. Count Lattice Points Inside a Circle/2249-2.java

```
class Solution {
    public int countLatticePoints(int[][] circles) {
        Set<Pair<Integer, Integer>> points = new HashSet<>();

        // dx := relative to x
        // dy := relative to y
        // So, dx^2 + dy^2 = r^2.
        for (int[] c : circles) {
            final int x = c[0];
            final int y = c[1];
            final int r = c[2];
            for (int dx = -r; dx <= r; ++dx) {
                final int yMax = (int) Math.sqrt(r * r - dx * dx);
                for (int dy = -yMax; dy <= yMax; ++dy)
                    points.add(new Pair<>(x + dx, y + dy));
            }
        }
        return points.size();
    }
}
```

1292. 2249.java

Path: LeetCode-main/solutions/2249. Count Lattice Points Inside a Circle/2249.java

```
class Solution {
    public int countLatticePoints(int[][] circles) {
        int ans = 0;

        for (int x = 0; x < 201; ++x)
            for (int y = 0; y < 201; ++y)
                for (int[] c : circles)
                    if ((c[0] - x) * (c[0] - x) + (c[1] - y) * (c[1] - y) <= c[2] * c[2]) {
                        ++ans;
                        break;
                    }

        return ans;
    }
}
```

1293. 225.java

Path: LeetCode-main/solutions/225. Implement Stack using Queues/225.java

```
class MyStack {
    public void push(int x) {
        q.offer(x);
        for (int i = 0; i < q.size() - 1; ++i)
            q.offer(q.poll());
    }

    public int pop() {
        return q.poll();
    }

    public int top() {
        return q.peek();
    }

    public boolean empty() {
        return q.isEmpty();
    }

    private Queue<Integer> q = new ArrayDeque<>();
}
```

1294. 2250.java

Path: LeetCode-main/solutions/2250. Count Number of Rectangles Containing Each Point/2250.java

```
class Solution {
    public int[] countRectangles(int[][] rectangles, int[][] points) {
        int[] ans = new int[points.length];
        List<Integer>[] yToXs = new List[101];

        for (int i = 0; i < 101; ++i)
            yToXs[i] = new ArrayList<>();

        for (int[] r : rectangles)
            yToXs[r[1]].add(r[0]);

        for (List<Integer> xs : yToXs)
            Collections.sort(xs);

        for (int i = 0; i < points.length; ++i) {
            int count = 0;
            for (int y = points[i][1]; y < 101; ++y) {
                List<Integer> xs = yToXs[y];
                count += xs.size() - firstGreaterEqual(xs, points[i][0]);
            }
            ans[i] = count;
        }

        return ans;
    }

    private int firstGreaterEqual(List<Integer> indices, int target) {
        final int i = Collections.binarySearch(indices, target);
        return i < 0 ? -i - 1 : i;
    }
}
```

1295. 2251.java

Path: LeetCode-main/solutions/2251. Number of Flowers in Full Bloom/2251.java

```
class Solution {
    public int[] fullBloomFlowers(int[][] flowers, int[] persons) {
        int[] ans = new int[persons.length];
        List<Integer> starts = new ArrayList<>();
        List<Integer> ends = new ArrayList<>();

        for (int[] flower : flowers) {
            starts.add(flower[0]);
            ends.add(flower[1]);
        }

        Collections.sort(starts);
        Collections.sort(ends);

        for (int i = 0; i < persons.length; ++i) {
            final int started = firstGreater(starts, persons[i]);
            final int ended = firstGreaterEqual(ends, persons[i]);
            ans[i] = started - ended;
        }

        return ans;
    }

    private int firstGreater(List<Integer> arr, int target) {
        int l = 0;
        int r = arr.size();
        while (l < r) {
            final int m = (l + r) / 2;
            if (arr.get(m) > target)
                r = m;
            else
                l = m + 1;
        }
        return l;
    }

    private int firstGreaterEqual(List<Integer> arr, int target) {
        int l = 0;
        int r = arr.size();
        while (l < r) {
            final int m = (l + r) / 2;
            if (arr.get(m) >= target)
                r = m;
            else
                l = m + 1;
        }
        return l;
    }
}
```

1296. 2254.java

Path: LeetCode-main/solutions/2254. Design Video Sharing Platform/2254.java

```
class VideoSharingPlatform {
    public int upload(String video) {
        final int videoId = getVideoId();
        videoIdToVideo.put(videoId, video);
        return videoId;
    }

    public void remove(int videoId) {
        if (videoIdToVideo.containsKey(videoId)) {
            usedIds.offer(videoId);
            videoIdToVideo.remove(videoId);
            videoIdToViews.remove(videoId);
            videoIdToLikes.remove(videoId);
            videoIdToDislikes.remove(videoId);
        }
    }

    public String watch(int videoId, int startMinute, int endMinute) {
        if (!videoIdToVideo.containsKey(videoId))
            return "-1";
        videoIdToViews.merge(videoId, 1, Integer::sum);
        final String video = videoIdToVideo.get(videoId);
        return video.substring(startMinute, Math.min(endMinute + 1, video.length()));
    }

    public void like(int videoId) {
        if (videoIdToVideo.containsKey(videoId))
            videoIdToLikes.merge(videoId, 1, Integer::sum);
    }

    public void dislike(int videoId) {
        if (videoIdToVideo.containsKey(videoId))
            videoIdToDislikes.merge(videoId, 1, Integer::sum);
    }

    public int[] getLikesAndDislikes(int videoId) {
        return videoIdToVideo.containsKey(videoId)
            ? new int[] {videoIdToLikes.getOrDefault(videoId, 0),
                        videoIdToDislikes.getOrDefault(videoId, 0)}
            : new int[] {-1};
    }

    public int getViews(int videoId) {
        return videoIdToVideo.containsKey(videoId) ? videoIdToViews.getOrDefault(videoId, 0) : -1;
    }

    private int currVideoId = 0;
    private Queue<Integer> usedIds = new PriorityQueue<>();
    private Map<Integer, String> videoIdToVideo = new HashMap<>();
    private Map<Integer, Integer> videoIdToViews = new HashMap<>();
    private Map<Integer, Integer> videoIdToLikes = new HashMap<>();
    private Map<Integer, Integer> videoIdToDislikes = new HashMap<>();

    private int getVideoId() {
        if (usedIds.isEmpty())
            return currVideoId++;
        return usedIds.poll();
    }
}
```


1297. 2255.java

Path: LeetCode-main/solutions/2255. Count Prefixes of a Given String/2255.java

```
class Solution {  
    public int countPrefixes(String[] words, String s) {  
        return (int) Arrays.stream(words).filter(word -> s.startsWith(word)).count();  
    }  
}
```

1298. 2256.java

Path: LeetCode-main/solutions/2256. Minimum Average Difference/2256.java

```
class Solution {
    public int minimumAverageDifference(int[] nums) {
        final int n = nums.length;
        int ans = 0;
        int minDiff = Integer.MAX_VALUE;
        long prefix = 0;
        long suffix = Arrays.stream(nums).asLongStream().sum();

        for (int i = 0; i < nums.length; ++i) {
            prefix += nums[i];
            suffix -= nums[i];
            final int prefixAvg = (int) (prefix / (i + 1));
            final int suffixAvg = (i == n - 1) ? 0 : (int) (suffix / (n - 1 - i));
            final int diff = Math.abs(prefixAvg - suffixAvg);
            if (diff < minDiff) {
                ans = i;
                minDiff = diff;
            }
        }

        return ans;
    }
}
```

1299. 2257.java

Path: LeetCode-main/solutions/2257. Count Unguarded Cells in the Grid/2257.java

```

class Solution {
public int countUnguarded(int m, int n, int[][] guards, int[][] walls) {
    int ans = 0;
    char[][] grid = new char[m][n];
    char[][] left = new char[m][n];
    char[][] right = new char[m][n];
    char[][] up = new char[m][n];
    char[][] down = new char[m][n];

    for (int[] guard : guards)
        grid[guard[0]][guard[1]] = 'G';

    for (int[] wall : walls)
        grid[wall[0]][wall[1]] = 'W';

    for (int i = 0; i < m; ++i) {
        char lastCell = 0;
        for (int j = 0; j < n; ++j)
            if (grid[i][j] == 'G' || grid[i][j] == 'W')
                lastCell = grid[i][j];
            else
                left[i][j] = lastCell;
        lastCell = 0;
        for (int j = n - 1; j >= 0; --j)
            if (grid[i][j] == 'G' || grid[i][j] == 'W')
                lastCell = grid[i][j];
            else
                right[i][j] = lastCell;
    }

    for (int j = 0; j < n; ++j) {
        char lastCell = 0;
        for (int i = 0; i < m; ++i)
            if (grid[i][j] == 'G' || grid[i][j] == 'W')
                lastCell = grid[i][j];
            else
                up[i][j] = lastCell;
        lastCell = 0;
        for (int i = m - 1; i >= 0; --i)
            if (grid[i][j] == 'G' || grid[i][j] == 'W')
                lastCell = grid[i][j];
            else
                down[i][j] = lastCell;
    }

    for (int i = 0; i < m; ++i)
        for (int j = 0; j < n; ++j)
            if (grid[i][j] == 0 && left[i][j] != 'G' && right[i][j] != 'G' && up[i][j] != 'G' &&
                down[i][j] != 'G')
                ++ans;

    return ans;
}
}

```

1300. 2258.java

Path: LeetCode-main/solutions/2258. Escape the Spreading Fire/2258.java

```
class Solution {
    public int maximumMinutes(int[][] grid) {
        final int MAX = grid.length * grid[0].length;
        int[][] fireMinute = new int[grid.length][grid[0].length];
        Arrays.stream(fireMinute).forEach(A -> Arrays.fill(A, -1));
        buildFireGrid(grid, fireMinute);

        int ans = -1;
        int l = 0;
        int r = MAX;

        while (l <= r) {
            final int m = (l + r) / 2;
            if (canStayFor(grid, fireMinute, m)) {
                ans = m;
                l = m + 1;
            } else {
                r = m - 1;
            }
        }

        return ans == MAX ? 1_000_000_000 : ans;
    }

    private static final int[][] DIRS = {{0, 1}, {1, 0}, {0, -1}, {-1, 0}};

    private void buildFireGrid(int[][] grid, int[][] fireMinute) {
        Queue<Pair<Integer, Integer>> q = new ArrayDeque<>();

        for (int i = 0; i < grid.length; ++i)
            for (int j = 0; j < grid[0].length; ++j)
                if (grid[i][j] == 1) { // the fire
                    q.offer(new Pair<>(i, j));
                    fireMinute[i][j] = 0;
                }

        for (int minuteFromFire = 1; !q.isEmpty(); ++minuteFromFire)
            for (int sz = q.size(); sz > 0; --sz) {
                final int i = q.peek().getKey();
                final int j = q.poll().getValue();
                for (int[] dir : DIRS) {
                    final int x = i + dir[0];
                    final int y = j + dir[1];
                    if (x < 0 || x == grid.length || y < 0 || y == grid[0].length)
                        continue;
                    if (grid[x][y] == 2) // the wall
                        continue;
                    if (fireMinute[x][y] != -1)
                        continue;
                    fireMinute[x][y] = minuteFromFire;
                    q.offer(new Pair<>(x, y));
                }
            }
    }

    boolean canStayFor(int[][] grid, int[][] fireMinute, int minute) {
        Queue<Pair<Integer, Integer>> q =
            new ArrayDeque<>(List.of(new Pair<>(0, 0))); // the start position
        boolean[][] seen = new boolean[grid.length][grid[0].length];
        seen[0][0] = true;

        while (!q.isEmpty()) {
            ++minute;
            for (int sz = q.size(); sz > 0; --sz) {
                final int i = q.peek().getKey();
                final int j = q.poll().getValue();
                for (int[] dir : DIRS) {
                    final int x = i + dir[0];
                    final int y = j + dir[1];
                    if (x < 0 || x == grid.length || y < 0 || y == grid[0].length)
                        continue;
                    if (grid[x][y] == 2) // the wall
                        continue;
                    if (x == grid.length - 1 && y == grid[0].length - 1) {
                        if (fireMinute[x][y] != -1 && fireMinute[x][y] < minute)
                            continue;
                        return true;
                    }
                }
                if (fireMinute[x][y] != -1 && fireMinute[x][y] <= minute)
                    continue;
            }
        }
    }
}
```

```
        if (seen[x][y])
            continue;
        q.offer(new Pair<>(x, y));
        seen[x][y] = true;
    }
}
return false;
}
```

1301. 2259.java

Path: LeetCode-main/solutions/2259. Remove Digit From Number to Maximize Result/2259.java

```
class Solution {
    public String removeDigit(String number, char digit) {
        for (int i = 0; i + 1 < number.length(); ++i)
            if (number.charAt(i) == digit && digit < number.charAt(i + 1))
                return new StringBuilder(number).deleteCharAt(i).toString();
        return new StringBuilder(number).deleteCharAt(number.lastIndexOf(digit)).toString();
    }
}
```

1302. 226.java

Path: LeetCode-main/solutions/226. Invert Binary Tree/226.java

```
class Solution {  
    public TreeNode invertTree(TreeNode root) {  
        if (root == null)  
            return null;  
  
        TreeNode left = root.left;  
        TreeNode right = root.right;  
        root.left = invertTree(right);  
        root.right = invertTree(left);  
        return root;  
    }  
}
```

1303. 2260.java

Path: LeetCode-main/solutions/2260. Minimum Consecutive Cards to Pick Up/2260.java

```
class Solution {
    public int minimumCardPickup(int[] cards) {
        int ans = Integer.MAX_VALUE;
        Map<Integer, Integer> lastSeen = new HashMap<>();

        for (int i = 0; i < cards.length; ++i) {
            if (lastSeen.containsKey(cards[i]))
                ans = Math.min(ans, i - lastSeen.get(cards[i]) + 1);
            lastSeen.put(cards[i], i);
        }

        return ans == Integer.MAX_VALUE ? -1 : ans;
    }
}
```


1304. 2261.java

Path: LeetCode-main/solutions/2261. K Divisible Elements Subarrays/2261.java

```
class TrieNode {
    public Map<Integer, TrieNode> children = new HashMap<>();
    public int count = 0;
}

class Solution {
    public int countDistinct(int[] nums, int k, int p) {
        for (int i = 0; i < nums.length; ++i)
            insert(root, nums, i, k, p);
        return ans;
    }

    private int ans = 0;
    private TrieNode root = new TrieNode();

    private void insert(TrieNode node, int[] nums, int i, int k, int p) {
        if (i == nums.length || k - (nums[i] % p == 0 ? 1 : 0) < 0)
            return;
        if (!node.children.containsKey(nums[i])) {
            node.children.put(nums[i], new TrieNode());
            ++ans;
        }
        insert(node.children.get(nums[i]), nums, i + 1, k - (nums[i] % p == 0 ? 1 : 0), p);
    }
}
```

1305. 2262-2.java

Path: LeetCode-main/solutions/2262. Total Appeal of A String/2262-2.java

```
class Solution {
    public long appealSum(String s) {
        final int n = s.length();
        long ans = 0;
        int[] lastSeen = new int[26];
        Arrays.fill(lastSeen, -1);

        for (int i = 0; i < n; ++i) {
            final int c = s.charAt(i) - 'a';
            ans += (i - lastSeen[c]) * (n - i);
            lastSeen[c] = i;
        }

        return ans;
    }
}
```

1306. 2262.java

Path: LeetCode-main/solutions/2262. Total Appeal of A String/2262.java

```
class Solution {
    public long appealSum(String s) {
        long ans = 0;
        // the total appeal of all substrings ending in the index so far
        int dp = 0;
        int[] lastSeen = new int[26];
        Arrays.fill(lastSeen, -1);

        for (int i = 0; i < s.length(); ++i) {
            // the total appeal of all substrings ending in s[i]
            // = the total appeal of all substrings ending in s[i - 1]
            // + the number of substrings ending in s[i] that contain only this s[i]
            final int c = s.charAt(i) - 'a';
            dp += i - lastSeen[c];
            ans += dp;
            lastSeen[c] = i;
        }

        return ans;
    }
}
```

1307. 2263.java

Path: LeetCode-main/solutions/2263. Make Array Non-decreasing or Non-increasing/2263.java

```
class Solution {
    public int convertArray(int[] nums) {
        return Math.min(cost(nums), cost(negative(nums)));
    }

    private int cost(int[] nums) {
        int ans = 0;
        Queue<Integer> maxHeap = new PriorityQueue<>(Comparator.reverseOrder());

        // Greedily make `nums` non-decreasing.
        for (final int num : nums) {
            if (!maxHeap.isEmpty() && maxHeap.peek() > num) {
                ans += maxHeap.poll() - num;
                maxHeap.offer(num);
            }
            maxHeap.offer(num);
        }

        return ans;
    }

    private int[] negative(int[] nums) {
        int[] arr = nums.clone();
        for (int i = 0; i < arr.length; ++i)
            arr[i] *= -1;
        return arr;
    }
}
```

1308. 2264.java

Path: LeetCode-main/solutions/2264. Largest 3-Same-Digit Number in String/2264.java

```
class Solution {
    public String largestGoodInteger(String num) {
        String ans = "";

        for (int i = 2; i < num.length(); ++i)
            if (num.charAt(i - 2) == num.charAt(i - 1) && num.charAt(i - 1) == num.charAt(i) &&
                num.substring(i - 2, i + 1).compareTo(ans) > 0)
                ans = num.substring(i - 2, i + 1);

        return ans;
    }
}
```

1309. 2265.java

Path: LeetCode-main/solutions/2265. Count Nodes Equal to Average of Subtree/2265.java

```
class Solution {
    public int averageOfSubtree(TreeNode root) {
        dfs(root);
        return ans;
    }

    private int ans = 0;

    private Pair<Integer, Integer> dfs(TreeNode root) {
        if (root == null)
            return new Pair<>(0, 0);
        Pair<Integer, Integer> left = dfs(root.left);
        Pair<Integer, Integer> right = dfs(root.right);
        final int sum = root.val + left.getKey() + right.getKey();
        final int count = 1 + left.getValue() + right.getValue();
        if (sum / count == root.val)
            ++ans;
        return new Pair<>(sum, count);
    }
}
```

1310. 2266.java

Path: LeetCode-main/solutions/2266. Count Number of Texts/2266.java

```
class Solution {
    public int countTexts(String pressedKeys) {
        final int MOD = 1_000_000_007;
        final int n = pressedKeys.length();
        // dp[i] := the number of possible text messages of pressedKeys[i..n)
        long[] dp = new long[n + 1];
        dp[n] = 1; // ""

        for (int i = n - 1; i >= 0; --i) {
            dp[i] = dp[i + 1];
            if (isSame(pressedKeys, i, 2))
                dp[i] += dp[i + 2];
            if (isSame(pressedKeys, i, 3))
                dp[i] += dp[i + 3];
            if ((pressedKeys.charAt(i) == '7' || pressedKeys.charAt(i) == '9') &&
                isSame(pressedKeys, i, 4))
                dp[i] += dp[i + 4];
            dp[i] %= MOD;
        }

        return (int) dp[0];
    }

    // Returns true if s[i..i + k) are the same digits.
    private boolean isSame(final String s, int i, int k) {
        if (i + k > s.length())
            return false;
        for (int j = i + 1; j < i + k; ++j)
            if (s.charAt(j) != s.charAt(i))
                return false;
        return true;
    }
}
```

1311. 2267.java

Path: LeetCode-main/solutions/2267. Check if There Is a Valid Parentheses String Path/2267.java

```
class Solution {
    public boolean hasValidPath(char[][] grid) {
        final int m = grid.length;
        final int n = grid[0].length;
        Boolean[][][] mem = new Boolean[m][n][m + n];
        return hasValidPath(grid, 0, 0, 0, mem);
    }

    // Returns true if there's a path from grid[i][j] to grid[m - 1][n - 1], where
    // the number of '(' - the number of ')' == k.
    private boolean hasValidPath(char[][] grid, int i, int j, int k, Boolean[][][] mem) {
        if (i == grid.length || j == grid[0].length)
            return false;
        k += grid[i][j] == '(' ? 1 : -1;
        if (k < 0)
            return false;
        if (i == grid.length - 1 && j == grid[0].length - 1)
            return k == 0;
        if (mem[i][j][k] != null)
            return mem[i][j][k];
        return mem[i][j][k] = hasValidPath(grid, i + 1, j, k, mem) | //
            hasValidPath(grid, i, j + 1, k, mem);
    }
}
```


1312. 2268.java

Path: LeetCode-main/solutions/2268. Minimum Number of Keypresses/2268.java

```
class Solution {
    public int minimumKeypresses(String s) {
        int ans = 0;
        Integer[] count = new Integer[26];
        Arrays.fill(count, 0);

        for (final char c : s.toCharArray())
            ++count[c - 'a'];

        Arrays.sort(count, Comparator.reverseOrder());

        for (int i = 0; i < 26; ++i)
            ans += count[i] * (i / 9 + 1);

        return ans;
    }
}
```

1313. 2269.java

Path: LeetCode-main/solutions/2269. Find the K-Beauty of a Number/2269.java

```
class Solution {
    public int divisorSubstrings(int num, int k) {
        final String s = String.valueOf(num);
        int ans = 0;

        for (int i = 0; i + k <= s.length(); ++i) {
            final int x = Integer.parseInt(s.substring(i, i + k));
            if (x != 0 && num % x == 0)
                ++ans;
        }

        return ans;
    }
}
```

1314. 227-2.java

Path: LeetCode-main/solutions/227. Basic Calculator II/227-2.java

```
class Solution {
    public int calculate(String s) {
        int ans = 0;
        int currNum = 0;
        int prevNum = 0;
        char op = '+';

        for (int i = 0; i < s.length(); ++i) {
            final char c = s.charAt(i);
            if (Character.isDigit(c))
                currNum = currNum * 10 + (c - '0');
            if (!Character.isDigit(c) && c != ' ' || i == s.length() - 1) {
                if (op == '+' || op == '-') {
                    ans += prevNum;
                    prevNum = op == '+' ? currNum : -currNum;
                } else if (op == '*') {
                    prevNum *= currNum;
                } else if (op == '/') {
                    prevNum /= currNum;
                }
                op = c;
                currNum = 0;
            }
        }

        return ans + prevNum;
    }
}
```

1315. 227.java

Path: LeetCode-main/solutions/227. Basic Calculator II/227.java

```
class Solution {
    public int calculate(String s) {
        Deque<Integer> nums = new ArrayDeque<>();
        Deque<Character> ops = new ArrayDeque<>();

        for (int i = 0; i < s.length(); ++i) {
            final char c = s.charAt(i);
            if (Character.isDigit(c)) {
                int num = c - '0';
                while (i + 1 < s.length() && Character.isDigit(s.charAt(i + 1))) {
                    num = num * 10 + (s.charAt(i + 1) - '0');
                    ++i;
                }
                nums.push(num);
            } else if (c == '+' || c == '-' || c == '*' || c == '/') {
                while (!ops.isEmpty() && compare(ops.peek(), c))
                    nums.push(calculate(ops.pop(), nums.pop(), nums.pop()));
                ops.push(c);
            }
        }

        while (!ops.isEmpty())
            nums.push(calculate(ops.pop(), nums.pop(), nums.pop()));

        return nums.peek();
    }

    private int calculate(char op, int b, int a) {
        switch (op) {
            case '+':
                return a + b;
            case '-':
                return a - b;
            case '*':
                return a * b;
            case '/':
                return a / b;
        }
        throw new IllegalArgumentException();
    }

    // Returns true if priority(op1) >= priority(op2).
    private boolean compare(char op1, char op2) {
        return op1 == '*' || op1 == '/' || op2 == '+' || op2 == '-';
    }
}
```

1316. 2270.java

Path: LeetCode-main/solutions/2270. Number of Ways to Split Array/2270.java

```
class Solution {
    public int waysToSplitArray(int[] nums) {
        int ans = 0;
        long prefix = 0;
        long suffix = Arrays.stream(nums).asLongStream().sum();

        for (int i = 0; i < nums.length - 1; ++i) {
            prefix += nums[i];
            suffix -= nums[i];
            if (prefix >= suffix)
                ++ans;
        }

        return ans;
    }
}
```

1317. 2271.java

Path: LeetCode-main/solutions/2271. Maximum White Tiles Covered by a Carpet/2271.java

```
class Solution {
    public int maximumWhiteTiles(int[][] tiles, int carpetLen) {
        if (Arrays.stream(tiles).anyMatch(tile -> tile[1] - tile[0] + 1 >= carpetLen))
            return carpetLen;

        int ans = 0;
        List<Integer> starts = new ArrayList<>();
        int[] prefix = new int[tiles.length + 1];

        Arrays.sort(tiles, Comparator.comparingInt(tile -> tile[0]));

        for (int[] tile : tiles)
            starts.add(tile[0]);

        for (int i = 0; i < tiles.length; ++i) {
            final int length = tiles[i][1] - tiles[i][0] + 1;
            prefix[i + 1] = prefix[i] + length;
        }

        for (int i = 0; i < tiles.length; ++i) {
            final int s = tiles[i][0];
            final int carpetEnd = s + carpetLen - 1;
            final int endIndex = firstGreater(starts, carpetEnd) - 1;
            final int notCover = Math.max(0, tiles[endIndex][1] - carpetEnd);
            ans = Math.max(ans, prefix[endIndex + 1] - prefix[i] - notCover);
        }

        return ans;
    }

    private int firstGreater(List<Integer> arr, int target) {
        final int i = Collections.binarySearch(arr, target + 1);
        return i < 0 ? -i - 1 : i;
    }
}
```

1318. 2272.java

Path: LeetCode-main/solutions/2272. Substring With Largest Variance/2272.java

```
class Solution {
    public int largestVariance(String s) {
        int ans = 0;

        for (char a = 'a'; a <= 'z'; ++a)
            for (char b = 'a'; b <= 'z'; ++b)
                if (a != b)
                    ans = Math.max(ans, kadane(s, a, b));

        return ans;
    }

    // a := the letter with the higher frequency
    // b := the letter with the lower frequency
    private int kadane(final String s, char a, char b) {
        int ans = 0;
        int countA = 0;
        int countB = 0;
        boolean canExtendPrevB = false;

        for (final char c : s.toCharArray()) {
            if (c != a && c != b)
                continue;
            if (c == a)
                ++countA;
            else
                ++countB;
            if (countB > 0) {
                // An interval should contain at least one b.
                ans = Math.max(ans, countA - countB);
            } else if (countB == 0 && canExtendPrevB) {
                // edge case: consider the previous b
                ans = Math.max(ans, countA - 1);
            }
            // Reset if the number of b > the number of a.
            if (countB > countA) {
                countA = 0;
                countB = 0;
                canExtendPrevB = true;
            }
        }

        return ans;
    }
}
```

1319. 2273.java

Path: LeetCode-main/solutions/2273. Find Resultant Array After Removing Anagrams/2273.java

```
class Solution {
    public List<String> removeAnagrams(String[] words) {
        List<String> ans = new ArrayList<>();

        for (int i = 0; i < words.length; i) {
            int j = i + 1;
            while (j < words.length && isAnagram(words[i], words[j]))
                ++j;
            ans.add(words[i]);
            i = j;
        }

        return ans;
    }

    private boolean isAnagram(final String a, final String b) {
        if (a.length() != b.length())
            return false;

        int[] count = new int[26];

        for (final char c : a.toCharArray())
            ++count[c - 'a'];

        for (final char c : b.toCharArray())
            --count[c - 'a'];

        return Arrays.stream(count).allMatch(c -> c == 0);
    }
}
```


1320. 2274.java

Path: LeetCode-main/solutions/2274. Maximum Consecutive Floors Without Special Floors/2274.java

```
class Solution {
    public int maxConsecutive(int bottom, int top, int[] special) {
        int ans = 0;

        Arrays.sort(special);

        for (int i = 1; i < special.length; ++i)
            ans = Math.max(ans, special[i] - special[i - 1] - 1);

        return Math.max(ans, Math.max(special[0] - bottom, top - special[special.length - 1]));
    }
}
```

1321. 2275.java

Path: LeetCode-main/solutions/2275. Largest Combination With Bitwise AND Greater Than Zero/2275.java

```
class Solution {
    public int largestCombination(int[] candidates) {
        final int MAX_BIT = 24;
        int ans = 0;

        for (int i = 0; i < MAX_BIT; ++i) {
            int count = 0;
            for (final int candidate : candidates)
                if ((candidate >> i & 1) == 1)
                    ++count;
            ans = Math.max(ans, count);
        }

        return ans;
    }
}
```

1322. 2276.java

Path: LeetCode-main/solutions/2276. Count Integers in Intervals/2276.java

```
class CountIntervals {
    public void add(int left, int right) {
        while (isOverlapped(left, right)) {
            final int l = intervals.floorKey(right);
            final int r = intervals.get(l);
            left = Math.min(left, l);
            right = Math.max(right, r);
            intervals.remove(l);
            count -= r - l + 1;
        }

        intervals.put(left, right);
        count += right - left + 1;
    }

    public int count() {
        return count;
    }

    private TreeMap<Integer, Integer> intervals = new TreeMap<>();
    private int count = 0;

    private boolean isOverlapped(int left, int right) {
        // L := the maximum number <= end
        Integer l = intervals.floorKey(right);
        return l != null && intervals.get(l) >= left;
    }
}
```

1323. 2277.java

Path: LeetCode-main/solutions/2277. Closest Node to Path in Tree/2277.java

```
class Solution {
    public int[] closestNode(int n, int[][] edges, int[][] query) {
        int[] ans = new int[query.length];
        List<Integer>[] tree = new List[n];
        int[][] dist = new int[n][n];
        Arrays.stream(dist).forEach(A -> Arrays.fill(A, -1));

        for (int i = 0; i < n; ++i)
            tree[i] = new ArrayList<>();

        for (int[] edge : edges) {
            final int u = edge[0];
            final int v = edge[1];
            tree[u].add(v);
            tree[v].add(u);
        }

        for (int i = 0; i < n; ++i)
            fillDist(tree, i, i, 0, dist);

        for (int i = 0; i < query.length; ++i) {
            final int start = query[i][0];
            final int end = query[i][1];
            final int node = query[i][2];
            ans[i] = findClosest(tree, dist, start, end, node, start);
        }

        return ans;
    }

    private void fillDist(List<Integer>[] tree, int start, int u, int d, int[][] dist) {
        dist[start][u] = d;
        for (final int v : tree[u])
            if (dist[start][v] == -1)
                fillDist(tree, start, v, d + 1, dist);
    }

    private int findClosest(List<Integer>[] tree, int[][] dist, int u, int end, int node, int ans) {
        for (final int v : tree[u])
            if (dist[v][end] < dist[u][end])
                return findClosest(tree, dist, v, end, node, dist[ans][node] < dist[v][node] ? ans : v);
        return ans;
    }
}
```

1324. 2278.java

Path: LeetCode-main/solutions/2278. Percentage of Letter in String/2278.java

```
class Solution {  
    public int percentageLetter(String s, char letter) {  
        return 100 * (int) s.chars().filter(c -> c == letter).count() / s.length();  
    }  
}
```

1325. 2279.java

Path: LeetCode-main/solutions/2279. Maximum Bags With Full Capacity of Rocks/2279.java

```
class Solution {
    public int maximumBags(int[] capacity, int[] rocks, int additionalRocks) {
        final int n = capacity.length;
        int[] diff = new int[n];

        for (int i = 0; i < n; ++i)
            diff[i] = capacity[i] - rocks[i];

        Arrays.sort(diff);

        for (int i = 0; i < n; ++i) {
            if (diff[i] > additionalRocks)
                return i;
            additionalRocks -= diff[i];
        }

        return n;
    }
}
```

1326. 228.java

Path: LeetCode-main/solutions/228. Summary Ranges/228.java

```
class Solution {
    public List<String> summaryRanges(int[] nums) {
        List<String> ans = new ArrayList<>();

        for (int i = 0; i < nums.length; ++i) {
            final int begin = nums[i];
            while (i + 1 < nums.length && nums[i] == nums[i + 1] - 1)
                ++i;
            final int end = nums[i];
            if (begin == end)
                ans.add("" + begin);
            else
                ans.add("" + begin + "->" + end);
        }

        return ans;
    }
}
```

1327. 2280.java

Path: LeetCode-main/solutions/2280. Minimum Lines to Represent a Line Chart/2280.java

```
class Solution {
    public int minimumLines(int[][] stockPrices) {
        int ans = 0;

        Arrays.sort(stockPrices, Comparator.comparingInt(stockPrice -> stockPrice[0]));

        for (int i = 2; i < stockPrices.length; ++i) {
            Pair<Integer, Integer> a = getSlope(stockPrices[i - 2], stockPrices[i - 1]);
            Pair<Integer, Integer> b = getSlope(stockPrices[i - 1], stockPrices[i]);
            if (a.getKey() != b.getKey() || a.getValue() != b.getValue())
                ++ans;
        }

        return ans + (stockPrices.length > 1 ? 1 : 0);
    }

    private Pair<Integer, Integer> getSlope(int[] p, int[] q) {
        final int dx = p[0] - q[0];
        final int dy = p[1] - q[1];
        if (dx == 0)
            return new Pair<>(0, p[0]);
        if (dy == 0)
            return new Pair<>(p[1], 0);
        final int d = gcd(dx, dy);
        return new Pair<>(dx / d, dy / d);
    }

    private int gcd(int a, int b) {
        return b == 0 ? a : gcd(b, a % b);
    }
}
```


1328. 2281.java

Path: LeetCode-main/solutions/2281. Sum of Total Strength of Wizards/2281.java

```
class Solution {
    public int totalStrength(int[] strength) {
        final int MOD = 1_000_000_007;
        final int n = strength.length;
        long[] prefix = new long[n];
        long[] prefixOfPrefix = new long[n + 1];
        // left[i] := the next index on the left (if any)
        //           s.t. nums[left[i]] <= nums[i]
        int[] left = new int[n];
        Arrays.fill(left, -1);
        // right[i] := the next index on the right (if any)
        //           s.t. nums[right[i]] < nums[i]
        int[] right = new int[n];
        Arrays.fill(right, n);
        Deque<Integer> stack = new ArrayDeque<>();

        for (int i = 0; i < n; ++i)
            prefix[i] = i == 0 ? strength[0] : (strength[i] + prefix[i - 1]) % MOD;

        for (int i = 0; i < n; ++i)
            prefixOfPrefix[i + 1] = (prefixOfPrefix[i] + prefix[i]) % MOD;

        for (int i = n - 1; i >= 0; --i) {
            while (!stack.isEmpty() && strength[stack.peek()] >= strength[i])
                left[stack.pop()] = i;
            stack.push(i);
        }

        stack.clear();

        for (int i = 0; i < n; ++i) {
            while (!stack.isEmpty() && strength[stack.peek()] > strength[i])
                right[stack.pop()] = i;
            stack.push(i);
        }

        long ans = 0;

        // For each strength[i] as minimum, calculate sum.
        for (int i = 0; i < n; ++i) {
            final int l = left[i];
            final int r = right[i];
            final long leftSum = prefixOfPrefix[i] - prefixOfPrefix[Math.max(0, l)];
            final long rightSum = prefixOfPrefix[r] - prefixOfPrefix[i];
            final int leftLen = i - l;
            final int rightLen = r - i;
            ans += strength[i] * (rightSum * leftLen % MOD - leftSum * rightLen % MOD + MOD) % MOD;
            ans %= MOD;
        }

        return (int) ans;
    }
}
```

1329. 2282.java

Path: LeetCode-main/solutions/2282. Number of People That Can Be Seen in a Grid/2282.java

```
class Solution {
    public int[][] seePeople(int[][] heights) {
        final int m = heights.length;
        final int n = heights[0].length;
        int[][] ans = new int[m][n];

        for (int i = 0; i < m; ++i) {
            Deque<Integer> stack = new ArrayDeque<>();
            for (int j = 0; j < n; ++j) {
                boolean hasEqualHeight = false;
                while (!stack.isEmpty() && heights[i][stack.peek()] <= heights[i][j]) {
                    if (heights[i][stack.peek()] == heights[i][j])
                        // edge case: [4, 2, 1, 1, 3]
                        hasEqualHeight = true;
                    ++ans[i][stack.pop()];
                }
                if (!stack.isEmpty() && !hasEqualHeight)
                    ++ans[i][stack.peek()];
                stack.push(j);
            }
        }

        for (int j = 0; j < n; ++j) {
            Deque<Integer> stack = new ArrayDeque<>();
            for (int i = 0; i < m; ++i) {
                boolean hasEqualHeight = false;
                while (!stack.isEmpty() && heights[stack.peek()][j] <= heights[i][j]) {
                    if (heights[stack.peek()][j] == heights[i][j])
                        hasEqualHeight = true;
                    ++ans[stack.pop()][j];
                }
                if (!stack.isEmpty() && !hasEqualHeight)
                    ++ans[stack.peek()][j];
                stack.push(i);
            }
        }

        return ans;
    }
}
```

1330. 2283.java

Path: LeetCode-main/solutions/2283. Check if Number Has Equal Digit Count and Digit Value/2283.java

```
class Solution {
    public boolean digitCount(String num) {
        int[] count = new int[10];

        for (final char c : num.toCharArray())
            ++count[c - '0'];

        for (int i = 0; i < num.length(); ++i)
            if (count[i] != num.charAt(i) - '0')
                return false;

        return true;
    }
}
```

1331. 2284.java

Path: LeetCode-main/solutions/2284. Sender With Largest Word Count/2284.java

```
class Solution {
    public String largestWordCount(String[] messages, String[] senders) {
        final int n = messages.length;
        String ans = "";
        int maxWordsSent = 0;
        Map<String, Integer> count = new HashMap<>(); // {sender, the number of words sent}

        for (int i = 0; i < n; ++i) {
            final String message = messages[i];
            final String sender = senders[i];
            final int wordsCount = (int) message.chars().filter(c -> c == ' ').count() + 1;
            final int numWordsSent = count.merge(sender, wordsCount, Integer::sum);
            if (numWordsSent > maxWordsSent) {
                ans = sender;
                maxWordsSent = numWordsSent;
            } else if (numWordsSent == maxWordsSent && sender.compareTo(ans) > 0) {
                ans = sender;
            }
        }

        return ans;
    }
}
```

1332. 2285.java

Path: LeetCode-main/solutions/2285. Maximum Total Importance of Roads/2285.java

```
class Solution {
    public long maximumImportance(int n, int[][] roads) {
        long ans = 0;
        long[] count = new long[n];

        for (int[] road : roads) {
            final int u = road[0];
            final int v = road[1];
            ++count[u];
            ++count[v];
        }

        Arrays.sort(count);

        for (int i = 0; i < n; ++i)
            ans += (i + 1) * count[i];

        return ans;
    }
}
```

1333. 2287.java

Path: LeetCode-main/solutions/2287. Rearrange Characters to Make Target String/2287.java

```
class Solution {
    public int rearrangeCharacters(String s, String target) {
        int ans = s.length();
        int[] counts = new int[128];
        int[] countT = new int[128];

        for (final char c : s.toCharArray())
            ++counts[c];

        for (final char c : target.toCharArray())
            ++countT[c];

        for (final char c : target.toCharArray())
            ans = Math.min(ans, counts[c] / countT[c]);

        return ans;
    }
}
```

1334. 2288.java

Path: LeetCode-main/solutions/2288. Apply Discount to Prices/2288.java

```
class Solution {
    public String discountPrices(String sentence, int discount) {
        final int PRECISION = 2;
        StringBuilder sb = new StringBuilder();

        for (final String word : sentence.split(" "))
            if (word.charAt(0) == '$' && word.length() > 1) {
                final String digits = word.substring(1);
                if (digits.chars().allMatch(c -> Character.isDigit(c))) {
                    final double val = Double.parseDouble(digits) * (100 - discount) / 100;
                    final String s = String.format("%.2f", val);
                    final String trimmed = s.substring(0, s.indexOf(".") + PRECISION + 1);
                    sb.append("$").append(trimmed).append(" ");
                } else {
                    sb.append(word).append(" ");
                }
            } else {
                sb.append(word).append(" ");
            }

        sb.deleteCharAt(sb.length() - 1);
        return sb.toString();
    }
}
```

1335. 2289.java

Path: LeetCode-main/solutions/2289. Steps to Make Array Non-decreasing/2289.java

```
class Solution {
    public int totalSteps(int[] nums) {
        // dp[i] := the number of steps to remove nums[i]
        int[] dp = new int[nums.length];
        Deque<Integer> stack = new ArrayDeque<>();

        for (int i = 0; i < nums.length; ++i) {
            int step = 1;
            while (!stack.isEmpty() && nums[stack.peek()] <= nums[i])
                step = Math.max(step, dp[stack.pop()] + 1);
            if (!stack.isEmpty())
                dp[i] = step;
            stack.push(i);
        }

        return Arrays.stream(dp).max().getAsInt();
    }
}
```


1336. 229.java

Path: LeetCode-main/solutions/229. Majority Element II/229.java

```
class Solution {
    public List<Integer> majorityElement(int[] nums) {
        List<Integer> ans = new ArrayList<>();
        int candidate1 = 0;
        int candidate2 = 1; // any number different from candidate1
        int countSoFar1 = 0; // the number of candidate1 so far
        int countSoFar2 = 0; // the number of candidate2 so far

        for (final int num : nums)
            if (num == candidate1) {
                ++countSoFar1;
            } else if (num == candidate2) {
                ++countSoFar2;
            } else if (countSoFar1 == 0) { // Assign the new candidate.
                candidate1 = num;
                ++countSoFar1;
            } else if (countSoFar2 == 0) { // Assign the new candidate.
                candidate2 = num;
                ++countSoFar2;
            } else { // Meet a new number, so pair with the previous counts.
                --countSoFar1;
                --countSoFar2;
            }

        int count1 = 0;
        int count2 = 0;

        for (final int num : nums)
            if (num == candidate1)
                ++count1;
            else if (num == candidate2)
                ++count2;

        if (count1 > nums.length / 3)
            ans.add(candidate1);
        if (count2 > nums.length / 3)
            ans.add(candidate2);
        return ans;
    }
}
```

1337. 2290.java

Path: LeetCode-main/solutions/2290. Minimum Obstacle Removal to Reach Corner/2290.java

```
class Solution {
    public int minimumObstacles(int[][] grid) {
        final int[][] DIRS = {{0, 1}, {1, 0}, {0, -1}, {-1, 0}};
        final int m = grid.length;
        final int n = grid[0].length;
        Queue<int[]> minHeap = new PriorityQueue<>(Comparator.comparingInt(a -> a[0])) {
            { offer(new int[] {grid[0][0], 0, 0}); } // (d, i, j)
        };
        int[][] dist = new int[m][n];
        Arrays.stream(dist).forEach(A -> Arrays.fill(A, Integer.MAX_VALUE));
        dist[0][0] = grid[0][0];

        while (!minHeap.isEmpty()) {
            final int d = minHeap.peek()[0];
            final int i = minHeap.peek()[1];
            final int j = minHeap.poll()[2];
            if (i == m - 1 && j == n - 1)
                return d;
            for (int[] dir : DIRS) {
                final int x = i + dir[0];
                final int y = j + dir[1];
                if (x < 0 || x == m || y < 0 || y == n)
                    continue;
                final int newDist = d + grid[i][j];
                if (newDist < dist[x][y]) {
                    dist[x][y] = newDist;
                    minHeap.offer(new int[] {newDist, x, y});
                }
            }
        }

        return dist[m - 1][n - 1];
    }
}
```

1338. 2291-2.java

Path: LeetCode-main/solutions/2291. Maximum Profit From Trading Stocks/2291-2.java

```
class Solution {
    public int maximumProfit(int[] present, int[] future, int budget) {
        final int n = present.length;
        // dp[i] := the maximum profit of buying present so far with i budget
        int[] dp = new int[budget + 1];

        for (int i = 0; i < n; ++i)
            for (int j = budget; j >= present[i]; --j)
                dp[j] = Math.max(dp[j], future[i] - present[i] + dp[j - present[i]]);

        return dp[budget];
    }
}
```

1339. 2291.java

Path: LeetCode-main/solutions/2291. Maximum Profit From Trading Stocks/2291.java

```
class Solution {
    public int maximumProfit(int[] present, int[] future, int budget) {
        final int n = present.length;
        // dp[i][j] := the maximum profit of buying present[0..i) with j budget
        int[][] dp = new int[n + 1][budget + 1];

        for (int i = 1; i <= n; ++i) {
            final int profit = future[i - 1] - present[i - 1];
            for (int j = 0; j <= budget; ++j)
                if (j < present[i - 1])
                    dp[i][j] = dp[i - 1][j];
                else
                    dp[i][j] = Math.max(dp[i - 1][j], profit + dp[i - 1][j - present[i - 1]]);
        }

        return dp[n][budget];
    }
}
```

1340. 2293.java

Path: LeetCode-main/solutions/2293. Min Max Game/2293.java

```
class Solution {
    public int minMaxGame(int[] nums) {
        if (nums.length == 1)
            return nums[0];

        int[] nextNums = new int[nums.length / 2];
        for (int i = 0; i < nextNums.length; ++i)
            nextNums[i] = i % 2 == 0 ? Math.min(nums[2 * i], nums[2 * i + 1])
                                     : Math.max(nums[2 * i], nums[2 * i + 1]);
        return minMaxGame(nextNums);
    }
}
```

1341. 2294.java

Path: LeetCode-main/solutions/2294. Partition Array Such That Maximum Difference Is K/2294.java

```
class Solution {
    public int partitionArray(int[] nums, int k) {
        Arrays.sort(nums);

        int ans = 1;
        int mn = nums[0];

        for (int i = 1; i < nums.length; ++i)
            if (mn + k < nums[i]) {
                ++ans;
                mn = nums[i];
            }

        return ans;
    }
}
```

1342. 2295.java

Path: LeetCode-main/solutions/2295. Replace Elements in an Array/2295.java

```
class Solution {
    public int[] arrayChange(int[] nums, int[][] operations) {
        Map<Integer, Integer> numToIndex = new HashMap<>();

        for (int i = 0; i < nums.length; ++i)
            numToIndex.put(nums[i], i);

        for (int[] o : operations) {
            final int original = o[0];
            final int replaced = o[1];
            final int index = numToIndex.get(original);
            nums[index] = replaced;
            numToIndex.remove(original);
            numToIndex.put(replaced, index);
        }

        return nums;
    }
}
```

1343. 2296.java

Path: LeetCode-main/solutions/2296. Design a Text Editor/2296.java

```
class TextEditor {
    public void addText(String text) {
        sb.append(text);
    }

    public int deleteText(int k) {
        final int numDeleted = Math.min(k, sb.length());
        for (int i = 0; i < numDeleted; ++i)
            sb.deleteCharAt(sb.length() - 1);
        return numDeleted;
    }

    public String cursorLeft(int k) {
        while (!sb.isEmpty() && k-- > 0) {
            stack.push(sb.charAt(sb.length() - 1));
            sb.deleteCharAt(sb.length() - 1);
        }
        return getString();
    }

    public String cursorRight(int k) {
        while (!stack.isEmpty() && k-- > 0)
            sb.append(stack.pop());
        return getString();
    }

    private String getString() {
        if (sb.length() < 10)
            return sb.toString();
        return sb.substring(sb.length() - 10).toString();
    }

    private StringBuilder sb = new StringBuilder();
    private Deque<Character> stack = new ArrayDeque<>();
}
```


1344. 2297.java

Path: LeetCode-main/solutions/2297. Jump Game IX/2297.java

```
class Solution {
    public long minCost(int[] nums, int[] costs) {
        final int n = nums.length;
        // dp[i] := the minimum cost to jump to i
        long[] dp = new long[n];
        Deque<Integer> maxStack = new ArrayDeque<>();
        Deque<Integer> minStack = new ArrayDeque<>();

        Arrays.fill(dp, Long.MAX_VALUE);
        dp[0] = 0;

        for (int i = 0; i < n; ++i) {
            while (!maxStack.isEmpty() && nums[i] >= nums[maxStack.peek()])
                dp[i] = Math.min(dp[i], dp[maxStack.pop()] + costs[i]);
            while (!minStack.isEmpty() && nums[i] < nums[minStack.peek()])
                dp[i] = Math.min(dp[i], dp[minStack.pop()] + costs[i]);
            maxStack.push(i);
            minStack.push(i);
        }

        return dp[n - 1];
    }
}
```

1345. 2299.java

Path: LeetCode-main/solutions/2299. Strong Password Checker II/2299.java

```
class Solution {
    public boolean strongPasswordCheckerII(String password) {
        if (password.length() < 8)
            return false;

        final boolean hasLowerCase = password.chars().anyMatch(c -> Character.isLowerCase(c));
        if (!hasLowerCase)
            return false;

        final boolean hasUpperCase = password.chars().anyMatch(c -> Character.isUpperCase(c));
        if (!hasUpperCase)
            return false;

        final boolean hasDigit = password.chars().anyMatch(c -> Character.isDigit(c));
        if (!hasDigit)
            return false;

        final boolean hasSpecial = password.chars().anyMatch(c -> "!@#$%^&*()-+\".indexOf(c) != -1);
        if (!hasSpecial)
            return false;

        for (int i = 1; i < password.length(); ++i)
            if (password.charAt(i) == password.charAt(i - 1))
                return false;
        return true;
    }
}
```

1346. 23.java

Path: LeetCode-main/solutions/23. Merge k Sorted Lists/23.java

```
class Solution {
    public ListNode mergeKLists(ListNode[] lists) {
        ListNode dummy = new ListNode(0);
        ListNode curr = dummy;
        Queue<ListNode> minHeap = new PriorityQueue<>(Comparator.comparingInt(a -> a.val));

        for (final ListNode list : lists)
            if (list != null)
                minHeap.offer(list);

        while (!minHeap.isEmpty()) {
            ListNode minNode = minHeap.poll();
            if (minNode.next != null)
                minHeap.offer(minNode.next);
            curr.next = minNode;
            curr = curr.next;
        }

        return dummy.next;
    }
}
```

1347. 230-2.java

Path: LeetCode-main/solutions/230. Kth Smallest Element in a BST/230-2.java

```
class Solution {
    public int kthSmallest(TreeNode root, int k) {
        traverse(root, k);
        return ans;
    }

    private int ans = -1;
    private int rank = 0;

    private void traverse(TreeNode root, int k) {
        if (root == null)
            return;

        traverse(root.left, k);
        if (++rank == k) {
            ans = root.val;
            return;
        }
        traverse(root.right, k);
    }
}
```

1348. 230-3.java

Path: LeetCode-main/solutions/230. Kth Smallest Element in a BST/230-3.java

```
class Solution {
    public int kthSmallest(TreeNode root, int k) {
        Deque<TreeNode> stack = new ArrayDeque<>();

        while (root != null) {
            stack.push(root);
            root = root.left;
        }

        for (int i = 0; i < k - 1; ++i) {
            root = stack.pop();
            root = root.right;
            while (root != null) {
                stack.push(root);
                root = root.left;
            }
        }

        return stack.peek().val;
    }
}
```

1349. 230.java

Path: LeetCode-main/solutions/230. Kth Smallest Element in a BST/230.java

```
class Solution {
    public int kthSmallest(TreeNode root, int k) {
        final int leftCount = countNodes(root.left);

        if (leftCount == k - 1)
            return root.val;
        if (leftCount >= k)
            return kthSmallest(root.left, k);
        return kthSmallest(root.right, k - 1 - leftCount); // leftCount < k
    }

    private int countNodes(TreeNode root) {
        if (root == null)
            return 0;
        return 1 + countNodes(root.left) + countNodes(root.right);
    }
}
```

1350. 2300.java

Path: LeetCode-main/solutions/2300. Successful Pairs of Spells and Potions/2300.java

```
class Solution {
    public int[] successfulPairs(int[] spells, int[] potions, long success) {
        int[] ans = new int[spells.length];
        Arrays.sort(potions);

        for (int i = 0; i < spells.length; ++i)
            ans[i] = potions.length - firstIndexSuccess(spells[i], potions, success);

        return ans;
    }

    // Returns the first index i s.t. spell * potions[i] >= success.
    private int firstIndexSuccess(int spell, int[] potions, long success) {
        int l = 0;
        int r = potions.length;
        while (l < r) {
            final int m = (l + r) / 2;
            if ((long) spell * potions[m] >= success)
                r = m;
            else
                l = m + 1;
        }
        return l;
    }
}
```

1351. 2301.java

Path: LeetCode-main/solutions/2301. Match Substring After Replacement/2301.java

```
class Solution {
    public boolean matchReplacement(String s, String sub, char[][] mappings) {
        boolean[][] isMapped = new boolean[128][128];

        for (char[] m : mappings) {
            final char old = m[0];
            final char _new = m[1];
            isMapped[old][_new] = true;
        }

        for (int i = 0; i < s.length(); ++i)
            if (canTransform(s, i, sub, isMapped))
                return true;

        return false;
    }

    private boolean canTransform(final String s, int start, final String sub, boolean[][] isMapped) {
        if (start + sub.length() > s.length())
            return false;

        for (int i = 0; i < sub.length(); ++i) {
            final char a = sub.charAt(i);
            final char b = s.charAt(start + i);
            if (a != b && !isMapped[a][b])
                return false;
        }

        return true;
    }
}
```


1352. 2302.java

Path: LeetCode-main/solutions/2302. Count Subarrays With Score Less Than K/2302.java

```
class Solution {
    public long countSubarrays(int[] nums, long k) {
        long ans = 0;
        long sum = 0;

        for (int l = 0, r = 0; r < nums.length; ++r) {
            sum += nums[r];
            while (sum * (r - l + 1) >= k)
                sum -= nums[l++];
            ans += r - l + 1;
        }

        return ans;
    }
}
```

1353. 2303.java

Path: LeetCode-main/solutions/2303. Calculate Amount Paid in Taxes/2303.java

```
class Solution {
    public double calculateTax(int[][] brackets, int income) {
        double ans = 0;
        int prev = 0;

        for (int[] b : brackets) {
            final int upper = b[0];
            final int percent = b[1];
            if (income < upper)
                return ans + (income - prev) * percent / 100.0;
            ans += (upper - prev) * percent / 100.0;
            prev = upper;
        }

        return ans;
    }
}
```

1354. 2304.java

Path: LeetCode-main/solutions/2304. Minimum Path Cost in a Grid/2304.java

```
class Solution {
    public int minPathCost(int[][] grid, int[][] moveCost) {
        final int m = grid.length;
        final int n = grid[0].length;
        // dp[i][j] := the minimum cost to reach grid[i][j]
        int[][] dp = new int[m][n];
        Arrays.stream(dp).forEach(A -> Arrays.fill(A, Integer.MAX_VALUE));
        dp[0] = grid[0];

        for (int i = 1; i < m; ++i)
            for (int j = 0; j < n; ++j)
                for (int k = 0; k < n; ++k)
                    dp[i][j] = Math.min(dp[i][j], dp[i - 1][k] + moveCost[grid[i - 1][k]][j] + grid[i][j]);

        return Arrays.stream(dp[m - 1]).min().getAsInt();
    }
}
```

1355. 2305.java

Path: LeetCode-main/solutions/2305. Fair Distribution of Cookies/2305.java

```
class Solution {
    public int distributeCookies(int[] cookies, int k) {
        dfs(cookies, 0, k, new int[k]);
        return ans;
    }

    private int ans = Integer.MAX_VALUE;

    private void dfs(int[] cookies, int s, int k, int[] children) {
        if (s == cookies.length) {
            ans = Math.min(ans, Arrays.stream(children).max().getAsInt());
            return;
        }

        for (int i = 0; i < k; ++i) {
            children[i] += cookies[s];
            dfs(cookies, s + 1, k, children);
            children[i] -= cookies[s];
        }
    }
}
```

1356. 2306.java

Path: LeetCode-main/solutions/2306. Naming a Company/2306.java

```
class Solution {
    public long distinctNames(String[] ideas) {
        long ans = 0;
        // suffixes[i] := the set of strings omitting the first letter, where the first letter is
        // ('a' + i)
        Set<String>[] suffixes = new Set[26];

        for (int i = 0; i < 26; ++i)
            suffixes[i] = new HashSet<>();

        for (final String idea : ideas)
            suffixes[idea.charAt(0) - 'a'].add(idea.substring(1));

        for (int i = 0; i < 25; ++i)
            for (int j = i + 1; j < 26; ++j) {
                int count = 0;
                for (final String suffix : suffixes[i])
                    if (suffixes[j].contains(suffix))
                        ++count;
                ans += 2 * (suffixes[i].size() - count) * (suffixes[j].size() - count);
            }

        return ans;
    }
}
```

1357. 2307.java

Path: LeetCode-main/solutions/2307. Check for Contradictions in Equations/2307.java

```
class Solution {
    public boolean checkContradictions(List<List<String>> equations, double[] values) {
        // Convert `string` to `int` for a better performance.
        Map<String, Integer> strToInt = new HashMap<>();

        for (List<String> equation : equations) {
            strToInt.putIfAbsent(equation.get(0), strToInt.size());
            strToInt.putIfAbsent(equation.get(1), strToInt.size());
        }

        List<Pair<Integer, Double>>[] graph = new List[strToInt.size()];
        double[] seen = new double[graph.length];

        for (int i = 0; i < graph.length; ++i)
            graph[i] = new ArrayList<>();

        for (int i = 0; i < equations.size(); ++i) {
            final int u = strToInt.get(equations.get(i).get(0));
            final int v = strToInt.get(equations.get(i).get(1));
            graph[u].add(new Pair<>(v, values[i]));
            graph[v].add(new Pair<>(u, 1 / values[i]));
        }

        for (int i = 0; i < graph.length; ++i)
            if (seen[i] != 0.0 && dfs(graph, i, seen, 1.0))
                return true;

        return false;
    }

    private boolean dfs(List<Pair<Integer, Double>>[] graph, int u, double[] seen, double val) {
        if (seen[u] != 0.0)
            return Math.abs(val / seen[u] - 1) > 1e-5;

        seen[u] = val;
        for (Pair<Integer, Integer> pair : graph[u]) {
            final int v = pair.getKey();
            final double w = pair.getValue();
            if (dfs(graph, v, seen, val / w))
                return true;
        }

        return false;
    }
}
```

1358. 2309.java

Path: LeetCode-main/solutions/2309. Greatest English Letter in Upper and Lower Case/2309.java

```
class Solution {
    public String greatestLetter(String s) {
        boolean[] seen = new boolean[128];

        for (final char c : s.toCharArray())
            seen[c] = true;

        for (int i = 25; i >= 0; --i)
            if (seen['a' + i] && seen['A' + i])
                return String.valueOf((char) ('A' + i));

        return "";
    }
}
```

1359. 231.java

Path: LeetCode-main/solutions/231. Power of Two/231.java

```
class Solution {  
    public boolean isPowerOfTwo(int n) {  
        return n >= 0 && Integer.bitCount(n) == 1;  
    }  
}
```


1360. 2310.java

Path: LeetCode-main/solutions/2310. Sum of Numbers With Units Digit K/2310.java

```
class Solution {
    public int minimumNumbers(int num, int k) {
        if (num == 0)
            return 0;

        // Assume the size of the set is n, and the numbers in the set are X1, X2,
        // ..., Xn. Since the units digit of each number is k,  $X_1 + X_2 + \dots + X_n =$ 
        //  $N * k + 10 * (x_1 + x_2 + \dots + x_n) = \text{num}$ . Therefore, the goal is to find
        // the n s.t.  $n * k \% 10 = \text{num} \% 10$ 
        for (int i = 1; i <= 10 && i * k <= num; ++i)
            if (i * k \% 10 == num \% 10)
                return i;

        return -1;
    }
}
```

1361. 2311.java

Path: LeetCode-main/solutions/2311. Longest Binary Subsequence Less Than or Equal to K/2311.java

```
class Solution {
    public int longestSubsequence(String s, int k) {
        int oneCount = 0;
        int num = 0;
        int pow = 1;

        // Take as many 1s as possible from the right.
        for (int i = s.length() - 1; i >= 0 && num + pow <= k; --i) {
            if (s.charAt(i) == '1') {
                ++oneCount;
                num += pow;
            }
            pow *= 2;
        }

        return (int) s.chars().filter(c -> c == '0').count() + oneCount;
    }
}
```

1362. 2312.java

Path: LeetCode-main/solutions/2312. Selling Pieces of Wood/2312.java

```
class Solution {
    public long sellingWood(int m, int n, int[][] prices) {
        // dp[i][j] := the maximum money of cutting i x j piece of wood
        long[][] dp = new long[m + 1][n + 1];

        for (int[] p : prices) {
            final int h = p[0];
            final int w = p[1];
            final int price = p[2];
            dp[h][w] = price;
        }

        for (int i = 1; i <= m; ++i)
            for (int j = 1; j <= n; ++j) {
                for (int h = 1; h <= i / 2; ++h)
                    dp[i][j] = Math.max(dp[i][j], dp[h][j] + dp[i - h][j]);
                for (int w = 1; w <= j / 2; ++w)
                    dp[i][j] = Math.max(dp[i][j], dp[i][w] + dp[i][j - w]);
            }

        return dp[m][n];
    }
}
```

1363. 2315.java

Path: LeetCode-main/solutions/2315. Count Asterisks/2315.java

```
class Solution {
    public int countAsterisks(String s) {
        int ans = 0;
        int bars = 0;

        for (final char c : s.toCharArray()) {
            if (c == '|')
                ++bars;
            else if (c == '*' && bars % 2 == 0)
                ++ans;
        }

        return ans;
    }
}
```

1364. 2316.java

Path: LeetCode-main/solutions/2316. Count Unreachable Pairs of Nodes in an Undirected Graph/2316.java

```
class Solution {
    public long countPairs(int n, int[][] edges) {
        long ans = 0;
        List<Integer>[] graph = new List[n];
        boolean[] seen = new boolean[n];
        int unreached = n;
        Arrays.setAll(graph, i -> new ArrayList<>());

        for (int[] edge : edges) {
            final int u = edge[0];
            final int v = edge[1];
            graph[u].add(v);
            graph[v].add(u);
        }

        for (int i = 0; i < n; ++i) {
            final int reached = dfs(graph, i, seen);
            unreached -= reached;
            ans += (long) unreached * reached;
        }
        return ans;
    }

    private int dfs(List<Integer>[] graph, int u, boolean[] seen) {
        if (seen[u])
            return 0;

        seen[u] = true;
        int ans = 1;
        for (final int v : graph[u])
            ans += dfs(graph, v, seen);
        return ans;
    }
}
```

1365. 2317.java

Path: LeetCode-main/solutions/2317. Maximum XOR After Operations/2317.java

```
class Solution {
    public int maximumXOR(int[] nums) {
        // 1. nums[i] & (nums[i] ^ x) enables you to turn 1-bit to 0-bit from
        //    nums[i] since x is arbitrary.
        // 2. The i-th bit of the XOR of all the elements is 1 if the i-th bit is 1
        //    for an odd number of elements.
        // 3. Therefore, the question is equivalent to: if you can convert any digit
        //    from 1 to 0 for any number, what is the maximum for XOR(nums[i]).
        // 4. The maximum we can get is of course to make every digit of the answer
        //    to be 1 if possible
        // 5. Therefore, OR(nums[i]) is an approach.
        return Arrays.stream(nums).reduce(0, (a, b) -> a | b);
    }
}
```

1366. 2318.java

Path: LeetCode-main/solutions/2318. Number of Distinct Roll Sequences/2318.java

```
class Solution {
    public int distinctSequences(int n) {
        int[][][] mem = new int[n + 1][7][7];
        return distinctSequences(n, 0, 0);
    }

    private static final int MOD = 1_000_000_007;

    // Returns the number of distinct sequences for n dices with `prev` and
    // `prevPrev`.
    private int distinctSequences(int n, int prev, int prevPrev, int[][][] mem) {
        if (n == 0)
            return 1;
        if (mem[n][prev][prevPrev] > 0)
            return mem[n][prev][prevPrev];

        for (int dice = 1; dice <= 6; ++dice)
            if (dice != prev && dice != prevPrev && (prev == 0 || gcd(dice, prev) == 1)) {
                mem[n][prev][prevPrev] += distinctSequences(n - 1, dice, prev, mem);
                mem[n][prev][prevPrev] %= MOD;
            }

        return mem[n][prev][prevPrev];
    }

    private int gcd(int a, int b) {
        return b == 0 ? a : gcd(b, a % b);
    }
}
```

1367. 2319.java

Path: LeetCode-main/solutions/2319. Check if Matrix Is X-Matrix/2319.java

```
class Solution {
    public boolean checkXMatrix(int[][] grid) {
        final int n = grid.length;

        for (int i = 0; i < n; ++i)
            for (int j = 0; j < n; ++j)
                if (i == j || i + j == n - 1) { // in diagonal
                    if (grid[i][j] == 0)
                        return false;
                } else if (grid[i][j] > 0) { // not in diagonal
                    return false;
                }
        return true;
    }
}
```


1368. 232.java

Path: LeetCode-main/solutions/232. Implement Queue using Stacks/232.java

```
class MyQueue {
    public void push(int x) {
        input.push(x);
    }

    public int pop() {
        peek();
        return output.pop();
    }

    public int peek() {
        if (output.isEmpty())
            while (!input.isEmpty())
                output.push(input.pop());
        return output.peek();
    }

    public boolean empty() {
        return input.isEmpty() && output.isEmpty();
    }

    private Deque<Integer> input = new ArrayDeque<>();
    private Deque<Integer> output = new ArrayDeque<>();
}
```

1369. 2320.java

Path: LeetCode-main/solutions/2320. Count Number of Ways to Place Houses/2320.java

```
class Solution {
    public int countHousePlacements(int n) {
        final int MOD = 1_000_000_007;
        int house = 1; // the number of ways ending in a house
        int space = 1; // the number of ways ending in a space
        int total = house + space;

        for (int i = 2; i <= n; ++i) {
            house = space;
            space = total;
            total = (house + space) % MOD;
        }

        return (int) ((long) total * total % MOD);
    }
}
```

1370. 2321.java

Path: LeetCode-main/solutions/2321. Maximum Score Of Spliced Array/2321.java

```
class Solution {
    public int maximumSplicedArray(int[] nums1, int[] nums2) {
        return Math.max(kadane(nums1, nums2), kadane(nums2, nums1));
    }

    // Returns the maximum gain of swapping some numbers in `nums1` with some numbers in `nums2`.
    private int kadane(int[] nums1, int[] nums2) {
        int gain = 0;
        int maxGain = 0;

        for (int i = 0; i < nums1.length; ++i) {
            gain = Math.max(0, gain + nums2[i] - nums1[i]);
            maxGain = Math.max(maxGain, gain);
        }

        return maxGain + Arrays.stream(nums1).sum();
    }
}
```

1371. 2322.java

Path: LeetCode-main/solutions/2322. Minimum Score After Removals on a Tree/2322.java

```
class Solution {
    public int minimumScore(int[] nums, int[][] edges) {
        final int n = nums.length;
        final int xors = getXors(nums);
        int[] subXors = nums.clone();
        List<Integer>[] tree = new List[n];
        Set<Integer>[] children = new Set[n];

        for (int i = 0; i < n; ++i)
            tree[i] = new ArrayList<>();

        for (int i = 0; i < n; ++i)
            children[i] = new HashSet<>(Arrays.asList(i));

        for (int[] edge : edges) {
            final int u = edge[0];
            final int v = edge[1];
            tree[u].add(v);
            tree[v].add(u);
        }

        dfs(tree, 0, -1, subXors, children);

        int ans = Integer.MAX_VALUE;

        for (int i = 0; i < edges.length; ++i) {
            int a = edges[i][0];
            int b = edges[i][1];
            if (children[a].contains(b)) {
                final int temp = a;
                a = b;
                b = temp;
            }
            for (int j = 0; j < i; ++j) {
                int c = edges[j][0];
                int d = edges[j][1];
                if (children[c].contains(d)) {
                    final int temp = c;
                    c = d;
                    d = temp;
                }
                int[] cands;
                if (a != c && children[a].contains(c))
                    cands = new int[] {subXors[c], subXors[a] ^ subXors[c], xors ^ subXors[a]};
                else if (a != c && children[c].contains(a))
                    cands = new int[] {subXors[a], subXors[c] ^ subXors[a], xors ^ subXors[c]};
                else
                    cands = new int[] {subXors[a], subXors[c], xors ^ subXors[a] ^ subXors[c]};
                ans = Math.min(ans, Arrays.stream(cands).max().getAsInt() -
                    Arrays.stream(cands).min().getAsInt());
            }
        }

        return ans;
    }

    private Pair<Integer, Set<Integer>> dfs(List<Integer>[] tree, int u, int prev, int[] subXors,
        Set<Integer>[] children) {
        for (final int v : tree[u]) {
            if (v == prev)
                continue;
            final Pair<Integer, Set<Integer>> pair = dfs(tree, v, u, subXors, children);
            final int vXor = pair.getKey();
            final Set<Integer> vChildren = pair.getValue();
            subXors[u] ^= vXor;
            for (final int child : vChildren)
                children[u].add(child);
        }
        return new Pair<>(subXors[u], children[u]);
    }

    private int getXors(int[] nums) {
        int xors = 0;
        for (final int num : nums)
            xors ^= num;
        return xors;
    }
}
```

1372. 2323.java

Path: LeetCode-main/solutions/2323. Find Minimum Time to Finish All Jobs II/2323.java

```
class Solution {
    public int minimumTime(int[] jobs, int[] workers) {
        int ans = 0;

        Arrays.sort(jobs);
        Arrays.sort(workers);

        for (int i = 0; i < jobs.length; ++i)
            ans = Math.max(ans, (jobs[i] - 1) / workers[i] + 1);

        return ans;
    }
}
```

1373. 2325.java

Path: LeetCode-main/solutions/2325. Decode the Message/2325.java

```
class Solution {
    public String decodeMessage(String key, String message) {
        StringBuilder sb = new StringBuilder();
        char[] keyToActual = new char[128];
        keyToActual[' '] = ' ';
        char currChar = 'a';

        for (final char c : key.toCharArray())
            if (keyToActual[c] == 0)
                keyToActual[c] = currChar++;

        for (final char c : message.toCharArray())
            sb.append(keyToActual[c]);

        return sb.toString();
    }
}
```

1374. 2326.java

Path: LeetCode-main/solutions/2326. Spiral Matrix IV/2326.java

```
class Solution {
    public int[][] spiralMatrix(int m, int n, ListNode head) {
        final int[][] DIRS = {{0, 1}, {1, 0}, {0, -1}, {-1, 0}};
        int[][] ans = new int[m][n];
        Arrays.stream(ans).forEach(A -> Arrays.fill(A, -1));
        int x = 0; // the current x position
        int y = 0; // the current y position
        int d = 0;

        for (ListNode curr = head; curr != null; curr = curr.next) {
            ans[x][y] = curr.val;
            if (x + DIRS[d][0] < 0 || x + DIRS[d][0] == m || y + DIRS[d][1] < 0 || //
                y + DIRS[d][1] == n || ans[x + DIRS[d][0]][y + DIRS[d][1]] != -1)
                d = (d + 1) % 4;
            x += DIRS[d][0];
            y += DIRS[d][1];
        }

        return ans;
    }
}
```

1375. 2327.java

Path: LeetCode-main/solutions/2327. Number of People Aware of a Secret/2327.java

```
class Solution {
    public int peopleAwareOfSecret(int n, int delay, int forget) {
        final int MOD = 1_000_000_007;
        long share = 0;
        // dp[i] := the number of people know the secret at day i
        int[] dp = new int[n]; // Maps day i to i + 1.
        dp[0] = 1;

        for (int i = 1; i < n; ++i) {
            if (i - delay >= 0)
                share += dp[i - delay];
            if (i - forget >= 0)
                share -= dp[i - forget];
            share += MOD;
            share %= MOD;
            dp[i] = (int) share;
        }

        // People before day `n - forget - 1` already forget the secret.
        int ans = 0;
        for (int i = n - forget; i < n; ++i)
            ans = (ans + dp[i]) % MOD;
        return ans;
    }
}
```


1376. 2328.java

Path: LeetCode-main/solutions/2328. Number of Increasing Paths in a Grid/2328.java

```
class Solution {
    public int countPaths(int[][] grid) {
        final int m = grid.length;
        final int n = grid[0].length;
        int ans = 0;
        Integer[][] mem = new Integer[m][n];

        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j) {
                ans += countPaths(grid, i, j, mem);
                ans %= MOD;
            }

        return ans;
    }

    private static final int MOD = 1_000_000_007;
    private static final int[][] DIRS = {{0, 1}, {1, 0}, {0, -1}, {-1, 0}};

    // Returns the number of increasing paths starting from (i, j).
    private int countPaths(int[][] grid, int i, int j, Integer[][] mem) {
        if (mem[i][j] != null)
            return mem[i][j];

        mem[i][j] = 1; // The current cell contributes 1 length.

        for (int[] dir : DIRS) {
            int x = i + dir[0];
            int y = j + dir[1];
            if (x < 0 || x == grid.length || y < 0 || y == grid[0].length)
                continue;
            if (grid[x][y] <= grid[i][j])
                continue;
            mem[i][j] += countPaths(grid, x, y, mem);
            mem[i][j] %= MOD;
        }

        return mem[i][j];
    }
}
```

1377. 233.java

Path: LeetCode-main/solutions/233. Number of Digit One/233.java

```
class Solution {
    public int countDigitOne(int n) {
        int ans = 0;

        for (long pow10 = 1; pow10 <= n; pow10 *= 10) {
            final long divisor = pow10 * 10;
            final int quotient = (int) (n / divisor);
            final int remainder = (int) (n % divisor);
            if (quotient > 0)
                ans += quotient * pow10;
            if (remainder >= pow10)
                ans += Math.min(remainder - pow10 + 1, pow10);
        }

        return ans;
    }
}
```

1378. 2330.java

Path: LeetCode-main/solutions/2330. Valid Palindrome IV/2330.java

```
class Solution {
    public boolean makePalindrome(String s) {
        int change = 0;
        int l = 0;
        int r = s.length() - 1;

        while (l < r) {
            if (s.charAt(l) != s.charAt(r) && ++change > 2)
                return false;
            ++l;
            --r;
        }

        return true;
    }
}
```

1379. 2331.java

Path: LeetCode-main/solutions/2331. Evaluate Boolean Binary Tree/2331.java

```
class Solution {
    public boolean evaluateTree(TreeNode root) {
        if (root.val < 2)
            return root.val == 1;
        if (root.val == 2) // OR
            return evaluateTree(root.left) || evaluateTree(root.right);
        // AND
        return evaluateTree(root.left) && evaluateTree(root.right);
    }
}
```

1380. 2332.java

Path: LeetCode-main/solutions/2332. The Latest Time to Catch a Bus/2332.java

```
class Solution {
    public int latestTimeCatchTheBus(int[] buses, int[] passengers, int capacity) {
        Arrays.sort(buses);
        Arrays.sort(passengers);

        if (passengers[0] > buses[buses.length - 1])
            return buses[buses.length - 1];

        int ans = passengers[0] - 1;
        int i = 0; // buses' index
        int j = 0; // passengers' index

        while (i < buses.length) {
            // Greedily make passengers catch `buses[i]`.
            int arrived = 0;
            while (arrived < capacity && j < passengers.length && passengers[j] <= buses[i]) {
                if (j > 0 && passengers[j] != passengers[j - 1] + 1)
                    ans = passengers[j] - 1;
                ++j;
                ++arrived;
            }
            // There's room for `buses[i]` to carry a passenger arriving at the
            // `buses[i]`.
            if (arrived < capacity && j > 0 && passengers[j - 1] != buses[i])
                ans = buses[i];
            ++i;
        }

        return ans;
    }
}
```

1381. 2333.java

Path: LeetCode-main/solutions/2333. Minimum Sum of Squared Difference/2333.java

```
class Solution {
    public long minSumSquareDiff(int[] nums1, int[] nums2, int k1, int k2) {
        int[] diff = getDiff(nums1, nums2);
        int k = k1 + k2;
        if (Arrays.stream(diff).asLongStream().sum() <= k)
            return 0;

        Map<Integer, Integer> count = new HashMap<>();
        // (num, freq)
        Queue<Pair<Integer, Integer>> maxHeap =
            new PriorityQueue<>(Comparator.comparing(Pair::getKey, Comparator.reverseOrder()));

        for (final int d : diff)
            if (d != 0)
                count.merge(d, 1, Integer::sum);

        for (Map.Entry<Integer, Integer> entry : count.entrySet())
            maxHeap.offer(new Pair<>(entry.getKey(), entry.getValue()));

        while (k > 0) {
            Pair<Integer, Integer> pair = maxHeap.poll();
            final int maxNum = pair.getKey();
            final int maxNumFreq = pair.getValue();
            // Buck decrease in this turn
            final int numDecreased = Math.min(k, maxNumFreq);
            k -= numDecreased;
            if (maxNumFreq > numDecreased)
                maxHeap.offer(new Pair<>(maxNum, maxNumFreq - numDecreased));
            if (!maxHeap.isEmpty() && maxHeap.peek().getKey() + 1 == maxNum) {
                Pair<Integer, Integer> secondNode = maxHeap.poll();
                final int secondMaxNum = secondNode.getKey();
                final int secondMaxNumFreq = secondNode.getValue();
                maxHeap.offer(new Pair<>(secondMaxNum, secondMaxNumFreq + numDecreased));
            } else if (maxNum > 1) {
                maxHeap.offer(new Pair<>(maxNum - 1, numDecreased));
            }
        }

        long ans = 0;
        while (!maxHeap.isEmpty()) {
            Pair<Integer, Integer> pair = maxHeap.poll();
            final int num = pair.getKey();
            final int freq = pair.getValue();
            ans += (long) num * num * freq;
        }

        return ans;
    }

    private int[] getDiff(int[] nums1, int[] nums2) {
        int[] diff = new int[nums1.length];
        for (int i = 0; i < nums1.length; ++i)
            diff[i] = Math.abs(nums1[i] - nums2[i]);
        return diff;
    }
}
```

1382. 2334.java

Path: LeetCode-main/solutions/2334. Subarray With Elements Greater Than Varying Threshold/2334.java

```
class Solution {
    // Similar to 907. Sum of Subarray Minimums
    public int validSubarraySize(int[] nums, int threshold) {
        final int n = nums.length;
        long ans = 0;
        // prev[i] := the index k s.t. nums[k] is the previous minimum in nums[0..n)
        int[] prev = new int[n];
        // next[i] := the index k s.t. nums[k] is the next minimum in nums[i + 1..n)
        int[] next = new int[n];
        Deque<Integer> stack = new ArrayDeque<>();

        Arrays.fill(prev, -1);
        Arrays.fill(next, n);

        for (int i = 0; i < nums.length; ++i) {
            while (!stack.isEmpty() && nums[stack.peek()] > nums[i]) {
                final int index = stack.pop();
                next[index] = i;
            }
            if (!stack.isEmpty())
                prev[i] = stack.peek();
            stack.push(i);
        }

        for (int i = 0; i < n; ++i) {
            // the number of `nums` in subarray containing nums[i] >= nums[i]
            final int k = (i - prev[i]) + (next[i] - i) - 1;
            if (nums[i] > threshold / (double) k)
                return k;
        }

        return -1;
    }
}
```

1383. 2335.java

Path: LeetCode-main/solutions/2335. Minimum Amount of Time to Fill Cups/2335.java

```
class Solution {  
    public int fillCups(int[] amount) {  
        final int mx = Arrays.stream(amount).max().getAsInt();  
        final int sum = Arrays.stream(amount).sum();  
        return Math.max(mx, (sum + 1) / 2);  
    }  
}
```


1384. 2336.java

Path: LeetCode-main/solutions/2336. Smallest Number in Infinite Set/2336.java

```
class SmallestInfiniteSet {
    public int popSmallest() {
        if (added.isEmpty())
            return curr++;
        final int mn = added.first();
        added.remove(mn);
        return mn;
    }

    public void addBack(int num) {
        if (num < curr)
            added.add(num);
    }

    private int curr = 1;
    private TreeSet<Integer> added = new TreeSet<>();
}
```

1385. 2337.java

Path: LeetCode-main/solutions/2337. Move Pieces to Obtain a String/2337.java

```
class Solution {
    public boolean canChange(String start, String target) {
        final int n = start.length();
        int i = 0; // start's index
        int j = 0; // target's index

        while (i <= n && j <= n) {
            while (i < n && start.charAt(i) == '_')
                ++i;
            while (j < n && target.charAt(j) == '_')
                ++j;
            if (i == n || j == n)
                return i == n && j == n;
            if (start.charAt(i) != target.charAt(j))
                return false;
            if (start.charAt(i) == 'R' && i > j)
                return false;
            if (start.charAt(i) == 'L' && i < j)
                return false;
            ++i;
            ++j;
        }
        return true;
    }
}
```

1386. 234.java

Path: LeetCode-main/solutions/234. Palindrome Linked List/234.java

```
class Solution {
    public boolean isPalindrome(ListNode head) {
        ListNode slow = head;
        ListNode fast = head;

        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
        }

        if (fast != null)
            slow = slow.next;
        slow = reverseList(slow);

        while (slow != null) {
            if (slow.val != head.val)
                return false;
            slow = slow.next;
            head = head.next;
        }

        return true;
    }

    private ListNode reverseList(ListNode head) {
        ListNode prev = null;
        while (head != null) {
            ListNode next = head.next;
            head.next = prev;
            prev = head;
            head = next;
        }
        return prev;
    }
}
```

1387. 2340.java

Path: LeetCode-main/solutions/2340. Minimum Adjacent Swaps to Make a Valid Array/2340.java

```
class Solution {
    public int minimumSwaps(int[] nums) {
        final int minIndex = getLeftmostMinIndex(nums);
        final int maxIndex = getRightmostMaxIndex(nums);
        final int swaps = minIndex + (nums.length - 1 - maxIndex);
        return minIndex <= maxIndex ? swaps : swaps - 1;
    }

    private int getLeftmostMinIndex(int[] nums) {
        int mn = nums[0];
        int minIndex = 0;
        for (int i = 1; i < nums.length; ++i)
            if (nums[i] < mn) {
                mn = nums[i];
                minIndex = i;
            }
        return minIndex;
    }

    int getRightmostMaxIndex(int[] nums) {
        int mx = nums[nums.length - 1];
        int maxIndex = nums.length - 1;
        for (int i = nums.length - 2; i >= 0; --i)
            if (nums[i] > mx) {
                mx = nums[i];
                maxIndex = i;
            }
        return maxIndex;
    }
}
```

1388. 2341.java

Path: LeetCode-main/solutions/2341. Maximum Number of Pairs in Array/2341.java

```
class Solution {
    public int[] numberOfPairs(int[] nums) {
        final int MAX = 100;
        int[] ans = new int[2];
        int[] count = new int[MAX + 1];

        for (final int num : nums)
            ++count[num];

        for (int i = 0; i <= MAX; ++i) {
            ans[0] += count[i] / 2;
            if (count[i] % 2 == 1)
                ++ans[1];
        }

        return ans;
    }
}
```

1389. 2342.java

Path: LeetCode-main/solutions/2342. Max Sum of a Pair With Equal Sum of Digits/2342.java

```
class Solution {
    public int maximumSum(int[] nums) {
        final int MAX = 9 * 9; // 999,999,999
        int ans = -1;
        List<Integer>[] count = new List[MAX + 1];

        for (int i = 0; i <= MAX; ++i)
            count[i] = new ArrayList<>();

        for (final int num : nums)
            count[getDigitSum(num)].add(num);

        for (List<Integer> groupNums : count) {
            if (groupNums.size() < 2)
                continue;
            Collections.sort(groupNums, Collections.reverseOrder());
            ans = Math.max(ans, groupNums.get(0) + groupNums.get(1));
        }

        return ans;
    }

    private int getDigitSum(int num) {
        int digitSum = 0;
        while (num > 0) {
            digitSum += num % 10;
            num /= 10;
        }
        return digitSum;
    }
}
```

1390. 2344.java

Path: LeetCode-main/solutions/2344. Minimum Deletions to Make Array Divisible/2344.java

```
class Solution {
    public int minOperations(int[] nums, int[] numsDivide) {
        final int gcd = getGCD(numsDivide);

        Arrays.sort(nums);

        for (int i = 0; i < nums.length; ++i)
            if (gcd % nums[i] == 0)
                return i;

        return -1;
    }

    private int getGCD(int[] nums) {
        int g = nums[0];
        for (final int num : nums)
            g = gcd(g, num);
        return g;
    }

    private int gcd(int a, int b) {
        return b == 0 ? a : gcd(b, a % b);
    }
}
```

1391. 2345.java

Path: LeetCode-main/solutions/2345. Finding the Number of Visible Mountains/2345.java

```
class Solution {
    public int visibleMountains(int[][] peaks) {
        int ans = 0;
        int maxRightFoot = 0;

        Arrays.sort(peaks, Comparator.comparingInt((int[] peak) -> peak[0] - peak[1])
            .thenComparing((int[] peak) -> peak[0], Comparator.reverseOrder()));

        for (int i = 0; i < peaks.length; ++i) {
            final boolean overlapWithNext = i + 1 < peaks.length && Arrays.equals(peaks[i], peaks[i + 1]);
            final int currRightFoot = peaks[i][0] + peaks[i][1];
            if (currRightFoot > maxRightFoot) {
                if (!overlapWithNext)
                    ++ans;
                maxRightFoot = currRightFoot;
            }
        }

        return ans;
    }
}
```


1392. 2347.java

Path: LeetCode-main/solutions/2347. Best Poker Hand/2347.java

```
class Solution {
    public String bestHand(int[] ranks, char[] suits) {
        if (new String(suits).chars().allMatch(s -> s == suits[0]))
            return "Flush";

        final int MAX = 13;
        int[] count = new int[MAX + 1];

        for (final int rank : ranks)
            ++count[rank];

        final int mx = Arrays.stream(count).max().getAsInt();
        if (mx > 2)
            return "Three of a Kind";
        if (mx == 2)
            return "Pair";
        return "High Card";
    }
}
```

1393. 2348.java

Path: LeetCode-main/solutions/2348. Number of Zero-Filled Subarrays/2348.java

```
class Solution {
    public long zeroFilledSubarray(int[] nums) {
        long ans = 0;
        int indexBeforeZero = -1;

        for (int i = 0; i < nums.length; ++i)
            if (nums[i] == 0)
                ans += i - indexBeforeZero;
            else
                indexBeforeZero = i;

        return ans;
    }
}
```

1394. 2349.java

Path: LeetCode-main/solutions/2349. Design a Number Container System/2349.java

```
class NumberContainers {
    public void change(int index, int number) {
        if (indexToNumbers.containsKey(index)) {
            final int originalNumber = indexToNumbers.get(index);
            numberToIndices.get(originalNumber).remove(index);
            if (numberToIndices.get(originalNumber).isEmpty())
                numberToIndices.remove(originalNumber);
        }
        indexToNumbers.put(index, number);
        numberToIndices.putIfAbsent(number, new TreeSet<>());
        numberToIndices.get(number).add(index);
    }

    public int find(int number) {
        if (numberToIndices.containsKey(number))
            return numberToIndices.get(number).first();
        return -1;
    }

    private Map<Integer, TreeSet<Integer>> numberToIndices = new HashMap<>();
    private Map<Integer, Integer> indexToNumbers = new HashMap<>();
}
```

1395. 235.java

Path: LeetCode-main/solutions/235. Lowest Common Ancestor of a Binary Search Tree/235.java

```
class Solution {
    public TreeNode lowestCommonAncestor(TreeNode root, TreeNode p, TreeNode q) {
        if (root.val > Math.max(p.val, q.val))
            return lowestCommonAncestor(root.left, p, q);
        if (root.val < Math.min(p.val, q.val))
            return lowestCommonAncestor(root.right, p, q);
        return root;
    }
}
```

1396. 2350.java

Path: LeetCode-main/solutions/2350. Shortest Impossible Sequence of Rolls/2350.java

```
class Solution {
    public int shortestSequence(int[] rolls, int k) {
        int ans = 1; // the the next target length
        Set<Integer> seen = new HashSet<>();

        for (final int roll : rolls) {
            seen.add(roll);
            if (seen.size() == k) {
                // Have all combinations that form `ans` length, and we are going to
                // extend the sequence to `ans + 1` length.
                ++ans;
                seen.clear();
            }
        }
        return ans;
    }
}
```

1397. 2351.java

Path: LeetCode-main/solutions/2351. First Letter to Appear Twice/2351.java

```
class Solution {
    public char repeatedCharacter(String s) {
        boolean[] seen = new boolean[26];

        for (final char c : s.toCharArray()) {
            if (seen[c - 'a'])
                return c;
            seen[c - 'a'] = true;
        }

        throw new IllegalArgumentException();
    }
}
```

1398. 2352.java

Path: LeetCode-main/solutions/2352. Equal Row and Column Pairs/2352.java

```
class Solution {
    public int equalPairs(int[][] grid) {
        final int n = grid.length;
        int ans = 0;

        for (int i = 0; i < n; ++i)
            for (int j = 0; j < n; ++j) {
                int k = 0;
                for (; k < n; ++k)
                    if (grid[i][k] != grid[k][j])
                        break;
                if (k == n) // R[i] == C[j]
                    ++ans;
            }
        return ans;
    }
}
```

1399. 2353.java

Path: LeetCode-main/solutions/2353. Design a Food Rating System/2353.java

```
class FoodRatings {
    public FoodRatings(String[] foods, String[] cuisines, int[] ratings) {
        for (int i = 0; i < foods.length; ++i) {
            cuisineToRatingAndFoods.putIfAbsent(
                cuisines[i],
                new TreeSet<>(
                    Comparator.comparing(Pair<Integer, String>::getKey, Comparator.reverseOrder())
                        .thenComparing(Pair<Integer, String>::getValue));
            cuisineToRatingAndFoods.get(cuisines[i]).add(new Pair<>(ratings[i], foods[i]));
            foodToCuisine.put(foods[i], cuisines[i]);
            foodToRating.put(foods[i], ratings[i]);
        }
    }

    public void changeRating(String food, int newRating) {
        final String cuisine = foodToCuisine.get(food);
        final int oldRating = foodToRating.get(food);
        TreeSet<Pair<Integer, String>> ratingAndFoods = cuisineToRatingAndFoods.get(cuisine);
        ratingAndFoods.remove(new Pair<>(oldRating, food));
        ratingAndFoods.add(new Pair<>(newRating, food));
        foodToRating.put(food, newRating);
    }

    public String highestRated(String cuisine) {
        return cuisineToRatingAndFoods.get(cuisine).first().getValue();
    }
}

// {cuisine: {(rating, food)}}
Map<String, TreeSet<Pair<Integer, String>>> cuisineToRatingAndFoods = new HashMap<>();
Map<String, String> foodToCuisine = new HashMap<>();
Map<String, Integer> foodToRating = new HashMap<>();
```


1400. 2354.java

Path: LeetCode-main/solutions/2354. Number of Excellent Pairs/2354.java

```
class Solution {
    public long countExcellentPairs(int[] nums, int k) {
        final int MAX_BIT = 30;
        // bits(num1 | num2) + bits(num1 & num2) = bits(num1) + bits(num2)
        long ans = 0;
        long[] count = new long[MAX_BIT];

        for (final int num : Arrays.stream(nums).boxed().collect(Collectors.toSet()))
            ++count[Integer.bitCount(num)];

        for (int i = 0; i < MAX_BIT; ++i)
            for (int j = 0; j < MAX_BIT; ++j)
                if (i + j >= k)
                    ans += count[i] * count[j];

        return ans;
    }
}
```

1401. 2355.java

Path: LeetCode-main/solutions/2355. Maximum Number of Books You Can Take/2355.java

```
class Solution {
    public long maximumBooks(int[] books) {
        // dp[i] := the maximum number of books we can take from books[0..i] with taking all of
        // books[i]
        long[] dp = new long[books.length];
        Deque<Integer> stack = new ArrayDeque<>(); // the possible indices we can reach

        for (int i = 0; i < books.length; ++i) {
            // We may take all of books[j], where books[j] < books[i] - (i - j).
            while (!stack.isEmpty() && books[stack.peek()] >= books[i] - (i - stack.peek()))
                stack.pop();
            // We can now take books[j + 1..i].
            final int j = stack.isEmpty() ? -1 : stack.peek();
            final int lastTook = books[i] - (i - j) + 1;
            if (lastTook > 1)
                // books[i] + (books[i] - 1) + ... + (books[i] - (i - j) + 1)
                dp[i] = (long) (books[i] + lastTook) * (i - j) / 2;
            else
                // 1 + 2 + ... + books[i]
                dp[i] = (long) books[i] * (books[i] + 1) / 2;
            if (j >= 0)
                dp[i] += dp[j];
            stack.push(i);
        }

        return Arrays.stream(dp).max().getAsLong();
    }
}
```

1402. 2357.java

Path: LeetCode-main/solutions/2357. Make Array Zero by Subtracting Equal Amounts/2357.java

```
class Solution {  
    public int minimumOperations(int[] nums) {  
        Set<Integer> seen = Arrays.stream(nums).boxed().collect(Collectors.toSet());  
        return seen.size() - (seen.contains(0) ? 1 : 0);  
    }  
}
```

1403. 2358.java

Path: LeetCode-main/solutions/2358. Maximum Number of Groups Entering a Competition/2358.java

```
class Solution {
    public int maximumGroups(int[] grades) {
        // Sort grades, then we can separate the students into groups of sizes 1, 2,
        // 3, ..., k, s.t. the i-th group < the (i + 1)-th group for both sum and
        // size. So, we can rephrase the problem into:
        // Find the maximum k s.t.  $1 + 2 + 3 + \dots + k \leq n$ 

        //  $1 + 2 + 3 + \dots + k \leq n$ 
        //  $k(k + 1) / 2 \leq n$ 
        //  $k^2 + k \leq 2n$ 
        //  $(k + 0.5)^2 - 0.25 \leq 2n$ 
        //  $(k + 0.5)^2 \leq 2n + 0.25$ 
        //  $k \leq \sqrt{2n + 0.25} - 0.5$ 
        return (int) (Math.sqrt(grades.length * 2 + 0.25) - 0.5);
    }
}
```

1404. 2359.java

Path: LeetCode-main/solutions/2359. Find Closest Node to Given Two Nodes/2359.java

```
class Solution {
    public int closestMeetingNode(int[] edges, int node1, int node2) {
        final int MAX = 10000;
        final int[] dist1 = getDist(edges, node1);
        final int[] dist2 = getDist(edges, node2);
        int minDist = MAX;
        int ans = -1;

        for (int i = 0; i < edges.length; ++i)
            if (Math.min(dist1[i], dist2[i]) >= 0) {
                final int maxDist = Math.max(dist1[i], dist2[i]);
                if (maxDist < minDist) {
                    minDist = maxDist;
                    ans = i;
                }
            }

        return ans;
    }

    private int[] getDist(int[] edges, int u) {
        int[] dist = new int[edges.length];
        Arrays.fill(dist, -1);
        int d = 0;
        while (u != -1 && dist[u] == -1) {
            dist[u] = d++;
            u = edges[u];
        }
        return dist;
    }
}
```

1405. 236-2.java

Path: LeetCode-main/solutions/236. Lowest Common Ancestor of a Binary Tree/236-2.java

```
class Solution {
    public TreeNode lowestCommonAncestor(TreeNode root, TreeNode p, TreeNode q) {
        Queue<TreeNode> q_ = new ArrayDeque<>(List.of(root));
        Map<TreeNode, TreeNode> parent = new HashMap<>();
        parent.put(root, null);
        Set<TreeNode> ancestors = new HashSet<>(); // p's ancestors

        // Iterate until we find both p and q.
        while (!parent.containsKey(p) || !parent.containsKey(q)) {
            root = q_.poll();
            if (root.left != null) {
                parent.put(root.left, root);
                q_.offer(root.left);
            }
            if (root.right != null) {
                parent.put(root.right, root);
                q_.offer(root.right);
            }
        }

        // Insert all the p's ancestors.
        while (p != null) {
            ancestors.add(p);
            p = parent.get(p); // `p` becomes null in the end.
        }

        // Go up from q until we meet any of p's ancestors.
        while (!ancestors.contains(q))
            q = parent.get(q);

        return q;
    }
}
```

1406. 236.java

Path: LeetCode-main/solutions/236. Lowest Common Ancestor of a Binary Tree/236.java

```
class Solution {
    public TreeNode lowestCommonAncestor(TreeNode root, TreeNode p, TreeNode q) {
        if (root == null || root == p || root == q)
            return root;
        TreeNode left = lowestCommonAncestor(root.left, p, q);
        TreeNode right = lowestCommonAncestor(root.right, p, q);
        if (left != null && right != null)
            return root;
        return left == null ? right : left;
    }
}
```

1407. 2360.java

Path: LeetCode-main/solutions/2360. Longest Cycle in a Graph/2360.java

```
class Solution {
    public int longestCycle(int[] edges) {
        int ans = -1;
        int time = 1;
        int[] timeVisited = new int[edges.length];

        for (int i = 0; i < edges.length; ++i) {
            if (timeVisited[i] > 0)
                continue;
            final int startTime = time;
            int u = i;
            while (u != -1 && timeVisited[u] == 0) {
                timeVisited[u] = time++;
                u = edges[u]; // Move to the next node.
            }
            if (u != -1 && timeVisited[u] >= startTime)
                ans = Math.max(ans, time - timeVisited[u]);
        }

        return ans;
    }
}
```


1408. 2361.java

Path: LeetCode-main/solutions/2361. Minimum Costs Using the Train Line/2361.java

```
class Solution {
    public long[] minimumCosts(int[] regular, int[] express, int expressCost) {
        final int n = regular.length;
        long[] ans = new long[n];
        // the minimum cost to reach the current stop in a regular route
        long dpReg = 0;
        // the minimum cost to reach the current stop in an express route
        long dpExp = expressCost;

        for (int i = 0; i < n; ++i) {
            final long prevReg = dpReg;
            final long prevExp = dpExp;
            dpReg = Math.min(prevReg + regular[i], prevExp + 0 + regular[i]);
            dpExp = Math.min(prevReg + expressCost + express[i], prevExp + express[i]);
            ans[i] = Math.min(dpReg, dpExp);
        }

        return ans;
    }
}
```

1409. 2363.java

Path: LeetCode-main/solutions/2363. Merge Similar Items/2363.java

```
class Solution {
    public List<List<Integer>> mergeSimilarItems(int[][] items1, int[][] items2) {
        final int MAX = 1000;
        List<List<Integer>> ans = new ArrayList<>();
        int[] count = new int[MAX + 1];

        for (int[] item : items1)
            count[item[0]] += item[1];

        for (int[] item : items2)
            count[item[0]] += item[1];

        for (int i = 1; i <= MAX; ++i)
            if (count[i] > 0)
                ans.add(Arrays.asList(i, count[i]));

        return ans;
    }
}
```

1410. 2364.java

Path: LeetCode-main/solutions/2364. Count Number of Bad Pairs/2364.java

```
class Solution {
    public long countBadPairs(int[] nums) {
        long ans = 0;
        Map<Integer, Long> count = new HashMap<>(); // (nums[i] - i)

        for (int i = 0; i < nums.length; ++i) {
            // count[nums[i] - i] := the number of good pairs
            // i - count[nums[i] - i] := the number of bad pairs
            ans += i - countOrDefault(nums[i] - i, 0L);
            count.merge(nums[i] - i, 1L, Long::sum);
        }

        return ans;
    }
}
```

1411. 2365.java

Path: LeetCode-main/solutions/2365. Task Scheduler II/2365.java

```
class Solution {
    public long taskSchedulerII(int[] tasks, int space) {
        Map<Integer, Long> taskToNextAvailable = new HashMap<>();
        long ans = 0;

        for (final int task : tasks) {
            ans = Math.max(ans + 1, taskToNextAvailable.getOrDefault(task, 0L));
            taskToNextAvailable.put(task, ans + space + 1);
        }

        return ans;
    }
}
```

1412. 2366.java

Path: LeetCode-main/solutions/2366. Minimum Replacements to Sort the Array/2366.java

```
class Solution {
    public long minimumReplacement(int[] nums) {
        long ans = 0;
        int mx = nums[nums.length - 1];

        for (int i = nums.length - 2; i >= 0; --i) {
            final int ops = (nums[i] - 1) / mx;
            ans += ops;
            mx = nums[i] / (ops + 1);
        }

        return ans;
    }
}
```

1413. 2367.java

Path: LeetCode-main/solutions/2367. Number of Arithmetic Triplets/2367.java

```
class Solution {
    public int arithmeticTriplets(int[] nums, int diff) {
        final int MAX = 200;
        int ans = 0;
        boolean[] count = new boolean[MAX + 1];

        for (final int num : nums) {
            if (num >= 2 * diff && count[num - diff] && count[num - 2 * diff])
                ++ans;
            count[num] = true;
        }

        return ans;
    }
}
```

1414. 2368.java

Path: LeetCode-main/solutions/2368. Reachable Nodes With Restrictions/2368.java

```
class Solution {
    public int reachableNodes(int n, int[][] edges, int[] restricted) {
        List<Integer>[] tree = new List[n];
        boolean[] seen = new boolean[n];

        for (int i = 0; i < n; ++i)
            tree[i] = new ArrayList<>();

        for (int[] edge : edges) {
            final int u = edge[0];
            final int v = edge[1];
            tree[u].add(v);
            tree[v].add(u);
        }

        for (final int r : restricted)
            seen[r] = true;

        return dfs(tree, 0, seen);
    }

    private int dfs(List<Integer>[] tree, int u, boolean[] seen) {
        if (seen[u])
            return 0;

        seen[u] = true;
        int ans = 1;

        for (final int v : tree[u])
            ans += dfs(tree, v, seen);

        return ans;
    }
}
```

1415. 2369.java

Path: LeetCode-main/solutions/2369. Check if There is a Valid Partition For The Array/2369.java

```
class Solution {
    public boolean validPartition(int[] nums) {
        final int n = nums.length;
        // dp[i] := true if there's a valid partition for the first i numbers
        boolean[] dp = new boolean[n + 1];
        dp[0] = true;
        dp[2] = nums[0] == nums[1];

        for (int i = 3; i <= n; ++i)
            dp[i] = (dp[i - 2] && nums[i - 2] == nums[i - 1]) ||
                    (dp[i - 3] && ((nums[i - 3] == nums[i - 2] && nums[i - 2] == nums[i - 1]) ||
                                   (nums[i - 3] + 1 == nums[i - 2] && nums[i - 2] + 1 == nums[i - 1]))));

        return dp[n];
    }
}
```


1416. 237.java

Path: LeetCode-main/solutions/237. Delete Node in a Linked List/237.java

```
class Solution {  
    public void deleteNode(ListNode node) {  
        node.val = node.next.val;  
        node.next = node.next.next;  
    }  
}
```

1417. 2370.java

Path: LeetCode-main/solutions/2370. Longest Ideal Subsequence/2370.java

```
class Solution {
    public int longestIdealString(String s, int k) {
        // dp[i] := the longest subsequence that ends in ('a' + i)
        int[] dp = new int[26];

        for (final char c : s.toCharArray()) {
            final int i = c - 'a';
            dp[i] = 1 + getMaxReachable(dp, i, k);
        }

        return Arrays.stream(dp).max().getAsInt();
    }

    private int getMaxReachable(int[] dp, int i, int k) {
        final int first = Math.max(0, i - k);
        final int last = Math.min(25, i + k);
        int maxReachable = 0;
        for (int j = first; j <= last; ++j)
            maxReachable = Math.max(maxReachable, dp[j]);
        return maxReachable;
    }
}
```

1418. 2371.java

Path: LeetCode-main/solutions/2371. Minimize Maximum Value in a Grid/2371.java

```
class Solution {
    public int[][] minScore(int[][] grid) {
        final int m = grid.length;
        final int n = grid[0].length;
        int[][] ans = new int[m][n];
        List<int[]> valAndIndices = new ArrayList<>();
        int[] rows = new int[m]; // rows[i] := the maximum used number so far
        int[] cols = new int[n]; // cols[j] := the maximum used number so far

        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j)
                valAndIndices.add(new int[] {grid[i][j], i, j});

        Collections.sort(valAndIndices, Comparator.comparingInt(valAndIndex -> valAndIndex[0]));

        for (int[] valAndIndex : valAndIndices) {
            final int i = valAndIndex[1];
            final int j = valAndIndex[2];
            final int nextAvailable = Math.max(rows[i], cols[j]) + 1;
            ans[i][j] = nextAvailable;
            rows[i] = nextAvailable;
            cols[j] = nextAvailable;
        }

        return ans;
    }
}
```

1419. 2373.java

Path: LeetCode-main/solutions/2373. Largest Local Values in a Matrix/2373.java

```
class Solution {
    public int[][] largestLocal(int[][] grid) {
        final int n = grid.length;
        int[][] ans = new int[n - 2][n - 2];

        for (int i = 0; i < n - 2; ++i)
            for (int j = 0; j < n - 2; ++j)
                for (int x = i; x < i + 3; ++x)
                    for (int y = j; y < j + 3; ++y)
                        ans[i][j] = Math.max(ans[i][j], grid[x][y]);

        return ans;
    }
}
```

1420. 2374.java

Path: LeetCode-main/solutions/2374. Node With Highest Edge Score/2374.java

```
class Solution {
    public int edgeScore(int[] edges) {
        int ans = 0;
        long[] scores = new long[edges.length];

        for (int i = 0; i < edges.length; ++i)
            scores[edges[i]] += i;

        for (int i = 1; i < scores.length; ++i)
            if (scores[i] > scores[ans])
                ans = i;

        return ans;
    }
}
```

1421. 2375.java

Path: LeetCode-main/solutions/2375. Construct Smallest Number From DI String/2375.java

```
class Solution {
    public String smallestNumber(String pattern) {
        StringBuilder sb = new StringBuilder();
        Deque<Character> stack = new ArrayDeque<>(List.of('1'));

        for (final char c : pattern.toCharArray()) {
            char maxSorFar = stack.peek();
            if (c == 'I')
                while (!stack.isEmpty()) {
                    maxSorFar = (char) Math.max(maxSorFar, stack.peek());
                    sb.append(stack.pop());
                }
            stack.push((char) (maxSorFar + 1));
        }

        while (!stack.isEmpty())
            sb.append(stack.pop());

        return sb.toString();
    }
}
```

1422. 2376.java

Path: LeetCode-main/solutions/2376. Count Special Integers/2376.java

```
class Solution {
    // Same as 1012. Numbers With Repeated Digits
    public int countSpecialNumbers(int n) {
        final int digitSize = (int) Math.log10(n) + 1;
        Integer[][][] mem = new Integer[digitSize + 1][1 << 10][2];
        return count(String.valueOf(n), 0, 0, true, mem) - 1; // - 0;
    }

    // Returns the number of special integers, considering the i-th digit, where
    // `used` is the bitmask of the used digits, and `tight` indicates if the
    // current digit is tightly bound.
    private int count(final String s, int i, int used, boolean tight, Integer[][][] mem) {
        if (i == s.length())
            return 1;
        if (mem[i][used][tight ? 1 : 0] != null)
            return mem[i][used][tight ? 1 : 0];

        int res = 0;
        final int maxDigit = tight ? s.charAt(i) - '0' : 9;

        for (int d = 0; d <= maxDigit; ++d) {
            // `d` is used.
            if ((used >> d & 1) == 1)
                continue;
            // Use `d` now.
            final boolean nextTight = tight && (d == maxDigit);
            if (used == 0 && d == 0) // Don't count leading 0s as used.
                res += count(s, i + 1, used, nextTight, mem);
            else
                res += count(s, i + 1, used | 1 << d, nextTight, mem);
        }

        return mem[i][used][tight ? 1 : 0] = res;
    }
}
```

1423. 2378.java

Path: LeetCode-main/solutions/2378. Choose Edges to Maximize Score in a Tree/2378.java

```
class Solution {
    public long maxScore(int[][] edges) {
        final int n = edges.length;
        List
```


1424. 2379.java

Path: LeetCode-main/solutions/2379. Minimum Recolors to Get K Consecutive Black Blocks/2379.java

```
class Solution {
    public int minimumRecolors(String blocks, int k) {
        int countB = 0;
        int maxCountB = 0;

        for (int i = 0; i < blocks.length(); ++i) {
            if (blocks.charAt(i) == 'B')
                ++countB;
            if (i >= k && blocks.charAt(i - k) == 'B')
                --countB;
            maxCountB = Math.max(maxCountB, countB);
        }

        return k - maxCountB;
    }
}
```

1425. 238-2.java

Path: LeetCode-main/solutions/238. Product of Array Except Self/238-2.java

```
class Solution {
    public int[] productExceptSelf(int[] nums) {
        final int n = nums.length;
        int[] ans = new int[n];
        ans[0] = 1;

        // Use ans as the prefix product array.
        for (int i = 1; i < n; ++i)
            ans[i] = ans[i - 1] * nums[i - 1];

        int suffix = 1; // suffix product
        for (int i = n - 1; i >= 0; --i) {
            ans[i] *= suffix;
            suffix *= nums[i];
        }

        return ans;
    }
}
```

1426. 238.java

Path: LeetCode-main/solutions/238. Product of Array Except Self/238.java

```
class Solution {
    public int[] productExceptSelf(int[] nums) {
        final int n = nums.length;
        int[] ans = new int[n]; // Can also use `nums` as the ans array.
        int[] prefix = new int[n]; // prefix product
        int[] suffix = new int[n]; // suffix product

        prefix[0] = 1;
        for (int i = 1; i < n; ++i)
            prefix[i] = prefix[i - 1] * nums[i - 1];

        suffix[n - 1] = 1;
        for (int i = n - 2; i >= 0; --i)
            suffix[i] = suffix[i + 1] * nums[i + 1];

        for (int i = 0; i < n; ++i)
            ans[i] = prefix[i] * suffix[i];

        return ans;
    }
}
```

1427. 2380.java

Path: LeetCode-main/solutions/2380. Time Needed to Rearrange a Binary String/2380.java

```
class Solution {
    public int secondsToRemoveOccurrences(String s) {
        int ans = 0;
        int zeros = 0;

        for (final char c : s.toCharArray())
            if (c == '0')
                ++zeros;
            else if (zeros > 0) // c == '1'
                ans = Math.max(ans + 1, zeros);

        return ans;
    }
}
```

1428. 2381.java

Path: LeetCode-main/solutions/2381. Shifting Letters II/2381.java

```
class Solution {
    public String shiftingLetters(String s, int[][] shifts) {
        StringBuilder sb = new StringBuilder();
        int currShift = 0;
        int[] line = new int[s.length() + 1];

        for (int[] shift : shifts) {
            final int start = shift[0];
            final int end = shift[1];
            final int direction = shift[2];
            final int diff = direction == 1 ? 1 : -1;
            line[start] += diff;
            line[end + 1] -= diff;
        }

        for (int i = 0; i < s.length(); ++i) {
            currShift = (currShift + line[i]) % 26;
            final int num = (s.charAt(i) - 'a' + currShift + 26) % 26;
            sb.append((char) ('a' + num));
        }

        return sb.toString();
    }
}
```

1429. 2382.java

Path: LeetCode-main/solutions/2382. Maximum Segment Sum After Removals/2382.java

```
class Solution {
    public long[] maximumSegmentSum(int[] nums, int[] removeQueries) {
        final int n = nums.length;
        long maxSum = 0;
        long[] ans = new long[n];
        // For the segment [l, r], record its sum in sum[l] and sum[r]
        long[] sum = new long[n];
        // For the segment [l, r], record its count in count[l] and count[r]
        int[] count = new int[n];

        for (int i = n - 1; i >= 0; --i) {
            ans[i] = maxSum;
            final int j = removeQueries[i];

            // Calculate `segmentSum`.
            final long leftSum = j > 0 ? sum[j - 1] : 0;
            final long rightSum = j + 1 < n ? sum[j + 1] : 0;
            final long segmentSum = nums[j] + leftSum + rightSum;

            // Calculate `segmentCount`.
            final int leftCount = j > 0 ? count[j - 1] : 0;
            final int rightCount = j + 1 < n ? count[j + 1] : 0;
            final int segmentCount = 1 + leftCount + rightCount;

            // Update the sum and count of the segment [l, r].
            final int l = j - leftCount;
            final int r = j + rightCount;
            sum[l] = segmentSum;
            sum[r] = segmentSum;
            count[l] = segmentCount;
            count[r] = segmentCount;
            maxSum = Math.max(maxSum, segmentSum);
        }

        return ans;
    }
}
```

1430. 2383.java

Path: LeetCode-main/solutions/2383. Minimum Hours of Training to Win a Competition/2383.java

```
class Solution {
    public int minNumberOfHours(int initialEnergy, int initialExperience, int[] energy,
                                int[] experience) {
        return getRequiredEnergy(initialEnergy, energy) +
            getRequiredExperience(initialExperience, experience);
    }

    private int getRequiredEnergy(int initialEnergy, int[] energy) {
        return Math.max(0, Arrays.stream(energy).sum() + 1 - initialEnergy);
    }

    private int getRequiredExperience(int currentExperience, int[] experience) {
        int requiredExperience = 0;
        for (final int e : experience) {
            if (e >= currentExperience) {
                requiredExperience += e + 1 - currentExperience;
                currentExperience += e + 1 - currentExperience;
            }
            currentExperience += e;
        }
        return requiredExperience;
    }
}
```

1431. 2384.java

Path: LeetCode-main/solutions/2384. Largest Palindromic Number/2384.java

```
class Solution {
    public String largestPalindromic(String num) {
        Map<Character, Integer> count = new HashMap<>();

        for (final char c : num.toCharArray())
            count.merge(c, 1, Integer::sum);

        final String firstHalf = getFirstHalf(count);
        final String mid = getMid(count);
        final String ans = firstHalf + mid + reversed(firstHalf);
        return ans.isEmpty() ? "0" : ans;
    }

    private String getFirstHalf(Map<Character, Integer> count) {
        StringBuilder sb = new StringBuilder();
        for (char c = '9'; c >= '0'; --c) {
            final int freq = count.getOrDefault(c, 0);
            sb.append(String.valueOf(c).repeat(freq / 2));
        }
        final int index = firstNotZeroIndex(sb);
        return index == -1 ? "" : sb.substring(index);
    }

    private int firstNotZeroIndex(StringBuilder sb) {
        for (int i = 0; i < sb.length(); ++i)
            if (sb.charAt(i) != '0')
                return i;
        return -1;
    }

    private String getMid(Map<Character, Integer> count) {
        StringBuilder sb = new StringBuilder();
        for (char c = '9'; c >= '0'; --c) {
            final int freq = count.getOrDefault(c, 0);
            if (freq % 2 == 1)
                return String.valueOf(c);
        }
        return "";
    }

    private String reversed(final String s) {
        return new StringBuilder(s).reverse().toString();
    }
}
```


1432. 2385.java

Path: LeetCode-main/solutions/2385. Amount of Time for Binary Tree to Be Infected/2385.java

```
class Solution {
    public int amountOfTime(TreeNode root, int start) {
        int ans = -1;
        Map<Integer, List<Integer>> graph = getGraph(root);
        Queue<Integer> q = new ArrayDeque<>(List.of(start));
        Set<Integer> seen = new HashSet<>(Arrays.asList(start));

        for (; !q.isEmpty(); ++ans) {
            for (int sz = q.size(); sz > 0; --sz) {
                final int u = q.poll();
                if (!graph.containsKey(u))
                    continue;
                for (final int v : graph.get(u)) {
                    if (seen.contains(v))
                        continue;
                    q.offer(v);
                    seen.add(v);
                }
            }
        }

        return ans;
    }

    private Map<Integer, List<Integer>> getGraph(TreeNode root) {
        Map<Integer, List<Integer>> graph = new HashMap<>();
        // (node, parent)
        Queue<Pair<TreeNode, Integer>> q = new ArrayDeque<>(List.of(new Pair<>(root, -1)));

        while (!q.isEmpty()) {
            Pair<TreeNode, Integer> pair = q.poll();
            TreeNode node = pair.getKey();
            final int parent = pair.getValue();
            if (parent != -1) {
                graph.putIfAbsent(parent, new ArrayList<>());
                graph.putIfAbsent(node.val, new ArrayList<>());
                graph.get(parent).add(node.val);
                graph.get(node.val).add(parent);
            }
            if (node.left != null)
                q.add(new Pair<>(node.left, node.val));
            if (node.right != null)
                q.add(new Pair<>(node.right, node.val));
        }

        return graph;
    }
}
```

1433. 2386.java

Path: LeetCode-main/solutions/2386. Find the K-Sum of an Array/2386.java

```
class Solution {
    public long kSum(int[] nums, int k) {
        final long maxSum = getMaxSum(nums);
        final int[] absNums = getAbsNums(nums);
        long ans = maxSum;
        // (the next maximum sum, the next index i)
        Queue<Pair<Long, Integer>> maxHeap =
            new PriorityQueue<>((a, b) -> Long.compare(b.getKey(), a.getKey()));
        maxHeap.offer(new Pair<>(maxSum - absNums[0], 0));

        for (int j = 0; j < k - 1; ++j) {
            Pair<Long, Integer> pair = maxHeap.poll();
            final long nextMaxSum = pair.getKey();
            final int i = pair.getValue();
            ans = nextMaxSum;
            if (i + 1 < absNums.length) {
                maxHeap.offer(new Pair<>(nextMaxSum - absNums[i + 1], i + 1));
                maxHeap.offer(new Pair<>(nextMaxSum - absNums[i + 1] + absNums[i], i + 1));
            }
        }

        return ans;
    }

    private long getMaxSum(int[] nums) {
        long maxSum = 0;
        for (final int num : nums)
            if (num > 0)
                maxSum += num;
        return maxSum;
    }

    private int[] getAbsNums(int[] nums) {
        for (int i = 0; i < nums.length; ++i)
            nums[i] = Math.abs(nums[i]);
        Arrays.sort(nums);
        return nums;
    }
}
```

1434. 2387.java

Path: LeetCode-main/solutions/2387. Median of a Row Wise Sorted Matrix/2387.java

```
class Solution {
    public int matrixMedian(int[][] grid) {
        final int noGreaterThanMedianCount = grid.length * grid[0].length / 2 + 1;
        int l = 1;
        int r = 1_000_000;

        while (l < r) {
            final int m = (l + r) / 2;
            if (numsNoGreaterThan(grid, m) >= noGreaterThanMedianCount)
                r = m;
            else
                l = m + 1;
        }

        return l;
    }

    private int numsNoGreaterThan(int[][] grid, int m) {
        int count = 0;
        for (int[] row : grid)
            count += firstGreater(row, m);
        return count;
    }

    private int firstGreater(int[] arr, int target) {
        int l = 0;
        int r = arr.length;
        while (l < r) {
            final int m = (l + r) / 2;
            if (arr[m] > target)
                r = m;
            else
                l = m + 1;
        }
        return l;
    }
}
```

1435. 2389.java

Path: LeetCode-main/solutions/2389. Longest Subsequence With Limited Sum/2389.java

```
class Solution {
    public int[] answerQueries(int[] nums, int[] queries) {
        int[] ans = new int[queries.length];

        Arrays.sort(nums);

        for (int i = 0; i < queries.length; ++i)
            ans[i] = numOfElementsLessThan(nums, queries[i]);

        return ans;
    }

    private int numOfElementsLessThan(int[] nums, int query) {
        int sum = 0;
        for (int i = 0; i < nums.length; ++i) {
            sum += nums[i];
            if (sum > query)
                return i;
        }
        return nums.length;
    }
}
```

1436. 239.java

Path: LeetCode-main/solutions/239. Sliding Window Maximum/239.java

```
class Solution {
    public int[] maxSlidingWindow(int[] nums, int k) {
        int[] ans = new int[nums.length - k + 1];
        Deque<Integer> maxQ = new ArrayDeque<>();

        for (int i = 0; i < nums.length; ++i) {
            while (!maxQ.isEmpty() && maxQ.peekLast() < nums[i])
                maxQ.pollLast();
            maxQ.offerLast(nums[i]);
            if (i >= k && nums[i - k] == maxQ.peekFirst()) // out-of-bounds
                maxQ.pollFirst();
            if (i >= k - 1)
                ans[i - k + 1] = maxQ.peekFirst();
        }

        return ans;
    }
}
```

1437. 2390.java

Path: LeetCode-main/solutions/2390. Removing Stars From a String/2390.java

```
class Solution {  
    public String removeStars(String s) {  
        StringBuilder sb = new StringBuilder();  
        for (final char c : s.toCharArray())  
            if (c == '*')  
                sb.deleteCharAt(sb.length() - 1);  
            else  
                sb.append(c);  
        return sb.toString();  
    }  
}
```

1438. 2391.java

Path: LeetCode-main/solutions/2391. Minimum Amount of Time to Collect Garbage/2391.java

```
class Solution {
    public int garbageCollection(String[] garbage, int[] travel) {
        int[] prefix = new int[travel.length];
        prefix[0] = travel[0];
        for (int i = 1; i < prefix.length; ++i)
            prefix[i] += prefix[i - 1] + travel[i];
        final int timeM = getTime(garbage, prefix, 'M');
        final int timeP = getTime(garbage, prefix, 'P');
        final int timeG = getTime(garbage, prefix, 'G');
        return timeM + timeP + timeG;
    }

    private int getTime(String[] garbage, int[] prefix, char c) {
        int characterCount = 0;
        int lastIndex = -1;
        for (int i = 0; i < garbage.length; ++i) {
            final String s = garbage[i];
            if (s.chars().anyMatch(g -> g == c))
                lastIndex = i;
            characterCount += (int) s.chars().filter(g -> g == c).count();
        }
        return characterCount + (lastIndex <= 0 ? 0 : prefix[lastIndex - 1]);
    }
}
```

1439. 2392.java

Path: LeetCode-main/solutions/2392. Build a Matrix With Conditions/2392.java

```
class Solution {
    public int[][] buildMatrix(int k, int[][] rowConditions, int[][] colConditions) {
        List<Integer> rowOrder = topologicalSort(rowConditions, k);
        if (rowOrder.isEmpty())
            return new int[][] {};

        List<Integer> colOrder = topologicalSort(colConditions, k);
        if (colOrder.isEmpty())
            return new int[][] {};

        int[][] ans = new int[k][k];
        int[] nodeToRowIndex = new int[k + 1];

        for (int i = 0; i < k; ++i)
            nodeToRowIndex[rowOrder.get(i)] = i;

        for (int j = 0; j < k; ++j) {
            final int node = colOrder[j];
            final int i = nodeToRowIndex[node];
            ans[i][j] = node;
        }

        return ans;
    }

    private List<Integer> topologicalSort(int[][] conditions, int n) {
        List<Integer> order = new ArrayList<>();
        List<Integer>[] graph = new List[n + 1];
        int[] inDegrees = new int[n + 1];
        Queue<Integer> q = new ArrayDeque<>();

        for (int i = 1; i <= n; ++i)
            graph[i] = new ArrayList<>();

        // Build the graph.
        for (int[] condition : conditions) {
            final int u = condition[0];
            final int v = condition[1];
            graph[u].add(v);
            ++inDegrees[v];
        }

        // Perform topological sorting.
        for (int i = 1; i <= n; ++i)
            if (inDegrees[i] == 0)
                q.offer(i);

        while (!q.isEmpty()) {
            final int u = q.poll();
            order.add(u);
            for (final int v : graph[u])
                if (--inDegrees[v] == 0)
                    q.offer(v);
        }

        return order.size() == n ? order : new ArrayList<>();
    }
}
```


1440. 2393.java

Path: LeetCode-main/solutions/2393. Count Strictly Increasing Subarrays/2393.java

```
class Solution {
    public long countSubarrays(int[] nums) {
        long ans = 0;

        for (int i = 0, j = -1; i < nums.length; ++i) {
            if (i > 0 && nums[i] <= nums[i - 1])
                j = i - 1;
            ans += i - j;
        }

        return ans;
    }
}
```

1441. 2395.java

Path: LeetCode-main/solutions/2395. Find Subarrays With Equal Sum/2395.java

```
class Solution {
    public boolean findSubarrays(int[] nums) {
        Set<Integer> seen = new HashSet<>();

        for (int i = 1; i < nums.length; ++i) {
            final int sum = nums[i - 1] + nums[i];
            if (seen.contains(sum))
                return true;
            seen.add(sum);
        }

        return false;
    }
}
```

1442. 2396.java

Path: LeetCode-main/solutions/2396. Strictly Palindromic Number/2396.java

```
class Solution {  
    public boolean isStrictlyPalindromic(int n) {  
        return false;  
    }  
}
```

1443. 2397.java

Path: LeetCode-main/solutions/2397. Maximum Rows Covered by Columns/2397.java

```
class Solution {
    public int maximumRows(int[][] matrix, int numSelect) {
        dfs(matrix, /*colIndex=*/0, numSelect, /*mask=*/0);
        return ans;
    }

    private int ans = 0;

    private void dfs(int[][] matrix, int colIndex, int leftColsCount, int mask) {
        if (leftColsCount == 0) {
            ans = Math.max(ans, getAllZerosRowCount(matrix, mask));
            return;
        }
        if (colIndex == matrix[0].length)
            return;

        // Choose this column.
        dfs(matrix, colIndex + 1, leftColsCount - 1, mask | 1 << colIndex);
        // Don't choose this column.
        dfs(matrix, colIndex + 1, leftColsCount, mask);
    }

    int getAllZerosRowCount(int[][] matrix, int mask) {
        int count = 0;
        for (int[] row : matrix) {
            boolean isAllZeros = true;
            for (int i = 0; i < row.length; ++i) {
                if (row[i] == 1 && (mask >> i & 1) == 0) {
                    isAllZeros = false;
                    break;
                }
            }
            if (isAllZeros)
                ++count;
        }
        return count;
    }
}
```

1444. 2398.java

Path: LeetCode-main/solutions/2398. Maximum Number of Robots Within Budget/2398.java

```
class Solution {
    public int maximumRobots(int[] chargeTimes, int[] runningCosts, long budget) {
        long cost = 0;
        // Stores `chargeTimes[i]`.
        Deque<Integer> maxQ = new ArrayDeque<>();

        int j = 0; // window's range := [i..j], so k = i - j + 1
        for (int i = 0; i < chargeTimes.length; ++i) {
            cost += runningCosts[i];
            while (!maxQ.isEmpty() && maxQ.peekLast() < chargeTimes[i])
                maxQ.pollLast();
            maxQ.offerLast(chargeTimes[i]);
            if (maxQ.peekFirst() + (i - j + 1) * cost > budget) {
                if (maxQ.peekFirst() == chargeTimes[j])
                    maxQ.pollFirst();
                cost -= runningCosts[j++];
            }
        }
        return chargeTimes.length - j;
    }
}
```

1445. 2399.java

Path: LeetCode-main/solutions/2399. Check Distances Between Same Letters/2399.java

```
class Solution {
    public boolean checkDistances(String s, int[] distance) {
        int[] firstSeenIndex = new int[26];
        Arrays.fill(firstSeenIndex, -1);

        for (int i = 0; i < s.length(); ++i) {
            final int j = s.charAt(i) - 'a';
            final int prevIndex = firstSeenIndex[j];
            if (prevIndex != -1 && i - prevIndex - 1 != distance[j])
                return false;
            firstSeenIndex[j] = i;
        }

        return true;
    }
}
```

1446. 24.java

Path: LeetCode-main/solutions/24. Swap Nodes in Pairs/24.java

```
class Solution {
    public ListNode swapPairs(ListNode head) {
        final int length = getLength(head);
        ListNode dummy = new ListNode(0, head);
        ListNode prev = dummy;
        ListNode curr = head;

        for (int i = 0; i < length / 2; ++i) {
            ListNode next = curr.next;
            curr.next = next.next;
            next.next = curr;
            prev.next = next;
            prev = curr;
            curr = curr.next;
        }

        return dummy.next;
    }

    private int getLength(ListNode head) {
        int length = 0;
        for (ListNode curr = head; curr != null; curr = curr.next)
            ++length;
        return length;
    }
}
```

1447. 240.java

Path: LeetCode-main/solutions/240. Search a 2D Matrix II/240.java

```
class Solution {
    public boolean searchMatrix(int[][] matrix, int target) {
        int r = 0;
        int c = matrix[0].length - 1;

        while (r <= matrix.length && c >= 0) {
            if (matrix[r][c] == target)
                return true;
            if (matrix[r][c] > target)
                --c;
            else
                ++r;
        }
        return false;
    }
}
```


1448. 2400.java

Path: LeetCode-main/solutions/2400. Number of Ways to Reach a Position After Exactly k Steps/2400.java

```
class Solution {
    public int numberOfWays(int startPos, int endPos, int k) {
        // leftStep + rightStep = k
        // rightStep - leftStep = endPos - startPos
        //      2 * rightStep = k + endPos - startPos
        //      rightStep = (k + endPos - startPos) / 2
        final int val = k + endPos - startPos;
        if (val < 0 || val % 2 == 1)
            return 0;
        final int rightStep = val / 2;
        final int leftStep = k - rightStep;
        if (leftStep < 0)
            return 0;
        return nCk(leftStep + rightStep, Math.min(leftStep, rightStep));
    }

    // C(n, k) = C(n - 1, k) + C(n - 1, k - 1)
    private int nCk(int n, int k) {
        final int MOD = 1_000_000_007;
        // dp[i] := C(n so far, i)
        int[] dp = new int[k + 1];
        dp[0] = 1;

        while (n-- > 0) // Calculate n times.
            for (int j = k; j > 0; --j) {
                dp[j] += dp[j - 1];
                dp[j] %= MOD;
            }

        return dp[k];
    }
}
```

1449. 2401.java

Path: LeetCode-main/solutions/2401. Longest Nice Subarray/2401.java

```
class Solution {
    public int longestNiceSubarray(int[] nums) {
        int ans = 0;
        int used = 0;

        for (int l = 0, r = 0; r < nums.length; ++r) {
            while ((used & nums[r]) > 0)
                used ^= nums[l++];
            used |= nums[r];
            ans = Math.max(ans, r - l + 1);
        }

        return ans;
    }
}
```

1450. 2402.java

Path: LeetCode-main/solutions/2402. Meeting Rooms III/2402.java

```
class Solution {
    public int mostBooked(int n, int[][] meetings) {
        record T(long endTime, int roomId) {}
        int[] count = new int[n];

        Arrays.sort(meetings, Comparator.comparingInt(meeting -> meeting[0]));

        Queue<T> occupied =
            new PriorityQueue<>(Comparator.comparingLong(T::endTime).thenComparingInt(T::roomId));
        Queue<Integer> availableRoomIds = new PriorityQueue<>();

        for (int i = 0; i < n; ++i)
            availableRoomIds.offer(i);

        for (int[] meeting : meetings) {
            final int start = meeting[0];
            final int end = meeting[1];
            // Push meetings ending before this `meeting` in occupied to the
            // `availableRoomIds`.
            while (!occupied.isEmpty() && occupied.peek().endTime <= start)
                availableRoomIds.offer(occupied.poll().roomId);
            if (availableRoomIds.isEmpty()) {
                T t = occupied.poll();
                ++count[t.roomId];
                occupied.offer(new T(t.endTime + (end - start), t.roomId));
            } else {
                final int roomId = availableRoomIds.poll();
                ++count[roomId];
                occupied.offer(new T(end, roomId));
            }
        }

        int maxIndex = 0;
        for (int i = 0; i < n; ++i)
            if (count[i] > count[maxIndex])
                maxIndex = i;
        return maxIndex;
    }
}
```

1451. 2403.java

Path: LeetCode-main/solutions/2403. Minimum Time to Kill All Monsters/2403.java

```
class Solution {
    public long minimumTime(int[] power) {
        final int n = power.length;
        final int maxMask = 1 << n;
        // dp[i] := the minimum number of days needed to defeat the monsters, where
        // i is the bitmask of the monsters
        long[] dp = new long[maxMask];
        Arrays.fill(dp, Long.MAX_VALUE);
        dp[0] = 0;

        for (int mask = 1; mask < maxMask; ++mask) {
            final int currentGain = Integer.bitCount(mask);
            for (int i = 0; i < n; ++i)
                if ((mask >> i & 1) == 1)
                    dp[mask] = Math.min(dp[mask], dp[mask & ~(1 << i)] +
                                         (int) (Math.ceil(power[i] * 1.0 / currentGain)));
        }

        return dp[maxMask - 1];
    }
}
```

1452. 2404.java

Path: LeetCode-main/solutions/2404. Most Frequent Even Element/2404.java

```
class Solution {
    public int mostFrequentEven(int[] nums) {
        int ans = -1;
        Map<Integer, Integer> count = new HashMap<>();

        for (final int num : nums) {
            if (num % 2 == 1)
                continue;
            final int newCount = count.merge(num, 1, Integer::sum);
            final int maxCount = count.getOrDefault(ans, 0);
            if (newCount > maxCount || newCount == maxCount && num < ans)
                ans = num;
        }
        return ans;
    }
}
```

1453. 2405.java

Path: LeetCode-main/solutions/2405. Optimal Partition of String/2405.java

```
class Solution {
    public int partitionString(String s) {
        int ans = 1;
        int used = 0;

        for (final char c : s.toCharArray()) {
            final int i = c - 'a';
            if ((used >> i & 1) == 1) {
                used = 1 << i;
                ++ans;
            } else {
                used |= 1 << i;
            }
        }
        return ans;
    }
}
```

1454. 2406.java

Path: LeetCode-main/solutions/2406. Divide Intervals Into Minimum Number of Groups/2406.java

```
class Solution {
    // Similar to 253. Meeting Rooms II
    public int minGroups(int[][] intervals) {
        // Stores `right`s.
        Queue<Integer> minHeap = new PriorityQueue<>();

        Arrays.sort(intervals, Comparator.comparingInt(interval -> interval[0]));

        for (int[] interval : intervals) {
            // There's no overlap, so we can reuse the same group.
            if (!minHeap.isEmpty() && interval[0] > minHeap.peek())
                minHeap.poll();
            minHeap.offer(interval[1]);
        }

        return minHeap.size();
    }
}
```

1455. 2408.java

Path: LeetCode-main/solutions/2408. Design SQL/2408.java

```
class SQL {
    public SQL(List<String> names, List<Integer> columns) {}

    public void insertRow(String name, List<String> row) {
        db.putIfAbsent(name, new ArrayList<>());
        db.get(name).add(row);
    }

    public void deleteRow(String name, int rowId) {}

    public String selectCell(String name, int rowId, int columnId) {
        return db.get(name).get(rowId - 1).get(columnId - 1);
    }

    private Map<String, List<List<String>>> db = new HashMap<>();
}
```


1456. 2409.java

Path: LeetCode-main/solutions/2409. Count Days Spent Together/2409.java

```
class Solution {
    public int countDaysTogether(String arriveAlice, String leaveAlice, String arriveBob,
                                String leaveBob) {
        final int arriveA = toDays(arriveAlice);
        final int leaveA = toDays(leaveAlice);
        final int arriveB = toDays(arriveBob);
        final int leaveB = toDays(leaveBob);
        int ans = 0;

        for (int day = 1; day <= 365; ++day)
            if (arriveA <= day && day <= leaveA && arriveB <= day && day <= leaveB)
                ++ans;

        return ans;
    }

    private final int[] days = {0, 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};

    private int toDays(final String s) {
        final int month = (s.charAt(0) - '0') * 10 + (s.charAt(1) - '0');
        final int day = (s.charAt(3) - '0') * 10 + (s.charAt(4) - '0');
        int prevDays = 0;
        for (int m = 1; m < month; ++m)
            prevDays += days[m];
        return prevDays + day;
    }
}
```

1457. 241.java

Path: LeetCode-main/solutions/241. Different Ways to Add Parentheses/241.java

```
class Solution {
    public List<Integer> diffWaysToCompute(String expression) {
        return ways(expression, new HashMap<>());
    }

    private List<Integer> ways(final String s, Map<String, List<Integer>> mem) {
        if (mem.containsKey(s))
            return mem.get(s);

        List<Integer> ans = new ArrayList<>();

        for (int i = 0; i < s.length(); ++i)
            if (!Character.isDigit(s.charAt(i)))
                for (final int a : ways(s.substring(0, i), mem))
                    for (final int b : ways(s.substring(i + 1), mem))
                        if (s.charAt(i) == '+')
                            ans.add(a + b);
                        else if (s.charAt(i) == '-')
                            ans.add(a - b);
                        else
                            ans.add(a * b);

        if (ans.isEmpty()) { // Single number
            mem.put(s, Arrays.asList(Integer.parseInt(s)));
            return mem.get(s);
        }
        mem.put(s, ans);
        return ans;
    }
}
```

1458. 2410.java

Path: LeetCode-main/solutions/2410. Maximum Matching of Players With Trainers/2410.java

```
class Solution {
    public int matchPlayersAndTrainers(int[] players, int[] trainers) {
        int ans = 0;

        Arrays.sort(players);
        Arrays.sort(trainers);

        for (int i = 0; i < trainers.length; ++i)
            if (players[ans] <= trainers[i] && ++ans == players.length)
                return ans;

        return ans;
    }
}
```

1459. 2411.java

Path: LeetCode-main/solutions/2411. Smallest Subarrays With Maximum Bitwise OR/2411.java

```
class Solution {
    public int[] smallestSubarrays(int[] nums) {
        final int MAX_BIT = 30;
        int[] ans = new int[nums.length];
        // closest[j] := the closest index i s.t. the j-th bit of nums[i] is 1
        int[] closest = new int[MAX_BIT];

        Arrays.fill(ans, 1);

        for (int i = nums.length - 1; i >= 0; --i)
            for (int j = 0; j < MAX_BIT; ++j) {
                if ((nums[i] >> j & 1) == 1)
                    closest[j] = i;
                ans[i] = Math.max(ans[i], closest[j] - i + 1);
            }

        return ans;
    }
}
```

1460. 2412.java

Path: LeetCode-main/solutions/2412. Minimum Money Required Before Transactions/2412.java

```
class Solution {
    public long minimumMoney(int[][] transactions) {
        long ans = 0;
        long losses = 0;

        // Before picking the final transaction, perform any transaction that raises
        // the required money.
        for (int[] t : transactions) {
            final int cost = t[0];
            final int cashback = t[1];
            losses += Math.max(0, cost - cashback);
        }

        // Now, pick a transaction to be the final one.
        for (int[] t : transactions) {
            final int cost = t[0];
            final int cashback = t[1];
            if (cost > cashback)
                // The losses except this transaction: losses - (cost - cashback), so
                // add the cost of this transaction = losses - (cost - cashback) + cost.
                ans = Math.max(ans, losses + cashback);
            else
                // The losses except this transaction: losses, so add the cost of this
                // transaction = losses + cost.
                ans = Math.max(ans, losses + cost);
        }

        return ans;
    }
}
```

1461. 2413.java

Path: LeetCode-main/solutions/2413. Smallest Even Multiple/2413.java

```
class Solution {  
    public int smallestEvenMultiple(int n) {  
        return n * (n % 2 + 1);  
    }  
}
```

1462. 2414.java

Path: LeetCode-main/solutions/2414. Length of the Longest Alphabetical Continuous Substring/2414.java

```
class Solution {
    public int longestContinuousSubstring(String s) {
        int ans = 1;
        int runningLen = 1;

        for (int i = 1; i < s.length(); ++i)
            if (s.charAt(i) == s.charAt(i - 1) + 1)
                ans = Math.max(ans, ++runningLen);
            else
                runningLen = 1;

        return ans;
    }
}
```

1463. 2415.java

Path: LeetCode-main/solutions/2415. Reverse Odd Levels of Binary Tree/2415.java

```
class Solution {
    public TreeNode reverseOddLevels(TreeNode root) {
        dfs(root.left, root.right, true);
        return root;
    }

    private void dfs(TreeNode left, TreeNode right, boolean isOddLevel) {
        if (left == null)
            return;
        if (isOddLevel) {
            final int val = left.val;
            left.val = right.val;
            right.val = val;
        }
        dfs(left.left, right.right, !isOddLevel);
        dfs(left.right, right.left, !isOddLevel);
    }
}
```


1464. 2416.java

Path: LeetCode-main/solutions/2416. Sum of Prefix Scores of Strings/2416.java

```
class TrieNode {
    public TrieNode[] children = new TrieNode[26];
    public int count = 0;
}

class Solution {
    public int[] sumPrefixScores(String[] words) {
        int[] ans = new int[words.length];

        for (final String word : words)
            insert(word);

        for (int i = 0; i < words.length; ++i)
            ans[i] = getScore(words[i]);

        return ans;
    }

    private TrieNode root = new TrieNode();

    private void insert(final String word) {
        TrieNode node = root;
        for (final char c : word.toCharArray()) {
            final int i = c - 'a';
            if (node.children[i] == null)
                node.children[i] = new TrieNode();
            node = node.children[i];
            ++node.count;
        }
    }

    private int getScore(final String word) {
        TrieNode node = root;
        int score = 0;
        for (final char c : word.toCharArray()) {
            node = node.children[c - 'a'];
            score += node.count;
        }
        return score;
    }
}
```

1465. 2417.java

Path: LeetCode-main/solutions/2417. Closest Fair Integer/2417.java

```
class Solution {
    public int closestFair(int n) {
        final int digitsCount = String.valueOf(n).length();
        return digitsCount % 2 == 0 ? getEvenDigits(n) : (int) getOddDigits(digitsCount);
    }

    private long getOddDigits(int digitsCount) {
        final int zeros = (digitsCount + 1) / 2;
        final int ones = (digitsCount - 1) / 2;
        return Integer.valueOf('1' + "0".repeat(zeros) + "1".repeat(ones));
    }

    private int getEvenDigits(int n) {
        final int digitsCount = String.valueOf(n).length();
        final long maxNum = Long.valueOf('1' + "0".repeat(digitsCount));
        for (long num = n; num < maxNum; ++num)
            if (isValidNum(num))
                return (int) num;
        return (int) getOddDigits(digitsCount + 1);
    }

    private boolean isValidNum(long num) {
        int count = 0;
        for (final char c : String.valueOf(num).toCharArray())
            count += (c - '0') % 2 == 0 ? 1 : -1;
        return count == 0;
    }
}
```

1466. 2418.java

Path: LeetCode-main/solutions/2418. Sort the People/2418.java

```
class Solution {
    public String[] sortPeople(String[] names, int[] heights) {
        List<Pair<Integer, String>> heightAndNames = new ArrayList<>();

        for (int i = 0; i < names.length; ++i)
            heightAndNames.add(new Pair<>(heights[i], names[i]));

        Collections.sort(heightAndNames, (a, b) -> b.getKey().compareTo(a.getKey()));

        for (int i = 0; i < heightAndNames.size(); ++i)
            names[i] = heightAndNames.get(i).getValue();

        return names;
    }
}
```

1467. 2419.java

Path: LeetCode-main/solutions/2419. Longest Subarray With Maximum Bitwise AND/2419.java

```
class Solution {
    public int longestSubarray(int[] nums) {
        int ans = 0;
        int maxIndex = 0;
        int sameNumLength = 0;

        for (int i = 0; i < nums.length; ++i)
            if (nums[i] == nums[maxIndex]) {
                ans = Math.max(ans, ++sameNumLength);
            } else if (nums[i] > nums[maxIndex]) {
                maxIndex = i;
                sameNumLength = 1;
                ans = 1;
            } else {
                sameNumLength = 0;
            }

        return ans;
    }
}
```

1468. 242.java

Path: LeetCode-main/solutions/242. Valid Anagram/242.java

```
class Solution {
    public boolean isAnagram(String s, String t) {
        if (s.length() != t.length())
            return false;

        int[] count = new int[26];

        for (final char c : s.toCharArray())
            ++count[c - 'a'];

        for (final char c : t.toCharArray()) {
            if (count[c - 'a'] == 0)
                return false;
            --count[c - 'a'];
        }

        return true;
    }
}
```

1469. 2420.java

Path: LeetCode-main/solutions/2420. Find All Good Indices/2420.java

```
class Solution {
    // Same as 2100. Find Good Days to Rob the Bank
    public List<Integer> goodIndices(int[] nums, int k) {
        final int n = nums.length;
        List<Integer> ans = new ArrayList<>();
        // dec[i] := 1 + the number of continuous decreasing numbers before i
        int[] dec = new int[n];
        // inc[i] := 1 + the number of continuous increasing numbers after i
        int[] inc = new int[n];

        Arrays.fill(dec, 1);
        Arrays.fill(inc, 1);

        for (int i = 1; i < n; ++i)
            if (nums[i - 1] >= nums[i])
                dec[i] = dec[i - 1] + 1;

        for (int i = n - 2; i >= 0; --i)
            if (nums[i] <= nums[i + 1])
                inc[i] = inc[i + 1] + 1;

        for (int i = k; i < n - k; ++i)
            if (dec[i - 1] >= k && inc[i + 1] >= k)
                ans.add(i);

        return ans;
    }
}
```

1470. 2421.java

Path: LeetCode-main/solutions/2421. Number of Good Paths/2421.java

```
class UnionFind {
    public UnionFind(int n) {
        id = new int[n];
        rank = new int[n];
        for (int i = 0; i < n; ++i)
            id[i] = i;
    }

    public void unionByRank(int u, int v) {
        final int i = find(u);
        final int j = find(v);
        if (i == j)
            return;
        if (rank[i] < rank[j]) {
            id[i] = j;
        } else if (rank[i] > rank[j]) {
            id[j] = i;
        } else {
            id[i] = j;
            ++rank[j];
        }
    }

    public int find(int u) {
        return id[u] == u ? u : (id[u] = find(id[u]));
    }

    private int[] id;
    private int[] rank;
}

class Solution {
    public int numberOfGoodPaths(int[] vals, int[][] edges) {
        final int n = vals.length;
        int ans = 0;
        UnionFind uf = new UnionFind(n);
        List<Integer>[] tree = new List[n];
        Map<Integer, List<Integer>> valToNodes = new TreeMap<>();

        for (int i = 0; i < n; ++i)
            tree[i] = new ArrayList<>();

        for (int[] edge : edges) {
            final int u = edge[0];
            final int v = edge[1];
            if (vals[v] <= vals[u])
                tree[u].add(v);
            if (vals[u] <= vals[v])
                tree[v].add(u);
        }

        for (int i = 0; i < vals.length; ++i) {
            valToNodes.putIfAbsent(vals[i], new ArrayList<>());
            valToNodes.get(vals[i]).add(i);
        }

        for (Map.Entry<Integer, List<Integer>> entry : valToNodes.entrySet()) {
            final int val = entry.getKey();
            List<Integer> nodes = entry.getValue();
            for (final int u : nodes)
                for (final int v : tree[u])
                    uf.unionByRank(u, v);
            Map<Integer, Integer> rootCount = new HashMap<>();
            for (final int u : nodes)
                rootCount.merge(uf.find(u), 1, Integer::sum);
            // For each group, C(count, 2) := count * (count - 1) / 2
            for (final int count : rootCount.values())
                ans += count * (count - 1) / 2;
        }

        return ans;
    }
}
```

1471. 2422.java

Path: LeetCode-main/solutions/2422. Merge Operations to Turn Array Into a Palindrome/2422.java

```
class Solution {
    public int minimumOperations(int[] nums) {
        int ans = 0;
        int l = 0;
        int r = nums.length - 1;
        long leftSum = nums[l];
        long rightSum = nums[r];

        while (l < r)
            if (leftSum < rightSum) {
                leftSum += nums[++l];
                ++ans;
            } else if (leftSum > rightSum) {
                rightSum += nums[--r];
                ++ans;
            } else { // LeftSum == rightSum
                leftSum = nums[++l];
                rightSum = nums[--r];
            }

        return ans;
    }
}
```


1472. 2423.java

Path: LeetCode-main/solutions/2423. Remove Letter To Equalize Frequency/2423.java

```
class Solution {
    public boolean equalFrequency(String word) {
        int[] count = new int[26];

        for (final char c : word.toCharArray())
            ++count[c - 'a'];

        // Try to remove each letter, then check if the frequency of all the letters
        // in `word` are equal.
        for (final char c : word.toCharArray()) {
            --count[c - 'a'];
            if (equalFreq(count))
                return true;
            ++count[c - 'a'];
        }

        return false;
    }

    private static final int MAX = 101;

    private boolean equalFreq(int[] count) {
        int minfreq = MAX;
        int maxfreq = 0;
        for (final int freq : count)
            if (freq > 0) {
                minfreq = Math.min(minfreq, freq);
                maxfreq = Math.max(maxfreq, freq);
            }
        return minfreq == maxfreq;
    }
}
```

1473. 2424.java

Path: LeetCode-main/solutions/2424. Longest Uploaded Prefix/2424.java

```
class LUPrefix {
    public LUPrefix(int n) {}

    public void upload(int video) {
        seen.add(video);
        while (seen.contains(longestPrefix + 1))
            ++longestPrefix;
    }

    public int longest() {
        return longestPrefix;
    }

    private Set<Integer> seen = new HashSet<>();
    private int longestPrefix = 0;
}
```

1474. 2425.java

Path: LeetCode-main/solutions/2425. Bitwise XOR of All Pairings/2425.java

```
class Solution {
    public int xorAllNums(int[] nums1, int[] nums2) {
        // If the size of nums1 is m and the size of nums2 is n, then each number in
        // nums1 is repeated n times and each number in nums2 is repeated m times.
        final int xors1 = Arrays.stream(nums1).reduce((a, b) -> a ^ b).getAsInt();
        final int xors2 = Arrays.stream(nums2).reduce((a, b) -> a ^ b).getAsInt();
        return (nums1.length % 2 * xors2) ^ (nums2.length % 2 * xors1);
    }
}
```

1475. 2427.java

Path: LeetCode-main/solutions/2427. Number of Common Factors/2427.java

```
class Solution {
    public int commonFactors(int a, int b) {
        int ans = 1;
        final int gcd = gcd(a, b);
        for (int i = 2; i <= gcd; ++i)
            if (a % i == 0 && b % i == 0)
                ++ans;
        return ans;
    }

    private int gcd(int a, int b) {
        return b == 0 ? a : gcd(b, a % b);
    }
}
```

1476. 2428.java

Path: LeetCode-main/solutions/2428. Maximum Sum of an Hourglass/2428.java

```
class Solution {
    public int maxSum(int[][] grid) {
        int ans = 0;

        for (int i = 1; i + 1 < grid.length; ++i)
            for (int j = 1; j + 1 < grid[0].length; ++j)
                ans = Math.max(ans, grid[i - 1][j - 1] + grid[i - 1][j] + grid[i - 1][j + 1] + grid[i][j] +
                                grid[i + 1][j - 1] + grid[i + 1][j] + grid[i + 1][j + 1]);

        return ans;
    }
}
```

1477. 2429.java

Path: LeetCode-main/solutions/2429. Minimize XOR/2429.java

```
class Solution {
    public int minimizeXor(int num1, int num2) {
        final int MAX_BIT = 30;
        int bits = Integer.bitCount(num2);
        // Can turn off all the bits in `num1`.
        if (Integer.bitCount(num1) == bits)
            return num1;

        int ans = 0;

        // Turn off the MSB if we have `bits` quota.
        for (int i = MAX_BIT - 1; i >= 0; --i)
            if ((num1 >> i & 1) == 1) {
                ans |= 1 << i;
                if (--bits == 0)
                    return ans;
            }

        // Turn on the LSB if we still have `bits`.
        for (int i = 0; i < MAX_BIT; ++i)
            if ((num1 >> i & 1) == 0) {
                ans |= 1 << i;
                if (--bits == 0)
                    return ans;
            }

        return ans;
    }
}
```

1478. 243.java

Path: LeetCode-main/solutions/243. Shortest Word Distance/243.java

```
class Solution {
    public int shortestDistance(String[] wordsDict, String word1, String word2) {
        int ans = wordsDict.length;
        int index1 = -1; // wordsDict[index1] == word1
        int index2 = -1; // wordsDict[index2] == word2

        for (int i = 0; i < wordsDict.length; ++i) {
            if (wordsDict[i].equals(word1)) {
                index1 = i;
                if (index2 != -1)
                    ans = Math.min(ans, index1 - index2);
            }
            if (wordsDict[i].equals(word2)) {
                index2 = i;
                if (index1 != -1)
                    ans = Math.min(ans, index2 - index1);
            }
        }

        return ans;
    }
}
```

1479. 2430.java

Path: LeetCode-main/solutions/2430. Maximum Deletions on a String/2430.java

```
class Solution {
    public int deleteString(String s) {
        final int n = s.length();
        // lcs[i][j] := the number of the same letters of s[i..n) and s[j..n)
        int[][] lcs = new int[n + 1][n + 1];
        // dp[i] := the maximum number of operations needed to delete s[i..n)
        int[] dp = new int[n];
        Arrays.fill(dp, 1);

        for (int i = n - 1; i >= 0; --i)
            for (int j = i + 1; j < n; ++j) {
                if (s.charAt(i) == s.charAt(j))
                    lcs[i][j] = lcs[i + 1][j + 1] + 1;
                if (lcs[i][j] >= j - i)
                    dp[i] = Math.max(dp[i], dp[j] + 1);
            }

        return dp[0];
    }
}
```


1480. 2431-2.java

Path: LeetCode-main/solutions/2431. Maximize Total Tastiness of Purchased Fruits/2431-2.java

```
class Solution {
    public int maxTastiness(int[] price, int[] tastiness, int maxAmount, int maxCoupons) {
        // dp[j][k] := the maximum tastiness of price so far with j amount of money and k
        // coupons
        int[][] dp = new int[maxAmount + 1][maxCoupons + 1];

        for (int i = 0; i < price.length; ++i)
            for (int j = maxAmount; j >= price[i] / 2; --j)
                for (int k = maxCoupons; k >= 0; --k) {
                    final int buyWithCoupon = k == 0 ? 0 : dp[j - price[i] / 2][k - 1] + tastiness[i];
                    final int buyWithoutCoupon = j < price[i] ? 0 : dp[j - price[i]][k] + tastiness[i];
                    dp[j][k] = Math.max(dp[j][k], Math.max(buyWithCoupon, buyWithoutCoupon));
                }

        return dp[maxAmount][maxCoupons];
    }
}
```

1481. 2431.java

Path: LeetCode-main/solutions/2431. Maximize Total Tastiness of Purchased Fruits/2431.java

```
class Solution {
    public int maxTastiness(int[] price, int[] tastiness, int maxAmount, int maxCoupons) {
        final int n = price.length;
        // dp[i][j][k] := the maximum tastiness of the first i price with j amount
        // of money and k coupons
        int[][][] dp = new int[price.length + 1][maxAmount + 1][maxCoupons + 1];

        for (int i = 1; i <= n; ++i) {
            // 1-indexed
            final int currPrice = price[i - 1];
            final int currTastiness = tastiness[i - 1];
            for (int amount = 0; amount <= maxAmount; ++amount) {
                for (int coupon = 0; coupon <= maxCoupons; ++coupon) {
                    // 1. Don't buy, the tastiness will be the same as the first i - 1
                    // price.
                    dp[i][amount][coupon] = dp[i - 1][amount][coupon];

                    // 2. Buy without coupon if have enough money.
                    if (amount >= currPrice)
                        dp[i][amount][coupon] = Math.max(dp[i][amount][coupon],
                                                            dp[i - 1][amount - currPrice][coupon] + currTastiness);

                    // 3. Buy with coupon if have coupon and enough money.
                    if (coupon > 0 && amount >= currPrice / 2)
                        dp[i][amount][coupon] =
                            Math.max(dp[i][amount][coupon],
                                        dp[i - 1][amount - currPrice / 2][coupon - 1] + currTastiness);
                }
            }
        }

        return dp[n][maxAmount][maxCoupons];
    }
}
```

1482. 2432.java

Path: LeetCode-main/solutions/2432. The Employee That Worked on the Longest Task/2432.java

```
class Solution {
    public int hardestWorker(int n, int[][] logs) {
        int ans = logs[0][0];
        int maxWorkingTime = logs[0][1];

        for (int i = 1; i < logs.length; ++i) {
            final int id = logs[i][0];
            final int workingTime = logs[i][1] - logs[i - 1][1];
            if (workingTime > maxWorkingTime) {
                ans = id;
                maxWorkingTime = workingTime;
            } else if (workingTime == maxWorkingTime) {
                ans = Math.min(ans, id);
            }
        }

        return ans;
    }
}
```

1483. 2433.java

Path: LeetCode-main/solutions/2433. Find The Original Array of Prefix Xor/2433.java

```
class Solution {  
    public int[] findArray(int[] pref) {  
        int[] ans = new int[pref.length];  
  
        ans[0] = pref[0];  
        for (int i = 1; i < ans.length; ++i)  
            ans[i] = pref[i] ^ pref[i - 1];  
  
        return ans;  
    }  
}
```

1484. 2434.java

Path: LeetCode-main/solutions/2434. Using a Robot to Print the Lexicographically Smallest String/2434.java

```
class Solution {
    public String robotWithString(String s) {
        StringBuilder sb = new StringBuilder();
        int[] count = new int[26];
        Deque<Character> stack = new ArrayDeque<>();

        for (final char c : s.toCharArray())
            ++count[c - 'a'];

        for (final char c : s.toCharArray()) {
            stack.push(c);
            --count[c - 'a'];
            final char minChar = getMinChar(count);
            while (!stack.isEmpty() && stack.peek() <= minChar)
                sb.append(stack.pop());
        }

        while (!stack.isEmpty())
            sb.append(stack.pop());

        return sb.toString();
    }

    private char getMinChar(int[] count) {
        for (int i = 0; i < 26; ++i)
            if (count[i] > 0)
                return (char) ('a' + i);
        return 'a';
    }
}
```

1485. 2435-2.java

Path: LeetCode-main/solutions/2435. Paths in Matrix Whose Sum Is Divisible by K/2435-2.java

```
class Solution {
    public int numberOfPaths(int[][] grid, int k) {
        final int MOD = 1_000_000_007;
        final int m = grid.length;
        final int n = grid[0].length;
        // dp[i][j][sum] := the number of paths to(i, j), where the sum / k == sum
        int[][][] dp = new int[m][n][k];
        dp[0][0][grid[0][0] % k] = 1;

        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j)
                for (int sum = 0; sum < k; ++sum) {
                    final int newSum = (sum + grid[i][j]) % k;
                    if (i > 0)
                        dp[i][j][newSum] += dp[i - 1][j][sum];
                    if (j > 0)
                        dp[i][j][newSum] += dp[i][j - 1][sum];
                    dp[i][j][newSum] %= MOD;
                }

        return dp[m - 1][n - 1][0];
    }
}
```

1486. 2435.java

Path: LeetCode-main/solutions/2435. Paths in Matrix Whose Sum Is Divisible by K/2435.java

```
class Solution {
    public int numberOfPaths(int[][] grid, int k) {
        Integer[][][] mem = new Integer[grid.length][grid[0].length][k];
        return numberOfPaths(grid, 0, 0, 0, k, mem);
    }

    private static final int MOD = 1_000_000_007;

    // Returns the number of paths to (i, j), where the sum / k == `sum`.
    private int numberOfPaths(int[][] grid, int i, int j, int sum, int k, Integer[][][] mem) {
        if (i == grid.length || j == grid[0].length)
            return 0;
        if (i == grid.length - 1 && j == grid[0].length - 1)
            return (sum + grid[i][j]) % k == 0 ? 1 : 0;
        if (mem[i][j][sum] != null)
            return mem[i][j][sum];
        final int newSum = (sum + grid[i][j]) % k;
        return mem[i][j][sum] = (numberOfPaths(grid, i + 1, j, newSum, k, mem) +
                                numberOfPaths(grid, i, j + 1, newSum, k, mem)) %
                                MOD;
    }
}
```

1487. 2436.java

Path: LeetCode-main/solutions/2436. Minimum Split Into Subarrays With GCD Greater Than One/2436.java

```
class Solution {
    public int minimumSplits(int[] nums) {
        int ans = 1;
        int gcd = nums[0];

        for (final int num : nums) {
            final int newGcd = gcd(gcd, num);
            if (newGcd > 1) {
                gcd = newGcd;
            } else {
                gcd = num;
                ++ans;
            }
        }

        return ans;
    }

    private int gcd(int a, int b) {
        return b == 0 ? a : gcd(b, a % b);
    }
}
```


1488. 2437.java

Path: LeetCode-main/solutions/2437. Number of Valid Clock Times/2437.java

```
class Solution {
    public int countTime(String time) {
        int ans = 1;
        if (time.charAt(3) == '?')
            ans *= 6;
        if (time.charAt(4) == '?')
            ans *= 10;

        if (time.charAt(0) == '?' && time.charAt(1) == '?')
            return ans * 24;
        if (time.charAt(0) == '?')
            return time.charAt(1) < '4' ? ans * 3 : ans * 2;
        if (time.charAt(1) == '?')
            return time.charAt(0) == '2' ? ans * 4 : ans * 10;
        return ans;
    }
}
```

1489. 2438.java

Path: LeetCode-main/solutions/2438. Range Product Queries of Powers/2438.java

```
class Solution {
    public int[] productQueries(int n, int[][] queries) {
        final int MOD = 1_000_000_007;
        final int MAX_BIT = 30;
        int[] ans = new int[queries.length];
        List<Integer> pows = new ArrayList<>();

        for (int i = 0; i < MAX_BIT; ++i)
            if ((n >> i & 1) == 1)
                pows.add(1 << i);

        for (int i = 0; i < queries.length; ++i) {
            final int left = queries[i][0];
            final int right = queries[i][1];
            long prod = 1;
            for (int j = left; j <= right; ++j) {
                prod *= pows.get(j);
                prod %= MOD;
            }
            ans[i] = (int) prod;
        }

        return ans;
    }
}
```

1490. 2439.java

Path: LeetCode-main/solutions/2439. Minimize Maximum of Array/2439.java

```
class Solution {
    public int minimizeArrayValue(int[] nums) {
        long ans = 0;
        long prefix = 0;

        for (int i = 0; i < nums.length; ++i) {
            prefix += nums[i];
            final long prefixAvg = (long) Math.ceil(prefix / (double) (i + 1));
            ans = Math.max(ans, prefixAvg);
        }

        return (int) ans;
    }
}
```

1491. 244.java

Path: LeetCode-main/solutions/244. Shortest Word Distance II/244.java

```
class WordDistance {
    public WordDistance(String[] words) {
        for (int i = 0; i < words.length; ++i) {
            wordToIndices.putIfAbsent(words[i], new ArrayList<>());
            wordToIndices.get(words[i]).add(i);
        }
    }

    public int shortest(String word1, String word2) {
        List<Integer> indices1 = wordToIndices.get(word1);
        List<Integer> indices2 = wordToIndices.get(word2);
        int ans = Integer.MAX_VALUE;

        for (int i = 0, j = 0; i < indices1.size() && j < indices2.size(); ) {
            ans = Math.min(ans, Math.abs(indices1.get(i) - indices2.get(j)));
            if (indices1.get(i) < indices2.get(j))
                ++i;
            else
                ++j;
        }

        return ans;
    }

    private Map<String, List<Integer>> wordToIndices = new HashMap<>();
}
```

1492. 2440.java

Path: LeetCode-main/solutions/2440. Create Components With Same Value/2440.java

```
class Solution {
    public int componentValue(int[] nums, int[][] edges) {
        final int n = nums.length;
        final int sum = Arrays.stream(nums).sum();
        List<Integer>[] tree = new List[n];

        for (int i = 0; i < tree.length; ++i)
            tree[i] = new ArrayList<>();

        for (int[] edge : edges) {
            final int u = edge[0];
            final int v = edge[1];
            tree[u].add(v);
            tree[v].add(u);
        }

        for (int i = n; i > 1; --i)
            // Split the tree into i parts, i.e. delete (i - 1) edges.
            if (sum % i == 0 && dfs(nums, tree, 0, sum / i, new boolean[n]) == 0)
                return i - 1;

        return 0;
    }

    private static final int MAX = 1_000_000_000;

    // Returns the sum of the subtree rooted at u subtracting the sum of the deleted subtrees.
    private int dfs(int[] nums, List<Integer>[] tree, int u, int target, boolean[] seen) {
        int sum = nums[u];
        seen[u] = true;

        for (final int v : tree[u]) {
            if (seen[v])
                continue;
            sum += dfs(nums, tree, v, target, seen);
            if (sum > target)
                return MAX;
        }

        // Delete the tree that has sum == target.
        if (sum == target)
            return 0;
        return sum;
    }
}
```

1493. 2441.java

Path: LeetCode-main/solutions/2441. Largest Positive Integer That Exists With Its Negative/2441.java

```
class Solution {
    public int findMaxK(int[] nums) {
        int ans = -1;
        Set<Integer> seen = new HashSet<>();

        for (final int num : nums)
            if (seen.contains(-num))
                ans = Math.max(ans, Math.abs(num));
            else
                seen.add(num);

        return ans;
    }
}
```

1494. 2444.java

Path: LeetCode-main/solutions/2444. Count Subarrays With Fixed Bounds/2444.java

```
class Solution {
    public long countSubarrays(int[] nums, int minK, int maxK) {
        long ans = 0;
        int j = -1;
        int prevMinKIndex = -1;
        int prevMaxKIndex = -1;

        for (int i = 0; i < nums.length; ++i) {
            if (nums[i] < minK || nums[i] > maxK)
                j = i;
            if (nums[i] == minK)
                prevMinKIndex = i;
            if (nums[i] == maxK)
                prevMaxKIndex = i;
            // Any index k in [j + 1, min(prevMinKIndex, prevMaxKIndex)] can be the
            // start of the subarray s.t. nums[k..i] satisfies the conditions.
            ans += Math.max(0, Math.min(prevMinKIndex, prevMaxKIndex) - j);
        }

        return ans;
    }
}
```

1495. 2445.java

Path: LeetCode-main/solutions/2445. Number of Nodes With Value One/2445.java

```
class Solution {
    public int numberOfNodes(int n, int[] queries) {
        // flipped[i] := true if we should flip all the values in the subtree rooted
        // at i
        boolean[] flipped = new boolean[n + 1];

        for (final int query : queries)
            flipped[query] = flipped[query] ^ true;

        return dfs(1, 0, n, flipped);
    }

    private int dfs(int label, int value, int n, boolean[] flipped) {
        if (label > n)
            return 0;
        value ^= flipped[label] ? 1 : 0;
        return value +
            dfs(label * 2, value, n, flipped) + //
            dfs(label * 2 + 1, value, n, flipped);
    }
}
```


1496. 2446.java

Path: LeetCode-main/solutions/2446. Determine if Two Events Have Conflict/2446.java

```
class Solution {  
    public boolean haveConflict(String[] event1, String[] event2) {  
        return event1[0].compareTo(event2[1]) <= 0 && event2[0].compareTo(event1[1]) <= 0;  
    }  
}
```

1497. 2447.java

Path: LeetCode-main/solutions/2447. Number of Subarrays With GCD Equal to K/2447.java

```
class Solution {
    public int subarrayGCD(int[] nums, int k) {
        int ans = 0;
        Map<Integer, Integer> gcds = new HashMap<>();

        for (final int num : nums)
            if (num % k == 0) {
                Map<Integer, Integer> nextGcds = new HashMap<>();
                nextGcds.put(num, 1);
                for (Map.Entry<Integer, Integer> entry : gcds.entrySet()) {
                    final int prevGcd = entry.getKey();
                    final int count = entry.getValue();
                    nextGcds.merge(gcd(prevGcd, num), count, Integer::sum);
                }
                ans += nextGcds.getOrDefault(k, 0);
                gcds = nextGcds;
            } else {
                // The GCD streak stops, so fresh start from the next number.
                gcds.clear();
            }

        return ans;
    }

    private int gcd(int a, int b) {
        return b == 0 ? a : gcd(b, a % b);
    }
}
```

1498. 2448.java

Path: LeetCode-main/solutions/2448. Minimum Cost to Make Array Equal/2448.java

```
class Solution {
    public long minCost(int[] nums, int[] cost) {
        long ans = 0;
        int l = Arrays.stream(nums).min().getAsInt();
        int r = Arrays.stream(nums).max().getAsInt();

        while (l < r) {
            final int m = (l + r) / 2;
            final long cost1 = getCost(nums, cost, m);
            final long cost2 = getCost(nums, cost, m + 1);
            ans = Math.min(cost1, cost2);
            if (cost1 < cost2)
                r = m;
            else
                l = m + 1;
        }

        return ans;
    }

    private long getCost(int[] nums, int[] cost, int target) {
        long res = 0;
        for (int i = 0; i < nums.length; ++i)
            res += Math.abs(nums[i] - target) * (long) cost[i];
        return res;
    }
}
```

1499. 2449.java

Path: LeetCode-main/solutions/2449. Minimum Number of Operations to Make Arrays Similar/2449.java

```
class Solution {
    public long makeSimilar(int[] nums, int[] target) {
        long ans = 0;
        // A[0] := even nums, A[1] := odd nums
        List<Integer>[] A = new List[] {new ArrayList<>(), new ArrayList<>()};
        // B[0] := even target, B[1] := odd target
        List<Integer>[] B = new List[] {new ArrayList<>(), new ArrayList<>()};

        for (final int num : nums)
            A[num % 2].add(num);

        for (final int num : target)
            B[num % 2].add(num);

        Collections.sort(A[0]);
        Collections.sort(A[1]);
        Collections.sort(B[0]);
        Collections.sort(B[1]);

        for (int i = 0; i < 2; ++i)
            for (int j = 0; j < A[i].size(); ++j)
                ans += Math.abs(A[i].get(j) - B[i].get(j)) / 2;

        return ans / 2;
    }
}
```

1500. 245.java

Path: LeetCode-main/solutions/245. Shortest Word Distance III/245.java

```
class Solution {
    public int shortestWordDistance(String[] words, String word1, String word2) {
        final boolean isSame = word1.equals(word2);
        int ans = Integer.MAX_VALUE;
        int index1 = words.length; // If word1 == word2, index1 is the newest index.
        int index2 = -words.length; // If word1 == word2, index2 is the previous index.

        for (int i = 0; i < words.length; ++i) {
            if (words[i].equals(word1)) {
                if (isSame)
                    index2 = index1;
                index1 = i;
            } else if (words[i].equals(word2)) {
                index2 = i;
            }
            ans = Math.min(ans, Math.abs(index1 - index2));
        }
        return ans;
    }
}
```

1501. 2450.java

Path: LeetCode-main/solutions/2450. Number of Distinct Binary Strings After Applying Operations/2450.java

```
class Solution {
    public int countDistinctStrings(String s, int k) {
        // Since the content of `s` doesn't matter, for each i in [0, n - k], we can
        // flip s[i..i + k] or don't flip it. Therefore, there's  $2^{(n - k + 1)}$  ways.
        return (int) modPow(2, s.length() - k + 1);
    }

    private static final int MOD = 1_000_000_007;

    private long modPow(long x, long n) {
        if (n == 0)
            return 1;
        if (n % 2 == 1)
            return x * modPow(x, n - 1) % MOD;
        return modPow(x * x % MOD, n / 2);
    }
}
```

1502. 2451.java

Path: LeetCode-main/solutions/2451. Odd String Difference/2451.java

```
class Solution {
    public String oddString(String[] words) {
        List<Pair<String, Integer>> wordAndHashCodes = getWordAndHashCodes(words);
        Map<Integer, Integer> hashCodeCount = new HashMap<>();

        for (Pair<String, Integer> wordAndHashCode : wordAndHashCodes)
            hashCodeCount.merge(wordAndHashCode.getValue(), 1, Integer::sum);

        for (Pair<String, Integer> wordAndHashCode : wordAndHashCodes)
            if (hashCodeCount.get(wordAndHashCode.getValue()) == 1)
                return wordAndHashCode.getKey();

        throw new IllegalArgumentException();
    }

    // Returns the pairs of the word and its corresponding difference string.
    // e.g. [("adc", "3#-1#"), ("wzy", "3#-1#"), ("abc", "1#1#")]
    private List<Pair<String, Integer>> getWordAndHashCodes(String[] words) {
        List<Pair<String, Integer>> wordAndHashCodes = new ArrayList<>();
        for (final String word : words)
            wordAndHashCodes.add(new Pair<>(word, getDiffStr(word).hashCode()));
        return wordAndHashCodes;
    }

    // Returns the difference string of `s`.
    // e.g. getDiffStr("adc") -> "3#-1#"
    private String getDiffStr(final String s) {
        StringBuilder sb = new StringBuilder();
        for (int i = 1; i < s.length(); ++i)
            sb.append(s.charAt(i) - s.charAt(i - 1)).append("#");
        return sb.toString();
    }
}
```

1503. 2452.java

Path: LeetCode-main/solutions/2452. Words Within Two Edits of Dictionary/2452.java

```
class Solution {
    public List<String> twoEditWords(String[] queries, String[] dictionary) {
        List<String> ans = new ArrayList<>();

        for (final String query : queries)
            for (final String word : dictionary)
                if (getDiff(query, word) < 3) {
                    ans.add(query);
                    break;
                }

        return ans;
    }

    private int getDiff(final String query, final String word) {
        int diff = 0;
        for (int i = 0; i < query.length(); ++i)
            if (query.charAt(i) != word.charAt(i))
                ++diff;
        return diff;
    }
}
```


1504. 2453.java

Path: LeetCode-main/solutions/2453. Destroy Sequential Targets/2453.java

```
class Solution {
    public int destroyTargets(int[] nums, int space) {
        int ans = Integer.MAX_VALUE;
        int maxCount = 0;
        Map<Integer, Integer> count = new HashMap<>();

        for (final int num : nums)
            maxCount = Math.max(maxCount, count.merge(num % space, 1, Integer::sum));

        for (final int num : nums)
            if (count.get(num % space) == maxCount)
                ans = Math.min(ans, num);

        return ans;
    }
}
```

1505. 2454.java

Path: LeetCode-main/solutions/2454. Next Greater Element IV/2454.java

```
class Solution {
    public int[] secondGreaterElement(int[] nums) {
        int[] ans = new int[nums.length];
        Arrays.fill(ans, -1);
        // a decreasing stack that stores indices that met the first greater number
        Deque<Integer> prevStack = new ArrayDeque<>();
        // a decreasing stack that stores indices
        Deque<Integer> currStack = new ArrayDeque<>();

        for (int i = 0; i < nums.length; ++i) {
            // Indices in prevStack meet the second greater num.
            while (!prevStack.isEmpty() && nums[prevStack.peek()] < nums[i])
                ans[prevStack.poll()] = nums[i];
            // Push indices that meet the first greater number from `currStack` to
            // `prevStack`. We need a temporary array to make the indices in the
            // `prevStack` increasing.
            Deque<Integer> decreasingIndices = new ArrayDeque<>();
            while (!currStack.isEmpty() && nums[currStack.peek()] < nums[i])
                decreasingIndices.push(currStack.poll());
            while (!decreasingIndices.isEmpty())
                prevStack.push(decreasingIndices.poll());
            currStack.push(i);
        }

        return ans;
    }
}
```

1506. 2455.java

Path: LeetCode-main/solutions/2455. Average Value of Even Numbers That Are Divisible by Three/2455.java

```
class Solution {
    public int averageValue(int[] nums) {
        int sum = 0;
        int count = 0;

        for (final int num : nums)
            if (num % 6 == 0) {
                sum += num;
                ++count;
            }

        return count == 0 ? 0 : sum / count;
    }
}
```

1507. 2456.java

Path: LeetCode-main/solutions/2456. Most Popular Video Creator/2456.java

```
class Creator {
    public long popularity; // the popularity sum
    public String videoId; // the video id that has the maximum view
    public int maxView; // the maximum view of the creator
    public Creator(long popularity, String videoId, int maxView) {
        this.popularity = popularity;
        this.videoId = videoId;
        this.maxView = maxView;
    }
}

class Solution {
    public List<List<String>> mostPopularCreator(String[] creators, String[] ids, int[] views) {
        List<List<String>> ans = new ArrayList<>();
        Map<String, Creator> nameToCreator = new HashMap<>();
        long maxPopularity = 0;

        for (int i = 0; i < creators.length; ++i) {
            if (!nameToCreator.containsKey(creators[i])) {
                nameToCreator.put(creators[i], new Creator(views[i], ids[i], views[i]));
                maxPopularity = Math.max(maxPopularity, (long) views[i]);
                continue;
            }
            Creator creator = nameToCreator.get(creators[i]);
            creator.popularity += views[i];
            maxPopularity = Math.max(maxPopularity, (long) creator.popularity);
            if (creator.maxView < views[i] ||
                creator.maxView == views[i] && creator.videoId.compareTo(ids[i]) > 0) {
                creator.videoId = ids[i];
                creator.maxView = views[i];
            }
        }

        for (Map.Entry<String, Creator> entry : nameToCreator.entrySet())
            if (entry.getValue().popularity == maxPopularity)
                ans.add(Arrays.asList(entry.getKey(), entry.getValue().videoId));

        return ans;
    }
}
```

1508. 2457.java

Path: LeetCode-main/solutions/2457. Minimum Addition to Make Integer Beautiful/2457.java

```
class Solution {
    public long makeIntegerBeautiful(long n, int target) {
        long ans = 0;
        long power = 1;

        // e.g. n = 123. After turning off the last bit by adding 7, n = 130.
        // Effectively, we can think n as 13. That's why we do n = (n / 10) + 1.
        while (sum(n) > target) {
            // the cost to turn off the last digit
            ans += power * (10 - n % 10);
            n = n / 10 + 1;
            power *= 10;
        }

        return ans;
    }

    private int sum(long n) {
        int res = 0;
        while (n > 0) {
            res += n % 10;
            n /= 10;
        }
        return res;
    }
}
```

1509. 2458.java

Path: LeetCode-main/solutions/2458. Height of Binary Tree After Subtree Removal Queries/2458.java

```
class Solution {
    public int[] treeQueries(TreeNode root, int[] queries) {
        int[] ans = new int[queries.length];

        dfs(root, 0, 0);

        for (int i = 0; i < queries.length; ++i)
            ans[i] = valToMaxHeight.get(queries[i]);

        return ans;
    }

    // valToMaxHeight[val] := the maximum height without the node with `val`
    private Map<Integer, Integer> valToMaxHeight = new HashMap<>();
    // valToHeight[val] := the height of the node with `val`
    private Map<Integer, Integer> valToHeight = new HashMap<>();

    private int height(TreeNode root) {
        if (root == null)
            return 0;
        if (valToHeight.containsKey(root.val))
            return valToHeight.get(root.val);
        final int h = 1 + Math.max(height(root.left), height(root.right));
        valToHeight.put(root.val, h);
        return h;
    }

    // maxHeight := the maximum height without the current node `root`
    private void dfs(TreeNode root, int depth, int maxHeight) {
        if (root == null)
            return;
        valToMaxHeight.put(root.val, maxHeight);
        dfs(root.left, depth + 1, Math.max(maxHeight, depth + height(root.right)));
        dfs(root.right, depth + 1, Math.max(maxHeight, depth + height(root.left)));
    }
}
```

1510. 246.java

Path: LeetCode-main/solutions/246. Strobogrammatic Number/246.java

```
class Solution {
    public boolean isStrobogrammatic(String num) {
        final char[] rotated = {'0', '1', 'x', 'x', 'x', 'x', '9', 'x', '8', '6'};
        int l = 0;
        int r = num.length() - 1;

        while (l <= r)
            if (num.charAt(l++) != rotated[num.charAt(r--) - '0'])
                return false;

        return true;
    }
}
```

1511. 2460-2.java

Path: LeetCode-main/solutions/2460. Apply Operations to an Array/2460-2.java

```
class Solution {
    public int[] applyOperations(int[] nums) {
        for (int i = 0, j = 0; i < nums.length; ++i) {
            if (i + 1 < nums.length && nums[i] == nums[i + 1]) {
                nums[i] *= 2;
                nums[i + 1] = 0;
            }
            if (nums[i] > 0) {
                swap(nums, i, j++);
            }
        }
        return nums;
    }

    private void swap(int[] nums, int i, int j) {
        final int temp = nums[i];
        nums[i] = nums[j];
        nums[j] = temp;
    }
}
```


1512. 2460.java

Path: LeetCode-main/solutions/2460. Apply Operations to an Array/2460.java

```
class Solution {
    public int[] applyOperations(int[] nums) {
        int[] ans = new int[nums.length];

        for (int i = 0; i + 1 < nums.length; ++i)
            if (nums[i] == nums[i + 1]) {
                nums[i] *= 2;
                nums[i + 1] = 0;
            }

        int i = 0;
        for (final int num : nums)
            if (num > 0)
                ans[i++] = num;

        return ans;
    }
}
```

1513. 2461.java

Path: LeetCode-main/solutions/2461. Maximum Sum of Distinct Subarrays With Length K/2461.java

```
class Solution {
    public long maximumSubarraySum(int[] nums, int k) {
        long ans = 0;
        long sum = 0;
        int distinct = 0;
        Map<Integer, Integer> count = new HashMap<>();

        for (int i = 0; i < nums.length; ++i) {
            sum += nums[i];
            if (count.merge(nums[i], 1, Integer::sum) == 1)
                ++distinct;
            if (i >= k) {
                if (count.merge(nums[i - k], -1, Integer::sum) == 0)
                    --distinct;
                sum -= nums[i - k];
            }
            if (i >= k - 1 && distinct == k)
                ans = Math.max(ans, sum);
        }

        return ans;
    }
}
```

1514. 2462.java

Path: LeetCode-main/solutions/2462. Total Cost to Hire K Workers/2462.java

```
class Solution {
    public long totalCost(int[] costs, int k, int candidates) {
        long ans = 0;
        int i = 0;
        int j = costs.length - 1;
        Queue<Integer> minHeapL = new PriorityQueue<>();
        Queue<Integer> minHeapR = new PriorityQueue<>();

        for (int hired = 0; hired < k; ++hired) {
            while (minHeapL.size() < candidates && i <= j)
                minHeapL.offer(costs[i++]);
            while (minHeapR.size() < candidates && i <= j)
                minHeapR.offer(costs[j--]);
            if (minHeapL.isEmpty())
                ans += minHeapR.poll();
            else if (minHeapR.isEmpty())
                ans += minHeapL.poll();
            // Both `minHeapL` and `minHeapR` are not empty.
            else if (minHeapL.peek() <= minHeapR.peek())
                ans += minHeapL.poll();
            else
                ans += minHeapR.poll();
        }

        return ans;
    }
}
```

1515. 2463.java

Path: LeetCode-main/solutions/2463. Minimum Total Distance Traveled/2463.java

```
class Solution {
    public long minimumTotalDistance(List<Integer> robot, int[][] factory) {
        Collections.sort(robot);
        Arrays.sort(factory, Comparator.comparingInt(a -> a[0]));
        long[][][] mem = new long[robot.size()][factory.length][robot.size()];
        return minimumTotalDistance(robot, factory, 0, 0, 0, mem);
    }

    private long minimumTotalDistance(List<Integer> robot, int[][] factory, int i, int j, int k,
                                       long[][][] mem) {
        if (i == robot.size())
            return 0;
        if (j == factory.length)
            return Long.MAX_VALUE;
        if (mem[i][j][k] > 0)
            return mem[i][j][k];
        final long skipFactory = minimumTotalDistance(robot, factory, i, j + 1, 0, mem);
        final int position = factory[j][0];
        final int limit = factory[j][1];
        final long useFactory = limit > k ? minimumTotalDistance(robot, factory, i + 1, j, k + 1, mem) +
                                           Math.abs(robot.get(i) - position)
                                           : Long.MAX_VALUE / 2;
        return mem[i][j][k] = Math.min(skipFactory, useFactory);
    }
}
```

1516. 2464.java

Path: LeetCode-main/solutions/2464. Minimum Subarrays in a Valid Split/2464.java

```
class Solution {
    public int validSubarraySplit(int[] nums) {
        final int MAX = Integer.MAX_VALUE / 2;
        final int n = nums.length;
        // dp[i] := the minimum number of subarrays to validly split nums[0..i]
        int[] dp = new int[n];
        Arrays.fill(dp, MAX);

        for (int i = 0; i < n; ++i)
            for (int j = 0; j <= i; ++j)
                if (gcd(nums[j], nums[i]) > 1)
                    dp[i] = Math.min(dp[i], j == 0 ? 1 : dp[j - 1] + 1);

        return dp[n - 1] == MAX ? -1 : dp[n - 1];
    }

    private int gcd(int a, int b) {
        return b == 0 ? a : gcd(b, a % b);
    }
}
```

1517. 2465.java

Path: LeetCode-main/solutions/2465. Number of Distinct Averages/2465.java

```
class Solution {
    public int distinctAverages(int[] nums) {
        final int n = nums.length;
        Set<Integer> sums = new HashSet<>();

        Arrays.sort(nums);

        for (int i = 0; i < n / 2; ++i)
            sums.add(nums[i] + nums[n - 1 - i]);

        return sums.size();
    }
}
```

1518. 2466.java

Path: LeetCode-main/solutions/2466. Count Ways To Build Good Strings/2466.java

```
class Solution {
    public int countGoodStrings(int low, int high, int zero, int one) {
        final int MOD = 1_000_000_007;
        int ans = 0;
        // dp[i] := the number of good strings with length i
        int[] dp = new int[high + 1];
        dp[0] = 1;

        for (int i = 1; i <= high; ++i) {
            if (i >= zero)
                dp[i] = (dp[i] + dp[i - zero]) % MOD;
            if (i >= one)
                dp[i] = (dp[i] + dp[i - one]) % MOD;
            if (i >= low)
                ans = (ans + dp[i]) % MOD;
        }

        return ans;
    }
}
```

1519. 2467.java

Path: LeetCode-main/solutions/2467. Most Profitable Path in a Tree/2467.java

```
class Solution {
    public int mostProfitablePath(int[][] edges, int bob, int[] amount) {
        final int n = amount.length;
        List<Integer>[] tree = new List[n];
        int[] parent = new int[n];
        int[] aliceDist = new int[n];
        Arrays.fill(aliceDist, -1);

        for (int i = 0; i < n; ++i)
            tree[i] = new ArrayList<>();

        for (int[] edge : edges) {
            final int u = edge[0];
            final int v = edge[1];
            tree[u].add(v);
            tree[v].add(u);
        }

        dfs(tree, 0, -1, 0, parent, aliceDist);

        // Modify the amount along the path from node Bob to node 0.
        // For each node,
        // 1. If Bob reaches earlier than Alice does, change the amount to 0.
        // 2. If Bob and Alice reach simultaneously, devide the amount by 2.
        for (int u = bob, bobDist = 0; u != 0; u = parent[u], ++bobDist)
            if (bobDist < aliceDist[u])
                amount[u] = 0;
            else if (bobDist == aliceDist[u])
                amount[u] /= 2;

        return getMoney(tree, 0, -1, amount);
    }

    // Fills `parent` and `dist`.
    private void dfs(List<Integer>[] tree, int u, int prev, int d, int[] parent, int[] dist) {
        parent[u] = prev;
        dist[u] = d;
        for (final int v : tree[u]) {
            if (dist[v] == -1)
                dfs(tree, v, u, d + 1, parent, dist);
        }
    }

    private int getMoney(List<Integer>[] tree, int u, int prev, int[] amount) {
        // a leaf node
        if (tree[u].size() == 1 && tree[u].get(0) == prev)
            return amount[u];

        int maxPath = Integer.MIN_VALUE;
        for (final int v : tree[u])
            if (v != prev)
                maxPath = Math.max(maxPath, getMoney(tree, v, u, amount));

        return amount[u] + maxPath;
    }
}
```


1520. 2468.java

Path: LeetCode-main/solutions/2468. Split Message Based on Limit/2468.java

```
class Solution {
    public String[] splitMessage(String message, int limit) {
        final int MESSAGE_LENGTH = message.length();
        int b = 1;
        // the total length of a: initialized with the length of "1"
        int aLength = sz(1);

        // the total length of b := b * sz(b)
        // The total length of "</>" := b * 3
        while (b * limit < b * (sz(b) + 3) + aLength + MESSAGE_LENGTH) {
            // If the length of the last suffix "<b/b>" := sz(b) * 2 + 3 >= limit,
            // then it's impossible that the length of "*<b/b>" <= limit.
            if (sz(b) * 2 + 3 >= limit)
                return new String[] {};
            aLength += sz(++b);
        }

        String[] ans = new String[b];

        for (int i = 0, a = 1; a <= b; ++a) {
            // the length of "<a/b>" := sz(a) + sz(b) + 3
            final int j = limit - (sz(a) + sz(b) + 3);
            ans[a - 1] = message.substring(i, Math.min(message.length(), i + j)) + "<" +
                String.valueOf(a) + "/" + String.valueOf(b) + ">";
            i += j;
        }

        return ans;
    }

    private int sz(int num) {
        return String.valueOf(num).length();
    }
}
```

1521. 2469.java

Path: LeetCode-main/solutions/2469. Convert the Temperature/2469.java

```
class Solution {  
    public double[] convertTemperature(double celsius) {  
        return new double[] {celsius + 273.15, celsius * 1.8 + 32};  
    }  
}
```

1522. 247.java

Path: LeetCode-main/solutions/247. Strobogrammatic Number II/247.java

```
class Solution {
    public List<String> findStrobogrammatic(int n) {
        return helper(n, n);
    }

    private List<String> helper(int n, int k) {
        if (n == 0)
            return List.of("");
        if (n == 1)
            return List.of("0", "1", "8");

        List<String> ans = new ArrayList<>();

        for (final String inner : helper(n - 2, k)) {
            if (n < k)
                ans.add("0" + inner + "0");
            ans.add("1" + inner + "1");
            ans.add("6" + inner + "9");
            ans.add("8" + inner + "8");
            ans.add("9" + inner + "6");
        }

        return ans;
    }
}
```

1523. 2470.java

Path: LeetCode-main/solutions/2470. Number of Subarrays With LCM Equal to K/2470.java

```
class Solution {
    public int subarrayLCM(int[] nums, int k) {
        int ans = 0;

        for (int i = 0; i < nums.length; ++i) {
            int runningLcm = nums[i];
            for (int j = i; j < nums.length; ++j) {
                runningLcm = lcm(runningLcm, nums[j]);
                if (runningLcm > k)
                    break;
                if (runningLcm == k)
                    ++ans;
            }
        }

        return ans;
    }

    private int gcd(int a, int b) {
        return b == 0 ? a : gcd(b, a % b);
    }

    private int lcm(int a, int b) {
        return a * (b / gcd(a, b));
    }
}
```

1524. 2471.java

Path: LeetCode-main/solutions/2471. Minimum Number of Operations to Sort a Binary Tree by Level/2471.java

```
class Solution {
    public int minimumOperations(TreeNode root) {
        int ans = 0;
        Queue<TreeNode> q = new LinkedList<>(Arrays.asList(root));

        // e.g. vals = [7, 6, 8, 5]
        // [2, 1, 3, 0]: Initialize the ids based on the order of vals.
        // [3, 1, 2, 0]: Swap 2 with 3, so 2 is in the right place (i == ids[i]).
        // [0, 1, 2, 3]: Swap 3 with 0, so 3 is in the right place.
        while (!q.isEmpty()) {
            List<Integer> vals = new ArrayList<>();
            List<Integer> ids = new ArrayList<>();
            for (int sz = q.size(); sz > 0; --sz) {
                TreeNode node = q.poll();
                vals.add(node.val);
                if (node.left != null)
                    q.offer(node.left);
                if (node.right != null)
                    q.offer(node.right);
            }
            for (int i = 0; i < vals.size(); ++i)
                ids.add(i);
            Collections.sort(ids, (i, j) -> vals.get(i) - vals.get(j));
            for (int i = 0; i < ids.size(); ++i)
                for (; ids.get(i) != i; ++ans)
                    swap(ids, i, ids.get(i));
        }

        return ans;
    }

    private void swap(List<Integer> ids, int i, int j) {
        final int temp = ids.get(i);
        ids.set(i, ids.get(j));
        ids.set(j, temp);
    }
}
```

1525. 2472.java

Path: LeetCode-main/solutions/2472. Maximum Number of Non-overlapping Palindrome Substrings/2472.java

```
class Solution {
    public int maxPalindromes(String s, int k) {
        final int n = s.length();
        // dp[i] := the maximum number of substrings in the first i chars of s
        int[] dp = new int[n + 1];

        // If a palindrome is a subString of another palindrome, then considering
        // the longer palindrome won't increase the number of non-overlapping
        // palindromes. So, we only need to consider the shorter one. Also,
        // considering palindromes with both k length and k + 1 length ensures that
        // we look for both even and odd length palindromes.
        for (int i = k; i <= n; ++i) {
            dp[i] = dp[i - 1];
            // Consider palindrome with length k.
            if (isPalindrome(s, i - k, i - 1))
                dp[i] = Math.max(dp[i], 1 + dp[i - k]);
            // Consider palindrome with length k + 1.
            if (isPalindrome(s, i - k - 1, i - 1))
                dp[i] = Math.max(dp[i], 1 + dp[i - k - 1]);
        }

        return dp[n];
    }

    // Returns true if s[i..j] is a palindrome.
    private boolean isPalindrome(String s, int l, int r) {
        if (l < 0)
            return false;
        while (l < r)
            if (s.charAt(l++) != s.charAt(r--))
                return false;
        return true;
    }
}
```

1526. 2473.java

Path: LeetCode-main/solutions/2473. Minimum Cost to Buy Apples/2473.java

```
class Solution {
    public long[] minCost(int n, int[][] roads, int[] appleCost, int k) {
        long[] ans = new long[n];
        List
```

1527. 2475.java

Path: LeetCode-main/solutions/2475. Number of Unequal Triplets in Array/2475.java

```
// Assume that we have 4 kinds of numbers a, b, c, and d in the count map.
//
// What we want is:
// cnt[a] * cnt[b] * cnt[c]
// cnt[a] * cnt[b] * cnt[d]
// cnt[a] * cnt[c] * cnt[d]
// cnt[b] * cnt[c] * cnt[d]
//
// The above combinations can be reduced as:
//
// prev | curr | next
// -----
// (0) * cnt[a] * (cnt[b] + cnt[c] + cnt[d])
// (cnt[a]) * cnt[b] * (cnt[c] + cnt[d])
// (cnt[a] + cnt[b]) * cnt[c] * (cnt[d])
// (cnt[a] + cnt[b] + cnt[c]) * cnt[d] * (0)

class Solution {
    public int unequalTriplets(int[] nums) {
        int ans = 0;
        int prev = 0;
        int next = nums.length;
        Map<Integer, Integer> count = new HashMap<>();

        for (final int num : nums)
            count.merge(num, 1, Integer::sum);

        for (final int freq : count.values()) {
            next -= freq;
            ans += prev * freq * next;
            prev += freq;
        }

        return ans;
    }
}
```


1528. 2476.java

Path: LeetCode-main/solutions/2476. Closest Nodes Queries in a Binary Search Tree/2476.java

```
class Solution {
    public List<List<Integer>> closestNodes(TreeNode root, List<Integer> queries) {
        List<List<Integer>> ans = new ArrayList<>();
        List<Integer> sortedVals = new ArrayList<>();

        inorder(root, sortedVals);

        for (final int query : queries) {
            final int i = firstGreaterEqual(sortedVals, query);
            // query is presented in the tree, so just use {query, query}.
            if (i != sortedVals.size() && sortedVals.get(i) == query)
                ans.add(Arrays.asList(query, query));
            // query isn't presented in the tree, so find the closest one if possible.
            else
                ans.add(Arrays.asList(i == 0 ? -1 : sortedVals.get(i - 1),
                                      i == sortedVals.size() ? -1 : sortedVals.get(i)));
        }

        return ans;
    }

    // Walks the BST to collect the sorted numbers.
    private void inorder(TreeNode root, List<Integer> sortedVals) {
        if (root == null)
            return;
        inorder(root.left, sortedVals);
        sortedVals.add(root.val);
        inorder(root.right, sortedVals);
    }

    private int firstGreaterEqual(List<Integer> A, int target) {
        final int i = Collections.binarySearch(A, target);
        return i < 0 ? -i - 1 : i;
    }
}
```

1529. 2477.java

Path: LeetCode-main/solutions/2477. Minimum Fuel Cost to Report to the Capital/2477.java

```
class Solution {
    public long minimumFuelCost(int[][] roads, int seats) {
        List<Integer>[] tree = new List[roads.length + 1];

        for (int i = 0; i < tree.length; ++i)
            tree[i] = new ArrayList<>();

        for (int[] road : roads) {
            final int u = road[0];
            final int v = road[1];
            tree[u].add(v);
            tree[v].add(u);
        }

        dfs(tree, 0, -1, seats);
        return ans;
    }

    private long ans = 0;

    private int dfs(List<Integer>[] tree, int u, int prev, int seats) {
        int people = 1;
        for (final int v : tree[u])
            if (v != prev)
                people += dfs(tree, v, u, seats);
        if (u > 0)
            // the number of cars needed = ceil(people / seats)
            ans += (people + seats - 1) / seats;
        return people;
    }
}
```

1530. 2478.java

Path: LeetCode-main/solutions/2478. Number of Beautiful Partitions/2478.java

```
class Solution {
    public int beautifulPartitions(String s, int k, int minLength) {
        if (!isPrime(s.charAt(0)) || isPrime(s.charAt(s.length() - 1)))
            return 0;
        Integer[][] mem = new Integer[s.length()][k];
        return beautifulPartitions(s, minLength, k - 1, minLength, mem);
    }

    private static final int MOD = 1_000_000_007;

    // Returns the number of beautiful partitions of s[i..n) with k bars (|) left.
    private int beautifulPartitions(final String s, int i, int k, int minLength, Integer[][] mem) {
        if (i <= s.length() && k == 0)
            return 1;
        if (i >= s.length())
            return 0;
        if (mem[i][k] != null)
            return mem[i][k];

        // Don't split between s[i - 1] and s[i].
        int ans = beautifulPartitions(s, i + 1, k, minLength, mem) % MOD;

        // Split between s[i - 1] and s[i].
        if (isPrime(s.charAt(i)) && !isPrime(s.charAt(i - 1)))
            ans += beautifulPartitions(s, i + minLength, k - 1, minLength, mem);

        return mem[i][k] = ans % MOD;
    }

    private boolean isPrime(char c) {
        return c == '2' || c == '3' || c == '5' || c == '7';
    }
}
```

1531. 2479.java

Path: LeetCode-main/solutions/2479. Maximum XOR of Two Non-Overlapping Subtrees/2479.java

```
class TrieNode {
    public TrieNode[] children = new TrieNode[2];
}

class BitTrie {
    public BitTrie(int maxBit) {
        this.maxBit = maxBit;
    }

    public void insert(long num) {
        TrieNode node = root;
        for (int i = maxBit; i >= 0; --i) {
            final int bit = (int) (num >> i & 1);
            if (node.children[bit] == null)
                node.children[bit] = new TrieNode();
            node = node.children[bit];
        }
    }

    public long getMaxXor(long num) {
        long maxXor = 0;
        TrieNode node = root;
        for (int i = maxBit; i >= 0; --i) {
            final int bit = (int) (num >> i & 1);
            final int toggleBit = bit ^ 1;
            if (node.children[toggleBit] != null) {
                maxXor = maxXor | 1L << i;
                node = node.children[toggleBit];
            } else if (node.children[bit] != null) {
                node = node.children[bit];
            } else { // There's nothing in the Bit Trie.
                return 0;
            }
        }
        return maxXor;
    }

    private int maxBit;
    private TrieNode root = new TrieNode();
}

class Solution {
    public long maxXor(int n, int[][] edges, int[] values) {
        List<Integer>[] tree = new List[n];
        long[] treeSums = new long[n];

        for (int i = 0; i < n; ++i)
            tree[i] = new ArrayList<>();

        for (int[] edge : edges) {
            final int u = edge[0];
            final int v = edge[1];
            tree[u].add(v);
            tree[v].add(u);
        }

        getTreeSum(tree, 0, -1, treeSums, values);
        final long maxSubTreeSum = getMaxSubTreeSum(treeSums);
        final int maxBit = (int) (Math.log(maxSubTreeSum) / Math.log(2));
        // Similar to 421. Maximum XOR of Two Numbers in an Array
        dfs(tree, 0, -1, treeSums, new BitTrie(maxBit));
        return ans;
    }

    private long ans = 0;

    // Gets the tree sum rooted at node u.
    private long getTreeSum(List<Integer>[] tree, int u, int prev, long[] treeSums, int[] values) {
        long treeSum = values[u];
        for (final int v : tree[u])
            if (v != prev)
                treeSum += getTreeSum(tree, v, u, treeSums, values);
        treeSums[u] = treeSum;
        return treeSum;
    }

    private long getMaxSubTreeSum(long[] treeSums) {
        long maxSubTreeSum = 0;
        for (int i = 1; i < treeSums.length; ++i)
            maxSubTreeSum = Math.max(maxSubTreeSum, treeSums[i]);
    }
}
```

```

        return maxSubTreeSum;
    }

    private void dfs(List<Integer>[] tree, int u, int prev, long[] treeSums, BitTrie bitTrie) {
        for (final int v : tree[u]) {
            if (v == prev)
                continue;
            // Preorder to get the ans.
            ans = Math.max(ans, bitTrie.getMaxXor(treeSums[v]));
            // Recursively call on the subtree rooted at node v.
            dfs(tree, v, u, treeSums, bitTrie);
            // Postorder to insert the tree sum rooted at node v.
            bitTrie.insert(treeSums[v]);
        }
    }
}

```

1532. 248.java

Path: LeetCode-main/solutions/248. Strobogrammatic Number III/248.java

```
class Solution {
    public int strobogrammaticInRange(String low, String high) {
        for (int n = low.length(); n <= high.length(); ++n) {
            char[] c = new char[n];
            dfs(low, high, c, 0, n - 1);
        }

        return ans;
    }

    private int ans = 0;

    private static final char[][] pairs = {
        {'0', '0'}, {'1', '1'}, {'6', '9'}, {'8', '8'}, {'9', '6'}};

    private void dfs(final String low, final String high, char[] c, int l, int r) {
        if (l > r) {
            final String s = new String(c);
            if (s.length() == low.length() && s.compareTo(low) < 0)
                return;
            if (s.length() == high.length() && s.compareTo(high) > 0)
                return;
            ++ans;
            return;
        }

        for (char[] pair : pairs) {
            if (l == r && pair[0] != pair[1])
                continue;
            c[l] = pair[0];
            c[r] = pair[1];
            if (c.length > 1 && c[0] == '0')
                continue;
            dfs(low, high, c, l + 1, r - 1);
        }
    }
}
```

1533. 2481.java

Path: LeetCode-main/solutions/2481. Minimum Cuts to Divide a Circle/2481.java

```
class Solution {  
    public int numberOfCuts(int n) {  
        if (n == 1)  
            return 0;  
        return n % 2 == 0 ? n / 2 : n;  
    }  
}
```

1534. 2482.java

Path: LeetCode-main/solutions/2482. Difference Between Ones and Zeros in Row and Column/2482.java

```
class Solution {
    public int[][] onesMinusZeros(int[][] grid) {
        final int m = grid.length;
        final int n = grid[0].length;
        int[][] ans = new int[m][n];
        int[] onesRow = new int[m];
        int[] onesCol = new int[n];

        for (int i = 0; i < m; ++i)
            onesRow[i] = (int) Arrays.stream(grid[i]).filter(a -> a == 1).count();

        for (int j = 0; j < n; ++j) {
            int ones = 0;
            for (int i = 0; i < m; ++i)
                ones += grid[i][j];
            onesCol[j] = ones;
        }

        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j)
                ans[i][j] = onesRow[i] + onesCol[j] - (n - onesRow[i]) - (m - onesCol[j]);

        return ans;
    }
}
```


1535. 2483.java

Path: LeetCode-main/solutions/2483. Minimum Penalty for a Shop/2483.java

```
class Solution {
    public int bestClosingTime(String customers) {
        // Instead of computing the minimum penalty, we can compute the maximum profit.
        int ans = 0;
        int profit = 0;
        int maxProfit = 0;

        for (int i = 0; i < customers.length(); ++i) {
            profit += customers.charAt(i) == 'Y' ? 1 : -1;
            if (profit > maxProfit) {
                maxProfit = profit;
                ans = i + 1;
            }
        }

        return ans;
    }
}
```

1536. 2484.java

Path: LeetCode-main/solutions/2484. Count Palindromic Subsequences/2484.java

```
class Solution {
    public int countPalindromes(String s) {
        final int MOD = 1_000_000_007;
        final int PATTERN_SIZE = 5;
        long ans = 0;

        for (char a = '0'; a <= '9'; ++a)
            for (char b = '0'; b <= '9'; ++b) {
                char[] pattern = {a, b, '.', b, a};
                // dp[i] := the number of subsequences of pattern[i..n) in s, where
                // pattern[2] can be any character
                long[] dp = new long[PATTERN_SIZE + 1];
                dp[PATTERN_SIZE] = 1;
                for (final char c : s.toCharArray())
                    for (int i = 0; i < PATTERN_SIZE; ++i)
                        if (pattern[i] == '.' || pattern[i] == c)
                            dp[i] += dp[i + 1];
                ans += dp[0];
                ans %= MOD;
            }
        return (int) ans;
    }
}
```

1537. 2485.java

Path: LeetCode-main/solutions/2485. Find the Pivot Integer/2485.java

```
class Solution {
    public int pivotInteger(int n) {
        // 1 + 2 + ... + x = x + ... + n
        // (1 + x) * x / 2 = (x + n) * (n - x + 1) / 2
        // x + x^2 = nx - x^2 + x + n^2 - nx + n
        // 2 * x^2 = n^2 + n
        // x = sqrt((n^2 + n) / 2)
        final int y = (n * n + n) / 2;
        final int x = (int) Math.sqrt(y);
        return x * x == y ? x : -1;
    }
}
```

1538. 2486.java

Path: LeetCode-main/solutions/2486. Append Characters to String to Make Subsequence/2486.java

```
class Solution {
    public int appendCharacters(String s, String t) {
        int i = 0; // t's index

        for (final char c : s.toCharArray())
            if (c == t.charAt(i))
                if (++i == t.length())
                    return 0;

        return t.length() - i;
    }
}
```

1539. 2487.java

Path: LeetCode-main/solutions/2487. Remove Nodes From Linked List/2487.java

```
class Solution {  
    public ListNode removeNodes(ListNode head) {  
        if (head == null)  
            return null;  
        head.next = removeNodes(head.next);  
        return head.next != null && head.val < head.next.val ? head.next : head;  
    }  
}
```

1540. 2488.java

Path: LeetCode-main/solutions/2488. Count Subarrays With Median K/2488.java

```
class Solution {
    public int countSubarrays(int[] nums, int k) {
        final int INDEX = find(nums, k);
        int ans = 0;
        Map<Integer, Integer> count = new HashMap<>();

        for (int i = INDEX, balance = 0; i >= 0; --i) {
            if (nums[i] < k)
                --balance;
            else if (nums[i] > k)
                ++balance;
            count.merge(balance, 1, Integer::sum);
        }

        for (int i = INDEX, balance = 0; i < nums.length; ++i) {
            if (nums[i] < k)
                --balance;
            else if (nums[i] > k)
                ++balance;
            // The subarray that has balance == 0 or 1 having median equal to k.
            // So, add count[0 - balance] and count[1 - balance] to `ans`.
            ans += count.getDefault(-balance, 0) + count.getDefault(1 - balance, 0);
        }

        return ans;
    }

    private int find(int[] nums, int k) {
        for (int i = 0; i < nums.length; ++i)
            if (nums[i] == k)
                return i;
        throw new IllegalArgumentException();
    }
}
```

1541. 2489.java

Path: LeetCode-main/solutions/2489. Number of Substrings With Fixed Ratio/2489.java

```
class Solution {
    public long fixedRatio(String s, int num1, int num2) {
        // Let x := the number of 0s and y := the number of 1s in the subarray.
        // We want x : y = num1 : num2, so our goal is to find number of subarrays
        // with x * num2 - y * num1 = 0. To achieve this, we can use a prefix count
        // map to record the count of the running x * num2 - y * num1. If the
        // running x * num2 - y * num1 = prefix, then add count[prefix] to the
        // `ans`.
        long ans = 0;
        long prefix = 0;
        Map<Long, Integer> prefixCount = new HashMap<>();
        prefixCount.put(0L, 1);

        for (final char c : s.toCharArray()) {
            if (c == '0')
                prefix += num2;
            else // c == '1'
                prefix -= num1;
            ans += prefixCount.getOrDefault(prefix, 0);
            prefixCount.merge(prefix, 1, Integer::sum);
        }

        return ans;
    }
}
```

1542. 249.java

Path: LeetCode-main/solutions/249. Group Shifted Strings/249.java

```
class Solution {
    public List<List<String>> groupStrings(String[] strings) {
        Map<String, List<String>> keyToStrings = new HashMap<>();

        for (final String s : strings)
            keyToStrings.computeIfAbsent(getKey(s), k -> new ArrayList<>()).add(s);

        return new ArrayList<>(keyToStrings.values());
    }

    // Returns the key of 's' by pairwise calculation of differences.
    // e.g. getKey("abc") -> "1,1" because diff(a, b) = 1 and diff(b, c) = 1.
    private String getKey(final String s) {
        StringBuilder sb = new StringBuilder();

        for (int i = 1; i < s.length(); ++i) {
            final int diff = (s.charAt(i) - s.charAt(i - 1) + 26) % 26;
            sb.append(diff).append(",");
        }

        return sb.toString();
    }
}
```


1543. 2490.java

Path: LeetCode-main/solutions/2490. Circular Sentence/2490.java

```
class Solution {  
    public boolean isCircularSentence(String sentence) {  
        for (int i = 0; i < sentence.length(); ++i)  
            if (sentence.charAt(i) == ' ' && sentence.charAt(i - 1) != sentence.charAt(i + 1))  
                return false;  
        return sentence.charAt(0) == sentence.charAt(sentence.length() - 1);  
    }  
}
```

1544. 2491.java

Path: LeetCode-main/solutions/2491. Divide Players Into Teams of Equal Skill/2491.java

```
class Solution {
    public long dividePlayers(int[] skill) {
        final int n = skill.length;
        final int teamSkill = Arrays.stream(skill).sum() / (n / 2);
        long ans = 0;
        Map<Integer, Integer> count = new HashMap<>();

        for (final int s : skill)
            count.merge(s, 1, Integer::sum);

        for (Map.Entry<Integer, Integer> entry : count.entrySet()) {
            final int s = entry.getKey();
            final int freq = entry.getValue();
            final int requiredSkill = teamSkill - s;
            if (count.getOrDefault(requiredSkill, 0) != freq)
                return -1;
            ans += (long) s * requiredSkill * freq;
        }

        return ans / 2;
    }
}
```

1545. 2492.java

Path: LeetCode-main/solutions/2492. Minimum Score of a Path Between Two Cities/2492.java

```
class Solution {
    public int minScore(int n, int[][] roads) {
        int ans = Integer.MAX_VALUE;
        List<Pair<Integer, Integer>>[] graph = new List[n]; // graph[u] := [(v, distance)]
        Queue<Integer> q = new ArrayDeque<>(List.of(0));
        boolean[] seen = new boolean[n];
        seen[0] = true;
        Arrays.setAll(graph, i -> new ArrayList<>());

        for (final int[] r : roads) {
            final int u = r[0] - 1;
            final int v = r[1] - 1;
            final int distance = r[2];
            graph[u].add(new Pair<>(v, distance));
            graph[v].add(new Pair<>(u, distance));
        }

        while (!q.isEmpty()) {
            final int u = q.poll();
            for (Pair<Integer, Integer> pair : graph[u]) {
                final int v = pair.getKey();
                final int d = pair.getValue();
                ans = Math.min(ans, d);
                if (seen[v])
                    continue;
                q.offer(v);
                seen[v] = true;
            }
        }

        return ans;
    }
}
```

1546. 2495.java

Path: LeetCode-main/solutions/2495. Number of Subarrays Having Even Product/2495.java

```
class Solution {
    public long evenProduct(int[] nums) {
        long ans = 0;
        int numsBeforeEven = 0; // inclusively

        // e.g. nums = [1, 0, 1, 1, 0].
        // After meeting the first 0, set `numsBeforeEven` to 2. So, the number
        // between index 1 to index 3 (the one before next 0) will contribute 2 to
        // `ans`.
        for (int i = 0; i < nums.length; ++i) {
            if (nums[i] % 2 == 0)
                numsBeforeEven = i + 1;
            ans += numsBeforeEven;
        }

        return ans;
    }
}
```

1547. 2496.java

Path: LeetCode-main/solutions/2496. Maximum Value of a String in an Array/2496.java

```
class Solution {  
    public int maximumValue(String[] strs) {  
        int ans = 0;  
        for (final String s : strs)  
            ans = Math.max(ans, s.chars().anyMatch(c -> Character.isAlphabetic(c)) ? s.length()  
                           : Integer.valueOf(s));  
        return ans;  
    }  
}
```

1548. 2497.java

Path: LeetCode-main/solutions/2497. Maximum Star Sum of a Graph/2497.java

```
class Solution {
    public int maxStarSum(int[] vals, int[][] edges, int k) {
        final int n = vals.length;
        int ans = Integer.MIN_VALUE;
        List
```

1549. 2498.java

Path: LeetCode-main/solutions/2498. Frog Jump II/2498.java

```
class Solution {
    public int maxJump(int[] stones) {
        // Let's denote the forwarding path as F and the backwarding path as B.
        // "F1 B2 B1 F2" is no better than "F1 B2 F2 B1" since the distance between
        // F1 and F2 increase, resulting a larger `ans`.
        int ans = stones[1] - stones[0]; // If there're only two stones.
        for (int i = 2; i < stones.length; ++i)
            ans = Math.max(ans, stones[i] - stones[i - 2]);
        return ans;
    }
}
```

1550. 2499.java

Path: LeetCode-main/solutions/2499. Minimum Total Cost to Make Arrays Unequal/2499.java

```
class Solution {
    public long minimumTotalCost(int[] nums1, int[] nums2) {
        final int n = nums1.length;
        long ans = 0;
        int maxFreq = 0;
        int maxFreqNum = 0;
        int shouldBeSwapped = 0;
        int[] conflictedNumCount = new int[n + 1];

        // Collect the indices i s.t. nums1[i] == nums2[i] and record their `maxFreq`
        // and `maxFreqNum`.
        for (int i = 0; i < n; ++i)
            if (nums1[i] == nums2[i]) {
                final int conflictedNum = nums1[i];
                if (++conflictedNumCount[conflictedNum] > maxFreq) {
                    maxFreq = conflictedNumCount[conflictedNum];
                    maxFreqNum = conflictedNum;
                }
                ++shouldBeSwapped;
                ans += i;
            }

        // Collect the indices with nums1[i] != nums2[i] that contribute less cost.
        // This can be greedily achieved by iterating from 0 to n - 1.
        for (int i = 0; i < n; ++i) {
            // Since we have over `maxFreq * 2` spaces, `maxFreqNum` can be
            // successfully distributed, so no need to collect extra spaces.
            if (maxFreq * 2 <= shouldBeSwapped)
                break;
            if (nums1[i] == nums2[i])
                continue;
            // The numbers == `maxFreqNum` worsen the result since they increase the
            // `maxFreq`.
            if (nums1[i] == maxFreqNum || nums2[i] == maxFreqNum)
                continue;
            ++shouldBeSwapped;
            ans += i;
        }

        return maxFreq * 2 > shouldBeSwapped ? -1 : ans;
    }
}
```


1551. 25-2.java

Path: LeetCode-main/solutions/25. Reverse Nodes in k-Group/25-2.java

```
class Solution {
    public ListNode reverseKGroup(ListNode head, int k) {
        if (head == null || k == 1)
            return head;

        final int length = getLength(head);
        ListNode dummy = new ListNode(0, head);
        ListNode prev = dummy;
        ListNode curr = head;

        for (int i = 0; i < length / k; ++i) {
            for (int j = 0; j < k - 1; ++j) {
                ListNode next = curr.next;
                curr.next = next.next;
                next.next = prev.next;
                prev.next = next;
            }
            prev = curr;
            curr = curr.next;
        }

        return dummy.next;
    }

    private int getLength(ListNode head) {
        int length = 0;
        for (ListNode curr = head; curr != null; curr = curr.next)
            ++length;
        return length;
    }
}
```

1552. 25.java

Path: LeetCode-main/solutions/25. Reverse Nodes in k-Group/25.java

```
class Solution {
    public ListNode reverseKGroup(ListNode head, int k) {
        if (head == null)
            return null;

        ListNode tail = head;

        for (int i = 0; i < k; ++i) {
            // There are less than k nodes in the list, do nothing.
            if (tail == null)
                return head;
            tail = tail.next;
        }

        ListNode newHead = reverse(head, tail);
        head.next = reverseKGroup(tail, k);
        return newHead;
    }

    // Reverses [head, tail).
    private ListNode reverse(ListNode head, ListNode tail) {
        ListNode prev = null;
        ListNode curr = head;
        while (curr != tail) {
            ListNode next = curr.next;
            curr.next = prev;
            prev = curr;
            curr = next;
        }
        return prev;
    }
}
```

1553. 250.java

Path: LeetCode-main/solutions/250. Count Unival Subtrees/250.java

```
class Solution {
    public int countUnivalSubtrees(TreeNode root) {
        isUnival(root, Integer.MAX_VALUE);
        return ans;
    }

    private int ans = 0;

    private boolean isUnival(TreeNode root, int val) {
        if (root == null)
            return true;

        if (isUnival(root.left, root.val) & isUnival(root.right, root.val)) {
            ++ans;
            return root.val == val;
        }

        return false;
    }
}
```

1554. 2500.java

Path: LeetCode-main/solutions/2500. Delete Greatest Value in Each Row/2500.java

```
class Solution {
    public int deleteGreatestValue(int[][] grid) {
        int ans = 0;

        for (int[] row : grid)
            Arrays.sort(row);

        for (int j = 0; j < grid[0].length; ++j) {
            int maxOfColumn = 0;
            for (int i = 0; i < grid.length; ++i)
                maxOfColumn = Math.max(maxOfColumn, grid[i][j]);
            ans += maxOfColumn;
        }

        return ans;
    }
}
```

1555. 2502.java

Path: LeetCode-main/solutions/2502. Design Memory Allocator/2502.java

```
class Allocator {
    public Allocator(int n) {
        memory = new int[n];
        mIDToIndices = new List[1001];
        for (int i = 1; i <= 1000; ++i)
            mIDToIndices[i] = new ArrayList<>();
    }

    public int allocate(int size, int mID) {
        int consecutiveFree = 0;
        for (int i = 0; i < memory.length; ++i) {
            consecutiveFree = memory[i] == 0 ? consecutiveFree + 1 : 0;
            if (consecutiveFree == size) {
                for (int j = i - consecutiveFree + 1; j <= i; ++j) {
                    memory[j] = mID;
                    mIDToIndices[mID].add(j);
                }
                return i - consecutiveFree + 1;
            }
        }
        return -1;
    }

    public int free(int mID) {
        List<Integer> indices = mIDToIndices[mID];
        final int freedUnits = indices.size();
        for (final int index : indices)
            memory[index] = 0;
        indices.clear();
        return freedUnits;
    }

    private int[] memory;
    private List<Integer>[] mIDToIndices;
}
```

1556. 2503.java

Path: LeetCode-main/solutions/2503. Maximum Number of Points From Grid Queries/2503.java

```
class Solution {
    public int[] maxPoints(int[][] grid, int[] queries) {
        record T(int i, int j, int val) {}
        final int[][] DIRS = {{0, 1}, {1, 0}, {0, -1}, {-1, 0}};
        final int m = grid.length;
        final int n = grid[0].length;
        int[] ans = new int[queries.length];
        Queue<T> minHeap = new PriorityQueue<>(Comparator.comparingInt(T::val));
        boolean[][] seen = new boolean[m][n];

        minHeap.offer(new T(0, 0, grid[0][0]));
        seen[0][0] = true;
        int accumulate = 0;

        for (IndexedQuery indexedQuery : getIndexedQueries(queries)) {
            final int queryIndex = indexedQuery.queryIndex;
            final int query = indexedQuery.query;
            while (!minHeap.isEmpty()) {
                final int i = minHeap.peek().i;
                final int j = minHeap.peek().j;
                final int val = minHeap.poll().val;
                if (val >= query) {
                    // The smallest neighbor is still larger than `query`, so no need to
                    // keep exploring. Re-push (i, j, grid[i][j]) back to the `minHeap`.
                    minHeap.offer(new T(i, j, val));
                    break;
                }
                ++accumulate;
                for (int[] dir : DIRS) {
                    final int x = i + dir[0];
                    final int y = j + dir[1];
                    if (x < 0 || x == m || y < 0 || y == n)
                        continue;
                    if (seen[x][y])
                        continue;
                    minHeap.offer(new T(x, y, grid[x][y]));
                    seen[x][y] = true;
                }
            }
            ans[queryIndex] = accumulate;
        }

        return ans;
    }

    private record IndexedQuery(int queryIndex, int query) {}

    private IndexedQuery[] getIndexedQueries(int[] queries) {
        IndexedQuery[] indexedQueries = new IndexedQuery[queries.length];
        for (int i = 0; i < queries.length; ++i)
            indexedQueries[i] = new IndexedQuery(i, queries[i]);
        Arrays.sort(indexedQueries, Comparator.comparingInt(IndexedQuery::query));
        return indexedQueries;
    }
}
```

1557. 2505.java

Path: LeetCode-main/solutions/2505. Bitwise OR of All Subsequence Sums/2505.java

```
class Solution {
    public long subsequenceSumOr(int[] nums) {
        long ans = 0;
        long prefix = 0;

        for (final int num : nums) {
            prefix += num;
            ans |= num | prefix;
        }

        return ans;
    }
}
```

1558. 2506.java

Path: LeetCode-main/solutions/2506. Count Pairs Of Similar Strings/2506.java

```
class Solution {
    public int similarPairs(String[] words) {
        int ans = 0;
        int[] masks = new int[words.length];

        for (int i = 0; i < words.length; ++i)
            masks[i] = getMask(words[i]);

        for (int i = 0; i < masks.length; ++i)
            for (int j = i + 1; j < masks.length; ++j)
                if (masks[i] == masks[j])
                    ++ans;

        return ans;
    }

    private int getMask(final String word) {
        int mask = 0;
        for (final char c : word.toCharArray())
            mask |= 1 << c - 'a';
        return mask;
    }
}
```


1559. 2507.java

Path: LeetCode-main/solutions/2507. Smallest Value After Replacing With Sum of Prime Factors/2507.java

```
class Solution {
    public int smallestValue(int n) {
        int primeSum = getPrimeSum(n);
        while (n != primeSum) {
            n = primeSum;
            primeSum = getPrimeSum(n);
        }
        return n;
    }

    private int getPrimeSum(int n) {
        int primeSum = 0;
        for (int i = 2; i <= n; ++i)
            while (n % i == 0) {
                n /= i;
                primeSum += i;
            }
        return primeSum;
    }
}
```

1560. 2508.java

Path: LeetCode-main/solutions/2508. Add Edges to Make Degrees of All Nodes Even/2508.java

```
class Solution {
    public boolean isPossible(int n, List<List<Integer>> edges) {
        Set<Integer>[] graph = new Set[n];

        for (int i = 0; i < n; ++i)
            graph[i] = new HashSet<>();

        for (List<Integer> edge : edges) {
            final int u = edge.get(0) - 1;
            final int v = edge.get(1) - 1;
            graph[u].add(v);
            graph[v].add(u);
        }

        List<Integer> oddNodes = getOddNodes(graph);
        if (oddNodes.isEmpty())
            return true;
        if (oddNodes.size() == 2) {
            final int a = oddNodes.get(0);
            final int b = oddNodes.get(1);
            for (int i = 0; i < n; ++i)
                // Can connect i with a and i with b.
                if (!graph[i].contains(a) && !graph[i].contains(b))
                    return true;
        }
        if (oddNodes.size() == 4) {
            final int a = oddNodes.get(0);
            final int b = oddNodes.get(1);
            final int c = oddNodes.get(2);
            final int d = oddNodes.get(3);
            return (!graph[a].contains(b) && !graph[c].contains(d)) ||
                (!graph[a].contains(c) && !graph[b].contains(d)) ||
                (!graph[a].contains(d) && !graph[b].contains(c));
        }
        return false;
    }

    private List<Integer> getOddNodes(Set<Integer>[] graph) {
        List<Integer> oddNodes = new ArrayList<>();
        for (int i = 0; i < graph.length; ++i)
            if (graph[i].size() % 2 == 1)
                oddNodes.add(i);
        return oddNodes;
    }
}
```

1561. 2509.java

Path: LeetCode-main/solutions/2509. Cycle Length Queries in a Tree/2509.java

```
class Solution {
    public int[] cycleLengthQueries(int n, int[][] queries) {
        int[] ans = new int[queries.length];

        for (int i = 0; i < queries.length; ++i) {
            ++ans[i];
            int a = queries[i][0];
            int b = queries[i][1];
            while (a != b) {
                if (a > b)
                    a /= 2;
                else
                    b /= 2;
                ++ans[i];
            }
        }

        return ans;
    }
}
```

1562. 251.java

Path: LeetCode-main/solutions/251. Flatten 2D Vector/251.java

```
class Vector2D {
    public Vector2D(int[][] vec) {
        for (int[] arr : vec)
            for (final int a : arr)
                this.vec.add(a);
    }

    public int next() {
        return vec.get(i++);
    }

    public boolean hasNext() {
        return i < vec.size();
    }

    private List<Integer> vec = new ArrayList<>();
    private int i = 0;
}
```

1563. 2510.java

Path: LeetCode-main/solutions/2510. Check if There is a Path With Equal Number of 0's And 1's/2510.java

```
class Solution {
    public boolean isThereAPath(int[][] grid) {
        final int m = grid.length;
        final int n = grid[0].length;
        // Map negative (the number of 0s - the number of 1s) to non-negative one.
        final int cells = m + n - 1;
        if (cells % 2 == 1)
            return false;
        Boolean[][][] mem = new Boolean[m][n][cells * 2 + 1];
        return isThereAPath(grid, 0, 0, 0, cells, mem);
    }

    // Returns 1 if there's a path to grid[i][j]
    // s.t. `sum` = (the number of 0s - the number of 1s).
    private boolean isThereAPath(int[][] grid, int i, int j, int sum, final int cells,
        Boolean[][][] mem) {
        if (i == grid.length || j == grid[0].length)
            return false;
        sum += grid[i][j] == 0 ? 1 : -1;
        if (i == grid.length - 1 && j == grid[0].length - 1)
            return sum == 0;
        final int k = cells + sum;
        if (mem[i][j][k] != null)
            return mem[i][j][k];
        return mem[i][j][k] = isThereAPath(grid, i + 1, j, sum, cells, mem) ||
            isThereAPath(grid, i, j + 1, sum, cells, mem);
    }
}
```

1564. 2511.java

Path: LeetCode-main/solutions/2511. Maximum Enemy Forts That Can Be Captured/2511.java

```
class Solution {
    public int captureForts(int[] forts) {
        int ans = 0;

        for (int i = 0, j = 0; i < forts.length; ++i)
            if (forts[i] != 0) { // -1 or 1
                if (forts[i] == -forts[j])
                    ans = Math.max(ans, i - j - 1);
                j = i;
            }
        return ans;
    }
}
```

1565. 2512.java

Path: LeetCode-main/solutions/2512. Reward Top K Students/2512.java

```
class Solution {
    public List<Integer> topStudents(String[] positive_feedback, String[] negative_feedback,
                                     String[] report, int[] student_id, int k) {
        List<Integer> ans = new ArrayList<>();
        Pair<Integer, Integer>[] scoreAndIds = new Pair[report.length];
        Set<String> pos = Arrays.stream(positive_feedback).collect(Collectors.toSet());
        Set<String> neg = Arrays.stream(negative_feedback).collect(Collectors.toSet());

        for (int i = 0; i < report.length; ++i) {
            int score = 0;
            for (final String word : report[i].split(" ")) {
                if (pos.contains(word))
                    score += 3;
                if (neg.contains(word))
                    score -= 1;
            }
            scoreAndIds[i] = new Pair<>(score, student_id[i]);
        }

        Arrays.sort(scoreAndIds,
                    Comparator.comparing(Pair<Integer, Integer>::getKey, Comparator.reverseOrder())
                               .thenComparing(Pair<Integer, Integer>::getValue));

        for (int i = 0; i < k; ++i)
            ans.add(scoreAndIds[i].getValue());
        return ans;
    }
}
```

1566. 2513.java

Path: LeetCode-main/solutions/2513. Minimize the Maximum of Two Arrays/2513.java

```
class Solution {
    public int minimizeSet(int divisor1, int divisor2, int uniqueCnt1, int uniqueCnt2) {
        final long divisorLcm = lcm(divisor1, divisor2);
        long l = 0;
        long r = Integer.MAX_VALUE;

        while (l < r) {
            final long m = (l + r) / 2;
            if (isPossible(m, divisorLcm, divisor1, divisor2, uniqueCnt1, uniqueCnt2))
                r = m;
            else
                l = m + 1;
        }

        return (int) l;
    }

    // Returns true if we can take uniqueCnt1 integers from [1..m] to arr1 and
    // take uniqueCnt2 integers from [1..m] to arr2.
    private boolean isPossible(long m, long divisorLcm, int divisor1, int divisor2, int uniqueCnt1,
                               int uniqueCnt2) {
        final long cnt1 = m - m / divisor1;
        final long cnt2 = m - m / divisor2;
        final long totalCnt = m - m / divisorLcm;
        return cnt1 >= uniqueCnt1 && cnt2 >= uniqueCnt2 && totalCnt >= uniqueCnt1 + uniqueCnt2;
    }

    private long gcd(int a, int b) {
        return b == 0 ? a : gcd(b, a % b);
    }

    private long lcm(int a, int b) {
        return a * (b / gcd(a, b));
    }
}
```


1567. 2514.java

Path: LeetCode-main/solutions/2514. Count Anagrams/2514.java

```
class Solution {
    public int countAnagrams(String s) {
        final int n = s.length();
        final long[][] factAndInvFact = getFactAndInvFact(n);
        final long[] fact = factAndInvFact[0];
        final long[] invFact = factAndInvFact[1];
        long ans = 1;

        for (final String word : s.split(" ")) {
            ans = ans * fact[word.length()] % MOD;
            int[] count = new int[26];
            for (final char c : word.toCharArray())
                ++count[c - 'a'];
            for (final int freq : count)
                ans = ans * invFact[freq] % MOD;
        }

        return (int) ans;
    }

    private static final int MOD = 1_000_000_007;

    private long[][] getFactAndInvFact(int n) {
        long[] fact = new long[n + 1];
        long[] invFact = new long[n + 1];
        long[] inv = new long[n + 1];
        fact[0] = invFact[0] = 1;
        inv[0] = inv[1] = 1;
        for (int i = 1; i <= n; ++i) {
            if (i >= 2)
                inv[i] = MOD - MOD / i * inv[MOD % i] % MOD;
            fact[i] = fact[i - 1] * i % MOD;
            invFact[i] = invFact[i - 1] * inv[i] % MOD;
        }
        return new long[][] {fact, invFact};
    }
}
```

1568. 2515.java

Path: LeetCode-main/solutions/2515. Shortest Distance to Target String in a Circular Array/2515.java

```
class Solution {
    public int closetTarget(String[] words, String target, int startIndex) {
        final int n = words.length;

        for (int i = 0; i < n; ++i) {
            if (words[(startIndex + i + n) % n].equals(target))
                return i;
            if (words[(startIndex - i + n) % n].equals(target))
                return i;
        }

        return -1;
    }
}
```

1569. 2516.java

Path: LeetCode-main/solutions/2516. Take K of Each Character From Left and Right/2516.java

```
class Solution {
    public int takeCharacters(String s, int k) {
        final int n = s.length();
        int ans = n;
        int[] count = new int[3];

        for (final char c : s.toCharArray())
            ++count[c - 'a'];

        if (count[0] < k || count[1] < k || count[2] < k)
            return -1;

        for (int l = 0, r = 0; r < n; ++r) {
            --count[s.charAt(r) - 'a'];
            while (count[s.charAt(r) - 'a'] < k)
                ++count[s.charAt(l++) - 'a'];
            ans = Math.min(ans, n - (r - l + 1));
        }

        return ans;
    }
}
```

1570. 2517.java

Path: LeetCode-main/solutions/2517. Maximum Tastiness of Candy Basket/2517.java

```
class Solution {
    public int maximumTastiness(int[] price, int k) {
        Arrays.sort(price);

        int l = 0;
        int r = Arrays.stream(price).max().getAsInt() - Arrays.stream(price).min().getAsInt() + 1;

        while (l < r) {
            final int m = (l + r) / 2;
            if (numBaskets(price, m) >= k)
                l = m + 1;
            else
                r = m;
        }

        return l - 1;
    }

    // Returns the number of baskets we can pick for m tastiness.
    private int numBaskets(int[] price, int m) {
        int baskets = 0;
        int prevPrice = -m;
        for (final int p : price)
            if (p >= prevPrice + m) {
                prevPrice = p;
                ++baskets;
            }
        return baskets;
    }
}
```

1571. 2518.java

Path: LeetCode-main/solutions/2518. Number of Great Partitions/2518.java

```
class Solution {
    public int countPartitions(int[] nums, int k) {
        final int MOD = 1_000_000_007;
        final long sum = Arrays.stream(nums).asLongStream().sum();
        long ans = modPow(2, nums.length, MOD); //  $2^n \% MOD$ 
        long[] dp = new long[k + 1];
        dp[0] = 1;

        for (final int num : nums)
            for (int i = k; i >= num; --i) {
                dp[i] += dp[i - num];
                dp[i] %= MOD;
            }

        // Subtract the cases that're not satisfied.
        for (int i = 0; i < k; ++i)
            if (sum - i < k) // Both group1 and group2 < k.
                ans -= dp[i];
            else
                ans -= dp[i] * 2;

        return (int) ((ans % MOD + MOD) % MOD);
    }

    private int modPow(int x, int n, int MOD) {
        int res = 1;
        for (int i = 0; i < n; ++i)
            res = res * x % MOD;
        return res;
    }
}
```

1572. 2519.java

Path: LeetCode-main/solutions/2519. Count the Number of K-Big Indices/2519.java

```
class FenwickTree {
    public FenwickTree(int n) {
        sums = new int[n + 1];
    }

    public void add(int i, int delta) {
        while (i < sums.length) {
            sums[i] += delta;
            i += lowbit(i);
        }
    }

    public int get(int i) {
        int sum = 0;
        while (i > 0) {
            sum += sums[i];
            i -= lowbit(i);
        }
        return sum;
    }

    private int[] sums;

    private static int lowbit(int i) {
        return i & -i;
    }
}

class Solution {
    public int kBigIndices(int[] nums, int k) {
        final int n = nums.length;
        int ans = 0;
        FenwickTree leftTree = new FenwickTree(n);
        FenwickTree rightTree = new FenwickTree(n);
        // left[i] := the number of `nums` < nums[i] with index < i
        int[] left = new int[n];
        // right[i] := the number of `nums` < nums[i] with index > i
        int[] right = new int[n];

        for (int i = 0; i < n; ++i) {
            left[i] = leftTree.get(nums[i] - 1);
            leftTree.add(nums[i], 1);
        }

        for (int i = n - 1; i >= 0; --i) {
            right[i] = rightTree.get(nums[i] - 1);
            rightTree.add(nums[i], 1);
        }

        for (int i = 0; i < n; ++i)
            if (left[i] >= k && right[i] >= k)
                ++ans;

        return ans;
    }
}
```

1573. 252.java

Path: LeetCode-main/solutions/252. Meeting Rooms/252.java

```
class Solution {  
    public boolean canAttendMeetings(int[][] intervals) {  
        Arrays.sort(intervals, Comparator.comparingInt(interval -> interval[0]));  
  
        for (int i = 1; i < intervals.length; ++i)  
            if (intervals[i - 1][1] > intervals[i][0])  
                return false;  
  
        return true;  
    }  
}
```

1574. 2520.java

Path: LeetCode-main/solutions/2520. Count the Digits That Divide a Number/2520.java

```
class Solution {  
    public int countDigits(int num) {  
        int ans = 0;  
  
        for (int n = num; n > 0; n /= 10)  
            if (num % (n % 10) == 0)  
                ++ans;  
  
        return ans;  
    }  
}
```


1576. 2522.java

Path: LeetCode-main/solutions/2522. Partition String Into Substrings With Values at Most K/2522.java

```
class Solution {
    public int minimumPartition(String s, int k) {
        int ans = 1;
        long curr = 0;

        for (final char c : s.toCharArray()) {
            curr = curr * 10 + c - '0';
            if (curr > k) {
                curr = c - '0';
                ++ans;
            }
            if (curr > k)
                return -1;
        }
        return ans;
    }
}
```

1577. 2523.java

Path: LeetCode-main/solutions/2523. Closest Prime Numbers in Range/2523.java

```
class Solution {
    public int[] closestPrimes(int left, int right) {
        final boolean[] isPrime = sieveEratosthenes(right + 1);
        List<Integer> primes = new ArrayList<>();

        for (int i = left; i <= right; ++i)
            if (isPrime[i])
                primes.add(i);

        if (primes.size() < 2)
            return new int[] {-1, -1};

        int minDiff = Integer.MAX_VALUE;
        int num1 = -1;
        int num2 = -1;

        for (int i = 1; i < primes.size(); ++i) {
            final int diff = prime.get(i) - prime.get(i - 1);
            if (diff < minDiff) {
                minDiff = diff;
                num1 = prime.get(i - 1);
                num2 = prime.get(i);
            }
        }

        return new int[] {num1, num2};
    }

    private boolean[] sieveEratosthenes(int n) {
        boolean[] isPrime = new boolean[n];
        Arrays.fill(isPrime, true);
        isPrime[0] = false;
        isPrime[1] = false;
        for (int i = 2; i * i < n; ++i)
            if (isPrime[i])
                for (int j = i * i; j < n; j += i)
                    isPrime[j] = false;
        return isPrime;
    }
}
```

1578. 2524.java

Path: LeetCode-main/solutions/2524. Maximum Frequency Score of a Subarray/2524.java

```
class Solution {
    public int maxFrequencyScore(int[] nums, int k) {
        Map<Integer, Integer> count = new HashMap<>();

        for (int i = 0; i < k; ++i)
            count.merge(nums[i], 1, Integer::sum);

        int sum = getInitialSum(count);
        int ans = sum;

        for (int i = k; i < nums.length; ++i) {
            // Remove the leftmost number that's out-of-window.
            final int leftNum = nums[i - k];
            sum = (sum - modPow(leftNum, count.get(leftNum)) + MOD) % MOD;
            // After decreasing its frequency, if it's still > 0, then add it back.
            if (count.merge(leftNum, -1, Integer::sum) > 0)
                sum = (sum + modPow(leftNum, count.get(leftNum))) % MOD;
            // Otherwise, remove it from the count map.
            else
                count.remove(leftNum);
            // Add the current number. Similarly, remove the current score like above.
            final int rightNum = nums[i];
            if (count.getOrDefault(rightNum, 0) > 0)
                sum = (sum - modPow(rightNum, count.get(rightNum)) + MOD) % MOD;
            sum = (sum + modPow(rightNum, count.merge(rightNum, 1, Integer::sum))) % MOD;
            ans = Math.max(ans, sum);
        }

        return ans;
    }

    private static final int MOD = 1_000_000_007;

    private int getInitialSum(Map<Integer, Integer> count) {
        int sum = 0;
        for (Map.Entry<Integer, Integer> entry : count.entrySet()) {
            final int num = entry.getKey();
            final int freq = entry.getValue();
            sum = (sum + modPow(num, freq)) % MOD;
        }
        return sum;
    }

    private int modPow(long x, long n) {
        if (n == 0)
            return 1;
        if (n % 2 == 1)
            return (int) (x * modPow(x % MOD, (n - 1)) % MOD);
        return modPow(x * x % MOD, (n / 2)) % MOD;
    }
}
```

1579. 2525.java

Path: LeetCode-main/solutions/2525. Categorize Box According to Criteria/2525.java

```
class Solution {
    public String categorizeBox(int length, int width, int height, int mass) {
        final boolean isBulky = length >= 10000 || width >= 10000 || height >= 10000 ||
            (long) length * width * height >= 1_000_000_000;
        final boolean isHeavy = mass >= 100;
        if (isBulky && isHeavy)
            return "Both";
        if (isBulky)
            return "Bulky";
        if (isHeavy)
            return "Heavy";
        return "Neither";
    }
}
```

1580. 2526.java

Path: LeetCode-main/solutions/2526. Find Consecutive Integers from a Data Stream/2526.java

```
class DataStream {
    public DataStream(int value, int k) {
        this.value = value;
        this.k = k;
    }

    public boolean consec(int num) {
        if (q.size() == k && q.poll() == value)
            --count;
        if (num == value)
            ++count;
        q.offer(num);
        return count == k;
    }

    private int value;
    private int k;
    private Queue<Integer> q = new ArrayDeque<>();
    private int count = 0;
}
```

1581. 2527.java

Path: LeetCode-main/solutions/2527. Find Xor-Beauty of Array/2527.java

```
class Solution {  
    public int xorBeauty(int[] nums) {  
        return Arrays.stream(nums).reduce((a, b) -> a ^ b).getAsInt();  
    }  
}
```

1582. 2528.java

Path: LeetCode-main/solutions/2528. Maximize the Minimum Powered City/2528.java

```
class Solution {
    public long maxPower(int[] stations, int r, int k) {
        long left = Arrays.stream(stations).min().getAsInt();
        long right = Arrays.stream(stations).asLongStream().sum() + k + 1;

        while (left < right) {
            final long mid = (left + right) / 2;
            if (check(stations.clone(), r, k, mid))
                left = mid + 1;
            else
                right = mid;
        }

        return left - 1;
    }

    // Returns true if each city can have at least `minPower`.
    boolean check(int[] stations, int r, int additionalStations, long minPower) {
        final int n = stations.length;
        // Initilaize `power` as the 0-th city's power - stations[r].
        long power = 0;

        for (int i = 0; i < r; ++i)
            power += stations[i];

        for (int i = 0; i < n; ++i) {
            if (i + r < n)
                power += stations[i + r]; // `power` = sum(stations[i - r..i + r]).
            if (power < minPower) {
                final long requiredPower = minPower - power;
                // There're not enough stations to plant.
                if (requiredPower > additionalStations)
                    return false;
                // Greedily plant `requiredPower` power stations in the farthest place
                // to cover as many cities as possible.
                stations[Math.min(n - 1, i + r)] += requiredPower;
                additionalStations -= requiredPower;
                power += requiredPower;
            }
            if (i - r >= 0)
                power -= stations[i - r];
        }

        return true;
    }
}
```


1583. 2529.java

Path: LeetCode-main/solutions/2529. Maximum Count of Positive Integer and Negative Integer/2529.java

```
class Solution {  
    public int maximumCount(int[] nums) {  
        return (int) Math.max(Arrays.stream(nums).filter(num -> num > 0).count(),  
                               Arrays.stream(nums).filter(num -> num < 0).count());  
    }  
}
```

1584. 253-2.java

Path: LeetCode-main/solutions/253. Meeting Rooms II/253-2.java

```
class Solution {
    public int minMeetingRooms(int[][] intervals) {
        final int n = intervals.length;
        int ans = 0;
        int[] starts = new int[n];
        int[] ends = new int[n];

        for (int i = 0; i < n; ++i) {
            starts[i] = intervals[i][0];
            ends[i] = intervals[i][1];
        }

        Arrays.sort(starts);
        Arrays.sort(ends);

        // J points to ends
        for (int i = 0, j = 0; i < n; ++i)
            if (starts[i] < ends[j])
                ++ans;
            else
                ++j;

        return ans;
    }
}
```

1585. 253.java

Path: LeetCode-main/solutions/253. Meeting Rooms II/253.java

```
class Solution {
    public int minMeetingRooms(int[][] intervals) {
        // Store the end times of each room.
        Queue<Integer> minHeap = new PriorityQueue<>();

        Arrays.sort(intervals, Comparator.comparingInt(interval -> interval[0]));

        for (int[] interval : intervals) {
            // There's no overlap, so we can reuse the same room.
            if (!minHeap.isEmpty() && interval[0] >= minHeap.peek())
                minHeap.poll();
            minHeap.offer(interval[1]);
        }

        return minHeap.size();
    }
}
```

1586. 2530.java

Path: LeetCode-main/solutions/2530. Maximal Score After Applying K Operations/2530.java

```
class Solution {
    public long maxKelements(int[] nums, int k) {
        long ans = 0;
        Queue<Integer> maxHeap = new PriorityQueue<>(Collections.reverseOrder());

        for (final int num : nums)
            maxHeap.offer(num);

        for (int i = 0; i < k; ++i) {
            final int num = maxHeap.poll();
            ans += num;
            maxHeap.offer((num + 2) / 3);
        }

        return ans;
    }
}
```

1587. 2531.java

Path: LeetCode-main/solutions/2531. Make Number of Distinct Characters Equal/2531.java

```
class Solution {
    public boolean isItPossible(String word1, String word2) {
        final int[] count1 = getCount(word1);
        final int[] count2 = getCount(word2);
        final int distinct1 = getDistinct(count1);
        final int distinct2 = getDistinct(count2);

        for (int i = 0; i < 26; ++i)
            for (int j = 0; j < 26; ++j) {
                if (count1[i] == 0 || count2[j] == 0)
                    continue;
                if (i == j) {
                    // Swapping the same letters won't change the number of distinct
                    // letters in each string, so just check if `distinct1 == distinct2`.
                    if (distinct1 == distinct2)
                        return true;
                    continue;
                }
                // The calculation is meaningful only when i != j.
                // Swap ('a' + i) in word1 with ('a' + j) in word2.
                final int distinctAfterSwap1 =
                    distinct1 - (count1[i] == 1 ? 1 : 0) + (count1[j] == 0 ? 1 : 0);
                final int distinctAfterSwap2 =
                    distinct2 - (count2[j] == 1 ? 1 : 0) + (count2[i] == 0 ? 1 : 0);
                if (distinctAfterSwap1 == distinctAfterSwap2)
                    return true;
            }

        return false;
    }

    private int[] getCount(final String s) {
        int[] count = new int[26];
        for (final char c : s.toCharArray())
            ++count[c - 'a'];
        return count;
    }

    private int getDistinct(int[] count) {
        return (int) Arrays.stream(count).filter(c -> c > 0).count();
    }
}
```

1588. 2532.java

Path: LeetCode-main/solutions/2532. Time to Cross a Bridge/2532.java

```
class Solution {
    public int findCrossingTime(int n, int k, int[][] time) {
        int ans = 0;
        // (leftToRight + rightToLeft, i)
        Queue<Pair<Integer, Integer>> leftBridgeQueue = createMaxHeap();
        Queue<Pair<Integer, Integer>> rightBridgeQueue = createMaxHeap();
        // (time to be idle, i)
        Queue<Pair<Integer, Integer>> leftWorkers = createMinHeap();
        Queue<Pair<Integer, Integer>> rightWorkers = createMinHeap();

        for (int i = 0; i < k; ++i)
            leftBridgeQueue.offer(new Pair<> (
                /*leftToRight*/ time[i][0] + /*rightToLeft*/ time[i][2], i));

        while (n > 0 || !rightBridgeQueue.isEmpty() || !rightWorkers.isEmpty()) {
            // Idle left workers get on the left bridge.
            while (!leftWorkers.isEmpty() && leftWorkers.peek().getKey() <= ans) {
                final int i = leftWorkers.poll().getValue();
                leftBridgeQueue.offer(new Pair<> (
                    /*leftToRight*/ time[i][0] + /*rightToLeft*/ time[i][2], i));
            }
            // Idle right workers get on the right bridge.
            while (!rightWorkers.isEmpty() && rightWorkers.peek().getKey() <= ans) {
                final int i = rightWorkers.poll().getValue();
                rightBridgeQueue.offer(new Pair<> (
                    /*leftToRight*/ time[i][0] + /*rightToLeft*/ time[i][2], i));
            }

            if (!rightBridgeQueue.isEmpty()) {
                // If the bridge is free, the worker waiting on the right side of the
                // bridge gets to cross the bridge. If more than one worker is waiting
                // on the right side, the one with the lowest efficiency crosses first.
                final int i = rightBridgeQueue.poll().getValue();
                ans += /*rightToLeft*/ time[i][2];
                leftWorkers.offer(new Pair<> (ans + /*putNew*/ time[i][3], i));
            } else if (!leftBridgeQueue.isEmpty() && n > 0) {
                // If the bridge is free and no worker is waiting on the right side, and
                // at least one box remains at the old warehouse, the worker on the left
                // side of the river gets to cross the bridge. If more than one worker
                // is waiting on the left side, the one with the lowest efficiency
                // crosses first.
                final int i = leftBridgeQueue.poll().getValue();
                ans += /*leftToRight*/ time[i][0];
                rightWorkers.offer(new Pair<> (ans + /*pickOld*/ time[i][1], i));
                --n;
            } else {
                // Advance the time of the last crossing worker.
                ans = Math.min(!leftWorkers.isEmpty() && n > 0 ? leftWorkers.peek().getKey()
                    : Integer.MAX_VALUE,
                    !rightWorkers.isEmpty() ? rightWorkers.peek().getKey() : Integer.MAX_VALUE);
            }
        }

        return ans;
    }

    private Queue<Pair<Integer, Integer>> createMaxHeap() {
        return new PriorityQueue<> (
            Comparator.comparing(Pair<Integer, Integer>::getKey, Comparator.reverseOrder())
                .thenComparing(Pair<Integer, Integer>::getValue, Comparator.reverseOrder()));
    }

    private Queue<Pair<Integer, Integer>> createMinHeap() {
        return new PriorityQueue<> (Comparator.comparingInt(Pair<Integer, Integer>::getKey)
            .thenComparingInt(Pair<Integer, Integer>::getValue));
    }
}
```

1589. 2533.java

Path: LeetCode-main/solutions/2533. Number of Good Binary Strings/2533.java

```
class Solution {
    public int goodBinaryStrings(int minLength, int maxLength, int oneGroup, int zeroGroup) {
        final int MOD = 1_000_000_007;
        // dp[i] := the number of good binary strings with length i
        int[] dp = new int[maxLength + 1];
        dp[0] = 1; // ""

        for (int i = 0; i <= maxLength; ++i)
            // There are good binary strings with length i, so we can append
            // consecutive 0s or 1s after it.
            if (dp[i] > 0) {
                final int appendZeros = i + zeroGroup;
                if (appendZeros <= maxLength) {
                    dp[appendZeros] += dp[i];
                    dp[appendZeros] %= MOD;
                }
                final int appendOnes = i + oneGroup;
                if (appendOnes <= maxLength) {
                    dp[appendOnes] += dp[i];
                    dp[appendOnes] %= MOD;
                }
            }

        int ans = 0;
        for (int i = minLength; i <= maxLength; ++i) {
            ans += dp[i];
            ans %= MOD;
        }
        return ans;
    }
}
```

1590. 2534.java

Path: LeetCode-main/solutions/2534. Time Taken to Cross the Door/2534.java

```
class Solution {
    public int[] timeTaken(int[] arrival, int[] state) {
        final int n = arrival.length;
        int[] ans = new int[n];
        Queue<Integer>[] qs = new Queue[2];
        qs[0] = new LinkedList<>(); // enter queue
        qs[1] = new LinkedList<>(); // exit queue

        for (int i = 0; i < n; ++i) {
            popQueues(arrival[i], qs, ans);
            // If the door was not used in the previous second, then the person who
            // wants to exit goes first.
            if (arrival[i] > time) {
                time = arrival[i]; // Forward `time` to now.
                d = 1;             // Reset priority to exit.
            }
            qs[state[i]].offer(i);
        }

        popQueues(2000000, qs, ans);
        return ans;
    }

    private int time = 0;
    private int d = 1; // 0: enter, 1: exit

    private void popQueues(int arrivalTime, Queue<Integer>[] qs, int[] ans) {
        while (arrivalTime > time && (!qs[0].isEmpty() || !qs[1].isEmpty())) {
            if (qs[d].isEmpty())
                d ^= 1; // Toggle between enter (0) and exit (1)
            ans[qs[d].poll()] = time++;
        }
    }
}
```


1591. 2535.java

Path: LeetCode-main/solutions/2535. Difference Between Element Sum and Digit Sum of an Array/2535.java

```
class Solution {
    public int differenceOfSum(int[] nums) {
        final int elementSum = Arrays.stream(nums).sum();
        final int allDigitSum = getAllDigitSum(nums);
        return Math.abs(elementSum - allDigitSum);
    }

    private int getAllDigitSum(int[] nums) {
        int allDigitSum = 0;
        for (final int num : nums)
            allDigitSum += getDigitSum(num);
        return allDigitSum;
    }

    private int getDigitSum(int num) {
        int digitSum = 0;
        while (num > 0) {
            digitSum += num % 10;
            num /= 10;
        }
        return digitSum;
    }
}
```

1592. 2538.java

Path: LeetCode-main/solutions/2538. Difference Between Maximum and Minimum Price Sum/2538.java

```
class Solution {
    public long maxOutput(int n, int[][] edges, int[] price) {
        List<Integer>[] tree = new List[n];
        // maxSums[i] := the maximum the sum of path rooted at i
        int[] maxSums = new int[n];

        for (int i = 0; i < n; ++i)
            tree[i] = new ArrayList<>();

        for (int[] edge : edges) {
            final int u = edge[0];
            final int v = edge[1];
            tree[u].add(v);
            tree[v].add(u);
        }

        // Precalculate `maxSums`.
        maxSum(tree, 0, /*prev=*/-1, maxSums, price);
        reroot(tree, 0, /*prev=*/-1, /*parentSum=*/0, maxSums, price);
        return (long) ans;
    }

    private int ans = 0;

    private int maxSum(List<Integer>[] tree, int u, int prev, int[] maxSums, int[] price) {
        int maxChildSum = 0;
        for (final int v : tree[u])
            if (v != prev)
                maxChildSum = Math.max(maxChildSum, maxSum(tree, v, u, maxSums, price));
        return maxSums[u] = price[u] + maxChildSum;
    }

    private void reroot(List<Integer>[] tree, int u, int prev, int parentSum, int[] maxSums,
                        int[] price) {
        // Get top two sums and top one node index.
        int maxSubtreeSum1 = 0;
        int maxSubtreeSum2 = 0;
        int maxNode = -1;
        for (final int v : tree[u]) {
            if (v == prev)
                continue;
            if (maxSums[v] > maxSubtreeSum1) {
                maxSubtreeSum2 = maxSubtreeSum1;
                maxSubtreeSum1 = maxSums[v];
                maxNode = v;
            } else if (maxSums[v] > maxSubtreeSum2) {
                maxSubtreeSum2 = maxSums[v];
            }
        }

        if (tree[u].size() == 1)
            ans = Math.max(ans, Math.max(parentSum, maxSubtreeSum1));

        for (final int v : tree[u]) {
            if (v == prev)
                continue;
            final int nextParentSum = (v == maxNode ? price[u] + Math.max(parentSum, maxSubtreeSum2)
                                     : price[u] + Math.max(parentSum, maxSubtreeSum1));
            reroot(tree, v, u, nextParentSum, maxSums, price);
        }
    }
}
```

1593. 2539.java

Path: LeetCode-main/solutions/2539. Count the Number of Good Subsequences/2539.java

```
class Solution {
    public int countGoodSubsequences(String s) {
        // For each frequency f in [1, max(freq)], start with "" and calculate how
        // many subsequences can be finalructed with each letter's frequency = f.
        //
        // e.g. s = "abb", so f = max(freq) = 2.
        //
        // For f = 1, with 1 way to build "", choose any 'a' to finalruct a good
        // subseq, so # of good subsequences = 1 + 1 * (1, 1) = 2 ("", "a"). Next, add
        // 'b' and # of good subsequences = 2 + 2 * (2, 1) = 6 ("", "a", "b1", "b2",
        // "ab1", "ab2"). So, the number of good subsequences for f = 1 is 5 since we need to
        // exclude "".
        //
        // For f = 2, with 1 way to build "", choose any two 'b's to finalruct a
        // good subseq, so # of good subsequences = 1 + 1 * (2, 2) is 2 ("", "bb"). So,
        // the number of good subsequences for f = 2 = 1 since we need to exclude "".
        //
        // Therefore, the number of good subsequences for "aab" = 5 + 1 = 6.
        final int MOD = 1_000_000_007;
        int ans = 0;
        int[] count = new int[26];

        for (final char c : s.toCharArray())
            ++count[c - 'a'];

        final int maxFreq = Arrays.stream(count).max().getAsInt();
        final long[][] factAndInvFact = getFactAndInvFact(maxFreq);
        final long[] fact = factAndInvFact[0];
        final long[] invFact = factAndInvFact[1];

        for (int freq = 1; freq <= maxFreq; ++freq) {
            long numSubseqs = 1; // ""
            for (final int charFreq : count)
                if (charFreq >= freq)
                    numSubseqs = (numSubseqs + //
                        numSubseqs * nCk(charFreq, freq, fact, invFact)) %
                        MOD;
            ans += numSubseqs - 1; // Minus "".
            ans %= MOD;
        }

        return ans;
    }

    private static final int MOD = 1_000_000_007;

    private long[][] getFactAndInvFact(int n) {
        long[] fact = new long[n + 1];
        long[] invFact = new long[n + 1];
        long[] inv = new long[n + 1];
        fact[0] = invFact[0] = 1;
        inv[0] = inv[1] = 1;
        for (int i = 1; i <= n; ++i) {
            if (i >= 2)
                inv[i] = MOD - MOD / i * inv[MOD % i] % MOD;
            fact[i] = fact[i - 1] * i % MOD;
            invFact[i] = invFact[i - 1] * inv[i] % MOD;
        }
        return new long[][] {fact, invFact};
    }

    private int nCk(int n, int k, long[] fact, long[] invFact) {
        return (int) (fact[n] * invFact[k] % MOD * invFact[n - k] % MOD);
    }
}
```

1594. 254.java

Path: LeetCode-main/solutions/254. Factor Combinations/254.java

```
class Solution {
    public List<List<Integer>> getFactors(int n) {
        List<List<Integer>> ans = new ArrayList<>();
        dfs(n, 2, new ArrayList<>(), ans); // The minimum factor is 2.
        return ans;
    }

    private void dfs(int n, int s, List<Integer> path, List<List<Integer>> ans) {
        if (n == 1) {
            if (path.size() > 1)
                ans.add(new ArrayList<>(path));
            return;
        }

        for (int i = s; i <= n; ++i)
            if (n % i == 0) {
                path.add(i);
                dfs(n / i, i, path, ans);
                path.remove(path.size() - 1);
            }
    }
}
```

1595. 2540.java

Path: LeetCode-main/solutions/2540. Minimum Common Value/2540.java

```
class Solution {
    public int getCommon(int[] nums1, int[] nums2) {
        int i = 0; // nums1's index
        int j = 0; // nums2's index

        while (i < nums1.length && j < nums2.length) {
            if (nums1[i] == nums2[j])
                return nums1[i];
            if (nums1[i] < nums2[j])
                ++i;
            else
                ++j;
        }
        return -1;
    }
}
```

1596. 2541.java

Path: LeetCode-main/solutions/2541. Minimum Operations to Make Array Equal II/2541.java

```
class Solution {
    public long minOperations(int[] nums1, int[] nums2, int k) {
        if (k == 0)
            return Arrays.equals(nums1, nums2) ? 0 : -1;

        long ans = 0;
        long opsDiff = 0; // the number of increments - the number of decrements

        for (int i = 0; i < nums1.length; ++i) {
            final int diff = nums1[i] - nums2[i];
            if (diff == 0)
                continue;
            if (diff % k != 0)
                return -1;
            final int ops = diff / k;
            opsDiff += ops;
            ans += Math.abs(ops);
        }

        return opsDiff == 0 ? ans / 2 : -1;
    }
}
```

1597. 2542.java

Path: LeetCode-main/solutions/2542. Maximum Subsequence Score/2542.java

```
class Solution {
    // Same as 1383. Maximum Performance of a Team
    public long maxScore(int[] nums1, int[] nums2, int k) {
        long ans = 0;
        long sum = 0;
        // (nums2[i], nums1[i]) sorted by nums2[i] in descending order
        Pair<Integer, Integer>[] A = new Pair[nums1.length];
        Queue<Integer> minHeap = new PriorityQueue<>();

        for (int i = 0; i < nums1.length; ++i)
            A[i] = new Pair<>(nums2[i], nums1[i]);

        Arrays.sort(A, Comparator.comparing(Pair::getKey, Comparator.reverseOrder()));

        for (Pair<Integer, Integer> a : A) {
            final int num2 = a.getKey();
            final int num1 = a.getValue();
            minHeap.offer(num1);
            sum += num1;
            if (minHeap.size() > k)
                sum -= minHeap.poll();
            if (minHeap.size() == k)
                ans = Math.max(ans, sum * num2);
        }

        return ans;
    }
}
```

1598. 2543.java

Path: LeetCode-main/solutions/2543. Check if Point Is Reachable/2543.java

```
class Solution {  
    public boolean isReachable(int targetX, int targetY) {  
        return Integer.bitCount(gcd(targetX, targetY)) == 1;  
    }  
  
    private int gcd(int a, int b) {  
        return b == 0 ? a : gcd(b, a % b);  
    }  
}
```


1599. 2544.java

Path: LeetCode-main/solutions/2544. Alternating Digit Sum/2544.java

```
class Solution {  
    public int alternateDigitSum(int n) {  
        int ans = 0;  
        int sign = 1;  
  
        for (; n > 0; n /= 10) {  
            sign *= -1;  
            ans += sign * n % 10;  
        }  
  
        return sign * ans;  
    }  
}
```

1600. 2545.java

Path: LeetCode-main/solutions/2545. Sort the Students by Their Kth Score/2545.java

```
class Solution {  
    public int[][] sortTheStudents(int[][] score, int k) {  
        Arrays.sort(score, Comparator.comparingInt(a -> - a[k]));  
        return score;  
    }  
}
```

1601. 2546.java

Path: LeetCode-main/solutions/2546. Apply Bitwise Operations to Make Strings Equal/2546.java

```
class Solution {  
    public boolean makeStringsEqual(String s, String target) {  
        return s.contains("1") == target.contains("1");  
    }  
}
```

1602. 2547.java

Path: LeetCode-main/solutions/2547. Minimum Cost to Split an Array/2547.java

```
class Solution {
    public int minCost(int[] nums, int k) {
        final int MAX = 1001;
        final int n = nums.length;
        // trimmedLength[i][j] := trimmed(nums[i..j]).length
        int[][] trimmedLength = new int[n][n];
        // dp[i] := the minimum cost to split nums[i..n)
        int[] dp = new int[n + 1];
        Arrays.fill(dp, Integer.MAX_VALUE / 2);

        for (int i = 0; i < n; ++i) {
            int length = 0;
            int[] count = new int[MAX];
            for (int j = i; j < n; ++j) {
                if (++count[nums[j]] == 2)
                    length += 2;
                else if (count[nums[j]] > 2)
                    ++length;
                trimmedLength[i][j] = length;
            }
        }

        dp[n] = 0;

        for (int i = n - 1; i >= 0; --i)
            for (int j = i; j < n; ++j)
                dp[i] = Math.min(dp[i], k + trimmedLength[i][j] + dp[j + 1]);

        return dp[0];
    }
}
```

1603. 2548.java

Path: LeetCode-main/solutions/2548. Maximum Price to Fill a Bag/2548.java

```
class Solution {
    public double maxPrice(int[][] items, int capacity) {
        double ans = 0;

        // Sort items based on price/weight.
        Arrays.sort(items, new Comparator<int[]>() {
            @Override
            public int compare(int[] a, int[] b) {
                return Double.compare((double) b[0] / b[1], (double) a[0] / a[1]);
            }
        });

        for (int[] item : items) {
            final int price = item[0];
            final int weight = item[1];
            // The bag is filled.
            if (capacity <= weight)
                return ans + price * capacity / (double) weight;
            ans += price;
            capacity -= weight;
        }

        return -1;
    }
}
```

1604. 2549.java

Path: LeetCode-main/solutions/2549. Count Distinct Numbers on Board/2549.java

```
class Solution {  
    public int distinctIntegers(int n) {  
        return Math.max(n - 1, 1);  
    }  
}
```

1605. 255-2.java

Path: LeetCode-main/solutions/255. Verify Preorder Sequence in Binary Search Tree/255-2.java

```
class Solution {
    public boolean verifyPreorder(int[] preorder) {
        int low = Integer.MIN_VALUE;
        Deque<Integer> stack = new ArrayDeque<>();

        for (final int p : preorder) {
            if (p < low)
                return false;
            while (!stack.isEmpty() && stack.peek() < p)
                low = stack.pop();
            stack.push(p);
        }

        return true;
    }
}
```

1606. 255-3.java

Path: LeetCode-main/solutions/255. Verify Preorder Sequence in Binary Search Tree/255-3.java

```
class Solution {
    public boolean verifyPreorder(int[] preorder) {
        int low = Integer.MIN_VALUE;
        int i = -1;

        for (final int p : preorder) {
            if (p < low)
                return false;
            while (i >= 0 && preorder[i] < p)
                low = preorder[i--];
            preorder[++i] = p;
        }

        return true;
    }
}
```


1607. 255.java

Path: LeetCode-main/solutions/255. Verify Preorder Sequence in Binary Search Tree/255.java

```
class Solution {
    public boolean verifyPreorder(int[] preorder) {
        dfs(preorder, Integer.MIN_VALUE, Integer.MAX_VALUE);
        return i == preorder.length;
    }

    private int i = 0;

    private void dfs(int[] preorder, int mn, int mx) {
        if (i == preorder.length)
            return;
        if (preorder[i] < mn || preorder[i] > mx)
            return;

        final int val = preorder[i++];
        dfs(preorder, mn, val);
        dfs(preorder, val, mx);
    }
}
```

1608. 2550.java

Path: LeetCode-main/solutions/2550. Count Collisions of Monkeys on a Polygon/2550.java

```
class Solution {
    public int monkeyMove(int n) {
        final int res = (int) modPow(2, n) - 2;
        return res < 0 ? res + MOD : res;
    }

    private static final int MOD = 1_000_000_007;

    private long modPow(long x, long n) {
        if (n == 0)
            return 1;
        if (n % 2 == 1)
            return x * modPow(x, n - 1) % MOD;
        return modPow(x * x % MOD, n / 2);
    }
}
```

1609. 2551.java

Path: LeetCode-main/solutions/2551. Put Marbles in Bags/2551.java

```
class Solution {
    public long putMarbles(int[] weights, int k) {
        // To distribute marbles into k bags, there will be k - 1 cuts. If there's a
        // cut after weights[i], then weights[i] and weights[i + 1] will be added to
        // the cost. Also, no matter how we cut, weights[0] and weights[n - 1] will
        // be counted. So, the goal is to find the max/min k - 1 weights[i] +
        // weights[i + 1].
        int[] arr = new int[weights.length - 1]; // weights[i] + weights[i + 1]
        long mn = 0;
        long mx = 0;

        for (int i = 0; i < arr.length; ++i)
            arr[i] = weights[i] + weights[i + 1];

        Arrays.sort(arr);

        for (int i = 0; i < k - 1; ++i) {
            mn += arr[i];
            mx += arr[arr.length - 1 - i];
        }

        return mx - mn;
    }
}
```

1610. 2552.java

Path: LeetCode-main/solutions/2552. Count Increasing Quadruplets/2552.java

```
class Solution {
    public long countQuadruplets(int[] nums) {
        long ans = 0;
        // dp[j] := the number of triplets (i, j, k) where i < j < k and nums[i] < nums[k] <
        // nums[j]. Keep this information for l to use later.
        int[] dp = new int[nums.size()];

        // k can be treated as l.
        for (int k = 2; k < nums.length; ++k)
            // j can be treated as i.
            for (int j = 0, numLessThanK = 0; j < k; ++j)
                if (nums[j] < nums[k]) {
                    ++numLessThanK; // nums[i] < nums[k]
                    // nums[j] < nums[l], so we should add dp[j] since we find a new
                    // quadruplets for (i, j, k, l).
                    ans += dp[j];
                } else if (nums[j] > nums[k]) {
                    dp[j] += numLessThanK;
                }

        return ans;
    }
}
```

1611. 2553.java

Path: LeetCode-main/solutions/2553. Separate the Digits in an Array/2553.java

```
class Solution {
    public int[] separateDigits(int[] nums) {
        List<Integer> ans = new ArrayList<>();

        for (final int num : nums)
            for (final char c : String.valueOf(num).toCharArray())
                ans.add(c - '0');

        return ans.stream().mapToInt(Integer::intValue).toArray();
    }
}
```

1612. 2554.java

Path: LeetCode-main/solutions/2554. Maximum Number of Integers to Choose From a Range I/2554.java

```
class Solution {
    public int maxCount(int[] banned, int n, int maxSum) {
        int ans = 0;
        int sum = 0;
        Set<Integer> bannedSet = Arrays.stream(banned).boxed().collect(Collectors.toSet());

        for (int i = 1; i <= n; ++i)
            if (!bannedSet.contains(i) && sum + i <= maxSum) {
                ++ans;
                sum += i;
            }

        return ans;
    }
}
```

1613. 2555.java

Path: LeetCode-main/solutions/2555. Maximize Win From Two Segments/2555.java

```
class Solution {
    public int maximizeWin(int[] prizePositions, int k) {
        int ans = 0;
        // dp[i] := the maximum number of prizes to choose the first i
        // `prizePositions`
        int[] dp = new int[prizePositions.length + 1];

        for (int i = 0, j = 0; i < prizePositions.length; ++i) {
            while (prizePositions[i] - prizePositions[j] > k)
                ++j;
            final int covered = i - j + 1;
            dp[i + 1] = Math.max(dp[i], covered);
            ans = Math.max(ans, dp[j] + covered);
        }

        return ans;
    }
}
```

1614. 2556.java

Path: LeetCode-main/solutions/2556. Disconnect Path in a Binary Matrix by at Most One Flip/2556.java

```
class Solution {
    public boolean isPossibleToCutPath(int[][] grid) {
        if (!hasPath(grid, 0, 0))
            return true;
        // Reassign (0, 0) as 1.
        grid[0][0] = 1;
        return !hasPath(grid, 0, 0);
    }

    // Returns true if there's a path from (0, 0) to (m - 1, n - 1).
    // Also marks the visited path as 0 except (m - 1, n - 1).
    private boolean hasPath(int[][] grid, int i, int j) {
        if (i == grid.length || j == grid[0].length)
            return false;
        if (i == grid.length - 1 && j == grid[0].length - 1)
            return true;
        if (grid[i][j] == 0)
            return false;

        grid[i][j] = 0;
        // Go down first. Since we use OR logic, we'll only mark one path.
        return hasPath(grid, i + 1, j) || hasPath(grid, i, j + 1);
    }
}
```


1615. 2557.java

Path: LeetCode-main/solutions/2557. Maximum Number of Integers to Choose From a Range II/2557.java

```
class Solution {
    public int maxCount(int[] banned, int n, long maxSum) {
        Set<Integer> bannedSet = Arrays.stream(banned).boxed().collect(Collectors.toSet());
        int l = 1;
        int r = n;

        while (l < r) {
            final int m = (l + r + 1) / 2;
            if (getSum(bannedSet, m) > maxSum)
                r = m - 1;
            else
                l = m;
        }

        int ans = l;
        for (final int b : bannedSet)
            if (b <= l)
                --ans;
        return ans;
    }

    // Returns sum([1..m]) - sum(bannedSet).
    private long getSum(Set<Integer> bannedSet, long m) {
        long sum = m * (m + 1) / 2; // sum([1..m])
        for (final int b : bannedSet)
            if (b <= m)
                sum -= b;
        return sum;
    }
}
```

1616. 2558.java

Path: LeetCode-main/solutions/2558. Take Gifts From the Richest Pile/2558.java

```
class Solution {
    public long pickGifts(int[] gifts, int k) {
        long ans = 0;
        Queue<Integer> maxHeap = new PriorityQueue<>(Collections.reverseOrder());

        for (final int gift : gifts)
            maxHeap.offer(gift);

        for (int i = 0; i < k; ++i) {
            final int squaredMax = (int) Math.sqrt(maxHeap.poll());
            maxHeap.offer(squaredMax);
        }

        while (!maxHeap.isEmpty())
            ans += maxHeap.poll();

        return ans;
    }
}
```

1617. 2559.java

Path: LeetCode-main/solutions/2559. Count Vowel Strings in Ranges/2559.java

```
class Solution {
    public int[] vowelStrings(String[] words, int[][] queries) {
        int[] ans = new int[queries.length];
        // prefix[i] := the number of the first i words that start with and end in a vowel
        int[] prefix = new int[words.length + 1];

        for (int i = 0; i < words.length; ++i)
            prefix[i + 1] += prefix[i] + (startsAndEndsWithVowel(words[i]) ? 1 : 0);

        for (int i = 0; i < queries.length; ++i) {
            final int l = queries[i][0];
            final int r = queries[i][1];
            ans[i] = prefix[r + 1] - prefix[l];
        }

        return ans;
    }

    private boolean startsAndEndsWithVowel(final String word) {
        return isVowel(word.charAt(0)) && isVowel(word.charAt(word.length() - 1));
    }

    private boolean isVowel(char c) {
        return "aeiou".indexOf(c) != -1;
    }
}
```

1618. 256.java

Path: LeetCode-main/solutions/256. Paint House/256.java

```
class Solution {
    public int minCost(int[][] costs) {
        final int n = costs.length;

        for (int i = 1; i < n; ++i) {
            costs[i][0] += Math.min(costs[i - 1][1], costs[i - 1][2]);
            costs[i][1] += Math.min(costs[i - 1][0], costs[i - 1][2]);
            costs[i][2] += Math.min(costs[i - 1][0], costs[i - 1][1]);
        }

        return Math.min(costs[n - 1][0], Math.min(costs[n - 1][1], costs[n - 1][2]));
    }
}
```

1619. 2560.java

Path: LeetCode-main/solutions/2560. House Robber IV/2560.java

```
class Solution {
    public int minCapability(int[] nums, int k) {
        int l = Arrays.stream(nums).min().getAsInt();
        int r = Arrays.stream(nums).max().getAsInt();

        while (l < r) {
            final int m = (l + r) / 2;
            if (numStolenHouses(nums, m) >= k)
                r = m;
            else
                l = m + 1;
        }

        return l;
    }

    private int numStolenHouses(int[] nums, int capacity) {
        int stolenHouses = 0;
        for (int i = 0; i < nums.length; ++i)
            if (nums[i] <= capacity) {
                ++stolenHouses;
                ++i;
            }
        return stolenHouses;
    }
}
```

1620. 2561.java

Path: LeetCode-main/solutions/2561. Rearranging Fruits/2561.java

```
class Solution {
    public long minCost(int[] basket1, int[] basket2) {
        long ans = 0;
        List<Integer> swapped = new ArrayList<>();
        Map<Integer, Integer> count = new HashMap<>();

        for (final int b : basket1)
            count.merge(b, 1, Integer::sum);

        for (final int b : basket2)
            count.merge(b, -1, Integer::sum);

        for (Map.Entry<Integer, Integer> entry : count.entrySet()) {
            final Integer num = entry.getKey();
            final Integer freq = entry.getValue();
            if (freq % 2 != 0)
                return -1;
            for (int i = 0; i < Math.abs(freq) / 2; ++i)
                swapped.add(num);
        }

        final int minNum =
            Math.min(Arrays.stream(basket1).min().getAsInt(), Arrays.stream(basket2).min().getAsInt());
        Collections.sort(swapped);

        for (int i = 0; i < swapped.size() / 2; ++i)
            ans += Math.min(minNum * 2, swapped.get(i));
        return ans;
    }
}
```

1621. 2563.java

Path: LeetCode-main/solutions/2563. Count the Number of Fair Pairs/2563.java

```
class Solution {
    public long countFairPairs(int[] nums, int lower, int upper) {
        // nums[i] + nums[j] == nums[j] + nums[i], so the condition that i < j
        // degrades to i != j and we can sort the array.
        Arrays.sort(nums);
        return countLess(nums, upper) - countLess(nums, lower - 1);
    }

    private long countLess(int[] nums, int sum) {
        long res = 0;
        for (int i = 0, j = nums.length - 1; i < j; ++i) {
            while (i < j && nums[i] + nums[j] > sum)
                --j;
            res += j - i;
        }
        return res;
    }
}
```

1622. 2564.java

Path: LeetCode-main/solutions/2564. Substring XOR Queries/2564.java

```
class Solution {
    public int[][] substringXorQueries(String s, int[][] queries) {
        final int MAX_BIT = 30;
        int[][] ans = new int[queries.length][2];
        // {val: (left, right)} := s[left..right]'s decimal value = val
        Map<Integer, Pair<Integer, Integer>> valToLeftAndRight = new HashMap<>();

        for (int left = 0; left < s.length(); ++left) {
            int val = 0;
            if (s.charAt(left) == '0') {
                // edge case: Save the index of the first 0.
                if (!valToLeftAndRight.containsKey(0))
                    valToLeftAndRight.put(val, new Pair<>(left, left));
                continue;
            }
            final int maxRight = Math.min(s.length(), left + MAX_BIT);
            for (int right = left; right < maxRight; ++right) {
                val = val * 2 + s.charAt(right) - '0';
                if (!valToLeftAndRight.containsKey(val))
                    valToLeftAndRight.put(val, new Pair<>(left, right));
            }
        }

        for (int i = 0; i < queries.length; ++i) {
            final int first = queries[i][0];
            final int second = queries[i][1];
            final int val = first ^ second;
            Pair<Integer, Integer> leftAndRight = valToLeftAndRight.get(val);
            if (leftAndRight == null) {
                ans[i] = new int[] {-1, -1};
            } else {
                final int left = leftAndRight.getKey();
                final int right = leftAndRight.getValue();
                ans[i] = new int[] {left, right};
            }
        }

        return ans;
    }
}
```


1623. 2566.java

Path: LeetCode-main/solutions/2566. Maximum Difference by Remapping a Digit/2566.java

```
class Solution {
    public int minMaxDifference(int num) {
        final String s = String.valueOf(num);
        final char to9 = s.charAt(firstNotNineIndex(s));
        final char to0 = s.charAt(0);
        return getMax(new StringBuilder(s), to9) - getMin(new StringBuilder(s), to0);
    }

    private int firstNotNineIndex(final String s) {
        for (int i = 0; i < s.length(); ++i)
            if (s.charAt(i) != '9')
                return i;
        return 0;
    }

    private int getMax(StringBuilder sb, char to9) {
        for (int i = 0; i < sb.length(); ++i)
            if (sb.charAt(i) == to9)
                sb.setCharAt(i, '9');
        return Integer.parseInt(sb.toString());
    }

    private int getMin(StringBuilder sb, char to0) {
        for (int i = 0; i < sb.length(); ++i)
            if (sb.charAt(i) == to0)
                sb.setCharAt(i, '0');
        return Integer.parseInt(sb.toString());
    }
}
```

1624. 2567.java

Path: LeetCode-main/solutions/2567. Minimum Score by Changing Two Elements/2567.java

```
class Solution {
    public int minimizeSum(int[] nums) {
        Arrays.sort(nums);
        // Can always change the number to any other number in `nums`, so `low` becomes 0.
        // Thus, rephrase the problem as finding the minimum `high`.
        final int n = nums.length;
        final int highOfChangingTwoMins = nums[n - 1] - nums[2];
        final int highOfChangingTwoMaxs = nums[n - 3] - nums[0];
        final int highOfChangingMinAndMax = nums[n - 2] - nums[1];
        return Math.min(Math.min(highOfChangingTwoMins, highOfChangingTwoMaxs),
            highOfChangingMinAndMax);
    }
}
```

1625. 2568.java

Path: LeetCode-main/solutions/2568. Minimum Impossible OR/2568.java

```
class Solution {
    public int minImpossibleOR(int[] nums) {
        int ans = 1;
        Set<Integer> numsSet = Arrays.stream(nums).boxed().collect(Collectors.toSet());

        while (numsSet.contains(ans))
            ans <<= 1;

        return ans;
    }
}
```

1626. 257.java

Path: LeetCode-main/solutions/257. Binary Tree Paths/257.java

```
class Solution {
    public List<String> binaryTreePaths(TreeNode root) {
        List<String> ans = new ArrayList<>();
        dfs(root, new StringBuilder(), ans);
        return ans;
    }

    private void dfs(TreeNode root, StringBuilder sb, List<String> ans) {
        if (root == null)
            return;
        if (root.left == null && root.right == null) {
            ans.add(sb.append(root.val).toString());
            return;
        }

        final int length = sb.length();
        dfs(root.left, sb.append(root.val).append("->"), ans);
        sb.setLength(length);
        dfs(root.right, sb.append(root.val).append("->"), ans);
        sb.setLength(length);
    }
}
```

1627. 2570.java

Path: LeetCode-main/solutions/2570. Merge Two 2D Arrays by Summing Values/2570.java

```
class Solution {
    public int[][] mergeArrays(int[][] nums1, int[][] nums2) {
        final int MAX = 1000;
        List<int[]> ans = new ArrayList<>();
        int[] count = new int[MAX + 1];

        addCount(nums1, count);
        addCount(nums2, count);

        for (int i = 1; i <= MAX; ++i)
            if (count[i] > 0)
                ans.add(new int[] {i, count[i]});

        return ans.stream().toArray(int[][] ::new);
    }

    private void addCount(int[][] nums, int[] count) {
        for (int[] idAndVal : nums) {
            final int id = idAndVal[0];
            final int val = idAndVal[1];
            count[id] += val;
        }
    }
}
```

1628. 2571.java

Path: LeetCode-main/solutions/2571. Minimum Operations to Reduce an Integer to 0/2571.java

```
class Solution {
    public int minOperations(int n) {
        // The strategy is that when the end of n is
        // 1. consecutive 1s, add 1 ( $2^0$ ).
        // 2. single 1, subtract 1 ( $2^0$ ).
        // 3. 0, subtract  $2^k$  to omit the last 1. Equivalently,  $n \gg 1$ .
        //
        // e.g.
        //
        //      n = 0b101
        // n -=  $2^0$  -> 0b100
        // n -=  $2^2$  -> 0b0
        //      n = 0b1011
        // n +=  $2^0$  -> 0b1100
        // n -=  $2^2$  -> 0b1000
        // n -=  $2^3$  -> 0b0
        int ans = 0;

        while (n > 0)
            if ((n & 3) == 3) {
                ++n;
                ++ans;
            } else if (n % 2 == 1) {
                --n;
                ++ans;
            } else {
                n >>= 1;
            }

        return ans;
    }
}
```

1629. 2572.java

Path: LeetCode-main/solutions/2572. Count the Number of Square-Free Subsets/2572.java

```
class Solution {
    public int squareFreeSubsets(int[] nums) {
        Integer[][] mem = new Integer[nums.length][1 << (PRIMES_COUNT + 1)];
        int[] masks = new int[nums.length];

        for (int i = 0; i < nums.length; ++i)
            masks[i] = getMask(nums[i]);

        // -1 means that we take no number.
        // `used` is initialized to 1 so that -1 & 1 = 1 instead of 0.
        return (squareFreeSubsets(masks, 0, /*used=*/1, mem) - 1 + MOD) % MOD;
    }

    private static final int MOD = 1_000_000_007;
    private static final int PRIMES_COUNT = 10;
    private static final int[] primes = {2, 3, 5, 7, 11, 13, 17, 19, 23, 29};

    private int squareFreeSubsets(int[] masks, int i, int used, Integer[][] mem) {
        if (i == masks.length)
            return 1;
        if (mem[i][used] != null)
            return mem[i][used];
        final int pick =
            (masks[i] & used) == 0 ? squareFreeSubsets(masks, i + 1, used | masks[i], mem) : 0;
        final int skip = squareFreeSubsets(masks, i + 1, used, mem);
        return mem[i][used] = (pick + skip) % MOD;
    }

    // e.g. num = 10 = 2 * 5, so mask = 0b101 -> 0b1010 (append a 0)
    //      num = 15 = 3 * 5, so mask = 0b110 -> 0b1100 (append a 0)
    //      num = 25 = 5 * 5, so mask = 0b-1 -> 0b1..1 (invalid)
    private int getMask(int num) {
        int mask = 0;
        for (int i = 0; i < primes.length; ++i) {
            int rootCount = 0;
            while (num % primes[i] == 0) {
                num /= primes[i];
                ++rootCount;
            }
            if (rootCount >= 2)
                return -1;
            if (rootCount == 1)
                mask |= 1 << i;
        }
        return mask << 1;
    }
}
```

1630. 2574-2.java

Path: LeetCode-main/solutions/2574. Left and Right Sum Differences/2574-2.java

```
class Solution {
    public int[] leftRigthDifference(int[] nums) {
        int[] ans = new int[nums.length];
        int leftSum = 0;
        int rightSum = Arrays.stream(nums).sum();

        for (int i = 0; i < nums.length; ++i) {
            rightSum -= nums[i];
            ans[i] = Math.abs(leftSum - rightSum);
            leftSum += nums[i];
        }

        return ans;
    }
}
```


1631. 2574.java

Path: LeetCode-main/solutions/2574. Left and Right Sum Differences/2574.java

```
class Solution {
    public int[] leftRigthDifference(int[] nums) {
        final int n = nums.length;
        int[] ans = new int[n];
        int[] leftSum = new int[n];
        int[] rightSum = new int[n];
        int prefix = 0;
        int suffix = 0;

        for (int i = 0; i < n; ++i) {
            if (i > 0)
                prefix += nums[i - 1];
            leftSum[i] = prefix;
        }

        for (int i = n - 1; i >= 0; --i) {
            if (i + 1 < n)
                suffix += nums[i + 1];
            rightSum[i] = suffix;
        }

        for (int i = 0; i < n; ++i)
            ans[i] = Math.abs(leftSum[i] - rightSum[i]);

        return ans;
    }
}
```

1632. 2575.java

Path: LeetCode-main/solutions/2575. Find the Divisibility Array of a String/2575.java

```
class Solution {
    public int[] divisibilityArray(String word, int m) {
        int[] ans = new int[word.length()];
        long prevRemainder = 0;

        for (int i = 0; i < word.length(); ++i) {
            final long remainder = (prevRemainder * 10 + (word.charAt(i) - '0')) % m;
            ans[i] = remainder == 0 ? 1 : 0;
            prevRemainder = remainder;
        }

        return ans;
    }
}
```

1633. 2576-2.java

Path: LeetCode-main/solutions/2576. Find the Maximum Number of Marked Indices/2576-2.java

```
class Solution {
    public int maxNumOfMarkedIndices(int[] nums) {
        Arrays.sort(nums);

        int i = 0;
        for (int j = nums.length / 2; j < nums.length; ++j)
            if (2 * nums[i] <= nums[j] && ++i == nums.length / 2)
                break;

        return i * 2;
    }
}
```

1634. 2576.java

Path: LeetCode-main/solutions/2576. Find the Maximum Number of Marked Indices/2576.java

```
class Solution {
    public int maxNumOfMarkedIndices(int[] nums) {
        Arrays.sort(nums);

        int l = 0;
        int r = nums.length / 2 + 1;

        while (l < r) {
            final int m = (l + r) / 2;
            if (isPossible(nums, m))
                l = m + 1;
            else
                r = m;
        }

        return (l - 1) * 2;
    }

    private boolean isPossible(int[] nums, int m) {
        for (int i = 0; i < m; ++i)
            if (2 * nums[i] > nums[nums.length - m + i])
                return false;
        return true;
    }
}
```

1635. 2577.java

Path: LeetCode-main/solutions/2577. Minimum Time to Visit a Cell In a Grid/2577.java

```
class Solution {
    public int minimumTime(int[][] grid) {
        if (grid[0][1] > 1 && grid[1][0] > 1)
            return -1;

        final int[][] DIRS = {{0, 1}, {1, 0}, {0, -1}, {-1, 0}};
        final int m = grid.length;
        final int n = grid[0].length;
        Queue<int[]> minHeap = new PriorityQueue<>(Comparator.comparingInt(a -> a[0])) {
            { offer(new int[] {0, 0, 0}); } // (time, i, j)
        };
        boolean[][] seen = new boolean[m][n];
        seen[0][0] = true;

        while (!minHeap.isEmpty()) {
            final int time = minHeap.peek()[0];
            final int i = minHeap.peek()[1];
            final int j = minHeap.poll()[2];
            if (i == m - 1 && j == n - 1)
                return time;
            for (int[] dir : DIRS) {
                final int x = i + dir[0];
                final int y = j + dir[1];
                if (x < 0 || x == m || y < 0 || y == n)
                    continue;
                if (seen[x][y])
                    continue;
                final int extraWait = (grid[x][y] - time) % 2 == 0 ? 1 : 0;
                final int nextTime = Math.max(time + 1, grid[x][y] + extraWait);
                minHeap.offer(new int[] {nextTime, x, y});
                seen[x][y] = true;
            }
        }

        throw new IllegalArgumentException();
    }
}
```

1636. 2578.java

Path: LeetCode-main/solutions/2578. Split With Minimum Sum/2578.java

```
class Solution {
    public int splitNum(int num) {
        StringBuilder sb1 = new StringBuilder();
        StringBuilder sb2 = new StringBuilder();
        char[] chars = String.valueOf(num).toCharArray();

        Arrays.sort(chars);

        for (int i = 0; i < chars.length; ++i)
            if (i % 2 == 0)
                sb1.append(chars[i]);
            else
                sb2.append(chars[i]);

        return Integer.parseInt(sb1.toString()) + Integer.parseInt(sb2.toString());
    }
}
```

1637. 2579.java

Path: LeetCode-main/solutions/2579. Count Total Number of Colored Cells/2579.java

```
class Solution {  
    public long coloredCells(int n) {  
        return 1L * n * n + 1L * (n - 1) * (n - 1);  
    }  
}
```

1638. 258.java

Path: LeetCode-main/solutions/258. Add Digits/258.java

```
class Solution {  
    public int addDigits(int num) {  
        return 1 + (num - 1) % 9;  
    }  
}
```


1639. 2580.java

Path: LeetCode-main/solutions/2580. Count Ways to Group Overlapping Ranges/2580.java

```
class Solution {
    public int countWays(int[][] ranges) {
        final int MOD = 1_000_000_007;
        int ans = 1;
        int prevEnd = -1;

        Arrays.sort(ranges, Comparator.comparingInt(range -> range[0]));

        for (int[] range : ranges) {
            final int start = range[0];
            final int end = range[1];
            if (start > prevEnd)
                ans = ans * 2 % MOD;
            prevEnd = Math.max(prevEnd, end);
        }

        return ans;
    }
}
```

1640. 2581.java

Path: LeetCode-main/solutions/2581. Count Number of Possible Root Nodes/2581.java

```
class Solution {
    public int rootCount(int[][] edges, int[][] guesses, int k) {
        final int n = edges.length + 1;
        List<Integer>[] graph = new List[n];
        Set<Integer>[] guessGraph = new Set[n];
        int[] parent = new int[n];

        for (int i = 0; i < n; ++i) {
            graph[i] = new ArrayList<>();
            guessGraph[i] = new HashSet<>();
        }

        for (int[] edge : edges) {
            final int u = edge[0];
            final int v = edge[1];
            graph[u].add(v);
            graph[v].add(u);
        }

        for (int[] guess : guesses) {
            final int u = guess[0];
            final int v = guess[1];
            guessGraph[u].add(v);
        }

        // Precalculate `parent`.
        dfs(graph, 0, /*prev=*/-1, parent);

        // Calculate `correctGuess` for tree rooted at 0.
        int correctGuess = 0;
        for (int i = 1; i < n; ++i)
            if (guessGraph[parent[i]].contains(i))
                ++correctGuess;

        reroot(graph, 0, /*prev=*/-1, correctGuess, guessGraph, k);
        return ans;
    }

    private int ans = 0;

    private void dfs(List<Integer>[] graph, int u, int prev, int[] parent) {
        parent[u] = prev;
        for (final int v : graph[u])
            if (v != prev)
                dfs(graph, v, u, parent);
    }

    private void reroot(List<Integer>[] graph, int u, int prev, int correctGuess,
                       Set<Integer>[] guessGraph, int k) {
        if (u != 0) {
            // The tree is rooted at u, so a guess edge (u, prev) will match the new
            // `parent` relationship.
            if (guessGraph[u].contains(prev))
                ++correctGuess;
            // A guess edge (prev, u) matching the old `parent` relationship will no
            // longer be true.
            if (guessGraph[prev].contains(u))
                --correctGuess;
        }
        if (correctGuess >= k)
            ++ans;
        for (final int v : graph[u])
            if (v != prev)
                reroot(graph, v, u, correctGuess, guessGraph, k);
    }
}
```

1641. 2582.java

Path: LeetCode-main/solutions/2582. Pass the Pillow/2582.java

```
class Solution {  
    public int passThePillow(int n, int time) {  
        // Repeat every (n - 1) * 2 seconds.  
        time %= (n - 1) * 2;  
        if (time < n) // Go forward from 1.  
            return 1 + time;  
        return n - (time - (n - 1)); // Go backward from n.  
    }  
}
```

1642. 2583.java

Path: LeetCode-main/solutions/2583. Kth Largest Sum in a Binary Tree/2583.java

```
class Solution {
    public long kthLargestLevelSum(TreeNode root, int k) {
        List<Long> levelSums = new ArrayList<>();
        dfs(root, 0, levelSums);
        if (levelSums.size() < k)
            return -1;

        Collections.sort(levelSums, Collections.reverseOrder());
        return levelSums.get(k - 1);
    }

    private void dfs(TreeNode root, int level, List<Long> levelSums) {
        if (root == null)
            return;
        if (levelSums.size() == level)
            levelSums.add(0L);
        levelSums.set(level, levelSums.get(level) + root.val);
        dfs(root.left, level + 1, levelSums);
        dfs(root.right, level + 1, levelSums);
    }
}
```

1643. 2584.java

Path: LeetCode-main/solutions/2584. Split the Array to Make Coprime Products/2584.java

```
class Solution {
    public int findValidSplit(int[] nums) {
        Map<Integer, Integer> leftPrimeFactors = new HashMap<>();
        Map<Integer, Integer> rightPrimeFactors = new HashMap<>();

        for (final int num : nums)
            for (final int primeFactor : getPrimeFactors(num))
                rightPrimeFactors.merge(primeFactor, 1, Integer::sum);

        for (int i = 0; i < nums.length - 1; ++i) {
            for (final int primeFactor : getPrimeFactors(nums[i])) {
                rightPrimeFactors.merge(primeFactor, -1, Integer::sum);
                if (rightPrimeFactors.get(primeFactor) == 0) {
                    // rightPrimeFactors[primeFactor] == 0, so no need to track
                    // leftPrimeFactors[primeFactor].
                    rightPrimeFactors.remove(primeFactor);
                    leftPrimeFactors.remove(primeFactor);
                } else {
                    // Otherwise, need to track leftPrimeFactors[primeFactor].
                    leftPrimeFactors.merge(primeFactor, 1, Integer::sum);
                }
            }
            if (leftPrimeFactors.isEmpty())
                return i;
        }

        return -1;
    }

    // Gets the prime factors under sqrt(10^6).
    private List<Integer> getPrimeFactors(int num) {
        List<Integer> primeFactors = new ArrayList<>();
        for (int divisor = 2; divisor <= Math.min(1000, num); ++divisor)
            if (num % divisor == 0) {
                primeFactors.add(divisor);
                while (num % divisor == 0)
                    num /= divisor;
            }
        // Handle the case that `num` contains a prime factor > 1000.
        if (num > 1)
            primeFactors.add(num);
        return primeFactors;
    }
}
```

1644. 2585-2.java

Path: LeetCode-main/solutions/2585. Number of Ways to Earn Points/2585-2.java

```
class Solution {
    public int waysToReachTarget(int target, int[][] types) {
        final int MOD = 1_000_000_007;
        // dp[j] := the number of ways to earn j points with the types so far
        int[] dp = new int[target + 1];
        dp[0] = 1;

        for (int[] type : types) {
            final int count = type[0];
            final int mark = type[1];
            for (int j = target; j >= 0; --j)
                for (int solved = 1; solved <= count; ++solved)
                    if (j - solved * mark >= 0) {
                        dp[j] += dp[j - solved * mark];
                        dp[j] %= MOD;
                    }
        }

        return dp[target];
    }
}
```

1645. 2585.java

Path: LeetCode-main/solutions/2585. Number of Ways to Earn Points/2585.java

```
class Solution {
    public int waysToReachTarget(int target, int[][] types) {
        final int MOD = 1_000_000_007;
        // dp[i][j] := the number of ways to earn j points with the first i types
        int[][] dp = new int[types.length + 1][target + 1];
        dp[0][0] = 1;

        for (int i = 1; i <= types.length; ++i) {
            final int count = types[i - 1][0];
            final int mark = types[i - 1][1];
            for (int j = 0; j <= target; ++j)
                for (int solved = 0; solved <= count; ++solved)
                    if (j - solved * mark >= 0) {
                        dp[i][j] += dp[i - 1][j - solved * mark];
                        dp[i][j] %= MOD;
                    }
        }

        return dp[types.length][target];
    }
}
```

1646. 2586.java

Path: LeetCode-main/solutions/2586. Count the Number of Vowel Strings in Range/2586.java

```
class Solution {
    public int vowelStrings(String[] words, int left, int right) {
        return (int) Arrays.asList(words)
            .subList(left, right + 1)
            .stream()
            .filter(word -> isVowel(word.charAt(0)) && isVowel(word.charAt(word.length() - 1)))
            .count();
    }

    private boolean isVowel(char c) {
        return "aeiou".indexOf(c) != -1;
    }
}
```


1647. 2587.java

Path: LeetCode-main/solutions/2587. Rearrange Array to Maximize Prefix Score/2587.java

```
class Solution {
    public int maxScore(int[] nums) {
        long prefix = 0;

        nums = Arrays.stream(nums)
            .boxed()
            .sorted(Collections.reverseOrder())
            .mapToInt(Integer::intValue)
            .toArray();

        for (int i = 0; i < nums.length; ++i) {
            prefix += nums[i];
            if (prefix <= 0)
                return i;
        }

        return nums.length;
    }
}
```

1648. 2588.java

Path: LeetCode-main/solutions/2588. Count the Number of Beautiful Subarrays/2588.java

```
class Solution {
    public long beautifulSubarrays(int[] nums) {
        // A subarray is beautiful if xor(subarray) = 0.
        long ans = 0;
        int prefix = 0;
        Map<Integer, Integer> prefixCount = new HashMap<>();
        prefixCount.put(0, 1);

        for (final int num : nums) {
            prefix ^= num;
            ans += prefixCount.getOrDefault(prefix, 0);
            prefixCount.merge(prefix, 1, Integer::sum);
        }

        return ans;
    }
}
```

1649. 2589.java

Path: LeetCode-main/solutions/2589. Minimum Time to Complete All Tasks/2589.java

```
class Solution {
    public int findMinimumTime(int[][] tasks) {
        final int MAX = 2000;
        boolean[] running = new boolean[MAX + 1];

        // Sort tasks by end.
        Arrays.sort(tasks, Comparator.comparingInt(task -> task[1]));

        for (int[] task : tasks) {
            final int start = task[0];
            final int end = task[1];
            final int duration = task[2];
            int neededDuration = duration;
            for (int i = start; i <= end; ++i)
                if (running[i])
                    --neededDuration;
            // Greedily run the task as late as possible so that later tasks can run
            // simultaneously.
            for (int i = end; neededDuration > 0; --i) {
                if (!running[i]) {
                    running[i] = true;
                    --neededDuration;
                }
            }
        }

        return (int) IntStream.range(0, running.length)
            .mapToObj(i -> running[i])
            .filter(r -> r)
            .count();
    }
}
```

1650. 259.java

Path: LeetCode-main/solutions/259. 3Sum Smaller/259.java

```
class Solution {
    public int threeSumSmaller(int[] nums, int target) {
        if (nums.length < 3)
            return 0;

        int ans = 0;

        Arrays.sort(nums);

        for (int i = 0; i + 2 < nums.length; ++i) {
            int l = i + 1;
            int r = nums.length - 1;
            while (l < r)
                if (nums[i] + nums[l] + nums[r] < target) {
                    // (nums[i], nums[l], nums[r])
                    // (nums[i], nums[l], nums[r - 1])
                    // ...,
                    // (nums[i], nums[l], nums[l + 1])
                    ans += r - l;
                    ++l;
                } else {
                    --r;
                }
        }

        return ans;
    }
}
```

1651. 2590.java

Path: LeetCode-main/solutions/2590. Design a Todo List/2590.java

```
class TodoList {
    public int addTask(int userId, String taskDescription, int dueDate, List<String> tags) {
        userIdToTaskIdToTasks.putIfAbsent(userId, new HashMap<>());
        userIdToTaskIdToTasks.get(userId).put(++taskId, new Task(taskDescription, dueDate, tags));
        taskIds.add(taskId);
        return taskId;
    }

    public List<String> getAllTasks(int userId) {
        List<String> res = new ArrayList<>();
        for (Task task : getTasksSortedByDueDate(userId))
            res.add(task.taskDescription);
        return res;
    }

    public List<String> getTasksForTag(int userId, String tag) {
        List<String> res = new ArrayList<>();
        for (Task task : getTasksSortedByDueDate(userId))
            if (task.tags.contains(tag))
                res.add(task.taskDescription);
        return res;
    }

    public void completeTask(int userId, int taskId) {
        if (!taskIds.contains(taskId))
            return;
        if (!userIdToTaskIdToTasks.containsKey(userId))
            return;
        Map<Integer, Task> taskIdToTasks = userIdToTaskIdToTasks.get(userId);
        if (!taskIdToTasks.containsKey(taskId))
            return;
        taskIdToTasks.remove(taskId);
    }

    private record Task(String taskDescription, int dueDate, List<String> tags) {}
    private int taskId = 0;
    private Set<Integer> taskIds = new HashSet<>();
    // {userId: {taskId: tasks}}
    private Map<Integer, Map<Integer, Task>> userIdToTaskIdToTasks = new HashMap<>();

    private List<Task> getTasksSortedByDueDate(int userId) {
        if (!userIdToTaskIdToTasks.containsKey(userId))
            return new ArrayList<>();
        TreeSet<Task> tasks = new TreeSet<>(Comparator.comparingInt(Task::dueDate));
        for (Task task : userIdToTaskIdToTasks.get(userId).values())
            tasks.add(task);
        return new ArrayList<>(tasks);
    }
}
```

1652. 2591.java

Path: LeetCode-main/solutions/2591. Distribute Money to Maximum Children/2591.java

```
class Solution {
    public int distMoney(int money, int children) {
        // Everyone must receive at least 1 dollar.
        money -= children;
        if (money < 0)
            return -1;

        final int count7 = money / 7;
        final int remaining = money % 7;

        // Distribute 8 dollars to every child.
        if (count7 == children && remaining == 0)
            return count7;

        // Need to move 1 dollar from the last child with 4 dollars to one of other
        // children. That's why we need to subtract 1.
        if (count7 == children - 1 && remaining == 3)
            return count7 - 1;

        // Though there might be child with 4 dollars, since count7 < children - 1,
        // we have "extra" spot to move money to if needed.
        return Math.min(children - 1, count7);
    }
}
```

1653. 2592.java

Path: LeetCode-main/solutions/2592. Maximize Greatness of an Array/2592.java

```
class Solution {
    public int maximizeGreatness(int[] nums) {
        int ans = 0;

        Arrays.sort(nums);

        for (final int num : nums)
            if (num > nums[ans])
                ++ans;

        return ans;
    }
}
```

1654. 2593.java

Path: LeetCode-main/solutions/2593. Find Score of an Array After Marking All Elements/2593.java

```
class Solution {
    public long findScore(int[] nums) {
        long ans = 0;
        TreeSet<Pair<Integer, Integer>> numAndIndexes =
            new TreeSet<>(Comparator.comparingInt(Pair<Integer, Integer>::getKey)
                .thenComparingInt(Pair<Integer, Integer>::getValue));
        boolean[] seen = new boolean[nums.length];

        for (int i = 0; i < nums.length; ++i)
            numAndIndexes.add(new Pair<>(nums[i], i));

        for (Pair<Integer, Integer> pair : numAndIndexes) {
            final int num = pair.getKey();
            final int i = pair.getValue();
            if (seen[i])
                continue;
            if (i > 0)
                seen[i - 1] = true;
            if (i + 1 < nums.length)
                seen[i + 1] = true;
            seen[i] = true;
            ans += num;
        }

        return ans;
    }
}
```


1655. 2594.java

Path: LeetCode-main/solutions/2594. Minimum Time to Repair Cars/2594.java

```
class Solution {
    public long repairCars(int[] ranks, int cars) {
        long l = 0;
        long r = (long) Arrays.stream(ranks).min().getAsInt() * cars * cars;

        while (l < r) {
            final long m = (l + r) / 2;
            if (numCarsFixed(ranks, m) >= cars)
                r = m;
            else
                l = m + 1;
        }

        return l;
    }

    private long numCarsFixed(int[] ranks, long minutes) {
        long carsFixed = 0;
        // r * n^2 = minutes
        // -> n = sqrt(minutes / r)
        for (final int rank : ranks)
            carsFixed += Math.sqrt(minutes / rank);
        return carsFixed;
    }
}
```

1656. 2595.java

Path: LeetCode-main/solutions/2595. Number of Even and Odd Bits/2595.java

```
class Solution {
    public int[] evenOddBit(int n) {
        int[] ans = new int[2];
        int i = 0; // 0 := even, 1 := odd

        while (n > 0) {
            ans[i] += n & 1;
            n >>= 1;
            i ^= 1;
        }

        return ans;
    }
}
```

1657. 2596.java

Path: LeetCode-main/solutions/2596. Check Knight Tour Configuration/2596.java

```
class Solution {
    public boolean checkValidGrid(int[][] grid) {
        if (grid[0][0] != 0)
            return false;

        final int n = grid.length;
        int i = 0;
        int j = 0;

        for (int target = 1; target < n * n; ++target) {
            Pair<Integer, Integer> pair = nextGrid(grid, i, j, target);
            final int x = pair.getKey();
            final int y = pair.getValue();
            if (x == -1 && y == -1)
                return false;
            // Move (x, y) to (i, j).
            i = x;
            j = y;
        }

        return true;
    }

    private static final int[][] DIRS = {{-2, 1}, {-1, 2}, {1, 2}, {2, 1},
                                           {2, -1}, {1, -2}, {-1, -2}, {-2, -1}};

    // Returns (x, y), where grid[x][y] == target if (i, j) can reach target.
    private Pair<Integer, Integer> nextGrid(int[][] grid, int i, int j, int target) {
        final int n = grid.length;
        for (int[] dir : DIRS) {
            final int x = i + dir[0];
            final int y = j + dir[1];
            if (x < 0 || x >= n || y < 0 || y >= n)
                continue;
            if (grid[x][y] == target)
                return new Pair<>(x, y);
        }
        return new Pair<>(-1, -1);
    }
}
```

1658. 2597.java

Path: LeetCode-main/solutions/2597. The Number of Beautiful Subsets/2597.java

```
// e.g. nums = [2, 3, 4, 4], k = 2
//
// subset[0] = [2, 4, 4']
// subset[1] = [1]
// count = {2: 1, 4: 2, 1: 1}
//
// Initially, skip = len([]) = 0, pick = len([]) = 0
//
// * For values in subset[0]:
//   After 2:
//     skip = skip + pick = len([]) = 0
//     pick = (2^count[2] - 1) * (1 + skip + pick)
//           = len([[2]]) * len([])
//           = len([[2]]) = 1
//   After 4:
//     skip = skip + pick = len([[2]]) = 1
//     pick = (2^count[4] - 1) * (1 + skip + pick)
//           = len([[4], [4']], [4, 4']]) * len([])
//           = len([[4], [4']], [4, 4']]) = 3
//
// * For values in subset[1]:
//   After 1:
//     skip = skip + pick
//           = len([[2], [4], [4']], [4, 4']]) = 4
//     pick = (2^count[1] - 1) * (1 + skip + pick)
//           = len([[1]]) * len([[2], [4], [4']], [4, 4']])
//           = len([[1], [1, 2], [1, 4], [1, 4']], [1, 4, 4']]) = 5
//
// So, ans = skip + pick = 9

class Solution {
    public int beautifulSubsets(int[] nums, int k) {
        final int MAX = 1000;
        int[] count = new int[MAX + 1];
        Map<Integer, Set<Integer>> modToSubset = new HashMap<>();

        for (final int num : nums) {
            ++count[num];
            modToSubset.putIfAbsent(num % k, new TreeSet<>());
            modToSubset.get(num % k).add(num);
        }

        int prevNum = -k;
        int skip = 0;
        int pick = 0;

        for (Set<Integer> subset : modToSubset.values()) {
            for (final int num : subset) {
                final int nonEmptyCount = (int) Math.pow(2, count[num]) - 1;
                final int cacheSkip = skip;
                skip += pick;
                pick = nonEmptyCount * (1 + cacheSkip + (num - prevNum == k ? 0 : pick));
                prevNum = num;
            }
            return skip + pick;
        }
    }
}
```

1659. 2598.java

Path: LeetCode-main/solutions/2598. Smallest Missing Non-negative Integer After Operations/2598.java

```
class Solution {
    public int findSmallestInteger(int[] nums, int value) {
        Map<Integer, Integer> count = new HashMap<>();

        for (final int num : nums)
            count.merge((num % value + value) % value, 1, Integer::sum);

        for (int i = 0; i < nums.length; ++i) {
            if (count.getOrDefault(i % value, 0) == 0)
                return i;
            count.merge(i % value, -1, Integer::sum);
        }

        return nums.length;
    }
}
```

1660. 2599.java

Path: LeetCode-main/solutions/2599. Make the Prefix Sum Non-negative/2599.java

```
class Solution {
    public int makePrefSumNonNegative(int[] nums) {
        int ans = 0;
        long prefix = 0;
        Queue<Integer> minHeap = new PriorityQueue<>();

        for (final int num : nums) {
            prefix += num;
            if (num < 0)
                minHeap.offer(num);
            while (prefix < 0) {
                prefix -= minHeap.poll();
                ++ans;
            }
        }

        return ans;
    }
}
```

1661. 26.java

Path: LeetCode-main/solutions/26. Remove Duplicates from Sorted Array/26.java

```
class Solution {  
    public int removeDuplicates(int[] nums) {  
        int i = 0;  
  
        for (final int num : nums)  
            if (i < 1 || num > nums[i - 1])  
                nums[i++] = num;  
  
        return i;  
    }  
}
```

1662. 260.java

Path: LeetCode-main/solutions/260. Single Number III/260.java

```
class Solution {
    public int[] singleNumber(int[] nums) {
        final int xors = Arrays.stream(nums).reduce((a, b) -> a ^ b).getAsInt();
        final int lowbit = xors & -xors;
        int[] ans = new int[2];

        // Seperate `nums` into two groups by `lowbit`.
        for (final int num : nums)
            if ((num & lowbit) > 0)
                ans[0] ^= num;
            else
                ans[1] ^= num;

        return ans;
    }
}
```


1663. 2600.java

Path: LeetCode-main/solutions/2600. K Items With the Maximum Sum/2600.java

```
class Solution {
    public int kItemsWithMaximumSum(int numOnes, int numZeros, int numNegOnes, int k) {
        if (k <= numOnes)
            return k;
        if (k <= numOnes + numZeros)
            return numOnes;
        return numOnes - (k - numOnes - numZeros);
    }
}
```

1664. 2601.java

Path: LeetCode-main/solutions/2601. Prime Subtraction Operation/2601.java

```
class Solution {
    public boolean primeSubOperation(int[] nums) {
        final int MAX = 1000;
        final List<Integer> primes = sieveEratosthenes(MAX);

        int prevNum = 0;
        for (int num : nums) {
            // Make nums[i] the smallest as possible and still > nums[i - 1].
            final int i = firstGreaterEqual(primes, num - prevNum);
            if (i > 0)
                num -= primes.get(i - 1);
            if (num <= prevNum)
                return false;
            prevNum = num;
        }

        return true;
    }

    private List<Integer> sieveEratosthenes(int n) {
        List<Integer> primes = new ArrayList<>();
        boolean[] isPrime = new boolean[n];
        Arrays.fill(isPrime, true);
        isPrime[0] = false;
        isPrime[1] = false;
        for (int i = 2; i * i < n; ++i)
            if (isPrime[i])
                for (int j = i * i; j < n; j += i)
                    isPrime[j] = false;
        for (int i = 2; i < n; ++i)
            if (isPrime[i])
                primes.add(i);
        return primes;
    }

    private int firstGreaterEqual(List<Integer> A, int target) {
        final int i = Collections.binarySearch(A, target);
        return i < 0 ? -i - 1 : i;
    }
}
```

1665. 2602.java

Path: LeetCode-main/solutions/2602. Minimum Operations to Make All Array Elements Equal/2602.java

```
class Solution {
    public List<Long> minOperations(int[] nums, int[] queries) {
        final int n = nums.length;
        List<Long> ans = new ArrayList<>();
        long[] prefix = new long[n + 1];

        Arrays.sort(nums);

        for (int i = 0; i < n; ++i)
            prefix[i + 1] = prefix[i] + nums[i];

        for (final int query : queries) {
            int i = Arrays.binarySearch(nums, query);
            if (i < 0)
                i = -i - 1;

            // Since nums[0..i) <= query, nums[i..n) > `query`, we should
            // - increase each num in nums[0..i) to `query` and
            // - decrease each num in nums[i..n) to `query`.
            final long inc = (long) query * i - prefix[i];
            final long dec = prefix[n] - prefix[i] - (long) query * (n - i);
            ans.add(inc + dec);
        }

        return ans;
    }
}
```

1666. 2603.java

Path: LeetCode-main/solutions/2603. Collect Coins in a Tree/2603.java

```
class Solution {
    public int collectTheCoins(int[] coins, int[][] edges) {
        final int n = coins.length;
        Set<Integer>[] tree = new Set[n];
        Deque<Integer> leavesToBeRemoved = new ArrayDeque<>();

        for (int i = 0; i < n; ++i)
            tree[i] = new HashSet<>();

        for (int[] edge : edges) {
            final int u = edge[0];
            final int v = edge[1];
            tree[u].add(v);
            tree[v].add(u);
        }

        for (int i = 0; i < n; ++i) {
            int u = i;
            // Remove the leaves that don't have coins.
            while (tree[u].size() == 1 && coins[u] == 0) {
                final int v = tree[u].iterator().next();
                tree[u].clear();
                tree[v].remove(u);
                u = v; // Walk up to its parent.
            }
            // After trimming leaves without coins, leaves with coins may satisfy
            // `leavesToBeRemoved`.
            if (tree[u].size() == 1)
                leavesToBeRemoved.offer(u);
        }

        // Remove each remaining leaf node and its parent. The remaining nodes are
        // the ones that must be visited.
        for (int i = 0; i < 2; ++i)
            for (int sz = leavesToBeRemoved.size(); sz > 0; --sz) {
                final int u = leavesToBeRemoved.poll();
                if (!tree[u].isEmpty()) {
                    final int v = tree[u].iterator().next();
                    tree[u].clear();
                    tree[v].remove(u);
                    if (tree[v].size() == 1)
                        leavesToBeRemoved.offer(v);
                }
            }

        return Arrays.stream(tree).mapToInt(children -> children.size()).sum();
    }
}
```

1667. 2604.java

Path: LeetCode-main/solutions/2604. Minimum Time to Eat All Grains/2604.java

```
class Solution {
    public int minimumTime(int[] hens, int[] grains) {
        Arrays.sort(hens);
        Arrays.sort(grains);

        final int maxPosition = Math.max(Arrays.stream(hens).max().getAsInt(), //
                                           Arrays.stream(grains).max().getAsInt());
        final int minPosition = Math.min(Arrays.stream(hens).min().getAsInt(), //
                                           Arrays.stream(grains).min().getAsInt());

        int l = 0;
        int r = (int) (1.5 * (maxPosition - minPosition));

        while (l < r) {
            final int m = (int) ((l + (long) r) / 2);
            if (canEat(hens, grains, m))
                r = m;
            else
                l = m + 1;
        }

        return (int) l;
    }

    // Returns true if `hens` can eat all `grains` within `time`.
    private boolean canEat(int[] hens, int[] grains, int time) {
        int i = 0; // grains[i] := next grain to be ate
        for (final int hen : hens) {
            int rightMoves = time;
            if (grains[i] < hen) {
                // `hen` needs go back to eat `grains[i]`.
                final int leftMoves = hen - grains[i];
                if (leftMoves > time)
                    return false;
                final int leftThenRight = time - 2 * leftMoves;
                final int rightThenLeft = (time - leftMoves) / 2;
                rightMoves = Math.max(0, Math.max(leftThenRight, rightThenLeft));
            }
            i = firstGreater(grains, hen + rightMoves);
            if (i == grains.length)
                return true;
        }
        return false;
    }

    private int firstGreater(int[] arr, int target) {
        final int i = Arrays.binarySearch(arr, target + 1);
        return i < 0 ? -i - 1 : i;
    }
}
```

1668. 2605.java

Path: LeetCode-main/solutions/2605. Form Smallest Number From Two Digit Arrays/2605.java

```
class Solution {
    public int minNumber(int[] nums1, int[] nums2) {
        int ans = 89; // the largest num we can have
        for (final int a : nums1)
            for (final int b : nums2)
                ans = Math.min(ans, a == b ? a : Math.min(a, b) * 10 + Math.max(a, b));
        return ans;
    }
}
```

1669. 2606.java

Path: LeetCode-main/solutions/2606. Find the Substring With Maximum Cost/2606.java

```
class Solution {
    public int maximumCostSubstring(String s, String chars, int[] vals) {
        int ans = 0;
        int cost = 0;
        int[] costs = new int[26]; // costs[i] := the cost of 'a' + i

        for (int i = 0; i < 26; ++i)
            costs[i] = i + 1;

        for (int i = 0; i < chars.length(); ++i)
            costs[chars.charAt(i) - 'a'] = vals[i];

        for (final char c : s.toCharArray()) {
            cost = Math.max(0, cost + costs[c - 'a']);
            ans = Math.max(ans, cost);
        }

        return ans;
    }
}
```

1670. 2607.java

Path: LeetCode-main/solutions/2607. Make K-Subarray Sums Equal/2607.java

```
class Solution {
    public long makeSubKSumEqual(int[] arr, int k) {
        // If the sum of each subarray of length k is equal, then `arr` must have a
        // repeated pattern of size k. e.g. arr = [1, 2, 3, ...] and k = 3, to have
        // sum([1, 2, 3]) == sum([2, 3, x]), x must be 1. Therefore, arr[i] ==
        // arr[(i + k) % n] for every i.
        final int n = arr.length;
        long ans = 0;
        boolean[] seen = new boolean[n];

        for (int i = 0; i < n; ++i) {
            List<Integer> groups = new ArrayList<>();
            int j = i;
            while (!seen[j]) {
                groups.add(arr[j]);
                seen[j] = true;
                j = (j + k) % n;
            }
            Collections.sort(groups);
            for (final int num : groups)
                ans += Math.abs(num - groups.get(groups.size() / 2));
        }

        return ans;
    }
}
```


1671. 2608.java

Path: LeetCode-main/solutions/2608. Shortest Cycle in a Graph/2608.java

```
class Solution {
    public int findShortestCycle(int n, int[][] edges) {
        int ans = INF;
        List<Integer>[] graph = new List[n];
        Arrays.setAll(graph, i -> new ArrayList<>());

        for (int[] edge : edges) {
            final int u = edge[0];
            final int v = edge[1];
            graph[u].add(v);
            graph[v].add(u);
        }

        for (int i = 0; i < n; ++i)
            ans = Math.min(ans, bfs(graph, i));

        return ans == INF ? -1 : ans;
    }

    private static final int INF = 1001;

    // Returns the length of the minimum cycle by starting BFS from node `i`.
    // Returns `INF` if there's no cycle.
    private int bfs(List<Integer>[] graph, int i) {
        int[] dist = new int[graph.length];
        Arrays.fill(dist, INF);
        Queue<Integer> q = new ArrayDeque<>(List.of(i));
        dist[i] = 0;
        while (!q.isEmpty()) {
            final int u = q.poll();
            for (final int v : graph[u]) {
                if (dist[v] == INF) {
                    dist[v] = dist[u] + 1;
                    q.offer(v);
                } else if (dist[v] + 1 != dist[u]) { // v is not a parent u.
                    return dist[v] + dist[u] + 1;
                }
            }
        }
        return INF;
    }
}
```

1672. 2609-2.java

Path: LeetCode-main/solutions/2609. Find the Longest Balanced Substring of a Binary String/2609-2.java

```
class Solution {
    public int findTheLongestBalancedSubstring(String s) {
        int ans = 0;
        int zeros = 0;
        int ones = 0;

        for (final char c : s.toCharArray()) {
            if (c == '0') {
                zeros = ones > 0 ? 1 : zeros + 1;
                ones = 0;
            } else { // c == '1'
                ++ones;
            }
            if (zeros >= ones)
                ans = Math.max(ans, ones);
        }

        return ans * 2;
    }
}
```

1673. 2609.java

Path: LeetCode-main/solutions/2609. Find the Longest Balanced Substring of a Binary String/2609.java

```
class Solution {
    public int findTheLongestBalancedSubstring(String s) {
        int ans = 0;

        for (int i = 0; i < s.length(); i) {
            int zeros = 0;
            int ones = 0;
            while (i < s.length() && s.charAt(i) == '0') {
                ++zeros;
                ++i;
            }
            while (i < s.length() && s.charAt(i) == '1') {
                ++ones;
                ++i;
            }
            ans = Math.max(ans, Math.min(zeros, ones));
        }
        return ans * 2;
    }
}
```

1674. 261-2.java

Path: LeetCode-main/solutions/261. Graph Valid Tree/261-2.java

```
class UnionFind {
    public UnionFind(int n) {
        count = n;
        id = new int[n];
        rank = new int[n];
        for (int i = 0; i < n; ++i)
            id[i] = i;
    }

    public void unionByRank(int u, int v) {
        final int i = find(u);
        final int j = find(v);
        if (i == j)
            return;
        if (rank[i] < rank[j]) {
            id[i] = j;
        } else if (rank[i] > rank[j]) {
            id[j] = i;
        } else {
            id[i] = j;
            ++rank[j];
        }
        --count;
    }

    public int getCount() {
        return count;
    }

    private int count;
    private int[] id;
    private int[] rank;

    private int find(int u) {
        return id[u] == u ? u : (id[u] = find(id[u]));
    }
}

class Solution {
    public boolean validTree(int n, int[][] edges) {
        if (n == 0 || edges.length != n - 1)
            return false;

        UnionFind uf = new UnionFind(n);

        for (int[] edge : edges) {
            final int u = edge[0];
            final int v = edge[1];
            uf.unionByRank(u, v);
        }

        return uf.getCount() == 1;
    }
}
```

1675. 261.java

Path: LeetCode-main/solutions/261. Graph Valid Tree/261.java

```
class Solution {
    public boolean validTree(int n, int[][] edges) {
        if (n == 0 || edges.length != n - 1)
            return false;

        List<Integer>[] graph = new List[n];
        Queue<Integer> q = new ArrayDeque<>(List.of(0));
        Set<Integer> seen = new HashSet<>(Arrays.asList(0));
        Arrays.setAll(graph, i -> new ArrayList<>());

        for (int[] edge : edges) {
            final int u = edge[0];
            final int v = edge[1];
            graph[u].add(v);
            graph[v].add(u);
        }

        while (!q.isEmpty()) {
            final int u = q.poll();
            for (final int v : graph[u])
                if (!seen.contains(v)) {
                    q.offer(v);
                    seen.add(v);
                }
        }

        return seen.size() == n;
    }
}
```

1676. 2610.java

Path: LeetCode-main/solutions/2610. Convert an Array Into a 2D Array With Conditions/2610.java

```
class Solution {
    public List<List<Integer>> findMatrix(int[] nums) {
        // The number of rows we need equals the maximum frequency.
        List<List<Integer>> ans = new ArrayList<>();
        int[] count = new int[nums.length + 1];

        for (final int num : nums) {
            // Construct `ans` on demand.
            if (++count[num] > ans.size())
                ans.add(new ArrayList<>());
            ans.get(count[num] - 1).add(num);
        }

        return ans;
    }
}
```

1677. 2611.java

Path: LeetCode-main/solutions/2611. Mice and Cheese/2611.java

```
class Solution {
    public int miceAndCheese(int[] reward1, int[] reward2, int k) {
        Queue<Integer> minHeap = new PriorityQueue<>();

        for (int i = 0; i < reward1.length; ++i) {
            minHeap.offer(reward1[i] - reward2[i]);
            if (minHeap.size() == k + 1)
                minHeap.poll();
        }

        return Arrays.stream(reward2).sum() + minHeap.stream().mapToInt(Integer::intValue).sum();
    }
}
```

1678. 2612.java

Path: LeetCode-main/solutions/2612. Minimum Reverse Operations/2612.java

```
class Solution {
    public int[] minReverseOperations(int n, int p, int[] banned, int k) {
        Set<Integer> bannedSet = Arrays.stream(banned).boxed().collect(Collectors.toSet());
        int[] ans = new int[n];
        Arrays.fill(ans, -1);
        // unseen[i] := the unseen numbers that % 2 == i
        TreeSet<Integer>[] unseen = new TreeSet[2];
        unseen[0] = new TreeSet<>();
        unseen[1] = new TreeSet<>();

        for (int num = 0; num < n; ++num)
            if (num != p && !bannedSet.contains(num))
                unseen[num % 2].add(num);

        // Perform BFS from `p`.
        Queue<Integer> q = new ArrayDeque<>(List.of(p));
        ans[p] = 0;

        while (!q.isEmpty()) {
            final int u = q.poll();
            final int lo = Math.max(u - k + 1, k - 1 - u);
            final int hi = Math.min(u + k - 1, n - 1 - (u - (n - k)));
            // Choose the correct set of numbers.
            TreeSet<Integer> nums = unseen[lo % 2];
            for (Integer num = nums.ceiling(lo); num != null && num <= hi;) {
                ans[num] = ans[u] + 1;
                q.offer(num);
                nums.remove(num);
                num = nums.higher(num);
            }
        }

        return ans;
    }
}
```


1679. 2614.java

Path: LeetCode-main/solutions/2614. Prime In Diagonal/2614.java

```
class Solution {
    public int diagonalPrime(int[][] nums) {
        int ans = 0;
        for (int i = 0; i < nums.length; ++i) {
            final int a = nums[i][i];
            final int b = nums[i][nums.length - i - 1];
            if (isPrime(a))
                ans = Math.max(ans, a);
            if (isPrime(b))
                ans = Math.max(ans, b);
        }
        return ans;
    }

    private boolean isPrime(int n) {
        if (n <= 1)
            return false;
        for (int i = 2; i * i <= n; ++i)
            if (n % i == 0)
                return false;
        return true;
    }
}
```

1680. 2615.java

Path: LeetCode-main/solutions/2615. Sum of Distances/2615.java

```
class Solution {
    public long[] distance(int[] nums) {
        long[] ans = new long[nums.length];
        Map<Integer, List<Integer>> numToIndices = new HashMap<>();

        for (int i = 0; i < nums.length; ++i) {
            numToIndices.putIfAbsent(nums[i], new ArrayList<>());
            numToIndices.get(nums[i]).add(i);
        }

        for (List<Integer> indices : numToIndices.values()) {
            final int n = indices.size();
            if (n == 1) {
                continue;
            }
            long sumSoFar = indices.stream().mapToLong(Integer::longValue).sum();
            int prevIndex = 0;
            for (int i = 0; i < n; ++i) {
                sumSoFar += (i - 1) * (indices.get(i) - prevIndex);
                sumSoFar -= (n - 1 - i) * (indices.get(i) - prevIndex);
                ans[indices.get(i)] = sumSoFar;
                prevIndex = indices.get(i);
            }
        }

        return ans;
    }
}
```

1681. 2616.java

Path: LeetCode-main/solutions/2616. Minimize the Maximum Difference of Pairs/2616.java

```
class Solution {
    public int minimizeMax(int[] nums, int p) {
        Arrays.sort(nums);

        int l = 0;
        int r = nums[nums.length - 1] - nums[0];

        while (l < r) {
            final int m = (l + r) / 2;
            if (numPairs(nums, m) >= p)
                r = m;
            else
                l = m + 1;
        }

        return l;
    }

    // Returns the number of pairs that can be obtained if the difference between each
    // pair <= `maxDiff`.
    private int numPairs(int[] nums, int maxDiff) {
        int pairs = 0;
        for (int i = 1; i < nums.length; ++i)
            // Greedily pair nums[i] with nums[i - 1].
            if (nums[i] - nums[i - 1] <= maxDiff) {
                ++pairs;
                ++i;
            }
        return pairs;
    }
}
```

1682. 263.java

Path: LeetCode-main/solutions/263. Ugly Number/263.java

```
class Solution {
    public boolean isUgly(int n) {
        if (n == 0)
            return false;

        for (final int prime : new int[] {2, 3, 5})
            while (n % prime == 0)
                n /= prime;

        return n == 1;
    }
}
```

1683. 2638.java

Path: LeetCode-main/solutions/2638. Count the Number of K-Free Subsets/2638.java

```
class Solution {
    public long countTheNumOfKFreeSubsets(int[] nums, int k) {
        Map<Integer, Set<Integer>> modToSubset = new HashMap<>();

        for (final int num : nums) {
            modToSubset.putIfAbsent(num % k, new TreeSet<>());
            modToSubset.get(num % k).add(num);
        }

        int prevNum = -k;
        long skip = 0;
        long pick = 0;

        for (Set<Integer> subset : modToSubset.values()) {
            for (final int num : subset) {
                final long cacheSkip = skip;
                skip += pick;
                pick = 1 + cacheSkip + (num - prevNum == k ? 0 : pick);
                prevNum = num;
            }
        }

        return 1 + skip + pick;
    }
}
```

1684. 2639.java

Path: LeetCode-main/solutions/2639. Find the Width of Columns of a Grid/2639.java

```
class Solution {
    public int[] findColumnWidth(int[][] grid) {
        int[] ans = new int[grid[0].length];

        for (int j = 0; j < grid[0].length; ++j) {
            ans[j] = 0;
            for (int i = 0; i < grid.length; ++i)
                ans[j] = Math.max(ans[j], String.valueOf(grid[i][j]).length());
        }

        return ans;
    }
}
```

1685. 264.java

Path: LeetCode-main/solutions/264. Ugly Number II/264.java

```
class Solution {
    public int nthUglyNumber(int n) {
        List<Integer> uglyNums = new ArrayList<>();
        uglyNums.add(1);
        int i2 = 0;
        int i3 = 0;
        int i5 = 0;

        while (uglyNums.size() < n) {
            final int next2 = uglyNums.get(i2) * 2;
            final int next3 = uglyNums.get(i3) * 3;
            final int next5 = uglyNums.get(i5) * 5;
            final int next = Math.min(next2, Math.min(next3, next5));
            if (next == next2)
                ++i2;
            if (next == next3)
                ++i3;
            if (next == next5)
                ++i5;
            uglyNums.add(next);
        }

        return uglyNums.get(uglyNums.size() - 1);
    }
}
```

1686. 2640.java

Path: LeetCode-main/solutions/2640. Find the Score of All Prefixes of an Array/2640.java

```
class Solution {
    public long[] findPrefixScore(int[] nums) {
        long[] ans = new long[nums.length];
        long prefix = 0;
        int mx = 0;

        for (int i = 0; i < nums.length; i++) {
            mx = Math.max(mx, nums[i]);
            prefix += nums[i] + mx;
            ans[i] = prefix;
        }

        return ans;
    }
}
```


1687. 2641.java

Path: LeetCode-main/solutions/2641. Cousins in Binary Tree II/2641.java

```
class Solution {
    public TreeNode replaceValueInTree(TreeNode root) {
        List<Integer> levelSums = new ArrayList<>();
        dfs(root, 0, levelSums);
        return replace(root, 0, new TreeNode(0), levelSums);
    }

    private void dfs(TreeNode root, int level, List<Integer> levelSums) {
        if (root == null)
            return;
        if (levelSums.size() == level)
            levelSums.add(0);
        levelSums.set(level, levelSums.get(level) + root.val);
        dfs(root.left, level + 1, levelSums);
        dfs(root.right, level + 1, levelSums);
    }

    private TreeNode replace(TreeNode root, int level, TreeNode curr, List<Integer> levelSums) {
        final int nextLevel = level + 1;
        final int nextLevelCousinsSum = nextLevel >= levelSums.size()
            ? 0
            : levelSums.get(nextLevel) -
                (root.left == null ? 0 : root.left.val) -
                (root.right == null ? 0 : root.right.val);

        if (root.left != null) {
            curr.left = new TreeNode(nextLevelCousinsSum);
            replace(root.left, level + 1, curr.left, levelSums);
        }
        if (root.right != null) {
            curr.right = new TreeNode(nextLevelCousinsSum);
            replace(root.right, level + 1, curr.right, levelSums);
        }
        return curr;
    }
}
```

1688. 2642.java

Path: LeetCode-main/solutions/2642. Design Graph With Shortest Path Calculator/2642.java

```
class Graph {
    public Graph(int n, int[][] edges) {
        graph = new List[n];
        Arrays.setAll(graph, i -> new ArrayList<>());
        for (int[] edge : edges)
            addEdge(edge);
    }

    public void addEdge(int[] edge) {
        final int u = edge[0];
        final int v = edge[1];
        final int w = edge[2];
        graph[u].add(new Pair<>(v, w));
    }

    public int shortestPath(int node1, int node2) {
        int[] dist = new int[graph.length];
        Arrays.fill(dist, Integer.MAX_VALUE);
        // (d, u)
        Queue<Pair<Integer, Integer>> minHeap =
            new PriorityQueue<>(Comparator.comparingInt(Pair::getKey));

        dist[node1] = 0;
        minHeap.offer(new Pair<>(dist[node1], node1));

        while (!minHeap.isEmpty()) {
            final int d = minHeap.peek().getKey();
            final int u = minHeap.poll().getValue();
            if (u == node2)
                return d;
            for (Pair<Integer, Integer> pair : graph[u]) {
                final int v = pair.getKey();
                final int w = pair.getValue();
                if (d + w < dist[v]) {
                    dist[v] = d + w;
                    minHeap.offer(new Pair<>(dist[v], v));
                }
            }
        }

        return -1;
    }

    private List<Pair<Integer, Integer>>[] graph;
}
```

1689. 2643.java

Path: LeetCode-main/solutions/2643. Row With Maximum Ones/2643.java

```
class Solution {
    public int[] rowAndMaximumOnes(int[][] mat) {
        int[] ans = new int[2];

        for (int i = 0; i < mat.length; ++i) {
            final int ones = (int) Arrays.stream(mat[i]).filter(a -> a == 1).count();
            if (ones > ans[1]) {
                ans[0] = i;
                ans[1] = ones;
            }
        }

        return ans;
    }
}
```

1690. 2644.java

Path: LeetCode-main/solutions/2644. Find the Maximum Divisibility Score/2644.java

```
class Solution {
    public int maxDivScore(int[] nums, int[] divisors) {
        int ans = -1;
        int maxScore = -1;

        for (final int divisor : divisors) {
            final int score = (int) Arrays.stream(nums).filter(num -> num % divisor == 0).count();
            if (score > maxScore) {
                ans = divisor;
                maxScore = score;
            } else if (score == maxScore) {
                ans = Math.min(ans, divisor);
            }
        }

        return ans;
    }
}
```

1691. 2645.java

Path: LeetCode-main/solutions/2645. Minimum Additions to Make Valid String/2645.java

```
class Solution {
    public int addMinimum(String word) {
        final char[] letters = new char[] {'a', 'b', 'c'};
        int ans = 0;
        int i = 0;

        while (i < word.length())
            for (final char c : letters) {
                if (i < word.length() && word.charAt(i) == c)
                    ++i;
                else
                    ++ans;
            }
        return ans;
    }
};
```

1692. 2646.java

Path: LeetCode-main/solutions/2646. Minimize the Total Price of the Trips/2646.java

```
class Solution {
    public int minimumTotalPrice(int n, int[][] edges, int[] price, int[][] trips) {
        List<Integer>[] graph = new List[n];

        for (int i = 0; i < n; i++)
            graph[i] = new ArrayList<>();

        for (int[] edge : edges) {
            final int u = edge[0];
            final int v = edge[1];
            graph[u].add(v);
            graph[v].add(u);
        }

        // count[i] := the number of times node i is traversed
        int[] count = new int[n];
        for (int[] trip : trips) {
            final int start = trip[0];
            final int end = trip[1];
            dfsCount(graph, start, -1, end, count, /*path=*/new ArrayList<>());
        }

        Integer[][] mem = new Integer[n][2];
        return dfs(graph, 0, -1, price, count, false, mem);
    }

    private void dfsCount(List<Integer>[] graph, int u, int prev, int end, int[] count,
                        List<Integer> path) {
        path.add(u);
        if (u == end) {
            for (final int i : path)
                ++count[i];
            return;
        }
        for (final int v : graph[u])
            if (v != prev)
                dfsCount(graph, v, u, end, count, path);
        path.remove(path.size() - 1);
    }

    // Returns the minimum price sum for the i-th node, where its parent is
    // halved parent or not halved not.
    private int dfs(List<Integer>[] graph, int u, int prev, int[] price, int[] count,
                    boolean parentHalved, Integer[][] mem) {
        if (mem[u][parentHalved ? 1 : 0] != null)
            return mem[u][parentHalved ? 1 : 0];

        int sumWithFullNode = price[u] * count[u];
        for (final int v : graph[u])
            if (v != prev)
                sumWithFullNode += dfs(graph, v, u, price, count, false, mem);

        if (parentHalved) // Can't halve this node if its parent was halved.
            return mem[u][1] = sumWithFullNode;

        int sumWithHalvedNode = (price[u] / 2) * count[u];
        for (int v : graph[u])
            if (v != prev)
                sumWithHalvedNode += dfs(graph, v, u, price, count, true, mem);

        return mem[u][parentHalved ? 1 : 0] = Math.min(sumWithFullNode, sumWithHalvedNode);
    }
}
```

1693. 2647.java

Path: LeetCode-main/solutions/2647. Color the Triangle Red/2647.java

```
class Solution {
    public int[][] colorRed(int n) {
        List<int[]> ans = new ArrayList<>();
        final int tipSize = n % 4;
        // The tip of the triangle is always painted red.
        if (tipSize >= 1)
            ans.add(new int[] {1, 1});

        // Paint the rightmost and the leftmost elements at the following rows.
        for (int i = 2; i <= tipSize; ++i) {
            ans.add(new int[] {i, 1});
            ans.add(new int[] {i, 2 * i - 1});
        }

        // Paint the 4-row chunks.
        for (int i = tipSize + 1; i < n; i += 4) {
            // Fill the first row of the chunk.
            ans.add(new int[] {i, 1});
            // Fill the second row.
            for (int j = 1; j <= i; ++j)
                ans.add(new int[] {i + 1, 2 * j + 1});
            // Fill the third row.
            ans.add(new int[] {i + 2, 2});
            // Fill the fourth row.
            for (int j = 0; j <= i + 2; ++j)
                ans.add(new int[] {i + 3, 2 * j + 1});
        }

        return ans.stream().toArray(int[][] ::new);
    }
}
```

1694. 265.java

Path: LeetCode-main/solutions/265. Paint House II/265.java

```
class Solution {
    public int minCostII(int[][] costs) {
        int prevIndex = -1; // the previous minimum index
        int prevMin1 = 0;   // the minimum cost so far
        int prevMin2 = 0;   // the second minimum cost so far

        for (int[] cost : costs) { // O(n)
            // the painted index that will achieve the minimum cost after painting the
            // current house
            int index = -1;
            // the minimum cost after painting the current house
            int min1 = Integer.MAX_VALUE;
            // the second minimum cost after painting the current house
            int min2 = Integer.MAX_VALUE;
            for (int i = 0; i < cost.length; ++i) { // O(k)
                final int theCost = cost[i] + (i == prevIndex ? prevMin2 : prevMin1);
                if (theCost < min1) {
                    index = i;
                    min2 = min1;
                    min1 = theCost;
                } else if (theCost < min2) { // min1 <= theCost < min2
                    min2 = theCost;
                }
            }
            prevIndex = index;
            prevMin1 = min1;
            prevMin2 = min2;
        }

        return prevMin1;
    }
}
```


1695. 2651.java

Path: LeetCode-main/solutions/2651. Calculate Delayed Arrival Time/2651.java

```
class Solution {  
    public int findDelayedArrivalTime(int arrivalTime, int delayedTime) {  
        return (arrivalTime + delayedTime) % 24;  
    }  
}
```

1696. 2652-2.java

Path: LeetCode-main/solutions/2652. Sum Multiples/2652-2.java

```
class Solution {
    public int sumOfMultiples(int n) {
        return sumOfMultiples(n, 3) + sumOfMultiples(n, 5) + sumOfMultiples(n, 7) -
            (sumOfMultiples(n, 15) + sumOfMultiples(n, 21) + sumOfMultiples(n, 35)) +
            sumOfMultiples(n, 105);
    }

    // Returns the sum of multiples of value in [1, n].
    private int sumOfMultiples(int n, int value) {
        final int lo = value;
        final int hi = n / value * value;
        final int count = (hi - lo) / value + 1;
        return (lo + hi) * count / 2;
    }
}
```

1697. 2652.java

Path: LeetCode-main/solutions/2652. Sum Multiples/2652.java

```
class Solution {
    public int sumOfMultiples(int n) {
        int ans = 0;
        for (int i = 1; i <= n; ++i)
            if (i % 3 == 0 || i % 5 == 0 || i % 7 == 0)
                ans += i;
        return ans;
    }
}
```

1698. 2653.java

Path: LeetCode-main/solutions/2653. Sliding Subarray Beauty/2653.java

```
class Solution {
    public int[] getSubarrayBeauty(int[] nums, int k, int x) {
        int[] ans = new int[nums.length - k + 1];
        int[] count = new int[50]; // count[i] := the frequency of (i + 50)

        for (int i = 0; i < nums.length; ++i) {
            if (nums[i] < 0)
                ++count[nums[i] + 50];
            if (i - k >= 0 && nums[i - k] < 0)
                --count[nums[i - k] + 50];
            if (i + 1 >= k)
                ans[i - k + 1] = getXthSmallestNum(count, x);
        }

        return ans;
    }

    private int getXthSmallestNum(int[] count, int x) {
        int prefix = 0;
        for (int i = 0; i < 50; ++i) {
            prefix += count[i];
            if (prefix >= x)
                return i - 50;
        }
        return 0;
    }
}
```

1699. 2654.java

Path: LeetCode-main/solutions/2654. Minimum Number of Operations to Make All Array Elements Equal to 1/2654.java

```
class Solution {
    public int minOperations(int[] nums) {
        final int n = nums.length;
        final int ones = (int) Arrays.stream(nums).filter(num -> num == 1).count();
        if (ones > 0)
            return n - ones;

        // the minimum operations to make the shortest subarray with a gcd == 1
        int minOps = Integer.MAX_VALUE;

        for (int i = 0; i < n; ++i) {
            int g = nums[i];
            for (int j = i + 1; j < n; ++j) {
                g = gcd(g, nums[j]);
                if (g == 1) { // gcd(nums[i..j]) == 1
                    minOps = Math.min(minOps, j - i);
                    break;
                }
            }
        }

        // After making the shortest subarray with `minOps`, need additional n - 1
        // operations to make the other numbers to 1.
        return minOps == Integer.MAX_VALUE ? -1 : minOps + n - 1;
    }

    private int gcd(int a, int b) {
        return b == 0 ? a : gcd(b, a % b);
    }
}
```

1700. 2655.java

Path: LeetCode-main/solutions/2655. Find Maximal Uncovered Ranges/2655.java

```
class Solution {
    public int[][] findMaximalUncoveredRanges(int n, int[][] ranges) {
        List<int[]> ans = new ArrayList<>();
        int start = 0;

        Arrays.sort(ranges, Comparator.comparingInt(range -> range[0]));

        for (int[] range : ranges) {
            final int l = range[0];
            final int r = range[1];
            if (start < l)
                ans.add(new int[] {start, l - 1});
            if (start <= r)
                start = r + 1;
        }

        if (start < n)
            ans.add(new int[] {start, n - 1});

        return ans.stream().toArray(int[][] ::new);
    }
}
```

1701. 2656.java

Path: LeetCode-main/solutions/2656. Maximum Sum With Exactly K Elements/2656.java

```
class Solution {
    public int maximizeSum(int[] nums, int k) {
        // If x = max(nums), ans = x + (x + 1) + .. + (x + k - 1).
        final int x = Arrays.stream(nums).max().getAsInt();
        return x * k + k * (k - 1) / 2;
    }
}
```

1702. 2657.java

Path: LeetCode-main/solutions/2657. Find the Prefix Common Array of Two Arrays/2657.java

```
class Solution {
    public int[] findThePrefixCommonArray(int[] A, int[] B) {
        final int n = A.length;
        int prefixCommon = 0;
        int[] ans = new int[n];
        int[] count = new int[n + 1];

        for (int i = 0; i < A.length; ++i) {
            if (++count[A[i]] == 2)
                ++prefixCommon;
            if (++count[B[i]] == 2)
                ++prefixCommon;
            ans[i] = prefixCommon;
        }

        return ans;
    }
}
```


1703. 2658.java

Path: LeetCode-main/solutions/2658. Maximum Number of Fish in a Grid/2658.java

```
class Solution {
    public int findMaxFish(int[][] grid) {
        int ans = 0;

        for (int i = 0; i < grid.length; ++i)
            for (int j = 0; j < grid[0].length; ++j)
                if (grid[i][j] > 0)
                    ans = Math.max(ans, dfs(grid, i, j));

        return ans;
    }

    private int dfs(int[][] grid, int i, int j) {
        if (i < 0 || i == grid.length || j < 0 || j == grid.length)
            return 0;
        if (grid[i][j] == 0)
            return 0;
        int caughtFish = grid[i][j];
        grid[i][j] = 0; // Mark 0 as visited
        return caughtFish +
            dfs(grid, i + 1, j) + dfs(grid, i - 1, j) + //
            dfs(grid, i, j + 1) + dfs(grid, i, j - 1);
    }
}
```

1704. 2659.java

Path: LeetCode-main/solutions/2659. Make Array Empty/2659.java

```
class Solution {
    public long countOperationsToEmptyArray(int[] nums) {
        final int n = nums.length;
        long ans = n;
        Map<Integer, Integer> numToIndex = new HashMap<>();

        for (int i = 0; i < n; ++i)
            numToIndex.put(nums[i], i);

        Arrays.sort(nums);

        for (int i = 1; i < n; ++i)
            // On the i-th step we've already removed the i - 1 smallest numbers and
            // can ignore them. If an element nums[i] has smaller index in origin
            // array than nums[i - 1], we should rotate the whole left array n - i
            // times to set nums[i] element on the first position.
            if (numToIndex.get(nums[i]) < numToIndex.get(nums[i - 1]))
                ans += n - i;

        return ans;
    }
}
```

1705. 266.java

Path: LeetCode-main/solutions/266. Palindrome Permutation/266.java

```
class Solution {
    public boolean canPermutePalindrome(String s) {
        Set<Character> seen = new HashSet<>();

        for (final char c : s.toCharArray())
            if (!seen.add(c))
                seen.remove(c);

        return seen.size() <= 1;
    }
}
```

1706. 2660.java

Path: LeetCode-main/solutions/2660. Determine the Winner of a Bowling Game/2660.java

```
class Solution {
    public int isWinner(int[] player1, int[] player2) {
        final int score1 = getScore(player1);
        final int score2 = getScore(player2);
        if (score1 > score2)
            return 1;
        if (score2 > score1)
            return 2;
        return 0;
    }

    private int getScore(int[] player) {
        final int INVALID = -3;
        int score = 0;
        int last10 = INVALID;
        for (int i = 0; i < player.length; ++i) {
            score += i - last10 > 2 ? player[i] : player[i] * 2;
            if (player[i] == 10)
                last10 = i;
        }
        return score;
    }
}
```

1707. 2661.java

Path: LeetCode-main/solutions/2661. First Completely Painted Row or Column/2661.java

```
class Solution {
    public int firstCompleteIndex(int[] arr, int[][] mat) {
        final int m = mat.length;
        final int n = mat[0].length;
        // rows[i] := the number of painted grid in the i-th row
        int[] rows = new int[m];
        // cols[j] := the number of painted grid in the j-th column
        int[] cols = new int[n];
        // numToRow[num] := the i-th row of `num` in `mat`
        int[] numToRow = new int[m * n + 1];
        // numToCol[num] := the j-th column of `num` in `mat`
        int[] numToCol = new int[m * n + 1];

        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j) {
                numToRow[mat[i][j]] = i;
                numToCol[mat[i][j]] = j;
            }

        for (int i = 0; i < arr.length; ++i) {
            if (++rows[numToRow[arr[i]]] == n)
                return i;
            if (++cols[numToCol[arr[i]]] == m)
                return i;
        }

        throw new IllegalArgumentException();
    }
}
```

1708. 2662.java

Path: LeetCode-main/solutions/2662. Minimum Cost of a Path With Special Roads/2662.java

```
class Solution {
    public int minimumCost(int[] start, int[] target, int[][] specialRoads) {
        return dijkstra(specialRoads, start[0], start[1], target[0], target[1]);
    }

    private int dijkstra(int[][] specialRoads, int srcX, int srcY, int dstX, int dstY) {
        final int n = specialRoads.length;
        // dist[i] := the minimum distance of (srcX, srcY) to
        // specialRoads[i](x2, y2)
        int[] dist = new int[n];
        Arrays.fill(dist, Integer.MAX_VALUE);
        // (d, u), where u := the i-th specialRoads
        Queue<Pair<Integer, Integer>> minHeap =
            new PriorityQueue<>(Comparator.comparingInt(Pair::getKey));

        // (srcX, srcY) -> (x1, y1) to cost -> (x2, y2)
        for (int u = 0; u < n; ++u) {
            final int x1 = specialRoads[u][0];
            final int y1 = specialRoads[u][1];
            final int cost = specialRoads[u][4];
            final int d = Math.abs(x1 - srcX) + Math.abs(y1 - srcY) + cost;
            dist[u] = d;
            minHeap.offer(new Pair<>(dist[u], u));
        }

        while (!minHeap.isEmpty()) {
            final int d = minHeap.peek().getKey();
            final int u = minHeap.poll().getValue();
            if (d > dist[u])
                continue;
            final int ux2 = specialRoads[u][2];
            final int uy2 = specialRoads[u][3];
            for (int v = 0; v < n; ++v) {
                if (v == u)
                    continue;
                final int vx1 = specialRoads[v][0];
                final int vy1 = specialRoads[v][1];
                final int vcost = specialRoads[v][4];
                // (ux2, uy2) -> (vx1, vy1) to vcost -> (vx2, vy2)
                final int newDist = d + Math.abs(vx1 - ux2) + Math.abs(vy1 - uy2) + vcost;
                if (newDist < dist[v]) {
                    dist[v] = newDist;
                    minHeap.offer(new Pair<>(dist[v], v));
                }
            }
        }

        int ans = Math.abs(dstX - srcX) + Math.abs(dstY - srcY);
        for (int u = 0; u < n; ++u) {
            final int x2 = specialRoads[u][2];
            final int y2 = specialRoads[u][3];
            // (srcX, srcY) -> (x2, y2) -> (dstX, dstY).
            ans = Math.min(ans, dist[u] + Math.abs(dstX - x2) + Math.abs(dstY - y2));
        }
        return ans;
    }
}
```

1709. 2663.java

Path: LeetCode-main/solutions/2663. Lexicographically Smallest Beautiful String/2663.java

```
class Solution {
    public String smallestBeautifulString(String s, int k) {
        StringBuilder sb = new StringBuilder(s);

        for (int i = s.length() - 1; i >= 0; --i) {
            do {
                sb.setCharAt(i, (char) (sb.charAt(i) + 1));
            } while (containsPalindrome(sb, i));
            if (sb.charAt(i) < 'a' + k)
                // If sb[i] is among the first k letters, then change the letters after
                // sb[i] to the smallest ones that don't form any palindrome substring.
                return changeSuffix(sb, i + 1);
        }

        return "";
    }

    // Returns true if sb[0..i] contains any palindrome.
    private boolean containsPalindrome(StringBuilder sb, int i) {
        return (i > 0 && sb.charAt(i) == sb.charAt(i - 1)) ||
            (i > 1 && sb.charAt(i) == sb.charAt(i - 2));
    }

    // Returns a string, where replacing sb[i..n) with the smallest possible
    // letters don't form any palindrome substring.
    private String changeSuffix(StringBuilder sb, int i) {
        for (int j = i; j < sb.length(); ++j)
            for (sb.setCharAt(j, 'a'); containsPalindrome(sb, j);
                sb.setCharAt(j, (char) (sb.charAt(j) + 1)))
                ;
        return sb.toString();
    }
}
```

1710. 2664.java

Path: LeetCode-main/solutions/2664. The Knight's Tour/2664.java

```
class Solution {
    public int[][] tourOfKnight(int m, int n, int r, int c) {
        int[][] ans = new int[m][n];
        Arrays.stream(ans).forEach(A -> Arrays.fill(A, -1));
        dfs(m, n, r, c, 0, ans);
        return ans;
    }

    private static final int[][] DIRS = {{1, 2}, {2, 1}, {2, -1}, {1, -2},
                                           {-1, -2}, {-2, -1}, {-2, 1}, {-1, 2}};

    private boolean dfs(int m, int n, int i, int j, int step, int[][] ans) {
        if (step == m * n)
            return true;
        if (i < 0 || i >= m || j < 0 || j >= n)
            return false;
        if (ans[i][j] != -1)
            return false;
        ans[i][j] = step;
        for (int[] dir : DIRS)
            if (dfs(m, n, i + dir[0], j + dir[1], step + 1, ans))
                return true;
        ans[i][j] = -1;
        return false;
    }
}
```


1711. 267.java

Path: LeetCode-main/solutions/267. Palindrome Permutation II/267.java

```
class Solution {
    public List<String> generatePalindromes(String s) {
        int odd = 0;
        Map<Character, Integer> count = new HashMap<>();

        for (final char c : s.toCharArray())
            count.merge(c, 1, Integer::sum);

        // Count odd ones.
        for (Map.Entry<Character, Integer> entry : count.entrySet())
            if (entry.getValue() % 2 == 1)
                ++odd;

        // Can't form any palindrome.
        if (odd > 1)
            return new ArrayList<>();

        List<String> ans = new ArrayList<>();
        List<Character> candidates = new ArrayList<>();
        StringBuilder mid = new StringBuilder();

        // Get the mid and the candidates characters.
        for (Map.Entry<Character, Integer> entry : count.entrySet()) {
            final char key = entry.getKey();
            final int value = entry.getValue();
            if (value % 2 == 1)
                mid.append(key);
            for (int i = 0; i < value / 2; ++i)
                candidates.add(key);
        }

        // Backtrack to generate the ans strings.
        dfs(candidates, mid, new boolean[candidates.size()], new StringBuilder(), ans);
        return ans;
    }

    // Generates all the unique palindromes from the candidates.
    private void dfs(List<Character> candidates, StringBuilder mid, boolean[] used, StringBuilder sb,
                     List<String> ans) {
        if (sb.length() == candidates.size()) {
            ans.add(sb.toString() + mid + sb.reverse().toString());
            sb.reverse();
            return;
        }

        for (int i = 0; i < candidates.size(); ++i) {
            if (used[i])
                continue;
            if (i > 0 && candidates.get(i) == candidates.get(i - 1) && !used[i - 1])
                continue;
            used[i] = true;
            sb.append(candidates.get(i));
            dfs(candidates, mid, used, sb, ans);
            sb.deleteCharAt(sb.length() - 1);
            used[i] = false;
        }
    }
}
```

1712. 2670.java

Path: LeetCode-main/solutions/2670. Find the Distinct Difference Array/2670.java

```
class Solution {
    public int[] distinctDifferenceArray(int[] nums) {
        final int MAX = 50;
        int[] ans = new int[nums.length];
        int[] prefixCount = new int[MAX + 1];
        int[] suffixCount = new int[MAX + 1];
        int distinctPrefix = 0;
        int distinctSuffix = 0;

        for (final int num : nums)
            if (++suffixCount[num] == 1)
                ++distinctSuffix;

        for (int i = 0; i < nums.length; ++i) {
            if (++prefixCount[nums[i]] == 1)
                ++distinctPrefix;
            if (--suffixCount[nums[i]] == 0)
                --distinctSuffix;
            ans[i] = distinctPrefix - distinctSuffix;
        }

        return ans;
    }
}
```

1713. 2671.java

Path: LeetCode-main/solutions/2671. Frequency Tracker/2671.java

```
class FrequencyTracker {
    public void add(int number) {
        if (count[number] > 0)
            --freqCount[count[number]];
        ++count[number];
        ++freqCount[count[number]];
    }

    public void deleteOne(int number) {
        if (count[number] == 0)
            return;
        --freqCount[count[number]];
        --count[number];
        ++freqCount[count[number]];
    }

    public boolean hasFrequency(int frequency) {
        return freqCount[frequency] > 0;
    }

    private int[] freqCount = new int[100_001];
    private int[] count = new int[100_001];
}
```

1714. 2672.java

Path: LeetCode-main/solutions/2672. Number of Adjacent Elements With the Same Color/2672.java

```
class Solution {
    public int[] colorTheArray(int n, int[][] queries) {
        int[] ans = new int[queries.length];
        int[] arr = new int[n];
        int sameColors = 0;

        for (int i = 0; i < queries.length; ++i) {
            final int j = queries[i][0];
            final int color = queries[i][1];
            if (j + 1 < n) {
                if (arr[j + 1] > 0 && arr[j + 1] == arr[j])
                    --sameColors;
                if (arr[j + 1] == color)
                    ++sameColors;
            }
            if (j > 0) {
                if (arr[j - 1] > 0 && arr[j - 1] == arr[j])
                    --sameColors;
                if (arr[j - 1] == color)
                    ++sameColors;
            }
            arr[j] = color;
            ans[i] = sameColors;
        }

        return ans;
    }
}
```

1715. 2673.java

Path: LeetCode-main/solutions/2673. Make Costs of Paths Equal in a Binary Tree/2673.java

```
class Solution {
    public int minIncrements(int n, int[] cost) {
        int ans = 0;

        for (int i = n / 2 - 1; i >= 0; --i) {
            final int l = i * 2 + 1;
            final int r = i * 2 + 2;
            ans += Math.abs(cost[l] - cost[r]);
            // Record the information in the parent from the children. So, there's need to actually
            // update the values in the children.
            cost[i] += Math.max(cost[l], cost[r]);
        }

        return ans;
    }
}
```

1716. 2674.java

Path: LeetCode-main/solutions/2674. Split a Circular Linked List/2674.java

```
class Solution {
    public ListNode[] splitCircularLinkedList(ListNode list) {
        ListNode slow = list;
        ListNode fast = list;

        // Point `slow` to the last node in the first half.
        while (fast.next != list && fast.next.next != list) {
            slow = slow.next;
            fast = fast.next.next;
        }

        // Circle back the second half.
        ListNode secondHead = slow.next;
        if (fast.next == list)
            fast.next = secondHead;
        else
            fast.next.next = secondHead;

        // Circle back the first half.
        slow.next = list;

        return new ListNode[] {list, secondHead};
    }
}
```

1717. 2678.java

Path: LeetCode-main/solutions/2678. Number of Senior Citizens/2678.java

```
class Solution {  
    public int countSeniors(String[] details) {  
        return (int) Arrays.stream(details)  
            .filter(detail -> Integer.parseInt(detail.substring(11, 13)) > 60)  
            .count();  
    }  
}
```

1718. 2679.java

Path: LeetCode-main/solutions/2679. Sum in a Matrix/2679.java

```
class Solution {
    public int matrixSum(int[][] nums) {
        int ans = 0;

        for (int[] row : nums)
            Arrays.sort(row);

        for (int j = 0; j < nums[0].length; ++j) {
            int mx = 0;
            for (int i = 0; i < nums.length; ++i)
                mx = Math.max(mx, nums[i][j]);
            ans += mx;
        }

        return ans;
    }
}
```


1719. 268.java

Path: LeetCode-main/solutions/268. Missing Number/268.java

```
class Solution {  
    public int missingNumber(int[] nums) {  
        int ans = nums.length;  
  
        for (int i = 0; i < nums.length; ++i)  
            ans ^= i ^ nums[i];  
  
        return ans;  
    }  
}
```

1720. 2680.java

Path: LeetCode-main/solutions/2680. Maximum OR/2680.java

```
class Solution {
    public long maximumOr(int[] nums, int k) {
        final int n = nums.length;
        long ans = 0;
        // prefix[i] := nums[0] | nums[1] | ... | nums[i - 1]
        long[] prefix = new long[n];
        // suffix[i] := nums[i + 1] | nums[i + 2] | ... | nums[n - 1]
        long[] suffix = new long[n];

        for (int i = 1; i < n; ++i)
            prefix[i] = prefix[i - 1] | nums[i - 1];

        for (int i = n - 2; i >= 0; --i)
            suffix[i] = suffix[i + 1] | nums[i + 1];

        // For each num, greedily shift it left by k bits. The bitwise or value is
        // prefix[i] | nums[i] << k | suffix[i]
        for (int i = 0; i < n; ++i)
            ans = Math.max(ans, prefix[i] | (long) nums[i] << k | suffix[i]);

        return ans;
    }
}
```

1721. 2681.java

Path: LeetCode-main/solutions/2681. Power of Heroes/2681.java

```
class Solution {
    public int sumOfPower(int[] nums) {
        final int MOD = 1_000_000_007;
        long ans = 0;
        long sum = 0;

        Arrays.sort(nums);

        for (final int num : nums) {
            ans = (ans + (num + sum) * num % MOD * num % MOD) % MOD;
            sum = (sum * 2 + num) % MOD;
        }

        return (int) ans;
    }
}
```

1722. 2682.java

Path: LeetCode-main/solutions/2682. Find the Losers of the Circular Game/2682.java

```
class Solution {
    public int[] circularGameLosers(int n, int k) {
        List<Integer> ans = new ArrayList<>();
        boolean[] seen = new boolean[n];

        for (int friendIndex = 0, turn = 1; !seen[friendIndex];) {
            seen[friendIndex] = true;
            friendIndex += turn++ * k;
            friendIndex %= n;
        }

        for (int friendIndex = 0; friendIndex < n; ++friendIndex)
            if (!seen[friendIndex])
                ans.add(friendIndex + 1);

        return ans.stream().mapToInt(Integer::intValue).toArray();
    }
}
```

1723. 2683.java

Path: LeetCode-main/solutions/2683. Neighboring Bitwise XOR/2683.java

```
class Solution {
    public boolean doesValidArrayExist(int[] derived) {
        //      original = [0[0], 0[1], ..., 0[n - 1]]
        //      derived = [0[0]^0[1], 0[1]^0[2], ..., 0[n - 1]^0[0]]
        // XOR(derived) = 0
        return Arrays.stream(derived).reduce((a, b) -> a ^ b).getAsInt() == 0;
    }
}
```

1724. 2684.java

Path: LeetCode-main/solutions/2684. Maximum Number of Moves in a Grid/2684.java

```
class Solution {
    public int maxMoves(int[][] grid) {
        final int m = grid.length;
        final int n = grid[0].length;
        int ans = 0;
        // dp[i][j] := the maximum number of moves you can perform from (i, j)
        int[][] dp = new int[m][n];

        for (int j = n - 2; j >= 0; --j)
            for (int i = 0; i < m; ++i) {
                if (grid[i][j + 1] > grid[i][j])
                    dp[i][j] = 1 + dp[i][j + 1];
                if (i > 0 && grid[i - 1][j + 1] > grid[i][j])
                    dp[i][j] = Math.max(dp[i][j], 1 + dp[i - 1][j + 1]);
                if (i + 1 < m && grid[i + 1][j + 1] > grid[i][j])
                    dp[i][j] = Math.max(dp[i][j], 1 + dp[i + 1][j + 1]);
            }

        for (int i = 0; i < m; ++i)
            ans = Math.max(ans, dp[i][0]);

        return ans;
    }
}
```

1725. 2685.java

Path: LeetCode-main/solutions/2685. Count the Number of Complete Components/2685.java

```
class UnionFind {
    public UnionFind(int n) {
        id = new int[n];
        rank = new int[n];
        nodeCount = new int[n];
        edgeCount = new int[n];
        for (int i = 0; i < n; ++i) {
            id[i] = i;
            nodeCount[i] = 1;
        }
    }

    public void unionByRank(int u, int v) {
        final int i = find(u);
        final int j = find(v);
        ++edgeCount[i];
        if (i == j)
            return;
        if (rank[i] < rank[j]) {
            id[i] = j;
            edgeCount[j] += edgeCount[i];
            nodeCount[j] += nodeCount[i];
        } else if (rank[i] > rank[j]) {
            id[j] = i;
            edgeCount[i] += edgeCount[j];
            nodeCount[i] += nodeCount[j];
        } else {
            id[i] = j;
            edgeCount[j] += edgeCount[i];
            nodeCount[j] += nodeCount[i];
            ++rank[j];
        }
    }

    public int find(int u) {
        return id[u] == u ? u : (id[u] = find(id[u]));
    }

    public boolean isComplete(int u) {
        return nodeCount[u] * (nodeCount[u] - 1) / 2 == edgeCount[u];
    }

    private int[] id;
    private int[] rank;
    private int[] nodeCount;
    private int[] edgeCount;
}

class Solution {
    public int countCompleteComponents(int n, int[][] edges) {
        int ans = 0;
        UnionFind uf = new UnionFind(n);
        Set<Integer> parents = new HashSet<>();

        for (int[] edge : edges) {
            final int u = edge[0];
            final int v = edge[1];
            uf.unionByRank(u, v);
        }

        for (int i = 0; i < n; ++i) {
            final int parent = uf.find(i);
            if (parents.add(parent) && uf.isComplete(parent))
                ++ans;
        }

        return ans;
    }
}
```

1726. 2689-2.java

Path: LeetCode-main/solutions/2689. Extract Kth Character From The Rope Tree/2689-2.java

```
class Solution {
    public char getKthCharacter(RopeTreeNode root, int k) {
        while (root.len > 0) {
            final int leftLen = root.left == null ? 0 : Math.max(root.left.len, root.left.val.length());
            if (leftLen >= k) {
                root = root.left;
            } else {
                root = root.right;
                k -= leftLen;
            }
        }
        return root.val.charAt(k - 1);
    }
}
```


1727. 2689.java

Path: LeetCode-main/solutions/2689. Extract Kth Character From The Rope Tree/2689.java

```
class Solution {
    public char getKthCharacter(RopeTreeNode root, int k) {
        if (root.len == 0)
            return root.val.charAt(k - 1);
        final int leftLen = root.left == null ? 0 : Math.max(root.left.len, root.left.val.length());
        return leftLen >= k ? getKthCharacter(root.left, k) : getKthCharacter(root.right, k - leftLen);
    }
}
```

1728. 269.java

Path: LeetCode-main/solutions/269. Alien Dictionary/269.java

```
class Solution {
    public String alienOrder(String[] words) {
        Map<Character, Set<Character>> graph = new HashMap<>();
        int[] inDegrees = new int[26];
        buildGraph(graph, words, inDegrees);
        return topology(graph, inDegrees);
    }

    private void buildGraph(Map<Character, Set<Character>> graph, String[] words, int[] inDegrees) {
        // Create a node for each character in each word.
        for (final String word : words)
            for (final char c : word.toCharArray())
                graph.putIfAbsent(c, new HashSet<>());

        for (int i = 1; i < words.length; ++i) {
            final String first = words[i - 1];
            final String second = words[i];
            final int length = Math.min(first.length(), second.length());
            for (int j = 0; j < length; ++j) {
                final char u = first.charAt(j);
                final char v = second.charAt(j);
                if (u != v) {
                    if (!graph.get(u).contains(v)) {
                        graph.get(u).add(v);
                        ++inDegrees[v - 'a'];
                    }
                    break; // The order of characters after this are meaningless.
                }
                // e.g. first = "ab", second = "a" -> invalid
                if (j == length - 1 && first.length() > second.length()) {
                    graph.clear();
                    return;
                }
            }
        }
    }

    private String topology(Map<Character, Set<Character>> graph, int[] inDegrees) {
        StringBuilder sb = new StringBuilder();
        Queue<Character> q = graph.keySet()
            .stream()
            .filter(c -> inDegrees[c - 'a'] == 0)
            .collect(Collectors.toCollection(ArrayDeque::new));

        while (!q.isEmpty()) {
            final char u = q.poll();
            sb.append(u);
            for (final char v : graph.get(u))
                if (--inDegrees[v - 'a'] == 0)
                    q.offer(v);
        }

        // e.g. words = ["z", "x", "y", "x"]
        return sb.length() == graph.size() ? sb.toString() : "";
    }
}
```

1729. 2696.java

Path: LeetCode-main/solutions/2696. Minimum String Length After Removing Substrings/2696.java

```
class Solution {
    public int minLength(String s) {
        Deque<Character> stack = new ArrayDeque<>();

        for (final char c : s.toCharArray())
            if (c == 'B' && match(stack, 'A'))
                stack.pop();
            else if (c == 'D' && match(stack, 'C'))
                stack.pop();
            else
                stack.push(c);

        return stack.size();
    }

    private boolean match(Deque<Character> stack, char c) {
        return !stack.isEmpty() && stack.peek() == c;
    }
}
```

1730. 2697.java

Path: LeetCode-main/solutions/2697. Lexicographically Smallest Palindrome/2697.java

```
class Solution {
    public String makeSmallestPalindrome(String s) {
        char[] chars = s.toCharArray();
        for (int i = 0, j = s.length() - 1; i < j; ++i, --j) {
            final char minChar = (char) Math.min(chars[i], chars[j]);
            chars[i] = minChar;
            chars[j] = minChar;
        }
        return new String(chars);
    }
}
```

1731. 2698.java

Path: LeetCode-main/solutions/2698. Find the Punishment Number of an Integer/2698.java

```
class Solution {
    public int punishmentNumber(int n) {
        int ans = 0;
        for (int i = 1; i <= n; ++i)
            if (isPossible(0, 0, String.valueOf(i * i), 0, i))
                ans += i * i;

        return ans;
    }

    // Returns true if the sum of any split of `numChars` equals to the target.
    private boolean isPossible(int accumulate, int running, String numChars, int s, int target) {
        if (s == numChars.length())
            return target == accumulate + running;
        final int d = numChars.charAt(s) - '0';
        return isPossible(accumulate, running * 10 + d, numChars, s + 1, target) ||
            isPossible(accumulate + running, d, numChars, s + 1, target);
    }
}
```

1732. 2699.java

Path: LeetCode-main/solutions/2699. Modify Graph Edge Weights/2699.java

```
class Solution {
    public int[][] modifiedGraphEdges(int n, int[][] edges, int source, int destination, int target) {
        final int MAX = 2_000_000_000;
        List<Pair<Integer, Integer>>[] graph = new List[n];

        for (int i = 0; i < n; i++)
            graph[i] = new ArrayList<>();

        for (int[] edge : edges) {
            final int u = edge[0];
            final int v = edge[1];
            final int w = edge[2];
            if (w == -1)
                continue;
            graph[u].add(new Pair<>(v, w));
            graph[v].add(new Pair<>(u, w));
        }

        int distToDestination = dijkstra(graph, source, destination);
        if (distToDestination < target)
            return new int[0][];
        if (distToDestination == target) {
            // Change the weights of negative edges to an impossible value.
            for (int[] edge : edges)
                if (edge[2] == -1)
                    edge[2] = MAX;
            return edges;
        }

        for (int i = 0; i < edges.length; ++i) {
            final int u = edges[i][0];
            final int v = edges[i][1];
            final int w = edges[i][2];
            if (w != -1)
                continue;
            edges[i][2] = 1;
            graph[u].add(new Pair<>(v, 1));
            graph[v].add(new Pair<>(u, 1));
            distToDestination = dijkstra(graph, source, destination);
            if (distToDestination <= target) {
                edges[i][2] += target - distToDestination;
                // Change the weights of negative edges to an impossible value.
                for (int j = i + 1; j < edges.length; ++j)
                    if (edges[j][2] == -1)
                        edges[j][2] = MAX;
                return edges;
            }
        }

        return new int[0][];
    }

    private int dijkstra(List<Pair<Integer, Integer>>[] graph, int src, int dst) {
        int[] dist = new int[graph.length];
        Arrays.fill(dist, Integer.MAX_VALUE);

        dist[src] = 0;
        Queue<Pair<Integer, Integer>> minHeap =
            new PriorityQueue<>(Comparator.comparingInt(Pair::getKey)) {
                { offer(new Pair<>(dist[src], src)); } // (d, u)
            };

        while (!minHeap.isEmpty()) {
            final int d = minHeap.peek().getKey();
            final int u = minHeap.poll().getValue();
            if (d > dist[u])
                continue;
            for (Pair<Integer, Integer> pair : graph[u]) {
                final int v = pair.getKey();
                final int w = pair.getValue();
                if (d + w < dist[v]) {
                    dist[v] = d + w;
                    minHeap.offer(new Pair<>(dist[v], v));
                }
            }
        }

        return dist[dst];
    }
}
```

1733. 27.java

Path: LeetCode-main/solutions/27. Remove Element/27.java

```
class Solution {  
    public int removeElement(int[] nums, int val) {  
        int i = 0;  
  
        for (final int num : nums)  
            if (num != val)  
                nums[i++] = num;  
  
        return i;  
    }  
}
```

1734. 270.java

Path: LeetCode-main/solutions/270. Closest Binary Search Tree Value/270.java

```
class Solution {
    public int closestValue(TreeNode root, double target) {
        // If target < root.val, search the left subtree.
        if (target < root.val && root.left != null) {
            final int left = closestValue(root.left, target);
            if (Math.abs(left - target) <= Math.abs(root.val - target))
                return left;
        }

        // If target > root.val, search the right subtree.
        if (target > root.val && root.right != null) {
            final int right = closestValue(root.right, target);
            if (Math.abs(right - target) < Math.abs(root.val - target))
                return right;
        }

        return root.val;
    }
}
```


1735. 2702.java

Path: LeetCode-main/solutions/2702. Minimum Operations to Make Numbers Non-positive/2702.java

```
class Solution {
    public int minOperations(int[] nums, int x, int y) {
        int l = 0;
        int r = Arrays.stream(nums).max().getAsInt();

        while (l < r) {
            final int m = (l + r) / 2;
            if (isPossible(nums, x, y, m))
                r = m;
            else
                l = m + 1;
        }

        return l;
    }

    // Returns true if it's possible to make all `nums` <= 0 using m operations.
    private boolean isPossible(int[] nums, int x, int y, int m) {
        long additionalOps = 0;
        // If we want m operations, first decrease all the numbers by y * m. Then,
        // we have m operations to select indices to decrease them by x - y.
        for (final int num : nums)
            additionalOps += Math.max(0L, (num - 1L * y * m + x - y - 1) / (x - y));
        return additionalOps <= m;
    }
}
```

1736. 2706.java

Path: LeetCode-main/solutions/2706. Buy Two Chocolates/2706.java

```
class Solution {
    public int buyChoco(int[] prices, int money) {
        int min1 = Integer.MAX_VALUE;
        int min2 = Integer.MAX_VALUE;

        for (final int price : prices)
            if (price <= min1) {
                min2 = min1;
                min1 = price;
            } else if (price < min2) {
                min2 = price;
            }

        final int minCost = min1 + min2;
        return minCost > money ? money : money - minCost;
    }
}
```

1737. 2707.java

Path: LeetCode-main/solutions/2707. Extra Characters in a String/2707.java

```
class Solution {
    // Similar to 139. Word Break
    public int minExtraChar(String s, String[] dictionary) {
        final int n = s.length();
        Set<String> dictionarySet = new HashSet<>(Arrays.asList(dictionary));
        // dp[i] := the minimum extra letters if breaking up s[0..i) optimally
        int[] dp = new int[n + 1];
        Arrays.fill(dp, n);
        dp[0] = 0;

        for (int i = 1; i <= n; i++)
            for (int j = 0; j < i; j++)
                // s[j..i) is in `dictionarySet`.
                if (dictionarySet.contains(s.substring(j, i)))
                    dp[i] = Math.min(dp[i], dp[j]);
                // s[j..i) are extra letters.
                else
                    dp[i] = Math.min(dp[i], dp[j] + i - j);

        return dp[n];
    }
}
```

1738. 2708.java

Path: LeetCode-main/solutions/2708. Maximum Strength of a Group/2708.java

```
class Solution {
    public long maxStrength(int[] nums) {
        long posProd = 1;
        long negProd = 1;
        int maxNeg = Integer.MIN_VALUE;
        int negCount = 0;
        boolean hasPos = false;
        boolean hasZero = false;

        for (final int num : nums)
            if (num > 0) {
                posProd *= num;
                hasPos = true;
            } else if (num < 0) {
                negProd *= num;
                maxNeg = Math.max(maxNeg, num);
                ++negCount;
            } else { // num == 0
                hasZero = true;
            }

        if (negCount == 0 && !hasPos)
            return 0;
        if (negCount % 2 == 0)
            return negProd * posProd;
        if (negCount >= 3)
            return negProd / maxNeg * posProd;
        if (hasPos)
            return posProd;
        if (hasZero)
            return 0;
        return maxNeg;
    }
}
```

1739. 2709.java

Path: LeetCode-main/solutions/2709. Greatest Common Divisor Traversal/2709.java

```
class UnionFind {
    public UnionFind(int n) {
        id = new int[n];
        sz = new int[n];
        for (int i = 0; i < n; ++i)
            id[i] = i;
        for (int i = 0; i < n; ++i)
            sz[i] = 1;
    }

    public void unionBySize(int u, int v) {
        final int i = find(u);
        final int j = find(v);
        if (i == j)
            return;
        if (sz[i] < sz[j]) {
            sz[j] += sz[i];
            id[i] = j;
        } else {
            sz[i] += sz[j];
            id[j] = i;
        }
    }

    public int getSize(int i) {
        return sz[i];
    }

    private int[] id;
    private int[] sz;

    private int find(int u) {
        return id[u] == u ? u : (id[u] = find(id[u]));
    }
}

class Solution {
    public boolean canTraverseAllPairs(int[] nums) {
        final int n = nums.length;
        final int mx = Arrays.stream(nums).max().getAsInt();
        final int[] minPrimeFactors = sieveEratosthenes(mx + 1);
        Map<Integer, Integer> primeToFirstIndex = new HashMap<>();
        UnionFind uf = new UnionFind(n);

        for (int i = 0; i < n; ++i)
            for (final int primeFactor : getPrimeFactors(nums[i], minPrimeFactors))
                // `primeFactor` already appeared in the previous indices.
                if (primeToFirstIndex.containsKey(primeFactor))
                    uf.unionBySize(primeToFirstIndex.get(primeFactor), i);
                else
                    primeToFirstIndex.put(primeFactor, i);

        for (int i = 0; i < n; ++i)
            if (uf.getSize(i) == n)
                return true;
        return false;
    }

    // Gets the minimum prime factor of i, where 1 < i <= n.
    private int[] sieveEratosthenes(int n) {
        int[] minPrimeFactors = new int[n + 1];
        for (int i = 2; i <= n; ++i)
            minPrimeFactors[i] = i;
        for (int i = 2; i * i < n; ++i)
            if (minPrimeFactors[i] == i) // `i` is prime.
                for (int j = i * i; j < n; j += i)
                    minPrimeFactors[j] = Math.min(minPrimeFactors[j], i);
        return minPrimeFactors;
    }

    private List<Integer> getPrimeFactors(int num, int[] minPrimeFactors) {
        List<Integer> primeFactors = new ArrayList<>();
        while (num > 1) {
            final int divisor = minPrimeFactors[num];
            primeFactors.add(divisor);
            while (num % divisor == 0)
                num /= divisor;
        }
        return primeFactors;
    }
}
```


1740. 271.java

Path: LeetCode-main/solutions/271. Encode and Decode Strings/271.java

```
public class Codec {
    // Encodes a list of strings to a single string.
    public String encode(List<String> strs) {
        StringBuilder encoded = new StringBuilder();

        for (final String s : strs)
            encoded.append(s.length()).append('/').append(s);

        return encoded.toString();
    }

    // Decodes a single string to a list of strings.
    public List<String> decode(String s) {
        List<String> decoded = new ArrayList<>();

        for (int i = 0; i < s.length(); ) {
            final int slash = s.indexOf('/', i);
            final int length = Integer.parseInt(s.substring(i, slash));
            i = slash + length + 1;
            decoded.add(s.substring(slash + 1, i));
        }

        return decoded;
    }
}
```

1741. 2710.java

Path: LeetCode-main/solutions/2710. Remove Trailing Zeros From a String/2710.java

```
class Solution {  
    public String removeTrailingZeros(String num) {  
        return num.replaceAll("0+$", "");  
    }  
}
```


1742. 2711.java

Path: LeetCode-main/solutions/2711. Difference of Number of Distinct Values on Diagonals/2711.java

```
class Solution {
    public int[][] differenceOfDistinctValues(int[][] grid) {
        final int m = grid.length;
        final int n = grid[0].length;
        int[][] ans = new int[m][n];

        for (int i = 0; i < m; ++i)
            fillInDiagonal(grid, i, 0, ans);

        for (int j = 1; j < n; ++j)
            fillInDiagonal(grid, 0, j, ans);

        return ans;
    }

    private void fillInDiagonal(int[][] grid, int i, int j, int[][] ans) {
        Set<Integer> topLeft = new HashSet<>();
        Set<Integer> bottomRight = new HashSet<>();

        // Fill in the diagonal from the top-left to the bottom-right.
        while (i < grid.length && j < grid[0].length) {
            ans[i][j] = topLeft.size();
            // Post-addition, so this information can be utilized in subsequent cells.
            topLeft.add(grid[i++][j++]);
        }

        --i;
        --j;

        // Fill in the diagonal from the bottom-right to the top-left.
        while (i >= 0 && j >= 0) {
            ans[i][j] = Math.abs(ans[i][j] - bottomRight.size());
            // Post-addition, so this information can be utilized in subsequent cells.
            bottomRight.add(grid[i--][j--]);
        }
    }
}
```

1743. 2712.java

Path: LeetCode-main/solutions/2712. Minimum Cost to Make All Characters Equal/2712.java

```
class Solution {
    public long minimumCost(String s) {
        final int n = s.length();
        long ans = 0;

        for (int i = 1; i < n; ++i)
            if (s.charAt(i) != s.charAt(i - 1))
                // Invert s[0..i - 1] or s[i..n - 1].
                ans += Math.min(i, n - i);

        return ans;
    }
}
```

1744. 2713.java

Path: LeetCode-main/solutions/2713. Maximum Strictly Increasing Cells in a Matrix/2713.java

```
class Solution {
    public int maxIncreasingCells(int[][] mat) {
        final int m = mat.length;
        final int n = mat[0].length;
        // rows[i] := the maximum path length for the i-th row
        int[] rows = new int[m];
        // cols[j] := the maximum path length for the j-th column
        int[] cols = new int[n];
        Map<Integer, ArrayList<Pair<Integer, Integer>>> valToIndices = new HashMap<>();
        // maxPathLength[i][j] := the maximum path length from mat[i][j]
        int[][] maxPathLength = new int[m][n];
        // Sort all the unique values in the matrix in non-increasing order.
        TreeSet<Integer> decreasingSet = new TreeSet<>(Comparator.reverseOrder());

        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j) {
                final int val = mat[i][j];
                valToIndices.putIfAbsent(val, new ArrayList<>());
                valToIndices.get(val).add(new Pair<>(i, j));
                decreasingSet.add(val);
            }

        for (final int val : decreasingSet) {
            for (Pair<Integer, Integer> pair : valToIndices.get(val)) {
                final int i = pair.getKey();
                final int j = pair.getValue();
                maxPathLength[i][j] = Math.max(rows[i], cols[j]) + 1;
            }
            for (Pair<Integer, Integer> pair : valToIndices.get(val)) {
                final int i = pair.getKey();
                final int j = pair.getValue();
                rows[i] = Math.max(rows[i], maxPathLength[i][j]);
                cols[j] = Math.max(cols[j], maxPathLength[i][j]);
            }
        }

        return Math.max(Arrays.stream(rows).max().getAsInt(), //
            Arrays.stream(cols).max().getAsInt());
    }
}
```

1745. 2714.java

Path: LeetCode-main/solutions/2714. Find Shortest Path with K Hops/2714.java

```
class Solution {
    // Similar to 787. Cheapest Flights Within K Stops
    public int shortestPathWithHops(int n, int[][] edges, int s, int d, int k) {
        List<Pair<Integer, Integer>>[] graph = new List[n];

        for (int i = 0; i < n; i++)
            graph[i] = new ArrayList<>();

        for (int[] edge : edges) {
            final int u = edge[0];
            final int v = edge[1];
            final int w = edge[2];
            graph[u].add(new Pair<>(v, w));
            graph[v].add(new Pair<>(u, w));
        }

        return dijkstra(graph, s, d, k);
    }

    private int dijkstra(List<Pair<Integer, Integer>>[] graph, int src, int dst, int k) {
        int[][] dist = new int[graph.length][k + 1];
        Arrays.stream(dist).forEach(A -> Arrays.fill(A, Integer.MAX_VALUE));

        dist[src][k] = 0;
        Queue<int[]> minHeap = new PriorityQueue<>(Comparator.comparingInt(a -> a[0])) {
            { offer(new int[] {dist[src][k], src, k}); } // (d, u, hops)
        };

        while (!minHeap.isEmpty()) {
            final int d = minHeap.peek()[0];
            final int u = minHeap.peek()[1];
            final int hops = minHeap.poll()[2];
            if (u == dst)
                return d;
            if (dist[u][hops] > d)
                continue;
            for (Pair<Integer, Integer> pair : graph[u]) {
                final int v = pair.getKey();
                final int w = pair.getValue();
                // Go from u -> v with w cost.
                if (d + w < dist[v][hops]) {
                    dist[v][hops] = d + w;
                    minHeap.offer(new int[] {dist[v][hops], v, hops});
                }
                // Hop from u -> v with 0 cost.
                if (hops > 0 && d < dist[v][hops - 1]) {
                    dist[v][hops - 1] = d;
                    minHeap.offer(new int[] {dist[v][hops - 1], v, hops - 1});
                }
            }
        }

        throw new IllegalArgumentException();
    }
}
```

1746. 2716.java

Path: LeetCode-main/solutions/2716. Minimize String Length/2716.java

```
class Solution {  
    public int minimizedStringLength(String s) {  
        return new HashSet<>(Arrays.asList(s.split(""))).size();  
    }  
}
```

1747. 2717.java

Path: LeetCode-main/solutions/2717. Semi-Ordered Permutation/2717.java

```
class Solution {
    public int semiOrderedPermutation(int[] nums) {
        final int n = nums.length;
        int index1 = -1;
        int indexN = -1;
        for (int i = 0; i < nums.length; ++i) {
            if (nums[i] == 1)
                index1 = i;
            else if (nums[i] == n)
                indexN = i;
        }
        return index1 + (n - 1 - indexN) - (index1 > indexN ? 1 : 0);
    }
}
```

1748. 2718.java

Path: LeetCode-main/solutions/2718. Sum of Matrix After Queries/2718.java

```
class Solution {
    public long matrixSumQueries(int n, int[][] queries) {
        long ans = 0;
        // seen[0] := row, seen[1] := col
        boolean[][] seen = new boolean[2][n];
        // notSet[0] = row, notSet[1] := col
        int[] notSet = new int[2];
        Arrays.fill(notSet, n);

        // Late queries dominate.
        for (int i = queries.length - 1; i >= 0; --i) {
            final int type = queries[i][0];
            final int index = queries[i][1];
            final int val = queries[i][2];
            if (!seen[type][index]) {
                ans += val * notSet[type ^ 1];
                seen[type][index] = true;
                --notSet[type];
            }
        }
        return ans;
    }
}
```

1749. 272.java

Path: LeetCode-main/solutions/272. Closest Binary Search Tree Value II/272.java

```
class Solution {
    public List<Integer> closestKValues(TreeNode root, double target, int k) {
        Deque<Integer> dq = new ArrayDeque<>();

        inorder(root, dq);

        while (dq.size() > k)
            if (Math.abs(dq.peekFirst() - target) > Math.abs(dq.peekLast() - target))
                dq.pollFirst();
            else
                dq.pollLast();

        return new ArrayList<>(dq);
    }

    private void inorder(TreeNode root, Deque<Integer> dq) {
        if (root == null)
            return;

        inorder(root.left, dq);
        dq.offerLast(root.val);
        inorder(root.right, dq);
    }
}
```


1750. 2728.java

Path: LeetCode-main/solutions/2728. Count Houses in a Circular Street/2728.java

```
/**
 * Definition for a street.
 * class Street {
 *     public Street(int[] doors);
 *     public void openDoor();
 *     public void closeDoor();
 *     public boolean isDoorOpen();
 *     public void moveRight();
 *     public void moveLeft();
 * }
 */

class Solution {
    public int houseCount(Street street, int k) {
        for (int i = 0; i < k; ++i) {
            if (street.isDoorOpen())
                street.closeDoor();
            street.moveRight();
        }

        for (int ans = 0;; ++ans) {
            if (street.isDoorOpen())
                return ans;
            street.openDoor();
            street.moveRight();
        }
    }
}
```

1751. 2729.java

Path: LeetCode-main/solutions/2729. Check if The Number is Fascinating/2729.java

```
class Solution {  
    public boolean isFascinating(int n) {  
        String s = Integer.toString(n) + Integer.toString(2 * n) + Integer.toString(3 * n);  
        char[] charArray = s.toCharArray();  
        Arrays.sort(charArray);  
        return new String(charArray).equals("123456789");  
    }  
}
```

1752. 273.java

Path: LeetCode-main/solutions/273. Integer to English Words/273.java

```
class Solution {
    public String numberToWords(int num) {
        return num == 0 ? "Zero" : helper(num);
    }

    private final String[] belowTwenty = {"", "One", "Two", "Three", "Four",
                                           "Five", "Six", "Seven", "Eight", "Nine",
                                           "Ten", "Eleven", "Twelve", "Thirteen", "Fourteen",
                                           "Fifteen", "Sixteen", "Seventeen", "Eighteen", "Nineteen"};
    private final String[] tens = {"", "", "Twenty", "Thirty", "Forty",
                                   "Fifty", "Sixty", "Seventy", "Eighty", "Ninety"};

    private String helper(int num) {
        StringBuilder s = new StringBuilder();

        if (num < 20)
            s.append(belowTwenty[num]);
        else if (num < 100)
            s.append(tens[num / 10]).append(" ").append(belowTwenty[num % 10]);
        else if (num < 1000)
            s.append(helper(num / 100)).append(" Hundred ").append(helper(num % 100));
        else if (num < 1000000)
            s.append(helper(num / 1000)).append(" Thousand ").append(helper(num % 1000));
        else if (num < 1000000000)
            s.append(helper(num / 1000000)).append(" Million ").append(helper(num % 1000000));
        else
            s.append(helper(num / 1000000000)).append(" Billion ").append(helper(num % 1000000000));

        return s.toString().trim();
    }
}
```

1753. 2730.java

Path: LeetCode-main/solutions/2730. Find the Longest Semi-Repetitive Substring/2730.java

```
class Solution {
    public int longestSemiRepetitiveSubstring(String s) {
        int ans = 1;
        int prevStart = 0;
        int start = 0;

        for (int i = 1; i < s.length(); ++i) {
            if (s.charAt(i) == s.charAt(i - 1)) {
                if (prevStart > 0)
                    start = prevStart;
                prevStart = i;
            }
            ans = Math.max(ans, i - start + 1);
        }

        return ans;
    }
}
```

1754. 2731.java

Path: LeetCode-main/solutions/2731. Movement of Robots/2731.java

```
class Solution {
    public int sumDistance(int[] nums, String s, int d) {
        final int MOD = 1_000_000_007;
        final int n = nums.length;
        int ans = 0;
        int prefix = 0;
        int[] pos = new int[n];

        for (int i = 0; i < n; ++i)
            if (s.charAt(i) == 'L')
                pos[i] = nums[i] - d;
            else
                pos[i] = nums[i] + d;

        Arrays.sort(pos);

        for (int i = 0; i < n; ++i) {
            ans = (int) (((ans + 1L * i * pos[i] - prefix) % MOD + MOD) % MOD);
            prefix = (int) (((0L + prefix + pos[i]) % MOD + MOD) % MOD);
        }

        return ans;
    }
}
```

1755. 2732.java

Path: LeetCode-main/solutions/2732. Find a Good Subset of the Matrix/2732.java

```
class Solution {
    public List<Integer> goodSubsetofBinaryMatrix(int[][] grid) {
        final int MAX_BIT = 30;
        Map<Integer, Integer> maskToIndex = new HashMap<>();

        for (int i = 0; i < grid.length; ++i) {
            final int mask = getMask(grid[i]);
            if (mask == 0)
                return List.of(i);
            for (int prevMask = 1; prevMask < MAX_BIT; ++prevMask)
                if ((mask & prevMask) == 0 && maskToIndex.containsKey(prevMask))
                    return List.of(maskToIndex.get(prevMask), i);
            maskToIndex.put(mask, i);
        }

        return new ArrayList<>();
    }

    private int getMask(int[] row) {
        int mask = 0;
        for (int i = 0; i < row.length; ++i)
            if (row[i] == 1)
                mask |= 1 << i;
        return mask;
    }
}
```

1756. 2733.java

Path: LeetCode-main/solutions/2733. Neither Minimum nor Maximum/2733.java

```
class Solution {
    public int findNonMinOrMax(int[] nums) {
        if (nums.length < 3)
            return -1;
        Arrays.sort(nums, 0, 3);
        return nums[1];
    }
}
```

1757. 2734.java

Path: LeetCode-main/solutions/2734. Lexicographically Smallest String After Substring Operation/2734.java

```
class Solution {
    public String smallestString(String s) {
        char[] charArray = s.toCharArray();
        final int n = s.length();
        int i = 0;

        while (i < n && charArray[i] == 'a')
            ++i;
        if (i == n) {
            charArray[n - 1] = 'z';
            return new String(charArray);
        }

        while (i < n && charArray[i] != 'a')
            --charArray[i++];

        return new String(charArray);
    }
}
```


1758. 2735.java

Path: LeetCode-main/solutions/2735. Collecting Chocolates/2735.java

```
class Solution {
    public long minCost(int[] nums, long x) {
        int n = nums.length;
        long ans = Long.MAX_VALUE;
        // minCost[i] := the minimum cost to collect the i-th type
        int[] minCost = new int[n];
        Arrays.fill(minCost, Integer.MAX_VALUE);

        for (int rotate = 0; rotate < n; ++rotate) {
            for (int i = 0; i < n; ++i)
                minCost[i] = Math.min(minCost[i], nums[(i - rotate + n) % n]);
            ans = Math.min(ans, Arrays.stream(minCost).asLongStream().sum() + rotate * x);
        }

        return ans;
    }
}
```

1759. 2736.java

Path: LeetCode-main/solutions/2736. Maximum Sum Queries/2736.java

```
class Solution {
    public int[] maximumSumQueries(int[] nums1, int[] nums2, int[][] queries) {
        MyPair[] pairs = getPairs(nums1, nums2);
        IndexedQuery[] indexedQueries = getIndexedQueries(queries);
        int[] ans = new int[queries.length];
        List<Pair<Integer, Integer>> stack = new ArrayList<>(); // [(y, x + y)]

        int pairsIndex = 0;
        for (IndexedQuery indexedQuery : indexedQueries) {
            final int queryIndex = indexedQuery.queryIndex;
            final int minX = indexedQuery.minX;
            final int minY = indexedQuery.minY;
            while (pairsIndex < pairs.length && pairs[pairsIndex].x >= minX) {
                MyPair pair = pairs[pairsIndex++];
                // x + y is a better candidate. Given that x is decreasing, the
                // condition "x + y >= stack.get(stack.size() - 1).getValue()" suggests
                // that y is relatively larger, thereby making it a better candidate.
                final int x = pair.x;
                final int y = pair.y;
                while (!stack.isEmpty() && x + y >= stack.get(stack.size() - 1).getValue())
                    stack.remove(stack.size() - 1);
                if (stack.isEmpty() || y > stack.get(stack.size() - 1).getKey())
                    stack.add(new Pair<>(y, x + y));
            }
            final int j = firstGreaterEqual(stack, minY);
            ans[queryIndex] = j == stack.size() ? -1 : stack.get(j).getValue();
        }

        return ans;
    }

    private record MyPair(int x, int y){};
    private record IndexedQuery(int queryIndex, int minX, int minY){};

    private int firstGreaterEqual(List<Pair<Integer, Integer>> A, int target) {
        int l = 0;
        int r = A.size();
        while (l < r) {
            final int m = (l + r) / 2;
            if (A.get(m).getKey() >= target)
                r = m;
            else
                l = m + 1;
        }
        return l;
    }

    private MyPair[] getPairs(int[] nums1, int[] nums2) {
        MyPair[] pairs = new MyPair[nums1.length];
        for (int i = 0; i < nums1.length; ++i)
            pairs[i] = new MyPair(nums1[i], nums2[i]);
        Arrays.sort(pairs, Comparator.comparing(MyPair::x, Comparator.reverseOrder()));
        return pairs;
    }

    private IndexedQuery[] getIndexedQueries(int[][] queries) {
        IndexedQuery[] indexedQueries = new IndexedQuery[queries.length];
        for (int i = 0; i < queries.length; ++i)
            indexedQueries[i] = new IndexedQuery(i, queries[i][0], queries[i][1]);
        Arrays.sort(indexedQueries,
            Comparator.comparing(IndexedQuery::minX, Comparator.reverseOrder()));
        return indexedQueries;
    }
}
```

1760. 2737.java

Path: LeetCode-main/solutions/2737. Find the Closest Marked Node/2737.java

```
class Solution {
    public int minimumDistance(int n, List<List<Integer>> edges, int s, int[] marked) {
        int ans = Integer.MAX_VALUE;
        List<Pair<Integer, Integer>>[] graph = new List[n];

        for (int i = 0; i < n; i++)
            graph[i] = new ArrayList<>();

        for (List<Integer> edge : edges) {
            final int u = edge.get(0);
            final int v = edge.get(1);
            final int w = edge.get(2);
            graph[u].add(new Pair<>(v, w));
        }

        int[] dist = dijkstra(graph, s);

        for (final int u : marked)
            ans = Math.min(ans, dist[u]);

        return ans == Integer.MAX_VALUE ? -1 : ans;
    }

    private int[] dijkstra(List<Pair<Integer, Integer>>[] graph, int src) {
        int[] dist = new int[graph.length];
        Arrays.fill(dist, Integer.MAX_VALUE);

        dist[src] = 0;
        Queue<Pair<Integer, Integer>> minHeap =
            new PriorityQueue<>(Comparator.comparingInt(Pair::getKey)) {
                {
                    offer(new Pair<>(dist[src], src)); // (d, u)
                }
            };

        while (!minHeap.isEmpty()) {
            final int d = minHeap.peek().getKey();
            final int u = minHeap.poll().getValue();
            if (d > dist[u])
                continue;
            for (Pair<Integer, Integer> pair : graph[u]) {
                final int v = pair.getKey();
                final int w = pair.getValue();
                if (d + w < dist[v]) {
                    dist[v] = d + w;
                    minHeap.offer(new Pair<>(dist[v], v));
                }
            }
        }

        return dist;
    }
}
```

1761. 2739.java

Path: LeetCode-main/solutions/2739. Total Distance Traveled/2739.java

```
class Solution {
    public int distanceTraveled(int mainTank, int additionalTank) {
        // M M M M M A M M M M A
        // 1 [2 3 4 5] 6 [7 8 9 10] 11
        return (mainTank + Math.min((mainTank - 1) / 4, additionalTank)) * 10;
    }
}
```

1762. 274-2.java

Path: LeetCode-main/solutions/274. H-Index/274-2.java

```
class Solution {
    public int hIndex(int[] citations) {
        final int n = citations.length;

        Arrays.sort(citations);

        for (int i = 0; i < n; ++i)
            if (citations[i] >= n - i)
                return n - i;

        return 0;
    }
}
```

1763. 274.java

Path: LeetCode-main/solutions/274. H-Index/274.java

```
class Solution {
    public int hIndex(int[] citations) {
        final int n = citations.length;
        int accumulate = 0;
        int[] count = new int[n + 1];

        for (final int citation : citations)
            ++count[Math.min(citation, n)];

        // To find the maximum h-index, loop from the back to the front.
        // i := the candidate's h-index
        for (int i = n; i >= 0; --i) {
            accumulate += count[i];
            if (accumulate >= i)
                return i;
        }

        throw new IllegalArgumentException();
    }
}
```

1764. 2740.java

Path: LeetCode-main/solutions/2740. Find the Value of the Partition/2740.java

```
class Solution {
    public int findValueOfPartition(int[] nums) {
        int ans = Integer.MAX_VALUE;

        Arrays.sort(nums);

        for (int i = 1; i < nums.length; ++i)
            ans = Math.min(ans, nums[i] - nums[i - 1]);

        return ans;
    }
}
```

1765. 2741.java

Path: LeetCode-main/solutions/2741. Special Permutations/2741.java

```
class Solution {
    public int specialPerm(int[] nums) {
        final int n = nums.length;
        final int maxMask = 1 << n;
        int ans = 0;
        int[][] mem = new int[n][maxMask];

        for (int i = 0; i < n; ++i) {
            ans += specialPerm(nums, i, 1 << i, maxMask, mem);
            ans %= MOD;
        }

        return ans;
    }

    private static final int MOD = 1_000_000_007;

    // Returns the number of special permutations, where the previous number is
    // nums[i] and `mask` is the bitmask of the used numbers.
    private int specialPerm(int[] nums, int prev, int mask, int maxMask, int[][] mem) {
        if (mask == maxMask - 1)
            return 1;
        if (mem[prev][mask] != 0)
            return mem[prev][mask];

        int res = 0;

        for (int i = 0; i < nums.length; ++i) {
            if ((mask >> i & 1) == 1)
                continue;
            if (nums[i] % nums[prev] == 0 || nums[prev] % nums[i] == 0) {
                res += specialPerm(nums, i, mask | 1 << i, maxMask, mem);
                res %= MOD;
            }
        }

        return mem[prev][mask] = res;
    }
}
```


1766. 2742-2.java

Path: LeetCode-main/solutions/2742. Painting the Walls/2742-2.java

```
class Solution {
    public int paintWalls(int[] cost, int[] time) {
        final int MAX = 500_000_000;
        final int n = cost.length;
        // dp[i] := the minimum cost to paint i walls by the painters so far
        int[] dp = new int[n + 1];
        Arrays.fill(dp, MAX);
        dp[0] = 0;

        for (int i = 0; i < n; ++i)
            for (int walls = n; walls > 0; --walls)
                dp[walls] = Math.min(dp[walls], dp[Math.max(walls - time[i] - 1, 0)] + cost[i]);

        return dp[n];
    }
}
```

1767. 2742.java

Path: LeetCode-main/solutions/2742. Painting the Walls/2742.java

```
class Solution {
    public int paintWalls(int[] cost, int[] time) {
        final int n = cost.length;
        int[][] mem = new int[n][n + 1];
        return paintWalls(cost, time, 0, time.length, mem);
    }

    private static final int MAX = 500_000_000;

    // Returns the minimum cost to paint j walls by painters[i..n).
    private int paintWalls(int[] cost, int[] time, int i, int walls, int[][] mem) {
        if (walls <= 0)
            return 0;
        if (i == cost.length)
            return MAX;
        if (mem[i][walls] > 0)
            return mem[i][walls];
        final int pick = cost[i] + paintWalls(cost, time, i + 1, walls - time[i] - 1, mem);
        final int skip = paintWalls(cost, time, i + 1, walls, mem);
        return mem[i][walls] = Math.min(pick, skip);
    }
}
```

1768. 2743.java

Path: LeetCode-main/solutions/2743. Count Substrings Without Repeating Character/2743.java

```
class Solution {
    public int numberOfSpecialSubstrings(String s) {
        int ans = 0;
        int[] count = new int[26];

        for (int l = 0, r = 0; r < s.length(); ++r) {
            ++count[s.charAt(r) - 'a'];
            while (count[s.charAt(r) - 'a'] == 2)
                --count[s.charAt(l++) - 'a'];
            ans += r - l + 1;
        }

        return ans;
    }
}
```

1769. 2744.java

Path: LeetCode-main/solutions/2744. Find Maximum Number of String Pairs/2744.java

```
class Solution {
    public int maximumNumberOfStringPairs(String[] words) {
        int ans = 0;
        boolean[] seen = new boolean[26 * 26];

        for (final String word : words) {
            if (seen[val(word.charAt(1)) * 26 + val(word.charAt(0))])
                ++ans;
            seen[val(word.charAt(0)) * 26 + val(word.charAt(1))] = true;
        }

        return ans;
    }

    private final int val(char c) {
        return c - 'a';
    }
}
```

1770. 2745.java

Path: LeetCode-main/solutions/2745. Construct the Longest New String/2745.java

```
class Solution {
    public int longestString(int x, int y, int z) {
        final int mn = Math.min(x, y);
        if (x == y)
            return (mn * 2 + z) * 2;
        return (mn * 2 + 1 + z) * 2;
    }
}
```


1772. 2747.java

Path: LeetCode-main/solutions/2747. Count Zero Request Servers/2747.java

```
class Solution {
    public int[] countServers(int n, int[][] logs, int x, int[] queries) {
        int[] ans = new int[queries.length];
        int[] count = new int[n + 1];

        Arrays.sort(logs, Comparator.comparingInt(log -> log[1]));

        int i = 0;
        int j = 0;
        int servers = 0;

        // For each query, we care about logs[i..j].
        for (IndexedQuery indexedQuery : getIndexedQueries(queries)) {
            final int queryIndex = indexedQuery.queryIndex;
            final int query = indexedQuery.query;
            for (; j < logs.length && logs[j][1] <= query; ++j)
                if (++count[logs[j][0]] == 1)
                    ++servers;
            for (; i < logs.length && logs[i][1] < query - x; ++i)
                if (--count[logs[i][0]] == 0)
                    --servers;
            ans[queryIndex] = n - servers;
        }

        return ans;
    }

    private record IndexedQuery(int queryIndex, int query){};

    private IndexedQuery[] getIndexedQueries(int[] queries) {
        IndexedQuery[] indexedQueries = new IndexedQuery[queries.length];
        for (int i = 0; i < queries.length; ++i)
            indexedQueries[i] = new IndexedQuery(i, queries[i]);
        Arrays.sort(indexedQueries, Comparator.comparingInt(IndexedQuery::query));
        return indexedQueries;
    }
}
```

1773. 2748.java

Path: LeetCode-main/solutions/2748. Number of Beautiful Pairs/2748.java

```
class Solution {
    public int countBeautifulPairs(int[] nums) {
        int ans = 0;

        for (int i = 0; i < nums.length; ++i)
            for (int j = i + 1; j < nums.length; ++j)
                if (gcd(firstDigit(nums[i]), lastDigit(nums[j])) == 1)
                    ++ans;

        return ans;
    }

    private int firstDigit(int num) {
        return Integer.parseInt(Integer.toString(num).substring(0, 1));
    }

    private int lastDigit(int num) {
        return num % 10;
    }

    private int gcd(int a, int b) {
        return b == 0 ? a : gcd(b, a % b);
    }
}
```


1774. 2749.java

Path: LeetCode-main/solutions/2749. Minimum Operations to Make the Integer Zero/2749.java

```
class Solution {
    public int makeTheIntegerZero(int num1, int num2) {
        // If k operations are used,  $\text{num1} - [(\text{num2} + 2^{i_1}) + (\text{num2} + 2^{i_2}) + \dots + (\text{num2} + 2^{i_k})] = 0$ . So,  $\text{num1} - k * \text{num2} = (2^{i_1} + 2^{i_2} + \dots + 2^{i_k})$ , where  $i_1, i_2, \dots, i_k$  are in the range  $[0, 60]$ .
        // Note that for any number x, we can use "x's bit count" operations to make x equal to 0. Additionally, we can also use x operations to deduct x by  $2^0$  (x times), which also results in 0.

        for (long ops = 0; ops <= 60; ++ops) {
            long target = num1 - ops * num2;
            if (Long.bitCount(target) <= ops && ops <= target)
                return (int) ops;
        }

        return -1;
    }
}
```

1775. 275.java

Path: LeetCode-main/solutions/275. H-Index II/275.java

```
class Solution {
    public int hIndex(int[] citations) {
        final int n = citations.length;
        int l = 0;
        int r = n;

        while (l < r) {
            final int m = (l + r) / 2;
            if (citations[m] + m >= n)
                r = m;
            else
                l = m + 1;
        }

        return n - l;
    }
}
```

1776. 2750.java

Path: LeetCode-main/solutions/2750. Ways to Split Array Into Good Subarrays/2750.java

```
class Solution {
    public int numberOfGoodSubarraySplits(int[] nums) {
        if (Arrays.stream(nums).filter(num -> num == 1).count() == 0)
            return 0;

        final int MOD = 1_000_000_007;
        int prev = -1; // the previous index of 1
        int ans = 1;

        for (int i = 0; i < nums.length; ++i)
            if (nums[i] == 1) {
                if (prev != -1)
                    ans = (int) ((long) ans * (i - prev) % MOD);
                prev = i;
            }

        return ans;
    }
}
```

1777. 2751.java

Path: LeetCode-main/solutions/2751. Robot Collisions/2751.java

```
class Robot {
    public int index;
    public int position;
    public int health;
    public char direction;
    public Robot(int index, int position, int health, char direction) {
        this.index = index;
        this.position = position;
        this.health = health;
        this.direction = direction;
    }
}

class Solution {
    public List<Integer> survivedRobotsHealths(int[] positions, int[] healths, String directions) {
        List<Integer> ans = new ArrayList<>();
        Robot[] robots = new Robot[positions.length];
        List<Robot> stack = new ArrayList<>(); // running robots

        for (int i = 0; i < positions.length; ++i)
            robots[i] = new Robot(i, positions[i], healths[i], directions.charAt(i));

        Arrays.sort(robots, Comparator.comparingInt((Robot robot) -> robot.position));

        for (Robot robot : robots) {
            if (robot.direction == 'R') {
                stack.add(robot);
                continue;
            }
            // Collide with robots going right if any.
            while (!stack.isEmpty() && stack.get(stack.size() - 1).direction == 'R' && robot.health > 0) {
                if (stack.get(stack.size() - 1).health == robot.health) {
                    stack.remove(stack.size() - 1);
                    robot.health = 0;
                } else if (stack.get(stack.size() - 1).health < robot.health) {
                    stack.remove(stack.size() - 1);
                    robot.health -= 1;
                } else { // stack[-1].health > robot.health
                    stack.get(stack.size() - 1).health -= 1;
                    robot.health = 0;
                }
            }
            if (robot.health > 0)
                stack.add(robot);
        }

        stack.sort(Comparator.comparingInt((Robot robot) -> robot.index));

        for (Robot robot : stack)
            ans.add(robot.health);

        return ans;
    }
}
```

1778. 2753.java

Path: LeetCode-main/solutions/2753. Count Houses in a Circular Street II/2753.java

```
/**
 * Definition for a street.
 * class Street {
 *     public Street(int[] doors);
 *     public void closeDoor();
 *     public boolean isDoorOpen();
 *     public void moveRight();
 * }
 */
class Solution {
    public int houseCount(Street street, int k) {
        int ans = 0;

        // Go to the first open door.
        while (!street.isDoorOpen())
            street.moveRight();

        street.moveRight();

        for (int count = 1; count <= k; ++count) {
            // Each time we encounter an open door, there's a possibility that it's
            // the first open door we intentionally left open.
            if (street.isDoorOpen()) {
                ans = count;
                street.closeDoor();
            }
            street.moveRight();
        }

        return ans;
    }
}
```

1779. 276.java

Path: LeetCode-main/solutions/276. Paint Fence/276.java

```
class Solution {
    public int numWays(int n, int k) {
        if (n == 0)
            return 0;
        if (n == 1)
            return k;
        if (n == 2)
            return k * k;

        // dp[i] := the number of ways to paint n posts with k colors
        int[] dp = new int[n + 1];
        dp[0] = 0;
        dp[1] = k;
        dp[2] = k * k;

        for (int i = 3; i <= n; ++i)
            dp[i] = (dp[i - 1] + dp[i - 2]) * (k - 1);

        return dp[n];
    }
}
```

1780. 2760.java

Path: LeetCode-main/solutions/2760. Longest Even Odd Subarray With Threshold/2760.java

```
class Solution {
    public int longestAlternatingSubarray(int[] nums, int threshold) {
        int ans = 0;
        int dp = 0;

        for (int i = 0; i < nums.length; ++i) {
            if (nums[i] > threshold)
                dp = 0;
            else if (i > 0 && dp > 0 && isOddEven(nums[i - 1], nums[i]))
                // Increase the size of the subarray.
                ++dp;
            else
                // Start a new subarray if the start is valid.
                dp = nums[i] % 2 == 0 ? 1 : 0;
            ans = Math.max(ans, dp);
        }

        return ans;
    }

    private boolean isOddEven(int a, int b) {
        return a % 2 != b % 2;
    }
}
```

1781. 2761.java

Path: LeetCode-main/solutions/2761. Prime Pairs With Target Sum/2761.java

```
class Solution {
    public List<List<Integer>> findPrimePairs(int n) {
        boolean[] isPrime = sieveEratosthenes(n + 1);
        List<List<Integer>> ans = new ArrayList<>();

        for (int i = 2; i <= n / 2; ++i)
            if (isPrime[i] && isPrime[n - i])
                ans.add(List.of(i, n - i));

        return ans;
    }

    private boolean[] sieveEratosthenes(int n) {
        boolean[] isPrime = new boolean[n];
        Arrays.fill(isPrime, true);
        isPrime[0] = false;
        isPrime[1] = false;
        for (int i = 2; i * i < n; ++i)
            if (isPrime[i])
                for (int j = i * i; j < n; j += i)
                    isPrime[j] = false;
        return isPrime;
    }
}
```


1782. 2762.java

Path: LeetCode-main/solutions/2762. Continuous Subarrays/2762.java

```
class Solution {
    public long continuousSubarrays(int[] nums) {
        long ans = 1; // [nums[0]]
        int left = nums[0] - 2;
        int right = nums[0] + 2;
        int l = 0;

        // nums[l..r] is a valid window with range in [left, right].
        for (int r = 1; r < nums.length; r++) {
            if (left <= nums[r] && nums[r] <= right) {
                left = Math.max(left, nums[r] - 2);
                right = Math.min(right, nums[r] + 2);
            } else {
                // nums[r] is out-of-bounds, so reconstruct the window.
                left = nums[r] - 2;
                right = nums[r] + 2;
                l = r;
                // If we consistently move leftward in each iteration, it implies that
                // the entire left subarray satisfies the given condition. For every
                // subarray with l in the range [0, r], the condition is met, preventing
                // the code from reaching the final "else" condition. Instead, it stops
                // at the "if" condition.
                while (nums[r] - 2 <= nums[l] && nums[l] <= nums[r] + 2) {
                    left = Math.max(left, nums[l] - 2);
                    right = Math.min(right, nums[l] + 2);
                    --l;
                }
                ++l;
            }
            // nums[l..r], nums[l + 1..r], ..., nums[r]
            ans += r - l + 1;
        }

        return ans;
    }
}
```

1783. 2763-2.java

Path: LeetCode-main/solutions/2763. Sum of Imbalance Numbers of All Subarrays/2763-2.java

```
class Solution {
    // If sorted(nums)[i + 1] - sorted(nums)[i] > 1, then there's a gap. Instead
    // of determining the number of gaps in each subarray, let's find out how many
    // subarrays contain each gap.
    public int sumImbalanceNumbers(int[] nums) {
        final int n = nums.length;
        int ans = 0;
        // Note that to avoid double counting, only `left` needs to check nums[i].
        // This adjustment ensures that i represents the position of the leftmost
        // element of nums[i] within the subarray.

        // left[i] := the maximum index l s.t. nums[l] = nums[i] or nums[i] + 1
        int[] left = new int[n];
        // right[i] := the minimum index r s.t. nums[r] = nums[i]
        int[] right = new int[n];
        int[] numToIndex = new int[n + 2];

        Arrays.fill(numToIndex, -1);
        for (int i = 0; i < n; ++i) {
            left[i] = Math.max(numToIndex[nums[i]], numToIndex[nums[i] + 1]);
            numToIndex[nums[i]] = i;
        }

        Arrays.fill(numToIndex, n);
        for (int i = n - 1; i >= 0; --i) {
            right[i] = numToIndex[nums[i] + 1];
            numToIndex[nums[i]] = i;
        }

        // The gap above nums[i] persists until encountering nums[i] or nums[i] + 1.
        // Consider subarrays nums[l..r] with l <= i <= r, where l in [left[i], i]
        // and r in [i, right[i] - 1]. There are (i - left[i]) * (right[i] - i)
        // subarrays satisfying this condition.
        for (int i = 0; i < n; ++i)
            ans += (i - left[i]) * (right[i] - i);

        // Subtract n * (n + 1) / 2 to account for the overcounting of elements
        // initially assumed to have a gap. This adjustment is necessary as the
        // maximum element of every subarray does not have a gap.
        return ans - n * (n + 1) / 2;
    }
}
```

1784. 2764.java

Path: LeetCode-main/solutions/2764. is Array a Preorder of Some ■Binary Tree/2764.java

```
class Solution {
    public boolean isPreorder(List<List<Integer>> nodes) {
        Deque<Integer> stack = new ArrayDeque<>(); // Stores `id`s.

        for (List<Integer> node : nodes) {
            final int id = node.get(0);
            final int parentId = node.get(1);
            if (parentId == -1) {
                stack.push(id);
                continue;
            }
            while (!stack.isEmpty() && stack.peek() != parentId)
                stack.pop();
            if (stack.isEmpty())
                return false;
            stack.push(id);
        }
        return true;
    }
}
```

1785. 2765.java

Path: LeetCode-main/solutions/2765. Longest Alternating Subarray/2765.java

```
class Solution {
    public int alternatingSubarray(int[] nums) {
        int ans = 1;
        int dp = 1;

        for (int i = 1; i < nums.length; ++i) {
            final int targetDiff = dp % 2 == 0 ? -1 : 1;
            // Append nums[i] to the current alternating subarray.
            if (nums[i] - nums[i - 1] == targetDiff)
                ++dp;
            // Reset the alternating subarray to nums[i - 1..i].
            else if (nums[i] - nums[i - 1] == 1)
                dp = 2;
            // Reset the alternating subarray to nums[i].
            else
                dp = 1;
            ans = Math.max(ans, dp);
        }

        return ans == 1 ? -1 : ans;
    }
}
```

1786. 2766.java

Path: LeetCode-main/solutions/2766. Relocate Marbles/2766.java

```
class Solution {
    public List<Integer> relocateMarbles(int[] nums, int[] moveFrom, int[] moveTo) {
        Set<Integer> numsSet = Arrays.stream(nums).boxed().collect(Collectors.toSet());

        for (int i = 0; i < moveFrom.length; ++i) {
            numsSet.remove(moveFrom[i]);
            numsSet.add(moveTo[i]);
        }

        return numsSet.stream().sorted().toList();
    }
}
```

1787. 2767.java

Path: LeetCode-main/solutions/2767. Partition String Into Minimum Beautiful Substrings/2767.java

```
class Solution {
    public int minimumBeautifulSubstrings(String s) {
        final int n = s.length();
        // dp[i] := the minimum number of beautiful substrings for the first i chars
        int[] dp = new int[n + 1];
        Arrays.fill(dp, n + 1);
        dp[0] = 0;

        for (int i = 1; i <= n; ++i) {
            if (s.charAt(i - 1) == '0')
                continue;
            int num = 0; // num of s[i - 1..j - 1]
            for (int j = i; j <= n; ++j) {
                num = (num << 1) + s.charAt(j - 1) - '0';
                if (isPowerOfFive(num))
                    dp[j] = Math.min(dp[j], dp[i - 1] + 1);
            }
        }

        return dp[n] == n + 1 ? -1 : dp[n];
    }

    private boolean isPowerOfFive(int num) {
        while (num % 5 == 0)
            num /= 5;
        return num == 1;
    }
}
```

1788. 2768.java

Path: LeetCode-main/solutions/2768. Number of Black Blocks/2768.java

```
class Solution {
    public long[] countBlackBlocks(int m, int n, int[][] coordinates) {
        long[] ans = new long[5];
        // count[i * n + j] := the number of black cells in
        // (i - 1, j - 1), (i - 1, j), (i, j - 1), (i, j)
        Map<Long, Integer> count = new HashMap<>();

        for (int[] coordinate : coordinates) {
            final int x = coordinate[0];
            final int y = coordinate[1];
            for (long i = x; i < x + 2; ++i)
                for (long j = y; j < y + 2; ++j)
                    // 2 x 2 submatrix with right-bottom conner being (i, j) contains the
                    // current black cell (x, y).
                    if (i - 1 >= 0 && i < m && j - 1 >= 0 && j < n)
                        count.merge(i * n + j, 1, Integer::sum);
        }

        for (final int freq : count.values())
            ++ans[freq];

        ans[0] = (m - 1L) * (n - 1) - Arrays.stream(ans).sum();
        return ans;
    }
}
```

1789. 2769.java

Path: LeetCode-main/solutions/2769. Find the Maximum Achievable Number/2769.java

```
class Solution {  
    public int theMaximumAchievableX(int num, int t) {  
        return num + 2 * t;  
    }  
}
```


1790. 277.java

Path: LeetCode-main/solutions/277. Find the Celebrity/277.java

```
public class Solution extends Relation {
    public int findCelebrity(int n) {
        int candidate = 0;

        // Everyone knows the celebrity.
        for (int i = 1; i < n; ++i)
            if (knows(candidate, i))
                candidate = i;

        // The candidate knows nobody and everyone knows the celebrity.
        for (int i = 0; i < n; ++i) {
            if (i < candidate && knows(candidate, i) || !knows(i, candidate))
                return -1;
            if (i > candidate && !knows(i, candidate))
                return -1;
        }

        return candidate;
    }
}
```

1791. 2770.java

Path: LeetCode-main/solutions/2770. Maximum Number of Jumps to Reach the Last Index/2770.java

```
class Solution {
    public int maximumJumps(int[] nums, int target) {
        final int n = nums.length;
        // dp[i] := the maximum number of jumps to reach i from 0
        int[] dp = new int[n];
        Arrays.fill(dp, -1);
        dp[0] = 0;

        for (int j = 1; j < n; ++j)
            for (int i = 0; i < j; ++i)
                if (dp[i] != -1 && Math.abs(nums[j] - nums[i]) <= target)
                    dp[j] = Math.max(dp[j], dp[i] + 1);

        return dp[n - 1];
    }
}
```

1792. 2771.java

Path: LeetCode-main/solutions/2771. Longest Non-decreasing Subarray From Two Arrays/2771.java

```
class Solution {
    public int maxNonDecreasingLength(int[] nums1, int[] nums2) {
        int ans = 1;
        int dp1 = 1; // the longest subarray that ends in nums1[i] so far
        int dp2 = 1; // the longest subarray that ends in nums2[i] so far

        for (int i = 1; i < nums1.length; ++i) {
            final int dp11 = nums1[i - 1] <= nums1[i] ? dp1 + 1 : 1;
            final int dp21 = nums2[i - 1] <= nums1[i] ? dp2 + 1 : 1;
            final int dp12 = nums1[i - 1] <= nums2[i] ? dp1 + 1 : 1;
            final int dp22 = nums2[i - 1] <= nums2[i] ? dp2 + 1 : 1;
            dp1 = Math.max(dp11, dp21);
            dp2 = Math.max(dp12, dp22);
            ans = Math.max(ans, Math.max(dp1, dp2));
        }

        return ans;
    }
}
```

1793. 2772.java

Path: LeetCode-main/solutions/2772. Apply Operations to Make All Array Elements Equal to Zero/2772.java

```
class Solution {
    public boolean checkArray(int[] nums, int k) {
        if (k == 1)
            return true;

        int needDecrease = 0;
        // Store nums[i - k + 1..i] with decreasing nums[i - k + 1].
        Deque<Integer> dq = new ArrayDeque<>();

        for (int i = 0; i < nums.length; ++i) {
            if (i >= k)
                needDecrease -= dq.pollFirst();
            if (nums[i] < needDecrease)
                return false;
            final int decreasedNum = nums[i] - needDecrease;
            dq.offerLast(decreasedNum);
            needDecrease += decreasedNum;
        }

        return dq.getLast() == 0;
    }
}
```

1794. 2773.java

Path: LeetCode-main/solutions/2773. Height of Special Binary Tree/2773.java

```
class Solution {
    public int heightOfTree(TreeNode root) {
        if (root == null)
            return 0;
        // a leaf node
        if (root.left != null && root.left.right == root)
            return 0;
        return 1 + Math.max(heightOfTree(root.left), heightOfTree(root.right));
    }
}
```

1795. 2778.java

Path: LeetCode-main/solutions/2778. Sum of Squares of Special Elements/2778.java

```
class Solution {
    public int sumOfSquares(int[] nums) {
        final int n = nums.length;
        int ans = 0;

        for (int i = 0; i < n; ++i)
            if (n % (i + 1) == 0)
                ans += nums[i] * nums[i];

        return ans;
    }
}
```

1796. 2779-2.java

Path: LeetCode-main/solutions/2779. Maximum Beauty of an Array After Applying Operation/2779-2.java

```
class Solution {
    public int maximumBeauty(int[] nums, int k) {
        // l and r track the maximum window instead of the valid window.
        int l = 0;
        int r = 0;

        Arrays.sort(nums);

        for (r = 0; r < nums.length; ++r)
            if (nums[r] - nums[l] > 2 * k)
                ++l;

        return r - l;
    }
}
```

1797. 2779.java

Path: LeetCode-main/solutions/2779. Maximum Beauty of an Array After Applying Operation/2779.java

```
class Solution {
    public int maximumBeauty(int[] nums, int k) {
        int ans = 0;

        Arrays.sort(nums);

        for (int l = 0, r = 0; r < nums.length; ++r) {
            while (nums[r] - nums[l] > 2 * k)
                ++l;
            ans = Math.max(ans, r - l + 1);
        }

        return ans;
    }
}
```


1798. 278.java

Path: LeetCode-main/solutions/278. First Bad Version/278.java

```
public class Solution extends VersionControl {
    public int firstBadVersion(int n) {
        int l = 1;
        int r = n;

        while (l < r) {
            final int m = l + (r - l) / 2;
            if (isBadVersion(m))
                r = m;
            else
                l = m + 1;
        }
        return l;
    }
}
```

1799. 2780.java

Path: LeetCode-main/solutions/2780. Minimum Index of a Valid Split/2780.java

```
class Solution {
    public int minimumIndex(List<Integer> nums) {
        final int n = nums.size();
        Map<Integer, Integer> count1 = new HashMap<>();
        Map<Integer, Integer> count2 = new HashMap<>();

        for (final int num : nums)
            count2.merge(num, 1, Integer::sum);

        for (int i = 0; i < n; ++i) {
            final int freq1 = count1.merge(nums.get(i), 1, Integer::sum);
            final int freq2 = count2.merge(nums.get(i), -1, Integer::sum);
            if (freq1 * 2 > i + 1 && freq2 * 2 > n - 1 - i)
                return i;
        }

        return -1;
    }
}
```

1800. 2781-2.java

Path: LeetCode-main/solutions/2781. Length of the Longest Valid Substring/2781-2.java

```
class TrieNode {
    public TrieNode[] children = new TrieNode[26];
    public boolean isWord = false;
}

class Trie {
    public void insert(final String word) {
        TrieNode node = root;
        for (final char c : word.toCharArray()) {
            final int i = c - 'a';
            if (node.children[i] == null)
                node.children[i] = new TrieNode();
            node = node.children[i];
        }
        node.isWord = true;
    }

    public boolean search(final String word, int l, int r) {
        TrieNode node = root;
        for (int j = l; j <= r; ++j) {
            final int i = word.charAt(j) - 'a';
            if (node.children[i] == null)
                return false;
            node = node.children[i];
        }
        return node.isWord;
    }

    private TrieNode root = new TrieNode();
}

class Solution {
    public int longestValidSubstring(String word, List<String> forbidden) {
        int ans = 0;
        Trie trie = new Trie();

        for (final String s : forbidden)
            trie.insert(s);

        // r is the rightmost index to make word[l..r] a valid substring.
        int r = word.length() - 1;
        for (int l = word.length() - 1; l >= 0; --l) {
            for (int end = l; end < Math.min(l + 10, r + 1); ++end)
                if (trie.search(word, l, end)) {
                    r = end - 1;
                    break;
                }
            ans = Math.max(ans, r - l + 1);
        }

        return ans;
    }
}
```

1801. 2781.java

Path: LeetCode-main/solutions/2781. Length of the Longest Valid Substring/2781.java

```
class Solution {
    public int longestValidSubstring(String word, List<String> forbidden) {
        int ans = 0;
        Set<String> forbiddenSet = new HashSet<>(forbidden);

        // r is the rightmost index to make word[l..r] a valid substring.
        int r = word.length() - 1;
        for (int l = word.length() - 1; l >= 0; --l) {
            for (int end = l; end < Math.min(l + 10, r + 1); ++end)
                if (forbiddenSet.contains(word.substring(l, end + 1))) {
                    r = end - 1;
                    break;
                }
            ans = Math.max(ans, r - l + 1);
        }

        return ans;
    }
}
```

1802. 2782.java

Path: LeetCode-main/solutions/2782. Number of Unique Categories/2782.java

```
/**
 * Definition for a category handler.
 * class CategoryHandler {
 *     public CategoryHandler(int[] categories);
 *     public boolean haveSameCategory(int a, int b);
 * };
 */

class Solution {
    public int numberOfCategories(int n, CategoryHandler categoryHandler) {
        int ans = 0;
        for (int i = 0; i < n; ++i)
            if (!haveSameCategoryPreviously(i, n, categoryHandler))
                ++ans;
        return ans;
    }

    private boolean haveSameCategoryPreviously(int i, int n, CategoryHandler categoryHandler) {
        for (int j = 0; j < i; ++j)
            if (categoryHandler.haveSameCategory(i, j))
                return true;
        return false;
    }
}
```

1803. 2784.java

Path: LeetCode-main/solutions/2784. Check if Array is Good/2784.java

```
class Solution {
    public boolean isGood(int[] nums) {
        final int MAX = 200;
        final int n = nums.length - 1;
        int[] count = new int[MAX + 1];

        for (final int num : nums)
            ++count[num];

        return (n == 0 || Arrays.stream(count, 1, n).allMatch(c -> c == 1)) && count[n] == 2;
    }
}
```

1804. 2785.java

Path: LeetCode-main/solutions/2785. Sort Vowels in a String/2785.java

```
class Solution {
    public String sortVowels(String s) {
        StringBuilder sb = new StringBuilder();
        List<Character> vowels = new ArrayList<>();

        for (final char c : s.toCharArray())
            if (isVowel(c))
                vowels.add(c);

        Collections.sort(vowels);

        int i = 0; // vowels' index
        for (final char c : s.toCharArray())
            sb.append(isVowel(c) ? vowels.get(i++) : c);

        return sb.toString();
    }

    private boolean isVowel(char c) {
        return "aeiouAEIOU".indexOf(c) != -1;
    }
}
```

1805. 2786.java

Path: LeetCode-main/solutions/2786. Visit Array Positions to Maximize Score/2786.java

```
class Solution {
    public long maxScore(int[] nums, int x) {
        // Note that we always need to take nums[0], so the initial definition might
        // not hold true.

        // dp0 := the maximum score so far with `nums` ending in an even number
        long dp0 = nums[0] - (nums[0] % 2 == 1 ? x : 0);
        // dp1 := the maximum score so far with `nums` ending in an odd number
        long dp1 = nums[0] - (nums[0] % 2 == 0 ? x : 0);

        for (int i = 1; i < nums.length; ++i)
            if (nums[i] % 2 == 0)
                dp0 = nums[i] + Math.max(dp0, dp1 - x);
            else
                dp1 = nums[i] + Math.max(dp1, dp0 - x);

        return Math.max(dp0, dp1);
    }
}
```


1806. 2787.java

Path: LeetCode-main/solutions/2787. Ways to Express an Integer as Sum of Powers/2787.java

```
class Solution {
    public int numberOfWays(int n, int x) {
        final int MOD = 1_000_000_007;
        // dp[i] := the number of ways to express i
        int[] dp = new int[n + 1];
        int ax; // a^x

        dp[0] = 1;

        for (int a = 1; (ax = (int) Math.pow(a, x)) <= n; ++a)
            for (int i = n; i >= ax; --i) {
                dp[i] += dp[i - ax];
                dp[i] %= MOD;
            }

        return dp[n];
    }
}
```

1807. 2788.java

Path: LeetCode-main/solutions/2788. Split Strings by Separator/2788.java

```
class Solution {  
    public List<String> splitWordsBySeparator(List<String> words, char separator) {  
        return words.stream()  
            .flatMap(word -> Arrays.stream(word.split("\\\" + separator)))  
            .filter(word -> !word.isEmpty())  
            .collect(Collectors.toList());  
    }  
}
```

1808. 2789.java

Path: LeetCode-main/solutions/2789. Largest Element in an Array after Merge Operations/2789.java

```
class Solution {
    public long maxArrayValue(int[] nums) {
        long ans = nums[nums.length - 1];

        for (int i = nums.length - 2; i >= 0; --i)
            if (nums[i] > ans)
                ans = nums[i];
            else
                ans += nums[i];

        return ans;
    }
}
```

1809. 279.java

Path: LeetCode-main/solutions/279. Perfect Squares/279.java

```
class Solution {
    public int numSquares(int n) {
        int[] dp = new int[n + 1];
        Arrays.fill(dp, n); // 1^2 x n
        dp[0] = 0;          // no way
        dp[1] = 1;          // 1^2

        for (int i = 2; i <= n; ++i)
            for (int j = 1; j * j <= i; ++j)
                dp[i] = Math.min(dp[i], dp[i - j * j] + 1);

        return dp[n];
    }
}
```

1810. 2790.java

Path: LeetCode-main/solutions/2790. Maximum Number of Groups With Increasing Length/2790.java

```
class Solution {
    public int maxIncreasingGroups(List<Integer> usageLimits) {
        int ans = 1; // the next target length
        long availableLimits = 0;

        Collections.sort(usageLimits);

        for (final int usageLimit : usageLimits) {
            availableLimits += usageLimit;
            // Can create groups 1, 2, ..., ans.
            if (availableLimits >= ans * (long) (ans + 1) / 2)
                ++ans;
        }
        return ans - 1;
    }
}
```

1811. 2791.java

Path: LeetCode-main/solutions/2791. Count Paths That Can Form a Palindrome in a Tree/2791.java

```
class Solution {
    public long countPalindromePaths(List<Integer> parent, String s) {
        // A valid (u, v) has at most 1 letter with odd frequency on its path. The
        // frequency of a letter on the u-v path is equal to the sum of its
        // frequencies on the root-u and root-v paths subtract twice of its
        // frequency on the root-LCA(u, v) path. Considering only the parity
        // (even/odd), the part involving root-LCA(u, v) can be ignored, making it
        // possible to calculate both parts easily using a simple DFS.
        List<Integer>[] tree = new List[parent.size()];

        for (int i = 0; i < parent.size(); ++i)
            tree[i] = new ArrayList<>();

        for (int i = 1; i < parent.size(); ++i)
            tree[parent.get(i)].add(i);

        return dfs(tree, 0, 0, s, new HashMap<>(Map.of(0, 1)));
    }

    // mask := 26 bits that represent the parity of each character in the alphabet
    // on the path from node 0 to node u
    private long dfs(List<Integer>[] tree, int u, int mask, String s,
                     Map<Integer, Integer> maskToCount) {
        long res = 0;
        if (u > 0) {
            mask ^= 1 << (s.charAt(u) - 'a');
            // Consider any u-v path with 1 bit set.
            for (int i = 0; i < 26; ++i)
                if (maskToCount.containsKey(mask ^ (1 << i)))
                    res += maskToCount.get(mask ^ (1 << i));
            // Consider u-v path with 0 bit set.
            res += maskToCount.getOrDefault(mask ^ 0, 0);
            maskToCount.merge(mask, 1, Integer::sum);
        }
        for (final int v : tree[u])
            res += dfs(tree, v, mask, s, maskToCount);
        return res;
    }
}
```

1812. 2792.java

Path: LeetCode-main/solutions/2792. Count Nodes That Are Great Enough/2792.java

```
class Solution {
    public int countGreatEnoughNodes(TreeNode root, int k) {
        dfs(root, k);
        return ans;
    }

    private int ans = 0;

    private Queue<Integer> dfs(TreeNode root, int k) {
        if (root == null)
            return new PriorityQueue<>(Collections.reverseOrder());

        Queue<Integer> kSmallest = dfs(root.left, k);
        Queue<Integer> kSmallestRight = dfs(root.right, k);
        kSmallest.addAll(kSmallestRight);

        while (kSmallest.size() > k)
            kSmallest.poll();
        if (kSmallest.size() == k && root.val > kSmallest.peek())
            ++ans;

        kSmallest.offer(root.val);
        return kSmallest;
    }
}
```

1813. 2798.java

Path: LeetCode-main/solutions/2798. Number of Employees Who Met the Target/2798.java

```
class Solution {  
    public int numberOfEmployeesWhoMetTarget(int[] hours, int target) {  
        return (int) Arrays.stream(hours).filter(hour -> hour >= target).count();  
    }  
}
```


1814. 2799.java

Path: LeetCode-main/solutions/2799. Count Complete Subarrays in an Array/2799.java

```
class Solution {
    public int countCompleteSubarrays(int[] nums) {
        final int MAX = 2000;
        final int totalDistinct = Arrays.stream(nums).boxed().collect(Collectors.toSet()).size();
        int ans = 0;
        int distinct = 0;
        int[] count = new int[MAX + 1];

        int l = 0;
        for (final int num : nums) {
            if (++count[num] == 1)
                ++distinct;
            while (distinct == totalDistinct)
                if (--count[nums[l++]] == 0)
                    --distinct;
            // Assume nums[r] = num,
            // nums[0..r], nums[1..r], ..., nums[l - 1..r] have k different values.
            ans += l;
        }
        return ans;
    }
}
```

1815. 28.java

Path: LeetCode-main/solutions/28. Implement strStr()/28.java

```
class Solution {
    public int strStr(String haystack, String needle) {
        final int m = haystack.length();
        final int n = needle.length();

        for (int i = 0; i < m - n + 1; ++i)
            if (haystack.substring(i, i + n).equals(needle))
                return i;

        return -1;
    }
}
```

1816. 280.java

Path: LeetCode-main/solutions/280. Wiggle Sort/280.java

```
class Solution {
    public void wiggleSort(int[] nums) {
        // 1. If i is even, then nums[i] <= nums[i - 1].
        // 2. If i is odd, then nums[i] >= nums[i - 1].
        for (int i = 1; i < nums.length; ++i)
            if (i % 2 == 0 && nums[i] > nums[i - 1] || //
                i % 2 == 1 && nums[i] < nums[i - 1])
                swap(nums, i, i - 1);
    }

    private void swap(int[] nums, int i, int j) {
        final int temp = nums[i];
        nums[i] = nums[j];
        nums[j] = temp;
    }
}
```

1817. 2800.java

Path: LeetCode-main/solutions/2800. Shortest String That Contains Three Strings/2800.java

```
class Solution {
    public String minimumString(String a, String b, String c) {
        final String abc = merge(a, merge(b, c));
        final String acb = merge(a, merge(c, b));
        final String bac = merge(b, merge(a, c));
        final String bca = merge(b, merge(c, a));
        final String cab = merge(c, merge(a, b));
        final String cba = merge(c, merge(b, a));
        return getMin(Arrays.asList(abc, acb, bac, bca, cab, cba));
    }

    // Merges a and b.
    private String merge(String a, String b) {
        if (b.contains(a)) // a is a substring of b.
            return b;
        for (int i = 0; i < a.length(); ++i) {
            final String aSuffix = a.substring(i);
            final String bPrefix = b.substring(0, Math.min(b.length(), aSuffix.length()));
            if (aSuffix.equals(bPrefix))
                return a + b.substring(bPrefix.length());
        }
        return a + b;
    }

    // Returns the lexicographically smallest string.
    private String getMin(List<String> words) {
        String res = words.get(0);
        for (int i = 1; i < words.size(); ++i)
            res = getMin(res, words.get(i));
        return res;
    }

    // Returns the lexicographically smaller string.
    private String getMin(String a, String b) {
        return (a.length() < b.length() || (a.length() == b.length() && a.compareTo(b) < 0)) ? a : b;
    }
}
```

1818. 2801.java

Path: LeetCode-main/solutions/2801. Count Stepping Numbers in Range/2801.java

```
class Solution {
    public int countSteppingNumbers(String low, String high) {
        final String lowWithLeadingZeros =
            String.valueOf('0').repeat(high.length() - low.length()) + low;
        Integer[][][] mem = new Integer[high.length()][11][2][2];
        return count(lowWithLeadingZeros, high, 0, 10, /*isLeadingZero=*/true, true, true, mem);
    }

    private static final int MOD = 1_000_000_007;

    // Returns the number of valid integers, considering the i-th digit, where
    // `prevDigit` is the previous digit, `tight1` indicates if the current
    // digit is tightly bound for `low`, and `tight2` indicates if the current
    // digit is tightly bound for `high`.
    private int count(final String low, final String high, int i, int prevDigit,
        boolean isLeadingZero, boolean tight1, boolean tight2, Integer[][][] mem) {
        if (i == high.length())
            return 1;
        if (mem[i][prevDigit][tight1 ? 1 : 0][tight2 ? 1 : 0] != null)
            return mem[i][prevDigit][tight1 ? 1 : 0][tight2 ? 1 : 0];

        int res = 0;
        final int minDigit = tight1 ? low.charAt(i) - '0' : 0;
        final int maxDigit = tight2 ? high.charAt(i) - '0' : 9;

        for (int d = minDigit; d <= maxDigit; ++d) {
            final boolean nextTight1 = tight1 && (d == minDigit);
            final boolean nextTight2 = tight2 && (d == maxDigit);
            if (isLeadingZero)
                // Can place any digit in [minDigit, maxDigit].
                res += count(low, high, i + 1, d, isLeadingZero && d == 0, nextTight1, nextTight2, mem);
            else if (Math.abs(d - prevDigit) == 1)
                // Can only place prevDigit - 1 or prevDigit + 1.
                res += count(low, high, i + 1, d, false, nextTight1, nextTight2, mem);
            res %= MOD;
        }

        return mem[i][prevDigit][tight1 ? 1 : 0][tight2 ? 1 : 0] = res;
    }
}
```

1819. 2802.java

Path: LeetCode-main/solutions/2802. Find The K-th Lucky Number/2802.java

```
class Solution {
    public String kthLuckyNumber(int k) {
        StringBuilder sb = new StringBuilder();

        for (int i = k + 1; i > 0; i /= 2)
            sb.append(i % 2 == 0 ? '4' : '7');

        return sb.reverse().substring(1);
    }
}
```

1820. 2806.java

Path: LeetCode-main/solutions/2806. Account Balance After Rounded Purchase/2806.java

```
class Solution {  
    public int accountBalanceAfterPurchase(int purchaseAmount) {  
        return 100 - ((purchaseAmount + 5) / 10) * 10;  
    }  
}
```

1821. 2807.java

Path: LeetCode-main/solutions/2807. Insert Greatest Common Divisors in Linked List/2807.java

```
class Solution {
    public ListNode insertGreatestCommonDivisors(ListNode head) {
        for (ListNode curr = head; curr.next != null;) {
            ListNode inserted = new ListNode(gcd(curr.val, curr.next.val), curr.next);
            curr.next = inserted;
            curr = inserted.next;
        }
        return head;
    }

    private int gcd(int a, int b) {
        return b == 0 ? a : gcd(b, a % b);
    }
}
```


1822. 2808.java

Path: LeetCode-main/solutions/2808. Minimum Seconds to Equalize a Circular Array/2808.java

```
class Solution {
    public int minimumSeconds(List<Integer> nums) {
        int n = nums.size();
        int ans = n;
        Map<Integer, List<Integer>> numToIndices = new HashMap<>();

        for (int i = 0; i < n; ++i) {
            numToIndices.putIfAbsent(nums.get(i), new ArrayList<>());
            numToIndices.get(nums.get(i)).add(i);
        }

        for (List<Integer> indices : numToIndices.values()) {
            int seconds = getSeconds(indices.get(0) + n, indices.get(indices.size() - 1));
            for (int i = 1; i < indices.size(); ++i)
                seconds = Math.max(seconds, getSeconds(indices.get(i), indices.get(i - 1)));
            ans = Math.min(ans, seconds);
        }

        return ans;
    }

    // Returns the number of seconds required to make nums[i..j] the same.
    private int getSeconds(int i, int j) {
        return (i - j) / 2;
    }
}
```

1823. 2809-2.java

Path: LeetCode-main/solutions/2809. Minimum Time to Make Array Sum At Most x/2809-2.java

```
class Solution {
    public int minimumTime(List<Integer> nums1, List<Integer> nums2, int x) {
        final int n = nums1.size();
        final int sum1 = nums1.stream().mapToInt(Integer::intValue).sum();
        final int sum2 = nums2.stream().mapToInt(Integer::intValue).sum();
        // dp[j] := the maximum reduced value if we do j operations on the numbers
        // so far
        int[] dp = new int[n + 1];
        List<Pair<Integer, Integer>> sortedNums = new ArrayList<>();

        for (int i = 0; i < n; ++i)
            sortedNums.add(new Pair<>(nums2.get(i), nums1.get(i)));

        sortedNums.sort(Comparator.comparingInt(Pair::getKey));

        for (int i = 1; i <= n; ++i) {
            final int num2 = sortedNums.get(i - 1).getKey();
            final int num1 = sortedNums.get(i - 1).getValue();
            for (int j = i; j > 0; --j)
                dp[j] = Math.max(
                    // the maximum reduced value if we do j operations on the first
                    // i - 1 numbers
                    dp[j],
                    // the maximum reduced value if we do j - 1 operations on the first
                    // i - 1 numbers + making the i-th number of `nums1` to 0 at the
                    // j-th operation
                    dp[j - 1] + num2 * j + num1);
        }

        for (int op = 0; op <= n; ++op)
            if (sum1 + sum2 * op - dp[op] <= x)
                return op;

        return -1;
    }
}
```

1824. 2809.java

Path: LeetCode-main/solutions/2809. Minimum Time to Make Array Sum At Most x/2809.java

```
class Solution {
    public int minimumTime(List<Integer> nums1, List<Integer> nums2, int x) {
        final int n = nums1.size();
        final int sum1 = nums1.stream().mapToInt(Integer::intValue).sum();
        final int sum2 = nums2.stream().mapToInt(Integer::intValue).sum();
        // dp[i][j] := the maximum reduced value if we do j operations on the first
        // i numbers
        int[][] dp = new int[n + 1][n + 1];
        List<Pair<Integer, Integer>> sortedNums = new ArrayList<>();

        for (int i = 0; i < n; ++i)
            sortedNums.add(new Pair<>(nums2.get(i), nums1.get(i)));

        sortedNums.sort(Comparator.comparingInt(Pair::getKey));

        for (int i = 1; i <= n; ++i) {
            final int num2 = sortedNums.get(i - 1).getKey();
            final int num1 = sortedNums.get(i - 1).getValue();
            for (int j = 1; j <= i; ++j)
                dp[i][j] = Math.max(
                    // the maximum reduced value if we do j ops on the first i - 1 nums
                    dp[i - 1][j],
                    // the maximum reduced value if we do j - 1 ops on the first i - 1
                    // nums + making i-th num of nums1 to 0 at j-th operation
                    dp[i - 1][j - 1] + num2 * j + num1);
        }

        for (int op = 0; op <= n; ++op)
            if (sum1 + sum2 * op - dp[n][op] <= x)
                return op;

        return -1;
    }
}
```

1825. 281.java

Path: LeetCode-main/solutions/281. Zigzag Iterator/281.java

```
public class ZigzagIterator {
    public ZigzagIterator(List<Integer> v1, List<Integer> v2) {
        if (!v1.isEmpty())
            q.offer(v1.iterator());
        if (!v2.isEmpty())
            q.offer(v2.iterator());
    }

    public int next() {
        final Iterator it = q.poll();
        final int next = (int) it.next();
        if (it.hasNext())
            q.offer(it);
        return next;
    }

    public boolean hasNext() {
        return !q.isEmpty();
    }

    private Queue<Iterator> q = new ArrayDeque<>();
}
```

1826. 2810.java

Path: LeetCode-main/solutions/2810. Faulty Keyboard/2810.java

```
class Solution {
    public String finalString(String s) {
        StringBuilder sb = new StringBuilder();
        Deque<Character> dq = new ArrayDeque<>();
        boolean inversed = false;

        for (final char c : s.toCharArray())
            if (c == 'i')
                inversed = !inversed;
            else if (inversed)
                dq.offerFirst(c);
            else
                dq.offerLast(c);

        while (!dq.isEmpty())
            sb.append(dq.pollFirst());

        return inversed ? sb.reverse().toString() : sb.toString();
    }
}
```

1827. 2811.java

Path: LeetCode-main/solutions/2811. Check if it is Possible to Split Array/2811.java

```
class Solution {
    public boolean canSplitArray(List<Integer> nums, int m) {
        if (nums.size() < 3)
            return true;

        for (int i = 1; i < nums.size(); ++i)
            if (nums.get(i) + nums.get(i - 1) >= m)
                return true;

        return false;
    }
}
```

1828. 2812.java

Path: LeetCode-main/solutions/2812. Find the Safest Path in a Grid/2812.java

```
class Solution {
    public int maximumSafenessFactor(List<List<Integer>> grid) {
        int[][] distToThief = getDistToThief(grid);
        int l = 0;
        int r = grid.size() * 2;

        while (l < r) {
            final int m = (l + r) / 2;
            if (hasValidPath(distToThief, m))
                l = m + 1;
            else
                r = m;
        }

        return l - 1;
    }

    private static final int[][] DIRS = {{0, 1}, {1, 0}, {0, -1}, {-1, 0}};

    private boolean hasValidPath(int[][] distToThief, int safeness) {
        if (distToThief[0][0] < safeness)
            return false;

        final int n = distToThief.length;
        Queue<Pair<Integer, Integer>> q = new ArrayDeque<>(List.of(new Pair<>(0, 0)));
        boolean[][] seen = new boolean[n][n];
        seen[0][0] = true;

        while (!q.isEmpty()) {
            final int i = q.peek().getKey();
            final int j = q.poll().getValue();
            if (distToThief[i][j] < safeness)
                continue;
            if (i == n - 1 && j == n - 1)
                return true;
            for (int[] dir : DIRS) {
                final int x = i + dir[0];
                final int y = j + dir[1];
                if (x < 0 || x == n || y < 0 || y == n)
                    continue;
                if (seen[x][y])
                    continue;
                q.offer(new Pair<>(x, y));
                seen[x][y] = true;
            }
        }

        return false;
    }

    private int[][] getDistToThief(List<List<Integer>> grid) {
        final int n = grid.size();
        int[][] distToThief = new int[n][n];
        Queue<Pair<Integer, Integer>> q = new ArrayDeque<>();
        boolean[][] seen = new boolean[n][n];

        for (int i = 0; i < n; ++i)
            for (int j = 0; j < n; ++j)
                if (grid.get(i).get(j) == 1) {
                    q.offer(new Pair<>(i, j));
                    seen[i][j] = true;
                }

        for (int dist = 0; !q.isEmpty(); ++dist) {
            for (int sz = q.size(); sz > 0; --sz) {
                final int i = q.peek().getKey();
                final int j = q.poll().getValue();
                distToThief[i][j] = dist;
                for (int[] dir : DIRS) {
                    final int x = i + dir[0];
                    final int y = j + dir[1];
                    if (x < 0 || x == n || y < 0 || y == n)
                        continue;
                    if (seen[x][y])
                        continue;
                    q.offer(new Pair<>(x, y));
                    seen[x][y] = true;
                }
            }
        }
    }
}
```

```
        return distToThief;  
    }  
}
```


1829. 2813.java

Path: LeetCode-main/solutions/2813. Maximum Elegance of a K-Length Subsequence/2813.java

```
class Solution {
    public long findMaximumElegance(int[][] items, int k) {
        long ans = 0;
        long totalProfit = 0;
        Set<Integer> seenCategories = new HashSet<>();
        Deque<Integer> decreasingDuplicateProfits = new ArrayDeque<>();

        Arrays.sort(items, Comparator.comparingInt(item -> - item[0]));

        for (int i = 0; i < k; ++i) {
            final int profit = items[i][0];
            final int category = items[i][1];
            totalProfit += profit;
            if (seenCategories.contains(category))
                decreasingDuplicateProfits.push(profit);
            else
                seenCategories.add(category);
        }

        ans = totalProfit + 1L * seenCategories.size() * seenCategories.size();

        for (int i = k; i < items.length; ++i) {
            final int profit = items[i][0];
            final int category = items[i][1];
            if (!seenCategories.contains(category) && !decreasingDuplicateProfits.isEmpty()) {
                // If this is a new category we haven't seen before, it's worth
                // considering taking it and replacing the one with the least profit
                // since it will increase the distinct_categories and potentially result
                // in a larger total_profit + distinct_categories^2.
                totalProfit -= decreasingDuplicateProfits.pop();
                totalProfit += profit;
                seenCategories.add(category);
                ans = Math.max(ans, totalProfit + 1L * seenCategories.size() * seenCategories.size());
            }
        }

        return ans;
    }
}
```

1830. 2814.java

Path: LeetCode-main/solutions/2814. Minimum Time Takes to Reach Destination Without Drowning/2814.java

```
class Solution {
    public int minimumSeconds(List<List<String>> land) {
        final int m = land.size();
        final int n = land.get(0).size();
        final int[][] floodDist = getFloodDist(land);
        Queue<Pair<Integer, Integer>> q = new LinkedList<>();
        boolean[][] seen = new boolean[m][n];

        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j)
                if (land.get(i).get(j).equals("S")) {
                    q.offer(new Pair<>(i, j));
                    seen[i][j] = true;
                }

        for (int step = 1; !q.isEmpty(); ++step)
            for (int sz = q.size(); sz > 0; --sz) {
                final int i = q.peek().getKey();
                final int j = q.poll().getValue();
                for (int[] dir : DIRS) {
                    final int x = i + dir[0];
                    final int y = j + dir[1];
                    if (x < 0 || x == m || y < 0 || y == n)
                        continue;
                    if (land.get(x).get(y).equals("D"))
                        return step;
                    if (floodDist[x][y] <= step || land.get(x).get(y).equals("X") || seen[x][y])
                        continue;
                    q.offer(new Pair<>(x, y));
                    seen[x][y] = true;
                }
            }

        return -1;
    }

    private final int[][] DIRS = {{0, 1}, {1, 0}, {0, -1}, {-1, 0}};

    private int[][] getFloodDist(List<List<String>> land) {
        final int m = land.size();
        final int n = land.get(0).size();
        int[][] dist = new int[m][n];
        Queue<Pair<Integer, Integer>> q = new LinkedList<>();
        boolean[][] seen = new boolean[m][n];

        Arrays.stream(dist).forEach(A -> Arrays.fill(A, Integer.MAX_VALUE));

        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j)
                if (land.get(i).get(j).equals("*")) {
                    q.offer(new Pair<>(i, j));
                    seen[i][j] = true;
                }

        for (int d = 0; !q.isEmpty(); ++d)
            for (int sz = q.size(); sz > 0; --sz) {
                final int i = q.peek().getKey();
                final int j = q.poll().getValue();
                dist[i][j] = d;
                for (int[] dir : DIRS) {
                    final int x = i + dir[0];
                    final int y = j + dir[1];
                    if (x < 0 || x == m || y < 0 || y == n)
                        continue;
                    if (land.get(x).get(y).equals("X") || land.get(x).get(y).equals("D") || seen[x][y])
                        continue;
                    q.offer(new Pair<>(x, y));
                    seen[x][y] = true;
                }
            }

        return dist;
    }
}
```

1831. 2815.java

Path: LeetCode-main/solutions/2815. Max Pair Sum in an Array/2815.java

```
class Solution {
    public int maxSum(int[] nums) {
        int ans = 0;
        // maxNum[i] := the maximum num we met so far with the maximum digit i
        int[] maxNum = new int[10];

        for (final int num : nums) {
            final int d = getMaxDigit(num);
            if (maxNum[d] > 0)
                ans = Math.max(ans, num + maxNum[d]);
            maxNum[d] = Math.max(maxNum[d], num);
        }

        return ans == 0 ? -1 : ans;
    }

    private int getMaxDigit(int num) {
        int maxDigit = 0;
        while (num > 0) {
            maxDigit = Math.max(maxDigit, num % 10);
            num /= 10;
        }
        return maxDigit;
    }
}
```

1832. 2816-2.java

Path: LeetCode-main/solutions/2816. Double a Number Represented as a Linked List/2816-2.java

```
class Solution {
    public ListNode doubleIt(ListNode head) {
        if (head.val >= 5)
            head = new ListNode(0, head);

        for (ListNode curr = head; curr != null; curr = curr.next) {
            curr.val *= 2;
            curr.val %= 10;
            if (curr.next != null && curr.next.val >= 5)
                ++curr.val;
        }

        return head;
    }
}
```

1833. 2816.java

Path: LeetCode-main/solutions/2816. Double a Number Represented as a Linked List/2816.java

```
class Solution {
    public ListNode doubleIt(ListNode head) {
        if (getCarry(head) == 1)
            return new ListNode(1, head);
        return head;
    }

    private int getCarry(ListNode node) {
        int val = node.val * 2;
        if (node.next != null)
            val += getCarry(node.next);
        node.val = val % 10;
        return val / 10;
    }
}
```

1834. 2817.java

Path: LeetCode-main/solutions/2817. Minimum Absolute Difference Between Elements With Constraint/2817.java

```
class Solution {
    public int minAbsoluteDifference(List<Integer> nums, int x) {
        int ans = Integer.MAX_VALUE;
        TreeSet<Integer> seen = new TreeSet<>();

        for (int i = x; i < nums.size(); ++i) {
            seen.add(nums.get(i - x));
            Integer hi = seen.ceiling(nums.get(i));
            if (hi != null)
                ans = Math.min(ans, hi - nums.get(i));
            Integer lo = seen.floor(nums.get(i));
            if (lo != null)
                ans = Math.min(ans, nums.get(i) - lo);
        }

        return ans;
    }
}
```

1835. 2818.java

Path: LeetCode-main/solutions/2818. Apply Operations to Maximize Score/2818.java

```
class Solution {
    public int maximumScore(List<Integer> nums, int k) {
        final int n = nums.size();
        final int mx = Collections.max(nums);
        final int[] minPrimeFactors = sieveEratosthenes(mx + 1);
        final int[] primeScores = getPrimeScores(nums, minPrimeFactors);
        int ans = 1;
        // left[i] := the next index on the left (if any)
        // s.t. primeScores[left[i]] >= primeScores[i]
        int[] left = new int[n];
        Arrays.fill(left, -1);
        // right[i] := the next index on the right (if any)
        // s.t. primeScores[right[i]] > primeScores[i]
        int[] right = new int[n];
        Arrays.fill(right, n);
        Deque<Integer> stack = new ArrayDeque<>();

        // Find the next indices on the left where `primeScores` are greater or equal.
        for (int i = n - 1; i >= 0; --i) {
            while (!stack.isEmpty() && primeScores[stack.peek()] <= primeScores[i])
                left[stack.pop()] = i;
            stack.push(i);
        }

        stack.clear();

        // Find the next indices on the right where `primeScores` are greater.
        for (int i = 0; i < n; ++i) {
            while (!stack.isEmpty() && primeScores[stack.peek()] < primeScores[i])
                right[stack.pop()] = i;
            stack.push(i);
        }

        Pair<Integer, Integer>[] numAndIndexes = new Pair[n];

        for (int i = 0; i < n; ++i)
            numAndIndexes[i] = new Pair<>(nums.get(i), i);

        Arrays.sort(numAndIndexes,
            Comparator.comparing(Pair<Integer, Integer>::getKey, Comparator.reverseOrder())
                .thenComparingInt(Pair<Integer, Integer>::getValue));

        for (Pair<Integer, Integer> numAndIndex : numAndIndexes) {
            final int num = numAndIndex.getKey();
            final int i = numAndIndex.getValue();
            // nums[i] is the maximum value in the range [left[i] + 1, right[i] - 1]
            // So, there are (i - left[i]) * (right[i] - 1) ranges where nums[i] will
            // be chosen.
            final long rangeCount = (long) (i - left[i]) * (right[i] - i);
            final long actualCount = Math.min(rangeCount, (long) k);
            k -= actualCount;
            ans = (int) ((1L * ans * modPow(num, actualCount)) % MOD);
        }

        return ans;
    }

    private static final int MOD = 1_000_000_007;

    private long modPow(long x, long n) {
        if (n == 0)
            return 1;
        if (n % 2 == 1)
            return x * modPow(x, n - 1) % MOD;
        return modPow(x * x % MOD, n / 2);
    }

    // Gets the minimum prime factor of i, where 1 < i <= n.
    private int[] sieveEratosthenes(int n) {
        int[] minPrimeFactors = new int[n + 1];
        for (int i = 2; i <= n; ++i)
            minPrimeFactors[i] = i;
        for (int i = 2; i * i < n; ++i)
            if (minPrimeFactors[i] == i) // `i` is prime.
                for (int j = i * i; j < n; j += i)
                    minPrimeFactors[j] = Math.min(minPrimeFactors[j], i);
        return minPrimeFactors;
    }

    private int[] getPrimeScores(List<Integer> nums, int[] minPrimeFactors) {
```

```

    int[] primeScores = new int[nums.size()];
    for (int i = 0; i < nums.size(); ++i)
        primeScores[i] = getPrimeScore(nums.get(i), minPrimeFactors);
    return primeScores;
}

private int getPrimeScore(int num, int[] minPrimeFactors) {
    Set<Integer> primeFactors = new HashSet<>();
    while (num > 1) {
        final int divisor = minPrimeFactors[num];
        primeFactors.add(divisor);
        while (num % divisor == 0)
            num /= divisor;
    }
    return primeFactors.size();
}
}

```


1836. 2819.java

Path: LeetCode-main/solutions/2819. Minimum Relative Loss After Buying Chocolates/2819.java

```
class Solution {
    long[] minimumRelativeLosses(int[] prices, int[][] queries) {
        final int n = prices.length;
        long[] ans = new long[queries.length];
        long[] prefix = new long[prices.length + 1];

        Arrays.sort(prices);

        for (int i = 0; i < prices.length; ++i)
            prefix[i + 1] = prefix[i] + prices[i];

        for (int i = 0; i < queries.length; ++i) {
            final int k = queries[i][0];
            final int m = queries[i][1];
            final int countFront = getCountFront(k, m, prices);
            final int countBack = m - countFront;
            ans[i] = getRelativeLoss(countFront, countBack, k, prefix);
        }

        return ans;
    }

    // Returns `countFront` for query (k, m) s.t. picking the first `countFront`
    // and the last `m - countFront` chocolates is optimal.
    //
    // Define loss[i] := the relative loss of picking `prices[i]`.
    // 1. For prices[i] <= k, Bob pays prices[i] while Alice pays 0.
    //    Thus, loss[i] = prices[i] - 0 = prices[i].
    // 2. For prices[i] > k, Bob pays k while Alice pays prices[i] - k.
    //    Thus, loss[i] = k - (prices[i] - k) = 2 * k - prices[i].
    // By observation, we deduce that it is always better to pick from the front
    // or the back since loss[i] is increasing for 1. and is decreasing for 2.
    //
    // Assume that picking `left` chocolates from the left and `right = m - left`
    // chocolates from the right is optimal. Therefore, we are selecting
    // chocolates from `prices[0..left - 1]` and `prices[n - right..n - 1]`.
    //
    // To determine the optimal `left` in each iteration, we simply compare
    // `loss[left]` with `loss[n - right]`; if `loss[left] < loss[n - right]`,
    // it's worth increasing `left`.
    private int getCountFront(int k, int m, int[] prices) {
        final int n = prices.length;
        final int countNoGreaterThanOrK = firstGreater(prices, k);
        int l = 0;
        int r = Math.min(countNoGreaterThanOrK, m);

        while (l < r) {
            final int mid = (l + r) / 2;
            final int right = m - mid;
            // Picking prices[mid] is better than picking prices[n - right].
            if (prices[mid] < 2L * k - prices[n - right])
                l = mid + 1;
            else
                r = mid;
        }

        return l;
    }

    private int firstGreater(int[] arr, int target) {
        final int i = Arrays.binarySearch(arr, target + 1);
        return i < 0 ? -i - 1 : i;
    }

    // Returns the relative loss of picking `countFront` and `countBack`
    // chocolates.
    private long getRelativeLoss(int countFront, int countBack, int k, long[] prefix) {
        final int n = prefix.length - 1;
        final long lossFront = prefix[countFront];
        final long lossBack = 2L * k * countBack - (prefix[n] - prefix[n - countBack]);
        return lossFront + lossBack;
    }
}
```

1837. 282.java

Path: LeetCode-main/solutions/282. Expression Add Operators/282.java

```
class Solution {
    public List<String> addOperators(String num, int target) {
        List<String> ans = new ArrayList<>();
        dfs(num, target, 0, 0, 0, new StringBuilder(), ans);
        return ans;
    }

    private void dfs(String num, int target, int s, long prev, long eval, StringBuilder sb,
                     List<String> ans) {
        if (s == num.length()) {
            if (eval == target)
                ans.add(sb.toString());
            return;
        }

        for (int i = s; i < num.length(); ++i) {
            if (i > s && num.charAt(s) == '0')
                return;
            final long curr = Long.parseLong(num.substring(s, i + 1));
            final int length = sb.length();
            if (s == 0) { // the first number
                dfs(num, target, i + 1, curr, curr, sb.append(curr), ans);
                sb.setLength(length);
            } else {
                dfs(num, target, i + 1, curr, eval + curr, sb.append("+").append(curr), ans);
                sb.setLength(length);
                dfs(num, target, i + 1, -curr, eval - curr, sb.append("-").append(curr), ans);
                sb.setLength(length);
                dfs(num, target, i + 1, prev * curr, eval - prev + prev * curr, sb.append("*").append(curr),
                    ans);
                sb.setLength(length);
            }
        }
    }
}
```


1839. 2825.java

Path: LeetCode-main/solutions/2825. Make String a Subsequence Using Cyclic Increments/2825.java

```
class Solution {
    public boolean canMakeSubsequence(String str1, String str2) {
        int i = 0; // str2's index

        for (final char c : str1.toCharArray())
            if (c == str2.charAt(i) || ('a' + ((c - 'a' + 1) % 26)) == str2.charAt(i))
                if (++i == str2.length())
                    return true;

        return false;
    }
}
```

1840. 2826.java

Path: LeetCode-main/solutions/2826. Sorting Three Groups/2826.java

```
class Solution {
    public int minimumOperations(List<Integer> nums) {
        // dp[i] := the longest non-decreasing subsequence so far with numbers in [1..i]
        int[] dp = new int[4];

        for (final int num : nums) {
            ++dp[num];
            dp[2] = Math.max(dp[2], dp[1]);
            dp[3] = Math.max(dp[3], dp[2]);
        }

        return nums.size() - dp[3];
    }
}
```

1841. 2828.java

Path: LeetCode-main/solutions/2828. Check if a String Is an Acronym of Words/2828.java

```
class Solution {  
    public boolean isAcronym(List<String> words, String s) {  
        if (words.size() != s.length())  
            return false;  
  
        for (int i = 0; i < words.size(); ++i)  
            if (words.get(i).charAt(0) != s.charAt(i))  
                return false;  
  
        return true;  
    }  
}
```

1842. 2829.java

Path: LeetCode-main/solutions/2829. Determine the Minimum Sum of a k-avoiding Array/2829.java

```
class Solution {
    public int minimumSum(int n, int k) {
        // These are the unique pairs that sum up to k:
        // (1, k - 1), (2, k - 2), ..., (ceil(k / 2), floor(k / 2)).
        // Our optimal strategy is to select 1, 2, ..., floor(k / 2), and then
        // choose k, k + 1, ... if necessary, as selecting any number in the range
        // [ceil(k / 2), k - 1] will result in a pair summing up to k.
        final int mid = k / 2; // floor(k / 2)
        if (n <= mid)
            return trapezoid(1, n);
        return trapezoid(1, mid) + trapezoid(k, k + (n - mid - 1));
    }

    // Returns sum(a..b).
    private int trapezoid(int a, int b) {
        return (a + b) * (b - a + 1) / 2;
    }
}
```

1843. 283.java

Path: LeetCode-main/solutions/283. Move Zeroes/283.java

```
class Solution {  
    public void moveZeroes(int[] nums) {  
        int i = 0;  
        for (final int num : nums)  
            if (num != 0)  
                nums[i++] = num;  
  
        while (i < nums.length)  
            nums[i++] = 0;  
    }  
}
```


1844. 2830.java

Path: LeetCode-main/solutions/2830. Maximize the Profit as the Salesman/2830.java

```
class Solution {
    public int maximizeTheProfit(int n, List<List<Integer>> offers) {
        // dp[i] := the maximum amount of gold of selling the first i houses
        int[] dp = new int[n + 1];
        List<Pair<Integer, Integer>>[] endToStartAndGolds = new List[n];

        for (int i = 0; i < n; ++i)
            endToStartAndGolds[i] = new ArrayList<>();

        for (List<Integer> offer : offers) {
            final int start = offer.get(0);
            final int end = offer.get(1);
            final int gold = offer.get(2);
            endToStartAndGolds[end].add(new Pair<>(start, gold));
        }

        for (int end = 1; end <= n; ++end) {
            // Get at least the same gold as selling the first `end - 1` houses.
            dp[end] = dp[end - 1];
            for (Pair<Integer, Integer> pair : endToStartAndGolds[end - 1]) {
                final Integer start = pair.getKey();
                final Integer gold = pair.getValue();
                dp[end] = Math.max(dp[end], dp[start] + gold);
            }
        }

        return dp[n];
    }
}
```

1845. 2831-2.java

Path: LeetCode-main/solutions/2831. Find the Longest Equal Subarray/2831-2.java

```
class Solution {
    public int longestEqualSubarray(List<Integer> nums, int k) {
        int ans = 0;
        Map<Integer, Integer> count = new HashMap<>();

        // l and r track the maximum window instead of the valid window.
        for (int l = 0, r = 0; r < nums.size(); ++r) {
            ans = Math.max(ans, count.merge(nums.get(r), 1, Integer::sum));
            if (r - l + 1 - k > ans)
                count.merge(nums.get(l++), -1, Integer::sum);
        }

        return ans;
    }
}
```

1846. 2831.java

Path: LeetCode-main/solutions/2831. Find the Longest Equal Subarray/2831.java

```
class Solution {
    public int longestEqualSubarray(List<Integer> nums, int k) {
        int ans = 0;
        Map<Integer, Integer> count = new HashMap<>();

        for (int l = 0, r = 0; r < nums.size(); ++r) {
            ans = Math.max(ans, count.merge(nums.get(r), 1, Integer::sum));
            while (r - l + 1 - k > ans)
                count.merge(nums.get(l++), -1, Integer::sum);
        }

        return ans;
    }
}
```

1847. 2832.java

Path: LeetCode-main/solutions/2832. Maximal Range That Each Element Is Maximum in It/2832.java

```
class Solution {
    public int[] maximumLengthOfRanges(int[] nums) {
        int[] ans = new int[nums.length];
        Deque<Integer> stack = new ArrayDeque<>(); // a decreasing stack

        for (int i = 0; i <= nums.length; ++i) {
            while (!stack.isEmpty() && (i == nums.length || nums[stack.peek()] < nums[i])) {
                final int index = stack.pop();
                final int left = stack.isEmpty() ? -1 : stack.peek();
                ans[index] = i - left - 1;
            }
            stack.push(i);
        }

        return ans;
    }
}
```

1848. 2833.java

Path: LeetCode-main/solutions/2833. Furthest Point From Origin/2833.java

```
class Solution {
    public int furthestDistanceFromOrigin(String moves) {
        int countL = 0;
        int countR = 0;
        int countUnderline = 0;

        for (final char c : moves.toCharArray())
            if (c == 'L')
                ++countL;
            else if (c == 'R')
                ++countR;
            else // c == '_'
                ++countUnderline;

        return Math.abs(countL - countR) + countUnderline;
    }
}
```

1849. 2834.java

Path: LeetCode-main/solutions/2834. Find the Minimum Possible Sum of a Beautiful Array/2834.java

```
class Solution {
    // Same as 2829. Determine the Minimum Sum of a k-avoiding Array
    public int minimumPossibleSum(int n, int target) {
        // These are the unique pairs that sum up to target (k):
        // (1, k - 1), (2, k - 2), ..., (ceil(k / 2), floor(k / 2)).
        // Our optimal strategy is to select 1, 2, ..., floor(k / 2), and then
        // choose k, k + 1, ... if necessary, as selecting any number in the range
        // [ceil(k / 2), k - 1] will result in a pair summing up to k.
        final int mid = target / 2; // floor(k / 2)
        if (n <= mid)
            return (int) trapezoid(1, n);
        return (int) ((trapezoid(1, mid) + trapezoid(target, target + (n - mid - 1))) % MOD);
    }

    private static final int MOD = 1_000_000_007;

    // Returns sum(a..b).
    private long trapezoid(long a, long b) {
        return (a + b) * (b - a + 1) / 2;
    }
}
```

1850. 2835.java

Path: LeetCode-main/solutions/2835. Minimum Operations to Form Subsequence With Target Sum/2835.java

```
class Solution {
    public int minOperations(List<Integer> nums, int target) {
        final int NO_MISSING_BIT = 31;
        final int maxBit = 31;
        int ans = 0;
        int minMissingBit = NO_MISSING_BIT;
        // count[i] := the number of occurrences of 2^i
        int[] count = new int[maxBit + 1];

        for (final int num : nums)
            ++count[(int) (Math.log(num) / Math.log(2))];

        for (int bit = 0; bit < maxBit; ++bit) {
            // Check if `bit` is in the target.
            if ((target >> bit & 1) == 1) {
                // If there are available bits, use one bit.
                if (count[bit] > 0)
                    --count[bit];
                else
                    minMissingBit = Math.min(minMissingBit, bit);
            }
            // If we previously missed a bit and there are available bits.
            if (minMissingBit != NO_MISSING_BIT && count[bit] > 0) {
                --count[bit];
                // Count the operations to break `bit` into `minMissingBit`.
                ans += bit - minMissingBit;
                minMissingBit = NO_MISSING_BIT; // Set it to an the invalid value.
            }
            // Combining smaller numbers costs nothing.
            count[bit + 1] += count[bit] / 2;
        }

        // Check if all target bits have been covered, otherwise return -1.
        return minMissingBit == NO_MISSING_BIT ? ans : -1;
    }
}
```

1851. 2836.java

Path: LeetCode-main/solutions/2836. Maximize Value of Function in a Ball Passing Game/2836.java

```
class Solution {
    public long getMaxFunctionValue(List<Integer> receiver, long k) {
        final int n = receiver.size();
        final int m = (int) (Math.log(k) / Math.log(2)) + 1;
        long ans = 0;
        // jump[i][j] := the the node you reach after jumping 2^j steps from i
        int[][] jump = new int[n][m];
        // sum[i][j] := the sum of the first 2^j nodes you reach when jumping from i
        long[][] sum = new long[n][m];

        for (int i = 0; i < n; ++i) {
            jump[i][0] = receiver.get(i);
            sum[i][0] = receiver.get(i);
        }

        // Calculate binary lifting.
        for (int j = 1; j < m; ++j)
            for (int i = 0; i < n; ++i) {
                final int midNode = jump[i][j - 1];
                // the the node you reach after jumping 2^j steps from i
                // = the node you reach after jumping 2^(j - 1) steps from i
                // + the node you reach after jumping another 2^(j - 1) steps
                jump[i][j] = jump[midNode][j - 1];
                // the sum of the first 2^j nodes you reach when jumping from i
                // = the sum of the first 2^(j - 1) nodes you reach when jumping from i
                // + the sum of another 2^(j - 1) nodes you reach
                sum[i][j] = sum[i][j - 1] + sum[midNode][j - 1];
            }

        for (int i = 0; i < n; ++i) {
            long currSum = i;
            int currPos = i;
            for (int j = 0; j < m; ++j)
                if ((k >> j & 1) == 1) {
                    currSum += sum[currPos][j];
                    currPos = jump[currPos][j];
                }
            ans = Math.max(ans, currSum);
        }

        return ans;
    }
}
```


1852. 2838.java

Path: LeetCode-main/solutions/2838. Maximum Coins Heroes Can Collect/2838.java

```
class Solution {
    public long[] maximumCoins(int[] heroes, int[] monsters, int[] coins) {
        Pair<Integer, Integer>[] monsterAndCoins = getSortedMonsterAndCoins(monsters, coins);
        long[] ans = new long[heroes.length];
        long[] coinsPrefix = new long[coins.length + 1];

        for (int i = 0; i < monsterAndCoins.length; ++i)
            coinsPrefix[i + 1] = coinsPrefix[i] + monsterAndCoins[i].getValue();

        for (int i = 0; i < heroes.length; ++i)
            ans[i] = coinsPrefix[firstGreaterEqual(monsterAndCoins, heroes[i])];

        return ans;
    }

    private Pair<Integer, Integer>[] getSortedMonsterAndCoins(int[] monsters, int[] coins) {
        Pair<Integer, Integer>[] monsterAndCoins = new Pair[monsters.length];
        for (int i = 0; i < monsters.length; ++i)
            monsterAndCoins[i] = new Pair<>(monsters[i], coins[i]);
        Arrays.sort(monsterAndCoins, Comparator.comparingInt(Pair::getKey));
        return monsterAndCoins;
    }

    private int firstGreaterEqual(Pair<Integer, Integer>[] monsterAndCoins, int hero) {
        int l = 0;
        int r = monsterAndCoins.length;
        while (l < r) {
            final int m = (l + r) / 2;
            if (monsterAndCoins[m].getKey() > hero)
                r = m;
            else
                l = m + 1;
        }
        return l;
    }
}
```

1853. 2839.java

Path: LeetCode-main/solutions/2839. Check if Strings Can be Made Equal With Operations I/2839.java

```
class Solution {
    public boolean canBeEqual(String s1, String s2) {
        for (final String a : swappedStrings(s1))
            for (final String b : swappedStrings(s2))
                if (a.equals(b))
                    return true;
        return false;
    }

    private List<String> swappedStrings(final String s) {
        List<String> res = new ArrayList<>();
        char[] chars = s.toCharArray();
        res.add(s);
        res.add(new String(new char[] {chars[2], chars[1], chars[0], chars[3]}));
        res.add(new String(new char[] {chars[0], chars[3], chars[2], chars[1]}));
        res.add(new String(new char[] {chars[2], chars[3], chars[0], chars[1]}));
        return res;
    }
}
```

1854. 284.java

Path: LeetCode-main/solutions/284. Peeking Iterator/284.java

```
// Java Iterator interface reference:
// https://docs.oracle.com/javase/8/docs/api/java/util/Iterator.html

class PeekingIterator implements Iterator<Integer> {
    public PeekingIterator(Iterator<Integer> iterator) {
        this.iterator = iterator;
        buffer = iterator.hasNext() ? iterator.next() : null;
    }

    // Returns the next element in the iteration without advancing the iterator.
    public Integer peek() {
        return buffer;
    }

    // hasNext() and next() should behave the same as in the Iterator interface.
    // Override them if needed.
    @Override
    public Integer next() {
        Integer next = buffer;
        buffer = iterator.hasNext() ? iterator.next() : null;
        return next;
    }

    @Override
    public boolean hasNext() {
        return buffer != null;
    }

    private Iterator<Integer> iterator;
    private Integer buffer;
}
```

1855. 2840.java

Path: LeetCode-main/solutions/2840. Check if Strings Can be Made Equal With Operations II/2840.java

```
class Solution {
    public boolean checkStrings(String s1, String s2) {
        int[][] count = new int[2][26];

        for (int i = 0; i < s1.length(); ++i) {
            ++count[i % 2][s1.charAt(i) - 'a'];
            --count[i % 2][s2.charAt(i) - 'a'];
        }

        for (int i = 0; i < 26; ++i)
            if (count[0][i] > 0 || count[1][i] > 0)
                return false;

        return true;
    }
}
```

1856. 2841.java

Path: LeetCode-main/solutions/2841. Maximum Sum of Almost Unique Subarray/2841.java

```
class Solution {
    public long maxSum(List<Integer> nums, int m, int k) {
        long ans = 0;
        long sum = 0;
        Map<Integer, Integer> count = new HashMap<>();

        for (int i = 0; i < nums.size(); ++i) {
            sum += nums.get(i);
            count.merge(nums.get(i), 1, Integer::sum);
            if (i >= k) {
                final int numToRemove = nums.get(i - k);
                sum -= numToRemove;
                count.merge(numToRemove, -1, Integer::sum);
                if (count.get(numToRemove) == 0)
                    count.remove(numToRemove);
            }
            if (count.size() >= m)
                ans = Math.max(ans, sum);
        }

        return ans;
    }
}
```

1857. 2842.java

Path: LeetCode-main/solutions/2842. Count K-Subsequences of a String With Maximum Beauty Solved/2842.java

```
class Solution {
    public int countKSubsequencesWithMaxBeauty(String s, int k) {
        Map<Character, Integer> count = new HashMap<>();
        for (final char c : s.toCharArray())
            count.merge(c, 1, Integer::sum);
        if (count.size() < k)
            return 0;

        long ans = 1;

        for (Pair<Integer, Integer> pair : getFreqCountPairs(count)) {
            final int fc = pair.getKey();
            final int numOfChars = pair.getValue();
            if (numOfChars >= k) {
                ans *= nCk(numOfChars, k) * modPow(fc, k);
                return (int) (ans % MOD);
            }
            ans *= modPow(fc, numOfChars);
            ans %= MOD;
            k -= numOfChars;
        }

        return (int) ans;
    }

    private static final int MOD = 1_000_000_007;

    private List<Pair<Integer, Integer>> getFreqCountPairs(Map<Character, Integer> count) {
        // freqCount := (f(c), # of chars with f(c))
        Map<Integer, Integer> freqCount = new HashMap<>();
        for (final int value : count.values())
            freqCount.merge(value, 1, Integer::sum);
        List<Pair<Integer, Integer>> freqCountPairs = new ArrayList<>();
        for (Map.Entry<Integer, Integer> entry : freqCount.entrySet())
            freqCountPairs.add(new Pair<>(entry.getKey(), entry.getValue()));
        freqCountPairs.sort((a, b) -> b.getKey().compareTo(a.getKey()));
        return freqCountPairs;
    }

    private long nCk(int n, int k) {
        long res = 1;
        for (int i = 1; i <= k; ++i)
            res = res * (n - i + 1) / i;
        return res;
    }

    private int modPow(long x, long n) {
        if (n == 0)
            return 1;
        if (n % 2 == 1)
            return (int) (x * modPow(x % MOD, (n - 1)) % MOD);
        return modPow(x * x % MOD, (n / 2)) % MOD;
    }
}
```

1858. 2843.java

Path: LeetCode-main/solutions/2843. Count Symmetric Integers/2843.java

```
class Solution {
    public int countSymmetricIntegers(int low, int high) {
        int ans = 0;

        for (int num = low; num <= high; ++num)
            if (isSymmetricInteger(num))
                ++ans;

        return ans;
    }

    private boolean isSymmetricInteger(int num) {
        if (num >= 10 && num <= 99)
            return num / 10 == num % 10;
        if (num >= 1000 && num <= 9999) {
            final int left = num / 100;
            final int right = num % 100;
            return left / 10 + left % 10 == right / 10 + right % 10;
        }
        return false;
    }
}
```

1859. 2844.java

Path: LeetCode-main/solutions/2844. Minimum Operations to Make a Special Number/2844.java

```
class Solution {
    public int minimumOperations(String num) {
        final int n = num.length();
        boolean seenFive = false;
        boolean seenZero = false;

        for (int i = n - 1; i >= 0; --i) {
            if (seenZero && num.charAt(i) == '0') // '00'
                return n - i - 2;
            if (seenZero && num.charAt(i) == '5') // '50'
                return n - i - 2;
            if (seenFive && num.charAt(i) == '2') // '25'
                return n - i - 2;
            if (seenFive && num.charAt(i) == '7') // '75'
                return n - i - 2;
            seenZero = seenZero || num.charAt(i) == '0';
            seenFive = seenFive || num.charAt(i) == '5';
        }

        return seenZero ? n - 1 : n;
    }
}
```


1860. 2845.java

Path: LeetCode-main/solutions/2845. Count of Interesting Subarrays/2845.java

```
class Solution {
    public long countInterestingSubarrays(List<Integer> nums, int modulo, int k) {
        long ans = 0;
        int prefix = 0; // (number of nums[i] % modulo == k so far) % modulo
        Map<Integer, Integer> prefixCount = new HashMap<>();
        prefixCount.put(0, 1);

        for (final int num : nums) {
            if (num % modulo == k)
                prefix = (prefix + 1) % modulo;
            ans += prefixCount.getDefault((prefix - k + modulo) % modulo, 0);
            prefixCount.merge(prefix, 1, Integer::sum);
        }

        return ans;
    }
}
```

1861. 2846.java

Path: LeetCode-main/solutions/2846. Minimum Edge Weight Equilibrium Queries in a Tree/2846.java

```
class Solution {
    public int[] minOperationsQueries(int n, int[][] edges, int[][] queries) {
        final int MAX = 26;
        final int m = (int) Math.ceil(Math.log(n) / Math.log(2));
        int[] ans = new int[queries.length];
        List
```

1862. 2847.java

Path: LeetCode-main/solutions/2847. Smallest Number With Given Digit Product/2847.java

```
class Solution {
    public String smallestNumber(long n) {
        if (n <= 9)
            return Long.toString(n);

        StringBuilder ans = new StringBuilder();

        for (int divisor = 9; divisor > 1; --divisor)
            while (n % divisor == 0) {
                ans.append(Integer.toString(divisor));
                n /= divisor;
            }

        return n > 1 ? "-1" : ans.reverse().toString();
    }
}
```

1863. 2848.java

Path: LeetCode-main/solutions/2848. Points That Intersect With Cars/2848.java

```
class Solution {
    public int numberOfPoints(List<List<Integer>> nums) {
        final int MAX = 100;
        int ans = 0;
        int runningSum = 0;
        int[] count = new int[MAX + 2];

        for (List<Integer> num : nums) {
            final int start = num.get(0);
            final int end = num.get(1);
            ++count[start];
            --count[end + 1];
        }

        for (int i = 1; i <= MAX; i++) {
            runningSum += count[i];
            if (runningSum > 0)
                ++ans;
        }

        return ans;
    }
}
```

1864. 2849.java

Path: LeetCode-main/solutions/2849. Determine if a Cell Is Reachable at a Given Time/2849.java

```
class Solution {  
    public boolean isReachableAtTime(int sx, int sy, int fx, int fy, int t) {  
        final int minStep = Math.max(Math.abs(sx - fx), Math.abs(sy - fy));  
        return minStep == 0 ? t != 1 : minStep <= t;  
    }  
}
```

1865. 285.java

Path: LeetCode-main/solutions/285. Inorder Successor in BST/285.java

```
class Solution {
    public TreeNode inorderSuccessor(TreeNode root, TreeNode p) {
        if (root == null)
            return null;
        if (root.val <= p.val)
            return inorderSuccessor(root.right, p);

        TreeNode left = inorderSuccessor(root.left, p);
        return left == null ? root : left;
    }
}
```


1867. 2851.java

Path: LeetCode-main/solutions/2851. String Transformation/2851.java

```
class Solution {
    // This dynamic programming table dp[k][i] represents the number of ways to
    // rearrange the String s after k steps such that it starts with s[i].
    // A String can be rotated from 1 to n - 1 times. The transition rule is
    // dp[k][i] = sum(dp[k - 1][j]) for all j != i. For example, when n = 4 and
    // k = 3, the table looks like this:
    //
    // -----
    // |           | i = 0 | i = 1 | i = 2 | i = 3 | sum = (n - 1)^k |
    // -----
    // | k = 0 |   1   |   0   |   0   |   0   |         1         |
    // | k = 1 |   0   |   1   |   1   |   1   |         3         |
    // | k = 2 |   3   |   2   |   2   |   2   |         9         |
    // | k = 3 |   6   |   7   |   7   |   7   |        27         |
    // -----
    //
    // By observation, we have
    // * dp[k][!0] = ((n - 1)^k - (-1)^k) / n
    // * dp[k][0] = dp[k][!0] + (-1)^k
    public int numberOfWays(String s, String t, long k) {
        final int n = s.length();
        final int negOnePowK = (k % 2 == 0 ? 1 : -1); // (-1)^k
        final int[] z = zFunction(s + t + t);
        final List<Integer> indices = getIndices(z, n);
        int[] dp = new int[2]; // dp[0] := dp[k][0]; dp[1] := dp[k][!0]
        dp[1] = (int) ((modPow(n - 1, k) - negOnePowK + MOD) % MOD * modPow(n, MOD - 2) % MOD);
        dp[0] = (int) ((dp[1] + negOnePowK + MOD) % MOD);
        int ans = 0;
        for (final int index : getIndices(z, n)) {
            ans += dp[index == 0 ? 0 : 1];
            ans %= MOD;
        }
        return ans;
    }

    private static final int MOD = 1_000_000_007;

    private long modPow(long x, long n) {
        if (n == 0)
            return 1;
        if (n % 2 == 1)
            return x * modPow(x, n - 1) % MOD;
        return modPow(x * x % MOD, n / 2);
    }

    // Returns the z array, where z[i] is the length of the longest prefix of
    // s[i..n) which is also a prefix of s.
    //
    // https://cp-algorithms.com/string/z-function.html#implementation
    private int[] zFunction(final String s) {
        final int n = s.length();
        int[] z = new int[n];
        int l = 0;
        int r = 0;
        for (int i = 1; i < n; ++i) {
            if (i < r)
                z[i] = Math.min(r - i, z[i - l]);
            while (i + z[i] < n && s.charAt(z[i]) == s.charAt(i + z[i]))
                ++z[i];
            if (i + z[i] > r) {
                l = i;
                r = i + z[i];
            }
        }
        return z;
    }

    // Returns the indices in `s` s.t. for each `i` in the returned indices,
    // `s[i..n) + s[0..i) = t`.
    private List<Integer> getIndices(int[] z, int n) {
        List<Integer> indices = new ArrayList<>();
        for (int i = n; i < n + n; ++i)
            if (z[i] >= n)
                indices.add(i - n);
        return indices;
    }
}
```


1868. 2852.java

Path: LeetCode-main/solutions/2852. Sum of Remoteness of All Cells/2852.java

```
class Solution {
    public long sumRemoteness(int[][] grid) {
        long ans = 0;
        long sum = 0;

        for (int[] row : grid)
            for (int cell : row)
                sum += Math.max(0, cell);

        for (int i = 0; i < grid.length; ++i)
            for (int j = 0; j < grid[0].length; ++j)
                if (grid[i][j] > 0) {
                    long[] res = dfs(grid, i, j);
                    final long count = res[0];
                    final long componentSum = res[1];
                    ans += (sum - componentSum) * count;
                }

        return ans;
    }

    private static final int[][] DIRS = {{0, 1}, {1, 0}, {0, -1}, {-1, 0}};

    // Returns the (count, componentSum) of the connected component that contains
    // (x, y).
    private long[] dfs(int[][] grid, int i, int j) {
        if (i < 0 || i == grid.length || j < 0 || j == grid[0].length)
            return new long[] {0, 0};
        if (grid[i][j] == -1)
            return new long[] {0, 0};

        long count = 1;
        long componentSum = grid[i][j];
        grid[i][j] = -1; // Mark as visited.;

        for (int[] dir : DIRS) {
            final int x = i + dir[0];
            final int y = j + dir[1];
            long[] nextRes = dfs(grid, x, y);
            final long nextCount = nextRes[0];
            final long nextComponentSum = nextRes[1];
            count += nextCount;
            componentSum += nextComponentSum;
        }

        return new long[] {count, componentSum};
    }
}
```

1869. 2855.java

Path: LeetCode-main/solutions/2855. Minimum Right Shifts to Sort the Array/2855.java

```
class Solution {
    public int minimumRightShifts(List<Integer> nums) {
        int pivot = -1;
        int count = 0;

        for (int i = 0; i + 1 < nums.size(); i++)
            if (nums.get(i) > nums.get(i + 1)) {
                ++count;
                pivot = i;
            }

        if (count == 0)
            return 0;
        if (count > 1 || nums.get(nums.size() - 1) > nums.get(0))
            return -1;
        return nums.size() - 1 - pivot;
    }
}
```

1870. 2856.java

Path: LeetCode-main/solutions/2856. Minimum Array Length After Pair Removals/2856.java

```
class Solution {
    public int minLengthAfterRemovals(List<Integer> nums) {
        final int n = nums.size();
        Map<Integer, Integer> count = new HashMap<>();
        int maxFreq = 0;

        for (final int num : nums)
            count.merge(num, 1, Integer::sum);

        for (final int freq : count.values())
            maxFreq = Math.max(maxFreq, freq);

        // The number with the maximum frequency cancel all the other numbers.
        if (maxFreq <= n / 2)
            return n % 2;
        // The number with the maximum frequency cancel all the remaining numbers.
        return maxFreq - (n - maxFreq);
    }
}
```

1871. 2857.java

Path: LeetCode-main/solutions/2857. Count Pairs of Points With Distance k/2857.java

```
class Solution {
    public int countPairs(List<List<Integer>> coordinates, int k) {
        int ans = 0;

        for (int x = 0; x <= k; ++x) {
            final int y = k - x;
            Map<Pair<Integer, Integer>, Integer> count = new HashMap<>();
            for (List<Integer> point : coordinates) {
                final int xi = point.get(0);
                final int yi = point.get(1);
                ans += count.getDefault(new Pair<>(xi ^ x, yi ^ y), 0);
                count.merge(new Pair<>(xi, yi), 1, Integer::sum);
            }
        }

        return ans;
    }
}
```

1872. 2858.java

Path: LeetCode-main/solutions/2858. Minimum Edge Reversals So Every Node Is Reachable/2858.java

```
class Solution {
    public int[] minEdgeReversals(int n, int[][] edges) {
        int[] ans = new int[n];
        List
```

1873. 2859.java

Path: LeetCode-main/solutions/2859. Sum of Values at Indices With K Set Bits/2859.java

```
class Solution {
    public int sumIndicesWithKSetBits(List<Integer> nums, int k) {
        int ans = 0;
        for (int i = 0; i < nums.size(); ++i)
            if (Integer.bitCount(i) == k)
                ans += nums.get(i);
        return ans;
    }
}
```

1874. 286.java

Path: LeetCode-main/solutions/286. Walls and Gates/286.java

```
class Solution {
    public void wallsAndGates(int[][] rooms) {
        final int[][] DIRS = {{0, 1}, {1, 0}, {0, -1}, {-1, 0}};
        final int m = rooms.length;
        final int n = rooms[0].length;
        Queue<Pair<Integer, Integer>> q = new ArrayDeque<>();

        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j)
                if (rooms[i][j] == 0)
                    q.offer(new Pair<>(i, j));

        while (!q.isEmpty()) {
            final int i = q.peek().getKey();
            final int j = q.poll().getValue();
            for (int[] dir : DIRS) {
                final int x = i + dir[0];
                final int y = j + dir[1];
                if (x < 0 || x == m || y < 0 || y == n)
                    continue;
                if (rooms[x][y] != Integer.MAX_VALUE)
                    continue;
                rooms[x][y] = rooms[i][j] + 1;
                q.offer(new Pair<>(x, y));
            }
        }
    }
}
```

1875. 2860.java

Path: LeetCode-main/solutions/2860. Happy Students/2860.java

```
class Solution {
    public int countWays(List<Integer> nums) {
        nums.add(-1);
        nums.add(Integer.MAX_VALUE);
        Collections.sort(nums);

        int ans = 0;

        // i := the number of the selected numbers
        for (int i = 0; i + 1 < nums.size(); ++i)
            if (nums.get(i) < i && i < nums.get(i + 1))
                ++ans;

        return ans;
    }
}
```


1876. 2861.java

Path: LeetCode-main/solutions/2861. Maximum Number of Alloys/2861.java

```
class Solution {
    public int maxNumberOfAlloys(int n, int k, int budget, List<List<Integer>> composition,
                                List<Integer> stock, List<Integer> cost) {
        int l = 1;
        int r = 1_000_000_000;

        while (l < r) {
            final int m = (l + r) / 2;
            if (isPossible(n, budget, composition, stock, cost, m))
                l = m + 1;
            else
                r = m;
        }

        return l - 1;
    }

    // Returns true if it's possible to create `m` alloys by using any machine.
    private boolean isPossible(int n, int budget, List<List<Integer>> composition,
                               List<Integer> stock, List<Integer> costs, int m) {
        // Try all the possible machines.
        for (List<Integer> machine : composition) {
            long requiredMoney = 0;
            for (int j = 0; j < n; ++j) {
                final long requiredUnits = Math.max(0L, (long) machine.get(j) * m - stock.get(j));
                requiredMoney += requiredUnits * costs.get(j);
            }
            if (requiredMoney <= budget)
                return true;
        }
        return false;
    }
}
```

1877. 2862-2.java

Path: LeetCode-main/solutions/2862. Maximum Element-Sum of a Complete Subset of Indices/2862-2.java

```
class Solution {
    public long maximumSum(List<Integer> nums) {
        long ans = 0;

        for (int oddPower = 1; oddPower <= nums.size(); ++oddPower) {
            long sum = 0;
            for (int num = 1; num * num * oddPower <= nums.size(); ++num)
                sum += nums.get(oddPower * num * num - 1);
            ans = Math.max(ans, sum);
        }

        return ans;
    }
}
```

1878. 2862.java

Path: LeetCode-main/solutions/2862. Maximum Element-Sum of a Complete Subset of Indices/2862.java

```
class Solution {
    public long maximumSum(List<Integer> nums) {
        long ans = 0;
        HashMap<Integer, Long> oddPowerToSum = new HashMap<>();

        for (int i = 0; i < nums.size(); ++i) {
            final int oddPower = divideSquares(i + 1);
            ans = Math.max(ans, oddPowerToSum.merge(oddPower, (long) nums.get(i), Long::sum));
        }

        return ans;
    }

    private int divideSquares(int val) {
        for (int num = 2; num * num <= val; ++num)
            while (val % (num * num) == 0)
                val /= num * num;
        return val;
    }
}
```

1879. 2863.java

Path: LeetCode-main/solutions/2863. Maximum Length of Semi-Decreasing Subarrays/2863.java

```
class Solution {
    public int maxSubarrayLength(int[] nums) {
        int ans = 0;
        Deque<Integer> stack = new ArrayDeque<>();

        for (int i = nums.length - 1; i >= 0; --i)
            // If nums[stack.peek()] <= nums[i], stack.peek() is better than i.
            // So, no need to push it.
            if (stack.isEmpty() || nums[stack.peek()] > nums[i])
                stack.push(i);

        for (int i = 0; i < nums.length; ++i)
            while (!stack.isEmpty() && nums[i] > nums[stack.peek()])
                ans = Math.max(ans, stack.pop() - i + 1);

        return ans;
    }
}
```

1880. 2864.java

Path: LeetCode-main/solutions/2864. Maximum Odd Binary Number/2864.java

```
class Solution {  
    public String maximumOddBinaryNumber(String s) {  
        final int zeros = (int) s.chars().filter(c -> c == '0').count();  
        final int ones = s.length() - zeros;  
        return "1".repeat(ones - 1) + "0".repeat(zeros) + "1";  
    }  
}
```

1881. 2865.java

Path: LeetCode-main/solutions/2865. Beautiful Towers I/2865.java

```
class Solution {
    public long maximumSumOfHeights(int[] heights) {
        final int n = heights.length;
        long[] maxSum = new long[n]; // maxSum[i] := the maximum sum with peak i

        Deque<Integer> stack = new ArrayDeque<>(List.of(-1));
        long sum = 0;

        for (int i = 0; i < n; i++) {
            sum = process(stack, heights, i, sum);
            maxSum[i] = sum;
        }

        stack = new ArrayDeque<>(List.of(n));
        sum = 0;

        for (int i = n - 1; i >= 0; i--) {
            sum = process(stack, heights, i, sum);
            maxSum[i] += sum - heights[i];
        }

        return Arrays.stream(maxSum).max().getAsLong();
    }

    private long process(Deque<Integer> stack, int[] heights, int i, long sum) {
        while (stack.size() > 1 && heights[stack.peek()] > heights[i]) {
            int j = stack.pop();
            // The last abs(j - stack.peek()) heights are `heights[j]`.
            sum -= (long) Math.abs(j - stack.peek()) * heights[j];
        }
        // Put abs(i - stack.peek()) * heights[i] in `heights`.
        sum += (long) Math.abs(i - stack.peek()) * heights[i];
        stack.push(i);
        return sum;
    }
}
```

1882. 2866.java

Path: LeetCode-main/solutions/2866. Beautiful Towers II/2866.java

```
class Solution {
    // Same as 2865. Beautiful Towers I
    public long maximumSumOfHeights(List<Integer> maxHeights) {
        final int n = maxHeights.size();
        long[] maxSum = new long[n]; // maxSum[i] := the maximum sum with peak i

        Deque<Integer> stack = new ArrayDeque<>(List.of(-1));
        long sum = 0;

        for (int i = 0; i < n; ++i) {
            sum = process(stack, maxHeights, i, sum);
            maxSum[i] = sum;
        }

        stack = new ArrayDeque<>(List.of(n));
        sum = 0;

        for (int i = n - 1; i >= 0; --i) {
            sum = process(stack, maxHeights, i, sum);
            maxSum[i] += sum - maxHeights.get(i);
        }

        return Arrays.stream(maxSum).max().getAsLong();
    }

    private long process(Deque<Integer> stack, List<Integer> maxHeights, int i, long sum) {
        while (stack.size() > 1 && maxHeights.get(stack.peek()) > maxHeights.get(i)) {
            int j = stack.pop();
            // The last abs(j - stack.peek()) maxHeights are `maxHeights[j]`.
            sum -= (long) Math.abs(j - stack.peek()) * maxHeights.get(j);
        }
        // Put abs(i - stack.peek()) * maxHeights[i] in `maxHeights`.
        sum += (long) Math.abs(i - stack.peek()) * maxHeights.get(i);
        stack.push(i);
        return sum;
    }
}
```

1883. 2867.java

Path: LeetCode-main/solutions/2867. Count Valid Paths in a Tree/2867.java

```
class Solution {
    public long countPaths(int n, int[][] edges) {
        final boolean[] isPrime = sieveEratosthenes(n + 1);
        List<Integer>[] graph = new List[n + 1];

        for (int i = 1; i <= n; ++i)
            graph[i] = new ArrayList<>();

        for (int[] edge : edges) {
            final int u = edge[0];
            final int v = edge[1];
            graph[u].add(v);
            graph[v].add(u);
        }

        dfs(graph, 1, /*prev=*/-1, isPrime);
        return ans;
    }

    private long ans = 0;

    private Pair<Long, Long> dfs(List<Integer>[] graph, int u, int prev, boolean[] isPrime) {
        long countZeroPrimePath = isPrime[u] ? 0 : 1;
        long countOnePrimePath = isPrime[u] ? 1 : 0;

        for (final int v : graph[u]) {
            if (v == prev)
                continue;
            Pair<Long, Long> pair = dfs(graph, v, u, isPrime);
            final long countZeroPrimeChildPath = pair.getKey();
            final long countOnePrimeChildPath = pair.getValue();
            ans +=
                countZeroPrimePath * countOnePrimeChildPath + countOnePrimePath * countZeroPrimeChildPath;
            if (isPrime[u]) {
                countOnePrimePath += countZeroPrimeChildPath;
            } else {
                countZeroPrimePath += countZeroPrimeChildPath;
                countOnePrimePath += countOnePrimeChildPath;
            }
        }

        return new Pair<>(countZeroPrimePath, countOnePrimePath);
    }

    private boolean[] sieveEratosthenes(int n) {
        boolean[] isPrime = new boolean[n];
        Arrays.fill(isPrime, true);
        isPrime[0] = false;
        isPrime[1] = false;
        for (int i = 2; i * i < n; ++i)
            if (isPrime[i])
                for (int j = i * i; j < n; j += i)
                    isPrime[j] = false;
        return isPrime;
    }
}
```


1884. 2868.java

Path: LeetCode-main/solutions/2868. The Wording Game/2868.java

```
class Solution {
    public boolean canAliceWin(String[] a, String[] b) {
        // words[0][i] := the biggest word starting with ('a' + i) for Alice
        // words[1][i] := the biggest word starting with ('a' + i) for Bob
        String[][] words = new String[2][26];

        // For each letter, only the biggest word is useful.
        for (final String word : a)
            words[0][word.charAt(0) - 'a'] = word;

        for (final String word : b)
            words[1][word.charAt(0) - 'a'] = word;

        // Find Alice's smallest word.
        int i = 0;
        while (words[0][i] == null)
            ++i;

        // Iterate through each letter until we find a winner.
        // Start with Alice's turn (0), so it's Bob's turn (1) now.
        for (int turn = 1; true; turn = turn ^ 1)
            // If the current player has a word that having the letter that is greater
            // than the opponent's word, choose it.
            if (words[turn][i] != null && words[turn][i].compareTo(words[1 - turn][i]) > 0) {
                // Choose the current words[turn][i].
                // Choose the next words[turn][i + 1].
            } else if (words[turn][i + 1] != null) {
                // Choose the next words[turn][i + 1].
                ++i;
            } else {
                // Game over. If it's Bob's turn, Alice wins, and vice versa.
                return turn == 1;
            }
    }
}
```

1885. 2869.java

Path: LeetCode-main/solutions/2869. Minimum Operations to Collect Elements/2869.java

```
class Solution {
    public int minOperations(List<Integer> nums, int k) {
        Set<Integer> seen = new HashSet<>();

        for (int i = nums.size() - 1; i >= 0; --i) {
            if (nums.get(i) > k)
                continue;
            seen.add(nums.get(i));
            if (seen.size() == k)
                return nums.size() - i;
        }

        throw new IllegalArgumentException();
    }
}
```

1886. 287.java

Path: LeetCode-main/solutions/287. Find the Duplicate Number/287.java

```
class Solution {
    public int findDuplicate(int[] nums) {
        int slow = nums[nums[0]];
        int fast = nums[nums[nums[0]]];

        while (slow != fast) {
            slow = nums[slow];
            fast = nums[nums[fast]];
        }

        slow = nums[0];

        while (slow != fast) {
            slow = nums[slow];
            fast = nums[fast];
        }

        return slow;
    }
}
```

1887. 2870.java

Path: LeetCode-main/solutions/2870. Minimum Number of Operations to Make Array Empty/2870.java

```
class Solution {
    public int minOperations(int[] nums) {
        int ans = 0;
        Map<Integer, Integer> count = new HashMap<>();

        for (final int num : nums)
            count.merge(num, 1, Integer::sum);

        for (final int freq : count.values()) {
            // If freq == 3k, need k operations.
            // If freq == 3k + 1 = 3*(k - 1) + 2*2, need k + 1 operations.
            // If freq == 3k + 2, need k + 1 operations.
            if (freq == 1)
                return -1;
            ans += (freq + 2) / 3;
        }

        return ans;
    }
}
```

1888. 2871.java

Path: LeetCode-main/solutions/2871. Split Array Into Maximum Number of Subarrays/2871.java

```
class Solution {
    public int maxSubarrays(int[] nums) {
        int ans = 0;
        int score = 0;

        for (final int num : nums) {
            score = score == 0 ? num : score & num;
            if (score == 0)
                ++ans;
        }

        return Math.max(1, ans);
    }
}
```

1889. 2872.java

Path: LeetCode-main/solutions/2872. Maximum Number of K-Divisible Components/2872.java

```
class Solution {
    public int maxKDivisibleComponents(int n, int[][] edges, int[] values, int k) {
        List<Integer>[] graph = new List[n];

        for (int i = 0; i < n; i++)
            graph[i] = new ArrayList<>();

        for (int[] edge : edges) {
            final int u = edge[0];
            final int v = edge[1];
            graph[u].add(v);
            graph[v].add(u);
        }

        dfs(graph, 0, /*prev=*/-1, values, k);
        return ans;
    }

    private int ans = 0;

    private long dfs(List<Integer>[] graph, int u, int prev, int[] values, int k) {
        long treeSum = values[u];

        for (int v : graph[u])
            if (v != prev)
                treeSum += dfs(graph, v, u, values, k);

        if (treeSum % k == 0)
            ++ans;
        return treeSum;
    }
}
```

1890. 2873.java

Path: LeetCode-main/solutions/2873. Maximum Value of an Ordered Triplet I/2873.java

```
class Solution {
    public long maximumTripletValue(int[] nums) {
        long ans = 0;
        int maxDiff = 0; // max(nums[i] - nums[j])
        int maxNum = 0; // max(nums[i])

        for (final int num : nums) {
            ans = Math.max(ans, (long) maxDiff * num); // num := nums[k]
            maxDiff = Math.max(maxDiff, maxNum - num); // num := nums[j]
            maxNum = Math.max(maxNum, num);           // num := nums[i]
        }

        return ans;
    }
}
```

1891. 2874.java

Path: LeetCode-main/solutions/2874. Maximum Value of an Ordered Triplet II/2874.java

```
class Solution {
    // Same as 2873. Maximum Value of an Ordered Triplet I
    public long maximumTripletValue(int[] nums) {
        long ans = 0;
        int maxDiff = 0; // max(nums[i] - nums[j])
        int maxNum = 0; // max(nums[i])

        for (final int num : nums) {
            ans = Math.max(ans, (long) maxDiff * num); // num := nums[k]
            maxDiff = Math.max(maxDiff, maxNum - num); // num := nums[j]
            maxNum = Math.max(maxNum, num);           // num := nums[i]
        }

        return ans;
    }
}
```


1892. 2875.java

Path: LeetCode-main/solutions/2875. Minimum Size Subarray in Infinite Array/2875.java

```
class Solution {
    public int minSizeSubarray(int[] nums, int target) {
        final long sum = Arrays.stream(nums).asLongStream().sum();
        final int n = nums.length;
        final int remainingTarget = (int) (target % sum);
        final int repeatLength = (int) (target / sum) * n;
        if (remainingTarget == 0)
            return repeatLength;

        int suffixPlusPrefixLength = n;
        long prefix = 0;
        HashMap<Long, Integer> prefixToIndex = new HashMap<>();
        prefixToIndex.put(0L, -1);

        for (int i = 0; i < 2 * n; ++i) {
            prefix += nums[i % n];
            if (prefixToIndex.containsKey(prefix - remainingTarget))
                suffixPlusPrefixLength =
                    Math.min(suffixPlusPrefixLength, i - prefixToIndex.get(prefix - remainingTarget));
            prefixToIndex.put(prefix, i);
        }

        return suffixPlusPrefixLength == n ? -1 : repeatLength + suffixPlusPrefixLength;
    }
}
```

1893. 2876.java

Path: LeetCode-main/solutions/2876. Count Visited Nodes in a Directed Graph/2876.java

```
class Solution {
    public int[] countVisitedNodes(List<Integer> edges) {
        final int n = edges.size();
        int[] ans = new int[n];
        int[] inDegrees = new int[n];
        boolean[] seen = new boolean[n];
        Queue<Integer> q = new ArrayDeque<>();
        Stack<Integer> stack = new Stack<>();

        for (int v : edges)
            ++inDegrees[v];

        // Perform topological sorting.
        for (int i = 0; i < n; ++i)
            if (inDegrees[i] == 0)
                q.add(i);

        // Push non-cyclic nodes to stack.
        while (!q.isEmpty()) {
            final int u = q.poll();
            if (--inDegrees[edges.get(u)] == 0)
                q.add(edges.get(u));
            stack.push(u);
            seen[u] = true;
        }

        // Fill the length of cyclic nodes.
        for (int i = 0; i < n; ++i)
            if (!seen[i])
                fillCycle(edges, i, seen, ans);

        // Fill the length of non-cyclic nodes.
        while (!stack.isEmpty()) {
            final int u = stack.pop();
            ans[u] = ans[edges.get(u)] + 1;
        }

        return ans;
    }

    private void fillCycle(List<Integer> edges, int start, boolean[] seen, int[] ans) {
        int cycleLength = 0;
        for (int u = start; !seen[u]; u = edges.get(u)) {
            ++cycleLength;
            seen[u] = true;
        }
        ans[start] = cycleLength;
        for (int u = edges.get(start); u != start; u = edges.get(u))
            ans[u] = cycleLength;
    }
}
```

1894. 288.java

Path: LeetCode-main/solutions/288. Unique Word Abbreviation/288.java

```
class ValidWordAbbr {
    public ValidWordAbbr(String[] dictionary) {
        dict = new HashSet<>(Arrays.asList(dictionary));

        for (final String word : dict) {
            final String abbr = getAbbr(word);
            abbrUnique.put(abbr, !abbrUnique.containsKey(abbr));
        }

        public boolean isUnique(String word) {
            final String abbr = getAbbr(word);
            final Boolean hasAbbr = abbrUnique.get(abbr);
            return hasAbbr == null || hasAbbr && dict.contains(word);
        }

        private Set<String> dict;
        private Map<String, Boolean> abbrUnique = new HashMap<>(); // T := unique, F := not unique

        private String getAbbr(final String s) {
            final int n = s.length();
            if (n <= 2)
                return s;
            return s.charAt(0) + Integer.toString(n - 2) + s.charAt(n - 1);
        }
    }
}
```

1895. 289.java

Path: LeetCode-main/solutions/289. Game of Life/289.java

```
class Solution {
    public void gameOfLife(int[][] board) {
        final int m = board.length;
        final int n = board[0].length;

        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j) {
                int ones = 0;
                for (int x = Math.max(0, i - 1); x < Math.min(m, i + 2); ++x)
                    for (int y = Math.max(0, j - 1); y < Math.min(n, j + 2); ++y)
                        ones += board[x][y] & 1;
                // Any live cell with two or three live neighbors lives on to the next
                // generation.
                if (board[i][j] == 1 && (ones == 3 || ones == 4))
                    board[i][j] |= 0b10;
                // Any dead cell with exactly three live neighbors becomes a live cell,
                // as if by reproduction.
                if (board[i][j] == 0 && ones == 3)
                    board[i][j] |= 0b10;
            }

        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j)
                board[i][j] >>= 1;
    }
}
```

1896. 2892.java

Path: LeetCode-main/solutions/2892. Minimizing Array After Replacing Pairs With Their Product/2892.java

```
class Solution {
    public int minArrayLength(int[] nums, int k) {
        int count = 0;
        long prod = -1;

        for (final int num : nums) {
            if (num == 0)
                return 1;
            if (prod != -1 && prod * num <= k) {
                prod *= num;
            } else {
                prod = num;
                count++;
            }
        }

        return count;
    }
}
```

1897. 2894.java

Path: LeetCode-main/solutions/2894. Divisible and Non-divisible Sums Difference/2894.java

```
class Solution {
    public int differenceOfSums(int n, int m) {
        final int sum = (1 + n) * n / 2;
        final int num2 = getDivisibleSum(n, m);
        final int num1 = sum - num2;
        return num1 - num2;
    }

    // Returns the sum of all the integers in [1, n] that are divisible by m.
    private int getDivisibleSum(int n, int m) {
        final int last = n / m * m;
        if (last == 0)
            return 0;
        final int first = m;
        final int count = (last - first) / m + 1;
        return (first + last) * count / 2;
    }
}
```

1898. 2895.java

Path: LeetCode-main/solutions/2895. Minimum Processing Time/2895.java

```
class Solution {
    public int minProcessingTime(List<Integer> processorTime, List<Integer> tasks) {
        int ans = 0;
        Collections.sort(processorTime);
        Collections.sort(tasks, Collections.reverseOrder());

        // It's optimal to run each 4 longer tasks with a smaller processor time.
        // Therefore, for each processor, take the maximum of the sum of the
        // processor time and the largest assigned tasks[i].
        for (int i = 0; i < processorTime.size(); ++i)
            ans = Math.max(ans, processorTime.get(i) + tasks.get(i * 4));

        return ans;
    }
}
```

1899. 2896-2.java

Path: LeetCode-main/solutions/2896. Apply Operations to Make Two Strings Equal/2896-2.java

```
class Solution {
    public int minOperations(String s1, String s2, int x) {
        List<Integer> diffIndices = getDiffIndices(s1, s2);
        if (diffIndices.isEmpty())
            return 0;
        // It's impossible to make two strings equal if there are odd number of
        // differences.
        if (diffIndices.size() % 2 == 1)
            return -1;

        double[] dp = new double[diffIndices.size() + 1];
        Arrays.fill(dp, Double.MAX_VALUE);
        dp[diffIndices.size()] = 0;
        dp[diffIndices.size() - 1] = x / 2.0;

        for (int i = diffIndices.size() - 2; i >= 0; --i)
            dp[i] = Math.min(dp[i + 1] + x / 2.0, //
                             dp[i + 2] + diffIndices.get(i + 1) - diffIndices.get(i));

        return (int) dp[0];
    }

    private List<Integer> getDiffIndices(final String s1, final String s2) {
        List<Integer> diffIndices = new ArrayList<>();
        for (int i = 0; i < s1.length(); ++i)
            if (s1.charAt(i) != s2.charAt(i))
                diffIndices.add(i);
        return diffIndices;
    }
}
```


1900. 2896-3.java

Path: LeetCode-main/solutions/2896. Apply Operations to Make Two Strings Equal/2896-3.java

```
class Solution {
    public int minOperations(String s1, String s2, int x) {
        List<Integer> diffIndices = getDiffIndices(s1, s2);
        if (diffIndices.isEmpty())
            return 0;
        // It's impossible to make two strings equal if there are odd number of
        // differences.
        if (diffIndices.size() % 2 == 1)
            return -1;

        double dp = 0;
        double dpNext = x / 2.0;
        double dpNextNext = 0;

        for (int i = diffIndices.size() - 2; i >= 0; --i) {
            dp = Math.min(dpNext + x / 2.0, //
                dpNextNext + diffIndices.get(i + 1) - diffIndices.get(i));
            dpNextNext = dpNext;
            dpNext = dp;
        }

        return (int) dp;
    }

    private List<Integer> getDiffIndices(final String s1, final String s2) {
        List<Integer> diffIndices = new ArrayList<>();
        for (int i = 0; i < s1.length(); ++i)
            if (s1.charAt(i) != s2.charAt(i))
                diffIndices.add(i);
        return diffIndices;
    }
}
```

1901. 2896.java

Path: LeetCode-main/solutions/2896. Apply Operations to Make Two Strings Equal/2896.java

```
class Solution {
    public int minOperations(String s1, String s2, int x) {
        List<Integer> diffIndices = getDiffIndices(s1, s2);
        if (diffIndices.isEmpty())
            return 0;
        // It's impossible to make two strings equal if there are odd number of
        // differences.
        if (diffIndices.size() % 2 == 1)
            return -1;
        Double[] mem = new Double[diffIndices.size()];
        return (int) minOperations(diffIndices, 0, x, mem);
    }

    private double minOperations(List<Integer> diffIndices, int i, double x, Double[] mem) {
        if (i == diffIndices.size())
            return 0;
        if (i == diffIndices.size() - 1)
            return x / 2;
        if (mem[i] != null)
            return mem[i];
        return mem[i] = Math.min(minOperations(diffIndices, i + 1, x, mem) + x / 2,
                                minOperations(diffIndices, i + 2, x, mem) + diffIndices.get(i + 1) -
                                diffIndices.get(i));
    }

    private List<Integer> getDiffIndices(final String s1, final String s2) {
        List<Integer> diffIndices = new ArrayList<>();
        for (int i = 0; i < s1.length(); ++i)
            if (s1.charAt(i) != s2.charAt(i))
                diffIndices.add(i);
        return diffIndices;
    }
}
```

1902. 2897.java

Path: LeetCode-main/solutions/2897. Apply Operations on Array to Maximize Sum of Squares/2897.java

```
class Solution {
    public int maxSum(List<Integer> nums, int k) {
        final int MOD = 1_000_000_007;
        final int MAX_BIT = 30;
        int ans = 0;
        // minIndices[i] := the minimum index in `optimalNums` that the i-th bit
        // should be moved to
        int[] minIndices = new int[MAX_BIT];
        int[] optimalNums = new int[nums.size()];

        for (final int num : nums)
            for (int i = 0; i < MAX_BIT; i++)
                if ((num >> i & 1) == 1)
                    optimalNums[minIndices[i]++] |= 1 << i;

        for (int i = 0; i < k; i++)
            ans = (int) ((ans + (long) optimalNums[i] * optimalNums[i]) % MOD);

        return ans;
    }
}
```

1903. 2898.java

Path: LeetCode-main/solutions/2898. Maximum Linear Stock Score/2898.java

```
class Solution {
    public long maxScore(int[] prices) {
        // prices[indices[j]] - prices[indices[j - 1]]
        // == indices[j] - indices[j - 1]
        // prices[indices[j]] - indices[j]
        // == prices[indices[j - 1]] - indices[j - 1]
        //
        // So, elements in the same subsequence must have the same prices[i] - i.
        Map<Integer, Long> groupIdToSum = new HashMap<>();

        for (int i = 0; i < prices.length; ++i)
            groupIdToSum.merge(prices[i] - i, (long) prices[i], Long::sum);

        return groupIdToSum.values().stream().max(Long::compare).orElse(0L);
    }
}
```

1904. 2899.java

Path: LeetCode-main/solutions/2899. Last Visited Integers/2899.java

```
class Solution {
    public List<Integer> lastVisitedIntegers(List<String> words) {
        List<Integer> ans = new ArrayList<>();
        List<Integer> nums = new ArrayList<>();
        int k = 0;

        for (final String word : words)
            if (word.equals("prev")) {
                ++k;
                ans.add(k > nums.size() ? -1 : nums.get(nums.size() - k));
            } else {
                k = 0;
                nums.add(Integer.valueOf(word));
            }

        return ans;
    }
}
```

1905. 29.java

Path: LeetCode-main/solutions/29. Divide Two Integers/29.java

```
class Solution {
    public int divide(long dividend, long divisor) {
        // -2^{31} / -1 = 2^{31} will overflow, so return 2^{31} - 1.
        if (dividend == Integer.MIN_VALUE && divisor == -1)
            return Integer.MAX_VALUE;

        final int sign = dividend > 0 ^ divisor > 0 ? -1 : 1;
        long dvd = Math.abs(dividend);
        long dvs = Math.abs(divisor);

        while (dvd >= dvs) {
            long k = 1;
            while (k * 2 * dvs <= dvd)
                k *= 2;
            dvd -= k * dvs;
            ans += k;
        }

        return sign * (int) ans;
    }
}
```

1906. 290.java

Path: LeetCode-main/solutions/290. Word Pattern/290.java

```
class Solution {
    public boolean wordPattern(String pattern, String str) {
        String[] words = str.split(" ");
        if (words.length != pattern.length())
            return false;

        Map<Character, Integer> charToIndex = new HashMap<>();
        Map<String, Integer> stringToIndex = new HashMap<>();

        for (Integer i = 0; i < pattern.length(); ++i)
            if (charToIndex.put(pattern.charAt(i), i) != stringToIndex.put(words[i], i))
                return false;

        return true;
    }
}
```

1907. 2900.java

Path: LeetCode-main/solutions/2900. Longest Unequal Adjacent Groups Subsequence I/2900.java

```
class Solution {
    public List<String> getWordsInLongestSubsequence(int n, String[] words, int[] groups) {
        List<String> ans = new ArrayList<>();
        int groupId = -1;

        for (int i = 0; i < n; ++i)
            if (groups[i] != groupId) {
                groupId = groups[i];
                ans.add(words[i]);
            }

        return ans;
    }
}
```


1908. 2901.java

Path: LeetCode-main/solutions/2901. Longest Unequal Adjacent Groups Subsequence II/2901.java

```
class Solution {
    public List<String> getWordsInLongestSubsequence(int n, String[] words, int[] groups) {
        List<String> ans = new ArrayList<>();
        // dp[i] := the length of the longest subsequence ending in `words[i]`
        int[] dp = new int[n];
        Arrays.fill(dp, 1);
        // prev[i] := the best index of words[i]
        int[] prev = new int[n];
        Arrays.fill(prev, -1);

        for (int i = 1; i < n; ++i)
            for (int j = 0; j < i; ++j) {
                if (groups[i] == groups[j])
                    continue;
                if (words[i].length() != words[j].length())
                    continue;
                if (hammingDist(words[i], words[j]) != 1)
                    continue;
                if (dp[i] < dp[j] + 1) {
                    dp[i] = dp[j] + 1;
                    prev[i] = j;
                }
            }

        // Find the last index of the subsequence.
        int index = getMaxIndex(dp);
        while (index != -1) {
            ans.add(words[index]);
            index = prev[index];
        }

        Collections.reverse(ans);
        return ans;
    }

    private int hammingDist(final String s1, final String s2) {
        int dist = 0;
        for (int i = 0; i < s1.length(); ++i)
            if (s1.charAt(i) != s2.charAt(i))
                ++dist;
        return dist;
    }

    private int getMaxIndex(int[] dp) {
        int maxIndex = 0;
        for (int i = 0; i < dp.length; ++i)
            if (dp[i] > dp[maxIndex])
                maxIndex = i;
        return maxIndex;
    }
}
```

1909. 2902.java

Path: LeetCode-main/solutions/2902. Count of Sub-Multisets With Bounded Sum/2902.java

```
class Solution {
    public int countSubMultisets(List<Integer> nums, int l, int r) {
        final int MOD = 1_000_000_007;
        // dp[i] := the number of submultisets of `nums` with sum i
        long[] dp = new long[r + 1];
        dp[0] = 1;
        Map<Integer, Integer> count = new HashMap<>();

        for (final int num : nums)
            count.merge(num, 1, Integer::sum);

        final int zeros = count.containsKey(0) ? count.remove(0) : 0;

        for (Map.Entry<Integer, Integer> entry : count.entrySet()) {
            final int num = entry.getKey();
            final int freq = entry.getValue();
            // stride[i] := dp[i] + dp[i - num] + dp[i - 2 * num] + ...
            long[] stride = dp.clone();
            for (int i = num; i <= r; ++i)
                stride[i] += stride[i - num];
            for (int i = r; i > 0; --i)
                if (i >= num * (freq + 1))
                    // dp[i] + dp[i - num] + dp[i - freq * num]
                    dp[i] = (stride[i] - stride[i - num * (freq + 1)]) % MOD;
                else
                    dp[i] = stride[i] % MOD;
        }

        long ans = 0;
        for (int i = l; i <= r; ++i)
            ans = (ans + dp[i]) % MOD;
        return (int) (((zeros + 1) * ans) % MOD);
    }
}
```

1910. 2903.java

Path: LeetCode-main/solutions/2903. Find Indices With Index and Value Difference I/2903.java

```
class Solution {
    public int[] findIndices(int[] nums, int indexDifference, int valueDifference) {
        // nums[minIndex] := the minimum number with enough index different from the
        // current number
        int minIndex = 0;
        // nums[maxIndex] := the maximum number with enough index different from the
        // current number
        int maxIndex = 0;

        for (int i = indexDifference; i < nums.length; ++i) {
            if (nums[i - indexDifference] < nums[minIndex])
                minIndex = i - indexDifference;
            if (nums[i - indexDifference] > nums[maxIndex])
                maxIndex = i - indexDifference;
            if (nums[i] - nums[minIndex] >= valueDifference)
                return new int[] {i, minIndex};
            if (nums[maxIndex] - nums[i] >= valueDifference)
                return new int[] {i, maxIndex};
        }

        return new int[] {-1, -1};
    }
}
```

1911. 2904.java

Path: LeetCode-main/solutions/2904. Shortest and Lexicographically Smallest Beautiful String/2904.java

```
class Solution {
    // Same as 76. Minimum Window Substring
    public String shortestBeautifulSubstring(String s, int k) {
        int bestLeft = -1;
        int minLength = s.length() + 1;
        int ones = 0;

        for (int l = 0, r = 0; r < s.length(); ++r) {
            if (s.charAt(r) == '1')
                ++ones;
            while (ones == k) {
                if (r - l + 1 < minLength) {
                    bestLeft = l;
                    minLength = r - l + 1;
                } else if (r - l + 1 == minLength &&
                    s.substring(l, l + minLength)
                        .compareTo(s.substring(bestLeft, bestLeft + minLength)) < 0) {
                    bestLeft = l;
                }
                if (s.charAt(l++) == '1')
                    --ones;
            }
        }

        return bestLeft == -1 ? "" : s.substring(bestLeft, bestLeft + minLength);
    }
}
```

1912. 2905.java

Path: LeetCode-main/solutions/2905. Find Indices With Index and Value Difference II/2905.java

```
class Solution {
    public int[] findIndices(int[] nums, int indexDifference, int valueDifference) {
        // nums[minIndex] := the minimum number with enough index different from the current number
        int minIndex = 0;
        // nums[maxIndex] := the maximum number with enough index different from the current number
        int maxIndex = 0;

        for (int i = indexDifference; i < nums.length; ++i) {
            if (nums[i - indexDifference] < nums[minIndex])
                minIndex = i - indexDifference;
            if (nums[i - indexDifference] > nums[maxIndex])
                maxIndex = i - indexDifference;
            if (nums[i] - nums[minIndex] >= valueDifference)
                return new int[] {i, minIndex};
            if (nums[maxIndex] - nums[i] >= valueDifference)
                return new int[] {i, maxIndex};
        }

        return new int[] {-1, -1};
    }
}
```

1913. 2906.java

Path: LeetCode-main/solutions/2906. Construct Product Matrix/2906.java

```
class Solution {
    public int[][] constructProductMatrix(int[][] grid) {
        final int MOD = 12345;
        final int m = grid.length;
        final int n = grid[0].length;
        int[][] ans = new int[m][n];
        List<Integer> prefix = new ArrayList<>(List.of(1));
        int suffix = 1;

        for (int[] row : grid)
            for (int num : row)
                prefix.add((int) ((long) prefix.get(prefix.size() - 1) * num % MOD));

        for (int i = m - 1; i >= 0; i--)
            for (int j = n - 1; j >= 0; j--) {
                ans[i][j] = (int) ((long) prefix.get(i * n + j) * suffix % MOD);
                suffix = (int) ((long) suffix * grid[i][j] % MOD);
            }

        return ans;
    }
}
```

1914. 2907.java

Path: LeetCode-main/solutions/2907. Maximum Profitable Triplets With Increasing Prices I/2907.java

```
class FenwickTree {
    public FenwickTree(int n) {
        vals = new int[n + 1];
    }

    public void maximize(int i, int val) {
        while (i < vals.length) {
            vals[i] = Math.max(vals[i], val);
            i += lowbit(i);
        }
    }

    public int get(int i) {
        int res = 0;
        while (i > 0) {
            res = Math.max(res, vals[i]);
            i -= lowbit(i);
        }
        return res;
    }

    private int[] vals;

    private static int lowbit(int i) {
        return i & -i;
    }
}

class Solution {
    public int maxProfit(int[] prices, int[] profits) {
        final int maxPrice = Arrays.stream(prices).max().getAsInt();
        int ans = -1;
        FenwickTree maxProfitTree1 = new FenwickTree(maxPrice);
        FenwickTree maxProfitTree2 = new FenwickTree(maxPrice);

        for (int i = 0; i < prices.length; ++i) {
            final int price = prices[i];
            final int profit = profits[i];
            // max(profits[i])
            final int maxProfit1 = maxProfitTree1.get(price - 1);
            // max(proftis[i]) + max(profits[j])
            final int maxProfit2 = maxProfitTree2.get(price - 1);
            maxProfitTree1.maximize(price, profit);
            if (maxProfit1 > 0)
                maxProfitTree2.maximize(price, profit + maxProfit1);
            if (maxProfit2 > 0)
                ans = Math.max(ans, profit + maxProfit2);
        }

        return ans;
    }
}
```

1915. 2908.java

Path: LeetCode-main/solutions/2908. Minimum Sum of Mountain Triplets I/2908.java

```
class Solution {
    public int minimumSum(int[] nums) {
        final int n = nums.length;
        int ans = Integer.MAX_VALUE;
        int[] minPrefix = new int[n];
        int[] minSuffix = new int[n];

        minPrefix[0] = nums[0];
        minSuffix[n - 1] = nums[n - 1];

        for (int i = 1; i < n; ++i)
            minPrefix[i] = Math.min(minPrefix[i - 1], nums[i]);

        for (int i = n - 2; i >= 0; --i)
            minSuffix[i] = Math.min(minSuffix[i + 1], nums[i]);

        for (int i = 0; i < n; ++i)
            if (nums[i] > minPrefix[i] && nums[i] > minSuffix[i])
                ans = Math.min(ans, nums[i] + minPrefix[i] + minSuffix[i]);

        return ans == Integer.MAX_VALUE ? -1 : ans;
    }
}
```


1916. 2909.java

Path: LeetCode-main/solutions/2909. Minimum Sum of Mountain Triplets II/2909.java

```
class Solution {
    // Same as 2908. Minimum Sum of Mountain Triplets I
    public int minimumSum(int[] nums) {
        final int n = nums.length;
        int ans = Integer.MAX_VALUE;
        int[] minPrefix = new int[n];
        int[] minSuffix = new int[n];

        minPrefix[0] = nums[0];
        minSuffix[n - 1] = nums[n - 1];

        for (int i = 1; i < n; ++i)
            minPrefix[i] = Math.min(minPrefix[i - 1], nums[i]);

        for (int i = n - 2; i >= 0; --i)
            minSuffix[i] = Math.min(minSuffix[i + 1], nums[i]);

        for (int i = 0; i < n; ++i)
            if (nums[i] > minPrefix[i] && nums[i] > minSuffix[i])
                ans = Math.min(ans, nums[i] + minPrefix[i] + minSuffix[i]);

        return ans == Integer.MAX_VALUE ? -1 : ans;
    }
}
```

1917. 291.java

Path: LeetCode-main/solutions/291. Word Pattern II/291.java

```
class Solution {
    public boolean wordPatternMatch(String pattern, String s) {
        return isMatch(pattern, 0, s, 0, new HashMap<>(), new HashSet<>());
    }

    private boolean isMatch(final String pattern, int i, final String s, int j,
        Map<Character, String> charToString, Set<String> seen) {
        if (i == pattern.length() && j == s.length())
            return true;
        if (i == pattern.length() || j == s.length())
            return false;

        final char c = pattern.charAt(i);

        if (charToString.containsKey(c)) {
            final String t = charToString.get(c);
            // See if we can match t with s[j..n).
            if (!s.startsWith(t, j))
                return false;

            // If there's a match, continue to match the rest.
            return isMatch(pattern, i + 1, s, j + t.length(), charToString, seen);
        }

        for (int k = j; k < s.length(); ++k) {
            final String t = s.substring(j, k + 1);

            // This string is mapped by another character.
            if (seen.contains(t))
                continue;

            charToString.put(c, t);
            seen.add(t);

            if (isMatch(pattern, i + 1, s, k + 1, charToString, seen))
                return true;

            // Backtrack.
            charToString.remove(c);
            seen.remove(t);
        }

        return false;
    }
}
```

1918. 2910.java

Path: LeetCode-main/solutions/2910. Minimum Number of Groups to Create a Valid Assignment/2910.java

```
class Solution {
    public int minGroupsForValidAssignment(int[] nums) {
        Map<Integer, Integer> count = new HashMap<>();
        int minFreq = nums.length;

        for (final int num : nums)
            count.merge(num, 1, Integer::sum);

        for (final int freq : count.values())
            minFreq = Math.min(minFreq, freq);

        for (int groupSize = minFreq; groupSize >= 1; --groupSize) {
            final int numGroups = getNumGroups(count, groupSize);
            if (numGroups > 0)
                return numGroups;
        }

        throw new IllegalArgumentException();
    }

    // Returns the number of groups if each group's size is `groupSize` or
    // `groupSize + 1`.
    private int getNumGroups(Map<Integer, Integer> count, int groupSize) {
        int numGroups = 0;
        for (final int freq : count.values()) {
            final int a = freq / (groupSize + 1);
            final int b = freq % (groupSize + 1);
            if (b == 0) {
                numGroups += a;
            } else if (groupSize - b <= a) {
                // Assign 1 number from `groupSize - b` out of `a` groups to this group,
                // so we'll have `a - (groupSize - b)` groups of size `groupSize + 1`
                // and `groupSize - b + 1` groups of size `groupSize`. In total, we have
                // `a + 1` groups.
                numGroups += a + 1;
            } else {
                return 0;
            }
        }
        return numGroups;
    }
}
```

1919. 2911.java

Path: LeetCode-main/solutions/2911. Minimum Changes to Make K Semi-palindromes/2911.java

```
class Solution {
    public int minimumChanges(String s, int k) {
        final int n = s.length();
        // factors[i] := factors of i
        List<Integer>[] factors = getFactors(n);
        // cost[i][j] := changes to make s[i..j] a semi-palindrome
        int[][] cost = getCost(s, n, factors);
        // dp[i][j] := the minimum changes to split s[i:] into j valid parts
        int[][] dp = new int[n + 1][k + 1];

        Arrays.stream(dp).forEach(A -> Arrays.fill(A, n));
        dp[n][0] = 0;

        for (int i = n - 1; i >= 0; --i)
            for (int j = 1; j <= k; ++j)
                for (int l = i + 1; l < n; ++l)
                    dp[i][j] = Math.min(dp[i][j], dp[l + 1][j - 1] + cost[i][l]);

        return dp[0][k];
    }

    private List<Integer>[] getFactors(int n) {
        List<Integer>[] factors = new List[n + 1];
        for (int i = 1; i <= n; ++i)
            factors[i] = new ArrayList<>(List.of(1));
        for (int d = 2; d < n; ++d)
            for (int i = d * 2; i <= n; i += d)
                factors[i].add(d);
        return factors;
    }

    private int[][] getCost(final String s, int n, List<Integer>[] factors) {
        int[][] cost = new int[n][n];
        for (int i = 0; i < n; ++i)
            for (int j = i + 1; j < n; ++j) {
                final int length = j - i + 1;
                int minCost = length;
                for (final int d : factors[length])
                    minCost = Math.min(minCost, getCost(s, i, j, d));
                cost[i][j] = minCost;
            }
        return cost;
    }

    // Returns the cost to make s[i..j] a semi-palindrome of `d`.
    private int getCost(final String s, int i, int j, int d) {
        int cost = 0;
        for (int offset = 0; offset < d; ++offset) {
            int l = i + offset;
            int r = j - d + 1 + offset;
            while (l < r) {
                if (s.charAt(l) != s.charAt(r))
                    ++cost;
                l += d;
                r -= d;
            }
        }
        return cost;
    }
}
```

1920. 2912-2.java

Path: LeetCode-main/solutions/2912. Number of Ways to Reach Destination in the Grid/2912-2.java

```
class Solution {
    public int numberOfWays(int n, int m, int k, int[] source, int[] dest) {
        final int MOD = 1_000_000_007;
        // the number of ways of `source` to `dest` using steps so far
        int ans = (source[0] == dest[0] && source[1] == dest[1]) ? 1 : 0;
        // the number of ways of `source` to dest's row using steps so far
        int row = (source[0] == dest[0] && source[1] != dest[1]) ? 1 : 0;
        // the number of ways of `source` to dest's col using steps so far
        int col = (source[0] != dest[0] && source[1] == dest[1]) ? 1 : 0;
        // the number of ways of `source` to others using steps so far
        int others = (source[0] != dest[0] && source[1] != dest[1]) ? 1 : 0;

        for (int i = 0; i < k; ++i) {
            final int nextAns = (row + col) % MOD;
            final int nextRow = (int) ((ans * (m - 1L) + // -self
                                         row * (m - 2L) + //-self, -center
                                         others) %
                                       MOD);
            final int nextCol = (int) ((ans * (n - 1L) + // -self
                                         col * (n - 2L) + // -self, -center
                                         others) %
                                       MOD);
            final int nextOthers = (int) ((row * (n - 1L) + // -self
                                           col * (m - 1L) + // -self
                                           others * (m + n - 1 - 3L)) % // -self, -row, -col
                                         MOD);

            ans = nextAns;
            row = nextRow;
            col = nextCol;
            others = nextOthers;
        }

        return ans;
    }
}
```

1921. 2912.java

Path: LeetCode-main/solutions/2912. Number of Ways to Reach Destination in the Grid/2912.java

```
class Solution {
    public int numberOfWays(int n, int m, int k, int[] source, int[] dest) {
        final int MOD = 1_000_000_007;
        // dp[i][0] := the the number of ways of `source` to `dest` using i steps
        // dp[i][1] := the the number of ways of `source` to dest's row using i steps
        // dp[i][2] := the the number of ways of `source` to dest's col using i steps
        // dp[i][3] := the the number of ways of `source` to others using i steps
        int[][] dp = new int[k + 1][4];
        if (Arrays.equals(source, dest))
            dp[0][0] = 1;
        else if (source[0] == dest[0])
            dp[0][1] = 1;
        else if (source[1] == dest[1])
            dp[0][2] = 1;
        else
            dp[0][3] = 1;

        for (int i = 1; i <= k; i++) {
            dp[i][0] = (dp[i - 1][1] + dp[i - 1][2]) % MOD;
            dp[i][1] = (int) ((dp[i - 1][0] * (m - 1L) + // -self
                             dp[i - 1][1] * (m - 2L) + // -self, -center
                             dp[i - 1][3]) %
                           MOD);
            dp[i][2] = (int) ((dp[i - 1][0] * (n - 1L) + // -self
                             dp[i - 1][2] * (n - 2L) + // -self, -center
                             dp[i - 1][3]) %
                           MOD);
            dp[i][3] = (int) ((dp[i - 1][1] * (n - 1L) + // -self
                             dp[i - 1][2] * (m - 1L) + // -self
                             dp[i - 1][3] * (m + n - 1 - 3L)) // -self, -row, -col
                              % MOD);
        }

        return dp[k][0];
    }
}
```

1922. 2914.java

Path: LeetCode-main/solutions/2914. Minimum Number of Changes to Make Binary String Beautiful/2914.java

```
class Solution {
    public int minChanges(String s) {
        int ans = 0;

        for (int i = 0; i + 1 < s.length(); i += 2)
            if (s.charAt(i) != s.charAt(i + 1))
                ++ans;

        return ans;
    }
}
```

1923. 2915-2.java

Path: LeetCode-main/solutions/2915. Length of the Longest Subsequence That Sums to Target/2915-2.java

```
class Solution {
    public int lengthOfLongestSubsequence(List<Integer> nums, int target) {
        // dp[i] := the maximum length of any subsequence of numbers so far that
        // sum to j
        int[] dp = new int[target + 1];

        for (final int num : nums)
            for (int i = target; i >= num; --i)
                if (i == num || dp[i - num] > 0)
                    dp[i] = Math.max(dp[i], 1 + dp[i - num]);

        return dp[target] > 0 ? dp[target] : -1;
    }
}
```


1924. 2915.java

Path: LeetCode-main/solutions/2915. Length of the Longest Subsequence That Sums to Target/2915.java

```
class Solution {
    public int lengthOfLongestSubsequence(List<Integer> nums, int target) {
        final int n = nums.size();
        // dp[i][j] := the maximum length of any subsequence of the first i numbers
        // that sum to j
        int[][] dp = new int[n + 1][target + 1];
        Arrays.stream(dp).forEach(A -> Arrays.fill(A, -1));

        for (int i = 0; i <= n; ++i)
            dp[i][0] = 0;

        for (int i = 1; i <= n; ++i) {
            final int num = nums.get(i - 1);
            for (int j = 1; j <= target; ++j) {
                // 1. Skip `num`.
                if (j < num || dp[i - 1][j - num] == -1)
                    dp[i][j] = dp[i - 1][j];
                // 2. Skip `num` or pick `num`.
                else
                    dp[i][j] = Math.max(dp[i - 1][j], 1 + dp[i - 1][j - num]);
            }
        }

        return dp[n][target];
    }
}
```

1925. 2917.java

Path: LeetCode-main/solutions/2917. Find the K-or of an Array/2917.java

```
class Solution {
    public int findKOr(int[] nums, int k) {
        final int MAX_BIT = 30;
        int ans = 0;

        for (int i = 0; i <= MAX_BIT; ++i) {
            final int finalI = i;
            final int count = (int) Arrays.stream(nums)
                .filter(num -> (num >> finalI & 1) == 1) //
                .count();

            if (count >= k)
                ans += Math.pow(2, i);
        }

        return ans;
    }
}
```

1926. 2918.java

Path: LeetCode-main/solutions/2918. Minimum Equal Sum of Two Arrays After Replacing Zeros/2918.java

```
class Solution {
    public long minSum(int[] nums1, int[] nums2) {
        final long sum1 = Arrays.stream(nums1).asLongStream().sum();
        final long sum2 = Arrays.stream(nums2).asLongStream().sum();
        final long zero1 = Arrays.stream(nums1).filter(num -> num == 0).count();
        final long zero2 = Arrays.stream(nums2).filter(num -> num == 0).count();
        if (zero1 == 0 && sum1 < sum2 + zero2)
            return -1;
        if (zero2 == 0 && sum2 < sum1 + zero1)
            return -1;
        return Math.max(sum1 + zero1, sum2 + zero2);
    }
}
```

1927. 2919.java

Path: LeetCode-main/solutions/2919. Minimum Increment Operations to Make Array Beautiful/2919.java

```
class Solution {
    public long minIncrementOperations(int[] nums, int k) {
        // the minimum operations to increase nums[i - 3] and nums[0..i - 3]
        long prev3 = 0;
        // the minimum operations to increase nums[i - 2] and nums[0..i - 2]
        long prev2 = 0;
        // the minimum operations to increase nums[i - 1] and nums[0..i - 1]
        long prev1 = 0;

        for (final int num : nums) {
            final long dp = Math.min(prev1, Math.min(prev2, prev3)) + Math.max(0, k - num);
            prev3 = prev2;
            prev2 = prev1;
            prev1 = dp;
        }

        return Math.min(prev1, Math.min(prev2, prev3));
    }
}
```

1928. 292.java

Path: LeetCode-main/solutions/292. Nim Game/292.java

```
class Solution {  
    public boolean canWinNim(int n) {  
        return n % 4 != 0;  
    }  
}
```

1929. 2920.java

Path: LeetCode-main/solutions/2920. Maximum Points After Collecting Coins From All Nodes/2920.java

```
class Solution {
    public int maximumPoints(int[][] edges, int[] coins, int k) {
        final int n = coins.length;
        List<Integer>[] graph = new List[n];
        Arrays.setAll(graph, i -> new ArrayList<>());

        Integer[][] mem = new Integer[n][MAX_HALVED + 1];

        for (int[] edge : edges) {
            final int u = edge[0];
            final int v = edge[1];
            graph[u].add(v);
            graph[v].add(u);
        }

        return dfs(graph, 0, /*prev=*/-1, coins, k, /*halved=*/0, mem);
    }

    private static final int MAX_COIN = 10000;
    private static final int MAX_HALVED = (int) (Math.log(MAX_COIN) / Math.log(2)) + 1;

    private int dfs(List<Integer>[] graph, int u, int prev, int[] coins, int k, int halved,
                    Integer[][] mem) {
        // All the children will be 0, so no need to explore.
        if (halved > MAX_HALVED)
            return 0;
        if (mem[u][halved] != null)
            return mem[u][halved];

        final int val = coins[u] / (1 << halved);
        int takeAll = val - k;
        int takeHalf = (int) Math.floor(val / 2.0);

        for (final int v : graph[u]) {
            if (v == prev)
                continue;
            takeAll += dfs(graph, v, u, coins, k, halved, mem);
            takeHalf += dfs(graph, v, u, coins, k, halved + 1, mem);
        }

        return mem[u][halved] = Math.max(takeAll, takeHalf);
    }
}
```

1930. 2921.java

Path: LeetCode-main/solutions/2921. Maximum Profitable Triplets With Increasing Prices II/2921.java

```
class FenwickTree {
    public FenwickTree(int n) {
        vals = new int[n + 1];
    }

    public void maximize(int i, int val) {
        while (i < vals.length) {
            vals[i] = Math.max(vals[i], val);
            i += lowbit(i);
        }
    }

    public int get(int i) {
        int res = 0;
        while (i > 0) {
            res = Math.max(res, vals[i]);
            i -= lowbit(i);
        }
        return res;
    }

    private int[] vals;

    private static int lowbit(int i) {
        return i & -i;
    }
}

class Solution {
    // Same as 2907. Maximum Profitable Triplets With Increasing Prices I
    public int maxProfit(int[] prices, int[] profits) {
        final int maxPrice = Arrays.stream(prices).max().getAsInt();
        int ans = -1;
        FenwickTree maxProfitTree1 = new FenwickTree(maxPrice);
        FenwickTree maxProfitTree2 = new FenwickTree(maxPrice);

        for (int i = 0; i < prices.length; ++i) {
            final int price = prices[i];
            final int profit = profits[i];
            // max(profits[i])
            final int maxProfit1 = maxProfitTree1.get(price - 1);
            // max(proftis[i]) + max(profits[j])
            final int maxProfit2 = maxProfitTree2.get(price - 1);
            maxProfitTree1.maximize(price, profit);
            if (maxProfit1 > 0)
                maxProfitTree2.maximize(price, profit + maxProfit1);
            if (maxProfit2 > 0)
                ans = Math.max(ans, profit + maxProfit2);
        }

        return ans;
    }
}
```

1931. 2923-2.java

Path: LeetCode-main/solutions/2923. Find Champion I/2923-2.java

```
class Solution {
    public int findChampion(int[][] grid) {
        for (int i = 0; i < grid.length; ++i)
            if (Arrays.stream(grid[i]).sum() == grid.length - 1)
                return i;
        return -1;
    }
}
```


1932. 2923.java

Path: LeetCode-main/solutions/2923. Find Champion I/2923.java

```
class Solution {
    public int findChampion(int[][] grid) {
        final int n = grid.length;
        int ans = -1;
        int count = 0;
        int[] inDegrees = new int[n];

        for (int i = 0; i < n; ++i)
            for (int j = 0; j < n; ++j) {
                if (i == j)
                    continue;
                if (grid[i][j] == 1)
                    ++inDegrees[j];
                else
                    ++inDegrees[i];
            }

        for (int i = 0; i < n; ++i)
            if (inDegrees[i] == 0) {
                ++count;
                ans = i;
            }

        return count > 1 ? -1 : ans;
    }
}
```

1933. 2924.java

Path: LeetCode-main/solutions/2924. Find Champion II/2924.java

```
class Solution {
    public int findChampion(int n, int[][] edges) {
        int ans = -1;
        int count = 0;
        int[] inDegrees = new int[n];

        for (int[] edge : edges) {
            final int v = edge[1];
            ++inDegrees[v];
        }

        for (int i = 0; i < n; ++i)
            if (inDegrees[i] == 0) {
                ++count;
                ans = i;
            }

        return count > 1 ? -1 : ans;
    }
}
```

1934. 2925.java

Path: LeetCode-main/solutions/2925. Maximum Score After Applying Operations on a Tree/2925.java

```
class Solution {
    public long maximumScoreAfterOperations(int[][] edges, int[] values) {
        List<Integer>[] tree = new ArrayList[values.length];

        for (int i = 0; i < values.length; ++i)
            tree[i] = new ArrayList<>();

        for (int[] edge : edges) {
            final int u = edge[0];
            final int v = edge[1];
            tree[u].add(v);
            tree[v].add(u);
        }

        return Arrays.stream(values).sum() - dfs(tree, 0, /*prev=*/-1, values);
    }

    private long dfs(List<Integer>[] tree, int u, int prev, int[] values) {
        if (u > 0 && tree[u].size() == 1)
            return values[u];
        long childrenSum = 0;
        for (final int v : tree[u])
            if (v != prev)
                childrenSum += dfs(tree, v, u, values);
        return Math.min(childrenSum, 1L * values[u]);
    }
}
```

1935. 2926.java

Path: LeetCode-main/solutions/2926. Maximum Balanced Subsequence Sum/2926.java

```
class FenwickTree {
    public FenwickTree(int n) {
        vals = new long[n + 1];
    }

    // Updates the maximum sum of subsequence ending in (i - 1) with `val`.
    public void maximize(int i, long val) {
        while (i < vals.length) {
            vals[i] = Math.max(vals[i], val);
            i += lowbit(i);
        }
    }

    // Returns the maximum sum of subsequence ending in (i - 1).
    public long get(int i) {
        long res = 0;
        while (i > 0) {
            res = Math.max(res, vals[i]);
            i -= lowbit(i);
        }
        return res;
    }

    private long[] vals;

    private static int lowbit(int i) {
        return i & -i;
    }
}

class Solution {
    public long maxBalancedSubsequenceSum(int[] nums) {
        // Let's define maxSum[i] := subsequence with the maximum sum ending in i
        // By observation:
        //     nums[i] - nums[j] >= i - j
        // => nums[i] - i >= nums[j] - j
        // So, if nums[i] - i >= nums[j] - j, where i > j,
        // maxSum[i] = max(maxSum[i], maxSum[j] + nums[i])
        long ans = Long.MIN_VALUE;
        FenwickTree tree = new FenwickTree(nums.length);

        for (Pair<Integer, Integer> pair : getPairs(nums)) {
            final int i = pair.getValue();
            final long subseqSum = tree.get(i) + nums[i];
            tree.maximize(i + 1, subseqSum);
            ans = Math.max(ans, subseqSum);
        }

        return ans;
    }

    private List<Pair<Integer, Integer>> getPairs(int[] nums) {
        List<Pair<Integer, Integer>> pairs = new ArrayList<>();
        for (int i = 0; i < nums.length; ++i)
            pairs.add(new Pair<>(nums[i] - i, i));
        pairs.sort((p1, p2) -> p1.getKey() - p2.getKey());
        return pairs;
    }
}
```

1936. 2927.java

Path: LeetCode-main/solutions/2927. Distribute Candies Among Children III/2927.java

```
class Solution {
    public long distributeCandies(int n, int limit) {
        final int limitPlusOne = limit + 1;
        final long oneChildExceedsLimit = ways(n - limitPlusOne);
        final long twoChildrenExceedLimit = ways(n - 2 * limitPlusOne);
        final long threeChildrenExceedLimit = ways(n - 3 * limitPlusOne);
        // Principle of Inclusion-Exclusion (PIE)
        return ways(n) - 3 * oneChildExceedsLimit + 3 * twoChildrenExceedLimit -
            threeChildrenExceedLimit;
    }

    private long ways(int n) {
        if (n < 0)
            return 0;
        // Stars and bars method:
        // e.g. '**|**|*' means to distribute 5 candies to 3 children, where
        // stars (*) := candies and bars (|) := dividers between children.
        return nCk(n + 2, 2);
    }

    private long nCk(int n, int k) {
        long res = 1;
        for (int i = 1; i <= k; ++i)
            res = res * (n - i + 1) / i;
        return res;
    }
}
```

1937. 2928.java

Path: LeetCode-main/solutions/2928. Distribute Candies Among Children I/2928.java

```
class Solution {
    // Returns the number of ways to distribute n candies to 3 children.
    public int distributeCandies(int n, int limit) {
        final int limitPlusOne = limit + 1;
        final int oneChildExceedsLimit = ways(n - limitPlusOne);
        final int twoChildrenExceedLimit = ways(n - 2 * limitPlusOne);
        final int threeChildrenExceedLimit = ways(n - 3 * limitPlusOne);
        // Principle of Inclusion-Exclusion (PIE)
        return ways(n) - 3 * oneChildExceedsLimit + 3 * twoChildrenExceedLimit -
            threeChildrenExceedLimit;
    }

    private int ways(int n) {
        if (n < 0)
            return 0;
        // Stars and bars method:
        // e.g. '***|**|*' means to distribute 5 candies to 3 children, where
        // stars (*) := candies and bars (|) := dividers between children.
        return nCk(n + 2, 2);
    }

    private int nCk(int n, int k) {
        int res = 1;
        for (int i = 1; i <= k; ++i)
            res = res * (n - i + 1) / i;
        return res;
    }
}
```

1938. 2929.java

Path: LeetCode-main/solutions/2929. Distribute Candies Among Children II/2929.java

```
class Solution {
    // Same as 2927. Distribute Candies Among Children III
    public long distributeCandies(int n, int limit) {
        final int limitPlusOne = limit + 1;
        final long oneChildExceedsLimit = ways(n - limitPlusOne);
        final long twoChildrenExceedLimit = ways(n - 2 * limitPlusOne);
        final long threeChildrenExceedLimit = ways(n - 3 * limitPlusOne);
        // Principle of Inclusion-Exclusion (PIE)
        return ways(n) - 3 * oneChildExceedsLimit + 3 * twoChildrenExceedLimit -
            threeChildrenExceedLimit;
    }

    private long ways(int n) {
        if (n < 0)
            return 0;
        // Stars and bars method:
        // e.g. '***|**|*' means to distribute 5 candies to 3 children, where
        // stars (*) := candies and bars (|) := dividers between children.
        return nCk(n + 2, 2);
    }

    private long nCk(int n, int k) {
        long res = 1;
        for (int i = 1; i <= k; ++i)
            res = res * (n - i + 1) / i;
        return res;
    }
}
```

1939. 293.java

Path: LeetCode-main/solutions/293. Flip Game/293.java

```
class Solution {
    public List<String> generatePossibleNextMoves(String currentState) {
        List<String> ans = new ArrayList<>();

        for (int i = 0; i + 1 < currentState.length(); ++i)
            if (currentState.charAt(i) == '+' && currentState.charAt(i + 1) == '+') {
                StringBuilder sb = new StringBuilder(currentState);
                sb.setCharAt(i, '-');
                sb.setCharAt(i + 1, '-');
                ans.add(sb.toString());
            }

        return ans;
    }
}
```


1940. 2930.java

Path: LeetCode-main/solutions/2930. Number of Strings Which Can Be Rearranged to Contain Substring/2930.java

```
class Solution {
    public int stringCount(int n) {
        // There're three invalid conditions:
        //   a. count('l') == 0
        //   b. count('e') < 2
        //   c. count('t') == 0
        //
        // By Principle of Inclusion-Exclusion (PIE):
        //   ans = allCount - a - b - c + ab + ac + bc - abc
        final long allCount = modPow(26, n);
        final long a = modPow(25, n);
        final long b = modPow(25, n);
        final long c = modPow(25, n) + n * modPow(25, n - 1);
        final long ab = modPow(24, n) + n * modPow(24, n - 1);
        final long ac = modPow(24, n);
        final long bc = modPow(24, n) + n * modPow(24, n - 1);
        final long abc = modPow(23, n) + n * modPow(23, n - 1);
        return (int) (((allCount - a - b - c + ab + ac + bc - abc) % MOD + MOD) % MOD);
    }

    private static final int MOD = 1_000_000_007;

    private long modPow(long x, long n) {
        if (n == 0)
            return 1;
        if (n % 2 == 1)
            return x * modPow(x, n - 1) % MOD;
        return modPow(x * x % MOD, n / 2);
    }
}
```

1941. 2931.java

Path: LeetCode-main/solutions/2931. Maximum Spending After Buying Items/2931.java

```
class Solution {
    public long maxSpending(int[][] values) {
        int[] sorted = Arrays.stream(values)
                                .flatMapToInt(Arrays::stream) //
                                .sorted() //
                                .toArray(); //
        return LongStream.range(0, sorted.length)
                        .map(i -> (i + 1) * sorted[(int) i]) //
                        .sum(); //
    }
}
```

1942. 2932.java

Path: LeetCode-main/solutions/2932. Maximum Strong Pair XOR I/2932.java

```
class TrieNode {
    public TrieNode[] children = new TrieNode[2];
    public int mn = Integer.MAX_VALUE;
    public int mx = Integer.MIN_VALUE;
}

class BitTrie {
    public BitTrie(int maxBit) {
        this.maxBit = maxBit;
    }

    public void insert(int num) {
        TrieNode node = root;
        for (int i = maxBit; i >= 0; --i) {
            final int bit = (int) (num >> i & 1);
            if (node.children[bit] == null)
                node.children[bit] = new TrieNode();
            node = node.children[bit];
            node.mn = Math.min(node.mn, num);
            node.mx = Math.max(node.mx, num);
        }
    }

    // Returns max(x ^ y), where |x - y| <= min(x, y).
    //
    // If x <= y, |x - y| <= min(x, y) can be written as y - x <= x.
    // So, y <= 2 * x.
    public int getMaxXor(int x) {
        int maxXor = 0;
        TrieNode node = root;
        for (int i = maxBit; i >= 0; --i) {
            final int bit = (int) (x >> i & 1);
            final int toggleBit = bit ^ 1;
            // If `node.children[toggleBit].mx > x`, it means there's a number in the
            // node that satisfies the condition to ensure that x <= y among x and y.
            // If `node.children[toggleBit].mn <= 2 * x`, it means there's a number
            // in the node that satisfies the condition for a valid y.
            if (node.children[toggleBit] != null && node.children[toggleBit].mx > x &&
                node.children[toggleBit].mn <= 2 * x) {
                maxXor = maxXor | 1 << i;
                node = node.children[toggleBit];
            } else if (node.children[bit] != null) {
                node = node.children[bit];
            } else { // There's nothing in the Bit Trie.
                return 0;
            }
        }
        return maxXor;
    }

    private int maxBit;
    private TrieNode root = new TrieNode();
}

class Solution {
    // Similar to 421. Maximum XOR of Two Numbers in an Array
    public int maximumStrongPairXor(int[] nums) {
        final int maxNum = Arrays.stream(nums).max().getAsInt();
        final int maxBit = (int) (Math.log(maxNum) / Math.log(2));
        int ans = 0;
        BitTrie bitTrie = new BitTrie(maxBit);

        for (final int num : nums)
            bitTrie.insert(num);

        for (final int num : nums)
            ans = Math.max(ans, bitTrie.getMaxXor(num));

        return ans;
    }
}
```

1943. 2933.java

Path: LeetCode-main/solutions/2933. High-Access Employees/2933.java

```
class Solution {
    public List<String> findHighAccessEmployees(List<List<String>> access_times) {
        Set<String> ans = new HashSet<>();

        Collections.sort(access_times,
            (a, b)
            -> a.get(0).equals(b.get(0)) ? a.get(1).compareTo(b.get(1))
            : a.get(0).compareTo(b.get(0)));

        for (int i = 0; i + 2 < access_times.size(); ++i) {
            String name = access_times.get(i).get(0);
            if (ans.contains(name))
                continue;
            if (!name.equals(access_times.get(i + 2).get(0)))
                continue;
            if (Integer.parseInt(access_times.get(i + 2).get(1)) -
                Integer.parseInt(access_times.get(i).get(1)) <
                100)
                ans.add(name);
        }

        return new ArrayList<>(ans);
    }
}
```

1944. 2934.java

Path: LeetCode-main/solutions/2934. Minimum Operations to Maximize Last Elements in Arrays/2934.java

```
class Solution {
    public int minOperations(int[] nums1, int[] nums2) {
        final int n = nums1.length;
        final int mn = Math.min(nums1[n - 1], nums2[n - 1]);
        final int mx = Math.max(nums1[n - 1], nums2[n - 1]);
        // the number of the minimum operations, where nums1[n - 1] is not swapped
        // with nums2[n - 1]
        int dp1 = 0;
        // the number of the minimum operations, where nums1[n - 1] is swapped with
        // nums2[n - 1]
        int dp2 = 0;

        for (int i = 0; i < n; ++i) {
            final int a = nums1[i];
            final int b = nums2[i];
            if (Math.min(a, b) > mn)
                return -1;
            if (Math.max(a, b) > mx)
                return -1;
            if (a > nums1[n - 1] || b > nums2[n - 1])
                ++dp1;
            if (a > nums2[n - 1] || b > nums1[n - 1])
                ++dp2;
        }

        return Math.min(dp1, dp2);
    }
}
```

1945. 2935.java

Path: LeetCode-main/solutions/2935. Maximum Strong Pair XOR II/2935.java

```
class TrieNode {
    public TrieNode[] children = new TrieNode[2];
    public int mn = Integer.MAX_VALUE;
    public int mx = Integer.MIN_VALUE;
}

class BitTrie {
    public BitTrie(int maxBit) {
        this.maxBit = maxBit;
    }

    public void insert(int num) {
        TrieNode node = root;
        for (int i = maxBit; i >= 0; --i) {
            final int bit = (int) (num >> i & 1);
            if (node.children[bit] == null)
                node.children[bit] = new TrieNode();
            node = node.children[bit];
            node.mn = Math.min(node.mn, num);
            node.mx = Math.max(node.mx, num);
        }
    }

    // Returns max(x ^ y), where |x - y| <= min(x, y).
    //
    // If x <= y, |x - y| <= min(x, y) can be written as y - x <= x.
    // So, y <= 2 * x.
    public int getMaxXor(int x) {
        int maxXor = 0;
        TrieNode node = root;
        for (int i = maxBit; i >= 0; --i) {
            final int bit = (int) (x >> i & 1);
            final int toggleBit = bit ^ 1;
            // If `node.children[toggleBit].mx > x`, it means there's a number in the
            // node that satisfies the condition to ensure that x <= y among x and y.
            // If `node.children[toggleBit].mn <= 2 * x`, it means there's a number
            // in the node that satisfies the condition for a valid y.
            if (node.children[toggleBit] != null && node.children[toggleBit].mx > x &&
                node.children[toggleBit].mn <= 2 * x) {
                maxXor = maxXor | 1 << i;
                node = node.children[toggleBit];
            } else if (node.children[bit] != null) {
                node = node.children[bit];
            } else { // There's nothing in the Bit Trie.
                return 0;
            }
        }
        return maxXor;
    }

    private int maxBit;
    private TrieNode root = new TrieNode();
}

class Solution {
    // Same as 2932. Maximum Strong Pair XOR I
    public int maximumStrongPairXor(int[] nums) {
        final int maxNum = Arrays.stream(nums).max().getAsInt();
        final int maxBit = (int) (Math.log(maxNum) / Math.log(2));
        int ans = 0;
        BitTrie bitTrie = new BitTrie(maxBit);

        for (final int num : nums)
            bitTrie.insert(num);

        for (final int num : nums)
            ans = Math.max(ans, bitTrie.getMaxXor(num));

        return ans;
    }
}
```

1946. 2936.java

Path: LeetCode-main/solutions/2936. Number of Equal Numbers Blocks/2936.java

```
/**
 * Definition for BigArray.
 * class BigArray {
 *     public BigArray(int[] elements);
 *     public int at(long index);
 *     public long size();
 * }
 */

class Solution {
    public int countBlocks(BigArray nums) {
        return countBlocks(nums, 0, nums.size() - 1, nums.at(0), nums.at(nums.size() - 1));
    }

    // Returns the number of maximal blocks in nums[l..r].
    private int countBlocks(BigArray nums, long l, long r, int leftValue, int rightValue) {
        if (leftValue == rightValue) // nums[l..r] are identical.
            return 1;
        if (l + 1 == r) // nums[l] != nums[r].
            return 2;
        final long m = (l + r) / 2;
        final int midValue = nums.at(m);
        // Subtract nums[m], which will be counted twice.
        return countBlocks(nums, l, m, leftValue, midValue) +
            countBlocks(nums, m, r, midValue, rightValue) - 1;
    }
}
```

1947. 2937.java

Path: LeetCode-main/solutions/2937. Make Three Strings Equal/2937.java

```
class Solution {
    public int findMinimumOperations(String s1, String s2, String s3) {
        final int minLength = Math.min(s1.length(), Math.min(s2.length(), s3.length()));
        int i = 0;
        while (i < minLength && s1.charAt(i) == s2.charAt(i) && s2.charAt(i) == s3.charAt(i))
            ++i;
        return i == 0 ? -1 : s1.length() + s2.length() + s3.length() - i * 3;
    }
}
```


1948. 2938.java

Path: LeetCode-main/solutions/2938. Separate Black and White Balls/2938.java

```
class Solution {
    public long minimumSteps(String s) {
        long ans = 0;
        int ones = 0;

        for (final char c : s.toCharArray())
            if (c == '1')
                ++ones;
            else // Move 1s to the front of the current '0'.
                ans += ones;

        return ans;
    }
}
```

1949. 2939.java

Path: LeetCode-main/solutions/2939. Maximum Xor Product/2939.java

```
class Solution {
    public int maximumXorProduct(long a, long b, int n) {
        final int MOD = 1_000_000_007;
        if (n > 0)
            for (long bit = 1L << (n - 1); bit > 0; bit >>= 1)
                // Pick a bit if it makes Math.min(a, b) larger.
                if ((Math.min(a, b) & bit) == 0) {
                    a ^= bit;
                    b ^= bit;
                }
        return (int) (a % MOD * (b % MOD) % MOD);
    }
}
```

1950. 294.java

Path: LeetCode-main/solutions/294. Flip Game II/294.java

```
class Solution {
    public boolean canWin(String currentState) {
        if (mem.containsKey(currentState))
            return mem.get(currentState);

        // If any of currentState[i:i + 2] == "++" and your friend can't win after
        // changing currentState[i:i + 2] to "--" (or "-"), then you can win.
        for (int i = 0; i + 1 < currentState.length(); ++i)
            if (currentState.charAt(i) == '+' && currentState.charAt(i + 1) == '+' &&
                !canWin(currentState.substring(0, i) + "-" + currentState.substring(i + 2))) {
                mem.put(currentState, true);
                return true;
            }

        mem.put(currentState, false);
        return false;
    }

    private Map<String, Boolean> mem = new HashMap<>();
}
```

1951. 2940.java

Path: LeetCode-main/solutions/2940. Find Building Where Alice and Bob Can Meet/2940.java

```
class Solution {
    // Similar to 2736. Maximum Sum Queries
    public int[] leftmostBuildingQueries(int[] heights, int[][] queries) {
        IndexedQuery[] indexedQueries = getIndexedQueries(queries);
        int[] ans = new int[queries.length];
        Arrays.fill(ans, -1);
        // Store indices (heightsIndex) of heights with heights[heightsIndex] in
        // descending order.
        List<Integer> stack = new ArrayList<>();

        // Iterate through queries and heights simultaneously.
        int heightsIndex = heights.length - 1;
        for (IndexedQuery indexedQuery : indexedQueries) {
            final int queryIndex = indexedQuery.queryIndex;
            final int a = indexedQuery.a;
            final int b = indexedQuery.b;
            if (a == b || heights[a] < heights[b]) {
                // 1. Alice and Bob are already in the same index (a == b) or
                // 2. Alice can jump from a -> b (heights[a] < heights[b]).
                ans[queryIndex] = b;
            } else {
                // Now, a < b and heights[a] >= heights[b].
                // Gradually add heights with an index > b to the monotonic stack.
                while (heightsIndex > b) {
                    // heights[heightsIndex] is a better candidate, given that
                    // heightsIndex is smaller than the indices in the stack and
                    // heights[heightsIndex] is larger or equal to the heights mapped in
                    // the stack.
                    while (!stack.isEmpty() && heights[stack.get(stack.size() - 1)] <= heights[heightsIndex])
                        stack.remove(stack.size() - 1);
                    stack.add(heightsIndex--);
                }
                // Binary search to find the smallest index j such that j > b and
                // heights[j] > heights[a], thereby ensuring heights[t] > heights[b].
                final int j = lastGreater(stack, a, heights);
                if (j != -1)
                    ans[queryIndex] = stack.get(j);
            }
        }
        return ans;
    }

    private record IndexedQuery(int queryIndex, int a, int b){};

    // Returns the last index i in A s.t. heights[A.get(i)] is > heights[target].
    private int lastGreater(List<Integer> A, int target, int[] heights) {
        int l = -1;
        int r = A.size() - 1;
        while (l < r) {
            final int m = (l + r + 1) / 2;
            if (heights[A.get(m)] > heights[target])
                l = m;
            else
                r = m - 1;
        }
        return l;
    }

    private IndexedQuery[] getIndexedQueries(int[][] queries) {
        IndexedQuery[] indexedQueries = new IndexedQuery[queries.length];
        for (int i = 0; i < queries.length; ++i) {
            // Make sure that a <= b.
            final int a = Math.min(queries[i][0], queries[i][1]);
            final int b = Math.max(queries[i][0], queries[i][1]);
            indexedQueries[i] = new IndexedQuery(i, a, b);
        }
        Arrays.sort(indexedQueries, Comparator.comparing(IndexedQuery::b, Comparator.reverseOrder()));
        return indexedQueries;
    }
}
```

1952. 2941.java

Path: LeetCode-main/solutions/2941. Maximum GCD-Sum of a Subarray/2941.java

```
class Solution {
    public long maxGcdSum(int[] nums, int k) {
        long ans = 0;
        // [(startIndex, gcd of subarray starting at startIndex)]
        List
```

1953. 2944-2.java

Path: LeetCode-main/solutions/2944. Minimum Number of Coins for Fruits/2944-2.java

```
class Solution {
    public int minimumCoins(int[] prices) {
        final int n = prices.length;
        int ans = 0;
        // Stores (dp[i], i), where dp[i] := the minimum number of coins to acquire
        // fruits[i:] (0-indexed).
        Queue<Pair<Integer, Integer>> minHeap =
            new PriorityQueue<>(Comparator.comparingInt(Pair::getKey)) {
                { offer(new Pair<>(0, n)); }
            };

        for (int i = n - 1; i >= 0; --i) {
            while (!minHeap.isEmpty() && minHeap.peek().getValue() > (i + 1) * 2)
                minHeap.poll();
            ans = prices[i] + minHeap.peek().getKey();
            minHeap.offer(new Pair<>(ans, i));
        }

        return ans;
    }
}
```

1954. 2944-3.java

Path: LeetCode-main/solutions/2944. Minimum Number of Coins for Fruits/2944-3.java

```
class Solution {
    public int minimumCoins(int[] prices) {
        final int n = prices.length;
        int ans = Integer.MAX_VALUE;
        // Stores (dp[i], i), where dp[i] := the minimum number of coins to acquire
        // fruits[i:] (0-indexed) in ascending order.
        Deque<Pair<Integer, Integer>> minQ = new ArrayDeque<>() {
            { offerFirst(new Pair<>(0, n)); }
        };

        for (int i = n - 1; i >= 0; --i) {
            while (!minQ.isEmpty() && minQ.peekFirst().getValue() > (i + 1) * 2)
                minQ.pollFirst();
            ans = prices[i] + minQ.peekFirst().getKey();
            while (!minQ.isEmpty() && minQ.peekLast().getKey() >= ans)
                minQ.pollLast();
            minQ.offerLast(new Pair<>(ans, i));
        }

        return ans;
    }
}
```

1955. 2944.java

Path: LeetCode-main/solutions/2944. Minimum Number of Coins for Fruits/2944.java

```
class Solution {
    public int minimumCoins(int[] prices) {
        final int n = prices.length;
        int[] dp = new int[n + 1];
        Arrays.fill(dp, Integer.MAX_VALUE);
        dp[n] = 0;

        for (int i = n - 1; i >= 0; --i)
            for (int j = i + 1; j <= Math.min((i + 1) * 2, n); ++j)
                dp[i] = Math.min(dp[i], prices[i] + dp[j]);

        return dp[0];
    }
}
```


1956. 2946.java

Path: LeetCode-main/solutions/2946. Matrix Similarity After Cyclic Shifts/2946.java

```
class Solution {
    public boolean areSimilar(int[][] mat, int k) {
        final int n = mat[0].length;
        for (int[] row : mat)
            for (int j = 0; j < n; ++j)
                if (row[j] != row[(j + k) % n])
                    return false;
        return true;
    }
}
```

1957. 2947.java

Path: LeetCode-main/solutions/2947. Count Beautiful Substrings I/2947.java

```
class Solution {
    public int beautifulSubstrings(String s, int k) {
        final int root = getRoot(k);
        int ans = 0;
        int vowels = 0;
        int vowelsMinusConsonants = 0;
        // {(vowels, vowelsMinusConsonants): count}
        Map<Pair<Integer, Integer>, Integer> prefixCount = new HashMap<>();
        prefixCount.put(new Pair<>(0, 0), 1);

        for (final char c : s.toCharArray()) {
            if (isVowel(c)) {
                vowels = (vowels + 1) % root;
                ++vowelsMinusConsonants;
            } else {
                --vowelsMinusConsonants;
            }
            Pair<Integer, Integer> prefix = new Pair<>(vowels, vowelsMinusConsonants);
            ans += prefixCount.getOrDefault(prefix, 0);
            prefixCount.merge(prefix, 1, Integer::sum);
        }

        return ans;
    }

    private boolean isVowel(char c) {
        return "aeiou".indexOf(c) != -1;
    }

    private int getRoot(int k) {
        for (int i = 1; i <= k; ++i)
            if (i * i % k == 0)
                return i;
        throw new IllegalArgumentException();
    }
}
```

1958. 2948.java

Path: LeetCode-main/solutions/2948. Make Lexicographically Smallest Array by Swapping Elements/2948.java

```
class Solution {
    public int[] lexicographicallySmallestArray(int[] nums, int limit) {
        int[] ans = new int[nums.length];
        List<List<Pair<Integer, Integer>>> numAndIndexesGroups = new ArrayList<>();

        for (Pair<Integer, Integer> numAndIndex : getNumAndIndexes(nums))
            if (numAndIndexesGroups.isEmpty() ||
                numAndIndex.getKey() -
                    numAndIndexesGroups.get(numAndIndexesGroups.size() - 1)
                        .get(numAndIndexesGroups.get(numAndIndexesGroups.size() - 1).size() - 1)
                            .getKey() >
                    limit) {
                // Start a new group.
                numAndIndexesGroups.add(new ArrayList<>(List.of(numAndIndex)));
            } else {
                // Append to the existing group.
                numAndIndexesGroups.get(numAndIndexesGroups.size() - 1).add(numAndIndex);
            }

        for (List<Pair<Integer, Integer>> numAndIndexesGroup : numAndIndexesGroups) {
            List<Integer> sortedNums = new ArrayList<>();
            List<Integer> sortedIndices = new ArrayList<>();
            for (Pair<Integer, Integer> pair : numAndIndexesGroup) {
                sortedNums.add(pair.getKey());
                sortedIndices.add(pair.getValue());
            }
            sortedIndices.sort(null);
            for (int i = 0; i < sortedNums.size(); ++i) {
                ans[sortedIndices.get(i)] = sortedNums.get(i);
            }
        }

        return ans;
    }

    private Pair<Integer, Integer>[] getNumAndIndexes(int[] nums) {
        Pair<Integer, Integer>[] numAndIndexes = new Pair[nums.length];
        for (int i = 0; i < nums.length; ++i)
            numAndIndexes[i] = new Pair<>(nums[i], i);
        Arrays.sort(numAndIndexes, Comparator.comparingInt(Pair::getKey));
        return numAndIndexes;
    }
}
```

1959. 2949.java

Path: LeetCode-main/solutions/2949. Count Beautiful Substrings II/2949.java

```
class Solution {
    // Same as 2947. Count Beautiful Substrings I
    public int beautifulSubstrings(String s, int k) {
        final int root = getRoot(k);
        int ans = 0;
        int vowels = 0;
        int vowelsMinusConsonants = 0;
        // {(vowels, vowelsMinusConsonants): count}
        Map<Pair<Integer, Integer>, Integer> prefixCount = new HashMap<>();
        prefixCount.put(new Pair<>(0, 0), 1);

        for (final char c : s.toCharArray()) {
            if (isVowel(c)) {
                vowels = (vowels + 1) % root;
                ++vowelsMinusConsonants;
            } else {
                --vowelsMinusConsonants;
            }
            Pair<Integer, Integer> prefix = new Pair<>(vowels, vowelsMinusConsonants);
            ans += prefixCount.getOrDefault(prefix, 0);
            prefixCount.merge(prefix, 1, Integer::sum);
        }

        return ans;
    }

    private boolean isVowel(char c) {
        return "aeiou".indexOf(c) != -1;
    }

    private int getRoot(int k) {
        for (int i = 1; i <= k; ++i)
            if (i * i % k == 0)
                return i;
        throw new IllegalArgumentException();
    }
}
```

1960. 295.java

Path: LeetCode-main/solutions/295. Find Median from Data Stream/295.java

```
class MedianFinder {
    public void addNum(int num) {
        if (maxHeap.isEmpty() || num <= maxHeap.peek())
            maxHeap.offer(num);
        else
            minHeap.offer(num);

        // Balance the two heaps s.t.
        // |maxHeap| >= |minHeap| and |maxHeap| - |minHeap| <= 1.
        if (maxHeap.size() < minHeap.size())
            maxHeap.offer(minHeap.poll());
        else if (maxHeap.size() - minHeap.size() > 1)
            minHeap.offer(maxHeap.poll());
    }

    public double findMedian() {
        if (maxHeap.size() == minHeap.size())
            return (double) (maxHeap.peek() + minHeap.peek()) / 2.0;
        return (double) maxHeap.peek();
    }

    private Queue<Integer> maxHeap = new PriorityQueue<>(Collections.reverseOrder());
    private Queue<Integer> minHeap = new PriorityQueue<>();
}
```

1961. 2950.java

Path: LeetCode-main/solutions/2950. Number of Divisible Substrings/2950.java

```
class Solution {
    public int countDivisibleSubstrings(String word) {
        // Let  $f(c) = d$ , where  $d = 1, 2, \dots, 9$ .
        // Rephrase the question to return the number of substrings that satisfy
        //  $f(c_1) + f(c_2) + \dots + f(c_k) / k = \text{avg}$ 
        //  $\Rightarrow f(c_1) + f(c_2) + \dots + f(c_k) - k * \text{avg}$ , where  $\text{avg}$  in  $[1, 9]$ .
        int ans = 0;

        for (int avg = 1; avg <= 9; ++avg) {
            int prefix = 0;
            Map<Integer, Integer> prefixCount = new HashMap<>();
            prefixCount.put(0, 1);
            for (final char c : word.toCharArray()) {
                prefix += f(c) - avg;
                ans += prefixCount.getOrDefault(prefix, 0);
                prefixCount.merge(prefix, 1, Integer::sum);
            }
        }

        return ans;
    }

    private int f(char c) {
        return 9 - ('z' - c) / 3;
    }
}
```

1962. 2951.java

Path: LeetCode-main/solutions/2951. Find the Peaks/2951.java

```
class Solution {
    public List<Integer> findPeaks(int[] mountain) {
        List<Integer> ans = new ArrayList<>();
        for (int i = 1; i + 1 < mountain.length; ++i)
            if (mountain[i] > mountain[i - 1] && mountain[i] > mountain[i + 1])
                ans.add(i);
        return ans;
    }
}
```

1963. 2952.java

Path: LeetCode-main/solutions/2952. Minimum Number of Coins to be Added/2952.java

```
class Solution {
    // Same as 330. Patching Array
    public int minimumAddedCoins(int[] coins, int target) {
        int ans = 0;
        int i = 0;        // coins' index
        long miss = 1; // the minimum sum in [1, n] we might miss

        Arrays.sort(coins);

        while (miss <= target)
            if (i < coins.length && coins[i] <= miss) {
                miss += coins[i++];
            } else {
                // Greedily add `miss` itself to increase the range from
                // [1, miss) to [1, 2 * miss).
                miss += miss;
                ++ans;
            }

        return ans;
    }
}
```


1964. 2953.java

Path: LeetCode-main/solutions/2953. Count Complete Substrings/2953.java

```
class Solution {
    public int countCompleteSubstrings(String word, int k) {
        final int uniqueLetters = word.chars().boxed().collect(Collectors.toSet()).size();
        int ans = 0;

        for (int windowSize = k; windowSize <= k * uniqueLetters && windowSize <= word.length();
            windowSize += k) {
            ans += countCompleteStrings(word, k, windowSize);
        }

        return ans;
    }

    // Returns the number of complete substrings of `windowSize` of `word`.
    private int countCompleteStrings(final String word, int k, int windowSize) {
        int res = 0;
        int countLetters = 0; // the number of letters in the running substring
        int[] count = new int[26];

        for (int i = 0; i < word.length(); ++i) {
            ++count[word.charAt(i) - 'a'];
            ++countLetters;
            if (i > 0 && Math.abs(word.charAt(i) - word.charAt(i - 1)) > 2) {
                count = new int[26];
                // Start a new substring starting at word[i].
                ++count[word.charAt(i) - 'a'];
                countLetters = 1;
            }
            if (countLetters == windowSize + 1) {
                --count[word.charAt(i - windowSize) - 'a'];
                --countLetters;
            }
            if (countLetters == windowSize)
                res += Arrays.stream(count).allMatch(freq -> freq == 0 || freq == k) ? 1 : 0;
        }

        return res;
    }
}
```

1965. 2954.java

Path: LeetCode-main/solutions/2954. Count the Number of Infection Sequences/2954.java

```
class Solution {
    public int numberOfSequence(int n, int[] sick) {
        final long[][] factAndInvFact = getFactAndInvFact(n - sick.length);
        final long[] fact = factAndInvFact[0];
        final long[] invFact = factAndInvFact[1];
        long ans = fact[n - sick.length]; // the number of infected children
        int prevSick = -1;

        for (int i = 0; i < sick.length; ++i) {
            // The segment [prevSick + 1, sick - 1] are the current non-infected
            // children.
            final int nonInfected = sick[i] - prevSick - 1;
            prevSick = sick[i];
            if (nonInfected == 0)
                continue;
            ans *= invFact[nonInfected];
            ans %= MOD;
            if (i > 0) {
                // There're two choices per second since the children at the two
                // endpoints can both be the infect candidates. So, there are
                //  $2^{\text{nonInfected} - 1}$  ways to infect all children in the current
                // segment.
                ans *= modPow(2, nonInfected - 1);
                ans %= MOD;
            }
        }

        final int nonInfected = n - sick[sick.length - 1] - 1;
        ans *= invFact[nonInfected];
        return (int) (ans % MOD);
    }

    private static final int MOD = 1_000_000_007;

    private long[][] getFactAndInvFact(int n) {
        long[] fact = new long[n + 1];
        long[] invFact = new long[n + 1];
        long[] inv = new long[n + 1];
        fact[0] = invFact[0] = 1;
        inv[0] = inv[1] = 1;
        for (int i = 1; i <= n; ++i) {
            if (i >= 2)
                inv[i] = MOD - MOD / i * inv[MOD % i] % MOD;
            fact[i] = fact[i - 1] * i % MOD;
            invFact[i] = invFact[i - 1] * inv[i] % MOD;
        }
        return new long[][] {fact, invFact};
    }

    private long modPow(long x, long n) {
        if (n == 0)
            return 1;
        if (n % 2 == 1)
            return x * modPow(x % MOD, (n - 1)) % MOD;
        return modPow(x * x % MOD, (n / 2)) % MOD;
    }
}
```

1966. 2955.java

Path: LeetCode-main/solutions/2955. Number of Same-End Substrings/2955.java

```
class Solution {
    public int[] sameEndSubstringCount(String s, int[][] queries) {
        int[] ans = new int[queries.length];
        int[] count = new int[26];
        // counts[i] := the count of s[0..i)
        int[][] counts = new int[s.length() + 1][26];

        for (int i = 0; i < s.length(); i++) {
            ++count[s.charAt(i) - 'a'];
            System.arraycopy(count, 0, counts[i + 1], 0, 26);
        }

        for (int i = 0; i < queries.length; ++i) {
            final int l = queries[i][0];
            final int r = queries[i][1];
            int sameEndCount = 0;
            for (char c = 'a'; c <= 'z'; ++c) {
                // the count of s[0..r + 1) - the count of s[0..l)
                // = the count of s[l..r]
                final int freq = counts[r + 1][c - 'a'] - counts[l][c - 'a'];
                // C(freq, 2) + freq
                // = freq * (freq - 1) / 2 + freq
                // = freq * (freq + 1) / 2
                sameEndCount += freq * (freq + 1) / 2;
            }
            ans[i] = sameEndCount;
        }

        return ans;
    }
}
```

1967. 2956.java

Path: LeetCode-main/solutions/2956. Find Common Elements Between Two Arrays/2956.java

```
class Solution {
    public int[] findIntersectionValues(int[] nums1, int[] nums2) {
        Set<Integer> nums1Set = Arrays.stream(nums1).boxed().collect(Collectors.toSet());
        Set<Integer> nums2Set = Arrays.stream(nums2).boxed().collect(Collectors.toSet());
        final int ans1 = (int) Arrays.stream(nums1).filter(nums2Set::contains).count();
        final int ans2 = (int) Arrays.stream(nums2).filter(nums1Set::contains).count();
        return new int[] {ans1, ans2};
    }
}
```

1968. 2957.java

Path: LeetCode-main/solutions/2957. Remove Adjacent Almost-Equal Characters/2957.java

```
class Solution {
    public int removeAlmostEqualCharacters(String word) {
        int ans = 0;

        int i = 1;
        while (i < word.length())
            if (Math.abs(word.charAt(i) - word.charAt(i - 1)) <= 1) {
                ++ans;
                i += 2;
            } else {
                i += 1;
            }

        return ans;
    }
}
```

1969. 2958.java

Path: LeetCode-main/solutions/2958. Length of Longest Subarray With at Most K Frequency/2958.java

```
class Solution {
    public int maxSubarrayLength(int[] nums, int k) {
        int ans = 0;
        Map<Integer, Integer> count = new HashMap<>();

        for (int l = 0, r = 0; r < nums.length; ++r) {
            count.merge(nums[r], 1, Integer::sum);
            while (count.get(nums[r]) == k + 1)
                count.merge(nums[l++], -1, Integer::sum);
            ans = Math.max(ans, r - l + 1);
        }

        return ans;
    }
}
```

1970. 2959.java

Path: LeetCode-main/solutions/2959. Number of Possible Sets of Closing Branches/2959.java

```
class Solution {
    public int numberOfSets(int n, int maxDistance, int[][] roads) {
        final int maxMask = 1 << n;
        int ans = 0;

        for (int mask = 0; mask < maxMask; ++mask)
            if (floydWarshall(n, maxDistance, roads, mask) <= maxDistance)
                ++ans;

        return ans;
    }

    private int floydWarshall(int n, int maxDistanceThreshold, int[][] roads, int mask) {
        int maxDistance = 0;
        int[][] dist = new int[n][n];
        Arrays.stream(dist).forEach(A -> Arrays.fill(A, maxDistanceThreshold + 1));

        for (int i = 0; i < n; ++i)
            if ((mask >> i & 1) == 1)
                dist[i][i] = 0;

        for (int[] road : roads) {
            final int u = road[0];
            final int v = road[1];
            final int w = road[2];
            if ((mask >> u & 1) == 1 && (mask >> v & 1) == 1) {
                dist[u][v] = Math.min(dist[u][v], w);
                dist[v][u] = Math.min(dist[v][u], w);
            }
        }

        for (int k = 0; k < n; ++k)
            if ((mask >> k & 1) == 1)
                for (int i = 0; i < n; ++i)
                    if ((mask >> i & 1) == 1)
                        for (int j = 0; j < n; ++j)
                            if ((mask >> j & 1) == 1)
                                dist[i][j] = Math.min(dist[i][j], dist[i][k] + dist[k][j]);

        for (int i = 0; i < n; ++i)
            if ((mask >> i & 1) == 1)
                for (int j = i + 1; j < n; ++j)
                    if ((mask >> j & 1) == 1)
                        maxDistance = Math.max(maxDistance, dist[i][j]);

        return maxDistance;
    }
}
```

1971. 296.java

Path: LeetCode-main/solutions/296. Best Meeting Point/296.java

```
class Solution {
    public int minTotalDistance(int[][] grid) {
        final int m = grid.length;
        final int n = grid[0].length;
        List<Integer> I = new ArrayList<>(); // i indices s.t. grid[i][j] == 1
        List<Integer> J = new ArrayList<>(); // j indices s.t. grid[i][j] == 1

        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j)
                if (grid[i][j] == 1)
                    I.add(i);

        for (int j = 0; j < n; ++j)
            for (int i = 0; i < m; ++i)
                if (grid[i][j] == 1)
                    J.add(j);

        // sum(i - median(I)) + sum(j - median(J))
        return minTotalDistance(I) + minTotalDistance(J);
    }

    private int minTotalDistance(List<Integer> grid) {
        int sum = 0;
        int i = 0;
        int j = grid.size() - 1;
        while (i < j)
            sum += grid.get(j--) - grid.get(i++);
        return sum;
    }
}
```


1972. 2960.java

Path: LeetCode-main/solutions/2960. Count Tested Devices After Test Operations/2960.java

```
class Solution {  
    public int countTestedDevices(int[] batteryPercentages) {  
        int ans = 0;  
  
        for (final int batteryPercentage : batteryPercentages)  
            if (batteryPercentage - ans > 0)  
                ++ans;  
  
        return ans;  
    }  
}
```

1973. 2961.java

Path: LeetCode-main/solutions/2961. Double Modular Exponentiation/2961.java

```
class Solution {
    public List<Integer> getGoodIndices(int[][] variables, int target) {
        List<Integer> ans = new ArrayList<>();
        for (int i = 0; i < variables.length; ++i) {
            final int a = variables[i][0];
            final int b = variables[i][1];
            final int c = variables[i][2];
            final int m = variables[i][3];
            if (modPow(modPow(a, b, 10), c, m) == target)
                ans.add(i);
        }
        return ans;
    }

    private long modPow(long x, long n, int mod) {
        if (n == 0)
            return 1;
        if (n % 2 == 1)
            return x * modPow(x % mod, (n - 1), mod) % mod;
        return modPow(x * x % mod, (n / 2), mod) % mod;
    }
}
```

1974. 2962.java

Path: LeetCode-main/solutions/2962. Count Subarrays Where Max Element Appears at Least K Times/2962.java

```
class Solution {
    public long countSubarrays(int[] nums, int k) {
        final int maxNum = Arrays.stream(nums).max().getAsInt();
        long ans = 0;
        int count = 0;

        for (int l = 0, r = 0; r < nums.length; ++r) {
            if (nums[r] == maxNum)
                ++count;
            // Keep the window to include k - 1 times of the maximum number.
            while (count == k)
                if (nums[l++] == maxNum)
                    --count;
            // If l > 0, nums[l..r] has k - 1 times of the maximum number. For any
            // subarray nums[i..r], where i < l, it will have at least k times of the
            // maximum number, since nums[l - 1] equals the maximum number.
            ans += l;
        }

        return ans;
    }
}
```

1975. 2963.java

Path: LeetCode-main/solutions/2963. Count the Number of Good Partitions/2963.java

```
class Solution {
    public int numberOfGoodPartitions(int[] nums) {
        final int MOD = 1_000_000_007;
        int ans = 1;

        // lastSeen[num] := the index of the last time `num` appeared
        HashMap<Integer, Integer> lastSeen = new HashMap<>();

        for (int i = 0; i < nums.length; ++i)
            lastSeen.put(nums[i], i);

        // Track the maximum right index of each running partition by ensuring that
        // the first and last occurrences of a number fall within the same
        // partition.
        int maxRight = 0;
        for (int i = 0; i < nums.length; ++i) {
            if (i > maxRight)
                // Start a new partition that starts from nums[i].
                // Each partition doubles the total number of good partitions.
                ans = (int) ((ans * 2L) % MOD);
            maxRight = Math.max(maxRight, lastSeen.get(nums[i]));
        }

        return ans;
    }
}
```

1976. 2964.java

Path: LeetCode-main/solutions/2964. Number of Divisible Triplet Sums/2964.java

```
class Solution {
    // Similar to 1995. Count Special Quadruplets
    public int divisibleTripletCount(int[] nums, int d) {
        int ans = 0;
        Map<Integer, Integer> count = new HashMap<>();

        for (int j = nums.length - 1; j > 0; --j) { // 'j' also represents k.
            for (int i = j - 1; i >= 0; --i)
                ans += count.getDefault((- (nums[i] + nums[j]) % d + d) % d, 0);
            count.merge(nums[j] % d, 1, Integer::sum); // j := k
        }

        return ans;
    }
}
```

1977. 2965.java

Path: LeetCode-main/solutions/2965. Find Missing and Repeated Values/2965.java

```
class Solution {
    public int[] findMissingAndRepeatedValues(int[][] grid) {
        final int n = grid.length;
        final int nSquared = n * n;
        int[] count = new int[nSquared + 1];

        for (int[] row : grid)
            for (final int num : row)
                ++count[num];

        int repeated = -1;
        int missing = -1;

        for (int i = 1; i <= nSquared; ++i) {
            if (count[i] == 2)
                repeated = i;
            if (count[i] == 0)
                missing = i;
        }

        return new int[] {repeated, missing};
    }
}
```

1978. 2966.java

Path: LeetCode-main/solutions/2966. Divide Array Into Arrays With Max Difference/2966.java

```
class Solution {
    public int[][] divideArray(int[] nums, int k) {
        int[][] ans = new int[nums.length / 3][3];

        Arrays.sort(nums);

        for (int i = 2; i < nums.length; i += 3) {
            if (nums[i] - nums[i - 2] > k)
                return new int[0][];
            ans[i / 3] = new int[] {nums[i - 2], nums[i - 1], nums[i]};
        }

        return ans;
    }
}
```

1979. 2967.java

Path: LeetCode-main/solutions/2967. Minimum Cost to Make Array Equalindromic/2967.java

```
class Solution {
    public long minimumCost(int[] nums) {
        Arrays.sort(nums);
        final int median = nums[nums.length / 2];
        final int nextPalindrome = getPalindrome(median, 1);
        final int prevPalindrome = getPalindrome(median, -1);
        return Math.min(cost(nums, nextPalindrome), cost(nums, prevPalindrome));
    }

    // Returns the cost to change all the numbers to `palindrome`.
    private long cost(int[] nums, int palindrome) {
        return Arrays.stream(nums).mapToLong(num -> Math.abs(palindrome - num)).sum();
    }

    // Returns the palindrome `p`, where  $p = \text{num} + a * \text{delta}$  and  $a > 0$ .
    private int getPalindrome(int num, int delta) {
        while (!isPalindrome(num))
            num += delta;
        return num;
    }

    private boolean isPalindrome(int num) {
        final String original = Integer.toString(num);
        final String reversed = new StringBuilder(original).reverse().toString();
        return original.equals(reversed);
    }
}
```


1980. 2968.java

Path: LeetCode-main/solutions/2968. Apply Operations to Maximize Frequency Score/2968.java

```
class Solution {
    public int maxFrequencyScore(int[] nums, long k) {
        int ans = 0;
        long cost = 0;

        Arrays.sort(nums);

        // For a window [l, r], the best choice to make the numbers in the range
        // equal is to make them all equal to the median in this range.
        for (int l = 0, r = 0; r < nums.length; ++r) {
            cost += nums[r] - nums[(l + r) / 2];
            while (cost > k)
                cost -= nums[(l + r + 1) / 2] - nums[l++];
            ans = Math.max(ans, r - l + 1);
        }

        return ans;
    }
}
```

1981. 2969-2.java

Path: LeetCode-main/solutions/2969. Minimum Number of Coins for Fruits II/2969-2.java

```
class Solution {
    // Same as 2944. Minimum Number of Coins for Fruits
    public int minimumCoins(int[] prices) {
        final int n = prices.length;
        int ans = 0;
        // Stores (dp[i], i), where dp[i] := the minimum number of coins to acquire
        // fruits[i:] (0-indexed).
        Queue<Pair<Integer, Integer>> minHeap =
            new PriorityQueue<>(Comparator.comparingInt(Pair::getKey)) {
                { offer(new Pair<>(0, n)); }
            };

        for (int i = n - 1; i >= 0; --i) {
            while (!minHeap.isEmpty() && minHeap.peek().getValue() > (i + 1) * 2)
                minHeap.poll();
            ans = prices[i] + minHeap.peek().getKey();
            minHeap.offer(new Pair<>(ans, i));
        }

        return ans;
    }
}
```

1982. 2969-3.java

Path: LeetCode-main/solutions/2969. Minimum Number of Coins for Fruits II/2969-3.java

```
class Solution {
    // Same as 2944. Minimum Number of Coins for Fruits
    public int minimumCoins(int[] prices) {
        final int n = prices.length;
        int ans = Integer.MAX_VALUE;
        // Stores (dp[i], i), where dp[i] := the minimum number of coins to acquire
        // fruits[i:] (0-indexed) in ascending order.
        Deque<Pair<Integer, Integer>> minQ = new ArrayDeque<>() {
            { offerFirst(new Pair<>(0, n)); }
        };

        for (int i = n - 1; i >= 0; --i) {
            while (!minQ.isEmpty() && minQ.peekFirst().getValue() > (i + 1) * 2)
                minQ.pollFirst();
            ans = prices[i] + minQ.peekFirst().getKey();
            while (!minQ.isEmpty() && minQ.peekLast().getKey() >= ans)
                minQ.pollLast();
            minQ.offerLast(new Pair<>(ans, i));
        }

        return ans;
    }
}
```

1983. 2969.java

Path: LeetCode-main/solutions/2969. Minimum Number of Coins for Fruits II/2969.java

```
class Solution {
    // Same as 2944. Minimum Number of Coins for Fruits
    public int minimumCoins(int[] prices) {
        final int n = prices.length;
        int[] dp = new int[n + 1];
        Arrays.fill(dp, Integer.MAX_VALUE);
        dp[n] = 0;

        for (int i = n - 1; i >= 0; --i)
            for (int j = i + 1; j <= Math.min((i + 1) * 2, n); ++j)
                dp[i] = Math.min(dp[i], prices[i] + dp[j]);

        return dp[0];
    }
}
```

1984. 297-2.java

Path: LeetCode-main/solutions/297. Serialize and Deserialize Binary Tree/297-2.java

```
public class Codec {
    // Encodes a tree to a single string.
    public String serialize(TreeNode root) {
        StringBuilder sb = new StringBuilder();
        preorder(root, sb);
        return sb.toString();
    }

    // Decodes your encoded data to tree.
    public TreeNode deserialize(String data) {
        final String[] vals = data.split(" ");
        Queue<String> q = new ArrayDeque<>(List.of(vals));
        return preorder(q);
    }

    private void preorder(TreeNode root, StringBuilder sb) {
        if (root == null) {
            sb.append("n ");
            return;
        }

        sb.append(root.val).append(" ");
        preorder(root.left, sb);
        preorder(root.right, sb);
    }

    private TreeNode preorder(Queue<String> q) {
        final String s = q.poll();
        if (s.equals("n"))
            return null;

        TreeNode root = new TreeNode(Integer.parseInt(s));
        root.left = preorder(q);
        root.right = preorder(q);
        return root;
    }
}
```

1985. 297.java

Path: LeetCode-main/solutions/297. Serialize and Deserialize Binary Tree/297.java

```
public class Codec {
    // Encodes a tree to a single string.
    public String serialize(TreeNode root) {
        if (root == null)
            return "";

        StringBuilder sb = new StringBuilder();
        Queue<TreeNode> q = new LinkedList<>(Arrays.asList(root));

        while (!q.isEmpty()) {
            TreeNode node = q.poll();
            if (node == null) {
                sb.append("n ");
            } else {
                sb.append(node.val).append(" ");
                q.offer(node.left);
                q.offer(node.right);
            }
        }

        return sb.toString();
    }

    // Decodes your encoded data to tree.
    public TreeNode deserialize(String data) {
        if (data.equals(""))
            return null;

        final String[] vals = data.split(" ");
        TreeNode root = new TreeNode(Integer.parseInt(vals[0]));
        Queue<TreeNode> q = new LinkedList<>(Arrays.asList(root));

        for (int i = 1; i < vals.length; i += 2) {
            TreeNode node = q.poll();
            if (!vals[i].equals("n")) {
                node.left = new TreeNode(Integer.parseInt(vals[i]));
                q.offer(node.left);
            }
            if (!vals[i + 1].equals("n")) {
                node.right = new TreeNode(Integer.parseInt(vals[i + 1]));
                q.offer(node.right);
            }
        }

        return root;
    }
}
```

1986. 2970-2.java

Path: LeetCode-main/solutions/2970. Count the Number of Incremovable Subarrays I/2970-2.java

```
class Solution {
    public int incremovableSubarrayCount(int[] nums) {
        final int n = nums.length;
        final int startIndex = getStartIndexOfSuffix(nums);
        // If the complete array is strictly increasing, the total number of ways we
        // can remove elements equals the total number of possible subarrays.
        if (startIndex == 0)
            return n * (n + 1) / 2;

        // The valid removals starting from nums[0] include nums[0..startIndex - 1],
        // nums[0..startIndex], ..., nums[0..n).
        int ans = n - startIndex + 1;

        // Enumerate each prefix subarray that is strictly increasing.
        for (int i = 0, j = startIndex; i < startIndex; ++i) {
            if (i > 0 && nums[i] <= nums[i - 1])
                break;
            // Since nums[0..i] is strictly increasing, move j to the place such that
            // nums[j] > nums[i]. The valid removals will then be nums[i + 1..j - 1],
            // nums[i + 1..j], ..., nums[i + 1..n).
            while (j < n && nums[i] >= nums[j])
                ++j;
            ans += n - j + 1;
        }

        return ans;
    }

    // Returns the start index i of the suffix subarray such that nums[i..n) is
    // strictly increasing.
    private int getStartIndexOfSuffix(int[] nums) {
        for (int i = nums.length - 2; i >= 0; --i)
            if (nums[i] >= nums[i + 1])
                return i + 1;
        return 0;
    }

    private int firstGreater(int[] arr, int startIndex, int target) {
        final int i = Arrays.binarySearch(arr, startIndex, arr.length, target + 1);
        return i < 0 ? -i - 1 : i;
    }
}
```

1987. 2970.java

Path: LeetCode-main/solutions/2970. Count the Number of Incremovable Subarrays I/2970.java

```
class Solution {
    public int incremovableSubarrayCount(int[] nums) {
        final int n = nums.length;
        final int startIndex = getStartIndexOfSuffix(nums);
        // If the complete array is strictly increasing, the total number of ways we
        // can remove elements equals the total number of possible subarrays.
        if (startIndex == 0)
            return n * (n + 1) / 2;

        // The valid removals starting from nums[0] include nums[0..startIndex - 1],
        // nums[0..startIndex], ..., nums[0..n).
        int ans = n - startIndex + 1;

        // Enumerate each prefix subarray that is strictly increasing.
        for (int i = 0; i < startIndex; ++i) {
            if (i > 0 && nums[i] <= nums[i - 1])
                break;
            // Since nums[0..i] is strictly increasing, find the first index j in
            // nums[startIndex..n) such that nums[j] > nums[i]. The valid removals
            // will then be nums[i + 1..j - 1], nums[i + 1..j], ..., nums[i + 1..n).
            ans += n - firstGreater(nums, startIndex, nums[i]) + 1;
        }

        return ans;
    }

    // Returns the start index i of the suffix subarray such that nums[i..n) is
    // strictly increasing.
    private int getStartIndexOfSuffix(int[] nums) {
        for (int i = nums.length - 2; i >= 0; --i)
            if (nums[i] >= nums[i + 1])
                return i + 1;
        return 0;
    }

    private int firstGreater(int[] arr, int startIndex, int target) {
        final int i = Arrays.binarySearch(arr, startIndex, arr.length, target + 1);
        return i < 0 ? -i - 1 : i;
    }
}
```


1988. 2971.java

Path: LeetCode-main/solutions/2971. Find Polygon With the Largest Perimeter/2971.java

```
class Solution {
    public long largestPerimeter(int[] nums) {
        long prefix = Arrays.stream(nums).asLongStream().sum();

        Arrays.sort(nums);

        for (int i = nums.length - 1; i >= 2; --i) {
            prefix -= nums[i];
            // Let nums[i] be the longest side. Check if the sum of all the edges with
            // length no longer than nums[i] > nums[i].
            if (prefix > nums[i])
                return prefix + nums[i];
        }

        return -1;
    }
}
```

1989. 2972-2.java

Path: LeetCode-main/solutions/2972. Count the Number of Incremovable Subarrays II/2972-2.java

```
class Solution {
    // Same as 2970. Count the Number of Incremovable Subarrays I
    public long incremovableSubarrayCount(int[] nums) {
        final int n = nums.length;
        final int startIndex = getStartIndexOfSuffix(nums);
        // If the complete array is strictly increasing, the total number of ways we
        // can remove elements equals the total number of possible subarrays.
        if (startIndex == 0)
            return (long) n * (n + 1) / 2;

        // The valid removals starting from nums[0] include nums[0..startIndex - 1],
        // nums[0..startIndex], ..., nums[0..n).
        long ans = n - startIndex + 1;

        // Enumerate each prefix subarray that is strictly increasing.
        for (int i = 0, j = startIndex; i < startIndex; ++i) {
            if (i > 0 && nums[i] <= nums[i - 1])
                break;
            // Since nums[0..i] is strictly increasing, move j to the place such that
            // nums[j] > nums[i]. The valid removals will then be nums[i + 1..j - 1],
            // nums[i + 1..j], ..., nums[i + 1..n).
            while (j < n && nums[i] >= nums[j])
                ++j;
            ans += n - j + 1;
        }

        return ans;
    }

    // Returns the start index i of the suffix subarray such that nums[i..n) is
    // strictly increasing.
    private int getStartIndexOfSuffix(int[] nums) {
        for (int i = nums.length - 2; i >= 0; --i)
            if (nums[i] >= nums[i + 1])
                return i + 1;
        return 0;
    }

    private int firstGreater(int[] arr, int startIndex, int target) {
        final int i = Arrays.binarySearch(arr, startIndex, arr.length, target + 1);
        return i < 0 ? -i - 1 : i;
    }
}
```

1990. 2972.java

Path: LeetCode-main/solutions/2972. Count the Number of Incremovable Subarrays II/2972.java

```
class Solution {
    // Same as 2970. Count the Number of Incremovable Subarrays I
    public long incremovableSubarrayCount(int[] nums) {
        final int n = nums.length;
        final int startIndex = getStartIndexOfSuffix(nums);
        // If the complete array is strictly increasing, the total number of ways we
        // can remove elements equals the total number of possible subarrays.
        if (startIndex == 0)
            return (long) n * (n + 1) / 2;

        // The valid removals starting from nums[0] include nums[0..startIndex - 1],
        // nums[0..startIndex], ..., nums[0..n).
        long ans = n - startIndex + 1;

        // Enumerate each prefix subarray that is strictly increasing.
        for (int i = 0; i < startIndex; ++i) {
            if (i > 0 && nums[i] <= nums[i - 1])
                break;
            // Since nums[0..i] is strictly increasing, find the first index j in
            // nums[startIndex..n) such that nums[j] > nums[i]. The valid removals
            // will then be nums[i + 1..j - 1], nums[i + 1..j], ..., nums[i + 1..n).
            ans += n - firstGreater(nums, startIndex, nums[i]) + 1;
        }

        return ans;
    }

    // Returns the start index i of the suffix subarray such that nums[i..n) is
    // strictly increasing.
    private int getStartIndexOfSuffix(int[] nums) {
        for (int i = nums.length - 2; i >= 0; --i)
            if (nums[i] >= nums[i + 1])
                return i + 1;
        return 0;
    }

    private int firstGreater(int[] arr, int startIndex, int target) {
        final int i = Arrays.binarySearch(arr, startIndex, arr.length, target + 1);
        return i < 0 ? -i - 1 : i;
    }
}
```

1991. 2973.java

Path: LeetCode-main/solutions/2973. Find Number of Coins to Place in Tree Nodes/2973.java

```
class ChildCost {
    public ChildCost(int cost) {
        if (cost > 0)
            maxPosCosts.add(cost);
        else
            minNegCosts.add(cost);
    }

    public void update(ChildCost childCost) {
        numNodes += childCost.numNodes;
        maxPosCosts.addAll(childCost.maxPosCosts);
        minNegCosts.addAll(childCost.minNegCosts);
        maxPosCosts.sort(Comparator.reverseOrder());
        minNegCosts.sort(Comparator.naturalOrder());
        if (maxPosCosts.size() > 3)
            maxPosCosts = maxPosCosts.subList(0, 3);
        if (minNegCosts.size() > 2)
            minNegCosts = minNegCosts.subList(0, 2);
    }

    public long maxProduct() {
        if (numNodes < 3)
            return 1;
        if (maxPosCosts.isEmpty())
            return 0;
        long res = 0;
        if (maxPosCosts.size() == 3)
            res = (long) maxPosCosts.get(0) * maxPosCosts.get(1) * maxPosCosts.get(2);
        if (minNegCosts.size() == 2)
            res = Math.max(res, (long) minNegCosts.get(0) * minNegCosts.get(1) * maxPosCosts.get(0));
        return res;
    }

    private int numNodes = 1;
    private List<Integer> maxPosCosts = new ArrayList<>();
    private List<Integer> minNegCosts = new ArrayList<>();
}

class Solution {
    public long[] placedCoins(int[][] edges, int[] cost) {
        final int n = cost.length;
        long[] ans = new long[n];
        List<Integer>[] tree = new List[n];

        for (int i = 0; i < n; i++)
            tree[i] = new ArrayList<>();

        for (int[] edge : edges) {
            final int u = edge[0];
            final int v = edge[1];
            tree[u].add(v);
            tree[v].add(u);
        }

        dfs(tree, 0, /*prev=*/-1, cost, ans);
        return ans;
    }

    private ChildCost dfs(List<Integer>[] tree, int u, int prev, int[] cost, long[] ans) {
        ChildCost res = new ChildCost(cost[u]);
        for (final int v : tree[u])
            if (v != prev)
                res.update(dfs(tree, v, u, cost, ans));
        ans[u] = res.maxProduct();
        return res;
    }
}
```

1992. 2974.java

Path: LeetCode-main/solutions/2974. Minimum Number Game/2974.java

```
class Solution {
    public int[] numberGame(int[] nums) {
        Arrays.sort(nums);

        for (int i = 0; i < nums.length; i += 2) {
            final int temp = nums[i];
            nums[i] = nums[i + 1];
            nums[i + 1] = temp;
        }

        return nums;
    }
}
```

1993. 2975.java

Path: LeetCode-main/solutions/2975. Maximum Square Area by Removing Fences From a Field/2975.java

```
class Solution {
    public int maximizeSquareArea(int m, int n, int[] hFences, int[] vFences) {
        final int MOD = 1_000_000_007;

        hFences = Arrays.copyOf(hFences, hFences.length + 2);
        vFences = Arrays.copyOf(vFences, vFences.length + 2);

        hFences[hFences.length - 2] = 1;
        hFences[hFences.length - 1] = m;
        vFences[vFences.length - 2] = 1;
        vFences[vFences.length - 1] = n;

        Arrays.sort(hFences);
        Arrays.sort(vFences);

        Set<Integer> hGaps = getGaps(hFences);
        Set<Integer> vGaps = getGaps(vFences);
        int maxGap = -1;

        for (final int hGap : hGaps)
            if (vGaps.contains(hGap))
                maxGap = Math.max(maxGap, hGap);

        return maxGap == -1 ? -1 : (int) ((long) maxGap * maxGap % MOD);
    }

    private Set<Integer> getGaps(int[] fences) {
        Set<Integer> gaps = new HashSet<>();
        for (int i = 0; i < fences.length; ++i)
            for (int j = 0; j < i; ++j)
                gaps.add(fences[i] - fences[j]);
        return gaps;
    }
}
```

1994. 2976.java

Path: LeetCode-main/solutions/2976. Minimum Cost to Convert String I/2976.java

```
class Solution {
    public long minimumCost(String source, String target, char[] original, char[] changed,
                             int[] cost) {
        long ans = 0;
        // dist[u][v] := the minimum distance to change ('a' + u) to ('a' + v)
        long[][] dist = new long[26][26];
        Arrays.stream(dist).forEach(A -> Arrays.fill(A, Long.MAX_VALUE));

        for (int i = 0; i < cost.length; ++i) {
            final int u = original[i] - 'a';
            final int v = changed[i] - 'a';
            dist[u][v] = Math.min(dist[u][v], cost[i]);
        }

        for (int k = 0; k < 26; ++k)
            for (int i = 0; i < 26; ++i)
                if (dist[i][k] < Long.MAX_VALUE)
                    for (int j = 0; j < 26; ++j)
                        if (dist[k][j] < Long.MAX_VALUE)
                            dist[i][j] = Math.min(dist[i][j], dist[i][k] + dist[k][j]);

        for (int i = 0; i < source.length(); ++i) {
            if (source.charAt(i) == target.charAt(i))
                continue;
            final int u = source.charAt(i) - 'a';
            final int v = target.charAt(i) - 'a';
            if (dist[u][v] == Long.MAX_VALUE)
                return -1;
            ans += dist[u][v];
        }

        return ans;
    }
}
```

1995. 2977.java

Path: LeetCode-main/solutions/2977. Minimum Cost to Convert String II/2977.java

```
class Solution {
    public long minimumCost(String source, String target, String[] original, String[] changed,
                             int[] cost) {
        Set<Integer> subLengths = getSubLengths(original);
        Map<String, Integer> subToId = getSubToId(original, changed);
        final int subCount = subToId.size();
        // dist[u][v] := the minimum distance to change the substring with id u to
        // the substring with id v
        long[][] dist = new long[subCount][subCount];
        Arrays.stream(dist).forEach(A -> Arrays.fill(A, Long.MAX_VALUE));
        // dp[i] := the minimum cost to change the first i letters of `source` into
        // `target`, leaving the suffix untouched
        long[] dp = new long[source.length() + 1];
        Arrays.fill(dp, Long.MAX_VALUE);

        for (int i = 0; i < cost.length; ++i) {
            final int u = subToId.get(original[i]);
            final int v = subToId.get(changed[i]);
            dist[u][v] = Math.min(dist[u][v], (long) cost[i]);
        }

        for (int k = 0; k < subCount; ++k)
            for (int i = 0; i < subCount; ++i)
                if (dist[i][k] < Long.MAX_VALUE)
                    for (int j = 0; j < subCount; ++j)
                        if (dist[k][j] < Long.MAX_VALUE)
                            dist[i][j] = Math.min(dist[i][j], dist[i][k] + dist[k][j]);

        dp[0] = 0;

        for (int i = 0; i < source.length(); ++i) {
            if (dp[i] == Long.MAX_VALUE)
                continue;
            if (target.charAt(i) == source.charAt(i))
                dp[i + 1] = Math.min(dp[i + 1], dp[i]);
            for (int subLength : subLengths) {
                if (i + subLength > source.length())
                    continue;
                String subSource = source.substring(i, i + subLength);
                String subTarget = target.substring(i, i + subLength);
                if (!subToId.containsKey(subSource) || !subToId.containsKey(subTarget))
                    continue;
                final int u = subToId.get(subSource);
                final int v = subToId.get(subTarget);
                if (dist[u][v] < Long.MAX_VALUE)
                    dp[i + subLength] = Math.min(dp[i + subLength], dp[i] + dist[u][v]);
            }
        }

        return dp[source.length()] == Long.MAX_VALUE ? -1 : dp[source.length()];
    }

    private Map<String, Integer> getSubToId(String[] original, String[] changed) {
        Map<String, Integer> subToId = new HashMap<>();
        for (final String s : original)
            subToId.putIfAbsent(s, subToId.size());
        for (final String s : changed)
            subToId.putIfAbsent(s, subToId.size());
        return subToId;
    }

    private Set<Integer> getSubLengths(String[] original) {
        Set<Integer> subLengths = new HashSet<>();
        for (final String s : original)
            subLengths.add(s.length());
        return subLengths;
    }
}
```


1996. 2979.java

Path: LeetCode-main/solutions/2979. Most Expensive Item That Can Not Be Bought/2979.java

```
class Solution {  
    public int mostExpensiveItem(int primeOne, int primeTwo) {  
        // https://en.wikipedia.org/wiki/Coin\_problem  
        return primeOne * primeTwo - primeOne - primeTwo;  
    }  
}
```

1997. 298.java

Path: LeetCode-main/solutions/298. Binary Tree Longest Consecutive Sequence/298.java

```
class Solution {
    public int longestConsecutive(TreeNode root) {
        if (root == null)
            return 0;
        return dfs(root, -1, 0, 1);
    }

    private int dfs(TreeNode root, int target, int length, int mx) {
        if (root == null)
            return mx;
        if (root.val == target)
            mx = Math.max(mx, ++length);
        else
            length = 1;
        return Math.max(dfs(root.left, root.val + 1, length, mx),
                        dfs(root.right, root.val + 1, length, mx));
    }
}
```

1998. 2980.java

Path: LeetCode-main/solutions/2980. Check if Bitwise OR Has Trailing Zeros/2980.java

```
class Solution {  
    public boolean hasTrailingZeros(int[] nums) {  
        int countEven = 0;  
  
        for (final int num : nums)  
            if (num % 2 == 0)  
                ++countEven;  
  
        return countEven >= 2;  
    }  
}
```

1999. 2981.java

Path: LeetCode-main/solutions/2981. Find Longest Special Substring That Occurs Thrice I/2981.java

```
class Solution {
    public int maximumLength(String s) {
        final int n = s.length();
        int ans = -1;
        int runningLen = 0;
        char prevLetter = '@';
        // counts[i][j] := the frequency of ('a' + i) repeating j times
        int[][] counts = new int[26][n + 1];

        for (final char c : s.toCharArray())
            if (c == prevLetter) {
                ++counts[c - 'a'][++runningLen];
            } else {
                runningLen = 1;
                ++counts[c - 'a'][runningLen];
                prevLetter = c;
            }

        for (int[] count : counts) {
            ans = Math.max(ans, getMaxFreq(count, n));
        }

        return ans;
    }

    // Returns the maximum frequency that occurs more than three times.
    private int getMaxFreq(int[] count, int maxFreq) {
        int times = 0;
        for (int freq = maxFreq; freq >= 1; --freq) {
            times += count[freq];
            if (times >= 3)
                return freq;
        }
        return -1;
    }
}
```

2000. 2982.java

Path: LeetCode-main/solutions/2982. Find Longest Special Substring That Occurs Thrice II/2982.java

```
class Solution {
    public int maximumLength(String s) {
        final int n = s.length();
        int ans = -1;
        int runningLen = 0;
        char prevLetter = '@';
        // counts[i][j] := the frequency of ('a' + i) repeating j times
        int[][] counts = new int[26][n + 1];

        for (final char c : s.toCharArray())
            if (c == prevLetter) {
                ++counts[c - 'a'][++runningLen];
            } else {
                runningLen = 1;
                ++counts[c - 'a'][runningLen];
                prevLetter = c;
            }

        for (int[] count : counts) {
            ans = Math.max(ans, getMaxFreq(count, n));
        }

        return ans;
    }

    // Returns the maximum frequency that occurs more than three times.
    private int getMaxFreq(int[] count, int maxFreq) {
        int times = 0;
        for (int freq = maxFreq; freq >= 1; --freq) {
            times += count[freq];
            if (times >= 3)
                return freq;
        }
        return -1;
    }
}
```

... 1371 more files omitted to avoid huge PDF ...